

WPU SABLAE Portal — Complete Source Code

Generated for CLASS / lms_scheduling. Monospace listing of all PHP, SQL, JS, CSS, and YAML source files.

WPU SABLAE Portal — Complete Source Code

Generated: 2026-07-27 08:29:51 CEST

Root: C:\xampp\htdocs\CLASS

Files: 124

FILE: add_schedule.php

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/schedule_helpers.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['dean', 'program_chair']);
$collegeId = dean_or_program_chair_college_id_or_fail();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
$scheduleFormOptions = [
    'unlock_program' => true,
    'all_college_programs' => true,
];
$effectiveProgramScope = $scheduleFormOptions['unlock_program'] ? null : $programScope;

if (in_array($SESSION['role'] ?? '', ['dean', 'program_chair'], true)) {
    $userId = (int) ($SESSION['user_id'] ?? 0);
    ensure_faculty_profile_for_teaching_role($userId);
}
$hasLabFlag = db_column_exists('courses', 'is_laboratory');
$hasProgramsTable = db_table_exists('programs');
$hasYearLevel = db_column_exists('courses', 'year_level');
$hasSection = db_column_exists('courses', 'section');
$hasGeScheduleTargets = db_table_exists('ge_schedule_targets');

$errors = [];
$old = $_POST ?? [];
if (isset($old['days']) && is_array($old['days'])) {
    $old['day_array'] = $old['days'];
}
if (isset($old['lab_days']) && is_array($old['lab_days'])) {
    $old['lab_day_array'] = $old['lab_days'];
}
if (!isset($old['lecture_slot_day']) || !is_array($old['lecture_slot_day'])) {
    $old['lecture_slot_day'] = [''];
}
if (!isset($old['lecture_slot_start_time']) || !is_array($old['lecture_slot_start_time'])) {
    $old['lecture_slot_start_time'] = [''];
}
if (!isset($old['lecture_slot_end_time']) || !is_array($old['lecture_slot_end_time'])) {
    $old['lecture_slot_end_time'] = [''];
}
if (!isset($old['lecture_slot_room_id']) || !is_array($old['lecture_slot_room_id'])) {
    $old['lecture_slot_room_id'] = [''];
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $courseCollegeId = $collegeId;
    $facultyId = (int) ($_POST['faculty_id'] ?? 0);
    $courseId = (int) ($_POST['course_id'] ?? 0);
    $roomId = (int) ($_POST['room_id'] ?? 0);
    $scheduleType = (string) ($_POST['schedule_type'] ?? 'Custom');
    $days = isset($_POST['days']) && is_array($_POST['days']) ? array_map('strval', $_POST['days']) : [];
    $startTime = substr((string) ($_POST['start_time'] ?? ''), 0, 5) . ':00';
    $endTime = substr((string) ($_POST['end_time'] ?? ''), 0, 5) . ':00';
    $semester = trim((string) ($_POST['semester'] ?? ''));
    $schoolYear = trim((string) ($_POST['school_year'] ?? ''));
    $academicYear = trim((string) ($_POST['academic_year'] ?? ''));

    $labRoomId = (int) ($_POST['lab_room_id'] ?? 0);
    $labDays = isset($_POST['lab_days']) && is_array($_POST['lab_days']) ? array_map('strval', $_POST['lab_days']) : [];
    $labStartTime = substr((string) ($_POST['lab_start_time'] ?? ''), 0, 5) . ':00';
    $labEndTime = substr((string) ($_POST['lab_end_time'] ?? ''), 0, 5) . ':00';
    $lectureSlotDays = isset($_POST['lecture_slot_day']) && is_array($_POST['lecture_slot_day']) ? array_map('strval',
$_POST['lecture_slot_day']) : [];
```

```

        $lectureSlotStartTimes = isset($_POST['lecture_slot_start_time']) && is_array($_POST['lecture_slot_start_time']) ? array_map('strval',
$_POST['lecture_slot_start_time']) : [];
        $lectureSlotEndTimes = isset($_POST['lecture_slot_end_time']) && is_array($_POST['lecture_slot_end_time']) ? array_map('strval',
$_POST['lecture_slot_end_time']) : [];
        $lectureSlotRoomIds = isset($_POST['lecture_slot_room_id']) && is_array($_POST['lecture_slot_room_id']) ? array_map('intval',
$_POST['lecture_slot_room_id']) : [];

        $requestOverride = !empty($_POST['request_admin_override']);
        $overrideReason = trim((string) ($_POST['override_reason'] ?? ''));
        $selectedProgramRaw = trim((string) ($_POST['program'] ?? ''));
        $selectedProgramParsed = schedule_program_option_parse($selectedProgramRaw);
        $selectedProgram = $selectedProgramParsed['program_name'];
        $selectedProgramCollegeId = $selectedProgramParsed['college_id'];

        if ($facultyId < 1 || $courseId < 1) {
            $errors[] = 'Please select faculty and course.';
        }
        if ($semester === '' || $schoolYear === '') {
            $errors[] = 'Semester and school year are required.';
        }

        $courseIsLab = false;
        $geProgramScope = is_ge_program_scope($programScope);
        $targetCollegeId = 0;
        $targetProgram = '';
        $targetYearLevel = '';
        $targetSection = '';
        if ($geProgramScope) {
            $targetCollegeId = (int) ($_POST['target_college_id'] ?? 0);
            $targetProgram = trim((string) ($_POST['target_program'] ?? ''));
            $targetYearLevel = trim((string) ($_POST['target_year_level'] ?? ''));
            $targetSection = trim((string) ($_POST['target_section'] ?? ''));
            if ($targetCollegeId < 1 || $targetProgram === '' || $targetYearLevel === '' || $targetSection === '') {
                $errors[] = 'Target college, program, year level, and section are required for GE scheduling.';
            }
            if (!$hasGeScheduleTargets) {
                $errors[] = 'Run upgrade_roles.php first to enable GE schedule targets.';
            }
        }
        if (!$errors) {
            if ($geProgramScope) {
                $sql = "SELECT COUNT(*) FROM faculty WHERE id=? AND college_id=? AND status='active'";
                $params = [$facultyId, $collegeId];
                if ($effectiveProgramScope !== null) {
                    $sql .= " AND (department=? OR department='')";
                    $params[] = $effectiveProgramScope;
                }
                $chk = db()->prepare($sql);
                $chk->execute($params);
                if ((int) $chk->fetchColumn() < 1) {
                    $errors[] = 'Selected faculty is not in your college GE program.';
                }
            } else {
                $sql = 'SELECT COUNT(*) FROM faculty WHERE id=? AND college_id=?';
                $params = [$facultyId, $collegeId];
                if ($effectiveProgramScope !== null) {
                    $sql .= " AND (department=? OR department='')";
                    $params[] = $effectiveProgramScope;
                }
                $chk = db()->prepare($sql);
                $chk->execute($params);
                if ((int) $chk->fetchColumn() < 1) {
                    $errors[] = $effectiveProgramScope !== null
                        ? 'Selected faculty is outside your assigned program.'
                        : 'Selected faculty is not in your college.';
                }
            }
        }

        $courseOk = false;
        if ($geProgramScope && db_column_exists('courses', 'is_gened') && db_table_exists('ge_course_colleges')) {
            $chk = db()->prepare(
                'SELECT c.is_laboratory FROM courses c
                INNER JOIN ge_course_colleges gcc ON gcc.course_id = c.id
                WHERE c.id=? AND COALESCE(c.is_gened,0)=1 AND gcc.college_id=?'
            );
            $chk->execute([$courseId, $collegeId]);
            $geRow = $chk->fetch(PDO::FETCH_ASSOC);
            if ($geRow) {
                $courseOk = true;
                $courseIsLab = $hasLabFlag && (int) ($geRow['is_laboratory'] ?? 0) === 1;
            }
        }
    }
}

```

```

    } elseif ($geProgramScope) {
        $errors[] = 'Selected course is not a GE course offered to your college.';
    }
}

$courseCollegeId = $collegeId;
if (!$courseOk && !$geProgramScope && $hasLabFlag) {
    $sql = 'SELECT is_laboratory, college_id FROM courses WHERE id=?';
    $params = [$courseId];
    if (!$scheduleFormOptions['all_college_programs']) {
        $sql .= ' AND college_id=?';
        $params[] = $collegeId;
    }
    if ($effectiveProgramScope !== null) {
        $sql .= ' AND department=?';
        $params[] = $effectiveProgramScope;
    }
    $chk = db()->prepare($sql);
    $chk->execute($params);
    $courseRow = $chk->fetch(PDO::FETCH_ASSOC);
    if ($courseRow === false) {
        $errors[] = $effectiveProgramScope !== null
            ? 'Selected course is outside your assigned program.'
            : ($scheduleFormOptions['all_college_programs']
                ? 'Selected course is invalid.'
                : 'Selected course is not in your college.');
```

```

    } else {
        $courseOk = true;
        $courseCollegeId = (int) ($courseRow['college_id'] ?? $collegeId);
        $courseIsLab = (int) ($courseRow['is_laboratory'] ?? 0) === 1;
    }
} elseif (!$courseOk) {
    $sql = 'SELECT college_id FROM courses WHERE id=?';
    $params = [$courseId];
    if (!$scheduleFormOptions['all_college_programs']) {
        $sql .= ' AND college_id=?';
        $params[] = $collegeId;
    }
    if ($effectiveProgramScope !== null) {
        $sql .= ' AND department=?';
        $params[] = $effectiveProgramScope;
    }
    $chk = db()->prepare($sql);
    $chk->execute($params);
    $courseCollegeId = (int) $chk->fetchColumn();
    if ($courseCollegeId < 1) {
        $errors[] = $effectiveProgramScope !== null
            ? 'Selected course is outside your assigned program.'
            : ($scheduleFormOptions['all_college_programs']
                ? 'Selected course is invalid.'
                : 'Selected course is not in your college.');
```

```

    } else {
        $courseOk = true;
    }
} else {
    $chk = db()->prepare('SELECT college_id FROM courses WHERE id=?');
    $chk->execute([$courseId]);
    $resolvedCollegeId = (int) $chk->fetchColumn();
    if ($resolvedCollegeId > 0) {
        $courseCollegeId = $resolvedCollegeId;
    }
}

if (!$geProgramScope && $hasProgramsTable && $selectedProgram !== '') {
    if ($selectedProgramCollegeId > 0) {
        $chk = db()->prepare("SELECT COUNT(*) FROM programs WHERE college_id=? AND program_name=? AND status='active'");
        $chk->execute([$selectedProgramCollegeId, $selectedProgram]);
    } else {
        $chk = db()->prepare("SELECT COUNT(*) FROM programs WHERE program_name=? AND status='active'");
        $chk->execute([$selectedProgram]);
    }
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'Selected program is invalid.';
    } else {
        $chk = db()->prepare('SELECT TRIM(department) FROM courses WHERE id=?');
        $chk->execute([$courseId]);
        $courseDept = trim((string) $chk->fetchColumn());
        if ($courseDept !== $selectedProgram) {
            $errors[] = 'Selected course does not match the selected program.';
        }
    }
}

```

```

    }
}

if (!$geProgramScope && $hasYearLevel && $hasSection) {
    $yearLevelPost = trim((string) ($_POST['year_level'] ?? ''));
    $sectionPost = trim((string) ($_POST['section'] ?? ''));
    if ($yearLevelPost === '' || $sectionPost === '') {
        $errors[] = 'Year level and section are required.';
    } else {
        $chk = db()->prepare('SELECT year_level, section FROM courses WHERE id=?');
        $chk->execute([$courseId]);
        $crow = $chk->fetch();
        if ($crow) {
            $cy = trim((string) ($crow['year_level'] ?? ''));
            $cs = trim((string) ($crow['section'] ?? ''));
            if ($cy !== $yearLevelPost || $cs !== $sectionPost) {
                $errors[] = 'Selected course does not match the chosen year level and section.';
            }
        }
    }
}

if ($geProgramScope && $targetCollegeId > 0 && !$errors) {
    $chk = db()->prepare("SELECT COUNT(*) FROM colleges WHERE id=? AND status='active'");
    $chk->execute([$targetCollegeId]);
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'Selected target college is invalid.';
    }
    $chk = db()->prepare("SELECT COUNT(*) FROM programs WHERE college_id=? AND program_name=? AND status='active'");
    $chk->execute([$targetCollegeId, $targetProgram]);
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'Selected target program is invalid for the chosen college.';
    }
}

if (!$errors) {
    $geTargetForLoad = null;
    if ($geProgramScope && $targetCollegeId > 0 && $targetProgram !== '' && $targetYearLevel !== '' && $targetSection !== '') {
        $geTargetForLoad = [
            'college_id' => $targetCollegeId,
            'program_name' => $targetProgram,
            'year_level' => $targetYearLevel,
            'section' => $targetSection,
        ];
    }
    $assignedFaculty = find_course_load_assigned_faculty($courseId, $semester, $schoolYear, $facultyId, $geTargetForLoad);
    if ($assignedFaculty !== null) {
        $errors[] = course_load_assignment_conflict_message($assignedFaculty, $courseId, $geTargetForLoad);
    }
}

$hasLectureDetails = ($roomId > 0) || ($days !== []);
$hasLabDetails = ($labRoomId > 0) || ($labDays !== []);

$lectureComplete = $roomId > 0 && $days !== [];
$labComplete = $labRoomId > 0 && $labDays !== [];

$segments = [];

if ($courseIsLab) {
    if (!$lectureComplete && !$labComplete) {
        $errors[] = 'For laboratory courses, set at least a laboratory schedule (room and days), or complete both lecture room and lecture days.';
    }
    if ($hasLectureDetails && !$lectureComplete) {
        $errors[] = 'Lecture schedule is incomplete. Please set both lecture room and lecture days, or clear lecture fields.';
    }
    if ($hasLabDetails && !$labComplete) {
        $errors[] = 'Laboratory schedule is incomplete. Please set both laboratory room and laboratory days.';
    }
} else {
    if (!$lectureComplete) {
        $errors[] = 'Please set lecture room and select at least one lecture day.';
    }
}

if ($lectureComplete) {
    $lectureRules = validate_schedule_rules($scheduleType, $days, $startTime, $endTime);
    foreach ($lectureRules['errors'] as $e) {

```

```

        $errors[] = $e;
    }
    $chk = db()->prepare('SELECT COUNT(*) FROM rooms WHERE id=?');
    $chk->execute([$roomId]);
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'Selected lecture room is invalid.';
    }
    $segments[] = [
        'label' => 'Lecture',
        'room_id' => $roomId,
        'days' => $days,
        'start_time' => $startTime,
        'end_time' => $endTime,
        'schedule_type' => $scheduleType,
    ];
}

$slotCount = max(count($lectureSlotDays), count($lectureSlotStartTimes), count($lectureSlotEndTimes), count($lectureSlotRoomIds));
for ($i = 0; $i < $slotCount; $i++) {
    $slotDay = trim((string) ($lectureSlotDays[$i] ?? ''));
    $slotStartRaw = trim((string) ($lectureSlotStartTimes[$i] ?? ''));
    $slotEndRaw = trim((string) ($lectureSlotEndTimes[$i] ?? ''));
    $slotRoomId = (int) ($lectureSlotRoomIds[$i] ?? 0);
    $isEmpty = $slotDay === '' && $slotStartRaw === '' && $slotEndRaw === '' && $slotRoomId < 1;
    if ($isEmpty) {
        continue;
    }
    if ($slotDay === '' || $slotStartRaw === '' || $slotEndRaw === '' || $slotRoomId < 1) {
        $errors[] = 'Each additional lecture slot must have day, start time, end time, and room.';
        continue;
    }
    $slotStart = substr($slotStartRaw, 0, 5) . ':00';
    $slotEnd = substr($slotEndRaw, 0, 5) . ':00';
    $slotRules = validate_schedule_rules('Custom', [$slotDay], $slotStart, $slotEnd);
    foreach ($slotRules['errors'] as $e) {
        $errors[] = 'Additional lecture slot: ' . $e;
    }
    $chk = db()->prepare('SELECT COUNT(*) FROM rooms WHERE id=?');
    $chk->execute([$slotRoomId]);
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'Selected room for additional lecture slot is invalid.';
    }
    $segments[] = [
        'label' => 'Lecture',
        'room_id' => $slotRoomId,
        'days' => [$slotDay],
        'start_time' => $slotStart,
        'end_time' => $slotEnd,
        'schedule_type' => 'Custom',
    ];
}

if ($courseIsLab && $labComplete) {
    $labRules = validate_schedule_rules('Custom', $labDays, $labStartTime, $labEndTime);
    foreach ($labRules['errors'] as $e) {
        $errors[] = 'Laboratory: ' . $e;
    }
    $chk = db()->prepare('SELECT COUNT(*) FROM rooms WHERE id=?');
    $chk->execute([$labRoomId]);
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'Selected laboratory room is invalid.';
    }
    $segments[] = [
        'label' => 'Laboratory',
        'room_id' => $labRoomId,
        'days' => $labDays,
        'start_time' => $labStartTime,
        'end_time' => $labEndTime,
        'schedule_type' => 'Custom',
    ];
}

$allConflicts = [];
$segmentCross = [];
if (!$errors) {
    foreach ($segments as $idx => $seg) {
        $conflicts = checkConflicts(
            $facultyId,
            (int) $seg['room_id'],
            $seg['days'],

```

```

        $seg['start_time'],
        $seg['end_time'],
        $semester,
        $schoolYear,
        null,
        $collegeId,
        true
    );
    $internal = [];
    $cross = [];
    foreach ($conflicts as $c) {
        if (($c['scope'] ?? '') === 'cross_college') {
            $cross[] = $c;
        } else {
            $internal[] = $c;
        }
    }
    foreach ($internal as $c) {
        $errors[] = $seg['label'] . ': ' . $c['description'];
    }
    if ($cross) {
        $segmentCross[$idx] = $cross;
    }
    $allConflicts[$idx] = $conflicts;

    if ($geProgramScope && $targetCollegeId > 0 && $targetProgram !== '') {
        $stBlock = db()->prepare(
            'SELECT s.id, s.day_of_week, s.start_time, s.end_time, c.course_code
            FROM schedules s
            INNER JOIN courses c ON c.id = s.course_id
            WHERE s.college_id = ?
            AND s.semester = ?
            AND s.school_year = ?
            AND c.department = ?
            AND c.year_level = ?
            AND c.section = ?'
        );
        $stBlock->execute([$targetCollegeId, $semester, $schoolYear, $targetProgram, $targetYearLevel, $targetSection]);
        $stMin = time_to_minutes((string) $seg['start_time']);
        $enMin = time_to_minutes((string) $seg['end_time']);
        foreach ($stBlock->fetchAll() as $br) {
            $existingDays = parse_day_set((string) $br['day_of_week']);
            if (!array_intersect($seg['days'], $existingDays)) {
                continue;
            }
            $rowStart = time_to_minutes((string) $br['start_time']);
            $rowEnd = time_to_minutes((string) $br['end_time']);
            if (intervals_overlap($stMin, $enMin, $rowStart, $rowEnd)) {
                $errors[] = $seg['label'] . ': Target block conflict - ' . $targetProgram . ' Y' . $targetYearLevel . '-' .
                $targetSection . ' already has ' . $br['course_code'] . ' at this time.';
                break;
            }
        }
    }
    if ($hasGeScheduleTargets && !$errors) {
        $stTarget = db()->prepare(
            'SELECT s.id, s.day_of_week, s.start_time, s.end_time, c.course_code
            FROM ge_schedule_targets gst
            INNER JOIN schedules s ON s.id = gst.schedule_id
            INNER JOIN courses c ON c.id = s.course_id
            WHERE gst.college_id = ?
            AND gst.program_name = ?
            AND gst.year_level = ?
            AND gst.section = ?
            AND s.semester = ?
            AND s.school_year = ?'
        );
        $stTarget->execute([$targetCollegeId, $targetProgram, $targetYearLevel, $targetSection, $semester, $schoolYear]);
        foreach ($stTarget->fetchAll() as $tr) {
            $existingDays = parse_day_set((string) $tr['day_of_week']);
            if (!array_intersect($seg['days'], $existingDays)) {
                continue;
            }
            $rowStart = time_to_minutes((string) $tr['start_time']);
            $rowEnd = time_to_minutes((string) $tr['end_time']);
            if (intervals_overlap($stMin, $enMin, $rowStart, $rowEnd)) {
                $errors[] = $seg['label'] . ': GE block conflict - another GE schedule (' . $tr['course_code'] . ') already
                targets ' . $targetProgram . ' Y' . $targetYearLevel . '-' . $targetSection . ' at this time.';
                break;
            }
        }
    }

```

```

    }
    }
}

if ($segmentCross && !$requestOverride) {
    foreach ($segmentCross as $idx => $crossList) {
        foreach ($crossList as $c) {
            $errors[] = $segments[$idx]['label'] . ': ' . $c['description'] . ' (Request admin override to proceed.)';
        }
    }
}

if ($segmentCross && $requestOverride && $overrideReason === '') {
    $errors[] = 'Please add a reason for admin override request.';
}

if (!$errors && $segmentCross && $requestOverride) {
    foreach ($segmentCross as $idx => $crossList) {
        $seg = $segments[$idx];
        create_conflict_request([
            'requested_by' => (int) $_SESSION['user_id'],
            'college_id' => $collegeId,
            'faculty_id' => $facultyId,
            'course_id' => $courseId,
            'room_id' => (int) $seg['room_id'],
            'schedule_type' => (string) $seg['schedule_type'],
            'day_of_week' => days_to_set($seg['days']),
            'start_time' => (string) $seg['start_time'],
            'end_time' => (string) $seg['end_time'],
            'semester' => $semester,
            'school_year' => $schoolYear,
            'academic_year' => $academicYear,
            'reason' => $overrideReason . ' [' . $seg['label'] . ']',
        ]);
    }
    log_dean_activity('schedule_override_request', 'Submitted cross-college conflict override request (with separated lab
days/time)');
    $_SESSION['flash'] = 'Conflict request sent to admin for approval.';
    header('Location: schedule.php');
    exit;
}

if (!$errors) {
    $uid = (int) $_SESSION['user_id'];
    $hasSessionKind = db_column_exists('schedules', 'session_kind');
    $scheduleInsertSql = 'INSERT INTO schedules (faculty_id, course_id, room_id, college_id, schedule_type, day_of_week, start_time,
end_time, semester, school_year, academic_year, created_by'
        . ($hasSessionKind ? ', session_kind' : '')
        . ') VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?'
        . ($hasSessionKind ? ', ?' : ''))
        . ')';
    $stmt = db()->prepare($scheduleInsertSql);
    $createdIds = [];
    foreach ($segments as $idx => $seg) {
        $insertParams = [
            $facultyId,
            $courseId,
            (int) $seg['room_id'],
            $collegeId,
            (string) $seg['schedule_type'],
            days_to_set($seg['days']),
            (string) $seg['start_time'],
            (string) $seg['end_time'],
            $semester,
            $schoolYear,
            $academicYear,
            $uid,
        ];
        if ($hasSessionKind) {
            $insertParams[] = schedule_segment_session_kind((string) $seg['label']);
        }
        $stmt->execute($insertParams);
        $newId = (int) db()->lastInsertId();
        $createdIds[] = $newId;
        if ($geProgramScope && $hasGeScheduleTargets && $targetCollegeId > 0 && $targetProgram !== '') {
            db()->prepare(
                'INSERT INTO ge_schedule_targets (schedule_id, college_id, program_name, year_level, section)
                VALUES (?, ?, ?, ?, ?)'
            );
        }
    }
}

```

```

        )->execute([$newId, $targetCollegeId, $targetProgram, $targetYearLevel, $targetSection]);
    }
    if (!empty($allConflicts[$idx])) {
        log_conflicts($newId, $allConflicts[$idx], false);
    }
}

log_dean_activity('schedule_create', 'Created schedule entries #' . implode(', ', $createdIds));
$summaryRows = [];
foreach ($createdIds as $cidLog) {
    $stl = db()->prepare(
        'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year, s.day_of_week, s.start_time,
s.end_time,
        f.full_name AS faculty_name, c.course_code, r.room_code
        FROM schedules s
        INNER JOIN faculty f ON f.id = s.faculty_id
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN rooms r ON r.id = s.room_id
        WHERE s.id = ? LIMIT 1'
    );
    $stl->execute([(int) $cidLog]);
    $r = $stl->fetch(PDO::FETCH_ASSOC);
    if ($r) {
        $summaryRows[] = $r;
    }
}
log_user_activity(
    'add',
    'Schedules',
    'New schedule row(s): #' . implode(', #', $createdIds),
    null,
    ['created_schedule_ids' => $createdIds, 'rows' => $summaryRows]
);
if ($courseIsLab) {
    $createdLecture = in_array('Lecture', array_column($segments, 'label'), true);
    $createdLaboratory = in_array('Laboratory', array_column($segments, 'label'), true);
    if ($createdLecture && $createdLaboratory) {
        $_SESSION['flash'] = 'Lecture and laboratory schedules created successfully.';
    } elseif ($createdLaboratory) {
        $_SESSION['flash'] = 'Laboratory schedule created successfully.';
    } else {
        $_SESSION['flash'] = 'Lecture schedule created successfully.';
    }
} else {
    $_SESSION['flash'] = 'Schedule created successfully.';
}
header('Location: schedule.php');
exit;
}
}

$pageTitle = 'Add schedule';
require_once __DIR__ . '/includes/header.php';
?>
<h1 class="h3 mb-4"><i class="fa-solid fa-plus me-2 text-primary"></i>Add schedule</h1>

<?php render_schedule_errors_warning_popup($errors); ?>

<div class="card shadow-sm">
    <div class="card-body">
        <form method="post" novalidate>
            <?php
                $defaults = $old;
                if ($_SERVER['REQUEST_METHOD'] !== 'POST' && $defaults === []) {
                    $defaults = [
                        'schedule_type' => 'Custom',
                        'semester' => '1st Semester',
                        'school_year' => date('Y') . '-' . (date('Y') + 1),
                        'start_time' => '08:00:00',
                        'end_time' => '09:00:00',
                        'lab_start_time' => '13:00:00',
                        'lab_end_time' => '16:00:00',
                    ];
                }
            render_schedule_form($defaults, $scheduleFormOptions);
            ?>
            <div class="mt-4 p-3 border rounded bg-light">
                <div class="d-flex align-items-center justify-content-between mb-2">
                    <h6 class="mb-0">Additional lecture time slots</h6>
                    <button type="button" class="btn btn-sm btn-outline-primary" id="addLectureSlotBtn">

```



```

        <i class="fa-solid fa-plus me-1"></i>Add slot
    </button>
</div>
<div class="small text-muted mb-3">
    Use this when lecture days have different times (e.g., Monday 8:00-9:30 and Wednesday 1:00-2:30).
</div>
<div id="lectureSlotsWrap">
    <?php
    $slotDays = is_array($old['lecture_slot_day'] ?? null) ? $old['lecture_slot_day'] : [''];
    $slotStarts = is_array($old['lecture_slot_start_time'] ?? null) ? $old['lecture_slot_start_time'] : [''];
    $slotEnds = is_array($old['lecture_slot_end_time'] ?? null) ? $old['lecture_slot_end_time'] : [''];
    $slotRooms = is_array($old['lecture_slot_room_id'] ?? null) ? $old['lecture_slot_room_id'] : [''];
    $slotTotal = max(1, count($slotDays), count($slotStarts), count($slotEnds), count($slotRooms));
    $dayOptions = schedule_days_list();
    $roomOptions = db()->query("SELECT id, room_code, room_name FROM rooms WHERE status IN ('available','tba') ORDER BY
room_code")->fetchAll();
    for ($i = 0; $i < $slotTotal; $i++):
        $slotDayValue = (string) ($slotDays[$i] ?? '');
        $slotStartValue = substr((string) ($slotStarts[$i] ?? ''), 0, 5);
        $slotEndValue = substr((string) ($slotEnds[$i] ?? ''), 0, 5);
        $slotRoomValue = (int) ($slotRooms[$i] ?? 0);
    ?>
    <div class="row g-2 mb-2 lecture-slot-row">
        <div class="col-md-3">
            <select name="lecture_slot_day[]" class="form-select">
                <option value="">Day</option>
                <?php foreach ($dayOptions as $d): ?>
                    <option value="<?= htmlspecialchars($d) ?>" <?= $slotDayValue === $d ? 'selected' : '' ?>><?>
htmlspecialchars($d) ?></option>
                <?php endforeach; ?>
            </select>
        </div>
        <div class="col-md-3">
            <input type="time" name="lecture_slot_start_time[]" class="form-control" value="<?=
htmlspecialchars($slotStartValue) ?>">
        </div>
        <div class="col-md-3">
            <input type="time" name="lecture_slot_end_time[]" class="form-control" value="<?=
htmlspecialchars($slotEndValue) ?>">
        </div>
        <div class="col-md-3">
            <div class="d-flex gap-2">
                <select name="lecture_slot_room_id[]" class="form-select">
                    <option value="">Room</option>
                    <?php foreach ($roomOptions as $r): ?>
                        <option value="<?= (int) $r['id'] ?>" <?= $slotRoomValue === (int) $r['id'] ? 'selected' : '' ?>
                            <?= htmlspecialchars($r['room_code'] . ($r['room_name'] ? ' - ' . $r['room_name'] : '')) ?>
                        </option>
                    <?php endforeach; ?>
                </select>
                <button type="button" class="btn btn-outline-danger remove-lecture-slot" title="Remove slot">
                    <i class="fa-solid fa-trash"></i>
                </button>
            </div>
        </div>
    </div>
    <?php endforeach; ?>
</div>
<div class="mt-3 p-3 border rounded bg-light">
    <div class="form-check">
        <input class="form-check-input" type="checkbox" id="request_admin_override" name="request_admin_override" value="1" <?=
!empty($old['request_admin_override']) ? 'checked' : '' ?>>
        <label class="form-check-label" for="request_admin_override">Request Admin Override for cross-college conflict</label>
    </div>
    <label class="form-label mt-2">Reason (required when requesting override)</label>
    <textarea name="override_reason" rows="2" class="form-control"><?= htmlspecialchars((string) ($old['override_reason'] ??
'')) ?></textarea>
    </div>
    <div class="mt-4">
        <button type="submit" class="btn btn-primary"><?= app_tooltip_attr('Creates the schedule row with the course, faculty, room,
and times you entered. Use this after filling all required fields.') ?>><i class="fa-solid fa-check me-1"></i>Save schedule</button>
        <a href="schedule.php" class="btn btn-outline-secondary"><?= app_tooltip_attr('Returns to the schedule list without saving
this draft.') ?>>Cancel</a>
    </div>
</form>
</div>
</div>
<script>

```

```

(function () {
    const wrap = document.getElementById('lectureSlotsWrap');
    const addBtn = document.getElementById('addLectureSlotBtn');
    if (!wrap || !addBtn) return;

    const rowTemplate = function () {
        const source = wrap.querySelector('.lecture-slot-row');
        if (!source) return null;
        const clone = source.cloneNode(true);
        clone.querySelectorAll('input').forEach((el) => {
            el.value = '';
        });
        clone.querySelectorAll('select').forEach((el) => {
            el.selectedIndex = 0;
        });
        return clone;
    };

    addBtn.addEventListener('click', function () {
        const row = rowTemplate();
        if (!row) return;
        wrap.appendChild(row);
    });

    wrap.addEventListener('click', function (event) {
        const btn = event.target.closest('.remove-lecture-slot');
        if (!btn) return;
        const rows = wrap.querySelectorAll('.lecture-slot-row');
        if (rows.length <= 1) {
            const row = btn.closest('.lecture-slot-row');
            if (!row) return;
            row.querySelectorAll('input').forEach((el) => {
                el.value = '';
            });
            row.querySelectorAll('select').forEach((el) => {
                el.selectedIndex = 0;
            });
            return;
        }
        const row = btn.closest('.lecture-slot-row');
        if (row) row.remove();
    });
})();
</script>

```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>
```

```
-----
FILE: admin_colleges.php
-----
```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['admin']);
$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if ($action === 'add') {
            db()->prepare('INSERT INTO colleges (college_code, college_name, status) VALUES (?, ?, ?)')
                ->execute([
                    trim((string) $_POST['college_code']),
                    trim((string) $_POST['college_name']),
                    (string) $_POST['status'],
                ]);
            $newId = (int) db()->lastInsertId();
            $after = [
                'id' => $newId,
                'college_code' => trim((string) $_POST['college_code']),
                'college_name' => trim((string) $_POST['college_name']),
                'status' => (string) $_POST['status'],
            ];
            log_admin_activity('add', 'Colleges', 'College #' . $newId, null, $after);
        }
    }
}

```

```

        $_SESSION['flash'] = 'College added.';
    } elseif ($action === 'edit') {
        $cid = (int) $_POST['id'];
        $stBefore = db()->prepare('SELECT id, college_code, college_name, status, dean_user_id FROM colleges WHERE id=?');
        $stBefore->execute([$cid]);
        $before = $stBefore->fetch(PDO::FETCH_ASSOC) ?: null;
        db()->prepare('UPDATE colleges SET college_code=?, college_name=?, status=?, dean_user_id=? WHERE id=?')
        ->execute([
            trim((string) $_POST['college_code']),
            trim((string) $_POST['college_name']),
            (string) $_POST['status'],
            ($_POST['dean_user_id'] ?? '') !== '' ? (int) $_POST['dean_user_id'] : null,
            $cid,
        ]);
        $deanId = ($_POST['dean_user_id'] ?? '') !== '' ? (int) $_POST['dean_user_id'] : null;
        if ($deanId) {
            db()->prepare('UPDATE users SET college_id=? WHERE id=? AND role="dean"')->execute([(int) $_POST['id'], $deanId]);
        }
        $after = [
            'id' => $cid,
            'college_code' => trim((string) $_POST['college_code']),
            'college_name' => trim((string) $_POST['college_name']),
            'status' => (string) $_POST['status'],
            'dean_user_id' => ($_POST['dean_user_id'] ?? '') !== '' ? (int) $_POST['dean_user_id'] : null,
        ];
        log_admin_activity('edit', 'Colleges', 'College #' . $cid, $before ? (array) $before : null, $after);
        $_SESSION['flash'] = 'College updated.';
    } elseif ($action === 'delete') {
        $cid = (int) $_POST['id'];
        $stBefore = db()->prepare('SELECT id, college_code, college_name, status, dean_user_id FROM colleges WHERE id=?');
        $stBefore->execute([$cid]);
        $before = $stBefore->fetch(PDO::FETCH_ASSOC) ?: null;
        db()->prepare('DELETE FROM colleges WHERE id=?')->execute([$cid]);
        log_admin_activity('delete', 'Colleges', 'College #' . $cid, $before ? (array) $before : null, null);
        $_SESSION['flash'] = 'College deleted.';
    }
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: admin_colleges.php');
exit;
}

$edit = null;
if (isset($_GET['edit'])) {
    $st = db()->prepare('SELECT * FROM colleges WHERE id=?');
    $st->execute([(int) $_GET['edit']]);
    $edit = $st->fetch() ?: null;
}

$deans = db()->query('SELECT id, full_name FROM users WHERE role="dean" ORDER BY full_name')->fetchAll();
$rows = db()->query(
    'SELECT c.*, u.full_name AS dean_name
    FROM colleges c
    LEFT JOIN users u ON u.id = c.dean_user_id
    ORDER BY c.college_code'
)->fetchAll();

$pageTitle = 'Manage Colleges';
require_once __DIR__ . '/includes/header.php';
?>
<style>
    @import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap');

    :root {
        --mc-bg: #f1f5f9;
        --mc-text: #0f172a;
        --mc-title: #0f2b3d;
        --mc-muted: #54708f;
        --mc-role-bg: #e9ecef;
        --mc-role-text: #1e4663;
        --mc-btn-bg: #fff;
        --mc-btn-border: #cbd5e1;
        --mc-btn-text: #1e293b;
        --mc-btn-hover-bg: #f8fafc;
        --mc-btn-hover-border: #94a3b8;
        --mc-card-bg: #fff;
        --mc-card-border: #eef2f6;
        --mc-card-header-border: #eef2f8;
        --mc-form-bg: #fefefe;
    }

```

```

--mc-form-border: #edf2f7;
--mc-input-bg: #fff;
--mc-input-border: #cfd9d8;
--mc-table-head-bg: #f2f2f2;
--mc-table-head-text: #2c4c6e;
--mc-table-row-border: #f0f4f8;
--mc-table-row-hover: #f2f2f2;
--mc-row-text: #1f2f3e;
--mc-empty-text: #7e95a6;
}

[data-bs-theme="dark"] {
  --mc-bg: #121a27;
  --mc-text: #e2e8f0;
  --mc-title: #f1f5f9;
  --mc-muted: #a9bdd3;
  --mc-role-bg: #223146;
  --mc-role-text: #d7e5f5;
  --mc-btn-bg: #1a2434;
  --mc-btn-border: #344861;
  --mc-btn-text: #d7e2f0;
  --mc-btn-hover-bg: #233249;
  --mc-btn-hover-border: #4a6588;
  --mc-card-bg: #182334;
  --mc-card-border: #2b3b53;
  --mc-card-header-border: #2b3b53;
  --mc-form-bg: #141f2f;
  --mc-form-border: #2b3b53;
  --mc-input-bg: #101a29;
  --mc-input-border: #3e5673;
  --mc-table-head-bg: #202f45;
  --mc-table-head-text: #c8d9ef;
  --mc-table-row-border: #2b3b53;
  --mc-table-row-hover: #223149;
  --mc-row-text: #d9e3ef;
  --mc-empty-text: #9eb4cb;
}

body {
  background: var(--mc-bg);
  font-family: 'Inter', sans-serif;
  color: var(--mc-text);
}

.dashboard-container {
  max-width: 1440px;
  margin: 0 auto;
}

.page-header {
  margin-bottom: 1.5rem;
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
}

.title-section h1 {
  font-size: 1.85rem;
  font-weight: 700;
  color: var(--mc-title);
  letter-spacing: -0.3px;
  display: flex;
  align-items: center;
  gap: 12px;
  margin: 0;
}

.title-section h1 i {
  color: #2c6e9e;
  font-size: 1.8rem;
}

.badge-role {
  background: var(--mc-role-bg);
  padding: 0.3rem 0.8rem;
  border-radius: 40px;
  font-size: 0.75rem;
  font-weight: 500;
  color: var(--mc-role-text);
  display: inline-flex;
  align-items: center;
  gap: 6px;
  margin-top: 8px;
}

```

```

}
.admin-actions { display: flex; gap: 12px; }
.btn-outline-light {
  background: var(--mc-btn-bg);
  border: 1px solid var(--mc-btn-border);
  padding: 0.5rem 1rem;
  border-radius: 60px;
  font-weight: 500;
  font-size: 0.85rem;
  color: var(--mc-btn-text);
  transition: all 0.2s;
  cursor: pointer;
  text-decoration: none;
}
.btn-outline-light:hover {
  background: var(--mc-btn-hover-bg);
  border-color: var(--mc-btn-hover-border);
  color: var(--mc-btn-text);
}
.modern-card {
  background: var(--mc-card-bg);
  border-radius: 28px;
  box-shadow: 0 8px 20px rgba(0,0,0,0.02), 0 2px 6px rgba(0,0,0,0.05);
  margin-bottom: 1.5rem;
  overflow: hidden;
  border: 1px solid var(--mc-card-border);
}
.modern-card-header {
  padding: 1.2rem 1.8rem;
  border-bottom: 1px solid var(--mc-card-header-border);
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 1rem;
}
.modern-card-header h2 {
  font-size: 1.3rem;
  font-weight: 600;
  color: var(--mc-text);
  display: flex;
  align-items: center;
  gap: 10px;
  margin: 0;
}
.modern-card-header h2 i { color: #2c6e9e; }
.form-grid {
  padding: 1.4rem 1.8rem 1.8rem;
  background: var(--mc-form-bg);
  border-bottom: 1px solid var(--mc-form-border);
}
.row-inputs {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
  align-items: flex-end;
}
.input-group { flex: 1 1 200px; min-width: 170px; }
.input-group label {
  display: block;
  font-size: 0.72rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.5px;
  color: var(--mc-muted);
  margin-bottom: 6px;
}
.input-group input,
.input-group select {
  width: 100%;
  padding: 0.7rem 0.9rem;
  border-radius: 16px;
  border: 1px solid var(--mc-input-border);
  background: var(--mc-input-bg);
  color: var(--mc-text);
  font-size: 0.9rem;
  transition: 0.2s;
  outline: none;
}
.input-group input:focus,

```

```

.input-group select:focus {
  border-color: #2c6e9e;
  box-shadow: 0 0 0 3px rgba(44,110,158,0.2);
}
.status-toggle {
  display: flex;
  align-items: center;
  gap: 12px;
  background: var(--mc-form-bg);
  padding: 0.45rem 1rem;
  border-radius: 40px;
  border: 1px solid var(--mc-form-border);
  margin-bottom: 0.4rem;
}
.status-toggle span { font-size: 0.85rem; font-weight: 500; }
.radio-group { display: flex; gap: 10px; }
.radio-group label {
  display: flex;
  align-items: center;
  gap: 5px;
  font-weight: 500;
  text-transform: none;
  font-size: 0.85rem;
  margin: 0;
}
.btn-save {
  background: #1f6390;
  border: none;
  padding: 0.7rem 1.5rem;
  border-radius: 40px;
  font-weight: 600;
  color: #fff;
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 0.85rem;
}
.btn-save:hover { background: #0f4b6e; color: #fff; }
.table-wrapper { overflow-x: auto; }
.colleges-table { width: 100%; border-collapse: collapse; font-size: 0.88rem; }
.colleges-table th {
  text-align: left;
  padding: 1rem 1.2rem;
  background-color: var(--mc-table-head-bg);
  font-weight: 600;
  color: var(--mc-table-head-text);
  border-bottom: 1px solid var(--mc-form-border);
  font-size: 0.8rem;
}
.colleges-table td {
  padding: 1rem 1.2rem;
  border-bottom: 1px solid var(--mc-table-row-border);
  vertical-align: middle;
  color: var(--mc-row-text);
}
.colleges-table tr:hover td { background-color: var(--mc-table-row-hover); }
.status-badge {
  display: inline-flex;
  align-items: center;
  background: #e3f6ec;
  color: #1e7b48;
  padding: 0.2rem 0.7rem;
  border-radius: 30px;
  font-size: 0.72rem;
  font-weight: 600;
  gap: 6px;
}
.status-badge.inactive { background: #ffe6e5; color: #b91c1c; }
.unassigned-text {
  color: #b38f40;
  background: #fff1e0;
  padding: 0.2rem 0.6rem;
  border-radius: 50px;
  font-size: 0.72rem;
  font-weight: 500;
  display: inline-block;
}
.action-icons {
  display: inline-flex;
  align-items: center;
}

```

```

        gap: 8px;
    }
    .action-icon-btn {
        border: none;
        background: transparent;
        color: #5c7f9c;
        width: 32px;
        height: 32px;
        border-radius: 20px;
        cursor: pointer;
    }
    .action-icon-btn:hover { background: #eef3fc; color: #1f6390; }
    .action-icon-btn.delete:hover { color: #d9534f; }
    .empty-row td {
        text-align: center;
        padding: 2rem;
        color: var(--mc-empty-text);
        font-style: italic;
    }
    }
    .toast-msg {
        position: fixed;
        bottom: 20px;
        right: 20px;
        background: #1e2f3e;
        color: #fff;
        padding: 12px 20px;
        border-radius: 60px;
        font-size: 0.85rem;
        font-weight: 500;
        display: flex;
        align-items: center;
        gap: 10px;
        box-shadow: 0 8px 20px rgba(0,0,0,0.12);
        z-index: 1000;
        opacity: 0;
        pointer-events: none;
        transition: opacity 0.2s ease;
    }
    .toast-msg.show { opacity: 1; }

    @media (max-width: 700px) {
        .row-inputs { flex-direction: column; align-items: stretch; }
        .modern-card-header { flex-direction: column; align-items: flex-start; }
    }
</style>

<div class="dashboard-container">
    <div class="page-header">
        <div class="title-section">
            <h1><i class="fa-solid fa-building-columns"></i> Manage Colleges</h1>
            <div class="badge-role">
                <i class="fa-solid fa-user-shield"></i> System Administrator · Western Philippines University
            </div>
        </div>
        <div class="admin-actions">
            <a class="btn-outline-light" href="admin_colleges.php"><i class="fa-solid fa-arrows-rotate"></i> Sync</a>
        </div>
    </div>

    <div class="modern-card">
        <div class="modern-card-header">
            <h2><i class="fa-solid fa-plus-circle"></i> <?= $edit ? 'Edit College' : 'Add New College' ?></h2>
            <span style="font-size:.75rem;color:#5e7e9e;"><i class="fa-solid fa-circle-info"></i> Fill all required fields</span>
        </div>
        <div class="form-grid">
            <form method="post" class="row-inputs">
                <input type="hidden" name="action" value="<?= $edit ? 'edit' : 'add' ?>">
                <?php if ($edit): ?><input type="hidden" name="id" value="<?= (int) $edit['id'] ?>"><?php endif; ?>

                <div class="input-group">
                    <label><i class="fa-solid fa-hashtag"></i> Code</label>
                    <input name="college_code" maxlength="20" required autocomplete="off" value="<?= htmlspecialchars((string) ($edit['college_code'] ?? '')) ?>">
                </div>
                <div class="input-group" style="flex:2 1 350px;">
                    <label><i class="fa-solid fa-font"></i> Name</label>
                    <input name="college_name" required autocomplete="off" value="<?= htmlspecialchars((string) ($edit['college_name'] ?? '')) ?>">
                </div>
                <div class="input-group">

```

```

        <label><i class="fa-solid fa-chalkboard-user"></i> Assigned Dean</label>
        <select name="dean_user_id">
            <option value="">Unassigned</option>
            <?php foreach ($deans as $d): ?>
                <option value="<?=(int) $d['id'] ?>" <?=(int) ($edit['dean_user_id'] ?? 0) === (int) $d['id'] ? 'selected' :
'' ?>><?=> htmlspecialchars($d['full_name']) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <div>
        <label style="display:block;font-size:.72rem;font-weight:600;text-transform:uppercase;letter-
spacing:.5px;color:#54708f;margin-bottom:6px;">Status</label>
        <div class="status-toggle">
            <span><i class="fa-solid fa-circle-info"></i></span>
            <div class="radio-group">
                <label><input type="radio" name="status" value="active" <?=( $edit['status'] ?? 'active') === 'active' ?
'checked' : '' ?>> Active</label>
                <label><input type="radio" name="status" value="inactive" <?=( $edit['status'] ?? '') === 'inactive' ? 'checked'
: '' ?>> Inactive</label>
            </div>
        </div>
    </div>
    <div>
        <button type="submit" class="btn-save"<?=> app_tooltip_attr($edit ? 'Saves changes to this college record.' : 'Creates the
college entry for deans and scheduling scope.') ?>>
            <i class="fa-solid <?=> $edit ? 'fa-pen' : 'fa-save' ?>"></i> <?=> $edit ? 'Update College' : 'Save College' ?>
        </button>
    </div>
</div>
</div>

<div class="modern-card">
    <div class="modern-card-header">
        <h2><i class="fa-solid fa-list-ul"></i> All Colleges</h2>
        <div><i class="fa-solid fa-school"></i> Total: <strong><?=> count($rows) ?></strong> records</div>
    </div>
    <div class="table-wrapper">
        <table class="colleges-table">
            <thead>
                <tr><th>Code</th><th>Name</th><th>Dean</th><th>Status</th><th style="width: 100px;">Actions</th></tr>
            </thead>
            <tbody>
                <?php if (!$rows): ?>
                    <tr class="empty-row"><td colspan="5">No colleges added yet.</td></tr>
                <?php endif; ?>
                <?php foreach ($rows as $r): ?>
                    <?php $isActive = ((string) $r['status']) === 'active'; ?>
                    <tr>
                        <td><strong><?=> htmlspecialchars($r['college_code']) ?></strong></td>
                        <td><?=> htmlspecialchars($r['college_name']) ?></td>
                        <td>
                            <?php if (empty($r['dean_name'])): ?>
                                <span class="unassigned-text"><i class="fa-solid fa-user-graduate"></i> Unassigned</span>
                            <?php else: ?>
                                <span><i class="fa-solid fa-user-tie"></i> <?=> htmlspecialchars((string) $r['dean_name']) ?></span>
                            <?php endif; ?>
                        </td>
                        <td>
                            <span class="status-badge<?=> $isActive ? '' : ' inactive' ?>">
                                <i class="fa-solid <?=> $isActive ? 'fa-check-circle' : 'fa-ban' ?>"></i> <?=> $isActive ? 'active' :
'inactive' ?>
                            </span>
                        </td>
                        <td>
                            <div class="action-icons">
                                <a href="admin_colleges.php?edit=<?=(int) $r['id'] ?>" class="action-icon-btn" title="Edit College"<?=>
app_tooltip_attr('Edits college code, name, or assigned dean.') ?>>
                                    <i class="fa-solid fa-pen"></i>
                                </a>
                                <form method="post" class="d-inline" onsubmit="return confirm('Delete this college?');">
                                    <input type="hidden" name="action" value="delete">
                                    <input type="hidden" name="id" value="<?=(int) $r['id'] ?>">
                                    <button type="submit" class="action-icon-btn delete" title="Delete College"<?=>
app_tooltip_attr('Deletes this college after confirmation when safe for your data.') ?>>
                                        <i class="fa-solid fa-trash-alt"></i>
                                    </button>
                                </form>
                            </div>
                        </td>
                    </tr>
                <?php endforeach; ?>
            </tbody>
        </table>
    </div>

```



```

        </tbody>
    </table>
</div>
</div>
</div>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: admin_deans.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/mail_helpers.php';
require_once __DIR__ . '/includes/account_registration_helpers.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['admin']);
$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);
$hasUserEmail = db_column_exists('users', 'email');

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if ($action === 'add') {
            $username = trim((string) $_POST['username']);
            $fullName = trim((string) $_POST['full_name']);
            $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
            $password = (string) ($_POST['password'] ?? '');
            $hasValidEmail = $hasUserEmail && $email !== '' && filter_var($email, FILTER_VALIDATE_EMAIL);

            if ($username === '' || $fullName === '') {
                throw new RuntimeException('Username and full name are required.');
            }
            if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
                throw new RuntimeException('Invalid email address.');
            }

            $cred = prepare_new_account_password($password, $hasValidEmail);
            $password = $cred['plain'];
            $plainForMail = $hasValidEmail ? $cred['plain'] : '';

            $exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username=?');
            $exists->execute([$username]);
            if ((int) $exists->fetchColumn() > 0) {
                $s = suggest_available_usernames($username, 3);
                $msg = 'Username already exists.';
                if ($s) {
                    $msg .= ' Try: ' . implode(', ', $s);
                }
                throw new RuntimeException($msg);
            }
        }

        if ($hasUserEmail) {
            db()->prepare('INSERT INTO users (username,password,full_name,email,role,college_id,is_active) VALUES (?,?,,?,,?)')
            ->execute([
                $username,
                $cred['hash'],
                $fullName,
                $email,
                'dean',
                ($_POST['college_id'] ?? '') !== '' ? (int) $_POST['college_id'] : null,
                !empty($_POST['is_active']) ? 1 : 0,
            ]);
        } else {
            db()->prepare('INSERT INTO users (username,password,full_name,role,college_id,is_active) VALUES (?,?,,?,,?)')
            ->execute([
                $username,
                $cred['hash'],
                $fullName,
                'dean',
                ($_POST['college_id'] ?? '') !== '' ? (int) $_POST['college_id'] : null,
            ]);
        }
    } catch (Exception $e) {
        $flash = $e->getMessage();
    }
}

```

```

        !empty($_POST['is_active']) ? 1 : 0,
    ];
}
$newId = (int) db()->lastInsertId();
if (!empty($_POST['college_id'])) {
    db()->prepare('UPDATE colleges SET dean_user_id=? WHERE id=?')->execute([$newId, (int) $_POST['college_id']]);
}

$stmtAfter = db()->prepare(
    'SELECT id, username, full_name, email, role, college_id, is_active FROM users WHERE id=? LIMIT 1'
);
$stmtAfter->execute([$newId]);
$afterRow = $stmtAfter->fetch(PDO::FETCH_ASSOC) ?: [];
if ($afterRow !== []) {
    $afterRow['password'] = '[set at creation]';
}
log_admin_activity('add', 'Deans', 'Dean user #' . $newId, null, $afterRow !== [] ? $afterRow : null);

$mailOk = $hasValidEmail
    ? notify_new_account_credentials($newId, $email, $fullName, $username, $plainForMail, 'dean')
    : null;
if (!$hasValidEmail) {
    mark_new_account_requires_password_change($newId);
}
$_SESSION['flash'] = registration_email_flash_message($mailOk, $email, 'Dean account');
} elseif ($action === 'edit') {
    $id = (int) $_POST['id'];
    $fullName = trim((string) $_POST['full_name']);
    $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
    $resetPassword = trim((string) ($_POST['reset_password'] ?? ''));
    $generateAndEmail = !empty($_POST['generate_temp_password_email']);

    if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        throw new RuntimeException('Invalid email address.');
```

```

        $id,
    ));
    if (!empty($_POST['email_reset_password']) && $hasUserEmail && $email !== '' && filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $plainForMail = $resetPassword;
    }
}

if ($plainForMail !== '') {
    $mailOk = send_account_credentials_mail($email, $displayName, $uname, $plainForMail, 'dean', $email);
    $_SESSION['flash'] = credentials_email_flash_message($mailOk, $email, 'Dean updated');
} else {
    $_SESSION['flash'] = 'Dean account updated.';
}

$stmtAfter = db()->prepare(
    'SELECT id, username, full_name, email, role, college_id, is_active FROM users WHERE id=? AND role="dean" LIMIT 1'
);
$stmtAfter->execute([$id]);
$afterDean = $stmtAfter->fetch(PDO::FETCH_ASSOC) ?: [];
if ($afterDean !== []) {
    if ($generateAndEmail || $resetPassword !== '') {
        $afterDean['password'] = '[changed]';
    }
}
log_admin_activity(
    'edit',
    'Deans',
    'Dean user #' . $id,
    $beforeDean ? (array) $beforeDean : null,
    $afterDean !== [] ? $afterDean : null
);
} elseif ($action === 'delete') {
    $id = (int) ($_POST['id'] ?? 0);
    if ($id < 1) {
        throw new RuntimeException('Invalid dean account.');
```

```
'SELECT u.*, c.college_name
FROM users u
LEFT JOIN colleges c ON c.id = u.college_id
WHERE u.role="dean"
ORDER BY u.full_name'
)->fetchAll();
```

```
$pageTitle = 'Manage Deans';
require_once __DIR__ . '/includes/header.php';
?>
```

```
<style>
.deans-dashboard {
    color: #0f172a;
}

.deans-top {
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: 1rem;
    margin-bottom: 1.5rem;
}

.deans-title h1 {
    margin: 0;
    font-size: 1.8rem;
    font-weight: 700;
}

.deans-title p {
    margin: 0.2rem 0 0;
    color: #475569;
    font-size: 0.92rem;
}

.deans-add-btn {
    border: 0;
    background: #1e462f;
    color: #fff;
    border-radius: 999px;
    padding: 0.65rem 1.15rem;
    font-weight: 600;
}

.deans-add-btn:hover {
    background: #163624;
}

.dean-form-card {
    border-radius: 22px;
    border: 1px solid #e6ebf2;
    box-shadow: 0 10px 24px rgba(15, 23, 42, 0.05);
    overflow: hidden;
}

.dean-form-header {
    border-bottom: 1px solid #edf2f7;
    background: #fbfdff;
    padding: 1rem 1.4rem;
    font-weight: 600;
}

.dean-form-body {
    padding: 1.4rem;
}

.dean-form-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
    gap: 1rem;
}

.dean-form-grid .form-label {
    font-size: 0.78rem;
    letter-spacing: 0.4px;
    text-transform: uppercase;
    color: #334155;
    font-weight: 700;
    margin-bottom: 0.45rem;
}
```

```

}

.dean-form-grid .form-control,
.dean-form-grid .form-select {
  border-radius: 14px;
  border: 1px solid #cbd5e1;
  padding: 0.6rem 0.85rem;
}

.dean-form-grid .form-control:focus,
.dean-form-grid .form-select:focus {
  border-color: #2c6e3c;
  box-shadow: 0 0 0 0.2rem rgba(44, 110, 60, 0.15);
}

.dean-form-actions {
  border-top: 1px solid #edf2f7;
  margin-top: 1.2rem;
  padding-top: 1rem;
  display: flex;
  justify-content: flex-end;
  gap: 0.75rem;
}

.deans-table-card {
  border-radius: 22px;
  border: 1px solid #e6ebf2;
  overflow: hidden;
  box-shadow: 0 4px 14px rgba(15, 23, 42, 0.03);
}

.deans-table {
  margin: 0;
}

.deans-table thead th {
  font-size: 0.82rem;
  letter-spacing: 0.2px;
  text-transform: uppercase;
  background: #f8fbff;
  color: #1e293b;
  border-bottom: 1px solid #e2e8f0;
}

.deans-table tbody td {
  vertical-align: middle;
}

.status-pill {
  display: inline-block;
  padding: 0.23rem 0.7rem;
  border-radius: 999px;
  font-size: 0.72rem;
  font-weight: 700;
}

.status-pill.active {
  background: #dcfce7;
  color: #166534;
}

.status-pill.inactive {
  background: #f1f5f9;
  color: #475569;
}

.deans-action {
  border: none;
  background: transparent;
  color: #64748b;
  padding: 0.2rem 0.35rem;
}

.deans-action:hover {
  color: #1f2937;
}

.deans-action.delete:hover {
  color: #b91c1c;
}

```

```

</style>

<div class="deans-dashboard">
  <div class="deans-top">
    <div class="deans-title">
      <h1><i class="fa-solid fa-user-tie me-2 text-success"></i>Manage Deans</h1>
      <p>Add, edit, or manage college deans across all campuses.</p>
    </div>
    <?php if (!$edit): ?>
      <button id="showDeanFormBtn" type="button" class="deans-add-btn"><i class="fa-solid fa-plus-circle me-1"></i>Add Dean</button>
    <?php endif; ?>
  </div>

  <?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
  <?php if (!$hasUserEmail): ?>
    <div class="alert alert-warning">The <code>users.email</code> column is missing. Run <a
href="upgrade_roles.php">upgrade_roles.php</a> once to add it, then reload this page.</div>
  <?php endif; ?>

  <div id="deanFormCard" class="card dean-form-card mb-4"<?=$edit ? ' ' : ' ' style="display:none;" ?>>
    <div class="dean-form-header">
      <i class="fa-solid fa-user-pen me-2 text-success"></i><?=$edit ? 'Edit Dean' : 'Add New Dean' ?>
    </div>
    <div class="dean-form-body">
      <form method="post" id="deanForm">
        <input type="hidden" name="action" value="<?=$edit ? 'edit' : 'add' ?>">
        <?php if ($edit): ?><input type="hidden" name="id" value="<?=(int) $edit['id'] ?>"><?php endif; ?>

        <div class="dean-form-grid">
          <div>
            <label class="form-label">Username</label>
            <input name="username" class="form-control" <?=$edit ? 'readonly' : 'required' ?> value="<?=
htmlspecialchars((string) ($edit['username'] ?? '')) ?>">
          </div>
          <div>
            <label class="form-label">Full Name</label>
            <input name="full_name" class="form-control" required value="<?= htmlspecialchars((string) ($edit['full_name'] ??
'')) ?>">
          </div>
          <?php if ($hasUserEmail): ?>
          <div>
            <label class="form-label">Email</label>
            <input type="email" name="email" class="form-control" placeholder="dean@school.edu" value="<?=
htmlspecialchars((string) ($edit['email'] ?? '')) ?>">
            <div class="form-text">Used to send temporary passwords.</div>
          </div>
          <?php endif; ?>
          <?php if (!$edit): ?>
          <div>
            <label class="form-label">Password</label>
            <input name="password" id="deanPassword" class="form-control" autocomplete="new-password" placeholder="If not
emailing">
            <?php if ($hasUserEmail): ?>
            <div class="form-check mt-2">
              <input type="checkbox" class="form-check-input" name="send_credentials_email" id="send_credentials_email"
value="1" checked>
              <label class="form-check-label" for="send_credentials_email">Generate temporary password and email it</label>
            </div>
            <?php endif; ?>
          </div>
          <?php else: ?>
          <div>
            <label class="form-label">Reset Password (Optional)</label>
            <input name="reset_password" id="deanResetPassword" class="form-control" autocomplete="new-password"
placeholder="New password">
            <?php if ($hasUserEmail): ?>
            <div class="form-check mt-2">
              <input type="checkbox" class="form-check-input" name="email_reset_password" id="email_reset_password" value="1">
              <label class="form-check-label" for="email_reset_password">Email this new password to the dean</label>
            </div>
            <div class="form-check mt-1">
              <input type="checkbox" class="form-check-input" name="generate_temp_password_email"
id="generate_temp_password_email" value="1">
              <label class="form-check-label" for="generate_temp_password_email">Generate temporary password and email
it</label>
            </div>
            <?php endif; ?>
          </div>
          <?php endif; ?>
        </div>
      </form>
    </div>
  </div>

```

```

        <label class="form-label">College</label>
        <select name="college_id" class="form-select">
            <option value="">Unassigned</option>
            <?php foreach ($colleges as $c): ?>
                <option value="<?= (int) $c['id'] ?>" <?= (int) ($edit['college_id'] ?? 0) === (int) $c['id'] ? 'selected' :
'' ?>>
                    <?= htmlspecialchars((string) $c['college_name']) ?>
                </option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="d-flex align-items-end">
        <div class="form-check mb-2">
            <input type="checkbox" class="form-check-input" name="is_active" id="is_active" value="1" <?=
isset($edit['is_active']) || (int) $edit['is_active'] === 1 ? 'checked' : '' ?>>
            <label class="form-check-label" for="is_active">Active</label>
        </div>
    </div>
</div>

<div class="dean-form-actions">
    <?php if (!$edit): ?>
        <button id="cancelDeanFormBtn" type="button" class="btn btn-light border">Cancel</button>
    <?php else: ?>
        <a class="btn btn-light border" href="admin_deans.php">Cancel</a>
    <?php endif; ?>
    <button class="btn btn-success" type="submit"><?= app_tooltip_attr($edit ? 'Saves changes to this dean account.' :
'Creates the dean user and links them to a college.') ?>>
        <i class="fa-solid fa-floppy-disk me-1"></i>Save Dean
    </button>
</div>
</form>
</div>
</div>

<div class="card deans-table-card">
    <div class="table-responsive">
        <table class="table deans-table">
            <thead>
                <tr>
                    <th>Username</th>
                    <th>Name</th>
                    <?php if ($hasUserEmail): ?><th>Email</th><?php endif; ?>
                    <th>College</th>
                    <th>Status</th>
                    <th style="width: 90px;">Actions</th>
                </tr>
            </thead>
            <tbody>
                <?php if (!$deans): ?>
                    <tr><td colspan="<?= $hasUserEmail ? '6' : '5' ?>" class="text-center text-muted py-4">No deans found.</td></tr>
                <?php endif; ?>
                <?php foreach ($deans as $d): ?>
                    <tr>
                        <td><strong><?= htmlspecialchars($d['username']) ?></strong></td>
                        <td><?= htmlspecialchars($d['full_name']) ?></td>
                        <?php if ($hasUserEmail): ?><td><?= htmlspecialchars((string) ($d['email'] ?? '')) ?></td><?php endif; ?>
                        <td><?= htmlspecialchars((string) ($d['college_name'] ?? 'Unassigned')) ?></td>
                        <td>
                            <span class="status-pill <?= (int) $d['is_active'] === 1 ? 'active' : 'inactive' ?>">
                                <?= (int) $d['is_active'] === 1 ? 'Active' : 'Inactive' ?>
                            </span>
                        </td>
                        <td>
                            <a class="deans-action" href="admin_deans.php?edit=<?= (int) $d['id'] ?>" title="Edit Dean"><?=
app_tooltip_attr('Edits this dean profile and account settings.') ?>>
                                <i class="fa-solid fa-pen-to-square"></i>
                            </a>
                            <form method="post" class="d-inline" onsubmit="return confirm('Delete this dean account?');">
                                <input type="hidden" name="action" value="delete">
                                <input type="hidden" name="id" value="<?= (int) $d['id'] ?>">
                                <button type="submit" class="deans-action delete" title="Delete Dean"><?= app_tooltip_attr('Removes this dean
account after confirmation.') ?>>
                                    <i class="fa-solid fa-trash"></i>
                                </button>
                            </form>
                        </td>
                    </tr>
                <?php endforeach; ?>
            </tbody>
        </table>
    </div>
</div>

```

```

        </table>
    </div>
</div>
</div>

<script>
(function () {
    const isEdit = <?=$edit ? 'true' : 'false' ?>;
    const formCard = document.getElementById('deanFormCard');
    const showBtn = document.getElementById('showDeanFormBtn');
    const cancelBtn = document.getElementById('cancelDeanFormBtn');
    const sendCredentials = document.getElementById('send_credentials_email');
    const deanPassword = document.getElementById('deanPassword');
    const generateTempPasswordEmail = document.getElementById('generate_temp_password_email');
    const deanResetPassword = document.getElementById('deanResetPassword');
    const emailResetPassword = document.getElementById('email_reset_password');

    if (showBtn && formCard) {
        showBtn.addEventListener('click', function () {
            formCard.style.display = 'block';
            showBtn.style.display = 'none';
            window.scrollTo({ top: formCard.offsetTop - 12, behavior: 'smooth' });
        });
    }

    if (cancelBtn && formCard && showBtn) {
        cancelBtn.addEventListener('click', function () {
            formCard.style.display = 'none';
            showBtn.style.display = 'inline-block';
        });
    }

    if (!isEdit && sendCredentials && deanPassword) {
        const syncAddPasswordState = function () {
            deanPassword.required = !sendCredentials.checked;
            deanPassword.disabled = sendCredentials.checked;
            if (sendCredentials.checked) {
                deanPassword.value = '';
            }
        };
        sendCredentials.addEventListener('change', syncAddPasswordState);
        syncAddPasswordState();
    }

    if (isEdit && generateTempPasswordEmail && deanResetPassword) {
        const syncEditPasswordState = function () {
            const disableManual = generateTempPasswordEmail.checked;
            deanResetPassword.disabled = disableManual;
            if (disableManual) {
                deanResetPassword.value = '';
                if (emailResetPassword) {
                    emailResetPassword.checked = false;
                }
            }
        };
        generateTempPasswordEmail.addEventListener('change', syncEditPasswordState);
        syncEditPasswordState();
    }
})();
</script>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: admin_gened.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/mail_helpers.php';
require_once __DIR__ . '/includes/account_registration_helpers.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['admin']);
$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);
$hasUserEmail = db_column_exists('users', 'email');

```



```

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    try {
        $st = db()->prepare('SELECT id FROM users WHERE role="gened" ORDER BY id LIMIT 1');
        $st->execute();
        $id = (int) ($st->fetchColumn() ?: 0);
        if ($id < 1) {
            throw new RuntimeException('GEN ED account not found. Run upgrade_roles.php first.');
```

```

$afterFull = $stAfter->fetch(PDO::FETCH_ASSOC) ?: [];
if ($afterFull !== [] && ($generateAndEmail || $resetPassword !== '')) {
    $afterFull['password'] = '[changed]';
}
log_admin_activity(
    'edit',
    'GEN ED account',
    'GEN ED user #' . $id,
    (array) $beforeFull,
    $afterFull !== [] ? $afterFull : null
);
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: admin_gened.php');
exit;
}

$gened = db()->query('SELECT * FROM users WHERE role="gened" ORDER BY id LIMIT 1')->fetch() ?: null;
$pageTitle = 'GEN ED Account';
require_once __DIR__ . '/includes/header.php';
?>
<h1 class="h3 mb-4"><i class="fa-solid fa-user-gear me-2 text-primary"></i>GEN ED Admin Account</h1>
<p class="small text-muted">Institution-wide General Education coordinator (GE schedule, GE faculty, GE courses).</p>
<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
<?php if (!$hasUserEmail): ?>
    <div class="alert alert-warning">The <code>users.email</code> column is missing. Run <a href="upgrade_roles.php">upgrade_roles.php</a>
to enable email for credentials.</div>
<?php endif; ?>
<?php if (!$gened): ?>
    <div class="alert alert-warning">No GEN ED account found. Run <a href="upgrade_roles.php">upgrade_roles.php</a> first.</div>
<?php else: ?>
<div class="card shadow-sm">
    <div class="card-body">
        <form method="post" class="row g-3">
            <div class="col-md-4"><label class="form-label">Username</label><input name="username" class="form-control" required value="<?=
htmlspecialchars((string) $gened['username']) ?>"></div>
            <div class="col-md-4"><label class="form-label">Full Name</label><input name="full_name" class="form-control" required value="<?=
htmlspecialchars((string) $gened['full_name']) ?>"></div>
            <?php if ($hasUserEmail): ?>
                <div class="col-md-4"><label class="form-label">Email</label><input type="email" name="email" class="form-control" value="<?=
htmlspecialchars((string) ($gened['email'] ?? '')) ?>"></div>
            <?php endif; ?>
            <div class="col-md-4"><label class="form-label">Reset Password (optional)</label><input name="reset_password" class="form-
control" autocomplete="new-password"></div>
            <?php if ($hasUserEmail): ?>
                <div class="col-md-4 d-flex flex-column justify-content-end">
                    <div class="form-check">
                        <input class="form-check-input" type="checkbox" name="email_reset_password" id="ag_email_reset_password" value="1">
                        <label class="form-check-label" for="ag_email_reset_password">Email the manual password above</label>
                    </div>
                    <div class="form-check mt-1">
                        <input class="form-check-input" type="checkbox" name="generate_temp_password_email"
id="ag_generate_temp_password_email" value="1">
                        <label class="form-check-label" for="ag_generate_temp_password_email">Generate temporary password and email
it</label>
                    </div>
                </div>
            <?php endif; ?>
            <div class="col-12"><div class="form-check"><input class="form-check-input" type="checkbox" name="is_active" value="1" <?= (int)
$gened['is_active'] === 1 ? 'checked' : '' ?>><label class="form-check-label">Active</label></div></div>
            <div class="col-12"><button type="submit" class="btn btn-primary"><?= app_tooltip_attr('Saves the GEN ED coordinator account and
optional password email.') ?>>Save</button></div>
        </form>
    </div>
</div>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

-----
FILE: admin_login_slideshow.php
-----

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

```

```

require_once __DIR__ . '/includes/login_slideshow_helpers.php';

require_role(['admin']);

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);
$tableReady = login_slideshow_table_ready();
$userId = (int) ($_SESSION['user_id'] ?? 0);

if ($_SERVER['REQUEST_METHOD'] === 'POST' && $tableReady) {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if ($action === 'upload') {
            $caption = trim((string) ($_POST['caption'] ?? ''));
            $result = login_slideshow_store_uploads($_FILES['slide_images'] ?? [], $caption, $userId);
            foreach ($result['ids'] as $newId) {
                log_admin_activity('add', 'Login slideshow', 'Slide #' . $newId, null, ['caption' => $caption]);
            }
            $msg = $result['uploaded'] === 1
                ? '1 image added to the login slideshow.'
                : $result['uploaded'] . ' images added to the login slideshow.';
            if ($result['errors'] !== []) {
                $msg .= ' Some files were skipped: ' . implode(' ', $result['errors']);
            }
            $_SESSION['flash'] = $msg;
        } elseif ($action === 'delete') {
            $id = (int) ($_POST['id'] ?? 0);
            $row = login_slideshow_image_row($id);
            login_slideshow_delete($id);
            log_admin_activity('delete', 'Login slideshow', 'Slide #' . $id, $row ? (array) $row : null, null);
            $_SESSION['flash'] = 'Image removed from the login slideshow.';
        } elseif ($action === 'toggle') {
            $id = (int) ($_POST['id'] ?? 0);
            $active = !empty($_POST['is_active']);
            $before = login_slideshow_image_row($id);
            login_slideshow_set_active($id, $active);
            $after = login_slideshow_image_row($id);
            log_admin_activity(
                'edit',
                'Login slideshow',
                'Slide #' . $id,
                $before ? (array) $before : null,
                $after ? (array) $after : null
            );
            $_SESSION['flash'] = $active ? 'Image is now visible on the login page.' : 'Image hidden from the login page.';
        } elseif ($action === 'caption') {
            $id = (int) ($_POST['id'] ?? 0);
            $caption = trim((string) ($_POST['caption'] ?? ''));
            $before = login_slideshow_image_row($id);
            login_slideshow_update_caption($id, $caption);
            $after = login_slideshow_image_row($id);
            log_admin_activity(
                'edit',
                'Login slideshow',
                'Slide #' . $id . ' caption',
                $before ? (array) $before : null,
                $after ? (array) $after : null
            );
            $_SESSION['flash'] = 'Caption updated.';
        } elseif ($action === 'move_up' || $action === 'move_down') {
            $id = (int) ($_POST['id'] ?? 0);
            login_slideshow_move($id, $action === 'move_up' ? 'up' : 'down');
            $_SESSION['flash'] = 'Slide order updated.';
        } else {
            throw new RuntimeException('Unknown action.');
```

Only active images are shown, in the order listed below.

</p>

```
<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
```

```
<?php if (!$tableReady): ?>
```

```
<div class="alert alert-warning">
```

The slideshow table is missing. Run upgrade_roles.php once to enable this feature.

```
</div>
```

```
<?php else: ?>
```

```
<div class="card shadow-sm mb-4">
```

```
<div class="card-header bg-white fw-semibold">Add images</div>
```

```
<div class="card-body">
```

```
<form method="post" enctype="multipart/form-data" class="row g-3">
```

```
<input type="hidden" name="action" value="upload">
```

```
<div class="col-md-6">
```

```
<label class="form-label" for="slide_images">Images</label>
```

```
<input type="file" name="slide_images[]" id="slide_images" class="form-control" accept="image/jpeg,image/png,image/webp"
```

```
multiple required>
```

```
<div class="form-text">Select one or more images (up to 20 at a time). JPEG, PNG, or WebP, max 5&nbsp;MB each. Images
```

```
are scaled to fit the login panel.</div>
```

```
</div>
```

```
<div class="col-md-4">
```

```
<label class="form-label" for="caption">Caption (optional)</label>
```

```
<input type="text" name="caption" id="caption" class="form-control" maxlength="255" placeholder="Applied to all images
```

```
in this batch">
```

```
</div>
```

```
<div class="col-md-2 d-flex align-items-end">
```

```
<button type="submit" class="btn btn-primary w-100"<? app_tooltip_attr('Uploads selected images to the login page
```

```
slideshow.') ?>>
```

```
<i class="fa-solid fa-upload me-1"></i>Upload
```

```
</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
<?php if ($slides === []): ?>
```

```
<div class="alert alert-secondary">No slideshow images yet. Upload one above to fill the login page hero area.</div>
```

```
<?php else: ?>
```

```
<div class="row g-3">
```

```
<?php foreach ($slides as $i => $slide):
```

```
$slideId = (int) $slide['id'];
```

```
$stored = trim((string) ($slide['stored_name'] ?? ''));
```

```
$previewUrl = is_file(login_slideshow_path($stored))
```

```
? 'login_slideshow_image.php?id=' . $slideId . '&preview=1'
```

```
: '';
```

```
$isActive = (int) ($slide['is_active'] ?? 0) === 1;
```

```
?>
```

```
<div class="col-12">
```

```
<div class="card shadow-sm">
```

```
<div class="card-body">
```

```
<div class="row g-3 align-items-center">
```

```
<div class="col-md-3 col-lg-2">
```

```
<?php if ($previewUrl !== '' && $isActive): ?>
```

```

```

```
<?php elseif ($previewUrl !== ''): ?>
```

```

```

```
<?php else: ?>
```

```
<div class="border rounded text-muted small p-3 text-center">File missing</div>
```

```
<?php endif; ?>
```

```
</div>
```

```
<div class="col-md-5 col-lg-6">
```

```
<form method="post" class="row g-2">
```

```
<input type="hidden" name="action" value="caption">
```

```
<input type="hidden" name="id" value="<? $slideId ?>">
```

```
<div class="col-12">
```

```
<label class="form-label small mb-1">Caption</label>
```

```
<div class="input-group input-group-sm">
```

```
<input type="text" name="caption" class="form-control" maxlength="255" value="<? =
```

```
htmlspecialchars((string) ($slide['caption'] ?? '')) ?>">
```

```
<button type="submit" class="btn btn-outline-secondary">Save</button>
```

```
</div>
```

```
</form>
```

```
<div class="small text-muted mt-2">
```

```
Order #<? $i + 1 ?>
```

```
<?php if (!$isActive): ?>
```

```

        <span class="badge text-bg-secondary ms-1">Hidden</span>
        <?php endif; ?>
    </div>
</div>
<div class="col-md-4 col-lg-4">
    <div class="d-flex flex-wrap gap-2 justify-content-md-end">
        <form method="post" class="d-inline">
            <input type="hidden" name="action" value="move_up">
            <input type="hidden" name="id" value="<?= $slideId ?>">
            <button type="submit" class="btn btn-sm btn-outline-secondary"<?= $i == 0 ? ' disabled' : '' ?>
title="Move earlier">↑</button>
        </form>
        <form method="post" class="d-inline">
            <input type="hidden" name="action" value="move_down">
            <input type="hidden" name="id" value="<?= $slideId ?>">
            <button type="submit" class="btn btn-sm btn-outline-secondary"<?= $i == count($slides) - 1 ? '
disabled' : '' ?> title="Move later">↓</button>
        </form>
        <form method="post" class="d-inline">
            <input type="hidden" name="action" value="toggle">
            <input type="hidden" name="id" value="<?= $slideId ?>">
            <input type="hidden" name="is_active" value="<?= $isActive ? '0' : '1' ?>">
            <button type="submit" class="btn btn-sm btn-outline-primary">
                <?= $isActive ? 'Hide' : 'Show' ?>
            </button>
        </form>
        <form method="post" class="d-inline" onsubmit="return confirm('Remove this image from the login
slideshow?');">
            <input type="hidden" name="action" value="delete">
            <input type="hidden" name="id" value="<?= $slideId ?>">
            <button type="submit" class="btn btn-sm btn-outline-danger">Delete</button>
        </form>
    </div>
</div>
</div>
</div>
</div>
</div>
<?php endforeach; ?>
</div>
<?php endif; ?>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: admin_requests.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';

require_role(['admin']);
header('Location: conflicts.php');
exit;

```

```

-----
FILE: api/check_course_load_assignment.php
-----

```

```

<?php
declare(strict_types=1);

require_once dirname(__DIR__) . '/includes/auth.php';
require_once dirname(__DIR__) . '/includes/functions.php';

header('Content-Type: application/json; charset=utf-8');

if (!in_array($_SESSION['role'] ?? '', ['dean', 'program_chair', 'admin'], true)) {
    http_response_code(403);
    echo json_encode(['error' => 'Forbidden'], JSON_UNESCAPED_UNICODE);
    exit;
}

$courseId = (int) ($_GET['course_id'] ?? 0);
$facultyId = (int) ($_GET['faculty_id'] ?? 0);
$semester = trim((string) ($_GET['semester'] ?? ''));
$schoolYear = trim((string) ($_GET['school_year'] ?? ''));

```

```

if ($courseId < 1 || $facultyId < 1 || $semester === '' || $schoolYear === '') {
    echo json_encode(['assigned' => false], JSON_UNESCAPED_UNICODE);
    exit;
}

$geTarget = null;
$targetCollegeId = (int) ($_GET['target_college_id'] ?? 0);
$targetProgram = trim((string) ($_GET['target_program'] ?? ''));
$targetYearLevel = trim((string) ($_GET['target_year_level'] ?? ''));
$targetSection = trim((string) ($_GET['target_section'] ?? ''));
if ($targetCollegeId > 0 && $targetProgram !== '' && $targetYearLevel !== '' && $targetSection !== '') {
    $geTarget = [
        'college_id' => $targetCollegeId,
        'program_name' => $targetProgram,
        'year_level' => $targetYearLevel,
        'section' => $targetSection,
    ];
}

$assigned = find_course_load_assigned_faculty($courseId, $semester, $schoolYear, $facultyId, $geTarget);
if ($assigned === null) {
    echo json_encode(['assigned' => false], JSON_UNESCAPED_UNICODE);
    exit;
}

$message = course_load_assignment_conflict_message($assigned, $courseId, $geTarget);
echo json_encode([
    'assigned' => true,
    'faculty_name' => $assigned['full_name'],
    'faculty_code' => $assigned['faculty_code'],
    'message' => $message,
], JSON_UNESCAPED_UNICODE);

```

FILE: api/classroom_inline_image_upload.php

```

<?php
declare(strict_types=1);

require_once dirname(__DIR__) . '/config/config.php';
require_once dirname(__DIR__) . '/includes/db.php';
require_once dirname(__DIR__) . '/includes/functions.php';
require_once dirname(__DIR__) . '/includes/auth.php';

if (session_status() === PHP_SESSION_NONE) {
    session_name(SESSION_NAME);
    session_start();
}

header('Content-Type: application/json; charset=utf-8');
header('X-Content-Type-Options: nosniff');
header('Cache-Control: private, no-store');

/**
 * @param array<string,mixed> $body
 * @never-return
 */
function classroom_inline_image_json(int $code, array $body): void
{
    http_response_code($code);
    echo json_encode($body, JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE);
    exit;
}

if (($_SERVER['REQUEST_METHOD'] ?? '') !== 'POST') {
    classroom_inline_image_json(405, ['ok' => false, 'error' => 'POST required.']);
}

$role = (string) ($_SESSION['role'] ?? '');
if (!in_array($role, ['faculty', 'program_chair', 'dean', 'gened'], true)) {
    classroom_inline_image_json(403, ['ok' => false, 'error' => 'Not allowed.']);
}

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user((int) ($_SESSION['user_id'] ?? 0)) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}

```

```

if ($facultyId < 1) {
    classroom_inline_image_json(403, ['ok' => false, 'error' => 'Faculty profile not linked.']);
}

if (!isset($_FILES['image']) || !is_array($_FILES['image'])) {
    classroom_inline_image_json(400, ['ok' => false, 'error' => 'Choose an image to upload.']);
}

$file = $_FILES['image'];
$error = (int) ($file['error'] ?? UPLOAD_ERR_NO_FILE);
if ($error !== UPLOAD_ERR_OK) {
    classroom_inline_image_json(400, ['ok' => false, 'error' => 'Image upload failed.']);
}

$size = (int) ($file['size'] ?? 0);
if ($size < 1) {
    classroom_inline_image_json(400, ['ok' => false, 'error' => 'Image file is empty.']);
}
if ($size > 5 * 1024 * 1024) {
    classroom_inline_image_json(400, ['ok' => false, 'error' => 'Image is too large (max 5 MB).']);
}

$tmp = (string) ($file['tmp_name'] ?? '');
$imageInfo = @getimagesize($tmp);
if ($imageInfo === false) {
    classroom_inline_image_json(400, ['ok' => false, 'error' => 'File must be a JPEG, PNG, GIF, or WebP image.']);
}

$mime = strtolower((string) ($imageInfo['mime'] ?? ''));
$ext = match ($mime) {
    'image/jpeg' => 'jpg',
    'image/png' => 'png',
    'image/gif' => 'gif',
    'image/webp' => 'webp',
    default => '',
};
if ($ext === '') {
    classroom_inline_image_json(400, ['ok' => false, 'error' => 'Only JPEG, PNG, GIF, and WebP images are allowed.']);
}

$dir = classroom_content_attachment_dir();
if (!is_dir($dir) && !mkdir($dir, 0775, true) && !is_dir($dir)) {
    classroom_inline_image_json(500, ['ok' => false, 'error' => 'Unable to create upload directory.']);
}

$storedName = 'inline_' . bin2hex(random_bytes(16)) . '.' . $ext;
$destination = classroom_content_attachment_storage_path($storedName);
if (!move_uploaded_file($tmp, $destination)) {
    classroom_inline_image_json(500, ['ok' => false, 'error' => 'Failed to save image.']);
}

classroom_inline_image_json(200, [
    'ok' => true,
    'src' => classroom_content_inline_image_href($storedName),
]);

```

FILE: api/faculty_assessment_generate_questions.php

```

<?php
declare(strict_types=1);

/**
 * POST – auto-generate assessment questions with answer keys (faculty only).
 */

require_once dirname(__DIR__) . '/config/config.php';
require_once dirname(__DIR__) . '/includes/db.php';
require_once dirname(__DIR__) . '/includes/functions.php';
require_once dirname(__DIR__) . '/includes/faculty_assessment_question_generator.php';

if (session_status() === PHP_SESSION_NONE) {
    session_name(SESSION_NAME);
    session_start();
}

/**
 * @param array<string, mixed> $body
 * @never-return

```

```

*/
function fagq_emit_json(int $httpCode, array $body): void
{
    header('Content-Type: application/json; charset=utf-8');
    header('X-Content-Type-Options: nosniff');
    header('Cache-Control: private, no-store');
    http_response_code($httpCode);

    echo json_encode(
        $body,
        JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE | JSON_THROW_ON_ERROR
    );
    exit;
}

function fagq_read_json_body(): ?array
{
    $raw = file_get_contents('php://input');
    if ($raw === false || $raw === '') {
        return [];
    }

    try {
        $decoded = json_decode($raw, true, 512, JSON_THROW_ON_ERROR);
    } catch (JsonException $e) {
        return null;
    }

    return is_array($decoded) ? $decoded : [];
}

function fagq_faculty_id(): int
{
    $role = (string) ($_SESSION['role'] ?? '');
    if (!in_array($role, ['faculty', 'program_chair', 'dean', 'gened'], true)) {
        return 0;
    }

    $facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
    if ($facultyId > 0) {
        return $facultyId;
    }

    $userId = (int) ($_SESSION['user_id'] ?? 0);
    if ($userId < 1) {
        return 0;
    }

    $resolved = resolve_faculty_id_for_user($userId) ?? 0;
    if ($resolved < 1 && in_array($role, ['program_chair', 'dean', 'gened'], true)) {
        $resolved = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    }
    if ($resolved > 0) {
        $_SESSION['faculty_id'] = $resolved;

        return $resolved;
    }

    return 0;
}

$method = strtoupper((string) ($_SERVER['REQUEST_METHOD'] ?? 'GET'));

if ($method === 'GET') {
    fagq_emit_json(200, [
        'ok' => true,
        'service' => 'faculty-assessment-generate-questions',
        'version' => '1.1.0',
        'ai_available' => wellness_ai_is_enabled(),
        'description' => 'Generates assessment questions from week materials (body + attachment text) when use_materials is true.',
        'endpoints' => [
            [
                'method' => 'POST',
                'auth' => ['session_cookie_faculty'],
                'request_body' => [
                    'classroom_id' => 1,
                    'assessment_type' => 'multiple_choice',
                    'title' => 'Quiz 1',
                    'description' => '',
                    'count' => 5,
                ],
            ],
        ],
    ]);
}

```



```

        'topic' => '',
        'credited_week' => 'Week 1',
        'use_materials' => true,
        'exclude' => [],
    ],
    'notes' => [
        'When use_materials is true, credited_week is required and questions are built from posted materials and attachments.',
        'Supported attachment extraction: txt, csv, docx, pptx, xlsx, pdf (pdftotext when available).',
    ],
],
    ],
]);
}

if ($method !== 'POST') {
    fagq_emit_json(405, ['ok' => false, 'error' => 'Method not allowed.']);
}

if (empty($_SESSION['user_id'])) {
    fagq_emit_json(401, ['ok' => false, 'error' => 'Login required.']);
}

$facultyId = fagq_faculty_id();
if ($facultyId < 1) {
    fagq_emit_json(403, ['ok' => false, 'error' => 'Faculty profile not linked to this account.']);
}

$payload = fagq_read_json_body();
if ($payload === null) {
    fagq_emit_json(400, ['ok' => false, 'error' => 'Invalid JSON body.']);
}

$classroomId = (int) ($payload['classroom_id'] ?? 0);
$assessmentType = (string) ($payload['assessment_type'] ?? 'essay');
$title = trim((string) ($payload['title'] ?? ''));
$description = (string) ($payload['description'] ?? '');
$count = max(1, min(20, (int) ($payload['count'] ?? 5)));
$topic = trim((string) ($payload['topic'] ?? ''));
$weekLabel = trim((string) ($payload['credited_week'] ?? $payload['week'] ?? ''));
$useMaterials = !empty($payload['use_materials']);
$exclude = [];
if (isset($payload['exclude']) && is_array($payload['exclude'])) {
    foreach ($payload['exclude'] as $questionText) {
        $text = trim((string) $questionText);
        if ($text !== '') {
            $exclude[] = $text;
        }
    }
}

if ($classroomId < 1) {
    fagq_emit_json(400, ['ok' => false, 'error' => 'classroom_id is required.']);
}

if ($useMaterials && $weekLabel === '') {
    fagq_emit_json(400, ['ok' => false, 'error' => 'Select a week under "From week materials" when including posted course materials.']);
}

try {
    $result = faculty_assessment_generator_generate(
        $facultyId,
        $classroomId,
        $assessmentType,
        $title,
        $description,
        $count,
        $topic,
        $weekLabel,
        $useMaterials,
        $exclude
    );
    fagq_emit_json(200, $result);
} catch (Throwable $e) {
    fagq_emit_json(400, ['ok' => false, 'error' => $e->getMessage()]);
}

```

FILE: api/faculty_end_live.php

```

<?php
declare(strict_types=1);

/**
 * POST schedule_id – end the LIVE indicator for a faculty schedule block.
 * Used by faculty_meet_live.php when Google Meet closes.
 */

require_once dirname(__DIR__) . '/config/config.php';
require_once dirname(__DIR__) . '/includes/db.php';
require_once dirname(__DIR__) . '/includes/functions.php';
require_once dirname(__DIR__) . '/includes/auth.php';

if (session_status() === PHP_SESSION_NONE) {
    session_name(SESSION_NAME);
    session_start();
}

header('Content-Type: application/json; charset=utf-8');
header('X-Content-Type-Options: nosniff');
header('Cache-Control: private, no-store');

$role = (string) ($_SESSION['role'] ?? '');
if (!in_array($role, ['faculty', 'program_chair', 'dean', 'gened'], true)) {
    http_response_code(403);
    echo json_encode(['ok' => false, 'error' => 'Forbidden'], JSON_THROW_ON_ERROR);
    exit;
}

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1 && $userId > 0) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
}
if ($facultyId < 1 && in_array($role, ['program_chair', 'dean', 'gened'], true) && $userId > 0) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
}
if ($facultyId < 1) {
    http_response_code(403);
    echo json_encode(['ok' => false, 'error' => 'Faculty profile required'], JSON_THROW_ON_ERROR);
    exit;
}

if (($_SERVER['REQUEST_METHOD'] ?? '') !== 'POST') {
    http_response_code(405);
    echo json_encode(['ok' => false, 'error' => 'Method not allowed'], JSON_THROW_ON_ERROR);
    exit;
}

$scheduleId = (int) ($_POST['schedule_id'] ?? 0);
if ($scheduleId < 1) {
    http_response_code(422);
    echo json_encode(['ok' => false, 'error' => 'schedule_id is required'], JSON_THROW_ON_ERROR);
    exit;
}

$ended = faculty_end_live_for_schedule($scheduleId, $facultyId);
echo json_encode(['ok' => true, 'ended' => $ended], JSON_THROW_ON_ERROR);

```

FILE: api/faculty_offline_upload.php

```

<?php
declare(strict_types=1);

/**
 * POST – sync a queued faculty offline upload (content post or syllabus).
 * Multipart fields mirror faculty_classroom.php actions.
 * Optional client_uuid for idempotent retries.
 */

require_once dirname(__DIR__) . '/config/config.php';
require_once dirname(__DIR__) . '/includes/db.php';
require_once dirname(__DIR__) . '/includes/functions.php';
require_once dirname(__DIR__) . '/includes/auth.php';

if (session_status() === PHP_SESSION_NONE) {
    session_name(SESSION_NAME);
    session_start();
}

```

```

}

/**
 * @param array<string, mixed> $body
 * @never-return
 */
function fou_emit_json(int $statusCode, array $body): void
{
    header('Content-Type: application/json; charset=utf-8');
    header('X-Content-Type-Options: nosniff');
    header('Cache-Control: private, no-store');
    http_response_code($statusCode);

    echo json_encode(
        $body,
        JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE | JSON_THROW_ON_ERROR
    );
    exit;
}

function fou_faculty_id(): int
{
    $role = (string) ($_SESSION['role'] ?? '');
    if (!in_array($role, ['faculty', 'program_chair', 'dean', 'gened'], true)) {
        return 0;
    }

    $facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
    if ($facultyId > 0) {
        return $facultyId;
    }

    $userId = (int) ($_SESSION['user_id'] ?? 0);
    if ($userId < 1) {
        return 0;
    }

    $resolved = resolve_faculty_id_for_user($userId) ?? 0;
    if ($resolved < 1 && in_array($role, ['program_chair', 'dean', 'gened'], true)) {
        $resolved = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    }
    if ($resolved > 0) {
        $_SESSION['faculty_id'] = $resolved;

        return $resolved;
    }

    return 0;
}

/**
 * @throws RuntimeException
 */
function fou_manage_url(string $raw): string
{
    $raw = trim($raw);
    if ($raw === '') {
        return '';
    }

    $url = filter_var($raw, FILTER_VALIDATE_URL);
    if ($url === false) {
        throw new RuntimeException('Please enter a valid URL.');
```

```

        db()->exec(
            'CREATE TABLE IF NOT EXISTS faculty_offline_sync (
                client_uuid VARCHAR(64) NOT NULL,
                faculty_id INT NOT NULL,
                action VARCHAR(40) NOT NULL,
                classroom_id INT NOT NULL,
                content_id INT NULL,
                created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
                PRIMARY KEY (client_uuid)
            ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci'
        );
    } catch (Throwable $e) {
        // Idempotency log is best-effort; sync still proceeds without it.
    }
}

/**
 * @return array{client_uuid:string,action:string,classroom_id:int,content_id:?int}|null
 */
function fou_find_sync(string $clientId): ?array
{
    if ($clientId === '' || !db_table_exists('faculty_offline_sync')) {
        return null;
    }

    $st = db()->prepare(
        'SELECT client_uuid, action, classroom_id, content_id
        FROM faculty_offline_sync
        WHERE client_uuid = ?
        LIMIT 1'
    );
    $st->execute([$clientId]);
    $row = $st->fetch();
    if (!$row) {
        return null;
    }

    return [
        'client_uuid' => (string) $row['client_uuid'],
        'action' => (string) $row['action'],
        'classroom_id' => (int) $row['classroom_id'],
        'content_id' => isset($row['content_id']) ? (int) $row['content_id'] : null,
    ];
}

function fou_remember_sync(string $clientId, int $facultyId, string $action, int $classroomId, ?int $contentId): void
{
    if ($clientId === '') {
        return;
    }

    try {
        fou_ensure_sync_table();
        if (!db_table_exists('faculty_offline_sync')) {
            return;
        }
        db()->prepare(
            'INSERT INTO faculty_offline_sync (client_uuid, faculty_id, action, classroom_id, content_id)
            VALUES (?, ?, ?, ?, ?)
            ON DUPLICATE KEY UPDATE content_id = VALUES(content_id)'
        )->execute([$clientId, $facultyId, $action, $classroomId, $contentId]);
    } catch (Throwable $e) {
        // Best-effort only.
    }
}

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    fou_emit_json(405, ['ok' => false, 'error' => 'Method not allowed. Use POST.']);
}

if (!isset($_SESSION['user_id'])) {
    fou_emit_json(401, ['ok' => false, 'error' => 'Please sign in again, then retry sync.']);
}

$facultyId = fou_faculty_id();
if ($facultyId < 1) {
    fou_emit_json(403, ['ok' => false, 'error' => 'Faculty profile is required to sync uploads.']);
}

$action = trim((string) ($_POST['action'] ?? ''));

```

```

$classroomId = (int) ($_POST['classroom_id'] ?? 0);
$clientUuid = trim((string) ($_POST['client_uuid'] ?? ''));
if (strlen($clientUuid) > 64) {
    $clientUuid = substr($clientUuid, 0, 64);
}
if ($clientUuid !== '' && !preg_match('/^[A-Za-z0-9._-]+$/', $clientUuid)) {
    fou_emit_json(400, ['ok' => false, 'error' => 'Invalid client_uuid.']);
}

if ($classroomId < 1) {
    fou_emit_json(400, ['ok' => false, 'error' => 'classroom_id is required.']);
}

if (!in_array($action, ['add_content', 'upload_syllabus'], true)) {
    fou_emit_json(400, ['ok' => false, 'error' => 'Unsupported action. Use add_content or upload_syllabus.']);
}

$existing = fou_find_sync($clientUuid);
if ($existing !== null) {
    fou_emit_json(200, [
        'ok' => true,
        'duplicate' => true,
        'action' => $existing['action'],
        'classroom_id' => $existing['classroom_id'],
        'content_id' => $existing['content_id'],
        'message' => 'Already synced.',
    ]);
}

$st = db()->prepare(
    'SELECT oc.id, oc.faculty_id, oc.syllabus_stored_name, oc.syllabus_original_name, oc.syllabus_mime
    FROM online_classrooms oc
    INNER JOIN schedules s ON s.id = oc.schedule_id
    WHERE oc.id = ? AND oc.faculty_id = ? AND s.faculty_id = ?
    LIMIT 1'
);
$st->execute([$classroomId, $facultyId, $facultyId]);
$classroom = $st->fetch() ?: null;
if (!$classroom) {
    fou_emit_json(404, ['ok' => false, 'error' => 'Classroom not found or not owned by you.']);
}

$hasSyllabusCols = db_column_exists('online_classrooms', 'syllabus_stored_name');
$hasContentAttachments = db_table_exists('classroom_content_attachments');
$hasContentWeeks = db_column_exists('classroom_content', 'weeks');
$hasContentDaysPerTopic = db_column_exists('classroom_content', 'days_per_topic');
$hasContentTopicSchedule = $hasContentWeeks && $hasContentDaysPerTopic;

try {
    if ($action === 'upload_syllabus') {
        if (!$hasSyllabusCols) {
            throw new RuntimeException('Run upgrade_roles.php to enable syllabus uploads.');
```

```

    }
}

db()->prepare(
    'UPDATE online_classrooms
    SET syllabus_stored_name = ?, syllabus_original_name = ?, syllabus_mime = ?
    WHERE id = ? AND faculty_id = ?'
)->execute([$basename, $origName, $mime, $classroomId, $facultyId]);

fou_remember_sync($clientId, $facultyId, $action, $classroomId, null);

fou_emit_json(200, [
    'ok' => true,
    'duplicate' => false,
    'action' => $action,
    'classroom_id' => $classroomId,
    'content_id' => null,
    'message' => 'Syllabus uploaded.',
    'syllabus_original_name' => $origName,
]);
}

// add_content
$contentType = (string) ($_POST['content_type'] ?? 'material');
$title = trim((string) ($_POST['title'] ?? ''));
$body = classroom_content_prepare_body((string) ($_POST['body'] ?? ''));
$weeks = trim((string) ($_POST['weeks'] ?? ''));
$daysPerTopic = trim((string) ($_POST['days_per_topic'] ?? ''));
$resourceUrl = fou_manage_url((string) ($_POST['resource_url'] ?? ''));
$uploadedFiles = isset($_FILES['attachments']) && is_array($_FILES['attachments'])
    ? classroom_content_normalize_uploads($_FILES['attachments'])
    : [];
$actualUploadedFiles = array_values(array_filter(
    $uploadedFiles,
    static fn (array $file): bool => (int) ($file['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_NO_FILE
));

if (!in_array($contentType, ['material', 'link', 'announcement'], true)) {
    $contentType = 'material';
}
if ($hasSyllabusCols) {
    $syName = trim((string) ($classroom['syllabus_stored_name'] ?? ''));
    if ($syName === '') {
        throw new RuntimeException('Upload the course syllabus before posting course content or announcements.');
```

```

        $weeks,
        $daysPerTopic,
        $legacyAttachmentResource ?? $resourceUrl,
    ));
} else {
    db()->prepare(
        'INSERT INTO classroom_content (classroom_id, faculty_id, content_type, title, body, resource_url)
        VALUES (?, ?, ?, ?, ?, ?)'
    )->execute([
        $classroomId,
        $facultyId,
        $contentType,
        $title,
        $body !== '' ? $body : null,
        $legacyAttachmentResource ?? $resourceUrl,
    ]);
}
$contentId = (int) db()->lastInsertId();

if ($attachments !== []) {
    $insertAttachment = db()->prepare(
        'INSERT INTO classroom_content_attachments (content_id, original_name, stored_name, mime)
        VALUES (?, ?, ?, ?)'
    );
    foreach ($attachments as $attachment) {
        $insertAttachment->execute([
            $contentId,
            $attachment['original_name'],
            $attachment['stored_name'],
            $attachment['mime'],
        ]);
    }
}

db()->commit();
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    if ($legacyAttachmentResource !== null) {
        $attachmentPath = classroom_content_attachment_path($legacyAttachmentResource);
        if (is_file($attachmentPath)) {
            @unlink($attachmentPath);
        }
    }
    foreach ($attachments as $attachment) {
        $attachmentPath = classroom_content_attachment_storage_path((string) ($attachment['stored_name'] ?? ''));
        if (is_file($attachmentPath)) {
            @unlink($attachmentPath);
        }
    }
    throw $e;
}

fou_remember_sync($clientId, $facultyId, $action, $classroomId, $contentId);

fou_emit_json(200, [
    'ok' => true,
    'duplicate' => false,
    'action' => $action,
    'classroom_id' => $classroomId,
    'content_id' => $contentId,
    'message' => $contentType === 'announcement'
        ? 'Announcement published for enrolled students.'
        : 'Course content added.',
]);
} catch (Throwable $e) {
    fou_emit_json(400, [
        'ok' => false,
        'error' => $e->getMessage() !== '' ? $e->getMessage() : 'Sync failed.',
    ]);
}
}

```

FILE: api/programs_by_college.php

```

<?php
declare(strict_types=1);

```

```

require_once __DIR__ . '/../config/config.php';
require_once __DIR__ . '/../includes/db.php';
require_once __DIR__ . '/../includes/student_registration_helpers.php';

header('Content-Type: application/json; charset=utf-8');

$collegeId = (int) ($_GET['college_id'] ?? 0);
$programs = active_programs_for_college($collegeId);
$yearLevelsByProgram = active_year_levels_by_program_for_college($collegeId);

```

```

echo json_encode([
    'programs' => $programs,
    'year_levels_by_program' => $yearLevelsByProgram,
], JSON_UNESCAPED_UNICODE);

```

```

-----
FILE: api/student_materials_review_questions.php
-----

```

```

<?php
declare(strict_types=1);

/**
 * POST – generate weekly practice questions from enrolled classroom materials (students only).
 */

```

```

require_once dirname(__DIR__) . '/config/config.php';
require_once dirname(__DIR__) . '/includes/db.php';
require_once dirname(__DIR__) . '/includes/functions.php';
require_once dirname(__DIR__) . '/includes/student_materials_reviewer.php';
require_once dirname(__DIR__) . '/includes/wellness_ai.php';

```

```

if (session_status() === PHP_SESSION_NONE) {
    session_name(SESSION_NAME);
    session_start();
}

```

```

/**
 * @param array<string, mixed> $body
 * @never-return
 */
function smrq_emit_json(int $httpCode, array $body): void
{
    header('Content-Type: application/json; charset=utf-8');
    header('X-Content-Type-Options: nosniff');
    http_response_code($httpCode);

    echo json_encode(
        $body,
        JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE | JSON_THROW_ON_ERROR
    );
    exit;
}

```

```

function smrq_read_json_body(): ?array
{
    $raw = file_get_contents('php://input');
    if ($raw === false || $raw === '') {
        return [];
    }

    try {
        $decoded = json_decode($raw, true, 512, JSON_THROW_ON_ERROR);
    } catch (JsonException $e) {
        return null;
    }

    return is_array($decoded) ? $decoded : [];
}

```

```

function smrq_student_id(): int
{
    $studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
    if ($studentId > 0) {
        return $studentId;
    }

    $userId = (int) ($_SESSION['user_id'] ?? 0);
    if ($userId < 1) {
        return 0;
    }
}

```



```

    }

    $resolved = resolve_student_id_for_user($userId);
    if ($resolved !== null && $resolved > 0) {
        $_SESSION['student_id'] = $resolved;

        return $resolved;
    }

    return 0;
}

$method = strtoupper((string) ($_SERVER['REQUEST_METHOD'] ?? 'GET'));

if ($method === 'GET') {
    smrq_emit_json(200, [
        'ok' => true,
        'service' => 'student-materials-review-questions',
        'version' => '1.0.0',
        'ai_available' => wellness_ai_is_enabled(),
        'endpoints' => [
            [
                'method' => 'POST',
                'auth' => ['session_cookie_student'],
                'request_body' => [
                    'classroom_id' => 1,
                    'week' => 'Week 1',
                    'count' => 0,
                    'exclude' => [],
                ],
            ],
        ],
    ]);
}

if ($method !== 'POST') {
    smrq_emit_json(405, ['ok' => false, 'error' => 'Method not allowed.']);
}

if (empty($_SESSION['user_id']) || ($_SESSION['role'] ?? '') !== 'student') {
    smrq_emit_json(401, ['ok' => false, 'error' => 'Student login required.']);
}

$payload = smrq_read_json_body();
if ($payload === null) {
    smrq_emit_json(400, ['ok' => false, 'error' => 'Invalid JSON body.']);
}

$studentId = smrq_student_id();
if ($studentId < 1) {
    smrq_emit_json(403, ['ok' => false, 'error' => 'Student profile not linked to this account.']);
}

$classroomId = (int) ($payload['classroom_id'] ?? 0);
$week = trim((string) ($payload['week'] ?? ''));
$count = (int) ($payload['count'] ?? 0);
$exclude = [];
if (isset($payload['exclude']) && is_array($payload['exclude'])) {
    foreach ($payload['exclude'] as $questionText) {
        $text = trim((string) $questionText);
        if ($text !== '') {
            $exclude[] = $text;
        }
    }
}

try {
    $result = student_materials_reviewer_generate($studentId, $classroomId, $week, $count, $exclude);
    smrq_emit_json(200, $result);
} catch (Throwable $e) {
    smrq_emit_json(400, ['ok' => false, 'error' => $e->getMessage()]);
}

```

FILE: api/student_offline_pack.php

```

<?php
declare(strict_types=1);

```

```

/**
 * GET – pack enrolled classroom faculty items for offline reading (students only).
 * Optional: ?classroom_id=N to pack one class; omit to pack all enrolled classes.
 */

require_once dirname(__DIR__) . '/config/config.php';
require_once dirname(__DIR__) . '/includes/db.php';
require_once dirname(__DIR__) . '/includes/functions.php';

if (session_status() === PHP_SESSION_NONE) {
    session_name(SESSION_NAME);
    session_start();
}

/**
 * @param array<string, mixed> $body
 * @never-return
 */
function sop_emit_json(int $httpCode, array $body): void
{
    header('Content-Type: application/json; charset=utf-8');
    header('X-Content-Type-Options: nosniff');
    header('Cache-Control: private, no-store');
    http_response_code($httpCode);

    echo json_encode(
        $body,
        JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE | JSON_THROW_ON_ERROR
    );
    exit;
}

function sop_student_id(): int
{
    $studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
    if ($studentId > 0) {
        return $studentId;
    }

    $userId = (int) ($_SESSION['user_id'] ?? 0);
    if ($userId < 1) {
        return 0;
    }

    $resolved = resolve_student_id_for_user($userId);
    if ($resolved !== null && $resolved > 0) {
        $_SESSION['student_id'] = $resolved;

        return $resolved;
    }

    return 0;
}

/**
 * @param array<int, array<string, mixed>> $items
 * @return array<int, array<string, mixed>>
 */
function sop_normalize_items(array $items, array $attachmentMap, bool $hasTopicSchedule): array
{
    $out = [];
    foreach ($items as $item) {
        $id = (int) ($item['id'] ?? 0);
        $resourceUrl = trim((string) ($item['resource_url'] ?? ''));
        $attachments = [];
        foreach ($attachmentMap[$id] ?? [] as $attachment) {
            $attachments[] = [
                'id' => (int) ($attachment['id'] ?? 0),
                'original_name' => (string) ($attachment['original_name'] ?? ''),
                'mime' => (string) ($attachment['mime'] ?? ''),
                'href' => classroom_content_attachment_href((int) ($attachment['id'] ?? 0)),
            ];
        }

        $resource = null;
        if ($resourceUrl !== '') {
            $isFile = classroom_content_is_attachment($resourceUrl);
            $resource = [
                'url' => $resourceUrl,
                'is_attachment' => $isFile,
            ];
        }
    }
}

```

```

        'label' => $isFile
            ? classroom_content_attachment_name($resourceUrl)
            : 'Open resource',
        'href' => classroom_content_resource_href($id, $resourceUrl),
    ];
}

$out[] = [
    'id' => $id,
    'content_type' => (string) ($item['content_type'] ?? 'material'),
    'title' => (string) ($item['title'] ?? ''),
    'body_html' => classroom_content_render_body((string) ($item['body'] ?? '')),
    'weeks' => $hasTopicSchedule ? (string) ($item['weeks'] ?? '') : '',
    'days_per_topic' => $hasTopicSchedule ? (string) ($item['days_per_topic'] ?? '') : '',
    'week_label' => classroom_content_week_label((string) ($item['weeks'] ?? '')),
    'resource' => $resource,
    'attachments' => $attachments,
    'created_at' => (string) ($item['created_at'] ?? ''),
];
}

return $out;
}

$method = strtoupper((string) ($_SERVER['REQUEST_METHOD'] ?? 'GET'));
if ($method !== 'GET') {
    sop_emit_json(405, ['ok' => false, 'error' => 'Method not allowed.']);
}

if (empty($_SESSION['user_id']) || (string) ($_SESSION['role'] ?? '') !== 'student') {
    sop_emit_json(401, ['ok' => false, 'error' => 'Student login required.']);
}

$studentId = sop_student_id();
if ($studentId < 1) {
    sop_emit_json(403, ['ok' => false, 'error' => 'Student profile not linked.']);
}

if (!db_table_exists('online_classrooms') || !db_table_exists('classroom_enrollments') || !db_table_exists('classroom_content')) {
    sop_emit_json(503, ['ok' => false, 'error' => 'Classroom features are not installed yet.']);
}

$classroomIdFilter = (int) ($_GET['classroom_id'] ?? 0);
$hasContentAttachments = db_table_exists('classroom_content_attachments');
$hasTopicSchedule = db_column_exists('classroom_content', 'weeks')
    && db_column_exists('classroom_content', 'days_per_topic');

$sql = 'SELECT oc.id, oc.title, oc.description, oc.status,
    c.course_code, c.course_name, f.full_name AS faculty_name,
    s.semester, s.school_year, s.day_of_week, s.start_time, s.end_time
    FROM classroom_enrollments ce
    INNER JOIN online_classrooms oc ON oc.id = ce.classroom_id
    INNER JOIN courses c ON c.id = oc.course_id
    INNER JOIN faculty f ON f.id = oc.faculty_id
    INNER JOIN schedules s ON s.id = oc.schedule_id
    WHERE ce.student_id = ?';
$params = [$studentId];
if ($classroomIdFilter > 0) {
    $sql .= ' AND oc.id = ?';
    $params[] = $classroomIdFilter;
}
$sql .= ' ORDER BY c.course_code ASC, oc.title ASC';

$stmt = db()->prepare($sql);
$stmt->execute($params);
$classrooms = $stmt->fetchAll() ?: [];

if ($classroomIdFilter > 0 && $classrooms === []) {
    sop_emit_json(404, ['ok' => false, 'error' => 'Classroom not found or you are not enrolled.']);
}

$pack = [];
$totalItems = 0;

foreach ($classrooms as $classroom) {
    $cid = (int) $classroom['id'];
    $cst = db()->prepare(
        'SELECT *
        FROM classroom_content
        WHERE classroom_id = ?
    ');

```

```

ORDER BY
    CASE WHEN content_type = "announcement" THEN 0 ELSE 1 END ASC,
    created_at DESC,
    id DESC
);
$cst->execute([$cid]);
$rawItems = $cst->fetchAll() ?: [];

$attachmentMap = [];
if ($hasContentAttachments && $rawItems !== []) {
    $attachmentMap = classroom_content_attachment_map(array_column($rawItems, 'id'));
}

$items = sop_normalize_items($rawItems, $attachmentMap, $hasTopicSchedule);
$totalItems += count($items);

$announcements = array_values(array_filter(
    $items,
    static fn (array $row): bool => ($row['content_type'] ?? '') === 'announcement'
));
$materials = array_values(array_filter(
    $items,
    static fn (array $row): bool => ($row['content_type'] ?? '') !== 'announcement'
));

$pack[] = [
    'id' => $cid,
    'title' => (string) ($classroom['title'] ?? ''),
    'description' => (string) ($classroom['description'] ?? ''),
    'status' => (string) ($classroom['status'] ?? ''),
    'course_code' => (string) ($classroom['course_code'] ?? ''),
    'course_name' => (string) ($classroom['course_name'] ?? ''),
    'faculty_name' => (string) ($classroom['faculty_name'] ?? ''),
    'semester' => (string) ($classroom['semester'] ?? ''),
    'school_year' => (string) ($classroom['school_year'] ?? ''),
    'day_of_week' => (string) ($classroom['day_of_week'] ?? ''),
    'start_time' => (string) ($classroom['start_time'] ?? ''),
    'end_time' => (string) ($classroom['end_time'] ?? ''),
    'announcements' => $announcements,
    'materials' => $materials,
    'item_count' => count($items),
];
}

sop_emit_json(200, [
    'ok' => true,
    'saved_at' => date('c'),
    'student_id' => $studentId,
    'classroom_count' => count($pack),
    'item_count' => $totalItems,
    'classrooms' => $pack,
    'note' => 'Text posts and links are available offline. Attachment file downloads still need internet.',
]);

```

FILE: api/student_wellness_chat.php

```

<?php
declare(strict_types=1);

/**
 * REST endpoint: student wellness companion (non-diagnostic; crisis escalation for PH).
 *
 * GET - service descriptor (public).
 * POST - chat completion (authenticated students only).
 */

require_once dirname(__DIR__) . '/config/config.php';
require_once dirname(__DIR__) . '/includes/wellness_chatbot.php';
require_once dirname(__DIR__) . '/includes/wellness_ai.php';
require_once dirname(__DIR__) . '/includes/wellness_engine.php';

if (session_status() === PHP_SESSION_NONE) {
    session_name(SESSION_NAME);
    session_start();
}

/**
 * @param array<string, mixed> $body

```

```

* @never-return
*/
function wellness_api_emit_json(int $httpCode, array $body): void
{
    header('Content-Type: application/json; charset=utf-8');
    header('X-Content-Type-Options: nosniff');
    http_response_code($httpCode);

    echo json_encode(
        $body,
        JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE | JSON_THROW_ON_ERROR
    );
    exit;
}

function wellness_api_read_json_body(): ?array
{
    $raw = file_get_contents('php://input');
    if ($raw === false || $raw === '') {
        return [];
    }

    try {
        $decoded = json_decode($raw, true, 512, JSON_THROW_ON_ERROR);
    } catch (JsonException $e) {
        return null;
    }

    return is_array($decoded) ? $decoded : [];
}

function wellness_api_is_student_logged_in(): bool
{
    return !empty($_SESSION['user_id']) && ($_SESSION['role'] ?? '') === 'student';
}

/**
 * @param array<string, mixed> $payload
 *
 * @return list<array{role: string, content: string}>
 */
function wellness_api_parse_history(array $payload): array
{
    if (!isset($payload['history']) || !is_array($payload['history'])) {
        return [];
    }

    return wellness_ai_sanitize_history($payload['history']);
}

/** @never-return */
function wellness_api_handle_get(): void
{
    $base = rtrim((string) (defined('APP_BASE_URL') ? APP_BASE_URL : ''), '/');
    $openapi = $base . '/api/wellness_chatbot.openapi.yaml';
    wellness_api_emit_json(200, [
        'ok' => true,
        'service' => 'student-wellness-chat',
        'version' => '1.3.0',
        'ai_available' => wellness_ai_is_enabled(),
        'ollama_reachable' => wellness_ollama_reachable(),
        'ai_provider' => wellness_ai_provider(),
        'groq' => wellness_groq_status(),
        'setup_hint' => wellness_ai_has_cloud_key()
            ? null
            : (wellness_ai_provider() === 'groq'
                ? 'Groq: open config/config.php and paste your API key from https://console.groq.com/keys (starts with gsk_).'
                : 'Set WELLNESS_AI_API_KEY in config/config.php or install Ollama for local AI.'),
        'constraints' => [
            'Question-aware replies matched to scenario (exams, sleep, loneliness, family, etc.).',
            'Provides general coping support—not therapy, diagnosis, or emergency intervention.',
            'Responds in English or Filipino/Tagalog heuristically, or honors client `language_hint`.',
            'May return `crisis: true` plus Philippines crisis hotlines when ideation cues are matched.',
        ],
        'openapi_url' => $openapi,
        'endpoints' => [
            [
                'name' => 'chat',
                'method' => 'POST',
                'path' => $base !== '' ? ($base . '/api/student_wellness_chat.php') : '/api/student_wellness_chat.php',
            ]
        ]
    ]);
}

```

```

        'auth' => ['session_cookie_student'],
        'request_body_example' => [
            'message' => 'School has been draining and I barely sleep lately.',
            'language_hint' => 'auto',
        ],
    ],
    'disclaimer_banner_en' => wellness_disclaimer_banner('en'),
    'philippines_crisis_sample' => wellness_ph_crisis_resources_public(),
]);
}

/** @never-return */
function wellness_api_handle_post(): void
{
    if (!wellness_api_is_student_logged_in()) {
        wellness_api_emit_json(401, [
            'ok' => false,
            'error' => 'Login required.',
            'code' => 'UNAUTHORIZED',
        ]);
    }

    $payload = wellness_api_read_json_body();
    if ($payload === null) {
        wellness_api_emit_json(400, [
            'ok' => false,
            'error' => 'Invalid JSON.',
            'code' => 'BAD_JSON',
        ]);
    }

    $messageRaw = isset($payload['message']) ? (string) $payload['message'] : '';
    if (mb_strlen($messageRaw, 'UTF-8') > 4000) {
        wellness_api_emit_json(400, [
            'ok' => false,
            'error' => 'Message exceeds 4000 characters.',
            'code' => 'MESSAGE_TOO_LONG',
        ]);
    }

    $hintLc = null;
    if (isset($payload['language_hint'])) {
        $h = mb_strtolower(trim((string) $payload['language_hint']), 'UTF-8');
        $hintLc = $h !== '' ? $h : null;
    } elseif (isset($payload['locale'])) {
        $h = mb_strtolower(trim((string) $payload['locale']), 'UTF-8');
        $hintLc = $h !== '' ? $h : null;
    }
    if ($hintLc === 'auto') {
        $hintLc = null;
    }

    $history = wellness_api_parse_history($payload);
    if ($history === [] && !empty($_SESSION['wellness_chat_history']) && is_array($_SESSION['wellness_chat_history'])) {
        $history = wellness_ai_sanitize_history($_SESSION['wellness_chat_history']);
    }

    $out = wellness_chat_orchestrate($messageRaw, $hintLc, $history);

    if ($messageRaw !== '' && !isset($out['crisis'])) {
        $_SESSION['wellness_chat_history'] = wellness_ai_sanitize_history(array_merge(
            $history,
            [
                ['role' => 'user', 'content' => $messageRaw],
                ['role' => 'assistant', 'content' => $out['reply_body']],
            ]
        ));
    }

    $reply = [
        'ok' => true,
        'reply' => $out['reply'],
        'reply_body' => $out['reply_body'],
        'reply_format' => 'markdown',
        'crisis' => $out['crisis'],
        'intent' => $out['intent'],
        'scenarios' => $out['scenarios'],
        'is_question' => $out['is_question'],
        'topics' => $out['topics'],
    ];
}

```

```

        'detected_language' => $out['language'],
        'disclaimer_banner' => $out['disclaimer_banner'],
        'footer_disclaimer' => $out['footer_disclaimer'],
        'philippines_crisis_resources' => $out['resources'] ?? null,
        'reply_source' => $out['reply_source'] ?? 'builtin',
    ];

    wellness_api_emit_json(200, $reply);
}

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';

if ($method === 'GET') {
    wellness_api_handle_get();
}

if ($method === 'POST') {
    wellness_api_handle_post();
}

wellness_api_emit_json(405, [
    'ok' => false,
    'error' => 'Method not allowed.',
    'code' => 'METHOD_NOT_ALLOWED',
]);

```

FILE: api/wellness_chatbot.openapi.yaml

```

openapi: 3.0.3
info:
  title: Student Wellness Chat Companion API
  description: >
    Authenticated REST API for bounded mental-health companionship replies.
    The implementation is deterministic (rule-based)—not clinical AI—designed for
    empathetic psychoeducation within scope: stress, anxiety, loneliness,
    and low mood. It must not diagnose, prescribe, or replace therapy.
    Crisis phrasing escalates with Philippines helpline payloads.
  version: 1.1.0
  contact:
    name: Institution IT / LMS operator

servers:
  - url: http://localhost/CLASS
    description: Local XAMPP (adjust if your virtual host differs)

paths:
  /api/student_wellness_chat.php:
    get:
      summary: Service descriptor & sample crisis resources
      description: >-
        Returns non-sensitive metadata suitable for onboarding API clients or
        student developers. Philippines helpline excerpts match the deterministic
        server copy (verify externally before production).
      operationId: getWellnessChatInfo
      responses:
        '200':
          description: Capability catalog
          content:
            application/json:
              schema:
                type: object
                properties:
                  ok:
                    type: boolean
                  service:
                    type: string
                  version:
                    type: string
                  constraints:
                    type: array
                    items:
                      type: string
                  openapi_url:
                    type: string
                  format: uri
                  endpoints:
                    type: array
                    items:

```

```

        type: object
    disclaimer_banner_en:
        type: string
    philippines_crisis_sample:
        type: object
post:
  summary: Non-diagnostic chat reply
  operationId: postWellnessChat
  security:
    - CampusSessionStudent: []
  requestBody:
    required: false
    content:
      application/json:
        schema:
          type: object
          properties:
            message:
              type: string
              description: Plain student message up to ~4000 UTF-8 graphemes (server-enforced length).
              maxLength: 4000
              example: I feel drained before every exam cycle and spiral about failing.
            language_hint:
              type: string
              nullable: true
              description: >
                Prefer `tl`/`fil`/Tagalog synonyms, `en`, or omit/`auto`.
            enum:
              - en
              - tl
              - fil
              - tagalog
              - english
              - auto
              - ''
            locale:
              type: string
              nullable: true
              description: Alias for `language_hint` when bilingual clients reuse generic field names.
          required: []
      examples:
        english:
          value:
            message: I barely sleep lately and dread group work.
            language_hint: en
        tagalog_heuristic:
          value:
            message: Nababalisa ako at parang nabibigatan sa lahat kahit hindi naman lagi may deadline.
            language_hint: auto
responses:
  '200':
    description: Supportive reply packaged with clinical-scope disclaimers
    content:
      application/json:
        schema:
          type: object
          required:
            - ok
            - reply
            - reply_body
            - crisis
            - intent
            - scenarios
            - is_question
            - topics
            - detected_language
            - disclaimer_banner
            - footer_disclaimer
          properties:
            ok:
              type: boolean
            reply:
              type: string
              description: Question-specific answer (Markdown). Same as reply_body.
            reply_body:
              type: string
              description: Main answer text without repeating disclaimers (show disclaimer_banner in UI separately).
            reply_format:
              type: string
              enum: [markdown]

```



```

intent:
  type: string
  description: e.g. question_coping, exam_school, crisis, prompt
scenarios:
  type: array
  items:
    type: string
    description: Matched themes ordered by relevance (exam_school, sleep, loneliness, etc.)
is_question:
  type: boolean
crisis:
  type: boolean
  description: True when suicidal ideation or self-harm crisis lexicon matched—the server substitutes hotline scaffolding.
topics:
  type: array
  items:
    type: string
    description: Lightweight emotional tags (stress, anxiety, loneliness, low_mood) when not in crisis escalation.
detected_language:
  type: string
  enum:
    - en
    - tl
disclaimer_banner:
  type: string
footer_disclaimer:
  type: string
philippines_crisis_resources:
  type: object
  nullable: true
  description: Structured helpline snippets when crisis=true; mirrored for SPA rendering.
error:
  type: string
'401':
  description: Valid campus session lacking student role—or unauthenticated altogether.
  content:
    application/json:
      schema:
        type: object
        properties:
          ok:
            type: boolean
            example: false
          error:
            type: string
          code:
            type: string
            example: UNAUTHORIZED
'400':
  description: Malformed payload or excessively long transcript.
  content:
    application/json:
      schema:
        type: object
        properties:
          ok:
            type: boolean
            example: false
          error:
            type: string
          code:
            type: string
            enum: [BAD_JSON, MESSAGE_TOO_LONG]
'405':
  description: Verb other than GET/POST (server closes with deterministic JSON envelope).

```

```

components:
  securitySchemes:
    CampusSessionStudent:
      type: apiKey
      in: cookie
      name: FSS_SESSION
      description: >
        PHP session cookie issued by WPU SABLAe Portal after student login.

```

```

-----
FILE: assets/css/print.css
-----

```

```

/* Print-friendly schedules */

```

```

@media print {
  body { background: #fff !important; font-size: 11pt; }
  .navbar, .btn, .no-print, form, .pagination { display: none !important; }
  main.container { max-width: 100% !important; padding: 0 !important; }
  .card { border: 1px solid #ccc !important; box-shadow: none !important; page-break-inside: avoid; }
  .table { font-size: 10pt; }
  a { color: #000 !important; text-decoration: none !important; }

  /* Weekly grid: always light ink on paper (even if screen was dark mode) */
  .schedule-weekly thead.table-primary th {
    background: #cfe2ff !important;
    color: #052c65 !important;
    border-color: #999 !important;
  }
  .schedule-weekly tbody td {
    background: #fff !important;
    color: #000 !important;
    border-color: #ccc !important;
  }
  .schedule-weekly--timed .schedule-time-col {
    background: #f1f5f9 !important;
    color: #000 !important;
    position: static !important;
  }
  .schedule-weekly--timed .schedule-cell--vacant {
    background: #fafafa !important;
  }
  .schedule-weekly--timed .schedule-vacant-label {
    color: #888 !important;
  }
  .schedule-block {
    color: #000 !important;
    border-left-color: #555 !important;
  }
  .schedule-block .text-muted {
    color: #444 !important;
  }
  .schedule-block.c0 { background: #e3f2fd !important; border-left-color: #1976d2 !important; }
  .schedule-block.c1 { background: #e8f5e9 !important; border-left-color: #388e3c !important; }
  .schedule-block.c2 { background: #fff3e0 !important; border-left-color: #f57c00 !important; }
  .schedule-block.c3 { background: #fce4ec !important; border-left-color: #c2185b !important; }
  .schedule-block.c4 { background: #f3e5f5 !important; border-left-color: #7b1fa2 !important; }
  .schedule-block.c5 { background: #e0f7fa !important; border-left-color: #00838f !important; }
}

```

FILE: assets/css/style.css

/* WPU SABLAE Portal – screen styles */

/* — App shell: light surface, typography, accessibility — */

```

:root {
  --app-bg: #f1f3f6;
  --app-surface: #ffffff;
  --app-border: rgba(15, 23, 42, 0.08);
  --app-border-strong: rgba(15, 23, 42, 0.12);
  --app-shadow-sm: 0 1px 2px rgba(15, 23, 42, 0.05);
  --app-shadow-md: 0 4px 16px rgba(15, 23, 42, 0.06);
  --app-radius: 12px;
  --app-radius-sm: 8px;
  --app-text: #0f172a;
  --app-text-muted: #64748b;
}

```

```

body.app-shell {
  background-color: var(--app-bg);
  color: var(--app-text);
  -webkit-font-smoothing: antialiased;
}

```

```

.app-main {
  max-width: 1280px;
}

```

```

.app-page-header {
  margin-bottom: 1.75rem;
}

```

```

.app-page-title {

```

```

font-size: clamp(1.35rem, 1.05rem + 1.1vw, 1.85rem);
font-weight: 650;
letter-spacing: -0.025em;
color: var(--app-text);
line-height: 1.2;
margin-bottom: 0.5rem;
}

.app-page-title .app-page-title-icon,
.app-role-icon {
  color: var(--sidebar-brand-icon-color, var(--bs-primary));
  opacity: 0.95;
}

.app-page-lead {
  font-size: 0.9375rem;
  color: var(--app-text-muted);
  max-width: 52rem;
  line-height: 1.55;
  margin-bottom: 0;
}

.app-meta-row {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem 1rem;
  margin-top: 0.75rem;
  font-size: 0.875rem;
  color: var(--app-text-muted);
}

.app-meta-row strong {
  color: var(--app-text);
  font-weight: 600;
}

.app-chip {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  padding: 0.25rem 0.65rem;
  border-radius: 999px;
  font-size: 0.8125rem;
  font-weight: 500;
  background: rgba(15, 23, 42, 0.04);
  border: 1px solid var(--app-border);
  color: var(--app-text);
}

.app-alert {
  border-radius: var(--app-radius-sm);
  border: 1px solid transparent;
  box-shadow: var(--app-shadow-sm);
}

.app-card {
  background: var(--app-surface);
  border: 1px solid var(--app-border);
  border-radius: var(--app-radius);
  box-shadow: var(--app-shadow-sm);
  margin-bottom: 1.5rem;
  overflow: hidden;
}

.app-card-header {
  padding: 1rem 1.25rem;
  border-bottom: 1px solid var(--app-border);
  background: linear-gradient(180deg, #fafbfc 0%, #f8fafc 100%);
  font-weight: 600;
  font-size: 0.9375rem;
  color: var(--app-text);
}

.app-card-body {
  padding: 1.25rem 1.35rem;
}

@media (min-width: 768px) {
  .app-card-body {
    padding: 1.35rem 1.5rem;
  }
}

```

```

    }
}

.app-table-shell {
  border-radius: var(--app-radius);
  border: 1px solid var(--app-border);
  box-shadow: var(--app-shadow-sm);
  background: var(--app-surface);
  overflow: hidden;
}

.app-table {
  margin-bottom: 0;
  vertical-align: middle;
}

.app-table thead th {
  font-size: 0.6875rem;
  text-transform: uppercase;
  letter-spacing: 0.055em;
  font-weight: 600;
  color: var(--app-text-muted);
  border-bottom: 1px solid var(--app-border-strong);
  background: #f8fafc;
  white-space: nowrap;
  padding-top: 0.85rem;
  padding-bottom: 0.85rem;
}

.app-table tbody td {
  padding-top: 0.85rem;
  padding-bottom: 0.85rem;
  border-color: rgba(15, 23, 42, 0.06);
}

.app-table tbody tr:hover {
  background: rgba(15, 23, 42, 0.025);
}

.app-table .app-table-actions {
  text-align: end;
  white-space: nowrap;
}

.app-empty-state {
  text-align: center;
  padding: 2.5rem 1rem;
  color: var(--app-text-muted);
  font-size: 0.9375rem;
}

.app-form-actions {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  align-items: center;
  padding-top: 0.25rem;
}

/* Focus visibility for keyboard users */
:where(a, button, input:not([type="hidden"]), select, textarea, [tabindex]:not([tabindex="-1"])):focus-visible {
  outline: 2px solid var(--bs-primary);
  outline-offset: 2px;
}

#main-content:focus-visible {
  outline: 2px solid var(--bs-primary);
  outline-offset: 4px;
}

/* Navbar on light background */
.navbar-light .navbar-brand:focus-visible {
  outline-offset: 4px;
}

.app-dropdown-menu .dropdown-item:hover,
.app-dropdown-menu .dropdown-item:focus {
  background: rgba(13, 110, 253, 0.09);
}

```

```

/* Faculty schedule: stacked cells on small screens */
.app-schedule-card .table td {
    vertical-align: top;
}

@media (max-width: 991.98px) {
    .app-schedule-card .table thead {
        display: none;
    }
    .app-schedule-card .table tbody tr {
        display: block;
        border-bottom: 1px solid var(--app-border);
        padding: 1rem 0;
    }
    .app-schedule-card .table tbody td {
        display: block;
        border: 0;
        padding: 0.35rem 1rem;
    }
    .app-schedule-card .table tbody td::before {
        content: attr(data-label);
        display: block;
        font-size: 0.65rem;
        text-transform: uppercase;
        letter-spacing: 0.06em;
        font-weight: 600;
        color: var(--app-text-muted);
        margin-bottom: 0.25rem;
    }
}

/* Join code highlight (faculty classroom) */
.fc-join-code-pill {
    font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, monospace;
    font-size: 1.35rem;
    font-weight: 600;
    letter-spacing: 0.04em;
    padding: 0.5rem 1rem;
    border-radius: var(--app-radius-sm);
    background: #f1f5f9;
    border: 1px solid var(--app-border);
    color: var(--app-text);
}

/* Week content cards */
.fc-content-item-card {
    border-radius: var(--app-radius-sm) !important;
    border: 1px solid var(--app-border) !important;
    box-shadow: var(--app-shadow-sm) !important;
    background: var(--app-surface) !important;
}

.fc-week-link.is-active-week {
    border-color: rgba(13, 110, 253, 0.45) !important;
    background: rgba(13, 110, 253, 0.06) !important;
}

.schedule-cell {
    min-height: 3rem;
    font-size: 0.85rem;
}

.schedule-block {
    border-radius: 0.35rem;
    padding: 0.35rem 0.5rem;
    margin-bottom: 0.25rem;
    border-left: 4px solid rgba(0, 0, 0, 0.15);
}

.schedule-block.c0 { background: #e3f2fd; border-left-color: #1976d2; }
.schedule-block.c1 { background: #e8f5e9; border-left-color: #388e3c; }
.schedule-block.c2 { background: #fff3e0; border-left-color: #f57c00; }
.schedule-block.c3 { background: #fce4ec; border-left-color: #c2185b; }
.schedule-block.c4 { background: #f3e5f5; border-left-color: #7b1fa2; }
.schedule-block.c5 { background: #e0f7fa; border-left-color: #00838f; }

/* Faculty course color picker (view_schedule.php) */
.course-color-panel {
    background: var(--app-surface, #fff);
}

.course-color-preview {
    display: inline-block;
}

```

```

width: 1.25rem;
height: 1.25rem;
flex-shrink: 0;
border-radius: 0.3rem;
border: 2px solid var(--swatch-border, #94a3b8);
background: var(--swatch-bg, #e2e8f0);
}
.course-color-preview--sm {
width: 0.9rem;
height: 0.9rem;
}
.course-color-legend-item {
display: inline-flex;
align-items: center;
gap: 0.35rem;
padding: 0.2rem 0.55rem;
border-radius: 999px;
background: rgba(15, 23, 42, 0.04);
}

html[data-bs-theme="dark"] .course-color-panel {
background: var(--bs-body-bg);
}
html[data-bs-theme="dark"] .course-color-legend-item {
background: rgba(255, 255, 255, 0.06);
}

/* Timed weekly grid: left time axis + vacant slots */
.schedule-weekly--timed .schedule-time-col {
width: 4.75rem;
min-width: 4.75rem;
max-width: 5.5rem;
white-space: nowrap;
vertical-align: top;
background: #f8fafc;
position: sticky;
left: 0;
z-index: 2;
}
.schedule-weekly--timed thead .schedule-time-col {
z-index: 3;
}
.schedule-weekly--timed .schedule-time-label {
font-size: 0.8rem;
line-height: 1.2;
}
.schedule-weekly--timed .schedule-time-end {
font-size: 0.7rem;
line-height: 1.2;
}
.schedule-weekly--timed tbody tr {
min-height: 4.5rem;
}
.schedule-weekly--timed .schedule-cell {
min-height: 4.5rem;
height: 1%;
}
.schedule-weekly--timed .schedule-cell--vacant {
background: repeating-linear-gradient(
-45deg,
transparent,
transparent 6px,
rgba(15, 23, 42, 0.03) 6px,
rgba(15, 23, 42, 0.03) 12px
);
}
.schedule-weekly--timed .schedule-vacant-label {
display: block;
font-size: 0.7rem;
color: #94a3b8;
letter-spacing: 0.02em;
text-transform: uppercase;
}
.schedule-weekly--timed .schedule-block--continued {
opacity: 0.85;
border-left-style: dashed;
padding-top: 0.25rem;
padding-bottom: 0.25rem;
}
.schedule-weekly--timed .schedule-cell--spanned {

```

```

    vertical-align: top;
}
.schedule-weekly--timed .schedule-block--spanned {
    min-height: calc(var(--block-span, 1) * 4.25rem - 0.5rem);
    height: calc(100% - 0.25rem);
    box-sizing: border-box;
}

/* Weekly schedule grid (view_schedule.php) - dark mode contrast */
html[data-bs-theme="dark"] .schedule-weekly {
    color: var(--bs-body-color);
    --sw-cell-border: rgba(255, 255, 255, 0.12);
}

html[data-bs-theme="dark"] .schedule-weekly > :not(caption) > * > * {
    border-color: var(--sw-cell-border);
}

html[data-bs-theme="dark"] .schedule-weekly thead.table-primary th {
    background: linear-gradient(180deg, #1e3a5f 0%, #152a45 100%);
    color: #e2e8f0;
    border-color: var(--sw-cell-border);
}

html[data-bs-theme="dark"] .schedule-weekly tbody td {
    background-color: var(--bs-body-bg);
    color: var(--bs-body-color);
}

html[data-bs-theme="dark"] .schedule-weekly--timed .schedule-time-col {
    background: #152a45;
    color: #e2e8f0;
}

html[data-bs-theme="dark"] .schedule-weekly--timed .schedule-cell--vacant {
    background: repeating-linear-gradient(
        -45deg,
        transparent,
        transparent 6px,
        rgba(255, 255, 255, 0.04) 6px,
        rgba(255, 255, 255, 0.04) 12px
    );
}

html[data-bs-theme="dark"] .schedule-weekly--timed .schedule-vacant-label {
    color: rgba(241, 245, 249, 0.45);
}

html[data-bs-theme="dark"] .schedule-cell {
    color: var(--bs-body-color);
}

html[data-bs-theme="dark"] .schedule-block {
    color: #f1f5f9;
    border-left-color: rgba(255, 255, 255, 0.35);
}

html[data-bs-theme="dark"] .schedule-block .text-muted {
    color: rgba(241, 245, 249, 0.78) !important;
}

html[data-bs-theme="dark"] .schedule-block .border-top.border-secondary-subtle,
html[data-bs-theme="dark"] .schedule-block .border-secondary-subtle {
    border-color: rgba(255, 255, 255, 0.18) !important;
}

html[data-bs-theme="dark"] .schedule-block.c0 {
    background: rgba(25, 118, 210, 0.32);
    border-left-color: #60a5fa;
}

html[data-bs-theme="dark"] .schedule-block.c1 {
    background: rgba(46, 125, 50, 0.32);
    border-left-color: #4ade80;
}

html[data-bs-theme="dark"] .schedule-block.c2 {
    background: rgba(230, 81, 0, 0.28);
    border-left-color: #fb923c;
}

```

```

html[data-bs-theme="dark"] .schedule-block.c3 {
  background: rgba(173, 20, 87, 0.3);
  border-left-color: #f472b6;
}

html[data-bs-theme="dark"] .schedule-block.c4 {
  background: rgba(106, 27, 154, 0.3);
  border-left-color: #c084fc;
}

html[data-bs-theme="dark"] .schedule-block.c5 {
  background: rgba(0, 131, 143, 0.3);
  border-left-color: #2dd4bf;
}

.stat-card {
  border: none;
  border-radius: 0.5rem;
  box-shadow: 0 0.125rem 0.25rem rgba(0, 0, 0, 0.075);
}

/* Responsive navbar/menu typography */
.navbar .navbar-brand {
  font-size: clamp(1rem, 0.9rem + 0.45vw, 1.25rem);
}

.navbar .nav-link {
  font-size: clamp(0.85rem, 0.8rem + 0.2vw, 1rem);
}

.navbar .navbar-text {
  font-size: clamp(0.75rem, 0.7rem + 0.2vw, 0.9rem);
}

/* Responsive, polished dropdown component */
.app-dropdown-menu .dropdown-toggle {
  border-radius: 999px;
  padding: 0.35rem 0.75rem;
}

.app-dropdown-menu .dropdown-menu {
  min-width: 260px;
  border-radius: 0.85rem;
  overflow: hidden;
  margin-top: 0.45rem;
}

.app-dropdown-menu .dropdown-item {
  padding: 0.55rem 0.95rem;
  font-size: 0.95rem;
}

.app-dropdown-menu .dropdown-item:hover,
.app-dropdown-menu .dropdown-item:focus {
  background: rgba(13, 110, 253, 0.08);
}

@media (max-width: 991.98px) {
  .app-dropdown-menu {
    width: 100%;
    margin-top: 0.5rem;
  }
  .app-dropdown-menu .dropdown-toggle {
    width: 100%;
    justify-content: center;
  }
  .app-dropdown-menu .dropdown-menu {
    width: 100%;
    min-width: 100%;
  }
}

.report-hub .card-header strong {
  letter-spacing: 0.2px;
}

.report-hub .list-group-item {
  font-size: 0.92rem;
}

```



```

.report-hub .list-group-item .btn {
  border-radius: 999px;
}

@keyframes livePulse {
  0%,
  100% {
    box-shadow: 0 0 0 0 rgba(220, 53, 69, 0.55);
  }
  50% {
    box-shadow: 0 0 0 6px rgba(220, 53, 69, 0);
  }
}

.live-pulse-badge {
  animation: livePulse 1.6s ease-in-out infinite;
}

.wordpad-shell {
  border: 1px solid var(--bs-border-color);
  border-radius: 0.5rem;
  background: #fff;
  overflow: hidden;
}

.wordpad-toolbar {
  display: flex;
  flex-wrap: wrap;
  gap: 0.4rem;
  padding: 0.5rem;
  border-bottom: 1px solid var(--bs-border-color);
  background: #f8f9fa;
}

.wordpad-toolbar .btn {
  min-width: 2.2rem;
}

.wordpad-select {
  width: auto;
  min-width: 9rem;
}

.wordpad-select--font {
  min-width: 8.5rem;
}

.wordpad-select--size {
  min-width: 7.5rem;
}

.wordpad-editor {
  min-height: 10rem;
  padding: 0.75rem;
  border: 0;
  border-radius: 0;
  box-shadow: none;
  overflow-y: auto;
}

.wordpad-editor:focus {
  outline: none;
  box-shadow: none;
}

.wordpad-editor:empty::before {
  content: attr(data-placeholder);
  color: #6c757d;
}

.classroom-content-body {
  line-height: 1.5;
}

.classroom-content-body p,
.classroom-content-body ul,
.classroom-content-body ol,
.classroom-content-body blockquote,
.classroom-content-body h2,

```

```

.classroom-content-body h3 {
  margin-bottom: 0.5rem;
}

.classroom-content-body p:last-child,
.classroom-content-body ul:last-child,
.classroom-content-body ol:last-child,
.classroom-content-body blockquote:last-child,
.classroom-content-body h2:last-child,
.classroom-content-body h3:last-child {
  margin-bottom: 0;
}

.classroom-content-body ul,
.classroom-content-body ol {
  padding-left: 1.25rem;
}

.classroom-content-body blockquote {
  padding-left: 0.75rem;
  border-left: 3px solid rgba(13, 110, 253, 0.25);
  color: #495057;
}

.classroom-content-body a {
  word-break: break-word;
}

.classroom-content-body img,
.classroom-content-attachment-image img,
.wordpad-editor img {
  max-width: 100%;
  height: auto;
  display: block;
  margin: 0.75rem 0;
  border-radius: 0.4rem;
}

.classroom-content-attachment-image {
  max-width: 40rem;
}

@media print {
  .no-print { display: none !important; }
}

/* Faculty online classroom – management UI (scoped under .fc-manage) */
.fc-manage {
  --fc-accent: #1a73e8;
  --fc-accent-soft: rgba(26, 115, 232, 0.08);
  --fc-card-radius: 12px;
}

.fc-manage .fc-section-card {
  border: 0;
  border-radius: var(--fc-card-radius);
  box-shadow: 0 1px 2px rgba(60, 64, 67, 0.12), 0 1px 4px rgba(60, 64, 67, 0.08);
}

.fc-manage .fc-section-card .card-header {
  background: #fff;
  border-bottom: 1px solid rgba(0, 0, 0, 0.06);
  padding: 1rem 1.25rem;
  border-radius: var(--fc-card-radius) var(--fc-card-radius) 0 0 !important;
}

.fc-manage .fc-section-title {
  font-size: 0.9375rem;
  font-weight: 600;
  letter-spacing: 0.02em;
  color: #3c4043;
  margin: 0;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.fc-manage .fc-section-title i {
  color: var(--fc-accent);
  width: 1.25rem;
}

```

```

    text-align: center;
}

.fc-manage .fc-subsection {
    margin-bottom: 1.5rem;
}

.fc-manage .fc-subsection:last-child {
    margin-bottom: 0;
}

.fc-manage .fc-subsection-label {
    font-size: 0.8125rem;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.04em;
    color: #5f6368;
    margin-bottom: 0.75rem;
    display: flex;
    align-items: center;
    gap: 0.35rem;
}

.fc-manage .fc-subsection-label i {
    opacity: 0.85;
    font-size: 0.9rem;
}

.fc-manage .fc-inner-panel {
    background: #f8f9fa;
    border-radius: 10px;
    padding: 1rem 1.125rem;
    border: 1px solid rgba(0, 0, 0, 0.06);
}

.fc-manage .fc-page-title {
    font-size: 1.5rem;
    font-weight: 600;
    color: #202124;
    letter-spacing: -0.02em;
}

.fc-manage .fc-meta-line {
    color: #5f6368;
    font-size: 0.875rem;
}

.fc-manage .fc-btn-primary-lg {
    font-weight: 600;
    padding: 0.5rem 1.5rem;
    border-radius: 999px;
    box-shadow: 0 1px 2px rgba(26, 115, 232, 0.35);
}

.fc-manage .fc-stat-tile {
    border-radius: 10px;
    border: 1px solid rgba(0, 0, 0, 0.06);
    background: #fff;
    padding: 1rem 1.125rem;
    height: 100%;
    transition: box-shadow 0.15s ease;
}

.fc-manage .fc-stat-tile:hover {
    box-shadow: 0 2px 8px rgba(60, 64, 67, 0.1);
}

.fc-manage .fc-stat-tile .fc-stat-icon {
    width: 2.25rem;
    height: 2.25rem;
    border-radius: 8px;
    display: flex;
    align-items: center;
    justify-content: center;
    background: var(--fc-accent-soft);
    color: var(--fc-accent);
    margin-bottom: 0.5rem;
}

.fc-manage .fc-stat-value {

```

```

font-size: 1.5rem;
font-weight: 700;
color: #202124;
line-height: 1.2;
}

.fc-manage .fc-stat-label {
font-size: 0.75rem;
font-weight: 600;
text-transform: uppercase;
letter-spacing: 0.05em;
color: #5f6368;
}

.fc-manage .fc-roster-empty,
.fc-manage .fc-placeholder-hint {
color: #80868b;
font-size: 0.875rem;
}

.fc-manage .table.fc-roster-table thead th {
font-size: 0.6875rem;
text-transform: uppercase;
letter-spacing: 0.06em;
font-weight: 600;
color: #5f6368;
border-bottom-width: 1px;
}

.fc-manage .fc-week-link {
border-radius: 8px;
border: 1px solid rgba(0, 0, 0, 0.08);
margin-bottom: 0.5rem;
padding: 0.75rem 1rem;
text-decoration: none;
color: inherit;
display: flex;
align-items: center;
justify-content: space-between;
background: #fff;
transition: background 0.12s ease, border-color 0.12s ease;
}

.fc-manage .fc-week-link:hover {
background: #f8f9fa;
border-color: rgba(26, 115, 232, 0.25);
color: inherit;
}

body.page-fc-week-edit-popup .admin-sidebar,
body.page-fc-week-edit-popup .admin-topbar,
body.page-fc-week-edit-popup .visually-hidden-focusable {
display: none !important;
}

body.page-fc-week-edit-popup .admin-layout {
min-height: auto;
}

body.page-fc-week-edit-popup #main-content {
max-width: 100%;
}

/* — App shell: sidebar, grouped nav, top bar (admin + all roles) — */
body.app-shell-sidebar,
body.app-admin-shell {
--admin-sidebar-w: 272px;
--admin-sidebar-bg: linear-gradient(180deg, #f8fafc 0%, #f1f5f9 100%);
--admin-sidebar-border: rgba(15, 23, 42, 0.08);
--sidebar-brand-icon-bg: rgba(37, 99, 235, 0.12);
--sidebar-brand-icon-color: #2563eb;
--admin-nav-icon: #5b8def;
--admin-nav-icon-active: var(--sidebar-brand-icon-color);
--admin-nav-link-hover-bg: rgba(37, 99, 235, 0.09);
--admin-nav-link-active-bg: rgba(37, 99, 235, 0.14);
--admin-nav-link-active-text: var(--sidebar-brand-icon-color);
}

body.app-shell-sidebar--super_admin {
--sidebar-brand-icon-bg: rgba(225, 29, 72, 0.12);

```

```

--sidebar-brand-icon-color: #e11d48;
--admin-nav-icon: #f43f5e;
--admin-nav-link-hover-bg: rgba(225, 29, 72, 0.09);
--admin-nav-link-active-bg: rgba(225, 29, 72, 0.14);
}
body.app-shell-sidebar--admin {
--sidebar-brand-icon-bg: rgba(37, 99, 235, 0.13);
--sidebar-brand-icon-color: #2563eb;
--admin-nav-icon: #5b8def;
--admin-nav-link-hover-bg: rgba(37, 99, 235, 0.09);
--admin-nav-link-active-bg: rgba(37, 99, 235, 0.14);
}
body.app-shell-sidebar--dean {
--sidebar-brand-icon-bg: rgba(67, 56, 202, 0.13);
--sidebar-brand-icon-color: #4338ca;
--admin-nav-icon: #6366f1;
--admin-nav-link-hover-bg: rgba(67, 56, 202, 0.09);
--admin-nav-link-active-bg: rgba(67, 56, 202, 0.14);
}
body.app-shell-sidebar--program_chair {
--sidebar-brand-icon-bg: rgba(109, 40, 217, 0.12);
--sidebar-brand-icon-color: #6d28d9;
--admin-nav-icon: #8b5cf6;
--admin-nav-link-hover-bg: rgba(109, 40, 217, 0.09);
--admin-nav-link-active-bg: rgba(109, 40, 217, 0.14);
}
body.app-shell-sidebar--gened {
--sidebar-brand-icon-bg: rgba(4, 120, 87, 0.12);
--sidebar-brand-icon-color: #047857;
--admin-nav-icon: #10b981;
--admin-nav-link-hover-bg: rgba(4, 120, 87, 0.09);
--admin-nav-link-active-bg: rgba(4, 120, 87, 0.14);
}
body.app-shell-sidebar--faculty {
--sidebar-brand-icon-bg: rgba(194, 65, 12, 0.12);
--sidebar-brand-icon-color: #c2410c;
--admin-nav-icon: #ea580c;
--admin-nav-link-hover-bg: rgba(194, 65, 12, 0.09);
--admin-nav-link-active-bg: rgba(194, 65, 12, 0.14);
}
body.app-shell-sidebar--student {
--sidebar-brand-icon-bg: rgba(2, 132, 199, 0.12);
--sidebar-brand-icon-color: #0284c7;
--admin-nav-icon: #0ea5e9;
--admin-nav-link-hover-bg: rgba(2, 132, 199, 0.09);
--admin-nav-link-active-bg: rgba(2, 132, 199, 0.14);
}

.admin-layout {
background: var(--app-bg);
}

.admin-sidebar {
width: var(--admin-sidebar-w);
min-height: 100vh;
height: 100vh;
position: sticky;
top: 0;
align-self: flex-start;
background: var(--admin-sidebar-bg);
border-right: 1px solid var(--admin-sidebar-border);
box-shadow: 2px 0 24px rgba(15, 23, 42, 0.04);
}

.admin-sidebar-brand {
padding: 1.25rem 1.1rem 1rem;
border-bottom: 1px solid var(--admin-sidebar-border);
}

.admin-sidebar-brand-link {
display: flex;
align-items: center;
gap: 0.65rem;
text-decoration: none;
color: var(--app-text);
font-weight: 650;
font-size: 1.02rem;
letter-spacing: -0.02em;
line-height: 1.25;
}

```

```

.admin-sidebar-brand-link:hover {
  color: var(--sidebar-brand-icon-color, var(--bs-primary));
}

.admin-sidebar-brand-icon {
  width: 2.25rem;
  height: 2.25rem;
  border-radius: 12px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  background: transparent;
  font-size: 1.05rem;
  flex-shrink: 0;
  overflow: hidden;
}

.admin-sidebar-brand-meta {
  margin-top: 0.35rem;
  margin-left: 2.9rem;
  font-size: 0.7rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  color: var(--app-text-muted);
}

.admin-sidebar-nav {
  padding: 0.85rem 0.65rem 1rem;
  overflow-y: auto;
  min-height: 0;
}

.admin-nav-section + .admin-nav-section {
  margin-top: 1.15rem;
}

.admin-nav-section-label {
  font-size: 0.65rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.09em;
  color: var(--app-text-muted);
  padding: 0.35rem 0.65rem 0.5rem;
}

.admin-nav-list li + li {
  margin-top: 0.15rem;
}

.admin-nav-link {
  display: flex;
  align-items: center;
  gap: 0.55rem;
  padding: 0.45rem 0.65rem;
  border-radius: 10px;
  text-decoration: none;
  color: var(--app-text);
  font-size: 0.9rem;
  font-weight: 500;
  transition: background 0.12s ease, color 0.12s ease;
}

.admin-nav-link:hover {
  background: var(--admin-nav-link-hover-bg, rgba(37, 99, 235, 0.09));
  color: var(--app-text);
}

.admin-nav-link.active {
  background: var(--admin-nav-link-active-bg, rgba(37, 99, 235, 0.14));
  color: var(--admin-nav-link-active-text, var(--sidebar-brand-icon-color, var(--bs-primary)));
  font-weight: 600;
}

.admin-nav-icon {
  width: 1.35rem;
  text-align: center;
  flex-shrink: 0;
  color: var(--admin-nav-icon);
}

```

```

    font-size: 0.95rem;
    transition: color 0.12s ease;
}

.admin-nav-link:hover .admin-nav-icon,
.admin-nav-link.active .admin-nav-icon {
    color: var(--admin-nav-icon-active, var(--sidebar-brand-icon-color, var(--bs-primary)));
}

.admin-nav-text {
    flex: 1 1 auto;
    min-width: 0;
}

.admin-nav-badge {
    font-size: 0.65rem;
    padding: 0.2em 0.45em;
}

.admin-sidebar-footer {
    margin-top: auto;
    padding: 0.85rem 1rem 1.1rem;
    border-top: 1px solid var(--admin-sidebar-border);
    background: rgba(255, 255, 255, 0.5);
    display: flex;
    flex-direction: column;
    gap: 0.25rem;
}

.admin-sidebar-foot-link {
    display: flex;
    align-items: center;
    padding: 0.45rem 0.65rem;
    border-radius: 10px;
    font-size: 0.88rem;
    font-weight: 500;
    text-decoration: none;
    color: var(--app-text-muted);
    transition: background 0.12s ease, color 0.12s ease;
}

.admin-sidebar-foot-link:not(.admin-sidebar-logout) i {
    color: var(--admin-nav-icon);
}

.admin-sidebar-foot-link:hover {
    background: rgba(15, 23, 42, 0.05);
    color: var(--app-text);
}

.admin-sidebar-logout {
    color: #b91c1c;
}

.admin-sidebar-logout:hover {
    background: rgba(185, 28, 28, 0.08);
    color: #991b1b;
}

.admin-offcanvas-nav {
    position: fixed;
    top: 0;
    left: 0;
    bottom: 0;
    height: 100%;
    width: min(320px, 92vw);
    max-width: 100%;
    display: flex;
    flex-direction: column;
    background: var(--admin-sidebar-bg, var(--app-surface, #fff));
    border-right: 1px solid var(--admin-sidebar-border, rgba(15, 23, 42, 0.08));
    box-shadow: 2px 0 24px rgba(15, 23, 42, 0.18);
    z-index: 1060;
    transform: translateX(-100%);
    transition: transform 0.3s ease;
    will-change: transform;
    overflow: hidden;
}

.admin-offcanvas-nav.show,

```

```

#adminNavToggle:checked ~ .admin-offcanvas-nav {
  transform: none;
}

.admin-nav-toggle-input {
  position: fixed;
  top: 0;
  left: -9999px;
  width: 1px;
  height: 1px;
  opacity: 0;
  pointer-events: none;
}

@media (min-width: 992px) {
  .admin-offcanvas-nav,
  .admin-offcanvas-backdrop,
  .admin-nav-toggle-input {
    display: none;
  }
}

.admin-offcanvas-backdrop {
  position: fixed;
  inset: 0;
  z-index: 1055;
  margin: 0;
  padding: 0;
  border: 0;
  background: rgba(15, 23, 42, 0.5);
  opacity: 0;
  pointer-events: none;
  transition: opacity 0.3s ease;
}

.admin-offcanvas-backdrop.show,
#adminNavToggle:checked ~ .admin-offcanvas-backdrop {
  opacity: 1;
  pointer-events: auto;
}

.admin-offcanvas-nav .offcanvas-header {
  padding: 1rem 1.1rem;
}

.admin-offcanvas-nav .offcanvas-body {
  flex: 1 1 auto;
  display: flex;
  flex-direction: column;
  min-height: 0;
  overflow: hidden;
  padding: 0;
}

.admin-offcanvas-nav .admin-sidebar-nav {
  flex: 1 1 auto;
  min-height: 0;
  overflow-x: hidden;
  overflow-y: auto;
  -webkit-overflow-scrolling: touch;
  overscroll-behavior: contain;
}

.admin-topbar {
  position: sticky;
  top: 0;
  z-index: 1040;
}

.admin-main-wrap {
  background: var(--app-bg);
}

.admin-topbar-title {
  font-size: clamp(1rem, 0.92rem + 0.35vw, 1.2rem);
  font-weight: 650;
  letter-spacing: -0.02em;
  color: var(--app-text);
  line-height: 1.25;
}

```



```

.admin-topbar-subtitle {
  letter-spacing: 0.02em;
}

.admin-topbar-avatar {
  width: 2.1rem;
  height: 2.1rem;
  background: var(--sidebar-brand-icon-bg, rgba(37, 99, 235, 0.12));
  color: var(--sidebar-brand-icon-color, var(--bs-primary));
  font-size: 0.95rem;
  border: 1.5px solid color-mix(in srgb, var(--sidebar-brand-icon-color, var(--bs-primary)) 22%, transparent);
  box-shadow: 0 1px 4px color-mix(in srgb, var(--sidebar-brand-icon-color, var(--bs-primary)) 14%, transparent);
}

.admin-topbar-avatar-icon {
  color: var(--sidebar-brand-icon-color, var(--bs-primary));
}

.admin-topbar-avatar-img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  display: block;
}

.settings-profile-preview {
  width: 5rem;
  height: 5rem;
  object-fit: cover;
}

.admin-topbar-user .dropdown-item {
  font-size: 0.9rem;
  padding: 0.45rem 0.85rem;
}

@media (min-width: 992px) {
  .admin-main-wrap > main.app-main {
    margin-left: auto;
    margin-right: auto;
  }
}

/* Contextual tooltips (cursor-follow; see assets/js/app_tooltips.js) */
.app-cursor-tooltip {
  position: fixed;
  z-index: 1090;
  max-width: 220px;
  padding: 0.5rem 0.75rem;
  border-radius: 8px;
  font-size: 0.8125rem;
  line-height: 1.38;
  pointer-events: none;
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.12);
  border: 1px solid #f59e0b;
  background: #fffbeb;
  color: #92400e;
  opacity: 0;
  transition: opacity 0.25s ease;
}

.app-cursor-tooltip[aria-hidden="false"] {
  opacity: 1;
}

html[data-bs-theme="dark"] body.app-shell .app-cursor-tooltip {
  background: #451a03;
  color: #fde68a;
  border-color: #d97706;
  box-shadow: 0 8px 28px rgba(0, 0, 0, 0.45);
}

/* — Global dark/light (top bar toggle; Bootstrap data-bs-theme on <html>) — */
.app-theme-toggle {
  border-radius: 999px;
  padding: 0.35rem 0.85rem;
  font-size: 0.8rem;
  font-weight: 500;
  border: 1px solid var(--bs-border-color);

```

```

background: var(--bs-body-bg);
color: var(--bs-body-color);
transition: background 0.15s ease, border-color 0.15s ease, color 0.15s ease;
}

.app-theme-toggle:hover {
background: var(--bs-secondary-bg);
border-color: var(--bs-secondary-border-subtle);
color: var(--bs-body-color);
}

html[data-bs-theme="dark"] {
color-scheme: dark;
--app-bg: #0f1419;
--app-surface: #151b24;
--app-border: rgba(255, 255, 255, 0.08);
--app-border-strong: rgba(255, 255, 255, 0.12);
--app-shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.4);
--app-shadow-md: 0 8px 24px rgba(0, 0, 0, 0.35);
--app-text: #e2e8f0;
--app-text-muted: #94a3b8;
}

html[data-bs-theme="dark"] body.app-shell-sidebar,
html[data-bs-theme="dark"] body.app-admin-shell {
--admin-sidebar-bg: linear-gradient(180deg, #101722 0%, #0c1018 100%);
--admin-sidebar-border: rgba(255, 255, 255, 0.08);
}

html[data-bs-theme="dark"] body.app-shell-sidebar--super_admin {
--sidebar-brand-icon-bg: rgba(244, 63, 94, 0.18);
--sidebar-brand-icon-color: #fb7185;
--admin-nav-icon: #fda4af;
}
html[data-bs-theme="dark"] body.app-shell-sidebar--admin {
--sidebar-brand-icon-bg: rgba(96, 165, 250, 0.16);
--sidebar-brand-icon-color: #60a5fa;
--admin-nav-icon: #93c5fd;
}
html[data-bs-theme="dark"] body.app-shell-sidebar--dean {
--sidebar-brand-icon-bg: rgba(129, 140, 248, 0.16);
--sidebar-brand-icon-color: #818cf8;
--admin-nav-icon: #a5b4fc;
}
html[data-bs-theme="dark"] body.app-shell-sidebar--program_chair {
--sidebar-brand-icon-bg: rgba(167, 139, 250, 0.16);
--sidebar-brand-icon-color: #a78bfa;
--admin-nav-icon: #c4b5fd;
}
html[data-bs-theme="dark"] body.app-shell-sidebar--gened {
--sidebar-brand-icon-bg: rgba(52, 211, 153, 0.14);
--sidebar-brand-icon-color: #34d399;
--admin-nav-icon: #6ee7b7;
}
html[data-bs-theme="dark"] body.app-shell-sidebar--faculty {
--sidebar-brand-icon-bg: rgba(251, 146, 60, 0.16);
--sidebar-brand-icon-color: #fb923c;
--admin-nav-icon: #fdb462;
}
html[data-bs-theme="dark"] body.app-shell-sidebar--student {
--sidebar-brand-icon-bg: rgba(56, 189, 248, 0.16);
--sidebar-brand-icon-color: #38bdf8;
--admin-nav-icon: #7dd3fc;
}

html[data-bs-theme="dark"] .admin-sidebar {
box-shadow: 2px 0 24px rgba(0, 0, 0, 0.35);
}

html[data-bs-theme="dark"] .admin-sidebar-footer {
background: rgba(0, 0, 0, 0.25);
}

html[data-bs-theme="dark"] .admin-nav-link:hover {
background: var(--admin-nav-link-hover-bg, rgba(96, 165, 250, 0.14));
}

html[data-bs-theme="dark"] .admin-nav-link.active {
background: var(--admin-nav-link-active-bg, rgba(96, 165, 250, 0.22));
}

```

```

html[data-bs-theme="dark"] .admin-sidebar-foot-link:hover {
  background: rgba(255, 255, 255, 0.06);
}

html[data-bs-theme="dark"] .app-card-header {
  background: linear-gradient(180deg, #1a2230 0%, #151b24 100%);
}

html[data-bs-theme="dark"] .app-chip {
  background: rgba(255, 255, 255, 0.06);
  border-color: var(--app-border);
}

html[data-bs-theme="dark"] .app-table thead th {
  background: #1e293b;
  color: var(--app-text-muted);
}

/* — Dark mode: contrast (avoid washed / wrong text vs background) — */
html[data-bs-theme="dark"] body.app-shell {
  background-color: var(--app-bg);
  color: var(--app-text);
}

html[data-bs-theme="dark"] .text-muted {
  color: var(--bs-secondary-color) !important;
}

/* Utility surfaces that stay "paper white" in markup */
html[data-bs-theme="dark"] .bg-white,
html[data-bs-theme="dark"] .card-header.bg-white,
html[data-bs-theme="dark"] .card-footer.bg-white {
  background-color: var(--bs-tertiary-bg) !important;
  color: var(--bs-body-color) !important;
  border-color: var(--bs-border-color);
}

html[data-bs-theme="dark"] .bg-light {
  background-color: var(--bs-secondary-bg) !important;
  color: var(--bs-body-color) !important;
}

html[data-bs-theme="dark"] .text-dark {
  color: var(--bs-emphasis-color) !important;
}

html[data-bs-theme="dark"] .bg-warning.text-dark,
html[data-bs-theme="dark"] .badge.bg-warning.text-dark,
html[data-bs-theme="dark"] .alert-warning .text-dark {
  color: #0f172a !important;
}

html[data-bs-theme="dark"] .card:not([class*="bg-"]):not(.dashboard-hero):not(.glass-card) {
  background-color: var(--bs-body-bg);
  color: var(--bs-body-color);
  border-color: var(--bs-border-color-translucent);
}

html[data-bs-theme="dark"] .card-header:not(.bg-primary):not(.bg-success):not(.bg-danger):not(.bg-warning):not(.bg-info):not(.bg-dark) {
  color: var(--bs-body-color);
}

html[data-bs-theme="dark"] .list-group-item {
  background-color: var(--bs-body-bg);
  color: var(--bs-body-color);
  border-color: var(--bs-border-color);
}

html[data-bs-theme="dark"] .table {
  --bs-table-bg: var(--bs-body-bg);
  --bs-table-color: var(--bs-body-color);
  --bs-table-border-color: var(--bs-border-color);
  color: var(--bs-body-color);
}

html[data-bs-theme="dark"] .table > :not(caption) > * > * {
  border-bottom-color: rgba(255, 255, 255, 0.08);
}

```

```

html[data-bs-theme="dark"] .table-striped > tbody > tr:nth-of-type(odd) > * {
  --bs-table-accent-bg: rgba(255, 255, 255, 0.04);
  color: var(--bs-body-color);
}

html[data-bs-theme="dark"] .table-hover > tbody > tr:hover > * {
  --bs-table-accent-bg: rgba(255, 255, 255, 0.07);
  color: var(--bs-body-color);
}

html[data-bs-theme="dark"] .table thead th {
  background-color: var(--bs-tertiary-bg);
  color: var(--bs-secondary-color);
}

html[data-bs-theme="dark"] .border,
html[data-bs-theme="dark"] .border-secondary {
  border-color: var(--bs-border-color) !important;
}

html[data-bs-theme="dark"] .border-light {
  border-color: rgba(255, 255, 255, 0.12) !important;
}

html[data-bs-theme="dark"] .shadow,
html[data-bs-theme="dark"] .shadow-sm {
  box-shadow: var(--app-shadow-sm) !important;
}

/* Dashboard (inline styles assume light chrome) */
html[data-bs-theme="dark"] .dashboard-shell .glass-card {
  border-color: rgba(255, 255, 255, 0.1);
  box-shadow: 0 16px 36px rgba(0, 0, 0, 0.35);
}

html[data-bs-theme="dark"] .dashboard-shell .glass-card .card-header {
  background: linear-gradient(180deg, #1a2230 0%, #151b24 100%);
  border-bottom-color: rgba(255, 255, 255, 0.1);
  color: var(--app-text);
}

html[data-bs-theme="dark"] .dashboard-shell .glass-card .card-body {
  background-color: var(--bs-body-bg);
  color: var(--bs-body-color);
}

html[data-bs-theme="dark"] .dashboard-shell .upcoming-table thead th {
  background: var(--bs-tertiary-bg);
  color: var(--bs-secondary-color);
  border-color: var(--bs-border-color);
}

html[data-bs-theme="dark"] .dashboard-shell .upcoming-table tbody tr:hover {
  background: rgba(255, 255, 255, 0.05);
}

html[data-bs-theme="dark"] .dashboard-shell .conflict-card .card-header {
  background: linear-gradient(180deg, rgba(251, 191, 36, 0.18) 0%, rgba(30, 41, 59, 0.95) 100%);
  border-bottom-color: rgba(251, 191, 36, 0.25);
  color: var(--app-text);
}

html[data-bs-theme="dark"] .dashboard-shell .conflict-card .card-body {
  background-color: var(--bs-body-bg);
  color: var(--bs-body-color);
}

/* Program chair students module (.pcs-app) */
html[data-bs-theme="dark"] .pcs-app {
  --pcs-text: #e2e8f0;
  --pcs-muted: #94a3b8;
  --pcs-border: rgba(255, 255, 255, 0.12);
  color: var(--pcs-text);
}

html[data-bs-theme="dark"] .pcs-app .pcs-card {
  background: #151b24;
  border-color: var(--pcs-border);
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.35);
}

```

```

html[data-bs-theme="dark"] .pcs-app .pcs-card-header {
  border-bottom-color: var(--pcs-border);
}

html[data-bs-theme="dark"] .pcs-app .pcs-field label {
  color: #cbd5e1;
}

html[data-bs-theme="dark"] .pcs-app .pcs-field label .pcs-opt {
  color: var(--pcs-muted);
}

html[data-bs-theme="dark"] .pcs-app .pcs-field input,
html[data-bs-theme="dark"] .pcs-app .pcs-field select {
  background: #0f1419;
  border-color: #334155;
  color: #e2e8f0;
}

html[data-bs-theme="dark"] .pcs-app .pcs-password-hint {
  color: var(--pcs-muted);
}

html[data-bs-theme="dark"] .pcs-app .pcs-btn-secondary {
  background: #1e293b;
  border-color: #475569;
  color: #e2e8f0;
}

html[data-bs-theme="dark"] .pcs-app .pcs-btn-secondary:hover {
  background: #334155;
  border-color: #64748b;
  color: #f8fafc;
}

html[data-bs-theme="dark"] .pcs-app .pcs-btn-outline {
  border-color: #475569;
  color: #e2e8f0;
}

html[data-bs-theme="dark"] .pcs-app .pcs-btn-outline:hover {
  background: rgba(255, 255, 255, 0.06);
  color: #f8fafc;
}

html[data-bs-theme="dark"] .pcs-app .pcs-college-badge {
  background: #1e293b;
  color: #e2e8f0;
  border-color: var(--pcs-border);
}

html[data-bs-theme="dark"] .pcs-app .pcs-nav-tabs {
  border-bottom-color: var(--pcs-border);
}

html[data-bs-theme="dark"] .pcs-app .pcs-nav-tab {
  color: #94a3b8;
}

html[data-bs-theme="dark"] .pcs-app .pcs-nav-tab:hover {
  color: #93c5fd;
  background: rgba(59, 130, 246, 0.12);
}

html[data-bs-theme="dark"] .pcs-app .pcs-nav-tab.pcs-active {
  color: #93c5fd;
  border-bottom-color: #3b82f6;
  background: rgba(59, 130, 246, 0.12);
}

html[data-bs-theme="dark"] .pcs-app .pcs-batch-zone,
html[data-bs-theme="dark"] .pcs-app .pcs-batch-inner {
  background: rgba(15, 23, 42, 0.6);
}

html[data-bs-theme="dark"] .pcs-app .pcs-fake-file {
  background: #1e293b;
  color: #cbd5e1;
  border-color: #475569;
}

```

```

}

html[data-bs-theme="dark"] .pcs-app .pcs-table th {
  background: #1e293b;
  color: #cbd5e1;
  border-bottom-color: var(--pcs-border);
}

html[data-bs-theme="dark"] .pcs-app .pcs-table td {
  border-bottom-color: rgba(255, 255, 255, 0.08);
  color: var(--pcs-text);
}

html[data-bs-theme="dark"] .pcs-app .pcs-status-off {
  background: #1e293b;
  color: #94a3b8;
}

html[data-bs-theme="dark"] .pcs-app .pcs-status-active {
  background: rgba(22, 163, 74, 0.2);
  color: #86efac;
}

html[data-bs-theme="dark"] .pcs-app .pcs-actions a,
html[data-bs-theme="dark"] .pcs-app .pcs-actions button {
  color: #94a3b8;
}

html[data-bs-theme="dark"] .pcs-app .pcs-actions a:hover,
html[data-bs-theme="dark"] .pcs-app .pcs-actions button:hover {
  background: rgba(255, 255, 255, 0.08);
}

html[data-bs-theme="dark"] .pcs-app .pcs-search {
  background: #0f1419;
  border-color: var(--pcs-border);
}

html[data-bs-theme="dark"] .pcs-app .pcs-search input {
  color: #e2e8f0;
}

html[data-bs-theme="dark"] .pcs-app .pcs-total-pill {
  background: rgba(99, 102, 241, 0.25);
  color: #c7d2fe;
}

html[data-bs-theme="dark"] .pcs-app .pcs-page-btn {
  background: #1e293b;
  border-color: var(--pcs-border);
  color: #e2e8f0;
}

html[data-bs-theme="dark"] .pcs-app .pcs-page-btn:hover {
  background: #334155;
}

html[data-bs-theme="dark"] .pcs-app .pcs-modal-card {
  background: #151b24;
  color: #e2e8f0;
}

html[data-bs-theme="dark"] .pcs-app .pcs-empty {
  color: var(--pcs-muted);
}

/* — Student account: mobile responsiveness — */
body.app-shell-sidebar--student .admin-main-wrap {
  overflow-x: clip;
  min-width: 0;
}

body.page-student-edutools .admin-main-wrap {
  overflow-x: visible;
}

body.app-shell-sidebar--student .student-class-meta {
  word-break: break-word;
  line-height: 1.45;
}

```

```

body.app-shell-sidebar--student .student-messages-back {
  display: none;
}

body.app-shell-sidebar--student .reviewer-question-card {
  border: 1px solid #e2e8f0;
  border-radius: 0.5rem;
  padding: 1rem;
  margin-bottom: 0.75rem;
  background: #fff;
  word-break: break-word;
  overflow-wrap: anywhere;
}

body.app-shell-sidebar--student .reviewer-question-card .reviewer-choice {
  display: block;
  margin-bottom: 0.35rem;
}

body.app-shell-sidebar--student .reviewer-answer {
  display: none;
  margin-top: 0.75rem;
  padding-top: 0.75rem;
  border-top: 1px dashed #cbd5e1;
}

body.app-shell-sidebar--student .reviewer-question-card.is-revealed .reviewer-answer {
  display: block;
}

html[data-bs-theme="dark"] body.app-shell-sidebar--student .reviewer-question-card {
  background: var(--bs-body-bg);
  border-color: rgba(255, 255, 255, 0.12);
}

html[data-bs-theme="dark"] body.app-shell-sidebar--student .reviewer-answer {
  border-top-color: rgba(255, 255, 255, 0.15);
}

body.app-shell-sidebar--student .student-calendar-wrap .table-responsive {
  -webkit-overflow-scrolling: touch;
}

body.app-shell-sidebar--student .student-calendar-wrap .schedule-day-col {
  min-width: 140px;
}

@media (max-width: 991.98px) {
  body.app-shell-sidebar--student .admin-main-wrap > main.app-main {
    padding-left: 0.875rem !important;
    padding-right: 0.875rem !important;
  }

  body.app-shell-sidebar--student .student-page-header {
    flex-direction: column;
    align-items: stretch !important;
  }

  body.app-shell-sidebar--student .student-page-header__actions {
    width: 100%;
    display: flex;
    flex-wrap: wrap;
    gap: 0.5rem;
  }

  body.app-shell-sidebar--student .student-page-header > .d-flex.gap-2:not(.student-page-header__actions):not(.min-w-0) {
    width: 100%;
    flex-wrap: wrap;
    gap: 0.5rem;
  }

  body.app-shell-sidebar--student .student-page-header > .d-flex.gap-2:not(.student-page-header__actions):not(.min-w-0) > .btn {
    flex: 1 1 calc(50% - 0.25rem);
    min-width: 0;
  }

  body.app-shell-sidebar--student .student-page-header__actions > .btn,
  body.app-shell-sidebar--student .student-page-header__actions > form .btn {
    flex: 1 1 calc(50% - 0.25rem);
  }
}

```

```

    min-width: 0;
}

body.app-shell-sidebar--student .student-page-header__actions > .student-attendance-panel {
    flex: 1 1 100%;
}

body.app-shell-sidebar--student .student-attendance-panel .d-flex.flex-wrap.gap-2 {
    width: 100%;
}

body.app-shell-sidebar--student .student-attendance-panel .btn {
    flex: 1 1 calc(50% - 0.25rem);
}

body.app-shell-sidebar--student .student-messages-layout.student-messages-thread-open .student-messages-sidebar {
    display: none;
}

body.app-shell-sidebar--student .student-messages-layout:not(.student-messages-thread-open) .student-messages-panel {
    display: none;
}

body.app-shell-sidebar--student .student-messages-back {
    display: inline-flex !important;
    align-items: center;
    gap: 0.35rem;
}

body.app-shell-sidebar--student #msgThread,
body.app-shell-sidebar--student #studentClassDiscussionThread {
    max-height: min(52vh, 420px);
}

body.app-shell-sidebar--student .student-edutools-page {
    overflow: visible;
    padding-bottom: 1rem !important;
}

body.app-shell-sidebar--student .student-edutools-root {
    flex: 1 1 auto;
    min-height: 0;
    display: flex;
    flex-direction: column;
}

body.app-shell-sidebar--student .student-edutools-root .app-container {
    flex: 1 1 auto;
    min-height: min(560px, calc(100dvh - 11rem));
}

body.app-shell-sidebar--student .student-edutools-root .main-area {
    min-height: min(560px, calc(100dvh - 11rem));
}

body.app-shell-sidebar--student .student-edutools-root .tool-viewport,
body.app-shell-sidebar--student .student-edutools-root #eduDynamicContent {
    min-height: min(480px, calc(100dvh - 15rem));
}

body.app-shell-sidebar--student .dashboard-hero .hero-title {
    font-size: clamp(1.25rem, 4.5vw, 1.85rem);
    line-height: 1.3;
}

body.app-shell-sidebar--student .student-calendar-wrap .schedule-day-col {
    min-width: 120px;
    font-size: 0.85rem;
}

body.app-shell-sidebar--student .student-calendar-wrap .schedule-block {
    font-size: 0.88rem;
}

body.app-shell-sidebar--student .student-reviewer-actions .btn {
    flex: 1 1 calc(50% - 0.25rem);
    min-width: 0;
}

body.app-shell-sidebar--student .student-dashboard-quick-actions .btn {

```



```

    flex: 1 1 calc(50% - 0.25rem);
    min-width: 0;
}

body.app-shell-sidebar--student .glass-card .card-header .btn {
    width: auto;
    max-width: 100%;
}

#adminNavOpenBtn {
    min-width: 44px;
    min-height: 44px;
    padding: 0.5rem 0.65rem;
    touch-action: manipulation;
}

.admin-offcanvas-nav .admin-nav-link,
.admin-offcanvas-nav .admin-sidebar-foot-link {
    min-height: 44px;
    align-items: center;
}

}

@media (max-width: 575.98px) {
    body.app-shell-sidebar--student .admin-topbar .app-theme-toggle #appThemeToggleLabel {
        display: none;
    }

    body.app-shell-sidebar--student .admin-topbar .px-3 {
        padding-left: 0.65rem !important;
        padding-right: 0.65rem !important;
        gap: 0.5rem !important;
    }

    body.app-shell-sidebar--student .student-page-header h1,
    body.app-shell-sidebar--student .student-page-header .h3 {
        font-size: 1.2rem;
        line-height: 1.35;
    }

    body.app-shell-sidebar--student .student-page-header__actions > .btn,
    body.app-shell-sidebar--student .student-page-header__actions > form .btn {
        flex: 1 1 100%;
    }

    body.app-shell-sidebar--student .student-page-header > .d-flex.gap-2:not(.student-page-header__actions):not(.min-w-0) > .btn {
        flex: 1 1 100%;
    }

    body.app-shell-sidebar--student .student-reviewer-actions .btn,
    body.app-shell-sidebar--student .student-dashboard-quick-actions .btn {
        flex: 1 1 100%;
    }

    body.app-shell-sidebar--student .student-calendar-wrap .schedule-day-col {
        min-width: 108px;
        font-size: 0.78rem;
    }

    body.app-shell-sidebar--student .student-calendar-wrap .card-header {
        flex-direction: column;
        align-items: stretch !important;
    }

    body.app-shell-sidebar--student .student-calendar-wrap .card-header .btn {
        width: 100%;
    }

    body.app-shell-sidebar--student .student-class-card-stats {
        display: flex;
        flex-direction: column;
        gap: 0.2rem;
    }

    body.app-shell-sidebar--student .student-class-card-stats > span {
        margin-right: 0 !important;
    }

    body.app-shell-sidebar--student .student-join-form .btn {
        width: 100%;
    }

```

```

}

body.app-shell-sidebar--student .card-body .d-flex.gap-2.flex-wrap .btn {
  flex: 1 1 100%;
  min-width: 0;
}

body.app-shell-sidebar--student .card[style*="max-width"] {
  max-width: 100% !important;
}
}

/* Classroom header banner (Google Classroom-style) */
.classroom-banner {
  --classroom-banner-image: none;
  position: relative;
  min-height: 11.5rem;
  border-radius: 0.85rem;
  overflow: hidden;
  color: #fff;
  background:
    linear-gradient(135deg, rgba(15, 76, 58, 0.92) 0%, rgba(31, 111, 67, 0.88) 45%, rgba(12, 74, 110, 0.9) 100%),
    var(--classroom-banner-image);
  background-size: cover;
  background-position: center;
  box-shadow: 0 0.35rem 1.25rem rgba(15, 23, 42, 0.12);
}

.classroom-banner.has-photo {
  background-image:
    linear-gradient(180deg, rgba(15, 23, 42, 0.15) 0%, rgba(15, 23, 42, 0.72) 100%),
    var(--classroom-banner-image);
}

.classroom-banner__overlay {
  position: relative;
  z-index: 1;
  display: flex;
  flex-direction: column;
  justify-content: flex-end;
  min-height: 11.5rem;
  padding: 1.25rem 1.35rem 1.15rem;
}

.classroom-banner__title {
  margin: 0;
  font-size: clamp(1.35rem, 2.4vw, 1.85rem);
  font-weight: 700;
  line-height: 1.25;
  letter-spacing: -0.02em;
  text-shadow: 0 1px 2px rgba(0, 0, 0, 0.35);
}

.classroom-banner__meta {
  margin-top: 0.4rem;
  font-size: 0.95rem;
  opacity: 0.95;
  text-shadow: 0 1px 2px rgba(0, 0, 0, 0.3);
}

.classroom-banner__sep {
  margin: 0 0.4rem;
  opacity: 0.7;
}

.classroom-banner__live-note {
  margin-top: 0.45rem;
  font-size: 0.875rem;
  font-weight: 600;
  color: #fecaca;
}

.classroom-banner__actions {
  position: absolute;
  top: 0.85rem;
  right: 0.85rem;
  z-index: 2;
  display: flex;
  flex-wrap: wrap;
  gap: 0.4rem;
}

```

```

    justify-content: flex-end;
}

.classroom-banner__upload-btn {
    font-weight: 600;
    box-shadow: 0 0.15rem 0.5rem rgba(0, 0, 0, 0.2);
}

.classroom-banner--preview {
    min-height: 8.5rem;
    max-width: 42rem;
}

.classroom-banner--preview .classroom-banner__overlay {
    min-height: 8.5rem;
    padding: 1rem 1.1rem;
}

.classroom-banner--preview .classroom-banner__title {
    font-size: 1.2rem;
}

.student-page-header--below-banner {
    margin-top: 0.25rem;
}

@media (max-width: 767.98px) {
    .classroom-banner,
    .classroom-banner__overlay {
        min-height: 9.5rem;
    }

    .classroom-banner__overlay {
        padding: 1rem;
        padding-top: 3rem;
    }

    .classroom-banner__meta {
        font-size: 0.85rem;
    }

    .classroom-banner__actions {
        top: 0.65rem;
        right: 0.65rem;
        left: 0.65rem;
        justify-content: flex-start;
    }
}

html[data-bs-theme="dark"] .classroom-banner:not(.has-photo) {
    background:
        linear-gradient(135deg, rgba(20, 83, 45, 0.95) 0%, rgba(21, 94, 117, 0.92) 55%, rgba(30, 58, 95, 0.95) 100%),
        var(--classroom-banner-image);
    box-shadow: 0 0.35rem 1.25rem rgba(0, 0, 0, 0.35);
}

```

FILE: assets/js/app_nav.js

```

(function () {
    'use strict';

    function ready(fn) {
        if (document.readyState === 'loading') {
            document.addEventListener('DOMContentLoaded', fn);
        } else {
            fn();
        }
    }

    ready(function () {
        var panel = document.getElementById('adminNavOffcanvas');
        var openBtn = document.getElementById('adminNavOpenBtn');
        if (!panel) {
            return;
        }

        var backdrop = null;

```

```

function ensureBackdrop() {
    if (backdrop) {
        return backdrop;
    }
    backdrop = document.createElement('div');
    backdrop.className = 'admin-offcanvas-backdrop no-print';
    document.body.appendChild(backdrop);
    backdrop.addEventListener('click', closePanel);
    return backdrop;
}

function openPanel() {
    ensureBackdrop();
    // Force a reflow so the transition runs from the off-screen state.
    void panel.offsetWidth;
    panel.classList.add('show');
    panel.removeAttribute('aria-hidden');
    backdrop.classList.add('show');
    document.body.classList.add('admin-offcanvas-open', 'overflow-hidden');
    if (openBtn) {
        openBtn.setAttribute('aria-expanded', 'true');
    }
}

function closePanel() {
    panel.classList.remove('show');
    panel.setAttribute('aria-hidden', 'true');
    if (backdrop) {
        backdrop.classList.remove('show');
    }
    document.body.classList.remove('admin-offcanvas-open', 'overflow-hidden');
    if (openBtn) {
        openBtn.setAttribute('aria-expanded', 'false');
    }
}

function togglePanel() {
    if (panel.classList.contains('show')) {
        closePanel();
    } else {
        openPanel();
    }
}

if (openBtn) {
    openBtn.addEventListener('click', function (event) {
        event.preventDefault();
        event.stopPropagation();
        togglePanel();
    });
}

panel.addEventListener('click', function (event) {
    if (event.target.closest('[data-offcanvas-dismiss]')) {
        closePanel();
        return;
    }
    var link = event.target.closest('.admin-nav-link, .admin-sidebar-foot-link');
    if (link && link.getAttribute('href') && link.getAttribute('href') !== '#') {
        closePanel();
    }
});

document.addEventListener('keydown', function (event) {
    if (event.key === 'Escape' && panel.classList.contains('show')) {
        closePanel();
    }
});

window.addEventListener('resize', function () {
    if (window.innerWidth >= 992 && panel.classList.contains('show')) {
        closePanel();
    }
});
})();
})();

```

```

-----
(function () {
    'use strict';

    function shellHasTooltipRole() {
        var c = document.body.classList;
        return (
            c.contains('app-shell-sidebar--student') ||
            c.contains('app-shell-sidebar--admin') ||
            c.contains('app-shell-sidebar--dean') ||
            c.contains('app-shell-sidebar--gened') ||
            c.contains('app-shell-sidebar--faculty')
        );
    }

    if (!shellHasTooltipRole()) {
        return;
    }

    var DELAY = 300;
    var AUTO_DISMISS = 3000;
    var tip = null;
    var timer = null;
    var dismissTimer = null;
    var hovered = null;
    var lastX = 0;
    var lastY = 0;

    function getTip() {
        if (!tip) {
            tip = document.createElement('div');
            tip.id = 'app-cursor-tooltip';
            tip.className = 'app-cursor-tooltip';
            tip.setAttribute('role', 'tooltip');
            tip.setAttribute('aria-hidden', 'true');
            document.body.appendChild(tip);
            tip.style.display = 'none';
        }
        return tip;
    }

    function place() {
        var el = getTip();
        if (el.style.display === 'none' || el.style.display === '') {
            return;
        }
        var pad = 14;
        var m = 10;
        el.style.left = '0px';
        el.style.top = '0px';
        el.style.visibility = 'hidden';
        el.style.display = 'block';
        var w = el.offsetWidth;
        var h = el.offsetHeight;
        var left = lastX + pad;
        var top = lastY + pad;
        if (left + w > window.innerWidth - m) {
            left = Math.max(m, window.innerWidth - w - m);
        }
        if (top + h > window.innerHeight - m) {
            top = Math.max(m, window.innerHeight - h - m);
        }
        el.style.left = left + 'px';
        el.style.top = top + 'px';
        el.style.visibility = 'visible';
    }

    function hideTip() {
        if (tip) {
            tip.style.display = 'none';
            tip.setAttribute('aria-hidden', 'true');
        }
        if (dismissTimer) {
            clearTimeout(dismissTimer);
            dismissTimer = null;
        }
    }

    function showTip(text) {
        var node = getTip();
    }
}

```

```

        node.textContent = text;
        node.style.display = 'block';
        node.setAttribute('aria-hidden', 'false');
        place();
        requestAnimationFrame(place);

        if (dismissTimer) {
            clearTimeout(dismissTimer);
        }
        dismissTimer = setTimeout(hideTip, AUTO_DISMISS);
    }
}

document.addEventListener(
    'mouseover',
    function (e) {
        var el = e.target.closest('[data-app-tooltip]');
        if (!el) {
            return;
        }
        if (hovered === el) {
            return;
        }
        hovered = el;
        if (timer) {
            clearTimeout(timer);
            timer = null;
        }
        lastX = e.clientX;
        lastY = e.clientY;
        timer = setTimeout(function () {
            timer = null;
            if (hovered !== el) {
                return;
            }
            var text = el.getAttribute('data-app-tooltip');
            if (!text) {
                return;
            }
            showTip(text);
        }, DELAY);
    },
    true
);

document.addEventListener(
    'mousemove',
    function (e) {
        lastX = e.clientX;
        lastY = e.clientY;
        if (!tip || tip.style.display === 'none') {
            return;
        }
        place();
    },
    true
);

document.addEventListener(
    'mouseout',
    function (e) {
        var el = e.target.closest('[data-app-tooltip]');
        if (!el) {
            return;
        }
        var rel = e.relatedTarget;
        if (rel && el.contains(rel)) {
            return;
        }
        if (hovered === el) {
            hovered = null;
        }
        if (timer) {
            clearTimeout(timer);
            timer = null;
        }
        hideTip();
    },
    true
);

```

```

document.addEventListener('click', function (e) {
    var el = e.target.closest('[data-app-tooltip]');
    if (el) {
        hideTip();
        hovered = null;
    }
}, true);

document.addEventListener('scroll', hideTip, true);
})();

```

FILE: assets/js/assessment_integrity.js

```

/**
 * Ends an in-progress assessment when the student leaves the window
 * (tab switch, minimize, blur, or close).
 */
(function () {
    if (window.__assessmentIntegrityLoaded) {
        return;
    }
    window.__assessmentIntegrityLoaded = true;

    var INTEGRITY_WARNING =
        'Warning: Your assessment has ended because you left or minimized the assessment window, closed it, or switched to another tab or application.\n\nYour answers will be submitted now.';
    var activeController = null;

    function storageKey(assessmentId, studentId) {
        return 'class_assessment_timer_' + studentId + '_' + assessmentId;
    }

    function formatDigits(totalSeconds) {
        var s = Math.max(0, Math.floor(totalSeconds));
        var hh = String(Math.floor(s / 3600)).padStart(2, '0');
        var mm = String(Math.floor((s % 3600) / 60)).padStart(2, '0');
        var ss = String(s % 60).padStart(2, '0');
        return hh + ':' + mm + ':' + ss;
    }

    function readStart(key) {
        try {
            var raw = localStorage.getItem(key);
            if (!raw) return null;
            var n = parseInt(raw, 10);
            return isNaN(n) ? null : n;
        } catch (e) {
            return null;
        }
    }

    function writeStart(key, ts) {
        try {
            localStorage.setItem(key, String(ts));
        } catch (e) {}
    }

    function clearStart(key) {
        try {
            localStorage.removeItem(key);
        } catch (e) {}
    }

    function showIntegrityWarning(onDone) {
        var warnEl = document.getElementById('assessmentIntegrityWarningModal');
        var finished = false;
        function finish() {
            if (finished) return;
            finished = true;
            onDone();
        }
        if (!warnEl || typeof bootstrap === 'undefined') {
            try {
                window.alert(INTEGRITY_WARNING);
            } catch (e) {}
            finish();
            return;
        }
    }
}

```

```

var warnModal = bootstrap.Modal.getOrCreateInstance(warnEl);
var ackBtn = warnEl.querySelector('[data-integrity-ack]');
function onAck() {
    if (ackBtn) ackBtn.removeEventListener('click', onAck);
    warnEl.removeEventListener('hidden.bs.modal', onHidden);
    try {
        warnModal.hide();
    } catch (e) {}
    finish();
}
function onHidden() {
    onAck();
}
if (ackBtn) ackBtn.addEventListener('click', onAck);
warnEl.addEventListener('hidden.bs.modal', onHidden);
warnModal.show();
setTimeout(onAck, 2500);
}

function setupModal(modal) {
    var timed = modal.getAttribute('data-timed-attempt') === '1';
    var alreadySubmitted = modal.getAttribute('data-already-submitted') === '1';
    var integrityLocked = modal.getAttribute('data-integrity-locked') === '1';
    var assessmentId = modal.getAttribute('data-assessment-id');
    var studentId = modal.getAttribute('data-student-id');
    var limitMinutes = parseInt(modal.getAttribute('data-time-limit-minutes') || '0', 10) || 0;
    var key = storageKey(assessmentId, studentId);
    var limitMs = limitMinutes * 60 * 1000;
    var gate = modal.querySelector('[data-timer-gate]');
    var body = modal.querySelector('[data-timer-body]');
    var display = modal.querySelector('[data-timer-display]');
    var digits = modal.querySelector('[data-timer-digits]');
    var startBtn = modal.querySelector('[data-timer-start]');
    var form = modal.querySelector('[data-assessment-form]');
    var integrityField = form ? form.querySelector('[data-integrity-field]') : null;
    var closeBtn = modal.querySelector('.btn-close');
    var tickTimer = null;
    var submitting = false;
    var monitoring = false;
    var blurTimer = null;
    var allowModalHide = false;

    var controller = {
        isMonitoring: function () {
            return monitoring && !submitting;
        },
        endForIntegrity: endForIntegrity
    };

    function stopTick() {
        if (tickTimer) {
            clearInterval(tickTimer);
            tickTimer = null;
        }
    }

    function isAttemptActive() {
        if (!form || submitting || alreadySubmitted || integrityLocked) return false;
        if (body && body.hidden) return false;
        return modal.classList.contains('show');
    }

    function stopMonitoring() {
        monitoring = false;
        if (activeController === controller) {
            activeController = null;
        }
        if (blurTimer) {
            clearTimeout(blurTimer);
            blurTimer = null;
        }
    }

    function startMonitoring() {
        if (monitoring || submitting || !form || alreadySubmitted || integrityLocked) return;
        monitoring = true;
        activeController = controller;
        modal.setAttribute('data-integrity-monitoring', '1');
    }
}

```



```

function prepareForcedSubmit(asIntegrity) {
    if (!form) return;
    if (integrityField) {
        integrityField.value = asIntegrity ? '1' : '0';
    }
    form.querySelectorAll('[required]').forEach(function (el) {
        el.removeAttribute('required');
    });
    form.setAttribute('novalidate', 'novalidate');
}

function doFormSubmit(asIntegrity) {
    if (!form) return;
    var isIntegrity = !!asIntegrity;

    var classroomId = '';
    var classroomInput = form.querySelector('input[name="classroom_id"]');
    if (classroomInput) {
        classroomId = String(classroomInput.value || '');
    }

    var redirectTo = 'student_classroom.php' + (classroomId !== '' ? ('?id=' + encodeURIComponent(classroomId)) : '');
    var postUrl = form.getAttribute('action') || window.location.pathname + window.location.search;

    try {
        if (integrityField) {
            integrityField.value = isIntegrity ? '1' : '0';
        }
        var fd = new FormData(form);
        fd.set('action', 'submit_assessment');
        fd.set('integrity_violation', isIntegrity ? '1' : '0');
        if (classroomId !== '') {
            fd.set('classroom_id', classroomId);
        }
        // keepalive allows the request to finish even when the tab is hidden/closing.
        fetch(postUrl, {
            method: 'POST',
            body: fd,
            credentials: 'same-origin',
            keepalive: true,
            redirect: 'follow'
        })
        .finally(function () {
            window.location.href = redirectTo;
        });
    } catch (e) {
        try {
            HTMLFormElement.prototype.submit.call(form);
        } catch (e2) {
            form.submit();
        }
    }
}

function forceSubmit(asIntegrity) {
    if (submitting || !form) return;
    submitting = true;
    stopTick();
    stopMonitoring();
    clearStart(key);
    prepareForcedSubmit(!asIntegrity);
    allowModalHide = true;
    doFormSubmit(!asIntegrity);
}

function endForIntegrity() {
    if (submitting || !form) return;
    if (!monitoring && !isAttemptActive()) return;
    submitting = true;
    stopTick();
    stopMonitoring();
    clearStart(key);
    prepareForcedSubmit(true);
    allowModalHide = true;
    modal.removeAttribute('data-integrity-monitoring');

    // Submit immediately in background – alert/modals are deferred when the tab is hidden.
    if (document.visibilityState === 'hidden' || !document.hasFocus()) {
        doFormSubmit(true);
        return;
    }
}

```

```

    }

    showIntegrityWarning(function () {
        doFormSubmit(true);
    });
}

function revealBody() {
    if (gate) gate.classList.add('d-none');
    if (body) body.hidden = false;
    if (display) display.classList.remove('d-none');
}

function updateDisplay(remainingSec) {
    if (!digits || !display) return;
    digits.textContent = formatDigits(remainingSec);
    display.classList.toggle('is-warning', remainingSec <= 300 && remainingSec > 60);
    display.classList.toggle('is-danger', remainingSec <= 60);
}

function autoSubmit() {
    if (digits) digits.textContent = '00:00:00';
    if (display) display.classList.add('is-danger');
    forceSubmit(false);
}

function beginCountdown(startedAt) {
    revealBody();
    startMonitoring();
    var endAt = startedAt + limitMs;

    function tick() {
        var remainingMs = endAt - Date.now();
        var remainingSec = Math.ceil(remainingMs / 1000);
        updateDisplay(remainingSec);
        if (remainingMs <= 0) autoSubmit();
    }

    stopTick();
    tick();
    tickTimer = setInterval(tick, 250);
}

function startAttempt() {
    var now = Date.now();
    writeStart(key, now);
    beginCountdown(now);
}

if (startBtn) {
    startBtn.addEventListener('click', function () {
        startAttempt();
    });
}

if (closeBtn) {
    closeBtn.addEventListener(
        'click',
        function (e) {
            if (!isAttemptActive() || submitting) return;
            e.preventDefault();
            e.stopImmediatePropagation();
            endForIntegrity();
        },
        true
    );
}

if (form) {
    form.addEventListener('submit', function () {
        submitting = true;
        stopMonitoring();
        clearStart(key);
        stopTick();
        allowModalHide = true;
    });
}

modal.addEventListener('hide.bs.modal', function (e) {
    if (allowModalHide || submitting) return;

```

```

        if (isAttemptActive()) {
            e.preventDefault();
            endForIntegrity();
        }
    });

    modal.addEventListener('shown.bs.modal', function () {
        if (modal.getAttribute('data-clear-timer') === '1') {
            clearStart(key);
        }
        if (timed && limitMinutes >= 1) {
            var startedAt = readStart(key);
            if (startedAt) {
                var remaining = startedAt + limitMs - Date.now();
                if (remaining <= 0) {
                    revealBody();
                    updateDisplay(0);
                    autoSubmit();
                } else {
                    beginCountdown(startedAt);
                }
            }
            return;
        }

        if (body && !body.hidden) {
            startMonitoring();
        }
    });

    modal.addEventListener('hidden.bs.modal', function () {
        stopTick();
        stopMonitoring();
        allowModalHide = false;
        modal.removeAttribute('data-integrity-monitoring');
    });
}

function boot() {
    var hash = window.location.hash;
    if (hash && hash.indexOf('#assessment-') === 0) {
        var id = hash.substring('#assessment-'.length);
        var modalEl = document.getElementById('assessmentModal' + id);
        if (modalEl && typeof bootstrap !== 'undefined') {
            bootstrap.Modal.getOrCreateInstance(modalEl).show();
        }
    }

    document.querySelectorAll('.assessment-take-modal').forEach(setupModal);

    document.addEventListener('visibilitychange', function () {
        if (activeController && activeController.isMonitoring() && document.visibilityState === 'hidden') {
            activeController.endForIntegrity();
        }
    });

    window.addEventListener('blur', function () {
        if (!activeController || !activeController.isMonitoring()) return;
        if (document.visibilityState === 'hidden') {
            activeController.endForIntegrity();
            return;
        }
        setTimeout(function () {
            if (!activeController || !activeController.isMonitoring()) return;
            if (document.visibilityState === 'hidden' || !document.hasFocus()) {
                activeController.endForIntegrity();
            }
        }, 50);
    });

    window.addEventListener('pagehide', function () {
        if (activeController && activeController.isMonitoring()) {
            activeController.endForIntegrity();
        }
    });
}

if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', boot);
} else {

```

```

        boot();
    }
})();

```

FILE: assets/js/faculty_offline.js

```

(function () {
    'use strict';

    var DB_NAME = 'class_faculty_offline';
    var DB_VERSION = 1;
    var STORE = 'queue';
    var API_URL = 'api/faculty_offline_upload.php';
    var MAX_FILE_BYTES = 10 * 1024 * 1024;
    var syncing = false;

    function openDb() {
        return new Promise(function (resolve, reject) {
            if (!window.indexedDB) {
                reject(new Error('This browser does not support offline storage.'));
                return;
            }
            var req = indexedDB.open(DB_NAME, DB_VERSION);
            req.onupgradeneeded = function () {
                var db = req.result;
                if (!db.objectStoreNames.contains(STORE)) {
                    db.createObjectStore(STORE, { keyPath: 'id' });
                }
            };
            req.onsuccess = function () {
                resolve(req.result);
            };
            req.onerror = function () {
                reject(req.error || new Error('Could not open offline storage.'));
            };
        });
    }

    function withStore(mode, fn) {
        return openDb().then(function (db) {
            return new Promise(function (resolve, reject) {
                var tx = db.transaction(STORE, mode);
                var store = tx.objectStore(STORE);
                var result;
                try {
                    result = fn(store);
                } catch (err) {
                    db.close();
                    reject(err);
                    return;
                }
                tx.oncomplete = function () {
                    db.close();
                    resolve(result);
                };
                tx.onerror = function () {
                    db.close();
                    reject(tx.error || new Error('Offline storage error.'));
                };
            });
        });
    }

    function idbPut(record) {
        return withStore('readwrite', function (store) {
            store.put(record);
            return record;
        });
    }

    function idbDelete(id) {
        return withStore('readwrite', function (store) {
            store.delete(id);
        });
    }

    function listQueue() {
        return openDb().then(function (db) {

```

```

return new Promise(function (resolve, reject) {
  var tx = db.transaction(STORE, 'readonly');
  var req = tx.objectStore(STORE).getAll();
  req.onsuccess = function () {
    var rows = req.result || [];
    rows.sort(function (a, b) {
      return String(a.created_at || '').localeCompare(String(b.created_at || ''));
    });
    resolve(rows);
  };
  req.onerror = function () {
    reject(req.error || new Error('Could not read offline queue.'));
  };
  tx.oncomplete = function () {
    db.close();
  };
  tx.onerror = function () {
    db.close();
  };
});
});
}

function readClassroomOptions() {
  var el = document.getElementById('offline-classroom-options');
  if (!el) {
    return [];
  }
  try {
    var parsed = JSON.parse(el.textContent || '[]');
    return Array.isArray(parsed) ? parsed : [];
  } catch (err) {
    return [];
  }
}

function classroomOptionLabel(option) {
  return String(option && option.label ? option.label : 'Classroom');
}

function reassignQueueItem(item, targetClassroomId) {
  var options = readClassroomOptions();
  var target = options.find(function (option) {
    return Number(option.id) === Number(targetClassroomId);
  });
  if (!target) {
    return Promise.reject(new Error('Choose a valid course.'));
  }
  if (Number(item.classroom_id) === Number(target.id)) {
    return Promise.resolve(item);
  }
  if (item.status === 'syncing') {
    return Promise.reject(new Error('Wait until this upload finishes syncing.'));
  }

  var code = String(target.course_code || '').trim();
  var name = String(target.course_name || '').trim();
  var classroomTitle = String(target.classroom_title || 'Classroom').trim();
  var label = classroomOptionLabel(target);

  item.classroom_id = Number(target.id);
  item.course_code = code;
  item.course_name = name;
  item.classroom_title = classroomTitle;
  item.classroom_label = label;
  item.status = 'pending';
  item.error = '';
  item.updated_at = new Date().toISOString();

  return idbPut(item);
}

function uuid() {
  if (window.crypto && typeof window.crypto.randomUUID === 'function') {
    return window.crypto.randomUUID();
  }
  return 'fou-' + Date.now().toString(36) + '-' + Math.random().toString(36).slice(2, 12);
}

function registerServiceWorker() {

```

```

    if (!('serviceWorker' in navigator)) {
        return Promise.resolve(null);
    }
    return navigator.serviceWorker.register('sw-faculty-offline.js').catch(function () {
        return null;
    });
}

function requestBackgroundSync() {
    if (!('serviceWorker' in navigator) || !navigator.serviceWorker.ready) {
        return Promise.resolve();
    }
    return navigator.serviceWorker.ready
        .then(function (reg) {
            if (reg.sync && typeof reg.sync.register === 'function') {
                return reg.sync.register('faculty-offline-upload-sync');
            }
            return null;
        })
        .catch(function () {
            return null;
        });
}

function setStatus(el, message, kind) {
    if (!el) return;
    el.textContent = message || '';
    el.classList.remove('text-success', 'text-danger', 'text-muted', 'text-primary', 'text-warning');
    if (kind === 'ok') el.classList.add('text-success');
    else if (kind === 'err') el.classList.add('text-danger');
    else if (kind === 'busy') el.classList.add('text-primary');
    else if (kind === 'warn') el.classList.add('text-warning');
    else el.classList.add('text-muted');
}

function formatWhen(iso) {
    if (!iso) return '-';
    var d = new Date(iso);
    if (Number.isNaN(d.getTime())) return String(iso);
    return d.toLocaleString();
}

function escapeHtml(value) {
    return String(value == null ? '' : value)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&#39;');
}

function fileListFromInput(input) {
    var files = [];
    if (!input || !input.files) return Promise.resolve(files);
    var list = Array.prototype.slice.call(input.files || []);
    return Promise.all(
        list.map(function (file) {
            if (file.size > MAX_FILE_BYTES) {
                return Promise.reject(new Error('"' + file.name + '" is larger than 10 MB.));
            }
            return file.arrayBuffer().then(function (buf) {
                return {
                    name: file.name,
                    type: file.type || 'application/octet-stream',
                    size: file.size,
                    buffer: buf,
                };
            });
        })
    );
}

function buffersToFiles(fileRecords) {
    return (fileRecords || []).map(function (rec) {
        var blob = new Blob([rec.buffer], { type: rec.type || 'application/octet-stream' });
        return new File([blob], rec.name || 'attachment', {
            type: rec.type || 'application/octet-stream',
            lastModified: Date.now(),
        });
    });
}

```

```

}

function offlineMetaFromPage() {
  var el = document.querySelector('[data-offline-classroom-label]');
  var courseCode = '';
  var courseName = '';
  var classroomTitle = '';
  var label = '';
  if (el) {
    courseCode = (el.getAttribute('data-offline-course-code') || '').trim();
    courseName = (el.getAttribute('data-offline-course-name') || '').trim();
    classroomTitle = (el.getAttribute('data-offline-classroom-title') || '').trim();
    label = (el.getAttribute('data-offline-classroom-label') || el.textContent || '').trim();
  }
  if (!label) {
    var h1 = document.querySelector('h1');
    label = h1 ? (h1.textContent || '').trim() : 'Classroom';
  }
  if (!courseCode && label.indexOf(' - ') !== -1) {
    courseCode = label.split(' - ')[0].trim();
  }
  if (!classroomTitle && label.indexOf(' - ') !== -1) {
    classroomTitle = label.split(' - ').slice(1).join(' - ').trim();
  }
  return {
    course_code: courseCode,
    course_name: courseName,
    classroom_title: classroomTitle || 'Classroom',
    classroom_label: label || 'Classroom',
  };
}

function courseGroupKey(item) {
  var code = String(item && item.course_code ? item.course_code : '').trim();
  var name = String(item && item.course_name ? item.course_name : '').trim();
  if (code || name) {
    return code + '|' + name;
  }
  var label = String(item && item.classroom_label ? item.classroom_label : '').trim();
  if (label.indexOf(' - ') !== -1) {
    var parsedCode = label.split(' - ')[0].trim();
    if (parsedCode) {
      return parsedCode + '|';
    }
  }
  if (label) {
    return 'label:' + label;
  }
  return 'classroom:' + String(item && item.classroom_id ? item.classroom_id : 'unknown');
}

function courseGroupHeading(item) {
  var code = String(item && item.course_code ? item.course_code : '').trim();
  var name = String(item && item.course_name ? item.course_name : '').trim();
  if (code && name) {
    return code + ' - ' + name;
  }
  if (code) {
    return code;
  }
  if (name) {
    return name;
  }
  var label = String(item && item.classroom_label ? item.classroom_label : '').trim();
  if (label.indexOf(' - ') !== -1) {
    return label.split(' - ')[0].trim() || label;
  }
  return label || 'Classroom #' + String(item && item.classroom_id ? item.classroom_id : '');
}

function groupQueueByCourse(items) {
  var groups = {};
  var order = [];
  (items || []).forEach(function (item) {
    var key = courseGroupKey(item);
    if (!groups[key]) {
      groups[key] = {
        key: key,
        heading: courseGroupHeading(item),
        items: [],
      };
    }
  });
}

```

```

    });
    order.push(key);
  }
  groups[key].items.push(item);
});
order.sort(function (a, b) {
  return String(groups[a].heading).localeCompare(String(groups[b].heading));
});
return order.map(function (key) {
  return groups[key];
});
}

function enqueueRecord(record) {
  return idbPut(record).then(function () {
    return registerServiceWorker().then(function () {
      return requestBackgroundSync();
    }).then(function () {
      updatePendingBadges();
      return record;
    });
  });
}

function enqueueAddContent(form) {
  var classroomId = parseInt(form.querySelector('[name="classroom_id"]')?.value || '0', 10);
  var title = (form.querySelector('[name="title"]')?.value || '').trim();
  var contentType = form.querySelector('[name="content_type"]')?.value || 'material';
  var body = form.querySelector('[name="body"]')?.value || '';
  var weeks = form.querySelector('[name="weeks"]')?.value || '';
  var daysPerTopic = form.querySelector('[name="days_per_topic"]')?.value || '';
  var resourceUrl = (form.querySelector('[name="resource_url"]')?.value || '').trim();
  var fileInput = form.querySelector('[name="attachments[]"]') || form.querySelector('#fc-attachments');
  var meta = offlineMetaFromPage();

  if (!classroomId) {
    return Promise.reject(new Error('Missing classroom.'));
  }
  if (!title) {
    return Promise.reject(new Error('Content title is required.'));
  }

  return fileListFromInput(fileInput).then(function (files) {
    if (!body.trim() && !resourceUrl && files.length === 0) {
      throw new Error('Add a short description, a resource URL, or at least one attachment.');
    }
    var record = {
      id: uuid(),
      action: 'add_content',
      classroom_id: classroomId,
      classroom_label: meta.classroom_label,
      course_code: meta.course_code,
      course_name: meta.course_name,
      classroom_title: meta.classroom_title,
      content_type: contentType,
      title: title,
      body: body,
      weeks: weeks,
      days_per_topic: daysPerTopic,
      resource_url: resourceUrl,
      files: files,
      status: 'pending',
      error: '',
      created_at: new Date().toISOString(),
      synced_at: null,
    };
    return enqueueRecord(record);
  });
}

function enqueueSyllabus(form) {
  var classroomId = parseInt(form.querySelector('[name="classroom_id"]')?.value || '0', 10);
  var fileInput = form.querySelector('[name="syllabus"]') || form.querySelector('#fc-syllabus-file');
  var meta = offlineMetaFromPage();
  if (!classroomId) {
    return Promise.reject(new Error('Missing classroom.'));
  }
  if (!fileInput || !fileInput.files || !fileInput.files.length) {
    return Promise.reject(new Error('Please choose a syllabus file to upload.'));
  }
}

```



```

return fileListFromInput(fileInput).then(function (files) {
  if (!files.length) {
    throw new Error('Please choose a syllabus file to upload.');
```

```

  }
  var record = {
    id: uuid(),
    action: 'upload_syllabus',
    classroom_id: classroomId,
    classroom_label: meta.classroom_label,
    course_code: meta.course_code,
    course_name: meta.course_name,
    classroom_title: meta.classroom_title,
    content_type: 'syllabus',
    title: files[0].name || 'Syllabus',
    body: '',
    weeks: '',
    days_per_topic: '',
    resource_url: '',
    files: files,
    status: 'pending',
    error: '',
    created_at: new Date().toISOString(),
    synced_at: null,
  };
  return enqueueRecord(record);
});
}

function resetAddContentForm(form) {
  form.reset();
  var editor = form.querySelector('.wordpad-editor');
  var textarea = form.querySelector('textarea[name="body"]');
  if (editor) editor.innerHTML = '';
  if (textarea) textarea.value = '';
}

function postQueueItem(item) {
  var fd = new FormData();
  fd.append('action', item.action);
  fd.append('classroom_id', String(item.classroom_id));
  fd.append('client_uuid', item.id);

  if (item.action === 'upload_syllabus') {
    var sylFiles = buffersToFiles(item.files);
    if (!sylFiles.length) {
      return Promise.reject(new Error('Missing syllabus file in offline queue.));
    }
    fd.append('syllabus', sylFiles[0], sylFiles[0].name);
  } else {
    fd.append('content_type', item.content_type || 'material');
    fd.append('title', item.title || '');
    fd.append('body', item.body || '');
    fd.append('weeks', item.weeks || '');
    fd.append('days_per_topic', item.days_per_topic || '');
    fd.append('resource_url', item.resource_url || '');
    buffersToFiles(item.files).forEach(function (file) {
      fd.append('attachments[]', file, file.name);
    });
  }
}

return fetch(API_URL, {
  method: 'POST',
  credentials: 'same-origin',
  body: fd,
  headers: { Accept: 'application/json' },
}).then(function (res) {
  return res.json().then(function (body) {
    if (!res.ok || !body || !body.ok) {
      throw new Error((body && body.error) || 'Sync failed.');
```

```

    }
    return body;
  });
});
}

function flushQueue() {
  if (syncing) {
    return Promise.resolve({ synced: 0, failed: 0, skipped: true });
  }
  if (!navigator.onLine) {

```

```

    return Promise.resolve({ synced: 0, failed: 0, offline: true });
}

syncing = true;
return listQueue()
    .then(function (items) {
        var pending = items.filter(function (item) {
            return item.status === 'pending' || item.status === 'error';
        });
        // Syllabus first so content posts are not blocked by the syllabus gate.
        pending.sort(function (a, b) {
            var aw = a.action === 'upload_syllabus' ? 0 : 1;
            var bw = b.action === 'upload_syllabus' ? 0 : 1;
            if (aw !== bw) return aw - bw;
            return String(a.created_at || '').localeCompare(String(b.created_at || ''));
        });
        var synced = 0;
        var failed = 0;

        var chain = Promise.resolve();
        pending.forEach(function (item) {
            chain = chain.then(function () {
                item.status = 'syncing';
                item.error = '';
                return idbPut(item)
                    .then(function () {
                        return postQueueItem(item);
                    })
                    .then(function () {
                        synced += 1;
                        return idbDelete(item.id);
                    })
                    .catch(function (err) {
                        failed += 1;
                        item.status = 'error';
                        item.error = err && err.message ? err.message : 'Sync failed.';
                        return idbPut(item);
                    });
            });
        });

        return chain.then(function () {
            return { synced: synced, failed: failed, skipped: false, pending: pending.length };
        });
    })
    .finally(function () {
        syncing = false;
        updatePendingBadges();
        var mount = document.querySelector('[data-offline-queue]');
        if (mount) {
            renderQueue(mount);
        }
    });
}

function updatePendingBadges() {
    listQueue()
        .then(function (items) {
            var count = items.filter(function (item) {
                return item.status === 'pending' || item.status === 'error' || item.status === 'syncing';
            }).length;
            document.querySelectorAll('[data-offline-pending-count]').forEach(function (el) {
                el.textContent = String(count);
                el.hidden = count < 1;
            });
            document.querySelectorAll('[data-offline-pending-label]').forEach(function (el) {
                if (count < 1) {
                    el.textContent = 'No pending offline uploads.';
                } else {
                    el.textContent =
                        count +
                        ' upload' +
                        (count === 1 ? '' : 's') +
                        ' waiting to sync' +
                        (navigator.onLine ? '' : ' (offline).');
                }
            });
        })
        .catch(function () {});
}

```

```

function renderQueue(mount) {
  if (!mount) return;
  mount.innerHTML = '<p class="text-muted">Loading offline upload queue...</p>';
  listQueue()
    .then(function (items) {
      var online = navigator.onLine;
      var html = '';
      html += '<div class="alert ' + (online ? 'alert-success' : 'alert-info') + '>';
      html += online
        ? '<i class="fa-solid fa-wifi me-1"></i>You are online. Pending uploads sync automatically.'
        : '<i class="fa-solid fa-plane me-1"></i>You are offline. New posts and files stay on this device until you reconnect.';
      html += '</div>';

      if (!items.length) {
        html +=
          '<div class="alert alert-light border mb-0">No queued uploads. On a classroom page, publish content or save a syllabus while
offline – they appear here, then sync when you are back online.</div>';
        mount.innerHTML = html;
        return;
      }

      html += '<p class="small text-muted mb-3">Open a course box to see its queued uploads. You can assign a queued item to another
course you teach before it syncs.</p>';
      var classroomOptions = readClassroomOptions();
      groupQueueByCourse(items).forEach(function (courseGroup, index) {
        var collapseId = 'offline-queue-course-' + index;
        html += '<div class="card shadow-sm mb-3 offline-course-box">';
        html +=
          '<button type="button" class="btn btn-link text-start text-decoration-none text-body w-100 p-0" data-bs-toggle="collapse" data-
bs-target="#' +
          escapeHtml(collapseId) +
          '" aria-expanded="false" aria-controls="' +
          escapeHtml(collapseId) +
          '">';
        html += '<div class="card-body d-flex justify-content-between gap-3 align-items-center">';
        html += '<div class="min-w-0">';
        html +=
          '<div class="fw-semibold mb-1"><i class="fa-solid fa-book-open me-2 text-primary"></i>' +
          escapeHtml(courseGroup.heading) +
          '</div>';
        html +=
          '<div class="small text-muted">' +
          escapeHtml(String(courseGroup.items.length)) +
          ' upload' +
          (courseGroup.items.length === 1 ? '' : 's') +
          ' waiting</div>';
        html += '</div>';
        html +=
          '<span class="badge text-bg-light border text-muted flex-shrink-0"><i class="fa-solid fa-chevron-down"></i></span>';
        html += '</div></button>';
        html += '<div class="collapse" id="' + escapeHtml(collapseId) + '>';
        html += '<div class="list-group list-group-flush border-top">';
        courseGroup.items.forEach(function (item) {
          var status = item.status || 'pending';
          var badgeClass =
            status === 'error'
              ? 'text-bg-danger'
              : status === 'syncing'
                ? 'text-bg-primary'
                : 'text-bg-warning';
          var fileNames = (item.files || []).
            .map(function (f) {
              return f.name;
            })
            .filter(Boolean);
          var classroomBit =
            item.classroom_title ||
            item.classroom_label ||
            'Classroom #' + item.classroom_id;
          html += '<div class="list-group-item" data-queue-id="' + escapeHtml(item.id) + '>';
          html += '<div class="d-flex justify-content-between gap-2 flex-wrap align-items-start">';
          html += '<div class="min-w-0">';
          html += '<div class="fw-semibold">' + escapeHtml(item.title || 'Untitled') + '</div>';
          html +=
            '<div class="small text-muted">' +
            escapeHtml(classroomBit) +
            ' . ' +
            escapeHtml(item.action === 'upload_syllabus' ? 'Syllabus' : item.content_type || 'material') +
            '</div>';

```

```

    if (fileNames.length) {
        html +=
            '<div class="small mt-1"><i class="fa-solid fa-paperclip me-1"></i> ' +
            escapeHtml(fileNames.join(', ')) +
            '</div>';
    }
    if (item.error) {
        html += '<div class="small text-danger mt-1">' + escapeHtml(item.error) + '</div>';
    }
    html += '<div class="small text-muted mt-1">Queued ' + escapeHtml(formatWhen(item.created_at)) + '</div>';
    html += '</div>';
    html += '<span class="badge ' + badgeClass + '">' + escapeHtml(status) + '</span>';
    html += '</div>';
    html +=
        '<div class="mt-2 d-flex flex-wrap gap-2 align-items-center"><button type="button" class="btn btn-sm btn-outline-danger" data-
offline-discard="' +
        escapeHtml(item.id) +
        '">Discard</button>';
    if (classroomOptions.length > 1 && status !== 'syncing') {
        html +=
            '<label class="small text-muted mb-0 me-1">Assign to</label><select class="form-select form-select-sm w-auto" data-offline-
reassign="' +
            escapeHtml(item.id) +
            '">';
        classroomOptions.forEach(function (option) {
            var selected = Number(option.id) === Number(item.classroom_id) ? ' selected' : '';
            html +=
                '<option value="' +
                escapeHtml(String(option.id)) +
                '"' +
                selected +
                '>' +
                escapeHtml(classroomOptionLabel(option)) +
                '</option>';
        });
        html += '</select>';
    }
    html += '</div>';
    html += '</div>';
});
html += '</div></div></div>';
});
mount.innerHTML = html;

mount.querySelectorAll('[data-offline-discard]').forEach(function (btn) {
    btn.addEventListener('click', function () {
        var id = btn.getAttribute('data-offline-discard');
        if (!id || !window.confirm('Discard this queued upload from this device?')) return;
        idbDelete(id).then(function () {
            updatePendingBadges();
            renderQueue(mount);
        });
    });
});

mount.querySelectorAll('[data-offline-reassign]').forEach(function (selectEl) {
    selectEl.setAttribute('data-previous-value', selectEl.value || '');
    selectEl.addEventListener('change', function () {
        var id = selectEl.getAttribute('data-offline-reassign');
        var targetId = parseInt(selectEl.value || '0', 10);
        var previousValue = selectEl.getAttribute('data-previous-value') || '';
        if (!id || targetId < 1) {
            return;
        }
        listQueue()
            .then(function (rows) {
                var item = rows.find(function (row) {
                    return String(row.id) === String(id);
                });
                if (!item) {
                    throw new Error('Queued upload not found.');
```

```

        window.alert(err && err.message ? err.message : 'Could not assign queued upload.');
```

```

    });
    });
    });
    .catch(function (err) {
        mount.innerHTML =
            '<div class="alert alert-danger mb-0">' +
            escapeHtml(err && err.message ? err.message : 'Could not load offline queue.') +
            '</div>';
    });
}

function bindAddContentForm(form) {
    if (!form || form.dataset.offlineBound === '1') return;
    form.dataset.offlineBound = '1';
    form.addEventListener(
        'submit',
        function (event) {
            if (navigator.onLine) {
                return;
            }
            event.preventDefault();
            event.stopPropagation();

            var shell = form.querySelector('[data-wordpad]');
            if (shell) {
                var editor = shell.querySelector('.wordpad-editor');
                var textarea = shell.querySelector('textarea[name="body"]');
                if (editor && textarea) {
                    textarea.value = editor.innerHTML
                        .replace(/<div><br><\div>/gi, '')
                        .replace(/&nbsp;/gi, ' ')
                        .trim();
                }
            }

            var statusEl = document.querySelector('[data-offline-status]');
            var submitBtn = form.querySelector('[type="submit"]');
            var original = submitBtn ? submitBtn.innerHTML : '';
            if (submitBtn) {
                submitBtn.disabled = true;
                submitBtn.innerHTML = '<i class="fa-solid fa-spinner fa-spin me-1"></i>Saving offline...';
            }
            setStatus(statusEl, 'Saving post and files on this device...', 'busy');

            enqueueAddContent(form)
                .then(function () {
                    resetAddContentForm(form);
                    setStatus(
                        statusEl,
                        'Saved offline. Open Offline uploads or reconnect to sync to the class.',
                        'ok'
                    );
                })
                .then(function () {
                    if (submitBtn) {
                        submitBtn.innerHTML = '<i class="fa-solid fa-check me-1"></i>Saved offline';
                        window.setTimeout(function () {
                            submitBtn.disabled = false;
                            submitBtn.innerHTML = original;
                        }, 1800);
                    }
                })
                .catch(function (err) {
                    setStatus(statusEl, err && err.message ? err.message : 'Could not save offline.', 'err');
                    if (submitBtn) {
                        submitBtn.disabled = false;
                        submitBtn.innerHTML = original;
                    }
                });
        },
        true
    );
}

function bindSyllabusForm(form) {
    if (!form || form.dataset.offlineBound === '1') return;
    form.dataset.offlineBound = '1';
    form.addEventListener(
        'submit',
        function (event) {

```

```

    if (navigator.onLine) {
        return;
    }
    event.preventDefault();
    event.stopPropagation();

    var statusEl = document.querySelector('[data-offline-status]');
    var submitBtn = form.querySelector('[type="submit"]');
    var original = submitBtn ? submitBtn.innerHTML : '';
    if (submitBtn) {
        submitBtn.disabled = true;
        submitBtn.innerHTML = '<i class="fa-solid fa-spinner fa-spin me-1"></i>Saving offline...';
    }
    setStatus(statusEl, 'Saving syllabus on this device...', 'busy');

    enqueueSyllabus(form)
        .then(function () {
            form.reset();
            setStatus(
                statusEl,
                'Syllabus saved offline. It will upload when you are back online.',
                'ok'
            );
            if (submitBtn) {
                submitBtn.innerHTML = '<i class="fa-solid fa-check me-1"></i>Saved offline';
                window.setTimeout(function () {
                    submitBtn.disabled = false;
                    submitBtn.innerHTML = original;
                }, 1800);
            }
        })
        .catch(function (err) {
            setStatus(statusEl, err && err.message ? err.message : 'Could not save offline.', 'err');
            if (submitBtn) {
                submitBtn.disabled = false;
                submitBtn.innerHTML = original;
            }
        });
    },
    true
);
}

function bindControls(root) {
    root = root || document;
    var addForm = root.querySelector('#fc-add-content-form');
    if (addForm) {
        bindAddContentForm(addForm);
    }
    root.querySelectorAll('form').forEach(function (form) {
        var action = form.querySelector('input[name="action"]');
        if (action && action.value === 'upload_syllabus') {
            bindSyllabusForm(form);
        }
    });
}

root.querySelectorAll('[data-offline-sync-now]').forEach(function (btn) {
    if (btn.dataset.offlineBound === '1') return;
    btn.dataset.offlineBound = '1';
    btn.addEventListener('click', function () {
        var statusEl = root.querySelector('[data-offline-status]');
        var original = btn.innerHTML;
        btn.disabled = true;
        btn.innerHTML = '<i class="fa-solid fa-spinner fa-spin me-1"></i>Syncing...';
        setStatus(statusEl, 'Uploading queued files...', 'busy');
        flushQueue()
            .then(function (result) {
                if (result.skipped && !navigator.onLine) {
                    setStatus(statusEl, 'Still offline – sync will run when you reconnect.', 'warn');
                } else if (result.failed > 0) {
                    setStatus(
                        statusEl,
                        'Synced ' + result.synced + ', failed ' + result.failed + '. Check errors below.',
                        'err'
                    );
                } else if (result.synced > 0) {
                    setStatus(statusEl, 'Synced ' + result.synced + ' upload(s).', 'ok');
                } else {
                    setStatus(statusEl, 'Nothing to sync.', 'muted');
                }
            })
    });
});

```

```

    })
    .catch(function (err) {
      setStatus(statusEl, err && err.message ? err.message : 'Sync failed.', 'err');
    })
    .finally(function () {
      btn.disabled = false;
      btn.innerHTML = original;
    });
  });
});

updatePendingBadges();
}

window.FacultyOffline = {
  registerServiceWorker: registerServiceWorker,
  flushQueue: flushQueue,
  listQueue: listQueue,
  bindControls: bindControls,
  renderQueue: renderQueue,
  formatWhen: formatWhen,
};

document.addEventListener('DOMContentLoaded', function () {
  registerServiceWorker();
  bindControls(document);
  var mount = document.querySelector('[data-offline-queue]');
  if (mount) {
    renderQueue(mount);
  }
  flushQueue().catch(function () {});
});

window.addEventListener('online', function () {
  flushQueue().catch(function () {});
});

if ('serviceWorker' in navigator) {
  navigator.serviceWorker.addEventListener('message', function (event) {
    if (event.data && event.data.type === 'faculty-offline-flush') {
      flushQueue().catch(function () {});
    }
  });
}
})();

```

FILE: assets/js/student_offline.js

```

(function () {
  'use strict';

  var DB_NAME = 'class_student_offline';
  var DB_VERSION = 1;
  var STORE = 'packs';
  var META_KEY = 'latest';
  var API_URL = 'api/student_offline_pack.php';

  function openDb() {
    return new Promise(function (resolve, reject) {
      if (!window.indexedDB) {
        reject(new Error('This browser does not support offline storage.'));
        return;
      }
      var req = indexedDB.open(DB_NAME, DB_VERSION);
      req.onupgradeneeded = function () {
        var db = req.result;
        if (!db.objectStoreNames.contains(STORE)) {
          db.createObjectStore(STORE, { keyPath: 'key' });
        }
      };
      req.onsuccess = function () {
        resolve(req.result);
      };
      req.onerror = function () {
        reject(req.error || new Error('Could not open offline storage.'));
      };
    });
  }
})();

```

```

function idbPut(record) {
  return openDb().then(function (db) {
    return new Promise(function (resolve, reject) {
      var tx = db.transaction(STORE, 'readwrite');
      tx.objectStore(STORE).put(record);
      tx.oncomplete = function () {
        db.close();
        resolve(record);
      };
      tx.onerror = function () {
        db.close();
        reject(tx.error || new Error('Could not save offline pack.'));
      };
    });
  });
}

function idbGet(key) {
  return openDb().then(function (db) {
    return new Promise(function (resolve, reject) {
      var tx = db.transaction(STORE, 'readonly');
      var req = tx.objectStore(STORE).get(key);
      req.onsuccess = function () {
        db.close();
        resolve(req.result || null);
      };
      req.onerror = function () {
        db.close();
        reject(req.error || new Error('Could not read offline pack.'));
      };
    });
  });
}

function registerServiceWorker() {
  if (!('serviceWorker' in navigator)) {
    return Promise.resolve(null);
  }
  return navigator.serviceWorker.register('sw-student-offline.js').catch(function () {
    return null;
  });
}

function fetchPack(classroomId) {
  var url = API_URL;
  if (classroomId && Number(classroomId) > 0) {
    url += '?classroom_id=' + encodeURIComponent(String(classroomId));
  }
  return fetch(url, {
    credentials: 'same-origin',
    headers: { Accept: 'application/json' },
  }).then(function (res) {
    return res.json().then(function (body) {
      if (!res.ok || !body || !body.ok) {
        throw new Error((body && body.error) || 'Could not download offline pack.');
      }
      return body;
    });
  });
}

function mergeClassrooms(existing, incoming) {
  var map = {};
  (existing || []).forEach(function (c) {
    map[String(c.id)] = c;
  });
  (incoming || []).forEach(function (c) {
    map[String(c.id)] = c;
  });
  return Object.keys(map)
    .map(function (k) {
      return map[k];
    })
    .sort(function (a, b) {
      return String(a.course_code || '').localeCompare(String(b.course_code || ''));
    });
}

function savePack(pack, options) {

```



```

options = options || {};
var merge = !!options.merge;
return idbGet(META_KEY).then(function (prev) {
    var classrooms = merge && prev && prev.data
        ? mergeClassrooms(prev.data.classrooms || [], pack.classrooms || [])
        : pack.classrooms || [];
    var itemCount = classrooms.reduce(function (sum, c) {
        return sum + (Number(c.item_count) || 0);
    }, 0);
    var record = {
        key: META_KEY,
        saved_at: pack.saved_at || new Date().toISOString(),
        data: {
            ok: true,
            saved_at: pack.saved_at || new Date().toISOString(),
            student_id: pack.student_id || null,
            classroom_count: classrooms.length,
            item_count: itemCount,
            classrooms: classrooms,
            note: pack.note || '',
        },
    };
    return idbPut(record).then(function () {
        return registerServiceWorker().then(function () {
            return record.data;
        });
    });
});
}

function loadPack() {
    return idbGet(META_KEY).then(function (row) {
        return row && row.data ? row.data : null;
    });
}

function downloadAndSave(classroomId, options) {
    return fetchPack(classroomId).then(function (pack) {
        return savePack(pack, options);
    });
}

function setStatus(el, message, kind) {
    if (!el) return;
    el.textContent = message || '';
    el.classList.remove('text-success', 'text-danger', 'text-muted', 'text-primary');
    if (kind === 'ok') el.classList.add('text-success');
    else if (kind === 'err') el.classList.add('text-danger');
    else if (kind === 'busy') el.classList.add('text-primary');
    else el.classList.add('text-muted');
}

function bindControls(root) {
    root = root || document;
    var statusEl = root.querySelector('[data-offline-status]');
    var buttons = root.querySelectorAll('[data-offline-save]');
    buttons.forEach(function (btn) {
        if (btn.dataset.offlineBound === '1') return;
        btn.dataset.offlineBound = '1';
        btn.addEventListener('click', function () {
            var classroomId = btn.getAttribute('data-offline-save') || '';
            var merge = btn.getAttribute('data-offline-merge') === '1';
            var original = btn.innerHTML;
            btn.disabled = true;
            btn.innerHTML = '<i class="fa-solid fa-spinner fa-spin me-1"></i>Saving...';
            setStatus(statusEl, 'Downloading faculty items for offline reading...', 'busy');
            downloadAndSave(classroomId, { merge: merge })
                .then(function (data) {
                    var msg =
                        'Saved offline: ' +
                        (data.classroom_count || 0) +
                        ' class(es), ' +
                        (data.item_count || 0) +
                        ' item(s).';
                    setStatus(statusEl, msg, 'ok');
                    btn.innerHTML = '<i class="fa-solid fa-check me-1"></i>Saved';
                    window.setTimeout(function () {
                        btn.disabled = false;
                        btn.innerHTML = original;
                    }, 1600);
                });
        });
    });
}

```

```

    })
    .catch(function (err) {
        setStatus(statusEl, err && err.message ? err.message : 'Save failed.', 'err');
        btn.disabled = false;
        btn.innerHTML = original;
    });
});
});

loadPack()
.then(function (data) {
    if (!data || !statusEl) return;
    if (statusEl.getAttribute('data-offline-status') === 'auto') {
        setStatus(
            statusEl,
            'Offline copy ready: ' +
            (data.classroom_count || 0) +
            ' class(es), ' +
            (data.item_count || 0) +
            ' item(s). Last saved ' +
            formatWhen(data.saved_at) +
            ', ',
            'ok'
        );
    }
})
.catch(function () {});
}

function formatWhen(iso) {
    if (!iso) return '-';
    var d = new Date(iso);
    if (Number.isNaN(d.getTime())) return String(iso);
    return d.toLocaleString();
}

function escapeHtml(value) {
    return String(value == null ? '' : value)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&#39;');
}

function renderItem(item) {
    var html = '<article class="border rounded-3 p-3 mb-3 offline-item">';
    html += '<div class="d-flex justify-content-between gap-2 flex-wrap">';
    html += '<h3 class="h6 mb-1">' + escapeHtml(item.title || 'Untitled') + '</h3>';
    html +=
        '<span class="badge text-bg-secondary">' +
        escapeHtml(item.content_type || 'material') +
        '</span>';
    html += '</div>';
    if (item.weeks || item.days_per_topic) {
        html += '<div class="small text-muted mb-2">';
        if (item.weeks) {
            html += '<span class="me-2"><i class="fa-regular fa-calendar me-1"></i>Weeks: ' + escapeHtml(item.weeks) + '</span>';
        }
        if (item.days_per_topic) {
            html +=
                '<span><i class="fa-regular fa-clock me-1"></i>Days/topic: ' +
                escapeHtml(item.days_per_topic) +
                '</span>';
        }
        html += '</div>';
    }
    if (item.body_html) {
        html += '<div class="classroom-content-body small">' + item.body_html + '</div>';
    }
    if (item.resource) {
        if (item.resource.is_attachment) {
            html +=
                '<div class="small mt-2 text-muted"><i class="fa-solid fa-paperclip me-1"></i>' +
                escapeHtml(item.resource.label || 'Attachment') +
                ' <em>(open online to download file)</em></div>';
        } else {
            html +=
                '<div class="small mt-2"><a href="' +
                escapeHtml(item.resource.href || item.resource.url) +

```

```

        '" target="_blank" rel="noopener noreferrer">Open resource</a></div>';
    }
}
(item.attachments || []).forEach(function (att) {
    html +=
        '<div class="small mt-2 text-muted"><i class="fa-solid fa-paperclip me-1"></i>' +
        escapeHtml(att.original_name || 'Attachment') +
        ' <em>(open online to download file)</em></div>';
});
if (item.created_at) {
    html += '<div class="small text-muted mt-2">' + escapeHtml(item.created_at) + '</div>';
}
html += '</article>';
return html;
}

function groupMaterialsByWeek(materials) {
    var groups = {};
    var order = [];
    (materials || []).forEach(function (item) {
        var label = item.week_label || item.weeks || 'General resources';
        if (!groups[label]) {
            groups[label] = [];
            order.push(label);
        }
        groups[label].push(item);
    });
    return order.map(function (label) {
        return { label: label, items: groups[label] };
    });
}

function courseGroupKey(classroom) {
    var code = String(classroom && classroom.course_code ? classroom.course_code : '').trim();
    var name = String(classroom && classroom.course_name ? classroom.course_name : '').trim();
    if (code || name) {
        return code + '|' + name;
    }
    return 'classroom:' + String(classroom && classroom.id ? classroom.id : 'unknown');
}

function courseGroupHeading(classroom) {
    var code = String(classroom && classroom.course_code ? classroom.course_code : '').trim();
    var name = String(classroom && classroom.course_name ? classroom.course_name : '').trim();
    if (code && name) {
        return code + ' - ' + name;
    }
    return code || name || classroom.title || 'Course';
}

function groupClassroomsByCourse(classrooms) {
    var groups = {};
    var order = [];
    (classrooms || []).forEach(function (classroom) {
        var key = courseGroupKey(classroom);
        if (!groups[key]) {
            groups[key] = {
                key: key,
                heading: courseGroupHeading(classroom),
                course_code: classroom.course_code || '',
                course_name: classroom.course_name || '',
                classrooms: [],
                item_count: 0,
            };
            order.push(key);
        }
        groups[key].classrooms.push(classroom);
        groups[key].item_count += Number(classroom.item_count) || 0;
        if (!classroom.item_count) {
            groups[key].item_count +=
                (classroom.announcements || []).length + (classroom.materials || []).length;
        }
    });
    return order.map(function (key) {
        return groups[key];
    });
}

function collectCourseReviewItems(courseGroup) {
    var entries = [];

```

```

(courseGroup.classrooms || []).forEach(function (classroom) {
  var classTitle = classroom.title || 'Classroom';
  (classroom.announcements || []).forEach(function (item) {
    entries.push({
      classroom_title: classTitle,
      section: 'Announcements',
      item: item,
    });
  });
});
groupMaterialsByWeek(classroom.materials || []).forEach(function (week) {
  week.items.forEach(function (item) {
    entries.push({
      classroom_title: classTitle,
      section: week.label,
      item: item,
    });
  });
});
});
return entries;
}

function renderCourseBox(courseGroup, index) {
  var entries = collectCourseReviewItems(courseGroup);
  var itemCount = entries.length;
  var classCount = (courseGroup.classrooms || []).length;
  var html = '<div class="card shadow-sm mb-3 offline-course-box" data-course-index="' + escapeHtml(String(index)) + '">';
  html +=
    '<button type="button" class="offline-course-open btn btn-link text-start text-decoration-none text-body w-100 p-0" aria-
expanded="false">';
  html += '<div class="card-body d-flex justify-content-between gap-3 align-items-center">';
  html += '<div class="min-w-0">';
  html +=
    '<div class="fw-semibold mb-1"><i class="fa-solid fa-book-open me-2 text-primary"></i> ' +
escapeHtml(courseGroup.heading) +
    '</div>';
  html +=
    '<div class="small text-muted">' +
escapeHtml(String(classCount)) +
    ' class' +
    (classCount === 1 ? '' : 'es') +
    ' . ' +
escapeHtml(String(itemCount)) +
    ' item' +
    (itemCount === 1 ? '' : 's') +
    ' . click to open</div>';
  html += '</div>';
  html +=
    '<span class="offline-course-chevron badge text-bg-light border text-muted flex-shrink-0"><i class="fa-solid fa-chevron-down"></i>
</span>';
  html += '</div></button>';

  html += '<div class="offline-course-panel border-top d-none">';
  html += '<div class="card-body">';
  if (!itemCount) {
    html += '<p class="small text-muted mb-0">No announcements or materials saved for this course.</p>';
  } else {
    html += '<p class="small text-muted mb-2">Choose an item to review:</p>';
    html += '<div class="list-group mb-3 offline-item-chooser">';
    entries.forEach(function (entry, itemIndex) {
      var item = entry.item || {};
      html +=
        '<button type="button" class="list-group-item list-group-item-action offline-choose-item" data-item-index="' +
escapeHtml(String(itemIndex)) +
        '">';
      html += '<div class="d-flex justify-content-between gap-2 align-items-start">';
      html += '<div class="min-w-0">';
      html += '<div class="fw-semibold">' + escapeHtml(item.title || 'Untitled') + '</div>';
      html +=
        '<div class="small text-muted mt-1">' +
escapeHtml(entry.classroom_title || '') +
        ' . ' +
escapeHtml(entry.section || '') +
        (item.content_type ? ' . ' + escapeHtml(item.content_type) : '') +
        '</div>';
      html += '</div>';
      html += '<i class="fa-solid fa-angle-right text-muted flex-shrink-0 mt-1"></i>';
      html += '</div></button>';
    });
  }
  html += '</div>';

```

```

html +=
  '<div class="offline-review-pane border rounded-3 p-3 bg-light d-none" data-review-pane>' +
  '<p class="small text-muted mb-0">Select an item above to review it here.</p>' +
  '</div>';
}
html += '</div></div></div>';
return { html: html, entries: entries };
}

function bindOfflineReader(mount, courseEntries) {
  if (!mount) return;

  mount.querySelectorAll('.offline-course-box').forEach(function (box) {
    var openBtn = box.querySelector('.offline-course-open');
    var panel = box.querySelector('.offline-course-panel');
    var chevron = box.querySelector('.offline-course-chevron i');
    var reviewPane = box.querySelector('[data-review-pane]');
    var courseIndex = parseInt(box.getAttribute('data-course-index') || '-1', 10);
    var entries = courseEntries[courseIndex] || [];

    if (openBtn && panel) {
      openBtn.addEventListener('click', function () {
        var willOpen = panel.classList.contains('d-none');
        // Close other course boxes so only one is open at a time.
        mount.querySelectorAll('.offline-course-box').forEach(function (other) {
          if (other === box) return;
          var otherPanel = other.querySelector('.offline-course-panel');
          var otherBtn = other.querySelector('.offline-course-open');
          var otherIcon = other.querySelector('.offline-course-chevron i');
          if (otherPanel) otherPanel.classList.add('d-none');
          if (otherBtn) otherBtn.setAttribute('aria-expanded', 'false');
          if (otherIcon) {
            otherIcon.classList.remove('fa-chevron-up');
            otherIcon.classList.add('fa-chevron-down');
          }
        });

        if (willOpen) {
          panel.classList.remove('d-none');
          openBtn.setAttribute('aria-expanded', 'true');
          if (chevron) {
            chevron.classList.remove('fa-chevron-down');
            chevron.classList.add('fa-chevron-up');
          }
        } else {
          panel.classList.add('d-none');
          openBtn.setAttribute('aria-expanded', 'false');
          if (chevron) {
            chevron.classList.remove('fa-chevron-up');
            chevron.classList.add('fa-chevron-down');
          }
        }
      });
    }
  });

  box.querySelectorAll('.offline-choose-item').forEach(function (btn) {
    btn.addEventListener('click', function () {
      var itemIndex = parseInt(btn.getAttribute('data-item-index') || '-1', 10);
      var entry = entries[itemIndex];
      if (!entry || !reviewPane) return;

      box.querySelectorAll('.offline-choose-item').forEach(function (row) {
        row.classList.remove('active');
      });
      btn.classList.add('active');

      reviewPane.classList.remove('d-none', 'bg-light');
      reviewPane.classList.add('bg-white');
      reviewPane.innerHTML =
        '<div class="d-flex justify-content-between align-items-center gap-2 mb-2">' +
        '<div class="small text-muted mb-0">Reviewing</div>' +
        '<button type="button" class="btn btn-sm btn-outline-secondary offline-clear-review">Close</button>' +
        '</div>' +
        renderItem(entry.item);

      var clearBtn = reviewPane.querySelector('.offline-clear-review');
      if (clearBtn) {
        clearBtn.addEventListener('click', function () {
          btn.classList.remove('active');
          reviewPane.classList.add('d-none', 'bg-light');
        });
      }
    });
  });
}

```

```

        reviewPane.classList.remove('bg-white');
        reviewPane.innerHTML =
            '<p class="small text-muted mb-0">Select an item above to review it here.</p>';
    });
}

reviewPane.scrollIntoView({ behavior: 'smooth', block: 'nearest' });
});
});
});
}

function renderOfflineReader(mount) {
    if (!mount) return;
    mount.innerHTML = '<p class="text-muted">Loading offline copy...</p>';
    loadPack()
        .then(function (data) {
            if (!data || !(data.classrooms || []).length) {
                mount.innerHTML =
                    '<div class="alert alert-warning mb-0">No offline copy yet. Go online, open <strong>My classes</strong>, and tap <strong>Save for offline</strong>.</div>';
                return;
            }

            var online = navigator.onLine;
            var html = '';
            html += '<div class="alert ' + (online ? 'alert-success' : 'alert-info') + '>';
            html += online
                ? '<i class="fa-solid fa-wifi me-1"></i>You are online. Showing your saved offline copy.'
                : '<i class="fa-solid fa-plane me-1"></i>You are offline. Reading saved faculty items.';
            html +=
                '<div class="small mt-1">Last saved: ' +
                escapeHtml(formatWhen(data.saved_at)) +
                ' . ' +
                escapeHtml(String(data.classroom_count || 0)) +
                ' class(es) . ' +
                escapeHtml(String(data.item_count || 0)) +
                ' item(s)</div>';
            if (data.note) {
                html += '<div class="small mt-1">' + escapeHtml(data.note) + '</div>';
            }
            html += '</div>';

            html +=
                '<p class="small text-muted mb-3">Click a course box to see its items, then choose one to review.</p>';

            var courseEntries = [];
            groupClassroomsByCourse(data.classrooms).forEach(function (courseGroup, index) {
                var built = renderCourseBox(courseGroup, index);
                html += built.html;
                courseEntries[index] = built.entries;
            });

            mount.innerHTML = html;
            bindOfflineReader(mount, courseEntries);
        })
        .catch(function (err) {
            mount.innerHTML =
                '<div class="alert alert-danger mb-0">' +
                escapeHtml(err && err.message ? err.message : 'Could not load offline copy.') +
                '</div>';
        });
}

window.StudentOffline = {
    registerServiceWorker: registerServiceWorker,
    downloadAndSave: downloadAndSave,
    loadPack: loadPack,
    bindControls: bindControls,
    renderOfflineReader: renderOfflineReader,
    formatWhen: formatWhen,
};

document.addEventListener('DOMContentLoaded', function () {
    registerServiceWorker();
    bindControls(document);
    var mount = document.querySelector('[data-offline-reader]');
    if (mount) {
        renderOfflineReader(mount);
    }
});

```

```
});  
})();
```

```
-----  
FILE: assets/js/theme-toggle.js  
-----
```

```
(function () {  
    'use strict';  
  
    var STORAGE_KEY = 'app_theme';  
    var LEGACY_KEY = 'schedule_theme';  
  
    function readStored() {  
        try {  
            var t = localStorage.getItem(STORAGE_KEY);  
            if (t === 'dark' || t === 'light') {  
                return t;  
            }  
        } catch (e) {}  
        return null;  
    }  
  
    function effectiveTheme() {  
        var s = readStored();  
        if (s) {  
            return s;  
        }  
        try {  
            if (window.matchMedia && matchMedia('(prefers-color-scheme: dark)').matches) {  
                return 'dark';  
            }  
        } catch (e) {}  
        return 'light';  
    }  
  
    function applyTheme(theme, persist) {  
        var t = theme === 'dark' ? 'dark' : 'light';  
        document.documentElement.setAttribute('data-bs-theme', t);  
        if (persist) {  
            try {  
                localStorage.setItem(STORAGE_KEY, t);  
                localStorage.setItem(LEGACY_KEY, t);  
            } catch (e) {}  
        }  
        updateToggleUI(t);  
    }  
  
    function updateToggleUI(theme) {  
        var btn = document.getElementById('appThemeToggle');  
        if (!btn) {  
            return;  
        }  
        var isDark = theme === 'dark';  
        btn.setAttribute('aria-pressed', isDark ? 'true' : 'false');  
        var label = document.getElementById('appThemeToggleLabel');  
        if (label) {  
            label.textContent = isDark ? 'Light' : 'Dark';  
        }  
        var moon = btn.querySelector('.app-theme-icon-dark');  
        var sun = btn.querySelector('.app-theme-icon-light');  
        if (moon) {  
            moon.classList.toggle('d-none', isDark);  
        }  
        if (sun) {  
            sun.classList.toggle('d-none', !isDark);  
        }  
        btn.title = isDark ? 'Switch to light mode' : 'Switch to dark mode';  
        btn.setAttribute('aria-label', isDark ? 'Switch to light mode' : 'Switch to dark mode');  
    }  
  
    function init() {  
        try {  
            if (!localStorage.getItem(STORAGE_KEY)) {  
                var leg = localStorage.getItem(LEGACY_KEY);  
                if (leg === 'dark' || leg === 'light') {  
                    localStorage.setItem(STORAGE_KEY, leg);  
                }  
            }  
        } catch (e) {}  
    }  
})
```

```

var attr = document.documentElement.getAttribute('data-bs-theme');
var t = attr === 'dark' || attr === 'light' ? attr : effectiveTheme();
document.documentElement.setAttribute('data-bs-theme', t);
updateToggleUI(t);

var btn = document.getElementById('appThemeToggle');
if (btn) {
    btn.addEventListener('click', function () {
        var cur = document.documentElement.getAttribute('data-bs-theme') === 'dark' ? 'dark' : 'light';
        applyTheme(cur === 'dark' ? 'light' : 'dark', true);
    });
}

try {
    matchMedia('(prefers-color-scheme: dark)').addEventListener('change', function (e) {
        if (readStored()) {
            return;
        }
        applyTheme(e.matches ? 'dark' : 'light', false);
    });
} catch (e) {}

if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', init);
} else {
    init();
}
})();

```

FILE: auto_schedule.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['dean']);
$collegeId = dean_college_id_or_fail();
$deanUserId = (int) $_SESSION['user_id'];
$collegeName = college_name_by_id($collegeId);
$hasLabFlag = db_column_exists('courses', 'is_laboratory');
$hasLectureUnits = db_column_exists('courses', 'lecture_units');
$hasLaboratoryUnits = db_column_exists('courses', 'laboratory_units');
$hasSpecializationTable = db()->query("SHOW TABLES LIKE 'faculty_specializations'")->fetchColumn() !== false;

function auto_candidate_starts(int $durationMinutes): array
{
    $starts = [];
    $min = time_to_minutes(TIME_MIN);
    $max = time_to_minutes(TIME_MAX) - $durationMinutes;
    for ($m = $min; $m <= $max; $m += 60) {
        $starts[] = minutes_to_time($m);
    }
    return $starts;
}

function auto_find_slot(
    int $facultyId,
    int $roomId,
    array $days,
    int $durationMinutes,
    string $semester,
    string $schoolYear,
    int $collegeId,
    bool $allowLongBlock = false
): ?array {
    foreach ($days as $day) {
        foreach (auto_candidate_starts($durationMinutes) as $start) {
            $stMin = time_to_minutes($start);
            $end = minutes_to_time($stMin + $durationMinutes);
            $conf = checkConflicts(
                $facultyId,
                $roomId,
                [$day],

```



```

        $start,
        $end,
        $semester,
        $schoolYear,
        null,
        $collegeId,
        $allowLongBlock
    );
    if ($conf === []) {
        return ['day' => $day, 'start' => $start, 'end' => $end];
    }
}
}
return null;
}

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);
$errors = [];
$warnings = [];
$results = [];
$specializationCourseIds = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $facultyId = (int) ($_POST['faculty_id'] ?? 0);
    $subjectCount = max(1, min(20, (int) ($_POST['subject_count'] ?? 1)));
    $semester = trim((string) ($_POST['semester'] ?? ''));
    $schoolYear = trim((string) ($_POST['school_year'] ?? ''));
    $academicYear = trim((string) ($_POST['academic_year'] ?? ''));
    $allowLabFallbackForLecture = !empty($_POST['allow_lab_fallback_for_lecture']);
    $specializationCourseIds = isset($_POST['specialization_course_ids']) && is_array($_POST['specialization_course_ids'])
        ? array_values(array_unique(array_map('intval', $_POST['specialization_course_ids'])))
        : [];

    if ($facultyId < 1 || $semester === '' || $schoolYear === '') {
        $errors[] = 'Faculty, semester, and school year are required.';
    } else {
        $st = db()->prepare('SELECT COUNT(*) FROM faculty WHERE id=? AND college_id=?');
        $st->execute([$facultyId, $collegeId]);
        if ((int) $st->fetchColumn() < 1) {
            $errors[] = 'Selected faculty is not under your college.';
        }
    }

    $courseScopeIds = [];
    if (!$errors && $hasSpecializationTable) {
        // Save posted specialization for this faculty (persistent mapping)
        if (isset($_POST['specialization_course_ids'])) {
            $validStmt = db()->prepare('SELECT id FROM courses WHERE college_id=?');
            $validStmt->execute([$collegeId]);
            $valid = array_fill_keys(array_map('intval', $validStmt->fetchAll(PDO::FETCH_COLUMN)), true);

            $clean = [];
            foreach ($specializationCourseIds as $cid) {
                if (isset($valid[$cid])) {
                    $clean[] = $cid;
                }
            }

            db()->beginTransaction();
            try {
                db()->prepare('DELETE FROM faculty_specializations WHERE faculty_id=?')->execute([$facultyId]);
                if ($clean) {
                    $insSpec = db()->prepare('INSERT INTO faculty_specializations (faculty_id, course_id) VALUES (?,?)');
                    foreach ($clean as $cid) {
                        $insSpec->execute([$facultyId, $cid]);
                    }
                }
                db()->commit();
                $specializationCourseIds = $clean;
            } catch (Throwable $e) {
                if (db()->inTransaction()) {
                    db()->rollBack();
                }
                throw $e;
            }
        }
    }

    if ($specializationCourseIds) {
        $courseScopeIds = $specializationCourseIds;
    }
}

```

```

    } else {
        $sp = db()->prepare('SELECT course_id FROM faculty_specializations WHERE faculty_id=?');
        $sp->execute([$facultyId]);
        $saved = array_map('intval', $sp->fetchAll(PDO::FETCH_COLUMN));
        if ($saved) {
            $courseScopeIds = $saved;
        }
    }
}

if (!$errors) {
    $orderBy = $hasLabFlag ? 'c.is_laboratory DESC, c.course_code ASC' : 'c.course_code ASC';
    if ($courseScopeIds) {
        $ph = implode(',', array_fill(0, count($courseScopeIds), '?'));
        $coursesStmt = db()->prepare(
            'SELECT c.*
            FROM courses c
            WHERE c.college_id=? AND c.id IN (' . $ph . ')
            ORDER BY ' . $orderBy . '
            LIMIT ' . $subjectCount
        );
        $coursesStmt->execute(array_merge([$collegeId], $courseScopeIds));
    } else {
        $coursesStmt = db()->prepare(
            'SELECT c.*
            FROM courses c
            WHERE c.college_id=?
            ORDER BY ' . $orderBy . '
            LIMIT ' . $subjectCount
        );
        $coursesStmt->execute([$collegeId]);
    }
    $courses = $coursesStmt->fetchAll();
    if (!$courses) {
        $errors[] = $courseScopeIds
            ? 'No specialization courses found for this faculty.'
            : 'No courses found for this college.';
    }
}

if (!$errors) {
    $roomLectureStmt = db()->prepare(
        "SELECT id, room_code FROM rooms WHERE status='available' AND college_id=? AND type IN ('lecture','conference') ORDER BY
room_code"
    );
    $roomLectureStmt->execute([$collegeId]);
    $lectureRooms = $roomLectureStmt->fetchAll();

    $roomLabStmt = db()->prepare(
        "SELECT id, room_code FROM rooms WHERE status='available' AND college_id=? AND type='laboratory' ORDER BY room_code"
    );
    $roomLabStmt->execute([$collegeId]);
    $labRooms = $roomLabStmt->fetchAll();

    if (!$lectureRooms) {
        if ($allowLabFallbackForLecture && $labRooms) {
            $lectureRooms = $labRooms;
            $warnings[] = 'No lecture/conference rooms found. Using laboratory rooms as fallback for lecture parts (toggle enabled).';
        } else {
            $errors[] = 'No available lecture/conference rooms in this college.'
                . ($labRooms ? ' Enable "Allow laboratory fallback for lecture parts" to continue.' : '');
        }
    }
}

if (!$errors) {
    db()->beginTransaction();
    try {
        $created = 0;
        $failed = 0;
        $createdScheduleIds = [];
        $hasSessionKind = db_column_exists('schedules', 'session_kind');
        $scheduleInsertSql = 'INSERT INTO schedules (faculty_id, course_id, room_id, college_id, schedule_type, day_of_week,
start_time, end_time, semester, school_year, academic_year, created_by'
            . ($hasSessionKind ? ', session_kind' : '')
            . ') VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?)'
            . ($hasSessionKind ? ',?' : '');
        $ins = db()->prepare($scheduleInsertSql);

        foreach ($courses as $course) {

```

```

$isLab = $hasLabFlag && (int) ($course['is_laboratory'] ?? 0) === 1;
$courseCode = (string) $course['course_code'];
$courseId = (int) $course['id'];
$legacyUnits = (float) ($course['units'] ?? 0);
$lectureUnits = $hasLectureUnits
    ? (float) ($course['lecture_units'] ?? 0)
    : $legacyUnits;
$laboratoryUnits = $hasLaboratoryUnits
    ? (float) ($course['laboratory_units'] ?? 0)
    : ($isLab ? $legacyUnits : 0.0);

$parts = [];
if ($isLab) {
    // New rule: 1 unit lecture = 1 hour, 1 unit lab = 3 hours
    if ($lectureUnits > 0) {
        $lectureMinutes = max(60, (int) round($lectureUnits * 60));
        $parts[] = ['label' => 'Lecture Part', 'duration' => $lectureMinutes, 'roomPool' => $lectureRooms, 'days' =>
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']];
    }
    if ($laboratoryUnits > 0) {
        $labMinutes = max(180, (int) round($laboratoryUnits * 180));
        $parts[] = ['label' => 'Laboratory Part', 'duration' => $labMinutes, 'roomPool' => $labRooms, 'days' =>
['Saturday', 'Friday', 'Thursday']];
    }
    if ($parts === []) {
        $results[] = "{$courseCode}: skipped (no lecture or laboratory units configured).";
        $failed++;
        continue;
    }
} else {
    // New rule: lecture hours are unit-based (1 unit = 1 hour)
    $effectiveLectureUnits = $lectureUnits > 0 ? $lectureUnits : $legacyUnits;
    $duration = max(60, (int) round($effectiveLectureUnits * 60));
    $parts[] = ['label' => 'Regular', 'duration' => $duration, 'roomPool' => $lectureRooms, 'days' => ['Monday',
'Tuesday', 'Wednesday', 'Thursday', 'Friday']];
}

$okForCourse = true;
foreach ($parts as $part) {
    if (empty($part['roomPool'])) {
        $results[] = "{$courseCode}: No room available for {$part['label']}. ";
        $okForCourse = false;
        break;
    }

    $scheduledPart = false;
    foreach ($part['roomPool'] as $room) {
        $slot = auto_find_slot(
            $facultyId,
            (int) $room['id'],
            $part['days'],
            (int) $part['duration'],
            $semester,
            $schoolYear,
            $collegeId,
            ((int) $part['duration']) > (MAX_CLASS_BLOCK_HOURS * 60)
        );
        if ($slot) {
            $insertParams = [
                $facultyId,
                $courseId,
                (int) $room['id'],
                $collegeId,
                'Custom',
                $slot['day'],
                $slot['start'],
                $slot['end'],
                $semester,
                $schoolYear,
                $academicYear,
                $deanUserId,
            ];
            if ($hasSessionKind) {
                $insertParams[] = schedule_segment_session_kind((string) $part['label']);
            }
            $ins->execute($insertParams);
            $created++;
            $createdScheduleIds[] = (int) db()->lastInsertId();
            $results[] = "{$courseCode} ({$part['label']}): {$slot['day']} " . substr($slot['start'], 0, 5) . '-' .
substr($slot['end'], 0, 5) . " @ {$room['room_code']}";

```

```

                $scheduledPart = true;
                break;
            }
        }
        if (!$scheduledPart) {
            $results[] = "{$courseCode}: No available slot for {$part['label']}.";
            $okForCourse = false;
            break;
        }
    }

    if (!$okForCourse) {
        $failed++;
    }
}

db()->commit();
if ($created > 0) {
    log_user_activity(
        'add',
        'Schedules (auto)',
        "Auto-generated {$created} schedule row(s)",
        null,
        [
            'created_count' => $created,
            'failed_courses' => $failed,
            'created_schedule_ids' => $createdScheduleIds,
            'preview' => array_slice($results, 0, 25),
        ]
    );
    log_dean_activity('auto_schedule', "Auto scheduling run: created {$created} entries, {$failed} course(s) incomplete.");
    $_SESSION['flash'] = "Auto scheduling completed. Created {$created} schedule entry/entries.";
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    throw $e;
}
}
}

$facultyStmt = db()->prepare('SELECT id, faculty_id, full_name FROM faculty WHERE college_id=? AND status="active" ORDER BY full_name');
$facultyStmt->execute([$collegeId]);
$facultyList = $facultyStmt->fetchAll();
$courseListStmt = db()->prepare('SELECT id, course_code, course_name FROM courses WHERE college_id=? ORDER BY course_code');
$courseListStmt->execute([$collegeId]);
$courseList = $courseListStmt->fetchAll();

$pageTitle = 'Automatic Scheduling';
require_once __DIR__ . '/includes/header.php';
?>
<div class="d-flex justify-content-between align-items-center mb-4">
    <h1 class="h3 mb-0"><i class="fa-solid fa-wand-magic-sparkles me-2 text-primary"></i>Automatic Scheduling</h1>
</div>
<p class="text-muted">
    College: <strong><?= htmlspecialchars($collegeName) ?></strong>.
    Unit-based time: <strong>Lecture Units x 1 hour</strong>, <strong>Laboratory Units x 3 hours</strong>.
</p>
<?php if (!$hasLectureUnits || !$hasLaboratoryUnits): ?>
    <div class="alert alert-warning">
        Split-unit columns are missing in your database. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once.
    </div>
<?php endif; ?>
<?php if (!$hasLabFlag): ?>
    <div class="alert alert-warning">
        Laboratory split is currently disabled because your database is missing `courses.is_laboratory`.
        Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then retry.
    </div>
<?php endif; ?>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
<?php if ($errors): ?>
    <div class="alert alert-warning">
        <ul class="mb-0">
            <?php foreach ($errors as $e): ?>
                <li><?= htmlspecialchars($e) ?></li>
            <?php endforeach; ?>
        </ul>
    </div>
</div>

```

```

    </div>
<?php endif; ?>
<?php if ($warnings): ?>
    <div class="alert alert-info">
        <ul class="mb-0">
            <?php foreach ($warnings as $w): ?>
                <li><?= htmlspecialchars($w) ?></li>
            <?php endforeach; ?>
        </ul>
    </div>
<?php endif; ?>

<div class="card shadow-sm mb-4">
    <div class="card-header bg-white"><strong>Generate Schedule Plan</strong></div>
    <div class="card-body">
        <form method="post" class="row g-3">
            <div class="col-md-4">
                <label class="form-label">Faculty</label>
                <select class="form-select" name="faculty_id" required>
                    <option value="">Select faculty</option>
                    <?php foreach ($facultyList as $f): ?>
                        <option value="<?=(int) $f['id'] ?>" <?=(int) ($_POST['faculty_id'] ?? 0) == (int) $f['id'] ? 'selected' : '' ?>>
                            <?= htmlspecialchars($f['full_name'] . ' (' . $f['faculty_id'] . ')') ?>
                        </option>
                    <?php endforeach; ?>
                </select>
            </div>
            <div class="col-md-2">
                <label class="form-label"># Subjects</label>
                <input type="number" class="form-control" name="subject_count" min="1" max="20" value="<?= htmlspecialchars((string)
($ _POST['subject_count'] ?? 5)) ?>" required>
            </div>
            <div class="col-md-2">
                <label class="form-label">Semester</label>
                <select class="form-select" name="semester" required>
                    <?php foreach (['1st Semester', '2nd Semester', 'Summer'] as $s): ?>
                        <option value="<?= htmlspecialchars($s) ?>" <?=( $_POST['semester'] ?? '1st Semester') == $s ? 'selected' : '' ?>>
                            <?= htmlspecialchars($s) ?></option>
                    <?php endforeach; ?>
                </select>
            </div>
            <div class="col-md-2">
                <label class="form-label">School Year</label>
                <input type="text" class="form-control" name="school_year" value="<?= htmlspecialchars((string) ($ _POST['school_year'] ??
(date('Y') . '-' . (date('Y') + 1)))) ?>" required>
            </div>
            <div class="col-md-2">
                <label class="form-label">Academic Year</label>
                <input type="text" class="form-control" name="academic_year" value="<?= htmlspecialchars((string) ($ _POST['academic_year']
?? '')) ?>" ?>
            </div>
            <div class="col-md-4 d-flex align-items-end">
                <div class="form-check">
                    <input class="form-check-input" type="checkbox" id="allow_lab_fallback_for_lecture"
name="allow_lab_fallback_for_lecture" value="1"
                    <?=(empty($_POST['allow_lab_fallback_for_lecture']) ? 'checked' : '' ?>>
                    <label class="form-check-label" for="allow_lab_fallback_for_lecture">
                        Allow laboratory fallback for lecture parts (optional)
                    </label>
                </div>
            </div>
            <div class="col-12">
                <label class="form-label">Faculty specialization courses (optional, saved per faculty)</label>
                <select class="form-select" name="specialization_course_ids[]" multiple size="6">
                    <?php foreach ($courseList as $c): ?>
                        <option value="<?=(int) $c['id'] ?>" <?=(in_array((int) $c['id'], $specializationCourseIds, true) ? 'selected' : ''
?>>
                            <?= htmlspecialchars($c['course_code'] . ' - ' . $c['course_name']) ?>
                        </option>
                    <?php endforeach; ?>
                </select>
                <div class="form-text">
                    If you select courses here, those become this faculty's specialization set and Auto Scheduling prioritizes them.
                </div>
            </div>
            <div class="col-12">
                <button type="submit" class="btn btn-primary"><?= app_tooltip_attr('Runs the automatic scheduler for your college with the
options you selected.') ?>><i class="fa-solid fa-gears me-1"></i>Auto-generate schedules</button>
                <a href="schedule.php" class="btn btn-outline-secondary"><?= app_tooltip_attr('Returns to the schedule list without running
another generation.') ?>>Back to schedules</a>
            </div>
        </form>
    </div>
</div>

```

```

        </div>
    </form>
</div>
</div>

<?php if ($results): ?>
    <div class="card shadow-sm">
        <div class="card-header bg-white"><strong>Generation Results</strong></div>
        <div class="card-body">
            <ul class="mb-0">
                <?php foreach ($results as $line): ?>
                    <li><? = htmlspecialchars($line) ?></li>
                <?php endforeach; ?>
            </ul>
        </div>
    </div>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: classroom_banner.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_login();

$classroomId = (int) ($_GET['id'] ?? 0);
if ($classroomId < 1) {
    http_response_code(400);
    exit('Invalid classroom.');
```

```

} elseif ($role === 'dean') {
    $cid = current_college_id();
    $schCid = $row['schedule_college_id'] != null ? (int) $row['schedule_college_id'] : 0;
    $allowed = $cid != null && $cid > 0 && $schCid === $cid;
} elseif ($role === 'program_chair') {
    $cid = current_college_id();
    $programScope = current_program_scope();
    $schCid = $row['schedule_college_id'] != null ? (int) $row['schedule_college_id'] : 0;
    if ($cid != null && $cid > 0 && $schCid === $cid && $programScope != null) {
        $courseDept = trim((string) ($row['course_department'] ?? ''));
        $schedProg = trim((string) ($row['schedule_program'] ?? ''));
        $allowed = ($courseDept != '' && strcasecmp($courseDept, $programScope) === 0)
            || ($schedProg != '' && strcasecmp($schedProg, $programScope) === 0);
    }
} elseif ($role === 'gened') {
    $allowed = (int) ($row['course_is_gened'] ?? 0) === 1;
}
} elseif ($role === 'student') {
    $studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
    if ($studentId < 1) {
        $studentId = resolve_student_id_for_user($userId) ?? 0;
        $_SESSION['student_id'] = $studentId > 0 ? $studentId : null;
    }
    if ($studentId > 0 && db_table_exists('classroom_enrollments')) {
        $chk = db()->prepare('SELECT COUNT(*) FROM classroom_enrollments WHERE classroom_id = ? AND student_id = ?');
        $chk->execute([$classroomId, $studentId]);
        $allowed = (int) $chk->fetchColumn() > 0;
    }
}

if (!$allowed) {
    http_response_code(403);
    exit('Access denied.');
```

```

}

$path = classroom_banner_path($storedName);
if (!is_file($path)) {
    http_response_code(404);
    exit('Banner file missing.');
```

```

}

$mime = trim((string) ($row['banner_mime'] ?? ''));
if ($mime === '' || !str_starts_with($mime, 'image/')) {
    $mime = classroom_banner_mime_for_stored($storedName);
}
```

```

header('Content-Type: ' . $mime);
header('Content-Length: ' . (string) filesize($path));
header('Cache-Control: private, max-age=3600');
header('X-Content-Type-Options: nosniff');
readfile($path);
exit;

-----
FILE: classroom_content_attachment.php
-----
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'student', 'program_chair', 'dean', 'gened', 'admin', 'super_admin']);

$attachmentId = (int) ($_GET['attachment_id'] ?? 0);
$contentId = (int) ($_GET['id'] ?? 0);
if ($attachmentId < 1 && $contentId < 1) {
    http_response_code(400);
    exit('Invalid attachment request.');
```

```

}

$role = (string) ($_SESSION['role'] ?? '');
$hasContentAttachments = db_table_exists('classroom_content_attachments');
$resourceUrl = '';
$storedName = '';
$originalName = '';
$mime = '';

if (in_array($role, ['admin', 'super_admin'], true)) {
```

```

if ($attachmentId > 0 && $hasContentAttachments) {
    $st = db()->prepare(
        'SELECT stored_name, original_name, mime
        FROM classroom_content_attachments
        WHERE id = ?
        LIMIT 1'
    );
    $st->execute([$attachmentId]);
    $attachment = $st->fetch() ?: null;
    if (is_array($attachment)) {
        $storedName = (string) ($attachment['stored_name'] ?? '');
        $originalName = (string) ($attachment['original_name'] ?? '');
        $mime = (string) ($attachment['mime'] ?? '');
    }
} elseif ($contentId > 0) {
    $st = db()->prepare('SELECT resource_url FROM classroom_content WHERE id = ? LIMIT 1');
    $st->execute([$contentId]);
    $resourceUrl = (string) ($st->fetchColumn() ?: '');
}
} elseif (in_array($role, ['faculty', 'program_chair', 'dean', 'gened'], true)) {
    $facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
    if ($facultyId < 1) {
        $facultyId = resolve_faculty_id_for_user((int) ($_SESSION['user_id'] ?? 0)) ?? 0;
        $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
    }
    if ($facultyId < 1) {
        http_response_code(403);
        exit('Faculty profile not linked to this account.');
```

```

    }

    if ($attachmentId > 0 && $hasContentAttachments) {
        $st = db()->prepare(
            'SELECT cca.stored_name, cca.original_name, cca.mime
            FROM classroom_content_attachments cca
            INNER JOIN classroom_content cc ON cc.id = cca.content_id
            INNER JOIN online_classrooms oc ON oc.id = cc.classroom_id
            WHERE cca.id = ? AND cc.faculty_id = ? AND oc.faculty_id = ?
            LIMIT 1'
        );
        $st->execute([$attachmentId, $facultyId, $facultyId]);
        $attachment = $st->fetch() ?: null;
        if (is_array($attachment)) {
            $storedName = (string) ($attachment['stored_name'] ?? '');
            $originalName = (string) ($attachment['original_name'] ?? '');
            $mime = (string) ($attachment['mime'] ?? '');
        }
    } elseif ($contentId > 0) {
        $st = db()->prepare(
            'SELECT cc.resource_url
            FROM classroom_content cc
            INNER JOIN online_classrooms oc ON oc.id = cc.classroom_id
            WHERE cc.id = ? AND cc.faculty_id = ? AND oc.faculty_id = ?
            LIMIT 1'
        );
        $st->execute([$contentId, $facultyId, $facultyId]);
        $resourceUrl = (string) ($st->fetchColumn() ?: '');
    }
} elseif ($role === 'student') {
    $studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
    if ($studentId < 1) {
        $studentId = resolve_student_id_for_user((int) ($_SESSION['user_id'] ?? 0)) ?? 0;
        $_SESSION['student_id'] = $studentId > 0 ? $studentId : null;
    }
    if ($studentId < 1) {
        http_response_code(403);
        exit('Student profile not linked to this account.');
```

```

    }

    if ($attachmentId > 0 && $hasContentAttachments) {
        $st = db()->prepare(
            'SELECT cca.stored_name, cca.original_name, cca.mime
            FROM classroom_content_attachments cca
            INNER JOIN classroom_content cc ON cc.id = cca.content_id
            INNER JOIN classroom_enrollments ce ON ce.classroom_id = cc.classroom_id
            WHERE cca.id = ? AND ce.student_id = ?
            LIMIT 1'
        );
        $st->execute([$attachmentId, $studentId]);
        $attachment = $st->fetch() ?: null;
        if (is_array($attachment)) {

```



```

        $storedName = (string) ($attachment['stored_name'] ?? '');
        $originalName = (string) ($attachment['original_name'] ?? '');
        $mime = (string) ($attachment['mime'] ?? '');
    }
} elseif ($contentId > 0) {
    $st = db()->prepare(
        'SELECT cc.resource_url
        FROM classroom_content cc
        INNER JOIN classroom_enrollments ce ON ce.classroom_id = cc.classroom_id
        WHERE cc.id = ? AND ce.student_id = ?
        LIMIT 1'
    );
    $st->execute([$contentId, $studentId]);
    $resourceUrl = (string) ($st->fetchColumn() ?: '');
}
} else {
    http_response_code(403);
    exit('You do not have access to this attachment.');
```

```

}

if ($storedName !== '') {
    $path = classroom_content_attachment_storage_path($storedName);
    if (!is_file($path)) {
        http_response_code(404);
        exit('Attachment file is missing.');
```

```

    }

    $downloadName = classroom_content_attachment_download_name($originalName, $storedName);
    $mime = trim($mime) !== '' ? $mime : (function_exists('mime_content_type') ? (string) mime_content_type($path) : 'application/octet-stream');
} else {
    if ($resourceUrl === '' || !classroom_content_is_attachment($resourceUrl)) {
        http_response_code(404);
        exit('Attachment not found.');
```

```

    }

    $path = classroom_content_attachment_path($resourceUrl);
    if (!is_file($path)) {
        http_response_code(404);
        exit('Attachment file is missing.');
```

```

    }

    $downloadName = classroom_content_attachment_name($resourceUrl);
    $mime = function_exists('mime_content_type') ? (string) mime_content_type($path) : 'application/octet-stream';
}
$mime = trim($mime) !== '' ? $mime : 'application/octet-stream';
```

```

$wantInline = (string) ($_GET['inline'] ?? '') === '1' || classroom_content_is_image_mime($mime);
$disposition = $wantInline && classroom_content_is_image_mime($mime) ? 'inline' : 'attachment';

header('Content-Type: ' . $mime);
header('Content-Length: ' . (string) filesize($path));
header('Content-Disposition: ' . $disposition . '; filename="' . addslashes($downloadName, '"\'') . '"');
header('X-Content-Type-Options: nosniff');
readfile($path);
exit;
```

```

-----
FILE: classroom_content_inline.php
-----
```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_login();

$storedName = basename(trim((string) ($_GET['n'] ?? '')));
if ($storedName === '' || !preg_match('/^inline_[A-Za-z0-9]{32}\. (jpg|jpeg|png|gif|webp)$/', $storedName)) {
    http_response_code(400);
    exit('Invalid image request.');
```

```

}

$path = classroom_content_attachment_storage_path($storedName);
if (!is_file($path)) {
    http_response_code(404);
    exit('Image not found.');
```

```

}
```

```

$mime = match (strtolower(pathinfo($storedName, PATHINFO_EXTENSION))) {
    'jpg', 'jpeg' => 'image/jpeg',
    'png' => 'image/png',
    'gif' => 'image/gif',
    'webp' => 'image/webp',
    default => 'application/octet-stream',
};

header('Content-Type: ' . $mime);
header('Content-Length: ' . (string) filesize($path));
header('Content-Disposition: inline; filename="' . addslashes($storedName, '"\'') . '"');
header('Cache-Control: private, max-age=3600');
header('X-Content-Type-Options: nosniff');
readfile($path);
exit;

```

```

-----
FILE: classroom_materials_monitor.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['super_admin', 'admin', 'dean', 'program_chair', 'gened']);

$role = (string) ($_SESSION['role'] ?? '');
$classroomId = (int) ($_GET['id'] ?? 0);
$monitorCollegeId = (int) ($_GET['monitor_college'] ?? 0);
$monitorProgram = trim((string) ($_GET['monitor_program'] ?? ''));
$selectedWeek = trim((string) ($_GET['week'] ?? ''));

if ($classroomId < 1) {
    http_response_code(400);
    exit('Invalid classroom.');
```

```

    $accessAllowed = $collegeId != null && $collegeId > 0 && $scheduleCollegeId == $collegeId;
} elseif ($role == 'program_chair') {
    $collegeId = current_college_id();
    $programScope = current_program_scope();
    $programMatches = $programScope != null && (
        ($courseDepartment != '' && strcasecmp($courseDepartment, $programScope) == 0)
        || ($scheduleProgram != '' && strcasecmp($scheduleProgram, $programScope) == 0)
    );
    $accessAllowed = $collegeId != null && $collegeId > 0 && $scheduleCollegeId == $collegeId && $programMatches;
} elseif ($role == 'gened') {
    if (!$isGeCourse) {
        $accessAllowed = false;
    } elseif ($monitorCollegeId > 0 && $monitorProgram != '') {
        $programMatches = ($courseDepartment != '' && strcasecmp($courseDepartment, $monitorProgram) == 0)
            || ($scheduleProgram != '' && strcasecmp($scheduleProgram, $monitorProgram) == 0);
        $accessAllowed = $scheduleCollegeId == $monitorCollegeId && $programMatches;
    } else {
        $accessAllowed = true;
    }
}

if (!$accessAllowed) {
    http_response_code(403);
    $denyMsg = $role == 'gened'
        ? 'Access denied. GE accounts can monitor GE courses only.'
        : 'You do not have access to monitor this classroom.';
    echo '<!DOCTYPE html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Access Denied</title>
<style>
*{margin:0;padding:0;box-sizing:border-box}
body{min-height:100vh;display:flex;align-items:center;justify-content:center;background:rgba(0,0,0,.05);font-family:-apple-
system,BlinkMacSystemFont,"Segoe UI",Roboto,sans-serif}
.notif{background:#fff;border-radius:12px;box-shadow:0 8px 32px rgba(0,0,0,.12);padding:28px 36px;text-align:center;max-
width:340px;animation:pop .3s ease}
.notif .icon{width:48px;height:48px;margin:0 auto 14px;border-radius:50%;background:#fee2e2;display:flex;align-items:center;justify-
content:center}
.notif .icon svg{width:24px;height:24px;color:#dc2626}
.notif h2{font-size:1rem;font-weight:600;color:#1f2937;margin-bottom:6px}
.notif p{font-size:.85rem;color:#6b7280;margin-bottom:18px}
.notif button{background:#3b82f6;color:#fff;border:none;border-radius:6px;padding:8px 20px;font-
size:.8rem;cursor:pointer;transition:background .2s}
.notif button:hover{background:#2563eb}
@keyframes pop{from{opacity:0;transform:scale(.9)}to{opacity:1;transform:scale(1)}}
</style></head><body>
<div class="notif">
<div class="icon"><svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" stroke-width="2" stroke="currentColor"><path
stroke-linecap="round" stroke-linejoin="round" d="M18.364 18.364A9 9 0 0 0 5.636 5.636m12.728 12.728A9 9 0 0 1 5.636 5.636m12.728
12.728L5.636 5.636"/></svg></div>
<h2>Access Denied</h2>
<p> . htmlspecialchars($denyMsg) . '</p>
<button onclick="window.close()">Close</button>
</div></body></html>';
    exit;
}

$hasContentWeeks = db_column_exists('classroom_content', 'weeks');
$hasContentDaysPerTopic = db_column_exists('classroom_content', 'days_per_topic');
$hasContentAttachments = db_table_exists('classroom_content_attachments');
$contentAttachmentMap = [];

$st = db()->prepare(
    'SELECT *
    FROM classroom_content
    WHERE classroom_id = ?
    ORDER BY created_at DESC'
);
$st->execute([$classroomId]);
$items = $st->fetchAll();
$contentWeeks = classroom_content_group_by_week($items);

if ($hasContentAttachments && $items != []) {
    $contentAttachmentMap = classroom_content_attachment_map(array_column($items, 'id'));
}

$weekLookup = [];
foreach ($contentWeeks as $group) {
    $weekLookup[(string) $group['label']] = $group;
}

if ($selectedWeek == '' && $contentWeeks != []) {

```

```

        $selectedWeek = (string) $contentWeeks[0]['label'];
    } elseif ($selectedWeek !== '' && $contentWeeks !== [] && !isset($weekLookup[$selectedWeek])) {
        $selectedWeek = (string) $contentWeeks[0]['label'];
    }

    $filteredItems = [];
    if ($selectedWeek !== '' && isset($weekLookup[$selectedWeek])) {
        $filteredItems = $weekLookup[$selectedWeek]['items'];
    } elseif ($contentWeeks === []) {
        $filteredItems = $items;
    }

    $monitorBaseQuery = ['id' => $classroomId];
    if ($monitorCollegeId > 0) {
        $monitorBaseQuery['monitor_college'] = $monitorCollegeId;
    }
    if ($monitorProgram !== '') {
        $monitorBaseQuery['monitor_program'] = $monitorProgram;
    }

    $monitorWeekHref = static function (string $weekLabel) use ($monitorBaseQuery): string {
        $query = $monitorBaseQuery;
        $query['week'] = $weekLabel;

        return 'classroom_materials_monitor.php?' . http_build_query($query);
    };

    $itemAttachmentSummary = static function (array $item) use ($contentAttachmentMap, $hasContentAttachments): string {
        $parts = [];
        $resource = trim((string) ($item['resource_url'] ?? ''));
        if ($resource !== '') {
            $parts[] = classroom_content_is_attachment($resource)
                ? classroom_content_attachment_name($resource)
                : 'External link';
        }
        if ($hasContentAttachments) {
            $attachments = $contentAttachmentMap[(int) ($item['id'] ?? 0)] ?? [];
            foreach ($attachments as $attachment) {
                $parts[] = classroom_content_attachment_download_name(
                    (string) ($attachment['original_name'] ?? ''),
                    (string) ($attachment['stored_name'] ?? '')
                );
            }
        }

        return implode(' ', $parts);
    };

    $pageTitle = 'Classroom materials monitor';
    require_once __DIR__ . '/includes/header.php';
    ?>

<style>
.monitor-week-card {
    border: 1px solid rgba(0, 0, 0, 0.08);
    border-radius: 0.5rem;
    padding: 0.75rem 1rem;
    text-decoration: none;
    color: inherit;
    display: block;
    height: 100%;
    background: var(--bs-body-bg);
    transition: background 0.12s ease, border-color 0.12s ease, box-shadow 0.12s ease;
}
.monitor-week-card:hover {
    border-color: var(--bs-primary);
    background: rgba(var(--bs-primary-rgb), 0.06);
    color: inherit;
}
.monitor-week-card.is-active {
    border-color: var(--bs-primary);
    background: rgba(var(--bs-primary-rgb), 0.08);
    box-shadow: 0 0 0 1px var(--bs-primary);
}
.monitor-item-row {
    width: 100%;
    text-align: left;
    border: 1px solid rgba(0, 0, 0, 0.08);
    border-radius: 0.5rem;
    padding: 0.85rem 1rem;
}

```

```

margin-bottom: 0.5rem;
background: var(--bs-body-bg);
cursor: pointer;
transition: background 0.12s ease, border-color 0.12s ease;
}
.monitor-item-row:hover,
.monitor-item-row:focus-visible {
border-color: var(--bs-primary);
background: rgba(var(--bs-primary-rgb), 0.04);
outline: none;
}
.monitor-item-row:last-child {
margin-bottom: 0;
}
}
</style>

<div class="d-flex flex-wrap justify-content-between align-items-start gap-2 mb-4">
  <div>
    <h1 class="h3 mb-1"><i class="fa-solid fa-folder-tree me-2 text-primary"></i>Classroom materials monitor</h1>
    <div class="text-muted">
      <?>= htmlspecialchars((string) $classroom['course_code']) ?> - <?>= htmlspecialchars((string) $classroom['course_name']) ?>
      | Faculty: <?>= htmlspecialchars((string) $classroom['faculty_name']) ?>
      | <?>= htmlspecialchars((string) $classroom['semester']) ?> / <?>= htmlspecialchars((string) $classroom['school_year']) ?>
    </div>
  </div>
  <div class="d-flex flex-wrap gap-2">
    <a href="view_schedule.php" class="btn btn-outline-secondary btn-sm"><i class="fa-solid fa-arrow-left me-1"></i>Back to Weekly
Schedule</a>
  </div>
</div>

<div class="card shadow-sm mb-4">
  <div class="card-header bg-white">
    <strong>Weekly topic coverage</strong>
  </div>
  <div class="card-body">
    <?php if ($contentWeeks === []): ?>
      <p class="text-muted mb-0">No posted materials/topics yet for this classroom.</p>
    <?php else: ?>
      <p class="small text-muted mb-3">Select a week to browse posted materials and topics for that period.</p>
      <div class="row g-2">
        <?php foreach ($contentWeeks as $group): ?>
          <?php $weekLabel = (string) $group['label']; ?>
          <?php $isActiveWeek = $selectedWeek === $weekLabel; ?>
          <div class="col-md-4 col-lg-3">
            <a
              class="monitor-week-card<?>= $isActiveWeek ? ' is-active' : '' ?>"
              href="<?>= htmlspecialchars($monitorWeekHref($weekLabel)) ?>"
              <?>= app_tooltip_attr('Show materials posted for ' . $weekLabel . '.') ?>
            >
              <div class="fw-semibold"><i class="fa-regular fa-calendar me-2 text-primary"></i><?>=
htmlspecialchars($weekLabel) ?></div>
              <div class="small text-muted"><?>= (int) $group['count'] ?> item<?>= (int) $group['count'] === 1 ? '' : 's' ?>
            </div>
          </a>
        </div>
      <?php endforeach; ?>
    </div>
  <?php endif; ?>
</div>

<div class="card shadow-sm">
  <div class="card-header bg-white d-flex justify-content-between align-items-center flex-wrap gap-2">
    <strong>
      Posted materials and topics
      <?php if ($selectedWeek !== '' && $contentWeeks !== []): ?>
        <span class="text-muted fw-normal">- <?>= htmlspecialchars($selectedWeek) ?></span>
      <?php endif; ?>
    </strong>
    <span class="badge text-bg-primary"><?>= count($filteredItems) ?> in view<?>= $contentWeeks !== [] ? ' . ' . count($items) . ' total'
: '' ?></span>
  </div>
  <div class="card-body">
    <?php if ($items === []): ?>
      <p class="text-muted mb-0">No classroom content to monitor yet.</p>
    <?php elseif ($filteredItems === []): ?>
      <p class="text-muted mb-0">No materials were posted for this week.</p>
    <?php else: ?>
      <p class="small text-muted mb-3">Click an item to open its full details in a popup.</p>
    </div>
  </div>
</div>

```

```

<?php foreach ($filteredItems as $item): ?>
    <?php
    $itemId = (int) ($item['id'] ?? 0);
    $attachmentSummary = $itemAttachmentSummary($item);
    $hasBody = trim((string) ($item['body'] ?? '')) !== '';
    ?>
    <button
        type="button"
        class="monitor-item-row"
        data-bs-toggle="modal"
        data-bs-target="#monitorContentModal"
        data-monitor-detail="monitor-detail-<?=$itemId ?>"
        data-monitor-title="<?=$item['title'] ?? 'Untitled', ENT_QUOTES ?>"
        <?=$app_tooltip_attr('Open full details for ' . (string) ($item['title'] ?? 'this item') . ' .') ?>
    >
        <div class="d-flex flex-wrap justify-content-between align-items-start gap-2">
            <div class="min-w-0">
                <div class="fw-semibold"><?=$item['title'] ?? 'Untitled' ?></div>
                <div class="small text-muted text-capitalize"><?=$item['content_type'] ??
'material') ?></div>
            </div>
            <div class="small text-muted text-nowrap"><?=$item['created_at'] ?? '' ?></div>
        </div>
        <div class="small text-muted mt-2 d-flex flex-wrap gap-2">
            <?php if ($hasBody): ?>
                <span><i class="fa-regular fa-file-lines me-1"></i>Has description</span>
            <?php endif; ?>
            <?php if ($attachmentSummary !== ''): ?>
                <span><i class="fa-solid fa-paperclip me-1"></i><?=$attachmentSummary ?></span>
            <?php endif; ?>
            <?php if (!$hasBody && $attachmentSummary === ''): ?>
                <span><i class="fa-regular fa-eye me-1"></i>View details</span>
            <?php endif; ?>
        </div>
    </button>

    <div id="monitor-detail-<?=$itemId ?>" class="d-none">
        <div class="d-flex flex-wrap justify-content-between align-items-start gap-2 mb-3">
            <div>
                <div class="text-muted small text-capitalize mb-1"><?=$item['content_type'] ??
'material') ?></div>
                <div class="small text-muted">Posted <?=$item['created_at'] ?? '' ?></div>
            </div>
        </div>

        <?php if ($hasContentWeeks): ?>
            <div class="small text-muted mb-3">
                <?php $weekLabel = trim((string) ($item['weeks'] ?? '')); ?>
                <span class="me-3">
                    <i class="fa-regular fa-calendar me-1"></i>Week: <?=$weekLabel !== '' ? $weekLabel :
'General resources') ?>
                </span>
            <?php if ($hasContentDaysPerTopic): ?>
                <?php $daysTopic = trim((string) ($item['days_per_topic'] ?? '')); ?>
                <?php if ($daysTopic !== ''): ?>
                    <span><i class="fa-regular fa-clock me-1"></i>Days/topic: <?=$daysTopic ?></span>
                <?php endif; ?>
            <?php endif; ?>
        </div>
    <?php endif; ?>

    <?php if ($hasBody): ?>
        <div class="classroom-content-body mb-3"><?=$classroom_content_render_body((string) $item['body']) ?></div>
    <?php endif; ?>

    <?php $resource = trim((string) ($item['resource_url'] ?? '')); ?>
    <?php if ($resource !== ''): ?>
        <div class="mb-2">
            <?php if (classroom_content_is_attachment($resource)): ?>
                <span class="text-muted"><i class="fa-solid fa-paperclip me-1"></i><?=$
htmlspecialchars(classroom_content_attachment_name($resource)) ?></span>
            <?php else: ?>
                <a href="<?=$resource ?>" target="_blank" rel="noopener noreferrer"><i class="fa-solid
fa-link me-1"></i>External resource</a>
            <?php endif; ?>
        </div>
    <?php endif; ?>

    <?php if ($hasContentAttachments): ?>
        <?php $attachments = $contentAttachmentMap[$itemId] ?? []; ?>

```

```

        <?php foreach ($attachments as $attachment): ?>
            <div class="mb-2">
                <span class="text-muted">
                    <i class="fa-solid fa-paperclip me-1"></i><?=  

htmlspecialchars(classroom_content_attachment_download_name(
                    (string) ($attachment['original_name'] ?? ''),
                    (string) ($attachment['stored_name'] ?? ''))
                </span>
            </div>
        <?php endforeach; ?>
    <?php endif; ?>

    <?php
    $detailAttachments = $hasContentAttachments ? ($contentAttachmentMap[$itemId] ?? []) : [];
    ?>
    <?php if (!$hasBody && $resource === '' && $detailAttachments === []): ?>
        <p class="text-muted mb-0">No additional details were posted for this item.</p>
    <?php endif; ?>
</div>
<?php endforeach; ?>
<?php endif; ?>
</div>
</div>

<div class="modal fade" id="monitorContentModal" tabindex="-1" aria-labelledby="monitorContentModalLabel" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered modal-lg modal-dialog-scrollable">
        <div class="modal-content rounded-4 border-0 shadow">
            <div class="modal-header border-0 pb-0">
                <h2 class="modal-title h5 fw-semibold" id="monitorContentModalLabel">Content detail</h2>
                <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"><? app_tooltip_attr('Close this detail  

view.') ?></button>
            </div>
            <div class="modal-body pt-2" id="monitorContentModalBody"></div>
            <div class="modal-footer border-0 pt-0">
                <button type="button" class="btn btn-outline-secondary rounded-pill" data-bs-dismiss="modal"><? app_tooltip_attr('Close this  

detail view.') ?>>Close</button>
            </div>
        </div>
    </div>
</div>
</div>

<script>
(function () {
    var modal = document.getElementById('monitorContentModal');
    if (!modal) {
        return;
    }

    modal.addEventListener('show.bs.modal', function (event) {
        var trigger = event.relatedTarget;
        if (!trigger) {
            return;
        }

        var detailId = trigger.getAttribute('data-monitor-detail') || '';
        var title = trigger.getAttribute('data-monitor-title') || 'Content detail';
        var source = detailId ? document.getElementById(detailId) : null;
        var titleEl = document.getElementById('monitorContentModalLabel');
        var bodyEl = document.getElementById('monitorContentModalBody');

        if (titleEl) {
            titleEl.textContent = title;
        }
        if (bodyEl) {
            bodyEl.innerHTML = source ? source.innerHTML : '<p class="text-muted mb-0">Details are not available.</p>';
        }
    });
})();
</script>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: classroom_syllabus.php

```

<?php
declare(strict_types=1);

```

```

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_login();

$classroomId = (int) ($_GET['id'] ?? 0);
if ($classroomId < 1) {
    http_response_code(400);
    exit('Invalid classroom.');
```



```

if (!$allowed) {
    http_response_code(403);
    echo '<!DOCTYPE html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Access Denied</title>
<style>
*{margin:0;padding:0;box-sizing:border-box}
body{min-height:100vh;display:flex;align-items:center;justify-content:center;background:rgba(0,0,0,.05);font-family:-apple-
system,BlinkMacSystemFont,"Segoe UI",Roboto,sans-serif}
.notif{background:#fff;border-radius:12px;box-shadow:0 8px 32px rgba(0,0,0,.12);padding:28px 36px;text-align:center;max-
width:340px;animation:pop .3s ease}
.notif .icon{width:48px;height:48px;margin:0 auto 14px;border-radius:50%;background:#fee2e2;display:flex;align-items:center;justify-
content:center}
.notif .icon svg{width:24px;height:24px;color:#dc2626}
.notif h2{font-size:1rem;font-weight:600;color:#1f2937;margin-bottom:6px}
.notif p{font-size:.85rem;color:#6b7280;margin-bottom:18px}
.notif button{background:#3b82f6;color:#fff;border:none;border-radius:6px;padding:8px 20px;font-
size:.8rem;cursor:pointer;transition:background .2s}
.notif button:hover{background:#2563eb}
@keyframes pop{from{opacity:0;transform:scale(.9)}to{opacity:1;transform:scale(1)}}
</style></head><body>
<div class="notif">
<div class="icon"><svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" stroke-width="2" stroke="currentColor"><path
stroke-linecap="round" stroke-linejoin="round" d="M18.364 18.364A9 9 0 0 0 5.636 5.636 5.636 5.636 12.728 12.728A9 9 0 0 1 5.636 5.636 12.728
12.728L5.636 5.636"/></svg></div>
<h2>Access Denied</h2>
<p>You do not have access to this syllabus.</p>
<button onclick="window.close()">Close</button>
</div></body></html>';
    exit;
}

$path = classroom_content_attachment_storage_path($storedName);
if (!is_file($path)) {
    http_response_code(404);
    exit('Syllabus file is missing.');
```

```

-----
FILE: config/config.example.php
-----
```

```

<?php
/**
 * Application configuration – copy to config.php and edit for your environment.
 */
declare(strict_types=1);

/** Application timezone (Philippine Standard Time). Keep this on production so times match PH. */
define('APP_TIMEZONE', 'Asia/Manila');
date_default_timezone_set(APP_TIMEZONE);

define('DB_HOST', '127.0.0.1');
/** MySQL port (default 3306). Use XAMPP Control Panel → MySQL → Config to confirm. */
define('DB_PORT', 3306);
define('DB_NAME', 'faculty_scheduling');
define('DB_USER', 'root');
define('DB_PASS', 'password');
```

```

define('DB_CHARSET', 'utf8mb4');

/** Default admin (used by install.php if no admin exists) */
define('DEFAULT_ADMIN_USERNAME', 'admin');
define('DEFAULT_ADMIN_PASSWORD', 'admin123');
define('DEFAULT_ADMIN_FULL_NAME', 'System Administrator');
/** Bootstrap account: creates admin users only; change password after first login. */
define('DEFAULT_SUPER_ADMIN_USERNAME', 'superadmin');
define('DEFAULT_SUPER_ADMIN_PASSWORD', 'superadmin123');
define('DEFAULT_SUPER_ADMIN_FULL_NAME', 'Super Administrator');
define('DEFAULT_GENED_USERNAME', 'gened');
define('DEFAULT_GENED_PASSWORD', 'gened123');
define('DEFAULT_GENED_FULL_NAME', 'General Education Admin');

/** GE faculty may message only this dean (user id). Set 0 to resolve by GE_DEAN_NAME_HINT. */
define('GE_DEAN_USER_ID', 0);
/** Dean full-name hint when GE_DEAN_USER_ID is 0 (all tokens must appear in the dean name). */
define('GE_DEAN_NAME_HINT', '');

/** Max messages kept per thread; oldest removed first when exceeded (FIFO). Set 0 for no limit. */
define('MESSAGING_THREAD_MAX_MESSAGES', 10);

define('SESSION_NAME', 'FSS_SESSION');
define('BASE_PATH', dirname(__DIR__));

/*
 * Email (registration credentials + password resets)
 */
* APP_BASE_URL: Site root in emails (no trailing slash). Used if MAIL_LOGIN_URL is not set.
* MAIL_LOGIN_URL: Full URL to login.php (used in credential emails; button + plain-text link).
* MAIL_ENABLED: Set false to skip sending (accounts still save; admins see email-failure notice).
* MAIL_FROM_ADDRESS / MAIL_FROM_NAME: Sender shown to recipients.
* MAIL_ORGANIZATION_NAME / MAIL_SUPPORT_CONTACT / MAIL_LOGO_URL: Registration email branding.
*
* Delivery options:
* 1) Leave MAIL_SMTP_HOST empty – uses PHP mail() (Unix sendmail, or Windows php.ini SMTP).
* 2) Set MAIL_SMTP_HOST only – sets ini SMTP/smtp_port for Windows (e.g. Mercury on 127.0.0.1:25).
* 3) Set MAIL_SMTP_HOST + MAIL_SMTP_USER + MAIL_SMTP_PASS – sends via SMTP AUTH (e.g. smtp.gmail.com:587).
*/
define('APP_BASE_URL', 'http://localhost/CLASS');
/** Full login page URL for credential emails (Sign in button). */
define('MAIL_LOGIN_URL', 'http://localhost/CLASS/login.php');
define('MAIL_ENABLED', false);
define('MAIL_FROM_ADDRESS', 'noreply@example.com');
define('MAIL_FROM_NAME', 'WPU SABLAE Portal');
/** Organization name shown in registration credential emails. */
define('MAIL_ORGANIZATION_NAME', 'WPU SABLAE Portal');
/** Technical support line in credential emails (phone, email, or department). */
define('MAIL_SUPPORT_CONTACT', 'IT Help Desk – helpdesk@example.com');
/** Optional absolute URL to logo image for HTML emails (defaults to APP_BASE_URL/assets/logo.png). */
define('MAIL_LOGO_URL', '');
/** Empty = default PHP mail(). Google Workspace / Gmail: smtp.gmail.com:587 + app password (no spaces). */
define('MAIL_SMTP_HOST', '');
define('MAIL_SMTP_PORT', 587);
define('MAIL_SMTP_USER', '');
define('MAIL_SMTP_PASS', '');
/** Use STARTTLS after EHLO (typical for port 587). */
define('MAIL_SMTP_TLS', true);
/** Windows mail(): optional envelope sender (some hosts require it). */
define('MAIL_SENDMAIL_FROM', '');
define('TIME_MIN', '06:00:00');
define('TIME_MAX', '22:00:00');
define('MIN_CLASS_MINUTES', 30);
/** Minimum minutes between class periods on the same day; 0 disables the spacing rule. */
define('MIN_GAP_MINUTES', 0);
define('MAX_CLASS_BLOCK_HOURS', 9);
define('MAX_CONSECUTIVE_HOURS', 3);

/*
 * Wellness companion (student_wellness.php) – Groq cloud
 */
* 1. Create free key: https://console.groq.com/keys
* 2. Paste below (starts with gsk_) OR set server env GROQ_API_KEY
* 3. Reload EduTools → Wellness check-in
*/
define('WELLNESS_AI_ENABLED', false);
define('WELLNESS_AI_PROVIDER', 'groq');
define('WELLNESS_AI_API_KEY', (static function (): string {
    $env = getenv('GROQ_API_KEY');
    if (is_string($env) && trim($env) !== '') {

```

```

        return trim($env);
    }
    return '';
})();
define('WELLNESS_AI_BASE_URL', 'https://api.groq.com/openai/v1');
/** Fast: llama-3.1-8b-instant | Best quality: llama-3.3-70b-versatile */
define('WELLNESS_AI_MODEL', 'llama-3.3-70b-versatile');
define('WELLNESS_OLLAMA_URL', 'http://127.0.0.1:11434');
define('WELLNESS_OLLAMA_MODEL', 'llama3.2');

```

FILE: conflicts.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['admin', 'dean', 'program_chair', 'gened', 'faculty']);
$role = (string) ($_SESSION['role'] ?? '');
$collegeId = current_college_id();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
$userId = (int) ($_SESSION['user_id'] ?? 0);
$flash = $_SESSION['flash'] ?? '';
$flashType = (string) ($_SESSION['flash_type'] ?? 'success');
unset($_SESSION['flash'], $_SESSION['flash_type']);
$makeupWarningPopup = [];
if (!empty($_SESSION['makeup_warning_popup']) && is_array($_SESSION['makeup_warning_popup'])) {
    $makeupWarningPopup = array_values(array_filter(array_map('strval', $_SESSION['makeup_warning_popup'])));
}
unset($_SESSION['makeup_warning_popup']);

if ($role === 'admin' && $_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['request_id'])) {
    $rid = (int) $_POST['request_id'];
    $action = (string) ($_POST['action'] ?? '');
    $remarks = trim((string) ($_POST['remarks'] ?? ''));
    $stmt = db()->prepare('SELECT * FROM conflict_requests WHERE id=? AND status="pending"');
    $stmt->execute([$rid]);
    $req = $stmt->fetch();
    if ($req) {
        if ($action === 'approve') {
            $beforeReq = [
                'id' => (int) $req['id'],
                'status' => (string) $req['status'],
                'college_id' => (int) $req['college_id'],
                'faculty_id' => (int) $req['faculty_id'],
                'course_id' => (int) $req['course_id'],
                'room_id' => (int) $req['room_id'],
                'semester' => (string) $req['semester'],
                'school_year' => (string) $req['school_year'],
            ];
            $ins = db()->prepare(
                'INSERT INTO schedules (faculty_id, course_id, room_id, college_id, schedule_type, day_of_week, start_time, end_time, semester, school_year, academic_year, created_by)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)'
            );
            $ins->execute([
                (int) $req['faculty_id'],
                (int) $req['course_id'],
                (int) $req['room_id'],
                (int) $req['college_id'],
                $req['schedule_type'],
                $req['day_of_week'],
                $req['start_time'],
                $req['end_time'],
                $req['semester'],
                $req['school_year'],
                $req['academic_year'],
                (int) $req['requested_by'],
            ]);
            $newId = (int) db()->lastInsertId();
            db()->prepare('UPDATE conflict_requests SET status="approved", admin_remarks=?, reviewed_by=?, reviewed_at=NOW() WHERE id=?')
            ->execute([$remarks, $userId, $rid]);
            log_admin_activity(
                'edit',
                'Conflict requests',
                'Override request #' . $rid,
            );

```

```

        $beforeReq,
        [
            'id' => $rid,
            'status' => 'approved',
            'admin_remarks' => $remarks,
            'reviewed_by' => $userId,
            'created_schedule_id' => $newId,
        ]
    );
    $stSched = db()->prepare(
        'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year,
            s.day_of_week, s.start_time, s.end_time, s.schedule_type,
            f.full_name AS faculty_name, c.course_code, r.room_code
        FROM schedules s
        INNER JOIN faculty f ON f.id = s.faculty_id
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN rooms r ON r.id = s.room_id
        WHERE s.id = ? LIMIT 1'
    );
    $stSched->execute([$newId]);
    $schedRow = $stSched->fetch(PDO::FETCH_ASSOC);
    log_admin_activity(
        'add',
        'Schedules',
        'Schedule #' . $newId . ' (approved override request #' . $rid . ' )',
        null,
        $schedRow ? (array) $schedRow : ['id' => $newId, 'note' => 'Row fetch after insert failed']
    );
    $_SESSION['flash'] = 'Override request approved and schedule #' . $newId . ' was created.';
} elseif ($action === 'reject') {
    $beforeReq = [
        'id' => (int) $req['id'],
        'status' => (string) $req['status'],
        'college_id' => (int) $req['college_id'],
        'faculty_id' => (int) $req['faculty_id'],
        'course_id' => (int) $req['course_id'],
        'room_id' => (int) $req['room_id'],
    ];
    db()->prepare('UPDATE conflict_requests SET status="rejected", admin_remarks=?, reviewed_by=?, reviewed_at=NOW() WHERE id=?')
    ->execute([$remarks, $userId, $rid]);
    log_admin_activity(
        'edit',
        'Conflict requests',
        'Override request #' . $rid,
        $beforeReq,
        [
            'id' => $rid,
            'status' => 'rejected',
            'admin_remarks' => $remarks,
            'reviewed_by' => $userId,
        ]
    );
    $_SESSION['flash'] = 'Override request rejected.';
}
}
header('Location: conflicts.php');
exit;
}

if ($role === 'program_chair' && $_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['scr_id'])) {
    $_SESSION['flash'] = 'Makeup class requests are reviewed by the dean.';
    $_SESSION['flash_type'] = 'danger';
    header('Location: conflicts.php');
    exit;
}

if (($role === 'dean' || $role === 'admin') && $_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['scr_id'])) {
    $sid = (int) $_POST['scr_id'];
    $action = (string) ($_POST['action'] ?? '');
    $remarks = trim((string) ($_POST['remarks'] ?? ''));
    if ($action === 'approve' || $action === 'approve_anyway') {
        $status = 'approved';
    } elseif ($action === 'reject') {
        $status = 'rejected';
    } else {
        $_SESSION['flash'] = 'Unknown action.';
        $_SESSION['flash_type'] = 'danger';
        header('Location: conflicts.php');
        exit;
    }
}

```

```

$hasMakeupSupport = ensure_makeup_schedule_support();
if ($role === 'admin') {
    $sqlFetchScr = 'SELECT scr.*
        FROM schedule_change_requests scr
        INNER JOIN schedules s ON s.id = scr.schedule_id
        INNER JOIN users u ON u.id = scr.faculty_user_id
        WHERE scr.id=? AND scr.status="pending" AND u.role="dean"';
    $parFetchScr = [$sid];
} else {
    $sqlFetchScr = 'SELECT scr.*
        FROM schedule_change_requests scr
        INNER JOIN schedules s ON s.id = scr.schedule_id
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN users u ON u.id = scr.faculty_user_id
        WHERE scr.id=? AND scr.status="pending" AND s.college_id=? AND u.role <> "dean"';
    $parFetchScr = [$sid, $collegeId];
}
$sqlFetchScr .= ' LIMIT 1';
$stmtScrB = db()->prepare($sqlFetchScr);
$stmtScrB->execute($parFetchScr);
$beforeScr = $stmtScrB->fetch(PDO::FETCH_ASSOC);

if (!$beforeScr) {
    $_SESSION['flash'] = 'Request not found or already reviewed.';
    header('Location: conflicts.php');
    exit;
}

$requestType = $hasMakeupSupport ? (string) ($beforeScr['request_type'] ?? 'change') : 'change';
$createdMakeupId = 0;
$forceMakeup = $action === 'approve_anyway';

if (($action === 'approve' || $forceMakeup) && $requestType === 'makeup') {
    $status = 'approved';
    $stmtBase = db()->prepare('SELECT * FROM schedules WHERE id=? LIMIT 1');
    $stmtBase->execute([(int) $beforeScr['schedule_id']]);
    $baseSchedule = $stmtBase->fetch(PDO::FETCH_ASSOC) ?: [];
    $created = create_makeup_schedule_from_request($beforeScr, $baseSchedule, $userId, $forceMakeup);
    if (empty($created['ok'])) {
        $msg = (string) ($created['error'] ?? 'Could not create makeup schedule.');
```

```

        $descriptions = [];
        if (!empty($created['conflict_messages']) && is_array($created['conflict_messages'])) {
            $descriptions = array_values(array_filter(array_map('strval', $created['conflict_messages'])));
        } elseif (!empty($created['conflicts']) && is_array($created['conflicts'])) {
            foreach ($created['conflicts'] as $c) {
                if (is_array($c) && isset($c['description'])) {
                    $descriptions[] = (string) $c['description'];
                } elseif (is_string($c)) {
                    $descriptions[] = $c;
                }
            }
        }
        if ($descriptions !== []) {
            $_SESSION['makeup_warning_popup'] = $descriptions;
            if (!str_contains($msg, $descriptions[0])) {
                $msg .= ' ' . implode(' ', $descriptions);
            }
        }
        $_SESSION['flash'] = $msg;
        $_SESSION['flash_type'] = 'danger';
        header('Location: conflicts.php');
        exit;
    }
    $createdMakeupId = (int) ($created['schedule_id'] ?? 0);
    if ($remarks === '') {
        $remarks = 'Makeup schedule #' . $createdMakeupId . ' created.';
    } else {
        $remarks .= ' (Makeup schedule #' . $createdMakeupId . ' created.)';
    }
} elseif ($forceMakeup) {
    $_SESSION['flash'] = '"Approve anyway" is only for makeup requests.';
    $_SESSION['flash_type'] = 'danger';
    header('Location: conflicts.php');
    exit;
}

if ($role === 'admin') {
    $sql = 'UPDATE schedule_change_requests scr
        INNER JOIN schedules s ON s.id = scr.schedule_id
        INNER JOIN users u ON u.id = scr.faculty_user_id

```

```

        SET scr.status=?, scr.dean_remarks=?, scr.reviewed_by=?, scr.reviewed_at=NOW()
        WHERE scr.id=? AND scr.status="pending" AND u.role="dean";
    $params = [$status, $remarks, $userId, $sid];
} else {
    $sql = 'UPDATE schedule_change_requests scr
    INNER JOIN schedules s ON s.id = scr.schedule_id
    INNER JOIN courses c ON c.id = s.course_id
    INNER JOIN users u ON u.id = scr.faculty_user_id
    SET scr.status=?, scr.dean_remarks=?, scr.reviewed_by=?, scr.reviewed_at=NOW()
    WHERE scr.id=? AND scr.status="pending" AND s.college_id=? AND u.role <> "dean"';
    $params = [$status, $remarks, $userId, $sid, $collegeId];
}
$stmt = db()->prepare($sql);
$stmt->execute($params);
if ($beforeScr && $stmt->rowCount() > 0) {
    log_user_activity(
        'edit',
        'Schedule change requests',
        'Request #' . $sid,
        [
            'id' => (int) $beforeScr['id'],
            'schedule_id' => (int) $beforeScr['schedule_id'],
            'faculty_user_id' => (int) $beforeScr['faculty_user_id'],
            'status' => (string) $beforeScr['status'],
            'message' => (string) $beforeScr['message'],
            'request_type' => $requestType,
        ],
        [
            'id' => $sid,
            'status' => $status,
            'dean_remarks' => $remarks,
            'reviewed_by' => $userId,
            'created_makeup_schedule_id' => $createdMakeupId ?: null,
        ]
    );
    if ($createdMakeupId > 0) {
        log_user_activity(
            'add',
            'Schedules',
            'Makeup schedule #' . $createdMakeupId . ' (request #' . $sid . ')',
            null,
            ['id' => $createdMakeupId, 'is_makeup' => 1, 'from_request' => $sid]
        );
    }
}
if ($role === 'admin') {
    log_admin_activity(
        'edit',
        'Schedule change requests',
        'Reviewed dean schedule change request #' . $sid . ': ' . $status,
        $beforeScr ? (array) $beforeScr : null,
        [
            'id' => $sid,
            'status' => $status,
            'dean_remarks' => $remarks,
            'reviewed_by' => $userId,
            'created_makeup_schedule_id' => $createdMakeupId ?: null,
        ]
    );
} else {
    log_dean_activity('schedule_change_review', "Reviewed faculty schedule change request #{$sid}: {$status}");
}
$_SESSION['flash'] = $createdMakeupId > 0
    ? 'Makeup request approved. Schedule #' . $createdMakeupId . ' was created. Delete it after the makeup class is held.'
    : 'Schedule change request updated.';
$_SESSION['flash_type'] = 'success';
header('Location: conflicts.php');
exit;
}

$pendingRequests = [];
$makeupRequests = [];
$myRequests = [];
$changeRequests = [];
$legacyLogs = [];
$hasMakeupSupport = ensure_makeup_schedule_support();

if ($role === 'admin') {
    $pendingRequests = db()->query(
        'SELECT cr.*, u.full_name AS dean_name, c.college_name, f.full_name AS faculty_name, co.course_code, r.room_code

```

```

FROM conflict_requests cr
INNER JOIN users u ON u.id = cr.requested_by
INNER JOIN colleges c ON c.id = cr.college_id
INNER JOIN faculty f ON f.id = cr.faculty_id
INNER JOIN courses co ON co.id = cr.course_id
INNER JOIN rooms r ON r.id = cr.room_id
WHERE cr.status="pending"
ORDER BY cr.created_at ASC'
)->fetchAll();

if ($hasMakeupSupport) {
    $sql = 'SELECT scr.*, f.full_name AS faculty_name, co.course_code, c.college_name, r.room_code AS proposed_room_code
FROM schedule_change_requests scr
INNER JOIN schedules s ON s.id = scr.schedule_id
INNER JOIN users u ON u.id = scr.faculty_user_id
INNER JOIN faculty f ON f.user_id = scr.faculty_user_id
INNER JOIN courses co ON co.id = s.course_id
INNER JOIN colleges c ON c.id = s.college_id
LEFT JOIN rooms r ON r.id = scr.proposed_room_id
WHERE scr.status="pending" AND u.role="dean"';
} else {
    $sql = 'SELECT scr.*, f.full_name AS faculty_name, co.course_code, c.college_name, NULL AS proposed_room_code
FROM schedule_change_requests scr
INNER JOIN schedules s ON s.id = scr.schedule_id
INNER JOIN users u ON u.id = scr.faculty_user_id
INNER JOIN faculty f ON f.user_id = scr.faculty_user_id
INNER JOIN courses co ON co.id = s.course_id
INNER JOIN colleges c ON c.id = s.college_id
WHERE scr.status="pending" AND u.role="dean"';
}
$changeRequests = db()->query($sql . ' ORDER BY scr.created_at DESC')->fetchAll();
} elseif ($role === 'dean' && $collegeId) {
    $st = db()->prepare(
        'SELECT cr.*, c.college_name
FROM conflict_requests cr
INNER JOIN colleges c ON c.id = cr.college_id
WHERE cr.college_id=?
ORDER BY cr.created_at DESC'
    );
    $st->execute([$collegeId]);
    $myRequests = $st->fetchAll();

    $stOwnScr = db()->prepare('SELECT * FROM schedule_change_requests WHERE faculty_user_id=? ORDER BY created_at DESC');
    $stOwnScr->execute([$userId]);
    $myRequests = array_merge($stOwnScr->fetchAll(PDO::FETCH_ASSOC), $myRequests);

    if ($hasMakeupSupport) {
        $sql = 'SELECT scr.*, f.full_name AS faculty_name, co.course_code, r.room_code AS proposed_room_code
FROM schedule_change_requests scr
INNER JOIN schedules s ON s.id = scr.schedule_id
INNER JOIN faculty f ON f.user_id = scr.faculty_user_id
INNER JOIN courses co ON co.id = s.course_id
INNER JOIN users u ON u.id = scr.faculty_user_id
LEFT JOIN rooms r ON r.id = scr.proposed_room_id
WHERE s.college_id=? AND scr.status="pending" AND u.role <> "dean"';
    } else {
        $sql = 'SELECT scr.*, f.full_name AS faculty_name, co.course_code, NULL AS proposed_room_code
FROM schedule_change_requests scr
INNER JOIN schedules s ON s.id = scr.schedule_id
INNER JOIN faculty f ON f.user_id = scr.faculty_user_id
INNER JOIN courses co ON co.id = s.course_id
INNER JOIN users u ON u.id = scr.faculty_user_id
WHERE s.college_id=? AND scr.status="pending" AND u.role <> "dean"';
    }
    $st = db()->prepare($sql . ' ORDER BY scr.created_at DESC');
    $st->execute([$collegeId]);
    $changeRequests = $st->fetchAll();
} elseif ($role === 'faculty') {
    $st = db()->prepare('SELECT * FROM schedule_change_requests WHERE faculty_user_id=? ORDER BY created_at DESC');
    $st->execute([$userId]);
    $myRequests = $st->fetchAll();
}

$legacyLogs = db()->query('SELECT * FROM conflict_logs ORDER BY created_at DESC LIMIT 100')->fetchAll();

$pageTitle = 'Conflicts and Requests';
require_once __DIR__ . '/includes/header.php';
?>
<h1 class="h3 mb-4"><i class="fa-solid fa-triangle-exclamation me-2 text-warning"></i>Conflicts and Requests</h1>

```

```

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if ($role === 'admin'): ?>
    <div class="card shadow-sm mb-4">
        <div class="card-header bg-white"><strong>Pending Admin Override Requests</strong></div>
        <div class="card-body p-0">
            <div class="table-responsive">
                <table class="table mb-0">
                    <thead class="table-light"><tr><th>College</th><th>Dean</th><th>Schedule</th><th>Reason</th><th>Action</th></tr></thead>
                    <tbody>
                        <?php if (!$pendingRequests): ?><tr><td colspan="5" class="p-3 text-muted">No pending requests.</td></tr><?php endif; ?>
                        <?php foreach ($pendingRequests as $r): ?>
                            <tr>
                                <td><?= htmlspecialchars($r['college_name']) ?></td>
                                <td><?= htmlspecialchars($r['dean_name']) ?></td>
                                <td><?= htmlspecialchars($r['course_code']) ?> / <?= htmlspecialchars($r['faculty_name']) ?> / <?=
htmlspecialchars($r['room_code']) ?> (<?= htmlspecialchars(substr($r['start_time'], 0, 5)) ?>-<?= htmlspecialchars(substr($r['end_time'], 0,
5)) ?></td>
                                <td><?= htmlspecialchars($r['reason']) ?></td>
                                <td>
                                    <form method="post" class="d-flex gap-1">
                                        <input type="hidden" name="request_id" value="<?= (int) $r['id'] ?>">
                                        <input type="text" name="remarks" class="form-control form-control-sm" placeholder="Remarks">
                                        <button name="action" value="approve" class="btn btn-sm btn-success"><?= app_tooltip_attr('Approves the
override and creates the schedule row. Use optional remarks for the audit trail.') ?>>Approve</button>
                                        <button name="action" value="reject" class="btn btn-sm btn-danger"><?= app_tooltip_attr('Rejects the
override request without creating a schedule. Use remarks to explain the decision.') ?>>Reject</button>
                                    </form>
                                </td>
                            </tr>
                        <?php endforeach; ?>
                    </tbody>
                </table>
            </div>
        </div>
    </div>
<?php endif; ?>

<?php if ($role === 'dean' || $role === 'admin'): ?>
    <div class="card shadow-sm mb-4">
        <div class="card-header bg-white"><strong><?= $role === 'admin' ? 'Dean Schedule Change / Makeup Requests' : 'Faculty Schedule
Change / Makeup Requests' ?></strong></div>
        <div class="card-body p-0">
            <div class="table-responsive">
                <table class="table mb-0">
                    <thead class="table-light"><tr><?php if ($role === 'admin'): ?><th>College</th><?php endif; ?><th>Faculty</th>
<th>Course</th><th>Type</th><th>Details</th><th>Action</th></tr></thead>
                    <tbody>
                        <?php if (!$changeRequests): ?><tr><td colspan="<?= $role === 'admin' ? 6 : 5 ?>" class="p-3 text-muted">No pending
change requests.</td></tr><?php endif; ?>
                        <?php foreach ($changeRequests as $r): ?>
                            <?php
                                $rtype = $hasMakeupSupport ? (string) ($r['request_type'] ?? 'change') : 'change';
                                $isMakeup = $rtype === 'makeup';
                                $detailParts = [(string) $r['message']];
                                $makeupConflictMsgs = [];
                                if ($isMakeup) {
                                    $day = str_replace(',', ' ', (string) ($r['proposed_day_of_week'] ?? ''));
                                    $st = substr((string) ($r['proposed_start_time'] ?? ''), 0, 5);
                                    $en = substr((string) ($r['proposed_end_time'] ?? ''), 0, 5);
                                    $room = (string) ($r['proposed_room_code'] ?? '');
                                    $slot = trim($day . ' ' . $st . ($st !== '' && $en !== '' ? '-' . $en : '')) . ($room !== '' ? ' @ ' . $room :
'');
                                }
                                if ($slot !== '') {
                                    array_unshift($detailParts, 'Proposed: ' . $slot);
                                }
                                $stBase = db()->prepare('SELECT faculty_id, college_id, semester, school_year FROM schedules WHERE id=? LIMIT
1');
                                $stBase->execute([(int) $r['schedule_id']]);
                                $base = $stBase->fetch(PDO::FETCH_ASSOC) ?: [];
                                if ($base) {
                                    $makeupConflictMsgs = makeup_hard_conflict_messages(
                                        (int) $base['faculty_id'],
                                        (int) ($r['proposed_room_id'] ?? 0) ?: 0,
                                        parse_day_set((string) ($r['proposed_day_of_week'] ?? '')),
                                        substr((string) ($r['proposed_start_time'] ?? ''), 0, 5) . ':00',
                                        substr((string) ($r['proposed_end_time'] ?? ''), 0, 5) . ':00',
                                        (string) $base['semester'],
                                        (string) $base['school_year'],
                                        isset($base['college_id']) ? (int) $base['college_id'] : null
                                    );
                                }
                            </?php>
                            <tr>
                                <th><?= $role === 'admin' ? 'College' : 'Faculty' ?></th>
                                <th><?= $role === 'admin' ? 'Faculty' : 'Course' ?></th>
                                <th><?= $role === 'admin' ? 'Type' : 'Details' ?></th>
                                <th><?= $role === 'admin' ? 'Action' : 'Details' ?></th>
                            </tr>
                        </tbody>
                    </table>
            </div>
        </div>
    </div>
<?php endif; ?>

```



```

        );
    }
    if ($makeupConflictMsgs !== []) {
        $detailParts[] = 'Conflicts: ' . implode(' | ', $makeupConflictMsgs);
    }
}
$conflictConfirm = $makeupConflictMsgs !== []
    ? ('Warning – This makeup conflicts with faculty or room schedule:\n\n- '
        . implode("\n- ", $makeupConflictMsgs)
        . "\n\nClick OK to try Approve (it will be blocked), or use Approve anyway to create it anyway.")
    : '';
?>
<tr<?=$makeupConflictMsgs !== [] ? ' class="table-warning" : '' ?>>
    <?php if ($role === 'admin'): ?>
        <td><?=$htmlspecialchars((string) ($r['college_name']) ?? '') ?></td>
    <?php endif; ?>
    <td><?=$htmlspecialchars((string) $r['faculty_name']) ?></td>
    <td><?=$htmlspecialchars((string) $r['course_code']) ?></td>
    <td>
        <?php if ($isMakeup): ?>
            <span class="badge bg-warning text-dark">Makeup</span>
            <?php if ($makeupConflictMsgs !== []): ?>
                <span class="badge bg-danger">Conflict</span>
            <?php endif; ?>
        <?php else: ?>
            <span class="badge bg-secondary">Change</span>
        <?php endif; ?>
    </td>
    <td>
        <?=$htmlspecialchars(implode(' - ', $detailParts)) ?>
        <?php if ($makeupConflictMsgs !== []): ?>
            <ul class="small text-danger mb-0 mt-1">
                <?php foreach ($makeupConflictMsgs as $cm): ?>
                    <li><?=$htmlspecialchars($cm) ?></li>
                <?php endforeach; ?>
            </ul>
        <?php endif; ?>
    </td>
    <td>
        <form method="post" class="d-flex flex-wrap gap-1 makeup-review-form"><?=$conflictConfirm !== '' ? ' data-conflict-warning="" . htmlspecialchars($conflictConfirm, ENT_QUOTES) . '' : '' ?>>
            <input type="hidden" name="scr_id" value="<?=(int) $r['id'] ?>">
            <input type="text" name="remarks" class="form-control form-control-sm" placeholder="Remarks" style="min-width:8rem">
            <button name="action" value="approve" class="btn btn-sm btn-success js-makeup-approve"><?=$app_tooltip_attr($isMakeup ? 'Approves the makeup request and creates a temporary schedule slot. Delete that slot after the makeup class is held.' : 'Approves the faculty member’s schedule change. Remarks are stored for documentation.') ?>>Approve</button>
            <?php if ($isMakeup): ?>
                <button name="action" value="approve_anyway" class="btn btn-sm btn-outline-warning js-makeup-approve-anyway"><?=$app_tooltip_attr('Creates the makeup schedule even if faculty/room conflicts exist. Use only when you intentionally allow the overlap.') ?>>Approve anyway</button>
            <?php endif; ?>
            <button name="action" value="reject" class="btn btn-sm btn-danger"><?=$app_tooltip_attr('Rejects the request so no schedule change is applied. Use remarks to tell faculty why.') ?>>Reject</button>
        </form>
    </td>
</tr>
<?php endforeach; ?>
</tbody>
</table>
</div>
</div>
</div>
<?php endif; ?>

<div class="card shadow-sm mb-4">
    <div class="card-header bg-white"><strong><?=$role === 'admin' ? 'Legacy Conflict Logs' : 'My Requests / Logs' ?></strong></div>
    <div class="card-body p-0">
        <div class="table-responsive">
            <table class="table mb-0">
                <thead class="table-light"><tr><th>Created</th><th>Type</th><th>Description</th><th>Status</th></tr></thead>
                <tbody>
                    <?php if ($myRequests): ?>
                        <?php foreach ($myRequests as $r): ?>
                            <tr>
                                <td><?=$htmlspecialchars((string) $r['created_at']) ?></td>
                                <td><?=$htmlspecialchars(
                                    $hasMakeupSupport && (($r['request_type'] ?? '') === 'makeup')
                                    ? 'Makeup'
                                    : ((isset($r['message']) ? 'Schedule request' : 'Request'))
                                )></td>

```

```

        ) ?></td>
        <td><?= htmlspecialchars((string) ($r['reason'] ?? $r['message'] ?? '')) ?></td>
        <td><?= htmlspecialchars((string) $r['status']) ?></td>
    </tr>
    <?php endforeach; ?>
<?php endif; ?>
<?php foreach ($legacyLogs as $r): ?>
    <tr>
        <td><?= htmlspecialchars((string) $r['created_at']) ?></td>
        <td><?= htmlspecialchars((string) $r['conflict_type']) ?></td>
        <td><?= htmlspecialchars((string) $r['conflict_description']) ?></td>
        <td><?= (int) $r['resolved'] === 1 ? 'Resolved' : 'Open' ?></td>
    </tr>
<?php endforeach; ?>
<?php if (!$myRequests && !$legacyLogs): ?>
    <tr><td colspan="4" class="p-3 text-muted">No entries.</td></tr>
<?php endif; ?>
</tbody>
</table>
</div>
</div>
</div>

<?php if ($makeupWarningPopup !== []): ?>
    <?php
    render_schedule_errors_warning_popup(
        $makeupWarningPopup,
        'Warning - Makeup conflicts with faculty or room schedule',
        'OK'
    );
    ?>
<?php endif; ?>

<script>
document.addEventListener('DOMContentLoaded', function () {
    document.querySelectorAll('form.makeup-review-form').forEach(function (form) {
        form.addEventListener('click', function (event) {
            var btn = event.target.closest('button[name="action"]');
            if (!btn || !form.contains(btn)) return;
            var warning = form.getAttribute('data-conflict-warning') || '';
            if (!warning) return;

            if (btn.value === 'approve') {
                event.preventDefault();
                window.alert(warning);
                return;
            }
            if (btn.value === 'approve_anyway') {
                var ok = window.confirm(
                    'Warning - This makeup conflicts with faculty or room schedule.\n\nCreate the makeup schedule anyway?'
                );
                if (!ok) {
                    event.preventDefault();
                }
            }
        });
    });
});
</script>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: courses.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['dean', 'program_chair']);
$collegeId = dean_or_program_chair_college_id_or_fail();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
$collegeName = college_name_by_id($collegeId);

$hasLabFlag = db_column_exists('courses', 'is_laboratory');
$hasLectureUnits = db_column_exists('courses', 'lecture_units');

```

```

$hasLaboratoryUnits = db_column_exists('courses', 'laboratory_units');
$hasYearLevel = db_column_exists('courses', 'year_level');
$hasSection = db_column_exists('courses', 'section');
$hasCourseBlock = $hasYearLevel && $hasSection;
$hasProgramsTable = db_table_exists('programs');
$hasClassroomCode = db_column_exists('courses', 'classroom_code');

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        // Auto-heal legacy DBs that still have UNIQUE(course_code).
        ensure_course_code_duplicates_allowed();
        $code = strtoupper(trim((string) ($_POST['course_code'] ?? '')));
        $name = trim((string) ($_POST['course_name'] ?? ''));
        $program = $programScope ?? trim((string) ($_POST['program'] ?? $_POST['department'] ?? ''));
        $yearLevel = trim((string) ($_POST['year_level'] ?? ''));
        $section = trim((string) ($_POST['section'] ?? ''));
        $isLab = !empty($_POST['is_laboratory']) ? 1 : 0;

        $lecUnits = $hasLectureUnits ? max(0.0, (float) ($_POST['lecture_units'] ?? 0)) : max(0.5, (float) ($_POST['units'] ?? 3.0));
        $labUnits = $hasLaboratoryUnits ? max(0.0, (float) ($_POST['laboratory_units'] ?? 0)) : ($isLab ? max(0.5, (float) ($_POST['units']
?? 3.0)) : 0.0);
        $legacyUnits = max(0.5, $lecUnits > 0 ? $lecUnits : ($isLab && $labUnits > 0 ? $labUnits : 3.0));
        if (($action === 'add' || $action === 'edit') && $hasCourseBlock && ($yearLevel === '' || $section === '')) {
            throw new RuntimeException('Year level and section are required.');
```

```

$ccCol = $hasClassroomCode ? ', classroom_code' : '';
$ccPh = $hasClassroomCode ? ',?' : '';

if ($action === 'add') {
    if ($hasLabFlag && $hasLectureUnits && $hasLaboratoryUnits && $hasCourseBlock) {
        $st = db()->prepare('INSERT INTO courses (course_code, course_name, units, lecture_units, laboratory_units, is_laboratory,
year_level, section, department, college_id' . $ccCol . ') VALUES (?,,?,,?,,?,,?,,?,,?' . $ccPh . ')');
        $params = [$code, $name, $legacyUnits, $lecUnits, $labUnits, $isLab, $yearLevel, $section, $program, $collegeId];
        if ($hasClassroomCode) {
            $params[] = $classroomCodeForWrite;
        }
        $st->execute($params);
    } elseif ($hasLabFlag && $hasLectureUnits && $hasLaboratoryUnits) {
        $st = db()->prepare('INSERT INTO courses (course_code, course_name, units, lecture_units, laboratory_units, is_laboratory,
department, college_id' . $ccCol . ') VALUES (?,,?,,?,,?,,?,,?' . $ccPh . ')');
        $params = [$code, $name, $legacyUnits, $lecUnits, $labUnits, $isLab, $program, $collegeId];
        if ($hasClassroomCode) {
            $params[] = $classroomCodeForWrite;
        }
        $st->execute($params);
    } elseif ($hasLabFlag && $hasCourseBlock) {
        $st = db()->prepare('INSERT INTO courses (course_code, course_name, units, is_laboratory, year_level, section, department,
college_id' . $ccCol . ') VALUES (?,,?,,?,,?,,?,,?' . $ccPh . ')');
        $params = [$code, $name, $legacyUnits, $isLab, $yearLevel, $section, $program, $collegeId];
        if ($hasClassroomCode) {
            $params[] = $classroomCodeForWrite;
        }
        $st->execute($params);
    } elseif ($hasLabFlag) {
        $st = db()->prepare('INSERT INTO courses (course_code, course_name, units, is_laboratory, department, college_id' . $ccCol .
') VALUES (?,,?,,?,,?,,?,,?' . $ccPh . ')');
        $params = [$code, $name, $legacyUnits, $isLab, $program, $collegeId];
        if ($hasClassroomCode) {
            $params[] = $classroomCodeForWrite;
        }
        $st->execute($params);
    } elseif ($hasCourseBlock) {
        $st = db()->prepare('INSERT INTO courses (course_code, course_name, units, year_level, section, department, college_id' .
$ccCol . ') VALUES (?,,?,,?,,?,,?,,?' . $ccPh . ')');
        $params = [$code, $name, $legacyUnits, $yearLevel, $section, $program, $collegeId];
        if ($hasClassroomCode) {
            $params[] = $classroomCodeForWrite;
        }
        $st->execute($params);
    } else {
        $st = db()->prepare('INSERT INTO courses (course_code, course_name, units, department, college_id' . $ccCol . ') VALUES
(?,,?,,?,,?,,?,,?' . $ccPh . ')');
        $params = [$code, $name, $legacyUnits, $program, $collegeId];
        if ($hasClassroomCode) {
            $params[] = $classroomCodeForWrite;
        }
        $st->execute($params);
    }
    $newCourseId = (int) db()->lastInsertId();
    $snap = db()->prepare('SELECT * FROM courses WHERE id = ? LIMIT 1');
    $snap->execute([$newCourseId]);
    $afterCourse = $snap->fetch(PDO::FETCH_ASSOC);
    log_user_activity('add', 'Courses', 'Course #' . $newCourseId, null, $afterCourse ? (array) $afterCourse : null);
    log_dean_activity('course_create', 'Created course ' . $code);
    $_SESSION['flash'] = 'Course added.';
    if ($hasClassroomCode && is_program_chair() && $classroomCodeForWrite !== null && $classroomCodeForWrite !== '') {
        $_SESSION['flash'] .= ' Classroom code for this subject: ' . $classroomCodeForWrite . '.';
    }
} elseif ($action === 'edit' && isset($_POST['id'])) {
    $id = (int) $_POST['id'];
    $sqlBefore = 'SELECT * FROM courses WHERE id=? AND college_id=?';
    $parBefore = [$id, $collegeId];
    if ($programScope !== null) {
        $sqlBefore .= ' AND department=?';
        $parBefore[] = $programScope;
    }
    $stBeforeC = db()->prepare($sqlBefore);
    $stBeforeC->execute($parBefore);
    $beforeCourse = $stBeforeC->fetch(PDO::FETCH_ASSOC);
    $ccSet = $hasClassroomCode ? ', classroom_code=?' : '';
    if ($hasLabFlag && $hasLectureUnits && $hasLaboratoryUnits && $hasCourseBlock) {
        $st = db()->prepare('UPDATE courses SET course_code=?, course_name=?, units=?, lecture_units=?, laboratory_units=?,
is_laboratory=?, year_level=?, section=?, department=?' . $ccSet . ' WHERE id=? AND college_id=?');
        $params = [$code, $name, $legacyUnits, $lecUnits, $labUnits, $isLab, $yearLevel, $section, $program];
        if ($hasClassroomCode) {

```

```

        $params[] = $classroomCodeForWrite;
    }
    $params[] = $id;
    $params[] = $collegeId;
    $st->execute($params);
} elseif ($hasLabFlag && $hasLectureUnits && $hasLaboratoryUnits) {
    $st = db()->prepare('UPDATE courses SET course_code=?, course_name=?, units=?, lecture_units=?, laboratory_units=?,
is_laboratory=?, department=?' . $ccSet . ' WHERE id=? AND college_id=?');
    $params = [$code, $name, $legacyUnits, $lecUnits, $labUnits, $isLab, $program];
    if ($hasClassroomCode) {
        $params[] = $classroomCodeForWrite;
    }
    $params[] = $id;
    $params[] = $collegeId;
    $st->execute($params);
} elseif ($hasLabFlag && $hasCourseBlock) {
    $st = db()->prepare('UPDATE courses SET course_code=?, course_name=?, units=?, is_laboratory=?, year_level=?, section=?,
department=?' . $ccSet . ' WHERE id=? AND college_id=?');
    $params = [$code, $name, $legacyUnits, $isLab, $yearLevel, $section, $program];
    if ($hasClassroomCode) {
        $params[] = $classroomCodeForWrite;
    }
    $params[] = $id;
    $params[] = $collegeId;
    $st->execute($params);
} elseif ($hasLabFlag) {
    $st = db()->prepare('UPDATE courses SET course_code=?, course_name=?, units=?, is_laboratory=?, department=?' . $ccSet . '
WHERE id=? AND college_id=?');
    $params = [$code, $name, $legacyUnits, $isLab, $program];
    if ($hasClassroomCode) {
        $params[] = $classroomCodeForWrite;
    }
    $params[] = $id;
    $params[] = $collegeId;
    $st->execute($params);
} elseif ($hasCourseBlock) {
    $st = db()->prepare('UPDATE courses SET course_code=?, course_name=?, units=?, year_level=?, section=?, department=?' .
$ccSet . ' WHERE id=? AND college_id=?');
    $params = [$code, $name, $legacyUnits, $yearLevel, $section, $program];
    if ($hasClassroomCode) {
        $params[] = $classroomCodeForWrite;
    }
    $params[] = $id;
    $params[] = $collegeId;
    $st->execute($params);
} else {
    $st = db()->prepare('UPDATE courses SET course_code=?, course_name=?, units=?, department=?' . $ccSet . ' WHERE id=? AND
college_id=?');
    $params = [$code, $name, $legacyUnits, $program];
    if ($hasClassroomCode) {
        $params[] = $classroomCodeForWrite;
    }
    $params[] = $id;
    $params[] = $collegeId;
    $st->execute($params);
}
$stAfterC = db()->prepare('SELECT * FROM courses WHERE id = ? LIMIT 1');
$stAfterC->execute([$id]);
$afterCourse = $stAfterC->fetch(PDO::FETCH_ASSOC);
log_user_activity('edit', 'Courses', 'Course #' . $id, $beforeCourse ? (array) $beforeCourse : null, $afterCourse ? (array)
$afterCourse : null);
log_dean_activity('course_update', 'Updated course ID #' . $id);
$_SESSION['flash'] = 'Course updated.';
if ($allocatedCourseClassroomCodeOnEdit !== null) {
    $_SESSION['flash'] .= ' Classroom code for this subject: ' . $allocatedCourseClassroomCodeOnEdit . ' .';
}
} elseif ($action === 'delete' && isset($_POST['id'])) {
    $delId = (int) $_POST['id'];
    $sqlSel = 'SELECT * FROM courses WHERE id=? AND college_id=?';
    $parSel = [$delId, $collegeId];
    if ($programScope !== null) {
        $sqlSel .= ' AND department=?';
        $parSel[] = $programScope;
    }
    $stSelDel = db()->prepare($sqlSel);
    $stSelDel->execute($parSel);
    $beforeDel = $stSelDel->fetch(PDO::FETCH_ASSOC);
    $sql = 'DELETE FROM courses WHERE id=? AND college_id=?';
    $params = [$delId, $collegeId];
    if ($programScope !== null) {

```

```

        $sql .= ' AND department=?';
        $params[] = $programScope;
    }
    $st = db()->prepare($sql);
    $st->execute($params);
    log_user_activity('delete', 'Courses', 'Course #' . $delId, $beforeDel ? (array) $beforeDel : null, null);
    log_dean_activity('course_delete', 'Deleted course ID #' . $delId);
    $_SESSION['flash'] = 'Course removed.';
} elseif ($action === 'delete_bulk') {
    $ids = isset($_POST['ids']) && is_array($_POST['ids'])
        ? array_values(array_unique(array_filter(array_map('intval', $_POST['ids']), static fn (int $id): bool => $id > 0)))
        : [];
    if ($ids === []) {
        throw new RuntimeException('Select at least one course to delete.');
```

```

    }
    $deleted = 0;
    foreach ($ids as $delId) {
        $sqlSel = 'SELECT * FROM courses WHERE id=? AND college_id=?';
        $parSel = [$delId, $collegeId];
        if ($programScope !== null) {
            $sqlSel .= ' AND department=?';
            $parSel[] = $programScope;
        }
        $stSelDel = db()->prepare($sqlSel);
        $stSelDel->execute($parSel);
        $beforeDel = $stSelDel->fetch(PDO::FETCH_ASSOC);
        if (!$beforeDel) {
            continue;
        }
        $sql = 'DELETE FROM courses WHERE id=? AND college_id=?';
        $params = [$delId, $collegeId];
        if ($programScope !== null) {
            $sql .= ' AND department=?';
            $params[] = $programScope;
        }
        db()->prepare($sql)->execute($params);
        log_user_activity('delete', 'Courses', 'Course #' . $delId, (array) $beforeDel, null);
        log_dean_activity('course_delete', 'Deleted course ID #' . $delId);
        $deleted++;
    }
    if ($deleted === 0) {
        throw new RuntimeException('No matching courses could be deleted.');
```

```

    }
    $_SESSION['flash'] = $deleted === 1 ? 'Course removed.' : $deleted . ' courses removed.';
}
} catch (Throwable $e) {
    $msg = $e->getMessage();
    if ($e instanceof PDOException && str_contains($msg, '1062') && str_contains(strtolower($msg), 'course_code')) {
        // Retry auto-heal message if DB user cannot alter schema.
        $msg = 'Your database still blocks duplicate course codes (legacy UNIQUE index). Please run upgrade_roles.php once.';
    }
    $_SESSION['flash'] = 'Error: ' . $msg;
}
$redirectUrl = 'courses.php';
if ($action === 'delete_bulk') {
    $redirectQuery = [];
    $returnQ = trim((string) ($_POST['return_q'] ?? ''));
    if ($returnQ !== '') {
        $redirectQuery['q'] = $returnQ;
    }
    $returnSort = trim((string) ($_POST['return_sort'] ?? ''));
    if ($returnSort !== '') {
        $redirectQuery['sort'] = $returnSort;
    }
    $returnDir = strtolower(trim((string) ($_POST['return_dir'] ?? '')));
    if ($returnDir === 'asc' || $returnDir === 'desc') {
        $redirectQuery['dir'] = $returnDir;
    }
    if ($redirectQuery !== []) {
        $redirectUrl .= '?' . http_build_query($redirectQuery);
    }
}
header('Location: ' . $redirectUrl);
exit;
}

$editRow = null;
if (isset($_GET['edit'])) {
    $sql = 'SELECT * FROM courses WHERE id=? AND college_id=?';
    $params = [(int) $_GET['edit'], $collegeId];

```

```

    if ($programScope != null) {
        $sql .= ' AND department=?';
        $params[] = $programScope;
    }
    $st = db()->prepare($sql);
    $st->execute($params);
    $editRow = $st->fetch() ?: null;
}

$search = trim((string) ($_GET['q'] ?? ''));
$sort = (string) ($_GET['sort'] ?? 'course_code');
$dir = strtolower((string) ($_GET['dir'] ?? 'asc')) === 'desc' ? 'DESC' : 'ASC';

$sortMap = [
    'course_code' => 'course_code',
    'course_name' => 'course_name',
    'program' => 'department',
    'lecture_units' => $hasLectureUnits ? 'lecture_units' : 'units',
    'laboratory_units' => $hasLaboratoryUnits ? 'laboratory_units' : 'units',
];
if ($hasYearLevel) {
    $sortMap['year_level'] = 'year_level';
}
if ($hasSection) {
    $sortMap['section'] = 'section';
}
if ($hasLabFlag) {
    $sortMap['type'] = 'is_laboratory';
}
if ($hasClassroomCode) {
    $sortMap['classroom_code'] = 'classroom_code';
}

$sortExpr = $sortMap[$sort] ?? 'course_code';

$programOptions = [];
if ($hasProgramsTable) {
    $sql = 'SELECT program_name, status FROM programs WHERE college_id=?';
    $params = [$collegeId];
    if ($programScope != null) {
        $sql .= ' AND program_name=?';
        $params[] = $programScope;
    }
    $sql .= ' ORDER BY (status = "active") DESC, program_name ASC';
    $st = db()->prepare($sql);
    $st->execute($params);
    $programOptions = $st->fetchAll();
}

$sql = 'SELECT * FROM courses WHERE college_id=?';
$params = [$collegeId];
if ($programScope != null) {
    $sql .= ' AND department=?';
    $params[] = $programScope;
}
if ($search != '') {
    $sql .= ' AND (course_code LIKE ? OR course_name LIKE ? OR department LIKE ?';
    $params[] = '%' . $search . '%';
    $params[] = '%' . $search . '%';
    $params[] = '%' . $search . '%';
    if ($hasYearLevel) {
        $sql .= ' OR year_level LIKE ?';
        $params[] = '%' . $search . '%';
    }
    if ($hasSection) {
        $sql .= ' OR section LIKE ?';
        $params[] = '%' . $search . '%';
    }
    if ($hasClassroomCode) {
        $sql .= ' OR classroom_code LIKE ?';
        $params[] = '%' . $search . '%';
    }
    $sql .= ' OR EXISTS (
        SELECT 1
        FROM schedules s
        INNER JOIN faculty f ON f.id = s.faculty_id
        WHERE s.course_id = courses.id
            AND (f.full_name LIKE ? OR f.faculty_id LIKE ?)
    )';
    $params[] = '%' . $search . '%';
}

```

```

$params[] = '%' . $search . '%';
$sql .= ')';
}
$sql .= " ORDER BY {$sortExpr} {$dir}, course_code ASC";
$stmt = db()->prepare($sql);
$stmt->execute($params);
$list = $stmt->fetchAll();

$geOfferings = [];
if (is_dean() && $programScope === null && db_column_exists('courses', 'is_gened') && db_table_exists('ge_course_colleges')) {
    $geOfferings = ge_courses_offered_to_college($collegeId);
}

$nextDirFor = static function (string $col, string $currentSort, string $currentDir): string {
    if ($col !== $currentSort) {
        return 'asc';
    }
    return strtoupper($currentDir) === 'ASC' ? 'desc' : 'asc';
};
$sortArrowFor = static function (string $col, string $currentSort, string $currentDir): string {
    if ($col !== $currentSort) {
        return '';
    }
    return strtoupper($currentDir) === 'ASC' ? '▲' : '▼';
};

$pageTitle = 'Courses';
require_once __DIR__ . '/includes/header.php';
?>
<style>
    .courses-page {
        min-width: 0;
        max-width: 100%;
    }

    .courses-page .app-page-title {
        font-size: 1.5rem;
    }

    .courses-page .app-card-header,
    .courses-page .app-table thead th {
        font-size: 0.875rem;
    }

    .courses-page .app-table thead th {
        letter-spacing: 0.03em;
    }

    .courses-page .app-table tbody td {
        font-size: 0.9rem;
    }

    .courses-page .app-table .app-table-actions .btn {
        font-size: 0.8rem;
    }

    .courses-page .courses-table-wrap {
        overflow-x: auto;
        -webkit-overflow-scrolling: touch;
        max-width: 100%;
    }

    .courses-page .courses-table-wrap .app-table {
        width: 100%;
        min-width: 1020px;
        table-layout: auto;
    }

    .courses-page .courses-table-wrap .app-table th,
    .courses-page .courses-table-wrap .app-table td {
        vertical-align: middle;
    }

    .courses-page .courses-col-code,
    .courses-page .courses-col-classroom,
    .courses-page .courses-col-units,
    .courses-page .courses-col-type,
    .courses-page .courses-col-year,
    .courses-page .courses-col-section,
    .courses-page .courses-col-actions {

```



```

    white-space: nowrap;
}

.courses-page .courses-col-code {
    min-width: 5.5rem;
}

.courses-page .courses-col-classroom {
    min-width: 7.5rem;
}

.courses-page .courses-col-title {
    min-width: 10rem;
    max-width: 22rem;
    white-space: normal;
    word-break: break-word;
    overflow-wrap: anywhere;
}

.courses-page .courses-col-units {
    min-width: 6.25rem;
    text-align: center;
}

.courses-page .courses-table-wrap .app-table thead th.courses-col-units {
    white-space: normal;
    line-height: 1.25;
}

.courses-page .courses-col-type {
    min-width: 5.5rem;
}

.courses-page .courses-col-year {
    min-width: 3.25rem;
    text-align: center;
}

.courses-page .courses-col-section {
    min-width: 4.5rem;
}

.courses-page .courses-col-program {
    min-width: 8rem;
    max-width: 18rem;
    white-space: normal;
    word-break: break-word;
    overflow-wrap: anywhere;
}

.courses-page .courses-col-actions {
    min-width: 6.5rem;
}

.courses-page .courses-col-select {
    width: 2.75rem;
    min-width: 2.75rem;
    text-align: center;
    vertical-align: middle;
}

.courses-page .courses-col-select .form-check-input {
    float: none;
    margin: 0;
}

.courses-page .courses-form-scroll {
    overflow-x: auto;
    -webkit-overflow-scrolling: touch;
    max-width: 100%;
}

.courses-page .courses-form-scroll > form.row {
    min-width: min(100%, 36rem);
}

@media (min-width: 768px) {
    .courses-page .courses-form-scroll > form.row {
        min-width: 52rem;
    }
}

```

```

}

.courses-page .courses-form-scroll .form-label {
    white-space: normal;
    line-height: 1.35;
}

.courses-page .courses-form-scroll [class*="col-"] {
    min-width: 0;
}

.courses-page .courses-form-scroll .form-control,
.courses-page .courses-form-scroll .form-select {
    width: 100%;
    min-width: 0;
}

@media (max-width: 767.98px) {
    .courses-page .app-page-title {
        font-size: 1.5rem;
        line-height: 1.25;
    }

    .courses-page .app-page-lead {
        font-size: 0.875rem;
    }

    .courses-page .app-card-body {
        padding: 1rem;
    }

    .courses-page .app-form-actions .btn {
        width: 100%;
    }
}
</style>
<div class="courses-page">
<header class="app-page-header">
    <h1 class="app-page-title"><i class="fa-solid fa-book me-2 app-page-title-icon" aria-hidden="true"></i>Courses</h1>
    <p class="app-page-lead">Maintain the course catalog for scheduling and online classrooms. Subject classroom codes help students join
the right class with their instructor.</p>
    <div class="app-meta-row" role="group" aria-label="Scope">
        <span class="app-chip"><i class="fa-solid fa-building-columns text-primary" aria-hidden="true"></i><?>
htmlspecialchars($collegeName) </span>
        <?php if ($programScope !== null): ?>
            <span class="app-chip"><i class="fa-solid fa-graduation-cap text-primary" aria-hidden="true"></i><?>
htmlspecialchars($programScope) </span>
        <?php endif; ?>
    </div>
</header>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
<?php if (!$hasLabFlag || !$hasLectureUnits || !$hasLaboratoryUnits): ?>
    <div class="alert alert-warning app-alert">Split unit fields are disabled until you run <a
href="upgrade_roles.php">upgrade_roles.php</a>.</div>
<?php endif; ?>
<?php if (!$hasCourseBlock): ?>
    <div class="alert alert-warning app-alert">Year level/section fields are disabled until you run <a
href="upgrade_roles.php">upgrade_roles.php</a>.</div>
<?php endif; ?>
<?php if (!$hasProgramsTable): ?>
    <div class="alert alert-warning app-alert">Program dropdown is disabled until you run <a href="upgrade_roles.php">upgrade_roles.php</a>.
</div>
<?php endif; ?>
<?php if ($programScope !== null && !$hasClassroomCode): ?>
    <div class="alert alert-warning app-alert">Subject classroom codes are not available until you run <a
href="upgrade_roles.php">upgrade_roles.php</a> once.</div>
<?php endif; ?>

<div class="app-card">
    <div class="app-card-header">Find and sort</div>
    <div class="app-card-body">
        <form method="get" class="row g-3 align-items-end">
            <div class="col-md-5">
                <label class="form-label" for="courses-q">Search</label>
                <input id="courses-q" type="search" name="q" class="form-control" placeholder="Code, course title, program, year, section,
faculty<?> $hasClassroomCode ? ', classroom code' : '' ?>" value="<?> htmlspecialchars($search) ?>" autocomplete="off">
            </div>
            <div class="col-md-3">
                <label class="form-label" for="courses-sort">Sort by</label>

```

```

        <select id="courses-sort" name="sort" class="form-select">
            <option value="course_code" <?= $sort === 'course_code' ? 'selected' : '' ?>>Code</option>
            <?php if ($hasClassroomCode): ?><option value="classroom_code" <?= $sort === 'classroom_code' ? 'selected' : '' ?>>Classroom
code</option><?php endif; ?>
            <option value="course_name" <?= $sort === 'course_name' ? 'selected' : '' ?>>Course Title</option>
            <option value="lecture_units" <?= $sort === 'lecture_units' ? 'selected' : '' ?>>Lecture units</option>
            <option value="laboratory_units" <?= $sort === 'laboratory_units' ? 'selected' : '' ?>>Laboratory units</option>
            <option value="program" <?= $sort === 'program' ? 'selected' : '' ?>>Program</option>
            <?php if ($hasYearLevel): ?><option value="year_level" <?= $sort === 'year_level' ? 'selected' : '' ?>>Year level</option><?
php endif; ?>
            <?php if ($hasSection): ?><option value="section" <?= $sort === 'section' ? 'selected' : '' ?>>Section</option><?php endif;
?>
            <?php if ($hasLabFlag): ?><option value="type" <?= $sort === 'type' ? 'selected' : '' ?>>Type</option><?php endif; ?>
        </select>
    </div>
    <div class="col-md-2">
        <label class="form-label" for="courses-dir">Direction</label>
        <select id="courses-dir" name="dir" class="form-select">
            <option value="asc" <?= $dir === 'ASC' ? 'selected' : '' ?>>Ascending</option>
            <option value="desc" <?= $dir === 'DESC' ? 'selected' : '' ?>>Descending</option>
        </select>
    </div>
    <div class="col-md-2 d-grid">
        <button type="submit" class="btn btn-primary rounded-pill"<?= app_tooltip_attr('Applies search, sort, and direction to the
course list. Use this after changing how you want the catalog ordered.') ?>>Apply</button>
    </div>
</form>
</div>
</div>

<div class="app-card">
    <div class="app-card-header"><?= $editRow ? 'Edit course' : 'Add course' ?></div>
    <div class="app-card-body courses-form-scroll">
        <form method="post" class="row g-3">
            <input type="hidden" name="action" value="<?= $editRow ? 'edit' : 'add' ?>">
            <?php if ($editRow): ?><input type="hidden" name="id" value="<?= (int) $editRow['id'] ?>"><?php endif; ?>
            <div class="col-md-2"><label class="form-label">Code</label><input name="course_code" class="form-control" required maxlength="20"
value="<?= htmlspecialchars((string) ($editRow['course_code'] ?? '')) ?>"></div>
            <div class="col-md-4"><label class="form-label">Course Title</label><input name="course_name" class="form-control" required
maxlength="100" value="<?= htmlspecialchars((string) ($editRow['course_name'] ?? '')) ?>"></div>
            <?php if ($hasLectureUnits): ?>
                <div class="col-md-2"><label class="form-label">Lecture Units</label><input name="lecture_units" type="number" step="0.1"
min="0" max="12" class="form-control" value="<?= htmlspecialchars((string) ($editRow['lecture_units'] ?? $editRow['units'] ?? '3.0')) ?>">
            </div>
            <?php else: ?>
                <div class="col-md-2"><label class="form-label">Units</label><input name="units" type="number" step="0.1" min="0.5" max="12"
class="form-control" value="<?= htmlspecialchars((string) ($editRow['units'] ?? '3.0')) ?>"></div>
            <?php endif; ?>
            <?php if ($hasLaboratoryUnits): ?>
                <div class="col-md-2"><label class="form-label">Laboratory Units</label><input name="laboratory_units" type="number" step="0.1"
min="0" max="12" class="form-control" value="<?= htmlspecialchars((string) ($editRow['laboratory_units'] ?? '0.0')) ?>"></div>
            <?php endif; ?>
            <div class="col-md-2 d-flex align-items-end"><?php if ($hasLabFlag): ?><div class="form-check"><input class="form-check-input"
type="checkbox" name="is_laboratory" value="1" <?= !empty($editRow['is_laboratory']) ? 'checked' : '' ?>><label class="form-check-
label">Laboratory course</label></div><?php endif; ?></div>
            <?php if ($hasYearLevel): ?>
                <div class="col-md-2"><label class="form-label">Year Level</label><select name="year_level" class="form-select" required><?php
foreach ([ '1', '2', '3', '4', '5' ] as $y1): ?><option value="<?= $y1 ?>" <?= (string) ($editRow['year_level'] ?? '') === $y1 ? 'selected' : '' ?
>><?= htmlspecialchars($y1) ?></option><?php endforeach; ?></select></div>
            <?php endif; ?>
            <?php if ($hasSection): ?>
                <div class="col-md-2"><label class="form-label">Section</label><input name="section" class="form-control" maxlength="20"
required value="<?= htmlspecialchars((string) ($editRow['section'] ?? '')) ?>"></div>
            <?php endif; ?>
            <?php if ($hasProgramsTable && $programScope === null): ?>
                <?php
                $selectedProgram = (string) ($editRow['department'] ?? '');
                $programValues = array_map(static fn ($p) => (string) ($p['program_name'] ?? ''), $programOptions);
                ?>
                <div class="col-md-4">
                    <label class="form-label">Program</label>
                    <select name="program" class="form-select" required>
                        <option value="">Select program</option>
                        <?php if ($selectedProgram !== '' && !in_array($selectedProgram, $programValues, true)): ?>
                            <option value="<?= htmlspecialchars($selectedProgram) ?>" selected><?= htmlspecialchars($selectedProgram) ?>
(legacy)</option>
                        <?php endif; ?>
                        <?php foreach ($programOptions as $programRow): ?>
                            <?php
                            $programName = (string) ($programRow['program_name'] ?? '');

```

```

        $programStatus = (string) ($programRow['status'] ?? 'active');
        $isSelected = $selectedProgram === $programName;
        ?>
        <option value="<?= htmlspecialchars($programName) ?>" <?= $isSelected ? 'selected' : '' ?>
            <?= htmlspecialchars($programName) ?><?= $programStatus === 'inactive' ? ' (inactive)' : '' ?>
        </option>
    </select>
</div>
<?php elseif ($programScope !== null): ?>
    <div class="col-md-4">
        <label class="form-label">Program</label>
        <input class="form-control" value="<?= htmlspecialchars($programScope) ?>" readonly>
        <input type="hidden" name="program" value="<?= htmlspecialchars($programScope) ?>">
    </div>
    <?php else: ?>
        <div class="col-md-4"><label class="form-label">Program</label><input name="program" class="form-control" placeholder="e.g. BS
Social Work, BS Information Systems" value="<?= htmlspecialchars((string) ($editRow['department'] ?? '')) ?>"></div>
    <?php endif; ?>
    <?php if ($hasClassroomCode): ?>
        <div class="col-md-3">
            <label class="form-label">Subject classroom code</label>
            <?php if ($editRow): ?>
                <?php $ccVal = trim((string) ($editRow['classroom_code'] ?? '')); ?>
                <?php if ($ccVal !== ''): ?>
                    <input class="form-control font-monospace" readonly value="<?= htmlspecialchars($ccVal) ?>">
                <?php elseif ($programScope !== null): ?>
                    <input class="form-control" value="Assigned when you save" readonly tabIndex="-1" style="background:#f8f9fa">
                <?php else: ?>
                    <input class="form-control text-muted" value="-" readonly tabIndex="-1" style="background:#f8f9fa" title="Generated
when a program chair creates or saves the course">
                <?php endif; ?>
            <?php elseif ($programScope !== null): ?>
                <input class="form-control" value="Generated when you save" readonly tabIndex="-1" style="background:#f8f9fa">
            <?php endif; ?>
            <?php if ($programScope !== null): ?>
                <div class="form-text">Auto-generated for each subject; share with faculty or students as needed.</div>
            <?php endif; ?>
        </div>
    <?php endif; ?>
    <div class="col-12 app-form-actions">
        <button type="submit" class="btn btn-primary px-4 rounded-pill"<?= app_tooltip_attr($editRow ? 'Saves changes to this course
record. Use this after editing codes, units, or program assignment.' : 'Creates the new course in the catalog. Use this when adding a
subject that scheduling will reference.') ?>>Save</button><?php if ($editRow): ?> <a href="courses.php" class="btn btn-outline-secondary
rounded-pill"<?= app_tooltip_attr('Discards unsaved edits and returns to the catalog without saving. Use this if you opened the wrong
course.') ?>>Cancel</a><?php endif; ?>
    </div>
</form>
</div>
</div>

<section class="app-card mb-0" aria-labelledby="courses-catalog-heading">
    <div class="app-card-header d-flex flex-wrap justify-content-between align-items-center gap-2">
        <span id="courses-catalog-heading">Course catalog</span>
        <div class="d-flex flex-wrap align-items-center gap-2">
            <?php if ($list !== []): ?>
                <form id="courses-bulk-delete-form" method="post" class="d-inline" onsubmit="return confirm('Delete the selected courses?
This cannot be undone.');">
                    <input type="hidden" name="action" value="delete_bulk">
                    <input type="hidden" name="return_q" value="<?= htmlspecialchars($search) ?>">
                    <input type="hidden" name="return_sort" value="<?= htmlspecialchars($sort) ?>">
                    <input type="hidden" name="return_dir" value="<?= $dir === 'DESC' ? 'desc' : 'asc' ?>">
                    <button type="submit" id="courses-bulk-delete-btn" class="btn btn-sm btn-outline-danger rounded-pill" disabled<?=
app_tooltip_attr('Deletes every checked course in the list. Select rows first, then use this to remove several at once.') ?>>
                        <i class="fa-solid fa-trash" aria-hidden="true"></i> Delete selected
                    </button>
                </form>
            <?php endif; ?>
            <span class="badge rounded-pill bg-light text-secondary border fw-normal"><?= count($list) ?> <?= count($list) === 1 ? 'course'
: 'courses' ?></span>
        </div>
    </div>
    <div class="table-responsive courses-table-wrap">
        <table class="table app-table mb-0">
            <thead><tr>
                <?php
                    $baseQuery = ['q' => $search];
                >
                <?php if ($list !== []): ?>
                    <th scope="col" class="courses-col-select">

```

```

        <input type="checkbox" class="form-check-input" id="courses-select-all" aria-label="Select all courses on this page">
    </th>
    <?php endif; ?>
    <th scope="col" class="courses-col-code"><a class="text-decoration-none text-body-secondary" href="?<?=  

htmlspecialchars(http_build_query($baseQuery + ['sort' => 'course_code', 'dir' => $nextDirFor('course_code', $sort, $dir)])) ?>">Code<?=  

htmlspecialchars($sortArrowFor('course_code', $sort, $dir)) ?></a></th>
    <?php if ($hasClassroomCode): ?>
        <th scope="col" class="courses-col-classroom"><a class="text-decoration-none text-body-secondary" href="?<?=  

htmlspecialchars(http_build_query($baseQuery + ['sort' => 'classroom_code', 'dir' => $nextDirFor('classroom_code', $sort, $dir)])) ?  

>">Classroom code<?=  

htmlspecialchars($sortArrowFor('classroom_code', $sort, $dir)) ?></a></th>
    <?php endif; ?>
    <th scope="col" class="courses-col-title"><a class="text-decoration-none text-body-secondary" href="?<?=  

htmlspecialchars(http_build_query($baseQuery + ['sort' => 'course_name', 'dir' => $nextDirFor('course_name', $sort, $dir)])) ?>">Course  

Title<?=  

htmlspecialchars($sortArrowFor('course_name', $sort, $dir)) ?></a></th>
    <th scope="col" class="courses-col-units"><a class="text-decoration-none text-body-secondary" href="?<?=  

htmlspecialchars(http_build_query($baseQuery + ['sort' => 'lecture_units', 'dir' => $nextDirFor('lecture_units', $sort, $dir)])) ?>">Lecture  

units<?=  

htmlspecialchars($sortArrowFor('lecture_units', $sort, $dir)) ?></a></th>
    <th scope="col" class="courses-col-units"><a class="text-decoration-none text-body-secondary" href="?<?=  

htmlspecialchars(http_build_query($baseQuery + ['sort' => 'laboratory_units', 'dir' => $nextDirFor('laboratory_units', $sort, $dir)])) ?  

>">Laboratory units<?=  

htmlspecialchars($sortArrowFor('laboratory_units', $sort, $dir)) ?></a></th>
    <?php if ($hasLabFlag): ?>
        <th scope="col" class="courses-col-type"><a class="text-decoration-none text-body-secondary" href="?<?=  

htmlspecialchars(http_build_query($baseQuery + ['sort' => 'type', 'dir' => $nextDirFor('type', $sort, $dir)])) ?>">Type<?=  

htmlspecialchars($sortArrowFor('type', $sort, $dir)) ?></a></th>
    <?php endif; ?>
    <?php if ($hasYearLevel): ?>
        <th scope="col" class="courses-col-year"><a class="text-decoration-none text-body-secondary" href="?<?=  

htmlspecialchars(http_build_query($baseQuery + ['sort' => 'year_level', 'dir' => $nextDirFor('year_level', $sort, $dir)])) ?>">Year<?=  

htmlspecialchars($sortArrowFor('year_level', $sort, $dir)) ?></a></th>
    <?php endif; ?>
    <?php if ($hasSection): ?>
        <th scope="col" class="courses-col-section"><a class="text-decoration-none text-body-secondary" href="?<?=  

htmlspecialchars(http_build_query($baseQuery + ['sort' => 'section', 'dir' => $nextDirFor('section', $sort, $dir)])) ?>">Section<?=  

htmlspecialchars($sortArrowFor('section', $sort, $dir)) ?></a></th>
    <?php endif; ?>
    <th scope="col" class="courses-col-program"><a class="text-decoration-none text-body-secondary" href="?<?=  

htmlspecialchars(http_build_query($baseQuery + ['sort' => 'program', 'dir' => $nextDirFor('program', $sort, $dir)])) ?>">Program<?=  

htmlspecialchars($sortArrowFor('program', $sort, $dir)) ?></a></th>
    <th scope="col" class="app-table-actions courses-col-actions"><span class="visually-hidden">Actions</span></th>
</tr></thead>
<tbody>
<?php foreach ($list as $r): ?>
    <tr>
        <td class="courses-col-select">
            <input type="checkbox" class="form-check-input courses-row-select" name="ids[]" form="courses-bulk-delete-form" value="  

<?=(int) $r['id'] ?>" aria-label="Select course <?=  

htmlspecialchars((string) $r['course_code']) ?>">
        </td>
        <td class="courses-col-code"><?=  

htmlspecialchars((string) $r['course_code']) ?></td>
        <?php if ($hasClassroomCode): ?>
            <td class="courses-col-classroom font-monospace"><?php $rcc = trim((string) ($r['classroom_code'] ?? '')); ?><?=  

$rcc  

!="" ?> <?=  

htmlspecialchars($rcc) : '-' ?></td>
        <?php endif; ?>
        <td class="courses-col-title"><?=  

htmlspecialchars((string) $r['course_name']) ?></td>
        <td class="courses-col-units"><?=  

htmlspecialchars((string) ($r['lecture_units'] ?? $r['units'])) ?></td>
        <td class="courses-col-units"><?=  

htmlspecialchars((string) ($r['laboratory_units'] ?? (empty($r['is_laboratory']) ?  

$r['units'] : 0))) ?></td>
        <?php if ($hasLabFlag): ?><td class="courses-col-type"><?=  

!empty($r['is_laboratory']) ? 'Laboratory' : 'Lecture' ?></td><?php  

endif; ?>
        <?php if ($hasYearLevel): ?><td class="courses-col-year"><?=  

htmlspecialchars((string) ($r['year_level'] ?? '')) ?></td><?php  

endif; ?>
        <?php if ($hasSection): ?><td class="courses-col-section"><?=  

htmlspecialchars((string) ($r['section'] ?? '')) ?></td><?php  

endif; ?>
        <td class="courses-col-program"><?=  

htmlspecialchars((string) $r['department']) ?></td>
        <td class="app-table-actions courses-col-actions text-nowrap"><a class="btn btn-sm btn-outline-primary rounded-pill"  

href="courses.php?edit=<?=(int) $r['id'] ?>" aria-label="Edit course"><?=  

app_tooltip_attr('Opens this course for editing. Use this to fix  

codes, units, program, or classroom metadata.') ?>><i class="fa-solid fa-pen-to-square" aria-hidden="true"></i><span class="visually-  

hidden">Edit</span></a>
            <form method="post" class="d-inline" onsubmit="return confirm('Delete this course?');"><input type="hidden"  

name="action" value="delete"><input type="hidden" name="id" value="<?=(int) $r['id'] ?>"><button type="submit" class="btn btn-sm btn-  

outline-danger rounded-pill" aria-label="Delete course"><?=  

app_tooltip_attr('Permanently removes this course after confirmation. Use this  

only if no schedules should reference it anymore.') ?>><i class="fa-solid fa-trash" aria-hidden="true"></i><span class="visually-  

hidden">Delete</span></button></form>
        </td>
    </tr>
<?php endforeach; ?>
</tbody>
</table>
</div>
</section>

```

```

<?php if (is_dean() && $programScope === null): ?>
<section class="app-card mt-4 mb-0" aria-labelledby="courses-ge-heading">
    <div class="app-card-header d-flex flex-wrap justify-content-between align-items-center gap-2">
        <span id="courses-ge-heading">GE courses (college offerings)</span>
        <span class="badge rounded-pill bg-light text-secondary border fw-normal"><? count($geOfferings) ?> <? count($geOfferings) === 1 ?
'course' : 'courses' ?></span>
    </div>
    <div class="app-card-body border-bottom">
        <p class="small text-muted mb-0">General Education subjects assigned to your college for scheduling and conflict monitoring.
Official GE timetables are built under <strong>GEN ED → GE schedule</strong>; use <strong>Weekly view</strong> to watch live GE blocks
alongside your college schedules.</p>
    </div>
    <div class="table-responsive courses-table-wrap">
        <table class="table app-table mb-0">
            <thead>
                <tr>
                    <th scope="col" class="courses-col-code">Code</th>
                    <th scope="col" class="courses-col-title">Name</th>
                    <th scope="col" class="courses-col-units">Lecture units</th>
                    <th scope="col" class="courses-col-units">Laboratory units</th>
                    <?php if ($hasLabFlag): ?><th scope="col" class="courses-col-type">Type</th><?php endif; ?>
                    <th scope="col" class="courses-col-program">Program</th>
                </tr>
            </thead>
            <tbody>
                <?php if ($geOfferings === []): ?>
                    <tr>
                        <td colspan="<? $hasLabFlag ? 6 : 5 ?>" class="text-center text-muted py-4">No GE courses are assigned to this college
yet. Ask the GEN ED coordinator to configure <strong>GE Offerings</strong>.</td>
                    </tr>
                <?php else: ?>
                    <?php foreach ($geOfferings as $ge): ?>
                        <tr>
                            <td class="courses-col-code"><? htmlspecialchars((string) $ge['course_code']) ?></td>
                            <td class="courses-col-title"><? htmlspecialchars((string) $ge['course_name']) ?></td>
                            <td class="courses-col-units"><? htmlspecialchars((string) ($ge['lecture_units'] ?? $ge['units'] ?? '')) ?></td>
                            <td class="courses-col-units"><? htmlspecialchars((string) ($ge['laboratory_units'] ??
(!empty($ge['is_laboratory'])) ? ($ge['units'] ?? '') : 0)) ?></td>
                            <?php if ($hasLabFlag): ?><td class="courses-col-type"><? !empty($ge['is_laboratory']) ? 'Laboratory' : 'Lecture' ?
></td><?php endif; ?>
                            <td class="courses-col-program"><? htmlspecialchars((string) ($ge['department'] ?? 'General Education')) ?></td>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</section>
<?php endif; ?>

</div>
<script>
(function () {
    var selectAll = document.getElementById('courses-select-all');
    var deleteBtn = document.getElementById('courses-bulk-delete-btn');
    if (!selectAll || !deleteBtn) {
        return;
    }

    var rowChecks = function () {
        return Array.prototype.slice.call(document.querySelectorAll('.courses-row-select'));
    };

    var syncBulkDelete = function () {
        var checks = rowChecks();
        var selected = checks.filter(function (el) { return el.checked; });
        deleteBtn.disabled = selected.length === 0;
        selectAll.indeterminate = selected.length > 0 && selected.length < checks.length;
        selectAll.checked = checks.length > 0 && selected.length === checks.length;
    };

    selectAll.addEventListener('change', function () {
        var checked = selectAll.checked;
        rowChecks().forEach(function (el) {
            el.checked = checked;
        });
        syncBulkDelete();
    });

    rowChecks().forEach(function (el) {

```

```

        el.addEventListener('change', syncBulkDelete);
    });

    syncBulkDelete();
})();
</script>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: dashboard.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_login();

$role = (string) ($_SESSION['role'] ?? '');
$collegeId = current_college_id();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
$today = date('l');
$studentSelfId = 0;
$studentScheduleClasses = [];
$studentScheduleByDay = [];
$hasStudentLiveAt = db_column_exists('schedules', 'online_live_at');
$isAdmin = $role === 'admin';
$stat1Label = 'Active faculty';
$stat2Label = 'Courses';
$stat3Label = 'Schedules';
$stat4Label = 'Open conflicts';
$superStats = [
    'totalSchedulesAllTime' => 0,
    'collegesDepartments' => 0,
    'facultyMembers' => 0,
    'studentEnrollments' => 0,
    'inactivePendingInvites' => 0,
    'activeConflictAlerts' => 0,
    'resolvedConflicts30d' => 0,
    'roomDoubleBookingRisks' => 0,
    'facultyTimeOverlap' => 0,
    'scheduledClassesCurrentTerm' => 0,
    'unassignedTimeSlots' => 0,
    'superAdminLastLogin' => 'N/A',
];

if ($isAdmin) {
    $activeFaculty = (int) db()->query("SELECT COUNT(*) FROM faculty WHERE status = 'active' AND college_id IS NOT NULL")->fetchColumn();
    $totalCourses = (int) db()->query('SELECT COUNT(*) FROM courses WHERE college_id IS NOT NULL')->fetchColumn();
    $totalSchedules = (int) db()->query('SELECT COUNT(*) FROM schedules WHERE college_id IS NOT NULL')->fetchColumn();
    $openConflicts = db_table_exists('conflict_requests')
        ? (int) db()->query("SELECT COUNT(*) FROM conflict_requests WHERE status = 'pending'")->fetchColumn()
        : 0;
} elseif (($role === 'dean' || $role === 'program_chair') && $collegeId) {
    $facultySql = "SELECT COUNT(*) FROM faculty WHERE status='active' AND college_id=?";
    $courseSql = "SELECT COUNT(*) FROM courses WHERE college_id=?";
    $scheduleSql = "SELECT COUNT(*) FROM schedules s INNER JOIN courses c ON c.id = s.course_id WHERE s.college_id=?";
    $params = [$collegeId];
    if ($programScope !== null) {
        $facultySql .= " AND department=?";
        $courseSql .= " AND department=?";
        $scheduleSql .= " AND c.department=?";
        $params[] = $programScope;
    }
    $st = db()->prepare($facultySql);
    $st->execute($params);
    $activeFaculty = (int) $st->fetchColumn();
    $st = db()->prepare($courseSql);
    $st->execute($params);
    $totalCourses = (int) $st->fetchColumn();
    $st = db()->prepare($scheduleSql);
    $st->execute($params);
    $totalSchedules = (int) $st->fetchColumn();
    if ($role === 'program_chair') {
        $openConflicts = 0;
    } elseif (db_table_exists('schedule_change_requests')) {
        $st = db()->prepare(
            "SELECT COUNT(*) FROM schedule_change_requests scr

```

```

        INNER JOIN schedules s ON s.id = scr.schedule_id
        WHERE s.college_id=? AND scr.status='pending'"
    );
    $st->execute([$collegeId]);
    $openConflicts = (int) $st->fetchColumn();
} else {
    $openConflicts = 0;
}
} elseif ($role === 'gened') {
    $activeFaculty = (int) db()->query("SELECT COUNT(*) FROM faculty WHERE status='active' AND is_gened=1")->fetchColumn();
    $totalCourses = (int) db()->query("SELECT COUNT(*) FROM courses WHERE is_gened=1")->fetchColumn();
    $totalSchedules = (int) db()->query(
        "SELECT COUNT(*) FROM schedules s INNER JOIN courses c ON c.id=s.course_id WHERE c.is_gened=1"
    )->fetchColumn();
    $openConflicts = 0;
} elseif ($role === 'student') {
    $studentSelfId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
    if ($studentSelfId < 1) {
        $studentSelfId = resolve_student_id_for_user((int) ($_SESSION['user_id'] ?? 0)) ?? 0;
        $_SESSION['student_id'] = $studentSelfId > 0 ? $studentSelfId : null;
    }

    $activeFaculty = 0;
    $totalCourses = 0;
    $totalSchedules = 0;
    $openConflicts = 0;
    $stat1Label = 'My classes';
    $stat2Label = 'Assessments';
    $stat3Label = 'Announcements';
    $stat4Label = 'Pending work';

    if ($studentSelfId > 0 && db_table_exists('classroom_enrollments')) {
        $st = db()->prepare('SELECT COUNT(*) FROM classroom_enrollments WHERE student_id = ?');
        $st->execute([$studentSelfId]);
        $activeFaculty = (int) $st->fetchColumn();

        if (db_table_exists('classroom_assessments')) {
            $st = db()->prepare(
                'SELECT COUNT(*)
                FROM classroom_assessments ca
                INNER JOIN classroom_enrollments ce ON ce.classroom_id = ca.classroom_id
                WHERE ce.student_id = ?'
            );
            $st->execute([$studentSelfId]);
            $totalCourses = (int) $st->fetchColumn();
        }

        if (db_table_exists('classroom_content')) {
            $st = db()->prepare(
                'SELECT COUNT(*)
                FROM classroom_content cc
                INNER JOIN classroom_enrollments ce ON ce.classroom_id = cc.classroom_id
                WHERE ce.student_id = ? AND cc.content_type = "announcement"'
            );
            $st->execute([$studentSelfId]);
            $totalSchedules = (int) $st->fetchColumn();
        }
    }

    if (db_table_exists('classroom_submissions') && db_table_exists('classroom_assessments')) {
        $st = db()->prepare(
            'SELECT COUNT(*)
            FROM classroom_assessments ca
            INNER JOIN classroom_enrollments ce ON ce.classroom_id = ca.classroom_id
            LEFT JOIN classroom_submissions sub
                ON sub.assessment_id = ca.id AND sub.student_id = ce.student_id
            WHERE ce.student_id = ? AND sub.id IS NULL'
        );
        $st->execute([$studentSelfId]);
        $openConflicts = (int) $st->fetchColumn();
    }

    if (db_table_exists('online_classrooms') && db_table_exists('schedules')) {
        $st = db()->prepare(
            'SELECT oc.id AS classroom_id, oc.title, oc.meet_link,
                c.course_code, c.course_name, c.id AS course_id,
                f.full_name AS faculty_name,
                s.semester, s.school_year, s.day_of_week, s.start_time, s.end_time,
                s.online_live_at, r.room_code
            FROM classroom_enrollments ce
            INNER JOIN online_classrooms oc ON oc.id = ce.classroom_id'
        );
    }
}

```



```

        INNER JOIN schedules s ON s.id = oc.schedule_id
        INNER JOIN courses c ON c.id = oc.course_id
        INNER JOIN faculty f ON f.id = oc.faculty_id
        INNER JOIN rooms r ON r.id = s.room_id
        WHERE ce.student_id = ?
        ORDER BY s.start_time, c.course_code'
    );
    $st->execute([$studentSelfId]);
    $studentScheduleClasses = $st->fetchAll();
    foreach (schedule_days_list() as $dayName) {
        $studentScheduleByDay[$dayName] = [];
    }
    foreach ($studentScheduleClasses as $schedRow) {
        foreach (parse_day_set((string) ($schedRow['day_of_week'] ?? '')) as $dayName) {
            if (isset($studentScheduleByDay[$dayName])) {
                $studentScheduleByDay[$dayName][] = $schedRow;
            }
        }
    }
    foreach ($studentScheduleByDay as $dayName => $dayList) {
        usort($studentScheduleByDay[$dayName], static function (array $a, array $b): int {
            return strcmp((string) $a['start_time'], (string) $b['start_time']);
        });
    }
}
}
} elseif ($role === 'super_admin') {
    $activeFaculty = (int) db()->query("SELECT COUNT(*) FROM users WHERE role = 'admin'")->fetchColumn();
    $totalCourses = (int) db()->query("SELECT COUNT(*) FROM users WHERE role = 'admin' AND is_active = 1")->fetchColumn();
    $totalSchedules = (int) db()->query("SELECT COUNT(*) FROM users WHERE role = 'admin' AND is_active = 0")->fetchColumn();
    $openConflicts = (int) db()->query("SELECT COUNT(*) FROM users WHERE role = 'super_admin'")->fetchColumn();
    $stat1Label = 'Administrator accounts';
    $stat2Label = 'Active admins';
    $stat3Label = 'Disabled admins';
    $stat4Label = 'Super Admin users';

    $superStats['totalSchedulesAllTime'] = db_table_exists('schedules')
        ? (int) db()->query("SELECT COUNT(*) FROM schedules")->fetchColumn()
        : 0;
    $superStats['facultyMembers'] = db_table_exists('faculty')
        ? (int) db()->query("SELECT COUNT(*) FROM faculty")->fetchColumn()
        : 0;
    $superStats['collegesDepartments'] = db_table_exists('colleges')
        ? (int) db()->query("SELECT COUNT(*) FROM colleges")->fetchColumn()
        : 0;
    $superStats['studentEnrollments'] = db_table_exists('classroom_enrollments')
        ? (int) db()->query("SELECT COUNT(*) FROM classroom_enrollments")->fetchColumn()
        : 0;
    $superStats['inactivePendingInvites'] = db_table_exists('users')
        ? (int) db()->query("SELECT COUNT(*) FROM users WHERE role = 'admin' AND is_active = 0")->fetchColumn()
        : 0;
    $superStats['activeConflictAlerts'] = db_table_exists('conflict_requests')
        ? (int) db()->query("SELECT COUNT(*) FROM conflict_requests WHERE status = 'pending'")->fetchColumn()
        : 0;
    if (db_table_exists('conflict_requests')) {
        $resolvedDateColumn = db_first_existing_column(
            'conflict_requests',
            ['updated_at', 'reviewed_at', 'created_at'],
            'created_at'
        );
        $superStats['resolvedConflicts30d'] = (int) db()->query(
            "SELECT COUNT(*) FROM conflict_requests WHERE status = 'resolved' AND {$resolvedDateColumn} >= (NOW() - INTERVAL 30 DAY)"
        )->fetchColumn();
    } else {
        $superStats['resolvedConflicts30d'] = 0;
    }
    if (db_table_exists('conflict_requests') && db_column_exists('conflict_requests', 'conflict_type')) {
        $superStats['roomDoubleBookingRisks'] = (int) db()->query(
            "SELECT COUNT(*) FROM conflict_requests WHERE status = 'pending' AND conflict_type LIKE '%room%'"
        )->fetchColumn();
        $superStats['facultyTimeOverlap'] = (int) db()->query(
            "SELECT COUNT(*) FROM conflict_requests WHERE status = 'pending' AND conflict_type LIKE '%faculty%'"
        )->fetchColumn();
    } else {
        $superStats['roomDoubleBookingRisks'] = 0;
        $superStats['facultyTimeOverlap'] = 0;
    }
}
$superStats['scheduledClassesCurrentTerm'] = db_table_exists('schedules')
    ? (int) db()->query("SELECT COUNT(*) FROM schedules")->fetchColumn()
    : 0;

```

```

$superStats['unassignedTimeSlots'] = db_table_exists('rooms')
    ? max(0, ((int) db()->query('SELECT COUNT(*) FROM rooms')->fetchColumn() * 60) - $superStats['scheduledClassesCurrentTerm'])
    : 0;
if (db_table_exists('users')) {
    $lastLoginColumn = db_first_existing_column(
        'users',
        ['last_login', 'last_seen_at', 'updated_at', 'created_at'],
        ''
    );
    if ($lastLoginColumn !== '') {
        $lastLoginRaw = db()->query(
            "SELECT {$lastLoginColumn} FROM users WHERE role = 'super_admin' ORDER BY {$lastLoginColumn} DESC LIMIT 1"
        )->fetchColumn();
        if (is_string($lastLoginRaw) && $lastLoginRaw !== '') {
            $stamp = strtotime($lastLoginRaw);
            if ($stamp !== false) {
                $superStats['superAdminLastLogin'] = date('M j, Y g:i A', $stamp);
            }
        }
    }
} else {
    $activeFaculty = 0;
    $totalCourses = 0;
    $totalSchedules = 0;
    $openConflicts = 0;
}

$upSql = "SELECT s.start_time, s.end_time, f.full_name AS faculty_name, c.course_code, c.course_name, r.room_code
FROM schedules s
INNER JOIN faculty f ON f.id = s.faculty_id
INNER JOIN courses c ON c.id = s.course_id
INNER JOIN rooms r ON r.id = s.room_id
WHERE FIND_IN_SET(?, s.day_of_week) > 0 AND s.start_time >= CURTIME()";
$upParams = [$today];
if ($isAdmin) {
    $upSql .= ' AND s.college_id IS NOT NULL';
}
if (($role === 'dean' || $role === 'program_chair') && $collegeId) {
    $upSql .= ' AND s.college_id = ?';
    $upParams[] = $collegeId;
}
if ($programScope !== null) {
    $upSql .= ' AND c.department = ?';
    $upParams[] = $programScope;
}
if ($role === 'faculty' && !empty($_SESSION['faculty_id'])) {
    $upSql .= ' AND s.faculty_id = ?';
    $upParams[] = (int) $_SESSION['faculty_id'];
}
if ($role === 'gened') {
    $upSql .= ' AND c.is_gened = 1';
}
$showUpcoming = $role !== 'student' && $role !== 'super_admin';
$upSql .= ' ORDER BY s.start_time ASC LIMIT 15';
$upcoming = [];
if ($showUpcoming) {
    $upStmt = db()->prepare($upSql);
    $upStmt->execute($upParams);
    $upcoming = $upStmt->fetchAll();
}

$dashboardOverviewTitle = $role === 'student'
    ? 'Today\'s remaining classes (' . htmlspecialchars($today) . ')'
    : ($showUpcoming
        ? 'Upcoming classes today (' . htmlspecialchars($today) . ')'
        : ($role === 'super_admin'
            ? 'Super Admin workspace'
            : 'Student overview'));

$formatTime12h = static function (?string $time): string {
    $raw = substr((string) $time, 0, 5);
    $dt = DateTime::createFromFormat('H:i', $raw);
    return $dt ? $dt->format('g:i A') : $raw;
};

$studentClassIsLive = static function (?string $liveAt) use ($hasStudentLiveAt): bool {
    if (!$hasStudentLiveAt || $liveAt === null || $liveAt === '') {
        return false;
    }
}

```

```

$liveTs = strtotime($liveAt);
if ($liveTs === false) {
    return false;
}
return (time() - $liveTs) <= 2 * 3600;
};

$studentTodayClasses = [];
$studentTimeSlots = [];
$studentScheduleOverlapsHour = static function (array $scheduleRow, int $hour): bool {
    $slotStart = $hour * 60;
    $slotEnd = ($hour + 1) * 60;
    $st = time_to_minutes(substr((string) ($scheduleRow['start_time'] ?? ''), 0, 8));
    $en = time_to_minutes(substr((string) ($scheduleRow['end_time'] ?? ''), 0, 8));
    return $st < $slotEnd && $en > $slotStart;
};
$studentScheduleStartsInHour = static function (array $scheduleRow, int $hour): bool {
    $st = time_to_minutes(substr((string) ($scheduleRow['start_time'] ?? ''), 0, 8));
    return $st >= ($hour * 60) && $st < (($hour + 1) * 60);
};

if ($role === 'student' && isset($studentScheduleByDay[$today])) {
    $nowTime = date('H:i:s');
    foreach ($studentScheduleByDay[$today] as $schedRow) {
        if ((string) ($schedRow['end_time'] ?? '') >= $nowTime) {
            $studentTodayClasses[] = $schedRow;
        }
    }
}

if ($role === 'student' && $studentScheduleClasses !== []) {
    $gridStartMin = defined('TIME_MIN') ? time_to_minutes((string) TIME_MIN) : 7 * 60;
    $gridEndMin = defined('TIME_MAX') ? time_to_minutes((string) TIME_MAX) : 21 * 60;
    foreach ($studentScheduleClasses as $schedRow) {
        $st = time_to_minutes(substr((string) ($schedRow['start_time'] ?? ''), 0, 8));
        $en = time_to_minutes(substr((string) ($schedRow['end_time'] ?? ''), 0, 8));
        if ($st < $gridStartMin) {
            $gridStartMin = $st;
        }
        if ($en > $gridEndMin) {
            $gridEndMin = $en;
        }
    }
    $gridStartHour = (int) floor($gridStartMin / 60);
    $gridEndHour = (int) max($gridStartHour + 1, (int) ceil($gridEndMin / 60));
    for ($h = $gridStartHour; $h < $gridEndHour; $h++) {
        $studentTimeSlots[] = $h;
    }
}

$roleLabel = match ($role) {
    'super_admin' => 'Super Administrator',
    'admin' => 'Administrator',
    'dean' => 'Dean',
    'program_chair' => 'Program Chair',
    'gened' => 'General Education',
    'faculty' => 'Faculty',
    'student' => 'Student',
    default => 'User',
};

$heroMessage = match ($role) {
    'super_admin' => 'Provision System Administrator accounts and review high-level access. Day-to-day scheduling tools remain on each administrator's own login.',
    'admin' => 'Monitor colleges, faculty activity, schedules, and system-wide academic coordination from one place.',
    'dean' => 'Stay on top of college schedules, faculty assignments, and pending coordination concerns with a smoother dashboard.',
    'program_chair' => 'Review your program workload, faculty activity, and schedule updates with a focused dashboard experience.',
    'gened' => 'Track General Education faculty, courses, and schedules with a cleaner and more polished view.',
    'faculty' => 'Manage your assigned classes, run online sessions, and work within the courses officially assigned to your faculty account.',
    'student' => 'Access your enrolled online classes, read announcements, submit assessments, and keep up with graded work in one place.',
    default => 'Manage your academic scheduling tasks with a streamlined dashboard.',
};

$sealDataUri = '';
$sealCandidates = [
    __DIR__ . '/assets/images/wpu-seal.png',
    'C:\Users\wpu-0\cursor\projects\c-xampp-htdocs-CLASS\assets\c__Users_wpu-0_AppData_Roaming_Cursor_User_workspaceStorage_f94ec644ac39e85d4f1f3f887fe3104f_images_A_LOGO-58c392b9-fcf1-44cf-828c-95229f4ba88a.png',
    'C:\Users\wpu-0\cursor\projects\c-xampp-htdocs-CLASS\assets\c__Users_wpu-

```

```

0_AppData_Roaming_Cursor_User_workspaceStorage_f94ec644ac39e85d4f1f3f887fe3104f_images_A_LOGO-ba32c997-80f6-4298-97e0-4ca3b192b5d0.png',
];
foreach ($sealCandidates as $sealPath) {
    if (is_file($sealPath)) {
        $sealRaw = @file_get_contents($sealPath);
        if ($sealRaw !== false) {
            $sealDataUri = 'data:image/png;base64,' . base64_encode($sealRaw);
            break;
        }
    }
}

$pageTitle = 'Dashboard';
require_once __DIR__ . '/includes/header.php';
?>
<style>
.dashboard-shell {
    --dash-deep: #0c2435;
    --dash-green: #1f6f43;
    --dash-soft: #f5f8f6;
    --dash-border: #dbe7e0;
}

.dashboard-hero {
    position: relative;
    overflow: hidden;
    border: 0;
    border-radius: 28px;
    background:
        radial-gradient(circle at top right, rgba(255,255,255,0.22), transparent 22%),
        linear-gradient(135deg, #0a2235 0%, #143a4d 48%, #1f6f43 100%);
    box-shadow: 0 22px 55px rgba(12, 36, 53, 0.18);
}

.dashboard-hero::after {
    content: "";
    position: absolute;
    right: -40px;
    bottom: -60px;
    width: 220px;
    height: 220px;
    border-radius: 50%;
    background: rgba(255, 255, 255, 0.08);
}

.dashboard-hero .card-body {
    padding: 2rem;
}

.hero-chip {
    display: inline-flex;
    align-items: center;
    gap: 8px;
    padding: 8px 14px;
    border-radius: 999px;
    color: #fff;
    background: rgba(255,255,255,0.12);
    border: 1px solid rgba(255,255,255,0.16);
    font-size: 0.82rem;
    text-transform: uppercase;
    letter-spacing: 0.06em;
}

.hero-title {
    color: #fff;
    font-size: clamp(1.7rem, 3vw, 2.6rem);
    font-weight: 700;
    line-height: 1.15;
    margin: 0.9rem 0 0.65rem;
}

.hero-text {
    color: rgba(255,255,255,0.8);
    max-width: 700px;
    margin-bottom: 0;
}

.hero-seal-wrap {
    width: 138px;
    height: 138px;
}

```

```

border-radius: 50%;
display: inline-flex;
align-items: center;
justify-content: center;
background: rgba(255,255,255,0.12);
border: 1px solid rgba(255,255,255,0.2);
box-shadow: 0 18px 40px rgba(0,0,0,0.18);
}

.hero-seal-wrap img {
width: 120px;
height: 120px;
object-fit: contain;
border-radius: 50%;
background: #fff;
padding: 5px;
}

.hero-seal-fallback {
color: #fff;
font-size: 3rem;
}

.dashboard-refresh {
color: rgba(255,255,255,0.82);
}

.dash-stat {
border: 0;
border-radius: 22px;
overflow: hidden;
box-shadow: 0 16px 34px rgba(15, 23, 42, 0.08);
}

.dash-stat .card-body {
padding: 1.25rem 1.3rem;
}

.dash-stat-label {
font-size: 0.92rem;
opacity: 0.82;
}

.dash-stat-value {
font-size: 2rem;
font-weight: 700;
line-height: 1.1;
}

.glass-card {
border: 1px solid var(--dash-border);
border-radius: 24px;
overflow: hidden;
box-shadow: 0 16px 36px rgba(15, 23, 42, 0.06);
}

.glass-card .card-header {
background: linear-gradient(180deg, rgba(255,255,255,0.96), rgba(247,249,248,0.96));
border-bottom: 1px solid var(--dash-border);
padding: 1rem 1.25rem;
}

.upcoming-table thead th {
background: #f5f8f6;
border-bottom-width: 1px;
}

.upcoming-table tbody tr:hover {
background: #f8fbf9;
}

.student-calendar-wrap .schedule-weekly th.is-today,
.student-calendar-wrap .schedule-cell.is-today:not(.schedule-cell--vacant) {
background: rgba(31, 111, 67, 0.08);
}

.student-calendar-wrap .schedule-weekly th.is-today {
box-shadow: inset 0 -3px 0 #1f6f43;
}

```

```

.student-calendar-wrap .schedule-cell.is-today.schedule-cell--vacant {
    background-color: rgba(31, 111, 67, 0.04);
}

.student-calendar-wrap .schedule-block {
    transition: transform 0.15s ease, box-shadow 0.15s ease;
}

.student-calendar-wrap .schedule-block:hover {
    transform: translateY(-1px);
    box-shadow: 0 4px 12px rgba(15, 23, 42, 0.1);
}

.student-calendar-legend {
    display: flex;
    flex-wrap: wrap;
    gap: 0.5rem 1rem;
    font-size: 0.82rem;
    color: #5c6f7f;
}

.student-calendar-legend span {
    display: inline-flex;
    align-items: center;
    gap: 0.35rem;
}

.student-calendar-legend i {
    color: #1f6f43;
}

.conflict-card {
    border: 1px solid rgba(255, 193, 7, 0.35);
    border-radius: 24px;
    box-shadow: 0 16px 36px rgba(255, 193, 7, 0.08);
}

.conflict-card .card-header {
    background: linear-gradient(180deg, rgba(255, 193, 7, 0.12), rgba(255, 255, 255, 0.8));
    border-bottom: 1px solid rgba(255, 193, 7, 0.2);
    padding: 1rem 1.25rem;
}

@media (max-width: 991.98px) {
    .dashboard-hero .card-body {
        padding: 1.5rem;
    }

    .hero-seal-wrap {
        width: 110px;
        height: 110px;
    }

    .hero-seal-wrap img {
        width: 96px;
        height: 96px;
    }
}
</style>

```

```

<div class="dashboard-shell">
<?php if ($role === 'super_admin'): ?>
<style>
    .sa-container {
        background: #f0f2f8;
        border-radius: 28px;
        padding: 28px 24px;
    }

    .sa-header h1 {
        font-size: 28px;
        font-weight: 700;
        background: linear-gradient(135deg, #0b2b3b 0%, #1a4a6f 100%);
        -webkit-background-clip: text;
        background-clip: text;
        color: transparent;
        letter-spacing: -0.3px;
        margin-bottom: 8px;
    }

```

```
.sa-sub {
  color: #2c5a6e;
  font-weight: 500;
  font-size: 15px;
  border-left: 3px solid #c9772e;
  padding-left: 12px;
  margin-bottom: 24px;
}

.sa-stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 18px;
  margin-bottom: 26px;
}

.sa-stat-card,
.sa-insight-card,
.sa-footer-alerts {
  background: #fff;
  border-radius: 24px;
  border: 1px solid #e9eef5;
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.04);
}

.sa-stat-card {
  padding: 18px;
}

.sa-stat-title {
  font-size: 13px;
  text-transform: uppercase;
  color: #5b6e8c;
  font-weight: 600;
  letter-spacing: 0.4px;
  margin-bottom: 8px;
  display: flex;
  align-items: center;
  gap: 8px;
}

.sa-badge {
  width: 30px;
  height: 30px;
  border-radius: 50%;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  background: #eef2fa;
}

.sa-stat-number {
  font-size: 34px;
  line-height: 1.1;
  font-weight: 800;
  color: #1e2f41;
}

.sa-stat-trend {
  font-size: 13px;
  color: #63809c;
  margin-top: 6px;
}

.sa-insight-row {
  display: flex;
  flex-wrap: wrap;
  gap: 18px;
  margin-bottom: 20px;
}

.sa-insight-card {
  flex: 1;
  min-width: 260px;
  padding: 20px;
}

.sa-insight-card h3 {
  font-size: 17px;
  color: #1e3a4d;
}
```

```

border-left: 4px solid #c9772e;
padding-left: 12px;
margin-bottom: 14px;
}

.sa-compact-row {
display: flex;
justify-content: space-between;
border-bottom: 1px solid #edf2f7;
padding: 10px 0;
gap: 8px;
}

.sa-compact-label {
color: #436278;
font-weight: 500;
font-size: 14px;
}

.sa-compact-value {
color: #173e54;
font-weight: 700;
font-size: 18px;
text-align: right;
}

.sa-note {
font-size: 12px;
color: #7a8eaa;
margin-top: 12px;
background: #f8fafd;
border-radius: 12px;
padding: 10px 12px;
}

.sa-action-bar {
display: flex;
flex-wrap: wrap;
gap: 12px;
align-items: center;
margin-bottom: 18px;
}

.sa-btn {
border: 1px solid #cfd9d8;
background: #fff;
color: #1e4660;
border-radius: 999px;
padding: 10px 22px;
font-size: 14px;
font-weight: 600;
cursor: pointer;
}

.sa-btn-dark {
background: #1c2f3f;
color: #fff;
border: 0;
}

.sa-btn-super {
border: 0;
color: #fff;
background: linear-gradient(95deg, #2c5a6e 0%, #1e3f50 100%);
}

.sa-role-badge {
background: #e9eff5;
border-radius: 999px;
padding: 6px 14px;
font-size: 12px;
font-family: monospace;
}

.sa-footer-alerts {
padding: 16px;
display: flex;
flex-wrap: wrap;
gap: 10px;
align-items: center;

```



```

        justify-content: space-between;
    }

    .sa-conflict-badge {
        background: #fff2e6;
        border-left: 4px solid #e67e22;
        border-radius: 999px;
        padding: 8px 14px;
        color: #b95a0c;
        font-weight: 600;
        font-size: 14px;
    }

    .sa-footer-note {
        color: #4b6f8c;
        font-size: 13px;
        background: #fff9ff;
        border-radius: 999px;
        padding: 8px 14px;
    }
}
</style>
<div class="sa-container" id="saContainer">
    <div class="sa-header">
        <h1>Dashboard - Western Philippines University</h1>
        <div class="sa-sub">Super Administrator Control Center - High-level analytics and system oversight</div>
    </div>

    <div class="sa-stats-grid">
        <div class="sa-stat-card"><div class="sa-stat-title"><span class="sa-badge">👤</span>Administrator accounts</div><div
class="sa-stat-number"><%= number_format($activeFaculty) %></div><div class="sa-stat-trend">Total registered system admins</div></div>
        <div class="sa-stat-card"><div class="sa-stat-title"><span class="sa-badge">🟢</span>Active admins</div><div class="sa-stat-
number"><%= number_format($totalCourses) %></div><div class="sa-stat-trend">Currently active</div></div>
        <div class="sa-stat-card"><div class="sa-stat-title"><span class="sa-badge">🔴</span>Disabled admins</div><div class="sa-stat-
number"><%= number_format($totalSchedules) %></div><div class="sa-stat-trend">Locked or disabled accounts</div></div>
        <div class="sa-stat-card"><div class="sa-stat-title"><span class="sa-badge">👑</span>Super Admin users</div><div class="sa-
stat-number"><%= number_format($openConflicts) %></div><div class="sa-stat-trend">Ultimate privilege level</div></div>
        <div class="sa-stat-card"><div class="sa-stat-title"><span class="sa-badge">📅</span>Total schedules</div><div class="sa-stat-
number"><%= number_format($superStats['totalSchedulesAllTime']) %></div><div class="sa-stat-trend">All-time schedule records</div></div>
        <div class="sa-stat-card"><div class="sa-stat-title"><span class="sa-badge">🏫</span>Colleges / Departments</div><div
class="sa-stat-number"><%= number_format($superStats['collegesDepartments']) %></div><div class="sa-stat-trend">Configured academic
units</div></div>
        <div class="sa-stat-card"><div class="sa-stat-title"><span class="sa-badge">👥</span>Faculty members</div><div class="sa-stat-
number"><%= number_format($superStats['facultyMembers']) %></div><div class="sa-stat-trend">Total faculty records</div></div>
        <div class="sa-stat-card"><div class="sa-stat-title"><span class="sa-badge">🎓</span>Student enrollments</div><div class="sa-
stat-number"><%= number_format($superStats['studentEnrollments']) %></div><div class="sa-stat-trend">Across online classrooms</div></div>
    </div>

    <div class="sa-insight-row">
        <div class="sa-insight-card">
            <h3>Admin and Access Analytics</h3>
            <div class="sa-compact-row"><span class="sa-compact-label">System admin accounts (provisioned)</span><span class="sa-
compact-value"><%= number_format($activeFaculty) %></span></div>
            <div class="sa-compact-row"><span class="sa-compact-label">Super admin power users</span><span class="sa-compact-value"><%=
number_format($openConflicts) %></span></div>
            <div class="sa-compact-row"><span class="sa-compact-label">Inactive or pending invites</span><span class="sa-compact-value">
<%= number_format($superStats['inactivePendingInvites']) %></span></div>
            <div class="sa-compact-row"><span class="sa-compact-label">Last login (Super Admin)</span><span class="sa-compact-value"><%=
htmlspecialchars($superStats['superAdminLastLogin']) %></span></div>
            <div class="sa-note">Super Admin role is strictly for system-wide configuration. Day-to-day scheduling tools require
standard admin login.</div>
        </div>
        <div class="sa-insight-card">
            <h3>Conflict and Scheduling Overview</h3>
            <div class="sa-compact-row"><span class="sa-compact-label">Active conflict alerts (pending)</span><span class="sa-compact-
value" style="color:#c96b1a;"><%= number_format($superStats['activeConflictAlerts']) %></span></div>
            <div class="sa-compact-row"><span class="sa-compact-label">Resolved conflicts (last 30d)</span><span class="sa-compact-
value"><%= number_format($superStats['resolvedConflicts30d']) %></span></div>
            <div class="sa-compact-row"><span class="sa-compact-label">Room double-booking risks</span><span class="sa-compact-value"><%=
number_format($superStats['roomDoubleBookingRisks']) %></span></div>
            <div class="sa-compact-row"><span class="sa-compact-label">Faculty time overlap</span><span class="sa-compact-value"><%=
number_format($superStats['facultyTimeOverlap']) %></span></div>
            <div class="sa-note">Super Admin has no conflict queue by design. Switch to an Administrator account to manage conflicts and
daily scheduling.</div>
        </div>
        <div class="sa-insight-card">
            <h3>System Utilization</h3>
            <div class="sa-compact-row"><span class="sa-compact-label">Total room usage (weekly avg)</span><span class="sa-compact-
value">N/A</span></div>
            <div class="sa-compact-row"><span class="sa-compact-label">Scheduled classes (current term)</span><span class="sa-compact-
value"><%= number_format($superStats['scheduledClassesCurrentTerm']) %></span></div>

```

```

        <div class="sa-compact-row"><span class="sa-compact-label">Unassigned time slots</span><span class="sa-compact-value"><?=
number_format($superStats['unassignedTimeSlots']) ?></span></div>
        <div class="sa-compact-row"><span class="sa-compact-label">Peak hour load (10 AM - 12 PM)</span><span class="sa-compact-
value">N/A</span></div>
        <div class="sa-note">Utilization indicators improve as room, term, and conflict data become more complete.</div>
    </div>
</div>

<div class="sa-action-bar">
    <button type="button" class="sa-btn sa-btn-dark" id="darkModeToggleBtn">Dark</button>
    <button type="button" class="sa-btn sa-btn-super" id="superAdminInfoBtn">Super Administrator - SUPER_ADMIN</button>
    <a href="super_admin_programs.php" class="sa-btn"><?= app_tooltip_attr('Opens the university-wide program offers catalog to view,
edit, or delete any college program.') ?>>Program offers</a>
    <span class="sa-role-badge">Elevated session - Full system audit rights</span>
</div>

<div class="sa-footer-alerts">
    <div class="sa-conflict-badge">Conflict alerts - <?= number_format($superStats['activeConflictAlerts']) ?> pending
resolution</div>
    <div class="sa-footer-note">Super Admin is only for Administrator accounts (system admins). Not a student account.</div>
    <div class="sa-footer-note">No conflict queue for Super Admin. Use Administrator accounts for day-to-day scheduling tools.</div>
</div>

<div class="modal fade" id="superAdminWarningModal" tabindex="-1" aria-labelledby="superAdminWarningModalLabel" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content border-warning">
            <div class="modal-header bg-warning-subtle">
                <h5 class="modal-title text-warning-emphasis" id="superAdminWarningModalLabel">
                    <i class="fa-solid fa-triangle-exclamation me-2"></i>Warning - SUPER_ADMIN
                </h5>
                <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
            </div>
            <div class="modal-body">
                <p class="mb-0">SUPER_ADMIN role: full system audit and account provisioning. Use standard admin login for daily
scheduling.</p>
            </div>
            <div class="modal-footer">
                <button type="button" class="btn btn-warning" data-bs-dismiss="modal">Understood</button>
            </div>
        </div>
    </div>
</div>
</div>
<?php else: ?>
    <div class="card dashboard-hero text-white mb-4">
        <div class="card-body">
            <div class="row align-items-center g-4">
                <div class="col-lg-8">
                    <div class="hero-chip">
                        <i class="fa-solid fa-gauge-high"></i>
                        <?= htmlspecialchars($roleLabel) ?> dashboard
                    </div>
                    <h1 class="hero-title"><?= $role === 'student' ? 'Western Philippines University LMS Dashboard' : 'Western Philippines
University Scheduling Dashboard' ?></h1>
                    <p class="hero-text"><?= htmlspecialchars($heroMessage) ?></p>
                </div>
                <div class="col-lg-4 text-lg-end">
                    <div class="hero-seal-wrap">
                        <?php if ($sealDataUri !== ''): ?>
                            
                        <?php else: ?>
                            
                        <?php endif; ?>
                    </div>
                    <div class="dashboard-refresh mt-3 text-lg-end">
                        <div class="small no-print"><i class="fa-regular fa-calendar me-1"></i><?= htmlspecialchars(date('l, F j, Y')) ?>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
</div>
</div>

<div class="row g-3 mb-4">
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-primary text-white">
            <div class="card-body">
                <div class="dash-stat-label"><?= htmlspecialchars($stat1Label) ?></div>
                <div class="dash-stat-value"><?= $activeFaculty ?></div>
            </div>
        </div>
    </div>
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-primary text-white">
            <div class="card-body">
                <div class="dash-stat-label"><?= htmlspecialchars($stat2Label) ?></div>
                <div class="dash-stat-value"><?= $activeFaculty ?></div>
            </div>
        </div>
    </div>
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-primary text-white">
            <div class="card-body">
                <div class="dash-stat-label"><?= htmlspecialchars($stat3Label) ?></div>
                <div class="dash-stat-value"><?= $activeFaculty ?></div>
            </div>
        </div>
    </div>
</div>

```

```

        </div>
    </div>
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-success text-white">
            <div class="card-body">
                <div class="dash-stat-label"><?= htmlspecialchars($stat2Label) ?></div>
                <div class="dash-stat-value"><?= $totalCourses ?></div>
            </div>
        </div>
    </div>
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-secondary text-white">
            <div class="card-body">
                <div class="dash-stat-label"><?= htmlspecialchars($stat3Label) ?></div>
                <div class="dash-stat-value"><?= $totalSchedules ?></div>
            </div>
        </div>
    </div>
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-warning text-dark">
            <div class="card-body">
                <div class="dash-stat-label"><?= htmlspecialchars($stat4Label) ?></div>
                <div class="dash-stat-value"><?= $openConflicts ?></div>
            </div>
        </div>
    </div>
</div>

<?php if ($role === 'faculty'): ?>
    <div class="card glass-card mb-4">
        <div class="card-header">
            <i class="fa-solid fa-user-check me-2 text-primary"></i>Faculty Account Access
        </div>
        <div class="card-body">
            <ul class="mb-0">
                <li>Can create online classrooms for assigned courses only.</li>
                <li>Can add or enroll students to online classes.</li>
                <li>Can manage course content, assignments, quizzes, and grades.</li>
                <li>Can conduct live video sessions using a Google Meet link.</li>
                <li>Cannot access courses that are not assigned to the faculty account.</li>
            </ul>
            <div class="mt-3 d-flex flex-wrap gap-2">
                <a href="faculty_classrooms.php" class="btn btn-primary btn-sm"<?= app_tooltip_attr('Opens your online classrooms for assigned courses. Use this to manage materials, Meet links, and student access.') ?>><i class="fa-solid fa-chalkboard me-1"></i>Open Classrooms</a>
                <a href="faculty_schedule.php" class="btn btn-outline-primary btn-sm"<?= app_tooltip_attr('Opens your teaching timetable and online class links. Use this to confirm times and join sessions.') ?>><i class="fa-solid fa-calendar-check me-1"></i>Open My Schedule</a>
            </div>
        </div>
    </div>
<?php endif; ?>

<?php if ($role === 'student'): ?>
    <div class="card glass-card mb-4">
        <div class="card-header">
            <i class="fa-solid fa-user-graduate me-2 text-primary"></i>Student Access
        </div>
        <div class="card-body">
            <ul class="mb-0">
                <li>View only the classes where your student account is enrolled.</li>
                <li>Read classroom announcements and course materials from your instructors.</li>
                <li>Answer assignments and quizzes using your student login.</li>
                <li>Review grades and feedback once your instructor evaluates your work.</li>
                <li>Cannot access classrooms or assessments where you are not enrolled.</li>
            </ul>
            <div class="mt-3 d-flex flex-wrap gap-2 student-dashboard-quick-actions">
                <a href="student_classrooms.php" class="btn btn-primary btn-sm"<?= student_tooltip_attr('Opens the list of online classes you are enrolled in. Use this to join Meet, open materials, or submit work.') ?>><i class="fa-solid fa-user-graduate me-1"></i>Open My Classes</a>
                <a href="student_edutools.php" class="btn btn-outline-primary btn-sm"<?= student_tooltip_attr('Opens EduTools: notebook, ChatGPT, Khan Academy, and other study helpers.') ?>><i class="fa-solid fa-wand-magic-sparkles me-1"></i>Learning tools</a>
                <a href="student_materials_reviewer.php" class="btn btn-outline-primary btn-sm"<?= student_tooltip_attr('Review weekly course materials and generate practice questions per week.') ?>><i class="fa-solid fa-clipboard-question me-1"></i>Materials reviewer</a>
                <a href="settings.php" class="btn btn-outline-secondary btn-sm"<?= student_tooltip_attr('Opens account settings to change your password. Use this when your password expires or you want a stronger one.') ?>><i class="fa-solid fa-key me-1"></i>Change Password</a>
            </div>
        </div>
    </div>
<?php endif; ?>

```

[illegible]

[illegible]

```

        <table class="table table-hover upcoming-table mb-0">
            <thead>
                <tr>
                    <th>Time</th>
                    <th>Course</th>
                    <th>Instructor</th>
                    <th>Room</th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($studentTodayClasses as $u): ?>
                    <?php
                        $liveAt = $hasStudentLiveAt ? (string) ($u['online_live_at'] ?? '') : '';
                        $isLive = $studentClassIsLive($liveAt);
                    <?php
                        <tr>
                            <td>
                                <? = htmlspecialchars($formatTime12h((string) ($u['start_time'] ?? ''))) ?>
                                -
                                <? = htmlspecialchars($formatTime12h((string) ($u['end_time'] ?? ''))) ?>
                                <?php if ($isLive): ?>
                                    <span class="badge bg-danger live-pulse-badge ms-1">LIVE</span>
                                <?php endif; ?>
                            </td>
                            <td><strong><? = htmlspecialchars((string) $u['course_code']) ?></strong> <? =
htmlspecialchars((string) $u['course_name']) ?></td>
                            <td><? = htmlspecialchars((string) $u['faculty_name']) ?></td>
                            <td><? = htmlspecialchars((string) $u['room_code']) ?></td>
                        </tr>
                    <?php endforeach; ?>
                </tbody>
            </table>
        </div>
        <?php endif; ?>
        <?php elseif ($role === 'super_admin'): ?>
            <div class="p-3">
                <p class="text-muted mb-2">Super Admin is only for <strong>Administrator accounts</strong> (system admins). This is
not a student account – do not use My Classes.</p>
            </div>
        <?php elseif (!$upcoming): ?>
            <p class="text-muted p-3 mb-0">No more classes scheduled for today after the current time.</p>
        <?php else: ?>
            <div class="table-responsive">
                <table class="table table-hover upcoming-table mb-0">
                    <thead>
                        <tr>
                            <th>Time</th>
                            <th>Course</th>
                            <th>Faculty</th>
                            <th>Room</th>
                        </tr>
                    </thead>
                    <tbody>
                        <?php foreach ($upcoming as $u): ?>
                            <tr>
                                <td><? = htmlspecialchars($formatTime12h((string) ($u['start_time'] ?? ''))) ?> - <? =
htmlspecialchars($formatTime12h((string) ($u['end_time'] ?? ''))) ?></td>
                                <td><strong><? = htmlspecialchars($u['course_code']) ?></strong> <? =
htmlspecialchars($u['course_name']) ?></td>
                                <td><? = htmlspecialchars($u['faculty_name']) ?></td>
                                <td><? = htmlspecialchars($u['room_code']) ?></td>
                            </tr>
                        <?php endforeach; ?>
                    </tbody>
                </table>
            </div>
        <?php endif; ?>
    </div>
</div>
<div class="col-lg-5">
    <div class="card conflict-card">
        <div class="card-header">
            <?php if ($role === 'student'): ?>
                <i class="fa-solid fa-list-check me-2 text-warning"></i>Pending work
            <?php else: ?>
                <i class="fa-solid fa-triangle-exclamation me-2 text-warning"></i>Conflict alerts
            <?php endif; ?>
        </div>
        <div class="card-body">

```

```

        <?php if ($role === 'super_admin'): ?>
            <p class="text-muted small mb-0">Super Admin has no conflict queue. Use <strong>Administrator accounts</strong> or sign
in as an <code>admin</code> user for day-to-day scheduling tools.</p>
        <?php elseif ($role === 'student'): ?>
            <?php if ($openConflicts === 0): ?>
                <p class="text-success mb-0"><i class="fa-solid fa-check-circle me-1"></i>All caught up – no pending assessments.
            </p>
            <?php else: ?>
                <p class="mb-2">You have <strong><?= $openConflicts ?></strong> assessment<?= $openConflicts === 1 ? ' ' : 's' ?>
waiting for your answers.</p>
                <a href="student_classrooms.php" class="btn btn-outline-warning btn-sm"><?= student_tooltip_attr('Opens My Classes so
you can find and submit pending assessments.') ?>>Open My Classes</a>
            <?php endif; ?>
            <?php elseif ($openConflicts === 0): ?>
                <p class="text-success mb-0"><i class="fa-solid fa-check-circle me-1"></i>No pending alerts.</p>
            <?php else: ?>
                <p class="mb-2">You have <strong><?= $openConflicts ?></strong> pending item(s).</p>
                <a href="conflicts.php" class="btn btn-outline-warning btn-sm"><?= app_tooltip_attr($role === 'student' ? 'Opens the
staff conflicts review page. If you are a student and see an error, use My Classes to finish pending assessments shown in your dashboard
counts.' : 'Opens the conflicts list to approve, reject, or resolve pending schedule coordination items.') ?>>Review conflicts</a>
            <?php endif; ?>
        </div>
    </div>
</div>
</div>
<?php endif; ?>

<script>
(function () {
    const darkToggle = document.getElementById('darkModeToggleBtn');
    const superAdminBtn = document.getElementById('superAdminInfoBtn');
    const container = document.getElementById('saContainer');
    if (!darkToggle || !superAdminBtn || !container) return;

    function applyDarkMode(isDark) {
        const cards = container.querySelectorAll('.sa-stat-card, .sa-insight-card, .sa-footer-alerts');
        if (isDark) {
            document.body.style.backgroundColor = '#121826';
            container.style.backgroundColor = '#121826';
            cards.forEach(function (el) {
                el.style.backgroundColor = '#1e2a36';
                el.style.borderColor = '#2d3e4e';
            });
        } else {
            document.body.style.backgroundColor = '#f0f2f8';
            container.style.backgroundColor = '#f0f2f8';
            cards.forEach(function (el) {
                el.style.backgroundColor = '#ffffff';
                el.style.borderColor = '#e9eef5';
            });
        }
    }

    let darkModeEnabled = localStorage.getItem('wpu_dark_mode') === 'true';
    applyDarkMode(darkModeEnabled);
    darkToggle.addEventListener('click', function () {
        darkModeEnabled = !darkModeEnabled;
        applyDarkMode(darkModeEnabled);
        localStorage.setItem('wpu_dark_mode', darkModeEnabled ? 'true' : 'false');
    });

    superAdminBtn.addEventListener('click', function () {
        const modalEl = document.getElementById('superAdminWarningModal');
        if (!modalEl || typeof bootstrap === 'undefined' || !bootstrap.Modal) {
            window.alert('Warning – SUPER_ADMIN\n\nSUPER_ADMIN role: full system audit and account provisioning. Use standard admin
login for daily scheduling.');
```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_login();

$role = (string) ($_SESSION['role'] ?? '');
$collegeId = current_college_id();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
$today = date('l');
$isAdmin = $role === 'admin';
$stat1Label = 'Active faculty';
$stat2Label = 'Courses';
$stat3Label = 'Schedules';
$stat4Label = 'Open conflicts';

if ($isAdmin) {
    $activeFaculty = (int) db()->query("SELECT COUNT(*) FROM faculty WHERE status = 'active' AND college_id IS NOT NULL")->fetchColumn();
    $totalCourses = (int) db()->query('SELECT COUNT(*) FROM courses WHERE college_id IS NOT NULL')->fetchColumn();
    $totalSchedules = (int) db()->query('SELECT COUNT(*) FROM schedules WHERE college_id IS NOT NULL')->fetchColumn();
    $openConflicts = db_table_exists('conflict_requests')
        ? (int) db()->query("SELECT COUNT(*) FROM conflict_requests WHERE status = 'pending'")->fetchColumn()
        : 0;
} elseif (($role === 'dean' || $role === 'program_chair') && $collegeId) {
    $facultySql = "SELECT COUNT(*) FROM faculty WHERE status='active' AND college_id=?";
    $courseSql = "SELECT COUNT(*) FROM courses WHERE college_id=?";
    $scheduleSql = "SELECT COUNT(*) FROM schedules s INNER JOIN courses c ON c.id = s.course_id WHERE s.college_id=?";
    $params = [$collegeId];
    if ($programScope !== null) {
        $facultySql .= " AND department=?";
        $courseSql .= " AND department=?";
        $scheduleSql .= " AND c.department=?";
        $params[] = $programScope;
    }
    $st = db()->prepare($facultySql);
    $st->execute($params);
    $activeFaculty = (int) $st->fetchColumn();
    $st = db()->prepare($courseSql);
    $st->execute($params);
    $totalCourses = (int) $st->fetchColumn();
    $st = db()->prepare($scheduleSql);
    $st->execute($params);
    $totalSchedules = (int) $st->fetchColumn();
    if ($role === 'program_chair' && db_table_exists('schedule_change_requests')) {
        $st = db()->prepare(
            "SELECT COUNT(*) FROM schedule_change_requests scr
            INNER JOIN schedules s ON s.id = scr.schedule_id
            INNER JOIN courses c ON c.id = s.course_id
            WHERE s.college_id=? AND c.department=? AND scr.status='pending'"
        );
        $st->execute([$collegeId, $programScope]);
        $openConflicts = (int) $st->fetchColumn();
    } elseif ($role === 'program_chair') {
        $openConflicts = 0;
    } elseif (db_table_exists('schedule_change_requests')) {
        $st = db()->prepare(
            "SELECT COUNT(*) FROM schedule_change_requests scr INNER JOIN schedules s ON s.id = scr.schedule_id WHERE s.college_id=? AND
            scr.status='pending'"
        );
        $st->execute([$collegeId]);
        $openConflicts = (int) $st->fetchColumn();
    } else {
        $openConflicts = 0;
    }
} elseif ($role === 'gened') {
    $activeFaculty = (int) db()->query("SELECT COUNT(*) FROM faculty WHERE status='active' AND is_gened=1")->fetchColumn();
    $totalCourses = (int) db()->query("SELECT COUNT(*) FROM courses WHERE is_gened=1")->fetchColumn();
    $totalSchedules = (int) db()->query(
        "SELECT COUNT(*) FROM schedules s INNER JOIN courses c ON c.id=s.course_id WHERE c.is_gened=1"
    )->fetchColumn();
    $openConflicts = 0;
} elseif ($role === 'student') {
    $studentSelfId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
    if ($studentSelfId < 1) {
        $studentSelfId = resolve_student_id_for_user((int) ($_SESSION['user_id'] ?? 0)) ?? 0;
        $_SESSION['student_id'] = $studentSelfId > 0 ? $studentSelfId : null;
    }
}

```



```

$activeFaculty = 0;
$totalCourses = 0;
$totalSchedules = 0;
$openConflicts = 0;
$stat1Label = 'My classes';
$stat2Label = 'Assessments';
$stat3Label = 'Announcements';
$stat4Label = 'Pending work';

if ($studentSelfId > 0 && db_table_exists('classroom_enrollments')) {
    $st = db()->prepare('SELECT COUNT(*) FROM classroom_enrollments WHERE student_id = ?');
    $st->execute([$studentSelfId]);
    $activeFaculty = (int) $st->fetchColumn();

    if (db_table_exists('classroom_assessments')) {
        $st = db()->prepare(
            'SELECT COUNT(*)
            FROM classroom_assessments ca
            INNER JOIN classroom_enrollments ce ON ce.classroom_id = ca.classroom_id
            WHERE ce.student_id = ?'
        );
        $st->execute([$studentSelfId]);
        $totalCourses = (int) $st->fetchColumn();
    }

    if (db_table_exists('classroom_content')) {
        $st = db()->prepare(
            'SELECT COUNT(*)
            FROM classroom_content cc
            INNER JOIN classroom_enrollments ce ON ce.classroom_id = cc.classroom_id
            WHERE ce.student_id = ? AND cc.content_type = "announcement"'
        );
        $st->execute([$studentSelfId]);
        $totalSchedules = (int) $st->fetchColumn();
    }

    if (db_table_exists('classroom_submissions') && db_table_exists('classroom_assessments')) {
        $st = db()->prepare(
            'SELECT COUNT(*)
            FROM classroom_assessments ca
            INNER JOIN classroom_enrollments ce ON ce.classroom_id = ca.classroom_id
            LEFT JOIN classroom_submissions sub
            ON sub.assessment_id = ca.id AND sub.student_id = ce.student_id
            WHERE ce.student_id = ? AND sub.id IS NULL'
        );
        $st->execute([$studentSelfId]);
        $openConflicts = (int) $st->fetchColumn();
    }
}
} else {
    $activeFaculty = 0;
    $totalCourses = 0;
    $totalSchedules = 0;
    $openConflicts = 0;
}

$upSql = "SELECT s.start_time, s.end_time, f.full_name AS faculty_name, c.course_code, c.course_name, r.room_code
FROM schedules s
INNER JOIN faculty f ON f.id = s.faculty_id
INNER JOIN courses c ON c.id = s.course_id
INNER JOIN rooms r ON r.id = s.room_id
WHERE FIND_IN_SET(?, s.day_of_week) > 0 AND s.start_time >= CURTIME()";
$upParams = [$today];
if ($isAdmin) {
    $upSql .= ' AND s.college_id IS NOT NULL';
}
if (($role === 'dean' || $role === 'program_chair') && $collegeId) {
    $upSql .= ' AND s.college_id = ?';
    $upParams[] = $collegeId;
}
if ($programScope !== null) {
    $upSql .= ' AND c.department = ?';
    $upParams[] = $programScope;
}
if ($role === 'faculty' && !empty($_SESSION['faculty_id'])) {
    $upSql .= ' AND s.faculty_id = ?';
    $upParams[] = (int) $_SESSION['faculty_id'];
}
if ($role === 'gened') {
    $upSql .= ' AND c.is_gened = 1';
}

```

```

}
$showUpcoming = $role !== 'student';
$upSql .= ' ORDER BY s.start_time ASC LIMIT 15';
$upStmt = db()->prepare($upSql);
$upStmt->execute($upParams);
$upcoming = $upStmt->fetchAll();

$formatTime12h = static function (?string $time): string {
    $raw = substr((string) $time, 0, 5);
    $dt = DateTime::createFromFormat('H:i', $raw);
    return $dt ? $dt->format('g:i A') : $raw;
};

$roleLabel = match ($role) {
    'admin' => 'Administrator',
    'dean' => 'Dean',
    'program_chair' => 'Program Chair',
    'gened' => 'General Education',
    'faculty' => 'Faculty',
    'student' => 'Student',
    default => 'User',
};

$heroMessage = match ($role) {
    'admin' => 'Monitor colleges, faculty activity, schedules, and system-wide academic coordination from one place.',
    'dean' => 'Stay on top of college schedules, faculty assignments, and pending coordination concerns with a smoother dashboard.',
    'program_chair' => 'Review your program workload, faculty activity, and schedule updates with a focused dashboard experience.',
    'gened' => 'Track General Education faculty, courses, and schedules with a cleaner and more polished view.',
    'faculty' => 'Manage your assigned classes, run online sessions, and work within the courses officially assigned to your faculty account.',
    'student' => 'Access your enrolled online classes, read announcements, submit assessments, and keep up with graded work in one place.',
    default => 'Manage your academic scheduling tasks with a streamlined dashboard.',
};

$sealDataUri = '';
$sealCandidates = [
    __DIR__ . '/assets/images/wpu-seal.png',
    'C:\Users\wpu-0\cursor\projects\c-xampp\htdocs\CLASS\assets\c__Users_wpu-0_AppData_Roaming_Cursor_User_workspaceStorage_f94ec644ac39e85d4f1f3f887fe3104f_images_A_LOGO-58c392b9-fcf1-44cf-828c-95229f4ba88a.png',
    'C:\Users\wpu-0\cursor\projects\c-xampp\htdocs\CLASS\assets\c__Users_wpu-0_AppData_Roaming_Cursor_User_workspaceStorage_f94ec644ac39e85d4f1f3f887fe3104f_images_A_LOGO-ba32c997-80f6-4298-97e0-4ca3b192b5d0.png',
];
foreach ($sealCandidates as $sealPath) {
    if (is_file($sealPath)) {
        $sealRaw = @file_get_contents($sealPath);
        if ($sealRaw !== false) {
            $sealDataUri = 'data:image/png;base64,' . base64_encode($sealRaw);
            break;
        }
    }
}

$pageTitle = 'Dashboard';
require_once __DIR__ . '/includes/header.php';
?>

<style>
.dashboard-shell {
    --dash-deep: #0c2435;
    --dash-green: #1f6f43;
    --dash-soft: #f5f8f6;
    --dash-border: #dbe7e0;
}

.dashboard-hero {
    position: relative;
    overflow: hidden;
    border: 0;
    border-radius: 28px;
    background:
        radial-gradient(circle at top right, rgba(255,255,255,0.22), transparent 22%),
        linear-gradient(135deg, #0a2235 0%, #143a4d 48%, #1f6f43 100%);
    box-shadow: 0 22px 55px rgba(12, 36, 53, 0.18);
}

.dashboard-hero::after {
    content: "";
    position: absolute;
    right: -40px;
    bottom: -60px;
    width: 220px;

```

```
    height: 220px;
    border-radius: 50%;
    background: rgba(255, 255, 255, 0.08);
  }

.dashboard-hero .card-body {
  padding: 2rem;
}

.hero-chip {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 8px 14px;
  border-radius: 999px;
  color: #fff;
  background: rgba(255,255,255,0.12);
  border: 1px solid rgba(255,255,255,0.16);
  font-size: 0.82rem;
  text-transform: uppercase;
  letter-spacing: 0.06em;
}

.hero-title {
  color: #fff;
  font-size: clamp(1.7rem, 3vw, 2.6rem);
  font-weight: 700;
  line-height: 1.15;
  margin: 0.9rem 0 0.65rem;
}

.hero-text {
  color: rgba(255,255,255,0.8);
  max-width: 700px;
  margin-bottom: 0;
}

.hero-seal-wrap {
  width: 138px;
  height: 138px;
  border-radius: 50%;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  background: rgba(255,255,255,0.12);
  border: 1px solid rgba(255,255,255,0.2);
  box-shadow: 0 18px 40px rgba(0,0,0,0.18);
}

.hero-seal-wrap img {
  width: 120px;
  height: 120px;
  object-fit: contain;
  border-radius: 50%;
  background: #fff;
  padding: 5px;
}

.hero-seal-fallback {
  color: #fff;
  font-size: 3rem;
}

.dashboard-refresh {
  color: rgba(255,255,255,0.82);
}

.dash-stat {
  border: 0;
  border-radius: 22px;
  overflow: hidden;
  box-shadow: 0 16px 34px rgba(15, 23, 42, 0.08);
}

.dash-stat .card-body {
  padding: 1.25rem 1.3rem;
}

.dash-stat-label {
  font-size: 0.92rem;
}
```

```

        opacity: 0.82;
    }

    .dash-stat-value {
        font-size: 2rem;
        font-weight: 700;
        line-height: 1.1;
    }

    .glass-card {
        border: 1px solid var(--dash-border);
        border-radius: 24px;
        overflow: hidden;
        box-shadow: 0 16px 36px rgba(15, 23, 42, 0.06);
    }

    .glass-card .card-header {
        background: linear-gradient(180deg, rgba(255,255,255,0.96), rgba(247,249,248,0.96));
        border-bottom: 1px solid var(--dash-border);
        padding: 1rem 1.25rem;
    }

    .upcoming-table thead th {
        background: #f5f8f6;
        border-bottom-width: 1px;
    }

    .upcoming-table tbody tr:hover {
        background: #f8fbf9;
    }

    .conflict-card {
        border: 1px solid rgba(255, 193, 7, 0.35);
        border-radius: 24px;
        box-shadow: 0 16px 36px rgba(255, 193, 7, 0.08);
    }

    .conflict-card .card-header {
        background: linear-gradient(180deg, rgba(255, 193, 7, 0.12), rgba(255, 255, 255, 0.8));
        border-bottom: 1px solid rgba(255, 193, 7, 0.2);
        padding: 1rem 1.25rem;
    }

    @media (max-width: 991.98px) {
        .dashboard-hero .card-body {
            padding: 1.5rem;
        }

        .hero-seal-wrap {
            width: 110px;
            height: 110px;
        }

        .hero-seal-wrap img {
            width: 96px;
            height: 96px;
        }
    }
}
</style>

<div class="dashboard-shell">
    <div class="card dashboard-hero text-white mb-4">
        <div class="card-body">
            <div class="row align-items-center g-4">
                <div class="col-lg-8">
                    <div class="hero-chip">
                        <i class="fa-solid fa-gauge-high"></i>
                        <?= htmlspecialchars($roleLabel) ?> dashboard
                    </div>
                    <h1 class="hero-title"><?= $role == 'student' ? 'Western Philippines University LMS Dashboard' : 'Western Philippines University Scheduling Dashboard' ?></h1>
                    <p class="hero-text"><?= htmlspecialchars($heroMessage) ?></p>
                </div>
                <div class="col-lg-4 text-lg-end">
                    <div class="hero-seal-wrap">
                        <?php if ($sealDataUri !== ''): ?>
                            
                        <?php else: ?>
                            <span class="hero-seal-fallback"><i class="fa-solid fa-building-columns"></i></span>
                        <?php endif; ?>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>

```

```

        </div>
        <div class="dashboard-refresh mt-3 text-lg-end">
            <div class="small no-print"><i class="fa-regular fa-calendar me-1"></i><? = htmlspecialchars(date('l, F j, Y')) ?>
</div>
        </div>
    </div>
</div>
</div>
</div>
</div>
</div>
<div class="row g-3 mb-4">
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-primary text-white">
            <div class="card-body">
                <div class="dash-stat-label"><? = htmlspecialchars($stat1Label) ?></div>
                <div class="dash-stat-value"><? = $activeFaculty ?></div>
            </div>
        </div>
    </div>
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-success text-white">
            <div class="card-body">
                <div class="dash-stat-label"><? = htmlspecialchars($stat2Label) ?></div>
                <div class="dash-stat-value"><? = $totalCourses ?></div>
            </div>
        </div>
    </div>
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-secondary text-white">
            <div class="card-body">
                <div class="dash-stat-label"><? = htmlspecialchars($stat3Label) ?></div>
                <div class="dash-stat-value"><? = $totalSchedules ?></div>
            </div>
        </div>
    </div>
    <div class="col-6 col-lg-3">
        <div class="card dash-stat h-100 bg-warning text-dark">
            <div class="card-body">
                <div class="dash-stat-label"><? = htmlspecialchars($stat4Label) ?></div>
                <div class="dash-stat-value"><? = $openConflicts ?></div>
            </div>
        </div>
    </div>
</div>
</div>
</div>
<?php if ($role === 'faculty'): ?>
    <div class="card glass-card mb-4">
        <div class="card-header">
            <i class="fa-solid fa-user-check me-2 text-primary"></i>Faculty Account Access
        </div>
        <div class="card-body">
            <ul class="mb-0">
                <li>Can create online classrooms for assigned courses only.</li>
                <li>Can add or enroll students to online classes.</li>
                <li>Can manage course content, assignments, quizzes, and grades.</li>
                <li>Can conduct live video sessions using a Google Meet link.</li>
                <li>Cannot access courses that are not assigned to the faculty account.</li>
            </ul>
            <div class="mt-3 d-flex flex-wrap gap-2">
                <a href="faculty_classrooms.php" class="btn btn-primary btn-sm"><i class="fa-solid fa-chalkboard me-1"></i>Open
Classrooms</a>
                <a href="faculty_schedule.php" class="btn btn-outline-primary btn-sm"><i class="fa-solid fa-calendar-check me-1"></i>Open My
Schedule</a>
            </div>
        </div>
    </div>
</div>
<?php endif; ?>

<?php if ($role === 'student'): ?>
    <div class="card glass-card mb-4">
        <div class="card-header">
            <i class="fa-solid fa-user-graduate me-2 text-primary"></i>Student Access
        </div>
        <div class="card-body">
            <ul class="mb-0">
                <li>View only the classes where your student account is enrolled.</li>
                <li>Read classroom announcements and course materials from your instructors.</li>
                <li>Answer assignments and quizzes using your student login.</li>
                <li>Review grades and feedback once your instructor evaluates your work.</li>
                <li>Cannot access classrooms or assessments where you are not enrolled.</li>
            </ul>
        </div>
    </div>
</div>

```

```

        </ul>
        <div class="mt-3 d-flex flex-wrap gap-2">
            <a href="student_classrooms.php" class="btn btn-primary btn-sm"><i class="fa-solid fa-user-graduate me-1"></i>Open My
Classes</a>
            <a href="settings.php" class="btn btn-outline-primary btn-sm"><i class="fa-solid fa-key me-1"></i>Change Password</a>
        </div>
    </div>
</div>
<?php endif; ?>

<div class="row g-4">
    <div class="col-lg-7">
        <div class="card glass-card">
            <div class="card-header d-flex justify-content-between align-items-center">
                <span><i class="fa-solid fa-clock me-2 text-primary"></i><?=$showUpcoming ? 'Upcoming classes today (' .
htmlspecialchars($today) . ')': 'Student overview' ?></span>
            </div>
            <div class="card-body p-0">
                <?php if (!$showUpcoming): ?>
                    <div class="p-3">
                        <p class="text-muted mb-2">Use your student account to open enrolled classrooms, receive announcements, and submit
assessment answers.</p>
                        <a href="student_classrooms.php" class="btn btn-outline-primary btn-sm">Go to My Classes</a>
                    </div>
                <?php elseif (!$upcoming): ?>
                    <p class="text-muted p-3 mb-0">No more classes scheduled for today after the current time.</p>
                <?php else: ?>
                    <div class="table-responsive">
                        <table class="table table-hover upcoming-table mb-0">
                            <thead>
                                <tr>
                                    <th>Time</th>
                                    <th>Course</th>
                                    <th>Faculty</th>
                                    <th>Room</th>
                                </tr>
                            </thead>
                            <tbody>
                                <?php foreach ($upcoming as $u): ?>
                                    <tr>
                                        <td><?=$formatTime12h((string) ($u['start_time'] ?? '')) ?> - <?=$
htmlspecialchars($formatTime12h((string) ($u['end_time'] ?? ''))) ?></td>
                                        <td><strong><?=$htmlspecialchars($u['course_code']) ?></strong> <?=$
htmlspecialchars($u['course_name']) ?></td>
                                        <td><?=$htmlspecialchars($u['faculty_name']) ?></td>
                                        <td><?=$htmlspecialchars($u['room_code']) ?></td>
                                    </tr>
                                <?php endforeach; ?>
                            </tbody>
                        </table>
                    </div>
                <?php endif; ?>
            </div>
        </div>
    </div>
    <div class="col-lg-5">
        <div class="card conflict-card">
            <div class="card-header">
                <i class="fa-solid fa-triangle-exclamation me-2 text-warning"></i>Conflict alerts
            </div>
            <div class="card-body">
                <?php if ($openConflicts === 0): ?>
                    <p class="text-success mb-0"><i class="fa-solid fa-check-circle me-1"></i>No pending alerts.</p>
                <?php else: ?>
                    <p class="mb-2">You have <strong><?=$openConflicts ?></strong> pending item(s).</p>
                    <a href="conflicts.php" class="btn btn-outline-warning btn-sm">Review conflicts</a>
                <?php endif; ?>
            </div>
        </div>
    </div>
</div>
</div>
</div>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: DB/faculty_scheduling.sql
-----

```

```

-- phpMyAdmin SQL Dump

```

```

-- version 5.2.1
-- https://www.phpmyadmin.net/
--
-- Host: 127.0.0.1
-- Generation Time: Apr 07, 2026 at 05:30 AM
-- Server version: 10.4.32-MariaDB
-- PHP Version: 8.2.12

SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
START TRANSACTION;
SET time_zone = "+00:00";

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;

--
-- Database: `faculty_scheduling`
--

--
-- Table structure for table `classroom_assessments`
--

CREATE TABLE `classroom_assessments` (
  `id` int(11) NOT NULL,
  `classroom_id` int(11) NOT NULL,
  `faculty_id` int(11) NOT NULL,
  `assessment_type` enum('written_work','performance_task','assignment','quiz') NOT NULL DEFAULT 'written_work',
  `title` varchar(150) NOT NULL,
  `description` text DEFAULT NULL,
  `total_points` decimal(8,2) NOT NULL DEFAULT 100.00,
  `due_at` datetime DEFAULT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for table `classroom_content`
--

CREATE TABLE `classroom_content` (
  `id` int(11) NOT NULL,
  `classroom_id` int(11) NOT NULL,
  `faculty_id` int(11) NOT NULL,
  `content_type` enum('material','link','announcement') NOT NULL DEFAULT 'material',
  `title` varchar(150) NOT NULL,
  `body` text DEFAULT NULL,
  `resource_url` varchar(512) NOT NULL DEFAULT '',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for table `classroom_content_attachments`
--

CREATE TABLE `classroom_content_attachments` (
  `id` int(11) NOT NULL,
  `content_id` int(11) NOT NULL,
  `original_name` varchar(255) NOT NULL DEFAULT '',
  `stored_name` varchar(255) NOT NULL DEFAULT '',
  `mime` varchar(120) NOT NULL DEFAULT '',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for table `classroom_enrollments`
--

CREATE TABLE `classroom_enrollments` (
  `id` int(11) NOT NULL,

```

```

`classroom_id` int(11) NOT NULL,
`student_id` int(11) NOT NULL,
`enrolled_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `classroom_enrollments`
--

INSERT INTO `classroom_enrollments` (`id`, `classroom_id`, `student_id`, `enrolled_at`) VALUES
(3, 1, 2, '2026-04-06 00:44:36');

-- -----

--
-- Table structure for table `classroom_messages`
--

CREATE TABLE `classroom_messages` (
  `id` int(11) NOT NULL,
  `classroom_id` int(11) NOT NULL,
  `sender_user_id` int(11) NOT NULL,
  `body` text NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `classroom_messages`
--

INSERT INTO `classroom_messages` (`id`, `classroom_id`, `sender_user_id`, `body`, `created_at`) VALUES
(1, 1, 10, 'sample ko lng', '2026-04-06 15:31:36'),
(2, 1, 22, 'thank you po mam', '2026-04-06 15:32:33'),
(3, 1, 10, 'dgdgfd', '2026-04-06 16:18:03');

-- -----

--
-- Table structure for table `classroom_scores`
--

CREATE TABLE `classroom_scores` (
  `id` int(11) NOT NULL,
  `assessment_id` int(11) NOT NULL,
  `student_id` int(11) NOT NULL,
  `score` decimal(8,2) DEFAULT NULL,
  `feedback` text DEFAULT NULL,
  `graded_at` datetime DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- -----

--
-- Table structure for table `classroom_students`
--

CREATE TABLE `classroom_students` (
  `id` int(11) NOT NULL,
  `student_number` varchar(30) NOT NULL DEFAULT '',
  `full_name` varchar(120) NOT NULL,
  `email` varchar(190) NOT NULL DEFAULT '',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `user_id` int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `classroom_students`
--

INSERT INTO `classroom_students` (`id`, `student_number`, `full_name`, `email`, `created_at`, `user_id`) VALUES
(1, '1001', 'Francis A Alterado', 'francisalterado@gmail.com', '2026-04-05 09:19:54', 21),
(2, '121', 'boyet A. Alterado', 'boyet@gmail.com', '2026-04-06 00:44:36', 22);

-- -----

--
-- Table structure for table `classroom_submissions`
--

CREATE TABLE `classroom_submissions` (

```



```

`id` int(11) NOT NULL,
`assessment_id` int(11) NOT NULL,
`student_id` int(11) NOT NULL,
`answer_text` longtext DEFAULT NULL,
`status` enum('submitted','resubmitted') NOT NULL DEFAULT 'submitted',
`submitted_at` datetime NOT NULL,
`updated_at` datetime NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for table `colleges`
--

CREATE TABLE `colleges` (
  `id` int(11) NOT NULL,
  `college_code` varchar(20) NOT NULL,
  `college_name` varchar(120) NOT NULL,
  `status` enum('active','inactive') NOT NULL DEFAULT 'active',
  `dean_user_id` int(11) DEFAULT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `colleges`
--

INSERT INTO `colleges` (`id`, `college_code`, `college_name`, `status`, `dean_user_id`, `created_at`) VALUES
(2, 'CAS', 'COLLEGE OF ARTS AND SCIENCES', 'active', NULL, '2026-04-03 02:45:36'),
(3, 'CCJE', 'College of Criminal Justice Education', 'active', NULL, '2026-04-03 05:16:45');

--
-- Table structure for table `college_programs`
--

CREATE TABLE `college_programs` (
  `id` int(11) NOT NULL,
  `college_id` int(11) NOT NULL,
  `program_code` varchar(30) NOT NULL,
  `program_name` varchar(120) NOT NULL DEFAULT '',
  `status` enum('active','inactive') NOT NULL DEFAULT 'active',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `college_programs`
--

INSERT INTO `college_programs` (`id`, `college_id`, `program_code`, `program_name`, `status`, `created_at`) VALUES
(1, 2, 'BSIS', 'BSIS', 'active', '2026-04-03 03:52:46');

--
-- Table structure for table `conflict_logs`
--

CREATE TABLE `conflict_logs` (
  `id` int(11) NOT NULL,
  `schedule_id` int(11) DEFAULT NULL,
  `conflict_type` enum('faculty','room','time') NOT NULL,
  `conflict_description` text NOT NULL,
  `resolved` tinyint(1) NOT NULL DEFAULT 0,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for table `conflict_requests`
--

CREATE TABLE `conflict_requests` (
  `id` int(11) NOT NULL,
  `requested_by` int(11) NOT NULL,
  `college_id` int(11) NOT NULL,
  `faculty_id` int(11) NOT NULL,

```

```

`course_id` int(11) NOT NULL,
`room_id` int(11) NOT NULL,
`scheduled_type` varchar(20) NOT NULL,
`day_of_week` varchar(120) NOT NULL,
`start_time` time NOT NULL,
`end_time` time NOT NULL,
`semester` varchar(20) NOT NULL,
`school_year` varchar(20) NOT NULL,
`academic_year` varchar(20) NOT NULL DEFAULT '',
`reason` text NOT NULL,
`status` enum('pending','approved','rejected') NOT NULL DEFAULT 'pending',
`admin_remarks` text DEFAULT NULL,
`reviewed_by` int(11) DEFAULT NULL,
`reviewed_at` datetime DEFAULT NULL,
`created_at` timestamp NOT NULL DEFAULT current_timestamp(),
`program` varchar(100) NOT NULL DEFAULT '',
`year_level` varchar(20) NOT NULL DEFAULT '',
`section` varchar(20) NOT NULL DEFAULT ''
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

-- -----

```

--
-- Table structure for table `courses`
--

```

```

CREATE TABLE `courses` (
  `id` int(11) NOT NULL,
  `course_code` varchar(20) NOT NULL,
  `course_name` varchar(100) NOT NULL,
  `units` decimal(3,1) NOT NULL DEFAULT 3.0,
  `department` varchar(100) NOT NULL DEFAULT '',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `college_id` int(11) DEFAULT NULL,
  `is_laboratory` tinyint(1) NOT NULL DEFAULT 0,
  `lecture_units` decimal(3,1) NOT NULL DEFAULT 3.0,
  `laboratory_units` decimal(3,1) NOT NULL DEFAULT 0.0,
  `is_gened` tinyint(1) NOT NULL DEFAULT 0,
  `year_level` varchar(20) NOT NULL DEFAULT '',
  `section` varchar(20) NOT NULL DEFAULT ''
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

--
-- Dumping data for table `courses`
--

```

```

INSERT INTO `courses` (`id`, `course_code`, `course_name`, `units`, `department`, `created_at`, `college_id`, `is_laboratory`,
`lecture_units`, `laboratory_units`, `is_gened`, `year_level`, `section`) VALUES
(1, 'IS105', 'FUNDAMENTAL OF IS', 3.0, 'General Education', '2026-03-31 15:36:22', NULL, 0, 3.0, 0.0, 0, '', ''),
(6, 'GE 2', 'Understanding the Self', 3.0, 'General Education', '2026-04-03 05:03:00', NULL, 0, 3.0, 0.0, 1, '1', 'SECTION A'),
(7, 'GE 3', 'Purposive Communication', 3.0, 'General Education', '2026-04-03 05:07:43', NULL, 0, 3.0, 0.0, 1, '1', 'SECTION A'),
(8, 'GE 5', 'Mathematics in the Modern World', 3.0, 'General Education', '2026-04-03 05:08:33', NULL, 0, 3.0, 0.0, 1, '1', 'SECTION A'),
(9, 'GEE 1', 'Gender and Society', 3.0, 'General Education', '2026-04-03 05:10:13', NULL, 0, 3.0, 0.0, 1, '1', 'SECTION A'),
(10, 'GE 10', 'The Life & Works of Rizal', 3.0, 'General Education', '2026-04-03 05:10:51', NULL, 0, 3.0, 0.0, 1, '1', 'SECTION A'),
(12, 'CC104', 'Data Structures & Algorithms', 2.0, 'BS Information Systems', '2026-04-03 05:57:53', 2, 1, 2.0, 1.0, 0, '2', 'SECTION A'),
(13, 'CC101', 'Introduction to Computing', 2.0, 'BS Information Systems', '2026-04-03 06:00:10', 2, 1, 2.0, 1.0, 0, '1', 'SECTION A'),
(14, 'CC102', 'Programming 1 (Fundamentals of Programming)', 2.0, 'BS Information Systems', '2026-04-03 06:04:20', 2, 1, 2.0, 1.0, 0, '1', 'SECTION A'),
(15, 'IS103', 'IT Infrastructure & Network', 2.0, 'BS Information Systems', '2026-04-03 06:07:12', 2, 1, 2.0, 1.0, 0, '2', 'SECTION A'),
(16, 'IS102', 'Professional Issues in IS', 3.0, 'BS Information Systems', '2026-04-03 06:08:43', 2, 0, 3.0, 0.0, 0, '2', 'SECTION A'),
(17, 'DM101', 'Organization & Management', 3.0, 'BS Information Systems', '2026-04-03 06:09:37', 2, 0, 3.0, 0.0, 0, '2', 'SECTION A'),
(18, 'ADV01', 'Enterprise Systems', 3.0, 'BS Information Systems', '2026-04-03 06:10:20', 2, 0, 3.0, 0.0, 0, '2', 'SECTION A'),
(22, 'ADV01', 'Enterprise Systems', 3.0, 'BS Information Systems', '2026-04-03 06:20:26', 2, 0, 3.0, 0.0, 0, '2', 'SECTION B'),
(23, 'CC104', 'Data Structures & Algorithms', 2.0, 'BS Information Systems', '2026-04-03 06:22:20', 2, 1, 2.0, 1.0, 0, '2', 'SECTION B'),
(24, 'IS103', 'IT Infrastructure & Network', 2.0, 'BS Information Systems', '2026-04-03 06:32:02', 2, 1, 2.0, 1.0, 0, '2', 'SECTION B'),
(25, 'IS102', 'Professional Issues in IS', 3.0, 'BS Information Systems', '2026-04-03 06:33:11', 2, 0, 3.0, 0.0, 0, '2', 'SECTION B'),
(26, 'DM101', 'Organization & Management', 3.0, 'BS Information Systems', '2026-04-03 06:33:59', 2, 0, 3.0, 0.0, 0, '2', 'SECTION B'),
(27, 'GE 4', 'Science, Technology & Society', 3.0, 'General Education', '2026-04-03 07:01:36', NULL, 0, 3.0, 0.0, 1, '1', 'SECTION A');

```

-- -----

```

--
-- Table structure for table `dean_activity_logs`
--

```

```

CREATE TABLE `dean_activity_logs` (
  `id` int(11) NOT NULL,
  `dean_user_id` int(11) NOT NULL,
  `college_id` int(11) NOT NULL,
  `action_type` varchar(60) NOT NULL,

```

```

`action_details` text NOT NULL,
`created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

--
-- Dumping data for table `dean_activity_logs`
--

```

```

INSERT INTO `dean_activity_logs` (`id`, `dean_user_id`, `college_id`, `action_type`, `action_details`, `created_at`) VALUES
(32, 2, 2, 'program_create', 'Created program BSIS', '2026-04-03 05:50:51'),
(33, 2, 2, 'program_delete', 'Deleted program ID #1', '2026-04-03 05:51:00'),
(34, 2, 2, 'program_create', 'Created program BS Social Work', '2026-04-03 05:51:18'),
(35, 2, 2, 'program_create', 'Created program BS Information Systems', '2026-04-03 05:51:37'),
(36, 2, 2, 'course_create', 'Created course CC104', '2026-04-03 05:57:53'),
(37, 2, 2, 'course_create', 'Created course CC101', '2026-04-03 06:00:10'),
(38, 2, 2, 'course_create', 'Created course CC102', '2026-04-03 06:04:20'),
(39, 2, 2, 'course_update', 'Updated course ID #14', '2026-04-03 06:04:34'),
(40, 2, 2, 'course_create', 'Created course IS103', '2026-04-03 06:07:12'),
(41, 2, 2, 'course_create', 'Created course IS102', '2026-04-03 06:08:43'),
(42, 2, 2, 'course_create', 'Created course DM101', '2026-04-03 06:09:37'),
(43, 2, 2, 'course_create', 'Created course ADV01', '2026-04-03 06:10:20'),
(44, 2, 2, 'course_create', 'Created course ADV01', '2026-04-03 06:20:26'),
(45, 2, 2, 'course_create', 'Created course CC104', '2026-04-03 06:22:20'),
(46, 2, 2, 'course_create', 'Created course IS103', '2026-04-03 06:32:02'),
(47, 2, 2, 'course_create', 'Created course IS102', '2026-04-03 06:33:11'),
(48, 2, 2, 'course_create', 'Created course DM101', '2026-04-03 06:33:59'),
(49, 2, 2, 'faculty_create', 'Created faculty IS01', '2026-04-03 14:10:26'),
(50, 2, 2, 'faculty_create', 'Created faculty IS02', '2026-04-03 14:11:15'),
(51, 2, 2, 'faculty_create', 'Created faculty IS03', '2026-04-03 14:12:09'),
(52, 2, 2, 'faculty_create', 'Created faculty IS04', '2026-04-03 14:38:52'),
(53, 2, 2, 'faculty_create', 'Created faculty IS05', '2026-04-03 14:40:24'),
(54, 2, 2, 'faculty_create', 'Created faculty IS06', '2026-04-03 14:41:11'),
(55, 2, 2, 'faculty_create', 'Created faculty IS07', '2026-04-03 14:42:18'),
(56, 2, 2, 'schedule_create', 'Created schedule entries #22, 23', '2026-04-03 14:45:30'),
(57, 2, 2, 'schedule_create', 'Created schedule entries #24', '2026-04-03 14:47:51'),
(58, 2, 2, 'schedule_change_review', 'Reviewed faculty schedule change request #1: approved', '2026-04-03 23:26:50'),
(59, 2, 2, 'schedule_delete', 'Deleted schedule #23', '2026-04-04 03:56:14'),
(60, 2, 2, 'schedule_create', 'Created schedule entries #25, 26', '2026-04-04 04:16:01'),
(61, 2, 2, 'schedule_delete', 'Deleted schedule #22', '2026-04-04 04:25:55'),
(62, 2, 2, 'schedule_create', 'Created schedule entries #27, 28', '2026-04-04 04:28:47'),
(63, 2, 2, 'schedule_update', 'Updated schedule #26', '2026-04-04 04:53:31'),
(64, 2, 2, 'schedule_update', 'Updated schedule #27', '2026-04-04 04:58:08'),
(65, 2, 2, 'program_chair_create', 'Created program chair for BS Information Systems', '2026-04-04 13:19:26'),
(66, 2, 2, 'schedule_delete', 'Deleted schedule #26', '2026-04-04 13:38:48'),
(67, 2, 2, 'schedule_delete', 'Deleted schedule #25', '2026-04-04 13:38:50'),
(68, 2, 2, 'schedule_delete', 'Deleted schedule #24', '2026-04-04 13:38:52'),
(69, 2, 2, 'schedule_delete', 'Deleted schedule #27', '2026-04-04 13:38:54'),
(70, 2, 2, 'schedule_delete', 'Deleted schedule #28', '2026-04-04 13:38:56');

```

```

-- -----

```

```

--
-- Table structure for table `faculty`
--

```

```

CREATE TABLE `faculty` (
  `id` int(11) NOT NULL,
  `faculty_id` varchar(20) NOT NULL,
  `full_name` varchar(100) NOT NULL,
  `department` varchar(100) NOT NULL DEFAULT '',
  `email` varchar(100) NOT NULL DEFAULT '',
  `max_hours_per_day` int(11) NOT NULL DEFAULT 8,
  `status` enum('active','inactive') NOT NULL DEFAULT 'active',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `user_id` int(11) DEFAULT NULL,
  `college_id` int(11) DEFAULT NULL,
  `is_gened` tinyint(1) NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

--
-- Dumping data for table `faculty`
--

```

```

INSERT INTO `faculty` (`id`, `faculty_id`, `full_name`, `department`, `email`, `max_hours_per_day`, `status`, `created_at`, `user_id`,
`college_id`, `is_gened`) VALUES
(1, '10001', 'Francis Alterado', 'General Education', '', 8, 'active', '2026-03-31 15:35:29', NULL, NULL, 0),
(3, '123', 'Juan Dela Cruz KO', 'BSIS', '', 16, 'active', '2026-04-02 23:53:29', 6, NULL, 0),
(5, '01', 'Instructor 1', 'General Education', '', 8, 'active', '2026-04-03 05:23:44', 10, NULL, 1),
(6, '02', 'Instructor 2', 'General Education', '', 8, 'active', '2026-04-03 05:24:12', 11, NULL, 1),
(7, 'IS01', 'Acosta, Christine Joy M.', 'BS Information Systems', '', 18, 'active', '2026-04-03 14:10:26', 12, 2, 0),

```

```
(8, 'IS02', 'Elijah A. Fresnillo', 'BS Information Systems', '', 18, 'active', '2026-04-03 14:11:15', 13, 2, 0),
(9, 'IS03', 'Santiago, Tristan Rex E.', 'BS Information Systems', '', 18, 'active', '2026-04-03 14:12:09', 14, 2, 0),
(10, 'IS04', 'Recalde, Jomari B.', 'BS Information Systems', '', 18, 'active', '2026-04-03 14:38:52', 15, 2, 0),
(11, 'IS05', 'Echague, Enkie Carl B.', 'BS Information Systems', '', 18, 'active', '2026-04-03 14:40:24', 16, 2, 0),
(12, 'IS06', 'Golez, Hannah Mae L.', 'BS Information Systems', '', 18, 'active', '2026-04-03 14:41:11', 17, 2, 0),
(13, 'IS07', 'Golez, Harmann', 'BS Information Systems', '', 18, 'active', '2026-04-03 14:42:18', 18, 2, 0);
```

```
-- -----
```

```
--
-- Table structure for table `faculty_specializations`
--
```

```
CREATE TABLE `faculty_specializations` (
  `id` int(11) NOT NULL,
  `faculty_id` int(11) NOT NULL,
  `course_id` int(11) NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- -----
```

```
--
-- Table structure for table `ge_course_colleges`
--
```

```
CREATE TABLE `ge_course_colleges` (
  `id` int(11) NOT NULL,
  `course_id` int(11) NOT NULL,
  `college_id` int(11) NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
--
-- Dumping data for table `ge_course_colleges`
--
```

```
INSERT INTO `ge_course_colleges` (`id`, `course_id`, `college_id`, `created_at`) VALUES
(3, 6, 3, '2026-04-03 05:19:40'),
(4, 7, 3, '2026-04-03 05:19:48'),
(6, 8, 3, '2026-04-03 05:20:11'),
(7, 9, 3, '2026-04-03 05:20:18'),
(8, 27, 2, '2026-04-03 07:05:15'),
(9, 10, 2, '2026-04-03 07:07:32');
```

```
-- -----
```

```
--
-- Table structure for table `ge_schedule_targets`
--
```

```
CREATE TABLE `ge_schedule_targets` (
  `id` int(11) NOT NULL,
  `schedule_id` int(11) NOT NULL,
  `college_id` int(11) NOT NULL,
  `program_name` varchar(120) NOT NULL,
  `year_level` varchar(20) NOT NULL DEFAULT '',
  `section` varchar(20) NOT NULL DEFAULT '',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
--
-- Dumping data for table `ge_schedule_targets`
--
```

```
INSERT INTO `ge_schedule_targets` (`id`, `schedule_id`, `college_id`, `program_name`, `year_level`, `section`, `created_at`) VALUES
(1, 29, 2, 'BS Information Systems', '1', 'SECTION A', '2026-04-04 13:39:53');
```

```
-- -----
```

```
--
-- Table structure for table `internal_messages`
--
```

```
CREATE TABLE `internal_messages` (
  `id` int(11) NOT NULL,
  `sender_user_id` int(11) NOT NULL,
  `recipient_user_id` int(11) NOT NULL,
  `body` text NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
```

```

`read_at` timestamp NULL DEFAULT NULL,
`subject` varchar(255) NOT NULL DEFAULT '',
`is_memo` tinyint(1) NOT NULL DEFAULT 0,
`attachment_original_name` varchar(255) NOT NULL DEFAULT '',
`attachment_stored_name` varchar(255) NOT NULL DEFAULT '',
`attachment_mime` varchar(120) NOT NULL DEFAULT ''
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `internal_messages`
--

INSERT INTO `internal_messages` (`id`, `sender_user_id`, `recipient_user_id`, `body`, `created_at`, `read_at`, `subject`, `is_memo`,
`attachment_original_name`, `attachment_stored_name`, `attachment_mime`) VALUES
(1, 2, 12, 'hi', '2026-04-03 14:58:52', '2026-04-03 14:59:11', '', 0, '', '', ''),
(2, 12, 2, 'yes sir', '2026-04-03 14:59:20', '2026-04-03 14:59:35', '', 0, '', '', '');

--
-- Table structure for table `online_classrooms`
--

CREATE TABLE `online_classrooms` (
  `id` int(11) NOT NULL,
  `schedule_id` int(11) NOT NULL,
  `faculty_id` int(11) NOT NULL,
  `course_id` int(11) NOT NULL,
  `title` varchar(150) NOT NULL,
  `description` text DEFAULT NULL,
  `meet_link` varchar(512) NOT NULL DEFAULT '',
  `status` enum('active','archived') NOT NULL DEFAULT 'active',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
  `written_work_percentage` decimal(5,2) NOT NULL DEFAULT 50.00,
  `performance_task_percentage` decimal(5,2) NOT NULL DEFAULT 50.00
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `online_classrooms`
--

INSERT INTO `online_classrooms` (`id`, `schedule_id`, `faculty_id`, `course_id`, `title`, `description`, `meet_link`, `status`,
`created_at`, `updated_at`, `written_work_percentage`, `performance_task_percentage`) VALUES
(1, 29, 5, 10, 'GE 10 - The Life & Works of Rizal', 'cvxcxvcb', 'https://meet.google.com/mdw-jkui-sov', 'active', '2026-04-05 09:17:27',
'2026-04-06 00:49:13', 50.00, 50.00);

--
-- Table structure for table `programs`
--

CREATE TABLE `programs` (
  `id` int(11) NOT NULL,
  `college_id` int(11) NOT NULL,
  `program_name` varchar(120) NOT NULL,
  `status` enum('active','inactive') NOT NULL DEFAULT 'active',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `programs`
--

INSERT INTO `programs` (`id`, `college_id`, `program_name`, `status`, `created_at`) VALUES
(2, 2, 'BS Social Work', 'active', '2026-04-03 05:51:18'),
(3, 2, 'BS Information Systems', 'active', '2026-04-03 05:51:37');

--
-- Table structure for table `program_sections`
--

CREATE TABLE `program_sections` (
  `id` int(11) NOT NULL,
  `year_level_id` int(11) NOT NULL,
  `section_name` varchar(20) NOT NULL,
  `status` enum('active','inactive') NOT NULL DEFAULT 'active',

```

```

`created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `program_sections`
--

INSERT INTO `program_sections` (`id`, `year_level_id`, `section_name`, `status`, `created_at`) VALUES
(1, 1, 'SECTION 1', 'active', '2026-04-03 03:53:07');

-- -----

--
-- Table structure for table `program_year_levels`
--

CREATE TABLE `program_year_levels` (
  `id` int(11) NOT NULL,
  `program_id` int(11) NOT NULL,
  `year_level` varchar(20) NOT NULL,
  `status` enum('active','inactive') NOT NULL DEFAULT 'active',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `program_year_levels`
--

INSERT INTO `program_year_levels` (`id`, `program_id`, `year_level`, `status`, `created_at`) VALUES
(1, 1, '1', 'active', '2026-04-03 03:52:55');

-- -----

--
-- Table structure for table `rooms`
--

CREATE TABLE `rooms` (
  `id` int(11) NOT NULL,
  `room_code` varchar(20) NOT NULL,
  `room_name` varchar(50) NOT NULL DEFAULT '',
  `capacity` int(11) NOT NULL DEFAULT 0,
  `type` enum('lecture','laboratory','conference','tba') NOT NULL DEFAULT 'lecture',
  `status` enum('available','tba','maintenance') NOT NULL DEFAULT 'available',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `college_id` int(11) DEFAULT NULL,
  `is_generated` tinyint(1) NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Dumping data for table `rooms`
--

INSERT INTO `rooms` (`id`, `room_code`, `room_name`, `capacity`, `type`, `status`, `created_at`, `college_id`, `is_generated`) VALUES
(1, '101', 'CCD', 50, 'lecture', 'available', '2026-03-31 15:38:00', NULL, 0),
(7, 'RM 101', 'CCD - RM 101', 50, 'lecture', 'available', '2026-04-03 02:49:07', NULL, 1),
(8, 'RM 102', 'CCD - RM 102', 30, 'lecture', 'available', '2026-04-03 07:28:17', NULL, 1),
(9, 'RM 103', 'CCD - RM 103', 30, 'lecture', 'tba', '2026-04-03 07:45:44', NULL, 1);

-- -----

--
-- Table structure for table `schedules`
--

CREATE TABLE `schedules` (
  `id` int(11) NOT NULL,
  `faculty_id` int(11) NOT NULL,
  `course_id` int(11) NOT NULL,
  `room_id` int(11) NOT NULL,
  `schedule_type` enum('MW', 'TTH', 'MWF', 'TTHS', 'Saturday', 'Sunday', 'MW_TTH', 'Custom') NOT NULL DEFAULT 'Custom',
  `day_of_week` set('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday') NOT NULL,
  `start_time` time NOT NULL,
  `end_time` time NOT NULL,
  `semester` varchar(20) NOT NULL,
  `school_year` varchar(20) NOT NULL,
  `academic_year` varchar(20) NOT NULL DEFAULT '',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `created_by` int(11) NOT NULL,
  `college_id` int(11) DEFAULT NULL,

```

```

`program` varchar(100) NOT NULL DEFAULT '',
`year_level` varchar(20) NOT NULL DEFAULT '',
`section` varchar(20) NOT NULL DEFAULT '',
`online_class_url` varchar(512) DEFAULT NULL,
`online_live_at` datetime DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

--
-- Dumping data for table `schedules`
--

```

```

INSERT INTO `schedules` (`id`, `faculty_id`, `course_id`, `room_id`, `schedule_type`, `day_of_week`, `start_time`, `end_time`, `semester`,
`school_year`, `academic_year`, `created_at`, `created_by`, `college_id`, `program`, `year_level`, `section`, `online_class_url`,
`online_live_at`) VALUES
(29, 5, 10, 7, 'MW', 'Monday,Wednesday', '08:00:00', '10:00:00', '1st Semester', '2026-2027', '', '2026-04-04 13:39:53', 7, 2, '', '', '',
'https://meet.google.com/mdw-jkui-sov', NULL);

```

```

-----

```

```

--
-- Table structure for table `schedule_change_requests`
--

```

```

CREATE TABLE `schedule_change_requests` (
  `id` int(11) NOT NULL,
  `faculty_user_id` int(11) NOT NULL,
  `schedule_id` int(11) NOT NULL,
  `message` text NOT NULL,
  `status` enum('pending','approved','rejected') NOT NULL DEFAULT 'pending',
  `dean_remarks` text DEFAULT NULL,
  `reviewed_by` int(11) DEFAULT NULL,
  `reviewed_at` datetime DEFAULT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-----

```

```

--
-- Table structure for table `users`
--

```

```

CREATE TABLE `users` (
  `id` int(11) NOT NULL,
  `username` varchar(50) NOT NULL,
  `password` varchar(255) NOT NULL,
  `full_name` varchar(100) NOT NULL,
  `role` enum('admin','dean','program_chair','faculty','gened','student') NOT NULL DEFAULT 'faculty',
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `college_id` int(11) DEFAULT NULL,
  `is_active` tinyint(1) NOT NULL DEFAULT 1,
  `email` varchar(190) NOT NULL DEFAULT '',
  `assigned_program` varchar(120) NOT NULL DEFAULT ''
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

--
-- Dumping data for table `users`
--

```

```

INSERT INTO `users` (`id`, `username`, `password`, `full_name`, `role`, `created_at`, `college_id`, `is_active`, `email`,
`assigned_program`) VALUES
(1, 'admin', '$2y$10$usFbHAW1b0nQyne.zYkQa0TuF1gl/NrfmAUB9p0E4fgn/kQy6Yab6', 'System Administrator', 'admin', '2026-03-31 15:14:18', NULL,
1, '', ''),
(2, 'boyet', '$2y$10$B7EUHcv35TVfCoz9/Yzfm.wc5ur8V13M7cxVNXvNRopyKYEmNseVS', 'Francis Alterado', 'dean', '2026-04-02 14:46:50', 2, 1, '',
''),
(3, 'juan', '$2y$10$N1jSzMa04WonbxUutV4vX.YIsdyESuBWHkMps.PkVm0AQpKCqWz12', 'Juan Dela Cruz', 'faculty', '2026-04-02 23:13:31', NULL, 1, '',
''),
(6, 'juank0', '$2y$10$im91niMRR/KZkw7zm3wuvuee1Wn73C0gGxjlACs9Gr3JAAZ0NiIG/m', 'Juan Dela Cruz K0', 'faculty', '2026-04-02 23:53:29', NULL,
1, '', ''),
(7, 'Gen', '$2y$10$GByRMkRA9nY5Cb64gwk5Su00Spw.7HKAjo/YmZszPFxG9KTZ16yfe', 'Jem', 'gened', '2026-04-03 02:40:37', NULL, 1, '', ''),
(9, 'pili', '$2y$10$LabZX3X38zrT.NOrfU03IObPST2v4hcBHQOgrpzoxsozwKUM9ciUy', 'Dr. Sheila Lou G. Pili', 'dean', '2026-04-03 05:15:54', 3, 1,
'', ''),
(10, 'sample1', '$2y$10$xnKm7gfpj/a2TfJYAg8SNOUKiMSIexZ8aY94s7iirQ7jlbrx2fK0q', 'Instructor 1', 'faculty', '2026-04-03 05:23:44', NULL, 1,
'', ''),
(11, 'sample2', '$2y$10$Sygvcwe0A3eBW9A6BmticugerNmgrwXyPi7T1DZq0.fC5S4gOGtwG', 'Instructor 2', 'faculty', '2026-04-03 05:24:12', NULL, 1,
'', ''),
(12, 'acosta', '$2y$10$mvU3AskpnxsJgXpGewzhE.sHqzb2wSXRaTdRujy9rwnNrQ8wBGG3q6', 'Acosta, Christine Joy M.', 'faculty', '2026-04-03 14:10:26',
2, 1, '', ''),
(13, 'Elijah', '$2y$10$4hh.xMTwg8rT650MYGvgf.CeAZ.X4umcT2J69Ut6G6NkZzXSCzUIu', 'Elijah A. Fresnillo', 'faculty', '2026-04-03 14:11:15', 2,
1, '', ''),
(14, 'Tristan', '$2y$10$47L5WITKyYB6ajzLpal1g.RRyijYPqx0Hkd4p671zbMyLdO0wzxTq', 'Santiago, Tristan Rex E.', 'faculty', '2026-04-03

```

```

14:12:09', 2, 1, '', ''),
(15, 'Jomari', '$2y$10$n9Ih0xMGLVw1baP.NNEeAeD0cXUFVh1NqGJXQiGIeo/h41gGZD05q', 'Recalde, Jomari B.', 'faculty', '2026-04-03 14:38:52', 2, 1,
'', ''),
(16, 'Carl', '$2y$10$X4dle0YZfovzbd8NuFEbh.ov15uIgJjLHeB8blQxjic6z3QIWuUSu', 'Echague, Enkie Carl B.', 'faculty', '2026-04-03 14:40:24', 2,
1, '', ''),
(17, 'Hannah', '$2y$10$b4EaU4LJ/7W0stnhgpwXP.I04iPYt7xZgufoHx320roeEqr2tiA/O', 'Golez,Hannah Mae L.', 'faculty', '2026-04-03 14:41:11', 2,
1, '', ''),
(18, 'Harmann', '$2y$10$NgNUCwmpRAsDfw0N2ESpveHTGawPM9FH4e1TN6ZQuj20Lqoc/746S', 'Golez, Harmann', 'faculty', '2026-04-03 14:42:18', 2, 1,
'', ''),
(20, 'tin', '$2y$10$ZfxRTXia.uDvUo60Mf3190S1rQg9quPlBSh4ChXXjbQc9u0Al4gje', 'Christine Acosta', 'program_chair', '2026-04-04 13:19:26', 2,
1, 'francisalterado@gmail.com', 'BS Information Systems'),
(21, 'student1', '$2y$10$WvnNF0gQvc7yhxv8Qquu4e9T1LDXBZvapbzqGtPbDdCkeK7piUVgm', 'Francis A Alterado', 'student', '2026-04-05 10:22:14', 2,
1, 'francisalterado@gmail.com', ''),
(22, 'boyet1', '$2y$10$rTiQPhoOQQS.t/0bdpWPn01scpBsUCJaxRtMSjugU9NKqgGneEEJC', 'boyet A. Alterado', 'student', '2026-04-06 00:44:36', 2, 1,
'boyet@gmail.com', '');

```

```

--
-- Indexes for dumped tables
--

```

```

--
-- Indexes for table `classroom_assessments`
--

```

```

ALTER TABLE `classroom_assessments`
  ADD PRIMARY KEY (`id`),
  ADD KEY `fk_ca_classroom` (`classroom_id`),
  ADD KEY `fk_ca_faculty` (`faculty_id`);

```

```

--
-- Indexes for table `classroom_content`
--

```

```

ALTER TABLE `classroom_content`
  ADD PRIMARY KEY (`id`),
  ADD KEY `fk_cc_classroom` (`classroom_id`),
  ADD KEY `fk_cc_faculty` (`faculty_id`);

```

```

--
-- Indexes for table `classroom_content_attachments`
--

```

```

ALTER TABLE `classroom_content_attachments`
  ADD PRIMARY KEY (`id`),
  ADD KEY `idx_cca_content` (`content_id`);

```

```

--
-- Indexes for table `classroom_enrollments`
--

```

```

ALTER TABLE `classroom_enrollments`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `uq_classroom_student` (`classroom_id`,`student_id`),
  ADD KEY `fk_ce_student` (`student_id`);

```

```

--
-- Indexes for table `classroom_messages`
--

```

```

ALTER TABLE `classroom_messages`
  ADD PRIMARY KEY (`id`),
  ADD KEY `idx_cm_classroom_time` (`classroom_id`,`created_at`),
  ADD KEY `fk_cm_sender` (`sender_user_id`);

```

```

--
-- Indexes for table `classroom_scores`
--

```

```

ALTER TABLE `classroom_scores`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `uq_assessment_student` (`assessment_id`,`student_id`),
  ADD KEY `fk_cs_student` (`student_id`);

```

```

--
-- Indexes for table `classroom_students`
--

```

```

ALTER TABLE `classroom_students`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `user_id` (`user_id`);

```

```

--
-- Indexes for table `classroom_submissions`
--

```

```

ALTER TABLE `classroom_submissions`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `uq_submission_assessment_student` (`assessment_id`,`student_id`),

```



```

    ADD KEY `fk_csub_student` (`student_id`);

--
-- Indexes for table `colleges`
--
ALTER TABLE `colleges`
    ADD PRIMARY KEY (`id`),
    ADD UNIQUE KEY `college_code` (`college_code`);

--
-- Indexes for table `college_programs`
--
ALTER TABLE `college_programs`
    ADD PRIMARY KEY (`id`),
    ADD UNIQUE KEY `uq_college_program_code` (`college_id`, `program_code`);

--
-- Indexes for table `conflict_logs`
--
ALTER TABLE `conflict_logs`
    ADD PRIMARY KEY (`id`),
    ADD KEY `fk_conf_sched` (`schedule_id`);

--
-- Indexes for table `conflict_requests`
--
ALTER TABLE `conflict_requests`
    ADD PRIMARY KEY (`id`),
    ADD KEY `fk_cr_user` (`requested_by`),
    ADD KEY `fk_cr_college` (`college_id`),
    ADD KEY `fk_cr_faculty` (`faculty_id`),
    ADD KEY `fk_cr_course` (`course_id`),
    ADD KEY `fk_cr_room` (`room_id`),
    ADD KEY `fk_cr_reviewer` (`reviewed_by`);

--
-- Indexes for table `courses`
--
ALTER TABLE `courses`
    ADD PRIMARY KEY (`id`),
    ADD KEY `fk_course_college` (`college_id`),
    ADD KEY `idx_course_code` (`course_code`);

--
-- Indexes for table `dean_activity_logs`
--
ALTER TABLE `dean_activity_logs`
    ADD PRIMARY KEY (`id`),
    ADD KEY `fk_dal_user` (`dean_user_id`),
    ADD KEY `fk_dal_college` (`college_id`);

--
-- Indexes for table `faculty`
--
ALTER TABLE `faculty`
    ADD PRIMARY KEY (`id`),
    ADD UNIQUE KEY `faculty_id` (`faculty_id`),
    ADD UNIQUE KEY `user_id` (`user_id`),
    ADD KEY `fk_faculty_college` (`college_id`);

--
-- Indexes for table `faculty_specializations`
--
ALTER TABLE `faculty_specializations`
    ADD PRIMARY KEY (`id`),
    ADD UNIQUE KEY `uq_faculty_course` (`faculty_id`, `course_id`),
    ADD KEY `fk_fs_course` (`course_id`);

--
-- Indexes for table `ge_course_colleges`
--
ALTER TABLE `ge_course_colleges`
    ADD PRIMARY KEY (`id`),
    ADD UNIQUE KEY `uq_ge_course_college` (`course_id`, `college_id`),
    ADD KEY `fk_gecc_college` (`college_id`);

--
-- Indexes for table `ge_schedule_targets`
--
ALTER TABLE `ge_schedule_targets`

```

```

ADD PRIMARY KEY (`id`),
ADD UNIQUE KEY `schedule_id` (`schedule_id`),
ADD KEY `idx_ge_target_scope` (`college_id`,`program_name`,`year_level`,`section`);

--
-- Indexes for table `internal_messages`
--
ALTER TABLE `internal_messages`
  ADD PRIMARY KEY (`id`),
  ADD KEY `idx_im_recipient_unread` (`recipient_user_id`,`read_at`),
  ADD KEY `idx_im_pair_time` (`sender_user_id`,`recipient_user_id`,`created_at`);

--
-- Indexes for table `online_classrooms`
--
ALTER TABLE `online_classrooms`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `schedule_id` (`schedule_id`),
  ADD KEY `fk_oc_faculty` (`faculty_id`),
  ADD KEY `fk_oc_course` (`course_id`);

--
-- Indexes for table `programs`
--
ALTER TABLE `programs`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `uq_program_college_name` (`college_id`,`program_name`);

--
-- Indexes for table `program_sections`
--
ALTER TABLE `program_sections`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `uq_year_section` (`year_level_id`,`section_name`);

--
-- Indexes for table `program_year_levels`
--
ALTER TABLE `program_year_levels`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `uq_program_year` (`program_id`,`year_level`);

--
-- Indexes for table `rooms`
--
ALTER TABLE `rooms`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `room_code` (`room_code`),
  ADD KEY `fk_room_college` (`college_id`);

--
-- Indexes for table `schedules`
--
ALTER TABLE `schedules`
  ADD PRIMARY KEY (`id`),
  ADD KEY `fk_sched_faculty` (`faculty_id`),
  ADD KEY `fk_sched_course` (`course_id`),
  ADD KEY `fk_sched_room` (`room_id`),
  ADD KEY `fk_sched_user` (`created_by`),
  ADD KEY `fk_sched_college` (`college_id`);

--
-- Indexes for table `schedule_change_requests`
--
ALTER TABLE `schedule_change_requests`
  ADD PRIMARY KEY (`id`),
  ADD KEY `fk_scr_user` (`faculty_user_id`),
  ADD KEY `fk_scr_sched` (`schedule_id`),
  ADD KEY `fk_scr_reviewer` (`reviewed_by`);

--
-- Indexes for table `users`
--
ALTER TABLE `users`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `username` (`username`),
  ADD KEY `fk_users_college` (`college_id`);

--
-- AUTO_INCREMENT for dumped tables

```

```
--
--
-- AUTO_INCREMENT for table `classroom_assessments`
--
ALTER TABLE `classroom_assessments`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=3;
--
-- AUTO_INCREMENT for table `classroom_content`
--
ALTER TABLE `classroom_content`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=3;
--
-- AUTO_INCREMENT for table `classroom_content_attachments`
--
ALTER TABLE `classroom_content_attachments`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;
--
-- AUTO_INCREMENT for table `classroom_enrollments`
--
ALTER TABLE `classroom_enrollments`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;
--
-- AUTO_INCREMENT for table `classroom_messages`
--
ALTER TABLE `classroom_messages`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;
--
-- AUTO_INCREMENT for table `classroom_scores`
--
ALTER TABLE `classroom_scores`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;
--
-- AUTO_INCREMENT for table `classroom_students`
--
ALTER TABLE `classroom_students`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=3;
--
-- AUTO_INCREMENT for table `classroom_submissions`
--
ALTER TABLE `classroom_submissions`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;
--
-- AUTO_INCREMENT for table `colleges`
--
ALTER TABLE `colleges`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;
--
-- AUTO_INCREMENT for table `college_programs`
--
ALTER TABLE `college_programs`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;
--
-- AUTO_INCREMENT for table `conflict_logs`
--
ALTER TABLE `conflict_logs`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;
--
-- AUTO_INCREMENT for table `conflict_requests`
--
ALTER TABLE `conflict_requests`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;
--
-- AUTO_INCREMENT for table `courses`
--
ALTER TABLE `courses`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=29;
```

```

--
-- AUTO_INCREMENT for table `dean_activity_logs`
--
ALTER TABLE `dean_activity_logs`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=71;

--
-- AUTO_INCREMENT for table `faculty`
--
ALTER TABLE `faculty`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=14;

--
-- AUTO_INCREMENT for table `faculty_specializations`
--
ALTER TABLE `faculty_specializations`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;

--
-- AUTO_INCREMENT for table `ge_course_colleges`
--
ALTER TABLE `ge_course_colleges`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=10;

--
-- AUTO_INCREMENT for table `ge_schedule_targets`
--
ALTER TABLE `ge_schedule_targets`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;

--
-- AUTO_INCREMENT for table `internal_messages`
--
ALTER TABLE `internal_messages`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=5;

--
-- AUTO_INCREMENT for table `online_classrooms`
--
ALTER TABLE `online_classrooms`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;

--
-- AUTO_INCREMENT for table `programs`
--
ALTER TABLE `programs`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;

--
-- AUTO_INCREMENT for table `program_sections`
--
ALTER TABLE `program_sections`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;

--
-- AUTO_INCREMENT for table `program_year_levels`
--
ALTER TABLE `program_year_levels`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;

--
-- AUTO_INCREMENT for table `rooms`
--
ALTER TABLE `rooms`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=10;

--
-- AUTO_INCREMENT for table `schedules`
--
ALTER TABLE `schedules`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=30;

--
-- AUTO_INCREMENT for table `schedule_change_requests`
--
ALTER TABLE `schedule_change_requests`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;

--
-- AUTO_INCREMENT for table `users`

```

```

--
ALTER TABLE `users`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=23;

--
-- Constraints for dumped tables
--

--
-- Constraints for table `classroom_assessments`
--
ALTER TABLE `classroom_assessments`
  ADD CONSTRAINT `fk_ca_classroom` FOREIGN KEY (`classroom_id`) REFERENCES `online_classrooms` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_ca_faculty` FOREIGN KEY (`faculty_id`) REFERENCES `faculty` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `classroom_content`
--
ALTER TABLE `classroom_content`
  ADD CONSTRAINT `fk_cc_classroom` FOREIGN KEY (`classroom_id`) REFERENCES `online_classrooms` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_cc_faculty` FOREIGN KEY (`faculty_id`) REFERENCES `faculty` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `classroom_content_attachments`
--
ALTER TABLE `classroom_content_attachments`
  ADD CONSTRAINT `fk_cca_content` FOREIGN KEY (`content_id`) REFERENCES `classroom_content` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `classroom_enrollments`
--
ALTER TABLE `classroom_enrollments`
  ADD CONSTRAINT `fk_ce_classroom` FOREIGN KEY (`classroom_id`) REFERENCES `online_classrooms` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_ce_student` FOREIGN KEY (`student_id`) REFERENCES `classroom_students` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `classroom_messages`
--
ALTER TABLE `classroom_messages`
  ADD CONSTRAINT `fk_cm_classroom` FOREIGN KEY (`classroom_id`) REFERENCES `online_classrooms` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_cm_sender` FOREIGN KEY (`sender_user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `classroom_scores`
--
ALTER TABLE `classroom_scores`
  ADD CONSTRAINT `fk_cs_assessment` FOREIGN KEY (`assessment_id`) REFERENCES `classroom_assessments` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_cs_student` FOREIGN KEY (`student_id`) REFERENCES `classroom_students` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `classroom_students`
--
ALTER TABLE `classroom_students`
  ADD CONSTRAINT `fk_cls_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE SET NULL;

--
-- Constraints for table `classroom_submissions`
--
ALTER TABLE `classroom_submissions`
  ADD CONSTRAINT `fk_csub_assessment` FOREIGN KEY (`assessment_id`) REFERENCES `classroom_assessments` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_csub_student` FOREIGN KEY (`student_id`) REFERENCES `classroom_students` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `college_programs`
--
ALTER TABLE `college_programs`
  ADD CONSTRAINT `fk_cp_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `conflict_logs`
--
ALTER TABLE `conflict_logs`
  ADD CONSTRAINT `fk_conf_sched` FOREIGN KEY (`schedule_id`) REFERENCES `schedules` (`id`) ON DELETE SET NULL;

--
-- Constraints for table `conflict_requests`
--
ALTER TABLE `conflict_requests`
  ADD CONSTRAINT `fk_cr_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_cr_course` FOREIGN KEY (`course_id`) REFERENCES `courses` (`id`) ON DELETE CASCADE,

```

```

ADD CONSTRAINT `fk_cr_faculty` FOREIGN KEY (`faculty_id`) REFERENCES `faculty` (`id`) ON DELETE CASCADE,
ADD CONSTRAINT `fk_cr_reviewer` FOREIGN KEY (`reviewed_by`) REFERENCES `users` (`id`) ON DELETE SET NULL,
ADD CONSTRAINT `fk_cr_room` FOREIGN KEY (`room_id`) REFERENCES `rooms` (`id`) ON DELETE CASCADE,
ADD CONSTRAINT `fk_cr_user` FOREIGN KEY (`requested_by`) REFERENCES `users` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `courses`
--
ALTER TABLE `courses`
  ADD CONSTRAINT `fk_course_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE SET NULL;

--
-- Constraints for table `dean_activity_logs`
--
ALTER TABLE `dean_activity_logs`
  ADD CONSTRAINT `fk_dal_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_dal_user` FOREIGN KEY (`dean_user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `faculty`
--
ALTER TABLE `faculty`
  ADD CONSTRAINT `fk_faculty_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE SET NULL,
  ADD CONSTRAINT `fk_faculty_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE SET NULL;

--
-- Constraints for table `faculty_specializations`
--
ALTER TABLE `faculty_specializations`
  ADD CONSTRAINT `fk_fs_course` FOREIGN KEY (`course_id`) REFERENCES `courses` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_fs_faculty` FOREIGN KEY (`faculty_id`) REFERENCES `faculty` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `ge_course_colleges`
--
ALTER TABLE `ge_course_colleges`
  ADD CONSTRAINT `fk_gecc_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_gecc_course` FOREIGN KEY (`course_id`) REFERENCES `courses` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `ge_schedule_targets`
--
ALTER TABLE `ge_schedule_targets`
  ADD CONSTRAINT `fk_gst_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_gst_schedule` FOREIGN KEY (`schedule_id`) REFERENCES `schedules` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `internal_messages`
--
ALTER TABLE `internal_messages`
  ADD CONSTRAINT `fk_im_recipient` FOREIGN KEY (`recipient_user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_im_sender` FOREIGN KEY (`sender_user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `online_classrooms`
--
ALTER TABLE `online_classrooms`
  ADD CONSTRAINT `fk_oc_course` FOREIGN KEY (`course_id`) REFERENCES `courses` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_oc_faculty` FOREIGN KEY (`faculty_id`) REFERENCES `faculty` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_oc_schedule` FOREIGN KEY (`schedule_id`) REFERENCES `schedules` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `programs`
--
ALTER TABLE `programs`
  ADD CONSTRAINT `fk_program_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `program_sections`
--
ALTER TABLE `program_sections`
  ADD CONSTRAINT `fk_ps_year` FOREIGN KEY (`year_level_id`) REFERENCES `program_year_levels` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `program_year_levels`
--
ALTER TABLE `program_year_levels`
  ADD CONSTRAINT `fk_pyl_program` FOREIGN KEY (`program_id`) REFERENCES `college_programs` (`id`) ON DELETE CASCADE;

--

```

```

-- Constraints for table `rooms`
--
ALTER TABLE `rooms`
  ADD CONSTRAINT `fk_room_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE SET NULL;

--
-- Constraints for table `schedules`
--
ALTER TABLE `schedules`
  ADD CONSTRAINT `fk_sched_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE SET NULL,
  ADD CONSTRAINT `fk_sched_course` FOREIGN KEY (`course_id`) REFERENCES `courses` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_sched_faculty` FOREIGN KEY (`faculty_id`) REFERENCES `faculty` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_sched_room` FOREIGN KEY (`room_id`) REFERENCES `rooms` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_sched_user` FOREIGN KEY (`created_by`) REFERENCES `users` (`id`);

--
-- Constraints for table `schedule_change_requests`
--
ALTER TABLE `schedule_change_requests`
  ADD CONSTRAINT `fk_scr_reviewer` FOREIGN KEY (`reviewed_by`) REFERENCES `users` (`id`) ON DELETE SET NULL,
  ADD CONSTRAINT `fk_scr_sched` FOREIGN KEY (`schedule_id`) REFERENCES `schedules` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_scr_user` FOREIGN KEY (`faculty_user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE;

--
-- Constraints for table `users`
--
ALTER TABLE `users`
  ADD CONSTRAINT `fk_users_college` FOREIGN KEY (`college_id`) REFERENCES `colleges` (`id`) ON DELETE SET NULL;
COMMIT;

/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;

```

```

-----
FILE: dean_gened_chair.php
-----

```

```

<?php
declare(strict_types=1);

/**
 * Dean: program chair account(s) for "General Education" within the college.
 * Separate from the institution-wide GEN ED coordinator (admin → GEN ED Account).
 */
const GE_PROGRAM_CHAIR_LABEL = 'General Education';

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/mail_helpers.php';
require_once __DIR__ . '/includes/account_registration_helpers.php';

require_role(['dean']);
$collegeId = dean_college_id_or_fail();
$collegeName = college_name_by_id($collegeId);
$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

$hasUserEmail = db_column_exists('users', 'email');
$hasAssignedProgram = db_column_exists('users', 'assigned_program');
$hasProgramsTable = db_table_exists('programs');

if ($hasProgramsTable) {
    ensure_college_program_name($collegeId, GE_PROGRAM_CHAIR_LABEL);
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if (!$hasAssignedProgram) {
            throw new RuntimeException('Run upgrade_roles.php first to enable Program Chair accounts.');
        }
        if (!$hasProgramsTable) {
            throw new RuntimeException('Programs table is missing. Run upgrade_roles.php first.');
        }
        ensure_college_program_name($collegeId, GE_PROGRAM_CHAIR_LABEL);

        if ($action === 'add') {
            $username = trim((string) ($_POST['username'] ?? ''));
            $fullName = trim((string) ($_POST['full_name'] ?? ''));

```

```

$assignedProgram = GE_PROGRAM_CHAIR_LABEL;
$email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
$password = (string) ($_POST['password'] ?? '');
$hasValidEmail = $hasUserEmail && $email !== '' && filter_var($email, FILTER_VALIDATE_EMAIL);
$plainForMail = '';

if ($username === '' || $fullName === '') {
    throw new RuntimeException('Username and full name are required.');
```

```

}
if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    throw new RuntimeException('Invalid email address.');
```

```

}

$cred = prepare_new_account_password($password, $hasValidEmail);
$password = $cred['plain'];
$plainForMail = $hasValidEmail ? $cred['plain'] : '';

$exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username=?');
$exists->execute([$username]);
if ((int) $exists->fetchColumn() > 0) {
    $s = suggest_available_usernames($username, 3);
    throw new RuntimeException('Username already exists.' . ($s ? ' Try: ' . implode(', ', $s) : ''));
}

$sql = $hasUserEmail
    ? 'INSERT INTO users (username,password,full_name,email,role,assigned_program,college_id,is_active) VALUES
(?,?,?,?,?,?,?,?)'
    : 'INSERT INTO users (username,password,full_name,role,assigned_program,college_id,is_active) VALUES (?,?,?,?,?,?,?)';
$params = $hasUserEmail
    ? [$username, $cred['hash'], $fullName, $email, 'program_chair', $assignedProgram, $collegeId, !empty($_POST['is_active']) ?
1 : 0]
    : [$username, $cred['hash'], $fullName, 'program_chair', $assignedProgram, $collegeId, !empty($_POST['is_active']) ? 1 : 0];
db()->prepare($sql)->execute($params);
$newGePcId = (int) db()->lastInsertId();
log_dean_activity('generated_program_chair_create', 'Created GE program chair for ' . GE_PROGRAM_CHAIR_LABEL);

$mailOk = $hasValidEmail
    ? notify_new_account_credentials($newGePcId, $email, $fullName, $username, $plainForMail, 'program_chair')
    : null;
if (!$hasValidEmail) {
    mark_new_account_requires_password_change($newGePcId);
}
$_SESSION['flash'] = registration_email_flash_message($mailOk, $email, 'General Education Program Chair account');
```

```

} elseif ($action === 'edit' && isset($_POST['id'])) {
    $id = (int) $_POST['id'];
    $fullName = trim((string) ($_POST['full_name'] ?? ''));
    $assignedProgram = GE_PROGRAM_CHAIR_LABEL;
    $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
    $resetPassword = trim((string) ($_POST['reset_password'] ?? ''));
    $generateAndEmail = !empty($_POST['generate_temp_password_email']);

    if ($fullName === '') {
        throw new RuntimeException('Full name is required.');
```

```

    }
    if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        throw new RuntimeException('Invalid email address.');
```

```

    }

    $st = db()->prepare('SELECT username, full_name FROM users WHERE id=? AND role="program_chair" AND college_id=? AND
assigned_program=?');
    $st->execute([$id, $collegeId, GE_PROGRAM_CHAIR_LABEL]);
    $row = $st->fetch();
    if (!$row) {
        throw new RuntimeException('General Education Program Chair account not found.');
```

```

    }

    $sql = $hasUserEmail
        ? 'UPDATE users SET full_name=?, email=?, assigned_program=?, is_active=? WHERE id=? AND role="program_chair" AND
college_id=? AND assigned_program=?'
        : 'UPDATE users SET full_name=?, assigned_program=?, is_active=? WHERE id=? AND role="program_chair" AND college_id=? AND
assigned_program=?';
    $params = $hasUserEmail
        ? [$fullName, $email, $assignedProgram, !empty($_POST['is_active']) ? 1 : 0, $id, $collegeId, GE_PROGRAM_CHAIR_LABEL]
        : [$fullName, $assignedProgram, !empty($_POST['is_active']) ? 1 : 0, $id, $collegeId, GE_PROGRAM_CHAIR_LABEL];
    db()->prepare($sql)->execute($params);

    $plainForMail = '';
    if ($generateAndEmail) {
        if (!$hasUserEmail || $email === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new RuntimeException('A valid email is required to generate and send a temporary password.');
```



```

    }
    $plainForMail = generate_temp_password();
    db()->prepare('UPDATE users SET password=? WHERE id=?')->execute([
        password_hash($plainForMail, PASSWORD_DEFAULT),
        $id,
    ]);
} elseif ($resetPassword !== '') {
    db()->prepare('UPDATE users SET password=? WHERE id=?')->execute([
        password_hash($resetPassword, PASSWORD_DEFAULT),
        $id,
    ]);
    if (!empty($_POST['email_reset_password']) && $hasUserEmail && $email !== '' && filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $plainForMail = $resetPassword;
    }
}

if ($plainForMail !== '') {
    $mailOk = send_account_credentials_mail($email, $fullName, (string) $row['username'], $plainForMail, 'program_chair',
$email);
    $_SESSION['flash'] = credentials_email_flash_message($mailOk, $email, 'General Education Program Chair updated');
} else {
    $_SESSION['flash'] = 'General Education Program Chair updated.';
}
log_dean_activity('gened_program_chair_update', 'Updated GE program chair #' . $id);
} elseif ($action === 'delete' && isset($_POST['id'])) {
    $id = (int) $_POST['id'];
    $st = db()->prepare('DELETE FROM users WHERE id=? AND role="program_chair" AND college_id=? AND assigned_program=?');
    $st->execute([$id, $collegeId, GE_PROGRAM_CHAIR_LABEL]);
    if ($st->rowCount() < 1) {
        throw new RuntimeException('General Education Program Chair account not found.');
```

```

        <div class="col-md-3"><label class="form-label">Username</label><input name="username" class="form-control" <?= $editRow ?
'readonly' : 'required' ?> value="<?= htmlspecialchars((string) ($editRow['username'] ?? '')) ?>"></div>
        <div class="col-md-3"><label class="form-label">Full Name</label><input name="full_name" class="form-control" required value="<?=
htmlspecialchars((string) ($editRow['full_name'] ?? '')) ?>"></div>
        <div class="col-md-3">
            <label class="form-label">Program</label>
            <input class="form-control" value="<?= htmlspecialchars(GE_PROGRAM_CHAIR_LABEL) ?>" readonly tabindex="-1"
style="background:#f8f9fa" title="Fixed for this page">
        </div>
        <?php if ($hasUserEmail): ?>
            <div class="col-md-3"><label class="form-label">Email</label><input type="email" name="email" class="form-control" value="<?=
htmlspecialchars((string) ($editRow['email'] ?? '')) ?>"></div>
            <?php endif; ?>
            <?php if (!$editRow): ?>
                <div class="col-md-3">
                    <label class="form-label">Password</label>
                    <input name="password" id="gePcPassword" class="form-control" autocomplete="new-password" placeholder="If not emailing">
                    <?php if ($hasUserEmail): ?>
                        <div class="form-check mt-2">
                            <input type="checkbox" class="form-check-input" name="send_credentials_email" id="ge_pc_send_credentials_email"
value="1" checked>
                            <label class="form-check-label" for="ge_pc_send_credentials_email">Generate temporary password and email
it</label>
                        </div>
                    <?php endif; ?>
                </div>
            <?php else: ?>
                <div class="col-md-3">
                    <label class="form-label">Reset password (optional)</label>
                    <input name="reset_password" class="form-control" autocomplete="new-password">
                    <?php if ($hasUserEmail): ?>
                        <div class="form-check mt-2">
                            <input type="checkbox" class="form-check-input" name="email_reset_password" id="ge_pc_email_reset_password"
value="1">
                            <label class="form-check-label" for="ge_pc_email_reset_password">Email this new password</label>
                        </div>
                        <div class="form-check mt-1">
                            <input type="checkbox" class="form-check-input" name="generate_temp_password_email"
id="ge_pc_generate_temp_password_email" value="1">
                            <label class="form-check-label" for="ge_pc_generate_temp_password_email">Generate temporary password and email
it</label>
                        </div>
                    <?php endif; ?>
                </div>
            <?php endif; ?>
        <div class="col-md-2 d-flex align-items-end">
            <div class="form-check"><input type="checkbox" class="form-check-input" name="is_active" value="1" <?=
!isset($editRow['is_active']) || (int) ($editRow['is_active'] ?? 1) === 1 ? 'checked' : '' ?>><label class="form-check-label">Active</label>
</div>
        </div>
        <div class="col-12">
            <button class="btn btn-primary" type="submit">Save</button>
            <?php if ($editRow): ?><a class="btn btn-outline-secondary" href="dean_gened_chair.php">Cancel</a><?php endif; ?>
        </div>
    </form>
    <?php if (!$editRow && $hasUserEmail): ?>
    <script>
        (function () {
            const cb = document.getElementById('ge_pc_send_credentials_email');
            const pw = document.getElementById('gePcPassword');
            if (!cb || !pw) return;
            function sync() {
                pw.required = !cb.checked;
                pw.disabled = cb.checked;
                if (cb.checked) pw.value = '';
            }
            cb.addEventListener('change', sync);
            sync();
        })();
    </script>
    <?php endif; ?>
</div>
</div>

<div class="card shadow-sm">
    <div class="table-responsive">
        <table class="table mb-0">
            <thead class="table-light"><tr><th>Username</th><th>Name</th><th>Program</th><?php if ($hasUserEmail): ?><th>Email</th><?php
endif; ?><th>Status</th><th></th></tr></thead>
            <tbody>

```

```

        <?php foreach ($chairs as $chair): ?>
            <tr>
                <td><?= htmlspecialchars((string) $chair['username']) ?></td>
                <td><?= htmlspecialchars((string) $chair['full_name']) ?></td>
                <td><?= htmlspecialchars((string) $chair['assigned_program']) ?></td>
                <?php if ($hasUserEmail): ?><td><?= htmlspecialchars((string) ($chair['email'] ?? '')) ?></td><?php endif; ?>
                <td><?= (int) $chair['is_active'] === 1 ? 'Active' : 'Disabled' ?></td>
                <td class="text-nowrap">
                    <a class="btn btn-sm btn-outline-primary" href="dean_gened_chair.php?edit=<?= (int) $chair['id'] ?>">Edit</a>
                    <form method="post" class="d-inline" onsubmit="return confirm('Delete this General Education Program Chair
account?');">
                        <input type="hidden" name="action" value="delete">
                        <input type="hidden" name="id" value="<?= (int) $chair['id'] ?>">
                        <button type="submit" class="btn btn-sm btn-outline-danger">Delete</button>
                    </form>
                </td>
            </tr>
        <?php endforeach; ?>
        <?php if ($chairs === []): ?>
            <tr><td colspan="<?= $hasUserEmail ? '6' : '5' ?>" class="text-center text-muted py-4">No General Education Program Chair
accounts yet.</td></tr>
        <?php endif; ?>
    </tbody>
</table>
</div>
</div>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: dean_programs.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['dean']);
$collegeId = dean_college_id_or_fail();
$collegeName = college_name_by_id($collegeId);
$hasProgramsTable = db_table_exists('programs');
$hasProgYlTable = db_table_exists('programs_year_levels');

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if (!$hasProgramsTable) {
            throw new RuntimeException('Programs module is unavailable until you run upgrade_roles.php.');
```

```

        if (!in_array($status, ['active', 'inactive'], true)) {
            $status = 'active';
        }
        $st = db()->prepare('UPDATE programs SET program_name=?, status=? WHERE id=? AND college_id=?');
        $st->execute([$programName, $status, $id, $collegeId]);
        if ($hasProgYlTable) {
            program_year_levels_replace($id, parse_dean_program_year_levels_post($_POST));
        }
        log_dean_activity('program_update', 'Updated program ID #' . $id);
        $_SESSION['flash'] = 'Program updated.';
    } elseif ($action === 'delete' && isset($_POST['id'])) {
        $id = (int) $_POST['id'];
        $st = db()->prepare('DELETE FROM programs WHERE id=? AND college_id=?');
        $st->execute([$id, $collegeId]);
        log_dean_activity('program_delete', 'Deleted program ID #' . $id);
        $_SESSION['flash'] = 'Program removed.';
    }
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: dean_programs.php');
exit;
}

$editRow = null;
if ($hasProgramsTable && isset($_GET['edit'])) {
    $st = db()->prepare('SELECT * FROM programs WHERE id=? AND college_id=?');
    $st->execute([(int) $_GET['edit'], $collegeId]);
    $editRow = $st->fetch() ?: null;
}

$list = [];
if ($hasProgramsTable) {
    $st = db()->prepare('SELECT * FROM programs WHERE college_id=? ORDER BY program_name ASC');
    $st->execute([$collegeId]);
    $list = $st->fetchAll();
}

$levelsByProgId = [];
if ($hasProgYlTable && $list !== []) {
    $ids = array_map(static fn ($r): int => (int) ($r['id'] ?? 0), $list);
    $ids = array_values(array_filter(array_unique($ids), static fn (int $i): bool => $i > 0));
    if ($ids !== []) {
        $holds = implode(',', array_fill(0, count($ids), '?'));
        $q = db()->prepare("SELECT program_id, year_level FROM programs_year_levels WHERE program_id IN ({$holds}) ORDER BY year_level");
        $q->execute($ids);
        while ($lr = $q->fetch(PDO::FETCH_ASSOC)) {
            $pid = (int) ($lr['program_id'] ?? 0);
            $yl = trim((string) ($lr['year_level'] ?? ''));
            if ($pid < 1 || $yl === '') {
                continue;
            }
            if (!isset($levelsByProgId[$pid])) {
                $levelsByProgId[$pid] = [];
            }
            $levelsByProgId[$pid][] = $yl;
        }
    }
}

$prefillCanon = [];
$prefillExtraCsv = '';
if ($hasProgYlTable && $editRow) {
    $defined = program_defined_year_levels((int) $editRow['id']);
    $canonLabels = ['1', '2', '3', '4', '5'];
    foreach ($defined as $d) {
        if (in_array($d, $canonLabels, true)) {
            $prefillCanon[] = $d;
        }
    }
    $extra = [];
    foreach ($defined as $d) {
        if (!in_array($d, $canonLabels, true)) {
            $extra[] = $d;
        }
    }
    $prefillExtraCsv = $extra !== [] ? implode(', ', $extra) : '';
}

$pageTitle = 'Programs';

```

```

require_once __DIR__ . '/includes/header.php';

$canonicalYl = ['1', '2', '3', '4', '5'];
?>
<h1 class="h3 mb-4"><i class="fa-solid fa-graduation-cap me-2 text-primary"></i>Programs</h1>
<p class="text-muted">Managing: <strong><?= htmlspecialchars($collegeName) ?></strong></p>
<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
<?php if (!$hasProgramsTable): ?>
    <div class="alert alert-warning">Programs table is missing. Please run <a href="upgrade_roles.php">upgrade_roles.php</a>.</div>
<?php else: ?>
<?php if (!$hasProgYlTable): ?>
    <div class="alert alert-warning">Run <a href="upgrade_roles.php">upgrade_roles.php</a> to enable program year levels so they propagate
to scheduling forms.</div>
<?php endif; ?>
<div class="card shadow-sm mb-4"><div class="card-body">
    <form method="post" class="row g-3">
        <input type="hidden" name="action" value="<?= $editRow ? 'edit' : 'add' ?>">
        <?php if ($editRow): ?><input type="hidden" name="id" value="<?= (int) $editRow['id'] ?>"><?php endif; ?>
        <div class="col-md-8">
            <label class="form-label">Program Name</label>
            <input name="program_name" class="form-control" required maxlength="120" placeholder="e.g. BS Social Work" value="<?=
htmlspecialchars((string) ($editRow['program_name'] ?? '')) ?>">
        </div>
        <div class="col-md-4">
            <label class="form-label">Status</label>
            <select name="status" class="form-select">
                <option value="active" <?= (($editRow['status'] ?? 'active') === 'active') ? 'selected' : '' ?>>Active</option>
                <option value="inactive" <?= (($editRow['status'] ?? '') === 'inactive') ? 'selected' : '' ?>>Inactive</option>
            </select>
        </div>
        <?php if ($hasProgYlTable): ?>
        <div class="col-12">
            <label class="form-label">Year levels offered</label>
            <div class="d-flex flex-wrap gap-3 mb-2">
                <?php foreach ($canonicalYl as $yl): ?>
                    <div class="form-check">
                        <input class="form-check-input" type="checkbox" name="program_year_levels[]" value="<?= htmlspecialchars($yl) ?>"
                            id="yl_<?= htmlspecialchars($yl) ?>" <?= in_array($yl, $prefillCanon, true) ? 'checked' : '' ?>>
                        <label class="form-check-label" for="yl_<?= htmlspecialchars($yl) ?>">Year <?= htmlspecialchars($yl) ?></label>
                    </div>
                <?php endforeach; ?>
            </div>
            <label class="form-label small text-muted">Additional levels (comma-separated)</label>
            <input type="text" name="program_year_level_extra" class="form-control" maxlength="200" placeholder="e.g. 6, grad1"
                value="<?= htmlspecialchars($prefillExtraCsv) ?>">
            <div class="form-text">These lists appear first in Add schedule for your college and when GEN ED coordinators target your
programs.</div>
        </div>
        <?php endif; ?>
        <div class="col-12"><button type="submit" class="btn btn-primary"><?= app_tooltip_attr($editRow ? 'Saves changes to this program
under your college.' : 'Adds a new program your college offers.') ?>>Save</button><?php if ($editRow): ?> <a class="btn btn-outline-
secondary" href="dean_programs.php"><?= app_tooltip_attr('Closes the editor without saving.') ?>>Cancel</a><?php endif; ?></div>
    </form>
</div></div>

<div class="card shadow-sm"><div class="table-responsive">
    <table class="table mb-0">
        <thead class="table-light"><tr><th>Program</th><?php if ($hasProgYlTable): ?><th>Year levels</th><?php endif; ?><th>Status</th><th>
</th></tr></thead>
        <tbody>
            <?php foreach ($list as $r): ?>
                <tr>
                    <td><?= htmlspecialchars((string) $r['program_name']) ?></td>
                    <?php if ($hasProgYlTable): ?>
                    <td>
                        <?php
                            $lid = (int) $r['id'];
                            $yrs = isset($levelsByProgId[$lid]) ? sort_schedule_year_levels($levelsByProgId[$lid]) : [];
                        ?>
                        <?php if ($yrs === []): ?>
                            <span class="text-muted small">— Uses course catalogs in scheduling until configured</span>
                        <?php else: ?>
                            <?php foreach ($yrs as $y): ?>
                                <span class="badge bg-primary-subtle text-primary-emphasis me-1 mb-1"><?= htmlspecialchars((string) $y) ?>
                            </span>
                            <?php endforeach; ?>
                        <?php endif; ?>
                    </td>
                    <?php endif; ?>
                </tr>
            <?php endforeach; ?>
            <td><span class="badge <?= ($r['status'] === 'active') ? 'bg-success-subtle text-success-emphasis' : 'bg-secondary-subtle

```

```

text-secondary-emphasis' ?><?>= htmlspecialchars((string) $r['status']) ?></span></td>
        <td class="text-nowrap">
            <a class="btn btn-sm btn-outline-primary" href="dean_programs.php?edit=<?>= (int) $r['id'] ?>"<?>= app_tooltip_attr('Edits
this program's name or status.') ?>>Edit</a>
            <form method="post" class="d-inline" onsubmit="return confirm('Delete this program?');">
                <input type="hidden" name="action" value="delete">
                <input type="hidden" name="id" value="<?>= (int) $r['id'] ?>">
                <button type="submit" class="btn btn-sm btn-outline-danger"<?>= app_tooltip_attr('Removes this program after
confirmation if no data blocks deletion.') ?>>Delete</button>
            </form>
        </td>
    </tr>
</table>
<?php endforeach; ?>
<?php if (!$list): ?>
    <tr><td colspan="<?>= $hasProgV1Table ? '4' : '3' ?>" class="text-center text-muted py-4">No programs yet. Add one to use Program
dropdown in Courses.</td></tr>
<?php endif; ?>
</tbody>
</table>
</div></div>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: dean_student_registrations.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/student_registration_helpers.php';

require_role(['dean']);
$collegeId = dean_college_id_or_fail();
$collegeName = college_name_by_id($collegeId);

if (!student_registration_table_ready()) {
    exit('Run upgrade_roles.php first to enable student registration approvals.');
```

```

        $collegeId,
        null
    );
    if (!empty($approval['already_approved'])) {
        $_SESSION['flash'] = 'This registration was already approved. The student can sign in with their existing account.';
    } else {
        log_dean_activity(
            'student_registration_approve',
            'Approved student registration (request #' . $requestId . ').'
        );
        $studentEmail = trim((string) ($approval['email'] ?? ''));
        if ($studentEmail !== '' && filter_var($studentEmail, FILTER_VALIDATE_EMAIL)) {
            $_SESSION['flash'] = !empty($approval['mail_sent'])
                ? 'Registration approved. Login credentials sent to ' . $studentEmail . ' .'
                : 'The account has been created successfully, but the email notification could not be sent.';
        } else {
            $_SESSION['flash'] = 'Registration approved. No email on file – share credentials manually with the student.';
        }
    }
} elseif ($action === 'reject' && $requestId > 0) {
    $reason = trim((string) ($_POST['rejection_reason'] ?? ''));
    reject_student_registration_request(
        $requestId,
        (int) $_SESSION['user_id'],
        $collegeId,
        null,
        $reason
    );
    log_dean_activity(
        'student_registration_reject',
        'Rejected student registration (request #' . $requestId . ').'
    );
    $_SESSION['flash'] = 'Registration request rejected.';
} else {
    throw new RuntimeException('Invalid action.');
```

```

}
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: ' . $redirectUrl);
exit;
}

$pending = pending_registrations_for_college($collegeId, $programScope);

$pageTitle = 'Student registration approvals';
$mainContainerClass = 'container-fluid py-4 py-md-5 app-main';
require_once __DIR__ . '/includes/header.php';
?>
<div class="container" style="max-width: 1100px;">
    <div class="d-flex flex-wrap justify-content-between align-items-center gap-3 mb-4">
        <div>
            <h1 class="h3 mb-1">Student registration approvals</h1>
            <p class="text-muted mb-0">
                <?= htmlspecialchars($collegeName) ?>
                <?php if ($programFilter !== ''): ?>
                    - <?= htmlspecialchars($programFilter) ?>
                <?php else: ?>
                    - all programs
                <?php endif; ?>
            </p>
        </div>
        <?php if ($pending !== []): ?>
            <form method="post" class="d-inline">
                <input type="hidden" name="action" value="approve_all">
                <input type="hidden" name="program_filter" value="<?= htmlspecialchars($programFilter) ?>">
                <button type="submit" class="btn btn-success">
                    onclick="return confirm('Approve all <?= (int) count($pending) ?> pending registration(s)? Each student will receive
an email with a temporary password when an email address is on file.');">
                    <i class="fa-solid fa-check-double me-1"></i> Approve all (<?= (int) count($pending) ?>)
                </button>
            </form>
        <?php endif; ?>
    </div>

    <?php if ($programs !== []): ?>
        <form method="get" class="row g-2 align-items-end mb-4">
            <div class="col-sm-8 col-md-5">
                <label class="form-label" for="program">Program</label>
                <select class="form-select" id="program" name="program" onchange="this.form.submit()">

```

```

        <option value="">All programs</option>
        <?php foreach ($programs as $programName): ?>
            <option value="<?= htmlspecialchars((string) $programName) ?>"
                <?= $programFilter == (string) $programName ? 'selected' : '' ?>>
                <?= htmlspecialchars((string) $programName) ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
</form>
<?php endif; ?>

<?php if ($flash != ''): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if ($pending == []): ?>
    <div class="card border-0 shadow-sm rounded-4">
        <div class="card-body text-center py-5 text-muted">
            <i class="fa-solid fa-inbox fa-2x mb-3 d-block opacity-50"></i>
            No pending student registrations<?= $programFilter != '' ? ' for this program' : ' in your college' ?>.
        </div>
    </div>
<?php else: ?>
    <div class="card border-0 shadow-sm rounded-4">
        <div class="table-responsive">
            <table class="table table-hover align-middle mb-0">
                <thead class="table-light">
                    <tr>
                        <th>Student</th>
                        <th>Program</th>
                        <th>Username</th>
                        <th>Student #</th>
                        <th>Year</th>
                        <th>Email</th>
                        <th>Submitted</th>
                        <th class="text-end">Actions</th>
                    </tr>
                </thead>
                <tbody>
                    <?php foreach ($pending as $row): ?>
                        <tr>
                            <td class="fw-semibold"><?= htmlspecialchars((string) $row['full_name']) ?></td>
                            <td><?= htmlspecialchars((string) ($row['program_name'] ?? '')) ?></td>
                            <td><code><?= htmlspecialchars((string) $row['username']) ?></code></td>
                            <td><?= htmlspecialchars((string) $row['student_number']) ?></td>
                            <td><?= htmlspecialchars((string) ($row['year_level'] ?? '')) ?>: '- ' ?></td>
                            <td><?= htmlspecialchars((string) ($row['email'] ?? '')) ?>: '- ' ?></td>
                            <td class="text-muted small"><?= htmlspecialchars((string) ($row['created_at'] ?? '')) ?></td>
                            <td class="text-end text-nowrap">
                                <form method="post" class="d-inline">
                                    <input type="hidden" name="action" value="approve">
                                    <input type="hidden" name="request_id" value="<?= (int) $row['id'] ?>">
                                    <input type="hidden" name="program_filter" value="<?= htmlspecialchars($programFilter) ?>">
                                    <button type="submit" class="btn btn-success btn-sm"
                                        onclick="return confirm('Approve this student registration? They will receive an email with
a temporary password to sign in.');">
                                        <i class="fa-solid fa-check me-1"></i> Approve
                                </form>
                                <button type="button" class="btn btn-outline-danger btn-sm ms-1"
                                    data-bs-toggle="modal" data-bs-target="#rejectModal"<?= (int) $row['id'] ?>">
                                    <i class="fa-solid fa-xmark me-1"></i> Reject
                                </button>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                </tbody>
            </table>
        </div>
    </div>
<?php foreach ($pending as $row): ?>
    <div class="modal fade" id="rejectModal"<?= (int) $row['id'] ?>" tabindex="-1" aria-hidden="true">
        <div class="modal-dialog">
            <div class="modal-content rounded-4">
                <form method="post">
                    <input type="hidden" name="action" value="reject">
                    <input type="hidden" name="request_id" value="<?= (int) $row['id'] ?>">
                    <input type="hidden" name="program_filter" value="<?= htmlspecialchars($programFilter) ?>">
                    <div class="modal-header">
                        <h5 class="modal-title">Reject registration</h5>
                        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
                    </div>
                </form>
            </div>
        </div>
    </div>
</div>

```



```

        </div>
        <div class="modal-body">
            <p>Reject registration for <strong><?= htmlspecialchars((string) $row['full_name']) ?></strong>?</p>
            <label class="form-label" for="reason<?= (int) $row['id'] ?>">Reason (optional)</label>
            <textarea class="form-control" id="reason<?= (int) $row['id'] ?>" name="rejection_reason" rows="3"
                placeholder="e.g. Wrong program or invalid student number"></textarea>
        </div>
        <div class="modal-footer">
            <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Cancel</button>
            <button type="submit" class="btn btn-danger">Reject</button>
        </div>
    </form>
</div>
</div>
</div>
<?php endforeach; ?>
<?php endif; ?>
</div>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: edit_schedule.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/schedule_helpers.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['dean', 'program_chair']);
$collegeId = dean_or_program_chair_college_id_or_fail();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
$hasLabFlag = db_column_exists('courses', 'is_laboratory');
$hasProgramsTable = db_table_exists('programs');
$hasYearLevel = db_column_exists('courses', 'year_level');
$hasSection = db_column_exists('courses', 'section');

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $id = (int) ($_POST['schedule_id'] ?? 0);
} else {
    $id = (int) ($_GET['id'] ?? 0);
}

if ($id < 1) {
    header('Location: schedule.php');
    exit;
}

$sql = 'SELECT s.*, COALESCE(u.role, "") AS creator_role
FROM schedules s
INNER JOIN courses c_scope ON c_scope.id = s.course_id
LEFT JOIN users u ON u.id = s.created_by
WHERE s.id = ? AND s.college_id = ?';
$params = [$id, $collegeId];
if ($programScope !== null) {
    $sql .= ' AND c_scope.department = ?';
    $params[] = $programScope;
}
$sql .= ' LIMIT 1';
$stmt = db()->prepare($sql);
$stmt->execute($params);
$scheduleRow = $stmt->fetch();
if (!$scheduleRow) {
    $_SESSION['flash'] = 'Schedule not found or not in your college.';
    header('Location: schedule.php');
    exit;
}
if ((string) ($scheduleRow['creator_role'] ?? '') === 'gened') {
    $_SESSION['flash'] = 'GE-created schedules cannot be edited here. Use GEN ED tools.';
    header('Location: schedule.php');
    exit;
}

$sql = 'SELECT * FROM courses WHERE id=? AND college_id=?';
$params = [(int) $scheduleRow['course_id'], $collegeId];
if ($programScope !== null) {
    $sql .= ' AND department=?';
    $params[] = $programScope;
}

```

```

}
$st = db()->prepare($sql);
$st->execute($params);
$courseRow = $st->fetch();
if (!$courseRow) {
    $_SESSION['flash'] = 'Course for this schedule is missing or not in your college.';
    header('Location: schedule.php');
    exit;
}

$errors = [];
$old = $_POST ?? [];
if (isset($old['days']) && is_array($old['days'])) {
    $old['day_array'] = $old['days'];
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $facultyId = (int) ($_POST['faculty_id'] ?? 0);
    $courseId = (int) ($_POST['course_id'] ?? 0);
    $roomId = (int) ($_POST['room_id'] ?? 0);
    $scheduleType = (string) ($_POST['schedule_type'] ?? 'Custom');
    $days = isset($_POST['days']) && is_array($_POST['days']) ? array_map('strval', $_POST['days']) : [];
    $startTime = substr((string) ($_POST['start_time'] ?? ''), 0, 5) . ':00';
    $endTime = substr((string) ($_POST['end_time'] ?? ''), 0, 5) . ':00';
    $semester = trim((string) ($_POST['semester'] ?? ''));
    $schoolYear = trim((string) ($_POST['school_year'] ?? ''));
    $academicYear = trim((string) ($_POST['academic_year'] ?? ''));

    $requestOverride = !empty($_POST['request_admin_override']);
    $overrideReason = trim((string) ($_POST['override_reason'] ?? ''));
    $selectedProgram = $programScope ?? trim((string) ($_POST['program'] ?? ''));

    if ($facultyId < 1 || $courseId < 1 || $roomId < 1) {
        $errors[] = 'Please select faculty, course, and room.';
    }
    if ($days === []) {
        $errors[] = 'Select at least one day.';
    }
    if ($semester === '' || $schoolYear === '') {
        $errors[] = 'Semester and school year are required.';
    }

    if (!$errors) {
        $sql = 'SELECT COUNT(*) FROM faculty WHERE id=? AND college_id=?';
        $params = [$facultyId, $collegeId];
        if ($programScope !== null) {
            $sql .= ' AND department=?';
            $params[] = $programScope;
        }
        $chk = db()->prepare($sql);
        $chk->execute($params);
        if ((int) $chk->fetchColumn() < 1) {
            $errors[] = $programScope !== null
                ? 'Selected faculty is outside your assigned program.'
                : 'Selected faculty is not in your college.';
        }

        if ($hasLabFlag) {
            $sql = 'SELECT is_laboratory FROM courses WHERE id=? AND college_id=?';
            $params = [$courseId, $collegeId];
            if ($programScope !== null) {
                $sql .= ' AND department=?';
                $params[] = $programScope;
            }
            $chk = db()->prepare($sql);
            $chk->execute($params);
            $v = $chk->fetchColumn();
            if ($v === false) {
                $errors[] = $programScope !== null
                    ? 'Selected course is outside your assigned program.'
                    : 'Selected course is not in your college.';
            }
        }
    } else {
        $sql = 'SELECT COUNT(*) FROM courses WHERE id=? AND college_id=?';
        $params = [$courseId, $collegeId];
        if ($programScope !== null) {
            $sql .= ' AND department=?';
            $params[] = $programScope;
        }
        $chk = db()->prepare($sql);
    }
}

```

```

        $chk->execute($params);
        if ((int) $chk->fetchColumn() < 1) {
            $errors[] = $programScope !== null
                ? 'Selected course is outside your assigned program.'
                : 'Selected course is not in your college.';
        }
    }

    $chk = db()->prepare('SELECT COUNT(*) FROM rooms WHERE id=?');
    $chk->execute([$roomId]);
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'Selected room is invalid.';
    }

    if ($hasProgramsTable && $selectedProgram !== '') {
        $chk = db()->prepare("SELECT COUNT(*) FROM programs WHERE college_id=? AND program_name=? AND status='active'");
        $chk->execute([$collegeId, $selectedProgram]);
        if ((int) $chk->fetchColumn() < 1) {
            $errors[] = 'Selected program is invalid for your college.';
        } else {
            $chk = db()->prepare('SELECT TRIM(department) FROM courses WHERE id=? AND college_id=?');
            $chk->execute([$courseId, $collegeId]);
            $courseDept = trim((string) $chk->fetchColumn());
            if ($courseDept !== $selectedProgram) {
                $errors[] = 'Selected course does not match the selected program.';
            }
        }
    }

    if ($hasYearLevel && $hasSection) {
        $yearLevelPost = trim((string) ($_POST['year_level'] ?? ''));
        $sectionPost = trim((string) ($_POST['section'] ?? ''));
        if ($yearLevelPost === '' || $sectionPost === '') {
            $errors[] = 'Year level and section are required.';
        } else {
            $chk = db()->prepare('SELECT year_level, section FROM courses WHERE id=? AND college_id=?');
            $chk->execute([$courseId, $collegeId]);
            $crow = $chk->fetch();
            if ($crow) {
                $cy = trim((string) ($crow['year_level'] ?? ''));
                $cs = trim((string) ($crow['section'] ?? ''));
                if ($cy !== $yearLevelPost || $cs !== $sectionPost) {
                    $errors[] = 'Selected course does not match the chosen year level and section.';
                }
            }
        }
    }
}

if (!$errors) {
    $assignedFaculty = find_course_load_assigned_faculty($courseId, $semester, $schoolYear, $facultyId);
    if ($assignedFaculty !== null) {
        $errors[] = course_load_assignment_conflict_message($assignedFaculty, $courseId);
    }
}

$lectureRules = validate_schedule_rules($scheduleType, $days, $startTime, $endTime);
foreach ($lectureRules['errors'] as $e) {
    $errors[] = $e;
}

$allConflicts = [];
$segmentCross = [];
if (!$errors) {
    $conflicts = checkConflicts(
        $facultyId,
        $roomId,
        $days,
        $startTime,
        $endTime,
        $semester,
        $schoolYear,
        $id,
        $collegeId
    );
    $internal = [];
    $cross = [];
    foreach ($conflicts as $c) {
        if (($c['scope'] ?? '') === 'cross_college') {
            $cross[] = $c;
        }
    }
}

```

```

        } else {
            $internal[] = $c;
        }
    }
    foreach ($internal as $c) {
        $errors[] = $c['description'];
    }
    if ($cross) {
        $segmentCross = $cross;
    }
    $allConflicts = $conflicts;

    if ($segmentCross && !$requestOverride) {
        foreach ($segmentCross as $c) {
            $errors[] = $c['description'] . ' (Request admin override to proceed.)';
        }
    }
    if ($segmentCross && $requestOverride && $overrideReason === '') {
        $errors[] = 'Please add a reason for admin override request.';
    }

    if (!$errors && $segmentCross && $requestOverride) {
        create_conflict_request([
            'requested_by' => (int) $_SESSION['user_id'],
            'college_id' => $collegeId,
            'faculty_id' => $facultyId,
            'course_id' => $courseId,
            'room_id' => $roomId,
            'schedule_type' => $scheduleType,
            'day_of_week' => days_to_set($days),
            'start_time' => $startTime,
            'end_time' => $endTime,
            'semester' => $semester,
            'school_year' => $schoolYear,
            'academic_year' => $academicYear,
            'reason' => $overrideReason . ' [Edit schedule #' . $id . ']',
        ]);
        log_dean_activity('schedule_override_request', 'Submitted cross-college conflict override request (edit schedule #' . $id .
    '));

    $_SESSION['flash'] = 'Conflict request sent to admin for approval.';
    header('Location: schedule.php');
    exit;
}

}

if (!$errors) {
    $beforeSched = [
        'id' => (int) $scheduleRow['id'],
        'faculty_id' => (int) $scheduleRow['faculty_id'],
        'course_id' => (int) $scheduleRow['course_id'],
        'room_id' => (int) $scheduleRow['room_id'],
        'schedule_type' => (string) $scheduleRow['schedule_type'],
        'day_of_week' => (string) $scheduleRow['day_of_week'],
        'start_time' => (string) $scheduleRow['start_time'],
        'end_time' => (string) $scheduleRow['end_time'],
        'semester' => (string) $scheduleRow['semester'],
        'school_year' => (string) $scheduleRow['school_year'],
        'academic_year' => (string) ($scheduleRow['academic_year'] ?? ''),
    ];
    $hasSessionKindCol = db_column_exists('schedules', 'session_kind');
    $updateSql = 'UPDATE schedules SET faculty_id=?, course_id=?, room_id=?, schedule_type=?, day_of_week=?, start_time=?, end_time=?,
semester=?, school_year=?, academic_year=?'
        . ($hasSessionKindCol ? ', session_kind=?' : '')
        . ' WHERE id=? AND college_id=?';
    $updateParams = [
        $facultyId,
        $courseId,
        $roomId,
        $scheduleType,
        days_to_set($days),
        $startTime,
        $endTime,
        $semester,
        $schoolYear,
        $academicYear,
    ];
    if ($hasSessionKindCol) {
        // Edit form is lecture segment only; lab rows are separate schedule entries.
        $updateParams[] = schedule_session_kind('Lecture');
    }
}

```

```

$updateParams[] = $id;
$updateParams[] = $collegeId;
$db()->prepare($updateSql)->execute($updateParams);
if ($allConflicts !== []) {
    log_conflicts($id, $allConflicts, false);
}
$stmtUp = $db()->prepare(
    'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.schedule_type, s.day_of_week, s.start_time, s.end_time, s.semester,
    s.school_year, s.academic_year,
        f.full_name AS faculty_name, c.course_code, r.room_code
    FROM schedules s
    INNER JOIN faculty f ON f.id = s.faculty_id
    INNER JOIN courses c ON c.id = s.course_id
    INNER JOIN rooms r ON r.id = s.room_id
    WHERE s.id = ? LIMIT 1'
);
$stmtUp->execute([$id]);
$afterSched = $stmtUp->fetch(PDO::FETCH_ASSOC);
log_user_activity('edit', 'Schedules', 'Schedule #' . $id, $beforeSched, $afterSched ? (array) $afterSched : null);
log_dean_activity('schedule_update', 'Updated schedule #' . $id);
$_SESSION['flash'] = 'Schedule updated.';
header('Location: schedule.php');
exit;
}
}

$pageTitle = 'Edit schedule';
require_once __DIR__ . '/includes/header.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST' && $errors !== []) {
    $defaults = $old;
    if (isset($old['days']) && is_array($old['days'])) {
        $defaults['day_array'] = $old['days'];
    }
} else {
    $defaults = [
        'faculty_id' => (int) $scheduleRow['faculty_id'],
        'course_id' => (int) $scheduleRow['course_id'],
        'room_id' => (int) $scheduleRow['room_id'],
        'schedule_type' => (string) $scheduleRow['schedule_type'],
        'day_of_week' => (string) $scheduleRow['day_of_week'],
        'start_time' => (string) $scheduleRow['start_time'],
        'end_time' => (string) $scheduleRow['end_time'],
        'semester' => (string) $scheduleRow['semester'],
        'school_year' => (string) $scheduleRow['school_year'],
        'academic_year' => (string) ($scheduleRow['academic_year'] ?? ''),
        'program' => trim((string) ($courseRow['department'] ?? '')),
        'year_level' => trim((string) ($courseRow['year_level'] ?? '')),
        'section' => trim((string) ($courseRow['section'] ?? '')),
    ];
}
?>
<h1 class="h3 mb-4"><i class="fa-solid fa-pen me-2 text-primary"></i>Edit schedule #<?=(int) $id ?></h1>
<p class="text-muted small">You are editing this schedule row only. For courses with separate lecture and laboratory rows, edit each row from the list.</p>

<?php render_schedule_errors_warning_popup($errors); ?>

<div class="card shadow-sm">
    <div class="card-body">
        <form method="post" novalidate>
            <input type="hidden" name="schedule_id" value="<?=(int) $id ?>">
            <?php render_schedule_form($defaults, ['edit_single_row' => true]); ?>
            <div class="mt-3 p-3 border rounded bg-light">
                <div class="form-check">
                    <input class="form-check-input" type="checkbox" id="request_admin_override" name="request_admin_override" value="1" <?=(int) $old['request_admin_override'] ? 'checked' : '' ?>>
                    <label class="form-check-label" for="request_admin_override">Request Admin Override for cross-college conflict</label>
                </div>
                <label class="form-label mt-2">Reason (required when requesting override)</label>
                <textarea name="override_reason" rows="2" class="form-control"><?=(string) ($old['override_reason'] ?? '') ?></textarea>
            </div>
            <div class="mt-4">
                <button type="submit" class="btn btn-primary"><?=(string) ($old['override_reason'] ?? '') ?> Saves edits to this schedule row. Use this after changing time, room, faculty, or meeting link.</button>
                <a href="schedule.php" class="btn btn-outline-secondary"><?=(string) ($old['override_reason'] ?? '') ?> Discards unsaved edits and returns to the schedule list.</a>
            </div>
        </form>
    </div>
</div>

```

```
</div>
</div>
```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>
```

```
-----
FILE: faculty.php
-----
```

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';
require_once __DIR__ . '/includes/account_registration_helpers.php';

require_role(['dean', 'program_chair']);
$collegeId = dean_or_program_chair_college_id_or_fail();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
$collegeName = college_name_by_id($collegeId);
$hasProgramsTable = db_table_exists('programs');
$hasEmploymentStatusColumn = db_column_exists('faculty', 'employment_status');
$deanProgramOptions = [];
if ($hasProgramsTable && $programScope === null) {
    $stProg = db()->prepare("SELECT program_name FROM programs WHERE college_id=? AND status='active' ORDER BY program_name");
    $stProg->execute([$collegeId]);
    $deanProgramOptions = array_map('strval', $stProg->fetchAll(PDO::FETCH_COLUMN)) ?: [];
}

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    $maxHoursPerDay = (int) ($_POST['max_hours_per_day'] ?? 8);
    if ($maxHoursPerDay < 1) {
        $maxHoursPerDay = 1;
    } elseif ($maxHoursPerDay > 36) {
        $maxHoursPerDay = 36;
    }

    try {
        if ($action === 'add') {
            $allowedEmploymentStatuses = ['Permanent', 'Contract of Service', 'Temporary'];
            $employmentStatus = trim((string) ($_POST['employment_status'] ?? 'Permanent'));
            if (!in_array($employmentStatus, $allowedEmploymentStatuses, true)) {
                throw new RuntimeException('Please select a valid employment status.');
            }
            $username = trim((string) ($_POST['username'] ?? ''));
            $password = (string) ($_POST['password'] ?? '');
            $facultyEmail = trim((string) ($_POST['email'] ?? ''));
            $fullName = trim((string) ($_POST['full_name'] ?? ''));
            $hasValidEmail = $facultyEmail !== '' && filter_var($facultyEmail, FILTER_VALIDATE_EMAIL);

            if ($username === '') {
                throw new RuntimeException('Username is required.');
            }
            if ($facultyEmail !== '' && !filter_var($facultyEmail, FILTER_VALIDATE_EMAIL)) {
                throw new RuntimeException('Invalid email address.');
            }

            $cred = prepare_new_account_password($password, $hasValidEmail);
            $password = $cred['plain'];
            $exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username=?');
            $exists->execute([$username]);
            if ((int) $exists->fetchColumn() > 0) {
                $s = suggest_available_usernames($username, 3);
                throw new RuntimeException('Username already exists.' . ($s ? ' Try: ' . implode(', ', $s) : ''));
            }
            $facultyCode = trim((string) ($_POST['faculty_id'] ?? ''));
            if ($facultyCode === '') {
                throw new RuntimeException('Faculty ID is required.');
            }
            $exists = db()->prepare('SELECT COUNT(*) FROM faculty WHERE faculty_id=?');
            $exists->execute([$facultyCode]);
            if ((int) $exists->fetchColumn() > 0) {
                throw new RuntimeException('Faculty ID already exists.');
            }
            $deptValAdd = trim((string) ($_POST['department'] ?? ''));
```

```

if ($programScope === null) {
    if ($hasProgramsTable && $deanProgramOptions !== []) {
        if (!in_array($deptValAdd, $deanProgramOptions, true)) {
            throw new RuntimeException('Please select a valid program from your college list.');
```

```

        }
    } elseif ($hasProgramsTable && $deanProgramOptions === [] && $deptValAdd === '') {
        throw new RuntimeException('Add at least one active program under Programs before assigning faculty.');
```

```

    } elseif ($deptValAdd === '') {
        throw new RuntimeException('Program is required.');
```

```

    }
}
```

```

db()->beginTransaction();
```

```

db()->prepare('INSERT INTO users (username,password,full_name,role,college_id,is_active) VALUES (?,?,,?,?,?)')
```

```

->execute([
    $username,
    $cred['hash'],
    $fullName,
    'faculty',
    $collegeId,
    (($_POST['status'] ?? '') === 'inactive') ? 0 : 1,
]);
```

```

$uid = (int) db()->lastInsertId();
```

```

if ($hasEmploymentStatusColumn) {
```

```

    db()->prepare(
```

```

        'INSERT INTO faculty (user_id, faculty_id, full_name, department, email, max_hours_per_day, college_id, status,
employment_status, is_generated)
        VALUES (?,?,,?,?,?,?,?,?,?)'
```

```

    )->execute([
        $uid,
        $facultyCode,
        $fullName,
        $programScope ?? trim((string) ($_POST['department'] ?? '')),
        $facultyEmail,
        $maxHoursPerDay,
        $collegeId,
        (($_POST['status'] ?? '') === 'inactive') ? 'inactive' : 'active',
        $employmentStatus,
    ]);
}
```

```

} else {
```

```

    db()->prepare(
```

```

        'INSERT INTO faculty (user_id, faculty_id, full_name, department, email, max_hours_per_day, college_id, status,
is_generated)
        VALUES (?,?,,?,?,?,?,?,?,?)'
```

```

    )->execute([
        $uid,
        $facultyCode,
        $fullName,
        $programScope ?? trim((string) ($_POST['department'] ?? '')),
        $facultyEmail,
        $maxHoursPerDay,
        $collegeId,
        (($_POST['status'] ?? '') === 'inactive') ? 'inactive' : 'active',
    ]);
}
```

```

db()->commit();
```

```

$newFacultyId = (int) db()->lastInsertId();
```

```

$stFa = db()->prepare(
```

```

    'SELECT f.id, f.user_id, f.faculty_id, f.full_name, f.department, f.email, f.max_hours_per_day, f.status, f.college_id,
u.username, u.is_active AS user_is_active
    FROM faculty f INNER JOIN users u ON u.id = f.user_id WHERE f.id = ? LIMIT 1'
```

```

);
```

```

$stFa->execute([$newFacultyId]);
```

```

$afterFac = $stFa->fetch(PDO::FETCH_ASSOC);
```

```

log_user_activity('add', 'Faculty', 'Faculty #' . $newFacultyId, null, $afterFac ? (array) $afterFac : null);
```

```

log_dean_activity('faculty_create', 'Created faculty ' . $facultyCode);
```

```

$mailOk = $hasValidEmail
```

```

    ? notify_new_account_credentials($uid, $facultyEmail, $fullName, $username, $password, 'faculty')
    : null;
```

```

if (!$hasValidEmail) {
```

```

    mark_new_account_requires_password_change($uid);
}
```

```

$_SESSION['flash'] = registration_email_flash_message($mailOk, $facultyEmail, 'Faculty account');
```

```

} elseif ($action === 'edit' && isset($_POST['id'])) {
```

```

    $allowedEmploymentStatuses = ['Permanent', 'Contract of Service', 'Temporary'];
```

```

    $employmentStatus = trim((string) ($_POST['employment_status'] ?? 'Permanent'));
    if (!in_array($employmentStatus, $allowedEmploymentStatuses, true)) {
```

```

        throw new RuntimeException('Please select a valid employment status.');
```

```

    }
```

```

    $fid = (int) $_POST['id'];
```

```

    $sqlBf = 'SELECT f.*, u.username, u.is_active AS user_is_active FROM faculty f INNER JOIN users u ON u.id = f.user_id WHERE
```

```

f.id=? AND f.college_id=? AND COALESCE(f.is_gened,0)=0';
$parBf = [$fid, $collegeId];
if ($programScope !== null) {
    $sqlBf .= ' AND f.department=?';
    $parBf[] = $programScope;
}
$stBf = db()->prepare($sqlBf);
$stBf->execute($parBf);
$beforeFac = $stBf->fetch(PDO::FETCH_ASSOC);
$sql = 'SELECT user_id FROM faculty WHERE id=? AND college_id=? AND COALESCE(is_gened,0)=0';
$params = [$fid, $collegeId];
if ($programScope !== null) {
    $sql .= ' AND department=?';
    $params[] = $programScope;
}
$chk = db()->prepare($sql);
$chk->execute($params);
$userId = (int) ($chk->fetchColumn() ?: 0);
if ($userId < 1) {
    throw new RuntimeException('Faculty record not found in your college.');
```

```

}
$newCode = trim((string) ($_POST['faculty_id'] ?? ''));
if ($newCode === '') {
    throw new RuntimeException('Faculty ID is required.');
```

```

}
$exists = db()->prepare('SELECT COUNT(*) FROM faculty WHERE faculty_id=? AND id<>?');
$exists->execute([$newCode, $fid]);
if ((int) $exists->fetchColumn() > 0) {
    throw new RuntimeException('Faculty ID already exists.');
```

```

}
$deptValEdit = trim((string) ($_POST['department'] ?? ''));
if ($programScope === null) {
    if ($hasProgramsTable && $deanProgramOptions !== []) {
        $legacyDept = trim((string) ($beforeFac['department'] ?? ''));
        $allowedDept = $deanProgramOptions;
        if ($legacyDept !== '' && !in_array($legacyDept, $allowedDept, true)) {
            $allowedDept[] = $legacyDept;
        }
        if (!in_array($deptValEdit, $allowedDept, true)) {
            throw new RuntimeException('Please select a valid program from your college list.');
```

```

        }
    } elseif ($deptValEdit === '') {
        throw new RuntimeException('Program is required.');
```

```

    }
}
if ($hasEmploymentStatusColumn) {
    db()->prepare(
        'UPDATE faculty SET faculty_id=?, full_name=?, department=?, email=?, max_hours_per_day=?, status=?, employment_status=?
WHERE id=? AND college_id=? AND COALESCE(is_gened,0)=0'
    )->execute([
        $newCode,
        trim((string) ($_POST['full_name'] ?? '')),
        $programScope ?? trim((string) ($_POST['department'] ?? '')),
        trim((string) ($_POST['email'] ?? '')),
        $maxHoursPerDay,
        (($_POST['status'] ?? '') === 'inactive') ? 'inactive' : 'active',
        $employmentStatus,
        $fid,
        $collegeId,
    ]);
} else {
    db()->prepare(
        'UPDATE faculty SET faculty_id=?, full_name=?, department=?, email=?, max_hours_per_day=?, status=? WHERE id=? AND
college_id=? AND COALESCE(is_gened,0)=0'
    )->execute([
        $newCode,
        trim((string) ($_POST['full_name'] ?? '')),
        $programScope ?? trim((string) ($_POST['department'] ?? '')),
        trim((string) ($_POST['email'] ?? '')),
        $maxHoursPerDay,
        (($_POST['status'] ?? '') === 'inactive') ? 'inactive' : 'active',
        $fid,
        $collegeId,
    ]);
}
db()->prepare('UPDATE users SET full_name=?, is_active=? WHERE id=?')
->execute([
    trim((string) ($_POST['full_name'] ?? '')),
    (($_POST['status'] ?? '') === 'inactive') ? 0 : 1,
    $userId,

```



```

    });
    if (!empty($_POST['reset_password'])) {
        db()->prepare('UPDATE users SET password=? WHERE id=?')
        ->execute([password_hash((string) $_POST['reset_password'], PASSWORD_DEFAULT), $userId]);
    }
    $stAf = db()->prepare(
        'SELECT f.id, f.user_id, f.faculty_id, f.full_name, f.department, f.email, f.max_hours_per_day, f.status, f.college_id,
u.username, u.is_active AS user_is_active
        FROM faculty f INNER JOIN users u ON u.id = f.user_id WHERE f.id = ? LIMIT 1'
    );
    $stAf->execute([$fid]);
    $afterFac = $stAf->fetch(PDO::FETCH_ASSOC);
    if ($afterFac !== false && $afterFac !== [] && (!empty($_POST['reset_password']))) {
        $afterFac['password'] = '[changed]';
    }
    log_user_activity('edit', 'Faculty', 'Faculty #' . $fid, $beforeFac ? (array) $beforeFac : null, $afterFac ? (array) $afterFac :
null);
    log_dean_activity('faculty_update', 'Updated faculty ID #' . $fid);
    $_SESSION['flash'] = 'Faculty updated.';
} elseif ($action === 'delete' && isset($_POST['id'])) {
    $fid = (int) $_POST['id'];
    $sqlDelSnap = 'SELECT f.*, u.username FROM faculty f INNER JOIN users u ON u.id = f.user_id WHERE f.id=? AND f.college_id=? AND
COALESCE(f.is_gened,0)=0';
    $parDelSnap = [$fid, $collegeId];
    if ($programScope !== null) {
        $sqlDelSnap .= ' AND f.department=?';
        $parDelSnap[] = $programScope;
    }
    $stDelSnap = db()->prepare($sqlDelSnap);
    $stDelSnap->execute($parDelSnap);
    $beforeDelFac = $stDelSnap->fetch(PDO::FETCH_ASSOC);
    $sql = 'SELECT user_id FROM faculty WHERE id=? AND college_id=? AND COALESCE(is_gened,0)=0';
    $params = [$fid, $collegeId];
    if ($programScope !== null) {
        $sql .= ' AND department=?';
        $params[] = $programScope;
    }
    $q = db()->prepare($sql);
    $q->execute($params);
    $userId = (int) ($q->fetchColumn() ?: 0);
    if ($userId < 1) {
        throw new RuntimeException('Faculty record not found.');
```

```

        FROM faculty f
        LEFT JOIN users u ON u.id = f.user_id
        WHERE f.college_id = ? AND COALESCE(f.is_gened,0) = 0';
$params = [$collegeId];
if ($programScope !== null) {
    $sql .= ' AND f.department = ?';
    $params[] = $programScope;
}
$sql .= ' ORDER BY f.full_name ASC';
$st = db()->prepare($sql);
$st->execute($params);
$list = $st->fetchAll();

$currentUser = current_user() ?? [];
$deanName = trim((string) ($currentUser['full_name'] ?? 'College Administrator'));
$deanInitials = '';
foreach (preg_split('/\s+/', $deanName) ? : [] as $part) {
    $token = trim((string) $part, " .\t\n\r\0\x0B");
    if ($token !== '') {
        $deanInitials .= strtoupper(substr($token, 0, 1));
    }
}
$deanInitials = $deanInitials !== '' ? substr($deanInitials, 0, 2) : 'FA';
$totalFacultyCount = count($list);
$permanentCount = 0;
$activeCount = 0;
foreach ($list as $facultyRow) {
    if ((string) ($facultyRow['employment_status'] ?? '') === 'Permanent') {
        $permanentCount++;
    }
    if ((string) ($facultyRow['status'] ?? '') === 'active') {
        $activeCount++;
    }
}

$pageTitle = 'Faculty';
require_once __DIR__ . '/includes/header.php';
?>
<style>
.faculty-dashboard-container { max-width: 1440px; margin: 0 auto; }
.faculty-university-header { display:flex; justify-content:space-between; align-items:flex-end; flex-wrap:wrap; margin-bottom:28px;
border-bottom:2px solid rgba(30,64,95,.2); padding-bottom:16px; gap: 12px; }
.faculty-title-section h1 { font-size:1.9rem; font-weight:700; background:linear-gradient(135deg,#1e405f,#2a6f8f); -webkit-background-clip:text; background-clip:text; color:transparent; letter-spacing:-.3px; }
.faculty-title-section p { font-size:.9rem; color:#4a627a; margin-top:6px; font-weight:500; }
.faculty-dean-card { background:#fff; padding:10px 24px; border-radius:60px; box-shadow:0 4px 10px rgba(0,0,0,.02),0 1px 2px
rgba(0,0,0,.05); display:flex; align-items:center; gap:16px; border:1px solid #e2edf2; }
.faculty-dean-avatar { background:#1e5a6f; width:44px; height:44px; border-radius:50%; display:flex; align-items:center; justify-
content:center; color:#fff; font-weight:600; font-size:1.2rem; }
.faculty-dean-info h4 { font-weight:600; font-size:.9rem; color:#2c4c6e; margin: 0; }
.faculty-dean-info p { font-size:.75rem; color:#4c6a82; font-weight:500; margin: 0; }
.faculty-stats-row { display:flex; gap:20px; margin-bottom:32px; flex-wrap:wrap; }
.faculty-stat-card { background:#fff; border-radius:24px; padding:14px 28px; box-shadow:0 2px 6px rgba(0,0,0,.04); display:flex; align-
items:center; gap:14px; border:1px solid #eef3f8; flex:1 1 auto; min-width: 200px; }
.faculty-stat-icon { background:#eaf4fc; width:48px; height:48px; border-radius:32px; display:flex; align-items:center; justify-
content:center; color:#1f6e8c; font-size:1.4rem; }
.faculty-stat-info h3 { font-size:1.6rem; font-weight:700; color:#1e405f; margin: 0; }
.faculty-stat-info span { font-size:.75rem; color:#5e7f9a; font-weight:500; }
.faculty-layout { display:flex; flex-direction:column; gap:32px; }
.faculty-card { background:#fff; border-radius:28px; box-shadow:0 12px 28px rgba(0,0,0,.05),0 0 0 1px rgba(0,0,0,.02); overflow:hidden; }
.faculty-card-header { padding:20px 28px 12px; border-bottom:1px solid #edf2f7; display:flex; justify-content:space-between; align-
items:center; flex-wrap:wrap; gap: 10px; }
.faculty-card-header h2 { font-size:1.35rem; font-weight:600; color:#1f3b4c; display:flex; align-items:center; gap:10px; margin: 0; }
.faculty-badge-manage { background:#eef2ff; padding:6px 14px; border-radius:30px; font-size:.7rem; font-weight:600; color:#2c5f8a; }
.faculty-form-grid { display:grid; grid-template-columns:repeat(auto-fit,minmax(280px,1fr)); gap:20px 28px; padding:28px; }
.faculty-input-group { display:flex; flex-direction:column; gap:8px; }
.faculty-input-group label { font-size:.75rem; font-weight:600; text-transform:uppercase; letter-spacing:.5px; color:#4a6b85; }
.faculty-input-group input, .faculty-input-group select { padding:12px 14px; border-radius:16px; border:1.5px solid #e2edf2;
background:#fefefe; font-size:.9rem; transition:all .2s; }
.faculty-input-group input:focus, .faculty-input-group select:focus { outline:none; border-color:#2c8cac; box-shadow:0 0 0 3px
rgba(44,140,172,.2); }
.faculty-row-flex { display:flex; gap:20px; flex-wrap:wrap; align-items:flex-end; grid-column: 1 / -1; }
.faculty-radio-group { display:flex; align-items:center; gap:18px; margin-top:8px; grid-column: 1 / -1; color:#2a445b; }
.faculty-save-btn { background:#1f6e8c; border:none; padding:12px 24px; border-radius:40px; font-weight:600; color:#fff; font-size:.9rem;
display:inline-flex; align-items:center; gap:12px; cursor:pointer; transition:.2s; }
.faculty-save-btn:hover { background:#0e556f; transform:translateY(-1px); box-shadow:0 6px 12px rgba(0,0,0,.05); }
.faculty-cancel-link { border-radius: 40px; padding: 10px 20px; }
.faculty-table-wrapper { overflow-x:auto; padding:0 0 8px; }
.faculty-table { width:100%; border-collapse:collapse; font-size:.85rem; }
.faculty-table th { text-align:left; padding:18px 16px; background:#f9fbfd; font-weight:600; color:#2c4c6e; border-bottom:1px solid

```

```

#e6edf2; }
.faculty-table td { padding:14px 16px; border-bottom:1px solid #eff3f8; vertical-align:middle; color:#2a445b; }
.faculty-table tr:hover td { background:#fafcff; }
.faculty-status-badge { display:inline-block; padding:4px 12px; border-radius:50px; font-size:.7rem; font-weight:600; text-
transform:capitalize; }
.faculty-status-active { background:#e0f2e9; color:#1f7840; }
.faculty-status-inactive { background:#ffe6e5; color:#b33; }
.faculty-action-buttons { display:flex; gap:8px; align-items:center; }
.faculty-icon-btn { border:none; border-radius:30px; background:transparent; color:#2c7da0; padding:6px 9px; }
.faculty-icon-btn.delete { color:#bc6c6c; }
.faculty-icon-btn:hover { background:#eef2ff; color:#0f5c7c; }
.faculty-icon-btn.delete:hover { background:#fff0f0; color:#b13e3e; }
.faculty-manage-programs-link { background:#f7f9fc; border-radius:20px; padding:8px 20px; font-size:.8rem; font-weight:500; color:#2a6f8f;
text-decoration:none; transition:.2s; }
.faculty-manage-programs-link:hover { background:#eaf3f8; color:#2a6f8f; }
.faculty-footer-note { margin-top:32px; text-align:center; font-size:.7rem; color:#6b89a0; border-top:1px solid #e0eaf0; padding-top:24px;
}
html[data-bs-theme="dark"] .faculty-title-section h1 { background: linear-gradient(135deg, #c5e6ff, #79c5e8); -webkit-background-
clip:text; background-clip:text; color: transparent; }
html[data-bs-theme="dark"] .faculty-title-section p { color: #9db6c8; }
html[data-bs-theme="dark"] .faculty-dean-card,
html[data-bs-theme="dark"] .faculty-stat-card,
html[data-bs-theme="dark"] .faculty-card { background: #16202a; border-color: #2a3947; box-shadow: 0 10px 24px rgba(0,0,0,.35); }
html[data-bs-theme="dark"] .faculty-dean-info h4,
html[data-bs-theme="dark"] .faculty-card-header h2,
html[data-bs-theme="dark"] .faculty-table th { color: #d3e7f6; }
html[data-bs-theme="dark"] .faculty-dean-info p,
html[data-bs-theme="dark"] .faculty-stat-info span,
html[data-bs-theme="dark"] .faculty-input-group label,
html[data-bs-theme="dark"] .faculty-footer-note { color: #9ab0c2; }
html[data-bs-theme="dark"] .faculty-stat-icon { background:#203346; color:#87c8ea; }
html[data-bs-theme="dark"] .faculty-stat-info h3 { color:#e3f3ff; }
html[data-bs-theme="dark"] .faculty-card-header { border-bottom-color: #2b3a49; }
html[data-bs-theme="dark"] .faculty-input-group input,
html[data-bs-theme="dark"] .faculty-input-group select { background:#0f1821; border-color:#2e4152; color:#deebf5; }
html[data-bs-theme="dark"] .faculty-input-group input:focus,
html[data-bs-theme="dark"] .faculty-input-group select:focus { border-color:#4ea4c6; box-shadow:0 0 3px rgba(78,164,198,.28); }
html[data-bs-theme="dark"] .faculty-radio-group { color:#c5d9e7; }
html[data-bs-theme="dark"] .faculty-manage-programs-link { background:#1d2b38; color:#9ed0eb; }
html[data-bs-theme="dark"] .faculty-manage-programs-link:hover { background:#253747; color:#b7ddf3; }
html[data-bs-theme="dark"] .faculty-badge-manage { background:#223243; color:#a9d2ea; }
html[data-bs-theme="dark"] .faculty-table th { background:#1b2834; border-bottom-color:#30414f; }
html[data-bs-theme="dark"] .faculty-table td { color:#c8dced; border-bottom-color:#2b3a47; }
html[data-bs-theme="dark"] .faculty-table tr:hover td { background:#1a2530; }
html[data-bs-theme="dark"] .faculty-status-active { background:#1d3a2b; color:#8fddb1; }
html[data-bs-theme="dark"] .faculty-status-inactive { background:#45262a; color:#f4aab1; }
html[data-bs-theme="dark"] .faculty-icon-btn { color:#90c7e5; }
html[data-bs-theme="dark"] .faculty-icon-btn:hover { background:#233344; color:#c4e6f9; }
html[data-bs-theme="dark"] .faculty-icon-btn.delete { color:#e09a9a; }
html[data-bs-theme="dark"] .faculty-icon-btn.delete:hover { background:#3f262b; color:#fffc4c; }
html[data-bs-theme="dark"] .faculty-footer-note { border-top-color:#2a3a49; }
@media (max-width: 700px) {
.faculty-card-header { flex-direction:column; align-items:flex-start; }
.faculty-stat-card { padding:10px 20px; }
}
</style>

<div class="faculty-dashboard-container">
  <div class="faculty-university-header">
    <div class="faculty-title-section">
      <h1><i class="fa-solid fa-chalkboard-user" style="color:#2c7da0; margin-right:8px;"></i>Faculty Management</h1>
      <p><?= htmlspecialchars($collegeName) ?> · Western Philippines University</p>
      <?php if ($programScope !== null): ?>
        <p>Program scope: <strong><?= htmlspecialchars($programScope) ?></strong></p>
      <?php endif; ?>
    </div>
    <div class="faculty-dean-card">
      <div class="faculty-dean-avatar"><?= htmlspecialchars($deanInitials) ?></div>
      <div class="faculty-dean-info">
        <h4><?= htmlspecialchars(strtoupper($deanName)) ?></h4>
        <p><?= is_program_chair() ? 'Program Chair' : 'Dean' ?>, <?= htmlspecialchars($collegeName) ?> <i class="fa-solid fa-circle-
check" style="font-size:.7rem; color:#1f8a4c;"></i></p>
      </div>
    </div>
  </div>

  <?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

  <?php if (!$hasEmploymentStatusColumn): ?>
    <div class="alert alert-warning py-2">

```

```

        Employment Status is in view-only fallback mode. Run <a href="upgrade_roles.php">upgrade_roles.php</a> to save status changes
(Permanent / Contract of Service / Temporary).
    </div>
    <?php endif; ?>

    <div class="faculty-stats-row">
        <div class="faculty-stat-card">
            <div class="faculty-stat-icon"><i class="fa-solid fa-users"></i></div>
            <div class="faculty-stat-info"><h3><?=$totalFacultyCount ?></h3><span>Total Faculty</span></div>
        </div>
        <div class="faculty-stat-card">
            <div class="faculty-stat-icon"><i class="fa-solid fa-user-check"></i></div>
            <div class="faculty-stat-info"><h3><?=$permanentCount ?></h3><span>Permanent</span></div>
        </div>
        <div class="faculty-stat-card">
            <div class="faculty-stat-icon"><i class="fa-solid fa-clock"></i></div>
            <div class="faculty-stat-info"><h3><?=$activeCount ?></h3><span>Active members</span></div>
        </div>
    </div>

    <div class="faculty-layout">
        <div class="faculty-card">
            <div class="faculty-card-header">
                <h2><i class="fa-solid fa-user-plus"></i> <?=$editRow ? 'Edit faculty' : 'Add faculty' ?></h2>
                <?php if ($programScope === null): ?>
                    <a href="dean_programs.php" class="faculty-manage-programs-link"><i class="fa-solid fa-layer-group me-1"></i>Manage
programs</a>
                <?php endif; ?>
            </div>
            <form method="post" class="faculty-form-grid">
                <input type="hidden" name="action" value="<?=$editRow ? 'edit' : 'add' ?>">
                <?php if ($editRow): ?><input type="hidden" name="id" value="<?=(int) $editRow['id'] ?>"<?php endif; ?>
                <div class="faculty-input-group"><label>Faculty ID</label><input type="text" name="faculty_id" required maxlength="20"
value="<?=$htmlspecialchars((string) ($editRow['faculty_id'] ?? '')) ?>"</div>
                <div class="faculty-input-group"><label>Username</label><input type="text" name="username" maxlength="50" <?=$editRow ?
'readonly' : 'required' ?> value="<?=$htmlspecialchars((string) ($editRow['username'] ?? '')) ?>"</div>
                <div class="faculty-input-group"><label><?=$editRow ? 'Reset Password (optional)' : 'Password' ?></label><input
type="password" name="<?=$editRow ? 'reset_password' : 'password' ?>" <?=$editRow ? ' : 'required' ?> autocomplete="new-password"></div>
                <div class="faculty-input-group"><label>Full Name</label><input type="text" name="full_name" required maxlength="100"
value="<?=$htmlspecialchars((string) ($editRow['full_name'] ?? '')) ?>"</div>

                <?php if ($programScope !== null): ?>
                    <div class="faculty-input-group">
                        <label>Program</label>
                        <input type="text" value="<?=$htmlspecialchars($programScope) ?>" readonly>
                        <input type="hidden" name="department" value="<?=$htmlspecialchars($programScope) ?>">
                    </div>
                <?php else: ?>
                    <?php
                    $currentDept = trim((string) ($editRow['department'] ?? ''));
                    $showProgramDropdown = $hasProgramsTable && $deanProgramOptions !== [];
                    $legacyProgram = $showProgramDropdown && $currentDept !== '' && !in_array($currentDept, $deanProgramOptions, true);
                    ?>
                    <div class="faculty-input-group">
                        <label>Program</label>
                        <?php if ($showProgramDropdown): ?>
                            <select name="department" required<?=$app_tooltip_attr('Active programs registered for your college under
Programs.') ?>>
                                <option value="" disabled<?=$currentDept === '' ? ' selected' : '' ?>>Select program...</option>
                                <?php foreach ($deanProgramOptions as $pn): ?>
                                    <option value="<?=$htmlspecialchars($pn) ?>"<?=$currentDept === $pn ? ' selected' : '' ?>><?=$
htmlspecialchars($pn) ?></option>
                                <?php endforeach; ?>
                                <?php if ($legacyProgram): ?>
                                    <option value="<?=$htmlspecialchars($currentDept) ?>" selected<?=$htmlspecialchars($currentDept) ?>
(current - not in active list)</option>
                                <?php endif; ?>
                            </select>
                        <?php else: ?>
                            <input type="text" name="department" maxlength="100" placeholder="e.g. BS Computer Science" value="<?=$
htmlspecialchars($currentDept) ?>"<?=$hasProgramsTable ? ' : ' required' ?>>
                        <?php endif; ?>
                    </div>
                <?php endif; ?>

                <div class="faculty-input-group"><label>Email</label><input type="email" name="email" maxlength="100" value="<?=$
htmlspecialchars((string) ($editRow['email'] ?? '')) ?>"</div>
                <div class="faculty-row-flex">
                    <div class="faculty-input-group" style="flex:1; min-width: 220px;"><label>Max Hrs/Day</label><input type="number"
name="max_hours_per_day" min="1" max="36" value="<?=(int) ($editRow['max_hours_per_day'] ?? 8) ?>"></div>

```

```

<div class="faculty-input-group" style="flex:1; min-width: 220px;">
    <label>Employment Status</label>
    <?php $selectedEmploymentStatus = (string) ($editRow['employment_status'] ?? 'Permanent'); ?>
    <select name="employment_status">
        <option value="Permanent" <?= $selectedEmploymentStatus === 'Permanent' ? 'selected' : '' ?>>Permanent</option>
        <option value="Contract of Service" <?= $selectedEmploymentStatus === 'Contract of Service' ? 'selected' : '' ?
>>Contract of Service</option>
        <option value="Temporary" <?= $selectedEmploymentStatus === 'Temporary' ? 'selected' : '' ?>>Temporary</option>
    </select>
</div>
</div>
<div class="faculty-radio-group">
    <label><strong>Status:</strong></label>
    <label><input type="radio" name="status" value="active" <?= ($editRow['status'] ?? 'active') === 'active' ? 'checked' :
'' ?>> Active</label>
    <label><input type="radio" name="status" value="inactive" <?= ($editRow['status'] ?? '') === 'inactive' ? 'checked' : ''
?>> Inactive</label>
</div>
<div style="display:flex; justify-content:flex-end; gap: 10px; grid-column: 1 / -1;">
    <?php if ($editRow): ?><a href="faculty.php" class="btn btn-outline-secondary faculty-cancel-link">Cancel</a><?php
endif; ?>
    <button type="submit" class="faculty-save-btn"<?= app_tooltip_attr($editRow ? 'Saves changes to this faculty record. Use
this after updating name, ID, or status.' : 'Adds the faculty member to your college roster. Use this before assigning courses or
accounts.') ?>><i class="fa-solid <?= $editRow ? 'fa-pen' : 'fa-save' ?>"></i> <?= $editRow ? 'Update Faculty' : 'Save Faculty' ?></button>
</div>
</form>
</div>

<div class="faculty-card">
    <div class="faculty-card-header">
        <h2><i class="fa-solid fa-list-ul"></i> Faculty roster</h2>
        <span class="faculty-badge-manage"><i class="fa-solid fa-pen me-1"></i>Manage faculty records</span>
    </div>
    <div class="faculty-table-wrapper">
        <table class="faculty-table">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Name</th>
                    <th>Program</th>
                    <?php if ($hasEmploymentStatusColumn): ?><th>Employment Status</th><?php endif; ?>
                    <th>Username</th>
                    <th>Email</th>
                    <th>Maxhrs/day</th>
                    <th>Status</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($list as $r): ?>
                    <tr>
                        <td><?= htmlspecialchars((string) $r['faculty_id']) ?></td>
                        <td><?= htmlspecialchars((string) $r['full_name']) ?></td>
                        <td><?= htmlspecialchars((string) ($r['department'] ?? '')) ?></td>
                        <?php if ($hasEmploymentStatusColumn): ?><td><?= htmlspecialchars((string) ($r['employment_status'] ??
'Permanent')) ?></td><?php endif; ?>
                        <td><?= htmlspecialchars((string) ($r['username'] ?? '')) ?></td>
                        <td><?= htmlspecialchars((string) ($r['email'] ?? '')) ?></td>
                        <td><?= (int) $r['max_hours_per_day'] ?></td>
                        <td>
                            <span class="faculty-status-badge <?= (($r['status'] ?? '') === 'active') ? 'faculty-status-active' :
'faculty-status-inactive' ?>">
                                <?= htmlspecialchars((string) $r['status']) ?>
                            </span>
                        </td>
                        <td class="faculty-action-buttons">
                            <a href="faculty.php?edit=<?= (int) $r['id'] ?>" class="faculty-icon-btn" title="Edit faculty"><i class="fa-
solid fa-pen"></i></a>
                            <form method="post" class="d-inline onsubmit="return confirm('Delete this faculty member? Their schedules
for this college will be removed too.');">
                                <input type="hidden" name="action" value="delete">
                                <input type="hidden" name="id" value="<?= (int) $r['id'] ?>">
                                <button type="submit" class="faculty-icon-btn delete" title="Delete faculty"><i class="fa-solid fa-
trash-can"></i></button>
                            </form>
                        </td>
                    </tr>
                </tbody>
            </table>
            <?php endforeach; ?>
            <?php if ($list === []): ?>
                <tr><td colspan="<?= $hasEmploymentStatusColumn ? '9' : '8' ?>" class="text-center text-muted py-4">No faculty yet.

```

```

Add one above.</td></tr>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>
</div>

<div class="faculty-footer-note">
    <i class="fa-solid fa-database"></i> Faculty directory · Real-time management | Western Philippines University
</div>
</div>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: faculty_classroom.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'program_chair', 'dean', 'gened']);

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}
if ($facultyId < 1 && in_array($_SESSION['role'] ?? '', ['program_chair', 'dean', 'gened'], true)) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    if ($facultyId > 0) {
        $_SESSION['faculty_id'] = $facultyId;
    }
}
if ($facultyId < 1) {
    exit('Faculty profile not linked to this account. Ask your dean to create/link your faculty profile.');
```

```

        throw new RuntimeException('Please enter a valid URL.');
```

```
    }
```

```

    $scheme = strtolower((string) (parse_url($url, PHP_URL_SCHEME) ?? ''));
    if (!in_array($scheme, ['http', 'https'], true)) {
        throw new RuntimeException('Only http and https URLs are allowed.');
```

```
    }
```

```
    return $url;
```

```
}
```

```
function faculty_datetime_save(?string $value): ?string
```

```
{
```

```
    $raw = trim((string) $value);
```

```
    if ($raw === '') {
        return null;
    }
```

```

    $raw = str_replace('T', ' ', $raw);
    if (strlen($raw) === 16) {
        $raw .= ':00';
    }
```

```
    return $raw;
```

```
}
```

```
$classroom = null;
```

```
if ($classroomId > 0 && $missingTables === []) {
```

```
    $st = db()->prepare(
        'SELECT oc.*, s.semester, s.school_year, s.day_of_week, s.start_time, s.end_time, s.id AS schedule_id, s.college_id,
           c.course_code, c.course_name
        FROM online_classrooms oc
        INNER JOIN schedules s ON s.id = oc.schedule_id
        INNER JOIN courses c ON c.id = oc.course_id
        WHERE oc.id = ? AND oc.faculty_id = ? AND s.faculty_id = ?
        LIMIT 1'
```

```
    );
    $st->execute([$classroomId, $facultyId, $facultyId]);
    $classroom = $st->fetch() ?: null;
```

```
}
```

```
if ($missingTables === [] && !$classroom) {
```

```
    http_response_code(404);
    exit('Classroom not found or you do not have access to it.');
```

```
}
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && $missingTables === [] && $classroom) {
    $action = (string) ($_POST['action'] ?? '');

```

```

    if ($action === 'open_meet') {
        try {
            faculty_open_meet_and_go_live($classroomId, $facultyId);
            header('Location: faculty_meet_live.php?classroom_id=' . $classroomId);
            exit;
        } catch (Throwable $e) {
            $_SESSION['flash'] = 'Error: ' . $e->getMessage();
            header('Location: faculty_classroom.php?id=' . $classroomId);
            exit;
        }
    }
}
```

```
try {
```

```

    if ($action === 'update_classroom') {
        $title = trim((string) ($_POST['title'] ?? ''));
        $description = trim((string) ($_POST['description'] ?? ''));
        $meetLink = faculty_classroom_manage_url((string) ($_POST['meet_link'] ?? ''));
        $status = (string) ($_POST['status'] ?? 'active');
        if ($title === '') {
            throw new RuntimeException('Classroom title is required.');
```

```
        }
```

```

        if (!in_array($status, ['active', 'archived'], true)) {
            $status = 'active';
        }
    }
```

```

    db()->prepare(
        'UPDATE online_classrooms
        SET title = ?, description = ?, meet_link = ?, status = ?
        WHERE id = ? AND faculty_id = ?'
    )->execute([
        $title,
```

```

        $description != '' ? $description : null,
        $meetLink,
        $status,
        $classroomId,
        $facultyId,
    ));

    if ($hasOnlineUrl) {
        db()->prepare('UPDATE schedules SET online_class_url = ? WHERE id = ? AND faculty_id = ?')
        ->execute([$meetLink != '' ? $meetLink : null, (int) $classroom['schedule_id'], $facultyId]);
    }

    $_SESSION['flash'] = 'Classroom details updated.';
} elseif ($action === 'regenerate_join_code') {
    if (!$hasJoinCode) {
        throw new RuntimeException('Run upgrade_roles.php to enable class join codes.');
```

their views.';


```

        WHERE id = ? AND faculty_id = ?'
    )->execute([$classroomId, $facultyId]);
    $classroom['syllabus_stored_name'] = null;
    $classroom['syllabus_original_name'] = null;
    $classroom['syllabus_mime'] = null;
    $_SESSION['flash'] = 'Syllabus removed. Upload a new syllabus before posting course content or announcements.';
} elseif ($action === 'upload_banner' || $action === 'delete_banner') {
    $bannerFlash = faculty_classroom_process_banner_post($classroomId, $facultyId, $classroom);
    if ($bannerFlash !== null) {
        $_SESSION['flash'] = $bannerFlash;
    }
} elseif ($action === 'add_content') {
    $contentType = (string) ($_POST['content_type'] ?? 'material');
    $title = trim((string) ($_POST['title'] ?? ''));
    $body = classroom_content_prepare_body((string) ($_POST['body'] ?? ''));
    $weeks = trim((string) ($_POST['weeks'] ?? ''));
    $daysPerTopic = trim((string) ($_POST['days_per_topic'] ?? ''));
    $resourceUrl = faculty_classroom_manage_url((string) ($_POST['resource_url'] ?? ''));
    $uploadedFiles = isset($_FILES['attachments']) && is_array($_FILES['attachments'])
        ? classroom_content_normalize_uploads($_FILES['attachments'])
        : [];
    $actualUploadedFiles = array_values(array_filter(
        $uploadedFiles,
        static fn (array $file): bool => (int) ($file['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_NO_FILE
    ));

    if (!in_array($contentType, ['material', 'link', 'announcement'], true)) {
        $contentType = 'material';
    }
    if ($hasSyllabusCols) {
        $sySt = db()->prepare(
            'SELECT syllabus_stored_name FROM online_classrooms WHERE id = ? AND faculty_id = ? LIMIT 1'
        );
        $sySt->execute([$classroomId, $facultyId]);
        $syName = trim((string) ($sySt->fetchColumn() ?? ''));
        if ($syName === '') {
            throw new RuntimeException('Upload the course syllabus before posting course content or announcements.');
```

```

        'INSERT INTO classroom_content (classroom_id, faculty_id, content_type, title, body, resource_url)
        VALUES (?, ?, ?, ?, ?, ?)'
    )->execute([
        $classroomId,
        $facultyId,
        $contentType,
        $title,
        $body !== '' ? $body : null,
        $legacyAttachmentResource ?? $resourceUrl,
    ]);
}
$contentId = (int) db()->lastInsertId();

if ($attachments !== []) {
    $insertAttachment = db()->prepare(
        'INSERT INTO classroom_content_attachments (content_id, original_name, stored_name, mime)
        VALUES (?, ?, ?, ?)'
    );
    foreach ($attachments as $attachment) {
        $insertAttachment->execute([
            $contentId,
            $attachment['original_name'],
            $attachment['stored_name'],
            $attachment['mime'],
        ]);
    }
}

db()->commit();
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    if ($legacyAttachmentResource !== null) {
        $attachmentPath = classroom_content_attachment_path($legacyAttachmentResource);
        if (is_file($attachmentPath)) {
            @unlink($attachmentPath);
        }
    }
    foreach ($attachments as $attachment) {
        $attachmentPath = classroom_content_attachment_storage_path((string) ($attachment['stored_name'] ?? ''));
        if (is_file($attachmentPath)) {
            @unlink($attachmentPath);
        }
    }
    throw $e;
}

$_SESSION['flash'] = $contentType === 'announcement'
    ? 'Announcement published for enrolled students.'
    : 'Course content added.';
} elseif ($action === 'delete_content') {
    $contentId = (int) ($_POST['content_id'] ?? 0);
    $st = db()->prepare(
        'SELECT resource_url
        FROM classroom_content
        WHERE id = ? AND classroom_id = ? AND faculty_id = ?
        LIMIT 1'
    );
    $st->execute([$contentId, $classroomId, $facultyId]);
    $resourceUrl = (string) ($st->fetchColumn() ?: '');
    $attachmentRows = [];
    if ($hasContentAttachments) {
        $st = db()->prepare(
            'SELECT stored_name
            FROM classroom_content_attachments
            WHERE content_id = ?'
        );
        $st->execute([$contentId]);
        $attachmentRows = $st->fetchAll();
    }

    db()->prepare(
        'DELETE FROM classroom_content WHERE id = ? AND classroom_id = ? AND faculty_id = ?'
    )->execute([$contentId, $classroomId, $facultyId]);

    if (classroom_content_is_attachment($resourceUrl)) {
        $attachmentPath = classroom_content_attachment_path($resourceUrl);
        if (is_file($attachmentPath)) {
            @unlink($attachmentPath);
        }
    }
}

```

```

    }
}
foreach ($attachmentRows as $attachmentRow) {
    $attachmentPath = classroom_content_attachment_storage_path((string) ($attachmentRow['stored_name'] ?? ''));
    if (is_file($attachmentPath)) {
        @unlink($attachmentPath);
    }
}
$_SESSION['flash'] = 'Content removed.';
} elseif ($action === 'add_assessment') {
    $assessmentType = (string) ($_POST['assessment_type'] ?? 'assignment');
    $title = trim((string) ($_POST['title'] ?? ''));
    $description = trim((string) ($_POST['description'] ?? ''));
    $totalPoints = (float) ($_POST['total_points'] ?? 0);
    $dueAt = faculty_datetime_save((string) ($_POST['due_at'] ?? ''));

    if (!in_array($assessmentType, ['assignment', 'quiz'], true)) {
        $assessmentType = 'assignment';
    }
    if ($title === '') {
        throw new RuntimeException('Assessment title is required.');
```

```

        ->execute([$assessmentId, $studentId]);
        continue;
    }

    $score = $scoreText === '' ? null : (float) $scoreText;
    db()->prepare(
        'INSERT INTO classroom_scores (assessment_id, student_id, score, feedback, graded_at)
        VALUES (?, ?, ?, ?, NOW())
        ON DUPLICATE KEY UPDATE score = VALUES(score), feedback = VALUES(feedback), graded_at = NOW()'
    )->execute([
        $assessmentId,
        $studentId,
        $score,
        $feedbackText !== '' ? $feedbackText : null,
    ]);
}

$_SESSION['flash'] = 'Grades saved successfully.';
}
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}

header('Location: faculty_classroom.php?id=' . $classroomId);
exit;
}

$contentItems = [];
$contentAttachmentMap = [];
$assessments = [];
$scoreMap = [];
$submissionMap = [];
$overallPoints = 0.0;
$contentWeeks = [];

if ($missingTables === [] && $classroom) {
    $st = db()->prepare(
        'SELECT *
        FROM classroom_content
        WHERE classroom_id = ? AND faculty_id = ?
        ORDER BY created_at DESC'
    );
    $st->execute([$classroomId, $facultyId]);
    $contentItems = $st->fetchAll();
    $contentWeeks = classroom_content_group_by_week($contentItems);
    if ($hasContentAttachments) {
        $contentAttachmentMap = classroom_content_attachment_map(array_column($contentItems, 'id'));
    }

    $st = db()->prepare(
        'SELECT *
        FROM classroom_assessments
        WHERE classroom_id = ? AND faculty_id = ?
        ORDER BY created_at DESC'
    );
    $st->execute([$classroomId, $facultyId]);
    $assessments = $st->fetchAll();

    $overallPoints = array_reduce(
        $assessments,
        static fn (float $carry, array $item): float => $carry + (float) $item['total_points'],
        0.0
    );
}

$pageTitle = $classroom ? 'Manage Classroom' : 'Classroom';
$syllabusBlocksContent = $classroom && $hasSyllabusCols && trim((string) ($classroom['syllabus_stored_name'] ?? '')) === '' ?
    $syllabusOriginalName = trim((string) ($classroom['syllabus_original_name'] ?? ''));
require_once __DIR__ . '/includes/header.php';

$assessmentQuizCount = count(array_filter(
    $assessments,
    static fn (array $a): bool => ($a['assessment_type'] ?? '') === 'quiz'
));
$assessmentAssignmentCount = count(array_filter(
    $assessments,
    static fn (array $a): bool => ($a['assessment_type'] ?? '') === 'assignment'
));
$meetHref = $classroom ? trim((string) $classroom['meet_link']) : '';
$statusIsActive = $classroom && (string) $classroom['status'] === 'active';

```

```

?>

<div class="fc-manage container-fluid px-0">
<?php if ($classroom): ?>
    <?php faculty_classroom_render_banner($classroom, ['form_id' => 'facultyClassroomBanner']); ?>
<?php else: ?>
    <h1 class="fc-page-title mb-4">
        <i class="fa-solid fa-chalkboard-user me-2" style="color: var(--fc-accent);"></i>Classroom
    </h1>
<?php endif; ?>

    <div class="d-flex flex-wrap justify-content-end align-items-start gap-2 mb-4">
        <a href="faculty_classrooms.php" class="btn btn-outline-secondary btn-sm rounded-pill"><? app_tooltip_attr('Returns to the list of
all your online classes.') ?><i class="fa-solid fa-arrow-left me-1"></i>All classrooms</a>
        <a href="faculty_classroom_attendance.php?id=<?=(int) $classroomId ?>" class="btn btn-outline-secondary btn-sm rounded-pill"><? app_tooltip_attr('Opens attendance records and auto-check for this class.') ?><i class="fa-solid fa-user-check me-1"></i>Attendance</a>
        <a href="faculty_classroom_assessments.php?id=<?=(int) $classroomId ?>" class="btn btn-outline-primary btn-sm rounded-pill"><? app_tooltip_attr('Manage quizzes, assignments, and student submissions for this class.') ?><i class="fa-solid fa-clipboard-list me-1">
</i>Grading & submissions</a>
        <?php if ($meetHref !== ''): ?>
            <form method="post" action="faculty_classroom.php?id=<?=(int) $classroomId ?>" target="_blank" rel="noopener noreferrer"
class="d-inline">
                <input type="hidden" name="action" value="open_meet">
                <input type="hidden" name="classroom_id" value="<?=(int) $classroomId ?>">
                <button type="submit" class="btn btn-success btn-sm rounded-pill"><? app_tooltip_attr('Opens Google Meet in a new tab and
marks this class as live so students can join.') ?><i class="fa-solid fa-video me-1"></i>Join Meet</button>
            </form>
        <?php endif; ?>
    </div>

<?php if ($hasJoinCode && $classroom): ?>
    <div class="card fc-section-card border-0 shadow-sm mb-4">
        <div class="card-body p-4">
            <h2 class="h6 text-muted text-uppercase mb-3"><i class="fa-solid fa-key me-2"></i>Student join code</h2>
            <p class="small text-muted mb-3">Students in your college sign in, open <strong>My Classes</strong>, and enter this code to add
this course to their list.</p>
            <div class="d-flex flex-wrap align-items-center gap-2 mb-3">
                <?php $jc = trim((string) ($classroom['join_code'] ?? '')); ?>
                <?php if ($jc !== ''): ?>
                    <code class="fc-join-code-pill" id="fc-join-code"><?= htmlspecialchars($jc) ?></code>
                    <button type="button" class="btn btn-outline-secondary btn-sm" id="fc-copy-join" data-code="<?= htmlspecialchars($jc) ?>
"><? app_tooltip_attr('Copies the join code so you can paste it into chat or email for students.') ?>>Copy</button>
                <?php else: ?>
                    <span class="text-muted">No code yet – generate one for students to use.</span>
                <?php endif; ?>
            </div>
            <form method="post" class="d-inline" onsubmit="return confirm('<?= $jc !== '' ? 'Generate a new code? The old code will stop
working.' : 'Generate a join code for this class?' ?>');">
                <input type="hidden" name="action" value="regenerate_join_code">
                <input type="hidden" name="classroom_id" value="<?=(int) $classroomId ?>">
                <button type="submit" class="btn btn-outline-warning btn-sm"><? app_tooltip_attr('$jc !== '' ? 'Creates a new join code and
invalidates the old one. Use if the code was shared too widely.' : 'Creates a join code students can enter under My Classes to enroll.') ?>>
<?= $jc !== '' ? 'Regenerate code' : 'Generate code' ?></button>
            </form>
        </div>
    </div>
</div>
<script>
(function () {
    var btn = document.getElementById('fc-copy-join');
    if (!btn) return;
    btn.addEventListener('click', function () {
        var c = btn.getAttribute('data-code') || '';
        if (!c) return;
        if (navigator.clipboard && navigator.clipboard.writeText) {
            navigator.clipboard.writeText(c).then(function () {
                btn.textContent = 'Copied';
                setTimeout(function () { btn.textContent = 'Copy'; }, 2000);
            });
        }
    });
})();
</script>
<?php endif; ?>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if ($missingTables !== []): ?>
    <div class="alert alert-warning border-0 shadow-sm">
        Classroom/student features are not installed yet. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this page.
    </div>

```

```

<?php else: ?>
    <form method="post" action="faculty_classroom.php?id=<?=(int) $classroomId ?>" target="_blank" rel="noopener noreferrer" class="d-none"
    id="fc-open-meet-form">
        <input type="hidden" name="action" value="open_meet">
        <input type="hidden" name="classroom_id" value="<?=(int) $classroomId ?>">
    </form>
    <form method="post" class="card fc-section-card mb-4">
        <input type="hidden" name="action" value="update_classroom">
        <input type="hidden" name="classroom_id" value="<?=(int) $classroomId ?>">
        <div class="card-header">
            <h2 class="fc-section-title"><i class="fa-solid fa-door-open" aria-hidden="true"></i>Classroom header</h2>
            <p class="small text-muted mb-0 mt-2">Title, visibility status, and live session link students see for this class.</p>
        </div>
        <div class="card-body p-4">
            <div class="row g-3 mb-4">
                <div class="col-md-6">
                    <label class="form-label fw-semibold" for="fc-class-title">Class title</label>
                    <input id="fc-class-title" type="text" name="title" class="form-control form-control-lg" maxlength="150" required
                    placeholder="e.g. BIO 101 – Laboratory Section A"
                    value="<?=(string) $classroom['title'] ?>">
                </div>
                <div class="col-md-3">
                    <label class="form-label fw-semibold" for="fc-class-status">Status</label>
                    <select id="fc-class-status" name="status" class="form-select form-select-lg" aria-describedby="fc-status-hint">
                        <option value="active" <?=(string) $classroom['status'] === 'active' ? 'selected' : '' ?>>Active – open to
                        students</option>
                        <option value="archived" <?=(string) $classroom['status'] === 'archived' ? 'selected' : '' ?>>Archived – read-
                        only</option>
                    </select>
                    <div id="fc-status-hint" class="form-text">Archived classes stay visible to you but are clearly marked for students.
                </div>
            </div>
            <div class="col-md-3">
                <label class="form-label fw-semibold" for="fc-meet-link">Meet link</label>
                <input id="fc-meet-link" type="url" name="meet_link" class="form-control form-control-lg"
                placeholder="https://meet.google.com/xxx-xxxx-xxx" value="<?=(string) $classroom['meet_link'] ?>" autocomplete="url">
            </div>
        </div>
        <?php if ($classroom): ?>
            <div class="d-flex flex-wrap align-items-center gap-2 mb-4 pb-3 border-bottom">
                <span class="small text-muted text-uppercase fw-semibold me-1">Preview</span>
                <?php if ($statusIsActive): ?>
                    <span class="badge rounded-pill bg-success-subtle text-success border border-success-subtle"><i class="fa-solid fa-
                    circle-check me-1"></i>Active</span>
                <?php else: ?>
                    <span class="badge rounded-pill bg-secondary-subtle text-secondary border"><i class="fa-solid fa-box-archive me-1">
                    </i>Archived</span>
                <?php endif; ?>
                <?php if ($meetHref !== ''): ?>
                    <button type="submit" form="fc-open-meet-form" class="btn btn-sm btn-outline-primary rounded-pill"<?=(
                    app_tooltip_attr('Opens Google Meet in a new tab and marks this class as live so students can join.') ?>><i class="fa-solid fa-video me-1">
                    </i>Open Meet</button>
                <?php else: ?>
                    <span class="fc-placeholder-hint"><i class="fa-regular fa-circle-question me-1"></i>No Meet link yet – add one
                    above.</span>
                <?php endif; ?>
            </div>
            <?php endif; ?>
            <div class="fc-subsection">
                <div class="fc-subsection-label" id="fc-desc-heading"><i class="fa-solid fa-align-left" aria-hidden="true">
                </i>Description</div>
                <label class="visually-hidden" for="fc-description">Class description</label>
                <textarea id="fc-description" name="description" class="form-control" rows="4" aria-labelledby="fc-desc-heading"
                placeholder="Briefly describe goals, expectations, office hours, or how students should use this space. Students
                see this on the class overview."><?=(string) ($classroom['description'] ?? '') ?></textarea>
                <?php if (trim((string) ($classroom['description'] ?? '')) === ''): ?>
                    <p class="fc-placeholder-hint mt-2 mb-0"><i class="fa-regular fa-lightbulb me-1"></i>Empty for now – add a short welcome
                    or syllabus summary.</p>
                <?php endif; ?>
            </div>
            <div class="d-flex flex-wrap gap-2 pt-2 border-top">
                <button type="submit" class="btn btn-primary fc-btn-primary-lg"<?=(app_tooltip_attr('Saves title, status, Meet link, and
                description shown to students.') ?>><i class="fa-solid fa-floppy-disk me-2"></i>Save class details</button>
            </div>
        </div>
    </form>

    <div class="card fc-section-card border-0 shadow-sm mb-4">

```

```

<div class="card-header">
  <h2 class="fc-section-title"><i class="fa-solid fa-file-contract" aria-hidden="true"></i>Course syllabus</h2>
  <p class="small text-muted mb-0 mt-2">Required before you can publish <strong>course content</strong> or
<strong>announcements</strong> (after the database step below). Students, dean, program chair, and admin can open it in a separate browser
tab.</p>
</div>
<div class="card-body p-4">
  <?php if (!$hasSyllabusCols): ?>
    <div class="alert alert-warning border-0 shadow-sm mb-0">
      <i class="fa-solid fa-screwdriver-wrench me-2"></i>Syllabus storage is not enabled on the database yet. Open <a
href="upgrade_roles.php" class="alert-link">upgrade_roles.php</a> once (submit the upgrade form), then reload this page – the upload form
and posting rules will appear here.
    </div>
  <?php else: ?>
    <?php if ($syllabusOriginalName != ''): ?>
      <div class="d-flex flex-wrap align-items-center gap-2 mb-3">
        <span class="small text-muted text-uppercase fw-semibold">Current file</span>
        <span class="fw-semibold"><? = htmlspecialchars($syllabusOriginalName) ?></span>
        <a href="<? = htmlspecialchars(classroom_syllabus_href((int) $classroomId)) ?>" target="_blank" rel="noopener
noreferrer" class="btn btn-sm btn-outline-primary rounded-pill"><? = app_tooltip_attr('Opens the syllabus in a new tab so you can confirm what
students and leadership will see.') ?><i class="fa-solid fa-up-right-from-square me-1"></i>Preview in new tab</a>
      </div>
    <?php else: ?>
      <div class="alert alert-warning border-0 shadow-sm mb-3">
        <i class="fa-solid fa-triangle-exclamation me-2"></i>No syllabus yet – upload one to unlock posting for this class.
      </div>
    <?php endif; ?>
    <form method="post" enctype="multipart/form-data" class="row g-3 align-items-end">
      <input type="hidden" name="action" value="upload_syllabus">
      <input type="hidden" name="classroom_id" value="<? = (int) $classroomId ?>">
      <div class="col-md-8">
        <label class="form-label fw-semibold" for="fc-syllabus-file">Upload or replace syllabus</label>
        <input id="fc-syllabus-file" type="file" name="syllabus" class="form-control"
accept=".pdf,.doc,.docx,.ppt,.pptx,.txt" required>
        <div class="form-text">PDF recommended for viewing in the browser (max 10 MB). Other types may download instead of
preview.</div>
      </div>
      <div class="col-md-4 d-flex flex-wrap gap-2 align-items-end">
        <button type="submit" class="btn btn-primary"><? = app_tooltip_attr('Saves this file as the official syllabus for
oversight and students.') ?><i class="fa-solid fa-upload me-1"></i>Save syllabus</button>
      </div>
    </form>
    <?php if ($syllabusOriginalName != ''): ?>
      <form method="post" class="mt-2" onsubmit="return confirm('Remove the syllabus? You must upload again before posting new
content or announcements.');">
        <input type="hidden" name="action" value="delete_syllabus">
        <input type="hidden" name="classroom_id" value="<? = (int) $classroomId ?>">
        <button type="submit" class="btn btn-sm btn-outline-danger"><? = app_tooltip_attr('Deletes the syllabus file from this
class.') ?><i class="fa-solid fa-trash me-1"></i>Remove syllabus</button>
      </form>
    <?php endif; ?>
  <?php endif; ?>
</div>
</div>

<div class="row g-4">
  <div class="col-12">
    <div class="card fc-section-card mb-4">
      <div class="card-header">
        <h2 class="fc-section-title"><i class="fa-solid fa-folder-open" aria-hidden="true"></i>Course content &
announcements</h2>
        <p class="small text-muted mb-0 mt-2">Post materials, links, or announcements. Choose a week so items group cleanly for
students.</p>
      </div>
      <div class="card-body p-4">
        <?php if ($syllabusBlocksContent): ?>
          <div class="alert alert-warning border-0 shadow-sm mb-3">
            <i class="fa-solid fa-lock me-2"></i>Posting is locked until you upload a <strong>course syllabus</strong> in
the section above.
          </div>
        <?php endif; ?>
        <span class="d-none"
data-offline-classroom-label="<? = htmlspecialchars(trim((string) ($classroom['course_code'] ?? '')) . ' - ' .
(string) ($classroom['title'] ?? 'Classroom')) ?>"
data-offline-course-code="<? = htmlspecialchars((string) ($classroom['course_code'] ?? '')) ?>"
data-offline-course-name="<? = htmlspecialchars((string) ($classroom['course_name'] ?? '')) ?>"
data-offline-classroom-title="<? = htmlspecialchars((string) ($classroom['title'] ?? 'Classroom')) ?>"></span>
        <p class="small mb-3" data-offline-status></p>
        <div class="d-flex flex-wrap align-items-center gap-2 mb-3">
          <a href="faculty_offline.php" class="btn btn-outline-secondary btn-sm"><? = app_tooltip_attr('View posts and files

```

```

queued on this device while offline, and sync them when you are online.') ?>>
    <i class="fa-solid fa-cloud-arrow-up me-1"></i>Offline uploads
    <span class="badge text-bg-warning ms-1" data-offline-pending-count hidden>0</span>
  </a>
  <span class="small text-muted">Works offline: publish or attach files without internet, then sync later.</span>
</div>
<form method="post" enctype="multipart/form-data" id="fc-add-content-form">
  <input type="hidden" name="action" value="add_content">
  <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">

  <div class="fc-subsection">
    <div class="fc-subsection-label"><i class="fa-solid fa-pen-to-square" aria-hidden="true"></i>Post details</div>
    <div class="row g-3">
      <div class="col-md-4">
        <label class="form-label fw-semibold" for="fc-content-type">Content type</label>
        <select id="fc-content-type" name="content_type" class="form-select">
          <option value="material">Material</option>
          <option value="link">Link</option>
          <option value="announcement">Announcement</option>
        </select>
      </div>
      <div class="col-md-8">
        <label class="form-label fw-semibold" for="fc-content-title">Title</label>
        <input id="fc-content-title" type="text" name="title" class="form-control" maxlength="150" required
placeholder="e.g. Week 3 readings or Important exam date">
      </div>
      <?php if ($hasContentTopicSchedule): ?>
        <div class="col-md-6">
          <label class="form-label fw-semibold" for="fc-content-weeks">Week</label>
          <select id="fc-content-weeks" name="weeks" class="form-select">
            <option value="">General resources (no specific week)</option>
            <?php for ($weekNo = 1; $weekNo <= 18; $weekNo++): ?>
              <option value="Week <?= $weekNo ?>">Week <?= $weekNo ?></option>
            <?php endfor; ?>
          </select>
          <div class="form-text">Match week labels so the week view stays organized.</div>
        </div>
        <div class="col-md-6">
          <label class="form-label fw-semibold" for="fc-days-topic">Days per topic</label>
          <input id="fc-days-topic" type="text" name="days_per_topic" class="form-control" maxlength="100"
placeholder="e.g. 2 class days">
        </div>
      <?php else: ?>
        <div class="col-12">
          <div class="alert alert-light border small mb-0"><i class="fa-solid fa-screwdriver-wrench me-2 text-
warning"></i>Run <code>upgrade_roles.php</code> once to enable <strong>Week</strong> and <strong>Days per topic</strong> fields.</div>
        </div>
      <?php endif; ?>
      <div class="col-12">
        <label class="form-label fw-semibold" for="fc-body-textarea">Description & notes</label>
        <div class="wordpad-shell" data-wordpad data-wordpad-name="body">
          <div class="wordpad-toolbar d-none" role="toolbar" aria-label="Formatting toolbar">
            <select class="form-select form-select-sm wordpad-select" data-wordpad-block aria-label="Text
style">
              <option value="<p>">Normal text</option>
              <option value="<h3>">Heading</option>
              <option value="<blockquote>">Quote</option>
            </select>
            <select class="form-select form-select-sm wordpad-select wordpad-select--font" data-wordpad-
font-family aria-label="Font"<?= app_tooltip_attr('Apply a standard font to the selected text.') ?>>
              <option value="">Font</option>
              <option value="Arial, Helvetica, sans-serif">Arial</option>
              <option value="'Times New Roman', Times, serif">Times New Roman</option>
              <option value="Georgia, serif">Georgia</option>
              <option value="'Courier New', Courier, monospace">Courier New</option>
              <option value="Verdana, sans-serif">Verdana</option>
              <option value="Tahoma, sans-serif">Tahoma</option>
            </select>
            <select class="form-select form-select-sm wordpad-select wordpad-select--size" data-wordpad-
font-size aria-label="Font size"<?= app_tooltip_attr('Change the font size of the selected text.') ?>>
              <option value="">Size</option>
              <option value="12px">Small (12)</option>
              <option value="14px">Normal (14)</option>
              <option value="16px">Medium (16)</option>
              <option value="18px">Large (18)</option>
              <option value="24px">Extra large (24)</option>
              <option value="32px">Title (32)</option>
            </select>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="bold" title="Bold"
<?= app_tooltip_attr('Makes selected text bold in the rich description area.') ?>><i class="fa-solid fa-bold"></i></button>

```



```

        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="italic"
title="Italic">?= app_tooltip_attr('Italicizes selected text for emphasis in the description.') ?>><i class="fa-solid fa-italic"></i>
</button>

        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="underline"
title="Underline">?= app_tooltip_attr('Underlines selected text in the description.') ?>><i class="fa-solid fa-underline"></i></button>
        <button type="button" class="btn btn-outline-secondary btn-sm" data-
command="insertUnorderedList" title="Bulleted list">?= app_tooltip_attr('Starts or continues a bulleted list in the description.') ?>><i
class="fa-solid fa-list-ul"></i></button>
        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="insertOrderedList"
title="Numbered list">?= app_tooltip_attr('Starts or continues a numbered list in the description.') ?>><i class="fa-solid fa-list-ol"></i>
</button>
        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="justifyLeft"
title="Align left">?= app_tooltip_attr('Aligns paragraph text to the left.') ?>><i class="fa-solid fa-align-left"></i></button>
        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="justifyCenter"
title="Align center">?= app_tooltip_attr('Centers paragraph text.') ?>><i class="fa-solid fa-align-center"></i></button>
        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="justifyRight"
title="Align right">?= app_tooltip_attr('Aligns paragraph text to the right.') ?>><i class="fa-solid fa-align-right"></i></button>
        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="createLink"
title="Insert link">?= app_tooltip_attr('Turns the selection into a hyperlink. Use for readings or external resources.') ?>><i class="fa-
solid fa-link"></i></button>
        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="unlink"
title="Remove link">?= app_tooltip_attr('Removes the link from the selected text without deleting the text.') ?>><i class="fa-solid fa-link-
slash"></i></button>
        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="insertImage"
title="Insert image">?= app_tooltip_attr('Insert a JPEG, PNG, GIF, or WebP image into the notes. You can also paste an image from the
clipboard.') ?>><i class="fa-solid fa-image"></i></button>
        <button type="button" class="btn btn-outline-secondary btn-sm" data-command="removeFormat"
title="Clear formatting">?= app_tooltip_attr('Strips bold, italics, and other formatting from the selection.') ?>><i class="fa-solid fa-
eraser"></i></button>
    </div>
    <div class="wordpad-editor form-control d-none" contenteditable="true" data-placeholder="Add
instructions, context, or a short message for students..."></div>
    <textarea id="fc-body-textarea" name="body" class="form-control" rows="5" placeholder="Plain text if
the rich editor is unavailable"></textarea>
    </div>
    <div class="form-text">Optional formatting for longer notes (bold, lists, links, images). Paste or
insert images – they are saved with the post.</div>
    </div>
</div>

    <div class="fc-subsection">
        <div class="fc-subsection-label"><i class="fa-solid fa-paperclip" aria-hidden="true"></i>File upload &amp;
links</div>
        <div class="fc-inner-panel">
            <div class="row g-3">
                <div class="col-12">
                    <label class="form-label fw-semibold" for="fc-resource-url">Resource URL</label>
                    <input id="fc-resource-url" type="url" name="resource_url" class="form-control"
placeholder="https://... (slides, video, reading)" autocomplete="url">
                    <div class="form-text">Link to an external page or file; optional if you attach files below.</div>
                </div>
                <div class="col-12">
                    <label class="form-label fw-semibold" for="fc-attachments">Attachments</label>
                    <input id="fc-attachments" type="file" name="attachments[]" class="form-control"
accept=".pdf,.doc,.docx,.xls,.xlsx,.ppt,.pptx,.txt,.csv,.jpg,.jpeg,.png" multiple>
                    <div class="form-text">PDF, Office, images – up to 10 MB each. Leave empty if you only use a URL or
description.</div>
                    <?php if (!$hasContentAttachments): ?>
                        <div class="form-text text-warning"><i class="fa-solid fa-triangle-exclamation me-1"></i>Run
<code>upgrade_roles.php</code> to allow multiple attachments per post.</div>
                    <?php endif; ?>
                </div>
            </div>
        </div>
    </div>

    <div class="d-flex flex-wrap align-items-center gap-2 pt-1">
        <button type="submit" class="btn btn-primary fc-btn-primary-lg">?= $syllabusBlocksContent ? ' disabled' : '' ?>
        <?= app_tooltip_attr('Publishes this post, files, and links to the class stream for enrolled students.') ?>><i class="fa-solid fa-paper-
plane me-2"></i>Publish to class</button>
        <span class="small text-muted">Students see new posts on their class stream.</span>
    </div>
</form>

<hr class="my-4 text-muted opacity-25">

<div class="fc-subsection mb-0">
    <div class="fc-subsection-label"><i class="fa-solid fa-calendar-week" aria-hidden="true"></i>Browse by week</div>
    <?php if ($contentWeeks === []): ?>

```

```

        <p class="fc-placeholder-hint mb-0"><i class="fa-regular fa-folder-open me-2"></i>No published items yet. After
you publish content with a week label, weeks appear here for quick editing.</p>
        <?php else: ?>
            <p class="small text-muted mb-3">Open a week to edit materials and topics for that period.</p>
            <?php foreach ($contentWeeks as $group): ?>
                <a class="fc-week-link" href="faculty_classroom_week.php?id=<?=(int) $classroomId ?>&week=<?=(int) $group['label'] ?>">
                    <span><i class="fa-regular fa-calendar me-2 text-primary"></i><strong><?=(int) $group['count'] ?> <?=(int) $group['count']
=== 1 ? 'item' : 'items' ?></span>
                </a>
            <?php endforeach; ?>
        <?php endif; ?>
    </div>
</div>

<div class="card fc-section-card mb-4">
    <div class="card-header d-flex flex-wrap justify-content-between align-items-start gap-2">
        <div>
            <h2 class="fc-section-title mb-0"><i class="fa-solid fa-clipboard-check" aria-hidden="true"></i>Assessments</h2>
            <p class="small text-muted mb-0 mt-2">Assignments, quizzes, total points, and grading live on the full assessments
page.</p>
        </div>
        <a href="faculty_classroom_assessments.php?id=<?=(int) $classroomId ?>" class="btn btn-outline-primary btn-sm rounded-
pill flex-shrink-0"><i class="fa-solid fa-arrow-up-right-from-square me-1"></i>Open assessments</a>
    </div>
    <div class="card-body p-4">
        <div class="row g-3">
            <div class="col-6 col-md-3">
                <div class="fc-stat-tile">
                    <div class="fc-stat-icon"><i class="fa-solid fa-list-check"></i></div>
                    <div class="fc-stat-label">Total</div>
                    <div class="fc-stat-value"><?=(int) $assessmentCount ?></div>
                    <div class="small text-muted">Assessments</div>
                </div>
            </div>
            <div class="col-6 col-md-3">
                <div class="fc-stat-tile">
                    <div class="fc-stat-icon"><i class="fa-solid fa-file-lines"></i></div>
                    <div class="fc-stat-label">Assignments</div>
                    <div class="fc-stat-value"><?=(int) $assessmentAssignmentCount ?></div>
                </div>
            </div>
            <div class="col-6 col-md-3">
                <div class="fc-stat-tile">
                    <div class="fc-stat-icon"><i class="fa-solid fa-circle-question"></i></div>
                    <div class="fc-stat-label">Quizzes</div>
                    <div class="fc-stat-value"><?=(int) $assessmentQuizCount ?></div>
                </div>
            </div>
            <div class="col-6 col-md-3">
                <div class="fc-stat-tile">
                    <div class="fc-stat-icon"><i class="fa-solid fa-star"></i></div>
                    <div class="fc-stat-label">Total points</div>
                    <div class="fc-stat-value"><?=(float) $overallPoints, 0 ?></div>
                </div>
            </div>
        </div>
        <p class="small text-muted mt-4 mb-0"><i class="fa-solid fa-info-circle me-1"></i>Create assignments and quizzes, review
submissions, and enter grades from the assessments workspace.</p>
    </div>
</div>
</div>
<?php endif; ?>
</div>

<script>
document.querySelectorAll('[data-wordpad]').forEach((shell) => {
    const fieldName = shell.getAttribute('data-wordpad-name') || 'body';
    const textarea = shell.querySelector('textarea[name="' + fieldName + '"]');
    const editor = shell.querySelector('.wordpad-editor');
    const toolbar = shell.querySelector('.wordpad-toolbar');
    const blockSelect = shell.querySelector('[data-wordpad-block]');
    const form = shell.closest('form');

    if (!textarea || !editor || !toolbar || !form) {
        return;
    }

```

```

}

toolbar.classList.remove('d-none');
editor.classList.remove('d-none');
editor.innerHTML = textarea.value.trim() !== '' ? textarea.value : '';
textarea.classList.add('d-none');

const syncEditor = () => {
  const html = editor.innerHTML
    .replace(/<div><br><\div>/gi, '')
    .replace(/&nbsp;/gi, ' ')
    .trim();
  textarea.value = html;
};

const runCommand = (command, value = null) => {
  editor.focus();
  document.execCommand('styleWithCSS', false, false);
  document.execCommand(command, false, value);
  syncEditor();
};

const insertImageFromFile = (file) => {
  if (!file || !/^image\/(jpeg|png|gif|webp)$/i.test(file.type || '')) {
    window.alert('Please choose a JPEG, PNG, GIF, or WebP image.');
```

return;

```

  }
  if (file.size > 5 * 1024 * 1024) {
    window.alert('Image is too large (max 5 MB).');
```

return;

```

  }
  const fd = new FormData();
  fd.append('image', file, file.name || 'image.jpg');
  fetch('api/classroom_inline_image_upload.php', {
    method: 'POST',
    body: fd,
    credentials: 'same-origin'
  })
    .then((res) => res.json().then((data) => ({ ok: res.ok, data })))
    .then((result) => {
      if (!result.ok || !result.data || !result.data.src) {
        throw new Error((result.data && result.data.error) || 'Image upload failed.');
```

}

```

      editor.focus();
      const safeSrc = String(result.data.src).replace(/"/g, '&quot;');
      document.execCommand('insertHTML', false, '<p></p>');
      syncEditor();
    })
    .catch((err) => {
      window.alert(err && err.message ? err.message : 'Image upload failed.');
```

});

```

});

const imageInput = document.createElement('input');
imageInput.type = 'file';
imageInput.accept = 'image/jpeg,image/png,image/gif,image/webp,.jpg,.jpeg,.png,.gif,.webp';
imageInput.className = 'd-none';
shell.appendChild(imageInput);
imageInput.addEventListener('change', () => {
  const file = imageInput.files && imageInput.files[0];
  if (file) {
    insertImageFromFile(file);
  }
  imageInput.value = '';
});

toolbar.addEventListener('click', (event) => {
  const button = event.target.closest('button[data-command]');
  if (!button) {
    return;
  }

  event.preventDefault();
  const command = button.getAttribute('data-command') || '';

  if (command === 'createLink') {
    const url = window.prompt('Enter link URL', 'https://');
    if (url) {
      runCommand('createLink', url);
    }
  }
}

```

```

        return;
    }

    if (command === 'insertImage') {
        imageInput.click();
        return;
    }

    runCommand(command);
});

blockSelect?.addEventListener('change', (event) => {
    const value = event.target.value || '<p>';
    runCommand('formatBlock', value);
});

const fontSizeSelect = shell.querySelector('[data-wordpad-font-size]');
const fontFamilySelect = shell.querySelector('[data-wordpad-font-family]');

const applyWordpadStyle = (styleProp, styleValue) => {
    editor.focus();
    const sel = window.getSelection();
    if (!sel || sel.rangeCount === 0) {
        window.alert('Select some text first.');
```

return false;

```
    }
    const range = sel.getRangeAt(0);
    if (range.collapsed) {
        window.alert('Select some text first.');
```

return false;

```
    }
    const span = document.createElement('span');
    span.style[styleProp] = styleValue;
    try {
        range.surroundContents(span);
    } catch (err) {
        const fragment = range.extractContents();
        span.appendChild(fragment);
        range.insertNode(span);
    }
    sel.removeAllRanges();
    syncEditor();
    return true;
};

fontSizeSelect?.addEventListener('change', (event) => {
    const value = event.target.value || '';
    if (value !== '') {
        applyWordpadStyle('fontSize', value);
    }
    event.target.value = '';
});

fontFamilySelect?.addEventListener('change', (event) => {
    const value = event.target.value || '';
    if (value !== '') {
        applyWordpadStyle('fontFamily', value);
    }
    event.target.value = '';
});

editor.addEventListener('paste', (event) => {
    const items = event.clipboardData && event.clipboardData.items;
    if (!items) {
        return;
    }
    for (let i = 0; i < items.length; i++) {
        const item = items[i];
        if (item && item.kind === 'file' && /^image\/\w/.test(item.type || '')) {
            event.preventDefault();
            const file = item.getAsFile();
            if (file) {
                insertImageFromFile(file);
            }
            return;
        }
    }
});

editor.addEventListener('input', syncEditor);
```

```

        editor.addEventListener('blur', syncEditor);
        form.addEventListener('submit', syncEditor);
    });
</script>
<script src="assets/js/faculty_offline.js" defer></script>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: faculty_classrooms.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'program_chair', 'dean', 'gened']);

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}
if ($facultyId < 1 && in_array($_SESSION['role'] ?? '', ['program_chair', 'dean', 'gened'], true)) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    if ($facultyId > 0) {
        $_SESSION['faculty_id'] = $facultyId;
    }
}
if ($facultyId < 1) {
    exit('Faculty profile not linked to this account. Ask your dean to create/link your faculty profile.');
```

```

        INNER JOIN courses c ON c.id = s.course_id
        WHERE s.id = ? AND s.faculty_id = ?
        LIMIT 1'
    );
    $st->execute([$scheduleId, $facultyId]);
    $schedule = $st->fetch();
    if (!$schedule) {
        throw new RuntimeException('You can only create classrooms for your assigned courses.');
```

```

    }

    $chk = db()->prepare('SELECT COUNT(*) FROM online_classrooms WHERE schedule_id = ?');
    $chk->execute([$scheduleId]);
    if ((int) $chk->fetchColumn() > 0) {
        throw new RuntimeException('An online classroom already exists for that assigned course.');
```

```

    }

    $meetLink = faculty_classroom_sanitize_url((string) ($_POST['meet_link'] ?? ''));
    if ($meetLink === '') {
        $meetLink = trim((string) ($schedule['online_class_url'] ?? ''));
    }
```

```

    $title = trim((string) ($_POST['title'] ?? ''));
    if ($title === '') {
        $title = trim((string) $schedule['course_code']) . ' - ' . trim((string) $schedule['course_name']);
    }
```

```

    $description = trim((string) ($_POST['description'] ?? ''));
```

```

    if ($hasJoinCode) {
        $joinCode = classroom_alloc_unique_join_code();
        db()->prepare(
            'INSERT INTO online_classrooms (schedule_id, faculty_id, course_id, title, description, meet_link, join_code)
            VALUES (?, ?, ?, ?, ?, ?, ?)'
        )->execute([
            $scheduleId,
            $facultyId,
            (int) $schedule['course_id'],
            $title,
            $description !== '' ? $description : null,
            $meetLink,
            $joinCode,
        ]);
    } else {
        db()->prepare(
            'INSERT INTO online_classrooms (schedule_id, faculty_id, course_id, title, description, meet_link)
            VALUES (?, ?, ?, ?, ?, ?)'
        )->execute([
            $scheduleId,
            $facultyId,
            (int) $schedule['course_id'],
            $title,
            $description !== '' ? $description : null,
            $meetLink,
        ]);
    }
}
```

```

    if ($hasOnlineUrl && $meetLink !== '') {
        db()->prepare('UPDATE schedules SET online_class_url = ? WHERE id = ? AND faculty_id = ?')
        ->execute([$meetLink, $scheduleId, $facultyId]);
    }
```

```

    $_SESSION['flash'] = $hasJoinCode
        ? 'Online classroom created successfully. Share the join code from Manage so students can enroll.'
        : 'Online classroom created successfully.';
```

```

} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
```

```

header('Location: faculty_classrooms.php');
exit;
```

```

}
```

```

if ($action === 'open_meet') {
    $classroomId = (int) ($_POST['classroom_id'] ?? 0);
```

```

    try {
        faculty_open_meet_and_go_live($classroomId, $facultyId);
        header('Location: faculty_meet_live.php?classroom_id=' . $classroomId);
        exit;
    } catch (Throwable $e) {
```

```

        $_SESSION['flash'] = 'Error: ' . $e->getMessage();
        header('Location: faculty_classrooms.php');
        exit;
    }
}

$assignedSchedules = [];
$classrooms = [];

if ($hasClassrooms) {
    $st = db()->prepare(
        'SELECT s.id, s.semester, s.school_year, s.day_of_week, s.start_time, s.end_time,
            c.course_code, c.course_name,
            oc.id AS classroom_id
        FROM schedules s
        INNER JOIN courses c ON c.id = s.course_id
        LEFT JOIN online_classrooms oc ON oc.schedule_id = s.id
        WHERE s.faculty_id = ?
        ORDER BY s.school_year DESC, s.semester, c.course_code, s.start_time'
    );
    $st->execute([$facultyId]);
    $assignedSchedules = $st->fetchAll();

    $st = db()->prepare(
        'SELECT oc.*, s.semester, s.school_year, s.day_of_week, s.start_time, s.end_time,
            c.course_code, c.course_name,
            (SELECT COUNT(*) FROM classroom_enrollments ce WHERE ce.classroom_id = oc.id) AS enrolled_count,
            (SELECT COUNT(*) FROM classroom_content cc WHERE cc.classroom_id = oc.id) AS content_count,
            (SELECT COUNT(*) FROM classroom_assessments ca WHERE ca.classroom_id = oc.id) AS assessment_count
        FROM online_classrooms oc
        INNER JOIN schedules s ON s.id = oc.schedule_id
        INNER JOIN courses c ON c.id = oc.course_id
        WHERE oc.faculty_id = ? AND s.faculty_id = ?
        ORDER BY oc.created_at DESC'
    );
    $st->execute([$facultyId, $facultyId]);
    $classrooms = $st->fetchAll();
}

$availableSchedules = array_values(array_filter(
    $assignedSchedules,
    static fn (array $row): bool => empty($row['classroom_id'])
));
$defaultAssessmentsLink = 'faculty_classrooms.php';
if ($classrooms !== []) {
    $defaultAssessmentsLink = 'faculty_classroom_assessments.php?id=' . (int) $classrooms[0]['id'];
}

$pageTitle = 'My Classrooms';
require_once __DIR__ . '/includes/header.php';
?>
<style>
    .faculty-hub {
        background: #f4f7fc;
        font-family: 'Inter', sans-serif;
        color: #1a2c3e;
        line-height: 1.4;
        padding: 2rem 1.5rem;
        border-radius: 22px;
    }
    .dashboard-container {
        max-width: 1400px;
        margin: 0 auto;
    }
    .uni-header {
        display: flex;
        justify-content: space-between;
        align-items: flex-end;
        flex-wrap: wrap;
        margin-bottom: 2rem;
        border-bottom: 2px solid #e2e8f0;
        padding-bottom: 1rem;
    }
    .brand h1 {
        font-size: 1.9rem;
        font-weight: 700;
        background: linear-gradient(135deg, #1e4a6e, #2c7a4d);
        background-clip: text;
        -webkit-background-clip: text;

```

```

    color: transparent;
    letter-spacing: -0.3px;
    margin: 0;
}
.brand p {
    font-size: 0.9rem;
    color: #2c5a6e;
    font-weight: 500;
    margin: 0.25rem 0 0;
}
}
.faculty-badge {
    background: #fff;
    padding: 0.55rem 1rem;
    border-radius: 60px;
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.04);
    font-size: 0.85rem;
    font-weight: 500;
    display: flex;
    align-items: center;
    gap: 8px;
    color: #1f5e3a;
}
}
.classroom-grid {
    display: grid;
    grid-template-columns: 1fr 1.6fr;
    gap: 2rem;
    margin-bottom: 2.5rem;
}
}
.hub-card {
    background: #fff;
    border-radius: 28px;
    box-shadow: 0 12px 28px rgba(0, 0, 0, 0.05), 0 0 0 1px rgba(0, 0, 0, 0.02);
    overflow: hidden;
}
}
.hub-card-header {
    padding: 1.25rem 1.75rem;
    border-bottom: 1px solid #ecf3f9;
    display: flex;
    align-items: center;
    gap: 10px;
}
}
.hub-card-header i {
    font-size: 1.6rem;
    color: #2c7a4d;
}
}
.hub-card-header h2 {
    font-size: 1.35rem;
    font-weight: 600;
    color: #0f2b3b;
    margin: 0;
}
}
.hub-card-body {
    padding: 1.5rem 1.75rem;
}
}
.form-group {
    margin-bottom: 1.4rem;
}
}
.faculty-hub label {
    font-weight: 600;
    font-size: 0.85rem;
    display: block;
    margin-bottom: 0.5rem;
    color: #2c5a6e;
}
}
.hub-input, .hub-select {
    width: 100%;
    padding: 0.85rem 1rem;
    border: 1.5px solid #e2edf2;
    border-radius: 20px;
    font-family: 'Inter', sans-serif;
    font-size: 0.95rem;
    background: #fefefe;
    transition: .2s;
    outline: none;
}
}
.hub-input:focus, .hub-select:focus {
    border-color: #2c7a4d;
    box-shadow: 0 0 0 3px rgba(44, 122, 77, 0.15);
}
}
.meet-link {

```



```

    background: #f8fafc;
    border-radius: 20px;
    padding: 0.6rem 1rem;
    font-size: 0.85rem;
    display: inline-flex;
    align-items: center;
    gap: 8px;
    border: 1px solid #dce5ec;
    width: 100%;
}
.meet-link a {
    text-decoration: none;
    font-weight: 500;
    color: #1e6f5c;
    word-break: break-all;
}
.info-note {
    font-size: 0.75rem;
    color: #6f8e9e;
    margin-top: 8px;
    display: flex;
    align-items: center;
    gap: 6px;
}
.btn-create {
    background: linear-gradient(100deg, #1d6f42, #2c8e55);
    border: 0;
    width: 100%;
    padding: 0.9rem;
    border-radius: 40px;
    font-weight: 700;
    font-size: 1rem;
    color: #fff;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 10px;
    cursor: pointer;
    transition: all 0.2s;
    margin-top: 1rem;
}
.btn-create:hover {
    background: linear-gradient(100deg, #166037, #1f7547);
    transform: translateY(-1px);
    box-shadow: 0 8px 18px rgba(32, 106, 63, 0.2);
}
.table-wrapper {
    overflow-x: auto;
    border-radius: 24px;
}
.classroom-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.85rem;
}
.classroom-table th {
    text-align: left;
    padding: 1rem 0.8rem;
    background-color: #f9fdf9;
    font-weight: 600;
    color: #1a4a5f;
    border-bottom: 2px solid #e2f0e6;
    white-space: nowrap;
}
.classroom-table td {
    padding: 1rem 0.8rem;
    border-bottom: 1px solid #edf3f0;
    vertical-align: middle;
}
.status-badge {
    background: #e1f7e8;
    color: #1e6a3b;
    font-weight: 600;
    font-size: 0.7rem;
    padding: 0.25rem 0.7rem;
    border-radius: 40px;
    display: inline-block;
    width: fit-content;
    text-transform: capitalize;
}

```

```

.status-badge.is-archived {
  background: #e2e8f0;
  color: #334155;
}
.join-code {
  font-family: monospace;
  font-weight: 700;
  background: #f1f5f9;
  padding: 0.2rem 0.5rem;
  border-radius: 12px;
  letter-spacing: 0.5px;
  display: inline-block;
}
.action-icons {
  display: flex;
  gap: 12px;
  align-items: center;
}
.action-icons a,
.action-icons button {
  border: 0;
  background: transparent;
  color: #6c8b9b;
  padding: 0;
  font-size: 1rem;
  transition: .2s;
}
.action-icons a:hover,
.action-icons button:hover {
  color: #2c7a4d;
  transform: scale(1.05);
}
.action-icons .text-danger:hover {
  color: #991b1b !important;
}
.empty-row td {
  text-align: center;
  padding: 2rem;
  color: #8ba5b4;
  font-style: italic;
}
.footer-actions {
  margin-top: 2rem;
  display: flex;
  gap: 1rem;
  justify-content: flex-end;
  border-top: 1px solid #dce9f0;
  padding-top: 1.8rem;
  flex-wrap: wrap;
}
.footer-btn {
  background: #fff;
  padding: 0.7rem 1.2rem;
  border-radius: 40px;
  font-weight: 600;
  font-size: 0.9rem;
  display: inline-flex;
  align-items: center;
  gap: 10px;
  border: 1px solid #cbdde6;
  cursor: pointer;
  color: #1f5e6e;
  text-decoration: none;
}
.footer-btn:hover {
  background: #eef3f7;
  border-color: #9fc1cf;
}
.toast-msg {
  position: fixed;
  bottom: 24px;
  left: 50%;
  transform: translateX(-50%);
  background: #1f2e3a;
  color: #fff;
  padding: 12px 24px;
  border-radius: 60px;
  font-size: 0.85rem;
  font-weight: 500;
  z-index: 1000;
}

```

```

        box-shadow: 0 6px 14px rgba(0, 0, 0, 0.15);
        opacity: 0;
        transition: opacity .2s;
        pointer-events: none;
    }
    @media (max-width: 880px) {
        .faculty-hub {
            padding: 1.2rem;
        }
        .classroom-grid {
            grid-template-columns: 1fr;
            gap: 1.5rem;
        }
        .uni-header {
            flex-direction: column;
            align-items: flex-start;
            gap: 1rem;
        }
    }
}
</style>

<div class="faculty-hub">
    <div class="dashboard-container">
        <div class="uni-header">
            <div class="brand">
                <h1><i class="fa-solid fa-chalkboard-user" style="color:#2c7a4d; margin-right: 8px;"></i>My Classrooms</h1>
                <p>Western Philippines University · Faculty Teaching Hub</p>
            </div>
            <div class="d-flex flex-wrap align-items-center gap-2">
                <a href="faculty_offline.php" class="footer-btn" style="padding: 0.45rem 0.9rem;"><? app_tooltip_attr('View and sync posts
or files queued on this device while offline.') ?>>
                    <i class="fa-solid fa-cloud-arrow-up"></i> Offline uploads
                    <span class="badge text-bg-warning" data-offline-pending-count hidden>0</span>
                </a>
                <div class="faculty-badge">
                    <i class="fa-solid fa-user-graduate"></i> Faculty Portal
                    <i class="fa-solid fa-chevron-down" style="font-size: 10px;"></i>
                </div>
            </div>
        </div>
    </div>

    <?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

    <?php if (!$hasClassrooms): ?>
        <div class="alert alert-warning">
            Classroom features are not installed yet. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this page.
        </div>
    <?php else: ?>
        <div class="classroom-grid">
            <div class="hub-card">
                <div class="hub-card-header">
                    <i class="fa-solid fa-plus-circle"></i>
                    <h2>Create Online Classroom</h2>
                </div>
                <div class="hub-card-body">
                    <?php if ($availableSchedules === []): ?>
                        <p class="text-muted mb-0">All of your assigned schedules already have online classrooms.</p>
                    <?php else: ?>
                        <form method="post">
                            <input type="hidden" name="action" value="create_classroom">
                            <div class="form-group">
                                <label for="schedule_id"><i class="fa-regular fa-calendar"></i> Assign course</label>
                                <select id="schedule_id" name="schedule_id" class="hub-select" required>
                                    <option value="">Select assigned course</option>
                                    <?php foreach ($availableSchedules as $row): ?>
                                        <option value="<?= (int) $row['id'] ?>">
                                            <?= htmlspecialchars($row['course_code'] . ' - ' . $row['course_name'] . ' (' .
$ row['semester'] . ' / ' . $row['school_year'] . ')') ?>
                                        </option>
                                    <?php endforeach; ?>
                                </select>
                                <div class="info-note"><i class="fa-solid fa-circle-info"></i> Only schedules assigned to your faculty
account can be turned into classrooms.</div>
                            </div>

                            <div class="form-group">
                                <label for="meet_link"><i class="fa-solid fa-video"></i> Google Meet link</label>
                                <div class="meet-link">
                                    <i class="fa-brands fa-google" style="color:#2c7a4d;"></i>
                                    <a href="https://meet.google.com/" target="_blank" rel="noopener

```

```

noreferer">https://meet.google.com/</a>
        <i class="fa-solid fa-external-link-alt" style="font-size: 11px; color:#94a3b8;"></i>
    </div>
    <input id="meet_link" type="url" name="meet_link" class="hub-input mt-2"
placeholder="https://meet.google.com/..." autocomplete="url">
    <div class="info-note"><i class="fa-solid fa-link"></i> Default meet generator, you can edit after
creation.</div>

    </div>

    <div class="form-group">
        <label for="title"><i class="fa-solid fa-heading"></i> Classroom title <span style="font-weight:
normal;">(Optional custom title)</span></label>
        <input id="title" type="text" name="title" class="hub-input" maxlength="150" placeholder="e.g., CC101 -
Live Lecture (Monday group)">
    </div>

    <div class="form-group">
        <label for="description"><i class="fa-regular fa-note-sticky"></i> Description <span style="font-weight:
normal;">(Optional)</span></label>
        <input id="description" type="text" name="description" class="hub-input" maxlength="255"
placeholder="Optional short classroom note">
    </div>

    <button type="submit" class="btn-create"><? app_tooltip_attr('Creates an online classroom shell for the
selected assigned course. Use this before posting content or enrollments.') ?>>
        <i class="fa-solid fa-chalkboard"></i> Create classroom
    </button>
</form>
<?php endif; ?>
</div>

<div class="hub-card">
    <div class="hub-card-header">
        <i class="fa-solid fa-door-open"></i>
        <h2>Existing Online Classrooms</h2>
    </div>
    <div class="hub-card-body" style="padding: 0 0 1rem 0;">
        <div class="table-wrapper">
            <table class="classroom-table">
                <thead>
                    <tr>
                        <th>Classroom</th>
                        <?php if ($hasJoinCode): ?><th>Join code</th><?php endif; ?>
                        <th>Schedule</th>
                        <th>Status</th>
                        <th style="width: 95px;"></th>
                    </tr>
                </thead>
                <tbody>
                    <?php if ($classrooms === []): ?>
                        <tr class="empty-row"><td colspan="5"><? $hasJoinCode ? 5 : 4 ?>>No classrooms yet. Create your first
online classroom.</td></tr>
                    <?php endif; ?>
                    <?php foreach ($classrooms as $row): ?>
                        <?php
                            $status = (string) $row['status'];
                            $isArchived = $status === 'archived';
                            $joinCode = (string) ($row['join_code'] ?? '');
                            $scheduleLine = (string) $row['semester'] . ' / ' . (string) $row['school_year'];
                            $dayLine = str_replace(',', ' ', (string) $row['day_of_week']);
                            $timeLine = substr((string) $row['start_time'], 0, 5) . ' - ' . substr((string) $row['end_time'], 0, 5);
                            ?>
                        <tr>
                            <td>
                                <div style="font-weight: 500;">
                                    <i class="fa-solid fa-school" style="color:#2c7a4d; margin-right:6px;"></i><? =
htmlspecialchars((string) $row['title']) ?>
                                </div>
                                <div class="text-muted" style="font-size: 0.8rem;"><? = htmlspecialchars((string)
$row['course_code']) ?> - <? = htmlspecialchars((string) $row['course_name']) ?></div>
                            </td>
                            <?php if ($hasJoinCode): ?>
                                <td><span class="join-code"><? = htmlspecialchars($joinCode !== '' ? $joinCode : '-') ?></span>
                            </td>
                            <?php endif; ?>
                            <td style="font-size: 0.8rem;">
                                <div><? = htmlspecialchars($scheduleLine) ?></div>
                                <div class="text-muted"><? = htmlspecialchars($dayLine) ?></div>
                                <div class="text-muted"><? = htmlspecialchars($timeLine) ?></div>

```

```

        </td>
        <td>
            <span class="status-badge<?= $isArchived ? ' is-archived' : '' ?>"
                <i class="fa-solid fa-circle" style="font-size: 0.5rem; vertical-align: middle; margin-
right: 4px;"></i><?= htmlspecialchars($status) ?>
            </span>
        </td>
        <td>
            <div class="action-icons">
                <?php if ($hasJoinCode && $joinCode !== ''): ?>
                    <button type="button" class="copy-join-code" data-code="<?= htmlspecialchars($joinCode)
?>" title="Copy join code"><i class="fa-solid fa-copy"></i></button>
                <?php endif; ?>
                <a href="faculty_classroom.php?id=<?= (int) $row['id'] ?>" title="Manage classroom"><i
class="fa-solid fa-gear"></i></a>
                <a href="faculty_classroom_assessments.php?id=<?= (int) $row['id'] ?>" title="Manage
assessments"><i class="fa-solid fa-clipboard-list"></i></a>
                <?php if (trim((string) $row['meet_link']) !== ''): ?>
                    <form method="post" action="faculty_classrooms.php" target="_blank" rel="noopener
noreferrer" class="d-inline">
                        <input type="hidden" name="action" value="open_meet">
                        <input type="hidden" name="classroom_id" value="<?= (int) $row['id'] ?>">
                        <button type="submit" title="Open Meet and go live"><?= app_tooltip_attr('Opens
Google Meet in a new tab and marks this class as live so students can join.') ?>><i class="fa-solid fa-video"></i></button>
                    </form>
                <?php endif; ?>
            </div>
        </td>
    </tr>
    <?php endforeach; ?>
</tbody>
</table>
</div>
</div>
</div>
</div>

<div class="footer-actions">
    <a href="<?= htmlspecialchars($defaultAssessmentsLink) ?>" class="footer-btn">
        <i class="fa-solid fa-clipboard-list"></i> Manage Assessments
    </a>
    <a href="https://meet.google.com/" target="_blank" rel="noopener noreferrer" class="footer-btn">
        <i class="fa-brands fa-google"></i> Open Google Meet
    </a>
    <a href="faculty_schedule.php" class="footer-btn"><?= app_tooltip_attr('Returns to your weekly teaching schedule and online
links. Use this when you are done managing class spaces.') ?>>
        <i class="fa-solid fa-calendar-check"></i> Back to My Schedule
    </a>
</div>
<?php endif; ?>
</div>
</div>

<div id="toastMsg" class="toast-msg"></div>

<script>
(function () {
    const toast = document.getElementById('toastMsg');
    let toastTimer = null;

    function showToast(message) {
        if (!toast) return;
        toast.textContent = message;
        toast.style.opacity = '1';
        if (toastTimer) {
            window.clearTimeout(toastTimer);
        }
        toastTimer = window.setTimeout(() => {
            toast.style.opacity = '0';
        }, 2200);
    }

    async function copyText(text) {
        try {
            await navigator.clipboard.writeText(text);
            return true;
        } catch (err) {
            const ta = document.createElement('textarea');
            ta.value = text;
            document.body.appendChild(ta);

```

```

        ta.select();
        const ok = document.execCommand('copy');
        document.body.removeChild(ta);
        return ok;
    }
}

document.querySelectorAll('.copy-join-code').forEach((btn) => {
    btn.addEventListener('click', async () => {
        const code = btn.getAttribute('data-code') || '';
        if (!code) return;
        const copied = await copyText(code);
        showToast(copied ? `Join code ${code} copied.` : 'Unable to copy join code.');
```

```

));

function faculty_assessment_datetime_save(?string $value): ?string
{
    $raw = trim((string) $value);
    if ($raw === '') {
        return null;
    }
    $raw = str_replace('T', ' ', $raw);
    if (strlen($raw) === 16) {
        $raw .= ':00';
    }
    return $raw;
}

/** @param array{score?: mixed, feedback?: mixed} $saved */
function faculty_submission_grading_status(array $submission, array $saved): string
{
    $hasSubmission = trim((string) ($submission['answer_text'] ?? '')) !== '';
    $scoreRaw = $saved['score'] ?? null;
    $hasScore = $scoreRaw !== null && $scoreRaw !== '' && is_numeric($scoreRaw);
    $hasFeedback = trim((string) ($saved['feedback'] ?? '')) !== '';
    if ($hasScore || $hasFeedback) {
        return 'graded';
    }
    if ($hasSubmission) {
        return 'submitted';
    }
    return 'pending';
}

function faculty_assessment_due_for_input(?string $dueAt): string
{
    if ($dueAt === null || trim($dueAt) === '') {
        return '';
    }
    $ts = strtotime((string) $dueAt);
    if ($ts === false) {
        return '';
    }
    return date('Y-m-d\TH:i', $ts);
}

/** Parse timer duration from POST (hours + minutes, or legacy minutes-only). Null = no timer. */
function faculty_assessment_time_limit_from_post(array $post): ?int
{
    $enabled = !empty($post['timer_enabled']);
    if (!$enabled) {
        return null;
    }
    $hours = max(0, (int) ($post['timer_hours'] ?? 0));
    $minutes = max(0, (int) ($post['timer_minutes'] ?? 0));
    if (isset($post['time_limit_minutes']) && (string) ($post['timer_hours'] ?? '') === '' && (string) ($post['timer_minutes'] ?? '') === '') {
        $minutes = max(0, (int) $post['time_limit_minutes']);
    }
    $total = ($hours * 60) + $minutes;
    if ($total < 1) {
        return null;
    }
    return min(720, $total);
}

function faculty_assess_lc(string $s): string
{
    return function_exists('mb_strtolower') ? mb_strtolower($s) : strtolower($s);
}

/**
 * @return list<array{
 *     question_type:string,
 *     question_text:string,
 *     points:float,
 *     options_json:?string,
 *     answer_key:?string,
 *     word_limit:?int,
 *     char_limit:?int,
 *     allow_steps:int
 *     }>
 */

```

```

function faculty_parse_questions_json(string $json, string $assessmentType): array
{
    $expectedType = classroom_assessment_normalize_type($assessmentType);
    $rows = json_decode($json, true);
    if (!is_array($rows)) {
        throw new RuntimeException('Question builder payload is invalid.');
```

```

    }
    $out = [];
    foreach ($rows as $row) {
        if (!is_array($row)) {
            continue;
        }
        $type = classroom_question_normalize_type((string) ($row['question_type'] ?? $expectedType));
        if ($type !== $expectedType) {
            throw new RuntimeException('All questions must match the selected assessment type: ' .
classroom_assessment_type_label($expectedType) . '.');
```

```

        }
        $questionText = trim((string) ($row['question_text'] ?? ''));
        $points = (float) ($row['points'] ?? 0);
        if ($questionText === '' || $points <= 0) {
            continue;
        }

        $options = null;
        $answerKey = trim((string) ($row['answer_key'] ?? ''));
        if ($type === 'multiple_choice') {
            $rawOptions = $row['options'] ?? [];
            $clean = [];
            if (is_array($rawOptions)) {
                foreach ($rawOptions as $opt) {
                    $text = trim((string) $opt);
                    if ($text !== '') {
                        $clean[] = $text;
                    }
                }
            }
            if (count($clean) < 2 || count($clean) > 6) {
                throw new RuntimeException('Multiple choice questions must have 2 to 6 options.');
```

```

            }
            if ($answerKey === '') {
                throw new RuntimeException('Multiple choice questions need an answer key.');
```

```

            }
            $options = json_encode(array_values($clean), JSON_UNESCAPED_UNICODE);
        } elseif ($type === 'true_false') {
            $keyNorm = strtolower($answerKey);
            if (!in_array($keyNorm, ['true', 'false'], true)) {
                throw new RuntimeException('True/False questions require an answer key of True or False.');
```

```

            }
            $answerKey = $keyNorm;
        } elseif ($type === 'problem_solving') {
            if ($answerKey === '') {
                throw new RuntimeException('Problem-solving questions require a final answer key.');
```

```

            }
        } else {
            $answerKey = null;
        }

        $wordLimit = isset($row['word_limit']) && (string) $row['word_limit'] !== '' ? max(0, (int) $row['word_limit']) : null;
        $charLimit = isset($row['char_limit']) && (string) $row['char_limit'] !== '' ? max(0, (int) $row['char_limit']) : null;
        $allowSteps = !empty($row['allow_steps']) ? 1 : 0;

        $out[] = [
            'question_type' => $type,
            'question_text' => $questionText,
            'points' => $points,
            'options_json' => $options,
            'answer_key' => $answerKey !== '' ? $answerKey : null,
            'word_limit' => $wordLimit,
            'char_limit' => $charLimit,
            'allow_steps' => $allowSteps,
        ];
    }
    if ($out === []) {
        throw new RuntimeException('Add at least one question.');
```

```

    }
    return $out;
}

$classroom = null;
if ($classroomId > 0 && $missingTables === []) {

```



```

$st = db()->prepare(
  'SELECT oc.*, s.semester, s.school_year, s.day_of_week, s.start_time, s.end_time, s.id AS schedule_id, s.college_id,
    c.course_code, c.course_name
  FROM online_classrooms oc
  INNER JOIN schedules s ON s.id = oc.schedule_id
  INNER JOIN courses c ON c.id = oc.course_id
  WHERE oc.id = ? AND oc.faculty_id = ? AND s.faculty_id = ?
  LIMIT 1'
);
$st->execute([$classroomId, $facultyId, $facultyId]);
$classroom = $st->fetch() ?: null;
}

if ($missingTables === [] && !$classroom) {
  http_response_code(404);
  exit('Classroom not found or you do not have access to it.');
```

```

}

if ($_SERVER['REQUEST_METHOD'] === 'POST' && $missingTables === [] && $classroom) {
  $action = (string) ($_POST['action'] ?? '');

  try {
    if ($action === 'add_assessment') {
      $assessmentType = classroom_assessment_normalize_type((string) ($_POST['assessment_type'] ?? 'essay'));
      $title = trim((string) ($_POST['title'] ?? ''));
      $description = classroom_content_prepare_body((string) ($_POST['description'] ?? ''));
      $totalPoints = (float) ($_POST['total_points'] ?? 0);
      $dueAt = faculty_assessment_datetime_save((string) ($_POST['due_at'] ?? ''));
      $timeLimitMinutes = $hasTimeLimit ? faculty_assessment_time_limit_from_post($_POST) : null;
      $creditedWeek = $hasCreditedWeek ? trim((string) ($_POST['credited_week'] ?? '')) : '';
      $questions = faculty_parse_questions_json((string) ($_POST['questions_json'] ?? '[]'), $assessmentType);

      if ($title === '') {
        throw new RuntimeException('Assessment title is required.');
```

```

      }
      $calculatedPoints = array_sum(array_map(static fn (array $q): float => (float) $q['points'], $questions));
      if ($calculatedPoints <= 0) {
        throw new RuntimeException('Total points must be greater than zero.');
```

```

      }
      $totalPoints = $calculatedPoints;

      db()->beginTransaction();
      if ($hasCreditedWeek && $hasTimeLimit) {
        db()->prepare(
          'INSERT INTO classroom_assessments (classroom_id, faculty_id, assessment_type, title, description, total_points, due_at,
time_limit_minutes, credited_week)
          VALUES (?, ?, ?, ?, ?, ?, ?, ?)'
        )->execute([
          $classroomId,
          $facultyId,
          $assessmentType,
          $title,
          $description,
          $totalPoints,
          $dueAt,
          $timeLimitMinutes,
          $creditedWeek,
        ]);
      } elseif ($hasTimeLimit) {
        db()->prepare(
          'INSERT INTO classroom_assessments (classroom_id, faculty_id, assessment_type, title, description, total_points, due_at,
time_limit_minutes)
          VALUES (?, ?, ?, ?, ?, ?, ?, ?)'
        )->execute([
          $classroomId,
          $facultyId,
          $assessmentType,
          $title,
          $description,
          $totalPoints,
          $dueAt,
          $timeLimitMinutes,
        ]);
      } elseif ($hasCreditedWeek) {
        db()->prepare(
          'INSERT INTO classroom_assessments (classroom_id, faculty_id, assessment_type, title, description, total_points, due_at,
credited_week)
          VALUES (?, ?, ?, ?, ?, ?, ?, ?)'
        )->execute([
          $classroomId,

```

```

        $facultyId,
        $assessmentType,
        $title,
        $description,
        $totalPoints,
        $dueAt,
        $creditedWeek,
    ));
} else {
    db()->prepare(
        'INSERT INTO classroom_assessments (classroom_id, faculty_id, assessment_type, title, description, total_points, due_at)
        VALUES (?, ?, ?, ?, ?, ?, ?)'
    )->execute([
        $classroomId,
        $facultyId,
        $assessmentType,
        $title,
        $description,
        $totalPoints,
        $dueAt,
    ]);
}
$assessmentId = (int) db()->lastInsertId();
$qStmt = db()->prepare(
    'INSERT INTO classroom_assessment_questions
    (assessment_id, question_type, question_text, options_json, answer_key, points, position, word_limit, char_limit,
allow_steps)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)'
);
foreach ($questions as $idx => $q) {
    $qStmt->execute([
        $assessmentId,
        $q['question_type'],
        $q['question_text'],
        $q['options_json'],
        $q['answer_key'],
        $q['points'],
        $idx + 1,
        $q['word_limit'],
        $q['char_limit'],
        $q['allow_steps'],
    ]);
}
db()->commit();

$_SESSION['flash'] = classroom_assessment_type_label($assessmentType) . ' added successfully.';
} elseif ($action === 'update_assessment') {
    $assessmentId = (int) ($_POST['assessment_id'] ?? 0);
    $assessmentType = classroom_assessment_normalize_type((string) ($_POST['assessment_type'] ?? 'essay'));
    $title = trim((string) ($_POST['title'] ?? ''));
    $description = classroom_content_prepare_body((string) ($_POST['description'] ?? ''));
    $totalPoints = (float) ($_POST['total_points'] ?? 0);
    $dueAt = faculty_assessment_datetime_save((string) ($_POST['due_at'] ?? ''));
    $timeLimitMinutes = $hasTimeLimit ? faculty_assessment_time_limit_from_post($_POST) : null;
    $creditedWeek = $hasCreditedWeek ? trim((string) ($_POST['credited_week'] ?? '')) : '';
    $questions = faculty_parse_questions_json((string) ($_POST['questions_json'] ?? '[]'), $assessmentType);

    if ($assessmentId < 1) {
        throw new RuntimeException('Invalid assessment.');
```

```

        $creditedWeek,
        $assessmentId,
        $classroomId,
        $facultyId,
    ));
} elseif ($hasTimeLimit) {
    $st = db()->prepare(
        'UPDATE classroom_assessments
        SET assessment_type = ?, title = ?, description = ?, total_points = ?, due_at = ?, time_limit_minutes = ?
        WHERE id = ? AND classroom_id = ? AND faculty_id = ?'
    );
    $st->execute([
        $assessmentType,
        $title,
        $description,
        $totalPoints,
        $dueAt,
        $timeLimitMinutes,
        $assessmentId,
        $classroomId,
        $facultyId,
    ]);
} elseif ($hasCreditedWeek) {
    $st = db()->prepare(
        'UPDATE classroom_assessments
        SET assessment_type = ?, title = ?, description = ?, total_points = ?, due_at = ?, credited_week = ?
        WHERE id = ? AND classroom_id = ? AND faculty_id = ?'
    );
    $st->execute([
        $assessmentType,
        $title,
        $description,
        $totalPoints,
        $dueAt,
        $creditedWeek,
        $assessmentId,
        $classroomId,
        $facultyId,
    ]);
} else {
    $st = db()->prepare(
        'UPDATE classroom_assessments
        SET assessment_type = ?, title = ?, description = ?, total_points = ?, due_at = ?
        WHERE id = ? AND classroom_id = ? AND faculty_id = ?'
    );
    $st->execute([
        $assessmentType,
        $title,
        $description,
        $totalPoints,
        $dueAt,
        $assessmentId,
        $classroomId,
        $facultyId,
    ]);
}
}
if ($st->rowCount() < 1) {
    $check = db()->prepare('SELECT id FROM classroom_assessments WHERE id = ? AND classroom_id = ? AND faculty_id = ? LIMIT 1');
    $check->execute([$assessmentId, $classroomId, $facultyId]);
    if (!$check->fetchColumn()) {
        throw new RuntimeException('Assessment not found.');
```

allow_steps)

```

        $q['char_limit'],
        $q['allow_steps'],
    ));
}
db()->commit();
$_SESSION['flash'] = 'Assessment updated.';
} elseif ($action === 'delete_assessment') {
    $assessmentId = (int) ($_POST['assessment_id'] ?? 0);
    db()->prepare(
        'DELETE FROM classroom_assessments WHERE id = ? AND classroom_id = ? AND faculty_id = ?'
    )->execute([$assessmentId, $classroomId, $facultyId]);
    $_SESSION['flash'] = 'Assessment deleted.';
} elseif ($action === 'assign_assessment') {
    $assessmentId = (int) ($_POST['assessment_id'] ?? 0);
    $targetClassroomId = (int) ($_POST['target_classroom_id'] ?? 0);
    classroom_assessment_copy_to_classroom($assessmentId, $classroomId, $targetClassroomId, $facultyId);
    $st = db()->prepare(
        'SELECT c.course_code, oc.title
        FROM online_classrooms oc
        INNER JOIN courses c ON c.id = oc.course_id
        WHERE oc.id = ?
        LIMIT 1'
    );
    $st->execute([$targetClassroomId]);
    $targetMeta = $st->fetch() ?: [];
    $targetLabel = trim((string) ($targetMeta['course_code'] ?? ''));
    if ($targetLabel === '') {
        $targetLabel = trim((string) ($targetMeta['title'] ?? 'the selected course'));
    }
    $_SESSION['flash'] = 'Assessment assigned to ' . $targetLabel . '. The original stays in this class.';
} elseif ($action === 'save_scores') {
    $assessmentId = (int) ($_POST['assessment_id'] ?? 0);

    $st = db()->prepare(
        'SELECT id FROM classroom_assessments WHERE id = ? AND classroom_id = ? AND faculty_id = ? LIMIT 1'
    );
    $st->execute([$assessmentId, $classroomId, $facultyId]);
    if (!$st->fetchColumn()) {
        throw new RuntimeException('Assessment not found.');
```

```

} elseif ($action === 'save_single_submission') {
    $assessmentId = (int) ($_POST['assessment_id'] ?? 0);
    $studentId = (int) ($_POST['student_id'] ?? 0);
    $scoreText = trim((string) ($_POST['score'] ?? ''));
    $feedbackText = trim((string) ($_POST['feedback'] ?? ''));

    $st = db()->prepare(
        'SELECT id FROM classroom_assessments WHERE id = ? AND classroom_id = ? AND faculty_id = ? LIMIT 1'
    );
    $st->execute([$assessmentId, $classroomId, $facultyId]);
    if (!$st->fetchColumn()) {
        throw new RuntimeException('Assessment not found.');
```

```

    }

    $allowedStudentIds = db()->prepare(
        'SELECT ce.student_id FROM classroom_enrollments ce WHERE ce.classroom_id = ? AND ce.student_id = ? LIMIT 1'
    );
    $allowedStudentIds->execute([$classroomId, $studentId]);
    if (!$allowedStudentIds->fetchColumn()) {
        throw new RuntimeException('Student not in this classroom.');
```

```

    }

    if ($scoreText === '' && $feedbackText === '') {
        db()->prepare('DELETE FROM classroom_scores WHERE assessment_id = ? AND student_id = ?')
        ->execute([$assessmentId, $studentId]);
    } else {
        $score = $scoreText === '' ? null : (float) $scoreText;
        db()->prepare(
            'INSERT INTO classroom_scores (assessment_id, student_id, score, feedback, graded_at)
            VALUES (?, ?, ?, ?, NOW())
            ON DUPLICATE KEY UPDATE score = VALUES(score), feedback = VALUES(feedback), graded_at = NOW()'
        )->execute([
            $assessmentId,
            $studentId,
            $score,
            $feedbackText !== '' ? $feedbackText : null,
        ]);
    }

    $_SESSION['flash'] = 'Saved grade for student.';
} elseif ($action === 'approve_retake_request') {
    $requestId = (int) ($_POST['request_id'] ?? 0);
    if ($requestId < 1) {
        throw new RuntimeException('Invalid retake request.');
```

```

    }

    approve_assessment_retake_request($requestId, $classroomId, $userId);
    $_SESSION['flash'] = 'Retake approved. The student may take the assessment again.';
} elseif ($action === 'reject_retake_request') {
    $requestId = (int) ($_POST['request_id'] ?? 0);
    $remarks = trim((string) ($_POST['faculty_remarks'] ?? ''));
    if ($requestId < 1) {
        throw new RuntimeException('Invalid retake request.');
```

```

    }

    reject_assessment_retake_request($requestId, $classroomId, $userId, $remarks);
    $_SESSION['flash'] = 'Retake request denied.';
} elseif ($action === 'upload_banner' || $action === 'delete_banner') {
    $bannerFlash = faculty_classroom_process_banner_post($classroomId, $facultyId, $classroom);
    if ($bannerFlash !== null) {
        $_SESSION['flash'] = $bannerFlash;
    }
}

} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}

$redirect = 'faculty_classroom_assessments.php?id=' . $classroomId;
$pick = (int) ($_POST['submissions_assessment_id'] ?? 0);
if ($pick > 0) {
    $redirect .= '&submissions=' . $pick;
}
$anchor = trim((string) ($_POST['scroll_anchor'] ?? ''));
if ($anchor !== '' && preg_match('/^[a-zA-Z0-9_-]+$/', $anchor)) {
    $redirect .= '#' . $anchor;
}
header('Location: ' . $redirect);
exit;
}

```

```

$enrollments = [];
$assessments = [];
$scoreMap = [];
$submissionMap = [];
$questionMap = [];
$pendingRetakeRequests = [];
$otherClassrooms = [];

if ($missingTables === [] && $classroom) {
    $st = db()->prepare(
        'SELECT ce.id AS enrollment_id, cs.id AS student_id, cs.user_id, cs.student_number, cs.full_name, cs.email, u.username
        FROM classroom_enrollments ce
        INNER JOIN classroom_students cs ON cs.id = ce.student_id
        LEFT JOIN users u ON u.id = cs.user_id
        WHERE ce.classroom_id = ?
        ORDER BY cs.full_name ASC'
    );
    $st->execute([$classroomId]);
    $enrollments = $st->fetchAll();

    $assessmentOrderBy = $hasCreditedWeek
        ? 'ORDER BY credited_week ASC, created_at DESC'
        : 'ORDER BY created_at DESC';
    $st = db()->prepare(
        "SELECT *
        FROM classroom_assessments
        WHERE classroom_id = ? AND faculty_id = ?
        {$assessmentOrderBy}"
    );
    $st->execute([$classroomId, $facultyId]);
    $assessments = $st->fetchAll();

    if ($assessments !== []) {
        $ids = array_map(static fn (array $r): int => (int) $r['id'], $assessments);
        $placeholders = implode(',', array_fill(0, count($ids), '?'));
        $qst = db()->prepare(
            "SELECT * FROM classroom_assessment_questions
            WHERE assessment_id IN ($placeholders)
            ORDER BY assessment_id ASC, position ASC, id ASC"
        );
        $qst->execute($ids);
        foreach ($qst->fetchAll() as $qr) {
            $aid = (int) ($qr['assessment_id'] ?? 0);
            if ($aid < 1) {
                continue;
            }
            $questionMap[$aid][] = $qr;
        }
    }

    $st = db()->prepare(
        'SELECT cs.assessment_id, cs.student_id, cs.score, cs.feedback
        FROM classroom_scores cs
        INNER JOIN classroom_assessments ca ON ca.id = cs.assessment_id
        WHERE ca.classroom_id = ?'
    );
    $st->execute([$classroomId]);
    foreach ($st->fetchAll() as $row) {
        $scoreMap[(int) $row['assessment_id']][(int) $row['student_id']] = [
            'score' => $row['score'],
            'feedback' => $row['feedback'],
        ];
    }

    $st = db()->prepare(
        'SELECT sub.assessment_id, sub.student_id, sub.answer_text, sub.status, sub.submitted_at'
        . ($hasIntegrityLocked ? ', sub.integrity_locked' : '') . '
        FROM classroom_submissions sub
        INNER JOIN classroom_assessments ca ON ca.id = sub.assessment_id
        WHERE ca.classroom_id = ?'
    );
    $st->execute([$classroomId]);
    foreach ($st->fetchAll() as $row) {
        $submissionMap[(int) $row['assessment_id']][(int) $row['student_id']] = [
            'answer_text' => $row['answer_text'],
            'status' => $row['status'],
            'submitted_at' => $row['submitted_at'],
            'integrity_locked' => $hasIntegrityLocked ? (int) ($row['integrity_locked'] ?? 0) : 0,
        ];
    }
}

```

```

}

if (assessment_retake_table_ready()) {
    $pendingRetakeRequests = pending_assessment_retake_requests_for_classroom($classroomId);
}

$otherClassrooms = faculty_owned_classrooms($facultyId, $classroomId);
}

$facultyUser = current_user();
$facultyDisplayName = trim((string) ($facultyUser['full_name'] ?? '')) !== ''
    ? (string) $facultyUser['full_name']
    : (trim((string) ($facultyUser['username'] ?? '')) !== '' ? (string) $facultyUser['username'] : 'Faculty');

$facultyInitials = '';
$initialParts = preg_split('/\s+/', trim($facultyDisplayName)) ?: [];
foreach ($initialParts as $ip) {
    if ($ip === '') {
        continue;
    }
    $facultyInitials .= strtoupper((function_exists('mb_substr') ? mb_substr($ip, 0, 1) : substr($ip, 0, 1)));
    if (strlen($facultyInitials) >= 2) {
        break;
    }
}
if ($facultyInitials === '') {
    $facultyInitials = 'F';
}

$submissionsPickId = (int) ($_GET['submissions'] ?? 0);
if ($submissionsPickId < 1 && $assessments !== []) {
    $submissionsPickId = (int) $assessments[0]['id'];
}

$assessmentAiAvailable = wellness_ai_is_enabled();

$pageTitle = $classroom ? 'Classroom Assessments' : 'Assessments';
require_once __DIR__ . '/includes/header.php';
?>
<style>
    @import url('https://fonts.googleapis.com/css2?
family=Inter:opsz,wght@14..32,300;14..32,400;14..32,500;14..32,600;14..32,700&display=swap');

    .fac-assess-dashboard {
        font-family: 'Inter', system-ui, sans-serif;
        color: #1e293b;
        line-height: 1.4;
        background: #f4f7fc;
        margin: -1.5rem -0.75rem 0;
        padding: 2rem 1.25rem 2.5rem;
    }
    @media (min-width: 576px) {
        .fac-assess-dashboard {
            margin: -1.5rem -1rem 0;
            padding: 2rem 1.5rem 2.5rem;
        }
    }
    .fac-assess-dashboard .dashboard-inner {
        max-width: 1400px;
        margin: 0 auto;
    }
    .fac-assess-dashboard .top-nav {
        background: white;
        border-radius: 1.5rem;
        padding: 1rem 1.8rem;
        box-shadow: 0 4px 10px rgba(0, 0, 0, 0.02), 0 1px 2px rgba(0, 0, 0, 0.05);
        margin-bottom: 2rem;
        display: flex;
        flex-wrap: wrap;
        justify-content: space-between;
        align-items: center;
        gap: 1rem;
    }
    .fac-assess-dashboard .course-info h1 {
        font-size: 1.5rem;
        font-weight: 600;
        letter-spacing: -0.3px;
        color: #0f3b5c;
        margin: 0;
    }

```

```

.fac-assess-dashboard .course-info .sub {
  font-size: 0.85rem;
  color: #5b6e8c;
  margin-top: 0.2rem;
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  align-items: center;
}
.fac-assess-dashboard .semester-badge {
  background: #eef2ff;
  padding: 0.2rem 0.7rem;
  border-radius: 20px;
  font-size: 0.75rem;
  font-weight: 500;
  color: #1e40af;
}
.fac-assess-dashboard .top-nav-actions {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  align-items: center;
}
.fac-assess-dashboard .top-nav-actions .fac-nav-link {
  font-size: 0.8rem;
  font-weight: 500;
  color: #2c7da0;
  text-decoration: none;
  padding: 0.35rem 0.9rem;
  border-radius: 2rem;
  background: #f1f5f9;
  border: none;
  transition: background 0.15s;
}
.fac-assess-dashboard .top-nav-actions .fac-nav-link:hover {
  background: #e2e8f0;
  color: #0f3b5c;
}
.fac-assess-dashboard .faculty-profile {
  display: flex;
  align-items: center;
  gap: 0.8rem;
  background: #f8fafc;
  padding: 0.4rem 1rem 0.4rem 0.8rem;
  border-radius: 40px;
}
.fac-assess-dashboard .avatar {
  width: 38px;
  height: 38px;
  background: #1e6f5c;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  color: white;
  font-weight: 600;
  font-size: 0.95rem;
  flex-shrink: 0;
}
.fac-assess-dashboard .faculty-name {
  font-weight: 600;
  font-size: 0.9rem;
}
.fac-assess-dashboard .faculty-role {
  font-size: 0.7rem;
  color: #5f6c84;
}
.fac-assess-dashboard .fac-dash-card {
  background: white;
  border-radius: 1.25rem;
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.03), 0 2px 4px rgba(0, 0, 0, 0.05);
  padding: 1.5rem;
  margin-bottom: 2rem;
}
.fac-assess-dashboard .section-title {
  font-size: 1.3rem;
  font-weight: 600;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

```



```

    margin-bottom: 1.25rem;
}
.fac-assess-dashboard .section-title i {
    color: #2c7da0;
    font-size: 1.2rem;
}
.fac-assess-dashboard .assessment-toolbar {
    display: flex;
    flex-wrap: wrap;
    justify-content: space-between;
    align-items: center;
    gap: 1rem;
    margin-bottom: 1rem;
}
.fac-assess-dashboard .assessment-table-wrapper {
    overflow-x: auto;
}
.fac-assess-dashboard .assessment-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.9rem;
}
.fac-assess-dashboard .assessment-table th {
    text-align: left;
    padding: 0.9rem 0.5rem 0.7rem 0;
    font-weight: 600;
    color: #4a5b7a;
    border-bottom: 1px solid #e2e8f0;
    user-select: none;
    white-space: nowrap;
}
.fac-assess-dashboard .assessment-table th.sortable { cursor: pointer; }
.fac-assess-dashboard .assessment-table th.sortable:hover { color: #1e6f5c; }
.fac-assess-dashboard .assessment-table td {
    padding: 0.9rem 0.5rem 0.9rem 0;
    border-bottom: 1px solid #f0f2f5;
    vertical-align: middle;
}
.fac-assess-dashboard .week-group-header td {
    border-bottom: none;
    padding-top: 1.25rem;
    padding-bottom: 0.4rem;
}
.fac-assess-dashboard .week-group-cell {
    background: #f1f5f9;
    border-radius: 0.6rem;
    padding: 0.55rem 1rem !important;
    font-size: 0.85rem;
    color: #0f3b5c;
    display: flex;
    align-items: center;
    gap: 0.5rem;
}
.fac-assess-dashboard .week-group-cell i {
    color: #2c7da0;
    font-size: 0.9rem;
}
.fac-assess-dashboard .week-count-badge {
    background: #e2e8f0;
    padding: 0.15rem 0.6rem;
    border-radius: 20px;
    font-size: 0.7rem;
    font-weight: 500;
    color: #475569;
    margin-left: 0.25rem;
}
.fac-assess-dashboard .type-badge {
    background: #eef2ff;
    padding: 0.2rem 0.7rem;
    border-radius: 30px;
    font-size: 0.75rem;
    font-weight: 500;
    display: inline-block;
    color: #1e3a8a;
}
.fac-assess-dashboard .type-badge.type-performance {
    background: #e0f2fe;
    color: #0369a1;
}
.fac-assess-dashboard .points {

```

```

    font-weight: 500;
}
.fac-assess-dashboard .action-icons {
    white-space: nowrap;
}
.fac-assess-dashboard .action-icons .fac-icon-btn {
    background: none;
    border: none;
    padding: 0 6px;
    color: #7e8b9f;
    cursor: pointer;
    transition: color 0.1s;
}
.fac-assess-dashboard .action-icons .fac-icon-btn:hover {
    color: #1e6f5c;
}
.fac-assess-dashboard .btn-add {
    background: #1e6f5c;
    color: white;
    border: none;
    padding: 0.5rem 1.2rem;
    border-radius: 2rem;
    font-weight: 500;
    font-size: 0.8rem;
    display: inline-flex;
    align-items: center;
    gap: 0.5rem;
    cursor: pointer;
    transition: background 0.2s;
}
.fac-assess-dashboard .btn-add:hover {
    background: #0f5a49;
    color: white;
}
.fac-assess-dashboard .filter-search input {
    border: 1px solid #cbd5e1;
    border-radius: 2rem;
    padding: 0.45rem 1rem;
    font-size: 0.8rem;
    width: 200px;
}
}
.fac-assess-dashboard .submissions-header {
    display: flex;
    flex-wrap: wrap;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 1.5rem;
    gap: 1rem;
}
}
.fac-assess-dashboard .assessment-tabs {
    display: flex;
    gap: 0.5rem;
    flex-wrap: wrap;
}
}
.fac-assess-dashboard .tab-btn {
    background: #f1f5f9;
    border: none;
    padding: 0.5rem 1.2rem;
    border-radius: 40px;
    font-weight: 500;
    font-size: 0.8rem;
    cursor: pointer;
    transition: all 0.2s;
    color: #2c3e66;
}
}
.fac-assess-dashboard .tab-btn.active {
    background: #1e6f5c;
    color: white;
    box-shadow: 0 2px 6px rgba(30, 111, 92, 0.2);
}
}
.fac-assess-dashboard .filter-search {
    display: flex;
    gap: 0.5rem;
    align-items: center;
}
}
.fac-assess-dashboard .submissions-grid {
    display: flex;
    flex-direction: column;
    gap: 1rem;
}
}

```

```

.fac-assess-dashboard .submission-card {
  background: #ffffff;
  border: 1px solid #eef2ff;
  border-radius: 1rem;
  padding: 1.2rem;
  transition: 0.1s;
  box-shadow: 0 1px 2px rgba(0, 0, 0, 0.02);
}
.fac-assess-dashboard .submission-card .fac-filter-hide {
  display: none !important;
}
.fac-assess-dashboard .submission-card .card-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  flex-wrap: wrap;
  gap: 0.5rem;
  margin-bottom: 0.75rem;
}
.fac-assess-dashboard .student-name {
  font-weight: 700;
  font-size: 1rem;
  display: flex;
  align-items: center;
  flex-wrap: wrap;
  gap: 0.5rem;
}
.fac-assess-dashboard .student-name .status {
  font-size: 0.7rem;
  font-weight: 600;
  padding: 0.2rem 0.7rem;
  border-radius: 20px;
  background: #fef9e3;
  color: #b45309;
}
.fac-assess-dashboard .student-name .status.graded {
  background: #e0f2e9;
  color: #1f6e43;
}
.fac-assess-dashboard .student-name .status.pending {
  background: #f1f5f9;
  color: #475569;
}
.fac-assess-dashboard .submission-meta {
  font-size: 0.75rem;
  color: #5f6c84;
  margin-bottom: 0.75rem;
  display: flex;
  gap: 1rem;
  flex-wrap: wrap;
}
.fac-assess-dashboard .submission-answer-box {
  font-size: 0.8rem;
  color: #475569;
  border: 1px solid #e2e8f0;
  border-radius: 0.8rem;
  padding: 0.75rem 1rem;
  margin-bottom: 1rem;
  max-height: 14rem;
  overflow-y: auto;
  background: #fafcff;
}
.fac-assess-dashboard .score-feedback-area {
  display: flex;
  flex-wrap: wrap;
  gap: 1.5rem;
  align-items: flex-start;
  margin-top: 0.5rem;
}
.fac-assess-dashboard .score-box {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  flex-wrap: wrap;
}
.fac-assess-dashboard .score-box label {
  font-weight: 500;
  font-size: 0.8rem;
}
.fac-assess-dashboard .score-input {

```

```

    width: 90px;
    padding: 0.45rem 0.6rem;
    border: 1px solid #cbd5e1;
    border-radius: 0.8rem;
    font-weight: 500;
  }
  .fac-assess-dashboard .feedback-box {
    flex: 2;
    min-width: 180px;
  }
  .fac-assess-dashboard .feedback-box textarea {
    width: 100%;
    border: 1px solid #cbd5e1;
    border-radius: 0.8rem;
    padding: 0.5rem;
    font-family: inherit;
    font-size: 0.8rem;
    resize: vertical;
  }
  .fac-assess-dashboard .char-count {
    font-size: 0.65rem;
    color: #7e8b9f;
    text-align: right;
    margin-top: 0.2rem;
  }
  .fac-assess-dashboard .action-buttons {
    display: flex;
    gap: 0.6rem;
    align-items: center;
    margin-left: auto;
  }
  .fac-assess-dashboard .save-btn {
    background: #1e6f5c;
    color: white;
    border: none;
    padding: 0.4rem 1rem;
    border-radius: 2rem;
    font-size: 0.75rem;
    font-weight: 500;
    cursor: pointer;
  }
  .fac-assess-dashboard .save-btn:hover {
    background: #0f5a49;
    color: white;
  }
  .fac-assess-dashboard .cancel-btn {
    background: #eef2ff;
    border: none;
    padding: 0.4rem 1rem;
    border-radius: 2rem;
    font-size: 0.75rem;
    cursor: pointer;
  }
  .fac-assess-dashboard .empty-hint {
    text-align: center;
    padding: 2rem;
    background: #fafcff;
    border-radius: 1rem;
    color: #64748b;
    font-size: 0.9rem;
    border: 1px dashed #e2e8f0;
  }
  #toastMsg.toast-msg {
    position: fixed;
    bottom: 20px;
    right: 20px;
    background: #1e293b;
    color: white;
    padding: 0.6rem 1.2rem;
    border-radius: 40px;
    font-size: 0.8rem;
    font-family: 'Inter', system-ui, sans-serif;
    z-index: 2000;
    opacity: 0;
    transition: opacity 0.2s;
    pointer-events: none;
    max-width: 90vw;
  }
  .fac-assess-dashboard .install-alert {
    background: #fffbeb;

```

```

border: 1px solid #fcd34d;
border-radius: 1rem;
padding: 1rem 1.25rem;
color: #92400e;
margin-bottom: 1.5rem;
}
@media (max-width: 700px) {
.fac-assess-dashboard .score-feedback-area { flex-direction: column; }
.fac-assess-dashboard .action-buttons { margin-left: 0; margin-top: 0.5rem; }
.fac-assess-dashboard .filter-search input { width: 150px; }
.fac-assess-dashboard .top-nav { flex-direction: column; align-items: flex-start; }
}
.fac-assess-modal .btn-primary {
background: #1e6f5c;
border-color: #1e6f5c;
}
.fac-assess-modal .btn-primary:hover {
background: #0f5a49;
border-color: #0f5a49;
}
.assessment-timer-preview {
display: inline-block;
font-family: "Consolas", "Courier New", monospace;
font-size: 1.35rem;
font-weight: 700;
letter-spacing: 0.08em;
background: #0b1220;
color: #22d3ee;
padding: 0.35rem 0.75rem;
border-radius: 0.5rem;
box-shadow: inset 0 0 1px rgba(34, 211, 238, 0.35);
min-width: 6.5rem;
text-align: center;
}
}
</style>

<div class="fac-assess-dashboard">
  <div class="dashboard-inner">
    <?php if ($classroom && $missingTables === []): ?>
      <?php faculty_classroom_render_banner($classroom, [
        'meta_extra' => 'Assessments & grading',
        'form_id' => 'facultyAssessBanner',
      ]); ?>
      <div class="top-nav">
        <div class="course-info">
          <h1 class="h5 mb-0"><i class="fa-solid fa-clipboard-list" style="margin-right: 8px; color:#1e6f5c;"></i> Assessment
tools</h1>

          <div class="sub">
            <span><i class="fa-solid fa-list-check"></i> Written work, performance tasks, submissions & grades</span>
          </div>
        </div>
        <div class="d-flex flex-wrap align-items-center gap-3">
          <div class="top-nav-actions">
            <a href="faculty_classroom.php?id=<?= (int) $classroomId ?>" class="fac-nav-link">Classroom</a>
            <a href="faculty_classrooms.php" class="fac-nav-link">All classes</a>
          </div>
          <div class="faculty-profile">
            <div class="avatar"><?= htmlspecialchars($facultyInitials) ?></div>
            <div>
              <div class="faculty-name"><?= htmlspecialchars($facultyDisplayName) ?></div>
              <div class="faculty-role">Course instructor</div>
            </div>
          </div>
        </div>
      </div>
    <?php else: ?>
      <div class="top-nav">
        <div class="course-info">
          <h1><i class="fa-solid fa-clipboard-list" style="margin-right: 8px; color:#1e6f5c;"></i> Assessments & grading</h1>
          <div class="sub"><span>Faculty scheduling</span></div>
        </div>
        <div class="faculty-profile">
          <div class="avatar"><?= htmlspecialchars($facultyInitials) ?></div>
          <div>
            <div class="faculty-name"><?= htmlspecialchars($facultyDisplayName) ?></div>
            <div class="faculty-role">Course instructor</div>
          </div>
        </div>
      </div>
    </div>
  <?php endif; ?>

```

```

<?php if ($missingTables !== []): ?>
    <div class="install-alert">
        Assessment features are not installed yet. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this
page.
    </div>
<?php else: ?>

    <?php if ($flash !== ''): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

    <?php if ($pendingRetakeRequests !== []): ?>
    <div class="fac-dash-card">
        <div class="section-title">
            <i class="fa-solid fa-rotate-left"></i> Retake requests
            <span class="badge bg-warning text-dark ms-1"><?php count($pendingRetakeRequests) ?> pending</span>
        </div>
        <p class="text-muted small mb-3">Students with locked assessments may request another attempt. Review their explanation and
approve or deny.</p>
        <?php foreach ($pendingRetakeRequests as $retakeReq): ?>
            <?php $reqId = (int) ($retakeReq['id'] ?? 0); ?>
            <div class="border rounded p-3 mb-3">
                <div class="d-flex flex-wrap justify-content-between align-items-start gap-2 mb-2">
                    <div>
                        <strong><?php htmlspecialchars((string) ($retakeReq['student_name'] ?? '')) ?></strong>
                        <?php if ((string) ($retakeReq['student_number'] ?? '') !== ''): ?>
                            <span class="text-muted small"><?php htmlspecialchars((string) $retakeReq['student_number']) ?></span>
                        <?php endif; ?>
                        <div class="small text-muted mt-1">
                            <i class="fa-solid fa-file-pen me-1"></i><?php htmlspecialchars((string) ($retakeReq['assessment_title']
?? '')) ?>
                                &bull; Requested <?php htmlspecialchars((string) ($retakeReq['created_at'] ?? '')) ?>
                            </div>
                        </div>
                        <span class="badge bg-danger">Integrity violation</span>
                    </div>
                    <div class="small mb-3 p-2 bg-light rounded">
                        <strong>Student explanation:</strong><br>
                        <?php nl2br(htmlspecialchars((string) ($retakeReq['reason'] ?? ''))) ?>
                    </div>
                    <div class="d-flex flex-wrap gap-2 align-items-start">
                        <form method="post" class="d-inline" onsubmit="return confirm('Approve this retake? The student\'s locked
submission and score will be cleared so they can try again.');">
                            <input type="hidden" name="action" value="approve_retake_request">
                            <input type="hidden" name="classroom_id" value="<?php (int) $classroomId ?>">
                            <input type="hidden" name="request_id" value="<?php $reqId ?>">
                            <button type="submit" class="btn btn-success btn-sm">
                                <i class="fa-solid fa-check me-1"></i>Approve retake
                            </button>
                        </form>
                        <form method="post" class="d-flex flex-wrap gap-2 align-items-start flex-grow-1">
                            <input type="hidden" name="action" value="reject_retake_request">
                            <input type="hidden" name="classroom_id" value="<?php (int) $classroomId ?>">
                            <input type="hidden" name="request_id" value="<?php $reqId ?>">
                            <input type="text" name="faculty_remarks" class="form-control form-control-sm" placeholder="Optional note to
student" maxlength="500" style="max-width:280px;">
                            <button type="submit" class="btn btn-outline-secondary btn-sm">
                                <i class="fa-solid fa-xmark me-1"></i>Deny
                            </button>
                        </form>
                    </div>
                </div>
            </div>
        <?php endforeach; ?>
    </div>

    <div class="fac-dash-card">
        <div class="section-title">
            <i class="fa-solid fa-clipboard-list"></i> Assessment manager
        </div>
        <div class="assessment-toolbar">
            <input type="search" class="filter-search" id="assessmentTableFilter" placeholder="Filter table..." style="border:1px
solid #cbd5e1;border-radius:2rem;padding:0.45rem 1rem;font-size:0.8rem;width:min(220px,100%);" autocomplete="off">
            <button type="button" class="btn-add" data-bs-toggle="modal" data-bs-target="#modalAddAssessment"><?php
app_tooltip_attr('Opens the form to add a quiz or assignment students will see in this class.') ?>>
                <i class="fa-solid fa-plus-circle"></i> Add assessment
            </button>
        </div>
    </div>
    <?php if ($assessments === []): ?>
        <div class="empty-hint">
            No assessments yet. Use <strong>Add assessment</strong> to create one. Student cards appear here once learners are

```

enrolled.

```
</div>
<?php else: ?>
<?php
$weekGroups = [];
foreach ($assessments as $row) {
    $wk = $hasCreditedWeek ? trim((string) ($row['credited_week'] ?? '')) : '';
    $label = $wk !== '' ? $wk : 'Unassigned';
    if (!isset($weekGroups[$label])) {
        $weekGroups[$label] = [];
    }
    $weekGroups[$label][] = $row;
}
uksort($weekGroups, static function (string $a, string $b): int {
    $na = $a === 'Unassigned' ? 999 : ((int) preg_replace('/\D/', '', $a) ?: 999);
    $nb = $b === 'Unassigned' ? 999 : ((int) preg_replace('/\D/', '', $b) ?: 999);
    return $na <=> $nb;
});
$tableCols = $hasCreditedWeek ? 6 : 6;
?>
<div class="assessment-table-wrapper">
    <table class="assessment-table" id="assessmentsDataTable">
        <thead>
            <tr>
                <th class="sortable" data-sort="type" data-type="string">Type</th>
                <th class="sortable" data-sort="title" data-type="string">Title</th>
                <th class="sortable" data-sort="points" data-type="number">Points</th>
                <th class="d-none d-md-table-cell">Description</th>
                <th class="sortable d-none d-lg-table-cell" data-sort="due" data-type="string">Due</th>
                <th style="width: 100px;"></th>
            </tr>
        </thead>
        <tbody>
            <?php foreach ($weekGroups as $weekLabel => $weekRows): ?>
                <?php if ($hasCreditedWeek): ?>
                    <tr class="week-group-header" data-filter="<?=> htmlspecialchars(faculty_assess_lc((string) $weekLabel))>
                        <td colspan="<?=> $tableCols ?>">
                            <div class="week-group-cell">
                                <i class="fa-regular fa-calendar-alt"></i>
                                <strong><?=> htmlspecialchars((string) $weekLabel) ?></strong>
                                <span class="week-count-badge"><?=> count($weekRows) ?> assessment<?=> count($weekRows) === 1
                                ? '' : 's' ?></span>
                            </div>
                        </td>
                    </tr>
                <?php endif; ?>
                <?php foreach ($weekRows as $row): ?>
                    <?php
                    $aid = (int) $row['id'];
                    $questionsForAssessment = $questionMap[$aid] ?? [];
                    $typeLabel = classroom_assessment_type_label((string) ($row['assessment_type'] ?? 'essay'));
                    $dueRaw = (string) ($row['due_at'] ?? '');
                    $desc = trim((string) ($row['description'] ?? ''));
                    $descPlain = $desc === '' ? '' : (preg_replace('/\s+/u', ' ',
trim(html_entity_decode(strip_tags(str_replace('&nbsp;', ' ', $desc)), ENT_QUOTES | ENT_HTML5, 'UTF-8')))) ?? '');
                    $isPerf = classroom_assessment_normalize_type((string) ($row['assessment_type'] ?? 'essay')) ===
'problem_solving';

                    $serializedQuestions = [];
                    foreach ($questionsForAssessment as $q) {
                        $serializedQuestions[] = [
                            'question_type' => (string) ($q['question_type'] ?? 'essay'),
                            'question_text' => (string) ($q['question_text'] ?? ''),
                            'points' => (float) ($q['points'] ?? 0),
                            'options' => json_decode((string) ($q['options_json'] ?? '[]'), true) ?: [],
                            'answer_key' => (string) ($q['answer_key'] ?? ''),
                            'word_limit' => $q['word_limit'],
                            'char_limit' => $q['char_limit'],
                            'allow_steps' => (int) ($q['allow_steps'] ?? 0),
                        ];
                    }
                    $questionCount = count($questionsForAssessment);
                    ?>
                    <tr
                        data-filter="<?=> htmlspecialchars(faculty_assess_lc($typeLabel . ' ' . (string) $row['title'] .
' ' . $row['total_points'] . ' ' . $dueRaw . ' ' . $descPlain . ' ' . $questionCount . ' questions ' . ($weekLabel !== 'Unassigned' ?
$weekLabel : ''))) ?>"
                        data-type="<?=> htmlspecialchars(faculty_assess_lc($typeLabel)) ?>"
                        data-title="<?=> htmlspecialchars(faculty_assess_lc((string) $row['title'])) ?>"
                        data-points="<?=> htmlspecialchars((string) (float) $row['total_points']) ?>"
                    >
```

```

data-due="<?>= htmlspecialchars($dueRow !== '' ? $dueRow : '9999-99-99') ?>"
data-week="<?>= htmlspecialchars((string) $weekLabel) ?>"
>
<td>
    <span class="type-badge <?>= $isPerf ? 'type-performance' : '' ?>"><?>=
htmlspecialchars($typeLabel) ?></span>
</td>
<td><strong><?>= htmlspecialchars((string) $row['title']) ?></strong></td>
<td><span class="points"><?>= number_format((float) $row['total_points'], 2) ?></span></td>
<td class="d-none d-md-table-cell">
    <?php if ($descPlain !== ''): ?>
        <?php
            $descPreview = $descPlain;
            if (function_exists('mb_strlen') && mb_strlen($descPreview) > 120) {
                $descPreview = mb_substr($descPreview, 0, 117) . '...';
            } elseif (strlen($descPreview) > 120) {
                $descPreview = substr($descPreview, 0, 117) . '...';
            }
        ?>
        <span style="color:#5b6e8c;font-size:0.85rem;"><?>= htmlspecialchars($descPreview) ?>
    </span>

    <?php else: ?>
        <span style="color:#94a3b8;font-size:0.8rem;font-style:italic;">No description</span>
    <?php endif: ?>
    <div style="color:#64748b;font-size:0.75rem;margin-top:0.25rem;"><?>= (int) $questionCount ?>
question<?>= $questionCount === 1 ? '' : 's' ?><?php
    $rowTimeLimit = (int) ($row['time_limit_minutes'] ?? 0);
    if ($hasTimeLimit && $rowTimeLimit > 0):
        $rh = intval($rowTimeLimit, 60);
        $rm = $rowTimeLimit % 60;
        $rLabel = $rh > 0 ? ($rh . 'h' . ($rm > 0 ? ' ' . $rm . 'm' : '')) : ($rm . ' min');
        echo ' . <i class="fa-regular fa-clock"></i> ' . htmlspecialchars($rLabel);
    endif;
?></div>
</td>
<td class="d-none d-lg-table-cell" style="color:#5b6e8c;font-size:0.85rem;"><?>= $dueRow !== '' ?
htmlspecialchars($dueRow) : '-' ?></td>
<td class="action-icons">
    <button type="button" class="fac-icon-btn" title="Edit"><?>= app_tooltip_attr('Opens the
editor to change title, points, due date, or instructions for this assessment.') ?>
    data-bs-toggle="modal" data-bs-target="#modalEditAssessment"
    data-id="<?>= $aid ?>"
    data-type="<?>= htmlspecialchars((string) ($row['assessment_type'] ?? 'essay')) ?>"
    data-title="<?>= htmlspecialchars((string) $row['title']) ?>"
    data-points="<?>= htmlspecialchars((string) $row['total_points']) ?>"
    data-due="<?>= htmlspecialchars(faculty_assessment_due_for_input($dueRow !== '' ? $dueRow
: null)) ?>"
    data-time-limit="<?>= (int) ($row['time_limit_minutes'] ?? 0) ?>"
    data-credited-week="<?>= htmlspecialchars((string) ($row['credited_week'] ?? '')) ?>"
    data-description="<?>= htmlspecialchars(str_replace(["\r", "\n", "\t"], ' ', $desc),
ENT_QUOTES, 'UTF-8') ?>"
    data-questions="<?>= htmlspecialchars((string) json_encode($serializedQuestions,
JSON_UNESCAPED_UNICODE), ENT_QUOTES, 'UTF-8') ?>"
    <i class="fa-solid fa-pen-to-square"></i>
</button>
    <?php if ($otherClassrooms !== []): ?>
        <button type="button"
            class="fac-icon-btn"
            title="Assign to another course"
            data-bs-toggle="modal"
            data-bs-target="#assignAssessmentModal"
            data-assessment-id="<?>= $aid ?>"
            data-assessment-title="<?>= htmlspecialchars((string) $row['title'], ENT_QUOTES)
?>"><?>= app_tooltip_attr('Copies this quiz or assignment into another class you teach. Grades and submissions are not copied.') ?>>
            <i class="fa-solid fa-share-from-square"></i>
        </button>
    <?php endif: ?>
    <form method="post" class="d-inline" onsubmit="return confirm('Delete this assessment and
all related grades and submissions?');">
        <input type="hidden" name="action" value="delete_assessment">
        <input type="hidden" name="classroom_id" value="<?>= (int) $classroomId ?>">
        <input type="hidden" name="assessment_id" value="<?>= $aid ?>">
        <button type="submit" class="fac-icon-btn" title="Delete"><?>= app_tooltip_attr('Deletes
this assessment and related grades after confirmation.') ?>><i class="fa-solid fa-trash-can"></i></button>
    </form>
</td>
</tr>
<?php endforeach; ?>
<?php endforeach; ?>
</tbody>

```



```

        </table>
    </div>
    <?php endif; ?>
</div>

<div class="fac-dash-card">
    <div class="section-title">
        <i class="fa-solid fa-inbox"></i> Submissions by assessment
    </div>
    <?php if ($assessments === []): ?>
        <div class="empty-hint">Add an assessment first; submissions and grades will show here.</div>
    <?php elseif ($enrollments === []): ?>
        <div class="empty-hint">Enroll students in this class to see one card per learner with submission text, score, and
feedback.</div>
    <?php else: ?>
        <div class="submissions-header">
            <div class="assessment-tabs" id="assessmentTabsContainer" role="tablist">
                <?php foreach ($assessments as $a): ?>
                    <?php $optId = (int) $a['id']; ?>
                    <button type="button" role="tab" class="tab-btn <?=$optId === $submissionsPickId ? 'active' : '' ?>"
                        data-assessment-tab="<?=$optId ?>"
                        aria-selected="<?=$optId === $submissionsPickId ? 'true' : 'false' ?>"
                        <?=$htmlspecialchars((string) $a['title']) ?> (<?=$number_format((float) $a['total_points'], 0) ?> pts)
                    </button>
                </div>
            <?php endforeach; ?>
            <div class="filter-search">
                <input type="text" id="searchSubmissionInput" placeholder="Search student..." autocomplete="off">
                <i class="fa-solid fa-magnifying-glass" style="color:#7e8b9f;font-size:0.85rem;"></i>
            </div>
            <div id="submissionsListContainer" class="submissions-grid">
                <?php foreach ($assessments as $a): ?>
                    <?php
                    $panelId = (int) $a['id'];
                    $maxPts = (float) $a['total_points'];
                    $maxPtsDisp = abs($maxPts - (float) (int) $maxPts) < 0.00001
                    ? (string) (int) $maxPts
                    : rtrim(rtrim(number_format($maxPts, 2, '.', ''), '0'), '.');
                    >
                    <div class="submissions-panel <?=$panelId === $submissionsPickId ? '' : 'd-none' ?>" data-assessment-panel="<?=$
panelId ?>">
                        <?php foreach ($enrollments as $student): ?>
                            <?php
                            $studentId = (int) $student['student_id'];
                            $saved = $scoreMap[$panelId][$studentId] ?? ['score' => '', 'feedback' => ''];
                            $submission = $submissionMap[$panelId][$studentId] ?? ['answer_text' => '', 'status' => '',
'submitted_at' => '', 'integrity_locked' => 0];
                            $integrityLocked = (int) ($submission['integrity_locked'] ?? 0) === 1
                            || stripos((string) ($saved['feedback'] ?? ''), 'left or minimized the assessment window') !==
false;

                            $status = faculty_submission_grading_status($submission, $saved);
                            $fbVal = (string) ($saved['feedback'] ?? '');
                            $scoreVal = $saved['score'] !== null && $saved['score'] !== '' ? (string) $saved['score'] : '';
                            $anchor = 's-' . $panelId . '-' . $studentId;
                            $filterHay = faculty_assess_lc((string) $student['full_name'] . ' ' . (string)
$student['student_number']);

                            $hasScoreShow = $scoreVal !== '';
                            if ($status === 'graded' && $hasScoreShow) {
                                $statusText = 'Graded: ' . $scoreVal . '/' . $maxPtsDisp;
                                $statusClass = 'graded';
                            } elseif ($status === 'graded') {
                                $statusText = 'Graded';
                                $statusClass = 'graded';
                            } elseif ($status === 'submitted') {
                                $statusText = 'Submitted (pending)';
                                $statusClass = '';
                            } else {
                                $statusText = 'Pending';
                                $statusClass = 'pending';
                            }
                        >
                        <div class="submission-card" id="<?=$htmlspecialchars($anchor) ?>" data-student-filter="<?=$
htmlspecialchars($filterHay) ?>">
                            <div class="card-header">
                                <div style="flex:1;min-width:0;">
                                    <div class="student-name">
                                        <i class="fa-solid fa-user-graduate" style="color:#2c7da0;"></i>
                                        <?=$htmlspecialchars((string) $student['full_name']) ?>
                                        <span class="status <?=$htmlspecialchars($statusClass) ?>"><?=$

```

```

htmlspecialchars($statusText) ?></span>

        <?php if ($integrityLocked): ?>
            <span class="badge bg-danger ms-1">Locked (violation)</span>
        <?php endif; ?>
    </div>
    <div class="submission-meta">
        <span><i class="fa-regular fa-clock"></i>
            <?php if (trim((string) ($submission['answer_text'] ?? '')) !== '' &&
!empty($submission['submitted_at'])): ?>

                Submitted: <?= htmlspecialchars((string) $submission['submitted_at']) ?>
            <?php else: ?>
                No submission on file
            <?php endif; ?>
        </span>
        <span><i class="fa-solid fa-file-lines"></i> Max points: <?=
htmlspecialchars($maxPtsDisp) ?></span>

        <?php if ((string) ($student['student_number'] ?? '') !== ''): ?>
            <span><i class="fa-solid fa-id-card"></i> <?= htmlspecialchars((string)
$student['student_number']) ?></span>

            <?php endif; ?>
        </div>
    </div>
</div>
<div class="submission-answer-box">
    <?php if (trim((string) ($submission['answer_text'] ?? '')) !== ''): ?>
        <?php
            $rawSubmissionAnswer = (string) $submission['answer_text'];
            $decodedSubmission = json_decode($rawSubmissionAnswer, true);
            ?>
        <?php if (is_array($decodedSubmission)): ?>
            <?php foreach ($decodedSubmission as $idx => $ans): ?>
                <div class="small mb-2">
                    <strong>Q<?= (int) ($idx + 1) ?>:</strong>
                    <span><?= htmlspecialchars((string) ($ans['answer_text'] ?? '')) ?></span>
                    <?php if (trim((string) ($ans['answer_steps'] ?? '')) !== ''): ?>
                        <div class="text-muted">Steps: <?= nl2br(htmlspecialchars((string)
$ans['answer_steps'])) ?></div>
                    <?php endif; ?>
                </div>
            <?php endforeach; ?>
        <?php else: ?>
            <div class="classroom-content-body small"><?=
classroom_content_render_body($rawSubmissionAnswer) ?></div>
        <?php endif; ?>
    <?php else: ?>
        <span style="color:#94a3b8;font-style:italic;">Student has not submitted yet. Their answer
will display here.</span>

    <?php endif; ?>
</div>
<form method="post" class="js-submission-form" data-original-score="<?= htmlspecialchars($scoreVal)
?>" data-original-feedback="<?= htmlspecialchars($fbVal) ?>">
    <input type="hidden" name="action" value="save_single_submission">
    <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
    <input type="hidden" name="assessment_id" value="<?= $panelId ?>">
    <input type="hidden" name="student_id" value="<?= $studentId ?>">
    <input type="hidden" name="submissions_assessment_id" value="<?= $panelId ?>">
    <input type="hidden" name="scroll_anchor" value="<?= htmlspecialchars($anchor) ?>">
    <div class="score-feedback-area">
        <div class="score-box">
            <label for="score-<?= $panelId ?>-<?= $studentId ?>">Score:</label>
            <input
                type="number"
                name="score"
                id="score-<?= $panelId ?>-<?= $studentId ?>"
                class="score-input js-score-input"
                min="0"
                step="0.01"
                max="<?= htmlspecialchars((string) $maxPts) ?>"
                value="<?= htmlspecialchars($scoreVal) ?>"
                inputmode="decimal"
                autocomplete="off"
                placeholder="-"
            >
        </div>
        <span style="font-size:0.7rem;color:#64748b;" / <?= htmlspecialchars($maxPtsDisp) ?>
    </div>
    <div class="feedback-box">
        <textarea
            name="feedback"
            id="fb-<?= $panelId ?>-<?= $studentId ?>"

```

```

                                class="js-feedback-input"
                                rows="2"
                                maxlength="4000"
                                placeholder="Write feedback..."
                                data-feedback-id="=(int) $panelId ?"-<?=(int) $studentId ?>"
                                ><?= htmlspecialchars($fbVal) ?></textarea>
                                <div class="char-count"><span class="js-char-count-num"><?=(int)
(function_exists('mb_strlen') ? mb_strlen($fbVal) : strlen($fbVal)) ?></span> characters</div>
                                </div>
                                <div class="action-buttons">
                                    <button type="submit" class="save-btn"><?= app_tooltip_attr('Saves the score and feedback
you entered for this student's submission.') ?>><i class="fa-solid fa-floppy-disk"></i> Save</button>
                                    <button type="button" class="cancel-btn js-cancel-edit"><?= app_tooltip_attr('Closes
inline grading without saving changes to this attempt.') ?>>Cancel</button>
                                </div>
                            </form>
                        </div>
                    <?php endforeach; ?>
                </div>
            <?php endforeach; ?>
        </div>
    <?php endif; ?>
</div>
</div>

<div id="toastMsg" class="toast-msg" aria-live="polite"></div>

<div class="modal fade" id="modalAddAssessment" tabindex="-1" aria-labelledby="modalAddAssessmentLabel" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered modal-lg fac-assess-modal">
        <div class="modal-content rounded-4 border-0 shadow">
            <div class="modal-header border-0 pb-0">
                <h2 class="modal-title h5 fw-semibold" id="modalAddAssessmentLabel">Add assessment</h2>
                <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"><?= app_tooltip_attr('Closes the dialog
without submitting the form.') ?>></button>
            </div>
            <div class="modal-body pt-2">
                <form method="post" id="formAddAssessment">
                    <input type="hidden" name="action" value="add_assessment">
                    <input type="hidden" name="classroom_id" value="=(int) $classroomId ?&gt;"&gt;
                    &lt;input type="hidden" name="questions_json" id="add_questions_json" value=""&gt;
                    &lt;div class="row g-3"&gt;
                        &lt;div class="col-md-4"&gt;
                            &lt;label class="form-label small"&gt;Type&lt;/label&gt;
                            &lt;select name="assessment_type" class="form-select"&gt;
                                &lt;option value="multiple_choice"&gt;Multiple Choice&lt;/option&gt;
                                &lt;option value="true_false"&gt;True or False&lt;/option&gt;
                                &lt;option value="essay"&gt;Essay&lt;/option&gt;
                                &lt;option value="problem_solving"&gt;Problem Solving&lt;/option&gt;
                            &lt;/select&gt;
                        &lt;/div&gt;
                        &lt;div class="col-md-8"&gt;
                            &lt;label class="form-label small"&gt;Title&lt;/label&gt;
                            &lt;input type="text" name="title" class="form-control" maxlength="150" required placeholder="e.g. Module 3 – Case
analysis"&gt;
                        &lt;/div&gt;
                        &lt;div class="col-md-4"&gt;
                            &lt;label class="form-label small"&gt;Points&lt;/label&gt;
                            &lt;input type="number" name="total_points" class="form-control" min="1" step="0.01" value="100" required&gt;
                        &lt;/div&gt;
                        &lt;div class="col-md-4"&gt;
                            &lt;label class="form-label small"&gt;Due date&lt;/label&gt;
                            &lt;input type="datetime-local" name="due_at" class="form-control"&gt;
                        &lt;/div&gt;
                    &lt;?php if ($hasCreditedWeek): ?&gt;
                        &lt;div class="col-md-4"&gt;
                            &lt;label class="form-label small"&gt;Credited week&lt;/label&gt;
                            &lt;select name="credited_week" class="form-select"&gt;
                                &lt;option value=""&gt;No specific week&lt;/option&gt;
                                &lt;?php for ($wn = 1; $wn &lt;= 18; $wn++): ?&gt;
                                    &lt;option value="Week &lt;?= $wn ?&gt;"&gt;Week &lt;?= $wn ?&gt;&lt;/option&gt;
                                &lt;?php endfor; ?&gt;
                            &lt;/select&gt;
                        &lt;/div&gt;
                    &lt;?php endif; ?&gt;
                    &lt;?php if ($hasTimeLimit): ?&gt;
                        &lt;div class="col-12"&gt;
</pre

```

```

<div class="border rounded-3 p-3 bg-light-subtle">
  <div class="form-check form-switch mb-2">
    <input class="form-check-input" type="checkbox" role="switch" name="timer_enabled" value="1"
id="add_timer_enabled">
    <label class="form-check-label fw-semibold" for="add_timer_enabled">Timer settings</label>
  </div>
  <div class="row g-2 align-items-end" id="addTimerFields" style="display:none;">
    <div class="col-auto">
      <label class="form-label small mb-0" for="add_timer_hours">Hours</label>
      <input type="number" name="timer_hours" id="add_timer_hours" class="form-control" min="0" max="12"
value="0" style="width:5.5rem;">
    </div>
    <div class="col-auto">
      <label class="form-label small mb-0" for="add_timer_minutes">Minutes</label>
      <input type="number" name="timer_minutes" id="add_timer_minutes" class="form-control" min="0"
max="59" value="30" style="width:5.5rem;">
    </div>
    <div class="col-auto pb-1">
      <span class="assessment-timer-preview" id="addTimerPreview" aria-hidden="true">00:30:00</span>
    </div>
  </div>
  <div class="form-text small mt-1">When enabled, students see a digital countdown after they start the
assessment. Leave off for untimed work.</div>
</div>
<?php endif; ?>
<div class="col-12">
  <label class="form-label small fw-semibold">Questions</label>
  <div class="border rounded-3 p-3 mb-2 bg-light-subtle" id="addAutoGeneratePanel"
data-classroom-id="<?=(int) $classroomId ?>"
data-api="<?= htmlspecialchars('api/faculty_assessment_generate_questions.php', ENT_QUOTES, 'UTF-8') ?>"
data-ai-available="<?= $assessmentAiAvailable ? '1' : '0' ?>"
  <div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-2">
    <span class="small fw-semibold"><i class="fa-solid fa-wand-magic-sparkles me-1 text-primary"></i>Auto-
generate from week materials</span>
    <?php if ($assessmentAiAvailable): ?>
      <span class="badge text-bg-primary-subtle border text-primary-emphasis">AI + attachments</span>
    <?php else: ?>
      <span class="badge text-bg-secondary-subtle border">Materials-based</span>
    <?php endif; ?>
  </div>
  <p class="small text-muted mb-2">Select a week to build questions from posted materials and file attachments
(PDF, DOCX, slides, text).</p>
  <div class="row g-2 align-items-end">
    <div class="col-sm-3">
      <label class="form-label small mb-0" for="add_auto_count">Count</label>
      <input type="number" class="form-control form-control-sm" id="add_auto_count" min="1" max="20"
value="5">
    </div>
    <div class="col-sm-5">
      <label class="form-label small mb-0" for="add_auto_topic">Topic (optional)</label>
      <input type="text" class="form-control form-control-sm" id="add_auto_topic" maxlength="200"
placeholder="Uses assessment title if blank">
    </div>
  <?php if ($hasCreditedWeek): ?>
    <div class="col-sm-4">
      <label class="form-label small mb-0" for="add_auto_week">From week materials</label>
      <select class="form-select form-select-sm" id="add_auto_week">
        <option value="">Select week...</option>
        <?php for ($wn = 1; $wn <= 18; $wn++): ?>
          <option value="Week <?= $wn ?>">Week <?= $wn ?></option>
        <?php endfor; ?>
      </select>
    </div>
  <?php endif; ?>
  <?php if ($hasCreditedWeek): ?>
    <div class="col-12">
      <div class="form-check">
        <input class="form-check-input" type="checkbox" value="1" id="add_auto_use_materials" checked>
        <label class="form-check-label small" for="add_auto_use_materials">Include posted materials and
attachments for the selected week</label>
      </div>
      <div class="small text-muted mt-1 d-none" id="add_auto_week_preview"></div>
    </div>
  <?php endif; ?>
  <div class="col-12 d-flex flex-wrap gap-2">
    <button type="button" class="btn btn-primary btn-sm" id="addAutoGenerateBtn"<?%=
app_tooltip_attr('Creates questions from the selected week materials and attachments.') ?>>
      <i class="fa-solid fa-wand-magic-sparkles me-1"></i>Generate questions
    </button>
  </div>

```

```

        <button type="button" class="btn btn-outline-primary btn-sm d-none" id="addAutoGenerateMoreBtn">=?
app_tooltip_attr('Adds more questions without removing existing ones.') ?>>
        <i class="fa-solid fa-plus me-1"></i>Generate more
    </button>
</div>
<div class="col-12">
    <div class="small text-muted d-none" id="addAutoGenerateStatus"></div>
</div>
</div>
<div class="small text-muted mb-1">Total items: <span id="addTotalItems">0</span></div>
<div class="border rounded-3 p-2" id="addQuestionBuilder"></div>
<button type="button" class="btn btn-outline-secondary btn-sm mt-2" id="addQuestionRowBtn">+ Add
question</button>

    <div class="form-text small">Choose a type above, select a week, then auto-generate or add questions manually.
Answer keys are filled for auto-graded types.</div>
</div>
<div class="col-12">
    <label class="form-label small" for="add-assessment-description">Description</label>
    <div class="wordpad-shell" data-wordpad data-wordpad-name="description">
        <div class="wordpad-toolbar d-none" role="toolbar" aria-label="Formatting toolbar">
            <select class="form-select form-select-sm wordpad-select" data-wordpad-block aria-label="Text style">
                <option value="p"><p></p></option>
                <option value="h3"><h3></h3></option>
                <option value="blockquote"><blockquote></blockquote></option>
            </select>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="bold" title="Bold">=?
app_tooltip_attr('Makes selected text bold in the description.') ?>><i class="fa-solid fa-bold"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="italic" title="Italic">=?
app_tooltip_attr('Italicizes selected text for emphasis.') ?>><i class="fa-solid fa-italic"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="underline"
title="Underline">=? app_tooltip_attr('Underlines selected text.') ?>><i class="fa-solid fa-underline"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="insertUnorderedList"
title="Bulleted list">=? app_tooltip_attr('Starts or continues a bulleted list.') ?>><i class="fa-solid fa-list-ul"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="insertOrderedList"
title="Numbered list">=? app_tooltip_attr('Starts or continues a numbered list.') ?>><i class="fa-solid fa-list-ol"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="justifyLeft" title="Align
left">=? app_tooltip_attr('Aligns paragraph text to the left.') ?>><i class="fa-solid fa-align-left"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="justifyCenter" title="Align
center">=? app_tooltip_attr('Centers paragraph text.') ?>><i class="fa-solid fa-align-center"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="justifyRight" title="Align
right">=? app_tooltip_attr('Aligns paragraph text to the right.') ?>><i class="fa-solid fa-align-right"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="createLink" title="Insert
link">=? app_tooltip_attr('Turns the selection into a hyperlink.') ?>><i class="fa-solid fa-link"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="unlink" title="Remove link"
>=? app_tooltip_attr('Removes the link from the selected text without deleting the text.') ?>><i class="fa-solid fa-link-slash"></i>
</button>

            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="removeFormat" title="Clear
formatting">=? app_tooltip_attr('Strips bold, italics, and other formatting from the selection.') ?>><i class="fa-solid fa-eraser"></i>
</button>

        </div>
        <div class="wordpad-editor form-control d-none" contenteditable="true" data-placeholder="Instructions and
expectations for students."></div>
        <textarea id="add-assessment-description" name="description" class="form-control" rows="5"
placeholder="Plain text if the rich editor is unavailable"></textarea>
    </div>
    <div class="form-text small">Bold, lists, alignment, and links—similar to WordPad.</div>
</div>
</div>
<div class="d-flex justify-content-end gap-2 mt-4">
    <button type="button" class="btn btn-outline-secondary rounded-pill" data-bs-dismiss="modal">=?
app_tooltip_attr('Closes the add dialog without creating an assessment.') ?>>Cancel</button>
    <button type="submit" class="btn btn-primary rounded-pill px-4">=? app_tooltip_attr('Creates the assessment with the
details you entered. Students can then submit work.') ?>>Create assessment</button>
</div>
</form>
</div>
</div>
</div>
</div>
</div>
<?php if ($otherClassrooms !== []): ?>
<div class="modal fade" id="assignAssessmentModal" tabindex="-1" aria-labelledby="assignAssessmentModalLabel" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content rounded-4 border-0 shadow">
            <form method="post">
                <input type="hidden" name="action" value="assign_assessment">
                <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
                <input type="hidden" name="assessment_id" id="assign-assessment-id" value="">
                <div class="modal-header border-0 pb-0">

```

```

        <h2 class="modal-title h5 fw-semibold" id="assignAssessmentModallabel">Assign to another course</h2>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
    </div>
    <div class="modal-body pt-2">
        <p class="small text-muted mb-3">Copy <strong id="assign-assessment-title">this assessment</strong> into another class
        you teach. Questions and settings are copied; grades and student submissions stay in this class.</p>
        <label class="form-label small fw-semibold" for="assign-target-assessment-classroom">Target course</label>
        <select id="assign-target-assessment-classroom" name="target_classroom_id" class="form-select" required>
            <option value="">Select a course</option>
            <?php foreach ($otherClassrooms as $other): ?>
                <?php
                $optionLabel = trim((string) ($other['course_code'] ?? ''));
                if ($optionLabel !== '' && trim((string) ($other['course_name'] ?? '')) !== '') {
                    $optionLabel .= ' - ' . trim((string) $other['course_name']);
                } elseif (trim((string) ($other['course_name'] ?? '')) !== '') {
                    $optionLabel = trim((string) $other['course_name']);
                } else {
                    $optionLabel = trim((string) ($other['title'] ?? 'Classroom'));
                }
                $termLabel = trim((string) ($other['semester'] ?? '')) . ' ' . (string) ($other['school_year'] ?? '');
                if ($termLabel !== '') {
                    $optionLabel .= ' ( ' . $termLabel . ' )';
                }
                <?php
                <option value="<?= (int) $other['id'] ?>"><?= htmlspecialchars($optionLabel) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="modal-footer border-0 pt-0">
        <button type="button" class="btn btn-outline-secondary rounded-pill" data-bs-dismiss="modal">Cancel</button>
        <button type="submit" class="btn btn-primary rounded-pill px-4">Assign copy</button>
    </div>
</form>
</div>
</div>
</div>
<?php endif; ?>

<div class="modal fade" id="modalEditAssessment" tabindex="-1" aria-labelledby="modalEditAssessmentLabel" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered modal-lg fac-assess-modal">
        <div class="modal-content rounded-4 border-0 shadow">
            <div class="modal-header border-0 pb-0">
                <h2 class="modal-title h5 fw-semibold" id="modalEditAssessmentLabel">Edit assessment</h2>
                <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"><?= app_tooltip_attr('Closes the edit
                dialog without saving.') ?></button>
            </div>
            <div class="modal-body pt-2">
                <form method="post" id="formEditAssessment">
                    <input type="hidden" name="action" value="update_assessment">
                    <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
                    <input type="hidden" name="assessment_id" id="edit_assessment_id" value="">
                    <input type="hidden" name="questions_json" id="edit_questions_json" value="[]">
                    <div class="row g-3">
                        <div class="col-md-4">
                            <label class="form-label small">Type</label>
                            <select name="assessment_type" class="form-select" id="edit_assessment_type">
                                <option value="multiple_choice">Multiple Choice</option>
                                <option value="true_false">True or False</option>
                                <option value="essay">Essay</option>
                                <option value="problem_solving">Problem Solving</option>
                            </select>
                        </div>
                        <div class="col-md-8">
                            <label class="form-label small">Title</label>
                            <input type="text" name="title" class="form-control" id="edit_title" maxlength="150" required>
                        </div>
                        <div class="col-md-4">
                            <label class="form-label small">Points</label>
                            <input type="number" name="total_points" class="form-control" id="edit_points" min="1" step="0.01" required>
                        </div>
                        <div class="col-md-4">
                            <label class="form-label small">Due date</label>
                            <input type="datetime-local" name="due_at" class="form-control" id="edit_due">
                        </div>
                        <?php if ($hasCreditedWeek): ?>
                        <div class="col-md-4">
                            <label class="form-label small">Credited week</label>
                            <select name="credited_week" id="edit_credited_week" class="form-select">
                                <option value="">No specific week</option>
                                <?php for ($wn = 1; $wn <= 18; $wn++): ?>

```

```

                <option value="Week <?= $wn ?>">Week <?= $wn ?></option>
            <?php endfor; ?>
        </select>
    </div>
    <?php endif; ?>
    <?php if ($hasTimeLimit): ?>
    <div class="col-12">
        <div class="border rounded-3 p-3 bg-light-subtle">
            <div class="form-check form-switch mb-2">
                <input class="form-check-input" type="checkbox" role="switch" name="timer_enabled" value="1"
id="edit_timer_enabled">
                <label class="form-check-label fw-semibold" for="edit_timer_enabled">Timer settings</label>
            </div>
            <div class="row g-2 align-items-end" id="editTimerFields" style="display:none;">
                <div class="col-auto">
                    <label class="form-label small mb-0" for="edit_timer_hours">Hours</label>
                    <input type="number" name="timer_hours" id="edit_timer_hours" class="form-control" min="0" max="12"
value="0" style="width:5.5rem;">
                </div>
                <div class="col-auto">
                    <label class="form-label small mb-0" for="edit_timer_minutes">Minutes</label>
                    <input type="number" name="timer_minutes" id="edit_timer_minutes" class="form-control" min="0"
max="59" value="30" style="width:5.5rem;">
                </div>
                <div class="col-auto pb-1">
                    <span class="assessment-timer-preview" id="editTimerPreview" aria-hidden="true">00:30:00</span>
                </div>
            </div>
            <div class="form-text small mt-1">When enabled, students see a digital countdown after they start the
assessment.</div>
        </div>
    </div>
    <?php endif; ?>
    <div class="col-12">
        <label class="form-label small fw-semibold">Questions</label>
        <div class="small text-muted mb-1">Total items: <span id="editTotalItems">0</span></div>
        <div class="border rounded-3 p-2" id="editQuestionBuilder"></div>
        <button type="button" class="btn btn-outline-secondary btn-sm mt-2" id="editQuestionRowBtn">+ Add
question</button>

        <div class="form-text small">Update questions and answer keys. Objective items auto-correct.</div>
    </div>
    <div class="col-12">
        <label class="form-label small" for="edit_description">Description</label>
        <div class="wordpad-shell" data-wordpad data-wordpad-name="description">
            <div class="wordpad-toolbar d-none" role="toolbar" aria-label="Formatting toolbar">
                <select class="form-select form-select-sm wordpad-select" data-wordpad-block aria-label="Text style">
                    <option value="<p>">Normal text</option>
                    <option value="<h3>">Heading</option>
                    <option value="<blockquote>">Quote</option>
                </select>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="bold" title="Bold"<?=
app_tooltip_attr('Makes selected text bold in the description.') ?>><i class="fa-solid fa-bold"></i></button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="italic" title="Italic"<?=
app_tooltip_attr('Italicizes selected text for emphasis.') ?>><i class="fa-solid fa-italic"></i></button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="underline"
title="Underline"<?= app_tooltip_attr('Underlines selected text.') ?>><i class="fa-solid fa-underline"></i></button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="insertUnorderedList"
title="Bulleted list"<?= app_tooltip_attr('Starts or continues a bulleted list.') ?>><i class="fa-solid fa-list-ul"></i></button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="insertOrderedList"
title="Numbered list"<?= app_tooltip_attr('Starts or continues a numbered list.') ?>><i class="fa-solid fa-list-ol"></i></button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="justifyLeft" title="Align
left"<?= app_tooltip_attr('Aligns paragraph text to the left.') ?>><i class="fa-solid fa-align-left"></i></button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="justifyCenter" title="Align
center"<?= app_tooltip_attr('Centers paragraph text.') ?>><i class="fa-solid fa-align-center"></i></button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="justifyRight" title="Align
right"<?= app_tooltip_attr('Aligns paragraph text to the right.') ?>><i class="fa-solid fa-align-right"></i></button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="createLink" title="Insert
link"<?= app_tooltip_attr('Turns the selection into a hyperlink.') ?>><i class="fa-solid fa-link"></i></button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="unlink" title="Remove link"
<?= app_tooltip_attr('Removes the link from the selected text without deleting the text.') ?>><i class="fa-solid fa-link-slash"></i>
</button>
                <button type="button" class="btn btn-outline-secondary btn-sm" data-command="removeFormat" title="Clear
formatting"<?= app_tooltip_attr('Strips bold, italics, and other formatting from the selection.') ?>><i class="fa-solid fa-eraser"></i>
</button>
            </div>
            <div class="wordpad-editor form-control d-none" contenteditable="true" data-placeholder="Instructions and
expectations for students."></div>
            <textarea name="description" class="form-control" id="edit_description" rows="5" placeholder="Plain text if
the rich editor is unavailable"></textarea>
        </div>
    </div>

```

```
</div>
</div>
<div class="d-flex justify-content-end gap-2 mt-4">
    <button type="button" class="btn btn-outline-secondary rounded-pill" data-bs-dismiss="modal"><?=  
app_tooltip_attr('Closes without saving edits to this assessment.') ?>>Cancel</button>  
    <button type="submit" class="btn btn-primary rounded-pill px-4"><? = app_tooltip_attr('Saves your changes to type,  
title, points, due date, and description.') ?>>Save changes</button>  
</div>  
</form>  
</div>  
</div>  
</div>  
</div>  
  
<?php if ($missingTables === [] && $classroom): ?>  
<script>  
(function () {  
    function showToast(msg, duration) {  
        var ms = typeof duration === 'number' ? duration : 2200;  
        var toast = document.getElementById('toastMsg');  
        if (!toast) return;  
        toast.textContent = msg;  
        toast.style.opacity = '1';  
        setTimeout(function () { toast.style.opacity = '0'; }, ms);  
    }  
  
    var filterInput = document.getElementById('assessmentTableFilter');  
    var table = document.getElementById('assessmentsDataTable');  
    if (filterInput && table) {  
        filterInput.addEventListener('input', function () {  
            var q = this.value.trim().toLowerCase();  
            var weekHeaders = {};  
            table.querySelectorAll('tbody tr:not(.week-group-header)').forEach(function (row) {  
                var hay = row.getAttribute('data-filter') || '';  
                var visible = q === '' || hay.indexOf(q) !== -1;  
                row.classList.toggle('d-none', !visible);  
                if (visible) {  
                    var wk = (row.getAttribute('data-week') || '').toLowerCase();  
                    if (wk) weekHeaders[wk] = true;  
                }  
            });  
            table.querySelectorAll('tbody tr.week-group-header').forEach(function (hdr) {  
                var hdrKey = (hdr.getAttribute('data-filter') || '').toLowerCase();  
                hdr.classList.toggle('d-none', q !== '' && !weekHeaders[hdrKey]);  
            });  
        });  
    }  
  
    if (table) {  
        var sortState = { col: null, dir: 1 };  
        table.querySelector('thead th.sortable').forEach(function (th) {  
            th.addEventListener('click', function () {  
                var key = th.getAttribute('data-sort');  
                var type = th.getAttribute('data-type') || 'string';  
                if (!key) return;  
                sortState.dir = sortState.col === key ? -sortState.dir : 1;  
                sortState.col = key;  
                var tbody = table.querySelector('tbody');  
                var rows = Array.prototype.slice.call(tbody.querySelectorAll('tr:not(.week-group-header)'));  
                rows.sort(function (a, b) {  
                    var av, bv;  
                    if (key === 'type') { av = a.getAttribute('data-type') || ''; bv = b.getAttribute('data-type') || ''; }  
                    else if (key === 'title') { av = a.getAttribute('data-title') || ''; bv = b.getAttribute('data-title') || ''; }  
                    else if (key === 'points') { av = parseFloat(a.getAttribute('data-points')) || '0'; bv =  
parseFloat(b.getAttribute('data-points')) || '0'; }  
                    else if (key === 'due') { av = a.getAttribute('data-due') || ''; bv = b.getAttribute('data-due') || ''; }  
                    else { return 0; }  
                    var cmp = 0;  
                    if (type === 'number') { cmp = av - bv; }  
                    else { cmp = av < bv ? -1 : (av > bv ? 1 : 0); }  
                    return cmp * sortState.dir;  
                });  
                rows.forEach(function (r) { tbody.appendChild(r); });  
            });  
        });  
    }  
  
    var searchInput = document.getElementById('searchSubmissionInput');  
    function applySubmissionSearch() {
```



```

    var q = (searchInput && searchInput.value || '').trim().toLowerCase();
    var visiblePanel = document.querySelector('.submissions-panel:not(.d-none)');
    if (!visiblePanel) return;
    visiblePanel.querySelectorAll('.submission-card').forEach(function (card) {
        var hay = (card.getAttribute('data-student-filter') || '').toLowerCase();
        card.classList.toggle('fac-filter-hide', q !== '' && hay.indexOf(q) === -1);
    });
}

function switchAssessmentPanel(id) {
    document.querySelectorAll('.submissions-panel').forEach(function (p) {
        p.classList.toggle('d-none', p.getAttribute('data-assessment-panel') !== id);
    });
    document.querySelectorAll('[data-assessment-tab]').forEach(function (btn) {
        var on = btn.getAttribute('data-assessment-tab') === id;
        btn.classList.toggle('active', on);
        btn.setAttribute('aria-selected', on ? 'true' : 'false');
    });
    applySubmissionSearch();
    try {
        var path = window.location.pathname || '';
        history.replaceState(null, '', path + '?id=<?=(int) $classroomId ?>&submissions=' + encodeURIComponent(id));
    } catch (e) {}
}

var tabContainer = document.getElementById('assessmentTabsContainer');
if (tabContainer) {
    tabContainer.querySelectorAll('[data-assessment-tab]').forEach(function (btn) {
        btn.addEventListener('click', function () {
            switchAssessmentPanel(btn.getAttribute('data-assessment-tab'));
        });
    });
}

if (searchInput) {
    searchInput.addEventListener('input', applySubmissionSearch);
}

document.querySelectorAll('.js-submission-form').forEach(function (form) {
    var ta = form.querySelector('.js-feedback-input');
    var countEl = form.querySelector('.js-char-count-num');
    function updateCount() {
        if (!ta || !countEl) return;
        countEl.textContent = String(ta.value.length);
    }
    if (ta) {
        ta.addEventListener('input', updateCount);
        updateCount();
    }
    var btnCancel = form.querySelector('.js-cancel-edit');
    if (btnCancel) {
        btnCancel.addEventListener('click', function () {
            var os = form.getAttribute('data-original-score') || '';
            var of = form.getAttribute('data-original-feedback') || '';
            var scoreIn = form.querySelector('.js-score-input');
            if (scoreIn) scoreIn.value = os;
            if (ta) ta.value = of;
            updateCount();
        });
    }
});

function buildQuestionRow(q, index) {
    var wrap = document.createElement('div');
    wrap.className = 'border rounded-3 p-2 mb-2';
    var optionsText = Array.isArray(q.options) ? q.options.join("\n") : '';
    wrap.innerHTML =
        '<div class="d-flex justify-content-between align-items-center mb-2">' +
        '<strong class="small">Question ' + (index + 1) + '</strong>' +
        '<button type="button" class="btn btn-sm btn-outline-danger js-q-remove">Remove</button>' +
        '</div>' +
        '<div class="row g-2">' +
        '<div class="col-md-4"><label class="form-label small">Type</label><select class="form-select form-select-sm js-q-type">' +
        '<option value="multiple_choice">Multiple Choice</option>' +
        '<option value="true_false">True/False</option>' +
        '<option value="essay">Essay</option>' +
        '<option value="problem_solving">Problem Solving</option>' +
        '</select></div>' +
        '<div class="col-md-2"><label class="form-label small">Points</label><input type="number" class="form-control form-control-sm' +
        'js-q-points" min="0.1" step="0.01" value="' + (q.points || 1) + '"></div>' +

```

```

        '<div class="col-md-3"><label class="form-label small">Word limit</label><input type="number" class="form-control form-control-sm js-q-word" min="0" step="1" value="' + (q.word_limit || '') + '"></div>' +
        '<div class="col-md-3"><label class="form-label small">Char limit</label><input type="number" class="form-control form-control-sm js-q-char" min="0" step="1" value="' + (q.char_limit || '') + '"></div>' +
        '<div class="col-12"><label class="form-label small">Question text</label><textarea class="form-control form-control-sm js-q-text" rows="2"></textarea></div>' +
        '<div class="col-12 js-q-options-wrap"><label class="form-label small">Options (one per line; 2-6)</label><textarea class="form-control form-control-sm js-q-options" rows="3"></textarea></div>' +
        '<div class="col-12"><label class="form-label small">Answer key</label><input type="text" class="form-control form-control-sm js-q-key" placeholder="e.g. B, true, 3.14, x=5"></div>' +
        '<div class="col-12"><div class="form-check"><input class="form-check-input js-q-steps" type="checkbox" id="q-steps-' + index + '"><label class="form-check-label small" for="q-steps-' + index + '">Allow optional step-by-step answer</label></div></div>' +
        '</div>';
    wrap.querySelector('.js-q-type').value = q.question_type || 'essay';
    wrap.querySelector('.js-q-text').value = q.question_text || '';
    wrap.querySelector('.js-q-key').value = q.answer_key || '';
    wrap.querySelector('.js-q-options').value = optionsText;
    wrap.querySelector('.js-q-steps').checked = !!q.allow_steps;
    return wrap;
}

function initBuilder(builderId, addBtnId, hiddenInputId, pointsInputSelector, assessmentTypeSelector, totalItemsSelector) {
    var builder = document.getElementById(builderId);
    var addBtn = document.getElementById(addBtnId);
    var hidden = document.getElementById(hiddenInputId);
    var form = hidden ? hidden.closest('form') : null;
    var pointsInput = form ? form.querySelector(pointsInputSelector) : null;
    var assessmentTypeInput = form ? form.querySelector(assessmentTypeSelector) : null;
    var totalItemsEl = document.querySelector(totalItemsSelector);
    if (!builder || !addBtn || !hidden || !form) return;

    function currentAssessmentType() {
        return assessmentTypeInput ? (assessmentTypeInput.value || 'essay') : 'essay';
    }

    function applyTypeLock(row) {
        var forced = currentAssessmentType();
        var typeSelect = row.querySelector('.js-q-type');
        if (!typeSelect) return;
        typeSelect.value = forced;
        typeSelect.disabled = true;
    }

    function collect() {
        var rows = [];
        var sum = 0;
        builder.querySelectorAll('.border.rounded-3').forEach(function (row) {
            var type = currentAssessmentType();
            var points = parseFloat(row.querySelector('.js-q-points').value || '0');
            var questionText = (row.querySelector('.js-q-text').value || '').trim();
            var options = (row.querySelector('.js-q-options').value || '').split(/\r?\n/).map(function (v) { return v.trim(); });
            var answerKey = (row.querySelector('.js-q-key').value || '').trim();
            var wordLimit = (row.querySelector('.js-q-word').value || '').trim();
            var charLimit = (row.querySelector('.js-q-char').value || '').trim();
            var allowSteps = row.querySelector('.js-q-steps').checked ? 1 : 0;
            if (!questionText || points <= 0) return;
            rows.push({
                question_type: type,
                question_text: questionText,
                points: points,
                options: options,
                answer_key: answerKey,
                word_limit: wordLimit === '' ? null : parseInt(wordLimit, 10),
                char_limit: charLimit === '' ? null : parseInt(charLimit, 10),
                allow_steps: allowSteps
            });
            sum += points;
        });
        hidden.value = JSON.stringify(rows);
        if (totalItemsEl) {
            totalItemsEl.textContent = String(rows.length);
        }
        if (pointsInput) {
            pointsInput.value = sum > 0 ? (Math.round(sum * 100) / 100).toString() : '';
            pointsInput.readOnly = true;
        }
        return rows;
    }
}

```

```

function renumber() {
    builder.querySelectorAll('.border.rounded-3').forEach(function (row, idx) {
        var label = row.querySelector('strong');
        if (label) label.textContent = 'Question ' + (idx + 1);
    });
}

function attachRowEvents(row) {
    applyTypeLock(row);
    row.addEventListener('input', collect);
    row.querySelector('.js-q-type').addEventListener('change', function () {
        var t = currentAssessmentType();
        var opts = row.querySelector('.js-q-options-wrap');
        var key = row.querySelector('.js-q-key');
        var steps = row.querySelector('.js-q-steps');
        var word = row.querySelector('.js-q-word');
        var chr = row.querySelector('.js-q-char');
        if (opts) opts.style.display = t === 'multiple_choice' ? '' : 'none';
        if (key) key.placeholder = t === 'true_false' ? 'true or false' : (t === 'essay' ? 'No answer key needed for essay' : 'e.g.
B, 3.14, x=5');
        if (steps) {
            var canSteps = t === 'problem_solving';
            steps.checked = canSteps ? steps.checked : false;
            steps.disabled = !canSteps;
        }
        if (word) word.disabled = t !== 'essay';
        if (chr) chr.disabled = t !== 'essay';
        collect();
    });
    row.querySelector('.js-q-remove').addEventListener('click', function () {
        row.remove();
        renumber();
        collect();
    });
    row.querySelector('.js-q-type').dispatchEvent(new Event('change'));
}

addBtn.addEventListener('click', function () {
    var row = buildQuestionRow({ question_type: currentAssessmentType(), points: 1, options: [] },
builder.querySelectorAll('.border.rounded-3').length);
    builder.appendChild(row);
    attachRowEvents(row);
    renumber();
    collect();
});

if (assessmentTypeInput) {
    assessmentTypeInput.addEventListener('change', function () {
        builder.querySelectorAll('.border.rounded-3').forEach(function (row) {
            applyTypeLock(row);
            row.querySelector('.js-q-type').dispatchEvent(new Event('change'));
        });
        collect();
    });
}

form.addEventListener('submit', function (event) {
    var rows = collect();
    if (!rows.length) {
        event.preventDefault();
        showToast('Add at least one question.', 2800);
    }
});

return {
    setRows: function (rows) {
        builder.innerHTML = '';
        (rows || []).forEach(function (r, idx) {
            var row = buildQuestionRow(r, idx);
            builder.appendChild(row);
            attachRowEvents(row);
        });
        if (!rows || !rows.length) {
            addBtn.click();
        } else {
            renumber();
            collect();
        }
    },
    appendRows: function (rows) {

```

```

        var existing = collect();
        var merged = existing.concat(rows || []);
        builder.innerHTML = '';
        merged.forEach(function (r, idx) {
            var row = buildQuestionRow(r, idx);
            builder.appendChild(row);
            attachRowEvents(row);
        });
        renumber();
        collect();
    },
    collect: collect
};

}

var addBuilderApi = initBuilder('addQuestionBuilder', 'addQuestionRowBtn', 'add_questions_json', 'input[name="total_points"]',
'select[name="assessment_type"]', '#addTotalItems');
var editBuilderApi = initBuilder('editQuestionBuilder', 'editQuestionRowBtn', 'edit_questions_json', 'input[name="total_points"]',
'#edit_assessment_type', '#editTotalItems');
if (addBuilderApi) {
    addBuilderApi.setRows([]);
}

function bindAutoGenerate(panelId, generateBtnId, moreBtnId, statusId, builderApi, formId, countId, topicId, weekId, useMaterialsId) {
    var panel = document.getElementById(panelId);
    var generateBtn = document.getElementById(generateBtnId);
    var moreBtn = document.getElementById(moreBtnId);
    var statusEl = document.getElementById(statusId);
    var form = document.getElementById(formId);
    if (!panel || !generateBtn || !builderApi || !form) {
        return;
    }

    function setStatus(text, isError) {
        if (!statusEl) return;
        statusEl.textContent = text;
        statusEl.classList.toggle('text-danger', !!isError);
        statusEl.classList.toggle('text-muted', !isError);
        statusEl.classList.toggle('d-none', !text);
    }

    function readDescription(formEl) {
        var ta = formEl.querySelector('textarea[name="description"]');
        if (!ta) return '';
        var shell = ta.closest('.wordpad-shell');
        var ed = shell && shell.querySelector('.wordpad-editor');
        if (ed && !ed.classList.contains('d-none')) {
            return (ed.innerText || ed.textContent || '').trim();
        }
        return (ta.value || '').trim();
    }

    async function requestQuestions(append) {
        var classroomId = parseInt(panel.dataset.classroomId || '0', 10);
        var api = panel.dataset.api || '';
        if (!classroomId || !api) {
            setStatus('Missing classroom context.', true);
            return;
        }

        var typeSelect = form.querySelector('select[name="assessment_type"]');
        var titleInput = form.querySelector('input[name="title"]');
        var countInput = document.getElementById(countId);
        var topicInput = document.getElementById(topicId);
        var weekSelect = weekId ? document.getElementById(weekId) : null;
        var useMaterialsInput = document.getElementById(useMaterialsId);

        var assessmentType = typeSelect ? typeSelect.value : 'essay';
        var title = titleInput ? (titleInput.value || '').trim() : '';
        var description = readDescription(form);
        var count = countInput ? parseInt(countInput.value || '5', 10) : 5;
        if (count < 1) count = 1;
        if (count > 20) count = 20;
        var topic = topicInput ? (topicInput.value || '').trim() : '';
        var week = weekSelect ? (weekSelect.value || '').trim() : '';
        var useMaterials = useMaterialsInput ? (useMaterialsInput.checked && week !== '') : false;

        if (useMaterialsInput && useMaterialsInput.checked && week === '') {
            setStatus('Select a week to generate questions from posted materials and attachments.', true);
            return;
        }
    }
}

```

```

}

var existing = append ? builderApi.collect() : [];
var exclude = existing.map(function (q) { return q.question_text || ''; });

generateBtn.disabled = true;
if (moreBtn) moreBtn.disabled = true;
setStatus(append ? 'Generating more questions...' : 'Generating questions...', false);

try {
  var res = await fetch(api, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'same-origin',
    body: JSON.stringify({
      classroom_id: classroomId,
      assessment_type: assessmentType,
      title: title,
      description: description,
      count: count,
      topic: topic,
      credited_week: week,
      use_materials: useMaterials,
      exclude: exclude
    })
  });
  var data = await res.json();
  if (!res.ok || !data.ok) {
    throw new Error(data.error || 'Could not generate questions.');
```

}

```

  var incoming = Array.isArray(data.questions) ? data.questions : [];
  if (!incoming.length) {
    throw new Error('No questions were generated. Try adding a title or selecting a week with materials.');
```

}

```

  var mapped = incoming.map(function (q) {
    return {
      question_type: q.question_type || assessmentType,
      question_text: q.question_text || '',
      points: q.points || 1,
      options: Array.isArray(q.options) ? q.options : [],
      answer_key: q.answer_key || '',
      word_limit: q.word_limit,
      char_limit: q.char_limit,
      allow_steps: q.allow_steps ? 1 : 0
    };
  });

  if (append && existing.length) {
    builderApi.appendRows(mapped);
  } else {
    builderApi.setRows(mapped);
  }

  if (moreBtn) {
    moreBtn.classList.remove('d-none');
  }

  var sourceNote = data.source === 'ai'
    ? 'AI-assisted from week materials'
    : (data.source === 'materials' ? 'Built from week materials' : 'template-based');
  var materialNote = '';
  if (data.from_week_materials) {
    materialNote = ' (' + (data.material_count || 0) + ' material(s)';
    if ((data.attachment_count || 0) > 0) {
      materialNote += ', ' + data.attachment_count + ' attachment(s)';
    }
    materialNote += ')';
  }
  setStatus('Added ' + mapped.length + ' question(s) (' + sourceNote + materialNote + '). Review and edit before saving.',
false);
  showToast('Generated ' + mapped.length + ' question(s).');
```

}

```

  catch (err) {
    setStatus(err.message || 'Something went wrong.', true);
  } finally {
    generateBtn.disabled = false;
    if (moreBtn) moreBtn.disabled = false;
  }
}
}

```

```

generateBtn.addEventListener('click', function () { requestQuestions(false); });
if (moreBtn) {
    moreBtn.addEventListener('click', function () { requestQuestions(true); });
}
}

bindAutoGenerate(
    'addAutoGeneratePanel',
    'addAutoGenerateBtn',
    'addAutoGenerateMoreBtn',
    'addAutoGenerateStatus',
    addBuilderApi,
    'formAddAssessment',
    'add_auto_count',
    'add_auto_topic',
    'add_auto_week',
    'add_auto_use_materials'
);

(function bindWeekMaterialsPreview() {
    var panel = document.getElementById('addAutoGeneratePanel');
    var weekSelect = document.getElementById('add_auto_week');
    var useMaterialsInput = document.getElementById('add_auto_use_materials');
    var previewEl = document.getElementById('add_auto_week_preview');
    if (!panel || !weekSelect || !previewEl) {
        return;
    }

    function syncPreview() {
        var week = (weekSelect.value || '').trim();
        var useMaterials = useMaterialsInput ? useMaterialsInput.checked : true;
        if (!useMaterials || week === '') {
            previewEl.classList.add('d-none');
            previewEl.textContent = '';
            return;
        }
        previewEl.textContent = 'Questions will be generated from materials and attachments posted for ' + week + '.';
        previewEl.classList.remove('d-none');
    }

    weekSelect.addEventListener('change', syncPreview);
    if (useMaterialsInput) {
        useMaterialsInput.addEventListener('change', syncPreview);
    }
    syncPreview();
})();

function formatTimerPreview(hours, minutes) {
    var h = Math.max(0, parseInt(hours, 10) || 0);
    var m = Math.max(0, parseInt(minutes, 10) || 0);
    if (m > 59) {
        h += Math.floor(m / 60);
        m = m % 60;
    }
    h = Math.min(12, h);
    var totalSecs = (h * 3600) + (m * 60);
    var hh = String(Math.floor(totalSecs / 3600)).padStart(2, '0');
    var mm = String(Math.floor((totalSecs % 3600) / 60)).padStart(2, '0');
    var ss = String(totalSecs % 60).padStart(2, '0');
    return hh + ':' + mm + ':' + ss;
}

function bindTimerSettings(enabledId, fieldsId, hoursId, minutesId, previewId) {
    var enabled = document.getElementById(enabledId);
    var fields = document.getElementById(fieldsId);
    var hours = document.getElementById(hoursId);
    var minutes = document.getElementById(minutesId);
    var preview = document.getElementById(previewId);
    if (!enabled || !fields || !hours || !minutes) {
        return;
    }
    var sync = function () {
        fields.style.display = enabled.checked ? '' : 'none';
        if (preview) {
            preview.textContent = formatTimerPreview(hours.value, minutes.value);
        }
    };
    enabled.addEventListener('change', sync);
    hours.addEventListener('input', sync);
}

```

```

        minutes.addEventListener('input', sync);
        sync();
    }

    bindTimerSettings('add_timer_enabled', 'addTimerFields', 'add_timer_hours', 'add_timer_minutes', 'addTimerPreview');
    bindTimerSettings('edit_timer_enabled', 'editTimerFields', 'edit_timer_hours', 'edit_timer_minutes', 'editTimerPreview');

    var editModal = document.getElementById('modalEditAssessment');
    var assignAssessmentModal = document.getElementById('assignAssessmentModal');
    if (assignAssessmentModal) {
        assignAssessmentModal.addEventListener('show.bs.modal', function (event) {
            var trigger = event.relatedTarget;
            if (!trigger) {
                return;
            }
            var assessmentId = trigger.getAttribute('data-assessment-id') || '';
            var assessmentTitle = trigger.getAttribute('data-assessment-title') || 'this assessment';
            var idField = document.getElementById('assign-assessment-id');
            var titleEl = document.getElementById('assign-assessment-title');
            var selectEl = document.getElementById('assign-target-assessment-classroom');
            if (idField) {
                idField.value = assessmentId;
            }
            if (titleEl) {
                titleEl.textContent = assessmentTitle;
            }
            if (selectEl) {
                selectEl.selectedIndex = 0;
            }
        });
    }
    if (editModal) {
        editModal.addEventListener('show.bs.modal', function (ev) {
            var btn = ev.relatedTarget;
            if (!btn || !btn.getAttribute) return;
            document.getElementById('edit_assessment_id').value = btn.getAttribute('data-id') || '';
            document.getElementById('edit_title').value = btn.getAttribute('data-title') || '';
            document.getElementById('edit_points').value = btn.getAttribute('data-points') || '';
            document.getElementById('edit_due').value = btn.getAttribute('data-due') || '';
            var editWeekSel = document.getElementById('edit_credited_week');
            if (editWeekSel) {
                editWeekSel.value = btn.getAttribute('data-credited-week') || '';
            }
            var editTimerEnabled = document.getElementById('edit_timer_enabled');
            var editTimerHours = document.getElementById('edit_timer_hours');
            var editTimerMinutes = document.getElementById('edit_timer_minutes');
            if (editTimerEnabled && editTimerHours && editTimerMinutes) {
                var limitMins = parseInt(btn.getAttribute('data-time-limit') || '0', 10) || 0;
                if (limitMins > 0) {
                    editTimerEnabled.checked = true;
                    editTimerHours.value = String(Math.floor(limitMins / 60));
                    editTimerMinutes.value = String(limitMins % 60);
                } else {
                    editTimerEnabled.checked = false;
                    editTimerHours.value = '0';
                    editTimerMinutes.value = '30';
                }
                editTimerEnabled.dispatchEvent(new Event('change'));
            }
            var editDescTa = document.getElementById('edit_description');
            if (editDescTa) {
                editDescTa.value = btn.getAttribute('data-description') || '';
                var shell = editDescTa.closest('.wordpad-shell');
                var ed = shell && shell.querySelector('.wordpad-editor');
                if (ed) {
                    var v = (editDescTa.value || '').trim();
                    ed.innerHTML = v !== '' ? editDescTa.value : '';
                }
            }
            var t = (btn.getAttribute('data-type') || 'essay').toLowerCase();
            var sel = document.getElementById('edit_assessment_type');
            if (sel) {
                var allowed = ['multiple_choice', 'true_false', 'essay', 'problem_solving'];
                sel.value = allowed.indexOf(t) !== -1 ? t : 'essay';
                sel.dispatchEvent(new Event('change'));
            }
            var qRaw = btn.getAttribute('data-questions') || '[]';
            if (editBuilderApi) {
                try {
                    editBuilderApi.setRows(JSON.parse(qRaw));
                }
            }
        });
    }

```

```

        } catch (e) {
            editBuilderApi.setRows([]);
        }
    }
});
}
})();

document.querySelectorAll('[data-wordpad]').forEach(function (shell) {
    var fieldName = shell.getAttribute('data-wordpad-name') || 'body';
    var textarea = shell.querySelector('textarea[name="' + fieldName + '"');
    var editor = shell.querySelector('.wordpad-editor');
    var toolbar = shell.querySelector('.wordpad-toolbar');
    var blockSelect = shell.querySelector('[data-wordpad-block]');
    var form = shell.closest('form');

    if (!textarea || !editor || !toolbar || !form) {
        return;
    }

    toolbar.classList.remove('d-none');
    editor.classList.remove('d-none');
    editor.innerHTML = textarea.value.trim() !== '' ? textarea.value : '';
    textarea.classList.add('d-none');

    var syncEditor = function () {
        var html = editor.innerHTML
            .replace(/<div><br><\div>/gi, '')
            .replace(/&nbsp;/gi, ' ')
            .trim();
        textarea.value = html;
    };

    var runCommand = function (command, value) {
        editor.focus();
        document.execCommand('styleWithCSS', false, false);
        document.execCommand(command, false, value);
        syncEditor();
    };

    toolbar.addEventListener('click', function (event) {
        var btnEl = event.target.closest('button[data-command]');
        if (!btnEl) {
            return;
        }

        event.preventDefault();
        var command = btnEl.getAttribute('data-command') || '';

        if (command === 'createLink') {
            var url = window.prompt('Enter link URL', 'https://');
            if (url) {
                runCommand(command, url);
            }
            return;
        }

        runCommand(command);
    });

    if (blockSelect) {
        blockSelect.addEventListener('change', function (event) {
            var value = event.target.value || '<p>';
            runCommand('formatBlock', value);
        });
    }

    editor.addEventListener('input', syncEditor);
    editor.addEventListener('blur', syncEditor);
    form.addEventListener('submit', syncEditor);
});
</script>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```



```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'program_chair', 'dean', 'gened']);

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}
if ($facultyId < 1 && in_array($_SESSION['role'] ?? '', ['program_chair', 'dean', 'gened'], true)) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    if ($facultyId > 0) {
        $_SESSION['faculty_id'] = $facultyId;
    }
}
if ($facultyId < 1) {
    exit('Faculty profile not linked to this account. Ask your dean to create/link your faculty profile.');
```

```

}

$classroomId = (int) ($_GET['id'] ?? $_POST['classroom_id'] ?? 0);
$attendanceDateInput = trim((string) ($_GET['attendance_date'] ?? $_POST['attendance_date'] ?? ''));
$attendanceDateExplicit = $attendanceDateInput !== '';
$attendanceDate = $attendanceDateInput !== '' ? $attendanceDateInput : date('Y-m-d');
$printMode = isset($_GET['print']) && (string) $_GET['print'] === '1';

$requiredTables = [
    'online_classrooms',
    'classroom_enrollments',
    'classroom_students',
    'classroom_attendance_sessions',
    'classroom_attendance_records',
];
$missingTables = array_values(array_filter(
    $requiredTables,
    static fn (string $table): bool => !db_table_exists($table)
));
$hasPresenceColumns = db_column_exists('users', 'last_login_at') && db_column_exists('users', 'last_seen_at');
$hasLogoutColumn = db_column_exists('users', 'last_logout_at');
$hasAttendanceLogoutColumn = db_column_exists('classroom_attendance_records', 'evidence_logout_at');

$classroom = null;
if ($classroomId > 0 && $missingTables === []) {
    $st = db()->prepare(
        'SELECT oc.*, s.day_of_week, s.start_time, s.end_time, s.semester, s.school_year, c.course_code, c.course_name
        FROM online_classrooms oc
        INNER JOIN schedules s ON s.id = oc.schedule_id
        INNER JOIN courses c ON c.id = oc.course_id
        WHERE oc.id = ? AND oc.faculty_id = ?
        LIMIT 1'
    );
    $st->execute([$classroomId, $facultyId]);
    $classroom = $st->fetch() ?: null;
}

if ($missingTables === [] && !$classroom) {
    http_response_code(404);
    exit('Classroom not found or you do not have access to it.');
```

```

}

$parseDate = static function (string $value): ?string {
    $dt = DateTime::createFromFormat('Y-m-d', $value);
    if (!$dt || $dt->format('Y-m-d') !== $value) {
        return null;
    }
    return $value;
};
$attendanceDate = $parseDate($attendanceDate) ?? date('Y-m-d');
$normalizeDayToken = static function (string $value): string {
    $lettersOnly = preg_replace('/[^a-z]/i', '', trim($value)) ?? '';
    return strtolower(substr($lettersOnly, 0, 3));
};
$formatTime12h = static function (?string $time): string {
    $raw = substr((string) $time, 0, 5);
    $dt = DateTime::createFromFormat('H:i', $raw);
    return $dt ? $dt->format('g:i A') : $raw;
}

```

```

});
$formatDateTime12h = static function (?string $dateTime): string {
    $raw = trim((string) $dateTime);
    if ($raw === '') {
        return '-';
    }
    $ts = strtotime($raw);
    return $ts === false ? $raw : date('M j, Y g:i A', $ts);
};

if ($_SERVER['REQUEST_METHOD'] === 'POST' && $missingTables === [] && $classroom && $hasPresenceColumns) {
    $action = (string) ($_POST['action'] ?? '');

    try {
        if ($action === 'run_auto_attendance') {
            $scheduleDays = parse_day_set((string) $classroom['day_of_week']);
            $dayName = date('l', strtotime($attendanceDate));
            $scheduleDayTokens = [];
            foreach ($scheduleDays as $scheduledDay) {
                $token = $normalizeDayToken((string) $scheduledDay);
                if ($token !== '') {
                    $scheduleDayTokens[$token] = true;
                }
            }
            if ($scheduleDayTokens === []) {
                throw new RuntimeException('Class schedule days are not configured yet for this classroom.');
            }
            $selectedToken = $normalizeDayToken($dayName);
            $isOffScheduleDate = !isset($scheduleDayTokens[$selectedToken]);

            $sessionStart = $attendanceDate . ' ' . substr((string) $classroom['start_time'], 0, 8);
            $sessionEnd = $attendanceDate . ' ' . substr((string) $classroom['end_time'], 0, 8);
            $now = date('Y-m-d H:i:s');
            $isWithinWindow = $now >= $sessionStart && $now <= $sessionEnd;
            $onlineRecentThreshold = date('Y-m-d H:i:s', time() - 10 * 60);

            db()->beginTransaction();
            try {
                db()->prepare(
source)
                    'INSERT INTO classroom_attendance_sessions (classroom_id, faculty_id, attendance_date, session_start_at, session_end_at,

                    VALUES (?, ?, ?, ?, ?, ?)
                    ON DUPLICATE KEY UPDATE
                        session_start_at = VALUES(session_start_at),
                        session_end_at = VALUES(session_end_at),
                        source = VALUES(source),
                        updated_at = CURRENT_TIMESTAMP'
                )->execute([$classroomId, $facultyId, $attendanceDate, $sessionStart, $sessionEnd, 'auto_login_online']);

                $sidStmt = db()->prepare('SELECT id FROM classroom_attendance_sessions WHERE classroom_id = ? AND attendance_date = ? LIMIT
1');
                $sidStmt->execute([$classroomId, $attendanceDate]);
                $sessionId = (int) ($sidStmt->fetchColumn() ?? 0);
                if ($sessionId < 1) {
                    throw new RuntimeException('Unable to initialize attendance session.');
                }

                $studentsSql = $hasLogoutColumn
                    ? 'SELECT cs.id AS student_id, u.last_login_at, u.last_seen_at, u.last_logout_at
                        FROM classroom_enrollments ce
                        INNER JOIN classroom_students cs ON cs.id = ce.student_id
                        LEFT JOIN users u ON u.id = cs.user_id
                        WHERE ce.classroom_id = ?'
                    : 'SELECT cs.id AS student_id, u.last_login_at, u.last_seen_at
                        FROM classroom_enrollments ce
                        INNER JOIN classroom_students cs ON cs.id = ce.student_id
                        LEFT JOIN users u ON u.id = cs.user_id
                        WHERE ce.classroom_id = ?';
                $studentsStmt = db()->prepare($studentsSql);
                $studentsStmt->execute([$classroomId]);
                $students = $studentsStmt->fetchAll();

                $upsert = $hasAttendanceLogoutColumn
                    ? db()->prepare(
notes)
                        'INSERT INTO classroom_attendance_records
                        (session_id, student_id, status, source, evidence_login_at, evidence_seen_at, evidence_logout_at, checked_at,

                        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
                        ON DUPLICATE KEY UPDATE
                            status = VALUES(status),

```

```

        source = VALUES(source),
        evidence_login_at = VALUES(evidence_login_at),
        evidence_seen_at = VALUES(evidence_seen_at),
        evidence_logout_at = VALUES(evidence_logout_at),
        checked_at = VALUES(checked_at),
        notes = VALUES(notes),
        updated_at = CURRENT_TIMESTAMP'
    )
    : db()->prepare(
        'INSERT INTO classroom_attendance_records
        (session_id, student_id, status, source, evidence_login_at, evidence_seen_at, checked_at, notes)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        ON DUPLICATE KEY UPDATE
            status = VALUES(status),
            source = VALUES(source),
            evidence_login_at = VALUES(evidence_login_at),
            evidence_seen_at = VALUES(evidence_seen_at),
            checked_at = VALUES(checked_at),
            notes = VALUES(notes),
            updated_at = CURRENT_TIMESTAMP'
    );

foreach ($students as $student) {
    $loginAt = $student['last_login_at'] !== null ? (string) $student['last_login_at'] : null;
    $seenAt = $student['last_seen_at'] !== null ? (string) $student['last_seen_at'] : null;
    $logoutAt = $hasLogoutColumn && ($student['last_logout_at'] ?? null) !== null ? (string) $student['last_logout_at'] :
null;

    $loggedDuringClass = $loginAt !== null && $loginAt >= $sessionStart && $loginAt <= $sessionEnd;
    $seenDuringClass = $seenAt !== null && $seenAt >= $sessionStart && $seenAt <= $sessionEnd;
    $currentlyOnlineInClass = $isWithinWindow && $seenAt !== null && $seenAt >= $onlineRecentThreshold;
    $isPresent = $loggedDuringClass || $seenDuringClass || $currentlyOnlineInClass;

    $note = ($isPresent
        ? 'Auto-marked present (login/online activity detected).'
        : 'Auto-marked absent (no login/online activity in class window).')
        . ($isOffScheduleDate ? ' Off-schedule date run by faculty.' : '');

    if ($hasAttendanceLogoutColumn) {
        $upsert->execute([$sessionId, (int) $student['student_id'], $isPresent ? 'present' : 'absent', 'auto', $loginAt,
$seenAt, $logoutAt, $now, $note]);
    } else {
        $upsert->execute([$sessionId, (int) $student['student_id'], $isPresent ? 'present' : 'absent', 'auto', $loginAt,
$seenAt, $now, $note]);
    }

    db()->commit();
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    throw $e;
}

$_SESSION['flash'] = $isOffScheduleDate
    ? 'Automatic attendance check completed for ' . $attendanceDate . ' (off-schedule date: ' . implode(' ', $scheduleDays) .
').'
    : 'Automatic attendance check completed for ' . $attendanceDate . '.';
} elseif ($action === 'set_manual_attendance') {
    $recordId = (int) ($_POST['record_id'] ?? 0);
    $status = (string) ($_POST['status'] ?? '');
    if (!in_array($status, ['present', 'absent'], true)) {
        throw new RuntimeException('Invalid attendance status.');
```

```

        INNER JOIN classroom_attendance_sessions s ON s.id = ar.session_id
        WHERE ar.id = ? AND s.classroom_id = ? AND s.faculty_id = ?'
    );
    $st->execute([$recordId, $classroomId, $facultyId]);
    $_SESSION['flash'] = $st->rowCount() > 0 ? 'Attendance record deleted.' : 'Attendance record not found.';
} elseif ($action === 'upload_banner' || $action === 'delete_banner') {
    $bannerFlash = faculty_classroom_process_banner_post($classroomId, $facultyId, $classroom);
    if ($bannerFlash !== null) {
        $_SESSION['flash'] = $bannerFlash;
    }
}
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}

header('Location: faculty_classroom_attendance.php?id=' . $classroomId . '&attendance_date=' . urlencode($attendanceDate));
exit;
}

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

$session = null;
$records = [];
$summary = ['present' => 0, 'absent' => 0, 'total' => 0];
$latestSessionDate = null;
$showDateHint = false;
if ($missingTables === [] && $classroom && $hasPresenceColumns) {
    $st = db()->prepare(
        'SELECT *
        FROM classroom_attendance_sessions
        WHERE classroom_id = ? AND attendance_date = ?
        LIMIT 1'
    );
    $st->execute([$classroomId, $attendanceDate]);
    $session = $st->fetch() ?: null;

    if (!$session) {
        $st = db()->prepare(
            'SELECT attendance_date
            FROM classroom_attendance_sessions
            WHERE classroom_id = ?
            ORDER BY attendance_date DESC, id DESC
            LIMIT 1'
        );
        $st->execute([$classroomId]);
        $latestSessionDate = $st->fetchColumn() ?: null;

        if (!$attendanceDateExplicit && $latestSessionDate !== null) {
            $attendanceDate = (string) $latestSessionDate;
            $st = db()->prepare(
                'SELECT *
                FROM classroom_attendance_sessions
                WHERE classroom_id = ? AND attendance_date = ?
                LIMIT 1'
            );
            $st->execute([$classroomId, $attendanceDate]);
            $session = $st->fetch() ?: null;
        } else {
            $showDateHint = $latestSessionDate !== null;
        }
    }

    if ($session) {
        $st = db()->prepare(
            'SELECT ar.*, cs.full_name, cs.student_number, u.last_logout_at AS user_last_logout_at
            FROM classroom_attendance_records ar
            INNER JOIN classroom_students cs ON cs.id = ar.student_id
            LEFT JOIN users u ON u.id = cs.user_id
            WHERE ar.session_id = ?
            ORDER BY cs.full_name ASC'
        );
        $st->execute([(int) $session['id']]);
        $records = $st->fetchAll();

        foreach ($records as $row) {
            $summary['total']++;
            if ((string) $row['status'] === 'present') {
                $summary['present']++;
            } else {

```

```

        $summary['absent']++;
    }
}
}

$pageTitle = $printMode ? 'Attendance Printout' : 'Class Attendance';
require_once __DIR__ . '/includes/header.php';
?>
<?php if ($classroom): ?>
    <?php faculty_classroom_render_banner($classroom, [
        'title' => (string) ($classroom['title']) ?? 'Classroom',
        'meta_extra' => 'Attendance',
        'allow_upload' => !$printMode,
        'form_id' => 'facultyAttendanceBanner',
    ]); ?>
<?php endif; ?>
<div class="d-flex flex-wrap justify-content-end align-items-center gap-2 mb-4">
    <div class="d-flex gap-2 no-print">
        <a href="faculty_classroom.php?id=<?=(int) $classroomId ?>" class="btn btn-outline-secondary btn-sm"<?=(int) $classroomId ?> app_tooltip_attr('Returns
to the class workspace and content tools.') ?>>Back to Classroom</a>
        <a href="faculty_classroom_attendance.php?id=<?=(int) $classroomId ?>&attendance_date=<?=(int) $classroomId ?>urlencode($attendanceDate) ?>&print=1"
target="_blank" rel="noopener noreferrer" class="btn btn-outline-primary btn-sm"<?=(int) $classroomId ?> app_tooltip_attr('Opens a printer-friendly login/logout
list for this date.') ?>>
            <i class="fa-solid fa-print me-1"></i>Print login/logout list
        </a>
    </div>
</div>

<?php if ($flash && !$printMode): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if ($missingTables !== []): ?>
    <div class="alert alert-warning">
        Attendance module needs a database update. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this page.
    </div>
<?php elseif (!$hasPresenceColumns): ?>
    <div class="alert alert-warning">
        User activity tracking columns are missing. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once to enable automatic
        login/online attendance detection.
    </div>
<?php else: ?>
    <?php if ($showDateHint && $latestSessionDate !== null): ?>
        <div class="alert alert-info">
            No attendance records found for <?=(int) $classroomId ?> htmlspecialchars($attendanceDate) ?>.
            Latest available attendance is on
            <a href="faculty_classroom_attendance.php?id=<?=(int) $classroomId ?>&attendance_date=<?=(int) $classroomId ?>urlencode((string)
$latestSessionDate) ?>">
                <?=(int) $classroomId ?> htmlspecialchars((string) $latestSessionDate) ?>
            </a>.
        </div>
    <?php endif; ?>

    <?php if ($printMode): ?>
        <style>
            @media print {
                .no-print { display: none !important; }
                .card { box-shadow: none !important; border: 1px solid #ddd !important; }
            }
        </style>
        <script>
            window.addEventListener('load', function () {
                window.print();
            });
        </script>
    <?php endif; ?>

    <?php if (!$printMode): ?>
        <div class="card shadow-sm mb-4">
            <div class="card-header bg-white"><strong>Run Auto Attendance</strong></div>
            <div class="card-body">
                <form method="post" class="row g-3 align-items-end">
                    <input type="hidden" name="action" value="run_auto_attendance">
                    <input type="hidden" name="classroom_id" value="<?=(int) $classroomId ?>">
                    <div class="col-md-4">
                        <label class="form-label">Attendance date</label>
                        <input type="date" name="attendance_date" class="form-control" value="<?=(int) $classroomId ?>htmlspecialchars($attendanceDate) ?>"
required>
                    </div>
                    <div class="col-md-8">
                        <button type="submit" class="btn btn-primary"<?=(int) $classroomId ?> app_tooltip_attr('Marks present/absent from login and activity logs for

```

```

the selected date. Run before manual overrides.') ?><i class="fa-solid fa-robot me-1"></i>Run automatic check</button>
    </div>
</form>
<div class="small text-muted mt-3">
    Students are marked <strong>present</strong> when login or online activity is detected within class time.
    Faculty can override each record manually below.
</div>
</div>
</div>

<div class="row g-3 mb-3">
    <div class="col-md-4">
        <div class="border rounded p-3 h-100 bg-body-tertiary">
            <div class="small text-muted text-uppercase">Total</div>
            <div class="fs-4 fw-semibold"><?= (int) $summary['total'] ?></div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="border rounded p-3 h-100 bg-body-tertiary">
            <div class="small text-muted text-uppercase">Present</div>
            <div class="fs-4 fw-semibold text-success"><?= (int) $summary['present'] ?></div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="border rounded p-3 h-100 bg-body-tertiary">
            <div class="small text-muted text-uppercase">Absent</div>
            <div class="fs-4 fw-semibold text-danger"><?= (int) $summary['absent'] ?></div>
        </div>
    </div>
</div>
</div>
<?php endif; ?>

<div class="card shadow-sm">
    <div class="card-header bg-white"><strong>Attendance Records</strong></div>
    <div class="card-body p-0">
        <div class="table-responsive">
            <table class="table table-sm mb-0">
                <thead class="table-light">
                    <tr>
                        <th>Student</th>
                        <th>Status</th>
                        <th>Login time</th>
                        <th>Logout time</th>
                        <th>Online activity</th>
                        <th>Source</th>
                        <?php if (!$printMode): ?><th class="text-end">Actions</th><?php endif; ?>
                    </tr>
                </thead>
                <tbody>
                    <?php if ($records === []): ?>
                        <tr><td colspan="<?= $printMode ? '6' : '7' ?>" class="p-3 text-muted">No attendance records yet. Run automatic
check first.</td></tr>
                    <?php endif; ?>
                    <?php foreach ($records as $row): ?>
                        <?php $logoutDisplay = (string) (($row['evidence_logout_at'] ?? '') != '' ? $row['evidence_logout_at'] :
($row['user_last_logout_at'] ?? '')); ?>
                        <tr>
                            <td>
                                <strong><?= htmlspecialchars((string) $row['full_name']) ?></strong><br>
                                <span class="small text-muted"><?= htmlspecialchars((string) $row['student_number']) ?></span>
                            </td>
                            <td>
                                <span class="badge <?= (string) $row['status'] === 'present' ? 'bg-success' : 'bg-danger' ?>">
                                    <?= htmlspecialchars(strtoupper((string) $row['status'])) ?>
                                </span>
                            </td>
                            <td class="small"><?= htmlspecialchars($formatDateTime12h((string) ($row['evidence_login_at'] ?? null))) ?></td>
                            <td class="small"><?= htmlspecialchars($formatDateTime12h($logoutDisplay != '' ? $logoutDisplay : null)) ?>
</td>
                            <td class="small"><?= htmlspecialchars($formatDateTime12h((string) ($row['evidence_seen_at'] ?? null))) ?></td>
                            <td class="small"><?= htmlspecialchars((string) $row['source']) ?></td>
                            <?php if (!$printMode): ?>
                                <td class="text-end">
                                    <div class="d-inline-flex gap-1">
                                        <form method="post">
                                            <input type="hidden" name="action" value="set_manual_attendance">
                                            <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
                                            <input type="hidden" name="attendance_date" value="<?= htmlspecialchars($attendanceDate) ?>">
                                            <input type="hidden" name="record_id" value="<?= (int) $row['id'] ?>">
                                            <input type="hidden" name="status" value="present">

```

```

                <button type="submit" class="btn btn-sm btn-outline-success">= app_tooltip_attr('Overrides this
student as present for the selected date.') ?>>Mark present</button>
            </form>
            <form method="post">
                <input type="hidden" name="action" value="set_manual_attendance">
                <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
                <input type="hidden" name="attendance_date" value="<?= htmlspecialchars($attendanceDate) ?>">
                <input type="hidden" name="record_id" value="<?= (int) $row['id'] ?>">
                <input type="hidden" name="status" value="absent">
                <button type="submit" class="btn btn-sm btn-outline-danger">= app_tooltip_attr('Overrides this
student as absent for the selected date.') ?>>Mark absent</button>
            </form>
            <form method="post" onsubmit="return confirm('Delete this attendance record?');">
                <input type="hidden" name="action" value="delete_attendance_record">
                <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
                <input type="hidden" name="attendance_date" value="<?= htmlspecialchars($attendanceDate) ?>">
                <input type="hidden" name="record_id" value="<?= (int) $row['id'] ?>">
                <button type="submit" class="btn btn-sm btn-outline-danger" title="Delete record" aria-
label="Delete attendance record">= app_tooltip_attr('Deletes this attendance row after confirmation. Use to fix mistaken auto entries.') ?
>>
                <i class="fa-solid fa-trash"></i>
            </button>
        </form>
    </div>
</td>
<?php endif; ?>
</tr>
<?php endforeach; ?>
</tbody>
</table>
</div>
</div>
</div>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: faculty_classroom_week.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'program_chair', 'dean', 'gened']);

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}
if ($facultyId < 1 && in_array($_SESSION['role'] ?? '', ['program_chair', 'dean', 'gened'], true)) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    if ($facultyId > 0) {
        $_SESSION['faculty_id'] = $facultyId;
    }
}
if ($facultyId < 1) {
    exit('Faculty profile not linked to this account. Ask your dean to create/link your faculty profile.');
```

```

$classroom = null;
if ($classroomId > 0 && $missingTables === []) {
    $st = db()->prepare(
        'SELECT oc.*, s.semester, s.school_year, c.course_code, c.course_name
        FROM online_classrooms oc
        INNER JOIN schedules s ON s.id = oc.schedule_id
        INNER JOIN courses c ON c.id = oc.course_id
        WHERE oc.id = ? AND oc.faculty_id = ?
        LIMIT 1'
    );
    $st->execute([$classroomId, $facultyId]);
    $classroom = $st->fetch() ?: null;
}

if ($missingTables === [] && !$classroom) {
    http_response_code(404);
    exit('Classroom not found or you do not have access to it.');
```

```

}

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST' && $missingTables === [] && $classroom) {
    $action = (string) ($_POST['action'] ?? '');

    try {
        if ($action === 'delete_content') {
            $contentId = (int) ($_POST['content_id'] ?? 0);
            $st = db()->prepare(
                'SELECT id, resource_url
                FROM classroom_content
                WHERE id = ? AND classroom_id = ? AND faculty_id = ?
                LIMIT 1'
            );
            $st->execute([$contentId, $classroomId, $facultyId]);
            $existing = $st->fetch();
            if (!$existing) {
                throw new RuntimeException('Content item not found.');
```

```

            }
            $resourceUrl = (string) ($existing['resource_url'] ?? '');

            $attachmentRows = [];
            if ($hasContentAttachments) {
                $st = db()->prepare(
                    'SELECT stored_name
                    FROM classroom_content_attachments
                    WHERE content_id = ?'
                );
                $st->execute([$contentId]);
                $attachmentRows = $st->fetchAll();
            }

            db()->prepare(
                'DELETE FROM classroom_content
                WHERE id = ? AND classroom_id = ? AND faculty_id = ?'
            )->execute([$contentId, $classroomId, $facultyId]);

            if (classroom_content_is_attachment($resourceUrl)) {
                $attachmentPath = classroom_content_attachment_path($resourceUrl);
                if (is_file($attachmentPath)) {
                    @unlink($attachmentPath);
                }
            }
            foreach ($attachmentRows as $attachmentRow) {
                $attachmentPath = classroom_content_attachment_storage_path((string) ($attachmentRow['stored_name'] ?? ''));
                if (is_file($attachmentPath)) {
                    @unlink($attachmentPath);
                }
            }

            $_SESSION['flash'] = 'Content removed.';
        } elseif ($action === 'assign_content') {
            $contentId = (int) ($_POST['content_id'] ?? 0);
            $targetClassroomId = (int) ($_POST['target_classroom_id'] ?? 0);
            classroom_content_copy_to_classroom($contentId, $classroomId, $targetClassroomId, $facultyId);
            $st = db()->prepare(
                'SELECT c.course_code, oc.title
                FROM online_classrooms oc
                INNER JOIN courses c ON c.id = oc.course_id

```



```

        WHERE oc.id = ?
        LIMIT 1'
    );
    $st->execute([$targetClassroomId]);
    $targetMeta = $st->fetch() ?: [];
    $targetLabel = trim((string) ($targetMeta['course_code'] ?? ''));
    if ($targetLabel === '') {
        $targetLabel = trim((string) ($targetMeta['title'] ?? 'the selected course'));
    }
    $_SESSION['flash'] = 'Content assigned to ' . $targetLabel . '. The original post stays in this class.';
} elseif ($action === 'upload_banner' || $action === 'delete_banner') {
    $bannerFlash = faculty_classroom_process_banner_post($classroomId, $facultyId, $classroom);
    if ($bannerFlash !== null) {
        $_SESSION['flash'] = $bannerFlash;
    }
}
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}

$redirectWeek = trim((string) ($_POST['current_week'] ?? $requestedWeek));
$redirectQuery = 'id=' . $classroomId;
if ($redirectWeek !== '') {
    $redirectQuery .= '&week=' . rawurlencode($redirectWeek);
}
header('Location: faculty_classroom_week.php?' . $redirectQuery);
exit;
}

$allItems = [];
$weekItems = [];
$weekGroups = [];
$contentAttachmentMap = [];
$otherClassrooms = [];
if ($missingTables === [] && $classroom) {
    $st = db()->prepare(
        'SELECT *
        FROM classroom_content
        WHERE classroom_id = ? AND faculty_id = ?
        ORDER BY created_at DESC'
    );
    $st->execute([$classroomId, $facultyId]);
    $allItems = $st->fetchAll();
    $weekGroups = classroom_content_group_by_week($allItems);

    $weekLookup = [];
    foreach ($weekGroups as $group) {
        $weekLookup[(string) $group['label']] = $group;
    }

    if ($requestedWeek === '') {
        $requestedWeek = $weekGroups !== [] ? (string) $weekGroups[0]['label'] : '';
    }
    if ($requestedWeek !== '' && isset($weekLookup[$requestedWeek])) {
        $weekItems = $weekLookup[$requestedWeek]['items'];
    } elseif ($allItems !== []) {
        http_response_code(404);
        exit('Week not found for this classroom.');
```

```

    }

    if ($hasContentAttachments) {
        $contentAttachmentMap = classroom_content_attachment_map(array_column($weekItems, 'id'));
    }

```

```

    $otherClassrooms = faculty_owned_classrooms($facultyId, $classroomId);
}

```

```

$pageTitle = $requestedWeek !== '' ? 'Week Content - ' . $requestedWeek : 'Week Content';
require_once __DIR__ . '/includes/header.php';
?>

```

```

<div class="fc-manage container-fluid px-0">
<?php if ($classroom): ?>
    <?php faculty_classroom_render_banner($classroom, [
        'title' => $requestedWeek !== '' ? $requestedWeek : (string) ($classroom['title'] ?? 'Classroom'),
        'meta_extra' => $requestedWeek !== '' ? (string) ($classroom['title'] ?? '') : '',
        'form_id' => 'facultyWeekBanner',
    ]); ?>
<?php endif; ?>
<div class="d-flex flex-wrap justify-content-end align-items-start gap-2 mb-4">

```

```

        <a href="faculty_classroom.php?id=<?= (int) $classroomId ?>" class="btn btn-outline-secondary btn-sm rounded-pill"<?=
app_tooltip_attr('Returns to the main classroom page with all weeks and posting tools.') ?><i class="fa-solid fa-arrow-left me-1"></i>Back
to class</a>
    </div>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if ($missingTables !== []): ?>
    <div class="alert alert-warning border-0 shadow-sm">
        Classroom features are not installed yet. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this page.
    </div>
<?php else: ?>
    <div class="row g-4">
        <div class="col-lg-4">
            <div class="card fc-section-card">
                <div class="card-header">
                    <h2 class="fc-section-title"><i class="fa-solid fa-calendar-days" aria-hidden="true"></i>Weeks</h2>
                    <p class="small text-muted mb-0 mt-2">Jump to another week to edit its materials.</p>
                </div>
                <div class="card-body p-4">
                    <?php if ($weekGroups === []): ?>
                        <p class="fc-placeholder-hint mb-0"><i class="fa-regular fa-folder-open me-2"></i>No week-tagged content yet.
Publish items from the main class page with a week label.</p>
                    <?php else: ?>
                        <?php foreach ($weekGroups as $group): ?>
                            <a class="fc-week-link <?= (string) $group['label'] === $requestedWeek ? 'is-active-week' : '' ?>"
href="faculty_classroom_week.php?id=<?= (int) $classroomId ?>&week=<?= rawurlencode((string) $group['label']) ?>"
                            <span><strong><?= htmlspecialchars((string) $group['label']) ?></strong></span>
                            <span class="badge text-bg-primary rounded-pill"><?= (int) $group['count'] ?></span>
                        </a>
                    <?php endforeach; ?>
                <?php endif; ?>
            </div>
        </div>
        <div class="col-lg-8">
            <div class="card fc-section-card">
                <div class="card-header">
                    <h2 class="fc-section-title"><i class="fa-solid fa-list-ul" aria-hidden="true"></i>This week's topics &
materials</h2>
                    <p class="small text-muted mb-0 mt-2">What enrolled students see for <?= htmlspecialchars($requestedWeek !== '' ?
$requestedWeek : 'this week') ?>.</p>
                </div>
                <div class="card-body p-4">
                    <?php if ($weekItems === []): ?>
                        <p class="fc-placeholder-hint mb-0"><i class="fa-regular fa-file-lines me-2"></i>No items for this week yet. Add
content from the class page or switch weeks using the list.</p>
                    <?php else: ?>
                        <?php foreach ($weekItems as $item): ?>
                            <div class="border rounded-3 p-3 mb-3 fc-content-item-card">
                                <div class="d-flex justify-content-between align-items-start gap-2">
                                    <div>
                                        <div class="fw-semibold"><?= htmlspecialchars((string) $item['title']) ?></div>
                                        <div class="small text-muted text-capitalize"><i class="fa-solid fa-tag me-1 opacity-75"></i><?=
htmlspecialchars((string) $item['content_type']) ?></div>
                                    </div>
                                    <div class="d-flex align-items-center gap-2 flex-shrink-0">
                                        <?php
                                            $editHref = 'faculty_classroom_week_edit.php?id=' . (int) $classroomId
                                                . '&content_id=' . (int) $item['id']
                                                . '&week=' . rawurlencode($requestedWeek);
                                        <?>
                                        <a href="<?= htmlspecialchars($editHref) ?>" class="btn btn-sm btn-outline-primary rounded-pill fc-
edit-popup-link" data-content-id="<?= (int) $item['id'] ?>" title="Edit content"<?= app_tooltip_attr('Opens a separate window to change this
item's title, body, or links for this week.') ?>>
                                            <i class="fa-solid fa-pen-to-square me-1"></i>Edit
                                        </a>
                                        <?php if ($otherClassrooms !== []): ?>
                                            <button type="button"
                                                class="btn btn-sm btn-outline-secondary rounded-pill"
                                                data-bs-toggle="modal"
                                                data-bs-target="#assignContentModal"
                                                data-content-id="<?= (int) $item['id'] ?>"
                                                data-content-title="<?= htmlspecialchars((string) $item['title'], ENT_QUOTES) ?>"
                                                title="Assign to another course"<?= app_tooltip_attr('Copies this material or
announcement into another class you teach. The original stays here.') ?>>
                                                <i class="fa-solid fa-share-from-square me-1"></i>Assign
                                            </button>
                                        <?php endif; ?>
                                    </div>
                                </div>
                            </div>
                        <?php endforeach; ?>
                    <?php else: ?>

```

```

        <form method="post" class="d-inline" onsubmit="return confirm('Delete this content item?');">
            <input type="hidden" name="action" value="delete_content">
            <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
            <input type="hidden" name="content_id" value="<?= (int) $item['id'] ?>">
            <input type="hidden" name="current_week" value="<?= htmlspecialchars($requestedWeek) ?>">
            <button type="submit" class="btn btn-sm btn-outline-danger rounded-pill" title="Delete content"
                <i class="fa-solid fa-trash"></i>
            </button>
        </form>
        <div class="small text-muted text-end d-none d-md-block"><?= htmlspecialchars((string)
$item['created_at']) ?></div>
    </div>
</div>
<?php if (trim((string) ($item['body'] ?? '')) !== ''): ?>
    <div class="small mt-2 classroom-content-body"><?= classroom_content_render_body((string) $item['body'])
?></div>
    <?php endif; ?>
    <?php if ($hasContentTopicSchedule && (trim((string) ($item['weeks'] ?? '')) !== '' || trim((string)
($item['days_per_topic'] ?? '')) !== '')): ?>
        <div class="small text-muted mt-2">
            <?php if (trim((string) ($item['weeks'] ?? '')) !== ''): ?>
                <span class="me-2"><i class="fa-regular fa-calendar me-1"></i>Weeks: <?=
htmlspecialchars((string) $item['weeks']) ?></span>
                <?php endif; ?>
                <?php if (trim((string) ($item['days_per_topic'] ?? '')) !== ''): ?>
                    <span><i class="fa-regular fa-clock me-1"></i>Days/topic: <?= htmlspecialchars((string)
$item['days_per_topic']) ?></span>
                    <?php endif; ?>
                </div>
            <?php endif; ?>
            <?php if (trim((string) ($item['resource_url'] ?? '')) !== ''): ?>
                <?php $resourceUrl = trim((string) $item['resource_url']); ?>
                <?php if (classroom_content_is_attachment($resourceUrl)): ?>
                    <?php
                    $legacyName = classroom_content_attachment_name($resourceUrl);
                    $legacyHref = classroom_content_resource_href((int) $item['id'], $resourceUrl);
                    $legacyIsImage = classroom_content_is_image_filename($legacyName);
                    ?>
                    <?php if ($legacyIsImage): ?>
                        <div class="mt-2 classroom-content-attachment-image">
                            <a href="<?= htmlspecialchars($legacyHref) ?>" target="_blank" rel="noopener noreferrer">
                                " loading="lazy" decoding="async">
                            </a>
                            <div class="small mt-1">
                                <a href="<?= htmlspecialchars($legacyHref) ?>"><i class="fa-solid fa-paperclip me-1">
</i><?= htmlspecialchars($legacyName) ?></a>
                            </div>
                        </div>
                    <?php else: ?>
                        <div class="small mt-2">
                            <a href="<?= htmlspecialchars($legacyHref) ?>">
                                <i class="fa-solid fa-paperclip me-1"></i><?= htmlspecialchars($legacyName) ?>
                            </a>
                        </div>
                    <?php endif; ?>
                <?php else: ?>
                    <div class="small mt-2"><a href="<?= htmlspecialchars(classroom_content_resource_href((int)
$item['id'], $resourceUrl)) ?>" target="_blank" rel="noopener noreferrer">Open resource</a></div>
                    <?php endif; ?>
                <?php endif; ?>
                <?php foreach ($contentAttachmentMap[(int) $item['id']] ?? [] as $attachment): ?>
                    <?php
                    $attachName = classroom_content_attachment_download_name((string) $attachment['original_name'], (string)
$attachment['stored_name']);
                    $attachHref = classroom_content_attachment_href((int) $attachment['id']);
                    $attachInlineHref = classroom_content_attachment_href((int) $attachment['id'], true);
                    ?>
                    <?php if (classroom_content_attachment_is_image($attachment)): ?>
                        <div class="mt-2 classroom-content-attachment-image">
                            <a href="<?= htmlspecialchars($attachHref) ?>" target="_blank" rel="noopener noreferrer">
                                " loading="lazy" decoding="async">
                            </a>
                            <div class="small mt-1">
                                <a href="<?= htmlspecialchars($attachHref) ?>"><i class="fa-solid fa-paperclip me-1"></i><?=
htmlspecialchars($attachName) ?></a>
                            </div>
                        </div>
                    <?php endif; ?>
                </div>
            </div>

```

```

                <?php else: ?>
                    <div class="small mt-2">
                        <a href="<?= htmlspecialchars($attachHref) ?>">
                            <i class="fa-solid fa-paperclip me-1"></i><?= htmlspecialchars($attachName) ?>
                        </a>
                    </div>
                <?php endif; ?>
            <?php endforeach; ?>
        </div>
    <?php endforeach; ?>
<?php endif; ?>
</div>
</div>
</div>
</div>
<?php endif; ?>
</div>

<?php if ($otherClassrooms !== []): ?>
<div class="modal fade" id="assignContentModal" tabindex="-1" aria-labelledby="assignContentModalLabel" aria-hidden="true">
    <div class="modal-dialog">
        <div class="modal-content">
            <form method="post">
                <input type="hidden" name="action" value="assign_content">
                <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
                <input type="hidden" name="content_id" id="assign-content-id" value="">
                <input type="hidden" name="current_week" value="<?= htmlspecialchars($requestedWeek) ?>">
                <div class="modal-header">
                    <h2 class="modal-title h5" id="assignContentModalLabel">Assign to another course</h2>
                    <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
                </div>
                <div class="modal-body">
                    <p class="small text-muted mb-3">Copy <strong id="assign-content-title">this item</strong> into another class you teach.
The original post stays in this course.</p>
                    <label class="form-label fw-semibold" for="assign-target-classroom">Target course</label>
                    <select id="assign-target-classroom" name="target_classroom_id" class="form-select" required>
                        <option value="">Select a course</option>
                        <?php foreach ($otherClassrooms as $other): ?>
                            <?php
                                $optionLabel = trim((string) ($other['course_code'] ?? ''));
                                if ($optionLabel !== '' && trim((string) ($other['course_name'] ?? '')) !== '') {
                                    $optionLabel .= ' - ' . trim((string) $other['course_name']);
                                } elseif (trim((string) ($other['course_name'] ?? '')) !== '') {
                                    $optionLabel = trim((string) $other['course_name']);
                                } else {
                                    $optionLabel = trim((string) ($other['title'] ?? 'Classroom'));
                                }
                                $termLabel = trim((string) ($other['semester'] ?? '')) . ' ' . (string) ($other['school_year'] ?? '');
                                if ($termLabel !== '') {
                                    $optionLabel .= ' (' . $termLabel . ')';
                                }
                            <?>
                            <option value="<?= (int) $other['id'] ?>"><?= htmlspecialchars($optionLabel) ?></option>
                        <?php endforeach; ?>
                    </select>
                    <div class="form-text mt-2">The target class must already have a syllabus uploaded before content can be assigned.</div>
                </div>
                <div class="modal-footer">
                    <button type="button" class="btn btn-outline-secondary" data-bs-dismiss="modal">Cancel</button>
                    <button type="submit" class="btn btn-primary">Assign copy</button>
                </div>
            </form>
        </div>
    </div>
</div>
<?php endif; ?>

<script>
(function () {
    document.querySelectorAll('.fc-edit-popup-link').forEach(function (link) {
        link.addEventListener('click', function (event) {
            event.preventDefault();
            var w = 920;
            var h = 780;
            var l = Math.max(0, Math.round((window.screen.width - w) / 2));
            var t = Math.max(0, Math.round((window.screen.height - h) / 2));
            var contentId = link.getAttribute('data-content-id') || '0';
            var features = 'width=' + w + ',height=' + h + ',left=' + l + ',top=' + t + ',scrollbars=yes,resizable=yes';
            window.open(link.href, 'fcWeekEdit' + contentId, features);
        });
    });
});

```

```

    });

    var assignModal = document.getElementById('assignContentModal');
    if (assignModal) {
        assignModal.addEventListener('show.bs.modal', function (event) {
            var trigger = event.relatedTarget;
            if (!trigger) {
                return;
            }
            var contentId = trigger.getAttribute('data-content-id') || '';
            var contentTitle = trigger.getAttribute('data-content-title') || 'this item';
            var idField = document.getElementById('assign-content-id');
            var titleEl = document.getElementById('assign-content-title');
            var selectEl = document.getElementById('assign-target-classroom');
            if (idField) {
                idField.value = contentId;
            }
            if (titleEl) {
                titleEl.textContent = contentTitle;
            }
            if (selectEl) {
                selectEl.selectedIndex = 0;
            }
        });
    }
})();
</script>

```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>
```

```
-----
FILE: faculty_classroom_week_edit.php
-----
```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'program_chair', 'dean', 'gened']);

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}
if ($facultyId < 1 && in_array($_SESSION['role'] ?? '', ['program_chair', 'dean', 'gened'], true)) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    if ($facultyId > 0) {
        $_SESSION['faculty_id'] = $facultyId;
    }
}
if ($facultyId < 1) {
    exit('Faculty profile not linked to this account. Ask your dean to create/link your faculty profile.');
```

```

    }

    $scheme = strtolower((string) (parse_url($url, PHP_URL_SCHEME) ?? ''));
    if (!in_array($scheme, ['http', 'https'], true)) {
        throw new RuntimeException('Only http and https URLs are allowed.');
```

```
    }
```

```
    return $url;
```

```
}
```

```
$classroom = null;
```

```
$item = null;
```

```
$error = '';
```

```
if ($classroomId > 0 && $contentId > 0 && db_table_exists('online_classrooms') && db_table_exists('classroom_content')) {
```

```
    $st = db()->prepare(
```

```
        'SELECT oc.*, s.semester, s.school_year, c.course_code, c.course_name
        FROM online_classrooms oc
        INNER JOIN schedules s ON s.id = oc.schedule_id
        INNER JOIN courses c ON c.id = oc.course_id
        WHERE oc.id = ? AND oc.faculty_id = ?
        LIMIT 1'
```

```
    );
```

```
    $st->execute([$classroomId, $facultyId]);
```

```
    $classroom = $st->fetch() ?: null;
```

```
    if ($classroom) {
```

```
        $st = db()->prepare(
```

```
            'SELECT *
            FROM classroom_content
            WHERE id = ? AND classroom_id = ? AND faculty_id = ?
            LIMIT 1'
```

```
        );
```

```
        $st->execute([$contentId, $classroomId, $facultyId]);
```

```
        $item = $st->fetch() ?: null;
```

```
    }
```

```
}
```

```
if (!$classroom || !$item) {
```

```
    http_response_code(404);
```

```
    exit('Content item not found or you do not have access to it.');
```

```
}
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
    $action = (string) ($_POST['action'] ?? '');
```

```
    try {
```

```
        if ($action !== 'update_content') {
```

```
            throw new RuntimeException('Invalid action.');
```

```
        }
```

```
        $contentType = (string) ($_POST['content_type'] ?? 'material');
```

```
        $title = trim((string) ($_POST['title'] ?? ''));
```

```
        $body = classroom_content_prepare_body((string) ($_POST['body'] ?? ''));
```

```
        $weeks = trim((string) ($_POST['weeks'] ?? ''));
```

```
        $daysPerTopic = trim((string) ($_POST['days_per_topic'] ?? ''));
```

```
        $resourceUrlRaw = trim((string) ($_POST['resource_url'] ?? ''));
```

```
        if (!in_array($contentType, ['material', 'link', 'announcement'], true)) {
```

```
            $contentType = 'material';
```

```
        }
```

```
        if ($title === '') {
```

```
            throw new RuntimeException('Content title is required.');
```

```
        }
```

```
        $existingResourceUrl = trim((string) ($item['resource_url'] ?? ''));
```

```
        $resourceUrl = $resourceUrlRaw !== '' ? faculty_week_edit_manage_url($resourceUrlRaw) : '';
```

```
        if ($resourceUrlRaw === '' && classroom_content_is_attachment($existingResourceUrl)) {
```

```
            $resourceUrl = $existingResourceUrl;
```

```
        }
```

```
        $hasAttachmentRows = false;
```

```
        if ($hasContentAttachments) {
```

```
            $st = db()->prepare('SELECT COUNT(*) FROM classroom_content_attachments WHERE content_id = ?');
```

```
            $st->execute([$contentId]);
```

```
            $hasAttachmentRows = (int) $st->fetchColumn() > 0;
```

```
        }
```

```
        if ($body === null && $resourceUrl === '' && !$hasAttachmentRows) {
```

```
            throw new RuntimeException('Add a short description, a resource URL, or at least one attachment.');
```

```

}

if ($hasContentTopicSchedule) {
    db()->prepare(
        'UPDATE classroom_content
        SET content_type = ?, title = ?, body = ?, weeks = ?, days_per_topic = ?, resource_url = ?
        WHERE id = ? AND classroom_id = ? AND faculty_id = ?'
    )->execute([
        $contentType,
        $title,
        $body !== '' ? $body : null,
        $weeks,
        $daysPerTopic,
        $resourceUrl,
        $contentId,
        $classroomId,
        $facultyId,
    ]);
} else {
    db()->prepare(
        'UPDATE classroom_content
        SET content_type = ?, title = ?, body = ?, resource_url = ?
        WHERE id = ? AND classroom_id = ? AND faculty_id = ?'
    )->execute([
        $contentType,
        $title,
        $body !== '' ? $body : null,
        $resourceUrl,
        $contentId,
        $classroomId,
        $facultyId,
    ]);
}

$redirectWeek = $requestedWeek;
if ($hasContentTopicSchedule) {
    $redirectWeek = classroom_content_week_label($weeks);
}

$openerUrl = 'faculty_classroom_week.php?id=' . $classroomId;
if ($redirectWeek !== '') {
    $openerUrl .= '&week=' . rawurlencode($redirectWeek);
}

header('Content-Type: text/html; charset=UTF-8');
echo '<!DOCTYPE html><html lang="en"><head><meta charset="UTF-8"><title>Saved</title></head><body>';
echo '<p>Saved. Closing editor...</p>';
echo '<script>';
echo 'if (window.opener && !window.opener.closed) { window.opener.location.href = ' . json_encode($openerUrl) . ' '; }';
echo 'window.close();';
echo 'setTimeout(function () { window.location.href = ' . json_encode($openerUrl) . ' '; }, 1500);';
echo '</script></body></html>';
exit;
} catch (Throwable $e) {
    $error = $e->getMessage();
}
}

$pageTitle = 'Edit content';
$bodyPageClass = 'page-fc-week-edit-popup';
$mainContainerClass = 'container-fluid px-3 py-3 app-main';
require_once __DIR__ . '/includes/header.php';
?>

<div class="fc-manage">
    <div class="d-flex flex-wrap justify-content-between align-items-start gap-2 mb-3">
        <div>
            <h1 class="h5 mb-1">Edit content</h1>
            <p class="small text-muted mb-0"><?php echo htmlspecialchars((string) ($classroom['course_code'] ?? '')) ?> . <?php echo htmlspecialchars($requestedWeek !== '' ? $requestedWeek : 'Week content') ?></p>
        </div>
        <button type="button" class="btn btn-outline-secondary btn-sm rounded-pill" onclick="window.close();" <?php echo app_tooltip_attr('Closes this editor without saving.') ?>>
            <i class="fa-solid fa-xmark me-1"></i>Close
        </button>
    </div>

    <?php if ($error !== ''): ?>
        <div class="alert alert-danger border-0 shadow-sm"><?php echo htmlspecialchars($error) ?></div>
    <?php endif: ?>

```

```

<div class="card fc-section-card">
  <div class="card-body p-4">
    <div class="fc-subsection-label mb-3"><i class="fa-solid fa-pen" aria-hidden="true"></i>Edit this item</div>
    <form method="post" class="row g-3" id="fcWeekEditForm">
      <input type="hidden" name="action" value="update_content">
      <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
      <input type="hidden" name="content_id" value="<?= (int) $contentId ?>">
      <input type="hidden" name="current_week" value="<?= htmlspecialchars($requestedWeek) ?>">
      <div class="col-md-4">
        <label class="form-label fw-semibold small">Content type</label>
        <select name="content_type" class="form-select form-select-sm">
          <option value="material" <?= (string) ($item['content_type'] ?? '') === 'material' ? 'selected' : '' ?
>>Material</option>
          <option value="link" <?= (string) ($item['content_type'] ?? '') === 'link' ? 'selected' : '' ?>>Link</option>
          <option value="announcement" <?= (string) ($item['content_type'] ?? '') === 'announcement' ? 'selected' : '' ?
>>Announcement</option>
        </select>
      </div>
      <div class="col-md-8">
        <label class="form-label fw-semibold small">Title</label>
        <input type="text" name="title" class="form-control form-control-sm" maxlength="150" required value="<?=
htmlspecialchars((string) ($item['title'] ?? '')) ?>" placeholder="Short title shown to students">
      </div>
      <?php if ($hasContentTopicSchedule): ?>
        <div class="col-md-6">
          <label class="form-label fw-semibold small">Week label</label>
          <input type="text" name="weeks" class="form-control form-control-sm" maxlength="100" value="<?=
htmlspecialchars((string) ($item['weeks'] ?? '')) ?>" placeholder="e.g. Week 3">
        </div>
        <div class="col-md-6">
          <label class="form-label fw-semibold small">Days per topic</label>
          <input type="text" name="days_per_topic" class="form-control form-control-sm" maxlength="100" value="<?=
htmlspecialchars((string) ($item['days_per_topic'] ?? '')) ?>" placeholder="e.g. 2 class days">
        </div>
      <?php endif; ?>
      <div class="col-12">
        <label class="form-label fw-semibold small">Description & notes</label>
        <div class="wordpad-shell" data-wordpad="data-wordpad-name=body">
          <div class="wordpad-toolbar d-none" role="toolbar" aria-label="Formatting toolbar">
            <select class="form-select form-select-sm wordpad-select" data-wordpad-block aria-label="Text style">
              <option value="p">Normal text</option>
              <option value="h3">Heading</option>
              <option value="blockquote">Quote</option>
            </select>
            <select class="form-select form-select-sm wordpad-select wordpad-select--font" data-wordpad-font-family aria-
label="Font">
              <option value="">Font</option>
              <option value="Arial, Helvetica, sans-serif">Arial</option>
              <option value="'Times New Roman', Times, serif">Times New Roman</option>
              <option value="Georgia, serif">Georgia</option>
              <option value="'Courier New', Courier, monospace">Courier New</option>
              <option value="Verdana, sans-serif">Verdana</option>
              <option value="Tahoma, sans-serif">Tahoma</option>
            </select>
            <select class="form-select form-select-sm wordpad-select wordpad-select--size" data-wordpad-font-size aria-
label="Font size">
              <option value="">Size</option>
              <option value="12px">Small (12)</option>
              <option value="14px">Normal (14)</option>
              <option value="16px">Medium (16)</option>
              <option value="18px">Large (18)</option>
              <option value="24px">Extra large (24)</option>
              <option value="32px">Title (32)</option>
            </select>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="bold" title="Bold"><i class="fa-
solid fa-bold"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="italic" title="Italic"><i
class="fa-solid fa-italic"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="underline" title="Underline"><i
class="fa-solid fa-underline"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="insertUnorderedList"
title="Bulleted list"><i class="fa-solid fa-list-ul"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="insertOrderedList" title="Numbered
list"><i class="fa-solid fa-list-ol"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="createLink" title="Insert link"><i
class="fa-solid fa-link"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="insertImage" title="Insert image">
<i class="fa-solid fa-image"></i></button>
            <button type="button" class="btn btn-outline-secondary btn-sm" data-command="removeFormat" title="Clear

```



```

formatting"><i class="fa-solid fa-eraser"></i></button>
    </div>
    <div class="wordpad-editor form-control d-none" contenteditable="true" data-placeholder="Instructions, context, or
notes (paste images OK)"></div>
    <textarea name="body" class="form-control form-control-sm" rows="8" placeholder="Instructions, context, or notes
(optional if you have a link or attachment)"><?> htmlspecialchars((string) ($item['body'] ?? '')) ?></textarea>
    </div>
    <div class="form-text">You can paste or insert images here; they are saved with the lesson.</div>
</div>
<div class="col-12">
    <label class="form-label fw-semibold small">Resource URL</label>
    <?php $editResourceUrl = trim((string) ($item['resource_url'] ?? '')); ?>
    <input type="url" name="resource_url" class="form-control form-control-sm" placeholder="https://... (optional)" value="<?=
classroom_content_is_attachment($editResourceUrl) ? '' : htmlspecialchars($editResourceUrl) ?>" autocomplete="url">
    <?php if (classroom_content_is_attachment($editResourceUrl)): ?>
        <div class="form-text">This item uses a legacy uploaded attachment. Leave URL blank to keep that file link.</div>
    <?php endif; ?>
</div>
<div class="col-12 pt-1 d-flex flex-wrap gap-2">
    <button type="submit" class="btn btn-primary fc-btn-primary-lg"><?> app_tooltip_attr('Saves your edits to this content
item.') ?>>
        <i class="fa-solid fa-floppy-disk me-2"></i>Save changes
    </button>
    <button type="button" class="btn btn-outline-secondary fc-btn-primary-lg" onclick="window.close();">Cancel</button>
</div>
</form>
</div>
</div>
</div>
</div>
<script>
(function () {
    function initWordpad(shell) {
        if (!shell || shell.getAttribute('data-wordpad-ready') === '1') {
            return;
        }

        var fieldName = shell.getAttribute('data-wordpad-name') || 'body';
        var textarea = shell.querySelector('textarea[name="' + fieldName + '"]');
        var editor = shell.querySelector('.wordpad-editor');
        var toolbar = shell.querySelector('.wordpad-toolbar');
        var blockSelect = shell.querySelector('[data-wordpad-block]');
        var form = shell.closest('form');

        if (!textarea || !editor || !toolbar || !form) {
            return;
        }

        toolbar.classList.remove('d-none');
        editor.classList.remove('d-none');
        editor.innerHTML = textarea.value;
        textarea.classList.add('d-none');
        shell.setAttribute('data-wordpad-ready', '1');

        var syncEditor = function () {
            if (shell.getAttribute('data-wordpad-ready') !== '1') {
                return;
            }
            var html = editor.innerHTML
                .replace(/<div><br></div>/gi, '')
                .replace(/&nbsp;/gi, ' ')
                .trim();
            if (html === '' && textarea.defaultValue.trim() !== '' && editor.childNodes.length === 0) {
                return;
            }
            textarea.value = html;
        };

        var runCommand = function (command, value) {
            editor.focus();
            document.execCommand('styleWithCSS', false, false);
            document.execCommand(command, false, value == null ? null : value);
            syncEditor();
        };

        var insertImageFromFile = function (file) {
            if (!file || !/^image\/(jpeg|png|gif|webp)$/i.test(file.type || '')) {
                window.alert('Please choose a JPEG, PNG, GIF, or WebP image.');
```

```

if (file.size > 5 * 1024 * 1024) {
    window.alert('Image is too large (max 5 MB).');
    return;
}

var fd = new FormData();
fd.append('image', file, file.name || 'image.jpg');

fetch('api/classroom_inline_image_upload.php', {
    method: 'POST',
    body: fd,
    credentials: 'same-origin'
})
.then(function (res) { return res.json().then(function (data) { return { ok: res.ok, data: data }; }); })
.then(function (result) {
    if (!result.ok || !result.data || !result.data.src) {
        throw new Error((result.data && result.data.error) || 'Image upload failed.');
```

```

        window.alert('Select some text first.');
```

```

        return false;
    }
    var span = document.createElement('span');
    span.style[styleProp] = styleValue;
    try {
        range.surroundContents(span);
    } catch (err) {
        var fragment = range.extractContents();
        span.appendChild(fragment);
        range.insertNode(span);
    }
    sel.removeAllRanges();
    syncEditor();
    return true;
};

if (fontSizeSelect) {
    fontSizeSelect.addEventListener('change', function (event) {
        var value = event.target.value || '';
        if (value !== '') {
            applyWordpadStyle('fontSize', value);
        }
        event.target.value = '';
    });
}

if (fontFamilySelect) {
    fontFamilySelect.addEventListener('change', function (event) {
        var value = event.target.value || '';
        if (value !== '') {
            applyWordpadStyle('fontFamily', value);
        }
        event.target.value = '';
    });
}

editor.addEventListener('paste', function (event) {
    var items = event.clipboardData && event.clipboardData.items;
    if (!items) {
        return;
    }
    for (var i = 0; i < items.length; i++) {
        var item = items[i];
        if (item && item.kind === 'file' && /^image\/\w+\.test(item.type) || '') {
            event.preventDefault();
            var file = item.getAsFile();
            if (file) {
                insertImageFromFile(file);
            }
            return;
        }
    }
});

editor.addEventListener('input', syncEditor);
editor.addEventListener('blur', syncEditor);
form.addEventListener('submit', syncEditor);
}

document.querySelectorAll('[data-wordpad]').forEach(initWordpad);
})();
</script>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: faculty_edutools.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'admin', 'program_chair', 'gened']);

$userId = (int) ($_SESSION['user_id'] ?? 0);
$storageNs = 'u' . ($userId > 0 ? (string) $userId : '0');

```

```

$pageTitle = 'EduTools (writing & research)';
$mainContainerClass = 'container-fluid px-3 px-lg-4 py-3 py-md-4 flex-grow-1 d-flex flex-column min-w-0 faculty-edutools-page';
require_once __DIR__ . '/includes/header.php';
?>
<div class="mb-3">
    <h1 class="h4 mb-1"><i class="fa-solid fa-wand-magic-sparkles me-2 text-primary"></i>EduTools Pro</h1>
    <p class="text-muted small mb-0">Writing, AI detection, research assistants, and a personal notebook. Use the menu to pick a source;
    <kbd class="small">Ctrl</kbd>+<kbd class="small">E</kbd> toggles the tool panel.</p>
</div>

<div class="faculty-edutools-root" data-storage-ns="<?>= htmlspecialchars($storageNs, ENT_QUOTES, 'UTF-8') ?>">
    <style>
        .faculty-edutools-root * {
            box-sizing: border-box;
        }

        .faculty-edutools-root .app-container {
            display: flex;
            width: 100%;
            position: relative;
            min-height: min(720px, calc(100vh - 14rem));
            border-radius: 12px;
            overflow: hidden;
            border: 1px solid #e2e8f0;
            background: #f1f5f9;
        }

        .faculty-edutools-root .tools-panel {
            width: 320px;
            max-width: 92vw;
            background: rgba(255, 255, 255, 0.97);
            backdrop-filter: blur(2px);
            border-right: 1px solid #e2e8f0;
            display: flex;
            flex-direction: column;
            transition: transform 0.25s ease, width 0.2s;
            z-index: 30;
            box-shadow: 2px 0 12px rgba(0, 0, 0, 0.03);
            flex-shrink: 0;
            height: auto;
            align-self: stretch;
            min-height: min(680px, calc(100vh - 15rem));
            overflow-y: auto;
        }

        .faculty-edutools-root .tools-panel.collapsed {
            transform: translateX(-100%);
            position: absolute;
            height: 100%;
            min-height: 100%;
        }

        .faculty-edutools-root .main-area {
            flex: 1;
            display: flex;
            flex-direction: column;
            overflow: hidden;
            background: #ffffff;
            transition: all 0.2s;
            min-width: 0;
            min-height: min(680px, calc(100vh - 15rem));
        }

        .faculty-edutools-root .top-bar {
            background: white;
            padding: 12px 20px;
            border-bottom: 1px solid #e9eef3;
            display: flex;
            align-items: center;
            gap: 16px;
            flex-wrap: wrap;
            box-shadow: 0 1px 2px rgba(0,0,0,0.02);
        }

        .faculty-edutools-root .menu-toggle-btn {
            background: #f8fafc;
            border: 1px solid #cbd5e1;
            border-radius: 40px;
            padding: 8px 14px;

```

```

font-size: 0.9rem;
font-weight: 500;
cursor: pointer;
display: flex;
align-items: center;
gap: 8px;
color: #1e293b;
transition: all 0.2s;
}

.faculty-edutools-root .menu-toggle-btn:hover {
background: #eef2ff;
border-color: #94a3b8;
}

.faculty-edutools-root .active-badge {
background: #eef2ff;
padding: 6px 14px;
border-radius: 40px;
font-size: 0.85rem;
color: #1e40af;
font-weight: 500;
display: flex;
align-items: center;
gap: 8px;
}

.faculty-edutools-root .active-badge span:first-child {
font-weight: 400;
color: #475569;
}

.faculty-edutools-root .key-hint {
background: #e2e8f0;
padding: 2px 8px;
border-radius: 20px;
font-family: ui-monospace, monospace;
font-size: 0.75rem;
color: #0f172a;
}

.faculty-edutools-root .tool-viewport {
flex: 1;
position: relative;
background: #fefefe;
overflow: auto;
min-height: 0;
}

.faculty-edutools-root iframe {
width: 100%;
height: 100%;
min-height: 420px;
border: none;
display: block;
}

.faculty-edutools-root .notebook-container {
padding: 24px 28px;
height: 100%;
overflow-y: auto;
background: #fefcf5;
}

.faculty-edutools-root .notebook-header {
display: flex;
justify-content: space-between;
align-items: center;
flex-wrap: wrap;
gap: 12px;
margin-bottom: 28px;
border-bottom: 2px solid #e2e8f0;
padding-bottom: 16px;
}

.faculty-edutools-root .notebook-header h2 {
font-size: 1.6rem;
font-weight: 600;
background: linear-gradient(135deg, #0f172a, #334155);
background-clip: text;

```

```

    -webkit-background-clip: text;
    color: transparent;
    margin: 0;
}

.faculty-edutools-root .note-actions {
    display: flex;
    gap: 10px;
    flex-wrap: wrap;
}

.faculty-edutools-root .note-btn {
    background: white;
    border: 1px solid #cbd5e1;
    padding: 6px 14px;
    border-radius: 30px;
    font-weight: 500;
    font-size: 0.8rem;
    cursor: pointer;
    transition: 0.2s;
}

.faculty-edutools-root .note-btn:hover {
    background: #f1f5f9;
}

.faculty-edutools-root textarea.notepad-textarea {
    width: 100%;
    min-height: 55vh;
    padding: 20px;
    font-family: 'Courier New', ui-monospace, monospace;
    font-size: 1rem;
    line-height: 1.5;
    border: 1px solid #e2e8f0;
    border-radius: 20px;
    background: #ffffff;
    resize: vertical;
}

.faculty-edutools-root .status-msg {
    font-size: 0.75rem;
    margin-top: 12px;
    color: #2d6a4f;
}

.faculty-edutools-root .menu-header {
    padding: 20px 20px 6px 20px;
    font-weight: 700;
    font-size: 0.7rem;
    letter-spacing: 1px;
    color: #5b6e8c;
    text-transform: uppercase;
}

.faculty-edutools-root .tool-list {
    list-style: none;
    margin: 4px 12px 12px 12px;
    padding: 0;
}

.faculty-edutools-root .tool-item {
    display: flex;
    align-items: center;
    gap: 14px;
    padding: 10px 14px;
    margin: 2px 0;
    border-radius: 14px;
    cursor: pointer;
    transition: all 0.15s;
    font-weight: 500;
    color: #1e293b;
}

.faculty-edutools-root .tool-item:hover {
    background: #f1f5f9;
}

.faculty-edutools-root .tool-item.active {
    background: #eef2ff;
    color: #1e40af;
}

```

```

border-left: 3px solid #3b82f6;
}

.faculty-edutools-root .tool-icon {
  font-size: 1.3rem;
  line-height: 1;
  min-width: 32px;
}

.faculty-edutools-root .tool-name {
  flex: 1;
  font-size: 0.9rem;
}

.faculty-edutools-root .badge-external {
  font-size: 0.65rem;
  background: #e2e8f0;
  padding: 2px 8px;
  border-radius: 30px;
  color: #334155;
}

.faculty-edutools-root .edutools-hr {
  margin: 8px 16px;
  border: 0;
  border-top: 1px solid #edf2f7;
}

.faculty-edutools-root .shortcut-toast {
  position: absolute;
  bottom: 12px;
  right: 12px;
  background: #0f172ad9;
  backdrop-filter: blur(8px);
  color: white;
  padding: 6px 12px;
  border-radius: 40px;
  font-size: 0.7rem;
  font-family: ui-monospace, monospace;
  pointer-events: none;
  z-index: 25;
  max-width: 90%;
}

@media (max-width: 680px) {
  .faculty-edutools-root .tools-panel {
    width: 280px;
  }
  .faculty-edutools-root .notebook-container {
    padding: 16px;
  }
}
</style>

<div class="app-container">
  <aside class="tools-panel" id="facToolsPanel" aria-label="Education sources and tools">
    <div style="padding: 20px 20px 0 20px;">
      <h2 style="font-size: 1.5rem; font-weight: 700; margin: 0; background: linear-gradient(145deg, #0f2b3d, #1e4a76); background-clip:
text; -webkit-background-clip: text; color: transparent;">EduTools Pro</h2>
      <p style="font-size: 0.7rem; color: #4b5563; margin-top: 4px;">AI detector · Writing · Research · Education sources</p>
    </div>
    <div class="menu-header">Writing & academic</div>
    <ul class="tool-list" id="facWritingToolsList"></ul>
    <div class="menu-header">AI & research</div>
    <ul class="tool-list" id="facAiResearchList"></ul>
    <div class="menu-header">Notebooks</div>
    <ul class="tool-list" id="facNotebooksList"></ul>
    <hr class="edutools-hr" />
    <div style="padding: 12px 20px 20px 20px; font-size: 0.7rem; color: #6c757d;">
      <kbd>Ctrl</kbd> + <kbd>E</kbd> toggles this panel
    </div>
  </aside>

  <div class="main-area">
    <div class="top-bar">
      <button type="button" class="menu-toggle-btn" id="facTogglePanelBtn"<? app_tooltip_attr('Shows or hides the EduTools sidebar. Same
as Ctrl+E.') ?>>
        <span aria-hidden="true">≡</span> Menu
      </button>
      <div class="active-badge" id="facActiveToolBadge">

```

```
<span>Active:</span> <strong id="facActiveToolName">Notebook</strong>  
</div>  
<div class="ms-auto d-flex gap-2 align-items-center">  
  <span class="key-hint">Ctrl+E</span>  
</div>  
</div>  
<div class="tool-viewport position-relative" id="facToolViewportOuter">  
  <div id="facDynamicContent"></div>  
  <div class="shortcut-toast" aria-hidden="true">Ctrl+E sidebar · AI Detector · Grammarly · QuillBot · Turnitin</div>  
</div>  
</div>  
</div>  
</div>  
  
<script>  
(function () {  
  const root = document.querySelector('.faculty-edutools-root');  
  if (!root) return;  
  const ns = root.getAttribute('data-storage-ns') || 'guest';  
  
  const NOTEBOOK_STORAGE_KEY = 'faculty_edu_' + ns + '_notebook_content';  
  const LAST_TOOL_KEY = 'faculty_edu_' + ns + '_last_tool';  
  const PANEL_COLLAPSED_KEY = 'faculty_edu_' + ns + '_panel_collapsed';  
  
  const toolsConfig = [  
    { id: 'turnitin', name: 'Turnitin', icon: '\uD83D\uDD0D', type: 'external', url: 'https://www.turnitin.com', openNewTab: true,  
      description: 'Plagiarism detection & feedback studio', category: 'writing' },  
    { id: 'grammarly', name: 'Grammarly', icon: '\u270D\uFE0F', type: 'external', url: 'https://www.grammarly.com', openNewTab: true,  
      description: 'Grammar checker & writing assistant', category: 'writing' },  
    { id: 'quillbot', name: 'QuillBot', icon: '\uD83E\uDD9C', type: 'external', url: 'https://quillbot.com', openNewTab: true, description:  
'Paraphraser, summarizer & grammar', category: 'writing' },  
    { id: 'aidetector', name: 'Free AI Detector', icon: '\uD83E\uDD16', type: 'external', url: 'https://www.aidetector.com', openNewTab:  
true, description: 'Check if text is AI-generated (ChatGPT, GPT-4)', category: 'writing' },  
    { id: 'chatgpt', name: 'ChatGPT', icon: '\uD83D\uDCA5', type: 'external', url: 'https://chat.openai.com', openNewTab: true, description:  
'OpenAI conversational AI', category: 'ai' },  
    { id: 'perplexity', name: 'Perplexity AI', icon: '\uD83D\uDD0D', type: 'external', url: 'https://www.perplexity.ai', openNewTab: true,  
      description: 'AI-powered search & answers', category: 'ai' },  
    { id: 'notebooklm', name: 'NotebookLM', icon: '\uD83E\uDE0B', type: 'external', url: 'https://notebooklm.google.com', openNewTab: true,  
      description: 'Google AI research assistant', category: 'ai' },  
    { id: 'wolfram', name: 'Wolfram Alpha', icon: '\uD83E\uDEEE', type: 'external', url: 'https://www.wolframalpha.com', openNewTab: true,  
      description: 'Computational knowledge engine', category: 'ai' },  
    { id: 'notebook', name: 'Built-in notebook', icon: '\uD83D\uDCD8', type: 'notebook', url: null, openNewTab: false, category: 'notebooks'  
},  
    { id: 'notion', name: 'Notion', icon: '\uD83D\uDCDD', type: 'external', url: 'https://www.notion.so', openNewTab: true, description:  
'All-in-one workspace', category: 'notebooks' },  
    { id: 'khan', name: 'Khan Academy', icon: '\uD83D\uDCDA', type: 'external', url: 'https://www.khanacademy.org', openNewTab: true,  
      description: 'Free courses & practice', category: 'notebooks' }  
  ];  
  
  let currentToolId = 'notebook';  
  let notebookTextarea = null;  
  let notebookContent = '';  
  
  const toolViewport = document.getElementById('facDynamicContent');  
  const toolsPanel = document.getElementById('facToolsPanel');  
  const toggleBtn = document.getElementById('facTogglePanelBtn');  
  const activeToolNameSpan = document.getElementById('facActiveToolName');  
  const writingToolsList = document.getElementById('facWritingToolsList');  
  const aiResearchList = document.getElementById('facAiResearchList');  
  const notebooksList = document.getElementById('facNotebooksList');  
  
  if (!toolViewport || !toolsPanel || !toggleBtn || !activeToolNameSpan || !writingToolsList || !aiResearchList || !notebooksList) return;  
  
  function escapeHtml(s) {  
    const d = document.createElement('div');  
    d.textContent = s;  
    return d.innerHTML;  
  }  
  
  function createMenuItem(tool) {  
    const li = document.createElement('li');  
    li.className = 'tool-item' + (currentToolId === tool.id ? ' active' : '');  
    li.setAttribute('data-tool-id', tool.id);  
    li.innerHTML =  
      '<span class="tool-icon">' + tool.icon + '</span>' +  
      '<span class="tool-name">' + escapeHtml(tool.name) + '</span>' +  
      '<span class="badge-external">' + (tool.openNewTab ? '\u2197\uFE0F' : '\uD83D\uDCD3') + '</span>';  
    li.addEventListener('click', function (e) {  
      e.stopPropagation();  
      activateTool(tool.id);  
    });  
  }  
</script>
```



```

    });
    return li;
}

function renderMenu() {
    writingToolsList.innerHTML = '';
    aiResearchList.innerHTML = '';
    notebooksList.innerHTML = '';
    toolsConfig.filter(function (t) { return t.category === 'writing'; }).forEach(function (tool) {
        writingToolsList.appendChild(createMenuItem(tool));
    });
    toolsConfig.filter(function (t) { return t.category === 'ai'; }).forEach(function (tool) {
        aiResearchList.appendChild(createMenuItem(tool));
    });
    toolsConfig.filter(function (t) { return t.category === 'notebooks'; }).forEach(function (tool) {
        notebooksList.appendChild(createMenuItem(tool));
    });
}

function renderNotebook() {
    const saved = localStorage.getItem(NOTEBOOK_STORAGE_KEY) ||
        '# Faculty notebook\n\nDraft syllabi, track sources, or paste text before checking with external tools.\n\n## Quick links\n- Turnitin,\nGrammarly, QuillBot, Free AI Detector (menu)\n- ChatGPT, Perplexity, NotebookLM (menu)\n\n---\n\n### Notes\n';
    notebookContent = saved;
    toolViewport.innerHTML =
        '<div class="notebook-container">' +
        '  <div class="notebook-header">' +
        '    <h2>Smart notebook</h2>' +
        '    <div class="note-actions">' +
        '      <button type="button" id="facExportMdBtn" class="note-btn">Export .md</button>' +
        '      <button type="button" id="facExportTxtBtn" class="note-btn">Export .txt</button>' +
        '      <button type="button" id="facClearNotebookBtn" class="note-btn">Clear</button>' +
        '    </div>' +
        '  </div>' +
        '  <textarea id="facNotebookTextarea" class="notepad-textarea" placeholder="Write notes, paste drafts for checking..."></textarea>' +
        '  <div class="status-msg" id="facNoteStatus">Auto-saves locally in this browser</div>' +
        '</div>';

    const textarea = document.getElementById('facNotebookTextarea');
    if (textarea) {
        textarea.value = notebookContent;
        notebookTextarea = textarea;
        var saveTimeout;
        textarea.addEventListener('input', function () {
            if (saveTimeout) clearTimeout(saveTimeout);
            saveTimeout = setTimeout(function () {
                var newContent = textarea.value;
                localStorage.setItem(NOTEBOOK_STORAGE_KEY, newContent);
                notebookContent = newContent;
                var statusDiv = document.getElementById('facNoteStatus');
                if (statusDiv) {
                    statusDiv.textContent = 'Saved at ' + new Date().toLocaleTimeString();
                    setTimeout(function () {
                        if (statusDiv) statusDiv.textContent = 'Auto-save active';
                    }, 1500);
                }
            }, 500);
        });
    }
    var exportMd = document.getElementById('facExportMdBtn');
    var exportTxt = document.getElementById('facExportTxtBtn');
    var clearBtn = document.getElementById('facClearNotebookBtn');
    if (exportMd) exportMd.addEventListener('click', function () { exportNotebook('md'); });
    if (exportTxt) exportTxt.addEventListener('click', function () { exportNotebook('txt'); });
    if (clearBtn) {
        clearBtn.addEventListener('click', function () {
            if (confirm('Clear all notebook content?')) {
                if (notebookTextarea) notebookTextarea.value = '';
                localStorage.setItem(NOTEBOOK_STORAGE_KEY, '');
                notebookContent = '';
                var statusDiv = document.getElementById('facNoteStatus');
                if (statusDiv) statusDiv.textContent = 'Cleared.';
            }
        });
    }
}

function exportNotebook(format) {
    var content = notebookTextarea ? notebookTextarea.value : (localStorage.getItem(NOTEBOOK_STORAGE_KEY) || '');
    var blob = new Blob([content], { type: format === 'md' ? 'text/markdown' : 'text/plain' });

```

```

var link = document.createElement('a');
link.href = URL.createObjectURL(blob);
link.download = 'faculty_notebook_export.' + (format === 'md' ? 'md' : 'txt');
link.click();
URL.revokeObjectURL(link.href);
}

function externalHelperMeta(tool) {
  var specialNote = '';
  var accentColor = '#0f172a';
  var gradientBg = '#fafcff';
  if (tool.id === 'turnitin') {
    specialNote = 'Turnitin is widely used by universities. Check papers for originality before submitting. (Institutional login may be required.)';
    accentColor = '#d32f2f';
    gradientBg = '#fff5f5';
  } else if (tool.id === 'grammarly') {
    specialNote = 'Grammarly catches grammar mistakes, enhances tone, and checks clarity. Free and premium plans available.';
    accentColor = '#24c0b0';
    gradientBg = '#f0fdfa';
  } else if (tool.id === 'quillbot') {
    specialNote = 'QuillBot offers paraphrasing, summarizer, grammar checker, and citation generator. Useful for rewording sentences.';
    accentColor = '#6f42c1';
    gradientBg = '#f8f3ff';
  } else if (tool.id === 'aidetector') {
    specialNote = 'Paste text to see if it may be AI-generated. Useful as one signal among many for academic integrity.';
    accentColor = '#e67e22';
    gradientBg = '#fff8f0';
  } else if (tool.id === 'notebooklm') {
    specialNote = 'Google's AI research notebook: upload sources (PDF, docs, websites) and get summaries, Q&A, and study guides.';
    accentColor = '#8b5cf6';
    gradientBg = '#f5f0ff';
  } else if (tool.id === 'chatgpt') {
    specialNote = 'Conversational AI for brainstorming, writing, coding, and research. Sign in with Google, Microsoft, or email.';
    accentColor = '#10a37f';
    gradientBg = '#f0fdf4';
  } else if (tool.id === 'perplexity') {
    specialNote = 'AI search with cited sources. Ask questions and get answers with references.';
    accentColor = '#1a56db';
    gradientBg = '#eff6ff';
  } else if (tool.id === 'wolfram') {
    specialNote = 'Computational answers across math, science, and data. Strong for formulas and facts.';
    accentColor = '#dd1100';
    gradientBg = '#fff8f5';
  } else if (tool.id === 'notion') {
    specialNote = 'Notes, wikis, and project boards in one place. Good for course planning and collaboration.';
    accentColor = '#000000';
    gradientBg = '#f7f7f5';
  } else if (tool.id === 'khan') {
    specialNote = 'Free lessons and practice across many subjects. Useful to recommend or review fundamentals.';
    accentColor = '#14b996';
    gradientBg = '#f0fdf9';
  }
  return { specialNote: specialNote, accentColor: accentColor, gradientBg: gradientBg };
}

function renderExternalHelper(tool) {
  var meta = externalHelperMeta(tool);
  var specialNote = meta.specialNote;
  var accentColor = meta.accentColor;
  var gradientBg = meta.gradientBg;
  toolViewport.innerHTML =
    '<div style="display:flex;align-items:center;justify-content:center;min-height:100%;background:' + gradientBg + ';padding:2rem;">' +
    '<div style="max-width:560px;text-align:center;background:white;border-radius:2rem;padding:2rem;box-shadow:0 20px 35px' +
    'rgba(0,0,0,0.1);border:1px solid #eef2ff;">' +
    '<div style="font-size:4rem;line-height:1;">' + tool.icon + '</div>' +
    '<h2 style="margin:1rem 0 0.5rem;color:' + accentColor + ';font-size:1.35rem;">' + escapeHtml(tool.name) + '</h2>' +
    '<p style="color:#4b5563;margin-bottom:0.5rem;">' + escapeHtml(tool.description || 'Academic & writing assistant') + '</p>' +
    (specialNote
      ? '<div style="background:' + gradientBg + ';padding:12px;border-radius:16px;margin:16px 0;font-size:0.85rem;color:#2d3e50;border-left:4px solid ' + accentColor + ';text-align:left;">' + escapeHtml(specialNote) + '</div>'
      : '') +
    '<button type="button" id="facOpenExternalBtn" style="background:' + accentColor + ';color:white;border:none;padding:12px' +
    '32px;border-radius:60px;font-weight:600;cursor:pointer;font-size:1rem;margin:8px 0;">Open ' + escapeHtml(tool.name) + ' \u2192</button>' +
    '<p style="margin-top:20px;font-size:0.7rem;color:#6c757d;">Opens in a new tab. Some tools require institutional access or a free' +
    'account.</p>' +
    '</div>' +
    '</div>';
  var openBtn = document.getElementById('facOpenExternalBtn');
  if (openBtn && tool.url) {

```

```

        openBtn.addEventListener('click', function () {
            window.open(tool.url, '_blank', 'noopener,noreferrer');
        });
    }
}

function activateTool(toolId) {
    var tool = toolsConfig.find(function (t) { return t.id === toolId; });
    if (!tool) return;
    currentToolId = toolId;
    activeToolNameSpan.textContent = tool.name;
    document.querySelectorAll('.faculty-edutools-root .tool-item').forEach(function (item) {
        if (item.getAttribute('data-tool-id') === toolId) {
            item.classList.add('active');
        } else {
            item.classList.remove('active');
        }
    });
    if (tool.type === 'notebook') {
        renderNotebook();
        setTimeout(function () {
            notebookTextarea = document.getElementById('facNotebookTextarea');
            if (notebookTextarea && notebookContent) notebookTextarea.value = notebookContent;
        }, 10);
    } else if (tool.type === 'external') {
        renderExternalHelper(tool);
    }
    localStorage.setItem(LAST_TOOL_KEY, toolId);
}

function togglePanel() {
    toolsPanel.classList.toggle('collapsed');
    localStorage.setItem(PANEL_COLLAPSED_KEY, toolsPanel.classList.contains('collapsed') ? 'true' : 'false');
}

function restoreState() {
    var lastTool = localStorage.getItem(LAST_TOOL_KEY);
    var panelCollapsed = localStorage.getItem(PANEL_COLLAPSED_KEY) === 'true';
    if (panelCollapsed) {
        toolsPanel.classList.add('collapsed');
    } else {
        toolsPanel.classList.remove('collapsed');
    }
    if (lastTool && toolsConfig.some(function (t) { return t.id === lastTool; })) {
        activateTool(lastTool);
    } else {
        activateTool('notebook');
    }
}

function handleKeydown(e) {
    if ((e.ctrlKey || e.metaKey) && (e.key === 'e' || e.key === 'E')) {
        e.preventDefault();
        togglePanel();
    }
}

window.addEventListener('beforeunload', function () {
    if (currentToolId === 'notebook' && notebookTextarea) {
        localStorage.setItem(NOTEBOOK_STORAGE_KEY, notebookTextarea.value);
    }
});

renderMenu();
restoreState();
toggleBtn.addEventListener('click', togglePanel);
window.addEventListener('keydown', handleKeydown);

setInterval(function () {
    if (currentToolId === 'notebook' && notebookTextarea && document.getElementById('facNotebookTextarea')) {
        var fresh = notebookTextarea.value;
        localStorage.setItem(NOTEBOOK_STORAGE_KEY, fresh);
        notebookContent = fresh;
    }
}, 2000);
})();
</script>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: faculty_meet_live.php

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'program_chair', 'dean', 'gened']);

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}
if ($facultyId < 1 && in_array($_SESSION['role'] ?? '', ['program_chair', 'dean', 'gened'], true)) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    if ($facultyId > 0) {
        $_SESSION['faculty_id'] = $facultyId;
    }
}
if ($facultyId < 1) {
    exit('Faculty profile not linked to this account.');
```

```
}

$classroomId = (int) ($_GET['classroom_id'] ?? 0);
if ($classroomId < 1) {
    http_response_code(400);
    exit('Missing classroom.');
```

```
}

$stmt = db()->prepare(
    'SELECT oc.id, oc.meet_link, oc.title, oc.schedule_id,
           s.day_of_week, s.start_time, s.end_time, s.online_live_at,
           c.course_code, c.course_name
    FROM online_classrooms oc
    INNER JOIN schedules s ON s.id = oc.schedule_id
    INNER JOIN courses c ON c.id = oc.course_id
    WHERE oc.id = ? AND oc.faculty_id = ? AND s.faculty_id = ?
    LIMIT 1'
);
$stmt->execute([$classroomId, $facultyId, $facultyId]);
$classroom = $stmt->fetch(PDO::FETCH_ASSOC);
if (!$classroom) {
    http_response_code(404);
    exit('Classroom not found.');
```

```
}

$meetLink = filter_var(trim((string) ($classroom['meet_link'] ?? '')), FILTER_VALIDATE_URL);
if ($meetLink === false) {
    http_response_code(400);
    exit('This classroom does not have a valid Meet link.');
```

```
}

$scheduleId = (int) $classroom['schedule_id'];
>window = classroom_attendance_login_allowed($classroom);
$sessionEndTs = strtotime($window['session_end']);
if ($sessionEndTs === false) {
    $sessionEndTs = time() + 7200;
}

$pageTitle = 'Live session - ' . (string) $classroom['course_code'];
require_once __DIR__ . '/includes/header.php';
?>

<div class="container py-4" style="max-width: 520px;">
    <div class="card border-0 shadow-sm">
        <div class="card-body p-4 text-center">
            <h1 class="h5 mb-2">
                <span class="badge bg-danger live-pulse-badger me-1">LIVE</span>
                <?= htmlspecialchars((string) $classroom['course_code']) ?>
            </h1>
            <p class="text-muted small mb-3"><?= htmlspecialchars((string) $classroom['course_name']) ?></p>
            <p id="meet-live-status" class="mb-3">Opening Google Meet...</p>
            <p class="small text-muted mb-0">When you leave or close Google Meet, the <strong>LIVE</strong> badge on the weekly schedule
turns off automatically.</p>
            <div class="mt-3 d-flex flex-wrap justify-content-center gap-2">
                <button type="button" class="btn btn-success btn-sm" id="meet-live-reopen"><i class="fa-solid fa-video me-1"></i>Open Meet
```

```

again</button>
        <button type="button" class="btn btn-outline-danger btn-sm" id="meet-live-end"><i class="fa-solid fa-stop me-1"></i>End live
now</button>
    </div>
</div>
</div>
</div>
</div>

<script>
(function () {
    var meetUrl = <?=json_encode($meetLink, JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT) ?>;
    var scheduleId = <?=(int) $scheduleId ?>;
    var sessionEndMs = <?=(int) ($sessionEndTs * 1000) ?>;
    var endUrl = <?=json_encode('api/faculty_end_live.php', JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT) ?>;
    var statusEl = document.getElementById('meet-live-status');
    var ended = false;
    var meetWin = null;

    function setStatus(text) {
        if (statusEl) {
            statusEl.textContent = text;
        }
    }

    function endLiveSession() {
        if (ended) {
            return;
        }
        ended = true;
        setStatus('Ending live session...');

        var body = new FormData();
        body.append('schedule_id', String(scheduleId));

        if (navigator.sendBeacon) {
            navigator.sendBeacon(endUrl, body);
        } else {
            fetch(endUrl, { method: 'POST', body: body, credentials: 'same-origin' }).catch(function () {});
        }

        setStatus('Live session ended. You may close this tab.');
```

```

        document.querySelectorAll('#meet-live-reopen, #meet-live-end').forEach(function (btn) {
            btn.disabled = true;
        });
    }

    function openMeet() {
        if (ended) {
            return;
        }
        meetWin = window.open(meetUrl, 'classMeetSession');
        if (!meetWin) {
            setStatus('Allow pop-ups for this site, then click "Open Meet again."');
            return;
        }
        setStatus('Google Meet is open. Close Meet when class ends to turn off LIVE.');
```

```

    }

    openMeet();

    var poll = window.setInterval(function () {
        if (ended) {
            window.clearInterval(poll);
            return;
        }
        if (meetWin && meetWin.closed) {
            window.clearInterval(poll);
            endLiveSession();
        }
        if (Date.now() >= sessionEndMs) {
            window.clearInterval(poll);
            endLiveSession();
        }
    }, 1000);

    window.addEventListener('pagehide', endLiveSession);

    var reopenBtn = document.getElementById('meet-live-reopen');
    if (reopenBtn) {
        reopenBtn.addEventListener('click', openMeet);
    }

```

```

    }
    var endBtn = document.getElementById('meet-live-end');
    if (endBtn) {
        endBtn.addEventListener('click', function () {
            if (meetWin && !meetWin.closed) {
                meetWin.close();
            }
            endLiveSession();
        });
    }
})();
</script>

```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>
```

```
-----
FILE: faculty_offline.php
-----
```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'program_chair', 'dean', 'gened']);

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}
if ($facultyId < 1 && in_array($_SESSION['role'] ?? '', ['program_chair', 'dean', 'gened'], true)) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    if ($facultyId > 0) {
        $_SESSION['faculty_id'] = $facultyId;
    }
}
if ($facultyId < 1) {
    exit('Faculty profile not linked to this account. Ask your dean to create/link your faculty profile.');
```

```

        $label .= ' (' . $term . ')';
    }

    return [
        'id' => (int) ($row['id'] ?? 0),
        'label' => $label,
        'course_code' => trim((string) ($row['course_code'] ?? '')),
        'course_name' => trim((string) ($row['course_name'] ?? '')),
        'classroom_title' => trim((string) ($row['title'] ?? 'Classroom')),
    ];
},
$assignableClassrooms
), JSON_UNESCAPED_UNICODE | JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT) ?></script>

<script src="assets/js/faculty_offline.js" defer></script>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: faculty_schedule.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'program_chair', 'dean', 'gened']);
$userRole = (string) ($_SESSION['role'] ?? '');
$scheduleReviewer = schedule_change_reviewer_label($userRole);
$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}
if ($facultyId < 1 && in_array($_SESSION['role'] ?? '', ['program_chair', 'dean', 'gened'], true)) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    if ($facultyId > 0) {
        $_SESSION['faculty_id'] = $facultyId;
    }
}
if ($facultyId < 1) {
    exit('Faculty profile not linked to this account. Ask your dean to create/link your faculty profile.');
```

```

if ($makeupRooms === []) {
    // Fallback: any available room so the form can still submit.
    $makeupRooms = db()->query(
        "SELECT id, room_code, room_name FROM rooms WHERE status IN ('available','tba') ORDER BY room_code"
    )->fetchAll(PDO::FETCH_ASSOC);
}
}

/**
 * @throws RuntimeException
 */
function faculty_schedule_sanitizе_online_url(string $raw): string
{
    $raw = trim($raw);
    if ($raw === '') {
        return '';
    }
    $u = filter_var($raw, FILTER_VALIDATE_URL);
    if ($u === false) {
        throw new RuntimeException('Please enter a valid URL (e.g. https://meet.google.com/...).');
    }
    $scheme = strtolower((string) (parse_url($u, PHP_URL_SCHEME) ?? ''));
    if (!in_array($scheme, ['http', 'https'], true)) {
        throw new RuntimeException('Only http and https links are allowed.');
```



```

        $window['reason'] !== ''
        ? $window['reason']
        : 'Go live is only available during the scheduled class time.'
    );
}
db()->prepare('UPDATE schedules SET online_live_at = NOW() WHERE id=? AND faculty_id=?' . $scheduleCollegeClause)
->execute(array_merge([$scheduleId, $facultyId], $scheduleCollegeParam));
if ($hasLiveSessions) {
    $st = db()->prepare('SELECT id FROM online_classrooms WHERE schedule_id = ? AND faculty_id = ? LIMIT 1');
    $st->execute([$scheduleId, $facultyId]);
    $classroomId = (int) ($st->fetchColumn() ?: 0);
    if ($classroomId > 0) {
        $st = db()->prepare(
            'SELECT id
            FROM classroom_live_sessions
            WHERE classroom_id = ? AND schedule_id = ? AND faculty_id = ? AND ended_at IS NULL
            ORDER BY id DESC
            LIMIT 1'
        );
        $st->execute([$classroomId, $scheduleId, $facultyId]);
        if ((int) ($st->fetchColumn() ?: 0) < 1) {
            db()->prepare(
                'INSERT INTO classroom_live_sessions (classroom_id, schedule_id, faculty_id, started_at)
                VALUES (?, ?, ?, NOW())'
            )->execute([$classroomId, $scheduleId, $facultyId]);
        }
    }
}
$_SESSION['flash'] = 'You are now marked as live for this class.';
} else {
    faculty_end_live_for_schedule($scheduleId, $facultyId);
    $_SESSION['flash'] = 'Live session ended.';
}
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: faculty_schedule.php');
exit;
}

```

```

if (isset($_POST['schedule_id'])) {
    $scheduleId = (int) $_POST['schedule_id'];
    $message = trim((string) ($_POST['message'] ?? ''));
    $requestType = (string) ($_POST['request_type'] ?? 'change');
    if ($requestType === 'makeup' && $hasMakeupSupport) {
        $day = trim((string) ($_POST['makeup_day'] ?? ''));
        $startRaw = substr((string) ($_POST['makeup_start_time'] ?? ''), 0, 5);
        $endRaw = substr((string) ($_POST['makeup_end_time'] ?? ''), 0, 5);
        $roomId = (int) ($_POST['makeup_room_id'] ?? 0);
        $allowedDays = array_flip(schedule_days_list());
        if ($message === '') {
            $_SESSION['flash'] = 'Please enter a reason for the makeup class.';
        } elseif (!isset($allowedDays[$day]) || $startRaw === '' || $endRaw === '' || $roomId < 1) {
            $_SESSION['flash'] = 'Makeup day, time, and room are required.';
        } else {
            $startTime = $startRaw . ':00';
            $endTime = $endRaw . ':00';
            $ruleCheck = validate_schedule_rules('Custom', [$day], $startTime, $endTime);
            if (empty($ruleCheck['ok'])) {
                $errs = $ruleCheck['errors'] ?? [];
                $_SESSION['flash'] = implode(' ', is_array($errs) ? $errs : ['Invalid makeup time.']);
                $_SESSION['makeup_warning_popup'] = is_array($errs) ? $errs : ['Invalid makeup time.'];
            } else {
                try {
                    $chk = db()->prepare(
                        'SELECT id, faculty_id, room_id, college_id, semester, school_year
                        FROM schedules WHERE id=? AND faculty_id=?' . $scheduleCollegeClause . ' LIMIT 1'
                    );
                    $chk->execute(array_merge([$scheduleId, $facultyId], $scheduleCollegeParam));
                    $baseRow = $chk->fetch(PDO::FETCH_ASSOC);
                    if (!$baseRow) {
                        $_SESSION['flash'] = 'Schedule not found.';
                    } else {
                        $collegeForCheck = isset($baseRow['college_id']) && $baseRow['college_id'] !== null
                            ? (int) $baseRow['college_id']
                            : $facultyCollegeId;
                        $conflictMsgs = makeup_hard_conflict_messages(
                            (int) $baseRow['faculty_id'],
                            $roomId,
                            [$day],

```

```

        $startTime,
        $endTime,
        (string) $baseRow['semester'],
        (string) $baseRow['school_year'],
        $collegeForCheck
    );
    $ackConflicts = !empty($_POST['ack_conflicts']);
    if ($conflictMsgs !== [] && !$ackConflicts) {
        $_SESSION['makeup_warning_popup'] = $conflictMsgs;
        $_SESSION['makeup_ack_draft'] = [
            'schedule_id' => $scheduleId,
            'makeup_day' => $day,
            'makeup_start_time' => $startRow,
            'makeup_end_time' => $endRow,
            'makeup_room_id' => $roomId,
            'message' => $message,
        ];
        $_SESSION['flash'] = 'This makeup slot conflicts with an existing faculty or room schedule. See the warning,
then confirm to submit anyway.';
    } else {
        db()->prepare(
            'INSERT INTO schedule_change_requests
            (faculty_user_id, schedule_id, request_type, message, proposed_day_of_week, proposed_start_time,
proposed_end_time, proposed_room_id)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)'
        )->execute([$userId, $scheduleId, 'makeup', $message, $day, $startTime, $endTime, $roomId]);
        $_SESSION['flash'] = $conflictMsgs !== []
            ? 'Makeup request submitted with known conflicts. ' . ucfirst($scheduleReviewer) . ' will review them.'
            : 'Makeup class request submitted to ' . $scheduleReviewer . '.';
    }
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Could not submit makeup request: ' . $e->getMessage();
}
}
} elseif ($message !== '') {
    $chk = db()->prepare('SELECT COUNT(*) FROM schedules WHERE id=? AND faculty_id=?' . $scheduleCollegeClause);
    $chk->execute(array_merge([$scheduleId, $facultyId], $scheduleCollegeParam));
    if ((int) $chk->fetchColumn() > 0) {
        if ($hasMakeupSupport) {
            db()->prepare(
                'INSERT INTO schedule_change_requests (faculty_user_id, schedule_id, request_type, message) VALUES (?, ?, ?, ?)'
            )->execute([$userId, $scheduleId, 'change', $message]);
        } else {
            db()->prepare('INSERT INTO schedule_change_requests (faculty_user_id, schedule_id, message) VALUES (?, ?, ?)'
            )->execute([$userId, $scheduleId, $message]);
        }
        $_SESSION['flash'] = 'Change request submitted to ' . $scheduleReviewer . '.';
    }
}
header('Location: faculty_schedule.php');
exit;
}
}

$sql = 'SELECT s.*, c.course_code, c.course_name, r.room_code
FROM schedules s
INNER JOIN courses c ON c.id = s.course_id
INNER JOIN rooms r ON r.id = s.room_id
WHERE s.faculty_id=?';
$params = [$facultyId];
if ($facultyCollegeId !== null) {
    $sql .= ' AND s.college_id=?';
    $params[] = $facultyCollegeId;
}
$sql .= ' ORDER BY s.school_year DESC, s.semester, s.start_time';
$st = db()->prepare($sql);
$st->execute($params);
$rows = $st->fetchAll();

$formatTime12h = static function (?string $time): string {
    $raw = substr((string) $time, 0, 5);
    $dt = DateTime::createFromFormat('H:i', $raw);
    return $dt ? $dt->format('g:i A') : $raw;
};

$pageTitle = 'My Schedule';
require_once __DIR__ . '/includes/header.php';
?>

```

```

<header class="app-page-header d-flex flex-wrap justify-content-between align-items-start gap-3">
    <div>
        <h1 class="app-page-title mb-2"><i class="fa-solid fa-calendar-check me-2 app-page-title-icon" aria-hidden="true"></i>My
schedule</h1>
        <p class="app-page-lead mb-0">Your assigned sections, meeting links, and live status. Request a schedule change or a makeup class
slot when needed.</p>
    </div>
    <a href="faculty_classrooms.php" class="btn btn-outline-primary btn-sm rounded-pill align-self-start">?= app_tooltip_attr('Opens your
online class list to manage content, students, and Meet links.') ?><i class="fa-solid fa-chalkboard me-1" aria-hidden="true"></i>My
classrooms</a>
</header>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
<?php if (!$hasOnlineUrl): ?>
    <div class="alert alert-warning app-alert">Online class links require a database update. Ask your administrator to run <a
href="upgrade_roles.php">upgrade_roles.php</a> once.</div>
<?php endif; ?>

<div class="app-card app-schedule-card mb-0">
    <div class="app-card-header">Weekly assignments</div>
    <div class="table-responsive">
        <table class="table app-table mb-0">
            <thead>
                <tr>
                    <th scope="col">Course</th>
                    <th scope="col">Days</th>
                    <th scope="col">Time</th>
                    <th scope="col">Room</th>
                    <?php if ($hasOnlineUrl): ?><th scope="col">Online class</th><?php endif; ?>
                    <th scope="col">Term</th>
                    <th scope="col">Requests</th>
                </tr>
            </thead>
            <tbody>
                <?php if (!$rows): ?>
                    <tr><td colspan="?">?= $hasOnlineUrl ? '7' : '6' ?>" class="app-empty-state">No schedule assigned.</td></tr>
                <?php endif; ?>
                <?php foreach ($rows as $r): ?>
                    <tr>
                        <td data-label="Course">
                            <span class="fw-medium"><?php htmlspecialchars($r['course_code']) ?></span>
                            <?php if ($hasMakeupSupport && (int) ($r['is_makeup'] ?? 0) === 1): ?>
                                <span class="badge bg-warning text-dark ms-1">Makeup</span>
                            <?php endif; ?>
                            <span class="text-muted d-block small"><?php htmlspecialchars($r['course_name']) ?></span>
                        </td>
                        <td data-label="Days"><?php htmlspecialchars(str_replace(',', ' ', (string) $r['day_of_week'])) ?></td>
                        <td data-label="Time"><?php htmlspecialchars($formatTime12h((string) $r['start_time'])) ?> - <?php
htmlspecialchars($formatTime12h((string) $r['end_time'])) ?></td>
                        <td data-label="Room"><?php htmlspecialchars($r['room_code']) ?></td>
                        <?php if ($hasOnlineUrl): ?>
                            <td class="align-top" style="min-width: 220px;" data-label="Online class">
                                <?php
                                    $link = trim((string) ($r['online_class_url'] ?? ''));
                                ?>
                                <?php if ($link !== ''): ?>
                                    <div class="mb-1">
                                        <a href="?">?= htmlspecialchars($link) ?>" target="_blank" rel="noopener noreferrer" class="btn btn-sm
btn-outline-primary">?= app_tooltip_attr('Opens your saved online class URL (e.g. Meet) in a new tab for this section.') ?>>
                                            <i class="fa-solid fa-video me-1"></i>Join online
                                        </a>
                                    </div>
                                <?php endif; ?>
                                <form method="post" class="d-flex flex-column gap-1">
                                    <input type="hidden" name="action" value="save_online_link">
                                    <input type="hidden" name="schedule_id" value="?">?= (int) $r['id'] ?>>
                                    <input type="url" name="online_class_url" class="form-control form-control-sm" placeholder="https://..."
value="?">?= htmlspecialchars($link) ?>" autocomplete="url">
                                    <div class="d-flex gap-1 flex-wrap">
                                        <button type="submit" class="btn btn-sm btn-primary">?= app_tooltip_attr('Saves the online meeting URL
for this section so students and deans can use it.') ?>>Save link</button>
                                        <?php if ($link !== ''): ?>
                                            <button type="submit" name="clear_online" value="1" class="btn btn-sm btn-outline-secondary"
title="Remove link">?= app_tooltip_attr('Removes the saved online link for this row. Use this when the meeting URL changes completely.') ?
>>Clear</button>
                                        <?php endif; ?>
                                    </div>
                                </form>
                                <?php if ($hasLiveAt && $link !== ''): ?>
                                    <?php

```

```

$liveAtRaw = $r['online_live_at'] ?? null;
$liveWindow = classroom_attendance_login_allowed($r);
$isWithinClassTime = !empty($liveWindow['allowed']);
$wasMarkedLive = $liveAtRaw !== null && $liveAtRaw !== '';
$isLiveNow = $wasMarkedLive && schedule_is_faculty_live((string) $liveAtRaw, null, $r);
$goLiveDisabledReason = (string) ($liveWindow['reason'] ?? '');
if ($goLiveDisabledReason === '') {
    $goLiveDisabledReason = 'Go live is only available during the scheduled class time.';
}
?>
<form method="post" class="mt-2">
    <input type="hidden" name="schedule_id" value="<?= (int) $r['id'] ?>">
    <?php if ($wasMarkedLive): ?>
        <input type="hidden" name="action" value="end_live">
        <button type="submit" class="btn btn-sm btn-outline-danger"><?= app_tooltip_attr('Stops the LIVE
indicator for this class on the weekly view. Use this when class ends.') ?><i class="fa-solid fa-stop me-1"></i>End live</button>
    <?php if ($isLiveNow): ?>
        <span class="badge bg-danger live-pulse-badge ms-1">LIVE</span>
    <?php else: ?>
        <span class="small text-muted ms-1">Class time ended – clear live status</span>
    <?php endif; ?>
    <?php else: ?>
        <input type="hidden" name="action" value="go_live">
        <button type="submit" class="btn btn-sm btn-success"><?= $isWithinClassTime ? '' : ' disabled' ?><?=
app_tooltip_attr($isWithinClassTime ? 'Marks this section as live so deans and students see the LIVE badge on the weekly schedule.' :
$goLiveDisabledReason) ?><i class="fa-solid fa-broadcast-tower me-1"></i>Go live</button>
        <?php if ($isWithinClassTime): ?>
            <span class="small text-muted ms-1">Deans see LIVE on weekly view</span>
        <?php else: ?>
            <span class="small text-muted ms-1 d-block mt-1"><?= htmlspecialchars($goLiveDisabledReason) ?>
        </span>
    <?php endif; ?>
    <?php endif; ?>
</form>
<?php endif; ?>
</td>
<?php endif; ?>
<td data-label="Term"><?= htmlspecialchars($r['semester']) ?> / <?= htmlspecialchars($r['school_year']) ?></td>
<td data-label="Requests" style="min-width: 280px;">
    <form method="post" class="d-flex flex-column gap-2 mb-3">
        <input type="hidden" name="schedule_id" value="<?= (int) $r['id'] ?>">
        <input type="hidden" name="request_type" value="change">
        <label class="visually-hidden" for="sched-req-<?= (int) $r['id'] ?>">Reason for schedule change</label>
        <input id="sched-req-<?= (int) $r['id'] ?>" type="text" name="message" class="form-control form-control-sm"
placeholder="Reason for change" required autocomplete="off">
        <button type="submit" class="btn btn-sm btn-outline-primary rounded-pill align-self-start"><?=
app_tooltip_attr('Sends a schedule change request to ' . $scheduleReviewer . ' with the reason you typed.') ?>>Request change</button>
    </form>
    <?php if ($hasMakeupSupport && (int) ($r['is_makeup'] ?? 0) !== 1): ?>
        <form method="post" class="d-flex flex-column gap-2 border-top pt-2">
            <input type="hidden" name="schedule_id" value="<?= (int) $r['id'] ?>">
            <input type="hidden" name="request_type" value="makeup">
            <div class="small fw-semibold text-muted">Makeup class</div>
            <label class="visually-hidden" for="makeup-day-<?= (int) $r['id'] ?>">Makeup day</label>
            <select id="makeup-day-<?= (int) $r['id'] ?>" name="makeup_day" class="form-select form-select-sm" required>
                <option value="">Day</option>
                <?php foreach (schedule_days_list() as $dayName): ?>
                    <option value="<?= htmlspecialchars($dayName) ?>"><?= htmlspecialchars($dayName) ?></option>
                <?php endforeach; ?>
            </select>
            <div class="d-flex gap-1">
                <label class="visually-hidden" for="makeup-start-<?= (int) $r['id'] ?>">Start time</label>
                <input id="makeup-start-<?= (int) $r['id'] ?>" type="time" name="makeup_start_time" class="form-control
form-control-sm" value="<?= htmlspecialchars(substr((string) $r['start_time'], 0, 5)) ?>" required>
                <label class="visually-hidden" for="makeup-end-<?= (int) $r['id'] ?>">End time</label>
                <input id="makeup-end-<?= (int) $r['id'] ?>" type="time" name="makeup_end_time" class="form-control
form-control-sm" value="<?= htmlspecialchars(substr((string) $r['end_time'], 0, 5)) ?>" required>
            </div>
            <label class="visually-hidden" for="makeup-room-<?= (int) $r['id'] ?>">Room</label>
            <select id="makeup-room-<?= (int) $r['id'] ?>" name="makeup_room_id" class="form-select form-select-sm"
required>
                <option value="">Room</option>
                <?php foreach ($makeupRooms as $room): ?>
                    <option value="<?= (int) $room['id'] ?>"><?= (int) $room['id'] === (int) $r['room_id'] ? ' selected'
: '' ?><?= htmlspecialchars((string) $room['room_code']) ?></option>
                <?php endforeach; ?>
            </select>
            <?php if ($makeupRooms === []): ?>
                <div class="small text-danger">No rooms available. Ask your dean to add rooms first.</div>
            <?php endif; ?>

```

```

                <label class="visually-hidden" for="makeup-msg-= (int) $r['id'] ?&gt;"&gt;Makeup reason&lt;/label&gt;
                &lt;input id="makeup-msg-<?= (int) $r['id'] ?&gt;" type="text" name="message" class="form-control form-control-sm"
placeholder="Reason for makeup" required autocomplete="off"&gt;
                &lt;button type="submit" class="btn btn-sm btn-warning rounded-pill align-self-start"&lt;?= $makeupRooms === [] ?
' disabled' : '' ?&gt;&lt;?= app_tooltip_attr('Requests a temporary makeup slot. ' . ucfirst($scheduleReviewer) . ' reviews it and, if approved,
adds it to the weekly schedule. Delete the makeup row after the class is held.') ?&gt;&gt;Request makeup&lt;/button&gt;
            &lt;/form&gt;
        &lt;?php endif; ?&gt;
    &lt;/td&gt;
&lt;/tr&gt;
&lt;?php endforeach; ?&gt;
&lt;/tbody&gt;
&lt;/table&gt;
&lt;/div&gt;
&lt;/div&gt;

&lt;?php if ($makeupAckDraft !== null): ?&gt;
    &lt;?php
    $draftId = (int) ($makeupAckDraft['schedule_id'] ?? 0);
    $draftDay = (string) ($makeupAckDraft['makeup_day'] ?? '');
    $draftStart = (string) ($makeupAckDraft['makeup_start_time'] ?? '');
    $draftEnd = (string) ($makeupAckDraft['makeup_end_time'] ?? '');
    $draftRoom = (int) ($makeupAckDraft['makeup_room_id'] ?? 0);
    $draftMsg = (string) ($makeupAckDraft['message'] ?? '');
    ?&gt;
    &lt;div class="modal fade" id="makeupAckConflictsModal" tabindex="-1" aria-labelledby="makeupAckConflictsModalLabel" aria-hidden="true"
data-bs-backdrop="static"&gt;
        &lt;div class="modal-dialog modal-dialog-centered"&gt;
            &lt;div class="modal-content border-warning"&gt;
                &lt;div class="modal-header bg-warning-subtle"&gt;
                    &lt;h5 class="modal-title text-warning-emphasis" id="makeupAckConflictsModalLabel"&gt;
                        &lt;i class="fa-solid fa-triangle-exclamation me-2"&gt;&lt;/i&gt;Warning – Makeup conflicts with faculty or room schedule
                    &lt;/h5&gt;
                    &lt;button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"&gt;&lt;/button&gt;
                &lt;/div&gt;
                &lt;div class="modal-body"&gt;
                    &lt;p class="mb-2"&gt;This makeup slot conflicts with an existing &lt;strong&gt;faculty&lt;/strong&gt; or &lt;strong&gt;room&lt;/strong&gt; schedule:</p
                    <ul class="mb-3">
                        <?php foreach ($makeupWarningPopup as $w): ?>
                            <li><?= htmlspecialchars($w) ?></li>
                        <?php endforeach; ?>
                    </ul>
                    <p class="mb-0 small text-muted">Your dean will still review the request. Only continue if the overlap is intentional.
                </div>
                <div class="modal-footer">
                    <button type="button" class="btn btn-outline-secondary" data-bs-dismiss="modal">Cancel</button>
                    <form method="post" class="d-inline">
                        <input type="hidden" name="schedule_id" value="<?= $draftId ?>">
                        <input type="hidden" name="request_type" value="makeup">
                        <input type="hidden" name="makeup_day" value="<?= htmlspecialchars($draftDay) ?>">
                        <input type="hidden" name="makeup_start_time" value="<?= htmlspecialchars($draftStart) ?>">
                        <input type="hidden" name="makeup_end_time" value="<?= htmlspecialchars($draftEnd) ?>">
                        <input type="hidden" name="makeup_room_id" value="<?= $draftRoom ?>">
                        <input type="hidden" name="message" value="<?= htmlspecialchars($draftMsg) ?>">
                        <input type="hidden" name="ack_conflicts" value="1">
                        <button type="submit" class="btn btn-warning">Submit anyway</button>
                    </form>
                </div>
            </div>
        </div>
    </div>
</div>
<script>
document.addEventListener('DOMContentLoaded', function () {
    var ackEl = document.getElementById('makeupAckConflictsModal');
    if (!ackEl) return;
    if (typeof bootstrap !== 'undefined' && bootstrap.Modal) {
        bootstrap.Modal.getOrCreateInstance(ackEl).show();
        return;
    }
    var items = [];
    ackEl.querySelectorAll('.modal-body li').forEach(function (li) {
        items.push(li.textContent.trim());
    });
    if (window.confirm('Warning – Makeup conflicts with faculty or room schedule:\n\n- ' + items.join('\n- ') + '\n\nSubmit anyway?')) {
        var form = ackEl.querySelector('form');
        if (form) form.submit();
    }
});

```

```

</script>
<?php elseif ($makeupWarningPopup != []): ?>
    <?php
        render_schedule_errors_warning_popup(
            $makeupWarningPopup,
            'Warning - Makeup conflicts with faculty or room schedule',
            'OK'
        );
    ?>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: faculty_student_list.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['faculty', 'program_chair', 'dean', 'gened']);

$facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($facultyId < 1) {
    $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
    $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
}
if ($facultyId < 1 && in_array($_SESSION['role'] ?? '', ['program_chair', 'dean', 'gened'], true)) {
    $facultyId = ensure_faculty_profile_for_teaching_role($userId) ?? 0;
    if ($facultyId > 0) {
        $_SESSION['faculty_id'] = $facultyId;
    }
}
if ($facultyId < 1) {
    exit('Faculty profile not linked to this account. Ask your dean to create/link your faculty profile.');
```

```

        (SELECT COUNT(*) FROM classroom_enrollments ce WHERE ce.classroom_id = oc.id) AS enrolled_count
        {$attendanceSelect}
    FROM online_classrooms oc
    INNER JOIN schedules s ON s.id = oc.schedule_id
    INNER JOIN courses c ON c.id = oc.course_id
    WHERE oc.faculty_id = ? AND s.faculty_id = ?
    ORDER BY s.school_year DESC, s.semester, c.course_code, oc.title"
);
$st->execute([$facultyId, $facultyId]);
$classrooms = $st->fetchAll();

foreach ($classrooms as $row) {
    $totals['enrolled'] += (int) ($row['enrolled_count'] ?? 0);
    $totals['present'] += (int) ($row['present_count'] ?? 0);
    $totals['absent'] += (int) ($row['absent_count'] ?? 0);
    $totals['sessions'] += (int) ($row['session_count'] ?? 0);
}

if ($classroomId > 0) {
    foreach ($classrooms as $row) {
        if ((int) $row['id'] === $classroomId) {
            $classroom = $row;
            break;
        }
    }
}

if ($classroom === null) {
    http_response_code(404);
    exit('Classroom not found or you do not have access to it.');
```

```

}

if ($hasAttendance) {
    $st = db()->prepare(
        "SELECT cs.id AS student_id, cs.student_number, cs.full_name, cs.email,
        COALESCE(SUM(ar.status = 'present'), 0) AS present_count,
        COALESCE(SUM(ar.status = 'absent'), 0) AS absent_count
        FROM classroom_enrollments ce
        INNER JOIN classroom_students cs ON cs.id = ce.student_id
        LEFT JOIN classroom_attendance_sessions cas
            ON cas.classroom_id = ce.classroom_id
        LEFT JOIN classroom_attendance_records ar
            ON ar.session_id = cas.id AND ar.student_id = cs.id
        WHERE ce.classroom_id = ?
        GROUP BY cs.id, cs.student_number, cs.full_name, cs.email
        ORDER BY cs.full_name ASC"
    );
} else {
    $st = db()->prepare(
        'SELECT cs.id AS student_id, cs.student_number, cs.full_name, cs.email,
        0 AS present_count, 0 AS absent_count
        FROM classroom_enrollments ce
        INNER JOIN classroom_students cs ON cs.id = ce.student_id
        WHERE ce.classroom_id = ?
        ORDER BY cs.full_name ASC'
    );
}
$st->execute([$classroomId]);
$students = $st->fetchAll();

if ($search !== '') {
    $needle = mb_strtolower($search);
    $students = array_values(array_filter(
        $students,
        static function (array $row) use ($needle): bool {
            $haystack = mb_strtolower(
                (string) $row['full_name'] . ' ' .
                (string) $row['student_number'] . ' ' .
                (string) ($row['email'] ?? '')
            );
            return str_contains($haystack, $needle);
        }
    ));
}

}

}

$pageTitle = $classroom
    ? ('Student list . ' . (string) $classroom['course_code'])
    : 'Student list';
require_once __DIR__ . '/includes/header.php';

```

?>

```
<style>
.fsl-hub {
  background: #f4f7fc;
  font-family: 'Inter', sans-serif;
  color: #1a2c3e;
  line-height: 1.4;
  padding: 2rem 1.5rem;
  border-radius: 22px;
}
.fsl-hub .dashboard-container {
  max-width: 1200px;
  margin: 0 auto;
}
.fsl-hub .uni-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-end;
  flex-wrap: wrap;
  gap: 1rem;
  margin-bottom: 1.5rem;
  border-bottom: 2px solid #e2e8f0;
  padding-bottom: 1rem;
}
.fsl-hub .brand h1 {
  font-size: 1.75rem;
  font-weight: 700;
  background: linear-gradient(135deg, #1e4a6e, #2c7a4d);
  background-clip: text;
  -webkit-background-clip: text;
  color: transparent;
  letter-spacing: -0.3px;
  margin: 0;
}
.fsl-hub .brand p {
  font-size: 0.9rem;
  color: #2c5a6e;
  font-weight: 500;
  margin: 0.25rem 0 0;
}
.fsl-stats {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 1rem;
  margin-bottom: 1.5rem;
}
.fsl-stat {
  background: #fff;
  border-radius: 16px;
  padding: 1rem 1.15rem;
  box-shadow: 0 8px 18px rgba(0, 0, 0, 0.04), 0 0 0 1px rgba(0, 0, 0, 0.02);
}
.fsl-stat .label {
  font-size: 0.72rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: #64748b;
  margin-bottom: 0.35rem;
}
.fsl-stat .value {
  font-size: 1.55rem;
  font-weight: 700;
  color: #0f172a;
  line-height: 1.1;
}
.fsl-stat.is-present .value { color: #15803d; }
.fsl-stat.is-absent .value { color: #b91c1c; }
.fsl-card {
  background: #fff;
  border-radius: 22px;
  box-shadow: 0 12px 28px rgba(0, 0, 0, 0.05), 0 0 0 1px rgba(0, 0, 0, 0.02);
  overflow: hidden;
  margin-bottom: 1.25rem;
}
.fsl-card-header {
  padding: 1.1rem 1.5rem;
  border-bottom: 1px solid #e8eef3;
  display: flex;
  justify-content: space-between;
```



```

    align-items: center;
    gap: 1rem;
    flex-wrap: wrap;
}
.fsl-card-header h2 {
    font-size: 1.05rem;
    font-weight: 700;
    margin: 0;
    color: #1e3a4c;
    display: flex;
    align-items: center;
    gap: 0.55rem;
}
.fsl-card-body { padding: 0; }
.fsl-table {
    width: 100%;
    border-collapse: collapse;
}
.fsl-table th {
    text-align: left;
    font-size: 0.72rem;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    color: #64748b;
    font-weight: 700;
    padding: 0.85rem 1.25rem;
    border-bottom: 1px solid #e8eef3;
    background: #f8fafc;
    white-space: nowrap;
}
.fsl-table td {
    padding: 0.95rem 1.25rem;
    border-bottom: 1px solid #f1f5f9;
    vertical-align: middle;
    font-size: 0.9rem;
}
.fsl-table tbody tr:hover { background: #f8fbfd; }
.fsl-table tbody tr:last-child td { border-bottom: none; }
.fsl-course-code {
    font-weight: 700;
    color: #1e4a6e;
}
.fsl-muted {
    color: #64748b;
    font-size: 0.8rem;
}
.fsl-pill {
    display: inline-flex;
    align-items: center;
    gap: 0.3rem;
    border-radius: 999px;
    padding: 0.25rem 0.65rem;
    font-size: 0.78rem;
    font-weight: 700;
    white-space: nowrap;
}
.fsl-pill.enrolled { background: #e8f0fe; color: #1a56db; }
.fsl-pill.present { background: #dcfce7; color: #15803d; }
.fsl-pill.absent { background: #fee2e2; color: #b91c1c; }
.fsl-actions {
    display: flex;
    gap: 0.45rem;
    flex-wrap: wrap;
}
.fsl-btn {
    display: inline-flex;
    align-items: center;
    gap: 0.4rem;
    border-radius: 999px;
    padding: 0.4rem 0.85rem;
    font-size: 0.8rem;
    font-weight: 600;
    text-decoration: none;
    border: 1px solid #cbdde6;
    background: #fff;
    color: #1f5e6e;
}
.fsl-btn:hover {
    background: #eef3f7;
    border-color: #9fc1cf;
}

```

```

        color: #1f5e6e;
    }
    .fsl-btn.primary {
        background: #2c7a4d;
        border-color: #2c7a4d;
        color: #fff;
    }
    .fsl-btn.primary:hover {
        background: #246640;
        border-color: #246640;
        color: #fff;
    }
    .fsl-toolbar {
        display: flex;
        gap: 0.75rem;
        flex-wrap: wrap;
        align-items: center;
    }
    .fsl-search {
        border: 1px solid #cbdde6;
        border-radius: 999px;
        padding: 0.45rem 0.9rem;
        min-width: 220px;
        font-size: 0.875rem;
    }
    .fsl-search:focus {
        outline: none;
        border-color: #2c7a4d;
        box-shadow: 0 0 0 3px rgba(44, 122, 77, 0.15);
    }
    .fsl-empty {
        text-align: center;
        padding: 2.5rem 1.5rem;
        color: #8ba5b4;
        font-style: italic;
    }
    .fsl-back {
        margin-bottom: 1rem;
    }
}
@media (max-width: 900px) {
    .fsl-stats { grid-template-columns: repeat(2, minmax(0, 1fr)); }
    .fsl-hub { padding: 1.2rem; }
}
@media (max-width: 560px) {
    .fsl-stats { grid-template-columns: 1fr; }
}
</style>

<div class="fsl-hub">
    <div class="dashboard-container">
        <div class="uni-header">
            <div class="brand">
                <h1><i class="fa-solid fa-users" style="color:#2c7a4d; margin-right: 8px;"></i>Student List</h1>
                <p>
                    <?php if ($classroom): ?>
                        <? = htmlspecialchars((string) $classroom['course_code']) ?> -
                        <? = htmlspecialchars((string) $classroom['course_name']) ?>
                    <?php else: ?>
                        Courses and registered students with attendance totals
                    <?php endif; ?>
                </p>
            </div>
            <div class="fsl-actions">
                <?php if ($classroom): ?>
                    <a href="faculty_student_list.php" class="fsl-btn"<? = app_tooltip_attr('Returns to the course list for all of your
online classrooms.') ?>>
                        <i class="fa-solid fa-arrow-left"></i> All courses
                    </a>
                    <a href="faculty_classroom_attendance.php?id=<? = (int) $classroom['id'] ?>" class="fsl-btn"<? = app_tooltip_attr('Opens
attendance marking for this classroom.') ?>>
                        <i class="fa-solid fa-clipboard-user"></i> Attendance
                    </a>
                    <a href="faculty_classroom.php?id=<? = (int) $classroom['id'] ?>" class="fsl-btn"<? = app_tooltip_attr('Opens the
classroom management page.') ?>>
                        <i class="fa-solid fa-gear"></i> Manage class
                    </a>
                <?php else: ?>
                    <a href="faculty_classrooms.php" class="fsl-btn"<? = app_tooltip_attr('Opens your online classrooms hub.') ?>>
                        <i class="fa-solid fa-chalkboard"></i> My classrooms
                    </a>
                </?php ?>
            </div>
        </div>
    </div>
</div>

```

```

        <?php endif; ?>
    </div>
</div>

<?php if ($missingTables !== []): ?>
    <div class="alert alert-warning mb-0">
        Classroom enrollment features are not installed yet. Run
        <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this page.
    </div>
<?php elseif ($classroom): ?>
    <?php
    $detailEnrolled = count($students);
    if ($search === '') {
        $detailEnrolled = (int) ($classroom['enrolled_count'] ?? count($students));
    }
    $detailPresent = (int) ($classroom['present_count'] ?? 0);
    $detailAbsent = (int) ($classroom['absent_count'] ?? 0);
    $detailSessions = (int) ($classroom['session_count'] ?? 0);
    ?>
    <div class="fsl-stats">
        <div class="fsl-stat">
            <div class="label">Registered</div>
            <div class="value"><?= (int) $detailEnrolled ?></div>
        </div>
        <div class="fsl-stat is-present">
            <div class="label">Present marks</div>
            <div class="value"><?= $detailPresent ?></div>
        </div>
        <div class="fsl-stat is-absent">
            <div class="label">Absent marks</div>
            <div class="value"><?= $detailAbsent ?></div>
        </div>
        <div class="fsl-stat">
            <div class="label">Sessions</div>
            <div class="value"><?= $detailSessions ?></div>
        </div>
    </div>

    <div class="fsl-card">
        <div class="fsl-card-header">
            <h2><i class="fa-solid fa-user-graduate" style="color:#2c7a4d;"></i> Registered students</h2>
            <form method="get" class="fsl-toolbar">
                <input type="hidden" name="id" value="<?= (int) $classroom['id'] ?>">
                <input
                    type="search"
                    name="q"
                    class="fsl-search"
                    value="<?= htmlspecialchars($search) ?>"
                    placeholder="Search name or student no."
                    <?= app_tooltip_attr('Filters the roster by student name, number, or email.') ?>
                >
                <button type="submit" class="fsl-btn primary">
                    <i class="fa-solid fa-magnifying-glass"></i> Search
                </button>
                <?php if ($search !== ''): ?>
                    <a href="faculty_student_list.php?id=<?= (int) $classroom['id'] ?>" class="fsl-btn">Clear</a>
                <?php endif; ?>
            </form>
        </div>
        <div class="fsl-card-body">
            <div class="table-responsive">
                <table class="fsl-table">
                    <thead>
                        <tr>
                            <th>#</th>
                            <th>Student</th>
                            <th>Student no.</th>
                            <th>Present</th>
                            <th>Absent</th>
                        </tr>
                    </thead>
                    <tbody>
                        <?php if ($students === []): ?>
                            <tr>
                                <td colspan="5" class="fsl-empty">
                                    <?= $search !== ''
                                        ? 'No students match your search.'
                                        : 'No students are registered in this course yet. Share the classroom join code so they can
enroll.' ?>
                                </td>
                            </tr>
                        </tbody>
                    </table>
                </div>
            </div>
        </div>
    </div>

```

```

        </tr>
    <?php else: ?>
        <?php foreach ($students as $i => $row): ?>
            <tr>
                <td><?=$i + 1 ?></td>
                <td>
                    <div style="font-weight:600;"><?=$htmlspecialchars((string) $row['full_name']) ?></div>
                    <?php if (trim((string) ($row['email'] ?? '')) != ''): ?>
                        <div class="fsl-muted"><?=$htmlspecialchars((string) $row['email']) ?></div>
                    <?php endif; ?>
                </td>
                <td><?=$htmlspecialchars((string) ($row['student_number'] != '' ? $row['student_number'] : '-')) ?>
            </td>
            <td>
                <span class="fsl-pill present">
                    <i class="fa-solid fa-check"></i>
                    <?=(int) $row['present_count'] ?>
                </span>
            </td>
            <td>
                <span class="fsl-pill absent">
                    <i class="fa-solid fa-xmark"></i>
                    <?=(int) $row['absent_count'] ?>
                </span>
            </td>
        </tr>
    <?php endforeach; ?>
<?php endif; ?>
</tbody>
</table>
</div>
</div>
</div>
<?php else: ?>
    <div class="fsl-stats">
        <div class="fsl-stat">
            <div class="label">Courses</div>
            <div class="value"><?=$count($classrooms) ?></div>
        </div>
        <div class="fsl-stat">
            <div class="label">Registered students</div>
            <div class="value"><?=(int) $totals['enrolled'] ?></div>
        </div>
        <div class="fsl-stat is-present">
            <div class="label">Present marks</div>
            <div class="value"><?=(int) $totals['present'] ?></div>
        </div>
        <div class="fsl-stat is-absent">
            <div class="label">Absent marks</div>
            <div class="value"><?=(int) $totals['absent'] ?></div>
        </div>
    </div>
</div>

<div class="fsl-card">
    <div class="fsl-card-header">
        <h2><i class="fa-solid fa-book-open" style="color:#2c7a4d;"></i> Your courses</h2>
    </div>
    <div class="fsl-card-body">
        <div class="table-responsive">
            <table class="fsl-table">
                <thead>
                    <tr>
                        <th>Course</th>
                        <th>Term</th>
                        <th>Registered</th>
                        <th>Present</th>
                        <th>Absent</th>
                        <th></th>
                    </tr>
                </thead>
                <tbody>
                    <?php if ($classrooms == []): ?>
                        <tr>
                            <td colspan="6" class="fsl-empty">
                                No online classrooms yet.
                                <a href="faculty_classrooms.php">Create a classroom</a> for an assigned course first.
                            </td>
                        </tr>
                    <?php else: ?>
                        <?php foreach ($classrooms as $row): ?>

```

```

<?php
$term = trim((string) $row['semester'] . ' / ' . (string) $row['school_year'], ' / ');
$dayLine = str_replace(',', ' ', (string) $row['day_of_week']);
$timeLine = substr((string) $row['start_time'], 0, 5) . '-' . substr((string) $row['end_time'], 0, 5);
?>
<tr>
    <td>
        <div class="fsl-course-code"><?> htmlspecialchars((string) $row['course_code']) ?></div>
        <div><?> htmlspecialchars((string) $row['course_name']) ?></div>
        <div class="fsl-muted"><?> htmlspecialchars((string) $row['title']) ?></div>
    </td>
    <td>
        <div><?> htmlspecialchars($term) ?></div>
        <div class="fsl-muted"><?> htmlspecialchars($dayLine) ?></div>
        <div class="fsl-muted"><?> htmlspecialchars($timeLine) ?></div>
    </td>
    <td>
        <span class="fsl-pill enrolled">
            <i class="fa-solid fa-users"></i>
            <?> (int) $row['enrolled_count'] ?>
        </span>
    </td>
    <td>
        <span class="fsl-pill present">
            <i class="fa-solid fa-check"></i>
            <?> (int) $row['present_count'] ?>
        </span>
    </td>
    <td>
        <span class="fsl-pill absent">
            <i class="fa-solid fa-xmark"></i>
            <?> (int) $row['absent_count'] ?>
        </span>
    </td>
    <td>
        <div class="fsl-actions">
            <a
                href="faculty_student_list.php?id=<?> (int) $row['id'] ?>"
                class="fsl-btn primary"
                <?> app_tooltip_attr('Shows every student registered in this course with present and
absent totals.') ?>
            >
                <i class="fa-solid fa-list"></i> View students
            </a>
        </div>
    </td>
</tr>
<?php endforeach; ?>
<?php endif; ?>
</tbody>
</table>
</div>
</div>
</div>
<?php endif; ?>
</div>
</div>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

-----
FILE: faculty_teaching_load.php
-----

<?php
declare(strict_types=1);

/**
 * Faculty teaching load (units) from scheduled offerings.
 * Scoped: admin (optional college/faculty filters), dean (college), program chair (program), gened (GE courses), faculty (own load only).
 */
require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['admin', 'dean', 'program_chair', 'gened', 'faculty']);

$role = (string) ($_SESSION['role'] ?? '');
$collegeId = current_college_id();
$programScope = null;
$facultySelfId = 0;

```

```

$facultySelfName = '';

if ($role === 'faculty') {
    $facultySelfId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
    $userId = (int) ($_SESSION['user_id'] ?? 0);
    if ($facultySelfId < 1) {
        $facultySelfId = resolve_faculty_id_for_user($userId) ?? 0;
        $_SESSION['faculty_id'] = $facultySelfId > 0 ? $facultySelfId : null;
    }
    if ($facultySelfId < 1) {
        exit('Faculty profile not linked to this account. Ask your dean to create/link your faculty profile.');
```

```

        {$courseCatalogBlockSelect}
        {$sessionKindSelect}
        {$targetSelect}
        FROM schedules s
        INNER JOIN faculty f ON f.id = s.faculty_id
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN rooms r ON r.id = s.room_id
        LEFT JOIN users uc ON uc.id = s.created_by
        LEFT JOIN colleges col ON col.id = s.college_id
        {$targetJoin}
        WHERE 1=1";
$params = [];

if ($role === 'admin') {
    if ($adminCollegeId > 0) {
        $sql .= ' AND s.college_id = ?';
        $params[] = $adminCollegeId;
    }
} elseif (is_dean() && $collegeId) {
    $hasIsGenedCourseCol = db_column_exists('courses', 'is_gened');
    $sql .= dean_schedule_scope_sql($collegeId, $hasIsGenedCourseCol, $params, $hasGeTargetsTable);
} elseif ($programScope !== null && $collegeId) {
    $sql .= program_chair_schedule_scope_sql($collegeId, $programScope, $hasGeTargetsTable, $params);
} elseif ($role === 'faculty' && $facultySelfId > 0) {
    $sql .= ' AND s.faculty_id = ?';
    $params[] = $facultySelfId;
}

if (is_gened()) {
    if ($hasIsGenedCourse) {
        $sql .= ' AND COALESCE(c.is_gened, 0) = 1';
    } else {
        $sql .= ' AND COALESCE(uc.role, "") = "gened"';
    }
}

if (is_gened() && $hasIsGenedFaculty && $geFacultyFilter === '1') {
    $sql .= ' AND COALESCE(f.is_gened, 0) = 1';
} elseif (is_gened() && $hasIsGenedFaculty && $geFacultyFilter === '0') {
    $sql .= ' AND COALESCE(f.is_gened, 0) = 0';
}

if (is_program_chair()) {
    if ($hasIsGenedCourse) {
        $sql .= ' AND COALESCE(c.is_gened, 0) = 0';
    } else {
        $sql .= ' AND COALESCE(uc.role, "") != "gened"';
    }
}

if ($dept !== '' && !is_program_chair()) {
    if ($hasGeTargetsTable) {
        $sql .= ' AND (f.department = ? OR c.department = ? OR gst.program_name = ?)';
        $params[] = $dept;
        $params[] = $dept;
        $params[] = $dept;
    } else {
        $sql .= ' AND (f.department = ? OR c.department = ?)';
        $params[] = $dept;
        $params[] = $dept;
    }
}

if ($sem !== '') {
    $sql .= ' AND s.semester = ?';
    $params[] = $sem;
}

if ($sy !== '') {
    $sql .= ' AND s.school_year = ?';
    $params[] = $sy;
}

if (($role === 'admin' || is_dean() || is_program_chair() || is_gened()) && $facultySearch !== '') {
    $sql .= ' AND (f.full_name LIKE ? OR f.faculty_id LIKE ?)';
    $params[] = '%' . $facultySearch . '%';
    $params[] = '%' . $facultySearch . '%';
}

$sql .= ' ORDER BY s.school_year DESC, s.semester, col.college_code, f.full_name, s.start_time';

$stmt = db()->prepare($sql);
$stmt->execute($params);

```

```

$rows = $stmt->fetchAll();

$sessionHoursBetween = static function (string $start, string $end): float {
    $rawS = substr($start, 0, 8);
    $rawE = substr($end, 0, 8);
    $ds = DateTime::createFromFormat('H:i:s', $rawS) ?: DateTime::createFromFormat('H:i', substr($start, 0, 5));
    $de = DateTime::createFromFormat('H:i:s', $rawE) ?: DateTime::createFromFormat('H:i', substr($end, 0, 5));
    if (!$ds || !$de) {
        return 0.0;
    }
    $secs = $de->getTimestamp() - $ds->getTimestamp();
    if ($secs <= 0) {
        return 0.0;
    }
    return round($secs / 3600, 2);
};

$scheduleListGroup = [];
foreach ($rows as $r) {
    $cid = (int) ($r['sched_college_id'] ?? 0);
    $k = (int) $r['faculty_id'] . '|' . (int) $r['course_id'] . '|' . (string) $r['semester'] . '|' . (string) $r['school_year'] . '|' .
    $cid;
    $scheduleListGroup[$k][] = $r;
}
foreach ($scheduleListGroup as &$g) {
    usort(
        $g,
        static function (array $a, array $b): int {
            $aLab = schedule_session_is_laboratory(
                (string) ($a['start_time'] ?? ''),
                (string) ($a['end_time'] ?? ''),
                (string) ($a['room_type'] ?? ''),
                isset($a['session_kind']) ? (string) $a['session_kind'] : null,
                array_key_exists('is_laboratory', $a) ? ((int) ($a['is_laboratory'] ?? 0) === 1) : null
            );
            $bLab = schedule_session_is_laboratory(
                (string) ($b['start_time'] ?? ''),
                (string) ($b['end_time'] ?? ''),
                (string) ($b['room_type'] ?? ''),
                isset($b['session_kind']) ? (string) $b['session_kind'] : null,
                array_key_exists('is_laboratory', $b) ? ((int) ($b['is_laboratory'] ?? 0) === 1) : null
            );
            if ($aLab !== $bLab) {
                return $aLab ? 1 : -1;
            }
            return strcmp((string) ($a['id'] ?? ''), (string) ($b['id'] ?? ''));
        }
    );
}
unset($g);

$courseOfferingUnits = static function (array $r) use ($hasLectureUnits, $hasLaboratoryUnits): float {
    if ($hasLectureUnits && $hasLaboratoryUnits) {
        return (float) ($r['lecture_units'] ?? 0) + (float) ($r['laboratory_units'] ?? 0);
    }

    return (float) ($r['course_units_total'] ?? 0);
};

$formatLoadUnits = static function (float $u): string {
    $s = number_format($u, 1, '.', '');
    return rtrim(rtrim($s, '0'), '.');
};

$formatTime12h = static function (?string $time): string {
    $raw = substr((string) $time, 0, 5);
    $dt = DateTime::createFromFormat('H:i', $raw);
    return $dt ? $dt->format('g:i A') : $raw;
};

$resolveProgramYearSection = static function (array $r) use (
    $hasGeTargetsTable,
    $hasCourseYearLevel,
    $hasCourseSection
): array {
    $programShow = trim((string) ($r['sched_program'] ?? ''));
    $yearShow = trim((string) ($r['sched_year_level'] ?? ''));
    $sectionShow = trim((string) ($r['sched_section'] ?? ''));
    if ($hasGeTargetsTable) {
        if ($programShow === '') {

```



```

        $programShow = trim((string) ($r['target_program'] ?? ''));
    }
    if ($yearShow === '') {
        $yearShow = trim((string) ($r['target_year_level'] ?? ''));
    }
    if ($sectionShow === '') {
        $sectionShow = trim((string) ($r['target_section'] ?? ''));
    }
}
if ($programShow === '') {
    $programShow = trim((string) ($r['course_department'] ?? ''));
}
if ($yearShow === '' && $hasCourseYearLevel) {
    $yearShow = trim((string) ($r['course_year_level'] ?? ''));
}
if ($sectionShow === '' && $hasCourseSection) {
    $sectionShow = trim((string) ($r['course_section'] ?? ''));
}

return [$programShow, $yearShow, $sectionShow];
};

$mergeScheduleGroupForDetail = static function (array $group) use (
    $formatTime12h,
    $sessionHoursBetween,
    $formatLoadUnits,
    $resolveProgramYearSection,
    $hasLectureUnits,
    $hasLaboratoryUnits
): array {
    $timeParts = [];
    $dayParts = [];
    $lecHours = 0.0;
    $labHours = 0.0;
    $roomOrder = [];
    $roomSeen = [];
    foreach ($group as $r) {
        $h = $sessionHoursBetween((string) ($r['start_time'] ?? ''), (string) ($r['end_time'] ?? ''));
        $isLabSlot = schedule_session_is_laboratory(
            (string) ($r['start_time'] ?? ''),
            (string) ($r['end_time'] ?? ''),
            (string) ($r['room_type'] ?? ''),
            isset($r['session_kind']) ? (string) $r['session_kind'] : null,
            array_key_exists('is_laboratory', $r) ? ((int) ($r['is_laboratory'] ?? 0) === 1) : null
        );
        $slotTag = $isLabSlot ? 'LAB' : 'LEC';
        $timeParts[] = $formatTime12h((string) ($r['start_time'] ?? '')) . ' - ' . $formatTime12h((string) ($r['end_time'] ?? '')) . ' (' .
        $slotTag . ')';
        $dayParts[] = format_day_set_abbrev((string) ($r['day_of_week'] ?? ''));
        $dayCount = count(parse_day_set((string) ($r['day_of_week'] ?? '')));
        if ($dayCount < 1) {
            $dayCount = 1;
        }
        $weeklyH = $h * $dayCount;
        if ($isLabSlot) {
            $labHours += $weeklyH;
        } else {
            $lecHours += $weeklyH;
        }
        $rc = trim((string) ($r['room_code'] ?? ''));
        $rn = trim((string) ($r['room_name'] ?? ''));
        $roomLabel = $rc === '' && $rn === ''
            ? ''
            : ($rn !== '' && strtolower($rn, $rc) !== 0 ? $rc . ' - ' . $rn : ($rc !== '' ? $rc : $rn));
        if ($roomLabel !== '' && !isset($roomSeen[$roomLabel])) {
            $roomSeen[$roomLabel] = true;
            $roomOrder[] = $roomLabel;
        }
    }
    $timeDisplay = implode(' ', array_values(array_unique($timeParts)));
    $dayDisplay = implode(' / ', array_values(array_unique($dayParts)));
    $roomDisplay = $roomOrder ? implode(' ', $roomOrder) : '-';

    $r0 = $group[0];
    if ($hasLectureUnits && $hasLaboratoryUnits) {
        $lecU = (float) ($r0['lecture_units'] ?? 0);
        $lecCell = $lecU > 0 ? $formatLoadUnits($lecU) : '-';
    } else {
        $lecCell = $lecHours > 0 ? $formatLoadUnits($lecHours) : '-';
    }
}

```

```

$labCell = $labHours > 0 ? $formatLoadUnits($labHours) : '-';

$programShow = '';
$yearShow = '';
$sectionShow = '';
foreach ($group as $gr) {
    [$p, $y, $s] = $resolveProgramYearSection($gr);
    if ($programShow === '' && $p !== '') {
        $programShow = $p;
    }
    if ($yearShow === '' && $y !== '') {
        $yearShow = $y;
    }
    if ($sectionShow === '' && $s !== '') {
        $sectionShow = $s;
    }
    if ($programShow !== '' && $yearShow !== '' && $sectionShow !== '') {
        break;
    }
}
$cyParts = [];
if ($programShow !== '') {
    $cyParts[] = $programShow;
}
if ($yearShow !== '') {
    $cyParts[] = $yearShow;
}
$courseYearLabel = $cyParts ? implode(' / ', $cyParts) : '-';

$lecHoursN = ($hasLectureUnits && $hasLaboratoryUnits)
    ? (float) ($r0['lecture_units'] ?? 0)
    : $lecHours;
$labHoursN = $labHours;

return [
    'time_display' => $timeDisplay !== '' ? $timeDisplay : '-',
    'day_display' => $dayDisplay !== '' ? $dayDisplay : '-',
    'lec_hours_display' => $lecCell,
    'lab_hours_display' => $labCell,
    'lec_hours_n' => $lecHoursN,
    'lab_hours_n' => $labHoursN,
    'room_display' => $roomDisplay,
    'course_year_display' => $courseYearLabel,
    'section_display' => $sectionShow !== '' ? $sectionShow : '-',
];
};

$facultyLoadSummary = [];
foreach ($scheduleListGroups as $group) {
    $r0 = $group[0];
    $fid = (int) $r0['faculty_id'];
    $units = $courseOfferingUnits($r0);
    if (!isset($facultyLoadSummary[$fid])) {
        $facultyLoadSummary[$fid] = [
            'faculty_name' => (string) $r0['faculty_name'],
            'college_id' => (int) ($r0['sched_college_id'] ?? 0),
            'total_units' => 0.0,
            'offerings' => [],
        ];
    }
    $facultyLoadSummary[$fid]['total_units'] += $units;
    $merged = $mergeScheduleGroupForDetail($group);
    $facultyLoadSummary[$fid]['offerings'][] = [
        'course_code' => (string) $r0['course_code'],
        'course_name' => (string) $r0['course_name'],
        'units' => $units,
        'semester' => (string) $r0['semester'],
        'school_year' => (string) $r0['school_year'],
        'college_id' => (int) ($r0['sched_college_id'] ?? 0),
        'college_label' => trim((string) ($r0['college_code'] ?? '')) . ' - ' . (string) ($r0['college_name'] ?? ''),
        'time_display' => $merged['time_display'],
        'day_display' => $merged['day_display'],
        'lec_hours_display' => $merged['lec_hours_display'],
        'lab_hours_display' => $merged['lab_hours_display'],
        'lec_hours_n' => $merged['lec_hours_n'],
        'lab_hours_n' => $merged['lab_hours_n'],
        'room_display' => $merged['room_display'],
        'course_year_display' => $merged['course_year_display'],
        'section_display' => $merged['section_display'],
    ];
};

```

```

}
uasort(
    $facultyLoadSummary,
    static function (array $a, array $b): int {
        return strcmp($a['faculty_name'], $b['faculty_name']);
    }
);

$grandTotalUnits = 0.0;
foreach ($facultyLoadSummary as $fl) {
    $grandTotalUnits += $fl['total_units'];
}

/** Detail table footer: totals for units + lec/lab hours when exactly one faculty is in view. */
$detailTableTotals = null;
if (count($facultyLoadSummary) === 1) {
    $onlyFl = reset($facultyLoadSummary);
    $sumUnits = 0.0;
    $sumLecN = 0.0;
    $sumLabN = 0.0;
    foreach ($onlyFl['offerings'] as $do) {
        $sumUnits += (float) ($do['units'] ?? 0);
        $sumLecN += (float) ($do['lec_hours_n'] ?? 0);
        $sumLabN += (float) ($do['lab_hours_n'] ?? 0);
    }
    $detailTableTotals = [
        'faculty_name' => (string) ($onlyFl['faculty_name'] ?? ''),
        'sum_units' => $sumUnits,
        'sum_lec_n' => $sumLecN,
        'sum_lab_n' => $sumLabN,
        'sum_total_n' => $sumLecN + $sumLabN,
    ];
}

/** Single-faculty snapshot for Teaching Load Memorandum print (same condition as detail totals row). */
$tlmMemoFaculty = null;
if ($detailTableTotals !== null) {
    $tlmMemoFaculty = reset($facultyLoadSummary);
}
$tlmMemoSchoolYear = $sy !== '' ? $sy : '_____';
$tlmMemoSemester = $sem !== '' ? $sem : '_____';
$tlmMemoDeanName = '_____';
if ($tlmMemoFaculty !== null && ($tlmMemoFaculty['offerings'] ?? []) !== []) {
    $tlmFirst = $tlmMemoFaculty['offerings'][0];
    if ($sy === '' && trim((string) ($tlmFirst['school_year'] ?? '')) !== '') {
        $tlmMemoSchoolYear = (string) $tlmFirst['school_year'];
    }
    if ($sem === '' && trim((string) ($tlmFirst['semester'] ?? '')) !== '') {
        $tlmMemoSemester = (string) $tlmFirst['semester'];
    }
}
if ($tlmMemoFaculty !== null) {
    $tlmMemoCollegeId = (int) ($tlmMemoFaculty['college_id'] ?? 0);
    if ($tlmMemoCollegeId <= 0 && ($tlmMemoFaculty['offerings'] ?? []) !== []) {
        $tlmMemoCollegeId = (int) ($tlmMemoFaculty['offerings'][0]['college_id'] ?? 0);
    }
    if ($tlmMemoCollegeId > 0) {
        $deanStmt = db()->prepare(
            'SELECT u.full_name
             FROM colleges c
             LEFT JOIN users u ON u.id = c.dean_user_id
             WHERE c.id = ?
             LIMIT 1'
        );
        $deanStmt->execute([$tlmMemoCollegeId]);
        $tlmMemoDeanNameDb = trim((string) ($deanStmt->fetchColumn() ?? ''));
        if ($tlmMemoDeanNameDb === '') {
            $fallbackDeanStmt = db()->prepare(
                'SELECT full_name
                 FROM users
                 WHERE role = "dean" AND college_id = ?
                 ORDER BY is_active DESC, id ASC
                 LIMIT 1'
            );
            $fallbackDeanStmt->execute([$tlmMemoCollegeId]);
            $tlmMemoDeanNameDb = trim((string) ($fallbackDeanStmt->fetchColumn() ?? ''));
        }
        if ($tlmMemoDeanNameDb !== '') {
            $tlmMemoDeanName = $tlmMemoDeanNameDb;
        }
    }
}

```

```

    }
}
$tlmMemoVpaaName = '_____';
$tlmMemoVpaaNameDb = vpaa_signatory_full_name();
if ($tlmMemoVpaaNameDb != '') {
    $tlmMemoVpaaName = $tlmMemoVpaaNameDb;
}

$collegeFilterOptions = [];
if ($role === 'admin') {
    $collegeFilterOptions = db()->query(
        'SELECT id, college_code, college_name FROM colleges ORDER BY college_code'
    )->fetchAll();
}

if (is_program_chair() && $programScope !== null) {
    $depts = [$programScope];
} elseif (is_dean() && $collegeId) {
    $d = db()->prepare('SELECT DISTINCT department FROM faculty WHERE college_id=? AND department != "" ORDER BY department');
    $d->execute([$collegeId]);
    $depts = $d->fetchAll(PDO::FETCH_COLUMN);
} else {
    $depts = db()->query('SELECT DISTINCT department FROM faculty WHERE department != "" ORDER BY department')->fetchAll(PDO::FETCH_COLUMN);
}
}
$sems = db()->query('SELECT DISTINCT semester FROM schedules ORDER BY semester')->fetchAll(PDO::FETCH_COLUMN);
$years = db()->query('SELECT DISTINCT school_year FROM schedules ORDER BY school_year DESC')->fetchAll(PDO::FETCH_COLUMN);

$filterColClass = is_program_chair() ? 'col-md-2' : 'col-md-3';
$showCollegeColumn = $role === 'admin' && $adminCollegeId === 0;
$showFacultyColumn = $role !== 'faculty';
$isFacultySelfView = $role === 'faculty';

$ftlScriptName = str_replace('\\', '/', (string) ($_SERVER['SCRIPT_NAME'] ?? ''));
$ftlScriptDir = rtrim(dirname($ftlScriptName), '/');
$ftlMemoLogoWebPath = ($ftlScriptDir === '' || $ftlScriptDir === '.' || $ftlScriptDir === '/')
    ? '/assets/wpu-logo.png'
    : '/' . ltrim(preg_replace('#/+#', '/', $ftlScriptDir . '/assets/wpu-logo.png'), '/');
$ftlWpuLogoAbsUrl = (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') ? 'https' : 'http'
    . '://' . ($_SERVER['HTTP_HOST'] ?? 'localhost')
    . $ftlMemoLogoWebPath;

$pageTitle = $isFacultySelfView ? 'My teaching load (units)' : 'Faculty teaching load (units)';
$mainContainerClass = 'container py-4 py-md-5 app-main';
require_once __DIR__ . '/includes/header.php';
?>
<style>
.ftl-page .ftl-scope-note { font-size: 0.9rem; }
.ftl-page .ftl-filters {
    background: var(--bs-body-bg);
    border: 1px solid var(--bs-border-color);
    border-radius: 1rem;
    padding: 1rem;
}
.ftl-page .schedule-faculty-load {
    background: var(--bs-body-bg);
    border: 1px solid var(--bs-border-color);
    border-radius: 1rem;
    padding: 1rem 1.1rem;
}
.ftl-page .schedule-faculty-load h2 {
    font-size: 1rem;
    font-weight: 600;
    margin-bottom: 0.35rem;
}
.ftl-page .schedule-faculty-load-card {
    background: var(--bs-secondary-bg);
    border: 1px solid var(--bs-border-color);
    border-radius: 0.85rem;
    padding: 0.85rem 1rem;
    height: 100%;
}
.ftl-page .schedule-faculty-load-card .faculty-load-course-list {
    max-height: 280px;
    overflow-y: auto;
    margin-top: 0.5rem;
    padding-right: 4px;
}
.ftl-page .schedule-faculty-load-card .faculty-load-course-list li {
    border-color: var(--bs-border-color) !important;
}

```

```

/* Detail table: compact type, wrap long text, avoid cramped overlap */
.ftl-detail-wrap {
    font-size: 0.6875rem;
    line-height: 1.35;
    -webkit-font-smoothing: antialiased;
}
.ftl-detail-wrap .ftl-detail-table {
    font-family: system-ui, "Segoe UI", Roboto, "Helvetica Neue", Arial, "Noto Sans", "Liberation Sans", sans-serif;
    margin-bottom: 0;
    table-layout: fixed;
    width: 100%;
}
.ftl-detail-wrap .ftl-detail-table > :is(thead, tbody) > tr > :is(th, td) {
    padding: 0.3rem 0.35rem;
    vertical-align: top;
    white-space: normal;
    word-break: break-word;
    overflow-wrap: anywhere;
    hyphens: auto;
}
.ftl-detail-wrap .ftl-detail-table thead th {
    font-weight: 600;
    font-size: 0.625rem;
    letter-spacing: 0.02em;
    text-transform: none;
    line-height: 1.25;
}
.ftl-detail-wrap .ftl-dt-faculty { width: 8%; }
.ftl-detail-wrap .ftl-dt-code { width: 5%; }
.ftl-detail-wrap .ftl-dt-title { width: 13%; }
.ftl-detail-wrap .ftl-dt-units { width: 5%; white-space: nowrap; }
.ftl-detail-wrap .ftl-dt-time { width: 10%; }
.ftl-detail-wrap .ftl-dt-day { width: 8%; }
.ftl-detail-wrap .ftl-dt-lec { width: 4%; white-space: nowrap; }
.ftl-detail-wrap .ftl-dt-lab { width: 4%; white-space: nowrap; }
.ftl-detail-wrap .ftl-dt-total { width: 4%; white-space: nowrap; }
.ftl-detail-wrap .ftl-dt-room { width: 8%; }
.ftl-detail-wrap .ftl-dt-cyear { width: 7%; }
.ftl-detail-wrap .ftl-dt-section { width: 4%; white-space: nowrap; }
.ftl-detail-wrap .ftl-dt-sem { width: 6%; }
.ftl-detail-wrap .ftl-dt-sy { width: 5%; white-space: nowrap; }
.ftl-detail-wrap .ftl-dt-college { width: 14%; }

@media print {
    .ftl-detail-wrap { font-size: 7.5pt; line-height: 1.3; }
    .ftl-detail-wrap .ftl-detail-table thead th { font-size: 7pt; }
}
.ftl-detail-wrap .ftl-detail-table tfoot td {
    border-top: 2px solid var(--bs-border-color);
    vertical-align: middle;
    font-weight: 600;
    background: var(--bs-secondary-bg);
}
/* Memorandum source: off-screen (used by print popup only) */
.ftl-page .ftl-tlm-print-source {
    position: absolute;
    width: 210mm;
    left: -9999px;
    top: 0;
    visibility: hidden;
    pointer-events: none;
}
</style>
<div class="ftl-page">
    <div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-3">
        <h1 class="h3 mb-0"><i class="fa-solid fa-scale-balanced me-2 text-primary"><i>= $isFacultySelfView ? 'My teaching load (units)'
: 'Faculty teaching load (units)' ?</h1>
        <div class="d-flex flex-wrap gap-2 align-items-center">
            <span class="badge text-bg-primary rounded-pill">Total Units in view: <?>= htmlspecialchars($formatLoadUnits($grandTotalUnits)) ?
> Units</span>
        </div>
    </div>

    <?php if ($programScope !== null): ?>
        <p class="text-muted ftl-scope-note mb-3">
            Program scope: <strong><?>= htmlspecialchars($programScope) ?></strong> (<?>= htmlspecialchars(college_name_by_id($collegeId)) ?
>).
        <?php if (is_program_chair()): ?>
            <?php if ($hasIsGenedCourse): ?>

```

```

        <span class="d-block mt-1">General Education (GE) catalog courses are excluded. Includes <strong>major</strong> courses
in this program and any loads for <strong>your program's faculty</strong> (including subjects the Dean assigned in other programs).
</span>

        <?php else: ??
            <span class="d-block mt-1">Schedules created by GEN ED accounts are excluded. Run <a
href="upgrade_roles.php">upgrade_roles.php</a> so GE can be flagged in the catalog for clearer filtering.</span>
            <?php endif; ??
        <?php endif; ??
    </p>
    <?php elseif (is_dean()): ??
        <p class="text-muted ftl-scope-note mb-3">College: <strong><?= htmlspecialchars(college_name_by_id($collegeId)) ?></strong></p>
    <?php elseif (is_gened()): ??
        <p class="text-muted ftl-scope-note mb-3">
            <?php if ($hasIsGenedCourse): ??
                Showing schedules for courses flagged as <strong>General Education</strong> in the catalog.
            <?php else: ??
                Showing schedules created by <strong>GEN ED</strong> coordinator accounts. Run upgrade_roles.php so courses can be flagged
as GE in the catalog for this report to follow those offerings instead.
            <?php endif; ??
        </p>
    <?php elseif ($isFacultySelfView): ??
        <p class="text-muted ftl-scope-note mb-3">
            Showing your personal teaching load<?= $facultySelfName !== ' ' ? ' : <strong>' . htmlspecialchars($facultySelfName) . '</strong>'
: ' ' ?>.
        </p>
    <?php endif; ??

    <form class="row g-2 mb-4 no-print align-items-end ftl-filters" method="get">
        <?php if ($role === 'admin'): ??
            <div class="<?= htmlspecialchars($filterColClass) ?>">
                <label class="form-label small mb-0">College</label>
                <select name="college_id" class="form-select form-select-sm">
                    <option value="0">All colleges</option>
                    <?php foreach ($collegeFilterOptions as $c): ??
                        <option value="<?= (int) $c['id'] ?>" <?= $adminCollegeId === (int) $c['id'] ? 'selected' : ' ' ?>>
                            <?= htmlspecialchars($c['college_code'] . ' - ' . $c['college_name']) ?>
                        </option>
                    <?php endforeach; ??
                </select>
            </div>
        <?php endif; ??
        <?php if (is_program_chair() && $programScope !== null): ??
            <div class="<?= htmlspecialchars($filterColClass) ?>">
                <label class="form-label small mb-0">Program</label>
                <input type="text" class="form-control form-control-sm bg-body-secondary" readonly value="<?=
htmlspecialchars($programScope) ?>" aria-readonly="true"<?= app_tooltip_attr('Your account is limited to this program. Teaching load
includes courses in this program and all assignments for faculty in this program (including cross-program loads assigned by the Dean).') ?>>
                <input type="hidden" name="dept" value="<?= htmlspecialchars($programScope) ?>">
            </div>
        <?php elseif (!$isFacultySelfView): ??
            <div class="<?= htmlspecialchars($filterColClass) ?>">
                <label class="form-label small mb-0">Program</label>
                <select name="dept" class="form-select form-select-sm">
                    <option value="">All</option>
                    <?php foreach ($depts as $d): ??
                        <option value="<?= htmlspecialchars((string) $d) ?>" <?= $dept === (string) $d ? 'selected' : ' ' ?>><?=
htmlspecialchars((string) $d) ?></option>
                    <?php endforeach; ??
                </select>
            </div>
        <?php endif; ??
        <div class="<?= htmlspecialchars($filterColClass) ?>">
            <label class="form-label small mb-0">Semester</label>
            <select name="semester" class="form-select form-select-sm">
                <option value="">All</option>
                <?php foreach ($sems as $s): ??
                    <option value="<?= htmlspecialchars((string) $s) ?>" <?= $sem === (string) $s ? 'selected' : ' ' ?>><?=
htmlspecialchars((string) $s) ?></option>
                <?php endforeach; ??
            </select>
        </div>
        <div class="<?= htmlspecialchars($filterColClass) ?>">
            <label class="form-label small mb-0">School year</label>
            <select name="school_year" class="form-select form-select-sm">
                <option value="">All</option>
                <?php foreach ($years as $y): ??
                    <option value="<?= htmlspecialchars((string) $y) ?>" <?= $sy === (string) $y ? 'selected' : ' ' ?>><?=
htmlspecialchars((string) $y) ?></option>
                <?php endforeach; ??
            </select>
        </div>
    </form>

```

```

</div>
<?php if (is_gened() && $hasIsGenedFaculty): ?>
    <div class="<?> htmlspecialchars($filterColClass) ?>">
        <label class="form-label small mb-0">Faculty GE restriction</label>
        <select name="ge_faculty" class="form-select form-select-sm"<?> app_tooltip_attr('GE-designated faculty are the pool allowed
on GEN ED schedules. Other instructors may still appear if they teach a flagged GE section.') ?>>
            <option value="">All instructors</option>
            <option value="1" <?> $geFacultyFilter === '1' ? 'selected' : '' ?>>GE-designated only</option>
            <option value="0" <?> $geFacultyFilter === '0' ? 'selected' : '' ?>>Not GE-designated</option>
        </select>
    </div>
<?php endif; ?>
<?php if (is_gened()): ?>
    <div class="<?> htmlspecialchars($filterColClass) ?>">
        <label class="form-label small mb-0">Faculty search</label>
        <input type="search" name="faculty_search" class="form-control form-control-sm" placeholder="Name or faculty ID" value="<?>
htmlspecialchars($facultySearch) ?>" maxlength="120" autocomplete="off"<?> app_tooltip_attr('Narrows rows to instructors whose name or ID
contains this text.') ?>>
    </div>
<?php endif; ?>
<?php if ($role === 'admin'): ?>
    <div class="col-md-3">
        <label class="form-label small mb-0">Faculty search</label>
        <input type="search" name="faculty_search" class="form-control form-control-sm" placeholder="Name or faculty ID" value="<?>
htmlspecialchars($facultySearch) ?>" maxlength="120" autocomplete="off">
    </div>
<?php endif; ?>
<?php if (is_program_chair()): ?>
    <div class="col-md-3">
        <label class="form-label small mb-0">Faculty name / ID</label>
        <input type="search" name="faculty_search" class="form-control form-control-sm" value="<?> htmlspecialchars($facultySearch)
?>" placeholder="Search faculty..." maxlength="120" autocomplete="off"<?> app_tooltip_attr('Narrows rows to instructors whose name or ID
contains this text.') ?>>
    </div>
<?php endif; ?>
<?php if (is_dean()): ?>
    <div class="<?> htmlspecialchars($filterColClass) ?>">
        <label class="form-label small mb-0">Faculty search</label>
        <input type="search" name="faculty_search" class="form-control form-control-sm" value="<?> htmlspecialchars($facultySearch)
?>" placeholder="Faculty name or ID" maxlength="120" autocomplete="off"<?> app_tooltip_attr('Narrows results to instructors whose name or
faculty ID contains this text.') ?>>
    </div>
<?php endif; ?>
<div class="col-md-3">
    <button type="submit" class="btn btn-outline-primary btn-sm"<?> app_tooltip_attr('Apply filters to the teaching load totals.') ?
>>Apply</button>
    <a href="faculty_teaching_load.php" class="btn btn-outline-secondary btn-sm"<?> app_tooltip_attr('Clear filters.') ?>>Reset</a>
</div>
</form>

<section class="schedule-faculty-load no-print mb-4" aria-label="Faculty teaching load summary">
    <h2><i class="fa-solid fa-chalkboard-user me-2 text-primary"></i><?> $isFacultySelfView ? 'Your load summary' : 'By faculty member'
?></h2>
    <p class="small text-muted mb-2">
        Each course section counts once per semester and school year. Lecture and laboratory timetable rows for the same section are
merged.
    <?php if ($hasLectureUnits && $hasLaboratoryUnits): ?>
        Units are <strong>lecture + laboratory</strong> from the course catalog.
    <?php else: ?>
        Units use the course catalog <strong>units</strong> field.
    <?php endif; ?>
</p>
<?php if (!$facultyLoadSummary): ?>
    <p class="text-muted small mb-0">No scheduled offerings match your filters.</p>
<?php else: ?>
    <div class="row g-3">
        <?php foreach ($facultyLoadSummary as $f1): ?>
            <div class="col-md-6 col-xl-4">
                <div class="schedule-faculty-load-card">
                    <div class="d-flex justify-content-between align-items-start gap-2">
                        <span class="fw-semibold"><?> htmlspecialchars($f1['faculty_name']) ?></span>
                        <span class="badge bg-primary rounded-pill">Total: <?>
htmlspecialchars($formatLoadUnits($f1['total_units'])) ?> Units</span>
                    </div>
                    <ul class="list-unstyled small faculty-load-course-list mb-0 text-muted">
                        <?php foreach ($f1['offerings'] as $o): ?>
                            <li class="d-flex justify-content-between gap-2 py-1 border-top">
                                <span>
                                    <strong class="text-body"><?> htmlspecialchars($o['course_code']) ?></strong>
                                    <span class="d-block small"><?> htmlspecialchars($o['course_name']) ?></span>
                                </span>
                            </li>
                        </?php>
                    </ul>
                </div>
            </div>
        </?php>
    </div>
</?php>

```

```

        <span class="small"><?= htmlspecialchars($o['semester']) ?> . <?=
htmlspecialchars($o['school_year']) ?></span>
        <?php if ($showCollegeColumn && ($o['college_label'] ?? '') != ''): ?>
            <span class="d-block small text-secondary"><?= htmlspecialchars($o['college_label']) ?>
</span>

            <?php endif; ?>
        </span>
        <span class="text-nowrap fw-semibold text-body"><?= htmlspecialchars($formatLoadUnits($o['units']))
?> Units</span>

    </li>
    <?php endforeach; ?>
</ul>
</div>
</div>
<?php endforeach; ?>
</div>
<?php endif; ?>
</section>

<?php if ($facultyLoadSummary): ?>
    <div class="card border shadow-sm no-print">
        <div class="card-header py-2">
            <div class="d-flex flex-wrap align-items-center justify-content-between gap-2">
                <div class="d-flex flex-wrap align-items-center gap-2">
                    <span class="fw-semibold small text-uppercase text-muted">Detail table</span>
                    <?php if ($tLmMemoFaculty != null): ?>
                        <button type="button" class="btn btn-primary btn-sm" id="ftlTlMPrintBtn" onclick="ftlPrintTeachingLoadMemo()"><?=
app_tooltip_attr('Opens a printable Teaching Load Memorandum for the one faculty in this view (WPU-style layout).') ?>>
                            <i class="fa-solid fa-print me-1" aria-hidden="true"></i>Print memorandum
                        </button>
                    <?php endif; ?>
                </div>
            </div>
        </div>
        <div class="table-responsive ftl-detail-wrap">
            <table class="table table-sm table-hover mb-0 align-middle ftl-detail-table">
                <thead class="table-light">
                    <tr>
                        <?php if ($showFacultyColumn): ?>
                            <th scope="col" class="ftl-dt-faculty">Faculty</th>
                        <?php endif; ?>
                        <th scope="col" class="ftl-dt-code">Code</th>
                        <th scope="col" class="ftl-dt-title">Title</th>
                        <th scope="col" class="text-end ftl-dt-units">Units</th>
                        <th scope="col" class="ftl-dt-time">Time</th>
                        <th scope="col" class="ftl-dt-day">Day</th>
                        <th scope="col" class="ftl-dt-lec">Lec h</th>
                        <th scope="col" class="ftl-dt-lab">Lab h</th>
                        <th scope="col" class="ftl-dt-total">Total h</th>
                        <th scope="col" class="ftl-dt-room">Room</th>
                        <th scope="col" class="ftl-dt-cyear">Course / Yr</th>
                        <th scope="col" class="ftl-dt-section">Sec.</th>
                        <th scope="col" class="ftl-dt-sem">Sem.</th>
                        <th scope="col" class="ftl-dt-sy">SY</th>
                        <?php if ($showCollegeColumn): ?>
                            <th scope="col" class="ftl-dt-college">College</th>
                        <?php endif; ?>
                    </tr>
                </thead>
                <tbody>
                    <?php foreach ($facultyLoadSummary as $fl): ?>
                        <?php foreach ($fl['offerings'] as $o): ?>
                            <tr>
                                <?php if ($showFacultyColumn): ?>
                                    <td class="ftl-dt-faculty"><?= htmlspecialchars($fl['faculty_name']) ?></td>
                                <?php endif; ?>
                                <td class="ftl-dt-code"><?= htmlspecialchars($o['course_code']) ?></td>
                                <td class="ftl-dt-title"><?= htmlspecialchars($o['course_name']) ?></td>
                                <td class="text-end fw-semibold ftl-dt-units"><?= htmlspecialchars($formatLoadUnits($o['units'])) ?>
</td>

                                <td class="ftl-dt-time"><?= htmlspecialchars($o['time_display']) ?></td>
                                <td class="ftl-dt-day"><?= htmlspecialchars($o['day_display']) ?></td>
                                <td class="ftl-dt-lec"><?= htmlspecialchars($o['lec_hours_display']) ?></td>
                                <td class="ftl-dt-lab"><?= htmlspecialchars($o['lab_hours_display']) ?></td>
                                <td class="ftl-dt-total"><?= htmlspecialchars($formatLoadUnits((float) ($o['lec_hours_n'] ?? 0) +
(float) ($o['lab_hours_n'] ?? 0))) ?></td>
                                <td class="ftl-dt-room"><?= htmlspecialchars($o['room_display']) ?></td>
                                <td class="ftl-dt-cyear"><?= htmlspecialchars($o['course_year_display']) ?></td>
                                <td class="ftl-dt-section"><?= htmlspecialchars($o['section_display']) ?></td>
                                <td class="ftl-dt-sem"><?= htmlspecialchars($o['semester']) ?></td>

```



```

        <td class="ftl-dt-sy"><?= htmlspecialchars($o['school_year']) ?></td>
        <?php if ($showCollegeColumn): ?>
            <td class="ftl-dt-college"><?= htmlspecialchars($o['college_label'] != '' ? $o['college_label'] :
'-' ) ?></td>
        <?php endif; ?>
    </tr>
    <?php endforeach; ?>
</tbody>
<?php if ($detailTableTotals != null): ?>
    <tfoot>
        <tr class="ftl-dt-total-row">
            <?php if ($showFacultyColumn): ?>
                <td class="ftl-dt-faculty"><?= htmlspecialchars($detailTableTotals['faculty_name']) ?> - total</td>
            <?php endif; ?>
            <td class="ftl-dt-code"><?= $showFacultyColumn ? '-' : 'Total' ?></td>
            <td class="ftl-dt-title"></td>
            <td class="text-end ftl-dt-units"><?= htmlspecialchars($formatLoadUnits($detailTableTotals['sum_units'])) ?>
</td>

            <td class="ftl-dt-time"></td>
            <td class="ftl-dt-day"></td>
            <td class="ftl-dt-lec"><?= (float) $detailTableTotals['sum_lec_n'] > 0 ?
htmlspecialchars($formatLoadUnits((float) $detailTableTotals['sum_lec_n'])) : '-' ?></td>
            <td class="ftl-dt-lab"><?= (float) $detailTableTotals['sum_lab_n'] > 0 ?
htmlspecialchars($formatLoadUnits((float) $detailTableTotals['sum_lab_n'])) : '-' ?></td>
            <td class="ftl-dt-total"><?= (float) $detailTableTotals['sum_total_n'] > 0 ?
htmlspecialchars($formatLoadUnits((float) $detailTableTotals['sum_total_n'])) : '-' ?></td>
            <td class="ftl-dt-room"></td>
            <td class="ftl-dt-cyear"></td>
            <td class="ftl-dt-section"></td>
            <td class="ftl-dt-sem"></td>
            <td class="ftl-dt-sy"></td>
            <?php if ($showCollegeColumn): ?>
                <td class="ftl-dt-college"></td>
            <?php endif; ?>
        </tr>
    </tfoot>
</table>
<?php endif; ?>
</div>
</div>
<?php endif; ?>

<?php if ($tlmMemoFaculty != null && $detailTableTotals != null): ?>
    <div id="ftl-tlm-print" class="ftl-tlm-print-source" aria-hidden="true">
        <div class="ftl-tlm-doc">
            <header class="ftl-tlm-head">
                <div class="ftl-tlm-head-left d-flex gap-3 align-items-start">
                    
                    <div>
                        <div class="ftl-tlm-phil small text-uppercase">Republic of the Philippines</div>
                        <div class="ftl-tlm-uni-name fw-bold">Western Philippines University</div>
                    </div>
                </div>
                <div class="ftl-tlm-head-right text-end">
                    <div class="ftl-tlm-tagline text-primary small fw-semibold text-uppercase" style="font-family: Arial, Helvetica,
sans-serif;">A strong partner for sustainable development</div>
                    <div class="ftl-tlm-aa text-primary fw-bold text-uppercase mt-1" style="font-family: Arial, Helvetica, sans-serif;
letter-spacing: 0.04em;">Academic Affairs</div>
                </div>
            </header>

            <h1 class="ftl-tlm-title text-center text-uppercase fw-bold my-3">Teaching Load Memorandum</h1>

            <p class="mb-2"><strong>TO:</strong> <?= htmlspecialchars($tlmMemoFaculty['faculty_name']) ?></p>
            <p class="mb-4">
                The following subjects have been scheduled for you to handle this
                <strong><?= htmlspecialchars($tlmMemoSemester) ?></strong> semester, school year
                <strong><?= htmlspecialchars($tlmMemoSchoolYear) ?></strong>.
            </p>

            <table class="ftl-tlm-table w-100">
                <thead>
                    <tr>
                        <th scope="col">Course Code</th>
                        <th scope="col">Course Title</th>
                        <th scope="col" class="text-center">Units</th>
                        <th scope="col">Time</th>
                        <th scope="col">Dav</th>
                    </tr>
                </thead>
            </table>
        </div>
    </div>
</?php>

```

```

        <th scope="col" class="text-center">Lec h</th>
        <th scope="col" class="text-center">Lab h</th>
        <th scope="col">Room</th>
        <th scope="col">Course/Year</th>
        <th scope="col" class="text-center">Section</th>
    </tr>
</thead>
<tbody>
    <?php foreach ($t1mMemoFaculty['offerings'] as $mo): ?>
        <tr>
            <td><?= htmlspecialchars($mo['course_code']) ?></td>
            <td><?= htmlspecialchars($mo['course_name']) ?></td>
            <td class="text-center"><?= htmlspecialchars($formatLoadUnits($mo['units'])) ?></td>
            <td><?= htmlspecialchars($mo['time_display']) ?></td>
            <td><?= htmlspecialchars($mo['day_display']) ?></td>
            <td class="text-center"><?= htmlspecialchars($mo['lec_hours_display']) ?></td>
            <td class="text-center"><?= htmlspecialchars($mo['lab_hours_display']) ?></td>
            <td><?= htmlspecialchars($mo['room_display']) ?></td>
            <td><?= htmlspecialchars($mo['course_year_display']) ?></td>
            <td class="text-center"><?= htmlspecialchars($mo['section_display']) ?></td>
        </tr>
    <?php endforeach; ?>
</tbody>
<tfoot>
    <?php
        $t1mMemoTotalLecShow = (float) $detailTableTotals['sum_lec_n'] > 0
            ? $formatLoadUnits((float) $detailTableTotals['sum_lec_n'])
            : '-';
        $t1mMemoTotalLabShow = (float) $detailTableTotals['sum_lab_n'] > 0
            ? $formatLoadUnits((float) $detailTableTotals['sum_lab_n'])
            : '-';
        $t1mMemoTotalHoursShow = (float) ($detailTableTotals['sum_total_n'] ?? ((float) $detailTableTotals['sum_lec_n'] +
(float) $detailTableTotals['sum_lab_n'])) > 0
            ? $formatLoadUnits((float) ($detailTableTotals['sum_total_n'] ?? ((float) $detailTableTotals['sum_lec_n'] +
(float) $detailTableTotals['sum_lab_n'])))
            : '-';
    >
    <tr>
        <td colspan="2"><strong>Total Units</strong></td>
        <td class="text-center fw-bold"><?= htmlspecialchars($formatLoadUnits($detailTableTotals['sum_units'])) ?></td>
        <td colspan="2"></td>
        <td class="text-center fw-bold"><?= htmlspecialchars($t1mMemoTotalLecShow) ?></td>
        <td class="text-center fw-bold"><?= htmlspecialchars($t1mMemoTotalLabShow) ?></td>
        <td class="text-center fw-bold"><?= htmlspecialchars($t1mMemoTotalHoursShow) ?></td>
        <td colspan="3"></td>
    </tr>
</tfoot>
</table>

<p class="mt-4 mb-5">
    You are advised to meet these classes beginning <span class="ftl-t1m-blank">_____</span> until the
end of the semester.
</p>

<div class="ftl-t1m-sigs text-center">
    <div class="ftl-t1m-sig mb-5">
        <div class="ftl-t1m-sig-line mx-auto"></div>
        <div class="small mt-1 fw-bold text-uppercase"><?= htmlspecialchars($t1mMemoDeanName) ?></div>
        <div class="small mt-1">Dean</div>
    </div>
    <div class="ftl-t1m-sig">
        <div class="ftl-t1m-sig-line mx-auto"></div>
        <div class="small mt-1 fw-bold text-uppercase"><?= htmlspecialchars($t1mMemoVpaaName) ?></div>
        <div class="small mt-1">Vice President for Academic Affairs</div>
    </div>
</div>
</div>
<script>
function ftlPrintTeachingLoadMemo() {
    var inner = document.querySelector('#ftl-t1m-print .ftl-t1m-doc');
    if (!inner) {
        return;
    }
    var css = [
        '@page { margin: 8mm; size: A4 portrait; }',
        '*, ::before, ::after { box-sizing: border-box; }',
        'html, body { margin: 0; padding: 0; font-family: Times New Roman, Times, serif; font-size: 8.5pt; line-
height: 1.35; color: #000; background: #fff; -webkit-print-color-adjust: exact; print-color-adjust: exact; }',
        '.ftl-t1m-doc { max-width: 100%; }',
    ]

```

```

        '.ftl-tlm-doc p{margin:0 0 0.35rem;line-height:1.35;font-size:8.5pt;}',
        '.ftl-tlm-doc .mb-4{margin-bottom:0.5rem !important;}',
        '.ftl-tlm-doc .my-3{margin-top:0.4rem !important;margin-bottom:0.4rem !important;}',
        '.ftl-tlm-doc .mt-4{margin-top:0.5rem !important;}',
        '.ftl-tlm-doc .mb-5{margin-bottom:0.65rem !important;}',
        '.ftl-tlm-head{display:flex;justify-content:space-between;align-items:flex-start;border-bottom:1.5px solid #000;padding-bottom:6px;margin-bottom:8px;gap:6px;}',
        '.ftl-tlm-head .gap-3{gap:6px !important;}',
        '.ftl-tlm-logo-img{width:44px;height:44px;min-width:44px;object-fit:contain;display:block;}',
        '.ftl-tlm-phil{font-size:7pt;letter-spacing:0.02em;line-height:1.25;}',
        '.ftl-tlm-uni-name{font-size:11pt;line-height:1.2;font-family:Times New Roman,Times,serif;}',
        '.ftl-tlm-tagline{font-size:6.5pt;max-width:11rem;margin-left:auto;line-height:1.2;}',
        '.ftl-tlm-aa{font-size:10pt;line-height:1.15;}',
        'h1.ftl-tlm-title{font-size:10pt !important;line-height:1.25 !important;letter-spacing:0.05em;margin:8px 0 !important;font-weight:bold !important;}',
        '.ftl-tlm-table{border-collapse:collapse;width:100%;margin-top:4px;table-layout:fixed;}',
        '.ftl-tlm-table th,.ftl-tlm-table td{border:1px solid #000;padding:2px 3px;vertical-align:top;word-break:break-word;overflow-wrap:anywhere;hyphens:auto;white-space:normal;}',
        '.ftl-tlm-table thead th{font-weight:bold;text-align:center;font-size:7.5pt;line-height:1.25;}',
        '.ftl-tlm-table tbody td{font-size:7.5pt;line-height:1.3;}',
        '.ftl-tlm-table tfoot td{font-size:7.5pt;border:1px solid #000;line-height:1.25;}',
        '.ftl-tlm-table th:nth-child(1),.ftl-tlm-table td:nth-child(1){width:8%;}',
        '.ftl-tlm-table th:nth-child(2),.ftl-tlm-table td:nth-child(2){width:20%;}',
        '.ftl-tlm-table th:nth-child(3),.ftl-tlm-table td:nth-child(3){width:5%;}',
        '.ftl-tlm-table th:nth-child(4),.ftl-tlm-table td:nth-child(4){width:13%;}',
        '.ftl-tlm-table th:nth-child(5),.ftl-tlm-table td:nth-child(5){width:10%;}',
        '.ftl-tlm-table th:nth-child(6),.ftl-tlm-table td:nth-child(6){width:8%;}',
        '.ftl-tlm-table th:nth-child(7),.ftl-tlm-table td:nth-child(7){width:8%;}',
        '.ftl-tlm-table th:nth-child(8),.ftl-tlm-table td:nth-child(8){width:10%;}',
        '.ftl-tlm-table th:nth-child(9),.ftl-tlm-table td:nth-child(9){width:10%;}',
        '.ftl-tlm-table th:nth-child(10),.ftl-tlm-table td:nth-child(10){width:8%;}',
        '.ftl-tlm-table tbody tr{break-inside:avoid;page-break-inside:avoid;}',
        '.ftl-tlm-blank{border-bottom:1px solid #000;padding:0 2px;}',
        '.ftl-tlm-sig-line{width:55%;border-top:1px solid #000;margin-top:28px;}',
        '.ftl-tlm-sig .small{font-size:7.5pt;line-height:1.2;}',
        '.ftl-tlm-sig.mb-5{margin-bottom:1.25rem !important;}'
    ].join('');
    var w = window.open('', '_blank');
    if (!w) {
        alert('Please allow pop-ups to print the memorandum.');
```

```

    return;
}
w.document.open();
w.document.write('<!DOCTYPE html><html lang="en"><head><meta charset="UTF-8"><title>Teaching Load Memorandum</title>');
w.document.write('<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">');
w.document.write('<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">');
w.document.write('<style>' + css + '</style></head><body>');
w.document.write(inner.innerHTML);
w.document.write('</body></html>');
w.document.close();
w.focus();
setTimeout(function () {
    w.print();
    w.close();
}, 350);
}
</script>
<?php endif; ?>
</div>
<?php
require_once __DIR__ . '/includes/footer.php';

```

```

-----
FILE: fix_room_code_scope.php
-----

```

```

<?php
declare(strict_types=1);

/**
 * One-time fix: remove global UNIQUE(room_code) and add scoped uniqueness (room_code_scope).
 * Run once via browser POST or: php fix_room_code_scope.php
 * Remove or protect this file after use (same security model as upgrade_roles.php).
 */
require_once __DIR__ . '/config/config.php';

/**
 * @return array{lines: string[], error: string}
 */
function migrate_room_code_scope(PDO $pdo): array

```

```

{
    $lines = [];
    $schema = DB_NAME;

    $run = static function (string $sql, bool $ignoreBenign = true) use ($pdo, &$lines): bool {
        try {
            $pdo->exec($sql);
            $lines[] = 'OK: ' . $sql;
            return true;
        } catch (Throwable $e) {
            $msg = strtolower($e->getMessage());
            if (
                $ignoreBenign
                && (
                    str_contains($msg, "can't drop")
                    || str_contains($msg, 'check that column/key exists')
                    || str_contains($msg, 'check that it exists')
                    || str_contains($msg, 'duplicate column')
                    || str_contains($msg, 'duplicate key name')
                    || str_contains($msg, 'already exists')
                )
            ) {
                $lines[] = 'Skip (expected): ' . $e->getMessage();
                return true;
            }
            $lines[] = 'FAIL: ' . $e->getMessage();
            return false;
        }
    };

    $run('ALTER TABLE rooms DROP INDEX room_code');
    $run('ALTER TABLE rooms DROP INDEX uq_rooms_scope');

    $st = $pdo->prepare(
        "SELECT DISTINCT INDEX_NAME
        FROM information_schema.STATISTICS
        WHERE TABLE_SCHEMA = ?
        AND TABLE_NAME = 'rooms'
        AND COLUMN_NAME = 'room_code'
        AND NON_UNIQUE = 0
        AND INDEX_NAME <> 'PRIMARY'"
    );
    $st->execute([$schema]);
    foreach ($st->fetchAll(PDO::FETCH_COLUMN) ? : [] as $idxName) {
        $safe = str_replace('`', '``', (string) $idxName);
        if (!$run("ALTER TABLE rooms DROP INDEX `${$safe}`")) {
            return ['lines' => $lines, 'error' => 'Could not drop unique index on room_code.'];
        }
    }

    $run('ALTER TABLE rooms ADD INDEX idx_room_code (room_code)');

    $chk = $pdo->prepare(
        "SELECT COUNT(*) FROM information_schema.COLUMNS
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = 'rooms' AND COLUMN_NAME = 'room_code_scope'"
    );
    $chk->execute([$schema]);
    if ((int) $chk->fetchColumn() === 0) {
        $sql = "ALTER TABLE rooms ADD COLUMN room_code_scope VARCHAR(64) GENERATED ALWAYS AS (
            IF(COALESCE(is_gened,0) = 1,
                CONCAT('G|', room_code),
                CONCAT('C|', IFNULL(college_id, 0), '|', room_code))
        ) STORED";
        if (!$run($sql, false)) {
            return [
                'lines' => $lines,
                'error' => 'Could not add room_code_scope column. Check MySQL/MariaDB version supports GENERATED STORED columns.',
            ];
        }
    } else {
        $lines[] = 'OK: room_code_scope column already present.';
    }

    if (!$run('ALTER TABLE rooms ADD UNIQUE KEY uq_rooms_scope (room_code_scope)')) {
        return [
            'lines' => $lines,
            'error' => 'Could not add UNIQUE uq_rooms_scope. If two rows share the same scope (e.g. duplicate GE code), fix data first.',
        ];
    }
}

```

```

        return ['lines' => $lines, 'error' => ''];
    }

    // ----- CLI -----
    if (PHP_SAPI === 'cli') {
        try {
            $pdo = new PDO(
                sprintf('mysql:host=%s;port=%d;dbname=%s;charset=%s', DB_HOST, DB_PORT, DB_NAME, DB_CHARSET),
                DB_USER,
                DB_PASS,
                [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
            );
            $out = migrate_room_code_scope($pdo);
            foreach ($out['lines'] as $line) {
                fwrite(STDOUT, $line . PHP_EOL);
            }
            if ($out['error'] !== '') {
                fwrite(STDERR, $out['error'] . PHP_EOL);
                exit(1);
            }
            fwrite(STDOUT, 'Room code scope migration finished.' . PHP_EOL);
            exit(0);
        } catch (Throwable $e) {
            fwrite(STDERR, $e->getMessage() . PHP_EOL);
            exit(1);
        }
    }

    // ----- Web -----
    $lines = [];
    $error = '';
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        try {
            $pdo = new PDO(
                sprintf('mysql:host=%s;port=%d;dbname=%s;charset=%s', DB_HOST, DB_PORT, DB_NAME, DB_CHARSET),
                DB_USER,
                DB_PASS,
                [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
            );
            $result = migrate_room_code_scope($pdo);
            $lines = $result['lines'];
            $error = $result['error'];
        } catch (Throwable $e) {
            $error = $e->getMessage();
        }
    }
    ?>
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Fix room code uniqueness</title>
    <?php require __DIR__ . '/includes/favicon.php'; ?>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
<div class="container py-5" style="max-width: 800px;">
    <h1 class="h4 mb-3">Fix room code uniqueness</h1>
    <p class="text-muted">Removes the old global unique index on <code>room_code</code> and adds scoped uniqueness so different colleges
    (and Gen Ed) can share the same code (e.g. 101).</p>
    <?php if ($error !== ''): ?>
        <div class="alert alert-danger"><?php echo htmlspecialchars($error); ?></div>
    <?php elseif ($lines !== []): ?>
        <div class="alert alert-success">Migration completed. You can add rooms again; delete or protect this script when done.</div>
    <?php endif; ?>
    <?php if ($lines !== []): ?>
        <pre class="bg-white border rounded p-3 small" style="max-height: 400px; overflow: auto;"><?php echo htmlspecialchars(implode("\n",
        $lines)); ?></pre>
    <?php endif; ?>
    <form method="post">
        <button class="btn btn-primary" type="submit">Run room code migration</button>
        <a class="btn btn-outline-secondary ms-2" href="upgrade_roles.php">Full role upgrade</a>
    </form>
</div>
</body>
</html>

```

FILE: gened_assignments.php

```
-----
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['gened']);
$hasIsGened = db_column_exists('courses', 'is_gened');
$hasAssignTable = true;
try {
    db()->query('SELECT 1 FROM ge_course_colleges LIMIT 1');
} catch (Throwable $e) {
    $hasAssignTable = false;
}

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    try {
        if (!$hasIsGened || !$hasAssignTable) {
            throw new RuntimeException('Run upgrade_roles.php first to enable GE course offerings.');
```

```
        }
        $courseId = (int) ($_POST['course_id'] ?? 0);
        $collegeIds = isset($_POST['college_ids']) && is_array($_POST['college_ids'])
            ? array_values(array_unique(array_map('intval', $_POST['college_ids'])))
            : [];
        if ($courseId < 1) {
            throw new RuntimeException('Please select a GE course.');
```

```
        }

        db()->prepare('DELETE FROM ge_course_colleges WHERE course_id=?')->execute([$courseId]);
        $ins = db()->prepare('INSERT INTO ge_course_colleges (course_id, college_id) VALUES (?,?)');
        foreach ($collegeIds as $cid) {
            if ($cid > 0) {
                $ins->execute([$courseId, $cid]);
            }
        }
        $_SESSION['flash'] = 'GE course offering assignments updated.';
    } catch (Throwable $e) {
        $_SESSION['flash'] = 'Error: ' . $e->getMessage();
    }
    header('Location: gened_assignments.php');
    exit;
}

$courses = $hasIsGened ? db()->query('SELECT id, course_code, course_name FROM courses WHERE is_gened=1 ORDER BY course_code')->fetchAll() :
[];
$colleges = db()->query("SELECT id, college_code, college_name FROM colleges WHERE status='active' ORDER BY college_code")->fetchAll();

$selectedCourseId = (int) ($_GET['course_id'] ?? ($courses[0]['id'] ?? 0));
$assigned = [];
if ($selectedCourseId > 0 && $hasAssignTable) {
    $st = db()->prepare('SELECT college_id FROM ge_course_colleges WHERE course_id=?');
    $st->execute([$selectedCourseId]);
    $assigned = array_map('intval', $st->fetchAll(PDO::FETCH_COLUMN));
}

$matrix = [];
if ($hasAssignTable) {
    $rows = db()->query(
        'SELECT gcc.course_id, c.course_code, c.course_name, col.college_code
        FROM ge_course_colleges gcc
        INNER JOIN courses c ON c.id = gcc.course_id
        INNER JOIN colleges col ON col.id = gcc.college_id
        WHERE c.is_gened=1
        ORDER BY c.course_code, col.college_code'
    )->fetchAll();
    foreach ($rows as $r) {
        $k = (int) $r['course_id'];
        if (!isset($matrix[$k])) {
            $matrix[$k] = [
                'course' => $r['course_code'] . ' - ' . $r['course_name'],
                'colleges' => [],
            ];
        }
        $matrix[$k]['colleges'][] = $r['college_code'];
    }
}
```

```

}

$geTableRows = [];
foreach ($courses as $c) {
    $cid = (int) $c['id'];
    $geTableRows[] = [
        'course' => $c['course_code'] . ' - ' . $c['course_name'],
        'colleges' => $matrix[$cid]['colleges'] ?? [],
    ];
}

$collegeNameByCode = [];
foreach ($colleges as $col) {
    $collegeNameByCode[(string) $col['college_code']] = (string) $col['college_name'];
}

$pageTitle = 'GE Course Offerings';
require_once __DIR__ . '/includes/header.php';
?>
<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<link href="https://fonts.googleapis.com/css2?
family=Inter:opsz,wght@14..32,300;14..32,400;14..32,500;14..32,600;14..32,700&display=swap" rel="stylesheet">
<style>
.ge-offerings-page {
    --ge-text: #1a2c3e;
    font-family: 'Inter', system-ui, sans-serif;
    color: var(--ge-text);
    background: linear-gradient(145deg, #eef2f7 0%, #d9e0e8 100%);
    margin: -1.5rem -0.75rem 0;
    padding: 2rem 1.5rem 2.5rem;
    border-radius: 0;
}
@media (min-width: 768px) {
    .ge-offerings-page {
        margin: -1.5rem -1rem 0;
        padding: 2rem 1.75rem 2.5rem;
        border-radius: 0 0 1rem 1rem;
    }
}
.ge-offerings-page * { box-sizing: border-box; }
.ge-offerings-page .ge-dashboard-container { max-width: 1400px; margin: 0 auto; }
.ge-offerings-page .ge-module-header { margin-bottom: 2rem; text-align: left; }
.ge-offerings-page .ge-module-header h1 {
    font-size: 1.9rem;
    font-weight: 700;
    background: linear-gradient(135deg, #1e4668, #0f2c44);
    background-clip: text;
    -webkit-background-clip: text;
    color: transparent;
    letter-spacing: -0.3px;
    display: inline-flex;
    align-items: center;
    gap: 12px;
}
.ge-offerings-page .ge-module-header h1 i {
    background: none;
    -webkit-text-fill-color: initial;
    color: #2c6e9e;
    font-size: 2rem;
}
.ge-offerings-page .ge-subhead {
    color: #2c4e6e;
    font-weight: 500;
    margin-top: 8px;
    font-size: 0.95rem;
    border-left: 4px solid #3b82f6;
    padding-left: 14px;
}
.ge-offerings-page .ge-grid {
    display: grid;
    grid-template-columns: 1fr 1.2fr;
    gap: 28px;
    margin-bottom: 40px;
}
.ge-offerings-page .ge-card {
    background: rgba(255, 255, 255, 0.96);
    border-radius: 32px;
    box-shadow: 0 20px 35px -12px rgba(0, 0, 0, 0.12), 0 1px 2px rgba(0,0,0,0.02);
    overflow: hidden;

```

```

        transition: box-shadow 0.2s;
        border: 1px solid rgba(255,255,255,0.5);
    }
    .ge-offerings-page .ge-card:hover { box-shadow: 0 25px 40px -16px rgba(0, 0, 0, 0.2); }
    .ge-offerings-page .ge-card-header {
        padding: 1.2rem 1.8rem;
        background: #ffffffdd;
        border-bottom: 1px solid #e9edf2;
        display: flex;
        align-items: center;
        gap: 12px;
    }
    .ge-offerings-page .ge-card-header i { font-size: 1.6rem; color: #2c7da0; }
    .ge-offerings-page .ge-card-header h2 {
        font-size: 1.35rem;
        font-weight: 600;
        color: #1f3b4c;
        margin: 0;
    }
    .ge-offerings-page .ge-card-body { padding: 1.6rem 1.8rem; }
    .ge-offerings-page .ge-selector { margin-bottom: 2rem; }
    .ge-offerings-page .ge-selector label {
        font-weight: 600;
        display: flex;
        align-items: center;
        gap: 8px;
        margin-bottom: 12px;
        color: #1f3e53;
        font-size: 0.9rem;
    }
    .ge-offerings-page .ge-custom-select {
        width: 100%;
        padding: 12px 16px;
        border-radius: 20px;
        border: 1.5px solid #cfdfed;
        background: white;
        font-family: 'Inter', system-ui, sans-serif;
        font-weight: 500;
        font-size: 0.95rem;
        color: #1f2f3c;
        cursor: pointer;
        transition: all 0.2s;
        outline: none;
    }
    .ge-offerings-page .ge-custom-select:focus {
        border-color: #3b82f6;
        box-shadow: 0 0 0 3px rgba(59,130,246,0.2);
    }
    .ge-offerings-page .ge-college-check-group {
        background: #f8fafd;
        border-radius: 24px;
        padding: 0.8rem 1rem;
        margin: 1.5rem 0 1.8rem;
        border: 1px solid #e2e8f0;
    }
    .ge-offerings-page .ge-college-check-group h3 {
        font-size: 0.85rem;
        text-transform: uppercase;
        letter-spacing: 1px;
        color: #4a6f8f;
        margin: 0 0 14px;
        font-weight: 600;
    }
    .ge-offerings-page .ge-checkbox-item {
        display: flex;
        align-items: center;
        gap: 12px;
        padding: 10px 6px;
        border-radius: 16px;
        transition: background 0.1s ease;
    }
    .ge-offerings-page .ge-checkbox-item:hover { background: #eef3fc; }
    .ge-offerings-page .ge-checkbox-item input {
        width: 18px;
        height: 18px;
        accent-color: #2c7da0;
        cursor: pointer;
        flex-shrink: 0;
    }
    .ge-offerings-page .ge-checkbox-item label {

```



```

    font-weight: 500;
    color: #1f3e53;
    cursor: pointer;
    font-size: 0.95rem;
    margin: 0;
}
.ge-offerings-page .ge-btn-save {
    background: linear-gradient(100deg, #1f6392, #134b6e);
    border: none;
    padding: 12px 24px;
    border-radius: 40px;
    font-weight: 600;
    font-size: 0.9rem;
    color: white;
    display: inline-flex;
    align-items: center;
    gap: 10px;
    cursor: pointer;
    transition: 0.2s;
    box-shadow: 0 5px 12px rgba(0,0,0,0.08);
    width: 100%;
    justify-content: center;
}
.ge-offerings-page .ge-btn-save:hover:not(:disabled) {
    background: linear-gradient(100deg, #0f547c, #0c3f5c);
    transform: translateY(-2px);
    box-shadow: 0 10px 20px -5px rgba(0,0,0,0.15);
}
.ge-offerings-page .ge-btn-save:active:not(:disabled) { transform: translateY(1px); }
.ge-offerings-page .ge-btn-save:disabled { opacity: 0.55; cursor: not-allowed; }
.ge-offerings-page .ge-offerings-table-wrapper { overflow-x: auto; border-radius: 24px; }
.ge-offerings-page .ge-offerings-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.9rem;
}
.ge-offerings-page .ge-offerings-table th {
    text-align: left;
    padding: 1rem 1.2rem;
    background: #eef3fa;
    font-weight: 600;
    color: #1f4662;
    border-bottom: 2px solid #cddfec;
}
.ge-offerings-page .ge-offerings-table td {
    padding: 1rem 1.2rem;
    border-bottom: 1px solid #e2edf5;
    vertical-align: middle;
    color: #1f2f3c;
}
.ge-offerings-page .ge-offerings-table tr:last-child td { border-bottom: none; }
.ge-offerings-page .ge-college-badge {
    background: #e0edf5;
    padding: 5px 12px;
    border-radius: 30px;
    font-size: 0.8rem;
    font-weight: 600;
    display: inline-block;
    color: #1c5679;
    margin: 2px 4px 2px 0;
}
.ge-offerings-page .ge-college-badge.ge-badge-multiple {
    background: #cbdde9;
    color: #1e4a6e;
}
.ge-offerings-page .ge-empty-message {
    text-align: center;
    padding: 2rem;
    color: #6c8eaa;
    font-style: italic;
}
.ge-offerings-page .ge-empty-inline {
    color: #6c8eaa;
    font-style: italic;
    font-weight: 400;
}
.ge-offerings-page .ge-options-legend {
    background: #f0f4f9;
    border-radius: 28px;
    padding: 1.2rem 1.5rem;
}

```

```

margin-top: 12px;
display: flex;
flex-wrap: wrap;
gap: 1rem;
align-items: center;
justify-content: flex-start;
border: 1px solid #e2eaf1;
}
.ge-offerings-page .ge-legend-title {
  font-weight: 700;
  font-size: 0.8rem;
  color: #2b5a7a;
  letter-spacing: 0.5px;
}
.ge-offerings-page .ge-legend-badge {
  background: white;
  padding: 5px 14px;
  border-radius: 40px;
  font-size: 0.8rem;
  font-weight: 500;
  color: #1e3e55;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05);
  border: 1px solid #cfdfec;
}
.ge-offerings-page .ge-toast-msg {
  position: fixed;
  bottom: 30px;
  left: 50%;
  transform: translateX(-50%);
  color: white;
  padding: 12px 26px;
  border-radius: 50px;
  font-size: 0.85rem;
  font-weight: 500;
  backdrop-filter: blur(6px);
  background: #1f3b4cee;
  box-shadow: 0 10px 20px rgba(0,0,0,0.1);
  z-index: 2000;
  transition: opacity 0.2s;
  pointer-events: none;
  opacity: 0;
  max-width: min(90vw, 32rem);
  text-align: center;
}
.ge-offerings-page .ge-toast-msg.ge-show { opacity: 1; }
.ge-offerings-page .ge-toast-msg.ge-toast--ok { background: rgba(43, 110, 71, 0.92); }
.ge-offerings-page .ge-toast-msg.ge-toast--err { background: rgba(139, 47, 47, 0.92); }
.ge-offerings-page .ge-flash-banner {
  border-radius: 20px;
  padding: 0.85rem 1.2rem;
  margin-bottom: 1.25rem;
  font-size: 0.9rem;
  font-weight: 500;
}
.ge-offerings-page .ge-flash-banner--warn {
  background: #fff8e6;
  border: 1px solid #e9d48a;
  color: #6b5a1a;
}
.ge-offerings-page .ge-flash-banner a { color: #0d5a9e; font-weight: 600; }
html[data-bs-theme="dark"] .ge-offerings-page {
  --ge-text: #d9e8f3;
  background: linear-gradient(145deg, #101b24 0%, #172633 100%);
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-module-header h1 {
  background: linear-gradient(135deg, #cae9ff, #90d3fb);
  background-clip: text;
  -webkit-background-clip: text;
  color: transparent;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-module-header h1 i { color: #69b8e8; }
html[data-bs-theme="dark"] .ge-offerings-page .ge-subhead {
  color: #9db4c5;
  border-left-color: #4a9de6;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-card {
  background: rgba(20, 33, 44, 0.95);
  border-color: #2c3f4f;
  box-shadow: 0 20px 35px -14px rgba(0, 0, 0, 0.45), 0 1px 2px rgba(0, 0, 0, 0.2);
}

```

```

html[data-bs-theme="dark"] .ge-offerings-page .ge-card:hover {
  box-shadow: 0 24px 42px -16px rgba(0, 0, 0, 0.52);
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-card-header {
  background: rgba(22, 37, 49, 0.95);
  border-bottom-color: #2d4150;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-card-header i { color: #6fb8de; }
html[data-bs-theme="dark"] .ge-offerings-page .ge-card-header h2 { color: #d1e6f4; }
html[data-bs-theme="dark"] .ge-offerings-page .ge-selector label,
html[data-bs-theme="dark"] .ge-offerings-page .ge-checkbox-item label { color: #c4d9e8; }
html[data-bs-theme="dark"] .ge-offerings-page .ge-custom-select {
  background: #101c27;
  border-color: #355167;
  color: #deecf7;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-custom-select:focus {
  border-color: #5da8ef;
  box-shadow: 0 0 3px rgba(93, 168, 239, 0.24);
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-college-check-group {
  background: #162633;
  border-color: #324959;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-college-check-group h3 { color: #a8c3d8; }
html[data-bs-theme="dark"] .ge-offerings-page .ge-checkbox-item:hover { background: #1f3444; }
html[data-bs-theme="dark"] .ge-offerings-page .ge-btn-save {
  background: linear-gradient(100deg, #1f6fa5, #124c72);
  box-shadow: 0 6px 16px rgba(0, 0, 0, 0.35);
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-btn-save:hover:not(:disabled) {
  background: linear-gradient(100deg, #1a6090, #103e5d);
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-offerings-table th {
  background: #1a2d3b;
  color: #c8deed;
  border-bottom-color: #355165;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-offerings-table td {
  color: #d4e7f4;
  border-bottom-color: #2a4050;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-college-badge {
  background: #274053;
  color: #b9ddf3;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-college-badge.ge-badge-multiple {
  background: #325166;
  color: #d1ecff;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-empty-message,
html[data-bs-theme="dark"] .ge-offerings-page .ge-empty-inline { color: #8fa8ba; }
html[data-bs-theme="dark"] .ge-offerings-page .ge-options-legend {
  background: #172938;
  border-color: #324a5a;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-legend-title { color: #b6d2e4; }
html[data-bs-theme="dark"] .ge-offerings-page .ge-legend-badge {
  background: #1f3545;
  border-color: #365268;
  color: #cce3f1;
  box-shadow: none;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-flash-banner--warn {
  background: #3a321f;
  border-color: #6d5a32;
  color: #f2dcaa;
}
html[data-bs-theme="dark"] .ge-offerings-page .ge-flash-banner a { color: #8ec5ff; }
@media (max-width: 800px) {
  .ge-offerings-page { padding: 1rem; margin-left: -0.5rem; margin-right: -0.5rem; }
  .ge-offerings-page .ge-grid { grid-template-columns: 1fr; gap: 20px; }
  .ge-offerings-page .ge-card-body { padding: 1.2rem; }
}
</style>

<div class="ge-offerings-page">
<div class="ge-dashboard-container">
  <div class="ge-module-header">
    <h1><i class="fas fa-chalkboard-user" aria-hidden="true"></i> GE Course Offerings · by College</h1>

```

```

        <div class="ge-subhead">Assign general education courses to academic colleges & manage cross-offerings</div>
    </div>

    <?php if (!$hasIsGened || !$hasAssignTable): ?>
        <div class="ge-flash-banner ge-flash-banner--warn">Run <a href="upgrade_roles.php">upgrade_roles.php</a> to enable GE assignments.
    </div>
    <?php endif; ?>

    <div class="ge-grid">
        <div class="ge-card">
            <div class="ge-card-header">
                <i class="fas fa-pen-ruler" aria-hidden="true"></i>
                <h2>Assign colleges to a GE course</h2>
            </div>
            <div class="ge-card-body">
                <form method="get" id="geCourseSelectForm" class="ge-selector">
                    <label for="geCourseSelect"><i class="fas fa-book-open" aria-hidden="true"></i> GE Course</label>
                    <select id="geCourseSelect" name="course_id" class="ge-custom-select" onchange="this.form.submit()" <?=$courses == []
? 'disabled' : '' ?>>
                        <?php foreach ($courses as $c): ?>
                            <option value="<?=(int) $c['id'] ?>" <?=$selectedCourseId == (int) $c['id'] ? 'selected' : '' ?>>
                                <?=$htmlspecialchars($c['course_code'] . ' - ' . $c['course_name']) ?>
                            </option>
                        <?php endforeach; ?>
                    </select>
                </form>
                <form method="post">
                    <input type="hidden" name="course_id" value="<?=(int) $selectedCourseId ?>">
                    <div class="ge-college-check-group">
                        <h3><i class="fas fa-building-columns" aria-hidden="true"></i> Colleges where this GE course is offered</h3>
                        <?php if ($courses == []): ?>
                            <p class="mb-0 small text-muted">No GE courses are marked in the catalog yet.</p>
                        <?php endif; ?>
                        <?php foreach ($colleges as $col): ?>
                            <div class="ge-checkbox-item">
                                <input type="checkbox" name="college_ids[]" value="<?=(int) $col['id'] ?>" id="col_<?=(int) $col['id'] ?>"
                                    <?=$in_array((int) $col['id'], $assigned, true) ? 'checked' : '' ?>
                                    <?=$(!$hasIsGened || !$hasAssignTable || $courses == []) ? 'disabled' : '' ?>
                                <label for="col_<?=(int) $col['id'] ?>"><strong><?=$htmlspecialchars($col['college_code']) ?></strong> - <?=$htmlspecialchars($col['college_name']) ?></label>
                            </div>
                        <?php endforeach; ?>
                    </div>
                    <button type="submit" class="ge-btn-save"><?=$(!$hasIsGened || !$hasAssignTable || $courses == []) ? 'disabled' : '' ?>
                        <?=$app_tooltip_attr('Saves which colleges or cohorts receive this GE offering. Use after adjusting checkboxes or targets.') ?>>
                        <i class="fas fa-save" aria-hidden="true"></i> Save offerings
                    </button>
                </form>
            </div>
        </div>

        <div class="ge-card">
            <div class="ge-card-header">
                <i class="fas fa-table-list" aria-hidden="true"></i>
                <h2>Current GE offerings</h2>
            </div>
            <div class="ge-card-body">
                <div class="ge-offerings-table-wrapper">
                    <table class="ge-offerings-table" id="offeringsTable">
                        <thead>
                            <tr><th>GE Course</th><th>Assigned Colleges</th></tr>
                        </thead>
                        <tbody>
                            <?php if ($courses == []): ?>
                                <tr><td colspan="2" class="ge-empty-message">No GE courses in the catalog yet.</td></tr>
                            <?php else: ?>
                                <?php foreach ($geTableRows as $row): ?>
                                    <tr>
                                        <td style="font-weight:500;"><?=$htmlspecialchars($row['course']) ?></td>
                                        <td>
                                            <?php if ($row['colleges'] == []): ?>
                                                <span class="ge-empty-inline">— Not assigned —</span>
                                            <?php else: ?>
                                                <?php foreach ($row['colleges'] as $code): ?>
                                                    <?php
                                                        $codeStr = (string) $code;
                                                        $tip = $collegeNameByCode[$codeStr] ?? $codeStr;
                                                    <?>
                                                    <span class="ge-college-badge"><?=$count($row['colleges']) > 1 ? 'ge-badge-multiple' : '' ?>
title="<?=$htmlspecialchars($tip) ?>"><?=$htmlspecialchars($codeStr) ?></span>

```

```

                <?php endforeach; ?>
            <?php endif; ?>
        </td>
    </tr>
<?php endforeach; ?>
<?php endif; ?>
</tbody>
</table>
</div>
</div>
</div>
</div>

<div class="ge-options-legend">
    <span class="ge-legend-title"><i class="fas fa-graduation-cap" aria-hidden="true"></i> College codes & names:</span>
    <?php foreach ($colleges as $col): ?>
        <span class="ge-legend-badge"><strong><?= htmlspecialchars($col['college_code']) ?></strong> - <?=
htmlspecialchars($col['college_name']) ?></span>
    <?php endforeach; ?>
</div>
</div>
<div id="geToastMessage" class="ge-toast-msg" role="status" aria-live="polite"></div>
</div>

```

```

<script>
(function () {
    var toast = document.getElementById('geToastMessage');
    var tmo = null;
    window.showGeToast = function (message, isError) {
        if (!toast) return;
        toast.textContent = message;
        toast.classList.remove('ge-toast--ok', 'ge-toast--err');
        toast.classList.add(isError ? 'ge-toast--err' : 'ge-toast--ok');
        toast.classList.add('ge-show');
        if (tmo) clearTimeout(tmo);
        tmo = setTimeout(function () { toast.classList.remove('ge-show'); }, 2600);
    };
})();
</script>

```

```

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: gened_courses.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['gened']);

$hasLabFlag = db_column_exists('courses', 'is_laboratory');
$hasLectureUnits = db_column_exists('courses', 'lecture_units');
$hasLaboratoryUnits = db_column_exists('courses', 'laboratory_units');
$hasIsGened = db_column_exists('courses', 'is_gened');
$hasYearLevel = db_column_exists('courses', 'year_level');
$hasSection = db_column_exists('courses', 'section');
$hasCourseBlock = $hasYearLevel && $hasSection;
$hasProgramsTable = db_table_exists('programs');

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

$programOptions = [];
if ($hasProgramsTable) {
    $programOptions = db()->query(
        "SELECT p.college_id, p.program_name, c.college_code
        FROM programs p
        INNER JOIN colleges c ON c.id = p.college_id
        WHERE p.status='active'
        ORDER BY c.college_code, p.program_name"
    )->fetchAll();
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {

```

```

// Auto-heal legacy DBs that still have UNIQUE(course_code).
ensure_course_code_duplicates_allowed();
$code = strtoupper(trim((string) ($_POST['course_code'] ?? '')));
$name = trim((string) ($_POST['course_name'] ?? ''));
$program = trim((string) ($_POST['program'] ?? $_POST['department'] ?? ''));
$isLab = !empty($_POST['is_laboratory']) ? 1 : 0;
$yearLevel = trim((string) ($_POST['year_level'] ?? ''));
$section = trim((string) ($_POST['section'] ?? ''));
$lecUnits = $hasLectureUnits ? max(0.0, (float) ($_POST['lecture_units'] ?? 0)) : max(0.5, (float) ($_POST['units'] ?? 3.0));
$labUnits = $hasLaboratoryUnits ? max(0.0, (float) ($_POST['laboratory_units'] ?? 0)) : ($isLab ? max(0.5, (float) ($_POST['units']
?? 3.0)) : 0.0);
$legacyUnits = max(0.5, $lecUnits > 0 ? $lecUnits : ($isLab && $labUnits > 0 ? $labUnits : 3.0));
if (($action === 'add' || $action === 'edit') && $hasCourseBlock && ($yearLevel === '' || $section === '')) {
    throw new RuntimeException('Year level and section are required.');
```

```

}
if (($action === 'add' || $action === 'edit') && $hasProgramsTable) {
    if ($program === '') {
        throw new RuntimeException('Program is required.');
```

```

    }
    $programParts = explode('|', $program, 2);
    $selectedCollegeId = count($programParts) === 2 ? (int) $programParts[0] : 0;
    $selectedProgramName = count($programParts) === 2 ? trim((string) $programParts[1]) : trim($program);
    if ($selectedProgramName === '') {
        throw new RuntimeException('Please select a valid program.');
```

```

    }
    if ($selectedCollegeId > 0) {
        $stmtProgram = db()->prepare('SELECT COUNT(*) FROM programs WHERE college_id = ? AND program_name = ?');
        $stmtProgram->execute([$selectedCollegeId, $selectedProgramName]);
    } else {
        $stmtProgram = db()->prepare('SELECT COUNT(*) FROM programs WHERE program_name = ?');
        $stmtProgram->execute([$selectedProgramName]);
    }
    if ((int) $stmtProgram->fetchColumn() < 1) {
        throw new RuntimeException('Please select a valid program from the Programs module.');
```

```

    }
    $program = $selectedProgramName;
} elseif ($program === '') {
    $program = 'General Education';
}

if ($action === 'add') {
    if ($hasLabFlag && $hasLectureUnits && $hasLaboratoryUnits && $hasIsGened && $hasCourseBlock) {
        db()->prepare(
            'INSERT INTO courses (course_code, course_name, units, lecture_units, laboratory_units, is_laboratory, is_gened,
year_level, section, department, college_id)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, NULL)'
        )->execute([$code, $name, $legacyUnits, $lecUnits, $labUnits, $isLab, 1, $yearLevel, $section, $program]);
    } elseif ($hasLabFlag && $hasLectureUnits && $hasLaboratoryUnits && $hasIsGened) {
        db()->prepare(
            'INSERT INTO courses (course_code, course_name, units, lecture_units, laboratory_units, is_laboratory, is_gened,
department, college_id)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, NULL)'
        )->execute([$code, $name, $legacyUnits, $lecUnits, $labUnits, $isLab, 1, $program]);
    } else {
        throw new RuntimeException('Run upgrade_roles.php first to enable GE course management.');
```

```

    }
    $_SESSION['flash'] = 'GE course added.';
} elseif ($action === 'edit' && isset($_POST['id'])) {
    $id = (int) $_POST['id'];
    if ($hasLabFlag && $hasLectureUnits && $hasLaboratoryUnits && $hasIsGened && $hasCourseBlock) {
        db()->prepare(
            'UPDATE courses SET course_code=?, course_name=?, units=?, lecture_units=?, laboratory_units=?, is_laboratory=?,
year_level=?, section=?, department=?
            WHERE id=? AND is_gened=1'
        )->execute([$code, $name, $legacyUnits, $lecUnits, $labUnits, $isLab, $yearLevel, $section, $program, $id]);
    } elseif ($hasLabFlag && $hasLectureUnits && $hasLaboratoryUnits && $hasIsGened) {
        db()->prepare(
            'UPDATE courses SET course_code=?, course_name=?, units=?, lecture_units=?, laboratory_units=?, is_laboratory=?,
department=?
            WHERE id=? AND is_gened=1'
        )->execute([$code, $name, $legacyUnits, $lecUnits, $labUnits, $isLab, $program, $id]);
    } else {
        throw new RuntimeException('Run upgrade_roles.php first to enable GE course management.');
```

```

    }
    $_SESSION['flash'] = 'GE course updated.';
} elseif ($action === 'delete' && isset($_POST['id'])) {
    db()->prepare('DELETE FROM courses WHERE id=? AND is_gened=1')->execute([(int) $_POST['id']]);
    $_SESSION['flash'] = 'GE course removed.';
}
} catch (Throwable $e) {

```

```

        $msg = $e->getMessage();
        if ($e instanceof PDOException && str_contains($msg, '1062') && str_contains(strtolower($msg), 'course_code')) {
            // Retry auto-heal message if DB user cannot alter schema.
            $msg = 'Your database still blocks duplicate course codes (legacy UNIQUE index). Please run upgrade_roles.php once.';
        }
        $_SESSION['flash'] = 'Error: ' . $msg;
    }
    header('Location: gened_courses.php');
    exit;
}

$editRow = null;
if (isset($_GET['edit'])) {
    $st = db()->prepare('SELECT * FROM courses WHERE id=? AND is_gened=1');
    $st->execute([(int) $_GET['edit']]);
    $editRow = $st->fetch() ?: null;
}

$search = trim((string) ($_GET['q'] ?? ''));
$programFilter = trim((string) ($_GET['program'] ?? ''));
$sort = (string) ($_GET['sort'] ?? 'course_code');
$dir = strtolower((string) ($_GET['dir'] ?? 'asc')) === 'desc' ? 'DESC' : 'ASC';

$sortMap = [
    'course_code' => 'course_code',
    'course_name' => 'course_name',
    'program' => 'department',
    'lecture_units' => $hasLectureUnits ? 'lecture_units' : 'units',
    'laboratory_units' => $hasLaboratoryUnits ? 'laboratory_units' : 'units',
];
if ($hasYearLevel) {
    $sortMap['year_level'] = 'year_level';
}
if ($hasSection) {
    $sortMap['section'] = 'section';
}
if ($hasLabFlag) {
    $sortMap['type'] = 'is_laboratory';
}
$sortExpr = $sortMap[$sort] ?? 'course_code';

$sql = 'SELECT * FROM courses WHERE is_gened=1';
$params = [];
if ($programFilter !== '') {
    $sql .= ' AND department = ?';
    $params[] = $programFilter;
}
if ($search !== '') {
    $sql .= ' AND (course_code LIKE ? OR course_name LIKE ? OR department LIKE ?';
    $params[] = '%' . $search . '%';
    $params[] = '%' . $search . '%';
    $params[] = '%' . $search . '%';
    if ($hasYearLevel) {
        $sql .= ' OR year_level LIKE ?';
        $params[] = '%' . $search . '%';
    }
    if ($hasSection) {
        $sql .= ' OR section LIKE ?';
        $params[] = '%' . $search . '%';
    }
    $sql .= ')';
}
$sql .= " ORDER BY {$sortExpr} {$dir}, course_code ASC";
$st = db()->prepare($sql);
$st->execute($params);
$list = $st->fetchAll();

$currentUser = current_user() ?? [];
$coordinatorName = trim((string) ($currentUser['full_name'] ?? 'GEN ED Coordinator'));
$coordinatorInitials = '';
foreach (preg_split('/\s+/', $coordinatorName) as $part) {
    $token = trim((string) $part, " ,.\t\n\r\0\x0B");
    if ($token !== '') {
        $coordinatorInitials .= strtoupper(substr($token, 0, 1));
    }
}
$coordinatorInitials = $coordinatorInitials !== '' ? substr($coordinatorInitials, 0, 2) : 'GE';

$tableColCount = 6 + ($hasYearLevel ? 1 : 0) + ($hasSection ? 1 : 0) + 1;

```

```

$pageTitle = 'GE Courses';
require_once __DIR__ . '/includes/header.php';
?>
<style>
    .faculty-dashboard-container { max-width: 1440px; margin: 0 auto; }
    .faculty-university-header { display:flex; justify-content:space-between; align-items:flex-end; flex-wrap:wrap; margin-bottom:28px;
border-bottom:2px solid rgba(30,64,95,.2); padding-bottom:16px; gap: 12px; }
    .faculty-title-section h1 { font-size:1.9rem; font-weight:700; background:linear-gradient(135deg,#1e405f,#2a6f8f); -webkit-background-
clip:text; background-clip:text; color:transparent; letter-spacing:-.3px; }
    .faculty-title-section p { font-size:.9rem; color:#4a627a; margin-top:6px; font-weight:500; }
    .faculty-dean-card { background:#fff; padding:10px 24px; border-radius:60px; box-shadow:0 4px 10px rgba(0,0,0,.02),0 1px 2px
rgba(0,0,0,.05); display:flex; align-items:center; gap:16px; border:1px solid #e2edf2; }
    .faculty-dean-avatar { background:#1e5a6f; width:44px; height:44px; border-radius:50%; display:flex; align-items:center; justify-
content:center; color:#fff; font-weight:600; font-size:1.2rem; }
    .faculty-dean-info h4 { font-weight:600; font-size:.9rem; color:#2c4c6e; margin: 0; }
    .faculty-dean-info p { font-size:.75rem; color:#4c6a82; font-weight:500; margin: 0; }
    .faculty-layout { display:flex; flex-direction:column; gap:32px; }
    .faculty-card { background:#fff; border-radius:28px; box-shadow:0 12px 28px rgba(0,0,0,.05),0 0 1px rgba(0,0,0,.02); overflow:hidden; }
    .faculty-card-header { padding:20px 28px 12px; border-bottom:1px solid #edf2f7; display:flex; justify-content:space-between; align-
items:center; flex-wrap:wrap; gap: 10px; }
    .faculty-card-header h2 { font-size:1.35rem; font-weight:600; color:#1f3b4c; display:flex; align-items:center; gap:10px; margin: 0; }
    .faculty-badge-manage { background:#eef2ff; padding:6px 14px; border-radius:30px; font-size:.7rem; font-weight:600; color:#2c5f8a; }
    .faculty-form-grid { display:grid; grid-template-columns:repeat(auto-fit,minmax(280px,1fr)); gap:20px 28px; padding:28px; }
    .faculty-input-group { display:flex; flex-direction:column; gap:8px; }
    .faculty-input-group label { font-size:.75rem; font-weight:600; text-transform:uppercase; letter-spacing:.5px; color:#4a6b85; }
    .faculty-input-group input, .faculty-input-group select { padding:12px 14px; border-radius:16px; border:1.5px solid #e2edf2;
background:#fefefe; font-size:.9rem; transition:all .2s; }
    .faculty-input-group input:focus, .faculty-input-group select:focus { outline:none; border-color:#2c8cac; box-shadow:0 0 3px
rgba(44,140,172,.2); }
    .faculty-row-flex { display:flex; gap:20px; flex-wrap:wrap; align-items:flex-end; grid-column: 1 / -1; }
    .faculty-checkbox-row { display:flex; align-items:center; gap:10px; grid-column: 1 / -1; color:#2a445b; }
    .faculty-checkbox-row input[type="checkbox"] { width:1.1rem; height:1.1rem; }
    .faculty-save-btn { background:#1f6e8c; border:none; padding:12px 24px; border-radius:40px; font-weight:600; color:#fff; font-size:.9rem;
display:inline-flex; align-items:center; gap:12px; cursor:pointer; transition:.2s; }
    .faculty-save-btn:hover { background:#0e556f; transform:translateY(-1px); box-shadow:0 6px 12px rgba(0,0,0,.05); }
    .faculty-apply-btn { background:#f0f6fa; border:1.5px solid #cfe3ee; padding:12px 24px; border-radius:40px; font-weight:600;
color:#1e5a6f; font-size:.9rem; cursor:pointer; transition:.2s; }
    .faculty-apply-btn:hover { background:#e4f0f7; border-color:#b8d4e4; }
    .faculty-cancel-link { border-radius: 40px; padding: 10px 20px; }
    .faculty-table-wrapper { overflow-x:auto; padding:0 0 8px; }
    .faculty-table { width:100%; border-collapse:collapse; font-size:.85rem; }
    .faculty-table th { text-align:left; padding:18px 16px; background:#f9fbfd; font-weight:600; color:#2c4c6e; border-bottom:1px solid
#e6edf2; }
    .faculty-table td { padding:14px 16px; border-bottom:1px solid #eff3f8; vertical-align:middle; color:#2a445b; }
    .faculty-table tr:hover td { background:#fafcff; }
    .faculty-status-badge { display:inline-block; padding:4px 12px; border-radius:50px; font-size:.7rem; font-weight:600; text-
transform:capitalize; }
    .faculty-type-lecture { background:#e8f4fc; color:#1a5f7a; }
    .faculty-type-lab { background:#f3e8ff; color:#5b2c8a; }
    .faculty-action-buttons { display:flex; gap:8px; align-items:center; }
    .faculty-icon-btn { border:none; border-radius:30px; background:transparent; color:#2c7da0; padding:6px 9px; }
    .faculty-icon-btn.delete { color:#bc6c6c; }
    .faculty-icon-btn:hover { background:#eef2ff; color:#0f5c7c; }
    .faculty-icon-btn.delete:hover { background:#fff0f0; color:#b13e3e; }
    .faculty-manage-programs-link { background:#f7f9fc; border-radius:20px; padding:8px 20px; font-size:.8rem; font-weight:500; color:#2a6f8f;
text-decoration:none; transition:.2s; }
    .faculty-manage-programs-link:hover { background:#eaf3f8; color:#2a6f8f; }
    .faculty-footer-note { margin-top:32px; text-align:center; font-size:.7rem; color:#6b89a0; border-top:1px solid #e0eaf0; padding-top:24px;
}

html[data-bs-theme="dark"] .faculty-title-section h1 { background: linear-gradient(135deg, #c5e6ff, #79c5e8); -webkit-background-
clip:text; background-clip:text; color: transparent; }
html[data-bs-theme="dark"] .faculty-title-section p { color: #9db6c8; }
html[data-bs-theme="dark"] .faculty-dean-card,
html[data-bs-theme="dark"] .faculty-card { background: #16202a; border-color: #2a3947; box-shadow: 0 10px 24px rgba(0,0,0,.35); }
html[data-bs-theme="dark"] .faculty-dean-info h4,
html[data-bs-theme="dark"] .faculty-card-header h2,
html[data-bs-theme="dark"] .faculty-table th { color: #d3e7f6; }
html[data-bs-theme="dark"] .faculty-dean-info p,
html[data-bs-theme="dark"] .faculty-input-group label,
html[data-bs-theme="dark"] .faculty-footer-note { color: #9ab0c2; }
html[data-bs-theme="dark"] .faculty-card-header { border-bottom-color: #2b3a49; }
html[data-bs-theme="dark"] .faculty-input-group input,
html[data-bs-theme="dark"] .faculty-input-group select { background:#0f1821; border-color:#2e4152; color:#deebf5; }
html[data-bs-theme="dark"] .faculty-input-group input:focus,
html[data-bs-theme="dark"] .faculty-input-group select:focus { border-color:#4ea4c6; box-shadow:0 0 3px rgba(78,164,198,.28); }
html[data-bs-theme="dark"] .faculty-checkbox-row { color:#c5d9e7; }
html[data-bs-theme="dark"] .faculty-manage-programs-link { background:#1d2b38; color:#9ed0eb; }
html[data-bs-theme="dark"] .faculty-manage-programs-link:hover { background:#253747; color:#b7ddf3; }
html[data-bs-theme="dark"] .faculty-badge-manage { background:#223243; color:#a9d2ea; }
html[data-bs-theme="dark"] .faculty-table th { background:#1b2834; border-bottom-color:#30414f; }

```



```

html[data-bs-theme="dark"] .faculty-table td { color:#c8dced; border-bottom-color:#2b3a47; }
html[data-bs-theme="dark"] .faculty-table tr:hover td { background:#1a2530; }
html[data-bs-theme="dark"] .faculty-type-lecture { background:#1a3040; color:#9ed4f0; }
html[data-bs-theme="dark"] .faculty-type-lab { background:#2d2440; color:#d4b8f0; }
html[data-bs-theme="dark"] .faculty-icon-btn { color:#90c7e5; }
html[data-bs-theme="dark"] .faculty-icon-btn:hover { background:#233344; color:#c4e6f9; }
html[data-bs-theme="dark"] .faculty-icon-btn.delete { color:#e09a9a; }
html[data-bs-theme="dark"] .faculty-icon-btn.delete:hover { background:#3f262b; color:#ffc4c4; }
html[data-bs-theme="dark"] .faculty-footer-note { border-top-color:#2a3a49; }
html[data-bs-theme="dark"] .faculty-apply-btn { background:#1d2b38; border-color:#304a5c; color:#9ed0eb; }
html[data-bs-theme="dark"] .faculty-apply-btn:hover { background:#253747; }
@media (max-width: 700px) {
    .faculty-card-header { flex-direction:column; align-items:flex-start; }
}
</style>

<div class="faculty-dashboard-container">
    <div class="faculty-university-header">
        <div class="faculty-title-section">
            <h1><i class="fa-solid fa-book" style="color:#2c7da0; margin-right:8px;"></i>Gen Ed Courses</h1>
            <p>General Education · Western Philippines University</p>
        </div>
        <div class="faculty-dean-card">
            <div class="faculty-dean-avatar"><?= htmlspecialchars($coordinatorInitials) ?></div>
            <div class="faculty-dean-info">
                <h4><?= htmlspecialchars(strtoupper($coordinatorName)) ?></h4>
                <p>GEN ED Coordinator <i class="fa-solid fa-circle-check" style="font-size:.7rem; color:#1f8a4c;"></i></p>
            </div>
        </div>
    </div>

    <?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

    <?php if (!$hasLabFlag || !$hasLectureUnits || !$hasLaboratoryUnits || !$hasIsGened): ?>
        <div class="alert alert-warning">Run <a href="upgrade_roles.php">upgrade_roles.php</a> to enable GE course features.</div>
    <?php endif; ?>
    <?php if (!$hasCourseBlock): ?>
        <div class="alert alert-warning">Year level/section fields are disabled until you run <a href="upgrade_roles.php">upgrade_roles.php</a>.</div>
    <?php endif; ?>
    <?php if (!$hasProgramsTable): ?>
        <div class="alert alert-warning">Program dropdown is disabled until you run <a href="upgrade_roles.php">upgrade_roles.php</a>.</div>
    <?php endif; ?>

    <div class="faculty-layout">
        <div class="faculty-card">
            <div class="faculty-card-header">
                <h2><i class="fa-solid fa-magnifying-glass"></i> Search & sort</h2>
                <span class="faculty-badge-manage"><i class="fa-solid fa-filter me-1"></i>Refine the roster</span>
            </div>
            <form method="get" class="faculty-form-grid">
                <div class="faculty-input-group">
                    <label>Search</label>
                    <input type="text" name="q" placeholder="Code, name, program, year, section" value="<?= htmlspecialchars($search) ?>">
                </div>
                <div class="faculty-input-group">
                    <label>Program</label>
                    <select name="program">
                        <option value="">All programs</option>
                        <?php foreach ($programOptions as $programRow): ?>
                            <?php
                                $programLabel = (string) ($programRow['college_code'] ?? '') . ' - ' . (string) ($programRow['program_name'] ?? '');
                                $programName = (string) ($programRow['program_name'] ?? '');
                            <?>
                            <option value="<?= htmlspecialchars($programName) ?>" <?= $programFilter === $programName ? 'selected' : '' ?>>
                                <?= htmlspecialchars($programLabel) ?>
                            </option>
                        <?php endforeach; ?>
                    </select>
                </div>
                <div class="faculty-input-group">
                    <label>Sort by</label>
                    <select name="sort">
                        <option value="course_code" <?= $sort === 'course_code' ? 'selected' : '' ?>>Code</option>
                        <option value="course_name" <?= $sort === 'course_name' ? 'selected' : '' ?>>Name</option>
                        <option value="program" <?= $sort === 'program' ? 'selected' : '' ?>>Program</option>
                        <option value="lecture_units" <?= $sort === 'lecture_units' ? 'selected' : '' ?>>Lecture units</option>
                        <option value="laboratory_units" <?= $sort === 'laboratory_units' ? 'selected' : '' ?>>Laboratory units</option>
                        <?php if ($hasYearLevel): ?><option value="year_level" <?= $sort === 'year_level' ? 'selected' : '' ?>>Year

```

```

level</option><?php endif; ?>
        <?php if ($hasSection): ?><option value="section" <?=$sort === 'section' ? 'selected' : '' ?>>Section</option><?php
endif; ?>
        <?php if ($hasLabFlag): ?><option value="type" <?=$sort === 'type' ? 'selected' : '' ?>>Type</option><?php endif; ?
>
        </select>
    </div>
    <div class="faculty-input-group">
        <label>Direction</label>
        <select name="dir">
            <option value="asc" <?=$dir === 'ASC' ? 'selected' : '' ?>>Ascending</option>
            <option value="desc" <?=$dir === 'DESC' ? 'selected' : '' ?>>Descending</option>
        </select>
    </div>
    <div class="faculty-row-flex" style="justify-content: flex-end;">
        <button type="submit" class="faculty-apply-btn" <?=$app_tooltip_attr('Applies search, sort, and filter to the GE course
list.') ?>><i class="fa-solid fa-check"></i> Apply</button>
    </div>
</form>
</div>

<div class="faculty-card">
    <div class="faculty-card-header">
        <h2><i class="fa-solid fa-book-open"></i> <?=$editRow ? 'Edit GE course' : 'Add GE course' ?></h2>
        <a href="gened_faculty.php" class="faculty-manage-programs-link"><i class="fa-solid fa-chalkboard-user me-1"></i>GE
faculty</a>
    </div>
    <form method="post" class="faculty-form-grid">
        <input type="hidden" name="action" value="<?=$editRow ? 'edit' : 'add' ?>">
        <?php if ($editRow): ?><input type="hidden" name="id" value="<?=(int) $editRow['id'] ?>"><?php endif; ?>
        <div class="faculty-input-group"><label>Course code</label><input name="course_code" required maxlength="20" value="<?=$
htmlspecialchars((string) ($editRow['course_code'] ?? '')) ?>"></div>
        <div class="faculty-input-group" style="grid-column: span 2; min-width: 280px;"><label>Course name</label><input
name="course_name" required maxlength="100" value="<?=$htmlspecialchars((string) ($editRow['course_name'] ?? '')) ?>"></div>
        <div class="faculty-input-group"><label>Lecture units</label><input name="lecture_units" type="number" step="0.1" min="0"
max="12" value="<?=$htmlspecialchars((string) ($editRow['lecture_units'] ?? $editRow['units'] ?? '3.0')) ?>"></div>
        <div class="faculty-input-group"><label>Laboratory units</label><input name="laboratory_units" type="number" step="0.1"
min="0" max="12" value="<?=$htmlspecialchars((string) ($editRow['laboratory_units'] ?? '0.0')) ?>"></div>
        <div class="faculty-checkbox-row">
            <input type="checkbox" name="is_laboratory" value="1" id="ge_is_lab" <?=$!empty($editRow['is_laboratory']) ? 'checked' :
'' ?>>
            <label for="ge_is_lab"><strong>Laboratory course</strong></label>
        </div>
        <?php if ($hasYearLevel): ?>
            <div class="faculty-input-group"><label>Year level</label>
            <select name="year_level" required>
                <?php foreach (['1','2','3','4','5'] as $y1): ?>
                    <option value="<?=$y1 ?>" <?=(string) ($editRow['year_level'] ?? '') === $y1 ? 'selected' : '' ?>><?=$
htmlspecialchars($y1) ?></option>
                <?php endforeach; ?>
            </select>
        </div>
        <?php endif; ?>
        <?php if ($hasSection): ?>
            <div class="faculty-input-group"><label>Section</label><input name="section" maxlength="20" required value="<?=$
htmlspecialchars((string) ($editRow['section'] ?? '')) ?>"></div>
        <?php endif; ?>
        <?php if ($hasProgramsTable): ?>
            <?php
            $selectedProgram = (string) ($editRow['department'] ?? '');
            $selectedProgramKey = '';
            foreach ($programOptions as $programRow) {
                if ((string) ($programRow['program_name'] ?? '') === $selectedProgram) {
                    $selectedProgramKey = (string) ($programRow['college_id'] ?? 0) . '|' . $selectedProgram;
                    break;
                }
            }
            <?>
            <div class="faculty-input-group" style="grid-column: span 2; min-width: 280px;">
                <label>Program</label>
                <select name="program" required>
                    <option value="" disabled<?=$selectedProgram === '' ? 'selected' : '' ?>>Select program...</option>
                    <?php if ($selectedProgram !== '' && $selectedProgramKey === ''): ?>
                        <option value="<?=$htmlspecialchars($selectedProgram) ?>" selected<?=$htmlspecialchars($selectedProgram) ?>
(legacy)</option>
                    <?php endif; ?>
                    <?php foreach ($programOptions as $programRow): ?>
                        <?php
                        $programName = (string) ($programRow['program_name'] ?? '');
                        $programKey = (string) ($programRow['college_id'] ?? 0) . '|' . $programName;

```

```

        $programLabel = (string) ($programRow['college_code'] ?? '') . ' - ' . $programName;
    }
    <option value="<?> htmlspecialchars($programKey) ?>" <?> $selectedProgramKey === $programKey ? 'selected' :
'' ?>>

        <?> htmlspecialchars($programLabel) ?>
    </option>
    <?php endforeach; ?>
</select>
</div>
<?php else: ?>
    <div class="faculty-input-group" style="grid-column: span 2; min-width: 280px;">
        <label>Program</label>
        <input name="program" value="<?> htmlspecialchars((string) ($editRow['department'] ?? '')) ?>" placeholder="e.g. BS
Information Systems">
    </div>
    <?php endif; ?>
    <div style="display:flex; justify-content:flex-end; gap: 10px; grid-column: 1 / -1;">
        <?php if ($editRow): ?><a href="gened_courses.php" class="btn btn-outline-secondary faculty-cancel-link"<?>
app_tooltip_attr('Discards unsaved edits and returns to the list.') ?>>Cancel</a><?php endif; ?>
        <button type="submit" class="faculty-save-btn"<?> app_tooltip_attr($editRow ? 'Saves changes to this GE course.' :
'Creates the GE course record for scheduling and catalog use.') ?>><i class="fa-solid <?> $editRow ? 'fa-pen' : 'fa-save' ?>"></i> <?>
$editRow ? 'Update course' : 'Save course' ?></button>
    </div>
</form>
</div>

<div class="faculty-card">
    <div class="faculty-card-header">
        <h2><i class="fa-solid fa-list-ul"></i> GE course catalog</h2>
        <span class="faculty-badge-manage"><i class="fa-solid fa-pen me-1"></i>Manage GE courses</span>
    </div>
    <div class="faculty-table-wrapper">
        <table class="faculty-table">
            <thead>
                <tr>
                    <th>Code</th>
                    <th>Course Title</th>
                    <th>Program</th>
                    <th>Lec units</th>
                    <th>Lab units</th>
                    <th>Type</th>
                    <?php if ($hasYearLevel): ?><th>Year</th><?php endif; ?>
                    <?php if ($hasSection): ?><th>Section</th><?php endif; ?>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($list as $r): ?>
                    <tr>
                        <td><?> htmlspecialchars((string) $r['course_code']) ?></td>
                        <td><?> htmlspecialchars((string) $r['course_name']) ?></td>
                        <td><?> htmlspecialchars((string) ($r['department'] ?? '')) ?></td>
                        <td><?> htmlspecialchars((string) ($r['lecture_units'] ?? $r['units'])) ?></td>
                        <td><?> htmlspecialchars((string) ($r['laboratory_units'] ?? 0)) ?></td>
                        <td>
                            <?php if (!empty($r['is_laboratory'])): ?>
                                <span class="faculty-status-badge faculty-type-lab">Laboratory</span>
                            <?php else: ?>
                                <span class="faculty-status-badge faculty-type-lecture">Lecture</span>
                            <?php endif; ?>
                        </td>
                        <?php if ($hasYearLevel): ?><td><?> htmlspecialchars((string) ($r['year_level'] ?? '')) ?></td><?php endif; ?>
                        <?php if ($hasSection): ?><td><?> htmlspecialchars((string) ($r['section'] ?? '')) ?></td><?php endif; ?>
                        <td class="faculty-action-buttons">
                            <a href="gened_courses.php?edit=<?> (int) $r['id'] ?>" class="faculty-icon-btn" title="Edit GE course"<?>
app_tooltip_attr('Opens this GE course for editing.') ?>><i class="fa-solid fa-pen"></i></a>
                            <form method="post" class="d-inline onsubmit="return confirm('Delete this GE course?');">
                                <input type="hidden" name="action" value="delete">
                                <input type="hidden" name="id" value="<?> (int) $r['id'] ?>">
                                <button type="submit" class="faculty-icon-btn delete" title="Delete GE course"<?>
app_tooltip_attr('Deletes this GE course after confirmation if no schedules depend on it.') ?>><i class="fa-solid fa-trash-can"></i>
                            </button>
                        </td>
                    </tr>
                </tbody>
            </table>
            <?php endforeach; ?>
            <?php if ($list === []): ?>
                <tr><td colspan="<?> (int) $tableColCount ?>" class="text-center text-muted py-4">No GE courses match your filters.
Add one above or adjust search.</td></tr>
            <?php endif; ?>

```

```

        </tbody>
    </table>
</div>
</div>
</div>

<div class="faculty-footer-note">
    <i class="fa-solid fa-database"></i> General Education course catalog · Western Philippines University
</div>
</div>

```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>
```

```
-----
FILE: gened_edit_schedule.php
-----
```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['gened']);
$hasGeScheduleTargets = db_table_exists('ge_schedule_targets');

$id = isset($_GET['id']) ? (int) $_GET['id'] : 0;
if ($id < 1) {
    header('Location: schedule.php');
    exit;
}

$targetSelect = $hasGeScheduleTargets
    ? ', gst.program_name AS target_program, gst.year_level AS target_year_level, gst.section AS target_section'
    : ', ' AS target_program, ' AS target_year_level, ' AS target_section';
$targetJoin = $hasGeScheduleTargets ? ' LEFT JOIN ge_schedule_targets gst ON gst.schedule_id = s.id ' : '';
$stmt = db()->prepare(
    "SELECT s.* {$targetSelect}
    FROM schedules s
    INNER JOIN users u ON u.id = s.created_by
    {$targetJoin}
    WHERE s.id=? AND u.role='gened' LIMIT 1"
);
$stmt->execute([$id]);
$row = $stmt->fetch();
if (!$row) {
    http_response_code(403);
    exit('Only GE-created schedules can be edited by GEN ED account.');
```

```

}

$errors = [];
$old = $_POST ?? [];
if (isset($old['days']) && is_array($old['days'])) {
    $old['day_array'] = $old['days'];
}

$hasIsGenedFaculty = db_column_exists('faculty', 'is_gened');
$hasIsGenedCourse = db_column_exists('courses', 'is_gened');
$hasIsGenedRoom = db_column_exists('rooms', 'is_gened');
$hasCourseYearLevel = db_column_exists('courses', 'year_level');
$hasCourseSection = db_column_exists('courses', 'section');
$hasProgramsTable = db_table_exists('programs');

$programRows = [];
if ($hasProgramsTable) {
    $programRows = db()->query(
        "SELECT p.college_id, p.program_name, c.college_code
        FROM programs p
        INNER JOIN colleges c ON c.id = p.college_id
        WHERE p.status='active'
        ORDER BY c.college_code, p.program_name"
    )->fetchAll();
}

$deanSectionRows = [];
if ($hasCourseYearLevel && $hasCourseSection) {
    $deanSectionRows = db()->query(
        "SELECT DISTINCT college_id, department, year_level, section
        FROM courses

```

```

        WHERE COALESCE(is_gened, 0) = 0
        AND college_id IS NOT NULL
        AND TRIM(COALESCE(department, '')) <> ''
        AND TRIM(COALESCE(section, '')) <> ''
        AND TRIM(COALESCE(year_level, '')) <> ''
        ORDER BY college_id, department, year_level, section"
    )->fetchAll();
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $collegeId = (int) ($_POST['college_id'] ?? 0);
    $facultyId = (int) ($_POST['faculty_id'] ?? 0);
    $courseId = (int) ($_POST['course_id'] ?? 0);
    $roomId = (int) ($_POST['room_id'] ?? 0);
    $scheduleType = (string) ($_POST['schedule_type'] ?? 'Custom');
    $days = isset($_POST['days']) && is_array($_POST['days']) ? array_map('strval', $_POST['days']) : [];
    $startTime = substr((string) ($_POST['start_time'] ?? ''), 0, 5) . ':00';
    $endTime = substr((string) ($_POST['end_time'] ?? ''), 0, 5) . ':00';
    $semester = trim((string) ($_POST['semester'] ?? ''));
    $schoolYear = trim((string) ($_POST['school_year'] ?? ''));
    $academicYear = trim((string) ($_POST['academic_year'] ?? ''));
    $targetProgram = trim((string) ($_POST['target_program'] ?? ''));
    $targetYearLevel = trim((string) ($_POST['target_year_level'] ?? ''));
    $targetSection = trim((string) ($_POST['target_section'] ?? ''));

    if ($collegeId < 1 || $facultyId < 1 || $courseId < 1 || $roomId < 1) {
        $errors[] = 'Please select college, GE faculty, GE course, and GE room.';
    }
    if ($days === []) {
        $errors[] = 'Select at least one day.';
    }
    if ($semester === '' || $schoolYear === '') {
        $errors[] = 'Semester and school year are required.';
    }
    if (!$hasIsGenedFaculty || !$hasIsGenedCourse || !$hasIsGenedRoom) {
        $errors[] = 'Run upgrade_roles.php first to enable GE scheduling.';
    }
    if (!$hasProgramsTable) {
        $errors[] = 'Run upgrade_roles.php first to sync Programs from Dean modules.';
    }
    if ($targetProgram === '' || $targetYearLevel === '' || $targetSection === '') {
        $errors[] = 'Program, year level, and section are required for GE scheduling.';
    }

    $rules = validate_schedule_rules($scheduleType, $days, $startTime, $endTime);
    foreach ($rules['errors'] as $e) {
        $errors[] = $e;
    }

    if (!$errors) {
        $chk = db()->prepare("SELECT COUNT(*) FROM colleges WHERE id=? AND status='active'");
        $chk->execute([$collegeId]);
        if ((int) $chk->fetchColumn() < 1) {
            $errors[] = 'Selected college is invalid.';
        }
        $chk = db()->prepare("SELECT COUNT(*) FROM faculty WHERE id=? AND is_gened=1 AND status='active'");
        $chk->execute([$facultyId]);
        if ((int) $chk->fetchColumn() < 1) {
            $errors[] = 'Selected faculty is not a GE faculty member.';
        }
        $chk = db()->prepare('SELECT COUNT(*) FROM courses WHERE id=? AND is_gened=1');
        $chk->execute([$courseId]);
        if ((int) $chk->fetchColumn() < 1) {
            $errors[] = 'Selected course is not a GE course.';
        }
        $chk = db()->prepare("SELECT COUNT(*) FROM rooms WHERE id=? AND is_gened=1 AND status IN ('available','tba')");
        $chk->execute([$roomId]);
        if ((int) $chk->fetchColumn() < 1) {
            $errors[] = 'Selected room is not an available GE room.';
        }

        $chk = db()->prepare('SELECT COUNT(*) FROM ge_course_colleges WHERE course_id=? AND college_id=?');
        $chk->execute([$courseId, $collegeId]);
        if ((int) $chk->fetchColumn() < 1) {
            $errors[] = 'This GE course is not assigned to the selected college. Configure it in GE Offerings first.';
        }

        if ($targetProgram !== '') {
            $chk = db()->prepare("SELECT COUNT(*) FROM programs WHERE college_id=? AND program_name=? AND status='active'");
            $chk->execute([$collegeId, $targetProgram]);
        }
    }
}

```

```

        if ((int) $chk->fetchColumn() < 1) {
            $errors[] = 'Selected program is invalid for the chosen college.';
        }
    }
}

$conflicts = [];
if (!$errors) {
    $conflicts = checkConflicts($facultyId, $roomId, $days, $startTime, $endTime, $semester, $schoolYear, $id, $collegeId);
    foreach ($conflicts as $c) {
        $errors[] = $c['description'];
    }
}

$stmtBlock = db()->prepare(
    'SELECT s.id, s.day_of_week, s.start_time, s.end_time, c.course_code
    FROM schedules s
    INNER JOIN courses c ON c.id = s.course_id
    WHERE s.college_id = ?
    AND s.semester = ?
    AND s.school_year = ?
    AND c.department = ?
    AND c.year_level = ?
    AND c.section = ?
    AND s.id <> ?'
);
$stmtBlock->execute([$collegeId, $semester, $schoolYear, $targetProgram, $targetYearLevel, $targetSection, $id]);
$blockRows = $stmtBlock->fetchAll();
$stmtMin = time_to_minutes($startTime);
$enMin = time_to_minutes($endTime);
foreach ($blockRows as $br) {
    $existingDays = parse_day_set((string) $br['day_of_week']);
    if (!array_intersect($days, $existingDays)) {
        continue;
    }
    $rowStart = time_to_minutes((string) $br['start_time']);
    $rowEnd = time_to_minutes((string) $br['end_time']);
    if (intervals_overlap($stmtMin, $enMin, $rowStart, $rowEnd)) {
        $errors[] = 'Target block conflict: ' . $targetProgram . ' Y' . $targetYearLevel . '-' . $targetSection
            . ' already has ' . $br['course_code'] . ' at this time.';
        break;
    }
}

if ($hasGeScheduleTargets) {
    $stTarget = db()->prepare(
        'SELECT s.id, s.day_of_week, s.start_time, s.end_time, c.course_code
        FROM ge_schedule_targets gst
        INNER JOIN schedules s ON s.id = gst.schedule_id
        INNER JOIN courses c ON c.id = s.course_id
        WHERE gst.college_id = ?
        AND gst.program_name = ?
        AND gst.year_level = ?
        AND gst.section = ?
        AND s.semester = ?
        AND s.school_year = ?
        AND s.id <> ?'
    );
    $stTarget->execute([$collegeId, $targetProgram, $targetYearLevel, $targetSection, $semester, $schoolYear, $id]);
    $targetRows = $stTarget->fetchAll();
    foreach ($targetRows as $tr) {
        $existingDays = parse_day_set((string) $tr['day_of_week']);
        if (!array_intersect($days, $existingDays)) {
            continue;
        }
        $rowStart = time_to_minutes((string) $tr['start_time']);
        $rowEnd = time_to_minutes((string) $tr['end_time']);
        if (intervals_overlap($stmtMin, $enMin, $rowStart, $rowEnd)) {
            $errors[] = 'GE block conflict: another GE schedule (' . $tr['course_code'] . ') already targets '
                . $targetProgram . ' Y' . $targetYearLevel . '-' . $targetSection . ' at this time.';
            break;
        }
    }
}

if (!$errors) {
    $beforeGe = [
        'id' => (int) $row['id'],
        'faculty_id' => (int) $row['faculty_id'],
        'course_id' => (int) $row['course_id'],
    ]
}

```

```

        'room_id' => (int) $row['room_id'],
        'college_id' => (int) $row['college_id'],
        'schedule_type' => (string) $row['schedule_type'],
        'day_of_week' => (string) $row['day_of_week'],
        'start_time' => (string) $row['start_time'],
        'end_time' => (string) $row['end_time'],
        'semester' => (string) $row['semester'],
        'school_year' => (string) $row['school_year'],
        'academic_year' => (string) ($row['academic_year'] ?? ''),
    ];
    db()->prepare(
        'UPDATE schedules
        SET faculty_id=?, course_id=?, room_id=?, college_id=?, schedule_type=?, day_of_week=?, start_time=?, end_time=?, semester=?,
school_year=?, academic_year=?
        WHERE id=?'
    );
    db()->execute([
        $facultyId,
        $courseId,
        $roomId,
        $collegeId,
        $scheduleType,
        days_to_set($days),
        $startTime,
        $endTime,
        $semester,
        $schoolYear,
        $academicYear,
        $id,
    ]);
    if ($hasGeScheduleTargets && $targetProgram !== '' && $targetYearLevel !== '' && $targetSection !== '') {
        db()->prepare(
            'INSERT INTO ge_schedule_targets (schedule_id, college_id, program_name, year_level, section)
            VALUES (?, ?, ?, ?, ?)
            ON DUPLICATE KEY UPDATE
            college_id = VALUES(college_id),
            program_name = VALUES(program_name),
            year_level = VALUES(year_level),
            section = VALUES(section)'
        );
        db()->execute([$id, $collegeId, $targetProgram, $targetYearLevel, $targetSection]);
    }
    if ($conflicts) {
        log_conflicts($id, $conflicts, false);
    }
    $stGeA = db()->prepare(
        'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year,
        s.day_of_week, s.start_time, s.end_time, s.schedule_type,
        f.full_name AS faculty_name, c.course_code, r.room_code
        FROM schedules s
        INNER JOIN faculty f ON f.id = s.faculty_id
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN rooms r ON r.id = s.room_id
        WHERE s.id = ? LIMIT 1'
    );
    $stGeA->execute([$id]);
    $afterGe = $stGeA->fetch(PDO::FETCH_ASSOC);
    log_user_activity('edit', 'Schedules (GEN ED)', 'GE schedule #' . $id, $beforeGe, $afterGe ? (array) $afterGe : null);
    $_SESSION['flash'] = 'GE schedule updated.';
    header('Location: schedule.php');
    exit;
}
} else {
    $old = $row;
    $old['day_array'] = parse_day_set((string) $row['day_of_week']);
}

$facultyList = $hasIsGenedFaculty
    ? db()->query("SELECT id, faculty_id, full_name FROM faculty WHERE status='active' AND is_gened=1 ORDER BY full_name")->fetchAll()
    : [];
$courseList = $hasIsGenedCourse
    ? db()->query("SELECT id, course_code, course_name FROM courses WHERE is_gened=1 ORDER BY course_code")->fetchAll()
    : [];
$roomList = $hasIsGenedRoom
    ? db()->query("SELECT id, room_code, room_name FROM rooms WHERE is_gened=1 AND status IN ('available', 'tba') ORDER BY room_code")->fetchAll()
    : [];
$collegeList = db()->query("SELECT id, college_code, college_name FROM colleges WHERE status='active' ORDER BY college_code")->fetchAll();
$days = schedule_days_list();
$semesters = ['1st Semester', '2nd Semester', 'Summer'];
$types = ['MW', 'TTH', 'MWF', 'TTHS', 'Saturday', 'Sunday', 'MW_TTH', 'Custom'];
$selDays = isset($old['day_array']) ? (array) $old['day_array'] : [];

```

```

$programsByCollege = [];
foreach ($programRows as $pr) {
    $cid = (int) $pr['college_id'];
    if (!isset($programsByCollege[$cid])) {
        $programsByCollege[$cid] = [];
    }
    $programsByCollege[$cid][] = [
        'name' => (string) $pr['program_name'],
        'college_code' => (string) $pr['college_code'],
    ];
}
$sectionsByScope = [];
foreach ($deanSectionRows as $sr) {
    $cid = (int) ($sr['college_id'] ?? 0);
    $program = (string) ($sr['department'] ?? '');
    $yl = (string) ($sr['year_level'] ?? '');
    $sec = (string) ($sr['section'] ?? '');
    if ($cid < 1 || $program === '' || $yl === '' || $sec === '') {
        continue;
    }
    if (!isset($sectionsByScope[$cid])) {
        $sectionsByScope[$cid] = [];
    }
    if (!isset($sectionsByScope[$cid][$program])) {
        $sectionsByScope[$cid][$program] = [];
    }
    if (!isset($sectionsByScope[$cid][$program][$yl])) {
        $sectionsByScope[$cid][$program][$yl] = [];
    }
    if (!in_array($sec, $sectionsByScope[$cid][$program][$yl], true)) {
        $sectionsByScope[$cid][$program][$yl][] = $sec;
    }
}

$pageTitle = 'Edit GE Schedule';
require_once __DIR__ . '/includes/header.php';
?>
<h1 class="h3 mb-4"><i class="fa-solid fa-pen me-2 text-primary"></i>Edit GE schedule</h1>
<?php render_schedule_errors_warning_popup($errors); ?>

<div class="card shadow-sm">
    <div class="card-body">
        <form method="post" class="row g-3">
            <div class="col-md-4">
                <label class="form-label">Target college</label>
                <select name="college_id" id="college_id" class="form-select" required>
                    <option value="">— Select —</option>
                    <?php foreach ($collegeList as $c): ?>
                        <option value="<?=(int) $c['id'] ?>" <?=(int) ($old['college_id'] ?? 0) === (int) $c['id'] ? 'selected' : '' ?>>
                            <?=(htmlspecialchars($c['college_code'] . ' - ' . $c['college_name'])) ?>
                        </option>
                    <?php endforeach; ?>
                </select>
            </div>
            <div class="col-md-4">
                <label class="form-label">Target program</label>
                <select name="target_program" id="target_program" class="form-select" required>
                    <option value="">— Select program —</option>
                </select>
            </div>
            <div class="col-md-2">
                <label class="form-label">Target year level</label>
                <select name="target_year_level" id="target_year_level" class="form-select" required>
                    <option value="">— Select —</option>
                    <?php foreach (['1','2','3','4','5'] as $yl): ?>
                        <option value="<?=$yl ?>" <?=((string) ($old['target_year_level'] ?? '')) === $yl ? 'selected' : '' ?>>
                            <?=(htmlspecialchars($yl)) ?></option>
                    <?php endforeach; ?>
                </select>
            </div>
            <div class="col-md-2">
                <label class="form-label">Target section</label>
                <?php if ($hasCourseYearLevel && $hasCourseSection): ?>
                    <select name="target_section" id="target_section" class="form-select" required>
                        <option value="">— Select section —</option>
                    </select>
                <?php else: ?>
                    <input type="text" name="target_section" class="form-control" maxlength="20" required value="<?=(
htmlspecialchars((string) ($old['target_section'] ?? '')) ?>)">
                <?php endif; ?>
            </div>
        </form>
    </div>
</div>

```



```

</div>
<div class="col-md-4">
    <label class="form-label">GE Faculty</label>
    <select name="faculty_id" class="form-select" required>
        <option value="">— Select —</option>
        <?php foreach ($facultyList as $f): ?>
            <option value="<?= (int) $f['id'] ?>" <?= (int) ($old['faculty_id'] ?? 0) === (int) $f['id'] ? 'selected' : '' ?>>
                <?= htmlspecialchars($f['full_name'] . ' (' . $f['faculty_id'] . ')') ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-4">
    <label class="form-label">GE Course</label>
    <select name="course_id" class="form-select" required>
        <option value="">— Select —</option>
        <?php foreach ($courseList as $c): ?>
            <option value="<?= (int) $c['id'] ?>" <?= (int) ($old['course_id'] ?? 0) === (int) $c['id'] ? 'selected' : '' ?>>
                <?= htmlspecialchars($c['course_code'] . ' - ' . $c['course_name']) ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-4">
    <label class="form-label">GE Room</label>
    <select name="room_id" class="form-select" required>
        <option value="">— Select —</option>
        <?php foreach ($roomList as $r): ?>
            <option value="<?= (int) $r['id'] ?>" <?= (int) ($old['room_id'] ?? 0) === (int) $r['id'] ? 'selected' : '' ?>>
                <?= htmlspecialchars($r['room_code'] . ($r['room_name'] ? ' - ' . $r['room_name'] : '')) ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-4">
    <label class="form-label">Schedule type</label>
    <select name="schedule_type" class="form-select">
        <?php foreach ($types as $t): ?>
            <option value="<?= $t ?>" <?= (($old['schedule_type'] ?? 'Custom') === $t) ? 'selected' : '' ?>><?=
htmlspecialchars($t) ?></option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-2"><label class="form-label">Start time</label><input type="time" name="start_time" class="form-control"
value="<?= htmlspecialchars(substr((string) ($old['start_time'] ?? '08:00'), 0, 5)) ?>"></div>
<div class="col-md-2"><label class="form-label">End time</label><input type="time" name="end_time" class="form-control" value="
<?= htmlspecialchars(substr((string) ($old['end_time'] ?? '09:00'), 0, 5)) ?>"></div>
<div class="col-12">
    <label class="form-label">Days</label>
    <div class="d-flex flex-wrap gap-3">
        <?php foreach ($days as $d): ?>
            <div class="form-check">
                <input class="form-check-input" type="checkbox" name="days[]" value="<?= htmlspecialchars($d) ?>" id="d_<?=
htmlspecialchars($d) ?>" <?= in_array($d, $selDays, true) ? 'checked' : '' ?>>
                <label class="form-check-label" for="d_<?= htmlspecialchars($d) ?>"><?= htmlspecialchars($d) ?></label>
            </div>
        <?php endforeach; ?>
    </div>
</div>
<div class="col-md-3">
    <label class="form-label">Semester</label>
    <select name="semester" class="form-select">
        <?php foreach ($semesters as $s): ?><option value="<?= htmlspecialchars($s) ?>" <?= (($old['semester'] ?? '') === $s) ?
'selected' : '' ?>><?= htmlspecialchars($s) ?></option><?php endforeach; ?>
    </select>
</div>
<div class="col-md-3"><label class="form-label">School year</label><input type="text" name="school_year" class="form-control"
value="<?= htmlspecialchars((string) ($old['school_year'] ?? (date('Y') . '-' . (date('Y') + 1)))) ?>"></div>
<div class="col-md-3"><label class="form-label">Academic year</label><input type="text" name="academic_year" class="form-
control" value="<?= htmlspecialchars((string) ($old['academic_year'] ?? '')) ?>"></div>
<div class="col-12"><div class="alert alert-info mb-0">GE schedules are targeted per college program/year/section to avoid block
conflicts with major schedules.</div></div>
<div class="col-12">
    <button type="submit" class="btn btn-primary"><?= app_tooltip_attr('Saves your edits to this GE schedule row.') ?>>Update GE
schedule</button>
    <a href="schedule.php" class="btn btn-outline-secondary"><?= app_tooltip_attr('Returns to the schedule list without further
edits here.') ?>>Back</a>
</div>
</form>
</div>

```

</div>

<script>

```
((() => {
  const data = <? json_encode($programsByCollege, JSON_UNESCAPED_SLASHES) ?>;
  const collegeSel = document.getElementById('college_id');
  const programSel = document.getElementById('target_program');
  const yearSel = document.getElementById('target_year_level');
  const sectionSel = document.getElementById('target_section');
  const sectionData = <? json_encode($sectionsByScope, JSON_UNESCAPED_SLASHES) ?>;
  const selectedProgram = <? json_encode((string) ($old['target_program'] ?? ''), JSON_UNESCAPED_SLASHES) ?>;
  const selectedSection = <? json_encode((string) ($old['target_section'] ?? ''), JSON_UNESCAPED_SLASHES) ?>;

  function renderPrograms() {
    const cid = collegeSel.value;
    const rows = data[cid] || [];
    programSel.innerHTML = '';
    const first = document.createElement('option');
    first.value = '';
    first.textContent = '- Select program -';
    programSel.appendChild(first);
    rows.forEach((r) => {
      const opt = document.createElement('option');
      opt.value = r.name;
      opt.textContent = r.name;
      if (selectedProgram && selectedProgram === r.name) {
        opt.selected = true;
      }
      programSel.appendChild(opt);
    });
  }

  function renderSections() {
    if (!sectionSel || !yearSel) {
      return;
    }
    const cid = collegeSel ? collegeSel.value : '';
    const program = programSel ? programSel.value : '';
    const y1 = yearSel.value;
    const rows = (((sectionData[cid] || {})[program] || {})[y1] || []);
    sectionSel.innerHTML = '';
    const first = document.createElement('option');
    first.value = '';
    first.textContent = '- Select section -';
    sectionSel.appendChild(first);
    rows.forEach((s) => {
      const opt = document.createElement('option');
      opt.value = s;
      opt.textContent = s;
      if (selectedSection && selectedSection === s) {
        opt.selected = true;
      }
      sectionSel.appendChild(opt);
    });
  }

  collegeSel.addEventListener('change', () => {
    while (programSel.firstChild) {
      programSel.removeChild(programSel.firstChild);
    }
    const rows = data[collegeSel.value] || [];
    const first = document.createElement('option');
    first.value = '';
    first.textContent = '- Select program -';
    programSel.appendChild(first);
    rows.forEach((r) => {
      const opt = document.createElement('option');
      opt.value = r.name;
      opt.textContent = r.name;
      programSel.appendChild(opt);
    });
  });

  if (yearSel) {
    yearSel.addEventListener('change', () => {
      if (!sectionSel) {
        return;
      }
      sectionSel.innerHTML = '';
      const cid = collegeSel ? collegeSel.value : '';
      const program = programSel ? programSel.value : '';
    });
  }
})
```

```

const rows = (((sectionData[cid] || {})[program] || {})[yearSel.value] || []);
const first = document.createElement('option');
first.value = '';
first.textContent = '- Select section -';
sectionSel.appendChild(first);
rows.forEach((s) => {
    const opt = document.createElement('option');
    opt.value = s;
    opt.textContent = s;
    sectionSel.appendChild(opt);
});
});
}
if (programSel) {
    programSel.addEventListener('change', renderSections);
}
if (collegeSel) {
    collegeSel.addEventListener('change', renderSections);
}

renderPrograms();
renderSections();
})();
</script>

```

```

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: gened_faculty.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';
require_once __DIR__ . '/includes/account_registration_helpers.php';

require_role(['gened']);
$hasEmploymentStatusColumn = db_column_exists('faculty', 'employment_status');

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    $maxHoursPerDay = (int) ($_POST['max_hours_per_day'] ?? 8);
    if ($maxHoursPerDay < 1) {
        $maxHoursPerDay = 1;
    } elseif ($maxHoursPerDay > 36) {
        $maxHoursPerDay = 36;
    }
}

try {
    if ($action === 'add') {
        $allowedEmploymentStatuses = ['Permanent', 'Contract of Service', 'Temporary'];
        $employmentStatus = trim((string) ($_POST['employment_status'] ?? 'Permanent'));
        if (!in_array($employmentStatus, $allowedEmploymentStatuses, true)) {
            throw new RuntimeException('Please select a valid employment status.');
```

```

        throw new RuntimeException('Username already exists.' . ($s ? ' Try: ' . implode(' ', $s) : ''));
    }
    $facultyCode = trim((string) ($_POST['faculty_id'] ?? ''));
    if ($facultyCode === '') {
        throw new RuntimeException('Faculty ID is required.');
```

```

    }
    $exists = db()->prepare('SELECT COUNT(*) FROM faculty WHERE faculty_id=?');
    $exists->execute([$facultyCode]);
    if ((int) $exists->fetchColumn() > 0) {
        throw new RuntimeException('Faculty ID already exists.');
```

```

    }
    db()->beginTransaction();
    db()->prepare('INSERT INTO users (username,password,full_name,role,college_id,is_active) VALUES (?,?,,?,?,?)')
    ->execute([
        $username,
        $cred['hash'],
        $fullName,
        'faculty',
        null,
        (($_POST['status'] ?? '') === 'inactive') ? 0 : 1,
    ]);
```

```

    $uid = (int) db()->lastInsertId();
```

```

    if ($hasEmploymentStatusColumn) {
```

```

        db()->prepare(
```

```

        'INSERT INTO faculty (user_id, faculty_id, full_name, department, email, max_hours_per_day, college_id, status,
employment_status, is_gened)
        VALUES (?,?,,?,?,?,?,?,?,1)'
```

```

    )->execute([
        $uid,
        $facultyCode,
        $fullName,
        'General Education',
        $facultyEmail,
        $maxHoursPerDay,
        null,
        (($_POST['status'] ?? '') === 'inactive') ? 'inactive' : 'active',
        $employmentStatus,
    ]);
```

```

    } else {
```

```

        db()->prepare(
```

```

        'INSERT INTO faculty (user_id, faculty_id, full_name, department, email, max_hours_per_day, college_id, status,
is_gened)
        VALUES (?,?,,?,?,?,?,?,?,1)'
```

```

    )->execute([
        $uid,
        $facultyCode,
        $fullName,
        'General Education',
        $facultyEmail,
        $maxHoursPerDay,
        null,
        (($_POST['status'] ?? '') === 'inactive') ? 'inactive' : 'active',
    ]);
```

```

    }
    db()->commit();
```

```

    $newFacultyId = (int) db()->lastInsertId();
```

```

    $stFa = db()->prepare(
```

```

    'SELECT f.id, f.user_id, f.faculty_id, f.full_name, f.department, f.email, f.max_hours_per_day, f.status, f.college_id,
u.username, u.is_active AS user_is_active
    FROM faculty f INNER JOIN users u ON u.id = f.user_id WHERE f.id = ? LIMIT 1'
```

```

    );
    $stFa->execute([$newFacultyId]);
```

```

    $afterFac = $stFa->fetch(PDO::FETCH_ASSOC);
```

```

    log_user_activity('add', 'GE Faculty', 'GE Faculty #' . $newFacultyId, null, $afterFac ? (array) $afterFac : null);
```

```

    $mailOk = $hasValidEmail
```

```

        ? notify_new_account_credentials($uid, $facultyEmail, $fullName, $username, $password, 'faculty')
        : null;
```

```

    if (!$hasValidEmail) {
```

```

        mark_new_account_requires_password_change($uid);
    }
```

```

    $_SESSION['flash'] = registration_email_flash_message($mailOk, $facultyEmail, 'GE faculty account');
```

```

} elseif ($action === 'edit' && isset($_POST['id'])) {
```

```

    $allowedEmploymentStatuses = ['Permanent', 'Contract of Service', 'Temporary'];
```

```

    $employmentStatus = trim((string) ($_POST['employment_status'] ?? 'Permanent'));
    if (!in_array($employmentStatus, $allowedEmploymentStatuses, true)) {
```

```

        throw new RuntimeException('Please select a valid employment status.');
```

```

    }
```

```

    $fid = (int) $_POST['id'];
```

```

    $stBf = db()->prepare(
```

```

    'SELECT f.*, u.username, u.is_active AS user_is_active FROM faculty f INNER JOIN users u ON u.id = f.user_id WHERE f.id=?'
```

```

AND COALESCE(f.is_gened,0)=1'
    );
    $stBf->execute([$fid]);
    $beforeFac = $stBf->fetch(PDO::FETCH_ASSOC);
    $chk = db()->prepare('SELECT user_id FROM faculty WHERE id=? AND COALESCE(is_gened,0)=1');
    $chk->execute([$fid]);
    $userId = (int) ($chk->fetchColumn() ?: 0);
    if ($userId < 1) {
        throw new RuntimeException('GE faculty record not found.');
```

```

    }
    $newCode = trim((string) ($_POST['faculty_id'] ?? ''));
    if ($newCode === '') {
        throw new RuntimeException('Faculty ID is required.');
```

```

    }
    $exists = db()->prepare('SELECT COUNT(*) FROM faculty WHERE faculty_id=? AND id<>?');
    $exists->execute([$newCode, $fid]);
    if ((int) $exists->fetchColumn() > 0) {
        throw new RuntimeException('Faculty ID already exists.');
```

```

    }
    if ($hasEmploymentStatusColumn) {
        db()->prepare(
            'UPDATE faculty SET faculty_id=?, full_name=?, email=?, max_hours_per_day=?, status=?, employment_status=? WHERE id=?
AND COALESCE(is_gened,0)=1'
        )->execute([
            $newCode,
            trim((string) ($_POST['full_name'] ?? '')),
            trim((string) ($_POST['email'] ?? '')),
            $maxHoursPerDay,
            (($_POST['status'] ?? '') === 'inactive') ? 'inactive' : 'active',
            $employmentStatus,
            $fid,
        ]);
    } else {
        db()->prepare(
            'UPDATE faculty SET faculty_id=?, full_name=?, email=?, max_hours_per_day=?, status=? WHERE id=? AND
COALESCE(is_gened,0)=1'
        )->execute([
            $newCode,
            trim((string) ($_POST['full_name'] ?? '')),
            trim((string) ($_POST['email'] ?? '')),
            $maxHoursPerDay,
            (($_POST['status'] ?? '') === 'inactive') ? 'inactive' : 'active',
            $fid,
        ]);
    }
}
```

```

db()->prepare('UPDATE users SET full_name=?, is_active=? WHERE id=?')
->execute([
    trim((string) ($_POST['full_name'] ?? '')),
    (($_POST['status'] ?? '') === 'inactive') ? 0 : 1,
    $userId,
]);
if (!empty($_POST['reset_password'])) {
    db()->prepare('UPDATE users SET password=? WHERE id=?')
    ->execute([password_hash((string) $_POST['reset_password'], PASSWORD_DEFAULT), $userId]);
}
```

```

$stAf = db()->prepare(
    'SELECT f.id, f.user_id, f.faculty_id, f.full_name, f.department, f.email, f.max_hours_per_day, f.status, f.college_id,
u.username, u.is_active AS user_is_active
FROM faculty f INNER JOIN users u ON u.id = f.user_id WHERE f.id = ? LIMIT 1'
);
$stAf->execute([$fid]);
$afterFac = $stAf->fetch(PDO::FETCH_ASSOC);
if ($afterFac !== false && $afterFac !== [] && (!empty($_POST['reset_password']))) {
    $afterFac['password'] = '[changed]';
}
log_user_activity('edit', 'GE Faculty', 'GE Faculty #' . $fid, $beforeFac ? (array) $beforeFac : null, $afterFac ? (array)
$afterFac : null);
$_SESSION['flash'] = 'GE faculty updated.';
} elseif ($action === 'delete' && isset($_POST['id'])) {
    $fid = (int) $_POST['id'];
    $stDelSnap = db()->prepare(
        'SELECT f.*, u.username FROM faculty f INNER JOIN users u ON u.id = f.user_id WHERE f.id=? AND COALESCE(f.is_gened,0)=1'
    );
    $stDelSnap->execute([$fid]);
    $beforeDelFac = $stDelSnap->fetch(PDO::FETCH_ASSOC);
    $q = db()->prepare('SELECT user_id FROM faculty WHERE id=? AND COALESCE(is_gened,0)=1');
    $q->execute([$fid]);
    $userId = (int) ($q->fetchColumn() ?: 0);
    if ($userId < 1) {
        throw new RuntimeException('GE faculty record not found.');
```

```

    }
    db()->prepare('DELETE FROM faculty WHERE id=? AND COALESCE(is_gened,0)=1')->execute([$fid]);
    db()->prepare('DELETE FROM users WHERE id=? AND role="faculty"')->execute([$userId]);
    log_user_activity('delete', 'GE Faculty', 'GE Faculty #' . $fid, $beforeDelFac ? (array) $beforeDelFac : null, null);
    $_SESSION['flash'] = 'GE faculty removed.';
}
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: gened_faculty.php');
exit;
}

$editRow = null;
if (isset($_GET['edit'])) {
    $st = db()->prepare(
        'SELECT f.*, u.username FROM faculty f LEFT JOIN users u ON u.id=f.user_id WHERE f.id=? AND COALESCE(f.is_gened,0)=1'
    );
    $st->execute([(int) $_GET['edit']]);
    $editRow = $st->fetch() ?: null;
}

$st = db()->query(
    'SELECT f.*, u.username, u.is_active
    FROM faculty f
    LEFT JOIN users u ON u.id = f.user_id
    WHERE COALESCE(f.is_gened,0)=1
    ORDER BY f.full_name ASC'
);
$list = $st->fetchAll();

$currentUser = current_user() ?? [];
$coordinatorName = trim((string) ($currentUser['full_name'] ?? 'GEN ED Coordinator'));
$coordinatorInitials = '';
foreach (preg_split('/\s+/', $coordinatorName) ?: [] as $part) {
    $token = trim((string) $part, " ,.\t\n\r\0\x0B");
    if ($token !== '') {
        $coordinatorInitials .= strtoupper(substr($token, 0, 1));
    }
}
$coordinatorInitials = $coordinatorInitials !== '' ? substr($coordinatorInitials, 0, 2) : 'GE';
$totalFacultyCount = count($list);
$permanentCount = 0;
$activeCount = 0;
foreach ($list as $facultyRow) {
    if ((string) ($facultyRow['employment_status'] ?? '') === 'Permanent') {
        $permanentCount++;
    }
    if ((string) ($facultyRow['status'] ?? '') === 'active') {
        $activeCount++;
    }
}

$pageTitle = 'GE Faculty';
require_once __DIR__ . '/includes/header.php';
?>
<style>
.faculty-dashboard-container { max-width: 1440px; margin: 0 auto; }
.faculty-university-header { display:flex; justify-content:space-between; align-items:flex-end; flex-wrap:wrap; margin-bottom:28px;
border-bottom:2px solid rgba(30,64,95,.2); padding-bottom:16px; gap: 12px; }
.faculty-title-section h1 { font-size:1.9rem; font-weight:700; background:linear-gradient(135deg,#1e405f,#2a6f8f); -webkit-background-clip:text; background-clip:text; color:transparent; letter-spacing:-.3px; }
.faculty-title-section p { font-size:.9rem; color:#4a627a; margin-top:6px; font-weight:500; }
.faculty-dean-card { background:#fff; padding:10px 24px; border-radius:60px; box-shadow:0 4px 10px rgba(0,0,0,.02),0 1px 2px
rgba(0,0,0,.05); display:flex; align-items:center; gap:16px; border:1px solid #e2edf2; }
.faculty-dean-avatar { background:#1e5a6f; width:44px; height:44px; border-radius:50%; display:flex; align-items:center; justify-
content:center; color:#fff; font-weight:600; font-size:1.2rem; }
.faculty-dean-info h4 { font-weight:600; font-size:.9rem; color:#2c4c6e; margin: 0; }
.faculty-dean-info p { font-size:.75rem; color:#4c6a82; font-weight:500; margin: 0; }
.faculty-stats-row { display:flex; gap:20px; margin-bottom:32px; flex-wrap:wrap; }
.faculty-stat-card { background:#fff; border-radius:24px; padding:14px 28px; box-shadow:0 2px 6px rgba(0,0,0,.04); display:flex; align-
items:center; gap:14px; border:1px solid #eef3f8; flex:1 1 auto; min-width: 200px; }
.faculty-stat-icon { background:#eaf4fc; width:48px; height:48px; border-radius:32px; display:flex; align-items:center; justify-
content:center; color:#1f6e8c; font-size:1.4rem; }
.faculty-stat-info h3 { font-size:1.6rem; font-weight:700; color:#1e405f; margin: 0; }
.faculty-stat-info span { font-size:.75rem; color:#5e7f9a; font-weight:500; }
.faculty-layout { display:flex; flex-direction:column; gap:32px; }

```

```

.faculty-card { background:#fff; border-radius:28px; box-shadow:0 12px 28px rgba(0,0,0,.05),0 0 0 1px rgba(0,0,0,.02); overflow:hidden; }
.faculty-card-header { padding:20px 28px 12px; border-bottom:1px solid #edf2f7; display:flex; justify-content:space-between; align-items:center; flex-wrap:wrap; gap: 10px; }
.faculty-card-header h2 { font-size:1.35rem; font-weight:600; color:#1f3b4c; display:flex; align-items:center; gap:10px; margin: 0; }
.faculty-badge-manage { background:#eef2ff; padding:6px 14px; border-radius:30px; font-size:.7rem; font-weight:600; color:#2c5f8a; }
.faculty-form-grid { display:grid; grid-template-columns:repeat(auto-fit,minmax(280px,1fr)); gap:20px 28px; padding:28px; }
.faculty-input-group { display:flex; flex-direction:column; gap:8px; }
.faculty-input-group label { font-size:.75rem; font-weight:600; text-transform:uppercase; letter-spacing:.5px; color:#4a6b85; }
.faculty-input-group input, .faculty-input-group select { padding:12px 14px; border-radius:16px; border:1.5px solid #e2edf2; background:#fefefe; font-size:.9rem; transition:all .2s; }
.faculty-input-group input:focus, .faculty-input-group select:focus { outline:none; border-color:#2c8cac; box-shadow:0 0 0 3px rgba(44,140,172,.2); }
.faculty-row-flex { display:flex; gap:20px; flex-wrap:wrap; align-items:flex-end; grid-column: 1 / -1; }
.faculty-radio-group { display:flex; align-items:center; gap:18px; margin-top:8px; grid-column: 1 / -1; color:#2a445b; }
.faculty-save-btn { background:#1f6e8c; border:none; padding:12px 24px; border-radius:40px; font-weight:600; color:#fff; font-size:.9rem; display:inline-flex; align-items:center; gap:12px; cursor:pointer; transition:.2s; }
.faculty-save-btn:hover { background:#0e556f; transform:translateY(-1px); box-shadow:0 6px 12px rgba(0,0,0,.05); }
.faculty-cancel-link { border-radius: 40px; padding: 10px 20px; }
.faculty-table-wrapper { overflow-x:auto; padding:0 0 8px; }
.faculty-table { width:100%; border-collapse:collapse; font-size:.85rem; }
.faculty-table th { text-align:left; padding:18px 16px; background:#f9fbfd; font-weight:600; color:#2c4c6e; border-bottom:1px solid #e6edf2; }
.faculty-table td { padding:14px 16px; border-bottom:1px solid #eff3f8; vertical-align:middle; color:#2a445b; }
.faculty-table tr:hover td { background:#fafcff; }
.faculty-status-badge { display:inline-block; padding:4px 12px; border-radius:50px; font-size:.7rem; font-weight:600; text-transform:capitalize; }
.faculty-status-active { background:#e0f2e9; color:#1f7840; }
.faculty-status-inactive { background:#ffe6e5; color:#b33; }
.faculty-action-buttons { display:flex; gap:8px; align-items:center; }
.faculty-icon-btn { border:none; border-radius:30px; background:transparent; color:#2c7da0; padding:6px 9px; }
.faculty-icon-btn.delete { color:#bc6c6c; }
.faculty-icon-btn:hover { background:#eef2ff; color:#0f5c7c; }
.faculty-icon-btn.delete:hover { background:#fff0f0; color:#b13e3e; }
.faculty-manage-programs-link { background:#f7f9fc; border-radius:20px; padding:8px 20px; font-size:.8rem; font-weight:500; color:#2a6f8f; text-decoration:none; transition:.2s; }
.faculty-manage-programs-link:hover { background:#eaf3f8; color:#2a6f8f; }
.faculty-footer-note { margin-top:32px; text-align:center; font-size:.7rem; color:#6b89a0; border-top:1px solid #e0eaf0; padding-top:24px; }

html[data-bs-theme="dark"] .faculty-title-section h1 { background: linear-gradient(135deg, #c5e6ff, #79c5e8); -webkit-background-clip:text; background-clip:text; color: transparent; }
html[data-bs-theme="dark"] .faculty-title-section p { color: #9db6c8; }
html[data-bs-theme="dark"] .faculty-dean-card,
html[data-bs-theme="dark"] .faculty-stat-card,
html[data-bs-theme="dark"] .faculty-card { background: #16202a; border-color: #2a3947; box-shadow: 0 10px 24px rgba(0,0,0,.35); }
html[data-bs-theme="dark"] .faculty-dean-info h4,
html[data-bs-theme="dark"] .faculty-card-header h2,
html[data-bs-theme="dark"] .faculty-table th { color: #d3e7f6; }
html[data-bs-theme="dark"] .faculty-dean-info p,
html[data-bs-theme="dark"] .faculty-stat-info span,
html[data-bs-theme="dark"] .faculty-input-group label,
html[data-bs-theme="dark"] .faculty-footer-note { color: #9ab0c2; }
html[data-bs-theme="dark"] .faculty-stat-icon { background:#203346; color:#87c8ea; }
html[data-bs-theme="dark"] .faculty-stat-info h3 { color:#e3f3ff; }
html[data-bs-theme="dark"] .faculty-card-header { border-bottom-color: #2b3a49; }
html[data-bs-theme="dark"] .faculty-input-group input,
html[data-bs-theme="dark"] .faculty-input-group select { background:#0f1821; border-color:#2e4152; color:#deebf5; }
html[data-bs-theme="dark"] .faculty-input-group input:focus,
html[data-bs-theme="dark"] .faculty-input-group select:focus { border-color:#4ea4c6; box-shadow:0 0 0 3px rgba(78,164,198,.28); }
html[data-bs-theme="dark"] .faculty-radio-group { color:#c5d9e7; }
html[data-bs-theme="dark"] .faculty-manage-programs-link { background:#1d2b38; color:#9ed0eb; }
html[data-bs-theme="dark"] .faculty-manage-programs-link:hover { background:#253747; color:#b7ddf3; }
html[data-bs-theme="dark"] .faculty-badge-manage { background:#223243; color:#a9d2ea; }
html[data-bs-theme="dark"] .faculty-table th { background:#1b2834; border-bottom-color:#30414f; }
html[data-bs-theme="dark"] .faculty-table td { color:#c8dced; border-bottom-color:#2b3a47; }
html[data-bs-theme="dark"] .faculty-table tr:hover td { background:#1a2530; }
html[data-bs-theme="dark"] .faculty-status-active { background:#1d3a2b; color:#8fddb1; }
html[data-bs-theme="dark"] .faculty-status-inactive { background:#45262a; color:#f4aab1; }
html[data-bs-theme="dark"] .faculty-icon-btn { color:#90c7e5; }
html[data-bs-theme="dark"] .faculty-icon-btn:hover { background:#233344; color:#c4e6f9; }
html[data-bs-theme="dark"] .faculty-icon-btn.delete { color:#e09a9a; }
html[data-bs-theme="dark"] .faculty-icon-btn.delete:hover { background:#3f262b; color:#ffc4c4; }
html[data-bs-theme="dark"] .faculty-footer-note { border-top-color:#2a3a49; }
@media (max-width: 700px) {
  .faculty-card-header { flex-direction:column; align-items:flex-start; }
  .faculty-stat-card { padding:10px 20px; }
}

```

```

</style>

<div class="faculty-dashboard-container">
  <div class="faculty-university-header">

```

```

<div class="faculty-title-section">
    <h1><i class="fa-solid fa-chalkboard-user" style="color:#2c7da0; margin-right:8px;"></i>GE Faculty Management</h1>
    <p>General Education · Western Philippines University</p>
</div>
<div class="faculty-dean-card">
    <div class="faculty-dean-avatar"><?>= htmlspecialchars($coordinatorInitials) ?></div>
    <div class="faculty-dean-info">
        <h4><?>= htmlspecialchars(strtoupper($coordinatorName)) ?></h4>
        <p>GEN ED Coordinator <i class="fa-solid fa-circle-check" style="font-size:.7rem; color:#1f8a4c;"></i></p>
    </div>
</div>
</div>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if (!$hasEmploymentStatusColumn): ?>
    <div class="alert alert-warning py-2">
        Employment Status is in view-only fallback mode. Run <a href="upgrade_roles.php">upgrade_roles.php</a> to save status changes
        (Permanent / Contract of Service / Temporary).
    </div>
<?php endif; ?>

<div class="faculty-stats-row">
    <div class="faculty-stat-card">
        <div class="faculty-stat-icon"><i class="fa-solid fa-users"></i></div>
        <div class="faculty-stat-info"><h3><?>= $totalFacultyCount ?></h3><span>Total GE Faculty</span></div>
    </div>
    <div class="faculty-stat-card">
        <div class="faculty-stat-icon"><i class="fa-solid fa-user-check"></i></div>
        <div class="faculty-stat-info"><h3><?>= $permanentCount ?></h3><span>Permanent</span></div>
    </div>
    <div class="faculty-stat-card">
        <div class="faculty-stat-icon"><i class="fa-solid fa-clock"></i></div>
        <div class="faculty-stat-info"><h3><?>= $activeCount ?></h3><span>Active members</span></div>
    </div>
</div>

<div class="faculty-layout">
    <div class="faculty-card">
        <div class="faculty-card-header">
            <h2><i class="fa-solid fa-user-plus"></i> <?>= $editRow ? 'Edit GE faculty' : 'Add GE faculty' ?></h2>
            <a href="gened_courses.php" class="faculty-manage-programs-link"><i class="fa-solid fa-book me-1"></i>GE courses</a>
        </div>
        <form method="post" class="faculty-form-grid">
            <input type="hidden" name="action" value="<?>= $editRow ? 'edit' : 'add' ?>">
            <?php if ($editRow): ?><input type="hidden" name="id" value="<?>= (int) $editRow['id'] ?>"><?php endif; ?>
            <div class="faculty-input-group"><label>Faculty ID</label><input type="text" name="faculty_id" required maxlength="20"
value="<?>= htmlspecialchars((string) ($editRow['faculty_id'] ?? '')) ?>"></div>
            <div class="faculty-input-group"><label>Username</label><input type="text" name="username" maxlength="50" <?>= $editRow ?
'readonly' : 'required' ?> value="<?>= htmlspecialchars((string) ($editRow['username'] ?? '')) ?>"></div>
            <div class="faculty-input-group"><label><?>= $editRow ? 'Reset Password (optional)' : 'Password' ?></label><input
type="password" name="<?>= $editRow ? 'reset_password' : 'password' ?>" <?>= $editRow ? ' : 'required' ?> autocomplete="new-password"></div>
            <div class="faculty-input-group"><label>Full Name</label><input type="text" name="full_name" required maxlength="100"
value="<?>= htmlspecialchars((string) ($editRow['full_name'] ?? '')) ?>"></div>
            <div class="faculty-input-group">
                <label>Program</label>
                <input type="text" value="General Education" readonly>
            </div>
            <div class="faculty-input-group"><label>Email</label><input type="email" name="email" maxlength="100" value="<?>=
htmlspecialchars((string) ($editRow['email'] ?? '')) ?>"></div>
            <div class="faculty-row-flex">
                <div class="faculty-input-group" style="flex:1; min-width: 220px;"><label>Max Hrs/Day</label><input type="number"
name="max_hours_per_day" min="1" max="36" value="<?>= (int) ($editRow['max_hours_per_day'] ?? 8) ?>"></div>
                <div class="faculty-input-group" style="flex:1; min-width: 220px;">
                    <label>Employment Status</label>
                    <?php $selectedEmploymentStatus = (string) ($editRow['employment_status'] ?? 'Permanent'); ?>
                    <select name="employment_status">
                        <option value="Permanent" <?>= $selectedEmploymentStatus === 'Permanent' ? 'selected' : '' ?>>Permanent</option>
                        <option value="Contract of Service" <?>= $selectedEmploymentStatus === 'Contract of Service' ? 'selected' : '' ?
>>Contract of Service</option>
                        <option value="Temporary" <?>= $selectedEmploymentStatus === 'Temporary' ? 'selected' : '' ?>>Temporary</option>
                    </select>
                </div>
            </div>
            <div class="faculty-radio-group">
                <label><strong>Status:</strong></label>
                <label><input type="radio" name="status" value="active" <?>= ($editRow['status'] ?? 'active') === 'active' ? 'checked' :
'' ?>> Active</label>
                <label><input type="radio" name="status" value="inactive" <?>= ($editRow['status'] ?? '') === 'inactive' ? 'checked' : ''
?>> Inactive</label>
            </div>
        </form>
    </div>
</div>

```



```

        </div>
        <div style="display:flex; justify-content:flex-end; gap: 10px; grid-column: 1 / -1;">
            <?php if ($editRow): ?><a href="gened_faculty.php" class="btn btn-outline-secondary faculty-cancel-link">Cancel</a><?php
endif; ?>
            <button type="submit" class="faculty-save-btn"<? app_tooltip_attr($editRow ? 'Saves changes to this GE faculty record.
Use this after updating name, ID, or status.' : 'Adds the faculty member to the General Education roster. Use this before assigning GE
courses or schedules.') ?><i class="fa-solid <? $editRow ? 'fa-pen' : 'fa-save' ?>"></i> <? $editRow ? 'Update GE Faculty' : 'Save GE
Faculty' ?></button>
        </div>
    </form>
</div>

<div class="faculty-card">
    <div class="faculty-card-header">
        <h2><i class="fa-solid fa-list-ul"></i> GE faculty roster</h2>
        <span class="faculty-badge-manage"><i class="fa-solid fa-pen me-1"></i>Manage GE faculty records</span>
    </div>
    <div class="faculty-table-wrapper">
        <table class="faculty-table">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Name</th>
                    <th>Program</th>
                    <?php if ($hasEmploymentStatusColumn): ?><th>Employment Status</th><?php endif; ?>
                    <th>Username</th>
                    <th>Email</th>
                    <th>Maxhrs/day</th>
                    <th>Status</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($list as $r): ?>
                    <tr>
                        <td><? htmlspecialchars((string) $r['faculty_id']) ?></td>
                        <td><? htmlspecialchars((string) $r['full_name']) ?></td>
                        <td><? htmlspecialchars((string) ($r['department'] ?? 'General Education')) ?></td>
                        <?php if ($hasEmploymentStatusColumn): ?><td><? htmlspecialchars((string) ($r['employment_status'] ??
'Permanent')) ?></td><?php endif; ?>
                        <td><? htmlspecialchars((string) ($r['username'] ?? '')) ?></td>
                        <td><? htmlspecialchars((string) ($r['email'] ?? '')) ?></td>
                        <td><? (int) $r['max_hours_per_day'] ?></td>
                        <td>
                            <span class="faculty-status-badge <? ((($r['status'] ?? '')) === 'active') ? 'faculty-status-active' :
'faculty-status-inactive' ?>">
                                <? htmlspecialchars((string) $r['status']) ?>
                            </span>
                        </td>
                        <td class="faculty-action-buttons">
                            <a href="gened_faculty.php?edit=<? (int) $r['id'] ?>" class="faculty-icon-btn" title="Edit GE faculty"><i
class="fa-solid fa-pen"></i></a>
                            <form method="post" class="d-inline" onsubmit="return confirm('Delete this GE faculty member? Their GE
schedules may be affected.');">
                                <input type="hidden" name="action" value="delete">
                                <input type="hidden" name="id" value="<? (int) $r['id'] ?>">
                                <button type="submit" class="faculty-icon-btn delete" title="Delete GE faculty"><i class="fa-solid fa-
trash-can"></i></button>
                            </form>
                        </td>
                    </tr>
                </tr>
                <?php endforeach; ?>
                <?php if ($list === []): ?>
                    <tr><td colspan="<? $hasEmploymentStatusColumn ? '9' : '8' ?>" class="text-center text-muted py-4">No GE faculty
yet. Add one above.</td></tr>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>
</div>

<div class="faculty-footer-note">
    <i class="fa-solid fa-database"></i> General Education faculty directory • Real-time management | Western Philippines University
</div>
</div>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: gened_rooms.php

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['gened']);
$hasIsGened = db_column_exists('rooms', 'is_gened');
$roomTypes = ['lecture', 'laboratory', 'conference', 'tba'];
$roomStatuses = ['available', 'tba', 'maintenance'];
$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        $type = strtolower(trim((string) ($_POST['type'] ?? 'lecture')));
        $status = strtolower(trim((string) ($_POST['status'] ?? 'available')));
        if (!in_array($type, $roomTypes, true)) {
            $type = 'lecture';
        }
        if (!in_array($status, $roomStatuses, true)) {
            $status = 'available';
        }
        if (!$hasIsGened) {
            throw new RuntimeException('Run upgrade_roles.php first to enable GE rooms.');
        }
        if ($action === 'add') {
            $useCustomCode = isset($_POST['custom_room_code']) && (string) $_POST['custom_room_code'] === '1';
            if ($useCustomCode) {
                $roomCode = trim((string) ($_POST['room_code'] ?? ''));
                if ($roomCode === '') {
                    throw new RuntimeException('Enter a room code, or leave Custom code unchecked for auto-generated code.');
                }
            } else {
                $roomCode = next_auto_room_code_gened();
            }
            if (room_code_taken_for_gened($roomCode)) {
                throw new RuntimeException(
                    'Room code "' . $roomCode . '" is already used for another Gen Ed room. Pick a different code.'
                );
            }
            db()->prepare(
                'INSERT INTO rooms (room_code, room_name, capacity, type, status, is_gened, college_id) VALUES (?, ?, ?, ?, ?, 1, NULL)'
            )->execute([
                $roomCode,
                trim((string) $_POST['room_name']),
                (int) $_POST['capacity'],
                $type,
                $status,
            ]);
            $_SESSION['flash'] = 'GE room added.';
        } elseif ($action === 'edit' && isset($_POST['id'])) {
            $roomId = (int) $_POST['id'];
            $roomCode = trim((string) $_POST['room_code']);
            if (room_code_taken_for_gened($roomCode, $roomId)) {
                throw new RuntimeException(
                    'Room code "' . $roomCode . '" is already used for another Gen Ed room. Pick a different code.'
                );
            }
            db()->prepare(
                'UPDATE rooms SET room_code=?, room_name=?, capacity=?, type=?, status=? WHERE id=? AND is_gened=1'
            )->execute([
                $roomCode,
                trim((string) $_POST['room_name']),
                (int) $_POST['capacity'],
                $type,
                $status,
                $roomId,
            ]);
            $_SESSION['flash'] = 'GE room updated.';
        } elseif ($action === 'delete' && isset($_POST['id'])) {
            db()->prepare('DELETE FROM rooms WHERE id=? AND is_gened=1')->execute([(int) $_POST['id']]);
            $_SESSION['flash'] = 'GE room removed.';
        }
    } catch (Throwable $e) {
        $_SESSION['flash'] = 'Error: ' . $e->getMessage();
    }
}
```

```

    }
    header('Location: gened_rooms.php');
    exit;
}

$editRow = null;
if (isset($_GET['edit'])) {
    $st = db()->prepare('SELECT * FROM rooms WHERE id=? AND is_gened=1');
    $st->execute([(int) $_GET['edit']]);
    $editRow = $st->fetch() ?: null;
}

$list = db()->query('SELECT * FROM rooms WHERE is_gened=1 ORDER BY room_code ASC')->fetchAll();

$nextAutoGenedCode = null;
if (!$editRow && $hasIsGened) {
    try {
        $nextAutoGenedCode = next_auto_room_code_gened();
    } catch (Throwable) {
        $nextAutoGenedCode = null;
    }
}

$pageTitle = 'GE Rooms';
require_once __DIR__ . '/includes/header.php';
?>
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?
family=Inter:opsz,wght@14..32,300;14..32,400;14..32,500;14..32,600;14..32,700&display=swap">
<style>
    .ge-rooms-page * { box-sizing: border-box; }
    .ge-rooms-page {
        background: linear-gradient(135deg, #f0f4fa 0%, #e9eef4 100%);
        font-family: 'Inter', sans-serif;
        padding: 2rem 1.5rem;
        border-radius: 20px;
        color: #1a2c3e;
    }
    .dashboard-container { max-width: 1400px; margin: 0 auto; }
    .header-section { display: flex; justify-content: space-between; align-items: flex-end; flex-wrap: wrap; margin-bottom: 2rem; gap: 1rem; }

    .title-badge h1 {
        font-size: 1.9rem; font-weight: 700; background: linear-gradient(135deg, #1f3b4c, #2a5f6e);
        background-clip: text; -webkit-background-clip: text; color: transparent; letter-spacing: -0.3px;
        display: inline-flex; align-items: center; gap: 12px; margin: 0;
    }
    .title-badge h1 i { color: #2c7a6e; font-size: 2rem; }
    .sub { font-size: 0.9rem; color: #5b6f82; margin-top: 6px; font-weight: 500; display: flex; align-items: center; gap: 8px; }
    .auto-code-card {
        background: rgba(255, 255, 255, 0.8); padding: 0.6rem 1.2rem; border-radius: 60px;
        box-shadow: 0 4px 12px rgba(0, 0, 0, 0.02), 0 1px 2px rgba(0, 0, 0, 0.05); font-size: 0.85rem;
        font-weight: 500; display: flex; align-items: center; gap: 12px; border: 1px solid rgba(72, 187, 150, 0.3);
    }
    .auto-code-card span:first-of-type { color: #3a5e6b; }
    .next-code {
        background: #eef2f7; padding: 0.2rem 0.8rem; border-radius: 40px; font-family: monospace;
        font-weight: 700; color: #1c6e5c; letter-spacing: 0.5px;
    }
    .form-card {
        background: rgba(255, 255, 255, 0.96); border-radius: 32px; box-shadow: 0 20px 35px -12px rgba(0, 0, 0, 0.08), 0 1px 3px rgba(0, 0, 0, 0.02);
        padding: 1.8rem 2rem; margin-bottom: 2.5rem; border: 1px solid rgba(166, 194, 187, 0.3);
    }
    .form-title {
        font-size: 1.3rem; font-weight: 600; margin-bottom: 1.6rem; display: flex; align-items: center;
        gap: 10px; color: #1f4e5c; border-left: 4px solid #2c9b7e; padding-left: 1rem;
    }
    .form-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); gap: 1.5rem; margin-bottom: 2rem; }
    .input-group { display: flex; flex-direction: column; gap: 8px; }
    .input-group label {
        font-weight: 600; font-size: 0.8rem; text-transform: uppercase; letter-spacing: 0.5px;
        color: #4b6a7c; display: flex; align-items: center; gap: 6px;
    }
    .input-group input, .input-group select {
        padding: 0.85rem 1rem; border-radius: 20px; border: 1.5px solid #e2e8f0; background: #fefefe;
        font-family: 'Inter', sans-serif; font-size: 0.95rem; transition: all 0.2s; outline: none; font-weight: 500;
    }
    .input-group input:focus, .input-group select:focus { border-color: #2c9b7e; box-shadow: 0 0 3px rgba(44, 155, 126, 0.2); }
    .input-group input:disabled { background: #eef2f7; color: #607283; cursor: not-allowed; }
    .checkbox-row { flex-direction: row; align-items: center; gap: 10px; margin-top: 1.8rem; }
    .checkbox-row label { text-transform: none; font-weight: 500; font-size: 0.85rem; margin: 0; }

```

```

.checkbox-row input[type="checkbox"] { width: 1rem; height: 1rem; margin: 0; accent-color: #2c9b7e; }
.btn-save {
  background: #1f5e5a; border: none; padding: 0.9rem 2rem; border-radius: 40px; font-weight: 600; font-size: 0.95rem; color: white;
  display: inline-flex; align-items: center; gap: 12px; cursor: pointer; transition: all 0.2s; box-shadow: 0 4px 8px rgba(0,0,0,0.05);
  text-decoration: none;
}
.btn-save:hover { background: #134e48; transform: translateY(-1px); box-shadow: 0 10px 20px -8px rgba(28, 78, 65, 0.4); color: #fff; }
.btn-cancel { background: #cddfd5; color: #2b5e4f; }
.btn-cancel:hover { background: #b6d2ca; color: #244e43; }
.table-wrapper {
  background: white; border-radius: 28px; box-shadow: 0 12px 30px rgba(0, 0, 0, 0.05);
  overflow-x: auto; padding: 0 0 0.2rem; border: 1px solid #eef2f0;
}
.rooms-table { width: 100%; border-collapse: collapse; font-size: 0.9rem; min-width: 680px; }
.rooms-table th {
  text-align: left; padding: 1.2rem 1rem; background-color: #f9fbfd; font-weight: 600; color: #2b5e6b;
  border-bottom: 1.5px solid #e2ede8; font-size: 0.85rem; letter-spacing: 0.3px;
}
.rooms-table td { padding: 1rem; border-bottom: 1px solid #edf2f1; vertical-align: middle; font-weight: 500; color: #1f3e48; }
.status-badge {
  display: inline-flex; align-items: center; gap: 6px; padding: 0.25rem 0.75rem; border-radius: 40px;
  font-size: 0.75rem; font-weight: 600; width: fit-content;
}
.status-badge.available { background: #dff3e6; color: #1c6e46; }
.status-badge.maintenance { background: #ffe6e3; color: #b13e3e; }
.status-badge.tba { background: #eef2f7; color: #516575; }
.action-btns { display: inline-flex; align-items: center; gap: 10px; }
.edit-btn, .delete-btn {
  border: none; font-size: 0.9rem; cursor: pointer; padding: 6px 10px; border-radius: 40px;
  transition: all 0.15s; font-weight: 500; display: inline-flex; align-items: center; gap: 6px; text-decoration: none;
}
.edit-btn { color: #2c7a6e; background: #eef8f5; }
.edit-btn:hover { background: #d4f0e8; color: #205e55; }
.delete-btn { color: #c25b5b; background: #fff3f0; }
.delete-btn:hover { background: #ffe0db; color: #a03e3e; }
.empty-row td { text-align: center; padding: 2.5rem; color: #8ea3af; font-style: italic; }
.ge-inline-alert {
  margin: 0 auto 1rem; max-width: 1400px; background: rgba(255, 255, 255, 0.95); border: 1px solid #dbe6eb;
  color: #29404b; border-radius: 16px; padding: 0.75rem 1rem;
}
.ge-inline-alert.warning { border-color: #f4d9a5; color: #7b5a1f; background: #fff8ec; }
html[data-bs-theme="dark"] .ge-rooms-page {
  background: linear-gradient(135deg, #111c25 0%, #162632 100%);
  color: #dbe8f2;
}
html[data-bs-theme="dark"] .title-badge h1 {
  background: linear-gradient(135deg, #d9edf7, #9cd4e3);
  background-clip: text;
  -webkit-background-clip: text;
  color: transparent;
}
html[data-bs-theme="dark"] .title-badge h1 i { color: #62bea8; }
html[data-bs-theme="dark"] .sub { color: #9eb3c2; }
html[data-bs-theme="dark"] .auto-code-card {
  background: rgba(20, 35, 46, 0.82);
  border-color: rgba(98, 190, 168, 0.35);
  color: #d6e7f2;
  box-shadow: 0 8px 18px rgba(0, 0, 0, 0.25);
}
html[data-bs-theme="dark"] .auto-code-card span:first-of-type { color: #9cc2d5; }
html[data-bs-theme="dark"] .next-code {
  background: #243746;
  color: #8de5c9;
}
html[data-bs-theme="dark"] .form-card {
  background: rgba(19, 33, 43, 0.96);
  border-color: rgba(117, 151, 163, 0.28);
  box-shadow: 0 16px 30px rgba(0, 0, 0, 0.28);
}
html[data-bs-theme="dark"] .form-title { color: #b9dbe7; border-left-color: #3bb495; }
html[data-bs-theme="dark"] .input-group label { color: #9cb6c6; }
html[data-bs-theme="dark"] .input-group input,
html[data-bs-theme="dark"] .input-group select {
  background: #111d27;
  border-color: #2f4454;
  color: #dbe8f2;
}
html[data-bs-theme="dark"] .input-group input::placeholder { color: #8ba1b0; }
html[data-bs-theme="dark"] .input-group input:disabled { background: #223341; color: #98afb7; }
html[data-bs-theme="dark"] .btn-save { background: #1c7467; color: #ecfffa; }

```

```

html[data-bs-theme="dark"] .btn-save:hover { background: #165d53; color: #ecfffa; }
html[data-bs-theme="dark"] .btn-cancel { background: #344a57; color: #d9e9f3; }
html[data-bs-theme="dark"] .btn-cancel:hover { background: #425c6b; color: #ebf7ff; }
html[data-bs-theme="dark"] .table-wrapper {
    background: #16242f;
    border-color: #2a3d4b;
    box-shadow: 0 14px 30px rgba(0, 0, 0, 0.25);
}
html[data-bs-theme="dark"] .rooms-table th {
    background: #1b2e3b;
    color: #b9d4e3;
    border-bottom-color: #2f4858;
}
html[data-bs-theme="dark"] .rooms-table td {
    color: #d4e5ef;
    border-bottom-color: #263a49;
}
html[data-bs-theme="dark"] .status-badge.available { background: #1f4738; color: #a3f0cd; }
html[data-bs-theme="dark"] .status-badge.maintenance { background: #4a2a2a; color: #ffb9b9; }
html[data-bs-theme="dark"] .status-badge.tba { background: #2d3f4d; color: #bed2df; }
html[data-bs-theme="dark"] .edit-btn { background: #244a45; color: #9ce2d0; }
html[data-bs-theme="dark"] .edit-btn:hover { background: #2c5a54; color: #baf2e4; }
html[data-bs-theme="dark"] .delete-btn { background: #4b2e2c; color: #ffb9b1; }
html[data-bs-theme="dark"] .delete-btn:hover { background: #633936; color: #ffd0ca; }
html[data-bs-theme="dark"] .empty-row td { color: #8ea6b5; }
html[data-bs-theme="dark"] .ge-inline-alert {
    background: #172630;
    border-color: #314553;
    color: #c9dce8;
}
html[data-bs-theme="dark"] .ge-inline-alert.warning {
    background: #3b3322;
    border-color: #6b5a34;
    color: #f0dba7;
}
@media (max-width: 640px) {
    .ge-rooms-page { padding: 1rem; }
    .form-card { padding: 1.2rem; }
}
</style>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
<?php if (!$hasIsGened): ?>
    <div class="ge-inline-alert warning">Run <a href="upgrade_roles.php">upgrade_roles.php</a> to enable GE rooms.</div>
<?php endif; ?>

<div class="ge-rooms-page">
    <div class="dashboard-container">
        <div class="header-section">
            <div class="title-badge">
                <h1><i class="fas fa-chalkboard-teacher"></i> GE Rooms</h1>
                <div class="sub"><i class="fas fa-door-open"></i> General Education Rooms · Smart Scheduling</div>
            </div>
            <div class="auto-code-card">
                <i class="fas fa-magic"></i>
                <span>Auto-code (if custom is off)</span>
                <span class="next-code"><?= htmlspecialchars((string) ($nextAutoGenedCode ?? 'N/A')) ?></span>
            </div>
        </div>

        <div class="form-card">
            <div class="form-title">
                <i class="fas <?= $editRow ? 'fa-pen-alt' : 'fa-plus-circle' ?>"></i>
                <span><?= $editRow ? 'Edit GE room' : 'Add GE room' ?></span>
            </div>
            <form method="post">
                <input type="hidden" name="action" value="<?= $editRow ? 'edit' : 'add' ?>">
                <?php if ($editRow): ?><input type="hidden" name="id" value="<?= (int) $editRow['id'] ?>"><?php endif; ?>
                <div class="form-grid">
                    <div class="input-group">
                        <label for="genedRoomCodeInput"><i class="fas fa-tag"></i> Code</label>
                        <input
                            type="text"
                            name="room_code"
                            id="genedRoomCodeInput"
                            maxlength="20"
                            value="<?= htmlspecialchars((string) ($editRow['room_code'] ?? '')) ?>"
                            placeholder="<?= $editRow ? 'Room code' : 'Auto if custom is off' ?>"
                            autocomplete="off"
                            <?= $editRow ? 'required' : 'disabled' ?>

```

```

        >
    </div>
    <div class="input-group">
        <label for="generatedRoomNameInput"><i class="fas fa-building"></i> Name</label>
        <input type="text" name="room_name" id="generatedRoomNameInput" maxlength="50" value="<?> htmlspecialchars((string)
($editRow['room_name'] ?? '')) ?>" placeholder="Room name (e.g., CCD)" required>
    </div>
    <div class="input-group">
        <label for="generatedCapacityInput"><i class="fas fa-users"></i> Capacity</label>
        <input type="number" name="capacity" id="generatedCapacityInput" min="0" max="500" value="<?> (int)
($editRow['capacity'] ?? 0) ?>">
    </div>
    <div class="input-group">
        <label for="generatedTypeInput"><i class="fas fa-chalkboard"></i> Type</label>
        <select name="type" id="generatedTypeInput">
            <?php foreach ($roomTypes as $t): ?>
                <?php $label = $t === 'tba' ? 'TBA (To be arranged)' : ucfirst($t); ?>
                <option value="<?> $t ?>" <?> ($editRow['type'] ?? 'lecture') === $t ? 'selected' : '' ?><?> $label ?>
            </option>

            <?php endforeach; ?>
        </select>
    </div>
    <div class="input-group">
        <label for="generatedStatusInput"><i class="fas fa-circle-info"></i> Status</label>
        <select name="status" id="generatedStatusInput">
            <option value="available" <?> ($editRow['status'] ?? '') === 'available' ? 'selected' : '' ?>>Available</option>
            <option value="tba" <?> ($editRow['status'] ?? '') === 'tba' ? 'selected' : '' ?>>TBA</option>
            <option value="maintenance" <?> ($editRow['status'] ?? '') === 'maintenance' ? 'selected' : '' ?
>>Maintenance</option>
        </select>
    </div>
    <?php if (!$editRow): ?>
        <div class="input-group checkbox-row">
            <input type="checkbox" name="custom_room_code" value="1" id="customGeneratedRoomCode" <?> !$hasIsGenerated ? 'disabled'
: '' ?>>

            <label for="customGeneratedRoomCode">Use custom code (unchecked = auto next code)</label>
        </div>
    <?php endif; ?>
</div>
<div style="display: flex; justify-content: flex-end; gap: 12px; flex-wrap: wrap;">
    <button type="submit" class="btn-save"><?> app_tooltip_attr($editRow ? 'Saves changes to this GE room.' : 'Adds a room
usable for GE scheduling.') ?>>
        <i class="fas fa-save"></i> <?> $editRow ? 'Save changes' : 'Save Room' ?>
    </button>
    <?php if ($editRow): ?>
        <a href="generated_rooms.php" class="btn-save btn-cancel"><?> app_tooltip_attr('Closes the editor without saving.') ?>><i
class="fas fa-times"></i> Cancel</a>
    <?php endif; ?>
</div>
</form>
</div>

<div class="table-wrapper">
    <table class="rooms-table">
        <thead>
            <tr>
                <th>Code</th>
                <th>Name</th>
                <th>Capacity</th>
                <th>Type</th>
                <th>Status</th>
                <th style="text-align: center;">Actions</th>
            </tr>
        </thead>
        <tbody>
            <?php if (!$list): ?>
                <tr class="empty-row"><td colspan="6"><i class="fas fa-door-closed"></i> No rooms created. Add your first GE room!
</td></tr>

            <?php endif; ?>
            <?php foreach ($list as $r): ?>
                <?php
                $status = strtolower((string) ($r['status'] ?? ''));
                $statusClass = in_array($status, ['available', 'maintenance', 'tba'], true) ? $status : 'tba';
                $statusIcon = 'fa-clock';
                if ($status === 'available') {
                    $statusIcon = 'fa-check-circle';
                } elseif ($status === 'maintenance') {
                    $statusIcon = 'fa-tools';
                }
                ?>

```

```

        <tr>
            <td><strong><?= htmlspecialchars((string) $r['room_code']) ?></strong></td>
            <td><?= htmlspecialchars((string) $r['room_name']) ?></td>
            <td><?= (int) $r['capacity'] ?></td>
            <td><?= htmlspecialchars(ucfirst((string) $r['type'])) ?></td>
            <td><span class="status-badge <?= $statusClass ?>"><i class="fas <?= $statusIcon ?>"></i> <?=
htmlspecialchars(ucfirst((string) $r['status'])) ?></span></td>
            <td style="text-align: center;">
                <div class="action-btns">
                    <a href="gened_rooms.php?edit=<?= (int) $r['id'] ?>" class="edit-btn"<?= app_tooltip_attr('Edits this GE
room's code, capacity, or status.') ?>><i class="fas fa-edit"></i> Edit</a>
                    <form method="post" onsubmit="return confirm('Delete this GE room?');">
                        <input type="hidden" name="action" value="delete">
                        <input type="hidden" name="id" value="<?= (int) $r['id'] ?>">
                        <button type="submit" class="delete-btn"<?= app_tooltip_attr('Removes this GE room after
confirmation.') ?>><i class="fas fa-trash-alt"></i> Delete</button>
                    </form>
                </div>
            </td>
        </tr>
    <?php endforeach; ?>
</tbody>
</table>
</div>
</div>
</div>

<?php if (!$editRow && $hasIsGened): ?>
<script>
(function () {
    var cb = document.getElementById('customGenedRoomCode');
    var inp = document.getElementById('genedRoomCodeInput');
    if (!cb || !inp) return;
    cb.addEventListener('change', function () {
        inp.disabled = !cb.checked;
        inp.required = cb.checked;
        if (!cb.checked) inp.value = '';
    });
})();
</script>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

-----
FILE: gened_schedule.php
-----

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['gened']);

ensure_faculty_profile_for_teaching_role((int) ($_SESSION['user_id'] ?? 0));

$errors = [];
$old = $_POST ?? [];
if (isset($old['days']) && is_array($old['days'])) {
    $old['day_array'] = $old['days'];
}
if (!isset($old['lecture_slot_day']) || !is_array($old['lecture_slot_day'])) {
    $old['lecture_slot_day'] = [''];
}
if (!isset($old['lecture_slot_start_time']) || !is_array($old['lecture_slot_start_time'])) {
    $old['lecture_slot_start_time'] = [''];
}
if (!isset($old['lecture_slot_end_time']) || !is_array($old['lecture_slot_end_time'])) {
    $old['lecture_slot_end_time'] = [''];
}
if (!isset($old['lecture_slot_room_id']) || !is_array($old['lecture_slot_room_id'])) {
    $old['lecture_slot_room_id'] = [''];
}

$hasIsGenedFaculty = db_column_exists('faculty', 'is_gened');
$hasIsGenedCourse = db_column_exists('courses', 'is_gened');
$hasIsGenedRoom = db_column_exists('rooms', 'is_gened');

```

```

$hasCourseYearLevel = db_column_exists('courses', 'year_level');
$hasCourseSection = db_column_exists('courses', 'section');
$hasProgramsTable = db_table_exists('programs');
$hasGeScheduleTargets = db_table_exists('ge_schedule_targets');

$programRows = [];
if ($hasProgramsTable) {
    $programRows = db()->query(
        "SELECT p.college_id, p.program_name, c.college_code
        FROM programs p
        INNER JOIN colleges c ON c.id = p.college_id
        WHERE p.status='active'
        ORDER BY c.college_code, p.program_name"
    )->fetchAll();
}

$deanSectionRows = [];
if ($hasCourseYearLevel && $hasCourseSection) {
    $deanSectionRows = db()->query(
        "SELECT DISTINCT college_id, department, year_level, section
        FROM courses
        WHERE COALESCE(is_gened, 0) = 0
        AND college_id IS NOT NULL
        AND TRIM(COALESCE(department, '')) <> ''
        AND TRIM(COALESCE(section, '')) <> ''
        AND TRIM(COALESCE(year_level, '')) <> ''
        ORDER BY college_id, department, year_level, section"
    )->fetchAll();
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $collegeId = (int) ($_POST['college_id'] ?? 0);
    $facultyId = (int) ($_POST['faculty_id'] ?? 0);
    $courseId = (int) ($_POST['course_id'] ?? 0);
    $roomId = (int) ($_POST['room_id'] ?? 0);
    $scheduleType = (string) ($_POST['schedule_type'] ?? 'Custom');
    $days = isset($_POST['days']) && is_array($_POST['days']) ? array_map('strval', $_POST['days']) : [];
    $startTime = substr((string) ($_POST['start_time'] ?? ''), 0, 5) . ':00';
    $endTime = substr((string) ($_POST['end_time'] ?? ''), 0, 5) . ':00';
    $semester = trim((string) ($_POST['semester'] ?? ''));
    $schoolYear = trim((string) ($_POST['school_year'] ?? ''));
    $academicYear = trim((string) ($_POST['academic_year'] ?? ''));
    $targetProgram = trim((string) ($_POST['target_program'] ?? ''));
    $targetYearLevel = trim((string) ($_POST['target_year_level'] ?? ''));
    $targetSection = trim((string) ($_POST['target_section'] ?? ''));
    $lectureSlotDays = isset($_POST['lecture_slot_day']) && is_array($_POST['lecture_slot_day']) ? array_map('strval',
$_POST['lecture_slot_day']) : [];
    $lectureSlotStartTimes = isset($_POST['lecture_slot_start_time']) && is_array($_POST['lecture_slot_start_time']) ? array_map('strval',
$_POST['lecture_slot_start_time']) : [];
    $lectureSlotEndTimes = isset($_POST['lecture_slot_end_time']) && is_array($_POST['lecture_slot_end_time']) ? array_map('strval',
$_POST['lecture_slot_end_time']) : [];
    $lectureSlotRoomIds = isset($_POST['lecture_slot_room_id']) && is_array($_POST['lecture_slot_room_id']) ? array_map('intval',
$_POST['lecture_slot_room_id']) : [];

    if ($collegeId < 1 || $facultyId < 1 || $courseId < 1 || $roomId < 1) {
        $errors[] = 'Please select college, GE faculty, GE course, and GE room.';
    }
    if ($days === []) {
        $errors[] = 'Select at least one day.';
    }
    if ($semester === '' || $schoolYear === '') {
        $errors[] = 'Semester and school year are required.';
    }
    if (!$hasIsGenedFaculty || !$hasIsGenedCourse || !$hasIsGenedRoom) {
        $errors[] = 'Run upgrade_roles.php first to enable GE scheduling.';
    }
    if (!$hasProgramsTable) {
        $errors[] = 'Run upgrade_roles.php first to sync Programs from Dean modules.';
    }
    if ($targetProgram === '' || $targetYearLevel === '' || $targetSection === '') {
        $errors[] = 'Program, year level, and section are required for GE scheduling.';
    }

    $rules = validate_schedule_rules($scheduleType, $days, $startTime, $endTime);
    foreach ($rules['errors'] as $e) {
        $errors[] = $e;
    }

    if (!$errors) {
        $chk = db()->prepare("SELECT COUNT(*) FROM colleges WHERE id=? AND status='active'");

```



```

$chk->execute([$collegeId]);
if ((int) $chk->fetchColumn() < 1) {
    $errors[] = 'Selected college is invalid.';
}
$chk = db()->prepare("SELECT COUNT(*) FROM faculty WHERE id=? AND is_gened=1 AND status='active'");
$chk->execute([$facultyId]);
if ((int) $chk->fetchColumn() < 1) {
    $errors[] = 'Selected faculty is not a GE faculty member.';
}
$chk = db()->prepare('SELECT COUNT(*) FROM courses WHERE id=? AND is_gened=1');
$chk->execute([$courseId]);
if ((int) $chk->fetchColumn() < 1) {
    $errors[] = 'Selected course is not a GE course.';
}
$chk = db()->prepare("SELECT COUNT(*) FROM rooms WHERE id=? AND is_gened=1 AND status IN ('available','tba')");
$chk->execute([$roomId]);
if ((int) $chk->fetchColumn() < 1) {
    $errors[] = 'Selected room is not an available GE room.';
}

try {
    $chk = db()->prepare('SELECT COUNT(*) FROM ge_course_colleges WHERE course_id=? AND college_id=?');
    $chk->execute([$courseId, $collegeId]);
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'This GE course is not assigned to the selected college. Configure it in GE Offerings first.';
    }
} catch (Throwable $e) {
    $errors[] = 'Run upgrade_roles.php first to create GE course offering mappings.';
}

if ($targetProgram !== '') {
    $chk = db()->prepare("SELECT COUNT(*) FROM programs WHERE college_id=? AND program_name=? AND status='active'");
    $chk->execute([$collegeId, $targetProgram]);
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'Selected program is invalid for the chosen college.';
    }
}
}

$segments = [
    'room_id' => $roomId,
    'days' => $days,
    'start_time' => $startTime,
    'end_time' => $endTime,
    'schedule_type' => $scheduleType,
];

$slotCount = max(count($lectureSlotDays), count($lectureSlotStartTimes), count($lectureSlotEndTimes), count($lectureSlotRoomIds));
for ($i = 0; $i < $slotCount; $i++) {
    $slotDay = trim((string) ($lectureSlotDays[$i] ?? ''));
    $slotStartRaw = trim((string) ($lectureSlotStartTimes[$i] ?? ''));
    $slotEndRaw = trim((string) ($lectureSlotEndTimes[$i] ?? ''));
    $slotRoomId = (int) ($lectureSlotRoomIds[$i] ?? 0);
    $isEmpty = $slotDay === '' && $slotStartRaw === '' && $slotEndRaw === '' && $slotRoomId < 1;
    if ($isEmpty) {
        continue;
    }
    if ($slotDay === '' || $slotStartRaw === '' || $slotEndRaw === '' || $slotRoomId < 1) {
        $errors[] = 'Each additional lecture slot must have day, start time, end time, and room.';
        continue;
    }
    $slotStart = substr($slotStartRaw, 0, 5) . ':00';
    $slotEnd = substr($slotEndRaw, 0, 5) . ':00';
    $slotRules = validate_schedule_rules('Custom', [$slotDay], $slotStart, $slotEnd);
    foreach ($slotRules['errors'] as $e) {
        $errors[] = 'Additional lecture slot: ' . $e;
    }
    $chk = db()->prepare("SELECT COUNT(*) FROM rooms WHERE id=? AND is_gened=1 AND status IN ('available','tba')");
    $chk->execute([$slotRoomId]);
    if ((int) $chk->fetchColumn() < 1) {
        $errors[] = 'Selected room for additional lecture slot is not an available GE room.';
    }
    $segments[] = [
        'room_id' => $slotRoomId,
        'days' => [$slotDay],
        'start_time' => $slotStart,
        'end_time' => $slotEnd,
        'schedule_type' => 'Custom',
    ];
}
}

```

```

$conflicts = [];
if (!$errors) {
    foreach ($segments as $seg) {
        $segConflicts = checkConflicts(
            $facultyId,
            (int) $seg['room_id'],
            (array) $seg['days'],
            (string) $seg['start_time'],
            (string) $seg['end_time'],
            $semester,
            $schoolYear,
            null,
            $collegeId
        );
        foreach ($segConflicts as $c) {
            $errors[] = $c['description'];
        }
    }
    $conflicts = array_merge($conflicts, $segConflicts);

    $st = db()->prepare(
        'SELECT s.id, s.day_of_week, s.start_time, s.end_time, c.course_code, c.course_name
        FROM schedules s
        INNER JOIN courses c ON c.id = s.course_id
        WHERE s.college_id = ?
            AND s.semester = ?
            AND s.school_year = ?
            AND c.department = ?
            AND c.year_level = ?
            AND c.section = ?'
    );
    $st->execute([$collegeId, $semester, $schoolYear, $targetProgram, $targetYearLevel, $targetSection]);
    $blockRows = $st->fetchAll();
    $stMin = time_to_minutes((string) $seg['start_time']);
    $enMin = time_to_minutes((string) $seg['end_time']);
    foreach ($blockRows as $br) {
        $existingDays = parse_day_set((string) $br['day_of_week']);
        if (!array_intersect((array) $seg['days'], $existingDays)) {
            continue;
        }
        $rowStart = time_to_minutes((string) $br['start_time']);
        $rowEnd = time_to_minutes((string) $br['end_time']);
        if (intervals_overlap($stMin, $enMin, $rowStart, $rowEnd)) {
            $errors[] = 'Target block conflict: ' . $targetProgram . ' Y' . $targetYearLevel . '-' . $targetSection
                . ' already has ' . $br['course_code'] . ' at this time.';
            break;
        }
    }
}

if ($hasGeScheduleTargets) {
    $st = db()->prepare(
        'SELECT s.id, s.day_of_week, s.start_time, s.end_time, c.course_code
        FROM ge_schedule_targets gst
        INNER JOIN schedules s ON s.id = gst.schedule_id
        INNER JOIN courses c ON c.id = s.course_id
        WHERE gst.college_id = ?
            AND gst.program_name = ?
            AND gst.year_level = ?
            AND gst.section = ?
            AND s.semester = ?
            AND s.school_year = ?'
    );
    $st->execute([$collegeId, $targetProgram, $targetYearLevel, $targetSection, $semester, $schoolYear]);
    $targetRows = $st->fetchAll();
    foreach ($targetRows as $tr) {
        $existingDays = parse_day_set((string) $tr['day_of_week']);
        if (!array_intersect((array) $seg['days'], $existingDays)) {
            continue;
        }
        $rowStart = time_to_minutes((string) $tr['start_time']);
        $rowEnd = time_to_minutes((string) $tr['end_time']);
        if (intervals_overlap($stMin, $enMin, $rowStart, $rowEnd)) {
            $errors[] = 'GE block conflict: another GE schedule (' . $tr['course_code'] . ') already targets '
                . $targetProgram . ' Y' . $targetYearLevel . '-' . $targetSection . ' at this time.';
            break;
        }
    }
}
}
}
}

```

```

if (!$errors) {
    $hasSessionKind = db_column_exists('schedules', 'session_kind');
    $scheduleInsertSql = 'INSERT INTO schedules (faculty_id, course_id, room_id, college_id, schedule_type, day_of_week, start_time,
end_time, semester, school_year, academic_year, created_by'
        . ($hasSessionKind ? ', session_kind' : '')
        . ') VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?'
        . ($hasSessionKind ? ', ?' : ''))
        . ')';
    $insert = db()->prepare($scheduleInsertSql);
    $createdIds = [];
    foreach ($segments as $seg) {
        $insertParams = [
            $facultyId,
            $courseId,
            (int) $seg['room_id'],
            $collegeId,
            (string) $seg['schedule_type'],
            days_to_set((array) $seg['days']),
            (string) $seg['start_time'],
            (string) $seg['end_time'],
            $semester,
            $schoolYear,
            $academicYear,
            (int) $_SESSION['user_id'],
        ];
        if ($hasSessionKind) {
            $insertParams[] = schedule_segment_session_kind((string) ($seg['label'] ?? 'Lecture'));
        }
        $insert->execute($insertParams);
        $newId = (int) db()->lastInsertId();
        $createdIds[] = $newId;
        if ($hasGeScheduleTargets && $targetProgram !== '' && $targetYearLevel !== '' && $targetSection !== '') {
            db()->prepare(
                'INSERT INTO ge_schedule_targets (schedule_id, college_id, program_name, year_level, section)
                VALUES (?, ?, ?, ?, ?)'
            )->execute([$newId, $collegeId, $targetProgram, $targetYearLevel, $targetSection]);
        }
        if ($conflicts) {
            log_conflicts($newId, $conflicts, false);
        }
        $stGe = db()->prepare(
            'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year,
            s.day_of_week, s.start_time, s.end_time, s.schedule_type,
            f.full_name AS faculty_name, c.course_code, r.room_code
            FROM schedules s
            INNER JOIN faculty f ON f.id = s.faculty_id
            INNER JOIN courses c ON c.id = s.course_id
            INNER JOIN rooms r ON r.id = s.room_id
            WHERE s.id = ? LIMIT 1'
        );
        $stGe->execute([$newId]);
        $afterGe = $stGe->fetch(PDO::FETCH_ASSOC);
        log_user_activity('add', 'Schedules (GEN ED)', 'GE schedule #' . $newId, null, $afterGe ? (array) $afterGe : null);
    }
    $_SESSION['flash'] = count($createdIds) > 1
        ? 'GE schedules created successfully.'
        : 'GE schedule created successfully.';
    header('Location: schedule.php');
    exit;
}
}

$facultyList = $hasIsGenedFaculty
    ? db()->query("SELECT id, faculty_id, full_name FROM faculty WHERE status='active' AND is_gened=1 ORDER BY full_name")->fetchAll()
    : [];
$courseList = $hasIsGenedCourse
    ? db()->query('SELECT id, course_code, course_name FROM courses WHERE is_gened=1 ORDER BY course_code')->fetchAll()
    : [];
$roomList = $hasIsGenedRoom
    ? db()->query("SELECT id, room_code, room_name FROM rooms WHERE is_gened=1 AND status IN ('available','tba') ORDER BY room_code")->fetchAll()
    : [];
$collegeList = db()->query("SELECT id, college_code, college_name FROM colleges WHERE status='active' ORDER BY college_code")->fetchAll();
$days = schedule_days_list();
$semesters = ['1st Semester', '2nd Semester', 'Summer'];
$types = ['MW', 'TTH', 'MWF', 'TTHS', 'Saturday', 'Sunday', 'MW_TTH', 'Custom'];
$selDays = isset($old['day_array']) ? (array) $old['day_array'] : [];
$hasProgramsYearLevels = db_table_exists('programs_year_levels');
$programYearLevelsByCollege = [];

```

```

if ($hasProgramsYearLevels) {
    $y1St = db()->query(
        'SELECT p.college_id AS cid, p.program_name AS pname, pyl.year_level AS yl
        FROM programs_year_levels pyl
        INNER JOIN programs p ON p.id = pyl.program_id'
    );
    while ($rw = $y1St->fetch(PDO::FETCH_ASSOC)) {
        $cid = (int) ($rw['cid'] ?? 0);
        $pname = trim((string) ($rw['pname'] ?? ''));
        $yl = trim((string) ($rw['yl'] ?? ''));
        if ($cid < 1 || $pname === '' || $yl === '') {
            continue;
        }
        if (!isset($programYearLevelsByCollege[$cid])) {
            $programYearLevelsByCollege[$cid] = [];
        }
        if (!isset($programYearLevelsByCollege[$cid][$pname])) {
            $programYearLevelsByCollege[$cid][$pname] = [];
        }
        if (!in_array($yl, $programYearLevelsByCollege[$cid][$pname], true)) {
            $programYearLevelsByCollege[$cid][$pname][] = $yl;
        }
    }
}

$programsByCollege = [];
foreach ($programRows as $pr) {
    $cid = (int) $pr['college_id'];
    if (!isset($programsByCollege[$cid])) {
        $programsByCollege[$cid] = [];
    }
    $nm = (string) $pr['program_name'];
    $deanYl = [];
    if (isset($programYearLevelsByCollege[$cid][$nm])) {
        $deanYl = sort_schedule_year_levels($programYearLevelsByCollege[$cid][$nm]);
    }
    $programsByCollege[$cid][] = [
        'name' => $nm,
        'college_code' => (string) $pr['college_code'],
        'year_levels' => ($deanYl !== []) ? $deanYl : ['1', '2', '3', '4', '5'],
    ];
}

$sectionsByScope = [];
foreach ($deanSectionRows as $sr) {
    $cid = (int) ($sr['college_id'] ?? 0);
    $program = (string) ($sr['department'] ?? '');
    $yl = (string) ($sr['year_level'] ?? '');
    $sec = (string) ($sr['section'] ?? '');
    if ($cid < 1 || $program === '' || $yl === '' || $sec === '') {
        continue;
    }
    if (!isset($sectionsByScope[$cid])) {
        $sectionsByScope[$cid] = [];
    }
    if (!isset($sectionsByScope[$cid][$program])) {
        $sectionsByScope[$cid][$program] = [];
    }
    if (!isset($sectionsByScope[$cid][$program][$yl])) {
        $sectionsByScope[$cid][$program][$yl] = [];
    }
    if (!in_array($sec, $sectionsByScope[$cid][$program][$yl], true)) {
        $sectionsByScope[$cid][$program][$yl][] = $sec;
    }
}

$pageTitle = 'Add GE Schedule';
require_once __DIR__ . '/includes/header.php';
?>

<h1 class="h3 mb-4"><i class="fa-solid fa-plus me-2 text-primary"></i>Add GE schedule</h1>
<?php render_schedule_errors_warning_popup($errors); ?>

<div class="card shadow-sm">
    <div class="card-body">
        <form method="post" class="row g-3">
            <div class="col-md-4">
                <label class="form-label">Target college</label>
                <select name="college_id" id="college_id" class="form-select" required>
                    <option value="">- Select -</option>
                    <?php foreach ($collegeList as $c): ?>
                        <option value="<?= (int) $c['id'] ?>" <?= (int) ($old['college_id'] ?? 0) === (int) $c['id'] ? 'selected' : '' ?>>

```

```

                <?= htmlspecialchars($c['college_code'] . ' - ' . $c['college_name']) ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-4">
    <label class="form-label">Target program</label>
    <select name="target_program" id="target_program" class="form-select" required>
        <option value="">- Select program -</option>
    </select>
</div>
<div class="col-md-2">
    <label class="form-label">Target year level</label>
    <select name="target_year_level" id="target_year_level" class="form-select" required>
        <option value="">- Select -</option>
    </select>
</div>
<div class="col-md-2">
    <label class="form-label">Target section</label>
    <?php if ($hasCourseYearLevel && $hasCourseSection): ?>
        <select name="target_section" id="target_section" class="form-select" required>
            <option value="">- Select section -</option>
        </select>
    <?php else: ?>
        <input type="text" name="target_section" class="form-control" maxlength="20" required value="<?=
htmlspecialchars((string) ($old['target_section'] ?? '')) ?>">
    <?php endif; ?>
</div>
<div class="col-md-4">
    <label class="form-label">GE Faculty</label>
    <select name="faculty_id" class="form-select" required>
        <option value="">- Select -</option>
        <?php foreach ($facultyList as $f): ?>
            <option value="<?= (int) $f['id'] ?>" <?= (int) ($old['faculty_id'] ?? 0) == (int) $f['id'] ? 'selected' : '' ?>>
                <?= htmlspecialchars($f['full_name'] . ' (' . $f['faculty_id'] . ')') ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-4">
    <label class="form-label">GE Course</label>
    <select name="course_id" class="form-select" required>
        <option value="">- Select -</option>
        <?php foreach ($courseList as $c): ?>
            <option value="<?= (int) $c['id'] ?>" <?= (int) ($old['course_id'] ?? 0) == (int) $c['id'] ? 'selected' : '' ?>>
                <?= htmlspecialchars($c['course_code'] . ' - ' . $c['course_name']) ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-4">
    <label class="form-label">GE Room</label>
    <select name="room_id" class="form-select" required>
        <option value="">- Select -</option>
        <?php foreach ($roomList as $r): ?>
            <option value="<?= (int) $r['id'] ?>" <?= (int) ($old['room_id'] ?? 0) == (int) $r['id'] ? 'selected' : '' ?>>
                <?= htmlspecialchars($r['room_code'] . ' (' . $r['room_name'] . ')') ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-4">
    <label class="form-label">Schedule type</label>
    <select name="schedule_type" class="form-select">
        <?php foreach ($types as $t): ?>
            <option value="<?= $t ?>" <?= (($old['schedule_type'] ?? 'Custom') == $t) ? 'selected' : '' ?>>
                <?= htmlspecialchars($t) ?></option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-2"><label class="form-label">Start time</label><input type="time" name="start_time" class="form-control"
value="<?= htmlspecialchars(substr((string) ($old['start_time'] ?? '08:00'), 0, 5)) ?>"></div>
<div class="col-md-2"><label class="form-label">End time</label><input type="time" name="end_time" class="form-control" value="
<?= htmlspecialchars(substr((string) ($old['end_time'] ?? '09:00'), 0, 5)) ?>"></div>
<div class="col-12">
    <label class="form-label">Days</label>
    <div class="d-flex flex-wrap gap-3">
        <?php foreach ($days as $d): ?>
            <div class="form-check">
                <input class="form-check-input" type="checkbox" name="days[]" value="<?= htmlspecialchars($d) ?>" id="d_<?=

```

```

htmlspecialchars($d) ?>" <?= in_array($d, $selDays, true) ? 'checked' : '' ?>>
    <label class="form-check-label" for="d_<?= htmlspecialchars($d) ?>"><?= htmlspecialchars($d) ?></label>
    </div>
    <?php endforeach; ?>
</div>
</div>
<div class="col-12">
    <div class="p-3 border rounded bg-light">
        <div class="d-flex align-items-center justify-content-between mb-2">
            <h6 class="mb-0">Additional lecture time slots</h6>
            <button type="button" class="btn btn-sm btn-outline-primary" id="addLectureSlotBtn">
                <i class="fa-solid fa-plus me-1"></i>Add slot
            </button>
        </div>
        <div class="small text-muted mb-3">
            Use this when GE lecture days have different times.
        </div>
        <div id="lectureSlotsWrap">
            <?php
                $slotDays = is_array($old['lecture_slot_day'] ?? null) ? $old['lecture_slot_day'] : [];
                $slotStarts = is_array($old['lecture_slot_start_time'] ?? null) ? $old['lecture_slot_start_time'] : [];
                $slotEnds = is_array($old['lecture_slot_end_time'] ?? null) ? $old['lecture_slot_end_time'] : [];
                $slotRooms = is_array($old['lecture_slot_room_id'] ?? null) ? $old['lecture_slot_room_id'] : [];
                $slotTotal = max(1, count($slotDays), count($slotStarts), count($slotEnds), count($slotRooms));
                for ($i = 0; $i < $slotTotal; $i++):
                    $slotDayValue = (string) ($slotDays[$i] ?? '');
                    $slotStartValue = substr((string) ($slotStarts[$i] ?? ''), 0, 5);
                    $slotEndValue = substr((string) ($slotEnds[$i] ?? ''), 0, 5);
                    $slotRoomValue = (int) ($slotRooms[$i] ?? 0);
                ?>
                <div class="row g-2 mb-2 lecture-slot-row">
                    <div class="col-md-3">
                        <select name="lecture_slot_day[]" class="form-select">
                            <option value="">Day</option>
                            <?php foreach ($days as $d): ?>
                                <option value="<?= htmlspecialchars($d) ?>" <?= $slotDayValue === $d ? 'selected' : '' ?><?=
htmlspecialchars($d) ?></option>
                            <?php endforeach; ?>
                        </select>
                    </div>
                    <div class="col-md-3">
                        <input type="time" name="lecture_slot_start_time[]" class="form-control" value="<?=
htmlspecialchars($slotStartValue) ?>">
                    </div>
                    <div class="col-md-3">
                        <input type="time" name="lecture_slot_end_time[]" class="form-control" value="<?=
htmlspecialchars($slotEndValue) ?>">
                    </div>
                    <div class="col-md-3">
                        <div class="d-flex gap-2">
                            <select name="lecture_slot_room_id[]" class="form-select">
                                <option value="">GE Room</option>
                                <?php foreach ($roomList as $r): ?>
                                    <option value="<?= (int) $r['id'] ?>" <?= $slotRoomValue === (int) $r['id'] ? 'selected' : '' ?
>>
                                    <?= htmlspecialchars($r['room_code'] . ($r['room_name'] ? ' - ' . $r['room_name'] : '')) ?>
                                </option>
                                <?php endforeach; ?>
                            </select>
                            <button type="button" class="btn btn-outline-danger remove-lecture-slot" title="Remove slot">
                                <i class="fa-solid fa-trash"></i>
                            </button>
                        </div>
                    </div>
                </div>
            <?php endforeach; ?>
        </div>
    </div>
</div>
<div class="col-md-3">
    <label class="form-label">Semester</label>
    <select name="semester" class="form-select">
        <?php foreach ($semesters as $s): ?><option value="<?= htmlspecialchars($s) ?>" <?= (($old['semester'] ?? '') === $s) ?
'selected' : '' ?><?= htmlspecialchars($s) ?></option><?php endforeach; ?>
    </select>
</div>
<div class="col-md-3"><label class="form-label">School year</label><input type="text" name="school_year" class="form-control"
value="<?= htmlspecialchars((string) ($old['school_year'] ?? (date('Y') . '-' . (date('Y') + 1)))) ?>"></div>
<div class="col-md-3"><label class="form-label">Academic year</label><input type="text" name="academic_year" class="form-
control" value="<?= htmlspecialchars((string) ($old['academic_year'] ?? '')) ?>"></div>

```

```

        <div class="col-12"><div class="alert alert-info mb-0">GE schedules are targeted per college program/year/section to avoid block
conflicts with major schedules.</div></div>
        <div class="col-12">
            <button type="submit" class="btn btn-primary">Creates the GE schedule entry with course, faculty, room,
and time. Use when adding a new GE section.</button>
            <a href="schedule.php" class="btn btn-outline-secondary">Returns to the main schedule list without
saving this form.</a>
        </div>
    </form>
</div>
</div>

```

```

<script>
(() => {
    const data = <?=json_encode($programsByCollege, JSON_UNESCAPED_SLASHES) ?>;
    const collegeSel = document.getElementById('college_id');
    const programSel = document.getElementById('target_program');
    const yearSel = document.getElementById('target_year_level');
    const sectionSel = document.getElementById('target_section');
    const sectionData = <?=json_encode($sectionsByScope, JSON_UNESCAPED_SLASHES) ?>;
    const selectedProgram = <?=json_encode((string) ($old['target_program'] ?? ''), JSON_UNESCAPED_SLASHES) ?>;
    const selectedSection = <?=json_encode((string) ($old['target_section'] ?? ''), JSON_UNESCAPED_SLASHES) ?>;
    const selectedYearInitial = <?=json_encode((string) ($old['target_year_level'] ?? ''), JSON_UNESCAPED_SLASHES) ?>;
    let yearInitialApplied = false;

    function yearLevelsForCurrentTarget() {
        const cid = collegeSel.value;
        const pname = programSel.value || '';
        const rows = data[cid] || [];
        for (let i = 0; i < rows.length; i++) {
            if (rows[i].name === pname) {
                return Array.isArray(rows[i].year_levels) && rows[i].year_levels.length ? rows[i].year_levels : ['1','2','3','4','5'];
            }
        }
        return ['1','2','3','4','5'];
    }

    function renderYearLevels() {
        if (!yearSel) {
            return;
        }
        const prev = yearSel.value || '';
        const list = yearLevelsForCurrentTarget();
        yearSel.innerHTML = '';
        const z = document.createElement('option');
        z.value = '';
        z.textContent = '\u2014 Select \u2014';
        yearSel.appendChild(z);
        for (let i = 0; i < list.length; i++) {
            const lvl = String(list[i]);
            const opt = document.createElement('option');
            opt.value = lvl;
            opt.textContent = lvl;
            yearSel.appendChild(opt);
        }
        let pick = '';
        if (!yearInitialApplied && selectedYearInitial && list.indexOf(selectedYearInitial) !== -1) {
            pick = selectedYearInitial;
            yearInitialApplied = true;
        } else if (prev && list.indexOf(prev) !== -1) {
            pick = prev;
        }
        yearSel.value = pick || '';
    }

    function renderPrograms() {
        const cid = collegeSel.value;
        const rows = data[cid] || [];
        programSel.innerHTML = '';
        const first = document.createElement('option');
        first.value = '';
        first.textContent = '\u2014 Select program \u2014';
        programSel.appendChild(first);
        rows.forEach((r) => {
            const opt = document.createElement('option');
            opt.value = r.name;
            opt.textContent = r.name;
            if (selectedProgram && selectedProgram === r.name) {
                opt.selected = true;
            }
        })
    }

```

```

        programSel.appendChild(opt);
    });
}

function renderSections() {
    if (!sectionSel || !yearSel) {
        return;
    }
    const cid = collegeSel ? collegeSel.value : '';
    const program = programSel ? programSel.value : '';
    const yl = yearSel.value;
    const rows = (((sectionData[cid] || {})[program] || {})[yl] || []);
    sectionSel.innerHTML = '';
    const first = document.createElement('option');
    first.value = '';
    first.textContent = '- Select section -';
    sectionSel.appendChild(first);
    rows.forEach((s) => {
        const opt = document.createElement('option');
        opt.value = s;
        opt.textContent = s;
        if (selectedSection && selectedSection === s) {
            opt.selected = true;
        }
        sectionSel.appendChild(opt);
    });
}

collegeSel.addEventListener('change', () => {
    renderPrograms();
    renderYearLevels();
    renderSections();
});

if (yearSel) {
    yearSel.addEventListener('change', () => {
        if (!sectionSel) {
            return;
        }
        sectionSel.innerHTML = '';
        const cid = collegeSel ? collegeSel.value : '';
        const program = programSel ? programSel.value : '';
        const rows = (((sectionData[cid] || {})[program] || {})[yearSel.value] || []);
        const first = document.createElement('option');
        first.value = '';
        first.textContent = '- Select section -';
        sectionSel.appendChild(first);
        rows.forEach((s) => {
            const opt = document.createElement('option');
            opt.value = s;
            opt.textContent = s;
            sectionSel.appendChild(opt);
        });
    });
}

if (programSel) {
    programSel.addEventListener('change', () => {
        renderYearLevels();
        renderSections();
    });
}

if (collegeSel) {
    collegeSel.addEventListener('change', renderSections);
}

renderPrograms();
renderYearLevels();
renderSections();

const wrap = document.getElementById('lectureSlotsWrap');
const addBtn = document.getElementById('addLectureSlotBtn');
if (wrap && addBtn) {
    const cloneTemplateRow = () => {
        const source = wrap.querySelector('.lecture-slot-row');
        if (!source) return null;
        const clone = source.cloneNode(true);
        clone.querySelectorAll('input').forEach((el) => {
            el.value = '';
        });
        clone.querySelectorAll('select').forEach((el) => {
            el.selectedIndex = 0;
        });
    };
}

```



```

    });
    return clone;
};

addBtn.addEventListener('click', () => {
    const row = cloneTemplateRow();
    if (row) {
        wrap.appendChild(row);
    }
});

wrap.addEventListener('click', (event) => {
    const btn = event.target.closest('.remove-lecture-slot');
    if (!btn) return;
    const rows = wrap.querySelectorAll('.lecture-slot-row');
    if (rows.length <= 1) {
        const row = btn.closest('.lecture-slot-row');
        if (!row) return;
        row.querySelectorAll('input').forEach((el) => {
            el.value = '';
        });
        row.querySelectorAll('select').forEach((el) => {
            el.selectedIndex = 0;
        });
        return;
    }
    const row = btn.closest('.lecture-slot-row');
    if (row) row.remove();
});
}
})();
</script>

```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>
```

```
-----
FILE: global_reports.php
-----
```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['admin']);

/** @return array{title:string,columns:string[],rows:list<array<string,mixed>>} */
function build_admin_report(string $reportKey, int $collegeId, int $facultyId, string $day): array
{
    $title = 'Report';
    $columns = [];
    $rows = [];
    $employmentStatusExpr = db_column_exists('faculty', 'employment_status')
        ? "COALESCE(NULLIF(TRIM(f.employment_status), ''), 'Permanent')"
        : "'Permanent'";

    switch ($reportKey) {
        case 'master_faculty_list':
            $title = 'Master Faculty List (All Colleges)';
            $columns = ['Faculty ID', 'Name', 'Department/Program', 'Employment Status', 'Email', 'College', 'Status'];
            $rows = db()->query(
                "SELECT f.faculty_id, f.full_name, f.department,
                    {$employmentStatusExpr} AS employment_status,
                    f.email,
                    COALESCE(c.college_code, 'Unassigned') AS college_code, f.status
                FROM faculty f
                LEFT JOIN colleges c ON c.id = f.college_id
                ORDER BY c.college_code, f.full_name"
            )->fetchAll();
            break;
        case 'faculty_per_college':
            $title = 'Faculty per College';
            $columns = ['College Code', 'College Name', 'Faculty Count'];
            $rows = db()->query(
                "SELECT c.college_code, c.college_name, COUNT(f.id) AS faculty_count
                FROM colleges c
                LEFT JOIN faculty f ON f.college_id = c.id
                GROUP BY c.id
                ORDER BY c.college_code"
            );
            break;
    }
}

```

```

    )->fetchAll();
    break;
case 'faculty_per_department':
case 'faculty_per_program':
    $title = $reportKey === 'faculty_per_department' ? 'Faculty per Department' : 'Faculty per Program';
    $columns = ['College', 'Department/Program', 'Faculty Count'];
    $rows = db()->query(
        "SELECT COALESCE(c.college_code, 'Unassigned') AS college_code,
            COALESCE(NULLIF(TRIM(f.department), ''), 'Unspecified') AS dept_program,
            COUNT(*) AS faculty_count
        FROM faculty f
        LEFT JOIN colleges c ON c.id = f.college_id
        GROUP BY c.college_code, dept_program
        ORDER BY c.college_code, dept_program"
    )->fetchAll();
    break;
case 'faculty_per_year_level':
    $title = 'Faculty per Year Level';
    $columns = ['Year Level', 'Faculty Count'];
    $rows = db()->query(
        "SELECT COALESCE(NULLIF(TRIM(c.year_level), ''), 'Unspecified') AS year_level,
            COUNT(DISTINCT s.faculty_id) AS faculty_count
        FROM schedules s
        INNER JOIN courses c ON c.id = s.course_id
        GROUP BY year_level
        ORDER BY year_level"
    )->fetchAll();
    break;
case 'active_inactive_faculty':
    $title = 'Active/Inactive Faculty';
    $columns = ['College', 'Status', 'Faculty Count'];
    $rows = db()->query(
        "SELECT COALESCE(c.college_code, 'Unassigned') AS college_code, f.status, COUNT(*) AS faculty_count
        FROM faculty f
        LEFT JOIN colleges c ON c.id = f.college_id
        GROUP BY c.college_code, f.status
        ORDER BY c.college_code, f.status"
    )->fetchAll();
    break;
case 'faculty_weekly_schedule':
    $title = 'Faculty Weekly Schedule (Individual)';
    $columns = ['Faculty', 'Course', 'Days', 'Time', 'Room', 'Term'];
    if ($facultyId > 0) {
        $st = db()->prepare(
            "SELECT f.full_name AS faculty_name, CONCAT(c.course_code, ' - ', c.course_name) AS course_title,
                s.day_of_week, CONCAT(SUBSTR(s.start_time,1,5), '-', SUBSTR(s.end_time,1,5)) AS schedule_time,
                r.room_code, CONCAT(s.semester, ' / ', s.school_year) AS term_label
            FROM schedules s
            INNER JOIN faculty f ON f.id = s.faculty_id
            INNER JOIN courses c ON c.id = s.course_id
            INNER JOIN rooms r ON r.id = s.room_id
            WHERE s.faculty_id = ?
            ORDER BY s.day_of_week, s.start_time"
        );
        $st->execute([$facultyId]);
        $rows = $st->fetchAll();
    }
    break;
case 'faculty_daily_schedule':
    $title = 'Faculty Daily Schedule';
    $columns = ['Faculty', 'Course', 'Day', 'Time', 'Room', 'Term'];
    $st = db()->prepare(
        "SELECT f.full_name AS faculty_name, CONCAT(c.course_code, ' - ', c.course_name) AS course_title,
            ? AS day_name, CONCAT(SUBSTR(s.start_time,1,5), '-', SUBSTR(s.end_time,1,5)) AS schedule_time,
            r.room_code, CONCAT(s.semester, ' / ', s.school_year) AS term_label
        FROM schedules s
        INNER JOIN faculty f ON f.id = s.faculty_id
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN rooms r ON r.id = s.room_id
        WHERE FIND_IN_SET(?, s.day_of_week) > 0
        ORDER BY f.full_name, s.start_time"
    );
    $st->execute([$day, $day]);
    $rows = $st->fetchAll();
    break;
case 'all_faculty_schedules':
    $title = 'All Faculty Schedules (Consolidated)';
    $columns = ['Faculty', 'Course', 'Days', 'Time', 'Room', 'Term'];
    $rows = db()->query(
        "SELECT f.full_name AS faculty_name, CONCAT(c.course_code, ' - ', c.course_name) AS course_title,

```

```

        s.day_of_week, CONCAT(SUBSTR(s.start_time,1,5), '-', SUBSTR(s.end_time,1,5)) AS schedule_time,
        r.room_code, CONCAT(s.semester, ' / ', s.school_year) AS term_label
    FROM schedules s
    INNER JOIN faculty f ON f.id = s.faculty_id
    INNER JOIN courses c ON c.id = s.course_id
    INNER JOIN rooms r ON r.id = s.room_id
    ORDER BY f.full_name, s.day_of_week, s.start_time"
)->fetchAll();
break;
case 'faculty_load_summary':
$title = 'Faculty Load Summary';
$columns = ['Faculty', 'College', 'Max Hours/Day', 'Total Weekly Hours'];
$rows = db()->query(
    "SELECT f.full_name AS faculty_name, COALESCE(c.college_code, 'Unassigned') AS college_code,
        f.max_hours_per_day,
        ROUND(SUM(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)) / 60, 2) AS total_weekly_hours
    FROM faculty f
    LEFT JOIN colleges c ON c.id = f.college_id
    LEFT JOIN schedules s ON s.faculty_id = f.id
    GROUP BY f.id
    ORDER BY total_weekly_hours DESC, f.full_name"
)->fetchAll();
break;
case 'faculty_availability':
$title = 'Faculty Availability Report';
$columns = ['Faculty', 'Max Hours/Day', 'Estimated Weekly Capacity', 'Assigned Weekly Hours', 'Availability'];
$rows = db()->query(
    "SELECT f.full_name AS faculty_name, f.max_hours_per_day,
        (f.max_hours_per_day * 5) AS weekly_capacity,
        ROUND(COALESCE(SUM(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)), 0) / 60, 2) AS assigned_weekly_hours,
        CASE
            WHEN ROUND(COALESCE(SUM(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)), 0) / 60, 2) >= (f.max_hours_per_day *
5) THEN 'Fully Loaded'
            WHEN ROUND(COALESCE(SUM(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)), 0) / 60, 2) >= (f.max_hours_per_day *
4) THEN 'Near Capacity'
            ELSE 'Available'
        END AS availability_status
    FROM faculty f
    LEFT JOIN schedules s ON s.faculty_id = f.id
    GROUP BY f.id
    ORDER BY assigned_weekly_hours DESC"
)->fetchAll();
break;
case 'total_faculty_per_college':
$title = 'Total Faculty per College';
$columns = ['College Code', 'College Name', 'Total Faculty'];
$rows = db()->query(
    "SELECT c.college_code, c.college_name, COUNT(f.id) AS total_faculty
    FROM colleges c
    LEFT JOIN faculty f ON f.college_id = c.id
    GROUP BY c.id
    ORDER BY c.college_code"
)->fetchAll();
break;
case 'faculty_load_distribution':
$title = 'Faculty Teaching Load Distribution';
$columns = ['Load Band', 'Faculty Count'];
$rows = db()->query(
    "SELECT
        CASE
            WHEN load_hours < 6 THEN 'Under 6 hrs/week'
            WHEN load_hours < 12 THEN '6-12 hrs/week'
            WHEN load_hours < 18 THEN '12-18 hrs/week'
            ELSE '18+ hrs/week'
        END AS load_band,
        COUNT(*) AS faculty_count
    FROM (
        SELECT f.id,
            ROUND(COALESCE(SUM(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)),0) / 60, 2) AS load_hours
        FROM faculty f
        LEFT JOIN schedules s ON s.faculty_id = f.id
        GROUP BY f.id
    ) t
    GROUP BY load_band
    ORDER BY MIN(load_hours)"
)->fetchAll();
break;
case 'faculty_per_course':
$title = 'Faculty per Course';
$columns = ['Course Code', 'Course Name', 'Assigned Faculty'];

```

```

$rows = db()->query(
    "SELECT c.course_code, c.course_name, COUNT(DISTINCT s.faculty_id) AS assigned_faculty
    FROM courses c
    LEFT JOIN schedules s ON s.course_id = c.id
    GROUP BY c.id
    ORDER BY assigned_faculty DESC, c.course_code"
)->fetchAll();
break;
case 'faculty_room_assignment':
$title = 'Faculty Room Assignment';
$columns = ['Room', 'Faculty', 'Assignments'];
$rows = db()->query(
    "SELECT r.room_code, f.full_name AS faculty_name, COUNT(*) AS assignments
    FROM schedules s
    INNER JOIN rooms r ON r.id = s.room_id
    INNER JOIN faculty f ON f.id = s.faculty_id
    GROUP BY r.id, f.id
    ORDER BY r.room_code, assignments DESC"
)->fetchAll();
break;
case 'college_faculty_list':
$title = 'College Faculty List';
$columns = ['College', 'Faculty ID', 'Name', 'Department/Program', 'Employment Status', 'Status'];
if ($collegeId > 0) {
    $st = db()->prepare(
        "SELECT CONCAT(c.college_code, ' - ', c.college_name) AS college_label,
        f.faculty_id, f.full_name, f.department,
        {$employmentStatusExpr} AS employment_status,
        f.status
        FROM faculty f
        INNER JOIN colleges c ON c.id = f.college_id
        WHERE f.college_id = ?
        ORDER BY f.full_name"
    );
    $st->execute([$collegeId]);
    $rows = $st->fetchAll();
}
break;
}

return ['title' => $title, 'columns' => $columns, 'rows' => $rows];
}

$reportCatalog = [
    'FACULTY REPORTS' => [
        'master_faculty_list' => 'Master Faculty List (All Colleges)',
        'faculty_per_college' => 'Faculty per College',
        'faculty_per_department' => 'Faculty per Department',
        'faculty_per_program' => 'Faculty per Program',
        'faculty_per_year_level' => 'Faculty per Year Level',
        'active_inactive_faculty' => 'Active/Inactive Faculty',
    ],
    'SCHEDULE REPORTS' => [
        'faculty_weekly_schedule' => 'Faculty Weekly Schedule (Individual)',
        'faculty_daily_schedule' => 'Faculty Daily Schedule',
        'all_faculty_schedules' => 'All Faculty Schedules (Consolidated)',
        'faculty_load_summary' => 'Faculty Load Summary',
        'faculty_availability' => 'Faculty Availability Report',
    ],
    'SUMMARY REPORTS' => [
        'total_faculty_per_college' => 'Total Faculty per College',
        'faculty_load_distribution' => 'Faculty Teaching Load Distribution',
        'faculty_per_course' => 'Faculty per Course',
        'faculty_room_assignment' => 'Faculty Room Assignment',
    ],
    'COLLEGE REPORTS' => [
        'college_faculty_list' => 'College Faculty List',
    ],
];

$flatCatalog = [];
foreach ($reportCatalog as $group => $items) {
    foreach ($items as $key => $label) {
        $flatCatalog[$key] = $label;
    }
}

$report = (string) ($_GET['report'] ?? 'master_faculty_list');
if (!isset($flatCatalog[$report])) {
    $report = 'master_faculty_list';
}

```

```

}
$collegeId = (int) ($_GET['college_id'] ?? 0);
$facultyId = (int) ($_GET['faculty_id'] ?? 0);
$day = (string) ($_GET['day'] ?? 'Monday');
$output = (string) ($_GET['output'] ?? '');

$colleges = db()->query('SELECT id, college_code, college_name FROM colleges ORDER BY college_code')->fetchAll();
$facultyFilterList = db()->query(
    "SELECT f.id, f.full_name, COALESCE(c.college_code, 'N/A') AS college_code
    FROM faculty f
    LEFT JOIN colleges c ON c.id = f.college_id
    ORDER BY f.full_name"
)->fetchAll();

$reportPayload = build_admin_report($report, $collegeId, $facultyId, $day);

if ($output === 'excel') {
    header('Content-Type: text/csv; charset=utf-8');
    header('Content-Disposition: attachment; filename="admin_report_' . $report . '_' . date('Ymd_His') . '.csv"');
    $fp = fopen('php://output', 'w');
    if ($fp !== false) {
        fputcsv($fp, [$reportPayload['title']]);
        fputcsv($fp, []);
        fputcsv($fp, $reportPayload['columns']);
        foreach ($reportPayload['rows'] as $r) {
            fputcsv($fp, array_values($r));
        }
        fclose($fp);
    }
    exit;
}

$pageTitle = 'Admin Print Module';
require_once __DIR__ . '/includes/header.php';
?>
<div class="report-hub">
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-4">
    <h1 class="h3 mb-0"><i class="fa-solid fa-chart-column me-2 text-primary"></i>Admin Print Module</h1>
    <span class="badge text-bg-primary px-3 py-2">Improved Reports Center</span>
</div>

<form method="get" class="card shadow-sm mb-4 no-print">
    <div class="card-body">
        <div class="row g-3">
            <div class="col-lg-6">
                <label class="form-label">Report Type</label>
                <select name="report" class="form-select">
                    <?php foreach ($reportCatalog as $group => $items): ?>
                        <optgroup label="<? = htmlspecialchars($group) ?>">
                            <?php foreach ($items as $key => $label): ?>
                                <option value="<? = htmlspecialchars($key) ?>" <? = $report === $key ? 'selected' : '' ?><? =
htmlspecialchars($label) ?></option>
                            <?php endforeach; ?>
                        </optgroup>
                    <?php endforeach; ?>
                </select>
            </div>
            <div class="col-lg-3">
                <label class="form-label">College Filter</label>
                <select name="college_id" class="form-select">
                    <option value="0">All / Not selected</option>
                    <?php foreach ($colleges as $c): ?>
                        <option value="<? = (int) $c['id'] ?>" <? = $collegeId === (int) $c['id'] ? 'selected' : '' ?>>
                            <? = htmlspecialchars($c['college_code'] . ' - ' . $c['college_name']) ?>
                        </option>
                    <?php endforeach; ?>
                </select>
            </div>
            <div class="col-lg-3">
                <label class="form-label">Faculty Filter</label>
                <select name="faculty_id" class="form-select">
                    <option value="0">All / Not selected</option>
                    <?php foreach ($facultyFilterList as $f): ?>
                        <option value="<? = (int) $f['id'] ?>" <? = $facultyId === (int) $f['id'] ? 'selected' : '' ?>>
                            <? = htmlspecialchars($f['full_name'] . ' (' . $f['college_code'] . ')') ?>
                        </option>
                    <?php endforeach; ?>
                </select>
            </div>
            <div class="col-lg-3">

```

```

<label class="form-label">Day (Daily report)</label>
<select name="day" class="form-select">
    <?php foreach (['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'] as $d): ?>
        <option value="<? $d ?>" <? $day == $d ? 'selected' : '' ?><? $d ?></option>
    <?php endforeach; ?>
</select>
</div>
<div class="col-lg-9 d-flex flex-wrap gap-2 align-items-end">
    <button type="submit" class="btn btn-primary"<? app_tooltip_attr('Builds the full report with your selected filters and
options.') ?><i class="fa-solid fa-gears me-1"></i>Generate Report</button>
    <button type="submit" class="btn btn-outline-primary"<? app_tooltip_attr('Shows an on-screen preview before printing or
exporting.') ?><i class="fa-regular fa-eye me-1"></i>Preview</button>
    <button type="button" class="btn btn-outline-secondary" onclick="window.print()"<? app_tooltip_attr('Opens the print dialog
for the current report view.') ?><i class="fa-solid fa-print me-1"></i>Print</button>
    <button type="button" class="btn btn-outline-secondary" onclick="window.print()"<? app_tooltip_attr('Same as Print—use your
browser’s Save as PDF for a PDF file.') ?><i class="fa-regular fa-file-pdf me-1"></i>Export to PDF</button>
    <button type="submit" class="btn btn-outline-success" name="output" value="excel"<? app_tooltip_attr('Downloads the report
data as a spreadsheet file.') ?><i class="fa-regular fa-file-excel me-1"></i>Export to Excel</button>
</div>
</div>
</div>
</form>

<div class="row g-3 mb-4 no-print">
    <?php foreach ($reportCatalog as $group => $items): ?>
        <div class="col-xl-3 col-md-6">
            <div class="card shadow-sm h-100">
                <div class="card-header bg-white"><strong><? htmlspecialchars($group) ?></strong></div>
                <ul class="list-group list-group-flush">
                    <?php foreach ($items as $key => $label): ?>
                        <li class="list-group-item d-flex justify-content-between align-items-center">
                            <span><? htmlspecialchars($label) ?></span>
                            <a class="btn btn-sm btn-outline-primary" href="?report=<? urlencode($key) ?>&college_id=<? (int) $collegeId ?
>&faculty_id=<? (int) $facultyId ?>&day=<? urlencode($day) ?>"<? app_tooltip_attr('Loads this report type with your current college,
faculty, and day filters.') ?>Use</a>
                        </li>
                    <?php endforeach; ?>
                </ul>
            </div>
        </div>
    <?php endforeach; ?>
</div>

<div class="card shadow-sm">
    <div class="card-header bg-white d-flex justify-content-between align-items-center">
        <strong><? htmlspecialchars($reportPayload['title']) ?></strong>
        <span class="small text-muted"><? count($reportPayload['rows']) ?> row(s)</span>
    </div>
    <div class="table-responsive">
        <table class="table table-striped mb-0">
            <thead class="table-light">
                <tr>
                    <?php foreach ($reportPayload['columns'] as $col): ?>
                        <th><? htmlspecialchars($col) ?></th>
                    <?php endforeach; ?>
                </tr>
            </thead>
            <tbody>
                <?php if ($reportPayload['rows'] == []): ?>
                    <tr><td colspan="<? max(1, count($reportPayload['columns']) ?>" class="text-muted p-3">No data for this report/filter
selection.</td></tr>
                <?php else: ?>
                    <?php foreach ($reportPayload['rows'] as $row): ?>
                        <tr>
                            <?php foreach (array_values($row) as $value): ?>
                                <td><? htmlspecialchars((string) $value) ?></td>
                            <?php endforeach; ?>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>
</div>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: includes/account_registration_helpers.php

```
-----
<?php
declare(strict_types=1);

require_once __DIR__ . '/functions.php';
require_once __DIR__ . '/mail_helpers.php';

/**
 * Ensure users.must_change_password exists (lazy migration for installs that skipped upgrade_roles).
 */
function ensure_must_change_password_column(): void
{
    static $done = false;
    if ($done) {
        return;
    }
    $done = true;
    if (!db_column_exists('users', 'must_change_password')) {
        try {
            db()->exec('ALTER TABLE users ADD COLUMN must_change_password TINYINT(1) NOT NULL DEFAULT 0');
        } catch (Throwable $e) {
            // Column may have been added concurrently; ignore duplicate-column errors.
        }
    }
}

function mark_new_account_requires_password_change(int $userId): void
{
    if ($userId < 1) {
        return;
    }
    ensure_must_change_password_column();
    if (!db_column_exists('users', 'must_change_password')) {
        return;
    }
    db()->prepare('UPDATE users SET must_change_password = 1 WHERE id = ?')->execute([$userId]);
}

function user_must_change_password(int $userId): bool
{
    if ($userId < 1 || !db_column_exists('users', 'must_change_password')) {
        return false;
    }
    $st = db()->prepare('SELECT must_change_password FROM users WHERE id = ? LIMIT 1');
    $st->execute([$userId]);

    return (int) $st->fetchColumn() === 1;
}

function clear_must_change_password(int $userId): void
{
    if ($userId < 1 || !db_column_exists('users', 'must_change_password')) {
        return;
    }
    db()->prepare('UPDATE users SET must_change_password = 0 WHERE id = ?')->execute([$userId]);
}

/**
 * Resolve password for a new account. Auto-generates when a valid email is provided and no password was entered.
 *
 * @return array{plain: string, hash: string}
 */
function prepare_new_account_password(string $manualPassword, bool $hasValidEmail): array
{
    if ($hasValidEmail) {
        $plain = ($manualPassword !== '' && strlen($manualPassword) >= 8)
            ? $manualPassword
            : generate_temp_password();
    } elseif ($manualPassword === '' || strlen($manualPassword) < 8) {
        throw new RuntimeException(
            'Password is required and must be at least 8 characters when no email address is provided.'
        );
    } else {
        $plain = $manualPassword;
    }

    return [
        'plain' => $plain,
        'hash' => password_hash($plain, PASSWORD_DEFAULT),
    ];
}
```

```

    ];
}

function log_registration_email_failure(int $userId, string $username, string $role, string $reason): void
{
    error_log(sprintf(
        '[CLASS] Registration credentials email failed for user %d (%s, role=%s): %s',
        $userId,
        $username,
        $role,
        $reason
    ));
}

/**
 * Send exactly one credentials email after successful account creation. Never logs the plain password.
 *
 * @return bool|null true = sent, false = failed, null = no valid email (skipped)
 */
function notify_new_account_credentials(
    int $userId,
    string $toEmail,
    string $fullName,
    string $username,
    string $plainPassword,
    string $role
): ?bool {
    mark_new_account_requires_password_change($userId);

    $toEmail = trim($toEmail);
    if ($toEmail === '' || !filter_var($toEmail, FILTER_VALIDATE_EMAIL)) {
        return null;
    }

    try {
        $sent = send_account_credentials_mail(
            $toEmail,
            $fullName,
            $username,
            $plainPassword,
            $role,
            $toEmail
        );
        if (!$sent) {
            log_registration_email_failure($userId, $username, $role, 'mail transport returned false');
        }

        return $sent;
    } catch (Throwable $e) {
        log_registration_email_failure($userId, $username, $role, $e->getMessage());

        return false;
    }
}

/**
 * Flash message after new account registration (never includes the password).
 */
function registration_email_flash_message(?bool $mailSent, string $email = '', string $entityLabel = 'Account'): string
{
    if ($mailSent === null) {
        return $entityLabel . ' created successfully.';
    }
    if ($mailSent) {
        return $entityLabel . ' created successfully. Login credentials sent to ' . $email . '.';
    }

    return 'The account has been created successfully, but the email notification could not be sent.';
}

/**
 * Flash message after password reset / credential re-send (never includes the password).
 */
function credentials_email_flash_message(?bool $mailSent, string $email, string $contextUpdated): string
{
    if ($mailSent === null) {
        return $contextUpdated . '.';
    }
    if ($mailSent) {
        return $contextUpdated . '. Login credentials sent to ' . $email . '.';
    }
}

```



```

    }

    return $contextUpdated . ', but the email notification could not be sent.';
}

function auth_enforce_password_change_if_required(): void
{
    if (empty($_SESSION['must_change_password'])) {
        return;
    }
    $script = basename((string) ($_SERVER['SCRIPT_NAME'] ?? ''));
    $allowed = ['settings.php', 'logout.php'];
    if (!in_array($script, $allowed, true)) {
        header('Location: settings.php?force_password=1');
        exit;
    }
}
}

```

```

-----
FILE: includes/admin_activity_log.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/db.php';

/** Logs older than this many days are removed (FIFO by creation time). */
const ADMIN_ACTIVITY_LOG_RETENTION_DAYS = 7;

/** @var list<string> */
const USER_ACTIVITY_LOG_ACTIONS = ['add', 'edit', 'delete', 'login', 'logout'];

function admin_activity_log_table_ready(): bool
{
    static $ready = null;
    if ($ready !== null) {
        return $ready;
    }
    try {
        $st = db()->prepare(
            'SELECT COUNT(*) FROM information_schema.TABLES
             WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?'
        );
        $st->execute([DB_NAME, 'admin_activity_logs']);
        $ready = (int) $st->fetchColumn() > 0;
    } catch (Throwable $e) {
        $ready = false;
    }

    return $ready;
}

function admin_activity_log_column_exists(string $columnName): bool
{
    if (!admin_activity_log_table_ready()) {
        return false;
    }
    try {
        $st = db()->prepare(
            'SELECT COUNT(*) FROM information_schema.COLUMNS
             WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ? AND COLUMN_NAME = ?'
        );
        $st->execute([DB_NAME, 'admin_activity_logs', $columnName]);

        return (int) $st->fetchColumn() > 0;
    } catch (Throwable $e) {
        return false;
    }
}

/**
 * Apply lightweight migrations for older installs (new column, wider action_type).
 */
function admin_activity_log_ensure_schema(): void
{
    static $ran = false;
    if ($ran || !admin_activity_log_table_ready()) {
        return;
    }
}

```

```

$ran = true;
try {
    if (!admin_activity_log_column_exists('user_role')) {
        db()->exec(
            "ALTER TABLE admin_activity_logs
            ADD COLUMN user_role VARCHAR(32) NOT NULL DEFAULT 'admin' AFTER admin_username"
        );
    }
} catch (Throwable $e) {
    // ignore
}
try {
    if (!admin_activity_log_column_exists('actor_full_name')) {
        db()->exec(
            "ALTER TABLE admin_activity_logs
            ADD COLUMN actor_full_name VARCHAR(100) NOT NULL DEFAULT '' AFTER admin_username"
        );
    }
} catch (Throwable $e) {
    // ignore
}
try {
    if (!admin_activity_log_column_exists('actor_log_title')) {
        db()->exec(
            "ALTER TABLE admin_activity_logs
            ADD COLUMN actor_log_title VARCHAR(120) NOT NULL DEFAULT '' AFTER actor_full_name"
        );
    }
} catch (Throwable $e) {
    // ignore
}
try {
    $st = db()->prepare(
        "SELECT COLUMN_TYPE FROM information_schema.COLUMNS
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = 'admin_activity_logs' AND COLUMN_NAME = 'action_type'"
    );
    $st->execute([DB_NAME]);
    $colType = strtolower((string) ($st->fetchColumn() ?: ''));
    if ($colType !== '' && str_contains($colType, 'enum')) {
        db()->exec(
            "ALTER TABLE admin_activity_logs
            MODIFY COLUMN action_type VARCHAR(20) NOT NULL"
        );
    }
} catch (Throwable $e) {
    // ignore
}
}

/**
 * Remove log rows past the retention window.
 * Safe to call often; runs at most once per HTTP request.
 */
function admin_activity_log_cleanup(): void
{
    static $done = false;
    if ($done || !admin_activity_log_table_ready()) {
        return;
    }
    $done = true;
    admin_activity_log_ensure_schema();
    try {
        $st = db()->prepare(
            "DELETE FROM admin_activity_logs WHERE created_at < DATE_SUB(NOW(), INTERVAL ? DAY)"
        );
        $st->execute([ADMIN_ACTIVITY_LOG_RETENTION_DAYS]);
    } catch (Throwable $e) {
        // Best effort; logging must not break primary operations.
    }
}

/**
 * @param array<string,mixed>|null $row
 * @return array<string,mixed>|null
 */
function admin_activity_log_sanitizе_row(?array $row): ?array
{
    if ($row === null) {
        return null;
    }
}

```

```

$out = [];
foreach ($row as $k => $v) {
    $key = strtolower((string) $k);
    if (str_contains($key, 'password')) {
        $out[$k] = ($v === null || $v === '') ? $v : '[redacted]';
    } else {
        $out[$k] = $v;
    }
}

return $out;
}

/**
 * Record a data or session event for the signed-in user (any role). Admins review in Settings.
 *
 * @param 'add' | 'edit' | 'delete' | 'login' | 'logout' $actionType
 * @param array<string,mixed>|null $before
 * @param array<string,mixed>|null $after
 */
function log_user_activity(
    string $actionType,
    string $targetModule,
    string $targetReference,
    ?array $before,
    ?array $after,
    ?int $actorUserId = null,
    ?string $actorUsername = null,
    ?string $actorRole = null
): void {
    if (!in_array($actionType, USER_ACTIVITY_LOG_ACTIONS, true)) {
        $actionType = 'edit';
    }
    if (!admin_activity_log_table_ready()) {
        return;
    }
    admin_activity_log_ensure_schema();

    $uid = $actorUserId ?? (int) ($_SESSION['user_id'] ?? 0);
    if ($uid < 1) {
        return;
    }
    $username = $actorUsername !== null && $actorUsername !== ''
        ? trim($actorUsername)
        : trim((string) ($_SESSION['username'] ?? ''));
    if ($username === '') {
        $username = 'user#' . (string) $uid;
    }
    $role = $actorRole !== null && $actorRole !== ''
        ? trim($actorRole)
        : trim((string) ($_SESSION['role'] ?? ''));
    if ($role === '') {
        $role = 'unknown';
    }
    $actorFullName = trim((string) ($_SESSION['full_name'] ?? ''));
    $actorLogTitle = trim((string) ($_SESSION['admin_log_title'] ?? ''));

    admin_activity_log_cleanup();

    $payload = [
        'before' => admin_activity_log_sanitise_row($before),
        'after' => admin_activity_log_sanitise_row($after),
    ];
    $json = json_encode($payload, JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE);
    if ($json === false) {
        $json = '{"before":null,"after":null}';
    }

    try {
        $hasActorCols = admin_activity_log_column_exists('actor_full_name')
            && admin_activity_log_column_exists('actor_log_title');
        if (admin_activity_log_column_exists('user_role')) {
            if ($hasActorCols) {
                $st = db()->prepare(
                    'INSERT INTO admin_activity_logs (admin_user_id, admin_username, actor_full_name, actor_log_title, user_role,
action_type, target_module, target_reference, details_json, created_at)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, NOW())'
                );
                $st->execute([
                    $uid,

```

```

        $username,
        $actorFullName,
        $actorLogTitle,
        $role,
        $actionType,
        $targetModule,
        $targetReference,
        $json,
    ]));
} else {
    $st = db()->prepare(
        'INSERT INTO admin_activity_logs (admin_user_id, admin_username, user_role, action_type, target_module,
target_reference, details_json, created_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, NOW())'
    );
    $st->execute([$uid, $username, $role, $actionType, $targetModule, $targetReference, $json]);
}
} else {
    $st = db()->prepare(
        'INSERT INTO admin_activity_logs (admin_user_id, admin_username, action_type, target_module, target_reference, details_json,
created_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, NOW())'
    );
    $st->execute([$uid, $username, $actionType, $targetModule, $targetReference, $json]);
}
} catch (Throwable $e) {
    // Non-fatal
}
}

/**
 * @param array<string,mixed>|null $before
 * @param array<string,mixed>|null $after
 */
function log_admin_activity(
    string $actionType,
    string $targetModule,
    string $targetReference,
    ?array $before,
    ?array $after
): void {
    log_user_activity($actionType, $targetModule, $targetReference, $before, $after);
}

/**
 * @return list<array<string,mixed>>
 */
function admin_activity_log_list_sorted(string $order): array
{
    if (!admin_activity_log_table_ready()) {
        return [];
    }
    admin_activity_log_ensure_schema();
    $dir = $order === 'oldest' ? 'ASC' : 'DESC';
    $roleCol = admin_activity_log_column_exists('user_role');
    $actorCols = admin_activity_log_column_exists('actor_full_name')
        && admin_activity_log_column_exists('actor_log_title');
    try {
        if ($roleCol && $actorCols) {
            $sql = 'SELECT id, admin_user_id, admin_username, actor_full_name, actor_log_title, user_role, action_type, target_module,
target_reference, details_json, created_at
            FROM admin_activity_logs
            ORDER BY created_at ' . $dir . ', id ' . $dir . '
            LIMIT 500';
        } elseif ($roleCol) {
            $sql = 'SELECT id, admin_user_id, admin_username, "" AS actor_full_name, "" AS actor_log_title, user_role, action_type,
target_module, target_reference, details_json, created_at
            FROM admin_activity_logs
            ORDER BY created_at ' . $dir . ', id ' . $dir . '
            LIMIT 500';
        } else {
            $sql = 'SELECT id, admin_user_id, admin_username, "" AS actor_full_name, "" AS actor_log_title, "admin" AS user_role,
action_type, target_module, target_reference, details_json, created_at
            FROM admin_activity_logs
            ORDER BY created_at ' . $dir . ', id ' . $dir . '
            LIMIT 500';
        }
        $st = db()->query($sql);

        return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
    }
}

```

```

    } catch (Throwable $e) {
        return [];
    }
}

/**
 * Decode details_json for the activity log UI. Supports the standard
 * {"before":..., "after":...} envelope and legacy flat objects.
 *
 * @return array{0: mixed, 1: mixed, 2: bool} Tuple: before, after, invalid_json
 */
function admin_activity_log_decode_details_for_display(string $detailsJson): array
{
    $trim = trim($detailsJson);
    if ($trim === '') {
        return [null, null, false];
    }
    $decoded = json_decode($trim, true);
    if (!is_array($decoded)) {
        if (json_last_error() !== JSON_ERROR_NONE) {
            return [null, null, true];
        }
        // Valid JSON scalar or object decoded without associative array (unlikely with `true`)

        return [null, $decoded, false];
    }
    $hasWrapper = array_key_exists('before', $decoded) || array_key_exists('after', $decoded);
    if ($hasWrapper) {
        return [$decoded['before'] ?? null, $decoded['after'] ?? null, false];
    }

    return [null, $decoded, false];
}

function admin_activity_log_detail_block_nonempty(mixed $value): bool
{
    if ($value === null) {
        return false;
    }
    if (is_array($value) && $value === []) {
        return false;
    }

    return true;
}

/**
 * Pretty JSON for audit blocks (password fields are already redacted at write time).
 */
function admin_activity_log_format_json_pretty(mixed $value): string
{
    if ($value === null) {
        return 'null';
    }
    $flags = JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE;
    $out = json_encode($value, $flags);

    return $out !== false ? $out : '';
}

function admin_activity_log_explain_trunc(string $s, int $max = 160): string
{
    $s = trim($s);
    if ($s === '') {
        return '';
    }
    if (strlen($s) <= $max) {
        return $s;
    }

    return substr($s, 0, $max - 1) . '...';
}

/**
 * @param array<string,mixed>|null $row
 */
function admin_activity_log_explain_schedule_row(?array $row): ?string
{
    if ($row === null) {
        return null;
    }

```

```

}
$cc = trim((string) ($row['course_code'] ?? ''));
$fn = trim((string) ($row['faculty_name'] ?? ''));
$rc = trim((string) ($row['room_code'] ?? ''));
$days = trim(str_replace(',', ' ', (string) ($row['day_of_week'] ?? '')));
$st = substr((string) ($row['start_time'] ?? ''), 0, 5);
$en = substr((string) ($row['end_time'] ?? ''), 0, 5);
$sem = trim((string) ($row['semester'] ?? ''));
$sy = trim((string) ($row['school_year'] ?? ''));
$parts = [];
if ($cc !== '') {
    $parts[] = $cc;
}
if ($fn !== '') {
    $parts[] = 'with ' . $fn;
}
if ($rc !== '') {
    $parts[] = 'in ' . $rc;
}
if ($days !== '') {
    $parts[] = 'on ' . $days;
}
if ($st !== '' || $en !== '') {
    $parts[] = $st . '-' . $en;
}
if ($sem !== '' || $sy !== '') {
    $parts[] = trim($sem . ' ' . $sy);
}
$line = trim(implode(' ', array_filter($parts, static fn (string $p): bool => $p !== '')));

return $line !== '' ? $line : null;
}

```

```

function admin_activity_log_explain_scalar(mixed $v): string
{
    if ($v === null) {
        return '(none)';
    }
    if (is_bool($v)) {
        return $v ? 'Yes' : 'No';
    }
    if (is_int($v) || is_float($v)) {
        return (string) $v;
    }
    if (is_array($v)) {
        $j = json_encode($v, JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE);
        return admin_activity_log_explain_trunc($j !== false ? $j : '', 120);
    }

    return admin_activity_log_explain_trunc(trim((string) $v), 200);
}

```

```

function admin_activity_log_label_for_key(string $key): string
{
    static $map = [
        'course_code' => 'Course code',
        'course_name' => 'Course name',
        'room_code' => 'Room',
        'room_name' => 'Room name',
        'faculty_id' => 'Faculty ID',
        'full_name' => 'Full name',
        'username' => 'Username',
        'email' => 'Email',
        'department' => 'Department / program',
        'assigned_program' => 'Assigned program',
        'units' => 'Units',
        'lecture_units' => 'Lecture units',
        'laboratory_units' => 'Laboratory units',
        'is_laboratory' => 'Laboratory course',
        'year_level' => 'Year level',
        'section' => 'Section',
        'capacity' => 'Capacity',
        'type' => 'Type',
        'status' => 'Status',
        'college_code' => 'College code',
        'college_name' => 'College name',
        'day_of_week' => 'Days',
        'start_time' => 'Start time',
        'end_time' => 'End time',
    ];
}

```

```

        'semester' => 'Semester',
        'school_year' => 'School year',
        'schedule_type' => 'Schedule type',
        'max_hours_per_day' => 'Max hours per day',
        'is_active' => 'Active',
        'classroom_code' => 'Classroom code',
        'admin_remarks' => 'Admin remarks',
        'admin_log_title' => 'Log title / office',
        'dean_remarks' => 'Dean / chair remarks',
        'message' => 'Message',
    ];
    $k = strtolower($key);

    return $map[$k] ?? ucfirst(str_replace('_', ' ', $key));
}

/**
 * @param array<string,mixed>|null $before
 * @param array<string,mixed>|null $after
 * @return list<string>
 */
function admin_activity_log_collect_diff_lines(?array $before, ?array $after): array
{
    if ($before === null || $after === null) {
        return [];
    }
    /** @var array<string,bool> */
    $ignore = [
        'id' => true,
        'college_id' => true,
        'user_id' => true,
        'faculty_id' => true,
        'course_id' => true,
        'room_id' => true,
        'created_by' => true,
        'academic_year' => true,
    ];
    $keys = array_unique(array_merge(array_keys($before), array_keys($after)));
    sort($keys);
    $out = [];
    foreach ($keys as $k) {
        if (isset($ignore[$k])) {
            continue;
        }
        $vb = $before[$k] ?? null;
        $va = $after[$k] ?? null;
        $encB = json_encode($vb, JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE);
        $encA = json_encode($va, JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE);
        if ($encB === $encA) {
            continue;
        }
        $label = admin_activity_log_label_for_key((string) $k);
        if (str_contains(strtolower((string) $k), 'password')) {
            $out[] = $label . ' was updated.';
            continue;
        }
        $out[] = $label . ': ' . admin_activity_log_explain_scalar($vb) . ' → ' . admin_activity_log_explain_scalar($va);
    }

    return array_slice($out, 0, 20);
}

/**
 * @param array<string,mixed> $snap
 * @return list<string>
 */
function admin_activity_log_summarize_entity_snapshot(array $snap): array
{
    $try = [
        ['course_code', 'Course'],
        ['course_name', 'Title'],
        ['room_code', 'Room'],
        ['room_name', 'Room name'],
        ['full_name', 'Name'],
        ['faculty_id', 'Faculty ID'],
        ['username', 'Username'],
        ['email', 'Email'],
        ['department', 'Department'],
        ['college_code', 'College code'],
        ['college_name', 'College name'],
    ];

```

```

        ['assigned_program', 'Program'],
        ['admin_log_title', 'Log title'],
    ];
    $out = [];
    foreach ($try as [$k, $lab]) {
        if (!array_key_exists($k, $snap)) {
            continue;
        }
        $v = $snap[$k];
        if ($v === null || $v === '') {
            continue;
        }
        $out[] = $lab . ': ' . admin_activity_log_explain_scalar($v);
        if (count($out) >= 6) {
            break;
        }
    }

    return $out;
}

/**
 * Plain-language lines for the activity log Details column.
 *
 * @param array<string,mixed> $log One row from admin_activity_logs
 * @return list<string>
 */
function admin_activity_log_explain_human_lines(array $log, mixed $before, mixed $after): array
{
    $action = strtolower(trim((string) ($log['action_type'] ?? '')));
    $module = trim((string) ($log['target_module'] ?? ''));
    $ref = trim((string) ($log['target_reference'] ?? ''));
    /** @var array<string,mixed>|null $b */
    $b = is_array($before) ? $before : null;
    /** @var array<string,mixed>|null $a */
    $a = is_array($after) ? $after : null;

    if ($action === 'login' && $a !== null) {
        $lines = ['Signed in to the system.'];
        $role = trim((string) ($a['role'] ?? ''));
        $ip = trim((string) ($a['ip'] ?? ''));
        if ($role !== '') {
            $lines[] = 'Role for this session: ' . $role . ' .';
        }
        if ($ip !== '') {
            $lines[] = 'Client IP address: ' . $ip . ' .';
        }
        $office = trim((string) ($a['log_title'] ?? ''));
        if ($office !== '') {
            $lines[] = 'Log title / office on file: ' . $office . ' .';
        }

        return $lines;
    }

    if ($action === 'logout') {
        return ['Signed out and ended the session.'];
    }

    if ($module === 'Account' && $action === 'edit' && $a !== null && array_key_exists('password', $a)) {
        return ['Changed the password on their own account.'];
    }

    if ($a !== null && isset($a['rows']) && is_array($a['rows'])) {
        $ids = $a['created_schedule_ids'] ?? [];
        $n = is_array($ids) ? count($ids) : 0;
        $lines = [
            $n > 0
                ? 'Added ' . $n . ' new class schedule row' . ($n === 1 ? '' : 's') . ' to the timetable.'
                : 'Added new class schedule rows to the timetable.',
        ];
        $shown = 0;
        foreach ($a['rows'] as $row) {
            if (!is_array($row)) {
                continue;
            }
            $slug = admin_activity_log_explain_schedule_row($row);
            if ($slug !== null) {
                $lines[] = '• ' . $slug;
                if (++$shown >= 8) {

```



```

        break;
    }
}
}
$totalRows = count($a['rows']);
if ($shown > 0 && $totalRows > $shown) {
    $lines[] = '• ...and ' . ($totalRows - $shown) . ' more row(s).';
}

return $lines;
}

if ($module === 'Schedules (auto)' && $a !== null) {
    $cc = (int) ($a['created_count'] ?? 0);
    $fc = (int) ($a['failed_courses'] ?? 0);
    $lines = [
        'Ran automatic timetable generation.',
        'Created ' . $cc . ' schedule row(s). ' . $fc . ' course(s) were not fully scheduled.',
    ];
    if (!empty($a['preview']) && is_array($a['preview'])) {
        foreach (array_slice($a['preview'], 0, 5) as $p) {
            $lines[] = '• ' . (string) $p;
        }
        if (count($a['preview']) > 5) {
            $lines[] = '• (Additional tool messages omitted here.)';
        }
    }

    return $lines;
}

if ($module === 'Conflict requests' && $b !== null && $a !== null) {
    $st = strtolower(trim((string) ($a['status'] ?? '')));
    if ($st === 'approved') {
        $lines = ['Approved a faculty room/time override request and allowed a new schedule to be created.'];
    } elseif ($st === 'rejected') {
        $lines = ['Rejected a faculty room/time override request.'];
    } else {
        $lines = ['Updated a conflict override request.'];
    }
    $rm = trim((string) ($a['admin_remarks'] ?? ''));
    if ($rm !== '') {
        $lines[] = 'Reviewer notes: ' . admin_activity_log_explain_trunc($rm, 220);
    }

    return $lines;
}

if ($module === 'Schedule change requests') {
    $lines = ['Reviewed a faculty request to change an existing class schedule.'];
    if ($a !== null && ($a['status'] ?? '') !== '') {
        $lines[] = 'Decision: ' . strtolower(trim((string) $a['status'])) . '.';
    }
    if ($a !== null) {
        $dr = trim((string) ($a['dean_remarks'] ?? ''));
        if ($dr !== '') {
            $lines[] = 'Remarks to faculty: ' . admin_activity_log_explain_trunc($dr, 220);
        }
    }
    if ($b !== null) {
        $msg = trim((string) ($b['message'] ?? ''));
        if ($msg !== '') {
            $lines[] = 'Faculty request text: ' . admin_activity_log_explain_trunc($msg, 220);
        }
    }

    return $lines;
}

$isScheduleModule = $module === 'Schedules'
    || str_starts_with($module, 'Schedules ');

if ($isScheduleModule && $action === 'delete' && $b !== null) {
    $lines = ['Removed a class schedule from the timetable.'];
    $slug = admin_activity_log_explain_schedule_row($b);
    if ($slug !== null) {
        $lines[] = 'What was removed: ' . $slug;
    }

    return $lines;
}

```

```

}

if ($isScheduleModule && $action === 'edit' && $b !== null && $a !== null) {
    $lines = [str_contains($module, 'GEN ED') ? 'Updated a General Education class schedule.' : 'Updated a class schedule.'];
    $diff = admin_activity_log_collect_diff_lines($b, $a);
    if ($diff !== []) {
        $lines = array_merge($lines, $diff);
    } else {
        $slug = admin_activity_log_explain_schedule_row($a);
        if ($slug !== null) {
            $lines[] = 'Saved state: ' . $slug;
        }
    }
}

return $lines;
}

if ($isScheduleModule && $action === 'add' && $a !== null) {
    $lines = [str_contains($module, 'GEN ED') ? 'Created a new General Education class schedule.' : 'Created a new class schedule
row.'];
    $slug = admin_activity_log_explain_schedule_row($a);
    if ($slug !== null) {
        $lines[] = 'Details: ' . $slug;
    }
}

return $lines;
}

$entityLabel = match ($module) {
    'Courses' => 'course',
    'Rooms' => 'room',
    'Faculty' => 'faculty member',
    'Colleges' => 'college',
    'Deans' => 'dean account',
    'Program chairs' => 'program chair account',
    'GEN ED account' => 'GEN ED coordinator account',
    'Administrator accounts' => 'administrator account',
    default => '',
};

if ($entityLabel !== '') {
    if ($action === 'add' && $a !== null) {
        $lines = ['Added a new ' . $entityLabel . '.'];
        $lines = array_merge($lines, admin_activity_log_summarize_entity_snapshot($a));

        return $lines;
    }
    if ($action === 'delete' && $b !== null) {
        $lines = ['Removed a ' . $entityLabel . ' from the system.'];
        $lines = array_merge($lines, admin_activity_log_summarize_entity_snapshot($b));

        return $lines;
    }
    if ($action === 'edit' && ($b !== null || $a !== null)) {
        $lines = ['Updated a ' . $entityLabel . '.'];
        if ($b !== null && $a !== null) {
            $lines = array_merge($lines, admin_activity_log_collect_diff_lines($b, $a));
        } elseif ($a !== null) {
            $lines = array_merge($lines, admin_activity_log_summarize_entity_snapshot($a));
        }
    }

    return $lines;
}
}

$verb = match ($action) {
    'add' => 'Added',
    'edit' => 'Updated',
    'delete' => 'Removed',
    default => 'Recorded',
};

$modDisp = $module !== '' ? $module : 'the system';
$lines = [$verb . ' data in ' . $modDisp . ($ref !== '' ? ' - ' . $ref : '') . '.'];
if ($action === 'edit' && $b !== null && $a !== null) {
    $diff = admin_activity_log_collect_diff_lines($b, $a);
    if ($diff !== []) {
        $lines = array_merge($lines, $diff);
    }
} elseif ($action === 'add' && $a !== null) {
    $snap = admin_activity_log_summarize_entity_snapshot($a);

```

```

        if ($snap !== []) {
            $lines = array_merge($lines, $snap);
        }
    } elseif ($action === 'delete' && $b !== null) {
        $snap = admin_activity_log_summarize_entity_snapshot($b);
        if ($snap !== []) {
            $lines = array_merge($lines, $snap);
        }
    }
}

return $lines;
}

```

FILE: includes/admin_nav.php

```

<?php
declare(strict_types=1);

/**
 * Shared sidebar navigation: grouped sections with icons (admin + role dashboards).
 *
 * @param array<string, list<array{file:string,href:string,icon:string,label:string,badge?:int,tooltip?:string,target?:string}>> $sections
 */
function render_nav_sections_markup(array $sections, string $currentPage, bool $dismissOffcanvas): void
{
    foreach ($sections as $title => $items) {
        echo '<div class="admin-nav-section" role="group" aria-label="' . htmlspecialchars($title, ENT_QUOTES, 'UTF-8') . '">';
        echo '<div class="admin-nav-section-label">' . htmlspecialchars($title, ENT_QUOTES, 'UTF-8') . '</div>';
        echo '<ul class="admin-nav-list list-unstyled mb-0">';
        foreach ($items as $item) {
            $isActive = $currentPage === $item['file'];
            $badge = isset($item['badge']) ? (int) $item['badge'] : 0;
            $classes = 'admin-nav-link' . ($isActive ? ' active' : '');
            echo '<li>';
            echo '<a class="' . htmlspecialchars($classes, ENT_QUOTES, 'UTF-8') . '" href="' . htmlspecialchars($item['href'], ENT_QUOTES, 'UTF-8') . '"';
            if ($isActive) {
                echo ' aria-current="page"';
            }
            if ($dismissOffcanvas) {
                echo ' data-bs-dismiss="offcanvas"';
            }
            if (!empty($item['tooltip'])) {
                echo ' data-app-tooltip="' . htmlspecialchars((string) $item['tooltip'], ENT_QUOTES, 'UTF-8') . '"';
            }
            if (!empty($item['target'])) {
                echo ' target="' . htmlspecialchars((string) $item['target'], ENT_QUOTES, 'UTF-8') . '" rel="noopener noreferrer"';
            }
            echo '>';
            echo '<span class="admin-nav-icon" aria-hidden="true"><i class="fa-solid ' . htmlspecialchars($item['icon'], ENT_QUOTES, 'UTF-8') . '"></i></span>';
            echo '<span class="admin-nav-text">' . htmlspecialchars($item['label'], ENT_QUOTES, 'UTF-8') . '</span>';
            if ($badge > 0) {
                echo '<span class="badge text-bg-danger rounded-pill admin-nav-badge">' . ($badge > 99 ? '99+' : (string) $badge) .
            }
            echo '</span>';
        }
        echo '</a></li>';
    }
    echo '</ul></div>';
}

/**
 * External link to the WPU E-Faculty Workload system (separate app under htdocs).
 *
 * @return array{file:string,href:string,icon:string,label:string,tooltip:string,target:string}
 */
function nav_faculty_workload_external_item(): array
{
    return [
        'file' => 'faculty_workload_external',
        'href' => '/FACULTY%20WORKLOAD/public/login.php',
        'icon' => 'fa-briefcase',
        'label' => 'E-Faculty Workload',
        'tooltip' => 'Opens the WPU E-Faculty Workload system in a new tab to submit and track official workload forms.',
        'target' => '_blank',
    ];
}

```

```

/**
 * Short label under the app name in the sidebar (per role).
 */
function app_sidebar_brand_meta(string $role): string
{
    return match ($role) {
        'super_admin' => 'Super administrator',
        'admin' => 'System administration',
        'dean' => 'College dean',
        'program_chair' => 'Program chair',
        'gened' => 'General education',
        'faculty' => 'Faculty',
        'student' => 'Student',
        default => 'Signed in',
    };
}

/**
 * Font Awesome icon class for the top bar avatar (no "fa-solid" prefix).
 */
function app_sidebar_topbar_icon_class(string $role): string
{
    return match ($role) {
        'super_admin' => 'fa-user-secret',
        'admin' => 'fa-user-shield',
        'dean' => 'fa-user-tie',
        'program_chair' => 'fa-clipboard-list',
        'gened' => 'fa-graduation-cap',
        'faculty' => 'fa-chalkboard-user',
        'student' => 'fa-user-graduate',
        default => 'fa-circle-user',
    };
}

/**
 * Top bar user avatar: profile photo when set, otherwise role icon.
 */
function app_render_topbar_avatar(string $role, ?string $profilePhotoUrl): void
{
    if ($profilePhotoUrl !== null && $profilePhotoUrl !== '') {
        echo '<span class="admin-topbar-avatar rounded-circle d-inline-flex align-items-center justify-content-center overflow-hidden">';
        echo '';
        echo '</span>';
        return;
    }

    $topbarIcon = app_sidebar_topbar_icon_class($role);
    echo '<span class="admin-topbar-avatar rounded-circle d-inline-flex align-items-center justify-content-center">';
    echo '<i class="fa-solid ' . htmlspecialchars($topbarIcon, ENT_QUOTES, 'UTF-8') . '" admin-topbar-avatar-icon aria-hidden="true"></i>';
    echo '</span>';
}

/**
 * Renders grouped admin navigation (sidebar / mobile drawer).
 *
 * @param string $currentPage basename of current script, e.g. dashboard.php
 * @param int $messagingUnread unread message count for badge
 * @param bool $dismissOffcanvas when true, links close the mobile drawer (Bootstrap offcanvas)
 */
/**
 * Super Admin: account oversight, reports, and institution-wide weekly schedule view.
 *
 * @param string $currentPage basename of current script
 * @param int $messagingUnread unread message count (unused; kept for signature parity)
 * @param bool $dismissOffcanvas when true, links close the mobile drawer
 */
function render_super_admin_nav_sections(string $currentPage, int $messagingUnread, bool $dismissOffcanvas = false): void
{
    unset($messagingUnread);
    $sections = [
        'Overview' => [
            [
                'file' => 'dashboard.php',
                'href' => 'dashboard.php',
                'icon' => 'fa-gauge-high',
                'label' => 'Dashboard',
                'tooltip' => 'Returns to the Super Admin home with shortcuts to administrator account management.',
            ],

```

```

[
    'file' => 'view_schedule.php',
    'href' => 'view_schedule.php',
    'icon' => 'fa-calendar-week',
    'label' => 'Weekly schedule',
    'tooltip' => 'Institution-wide week-at-a-glance schedule. Filter by college, program, term, faculty, or room.',
],
],
'Accounts' => [
    [
        'file' => 'super_admin_accounts.php',
        'href' => 'super_admin_accounts.php',
        'icon' => 'fa-users-gear',
        'label' => 'Account monitor',
        'tooltip' => 'Monitor all logins, reset passwords, and create accounts for any role (including Super Administrators).',
    ],
    [
        'file' => 'super_admin_admins.php',
        'href' => 'super_admin_admins.php',
        'icon' => 'fa-user-shield',
        'label' => 'Administrator accounts',
        'tooltip' => 'Create or update System Administrator accounts (scheduling admins). Each account can show a title in the
activity log.',
    ],
],
'Academic' => [
    [
        'file' => 'super_admin_programs.php',
        'href' => 'super_admin_programs.php',
        'icon' => 'fa-graduation-cap',
        'label' => 'Program offers',
        'tooltip' => 'View, edit, or delete all program offers across every college in the university.',
    ],
],
],
'Reports' => [
    [
        'file' => 'super_admin_faculty_inventory.php',
        'href' => 'super_admin_faculty_inventory.php',
        'icon' => 'fa-address-book',
        'label' => 'Faculty inventory',
        'tooltip' => 'All faculty records across colleges, including GE-tagged faculty. Filter by college, scope, or name.',
    ],
    [
        'file' => 'super_admin_teaching_load_report.php',
        'href' => 'super_admin_teaching_load_report.php',
        'icon' => 'fa-scale-balanced',
        'label' => 'Teaching load (under / over)',
        'tooltip' => 'All programs: weekly contact hours from schedules. Under load is below 18 hrs/week; overload is above 27.
Filter by college and term.',
    ],
],
    nav_faculty_workload_external_item(),
],
];
render_nav_sections_markup($sections, $currentPage, $dismissOffcanvas);
}

function render_admin_nav_sections(string $currentPage, int $messagingUnread, bool $dismissOffcanvas = false): void
{
    $sections = [
        'Overview' => [
            [
                'file' => 'dashboard.php', 'href' => 'dashboard.php', 'icon' => 'fa-gauge-high', 'label' => 'Dashboard',
                'tooltip' => 'Opens your administrator home with system-wide counts and shortcuts. Use this to return to the overview from
any page.',
            ],
            [
                'file' => 'messages.php', 'href' => 'messages.php', 'icon' => 'fa-comments', 'label' => 'Messages', 'badge' =>
$messagingUnread,
                'tooltip' => 'Opens internal messages and class threads. Use this to coordinate with deans, faculty, and staff across
colleges.',
            ],
            [
                'file' => 'schedule.php', 'href' => 'schedule.php', 'icon' => 'fa-list', 'label' => 'Schedules',
                'tooltip' => 'Opens the master schedule list. Use this to browse or drill into course times, rooms, and assignments.',
            ],
            [
                'file' => 'view_schedule.php', 'href' => 'view_schedule.php', 'icon' => 'fa-calendar-week', 'label' => 'Weekly view',
                'tooltip' => 'Shows schedules in a week-at-a-glance layout. Use this to spot patterns, gaps, or overlaps quickly.',
            ],
        ],
    ],
];

```

```

'Academic' => [
    [
        'file' => 'admin_colleges.php', 'href' => 'admin_colleges.php', 'icon' => 'fa-building-columns', 'label' => 'Colleges',
        'tooltip' => 'Manage colleges, codes, and dean links. Use this when setting up structure or fixing college metadata.',
    ],
    [
        'file' => 'admin_deans.php', 'href' => 'admin_deans.php', 'icon' => 'fa-user-tie', 'label' => 'Deans',
        'tooltip' => 'Create or edit dean accounts tied to colleges. Use this to onboard leadership or reset access.',
    ],
    [
        'file' => 'admin_gened.php', 'href' => 'admin_gened.php', 'icon' => 'fa-graduation-cap', 'label' => 'GEN ED',
        'tooltip' => 'Configure General Education coordinators and scope. Use this for cross-college GE administration.',
    ],
    [
        'file' => 'admin_login_slideshow.php', 'href' => 'admin_login_slideshow.php', 'icon' => 'fa-images', 'label' => 'Login
slideshow',
        'tooltip' => 'Upload and manage images that auto-scroll on the public login page.',
    ],
    [
        'file' => 'faculty_edutools.php', 'href' => 'faculty_edutools.php', 'icon' => 'fa-wand-magic-sparkles', 'label' =>
'EduTools',
        'tooltip' => 'Education sources: writing tools (Turnitin, Grammarly, QuillBot, AI detector), AI & research (ChatGPT,
Perplexity, NotebookLM, Wolfram), notebooks, Khan Academy, and Notion—plus a built-in notebook.',
    ],
],
'Management' => [
    [
        'file' => 'admin_requests.php', 'href' => 'admin_requests.php', 'icon' => 'fa-inbox', 'label' => 'Override requests',
        'tooltip' => 'Review scheduling override requests from faculty or chairs. Use this to approve or deny exceptions.',
    ],
    [
        'file' => 'conflicts.php', 'href' => 'conflicts.php', 'icon' => 'fa-triangle-exclamation', 'label' => 'Conflicts',
        'tooltip' => 'Lists scheduling conflicts and coordination items. Use this to resolve issues before publishing schedules.',
    ],
    [
        'file' => 'reports.php', 'href' => 'reports.php', 'icon' => 'fa-file-lines', 'label' => 'Reports',
        'tooltip' => 'Opens standard scheduling and workload reports. Use this for accreditation, planning, or audits.',
    ],
    [
        'file' => 'faculty_teaching_load.php', 'href' => 'faculty_teaching_load.php', 'icon' => 'fa-scale-balanced', 'label' =>
'Teaching load (units)',
        'tooltip' => 'Faculty teaching load in units from scheduled offerings. Filter by college, term, or instructor.',
    ],
    nav_faculty_workload_external_item(),
    [
        'file' => 'global_reports.php', 'href' => 'global_reports.php', 'icon' => 'fa-chart-column', 'label' => 'Global reports',
        'tooltip' => 'Cross-college analytics and exports. Use this for institution-wide summaries beyond a single college.',
    ],
],
];
render_nav_sections_markup($sections, $currentPage, $dismissOffcanvas);
}

/**
 * Grouped nav items for dean, program chair, GE, faculty, and student roles.
 */
* @return array<string, list<array{file:string,href:string,icon:string,label:string,badge?:int}>>
*/
function role_nav_sections(string $role, int $messagingUnread): array
{
    $dash = [
        'file' => 'dashboard.php', 'href' => 'dashboard.php', 'icon' => 'fa-gauge-high', 'label' => 'Dashboard',
        'tooltip' => 'Opens your home dashboard with summary counts and shortcuts. Use this to return to the overview from any page.',
    ];
    $msg = [
        'file' => 'messages.php', 'href' => 'messages.php', 'icon' => 'fa-comments', 'label' => 'Messages', 'badge' => $messagingUnread,
        'tooltip' => 'Opens internal messages and class conversation threads. Use this to coordinate with colleagues and students.',
    ];
    $sched = [
        'file' => 'schedule.php', 'href' => 'schedule.php', 'icon' => 'fa-list', 'label' => 'Schedules',
        'tooltip' => 'Opens the schedule list for your scope. Use this to review or edit when you manage or teach courses.',
    ];
    $week = [
        'file' => 'view_schedule.php', 'href' => 'view_schedule.php', 'icon' => 'fa-calendar-week', 'label' => 'Weekly view',
        'tooltip' => 'Shows the schedule in a weekly layout. Use this to see how classes spread across days.',
    ];
    $facultyWorkload = nav_faculty_workload_external_item();

    return match ($role) {
        'dean' => [

```

```

'Overview' => [$dash, $msg, $sched, $week],
'Academic' => [
  [
    'file' => 'faculty.php', 'href' => 'faculty.php', 'icon' => 'fa-chalkboard-user', 'label' => 'Faculty',
    'tooltip' => 'Manage faculty records for your college. Use this to add people, link accounts, or fix assignments.',
  ],
  [
    'file' => 'program_chairs.php', 'href' => 'program_chairs.php', 'icon' => 'fa-user-tie', 'label' => 'Program chairs',
    'tooltip' => 'Assign or edit program chairs for programs in your college. Use this to delegate scheduling within
departments.',
  ],
  [
    'file' => 'dean_programs.php', 'href' => 'dean_programs.php', 'icon' => 'fa-graduation-cap', 'label' => 'Programs',
    'tooltip' => 'Maintain program names and status under your college. Use this when curricula or program codes change.',
  ],
  [
    'file' => 'dean_student_registrations.php',
    'href' => 'dean_student_registrations.php',
    'icon' => 'fa-user-check',
    'label' => 'Registration approvals',
    'tooltip' => 'Review and approve student self-registration requests for any program in your college.',
  ],
  [
    'file' => 'courses.php', 'href' => 'courses.php', 'icon' => 'fa-book', 'label' => 'Courses',
    'tooltip' => 'Create and edit courses your college offers. Use this before building schedules each term.',
  ],
  [
    'file' => 'rooms.php', 'href' => 'rooms.php', 'icon' => 'fa-door-open', 'label' => 'Rooms',
    'tooltip' => 'Manage physical or virtual rooms for scheduling. Use this so auto-schedule and manual edits use correct
capacity.',
  ],
  [
    'file' => 'faculty_classrooms.php',
    'href' => 'faculty_classrooms.php',
    'icon' => 'fa-chalkboard',
    'label' => 'My classrooms',
    'tooltip' => 'Create and manage online classrooms for your own teaching load.',
  ],
  [
    'file' => 'faculty_offline.php',
    'href' => 'faculty_offline.php',
    'icon' => 'fa-cloud-arrow-up',
    'label' => 'Offline uploads',
    'tooltip' => 'Queue course posts and file uploads while offline, then sync them automatically when you reconnect.',
  ],
  [
    'file' => 'faculty_student_list.php',
    'href' => 'faculty_student_list.php',
    'icon' => 'fa-users',
    'label' => 'Student list',
    'tooltip' => 'Lists your courses and registered students with present and absent attendance totals.',
  ],
  [
    'file' => 'faculty_schedule.php', 'href' => 'faculty_schedule.php', 'icon' => 'fa-calendar-check', 'label' => 'My
schedule',
    'tooltip' => 'Shows your teaching timetable and online links. Use this to confirm times, rooms, and live session
links.',
  ],
],
'Scheduling' => [
  [
    'file' => 'add_schedule.php', 'href' => 'add_schedule.php', 'icon' => 'fa-plus', 'label' => 'Add schedule',
    'tooltip' => 'Create a new scheduled section manually. Use this for one-off classes or fine-tuning after auto-
schedule.',
  ],
  [
    'file' => 'auto_schedule.php', 'href' => 'auto_schedule.php', 'icon' => 'fa-wand-magic-sparkles', 'label' => 'Auto
schedule',
    'tooltip' => 'Runs automated placement of courses into rooms and times. Use this to generate a draft or refresh a
term.',
  ],
],
'Management' => [
  [
    'file' => 'reports.php', 'href' => 'reports.php', 'icon' => 'fa-file-lines', 'label' => 'Reports',
    'tooltip' => 'College-level scheduling and workload reports. Use this for reviews, accreditation, or planning.',
  ],
  [
    'file' => 'faculty_teaching_load.php', 'href' => 'faculty_teaching_load.php', 'icon' => 'fa-scale-balanced', 'label' =>
'Teaching load (units)',
  ],
]

```

```

        'tooltip' => 'Units per faculty from schedules in your college. Filter by department and term.',
    ],
    $facultyWorkload,
    [
        'file' => 'conflicts.php', 'href' => 'conflicts.php', 'icon' => 'fa-triangle-exclamation', 'label' => 'Conflicts',
        'tooltip' => 'Review room, faculty, or change-request conflicts. Use this to clear issues before finalizing schedules.',
    ],
    ],
    ],
    'program_chair' => [
        'Overview' => [$dash, $msg, $sched, $week],
        'Academic' => [
            ['file' => 'faculty.php', 'href' => 'faculty.php', 'icon' => 'fa-chalkboard-user', 'label' => 'Faculty'],
            ['file' => 'program_chair_students.php', 'href' => 'program_chair_students.php', 'icon' => 'fa-user-graduate', 'label' =>
'Students'],
            [
                'file' => 'program_chair_student_registrations.php',
                'href' => 'program_chair_student_registrations.php',
                'icon' => 'fa-user-check',
                'label' => 'Registration approvals',
                'tooltip' => 'Review and approve student self-registration requests for your program.',
            ],
            ['file' => 'courses.php', 'href' => 'courses.php', 'icon' => 'fa-book', 'label' => 'Courses'],
            [
                'file' => 'faculty_classrooms.php',
                'href' => 'faculty_classrooms.php',
                'icon' => 'fa-chalkboard',
                'label' => 'My classrooms',
                'tooltip' => 'Create and manage online classrooms for your own teaching load.',
            ],
            [
                'file' => 'faculty_offline.php',
                'href' => 'faculty_offline.php',
                'icon' => 'fa-cloud-arrow-up',
                'label' => 'Offline uploads',
                'tooltip' => 'Queue course posts and file uploads while offline, then sync them automatically when you reconnect.',
            ],
            [
                'file' => 'faculty_student_list.php',
                'href' => 'faculty_student_list.php',
                'icon' => 'fa-users',
                'label' => 'Student list',
                'tooltip' => 'Lists your courses and registered students with present and absent attendance totals.',
            ],
            [
                'file' => 'faculty_schedule.php', 'href' => 'faculty_schedule.php', 'icon' => 'fa-calendar-check', 'label' => 'My
schedule',
                'tooltip' => 'Shows your teaching timetable and online links. Use this to confirm times, rooms, and live session
links.',
            ],
            [
                'file' => 'faculty_edutools.php', 'href' => 'faculty_edutools.php', 'icon' => 'fa-wand-magic-sparkles', 'label' =>
'EduTools',
                'tooltip' => 'Education sources: writing tools (Turnitin, Grammarly, QuillBot, AI detector), AI & research (ChatGPT,
Perplexity, NotebookLM, Wolfram), notebooks, Khan Academy, and Notion—plus a built-in notebook.',
            ],
        ],
        'Scheduling' => [
            ['file' => 'add_schedule.php', 'href' => 'add_schedule.php', 'icon' => 'fa-plus', 'label' => 'Add schedule'],
        ],
        'Management' => [
            ['file' => 'reports.php', 'href' => 'reports.php', 'icon' => 'fa-file-lines', 'label' => 'Reports'],
            [
                'file' => 'faculty_teaching_load.php', 'href' => 'faculty_teaching_load.php', 'icon' => 'fa-scale-balanced', 'label' =>
'Teaching load (units)',
                'tooltip' => 'Units per faculty for your program scheduled offerings. Filter by term or search faculty.',
            ],
        ],
        $facultyWorkload,
        ['file' => 'conflicts.php', 'href' => 'conflicts.php', 'icon' => 'fa-triangle-exclamation', 'label' => 'Conflicts'],
    ],
    ],
    'gened' => [
        'Overview' => [$dash, $msg, $sched, $week],
        'Academic' => [
            [
                'file' => 'gened_schedule.php', 'href' => 'gened_schedule.php', 'icon' => 'fa-calendar-plus', 'label' => 'GE schedule',
                'tooltip' => 'Build and edit General Education schedule rows. Use this to place GE sections in time and rooms.',
            ],
            [
                'file' => 'gened_faculty.php', 'href' => 'gened_faculty.php', 'icon' => 'fa-chalkboard-user', 'label' => 'GE faculty',
            ],
        ],
    ],

```



```

        'tooltip' => 'Manage which instructors teach GE and their GE flags. Use this before assigning GE courses.',
    ],
    [
        'file' => 'gened_courses.php', 'href' => 'gened_courses.php', 'icon' => 'fa-book', 'label' => 'GE courses',
        'tooltip' => 'Maintain the GE course catalog. Use this when codes, titles, or GE status change.',
    ],
    [
        'file' => 'gened_rooms.php', 'href' => 'gened_rooms.php', 'icon' => 'fa-door-open', 'label' => 'GE rooms',
        'tooltip' => 'Rooms available for GE scheduling. Use this so GE placements respect capacity and type.',
    ],
    [
        'file' => 'gened_assignments.php', 'href' => 'gened_assignments.php', 'icon' => 'fa-building-user', 'label' => 'GE
offerings',
        'tooltip' => 'Map GE sections to colleges or cohorts. Use this to coordinate cross-college GE delivery.',
    ],
    [
        'file' => 'faculty_classrooms.php',
        'href' => 'faculty_classrooms.php',
        'icon' => 'fa-chalkboard',
        'label' => 'My classrooms',
        'tooltip' => 'Create and manage online classrooms for your own teaching load.',
    ],
    [
        'file' => 'faculty_offline.php',
        'href' => 'faculty_offline.php',
        'icon' => 'fa-cloud-arrow-up',
        'label' => 'Offline uploads',
        'tooltip' => 'Queue course posts and file uploads while offline, then sync them automatically when you reconnect.',
    ],
    [
        'file' => 'faculty_student_list.php',
        'href' => 'faculty_student_list.php',
        'icon' => 'fa-users',
        'label' => 'Student list',
        'tooltip' => 'Lists your courses and registered students with present and absent attendance totals.',
    ],
    [
        'file' => 'faculty_edutools.php', 'href' => 'faculty_edutools.php', 'icon' => 'fa-wand-magic-sparkles', 'label' =>
'EduTools',
        'tooltip' => 'Education sources: writing tools (Turnitin, Grammarly, QuillBot, AI detector), AI & research (ChatGPT,
Perplexity, NotebookLM, Wolfram), notebooks, Khan Academy, and Notion—plus a built-in notebook.',
    ],
    ],
    'Management' => [
        [
            'file' => 'reports.php', 'href' => 'reports.php', 'icon' => 'fa-file-lines', 'label' => 'Reports',
            'tooltip' => 'GE-focused scheduling reports. Use this to audit load, utilization, or compliance.',
        ],
        [
            'file' => 'faculty_teaching_load.php', 'href' => 'faculty_teaching_load.php', 'icon' => 'fa-scale-balanced', 'label' =>
'Teaching load (units)',
            'tooltip' => 'GE faculty load in units from scheduled GE offerings. Filter by term or department.',
        ],
        $facultyWorkload,
        [
            'file' => 'conflicts.php', 'href' => 'conflicts.php', 'icon' => 'fa-triangle-exclamation', 'label' => 'Conflicts',
            'tooltip' => 'Review conflicts affecting GE sections. Use this to resolve overlaps before publishing.',
        ],
    ],
    ],
    ],
    'faculty' => [
        'Overview' => [$dash, $msg, $sched, $week],
        'Academic' => [
            [
                'file' => 'faculty_classrooms.php', 'href' => 'faculty_classrooms.php', 'icon' => 'fa-chalkboard', 'label' => 'My
classrooms',
                'tooltip' => 'Opens your online class spaces for assigned courses. Use this to post content, run Meet, and track
students.',
            ],
            [
                'file' => 'faculty_offline.php',
                'href' => 'faculty_offline.php',
                'icon' => 'fa-cloud-arrow-up',
                'label' => 'Offline uploads',
                'tooltip' => 'Queue course posts and file uploads while offline, then sync them automatically when you reconnect.',
            ],
            [
                'file' => 'faculty_student_list.php', 'href' => 'faculty_student_list.php', 'icon' => 'fa-users', 'label' => 'Student
list',
                'tooltip' => 'Lists your courses and registered students with present and absent attendance totals.',
            ],
        ],
    ],

```

```

    ],
    [
        'file' => 'faculty_schedule.php', 'href' => 'faculty_schedule.php', 'icon' => 'fa-calendar-check', 'label' => 'My
schedule',
        'tooltip' => 'Shows your teaching timetable and online links. Use this to confirm times, rooms, and live session
links.',
    ],
    [
        'file' => 'faculty_edutools.php', 'href' => 'faculty_edutools.php', 'icon' => 'fa-wand-magic-sparkles', 'label' =>
'EduTools',
        'tooltip' => 'Education sources: writing tools (Turnitin, Grammarly, QuillBot, AI detector), AI & research (ChatGPT,
Perplexity, NotebookLM, Wolfram), notebooks, Khan Academy, and Notion—plus a built-in notebook.',
    ],
],
'Management' => [
    [
        'file' => 'reports.php', 'href' => 'reports.php', 'icon' => 'fa-file-lines', 'label' => 'Reports',
        'tooltip' => 'Reports available to faculty (e.g. workload). Use this when you need a printable or official summary.',
    ],
    [
        'file' => 'faculty_teaching_load.php', 'href' => 'faculty_teaching_load.php', 'icon' => 'fa-scale-balanced', 'label' =>
'Teaching load (units)',
        'tooltip' => 'Your personal teaching load in units from your scheduled offerings. Filter by term.',
    ],
    $facultyWorkload,
    [
        'file' => 'conflicts.php', 'href' => 'conflicts.php', 'icon' => 'fa-triangle-exclamation', 'label' => 'Conflicts',
        'tooltip' => 'Items that need coordination with the office. Use this to see if your classes have pending issues.',
    ],
],
],
'student' => [
    'Overview' => [
        array_merge($dash, [
            'tooltip' => 'Opens your home dashboard with summary counts and shortcuts. Use this to return to the overview after
visiting Messages or My Classes.',
        ]),
        array_merge($msg, [
            'tooltip' => 'Opens class conversations for your enrolled subjects. Use this to read and send messages shared with your
instructor and classmates.',
        ]),
    ],
    'Academic' => [
        [
            'file' => 'student_classrooms.php',
            'href' => 'student_classrooms.php',
            'icon' => 'fa-user-graduate',
            'label' => 'My classes',
            'tooltip' => 'Lists every online class you are enrolled in. Use this to open a class, join Meet, or enter a join code
from your instructor.',
        ],
        [
            'file' => 'student_offline.php',
            'href' => 'student_offline.php',
            'icon' => 'fa-cloud-arrow-down',
            'label' => 'Offline reading',
            'tooltip' => 'Read saved faculty announcements and course materials without internet. Save while online first, then open
this page offline.',
        ],
        [
            'file' => 'student_edutools.php',
            'href' => 'student_edutools.php',
            'icon' => 'fa-wand-magic-sparkles',
            'label' => 'Learning tools',
            'tooltip' => 'Opens EduTools: built-in notebook, NotebookLM, ChatGPT, Perplexity, Khan Academy, Wolfram Alpha, and
Notion for studying and research.',
        ],
        [
            'file' => 'student_materials_reviewer.php',
            'href' => 'student_materials_reviewer.php',
            'icon' => 'fa-clipboard-question',
            'label' => 'Materials reviewer',
            'tooltip' => 'Review course materials week by week and generate automatic practice questions from what your instructor
posted for each week.',
        ],
        [
            'file' => 'student_calculator.php',
            'href' => 'student_calculator.php',
            'icon' => 'fa-calculator',
            'label' => 'Scientific calculator',
        ]
    ]
]

```

```

        'tooltip' => 'A full-featured scientific calculator with trigonometry, logarithms, powers, roots, and more—right inside
the portal.',
    ],
    [
        'file' => 'student_wellness.php',
        'href' => 'student_wellness.php',
        'icon' => 'fa-heart-pulse',
        'label' => 'Wellness companion',
        'tooltip' => 'Private mental health support chat (stress, anxiety, loneliness). English/Tagalog. Not therapy or
emergency care—crisis hotlines shown when needed.',
    ],
],
],
default => [],
});
}

```

```

/**
 * Renders role-based grouped navigation (non-admin users).
 */
function render_role_nav_sections(string $role, string $currentPage, int $messagingUnread, bool $dismissOffcanvas = false): void
{
    $sections = role_nav_sections($role, $messagingUnread);
    if ($sections === []) {
        return;
    }
    render_nav_sections_markup($sections, $currentPage, $dismissOffcanvas);
}

```

FILE: includes/admin_offcanvas.php

```

<?php
declare(strict_types=1);

/**
 * Mobile navigation drawer (pure CSS toggle, no JavaScript required).
 * The hidden checkbox #adminNavToggle is flipped by <label> controls
 * (the topbar hamburger, the backdrop, and the close button), and CSS
 * sibling selectors slide the panel in/out. Rendered at body level.
 */
?>
<input type="checkbox" id="adminNavToggle" class="admin-nav-toggle-input no-print" tabindex="-1" aria-hidden="true">
<label for="adminNavToggle" class="admin-offcanvas-backdrop no-print" aria-label="Close navigation menu"></label>
<div class="admin-offcanvas-nav no-print" id="adminNavOffcanvas" aria-label="Mobile navigation">
    <div class="offcanvas-header border-bottom d-flex align-items-center justify-content-between">
        <div>
            <div class="fw-semibold" id="adminNavOffcanvasLabel">Menu</div>
            <div class="small text-muted text-uppercase"><?= htmlspecialchars($navRole) ?></div>
        </div>
        <label for="adminNavToggle" class="btn-close" role="button" tabindex="0" aria-label="Close menu"><?= $appCursorTooltips ?
app_tooltip_attr('Closes the slide-out navigation menu. Use this after you choose a page or if you opened the menu by mistake.') : '' ?>>
</label>
    </div>
    <div class="offcanvas-body p-0 d-flex flex-column">
        <nav class="admin-sidebar-nav flex-grow-1 px-2 pb-3" aria-label="Primary sections">
            <?php if ($navRole === 'admin'): ?>
                <?php render_admin_nav_sections($adminNavPage, $messagingNavUnread, true); ?>
            <?php elseif ($navRole === 'super_admin'): ?>
                <?php render_super_admin_nav_sections($adminNavPage, $messagingNavUnread, true); ?>
            <?php else: ?>
                <?php render_role_nav_sections($navRole, $adminNavPage, $messagingNavUnread, true); ?>
            <?php endif; ?>
        </nav>
        <div class="admin-sidebar-footer border-top px-3 py-3 mt-auto">
            <a class="admin-sidebar-foot-link d-block py-1" href="settings.php"><?= $appCursorTooltips ? app_tooltip_attr('Opens account
settings where you can change your password.') : '' ?>><i class="fa-solid fa-gear me-2" aria-hidden="true"></i>Settings</a>
            <a class="admin-sidebar-foot-link admin-sidebar-logout d-block py-1" href="logout.php"><?= $appCursorTooltips ?
app_tooltip_attr('Signs you out.') : '' ?>><i class="fa-solid fa-right-from-bracket me-2" aria-hidden="true"></i>Logout</a>
        </div>
    </div>
</div>

```

FILE: includes/assessment_retake_helpers.php

```

<?php
declare(strict_types=1);

```

```

require_once __DIR__ . '/functions.php';

function assessment_retake_table_ready(): bool
{
    return db_table_exists('assessment_retake_requests');
}

function assessment_retake_ensure_table(): void
{
    if (assessment_retake_table_ready()) {
        return;
    }
    if (!db_table_exists('online_classrooms') || !db_table_exists('classroom_assessments') || !db_table_exists('classroom_students')) {
        return;
    }
    try {
        db()->exec(
            "CREATE TABLE IF NOT EXISTS assessment_retake_requests (
                id INT AUTO_INCREMENT PRIMARY KEY,
                classroom_id INT NOT NULL,
                assessment_id INT NOT NULL,
                student_id INT NOT NULL,
                submission_id INT NULL,
                reason TEXT NOT NULL,
                status ENUM('pending','approved','rejected') NOT NULL DEFAULT 'pending',
                reviewed_by_user_id INT NULL,
                reviewed_at DATETIME NULL,
                faculty_remarks VARCHAR(500) NOT NULL DEFAULT '',
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                INDEX idx_arr_pending_classroom (status, classroom_id),
                INDEX idx_arr_student_assessment (student_id, assessment_id),
                CONSTRAINT fk_arr_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
                CONSTRAINT fk_arr_assessment FOREIGN KEY (assessment_id) REFERENCES classroom_assessments(id) ON DELETE CASCADE,
                CONSTRAINT fk_arr_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
            ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
        );
    } catch (Throwable $e) {
        // Table may already exist under a concurrent request.
    }
}

/**
 * @return array<string, array<string, mixed>>
 */
function assessment_retake_requests_for_student(int $studentId, array $assessmentIds): array
{
    if ($studentId < 1 || $assessmentIds === [] || !assessment_retake_table_ready()) {
        return [];
    }
    $placeholders = implode(',', array_fill(0, count($assessmentIds), '?'));
    $st = db()->prepare(
        "SELECT *
        FROM assessment_retake_requests
        WHERE student_id = ? AND assessment_id IN ($placeholders)
        ORDER BY created_at DESC"
    );
    $st->execute(array_merge([$studentId], $assessmentIds));
    $out = [];
    foreach ($st->fetchAll(PDO::FETCH_ASSOC) ?: [] as $row) {
        $aid = (int) ($row['assessment_id'] ?? 0);
        if ($aid > 0 && !isset($out[(string) $aid])) {
            $out[(string) $aid] = $row;
        }
    }
    return $out;
}

/**
 * @return list<array<string, mixed>>
 */
function pending_assessment_retake_requests_for_classroom(int $classroomId): array
{
    if ($classroomId < 1 || !assessment_retake_table_ready()) {
        return [];
    }
    $st = db()->prepare(
        "SELECT r.*, cs.full_name AS student_name, cs.student_number, ca.title AS assessment_title
        FROM assessment_retake_requests r
        INNER JOIN classroom_students cs ON cs.id = r.student_id"
    );

```

```

        INNER JOIN classroom_assessments ca ON ca.id = r.assessment_id
        WHERE r.classroom_id = ? AND r.status = 'pending'
        ORDER BY r.created_at ASC"
    );
    $st->execute([$classroomId]);
    return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
}

function create_assessment_retake_request(
    int $classroomId,
    int $assessmentId,
    int $studentId,
    string $reason
): void {
    if (!assessment_retake_table_ready()) {
        throw new RuntimeException('Retake requests are not available yet. Ask your instructor to run upgrade_roles.php.');
```

```

        'SELECT r.* FROM assessment_retake_requests r
        INNER JOIN online_classrooms oc ON oc.id = r.classroom_id
        WHERE r.id = ? AND r.classroom_id = ? LIMIT 1'
    );
    $st->execute([$requestId, $classroomId]);
    $req = $st->fetch(PDO::FETCH_ASSOC);
    if (!$req) {
        throw new RuntimeException('Retake request not found.');
```

```

    }
    if ((string) ($req['status'] ?? '') !== 'pending') {
        throw new RuntimeException('This retake request is no longer pending.');
```

```

    }
    $assessmentId = (int) ($req['assessment_id'] ?? 0);
    $studentId = (int) ($req['student_id'] ?? 0);
```

```

    db()->beginTransaction();
    try {
        db()->prepare(
            'DELETE FROM classroom_scores WHERE assessment_id = ? AND student_id = ?'
        )->execute([$assessmentId, $studentId]);

        db()->prepare(
            'DELETE FROM classroom_submissions WHERE assessment_id = ? AND student_id = ?'
        )->execute([$assessmentId, $studentId]);

        db()->prepare(
            "UPDATE assessment_retake_requests
            SET status = 'approved', reviewed_by_user_id = ?, reviewed_at = NOW()
            WHERE id = ?"
        )->execute([$facultyUserId, $requestId]);

        db()->commit();
    } catch (Throwable $e) {
        if (db()->inTransaction()) {
            db()->rollBack();
        }
        throw $e;
    }
}
```

```

function reject_assessment_retake_request(
    int $requestId,
    int $classroomId,
    int $facultyUserId,
    string $remarks = ''
): void {
    if (!assessment_retake_table_ready()) {
        throw new RuntimeException('Retake requests are not available.');
```

```

    }
    $st = db()->prepare(
        'SELECT id, status FROM assessment_retake_requests
        WHERE id = ? AND classroom_id = ? LIMIT 1'
    );
    $st->execute([$requestId, $classroomId]);
    $req = $st->fetch(PDO::FETCH_ASSOC);
    if (!$req) {
        throw new RuntimeException('Retake request not found.');
```

```

    }
    if ((string) ($req['status'] ?? '') !== 'pending') {
        throw new RuntimeException('This retake request is no longer pending.');
```

```

    }
    db()->prepare(
        "UPDATE assessment_retake_requests
        SET status = 'rejected', reviewed_by_user_id = ?, reviewed_at = NOW(), faculty_remarks = ?
        WHERE id = ?"
    )->execute([$facultyUserId, trim($remarks), $requestId]);
}
```

```

-----
FILE: includes/auth.php
-----
```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/db.php';
require_once __DIR__ . '/account_registration_helpers.php';
```

```

if (session_status() === PHP_SESSION_NONE) {
    session_name(SESSION_NAME);
    session_start();
}

function auth_user_activity_columns_ready(): bool
{
    static $ready = null;
    if ($ready !== null) {
        return $ready;
    }

    try {
        $st = db()->prepare(
            'SELECT COUNT(*) FROM information_schema.COLUMNS
            WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ? AND COLUMN_NAME IN (?, ?)'
        );
        $st->execute([DB_NAME, 'users', 'last_login_at', 'last_seen_at']);
        $ready = (int) $st->fetchColumn() === 2;
    } catch (Throwable $e) {
        $ready = false;
    }

    return $ready;
}

function auth_touch_user_presence(int $userId): void
{
    if ($userId < 1 || !auth_user_activity_columns_ready()) {
        return;
    }

    // Throttle DB writes: at most once per 2 minutes per session.
    $now = time();
    $last = (int) ($_SESSION['last_seen_touch_at'] ?? 0);
    if ($last > 0 && ($now - $last) < 120) {
        return;
    }
    $_SESSION['last_seen_touch_at'] = $now;

    try {
        db()->prepare('UPDATE users SET last_seen_at = NOW() WHERE id = ?')->execute([$userId]);
    } catch (Throwable $e) {
        // Presence updates are best effort.
    }
}

function require_login(): void
{
    if (empty($_SESSION['user_id'])) {
        header('Location: login.php');
        exit;
    }

    auth_touch_user_presence((int) ($_SESSION['user_id'] ?? 0));
    if ((string) ($_SESSION['role'] ?? '') === 'program_chair') {
        program_chair_sync_session_programs((int) ($_SESSION['user_id'] ?? 0));
        program_chair_handle_active_program_post();
    }
    auth_enforce_password_change_if_required();
}

function current_user(): ?array
{
    if (empty($_SESSION['user_id'])) {
        return null;
    }
    return [
        'id' => (int) $_SESSION['user_id'],
        'username' => $_SESSION['username'] ?? '',
        'full_name' => $_SESSION['full_name'] ?? '',
        'role' => $_SESSION['role'] ?? '',
        'assigned_program' => $_SESSION['assigned_program'] ?? '',
        'assigned_programs' => $_SESSION['assigned_programs'] ?? [],
        'admin_log_title' => $_SESSION['admin_log_title'] ?? '',
        'college_id' => isset($_SESSION['college_id']) ? (int) $_SESSION['college_id'] : null,
        'faculty_id' => isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : null,
        'student_id' => isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : null,
    ];
}

```

```

}

function is_super_admin(): bool
{
    return ($_SESSION['role'] ?? '') === 'super_admin';
}

function is_admin(): bool
{
    return ($_SESSION['role'] ?? '') === 'admin';
}

function is_dean(): bool
{
    return ($_SESSION['role'] ?? '') === 'dean';
}

function is_program_chair(): bool
{
    return ($_SESSION['role'] ?? '') === 'program_chair';
}

function is_faculty(): bool
{
    return ($_SESSION['role'] ?? '') === 'faculty';
}

function is_gened(): bool
{
    return ($_SESSION['role'] ?? '') === 'gened';
}

function is_student(): bool
{
    return ($_SESSION['role'] ?? '') === 'student';
}

function require_role(array $roles): void
{
    require_login();
    if (!in_array((string) ($_SESSION['role'] ?? ''), $roles, true)) {
        http_response_code(403);
        echo 'Forbidden';
        exit;
    }
}

function current_college_id(): ?int
{
    return isset($_SESSION['college_id']) ? (int) $_SESSION['college_id'] : null;
}

function current_program_scope(): ?string
{
    $program = trim((string) ($_SESSION['assigned_program'] ?? ''));
    return $program !== '' ? $program : null;
}

function program_chair_programs_table_ready(bool $refresh = false): bool
{
    static $ready = null;
    if ($refresh) {
        $ready = null;
    }
    if ($ready !== null) {
        return $ready;
    }
    try {
        $ready = db_table_exists('program_chair_programs');
    } catch (Throwable $e) {
        $ready = false;
    }

    return $ready;
}

/** Create program_chair_programs if missing (idempotent). */
function ensure_program_chair_programs_table(): bool
{
    if (program_chair_programs_table_ready()) {

```



```

        return true;
    }
    try {
        db()->exec(
            "CREATE TABLE IF NOT EXISTS program_chair_programs (
                user_id INT NOT NULL,
                program_name VARCHAR(120) NOT NULL,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                PRIMARY KEY (user_id, program_name),
                CONSTRAINT fk_pcp_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
            ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
        );
        return program_chair_programs_table_ready(true);
    } catch (Throwable $e) {
        return false;
    }
}

/**
 * Programs assigned to a Program Chair (junction table + legacy assigned_program fallback).
 *
 * @return list<string>
 */
function program_chair_assigned_programs(int $userId): array
{
    if ($userId < 1) {
        return [];
    }

    $programs = [];
    if (program_chair_programs_table_ready()) {
        $st = db()->prepare(
            'SELECT program_name FROM program_chair_programs WHERE user_id = ? ORDER BY program_name'
        );
        $st->execute([$userId]);
        foreach ($st->fetchAll(PDO::FETCH_COLUMN) ?: [] as $name) {
            $name = trim((string) $name);
            if ($name !== '') {
                $programs[] = $name;
            }
        }
    }

    if ($programs === [] && db_column_exists('users', 'assigned_program')) {
        $st = db()->prepare('SELECT assigned_program FROM users WHERE id = ? LIMIT 1');
        $st->execute([$userId]);
        $legacy = trim((string) ($st->fetchColumn() ?: ''));
        if ($legacy !== '') {
            $programs[] = $legacy;
        }
    }

    return array_values(array_unique($programs));
}

/**
 * Replace a Program Chair's assigned programs and sync users.assigned_program (primary/active).
 *
 * @param list<string> $programs
 */
function program_chair_set_assigned_programs(int $userId, array $programs, ?string $primaryProgram = null): void
{
    if ($userId < 1) {
        throw new RuntimeException('Invalid Program Chair user.');
```

```

    }

    $pdo = db();
    $pdo->beginTransaction();
    try {
        if (program_chair_programs_table_ready()) {
            $pdo->prepare('DELETE FROM program_chair_programs WHERE user_id = ?')->execute([$userId]);
            $ins = $pdo->prepare('INSERT INTO program_chair_programs (user_id, program_name) VALUES (?, ?)');
            foreach ($clean as $name) {
                $ins->execute([$userId, $name]);
            }
        }
        if (db_column_exists('users', 'assigned_program')) {
            $pdo->prepare('UPDATE users SET assigned_program = ? WHERE id = ?')->execute([$primary, $userId]);
        }
        $pdo->commit();
    } catch (Throwable $e) {
        if ($pdo->inTransaction()) {
            $pdo->rollBack();
        }
        throw $e;
    }
}

/**
 * Load assigned programs into the session and ensure assigned_program is one of them.
 *
 * @return list<string>
 */
function program_chair_sync_session_programs(?int $userId = null): array
{
    $uid = $userId ?? (int) ($_SESSION['user_id'] ?? 0);
    if ($uid < 1 || (string) ($_SESSION['role'] ?? '') !== 'program_chair') {
        return [];
    }

    $programs = program_chair_assigned_programs($uid);
    $_SESSION['assigned_programs'] = $programs;

    $active = trim((string) ($_SESSION['assigned_program'] ?? ''));
    if ($programs === []) {
        $_SESSION['assigned_program'] = '';
        return [];
    }
    if ($active === '' || !in_array($active, $programs, true)) {
        $_SESSION['assigned_program'] = $programs[0];
        if (db_column_exists('users', 'assigned_program')) {
            db()->prepare('UPDATE users SET assigned_program = ? WHERE id = ?')->execute([$programs[0], $uid]);
        }
    }

    return $programs;
}

function program_chair_switch_active_program(string $programName): void
{
    // Avoid require_role() here: it re-enters require_login() while the POST switch is handled.
    if (!is_program_chair()) {
        throw new RuntimeException('Only program chairs can switch active program.');
```

```

        return false;
    }
    foreach (program_chair_assigned_programs($userId) as $name) {
        if (strcasecmp($name, $programName) === 0) {
            return true;
        }
    }

    return false;
}

/** Home college for a faculty row; used to scope schedules and weekly views. */
function faculty_college_id(int $facultyId): ?int
{
    $st = db()->prepare('SELECT college_id FROM faculty WHERE id=?');
    $st->execute([$facultyId]);
    $v = $st->fetchColumn();
    return $v !== false && $v !== null ? (int) $v : null;
}

function resolve_faculty_id_for_user(int $userId): ?int
{
    $st = db()->prepare('SELECT id FROM faculty WHERE user_id=? LIMIT 1');
    $st->execute([$userId]);
    $fid = $st->fetchColumn();
    if ($fid !== false) {
        return (int) $fid;
    }

    $u = db()->prepare('SELECT id, role, full_name, college_id FROM users WHERE id=? LIMIT 1');
    $u->execute([$userId]);
    $user = $u->fetch();
    if (!$user || !in_array((string) $user['role'], ['faculty', 'program_chair', 'dean', 'gened'], true)) {
        return null;
    }

    $sql = 'SELECT id, college_id FROM faculty WHERE user_id IS NULL AND full_name = ?';
    $params = [(string) $user['full_name']];
    if ($user['college_id'] !== null) {
        $sql .= ' AND college_id = ?';
        $params[] = (int) $user['college_id'];
    }
    $sql .= ' ORDER BY id';
    $f = db()->prepare($sql);
    $f->execute($params);
    $rows = $f->fetchAll();

    if (count($rows) === 1) {
        $newFid = (int) $rows[0]['id'];
        db()->prepare('UPDATE faculty SET user_id=? WHERE id=? AND user_id IS NULL')->execute([$userId, $newFid]);
        if ($user['college_id'] === null && $rows[0]['college_id'] !== null) {
            db()->prepare('UPDATE users SET college_id=? WHERE id=?')->execute([(int) $rows[0]['college_id'], $userId]);
            $_SESSION['college_id'] = (int) $rows[0]['college_id'];
        }
        return $newFid;
    }

    return null;
}

/**
 * Auto-provision a faculty record for a non-faculty role (dean, program_chair, gened)
 * so they can use classroom features under their own teaching load.
 * Returns the faculty table PK or null on failure.
 */
function ensure_faculty_profile_for_teaching_role(int $userId): ?int
{
    $role = (string) ($_SESSION['role'] ?? '');
    if (!in_array($role, ['dean', 'program_chair', 'gened'], true)) {
        return null;
    }

    $hasIsGened = db_column_exists('faculty', 'is_gened');

    $st = db()->prepare('SELECT id, department, college_id . ($hasIsGened ? ', is_gened' : '') . ' FROM faculty WHERE user_id = ? LIMIT 1');
    $st->execute([$userId]);
    $existing = $st->fetch();
    if ($existing !== false) {
        $needsUpdate = false;
    }

```

```

$updates = [];
$params = [];

if ($role === 'program_chair' && trim((string) $existing['department']) === '') {
    $prog = trim((string) ($_SESSION['assigned_program'] ?? ''));
    if ($prog !== '') {
        $updates[] = 'department = ?';
        $params[] = $prog;
        $needsUpdate = true;
    }
}

if ($role === 'gened' && $hasIsGened && (int) ($existing['is_gened'] ?? 0) === 0) {
    $updates[] = 'is_gened = 1';
    $needsUpdate = true;
}

if ($needsUpdate && $updates !== []) {
    $params[] = (int) $existing['id'];
    db()->prepare('UPDATE faculty SET ' . implode(' ', $updates) . ' WHERE id = ?')->execute($params);
}

return (int) $existing['id'];
}

$u = db()->prepare('SELECT id, full_name, college_id, assigned_program FROM users WHERE id = ? LIMIT 1');
$u->execute([$userId]);
$user = $u->fetch();
if (!$user) {
    return null;
}

$prefixMap = ['dean' => 'DN', 'program_chair' => 'PC', 'gened' => 'GE'];
$prefix = $prefixMap[$role] ?? 'FC';
$facCode = $prefix . '-' . str_pad((string) $userId, 6, '0', STR_PAD_LEFT);

$chk = db()->prepare('SELECT COUNT(*) FROM faculty WHERE faculty_id = ?');
$chk->execute([$facCode]);
if ((int) $chk->fetchColumn() > 0) {
    $facCode = $prefix . '-' . $userId . '-' . time() % 10000;
}

$department = $role === 'program_chair' ? trim((string) ($user['assigned_program'] ?? '')) : '';
$isGened = ($role === 'gened') ? 1 : 0;

$cols = 'user_id, faculty_id, full_name, department, college_id, employment_status, status';
$placeholders = '? , ? , ? , ? , ? , ? , ?';
$values = [
    $userId,
    $facCode,
    (string) $user['full_name'],
    $department,
    $user['college_id'] !== null ? (int) $user['college_id'] : null,
    'Permanent',
    'active',
];

if ($hasIsGened) {
    $cols .= ', is_gened';
    $placeholders .= ', ?';
    $values[] = $isGened;
}

db()->prepare("INSERT INTO faculty ({cols}) VALUES ({placeholders})")->execute($values);

return (int) db()->lastInsertId();
}

/** @deprecated Use ensure_faculty_profile_for_teaching_role() instead */
function ensure_faculty_profile_for_program_chair(int $userId): ?int
{
    return ensure_faculty_profile_for_teaching_role($userId);
}

function resolve_student_id_for_user(int $userId): ?int
{
    $st = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.TABLES
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?'
    );
    $st->execute([DB_NAME, 'classroom_students']);
}

```

```

if ((int) $st->fetchColumn() < 1) {
    return null;
}

$st = db()->prepare(
    'SELECT COUNT(*) FROM information_schema.COLUMNS
    WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ? AND COLUMN_NAME = ?'
);
$st->execute([DB_NAME, 'classroom_students', 'user_id']);
if ((int) $st->fetchColumn() < 1) {
    return null;
}

$st = db()->prepare('SELECT id FROM classroom_students WHERE user_id = ? LIMIT 1');
$st->execute([$userId]);
$sid = $st->fetchColumn();
return $sid !== false ? (int) $sid : null;
}

function dean_college_id_or_fail(): int
{
    require_role(['dean']);
    $cid = current_college_id();
    if (!$cid) {
        http_response_code(403);
        echo 'Dean account is not assigned to a college.';
        exit;
    }
    return $cid;
}

function dean_or_program_chair_college_id_or_fail(): int
{
    require_role(['dean', 'program_chair']);
    $cid = current_college_id();
    if (!$cid) {
        http_response_code(403);
        echo 'This account is not assigned to a college.';
        exit;
    }
    return $cid;
}

function program_scope_or_fail(): string
{
    require_role(['program_chair']);
    program_chair_sync_session_programs((int) ($_SESSION['user_id'] ?? 0));
    $program = current_program_scope();
    if ($program === null) {
        http_response_code(403);
        echo 'Program Chair account is not assigned to a program.';
        exit;
    }
    return $program;
}

/**
 * Handle POST switch of the active program for multi-program chairs.
 * Call early on pages that include the header switcher.
 */
function program_chair_handle_active_program_post(): void
{
    if (
        $_SERVER['REQUEST_METHOD'] !== 'POST'
        || (string) ($_POST['action'] ?? '') !== 'switch_active_program'
        || !is_program_chair()
    ) {
        return;
    }
    try {
        program_chair_switch_active_program((string) ($_POST['active_program'] ?? ''));
        $_SESSION['flash'] = 'Active program switched to ' . (string) ($_SESSION['assigned_program'] ?? '') . '.';
    } catch (Throwable $e) {
        $_SESSION['flash'] = 'Error: ' . $e->getMessage();
    }
    $redirect = trim((string) ($_POST['redirect'] ?? ''));
    if ($redirect === '' || !preg_match('/^[a-zA-Z0-9_\-\./\?=&]+$', $redirect) || str_contains($redirect, '..')) {
        $redirect = basename((string) ($_SERVER['SCRIPT_NAME'] ?? 'dashboard.php'));
    }
    header('Location: ' . $redirect);
}

```

```

    exit;
}

function college_name_by_id(?int $id): string
{
    if (!$id) {
        return 'Unassigned';
    }
    $stmt = db()->prepare('SELECT college_name FROM colleges WHERE id = ?');
    $stmt->execute([$id]);
    return (string) ($stmt->fetchColumn() ?: 'Unknown college');
}

function log_dean_activity(string $actionType, string $details): void
{
    if (!is_dean() && !is_program_chair()) {
        return;
    }
    $uid = (int) ($_SESSION['user_id'] ?? 0);
    $cid = current_college_id();
    if ($uid < 1 || !$cid) {
        return;
    }
    $stmt = db()->prepare(
        'INSERT INTO dean_activity_logs (dean_user_id, college_id, action_type, action_details) VALUES (?, ?, ?, ?)'
    );
    $stmt->execute([$uid, $cid, $actionType, $details]);
}

function verify_admin_password(string $password): bool
{
    $stmt = db()->query("SELECT password FROM users WHERE role IN ('admin', 'super_admin') AND is_active = 1");
    while ($row = $stmt->fetch()) {
        if (password_verify($password, $row['password'])) {
            return true;
        }
    }
    return false;
}

/** Username variants for login (exact, compact, common Super Admin aliases). */
function login_username_lookup_variants(string $username): array
{
    $trimmed = trim($username);
    if ($trimmed === '') {
        return [];
    }

    $variants = [$trimmed];
    $compact = strtolower((string) preg_replace('/[\s_-]+/', '', $trimmed));
    if ($compact !== '' && $compact !== strtolower($trimmed)) {
        $variants[] = $compact;
    }
    if ($compact !== '' && defined('DEFAULT_SUPER_ADMIN_USERNAME')) {
        $defaultSuper = strtolower((string) DEFAULT_SUPER_ADMIN_USERNAME);
        if (in_array($compact, ['superadmin', 'superadministrator', 'super'], true) && $defaultSuper !== '') {
            $variants[] = (string) DEFAULT_SUPER_ADMIN_USERNAME;
        }
    }

    return array_values(array_unique($variants));
}

```

FILE: includes/classroom_discussion_helpers.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/db.php';
require_once __DIR__ . '/auth.php';
require_once __DIR__ . '/messaging_helpers.php';

function classroom_discussions_table_exists(): bool
{
    static $cache = null;
    if ($cache !== null) {
        return $cache;
    }

```

```

}

$st = db()->prepare(
    'SELECT COUNT(*) FROM information_schema.TABLES WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?'
);
$st->execute([DB_NAME, 'classroom_messages']);
$cache = (int) $st->fetchColumn() > 0;
return $cache;
}

function classroom_discussion_can_access(int $classroomId, int $userId): bool
{
    $role = (string) ($_SESSION['role'] ?? '');

    if ($role === 'faculty') {
        $facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
        if ($facultyId < 1) {
            $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
            $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
        }
        if ($facultyId < 1) {
            return false;
        }

        $st = db()->prepare('SELECT COUNT(*) FROM online_classrooms WHERE id = ? AND faculty_id = ?');
        $st->execute([$classroomId, $facultyId]);
        return (int) $st->fetchColumn() > 0;
    }

    if ($role === 'student') {
        $studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
        if ($studentId < 1) {
            $studentId = resolve_student_id_for_user($userId) ?? 0;
            $_SESSION['student_id'] = $studentId > 0 ? $studentId : null;
        }
        if ($studentId < 1) {
            return false;
        }

        $st = db()->prepare(
            'SELECT COUNT(*)
             FROM classroom_enrollments
             WHERE classroom_id = ? AND student_id = ?'
        );
        $st->execute([$classroomId, $studentId]);
        return (int) $st->fetchColumn() > 0;
    }

    return false;
}

function classroom_discussion_enforce_fifo(int $classroomId): void
{
    if (!classroom_discussions_table_exists()) {
        return;
    }
    $max = messaging_thread_max_messages();
    if ($max < 1) {
        return;
    }

    $st = db()->prepare('SELECT COUNT(*) FROM classroom_messages WHERE classroom_id = ?');
    $st->execute([$classroomId]);
    $count = (int) $st->fetchColumn();
    if ($count <= $max) {
        return;
    }

    $excess = $count - $max;
    $st = db()->prepare(
        'SELECT id FROM classroom_messages
         WHERE classroom_id = ?
         ORDER BY created_at ASC, id ASC
         LIMIT ' . $excess
    );
    $st->execute([$classroomId]);
    foreach ($st->fetchAll(PDO::FETCH_ASSOC) ?: [] as $row) {
        $messageId = (int) ($row['id'] ?? 0);
        if ($messageId < 1) {
            continue;
        }
    }
}

```

```

    }
    db()->prepare('DELETE FROM classroom_messages WHERE id = ?')->execute([$messageId]);
}

/**
 * @return list<array{id:int, sender_user_id:int, body:string, created_at:string, full_name:string, role:string, mine:bool}>
 */
function classroom_discussion_messages(int $classroomId, int $viewerUserId, ?int $limit = null): array
{
    if (!classroom_discussions_table_exists()) {
        return [];
    }

    $max = messaging_thread_max_messages();
    if ($limit === null) {
        $limit = $max > 0 ? $max : 500;
    }

    $st = db()->prepare(
        'SELECT cm.id, cm.sender_user_id, cm.body, cm.created_at, u.full_name, u.role
        FROM classroom_messages cm
        INNER JOIN users u ON u.id = cm.sender_user_id
        WHERE cm.classroom_id = ?
        ORDER BY cm.created_at ASC, cm.id ASC
        LIMIT ' . max(1, min(500, $limit))
    );
    $st->execute([$classroomId]);

    $messages = [];
    foreach ($st->fetchAll(PDO::FETCH_ASSOC) ?: [] as $row) {
        $senderUserId = (int) ($row['sender_user_id'] ?? 0);
        $messages[] = [
            'id' => (int) ($row['id'] ?? 0),
            'sender_user_id' => $senderUserId,
            'body' => (string) ($row['body'] ?? ''),
            'created_at' => (string) ($row['created_at'] ?? ''),
            'full_name' => (string) ($row['full_name'] ?? 'Unknown user'),
            'role' => (string) ($row['role'] ?? ''),
            'mine' => $senderUserId === $viewerUserId,
        ];
    }

    return $messages;
}

function classroom_discussion_post(int $classroomId, int $senderUserId, string $body): void
{
    if (!classroom_discussions_table_exists()) {
        throw new RuntimeException('Run upgrade_roles.php once to enable classroom discussion.');
```



```

*/
function classroom_discussion_threads_for_user(int $userId): array
{
    if (!classroom_discussions_table_exists()) {
        return [];
    }

    $role = (string) ($_SESSION['role'] ?? '');
    if ($role === 'faculty') {
        $facultyId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
        if ($facultyId < 1) {
            $facultyId = resolve_faculty_id_for_user($userId) ?? 0;
            $_SESSION['faculty_id'] = $facultyId > 0 ? $facultyId : null;
        }
        if ($facultyId < 1) {
            return [];
        }

        $st = db()->prepare(
            'SELECT oc.id AS classroom_id, oc.title, c.course_code, c.course_name,
                COALESCE(
                    SELECT cm.created_at
                    FROM classroom_messages cm
                    WHERE cm.classroom_id = oc.id
                    ORDER BY cm.created_at DESC, cm.id DESC
                    LIMIT 1
                ), "" AS last_at,
                COALESCE(
                    SELECT cm.body
                    FROM classroom_messages cm
                    WHERE cm.classroom_id = oc.id
                    ORDER BY cm.created_at DESC, cm.id DESC
                    LIMIT 1
                ), "" AS preview
            FROM online_classrooms oc
            INNER JOIN courses c ON c.id = oc.course_id
            WHERE oc.faculty_id = ?
            ORDER BY (last_at = "") ASC, last_at DESC, c.course_code ASC'
        );
        $st->execute([$facultyId]);
        return classroom_discussion_normalize_threads($st->fetchAll(PDO::FETCH_ASSOC) ?: []);
    }

    if ($role === 'student') {
        $studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
        if ($studentId < 1) {
            $studentId = resolve_student_id_for_user($userId) ?? 0;
            $_SESSION['student_id'] = $studentId > 0 ? $studentId : null;
        }
        if ($studentId < 1) {
            return [];
        }

        $st = db()->prepare(
            'SELECT oc.id AS classroom_id, oc.title, c.course_code, c.course_name,
                COALESCE(
                    SELECT cm.created_at
                    FROM classroom_messages cm
                    WHERE cm.classroom_id = oc.id
                    ORDER BY cm.created_at DESC, cm.id DESC
                    LIMIT 1
                ), "" AS last_at,
                COALESCE(
                    SELECT cm.body
                    FROM classroom_messages cm
                    WHERE cm.classroom_id = oc.id
                    ORDER BY cm.created_at DESC, cm.id DESC
                    LIMIT 1
                ), "" AS preview
            FROM classroom_enrollments ce
            INNER JOIN online_classrooms oc ON oc.id = ce.classroom_id
            INNER JOIN courses c ON c.id = oc.course_id
            WHERE ce.student_id = ?
            ORDER BY (last_at = "") ASC, last_at DESC, c.course_code ASC'
        );
        $st->execute([$studentId]);
        return classroom_discussion_normalize_threads($st->fetchAll(PDO::FETCH_ASSOC) ?: []);
    }

    return [];
}

```

```

}

/**
 * @param list<array<string,mixed>> $rows
 * @return list<array{classroom_id:int,title:string,course_code:string,course_name:string,last_at:string,preview:string}>
 */
function classroom_discussion_normalize_threads(array $rows): array
{
    $threads = [];
    foreach ($rows as $row) {
        $preview = trim((string) ($row['preview'] ?? ''));
        if (function_exists('mb_substr')) {
            $preview = mb_strlen($preview) > 80 ? mb_substr($preview, 0, 80) . '...' : $preview;
        } else {
            $preview = strlen($preview) > 80 ? substr($preview, 0, 80) . '...' : $preview;
        }

        $threads[] = [
            'classroom_id' => (int) ($row['classroom_id'] ?? 0),
            'title' => (string) ($row['title'] ?? ''),
            'course_code' => (string) ($row['course_code'] ?? ''),
            'course_name' => (string) ($row['course_name'] ?? ''),
            'last_at' => (string) ($row['last_at'] ?? ''),
            'preview' => $preview,
        ];
    }

    return $threads;
}

/** @return array{classroom_id:int,title:string,course_code:string,course_name:string}|null */
function classroom_discussion_thread_row(int $classroomId, int $userId): ?array
{
    if (!classroom_discussion_can_access($classroomId, $userId)) {
        return null;
    }

    $st = db()->prepare(
        'SELECT oc.id AS classroom_id, oc.title, c.course_code, c.course_name
        FROM online_classrooms oc
        INNER JOIN courses c ON c.id = oc.course_id
        WHERE oc.id = ?
        LIMIT 1'
    );
    $st->execute([$classroomId]);
    $row = $st->fetch(PDO::FETCH_ASSOC);
    if (!$row) {
        return null;
    }

    return [
        'classroom_id' => (int) ($row['classroom_id'] ?? 0),
        'title' => (string) ($row['title'] ?? ''),
        'course_code' => (string) ($row['course_code'] ?? ''),
        'course_name' => (string) ($row['course_name'] ?? ''),
    ];
}

```

FILE: includes/db.php

```

<?php
declare(strict_types=1);

require_once dirname(__DIR__) . '/config/config.php';

if (!defined('APP_TIMEZONE')) {
    define('APP_TIMEZONE', 'Asia/Manila');
}
date_default_timezone_set(APP_TIMEZONE);

function db(): PDO
{
    static $pdo = null;
    if ($pdo === null) {
        $dsn = sprintf(
            'mysql:host=%s;port=%d;dbname=%s;charset=%s',
            DB_HOST,
            DB_PORT,

```

```

        DB_NAME,
        DB_CHARSET
    );
    try {
        $pdo = new PDO($dsn, DB_USER, DB_PASS, [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]);
        // Keep MySQL NOW()/TIMESTAMP in Philippine Standard Time regardless of host TZ.
        $offset = (new DateTime('now', new DateTimeZone(APP_TIMEZONE)))->format('P');
        $pdo->exec('SET time_zone = ' . $pdo->quote($offset));
    } catch (PDOException $e) {
        $code = (string) $e->getCode();
        $msg = $e->getMessage();
        if (
            $code === '2002'
            || str_contains($msg, '2002')
            || str_contains(strtolower($msg), 'refused')
            || str_contains(strtolower($msg), 'could not find server')
        ) {
            throw new RuntimeException(
                'Cannot connect to MySQL (nothing is listening on '
                . DB_HOST . ':' . DB_PORT . '). '
                . 'Open the XAMPP Control Panel and click Start for MySQL, then reload this page. '
                . 'If you use a different port, set DB_PORT in config/config.php.',
                0,
                $e
            );
        }
        throw $e;
    }
    return $pdo;
}

```

FILE: includes/faculty_assessment_question_generator.php

```

<?php
declare(strict_types=1);

/**
 * Auto-generate assessment questions with answer keys for faculty.
 */

require_once __DIR__ . '/student_materials_reviewer.php';
require_once __DIR__ . '/wellness_ai.php';

/**
 * @return array<string, mixed>|null
 */
function faculty_assessment_generator_fetch_classroom(int $facultyId, int $classroomId): ?array
{
    if ($facultyId < 1 || $classroomId < 1) {
        return null;
    }

    $st = db()->prepare(
        'SELECT oc.id, c.course_code, c.course_name, f.full_name AS faculty_name
        FROM online_classrooms oc
        INNER JOIN schedules s ON s.id = oc.schedule_id
        INNER JOIN courses c ON c.id = oc.course_id
        INNER JOIN faculty f ON f.id = oc.faculty_id
        WHERE oc.id = ? AND oc.faculty_id = ? AND s.faculty_id = ?
        LIMIT 1'
    );
    $st->execute([$classroomId, $facultyId, $facultyId]);
    $row = $st->fetch();

    return is_array($row) ? $row : null;
}

function faculty_assessment_generator_topic(string $title, string $description, string $topicOverride = ''): string
{
    $topic = trim($topicOverride);
    if ($topic !== '') {
        return $topic;
    }
}

```

```

$title = trim($title);
if ($title !== '') {
    return $title;
}

$plain = student_materials_reviewer_plain_text($description);
if ($plain !== '') {
    return mb_substr($plain, 0, 120, 'UTF-8');
}

return 'this subject';
}

/**
 * @return list<array<string, mixed>>
 */
function faculty_assessment_generator_material_items(int $classroomId, string $weekLabel): array
{
    $weekLabel = trim($weekLabel);
    if ($classroomId < 1 || $weekLabel === '') {
        return [];
    }

    return student_materials_reviewer_week_items($classroomId, $weekLabel);
}

function faculty_assessment_generator_material_snippet(array $item, int $maxChars = 320): string
{
    $attachmentText = trim((string) ($item['attachment_text'] ?? ''));
    if ($attachmentText !== '') {
        return mb_strlen($attachmentText, 'UTF-8') > $maxChars
            ? mb_substr($attachmentText, 0, max(0, $maxChars - 1), 'UTF-8') . '...'
            : $attachmentText;
    }

    $body = student_materials_reviewer_plain_text((string) ($item['body'] ?? ''));
    if ($body === '') {
        return '';
    }

    return mb_strlen($body, 'UTF-8') > $maxChars
        ? mb_substr($body, 0, max(0, $maxChars - 1), 'UTF-8') . '...'
        : $body;
}

function faculty_assessment_generator_count_attachments(array $items): int
{
    $count = 0;
    foreach ($items as $item) {
        $count += max(0, (int) ($item['attachment_count'] ?? 0));
    }

    return $count;
}

function faculty_assessment_generator_build_context(
    array $classroom,
    string $title,
    string $description,
    string $topic,
    string $weekLabel,
    bool $useMaterials
): string {
    $lines = [
        'Course: ' . trim((string) ($classroom['course_code'] ?? '')) . ' - ' . trim((string) ($classroom['course_name'] ?? '')),
        'Assessment title: ' . trim($title),
        'Topic focus: ' . trim($topic),
    ];

    $plainDesc = student_materials_reviewer_plain_text($description);
    if ($plainDesc !== '') {
        $lines[] = 'Instructions: ' . mb_substr($plainDesc, 0, 1500, 'UTF-8');
    }

    if ($useMaterials && trim($weekLabel) !== '') {
        $items = faculty_assessment_generator_material_items((int) ($classroom['id'] ?? 0), $weekLabel);
        if ($items !== []) {
            $lines[] = '';
            $lines[] = '=== Week materials (including attachment content) ===';
            $lines[] = student_materials_reviewer_build_source_text($items, $classroom, $weekLabel);
        }
    }
}

```

```

    }
}

$text = trim(implode("\n", $lines));
if (mb_strlen($text, 'UTF-8') > 8000) {
    $text = mb_substr($text, 0, 7997, 'UTF-8') . '...';
}

return $text;
}

/**
 * @param list<array<string, mixed>> $items
 * @param list<string> $exclude
 * @return list<array<string, mixed>>
 */
function faculty_assessment_generator_materials_builtin(
    string $assessmentType,
    string $weekLabel,
    array $items,
    int $count,
    array $exclude = []
): array {
    $type = classroom_assessment_normalize_type($assessmentType);
    $count = max(1, min(20, $count));
    $excludeNorm = student_materials_reviewer_normalize_exclude_list($exclude);
    $pool = [];

    foreach ($items as $item) {
        $title = trim((string) ($item['title'] ?? 'Untitled'));
        if ($title === '') {
            $title = 'Untitled';
        }
        $snippet = faculty_assessment_generator_material_snippet($item);
        $hasAttachment = ((int) ($item['attachment_count'] ?? 0)) > 0;
        $sourceLabel = $hasAttachment ? 'the attachment for "' . $title . '" : "' . $title . '"';

        if ($type === 'true_false') {
            if ($snippet !== '') {
                $pool[] = [
                    'question_type' => 'true_false',
                    'question_text' => 'According to ' . $sourceLabel . ' in ' . $weekLabel . ', the posted material covers concepts
students must understand for this course.',
                    'points' => 1.0,
                    'options' => [],
                    'answer_key' => 'true',
                    'word_limit' => null,
                    'char_limit' => null,
                    'allow_steps' => 0,
                ];
                $pool[] = [
                    'question_type' => 'true_false',
                    'question_text' => 'Based on ' . $sourceLabel . ', students can skip reviewing the posted content and still fully
understand ' . $weekLabel . '.',
                    'points' => 1.0,
                    'options' => [],
                    'answer_key' => 'false',
                    'word_limit' => null,
                    'char_limit' => null,
                    'allow_steps' => 0,
                ];
            } else {
                $pool[] = [
                    'question_type' => 'true_false',
                    'question_text' => 'The instructor posted "' . $title . '" as part of ' . $weekLabel . ' materials.',
                    'points' => 1.0,
                    'options' => [],
                    'answer_key' => 'true',
                    'word_limit' => null,
                    'char_limit' => null,
                    'allow_steps' => 0,
                ];
            }
        } elseif ($type === 'multiple_choice') {
            if ($snippet !== '') {
                $shortSnippet = mb_strlen($snippet, 'UTF-8') > 140
                    ? mb_substr($snippet, 0, 137, 'UTF-8') . '...'
                    : $snippet;
                $pool[] = faculty_assessment_generator_mc_row(
                    'Which statement best reflects the content of ' . $sourceLabel . '?',

```

```

        [
            $shortSnippet,
            'The material is unrelated to ' . $weekLabel,
            'Students are not expected to read ' . $sourceLabel,
            'The attachment contains no course concepts',
        ],
        0
    );
}
$pool[] = faculty_assessment_generator_mc_row(
    'Which resource should students review for "' . $title . '" in ' . $weekLabel . '?',
    [
        'An unrelated website',
        $hasAttachment ? 'The posted attachment: ' . $title : 'The posted material: ' . $title,
        'Only the syllabus cover page',
        'Materials from a different week',
    ],
    1
);
} elseif ($type === 'problem_solving') {
    if (preg_match('/\d+(?:\.\d+)?/', $snippet, $m) === 1) {
        $num = (float) $m[0];
        $pool[] = [
            'question_type' => 'problem_solving',
            'question_text' => 'Using a value from ' . $sourceLabel . ', compute: ' . $num . ' × 2.',
            'points' => 1.0,
            'options' => [],
            'answer_key' => (string) ($num * 2),
            'word_limit' => null,
            'char_limit' => null,
            'allow_steps' => 1,
        ];
    }
    $pool[] = [
        'question_type' => 'problem_solving',
        'question_text' => 'If a student reviews ' . count($items) . ' posted material(s) for ' . $weekLabel . ', how many materials
should they study in 2 review sessions covering the full week?',
        'points' => 1.0,
        'options' => [],
        'answer_key' => (string) count($items),
        'word_limit' => null,
        'char_limit' => null,
        'allow_steps' => 1,
    ];
} else {
    $pool[] = [
        'question_type' => 'essay',
        'question_text' => $snippet !== ''
            ? 'Explain the main ideas from ' . $sourceLabel . ' and how they relate to ' . $weekLabel . '
            : 'Summarize the learning goals of "' . $title . '" for ' . $weekLabel . '.',
        'points' => 1.0,
        'options' => [],
        'answer_key' => null,
        'word_limit' => 220,
        'char_limit' => null,
        'allow_steps' => 0,
    ];
    if ($snippet !== '') {
        $pool[] = [
            'question_type' => 'essay',
            'question_text' => 'Using ' . $sourceLabel . ', describe one concept students must understand and give an example.',
            'points' => 1.0,
            'options' => [],
            'answer_key' => null,
            'word_limit' => 200,
            'char_limit' => null,
            'allow_steps' => 0,
        ];
    }
}
}
}

if ($pool === []) {
    return faculty_assessment_generator_builtin($type, $weekLabel, $count, ['id' => 0, 'course_name' => 'the course'], $weekLabel,
true);
}

$filtered = [];
foreach ($pool as $row) {
    if (student_materials_reviewer_question_excluded((string) ($row['question_text'] ?? ''), $excludeNorm)) {

```

```

        continue;
    }
    $filtered[] = $row;
    $excludeNorm[] = mb_strtolower((string) ($row['question_text'] ?? ''), 'UTF-8');
}

if ($filtered === []) {
    return faculty_assessment_generator_builtin($type, $weekLabel, $count, ['id' => 0, 'course_name' => 'the course'], $weekLabel,
true);
}

$questions = [];
for ($i = 0; $i < $count; ++$i) {
    $questions[] = $filtered[$i % count($filtered)];
}

return $questions;
}

/**
 * @param list<string> $options
 */
function faculty_assessment_generator_mc_key(int $correctIndex): string
{
    return chr(65 + max(0, min(5, $correctIndex)));
}

/**
 * @return list<array{
 *     question_type:string,
 *     question_text:string,
 *     points:float,
 *     options:array<int,string>,
 *     answer_key:string,
 *     word_limit:int,
 *     char_limit:int,
 *     allow_steps:int
 * }>
 */
function faculty_assessment_generator_builtin(
    string $assessmentType,
    string $topic,
    int $count,
    array $classroom,
    string $weekLabel,
    bool $useMaterials
): array {
    $type = classroom_assessment_normalize_type($assessmentType);
    $count = max(1, min(20, $count));
    $courseName = trim((string) ($classroom['course_name'] ?? 'the course'));
    $topicLabel = $topic !== '' ? $topic : $courseName;
    $materialTitles = [];

    if ($useMaterials && trim($weekLabel) !== '') {
        foreach (faculty_assessment_generator_material_items((int) ($classroom['id'] ?? 0), $weekLabel) as $item) {
            $title = trim((string) ($item['title'] ?? ''));
            if ($title !== '') {
                $materialTitles[] = $title;
            }
        }
    }

    $questions = [];
    $templates = faculty_assessment_generator_builtin_templates($type, $topicLabel, $courseName, $materialTitles, $weekLabel);

    for ($i = 0; $i < $count; ++$i) {
        $template = $templates[$i % count($templates)];
        $questions[] = $template;
    }

    return $questions;
}

/**
 * @param list<string> $materialTitles
 * @return list<array<string, mixed>>
 */
function faculty_assessment_generator_builtin_templates(
    string $type,
    string $topic,

```

```

string $courseName,
array $materialTitles,
string $weekLabel
): array {
    $material = $materialTitles[0] ?? $topic;
    $secondMaterial = $materialTitles[1] ?? $material;

    if ($type === 'true_false') {
        return [
            [
                'question_type' => 'true_false',
                'question_text' => 'The topic "' . $topic . '" is part of the learning goals in ' . $courseName . '.',',
                'points' => 1.0,
                'options' => [],
                'answer_key' => 'true',
                'word_limit' => null,
                'char_limit' => null,
                'allow_steps' => 0,
            ],
            [
                'question_type' => 'true_false',
                'question_text' => 'Students can ignore the core concepts of "' . $topic . '" and still master ' . $courseName . '.',',
                'points' => 1.0,
                'options' => [],
                'answer_key' => 'false',
                'word_limit' => null,
                'char_limit' => null,
                'allow_steps' => 0,
            ],
            [
                'question_type' => 'true_false',
                'question_text' => 'Understanding "' . $material . '" helps explain ideas covered in "' . $topic . '"',',
                'points' => 1.0,
                'options' => [],
                'answer_key' => 'true',
                'word_limit' => null,
                'char_limit' => null,
                'allow_steps' => 0,
            ],
            [
                'question_type' => 'true_false',
                'question_text' => 'The assessment on "' . $topic . '" requires no preparation from course materials.',',
                'points' => 1.0,
                'options' => [],
                'answer_key' => 'false',
                'word_limit' => null,
                'char_limit' => null,
                'allow_steps' => 0,
            ],
        ];
    }

    if ($type === 'multiple_choice') {
        return [
            faculty_assessment_generator_mc_row(
                'Which option best describes the main focus of "' . $topic . '"?',
                [
                    'A broad overview unrelated to ' . $courseName,
                    'The core concepts and skills for "' . $topic . '"',
                    'Only administrative policies',
                    'Topics from a different course entirely',
                ],
                1
            ),
            faculty_assessment_generator_mc_row(
                'What should students review first when preparing for "' . $topic . '"?',
                [
                    'Random internet sources only',
                    'Posted materials and notes on "' . $material . '"',
                    'Skip all readings',
                    'Wait until after the due date',
                ],
                1
            ),
            faculty_assessment_generator_mc_row(
                'In ' . $courseName . ', "' . $topic . '" is best understood by connecting it to:',
                [
                    'Unrelated hobbies',
                    'Prior lessons and "' . $secondMaterial . '"',
                    'Only the syllabus header',
                ],
            )
        ];
    }
}

```



```

        'Nothing else in the course',
    ],
    1
),
faculty_assessment_generator_mc_row(
    'Which study strategy fits an assessment on "' . $topic . '"?',
    [
        'Memorize labels without understanding',
        'Practice explaining key ideas in your own words',
        'Avoid all practice questions',
        'Copy answers without reading the prompt',
    ],
    1
),
];
}

if ($type === 'problem_solving') {
    $weekNum = 1;
    if (preg_match('/(\d+)/', $weekLabel, $m) === 1) {
        $weekNum = max(1, (int) $m[1]);
    }

    return [
        [
            'question_type' => 'problem_solving',
            'question_text' => 'If a class covers ' . $weekNum . ' topics per week on "' . $topic . '"', how many topics are covered in 4
weeks?',
            'points' => 1.0,
            'options' => [],
            'answer_key' => (string) ($weekNum * 4),
            'word_limit' => null,
            'char_limit' => null,
            'allow_steps' => 1,
        ],
        [
            'question_type' => 'problem_solving',
            'question_text' => 'A review session for "' . $material . '" lasts 45 minutes. How many minutes are used in 3 sessions?',
            'points' => 1.0,
            'options' => [],
            'answer_key' => '135',
            'word_limit' => null,
            'char_limit' => null,
            'allow_steps' => 1,
        ],
        [
            'question_type' => 'problem_solving',
            'question_text' => 'Solve for x:  $2x + 6 = 18$ . This models a two-step process in "' . $topic . '".',
            'points' => 1.0,
            'options' => [],
            'answer_key' => '6',
            'word_limit' => null,
            'char_limit' => null,
            'allow_steps' => 1,
        ],
        [
            'question_type' => 'problem_solving',
            'question_text' => 'A dataset has values 4, 8, and 12. What is the mean?',
            'points' => 1.0,
            'options' => [],
            'answer_key' => '8',
            'word_limit' => null,
            'char_limit' => null,
            'allow_steps' => 1,
        ],
    ],
];
}

return [
    [
        'question_type' => 'essay',
        'question_text' => 'Explain the main ideas of "' . $topic . '" and why they matter in ' . $courseName . '.',
        'points' => 1.0,
        'options' => [],
        'answer_key' => null,
        'word_limit' => 250,
        'char_limit' => null,
        'allow_steps' => 0,
    ],
    [

```

```

        'question_type' => 'essay',
        'question_text' => 'How does "' . $material . '" connect to the learning goals for "' . $topic . '"?',
        'points' => 1.0,
        'options' => [],
        'answer_key' => null,
        'word_limit' => 200,
        'char_limit' => null,
        'allow_steps' => 0,
    ],
    [
        'question_type' => 'essay',
        'question_text' => 'Give a real-world example that illustrates a key concept from "' . $topic . '"',
        'points' => 1.0,
        'options' => [],
        'answer_key' => null,
        'word_limit' => 180,
        'char_limit' => null,
        'allow_steps' => 0,
    ],
    [
        'question_type' => 'essay',
        'question_text' => 'What is still unclear to you about "' . $topic . '"', and how would you clarify it using course materials?',
        'points' => 1.0,
        'options' => [],
        'answer_key' => null,
        'word_limit' => 150,
        'char_limit' => null,
        'allow_steps' => 0,
    ],
];
}

/**
 * @param list<string> $options
 * @return array<string, mixed>
 */
function faculty_assessment_generator_mc_row(string $questionText, array $options, int $correctIndex): array
{
    $options = array_values(array_slice($options, 0, 6));
    while (count($options) < 4) {
        $options[] = 'Option ' . chr(65 + count($options));
    }

    return [
        'question_type' => 'multiple_choice',
        'question_text' => $questionText,
        'points' => 1.0,
        'options' => $options,
        'answer_key' => faculty_assessment_generator_mc_key($correctIndex),
        'word_limit' => null,
        'char_limit' => null,
        'allow_steps' => 0,
    ];
}

/**
 * @param list<string> $exclude
 * @return list<array<string, mixed>>
 */
function faculty_assessment_generator_parse_ai(string $raw, string $assessmentType, int $maxCount, array $exclude = []): array
{
    if (preg_match('/\[{"s\S"}*\]/', $raw, $m) === 1) {
        $raw = $m[0];
    }

    try {
        $decoded = json_decode($raw, true, 512, JSON_THROW_ON_ERROR);
    } catch (JsonException $e) {
        return [];
    }

    if (!is_array($decoded) || !isset($decoded['questions']) || !is_array($decoded['questions'])) {
        return [];
    }

    $expectedType = classroom_assessment_normalize_type($assessmentType);
    $excludeNorm = student_materials_reviewer_normalize_exclude_list($exclude);
    $out = [];

    foreach ($decoded['questions'] as $row) {

```

```

if (count($out) >= $maxCount || !is_array($row)) {
    continue;
}

$questionText = trim((string) ($row['question_text'] ?? $row['question'] ?? ''));
if ($questionText === '' || student_materials_reviewer_question_excluded($questionText, $excludeNorm)) {
    continue;
}

$type = classroom_question_normalize_type((string) ($row['question_type'] ?? $expectedType));
if ($type !== $expectedType) {
    $type = $expectedType;
}

$points = (float) ($row['points'] ?? 1);
if ($points <= 0) {
    $points = 1.0;
}

$options = [];
$rawOptions = $row['options'] ?? $row['choices'] ?? [];
if (is_array($rawOptions)) {
    foreach ($rawOptions as $opt) {
        $text = trim((string) $opt);
        if ($text !== '') {
            $options[] = $text;
        }
    }
}

$answerKey = trim((string) ($row['answer_key'] ?? $row['answer'] ?? ''));
$wordLimit = isset($row['word_limit']) && (string) $row['word_limit'] !== '' ? max(0, (int) $row['word_limit']) : null;
$charLimit = isset($row['char_limit']) && (string) $row['char_limit'] !== '' ? max(0, (int) $row['char_limit']) : null;
$allowSteps = !empty($row['allow_steps']) ? 1 : 0;

if ($type === 'multiple_choice') {
    if (count($options) < 2) {
        continue;
    }
    $options = array_slice($options, 0, 6);
    $answerKey = strtoupper($answerKey);
    if (!preg_match('/^[A-F]$/ ', $answerKey)) {
        $answerKey = 'A';
    }
    $maxLetterIndex = count($options) - 1;
    $letterIndex = ord($answerKey) - 65;
    if ($letterIndex < 0 || $letterIndex > $maxLetterIndex) {
        $answerKey = faculty_assessment_generator_mc_key(0);
    }
} elseif ($type === 'true_false') {
    $keyNorm = strtolower($answerKey);
    if (!in_array($keyNorm, ['true', 'false'], true)) {
        continue;
    }
    $answerKey = $keyNorm;
    $options = [];
} elseif ($type === 'problem_solving') {
    if ($answerKey === '') {
        continue;
    }
    $options = [];
} else {
    $answerKey = null;
    $options = [];
    if ($wordLimit === null) {
        $wordLimit = 200;
    }
}

$out[] = [
    'question_type' => $type,
    'question_text' => $questionText,
    'points' => $points,
    'options' => $options,
    'answer_key' => $answerKey,
    'word_limit' => $wordLimit,
    'char_limit' => $charLimit,
    'allow_steps' => $type === 'problem_solving' ? ($allowSteps ? 1) : 0,
];
$excludeNorm[] = mb_strtolower($questionText, 'UTF-8');
```

```

    }

    return $out;
}

/**
 * @param list<string> $exclude
 * @return list<array<string, mixed>>|null
 */
function faculty_assessment_generator_ai(
    string $sourceText,
    string $assessmentType,
    string $topic,
    int $count,
    array $exclude = [],
    bool $fromWeekMaterials = false,
    string $weekLabel = ''
): ?array {
    if (!wellness_ai_is_enabled() || trim($sourceText) === '') {
        return null;
    }

    $type = classroom_assessment_normalize_type($assessmentType);
    $count = max(1, min(20, $count));
    $typeLabel = classroom_assessment_type_label($type);

    $shapeHint = match ($type) {
        'multiple_choice' => 'For multiple_choice: provide exactly 4 options (plain text, no A/B prefix) and answer_key as a single letter
A-D matching the correct option index.',
        'true_false' => 'For true_false: answer_key must be "true" or "false". No options array.',
        'problem_solving' => 'For problem_solving: provide a concise numeric or expression answer_key (e.g. 12, 3.14, x=5). Set allow_steps
to 1.',
        default => 'For essay: no answer_key. Set word_limit around 150-300.',
    };

    $excludeBlock = '';
    if ($exclude !== []) {
        $sample = array_slice($exclude, -30);
        $excludeBlock = "\nDo NOT repeat or closely rephrase these existing questions:\n- "
            . implode("\n- ", array_map(static fn (string $q): string => mb_substr($q, 0, 200, 'UTF-8'), $sample));
    }

    $materialsRules = '';
    if ($fromWeekMaterials) {
        $weekFocus = $weekLabel !== '' ? $weekLabel : 'the selected week';
        $materialsRules = <<<RULES

IMPORTANT – Week materials mode:
- Base every question ONLY on the posted week materials and extracted attachment text in the context below.
- Prefer facts, definitions, examples, and concepts found in attachments (PDF, DOCX, slides, text files) over generic course wording.
- Reference specific material titles when helpful.
- Do not invent content that is not supported by {$weekFocus} materials.
RULES;
    }

    $system = <<<PROMPT
You create assessment questions for a university instructor. All questions must be type "{$type}" ({$typeLabel}).

Topic: {$topic}

Rules:
- Generate exactly {$count} questions grounded in the provided context.
- Every question must include question_type "{$type}" and points (use 1 unless the prompt suggests otherwise).
- {$shapeHint}{$materialsRules}
- Return ONLY valid JSON (no markdown fences) with this shape:
{"questions":[{"question_type":"{$type}","question_text":"...", "points":1,"options":
[], "answer_key":"...", "word_limit":null, "char_limit":null, "allow_steps":0}]}
PROMPT;

    $user = ($fromWeekMaterials ? "Week materials (body + attachments):\n\n" : "Assessment context:\n\n") . $sourceText . $excludeBlock;
    $raw = student_materials_reviewer_ai_call($system, $user, $count > 8);
    if ($raw === null || trim($raw) === '') {
        return null;
    }

    $parsed = faculty_assessment_generator_parse_ai($raw, $type, $count, $exclude);

    return $parsed !== [] ? $parsed : null;
}

```

```

/**
 * @param list<string> $exclude
 * @return array{
 *   ok: bool,
 *   source: string,
 *   ai_available: bool,
 *   material_count: int,
 *   attachment_count: int,
 *   credited_week: string,
 *   from_week_materials: bool,
 *   questions: list<array<string, mixed>>
 * }
 */
function faculty_assessment_generator_generate(
    int $facultyId,
    int $classroomId,
    string $assessmentType,
    string $title,
    string $description,
    int $count,
    string $topicOverride = '',
    string $weekLabel = '',
    bool $useMaterials = false,
    array $exclude = []
): array {
    $classroom = faculty_assessment_generator_fetch_classroom($facultyId, $classroomId);
    if ($classroom === null) {
        throw new RuntimeException('Classroom not found or you do not have access to it.');
```

```

    }
}

if ($questions === null) {
    if ($fromWeekMaterials) {
        $questions = faculty_assessment_generator_materials_builtin($type, $weekLabel, $materialItems, $count, $exclude);
        $source = 'materials';
    } else {
        $questions = faculty_assessment_generator_builtin($type, $topic, $count, $classroom, $weekLabel, $useMaterials);
    }
}

return [
    'ok' => true,
    'source' => $source,
    'ai_available' => $aiAvailable,
    'material_count' => $materialCount,
    'attachment_count' => $attachmentCount,
    'credited_week' => $weekLabel,
    'from_week_materials' => $fromWeekMaterials,
    'questions' => $questions,
];
}

```

FILE: includes/faculty_classroom_banner.php

```

<?php
declare(strict_types=1);

/** @var array<string,mixed> $classroom */
/** @var int $classroomId */
/** @var string|null $bannerUrl */
/** @var bool $hasBannerCols */
/** @var bool $allowUpload */
/** @var string $title */
/** @var string $metaExtra */
/** @var string $formId */
/** @var string $fileInputId */
/** @var string $uploadFormId */
/** @var string $courseCode */
/** @var string $courseName */
/** @var string $semester */
/** @var string $schoolYear */
?>

<div class="classroom-banner mb-3<?=$bannerUrl ? ' has-photo' : '' ?><?=$bannerUrl ? ' style="--classroom-banner-image: url(\'' .
htmlspecialchars($bannerUrl, ENT_QUOTES) . '\')"' : '' ?>>
    <div class="classroom-banner__overlay">
        <h1 class="classroom-banner__title"><?=$title ?></h1>
        <div class="classroom-banner__meta">
            <?php if ($courseCode !== '' || $courseName !== ''): ?>
                <?=$htmlspecialchars($courseCode) ?><?=$ ($courseCode !== '' && $courseName !== '') ? ' - ' : '' ?><?=$
htmlspecialchars($courseName) ?>
            <?php endif; ?>
            <?php if ($semester !== '' || $schoolYear !== ''): ?>
                <?php if ($courseCode !== '' || $courseName !== ''): ?>
                    <span class="classroom-banner__sep"></span>
                <?php endif; ?>
                <?=$ htmlspecialchars(trim($semester . ($semester !== '' && $schoolYear !== '' ? ' / ' : '')) . $schoolYear) ?>
            <?php endif; ?>
            <?php if ($metaExtra !== ''): ?>
                <span class="classroom-banner__sep"></span>
                <?=$ htmlspecialchars($metaExtra) ?>
            <?php endif; ?>
        </div>
    </div>
    <?php if ($allowUpload && $hasBannerCols): ?>
        <div class="classroom-banner__actions">
            <form method="post" enctype="multipart/form-data" class="classroom-banner__upload-form" id="<?=$uploadFormId
?>">
                <input type="hidden" name="action" value="upload_banner">
                <input type="hidden" name="classroom_id" value="<?=(int) $classroomId ?>">
                <label class="btn btn-light btn-sm classroom-banner__upload-btn" for="<?=$fileInputId ?>"><?=$
function_exists('app_tooltip_attr') ? app_tooltip_attr('Choose a JPEG, PNG, or WebP image for this class header background. Students see the
same banner.') : '' ?>>
                    <i class="fa-solid fa-image me-1"></i><?=$bannerUrl ? 'Change background' : 'Upload background' ?>
                </label>
                <input id="<?=$fileInputId ?>" type="file" name="banner" class="visually-hidden"
                accept="image/jpeg,image/png,image/webp,.jpg,.jpeg,.png,.webp" required>
            </div>
        </div>
    </div>

```

```

        </form>
        <?php if ($bannerUrl): ?>
            <form method="post" class="d-inline" onsubmit="return confirm('Remove the header background image?');">
                <input type="hidden" name="action" value="delete_banner">
                <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
                <button type="submit" class="btn btn-outline-light btn-sm"><?= function_exists('app_tooltip_attr') ?
app_tooltip_attr('Removes the custom background; the default gradient banner will show instead.') : '' ?><i class="fa-solid fa-trash me-1">
</i>Remove</button>
            </form>
        <?php endif; ?>
    </div>
    <script>
    (function () {
        var input = document.getElementById(<?= json_encode($fileInputId) ?>);
        var form = document.getElementById(<?= json_encode($uploadFormId) ?>);
        if (!input || !form) return;
        input.addEventListener('change', function () {
            if (input.files && input.files.length) {
                form.submit();
            }
        });
    })();
    </script>
    <?php elseif ($allowUpload && !$hasBannerCols): ?>
        <div class="classroom-banner__actions">
            <span class="badge text-bg-warning">Run upgrade_roles.php to enable background upload</span>
        </div>
    <?php endif; ?>
</div>

```

FILE: includes/favicon.php

```

<?php
declare(strict_types=1);

$faviconVersion = @filemtime(__DIR__ . '/../assets/wpu-logo.png') ?: 1;
?>
<link rel="icon" href="assets/wpu-logo.png?v=<?= (int) $faviconVersion ?>" type="image/png">
<link rel="apple-touch-icon" href="assets/wpu-logo.png?v=<?= (int) $faviconVersion ?>">

```

FILE: includes/footer.php

```

</main>
<?php
$appSidebarShell = $appSidebarShell ?? false;
if ($appSidebarShell):
    ?>
    </div>
</div>
<?php
require __DIR__ . '/admin_offcanvas.php';
endif;
?>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
<script src="assets/js/theme-toggle.js" defer></script>
<?php if (!empty($appCursorToolTips ?? false)): ?>
<script src="assets/js/app_tooltips.js" defer></script>
<?php endif; ?>
<?php if (!empty($footerScripts) && is_array($footerScripts)): ?>
    <?php foreach ($footerScripts as $footerScript): ?>
        <?php if (is_string($footerScript) && $footerScript !== ''): ?>
<script src="<?= htmlspecialchars($footerScript) ?>"></script>
        <?php endif; ?>
    <?php endforeach; ?>
<?php endif; ?>
</body>
</html>

```

FILE: includes/functions.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/db.php';

```

```

/** @return string[] */
function schedule_days_list(): array
{
    return ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'];
}

function sql_scope_for_college(string $alias, ?int $collegeId, bool $adminSeesAll = true): array
{
    if ($adminSeesAll && function_exists('is_admin') && is_admin()) {
        return ['sql' => '', 'params' => []];
    }
    if ($collegeId) {
        return ['sql' => " AND {$alias}.college_id = ?", 'params' => [$collegeId]];
    }
    return ['sql' => ' AND 1 = 0 ', 'params' => []];
}

/**
 * Suggest available usernames based on a preferred value.
 * @return string[]
 */
function suggest_available_usernames(string $preferred, int $limit = 3): array
{
    $base = strtolower(trim($preferred));
    $base = preg_replace('/[^a-z0-9._-]+/', '', $base ?? '');
    if ($base === '') {
        $base = 'user';
    }

    $st = db()->prepare('SELECT username FROM users WHERE username LIKE ?');
    $st->execute([$base . '%']);
    $existing = array_fill_keys(array_map('strtolower', $st->fetchAll(PDO::FETCH_COLUMN)), true);

    $suggestions = [];
    if (!isset($existing[$base])) {
        $suggestions[] = $base;
    }

    $n = 1;
    while (count($suggestions) < $limit && $n < 1000) {
        $candidate = $base . $n;
        if (!isset($existing[$candidate])) {
            $suggestions[] = $candidate;
        }
        $n++;
    }
    return $suggestions;
}

/**
 * Suggest available course codes within a scope.
 * Scope is college-specific for dean flows, or GE-specific for gened flows.
 *
 * @return string[]
 */
function suggest_available_course_codes(
    string $preferred,
    ?int $collegeId = null,
    bool $isGened = false,
    int $limit = 3
): array {
    $base = strtoupper(trim($preferred));
    $base = preg_replace('/[^A-Z0-9._-]+/', '', $base ?? '');
    if ($base === '') {
        $base = 'COURSE';
    }

    if ($isGened) {
        $st = db()->prepare('SELECT course_code FROM courses WHERE COALESCE(is_gened,0)=1 AND course_code LIKE ?');
        $st->execute([$base . '%']);
    } else {
        $st = db()->prepare('SELECT course_code FROM courses WHERE college_id = ? AND course_code LIKE ?');
        $st->execute([$collegeId, $base . '%']);
    }
    $existing = array_fill_keys(array_map('strtoupper', $st->fetchAll(PDO::FETCH_COLUMN)), true);

    $suggestions = [];
    if (!isset($existing[$base])) {
        $suggestions[] = $base;
    }
}

```



```

    }

    $n = 1;
    while (count($suggestions) < $limit && $n < 1000) {
        $candidate = $base . sprintf('%02d', $n);
        if (!isset($existing[$candidate])) {
            $suggestions[] = $candidate;
        }
        $n++;
    }
    return $suggestions;
}

function db_column_exists(string $table, string $column): bool
{
    $stmt = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.COLUMNS
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ? AND COLUMN_NAME = ?'
    );
    $stmt->execute([DB_NAME, $table, $column]);
    return (int) $stmt->fetchColumn() > 0;
}

/**
 * Return the first existing column name from a preferred list, else fallback.
 */
function db_first_existing_column(string $table, array $preferredColumns, string $fallback): string
{
    foreach ($preferredColumns as $column) {
        if (is_string($column) && $column !== '' && db_column_exists($table, $column)) {
            return $column;
        }
    }
    return $fallback;
}

/** Normalize a classroom join code for lookup (alphanumeric only, uppercase). */
function classroom_normalize_join_code(string $raw): string
{
    $clean = preg_replace('/[^A-Za-z0-9]+/', '', $raw);

    return strtoupper($clean === null ? '' : $clean);
}

/**
 * Allocate a short unique join code for online_classrooms.join_code.
 * @throws RuntimeException
 */
function classroom_alloc_unique_join_code(): string
{
    $chars = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
    for ($attempt = 0; $attempt < 50; $attempt++) {
        $code = '';
        for ($i = 0; $i < 8; $i++) {
            $code .= $chars[random_int(0, strlen($chars) - 1)];
        }
        $st = db()->prepare('SELECT COUNT(*) FROM online_classrooms WHERE join_code = ?');
        $st->execute([$code]);
        if ((int) $st->fetchColumn() === 0) {
            return $code;
        }
    }
    throw new RuntimeException('Could not allocate a unique class code. Try again.');
```

```

}

/**
 * Unique code for courses.classroom_code (Program Chair subject code).
 * Avoids collision with online_classrooms.join_code so student join-by-code stays unambiguous.
 *
 * @throws RuntimeException
 */
function course_alloc_unique_classroom_code(): string
{
    $chars = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
    for ($attempt = 0; $attempt < 50; $attempt++) {
        $code = '';
        for ($i = 0; $i < 8; $i++) {
            $code .= $chars[random_int(0, strlen($chars) - 1)];
        }
        $st = db()->prepare('SELECT COUNT(*) FROM courses WHERE classroom_code = ?');
```

```

    $st->execute([$code]);
    if ((int) $st->fetchColumn() > 0) {
        continue;
    }
    if (db_table_exists('online_classrooms') && db_column_exists('online_classrooms', 'join_code')) {
        $stOc = db()->prepare('SELECT COUNT(*) FROM online_classrooms WHERE join_code = ?');
        $stOc->execute([$code]);
        if ((int) $stOc->fetchColumn() > 0) {
            continue;
        }
    }
    return $code;
}
}
throw new RuntimeException('Could not allocate a unique classroom code. Try again.');
```

```

function db_table_exists(string $table): bool
{
    $stmt = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.TABLES
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?'
    );
    $stmt->execute([DB_NAME, $table]);
    return (int) $stmt->fetchColumn() > 0;
}

/** Ensure faculty_course_colors exists (idempotent; safe for local installs). */
function ensure_faculty_course_colors_table(): bool
{
    if (db_table_exists('faculty_course_colors')) {
        return true;
    }
    try {
        db()->exec(
            "CREATE TABLE IF NOT EXISTS faculty_course_colors (
                id INT AUTO_INCREMENT PRIMARY KEY,
                faculty_id INT NOT NULL,
                course_id INT NOT NULL,
                color_index TINYINT UNSIGNED NOT NULL DEFAULT 0,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
                UNIQUE KEY uq_fcc_faculty_course (faculty_id, course_id),
                CONSTRAINT fk_fcc_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE,
                CONSTRAINT fk_fcc_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
            ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
        );
    } catch (Throwable $e) {
        return false;
    }
    return db_table_exists('faculty_course_colors');
}

/**
 * Preset schedule block colors (index 0-5 → CSS classes c0-c5).
 *
 * @return list<array{index:int,label:string,bg:string,border:string}>
 */
function schedule_color_palette(): array
{
    return [
        ['index' => 0, 'label' => 'Blue', 'bg' => '#e3f2fd', 'border' => '#1976d2'],
        ['index' => 1, 'label' => 'Green', 'bg' => '#e8f5e9', 'border' => '#388e3c'],
        ['index' => 2, 'label' => 'Orange', 'bg' => '#fff3e0', 'border' => '#f57c00'],
        ['index' => 3, 'label' => 'Pink', 'bg' => '#fce4ec', 'border' => '#c2185b'],
        ['index' => 4, 'label' => 'Purple', 'bg' => '#f3e5f5', 'border' => '#7b1fa2'],
        ['index' => 5, 'label' => 'Teal', 'bg' => '#e0f7fa', 'border' => '#00838f'],
    ];
}

/**
 * Map of course_id => color_index for one faculty member.
 *
 * @return array<int,int>
 */
function faculty_course_color_map(int $facultyId): array
{
    if ($facultyId < 1 || !ensure_faculty_course_colors_table()) {
        return [];
    }
    $st = db()->prepare('SELECT course_id, color_index FROM faculty_course_colors WHERE faculty_id = ?');
    $st->execute([$facultyId]);

```

```

$map = [];
foreach ($st->fetchAll(PDO::FETCH_ASSOC) as $row) {
    $map[(int) $row['course_id']] = max(0, min(5, (int) $row['color_index']));
}
return $map;
}

/**
 * CSS class for a schedule block (c0-c5). Uses faculty preference when available.
 *
 * @param array<int,int>|null $colorMap course_id => color_index
 */
function schedule_block_color_class(int $courseId, ?array $colorMap = null): string
{
    if ($colorMap !== null && isset($colorMap[$courseId])) {
        return 'c' . max(0, min(5, (int) $colorMap[$courseId]));
    }
    return 'c' . ($courseId % 6);
}

/**
 * Save a faculty member's preferred color for a course they teach.
 *
 * @throws RuntimeException
 */
function save_faculty_course_color(int $facultyId, int $courseId, int $colorIndex, ?int $collegeId = null): void
{
    if ($facultyId < 1 || $courseId < 1) {
        throw new RuntimeException('Invalid course selection.');
```

```

        return [];
    }
    $st = db()->prepare(
        'SELECT p.program_name, pyl.year_level
        FROM programs_year_levels pyl
        INNER JOIN programs p ON p.id = pyl.program_id
        WHERE p.college_id = ?
        ORDER BY p.program_name, pyl.year_level'
    );
    $st->execute([$collegeId]);
    $out = [];
    while ($row = $st->fetch(PDO::FETCH_ASSOC)) {
        $pn = (string) ($row['program_name'] ?? '');
        $yl = trim((string) ($row['year_level'] ?? ''));
        if ($pn === '' || $yl === '') {
            continue;
        }
        if (!isset($out[$pn])) {
            $out[$pn] = [];
        }
        if (!in_array($yl, $out[$pn], true)) {
            $out[$pn][] = $yl;
        }
    }
    return $out;
}

/**
 * Normalize year-level labels for predictable ordering (1, 2... then others alphabetically).
 * @param list<string> $levels
 * @return list<string>
 */
function sort_schedule_year_levels(array $levels): array
{
    $uniq = [];
    foreach ($levels as $l) {
        $s = trim((string) $l);
        if ($s !== '' && !in_array($s, $uniq, true)) {
            $uniq[] = $s;
        }
    }
    usort(
        $uniq,
        static function (string $a, string $b): int {
            $ad = ctype_digit($a);
            $bd = ctype_digit($b);
            if ($ad && $bd) {
                return (int) $a <=> (int) $b;
            }
            if ($ad !== $bd) {
                return $ad ? -1 : 1;
            }
            return strcmp($a, $b);
        }
    );
    /** @var list<string> */
    return $uniq;
}

/** @return list<string> */
function program_defined_year_levels(int $programId): array
{
    if ($programId < 1 || !db_table_exists('programs_year_levels')) {
        return [];
    }
    $st = db()->prepare('SELECT year_level FROM programs_year_levels WHERE program_id=?');
    $st->execute([$programId]);
    $raw = [];
    foreach ($st->fetchAll(PDO::FETCH_COLUMN) as $v) {
        $raw[] = (string) $v;
    }
    return sort_schedule_year_levels($raw);
}

/**
 * Replace dean-configured year levels for programs.id (no-op when table absent).
 *
 * @param list<string>|array<int|string,string> $levels
 */
function program_year_levels_replace(int $programId, array $levels): void

```

```

{
    if (!db_table_exists('programs_year_levels') || $programId < 1) {
        return;
    }
    $pdo = db();
    $pdo->prepare('DELETE FROM programs_year_levels WHERE program_id=?')->execute([$programId]);
    $ins = $pdo->prepare('INSERT INTO programs_year_levels (program_id, year_level) VALUES (?,?)');
    $seen = [];
    foreach ($levels as $l) {
        $s = trim((string) $l);
        if ($s === '' || strlen($s) > 20 || isset($seen[$s])) {
            continue;
        }
        $seen[$s] = true;
        $ins->execute([$programId, $s]);
    }
}

/**
 * Parsed year levels from Programs form (checkboxes + optional comma list).
 *
 * @return list<string>
 */
function parse_dean_program_year_levels_post(array $post): array
{
    $levels = [];
    $raw = $post['program_year_levels'] ?? null;
    if (is_array($raw)) {
        foreach ($raw as $v) {
            $s = trim((string) $v);
            if ($s !== '' && strlen($s) <= 20) {
                $levels[] = $s;
            }
        }
    }
    $extra = trim((string) ($post['program_year_level_extra'] ?? ''));
    if ($extra !== '') {
        foreach (preg_split('/[,;]\s*/', $extra, -1, PREG_SPLIT_NO_EMPTY) ?: [] as $part) {
            $p = trim($part);
            if ($p !== '' && strlen($p) <= 20) {
                $levels[] = $p;
            }
        }
    }
    return sort_schedule_year_levels($levels);
}

function classroom_assessment_normalize_type(string $type): string
{
    return match (trim(strtolower($type))) {
        'multiple_choice' => 'multiple_choice',
        'true_false' => 'true_false',
        'problem_solving', 'performance_task' => 'problem_solving',
        'essay', 'assignment', 'written_work' => 'essay',
        'quiz' => 'multiple_choice',
        default => 'essay',
    };
}

function classroom_assessment_type_label(string $type): string
{
    return match (classroom_assessment_normalize_type($type)) {
        'multiple_choice' => 'Multiple Choice',
        'true_false' => 'True or False',
        'problem_solving' => 'Problem Solving',
        default => 'Essay',
    };
}

function classroom_assessment_type_badge_class(string $type): string
{
    return match (classroom_assessment_normalize_type($type)) {
        'multiple_choice' => 'text-bg-primary',
        'true_false' => 'text-bg-info',
        'problem_solving' => 'text-bg-success',
        default => 'text-bg-secondary',
    };
}

/** @return string[] */

```

```

function classroom_question_type_list(): array
{
    return ['multiple_choice', 'true_false', 'essay', 'problem_solving'];
}

function classroom_question_type_label(string $type): string
{
    return match (trim(strtolower($type))) {
        'multiple_choice' => 'Multiple Choice',
        'true_false' => 'True or False',
        'essay' => 'Essay',
        'problem_solving' => 'Problem Solving',
        default => 'Question',
    };
}

function classroom_question_normalize_type(string $type): string
{
    $type = trim(strtolower($type));
    return in_array($type, classroom_question_type_list(), true) ? $type : 'essay';
}

function classroom_problem_answer_normalize(string $value): string
{
    $value = trim($value);
    if ($value === '') {
        return '';
    }
    $value = preg_replace('/\s+/u', '', $value) ?? '';
    $value = strtolower($value);
    if (preg_match('/^~?\d+(?:\.\d+)?$/i', $value) === 1) {
        $value = rtrim(rtrim($value, '0'), '.');
    }
    return $value;
}

function classroom_problem_answer_matches(string $answer, string $key): bool
{
    return classroom_problem_answer_normalize($answer) === classroom_problem_answer_normalize($key);
}

/**
 * @param array<string,mixed> $question
 * @param array<string,mixed> $payload
 * @return array{answer_text:string,answer_steps:string,is_correct:int:auto_score:float,requires_manual:bool}
 */
function classroom_grade_question_submission(array $question, array $payload): array
{
    $type = classroom_question_normalize_type((string) ($question['question_type'] ?? 'essay'));
    $points = (float) ($question['points'] ?? 0);
    $answerRaw = trim((string) ($payload['answer_text'] ?? ''));
    $stepsRaw = trim((string) ($payload['answer_steps'] ?? ''));
    $keyRaw = trim((string) ($question['answer_key'] ?? ''));

    if ($type === 'multiple_choice') {
        $ok = $answerRaw !== '' && strcasecmp($answerRaw, $keyRaw) === 0;
        return [
            'answer_text' => $answerRaw,
            'answer_steps' => $stepsRaw !== '' ? $stepsRaw : null,
            'is_correct' => $ok ? 1 : 0,
            'auto_score' => $ok ? $points : 0.0,
            'requires_manual' => false,
        ];
    }

    if ($type === 'true_false') {
        $answerNorm = strtolower($answerRaw) === 'true' ? 'true' : (strtolower($answerRaw) === 'false' ? 'false' : '');
        $keyNorm = strtolower($keyRaw) === 'true' ? 'true' : (strtolower($keyRaw) === 'false' ? 'false' : '');
        $ok = $answerNorm !== '' && $keyNorm !== '' && $answerNorm === $keyNorm;
        return [
            'answer_text' => $answerNorm,
            'answer_steps' => $stepsRaw !== '' ? $stepsRaw : null,
            'is_correct' => $ok ? 1 : 0,
            'auto_score' => $ok ? $points : 0.0,
            'requires_manual' => false,
        ];
    }

    if ($type === 'problem_solving') {
        $ok = $answerRaw !== '' && $keyRaw !== '' && classroom_problem_answer_matches($answerRaw, $keyRaw);
    }
}

```

```

        return [
            'answer_text' => $answerRaw,
            'answer_steps' => $stepsRaw !== '' ? $stepsRaw : null,
            'is_correct' => $ok ? 1 : 0,
            'auto_score' => $ok ? $points : 0.0,
            'requires_manual' => false,
        ];
    }

    return [
        'answer_text' => $answerRaw,
        'answer_steps' => $stepsRaw !== '' ? $stepsRaw : null,
        'is_correct' => null,
        'auto_score' => null,
        'requires_manual' => true,
    ];
}

function classroom_content_attachment_dir(): string
{
    return BASE_PATH . DIRECTORY_SEPARATOR . 'uploads' . DIRECTORY_SEPARATOR . 'classroom_content';
}

function classroom_content_attachment_relative_dir(): string
{
    return 'uploads/classroom_content';
}

function classroom_content_is_attachment(string $resourceUrl): bool
{
    $resourceUrl = trim($resourceUrl);
    if ($resourceUrl === '') {
        return false;
    }

    $normalized = str_replace('\\', '/', $resourceUrl);
    $prefix = classroom_content_attachment_relative_dir() . '/';
    return str_starts_with($normalized, $prefix);
}

function classroom_content_attachment_path(string $resourceUrl): string
{
    return classroom_content_attachment_dir() . DIRECTORY_SEPARATOR . basename(str_replace('\\', '/', trim($resourceUrl)));
}

function classroom_content_attachment_storage_path(string $storedName): string
{
    return classroom_content_attachment_dir() . DIRECTORY_SEPARATOR . basename($storedName);
}

function classroom_content_attachment_name(string $resourceUrl): string
{
    $name = basename(str_replace('\\', '/', trim($resourceUrl)));
    $parts = explode('_', $name, 2);
    $downloadName = $parts[1] ?? $name;
    $downloadName = trim($downloadName);
    return $downloadName !== '' ? $downloadName : 'attachment';
}

function classroom_content_resource_href(int $contentId, string $resourceUrl): string
{
    if (classroom_content_is_attachment($resourceUrl)) {
        return 'classroom_content_attachment.php?id=' . $contentId;
    }

    return $resourceUrl;
}

function classroom_content_attachment_href(int $attachmentId, bool $inline = false): string
{
    $href = 'classroom_content_attachment.php?attachment_id=' . $attachmentId;
    if ($inline) {
        $href .= '&inline=1';
    }

    return $href;
}

function classroom_content_is_image_filename(string $name): bool
{

```

```

        $ext = strtolower(pathinfo(trim($name), PATHINFO_EXTENSION));

        return in_array($ext, ['jpg', 'jpeg', 'png', 'gif', 'webp'], true);
    }

function classroom_content_is_image_mime(string $mime): bool
{
    $mime = strtolower(trim($mime));

    return $mime !== '' && str_starts_with($mime, 'image/');
}

/**
 * Whether a content attachment row should be previewed as an inline image.
 *
 * @param array<string,mixed> $attachment
 */
function classroom_content_attachment_is_image(array $attachment): bool
{
    if (classroom_content_is_image_mime((string) ($attachment['mime'] ?? ''))) {
        return true;
    }

    return classroom_content_is_image_filename((string) ($attachment['original_name'] ?? ''))
        || classroom_content_is_image_filename((string) ($attachment['stored_name'] ?? ''));
}

function classroom_content_inline_image_href(string $storedName): string
{
    return 'classroom_content_inline.php?n=' . rawurlencode(basename($storedName));
}

/**
 * Persist a pasted/editor data-URI image and return a safe relative img src.
 *
 * @throws RuntimeException
 */
function classroom_content_store_data_uri_image(string $dataUri): string
{
    if (preg_match('#^data:image/(jpeg|jpg|png|gif|webp);base64,([A-Za-z0-9+/\s+=\s]+)$#i', trim($dataUri), $matches) !== 1) {
        throw new RuntimeException('Unsupported inline image format.');
```



```

        if (!is_dir($dir) && !mkdir($dir, 0775, true) && !is_dir($dir)) {
            throw new RuntimeException('Unable to create attachment directory.');
```

```

        }

        $storedName = 'inline_' . bin2hex(random_bytes(16)) . '.' . $ext;
        $destination = classroom_content_attachment_storage_path($storedName);
        if (file_put_contents($destination, $binary) === false) {
            throw new RuntimeException('Failed to save pasted image.');
```

```

        }

        return classroom_content_inline_image_href($storedName);
    }
}

/**
 * Normalize and validate an <img src> value for classroom HTML bodies.
 */
function classroom_content_sanitize_img_src(string $src): ?string
{
    $src = trim(html_entity_decode($src, ENT_QUOTES | ENT_HTML5, 'UTF-8'));
    if ($src === '') {
        return null;
    }

    if (str_starts_with(strtolower($src), 'data:image/')) {
        try {
            return classroom_content_store_data_uri_image($src);
        } catch (Throwable) {
            return null;
        }
    }

    if (preg_match('#^(?:(:?\.\/)?classroom_content_inline\.php(?:n=)([A-Za-z0-9_-]+)$#', $src, $matches) === 1) {
        $stored = basename((string) $matches[1]);
        if ($stored === '' || !is_file(classroom_content_attachment_storage_path($stored))) {
            return null;
        }

        return classroom_content_inline_image_href($stored);
    }

    if (preg_match('#^(?:(:?\.\/)?classroom_content_attachment\.php(?:attachment_id|id)=(\d+)(?:&inline=1)?$#', $src, $matches) === 1) {
        $id = (int) $matches[1];

        return $id > 0 ? 'classroom_content_attachment.php?attachment_id=' . $id . '&inline=1' : null;
    }

    $validated = filter_var($src, FILTER_VALIDATE_URL);
    if ($validated === false) {
        return null;
    }

    $scheme = strtolower((string) (parse_url($validated, PHP_URL_SCHEME) ?? ''));
    if (!in_array($scheme, ['http', 'https'], true)) {
        return null;
    }

    return $validated;
}

/** Open syllabus for a classroom (auth enforced in classroom_syllabus.php). */
function classroom_syllabus_href(int $classroomId): string
{
    return 'classroom_syllabus.php?id=' . $classroomId;
}

function classroom_banner_column_ready(): bool
{
    static $ready = null;
    if ($ready === null) {
        $ready = db_column_exists('online_classrooms', 'banner_stored_name');
    }
    return $ready;
}

function classroom_banner_dir(): string
{
    return BASE_PATH . DIRECTORY_SEPARATOR . 'uploads' . DIRECTORY_SEPARATOR . 'classroom_banners';
}

function classroom_banner_path(string $storedName): string
{

```

```

        return classroom_banner_dir() . DIRECTORY_SEPARATOR . basename($storedName);
    }

function classroom_banner_href(int $classroomId): string
{
    return 'classroom_banner.php?id=' . $classroomId;
}

function classroom_banner_url_for(array $classroom): ?string
{
    if (!classroom_banner_column_ready()) {
        return null;
    }
    $stored = trim((string) ($classroom['banner_stored_name'] ?? ''));
    if ($stored === '') {
        return null;
    }
    $path = classroom_banner_path($stored);
    if (!is_file($path)) {
        return null;
    }
    $id = (int) ($classroom['id'] ?? 0);
    return $id > 0 ? classroom_banner_href($id) : null;
}

/**
 * @param array<string,mixed> $file
 */
function classroom_banner_store(int $classroomId, int $facultyId, array $file): string
{
    if ($classroomId < 1) {
        throw new RuntimeException('Invalid classroom.');
```

```

        'image/webp' => 'webp',
        default => '',
    };
    if ($ext === '') {
        throw new RuntimeException('Only JPEG, PNG, and WebP images are allowed.');
```

```

    }

    $dir = classroom_banner_dir();
    if (!is_dir($dir) && !mkdir($dir, 0775, true) && !is_dir($dir)) {
        throw new RuntimeException('Unable to create banner storage directory.');
```

```

    }

    $st = db()->prepare(
        'SELECT banner_stored_name FROM online_classrooms WHERE id = ? LIMIT 1'
    );
    $st->execute([$classroomId]);
    $oldStored = trim((string) ($st->fetchColumn() ?: ''));
    if ($oldStored !== '') {
        $oldPath = classroom_banner_path($oldStored);
        if (is_file($oldPath)) {
            @unlink($oldPath);
        }
    }
}
```

```

$original = trim((string) ($file['name'] ?? ''));
$original = str_replace(["\r", "\n"], '', basename($original));
if ($original === '') {
    $original = 'banner.' . $ext;
}
}
```

```

$stored = 'class_' . $classroomId . '_' . bin2hex(random_bytes(12)) . '.' . $ext;
$dest = classroom_banner_path($stored);
if (!move_uploaded_file($tmp, $dest)) {
    throw new RuntimeException('Failed to save background image.');
```

```

}

$upd = db()->prepare(
    'UPDATE online_classrooms
     SET banner_stored_name = ?, banner_original_name = ?, banner_mime = ?
     WHERE id = ?'
);
$upd->execute([$stored, $original, $mime, $classroomId]);
if ($upd->rowCount() < 1) {
    @unlink($dest);
    throw new RuntimeException('Classroom not found.');
```

```

}

return $stored;
}
```

```

function classroom_banner_remove(int $classroomId, int $facultyId): void
{
    unset($facultyId);
    classroom_banner_remove_for_classroom($classroomId);
}
}
```

```

function classroom_banner_remove_for_classroom(int $classroomId): void
{
    if ($classroomId < 1) {
        throw new RuntimeException('Invalid classroom.');
```

```

    }
    if (!classroom_banner_column_ready()) {
        throw new RuntimeException('Classroom banners are not installed.');
```

```

    }
}

$st = db()->prepare(
    'SELECT banner_stored_name FROM online_classrooms WHERE id = ? LIMIT 1'
);
$st->execute([$classroomId]);
$oldStored = trim((string) ($st->fetchColumn() ?: ''));
if ($oldStored !== '') {
    $oldPath = classroom_banner_path($oldStored);
    if (is_file($oldPath)) {
        @unlink($oldPath);
    }
}
}
```

```

db()->prepare(
    'UPDATE online_classrooms
     SET banner_stored_name = NULL, banner_original_name = NULL, banner_mime = NULL

```

```

        WHERE id = ?'
    )->execute([$classroomId]);
}

function classroom_banner_mime_for_stored(string $storedName): string
{
    return match (strtolower(pathinfo($storedName, PATHINFO_EXTENSION))) {
        'jpg', 'jpeg' => 'image/jpeg',
        'png' => 'image/png',
        'webp' => 'image/webp',
        default => 'application/octet-stream',
    };
}

/**
 * Handle upload_banner / delete_banner POST for faculty manage-classroom pages.
 * Returns a flash message when handled, or null when the action is not a banner action.
 *
 * @param array<string,mixed>|null $classroom Updated in place on success
 */
function faculty_classroom_process_banner_post(int $classroomId, int $facultyId, ?array &$classroom): ?string
{
    if (($_SERVER['REQUEST_METHOD'] ?? '') !== 'POST' || $classroomId < 1 || $facultyId < 1 || !$classroom) {
        return null;
    }

    $action = (string) ($_POST['action'] ?? '');
    if ($action !== 'upload_banner' && $action !== 'delete_banner') {
        return null;
    }

    if (!classroom_banner_column_ready()) {
        throw new RuntimeException('Run upgrade_roles.php to enable classroom header backgrounds.');
```

```

        $title = (string) ($classroom['title'] ?? 'Classroom');
    }
    $metaExtra = trim((string) ($options['meta_extra'] ?? ''));
    $formId = preg_replace('/[^\a-zA-Z0-9_-]/', '', (string) ($options['form_id'] ?? 'facultyBanner')) ?: 'facultyBanner';
    $fileInputId = $formId . 'File';
    $uploadFormId = $formId . 'UploadForm';

    $courseCode = trim((string) ($classroom['course_code'] ?? ''));
    $courseName = trim((string) ($classroom['course_name'] ?? ''));
    $semester = trim((string) ($classroom['semester'] ?? ''));
    $schoolYear = trim((string) ($classroom['school_year'] ?? ''));

    require __DIR__ . '/faculty_classroom_banner.php';
}

function classroom_content_attachment_download_name(string $originalName, string $storedName = ''): string
{
    $name = trim($originalName);
    if ($name !== '') {
        return $name;
    }

    $storedName = basename(trim($storedName));
    if ($storedName === '') {
        return 'attachment';
    }

    $parts = explode('_', $storedName, 2);
    $fallback = trim($parts[1] ?? $storedName);
    return $fallback !== '' ? $fallback : 'attachment';
}

/**
 * @param array<string,mixed> $files
 * @return list<array{name:string,type:string,tmp_name:string,error:int,size:int}>
 */
function classroom_content_normalize_uploads(array $files): array
{
    $normalized = [];
    $names = $files['name'] ?? null;

    if (is_array($names)) {
        $count = count($names);
        for ($i = 0; $i < $count; $i++) {
            $normalized[] = [
                'name' => (string) ($files['name'][$i] ?? ''),
                'type' => (string) ($files['type'][$i] ?? ''),
                'tmp_name' => (string) ($files['tmp_name'][$i] ?? ''),
                'error' => (int) ($files['error'][$i] ?? UPLOAD_ERR_NO_FILE),
                'size' => (int) ($files['size'][$i] ?? 0),
            ];
        }

        return $normalized;
    }

    if ($names === null) {
        return [];
    }

    return [
        'name' => (string) ($files['name'] ?? ''),
        'type' => (string) ($files['type'] ?? ''),
        'tmp_name' => (string) ($files['tmp_name'] ?? ''),
        'error' => (int) ($files['error'] ?? UPLOAD_ERR_NO_FILE),
        'size' => (int) ($files['size'] ?? 0),
    ];
}

/**
 * @param array<string,mixed> $file
 * @return string|null
 */
function classroom_content_store_attachment(array $file): ?string
{
    if (($file['error'] ?? UPLOAD_ERR_NO_FILE) === UPLOAD_ERR_NO_FILE) {
        return null;
    }

    if (($file['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_OK) {
        throw new RuntimeException('Attachment upload failed.');
```

```

}

$size = (int) ($file['size'] ?? 0);
if ($size < 1) {
    throw new RuntimeException('Attachment is empty.');
```

```

}
if ($size > 10 * 1024 * 1024) {
    throw new RuntimeException('Attachment is too large (max 10 MB).');
```

```

}
$original = trim((string) ($file['name'] ?? ''));
if ($original === '') {
    throw new RuntimeException('Invalid attachment name.');
```

```

}
$original = str_replace(["\r", "\n"], '', basename($original));
$extension = strtolower(pathinfo($original, PATHINFO_EXTENSION));
$allowed = ['pdf', 'doc', 'docx', 'xls', 'xlsx', 'ppt', 'pptx', 'txt', 'csv', 'jpg', 'jpeg', 'png'];
if (!in_array($extension, $allowed, true)) {
    throw new RuntimeException('Unsupported attachment type.');
```

```

}
$baseName = pathinfo($original, PATHINFO_FILENAME);
$safeBaseName = preg_replace('/[^A-Za-z0-9._-]+/', '_', $baseName) ?? '';
$safeBaseName = trim($safeBaseName, '._-');
if ($safeBaseName === '') {
    $safeBaseName = 'attachment';
}
$safeBaseName = substr($safeBaseName, 0, 80);
```

```

$downloadName = $safeBaseName . ($extension !== '' ? '.' . $extension : '');
$storedName = bin2hex(random_bytes(16)) . '___' . $downloadName;
```

```

$dir = classroom_content_attachment_dir();
if (!is_dir($dir) && !mkdir($dir, 0775, true) && !is_dir($dir)) {
    throw new RuntimeException('Unable to create attachment directory.');
```

```

}
$destination = $dir . DIRECTORY_SEPARATOR . $storedName;
if (!move_uploaded_file((string) ($file['tmp_name'] ?? ''), $destination)) {
    throw new RuntimeException('Failed to save attachment.');
```

```

}
return classroom_content_attachment_relative_dir() . '/' . $storedName;
}

```

```

/**
 * @param array<string,mixed> $files
 * @return list<array{original_name:string,stored_name:string,mime:string}>
 */

```

```

function classroom_content_store_attachments(array $files): array
{

```

```

    $attachments = [];
    foreach (classroom_content_normalize_uploads($files) as $file) {
        if (($file['error'] ?? UPLOAD_ERR_NO_FILE) === UPLOAD_ERR_NO_FILE) {
            continue;
        }

```

```

        $resourceUrl = classroom_content_store_attachment($file);
        if ($resourceUrl === null) {
            continue;
        }

```

```

        $attachments[] = [
            'original_name' => classroom_content_attachment_name($resourceUrl),
            'stored_name' => basename(str_replace('\\', '/', $resourceUrl)),
            'mime' => trim((string) ($file['type'] ?? 'application/octet-stream')) ?: 'application/octet-stream',
        ];
    }

```

```

    return $attachments;
}

```

```

/**
 * @param array<int,mixed> $contentIds
 * @return array<int,list<array{id:int,content_id:int,original_name:string,stored_name:string,mime:string}>>
 */

```

```

function classroom_content_attachment_map(array $contentIds): array
{

```

```

    $ids = array_values(array_unique(array_filter(array_map('intval', $contentIds), static fn (int $id): bool => $id > 0)));

```

```

    if ($ids === []) {
        return [];
    }

    $placeholders = implode(',', array_fill(0, count($ids), '?'));
    $st = db()->prepare(
        "SELECT id, content_id, original_name, stored_name, mime
        FROM classroom_content_attachments
        WHERE content_id IN ($placeholders)
        ORDER BY created_at ASC, id ASC"
    );
    $st->execute($ids);

    $map = [];
    foreach ($st->fetchAll() as $row) {
        $contentId = (int) ($row['content_id'] ?? 0);
        if ($contentId < 1) {
            continue;
        }

        $map[$contentId][] = [
            'id' => (int) ($row['id'] ?? 0),
            'content_id' => $contentId,
            'original_name' => (string) ($row['original_name'] ?? ''),
            'stored_name' => (string) ($row['stored_name'] ?? ''),
            'mime' => (string) ($row['mime'] ?? ''),
        ];
    }

    return $map;
}

function classroom_content_attachment_extension(string $originalName, string $storedName = '', string $mime = ''): string
{
    foreach ([$originalName, $storedName] as $name) {
        $name = trim($name);
        if ($name === '') {
            continue;
        }
        $ext = strtolower(pathinfo($name, PATHINFO_EXTENSION));
        if ($ext !== '') {
            return $ext;
        }
    }

    $mime = strtolower(trim($mime));
    $mimeMap = [
        'text/plain' => 'txt',
        'text/csv' => 'csv',
        'application/pdf' => 'pdf',
        'application/vnd.openxmlformats-officedocument.wordprocessingml.document' => 'docx',
        'application/msword' => 'doc',
        'application/vnd.openxmlformats-officedocument.presentationml.presentation' => 'pptx',
        'application/vnd.ms-powerpoint' => 'ppt',
        'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet' => 'xlsx',
        'application/vnd.ms-excel' => 'xls',
    ];

    return $mimeMap[$mime] ?? '';
}

/**
 * @param array<string,mixed> $item
 * @param list<array{id:int,content_id:int,original_name:string,stored_name:string,mime:string}> $extraAttachments
 * @return list<array{original_name:string,stored_name:string,mime:string,extension:string}>
 */
function classroom_content_item_attachment_files(array $item, array $extraAttachments = []): array
{
    $files = [];

    $resourceUrl = trim((string) ($item['resource_url'] ?? ''));
    if ($resourceUrl !== '' && classroom_content_is_attachment($resourceUrl)) {
        $storedName = basename(str_replace('\\', '/', $resourceUrl));
        $originalName = classroom_content_attachment_name($resourceUrl);
        $files[] = [
            'original_name' => $originalName,
            'stored_name' => $storedName,
            'mime' => '',
            'extension' => classroom_content_attachment_extension($originalName, $storedName),
        ];
    }
}

```

```

}

foreach ($extraAttachments as $attachment) {
    if (!is_array($attachment)) {
        continue;
    }
    $storedName = trim((string) ($attachment['stored_name'] ?? ''));
    if ($storedName === '') {
        continue;
    }
    $originalName = classroom_content_attachment_download_name(
        (string) ($attachment['original_name'] ?? ''),
        $storedName
    );
    $files[] = [
        'original_name' => $originalName,
        'stored_name' => $storedName,
        'mime' => trim((string) ($attachment['mime'] ?? '')),
        'extension' => classroom_content_attachment_extension(
            $originalName,
            $storedName,
            (string) ($attachment['mime'] ?? '')
        ),
    ];
}

$seen = [];
$unique = [];
foreach ($files as $file) {
    $key = strtolower((string) ($file['stored_name'] ?? ''));
    if ($key === '' || isset($seen[$key])) {
        continue;
    }
    $seen[$key] = true;
    $unique[] = $file;
}

return $unique;
}

function classroom_content_attachment_extract_plain_text(string $storedName, string $originalName = '', string $mime = '', int $maxChars = 4000): string
{
    $storedName = basename(str_replace('\\', '/', trim($storedName)));
    if ($storedName === '') {
        return '';
    }

    $path = classroom_content_attachment_storage_path($storedName);
    if (!is_file($path) || !is_readable($path)) {
        return '';
    }

    $ext = classroom_content_attachment_extension($originalName, $storedName, $mime);
    $text = match ($ext) {
        'txt', 'csv', 'md' => classroom_content_attachment_read_text_file($path, $maxChars),
        'docx' => classroom_content_attachment_extract_docx_text($path, $maxChars),
        'pptx' => classroom_content_attachment_extract_pptx_text($path, $maxChars),
        'xlsx' => classroom_content_attachment_extract_xlsx_text($path, $maxChars),
        'pdf' => classroom_content_attachment_extract_pdf_text($path, $maxChars),
        default => '',
    };

    return classroom_content_attachment_normalize_extracted_text($text, $maxChars);
}

function classroom_content_attachment_read_text_file(string $path, int $maxChars): string
{
    $size = filesize($path);
    if ($size === false || $size < 1 || $size > 5_000_000) {
        return '';
    }

    $raw = file_get_contents($path);
    if ($raw === false || $raw === '') {
        return '';
    }

    if (!mb_check_encoding($raw, 'UTF-8')) {
        $converted = mb_convert_encoding($raw, 'UTF-8', 'UTF-8, ISO-8859-1, Windows-1252');
    }

```



```

        $raw = is_string($converted) ? $converted : $raw;
    }

    return mb_substr(trim($raw), 0, $maxChars, 'UTF-8');
}

function classroom_content_attachment_extract_docx_text(string $path, int $maxChars): string
{
    if (!class_exists('ZipArchive')) {
        return '';
    }

    $zip = new ZipArchive();
    if ($zip->open($path) !== true) {
        return '';
    }

    $xml = $zip->getFromName('word/document.xml');
    $zip->close();
    if ($xml === false || $xml === '') {
        return '';
    }

    $text = classroom_content_attachment_xml_to_text($xml);

    return mb_substr($text, 0, $maxChars, 'UTF-8');
}

function classroom_content_attachment_extract_pptx_text(string $path, int $maxChars): string
{
    if (!class_exists('ZipArchive')) {
        return '';
    }

    $zip = new ZipArchive();
    if ($zip->open($path) !== true) {
        return '';
    }

    $chunks = [];
    for ($i = 0; $i < $zip->numFiles; ++$i) {
        $name = (string) $zip->getNameIndex($i);
        if (!preg_match('#^ppt/slides/slide\d+\.xml$#', $name)) {
            continue;
        }
        $xml = $zip->getFromIndex($i);
        if ($xml !== false && $xml !== '') {
            $chunks[] = classroom_content_attachment_xml_to_text($xml);
        }
    }
    $zip->close();

    $text = trim(implode("\n", array_filter($chunks)));

    return mb_substr($text, 0, $maxChars, 'UTF-8');
}

function classroom_content_attachment_extract_xlsx_text(string $path, int $maxChars): string
{
    if (!class_exists('ZipArchive')) {
        return '';
    }

    $zip = new ZipArchive();
    if ($zip->open($path) !== true) {
        return '';
    }

    $sharedXml = $zip->getFromName('xl/sharedStrings.xml');
    $zip->close();
    if ($sharedXml === false || $sharedXml === '') {
        return '';
    }

    $text = classroom_content_attachment_xml_to_text($sharedXml);

    return mb_substr($text, 0, $maxChars, 'UTF-8');
}

function classroom_content_attachment_extract_pdf_text(string $path, int $maxChars): string

```

```

{
    $pdftotext = classroom_content_find_pdftotext_binary();
    if ($pdftotext !== null) {
        $cmd = escapeshellarg($pdftotext) . ' -layout -nopgbrk ' . escapeshellarg($path) . ' - 2>NUL';
        $output = shell_exec($cmd);
        if (is_string($output) && trim($output) !== '') {
            return classroom_content_attachment_normalize_extracted_text($output, $maxChars);
        }
    }

    $size = filesize($path);
    if ($size === false || $size < 1 || $size > 8_000_000) {
        return '';
    }

    $raw = file_get_contents($path);
    if ($raw === false || $raw === '') {
        return '';
    }

    $parts = [];
    if (preg_match_all('/\s(?:[^\s\\\r\n]{3,200})\s/s', $raw, $matches) === 1 && isset($matches[1])) {
        foreach ($matches[1] as $part) {
            $part = str_replace(['\\(', '\\)', '\\n', '\\r', '\\t'], ['(', ')', "\n", "\r", "\t"], $part);
            $part = trim($part);
            if ($part !== '' && preg_match('/[p{L}p{N}]/u', $part) === 1) {
                $parts[] = $part;
            }
        }
    }

    return classroom_content_attachment_normalize_extracted_text(implode(' ', $parts), $maxChars);
}

function classroom_content_find_pdftotext_binary(): ?string
{
    static $cached = null;
    if ($cached !== null) {
        return $cached !== '' ? $cached : null;
    }

    $candidates = ['pdftotext'];
    if (DIRECTORY_SEPARATOR === '\\') {
        $candidates[] = 'C:\\Program Files\\poppler\\Library\\bin\\pdftotext.exe';
        $candidates[] = 'C:\\poppler\\Library\\bin\\pdftotext.exe';
    }

    foreach ($candidates as $candidate) {
        $cmd = escapeshellarg($candidate) . ' -v 2>NUL';
        $result = shell_exec($cmd);
        if (is_string($result) && strpos($result, 'pdftotext') !== false) {
            $cached = $candidate;
        }
    }

    return $cached;
}

return null;
}

function classroom_content_attachment_xml_to_text(string $xml): string
{
    $previous = libxml_use_internal_errors(true);
    $dom = new DOMDocument();
    $loaded = $dom->loadXML($xml, LIBXML_NONET | LIBXML_NOERROR | LIBXML_NOWARNING);
    libxml_clear_errors();
    libxml_use_internal_errors($previous);

    if (!$loaded) {
        $text = strip_tags($xml);

        return classroom_content_attachment_normalize_extracted_text($text, 8000);
    }

    $chunks = [];
    $nodes = $dom->getElementsByTagName('*');
    foreach ($nodes as $node) {
        if (!$node instanceof DOMElement) {

```

```

        continue;
    }
    if (!in_array($node->localName, ['t', 'p', 'text'], true)) {
        continue;
    }
    $value = trim($node->textContent ?? '');
    if ($value !== '') {
        $chunks[] = $value;
    }
}

return classroom_content_attachment_normalize_extracted_text(implode("\n", $chunks), 8000);
}

function classroom_content_attachment_normalize_extracted_text(string $text, int $maxChars): string
{
    $text = html_entity_decode($text, ENT_QUOTES | ENT_HTML5, 'UTF-8');
    $text = preg_replace('/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]/u', ' ', $text) ?? $text;
    $text = preg_replace('/\s+/u', ' ', $text) ?? $text;
    $text = trim($text);
    if ($text === '') {
        return '';
    }

    return mb_strlen($text, 'UTF-8') > $maxChars
        ? mb_substr($text, 0, max(0, $maxChars - 1), 'UTF-8') . '...'
        : $text;
}

/**
 * @param list<array{original_name:string,stored_name:string,mime:string,extension:string}> $attachments
 */
function classroom_content_attachment_files_extract_text(array $attachments, int $maxCharsPerFile = 2500): string
{
    $lines = [];
    foreach ($attachments as $attachment) {
        $name = trim((string) ($attachment['original_name'] ?? 'Attachment'));
        $text = classroom_content_attachment_extract_plain_text(
            (string) ($attachment['stored_name'] ?? ''),
            $name,
            (string) ($attachment['mime'] ?? ''),
            $maxCharsPerFile
        );
        if ($text === '') {
            continue;
        }
        $lines[] = 'Attachment "' . $name . '": ' . $text;
    }

    return trim(implode("\n", $lines));
}

/**
 * @param list<array<string,mixed>> $items
 * @return list<array<string,mixed>>
 */
function classroom_content_enrich_week_items(array $items): array
{
    if ($items === []) {
        return [];
    }

    $contentIds = [];
    foreach ($items as $item) {
        $id = (int) ($item['id'] ?? 0);
        if ($id > 0) {
            $contentIds[] = $id;
        }
    }

    $attachmentMap = classroom_content_attachment_map($contentIds);
    $enriched = [];

    foreach ($items as $item) {
        $contentId = (int) ($item['id'] ?? 0);
        $attachments = classroom_content_item_attachment_files($item, $attachmentMap[$contentId] ?? []);
        $item['attachments'] = $attachments;
        $item['attachment_count'] = count($attachments);
        $item['attachment_text'] = classroom_content_attachment_files_extract_text($attachments);
        $enriched[] = $item;
    }
}

```

```

    }

    return $enriched;
}

function classroom_content_extract_alignment(string $style): string
{
    if (preg_match('/text-align\s*:\s*(left|center|right|justify)/i', $style, $matches) != 1) {
        return '';
    }

    return strtolower((string) ($matches[1] ?? ''));
}

function classroom_content_extract_font_size(string $style): string
{
    if (preg_match('/font-size\s*:\s*(\d+(?:\.\d+)?(?:px|pt|rem|em))/i', $style, $matches) != 1) {
        return '';
    }

    $size = strtolower((string) ($matches[1] ?? ''));
    if (preg_match('/^(\d+(?:\.\d+)?)(px|pt|rem|em)$/', $size, $parts) != 1) {
        return '';
    }

    $num = (float) $parts[1];
    $unit = (string) $parts[2];
    if ($unit === 'px' && ($num < 8 || $num > 72)) {
        return '';
    }
    if ($unit === 'pt' && ($num < 6 || $num > 54)) {
        return '';
    }
    if ($unit === 'rem' && ($num < 0.5 || $num > 4)) {
        return '';
    }
    if ($unit === 'em' && ($num < 0.5 || $num > 4)) {
        return '';
    }

    return $num . $unit;
}

function classroom_content_extract_font_family(string $style): string
{
    if (preg_match('/font-family\s*:\s*(^[^;]+)/i', $style, $matches) != 1) {
        return '';
    }

    $family = trim((string) ($matches[1] ?? ''));
    if ($family === '') {
        return '';
    }

    $allowed = [
        'arial, helvetica, sans-serif',
        "'times new roman', times, serif",
        'georgia, serif',
        "'courier new', courier, monospace",
        'verdana, sans-serif',
        'tahoma, sans-serif',
    ];
    $normalized = strtolower(str_replace("'", "", $family));
    foreach ($allowed as $allowedFamily) {
        if ($normalized === $allowedFamily) {
            return $family;
        }
    }

    return '';
}

function classroom_content_build_text_style(string $alignment, string $fontSize, string $fontFamily): string
{
    $parts = [];
    if ($alignment !== '') {
        $parts[] = 'text-align:' . $alignment;
    }
    if ($fontSize !== '') {
        $parts[] = 'font-size:' . $fontSize;
    }

```

```

    }
    if ($fontFamily !== '') {
        $parts[] = 'font-family:' . $fontFamily;
    }

    return $parts !== [] ? implode(';', $parts) . ';' : '';
}

function classroom_content_font_size_from_legacy(int $size): string
{
    return match ($size) {
        1 => '10px',
        2 => '12px',
        3 => '14px',
        4 => '16px',
        5 => '18px',
        6 => '24px',
        7 => '32px',
        default => '',
    };
}

function classroom_content_sanitize_html_node(DOMNode $node, DOMDocument $doc): ?DOMNode
{
    if ($node instanceof DOMText) {
        return $doc->createTextNode($node->nodeValue);
    }

    if (!$node instanceof DOMElement) {
        return null;
    }

    $tag = strtolower($node->tagName);
    $allowedTags = [
        'p' => true,
        'div' => true,
        'br' => true,
        'strong' => true,
        'b' => true,
        'em' => true,
        'i' => true,
        'u' => true,
        's' => true,
        'ul' => true,
        'ol' => true,
        'li' => true,
        'blockquote' => true,
        'a' => true,
        'h2' => true,
        'h3' => true,
        'img' => true,
        'span' => true,
        'font' => true,
    ];

    if (!isset($allowedTags[$tag])) {
        $fragment = $doc->createDocumentFragment();
        foreach ($node->childNodes as $child) {
            $cleanChild = classroom_content_sanitize_html_node($child, $doc);
            if ($cleanChild !== null) {
                $fragment->appendChild($cleanChild);
            }
        }

        return $fragment;
    }

    if ($tag === 'font') {
        $styleAttr = (string) $node->getAttribute('style');
        $fontSize = classroom_content_extract_font_size($styleAttr);
        $fontFamily = classroom_content_extract_font_family($styleAttr);
        if ($fontFamily === '') {
            $face = trim((string) $node->getAttribute('face'));
            if ($face !== '') {
                $fontFamily = classroom_content_extract_font_family('font-family:' . $face);
            }
        }
        if ($fontSize === '') {
            $legacySize = (int) $node->getAttribute('size');
            if ($legacySize > 0) {

```

```

        $fontSize = classroom_content_font_size_from_legacy($legacySize);
    }
}

$style = classroom_content_build_text_style('', $fontSize, $fontFamily);
if ($style === '') {
    $fragment = $doc->createDocumentFragment();
    foreach ($node->childNodes as $child) {
        $cleanChild = classroom_content_sanitize_html_node($child, $doc);
        if ($cleanChild !== null) {
            $fragment->appendChild($cleanChild);
        }
    }

    return $fragment;
}

$clean = $doc->createElement('span');
$clean->setAttribute('style', $style);
foreach ($node->childNodes as $child) {
    $cleanChild = classroom_content_sanitize_html_node($child, $doc);
    if ($cleanChild !== null) {
        $clean->appendChild($cleanChild);
    }
}

return $clean;
}

$clean = $doc->createElement($tag === 'span' ? 'span' : $tag);

if ($tag === 'a') {
    $href = trim((string) $node->getAttribute('href'));
    $validatedHref = filter_var($href, FILTER_VALIDATE_URL);
    $scheme = strtolower((string) (parse_url((string) $validatedHref, PHP_URL_SCHEME) ?? ''));
    if ($validatedHref !== false && in_array($scheme, ['http', 'https'], true)) {
        $clean->setAttribute('href', $validatedHref);
        $clean->setAttribute('target', '_blank');
        $clean->setAttribute('rel', 'noopener noreferrer');
    }
}

if ($tag === 'img') {
    $safeSrc = classroom_content_sanitize_img_src((string) $node->getAttribute('src'));
    if ($safeSrc === null) {
        return null;
    }
    $clean->setAttribute('src', $safeSrc);
    $alt = trim((string) $node->getAttribute('alt'));
    if ($alt !== '') {
        if (function_exists('mb_substr')) {
            $alt = mb_substr($alt, 0, 200);
        } else {
            $alt = substr($alt, 0, 200);
        }
        $clean->setAttribute('alt', $alt);
    } else {
        $clean->setAttribute('alt', '');
    }
    $clean->setAttribute('loading', 'lazy');
    $clean->setAttribute('decoding', 'async');
    $clean->setAttribute('style', 'max-width:100%;height:auto;');

    return $clean;
}

if ($tag === 'span') {
    $fontSize = classroom_content_extract_font_size((string) $node->getAttribute('style'));
    $fontFamily = classroom_content_extract_font_family((string) $node->getAttribute('style'));
    $style = classroom_content_build_text_style('', $fontSize, $fontFamily);
    if ($style === '') {
        $fragment = $doc->createDocumentFragment();
        foreach ($node->childNodes as $child) {
            $cleanChild = classroom_content_sanitize_html_node($child, $doc);
            if ($cleanChild !== null) {
                $fragment->appendChild($cleanChild);
            }
        }

        return $fragment;
    }
}

```

```

    }
    $clean->setAttribute('style', $style);
    foreach ($node->childNodes as $child) {
        $cleanChild = classroom_content_sanitize_html_node($child, $doc);
        if ($cleanChild !== null) {
            $clean->appendChild($cleanChild);
        }
    }
}

return $clean;
}

if (in_array($tag, ['p', 'div', 'li', 'blockquote', 'h2', 'h3'], true)) {
    $styleAttr = (string) $node->getAttribute('style');
    $alignment = classroom_content_extract_alignment($styleAttr);
    if ($alignment === '') {
        $alignment = classroom_content_extract_alignment('text-align:' . (string) $node->getAttribute('align'));
    }
    $fontSize = classroom_content_extract_font_size($styleAttr);
    $fontFamily = classroom_content_extract_font_family($styleAttr);
    $style = classroom_content_build_text_style($alignment, $fontSize, $fontFamily);
    if ($style !== '') {
        $clean->setAttribute('style', $style);
    }
}

foreach ($node->childNodes as $child) {
    $cleanChild = classroom_content_sanitize_html_node($child, $doc);
    if ($cleanChild !== null) {
        $clean->appendChild($cleanChild);
    }
}

return $clean;
}

function classroom_content_sanitize_html(string $html): string
{
    $html = trim($html);
    if ($html === '') {
        return '';
    }

    if (!class_exists('DOMDocument')) {
        return nl2br(htmlspecialchars(strip_tags($html)));
    }

    $previousErrors = libxml_use_internal_errors(true);

    $source = new DOMDocument('1.0', 'UTF-8');
    $wrapper = '<div data-classroom-content-root="1">' . $html . '</div>';
    $source->loadHTML(
        '<?xml encoding="utf-8" ?>' . $wrapper,
        LIBXML_HTML_NOIMPLIED | LIBXML_HTML_NODEFDTD
    );

    $xpath = new DOMXPath($source);
    $root = $xpath->query('//*[@data-classroom-content-root="1"]')->item(0);

    $cleanDoc = new DOMDocument('1.0', 'UTF-8');
    $cleanRoot = $cleanDoc->createElement('div');
    $cleanDoc->appendChild($cleanRoot);

    if ($root instanceof DOMElement) {
        foreach ($root->childNodes as $child) {
            $cleanChild = classroom_content_sanitize_html_node($child, $cleanDoc);
            if ($cleanChild !== null) {
                $cleanRoot->appendChild($cleanChild);
            }
        }
    }

    $output = '';
    foreach ($cleanRoot->childNodes as $child) {
        $output .= $cleanDoc->saveHTML($child);
    }

    libxml_clear_errors();
    libxml_use_internal_errors($previousErrors);
}

```

```

        return trim($output);
    }

function classroom_content_prepare_body(?string $body): ?string
{
    $raw = trim((string) $body);
    if ($raw === '') {
        return null;
    }

    $sanitized = classroom_content_sanitize_html($raw);
    if ($sanitized === '') {
        return null;
    }

    $plain = html_entity_decode(
        strip_tags(str_replace('&nbsp;', ' ', $sanitized)),
        ENT_QUOTES | ENT_HTML5,
        'UTF-8'
    );
    $plain = preg_replace('/\s+/u', ' ', $plain ?? '');
    $hasImage = preg_match('/<img\b/i', $sanitized) === 1;

    return trim($plain) === '' && !$hasImage ? null : $sanitized;
}

function classroom_content_render_body(?string $body): string
{
    $body = trim((string) $body);
    if ($body === '') {
        return '';
    }

    if (preg_match('/<\s*\/?\s*[a-z][^>]*>/i', $body) === 1) {
        return classroom_content_sanitize_html($body);
    }

    return nl2br(htmlspecialchars($body));
}

function classroom_content_week_label(?string $weeks): string
{
    $label = trim((string) $weeks);
    return $label !== '' ? $label : 'General resources';
}

/**
 * @param array<int,array<string,mixed>> $items
 * @return array<int,array{label:string,count:int,items:array<int,array<string,mixed>>>>
 */
function classroom_content_group_by_week(array $items): array
{
    $groups = [];
    foreach ($items as $item) {
        $label = classroom_content_week_label((string) ($item['weeks'] ?? ''));
        if (!isset($groups[$label])) {
            $groups[$label] = [
                'label' => $label,
                'count' => 0,
                'items' => [],
            ];
        }

        $groups[$label]['items'][] = $item;
        $groups[$label]['count']++;
    }

    return array_values($groups);
}

/**
 * @return list<array{id:int,title:string,course_code:string,course_name:string,semester:string,school_year:string}>
 */
function faculty_owned_classrooms(int $facultyId, ?int $excludeClassroomId = null): array
{
    if ($facultyId < 1 || !db_table_exists('online_classrooms')) {
        return [];
    }

    $sql = 'SELECT oc.id, oc.title, c.course_code, c.course_name, s.semester, s.school_year

```



```

        FROM online_classrooms oc
        INNER JOIN schedules s ON s.id = oc.schedule_id
        INNER JOIN courses c ON c.id = oc.course_id
        WHERE oc.faculty_id = ? AND s.faculty_id = ?';
$params = [$facultyId, $facultyId];
if ($excludeClassroomId !== null && $excludeClassroomId > 0) {
    $sql .= ' AND oc.id <> ?';
    $params[] = $excludeClassroomId;
}
$sql .= ' ORDER BY s.school_year DESC, s.semester, c.course_code, oc.title';

$stmt = db()->prepare($sql);
$stmt->execute($params);

return $stmt->fetchAll() ?: [];
}

/**
 * @throws RuntimeException
 */
function classroom_content_clone_stored_file(string $storedName): string
{
    $storedName = basename(str_replace('\\', '/', trim($storedName)));
    if ($storedName === '') {
        throw new RuntimeException('Invalid attachment.');
```

```

if (!db_table_exists('classroom_content')) {
    throw new RuntimeException('Classroom content is not available.');
```

```

}

$hasContentAttachments = db_table_exists('classroom_content_attachments');
$hasContentWeeks = db_column_exists('classroom_content', 'weeks');
$hasContentDaysPerTopic = db_column_exists('classroom_content', 'days_per_topic');
$hasContentTopicSchedule = $hasContentWeeks && $hasContentDaysPerTopic;
$hasSyllabusCols = db_column_exists('online_classrooms', 'syllabus_stored_name');
```

```

$st = db()->prepare(
    'SELECT *
     FROM classroom_content
     WHERE id = ? AND classroom_id = ? AND faculty_id = ?
     LIMIT 1'
);
$st->execute([$contentId, $sourceClassroomId, $facultyId]);
$source = $st->fetch();
if (!$source) {
    throw new RuntimeException('Content item not found.');
```

```

}

$st = db()->prepare(
    'SELECT oc.id, oc.title, oc.syllabus_stored_name, c.course_code, c.course_name
     FROM online_classrooms oc
     INNER JOIN schedules s ON s.id = oc.schedule_id
     INNER JOIN courses c ON c.id = oc.course_id
     WHERE oc.id = ? AND oc.faculty_id = ? AND s.faculty_id = ?
     LIMIT 1'
);
$st->execute([$targetClassroomId, $facultyId, $facultyId]);
$target = $st->fetch();
if (!$target) {
    throw new RuntimeException('Target classroom not found or you do not have access to it.');
```

```

}

if ($hasSyllabusCols && trim((string) ($target['syllabus_stored_name'] ?? '')) === '') {
    $targetLabel = trim((string) ($target['course_code'] ?? ''));
    if ($targetLabel === '') {
        $targetLabel = trim((string) ($target['title'] ?? 'that class'));
    }
    throw new RuntimeException(
        'Upload a syllabus for ' . $targetLabel . ' before assigning content to it.'
    );
}

$clonedResourceUrl = classroom_content_clone_resource_url((string) ($source['resource_url'] ?? ''));

$attachmentRows = [];
if ($hasContentAttachments) {
    $st = db()->prepare(
        'SELECT original_name, stored_name, mime
         FROM classroom_content_attachments
         WHERE content_id = ?
         ORDER BY created_at ASC, id ASC'
    );
    $st->execute([$contentId]);
    $attachmentRows = $st->fetchAll() ?: [];
}

$clonedAttachments = [];
foreach ($attachmentRows as $row) {
    $clonedAttachments[] = [
        'original_name' => (string) ($row['original_name'] ?? ''),
        'stored_name' => classroom_content_clone_stored_file((string) ($row['stored_name'] ?? '')),
        'mime' => trim((string) ($row['mime'] ?? 'application/octet-stream')) ?: 'application/octet-stream',
    ];
}

db()->beginTransaction();
try {
    if ($hasContentTopicSchedule) {
        db()->prepare(
            'INSERT INTO classroom_content (classroom_id, faculty_id, content_type, title, body, weeks, days_per_topic, resource_url)
             VALUES (?, ?, ?, ?, ?, ?, ?, ?)'
        )->execute([
            $targetClassroomId,
            $facultyId,
            (string) ($source['content_type'] ?? 'material'),
            (string) ($source['title'] ?? ''),

```

```

        ($source['body'] ?? null) != null && (string) $source['body'] != '' ? (string) $source['body'] : null,
        (string) ($source['weeks'] ?? ''),
        (string) ($source['days_per_topic'] ?? ''),
        $clonedResourceUrl,
    ]);
} else {
    db()->prepare(
        'INSERT INTO classroom_content (classroom_id, faculty_id, content_type, title, body, resource_url)
        VALUES (?, ?, ?, ?, ?, ?)'
    )->execute([
        $targetClassroomId,
        $facultyId,
        (string) ($source['content_type'] ?? 'material'),
        (string) ($source['title'] ?? ''),
        ($source['body'] ?? null) != null && (string) $source['body'] != '' ? (string) $source['body'] : null,
        $clonedResourceUrl,
    ]);
}

$newContentId = (int) db()->lastInsertId();
if ($newContentId < 1) {
    throw new RuntimeException('Failed to assign content.');
```

```

}

if ($clonedAttachments != []) {
    $insertAttachment = db()->prepare(
        'INSERT INTO classroom_content_attachments (content_id, original_name, stored_name, mime)
        VALUES (?, ?, ?, ?)'
    );
    foreach ($clonedAttachments as $attachment) {
        $insertAttachment->execute([
            $newContentId,
            $attachment['original_name'],
            $attachment['stored_name'],
            $attachment['mime'],
        ]);
    }
}

db()->commit();

return $newContentId;
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }

    if ($clonedResourceUrl != null && classroom_content_is_attachment($clonedResourceUrl)) {
        $path = classroom_content_attachment_path($clonedResourceUrl);
        if (is_file($path)) {
            @unlink($path);
        }
    }
    foreach ($clonedAttachments as $attachment) {
        $path = classroom_content_attachment_storage_path((string) ($attachment['stored_name'] ?? ''));
        if (is_file($path)) {
            @unlink($path);
        }
    }

    throw $e;
}
}

/**
 * Copy a quiz or assignment (including its questions) into another owned classroom.
 *
 * @throws RuntimeException
 */
function classroom_assessment_copy_to_classroom(
    int $assessmentId,
    int $sourceClassroomId,
    int $targetClassroomId,
    int $facultyId
): int {
    if ($assessmentId < 1 || $sourceClassroomId < 1 || $targetClassroomId < 1 || $facultyId < 1) {
        throw new RuntimeException('Invalid assessment or classroom.');
```

```

    }
    if ($sourceClassroomId === $targetClassroomId) {
        throw new RuntimeException('Choose a different course than the current class.');
```

```

}
if (!db_table_exists('classroom_assessments') || !db_table_exists('classroom_assessment_questions')) {
    throw new RuntimeException('Assessments are not available.');
```

```

}
```

```

$hasCreditedWeek = db_column_exists('classroom_assessments', 'credited_week');
$hasTimeLimit = db_column_exists('classroom_assessments', 'time_limit_minutes');
```

```

$st = db()->prepare(
    'SELECT *
    FROM classroom_assessments
    WHERE id = ? AND classroom_id = ? AND faculty_id = ?
    LIMIT 1'
```

```

);
$st->execute([$assessmentId, $sourceClassroomId, $facultyId]);
$source = $st->fetch();
if (!$source) {
    throw new RuntimeException('Assessment not found.');
```

```

}
```

```

$st = db()->prepare(
    'SELECT oc.id, oc.title, c.course_code, c.course_name
    FROM online_classrooms oc
    INNER JOIN schedules s ON s.id = oc.schedule_id
    INNER JOIN courses c ON c.id = oc.course_id
    WHERE oc.id = ? AND oc.faculty_id = ? AND s.faculty_id = ?
    LIMIT 1'
```

```

);
$st->execute([$targetClassroomId, $facultyId, $facultyId]);
$target = $st->fetch();
if (!$target) {
    throw new RuntimeException('Target classroom not found or you do not have access to it.');
```

```

}
```

```

$st = db()->prepare(
    'SELECT question_type, question_text, options_json, answer_key, points, position, word_limit, char_limit, allow_steps
    FROM classroom_assessment_questions
    WHERE assessment_id = ?
    ORDER BY position ASC, id ASC'
```

```

);
$st->execute([$assessmentId]);
$questions = $st->fetchAll() ?: [];

$assessmentType = classroom_assessment_normalize_type((string) ($source['assessment_type'] ?? 'essay'));
$title = (string) ($source['title'] ?? '');
$description = ($source['description'] ?? null) !== null && (string) $source['description'] !== ''
    ? (string) $source['description']
    : null;
$totalPoints = (float) ($source['total_points'] ?? 0);
$dueAt = ($source['due_at'] ?? null) !== null && (string) $source['due_at'] !== ''
    ? (string) $source['due_at']
    : null;
$timeLimitMinutes = $hasTimeLimit && ($source['time_limit_minutes'] ?? null) !== null
    ? (int) $source['time_limit_minutes']
    : null;
$creditedWeek = $hasCreditedWeek ? (string) ($source['credited_week'] ?? '') : '';
```

```

db()->beginTransaction();
```

```

try {
```

```
    if ($hasCreditedWeek && $hasTimeLimit) {
```

```
        db()->prepare(
```

```
            'INSERT INTO classroom_assessments (classroom_id, faculty_id, assessment_type, title, description, total_points, due_at,
```

```
time_limit_minutes, credited_week)
```

```
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)'
```

```
        )->execute([
```

```
            $targetClassroomId,
            $facultyId,
            $assessmentType,
            $title,
            $description,
            $totalPoints,
            $dueAt,
            $timeLimitMinutes,
            $creditedWeek,
```

```
        ]);
```

```
    } elseif ($hasTimeLimit) {
```

```
        db()->prepare(
```

```
            'INSERT INTO classroom_assessments (classroom_id, faculty_id, assessment_type, title, description, total_points, due_at,
```

```
time_limit_minutes)
```

```
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)'
```

```

        )->execute([
            $targetClassroomId,
            $facultyId,
            $assessmentType,
            $title,
            $description,
            $totalPoints,
            $dueAt,
            $timeLimitMinutes,
        ]);
    } elseif ($hasCreditedWeek) {
        db()->prepare(
            'INSERT INTO classroom_assessments (classroom_id, faculty_id, assessment_type, title, description, total_points, due_at,
credited_week)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)'
        )->execute([
            $targetClassroomId,
            $facultyId,
            $assessmentType,
            $title,
            $description,
            $totalPoints,
            $dueAt,
            $creditedWeek,
        ]);
    } else {
        db()->prepare(
            'INSERT INTO classroom_assessments (classroom_id, faculty_id, assessment_type, title, description, total_points, due_at)
            VALUES (?, ?, ?, ?, ?, ?, ?)'
        )->execute([
            $targetClassroomId,
            $facultyId,
            $assessmentType,
            $title,
            $description,
            $totalPoints,
            $dueAt,
        ]);
    }
}

$newAssessmentId = (int) db()->lastInsertId();
if ($newAssessmentId < 1) {
    throw new RuntimeException('Failed to assign assessment.');
```

```

}

if ($questions !== []) {
    $qStmt = db()->prepare(
        'INSERT INTO classroom_assessment_questions
        (assessment_id, question_type, question_text, options_json, answer_key, points, position, word_limit, char_limit,
allow_steps)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)'
    );
    foreach ($questions as $question) {
        $qStmt->execute([
            $newAssessmentId,
            (string) ($question['question_type'] ?? 'essay'),
            (string) ($question['question_text'] ?? ''),
            ($question['options_json'] ?? null) !== null ? (string) $question['options_json'] : null,
            ($question['answer_key'] ?? null) !== null && (string) $question['answer_key'] !== ''
                ? (string) $question['answer_key']
                : null,
            (float) ($question['points'] ?? 0),
            (int) ($question['position'] ?? 1),
            ($question['word_limit'] ?? null) !== null && (string) $question['word_limit'] !== ''
                ? (int) $question['word_limit']
                : null,
            ($question['char_limit'] ?? null) !== null && (string) $question['char_limit'] !== ''
                ? (int) $question['char_limit']
                : null,
            !empty($question['allow_steps']) ? 1 : 0,
        ]);
    }
}

db()->commit();

return $newAssessmentId;
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
}

```

```

    }

    throw $e;
}
}

/**
 * Remove any UNIQUE indexes that include courses.course_code.
 * Returns number of dropped indexes.
 */
function ensure_course_code_duplicates_allowed(): int
{
    if (!db_table_exists('courses') || !db_column_exists('courses', 'course_code')) {
        return 0;
    }

    $st = db()->prepare(
        'SELECT DISTINCT INDEX_NAME
        FROM information_schema.STATISTICS
        WHERE TABLE_SCHEMA = ?
        AND TABLE_NAME = ?
        AND COLUMN_NAME = ?
        AND NON_UNIQUE = 0
        AND INDEX_NAME <> "PRIMARY"'
    );
    $st->execute([DB_NAME, 'courses', 'course_code']);
    $indexes = $st->fetchAll(PDO::FETCH_COLUMN) ?: [];

    $dropped = 0;
    foreach ($indexes as $idx) {
        $indexName = str_replace('`', '', (string) $idx);
        db()->exec("ALTER TABLE courses DROP INDEX `${$indexName}`");
        $dropped++;
    }

    // Keep a normal non-unique index for lookups/sorts.
    if ($dropped > 0) {
        db()->exec('ALTER TABLE courses ADD INDEX idx_course_code (course_code)');
    }

    return $dropped;
}

/**
 * Parse MySQL SET or comma string into array of day names.
 * @return string[]
 */
function parse_day_set(?string $value): array
{
    if ($value === null || $value === '') {
        return [];
    }
    $parts = preg_split('/\s*,\s*/', $value) ?: [];
    return array_values(array_filter(array_map('trim', $parts)));
}

/**
 * Abbreviate a weekday name for schedule display (MW/TTH-style).
 */
function abbreviate_day_name(string $day): string
{
    static $map = [
        'Monday' => 'M',
        'Tuesday' => 'T',
        'Wednesday' => 'W',
        'Thursday' => 'TH',
        'Friday' => 'F',
        'Saturday' => 'S',
        'Sunday' => 'Su',
    ];
    $day = trim($day);
    return $map[$day] ?? $day;
}

/**
 * Format a MySQL SET or comma-separated day string as abbreviated labels (e.g. "T, TH").
 */
function format_day_set_abbrev(?string $value): string
{
    $days = parse_day_set($value);

```

```

    if ($days == []) {
        return '';
    }
    return implode(' ', array_map('abbreviate_day_name', $days));
}

/**
 * Whether a student may record attendance login for a class at the given moment.
 *
 * @param array{day_of_week?: string|null, start_time?: string|null, end_time?: string|null} $scheduleRow
 * @return array{allowed: bool, reason: string, is_scheduled_day: bool, is_within_window: bool, session_start: string, session_end: string}
 */
function classroom_attendance_login_allowed(array $scheduleRow, ?int $timestamp = null): array
{
    $timestamp = $timestamp ?? time();
    $todayDate = date('Y-m-d', $timestamp);
    $nowStr = date('Y-m-d H:i:s', $timestamp);
    $dayName = date('l', $timestamp);

    $normalizeDayToken = static function (string $value): string {
        $lettersOnly = preg_replace('/[^a-z]/i', '', trim($value)) ?? '';
        return strtolower(substr($lettersOnly, 0, 3));
    };

    $scheduleDayTokens = [];
    foreach (parse_day_set((string) ($scheduleRow['day_of_week'] ?? '')) as $scheduledDay) {
        $token = $normalizeDayToken((string) $scheduledDay);
        if ($token !== '') {
            $scheduleDayTokens[$token] = true;
        }
    }
    $isScheduledDay = isset($scheduleDayTokens[$normalizeDayToken($dayName)]);

    $startTime = substr((string) ($scheduleRow['start_time'] ?? '00:00:00'), 0, 8);
    $endTime = substr((string) ($scheduleRow['end_time'] ?? '23:59:59'), 0, 8);
    $sessionStart = $todayDate . ' ' . $startTime;
    $sessionEnd = $todayDate . ' ' . $endTime;
    $isWithinWindow = $nowStr >= $sessionStart && $nowStr <= $sessionEnd;

    $reason = '';
    if ($scheduleDayTokens === []) {
        $reason = 'Class schedule days are not configured yet.';
    } elseif (!$isScheduledDay) {
        $reason = 'Class is not scheduled today.';
    } elseif ($nowStr < $sessionStart) {
        $reason = 'Class has not started yet. You can log in when class time begins.';
    } elseif ($nowStr > $sessionEnd) {
        $reason = 'Class time has already ended.';
    }

    return [
        'allowed' => $scheduleDayTokens !== [] && $isScheduledDay && $isWithinWindow,
        'reason' => $reason,
        'is_scheduled_day' => $isScheduledDay,
        'is_within_window' => $isWithinWindow,
        'session_start' => $sessionStart,
        'session_end' => $sessionEnd,
    ];
}

/**
 * Whether faculty marked a section live (within two hours, and still within today's class window when schedule is known).
 *
 * @param array{day_of_week?: string|null, start_time?: string|null, end_time?: string|null}|null $scheduleRow
 */
function schedule_is_faculty_live(?string $liveAt, ?int $timestamp = null, ?array $scheduleRow = null): bool
{
    if ($liveAt === null || $liveAt === '') {
        return false;
    }
    $timestamp = $timestamp ?? time();
    $t = strtotime($liveAt);
    if ($t === false) {
        return false;
    }
    if (($timestamp - $t) > 2 * 3600) {
        return false;
    }
    if ($scheduleRow !== null) {
        $window = classroom_attendance_login_allowed($scheduleRow, $timestamp);
    }
}

```

```

        if (!empty($window['is_scheduled_day']) && empty($window['is_within_window'])) {
            $sessionEndTs = strtotime($window['session_end']);
            if ($sessionEndTs !== false && $timestamp > $sessionEndTs) {
                return false;
            }
        }
    }
}

return true;
}

/**
 * LIVE badge helper: resolves display state and clears stale DB flags when class time or the two-hour window has passed.
 *
 * @param array{id?: int|string, faculty_id?: int|string, online_live_at?: string|null, day_of_week?: string|null, start_time?: string|null, end_time?: string|null} $scheduleRow
 */
function schedule_display_is_faculty_live(array $scheduleRow): bool
{
    $liveAt = isset($scheduleRow['online_live_at']) ? (string) $scheduleRow['online_live_at'] : '';
    if ($liveAt === '') {
        return false;
    }
    if (schedule_is_faculty_live($liveAt, null, $scheduleRow)) {
        return true;
    }

    $scheduleId = (int) ($scheduleRow['id'] ?? 0);
    $facultyId = (int) ($scheduleRow['faculty_id'] ?? 0);
    if ($scheduleId > 0 && $facultyId > 0) {
        faculty_end_live_for_schedule($scheduleId, $facultyId);
    }

    return false;
}

/** Clears online_live_at and closes any open classroom_live_sessions row for one faculty schedule. */
function faculty_end_live_for_schedule(int $scheduleId, int $facultyId): bool
{
    if ($scheduleId < 1 || $facultyId < 1 || !db_column_exists('schedules', 'online_live_at')) {
        return false;
    }

    $facultyCollegeId = faculty_college_id($facultyId);
    $scheduleCollegeClause = $facultyCollegeId !== null ? ' AND college_id=?' : '';
    $scheduleCollegeParam = $facultyCollegeId !== null ? [$facultyCollegeId] : [];

    $chk = db()->prepare(
        'SELECT online_live_at FROM schedules WHERE id=? AND faculty_id=?' . $scheduleCollegeClause . ' LIMIT 1'
    );
    $chk->execute(array_merge([$scheduleId, $facultyId], $scheduleCollegeParam));
    $liveAt = $chk->fetchColumn();
    if ($liveAt === false || $liveAt === null || $liveAt === '') {
        return false;
    }

    db()->prepare('UPDATE schedules SET online_live_at = NULL WHERE id=? AND faculty_id=?' . $scheduleCollegeClause)
    ->execute(array_merge([$scheduleId, $facultyId], $scheduleCollegeParam));

    if (db_table_exists('classroom_live_sessions')) {
        $st = db()->prepare('SELECT id FROM online_classrooms WHERE schedule_id = ? AND faculty_id = ? LIMIT 1');
        $st->execute([$scheduleId, $facultyId]);
        $classroomId = (int) ($st->fetchColumn() ?? 0);
        if ($classroomId > 0) {
            db()->prepare(
                'UPDATE classroom_live_sessions
                SET ended_at = NOW()
                WHERE classroom_id = ? AND schedule_id = ? AND faculty_id = ? AND ended_at IS NULL'
            )->execute([$classroomId, $scheduleId, $facultyId]);
        }
    }

    return true;
}

/**
 * Sync schedule URL, mark live during class window, and return the Meet URL for a faculty classroom.
 *
 * @throws RuntimeException
 */

```



```

function faculty_open_meet_and_go_live(int $classroomId, int $facultyId): string
{
    if ($classroomId < 1) {
        throw new RuntimeException('Classroom not found.');
```

```
    }

    $st = db()->prepare(
        'SELECT oc.id, oc.meet_link, oc.schedule_id, s.day_of_week, s.start_time, s.end_time, s.online_class_url
        FROM online_classrooms oc
        INNER JOIN schedules s ON s.id = oc.schedule_id
        WHERE oc.id = ? AND oc.faculty_id = ? AND s.faculty_id = ?
        LIMIT 1'
    );
    $st->execute([$classroomId, $facultyId, $facultyId]);
    $classroomRow = $st->fetch(PDO::FETCH_ASSOC);
    if (!$classroomRow) {
        throw new RuntimeException('Classroom not found or you do not have access to it.');
```

```
    }

    $rawMeet = trim((string) ($classroomRow['meet_link'] ?? ''));
    if ($rawMeet === '') {
        throw new RuntimeException('This classroom does not have a Meet link yet.');
```

```
    }

    $meetLink = filter_var($rawMeet, FILTER_VALIDATE_URL);
    if ($meetLink === false) {
        throw new RuntimeException('Please enter a valid Meet URL.');
```

```
    }

    $scheme = strtolower((string) (parse_url($meetLink, PHP_URL_SCHEME) ?? ''));
    if (!in_array($scheme, ['http', 'https'], true)) {
        throw new RuntimeException('Only http and https meeting links are allowed.');
```

```
    }

    $scheduleId = (int) $classroomRow['schedule_id'];
    $hasOnlineUrl = db_column_exists('schedules', 'online_class_url');
    $hasLiveAt = db_column_exists('schedules', 'online_live_at');
    $hasLiveSessions = db_table_exists('classroom_live_sessions');
```

```


    if ($hasOnlineUrl) {
        db()->prepare('UPDATE schedules SET online_class_url = ? WHERE id = ? AND faculty_id = ?')
        ->execute([$meetLink, $scheduleId, $facultyId]);
    }

    if ($hasLiveAt && $hasOnlineUrl) {
        $facultyCollegeId = faculty_college_id($facultyId);
        $scheduleCollegeClause = $facultyCollegeId !== null ? ' AND college_id=?' : '';
        $scheduleCollegeParam = $facultyCollegeId !== null ? [$facultyCollegeId] : [];

        db()->prepare('UPDATE schedules SET online_live_at = NOW() WHERE id=? AND faculty_id=?' . $scheduleCollegeClause)
        ->execute(array_merge([$scheduleId, $facultyId], $scheduleCollegeParam));

        if ($hasLiveSessions) {
            $st = db()->prepare(
                'SELECT id
                FROM classroom_live_sessions
                WHERE classroom_id = ? AND schedule_id = ? AND faculty_id = ? AND ended_at IS NULL
                ORDER BY id DESC
                LIMIT 1'
            );
            $st->execute([$classroomId, $scheduleId, $facultyId]);
            if ((int) ($st->fetchColumn() ?: 0) < 1) {
                db()->prepare(
                    'INSERT INTO classroom_live_sessions (classroom_id, schedule_id, faculty_id, started_at)
                    VALUES (?, ?, ?, NOW())'
                )->execute([$classroomId, $scheduleId, $facultyId]);
            }
        }
    }

    return $meetLink;
}

/**
 * Convert array of days to MySQL SET string.
 */
function days_to_set(array $days): string
{
    $allowed = array_flip(schedule_days_list());
    $clean = [];

```

```

        foreach ($days as $d) {
            $d = trim((string) $d);
            if (isset($allowed[$d])) {
                $clean[$d] = true;
            }
        }
        return implode(',', array_keys($clean));
    }
}

/**
 * Whether a GEN ED account may see/join live on a weekly schedule block.
 *
 * @param array<string, mixed> $scheduleRow college_id, course_id, course_is_gened, course_department, program, ge_target_program
 */
function weekly_schedule_gened_can_join_online_live(
    array $scheduleRow,
    int $viewCollegeFilter = 0,
    string $viewProgramFilter = ''
): bool {
    if (!db_column_exists('courses', 'is_gened') || (int) ($scheduleRow['course_is_gened'] ?? 0) !== 1) {
        return false;
    }

    $scheduleCollegeId = (int) ($scheduleRow['college_id'] ?? 0);
    $courseId = (int) ($scheduleRow['course_id'] ?? 0);
    $courseDepartment = trim((string) ($scheduleRow['course_department'] ?? ''));
    $scheduleProgram = trim((string) ($scheduleRow['program'] ?? ''));
    $geTargetProgram = trim((string) ($scheduleRow['ge_target_program'] ?? ''));

    if ($courseId > 0 && $scheduleCollegeId > 0 && db_table_exists('ge_course_colleges')) {
        $chk = db()->prepare('SELECT COUNT(*) FROM ge_course_colleges WHERE course_id = ? AND college_id = ?');
        $chk->execute([$courseId, $scheduleCollegeId]);
        if ((int) $chk->fetchColumn() < 1) {
            return false;
        }
    }

    if ($viewCollegeFilter > 0 && $viewProgramFilter !== '') {
        $programMatches = ($courseDepartment !== '' && strcasecmp($courseDepartment, $viewProgramFilter) === 0)
            || ($scheduleProgram !== '' && strcasecmp($scheduleProgram, $viewProgramFilter) === 0)
            || ($geTargetProgram !== '' && strcasecmp($geTargetProgram, $viewProgramFilter) === 0);

        return $scheduleCollegeId === $viewCollegeFilter && $programMatches;
    }

    return true;
}

/** Program scope label for college GE program chairs (see dean_gened_chair.php). */
function ge_program_chair_label(): string
{
    return 'General Education';
}

function is_ge_program_scope(?string $program): bool
{
    return $program !== null && strcasecmp(trim($program), ge_program_chair_label()) === 0;
}

/**
 * College of Arts and Sciences (CAS) – only this dean may join live GE sessions.
 */
function is_cas_college(?int $collegeId): bool
{
    if ($collegeId === null || $collegeId < 1) {
        return false;
    }

    static $cache = [];
    if (array_key_exists($collegeId, $cache)) {
        return $cache[$collegeId];
    }

    $st = db()->prepare('SELECT college_code, college_name FROM colleges WHERE id = ? LIMIT 1');
    $st->execute([$collegeId]);
    $row = $st->fetch(PDO::FETCH_ASSOC);
    if (!$row) {
        $cache[$collegeId] = false;

        return false;
    }

```

```

    }

    $code = strtoupper(trim((string) ($row['college_code'] ?? '')));
    $name = strtolower(trim((string) ($row['college_name'] ?? '')));
    $cache[$collegeId] = $code === 'CAS' || str_contains($name, 'arts and sciences');

    return $cache[$collegeId];
}

/** Whether the logged-in dean may join a live GE class session (CAS dean only). */
function dean_can_join_ge_live_session(): bool
{
    return is_dean() && is_cas_college(current_college_id());
}

/**
 * SQL AND-clause so deans see their college schedules plus cross-college loads and GE rows.
 *
 * @param list<int|string> $params
 */
function dean_schedule_scope_sql(
    int $collegeId,
    bool $hasCourseIsGenedCol,
    array &$params,
    bool $hasGeTargetsTable = false
): string {
    if ($collegeId < 1) {
        return '';
    }

    $parts = ['s.college_id = ?', 'f.college_id = ?', 'c.college_id = ?'];
    $params[] = $collegeId;
    $params[] = $collegeId;
    $params[] = $collegeId;
    if ($hasGeTargetsTable) {
        $parts[] = 'gst.college_id = ?';
        $params[] = $collegeId;
    }
    if ($hasCourseIsGenedCol) {
        $parts[] = 'COALESCE(c.is_gened, 0) = 1';
    }

    return ' AND (' . implode(' OR ', $parts) . ')';
}

/**
 * SQL AND-clause so program chairs see schedules for their program courses and their program faculty.
 *
 * @param list<int|string> $params
 */
function program_chair_schedule_scope_sql(
    int $collegeId,
    string $programScope,
    bool $hasGeTargetsTable,
    array &$params
): string {
    if ($collegeId < 1 || trim($programScope) === '') {
        return '';
    }

    $sql = ' AND (s.college_id = ? OR f.college_id = ? OR c.college_id = ?';
    $params[] = $collegeId;
    $params[] = $collegeId;
    $params[] = $collegeId;
    if ($hasGeTargetsTable) {
        $sql .= ' OR gst.college_id = ?';
        $params[] = $collegeId;
    }
    $sql .= ')';
    if ($hasGeTargetsTable) {
        $sql .= ' AND (f.department = ? OR c.department = ? OR gst.program_name = ?)';
        $params[] = $programScope;
        $params[] = $programScope;
        $params[] = $programScope;
    } else {
        $sql .= ' AND (f.department = ? OR c.department = ?)';
        $params[] = $programScope;
        $params[] = $programScope;
    }
}

```

```

        return $sql;
    }

/**
 * GE catalog courses offered to a college (read-only / scheduling reference).
 *
 * @return list<array<string, mixed>>
 */
function ge_courses_offered_to_college(int $collegeId): array
{
    if ($collegeId < 1 || !db_column_exists('courses', 'is_gened') || !db_table_exists('ge_course_colleges')) {
        return [];
    }

    $hasLab = db_column_exists('courses', 'is_laboratory');
    $hasYl = db_column_exists('courses', 'year_level');
    $hasSec = db_column_exists('courses', 'section');
    $hasLec = db_column_exists('courses', 'lecture_units');
    $hasLabU = db_column_exists('courses', 'laboratory_units');

    $extra = ($hasLab ? ', c.is_laboratory' : '')
        . ($hasYl ? ', c.year_level' : '')
        . ($hasSec ? ', c.section' : '')
        . ($hasLec ? ', c.lecture_units' : '')
        . ($hasLabU ? ', c.laboratory_units' : '');

    $st = db()->prepare(
        "SELECT c.id, c.course_code, c.course_name, c.department, c.units{$extra}
        FROM courses c
        INNER JOIN ge_course_colleges gcc ON gcc.course_id = c.id
        WHERE COALESCE(c.is_gened, 0) = 1 AND gcc.college_id = ?
        ORDER BY c.course_code"
    );
    $st->execute([$collegeId]);

    return $st->fetchAll(PDO::FETCH_ASSOC);
}

/**
 * How online/live appears on the weekly schedule for one block.
 *
 * @param array<string, mixed> $scheduleRow
 * @return 'normal'|'hidden'|'unauthorized'
 */
function weekly_schedule_online_live_mode(
    string $role,
    array $scheduleRow,
    int $viewCollegeFilter = 0,
    string $viewProgramFilter = ''
): string {
    if ($role === 'gened') {
        return weekly_schedule_gened_can_join_online_live($scheduleRow, $viewCollegeFilter, $viewProgramFilter)
            ? 'normal'
            : 'hidden';
    }

    $isGeCourse = db_column_exists('courses', 'is_gened') && (int) ($scheduleRow['course_is_gened'] ?? 0) === 1;

    if ($role === 'program_chair' && $isGeCourse) {
        return 'unauthorized';
    }

    if ($role === 'dean' && $isGeCourse) {
        return dean_can_join_ge_live_session() ? 'normal' : 'unauthorized';
    }

    return 'normal';
}

function time_to_minutes(string $time): int
{
    $parts = explode(':', $time);
    $h = (int) ($parts[0] ?? 0);
    $m = (int) ($parts[1] ?? 0);
    $s = (int) ($parts[2] ?? 0);
    return $h * 60 + $m + (int) round($s / 60);
}

/**
 * Stored lecture/laboratory kind from schedule create/edit (segment label).

```

```

*/
function schedule_segment_session_kind(string $label): string
{
    return strcasecmp(trim($label), 'Laboratory') === 0 ? 'laboratory' : 'lecture';
}

/**
 * Whether a scheduled block counts as laboratory (vs lecture) for load/contact hours.
 * Uses stored session_kind when present. Pure lecture courses are never labeled Laboratory
 * from duration alone (a one-day 3-hour lecture stays Lecture).
 * For laboratory courses with no session_kind, 1 lab unit = 3 hours → typical 3–4 h blocks.
 *
 * @param bool|null $courseIsLaboratory When false, duration heuristic is skipped (always lecture).
 */
function schedule_session_is_laboratory(
    ?string $startTime,
    ?string $endTime,
    ?string $roomType = null,
    ?string $sessionKind = null,
    ?bool $courseIsLaboratory = null
): bool {
    unset($roomType);
    $kind = strtolower(trim((string) $sessionKind));
    if ($kind === 'laboratory') {
        return true;
    }
    if ($kind === 'lecture') {
        return false;
    }

    // Pure lecture course: never treat a long block as lab just because of duration.
    if ($courseIsLaboratory === false) {
        return false;
    }

    $st = time_to_minutes(substr((string) $startTime, 0, 8));
    $en = time_to_minutes(substr((string) $endTime, 0, 8));
    $durationMinutes = $en - $st;
    if ($durationMinutes <= 0) {
        return false;
    }
    $hours = $durationMinutes / 60.0;

    return $hours >= 3.0 && $hours <= 4.0;
}

function minutes_to_time(int $minutes): string
{
    $h = intval($minutes, 60);
    $m = $minutes % 60;
    return sprintf('%02d:%02d:00', $h, $m);
}

/**
 * Check overlap of two [start,end) intervals in minutes.
 */
function intervals_overlap(int $s1, int $e1, int $s2, int $e2): bool
{
    return $s1 < $e2 && $s2 < $e1;
}

/**
 * @param string[] $days
 * @return array{ok:bool,errors:string[]}
 */
function validate_schedule_rules(
    string $scheduleType,
    array $days,
    string $startTime,
    string $endTime
): array {
    $errors = [];
    $daySet = array_flip($days);
    $boundsMin = time_to_minutes(TIME_MIN);
    $boundsMax = time_to_minutes(TIME_MAX);

    $st = time_to_minutes(substr($startTime, 0, 5) . ':00');
    $en = time_to_minutes(substr($endTime, 0, 5) . ':00');

    if ($st >= $en) {

```

```

        $errors[] = 'Start time must be before end time.';
    }
    if ($st < $boundsMin || $en > $boundsMax) {
        $errors[] = 'Schedule must fall between 6:00 AM and 10:00 PM.';
    }
    $dur = $en - $st;
    if ($dur < MIN_CLASS_MINUTES) {
        $errors[] = 'Each class must be at least ' . MIN_CLASS_MINUTES . ' minutes long.';
    }
    if ($dur > MAX_CLASS_BLOCK_HOURS * 60) {
        $errors[] = 'A single class block cannot exceed ' . MAX_CLASS_BLOCK_HOURS . ' hours.';
    }
}

switch ($scheduleType) {
    case 'MW':
        if (!isset($daySet['Monday']) || !isset($daySet['Wednesday'])) {
            $errors[] = 'MW schedule requires Monday and Wednesday.';
        }
        break;
    case 'TTH':
        if (!isset($daySet['Tuesday']) || !isset($daySet['Thursday'])) {
            $errors[] = 'TTH schedule requires Tuesday and Thursday.';
        }
        break;
    case 'Saturday':
        if (count($days) != 1 || !isset($daySet['Saturday'])) {
            $errors[] = 'Saturday schedule must select Saturday only.';
        }
        break;
    case 'Sunday':
        if (count($days) != 1 || !isset($daySet['Sunday'])) {
            $errors[] = 'Sunday schedule must select Sunday only.';
        }
        break;
    case 'MWF':
        foreach (['Monday', 'Wednesday', 'Friday'] as $req) {
            if (!isset($daySet[$req])) {
                $errors[] = 'MWF schedule requires Monday, Wednesday, and Friday.';
                break;
            }
        }
        break;
    case 'TTHS':
        foreach (['Tuesday', 'Thursday', 'Saturday'] as $req) {
            if (!isset($daySet[$req])) {
                $errors[] = 'TTHS schedule requires Tuesday, Thursday, and Saturday.';
                break;
            }
        }
        break;
    case 'MW_TTH':
        $mw = isset($daySet['Monday'], $daySet['Wednesday']);
        $tth = isset($daySet['Tuesday'], $daySet['Thursday']);
        if (!$mw || !$tth) {
            $errors[] = 'MW_TTH requires Monday & Wednesday and Tuesday & Thursday.';
        }
        break;
    case 'Custom':
    default:
        break;
}

return ['ok' => $errors === [], 'errors' => $errors];
}

/**
 * Fetch schedules for faculty on a given day (same semester/school year), optional exclude id.
 * @return array<int,array<string,mixed>>
 */
function fetch_faculty_day_schedules(
    int $facultyId,
    string $day,
    string $semester,
    string $schoolYear,
    ?int $excludeId = null
): array {
    $sql = "SELECT id, start_time, end_time FROM schedules
        WHERE faculty_id = ? AND semester = ? AND school_year = ?
        AND FIND_IN_SET(?, day_of_week) > 0";
    $params = [$facultyId, $semester, $schoolYear, $day];

```

```

        if ($excludeId !== null) {
            $sql .= ' AND id != ?';
            $params[] = $excludeId;
        }
        $stmt = db()->prepare($sql);
        $stmt->execute($params);
        return $stmt->fetchAll();
    }

/**
 * Fetch schedules for room on a given day.
 * @return array<int,array<string,mixed>>
 */
function fetch_room_day_schedules(
    int $roomId,
    string $day,
    string $semester,
    string $schoolYear,
    ?int $excludeId = null
): array {
    $sql = "SELECT s.id, s.start_time, s.end_time, COALESCE(f.full_name, '') AS faculty_name
        FROM schedules s
        LEFT JOIN faculty f ON f.id = s.faculty_id
        WHERE s.room_id = ? AND s.semester = ? AND s.school_year = ?
        AND FIND_IN_SET(?, s.day_of_week) > 0";
    $params = [$roomId, $semester, $schoolYear, $day];
    if ($excludeId !== null) {
        $sql .= ' AND s.id != ?';
        $params[] = $excludeId;
    }
    $stmt = db()->prepare($sql);
    $stmt->execute($params);
    return $stmt->fetchAll();
}

function room_status_allows_overlap(int $roomId): bool
{
    $stmt = db()->prepare('SELECT status, type FROM rooms WHERE id = ? LIMIT 1');
    $stmt->execute([$roomId]);
    $row = $stmt->fetch();
    if (!$row) {
        return false;
    }
    $status = strtolower((string) ($row['status'] ?? ''));
    $type = strtolower((string) ($row['type'] ?? ''));
    return $status === 'tba' || $type === 'tba';
}

/**
 * Whether room_code is already used for another room in the same college (dean-managed rooms).
 */
function room_code_taken_for_college(string $roomCode, int $collegeId, ?int $exceptRoomId = null): bool
{
    $code = trim($roomCode);
    if ($code === '' || $collegeId < 1) {
        return false;
    }
    if ($exceptRoomId !== null && $exceptRoomId > 0) {
        $stmt = db()->prepare(
            'SELECT id FROM rooms WHERE COALESCE(is_gened,0) = 0 AND college_id = ? AND room_code = ? AND id <> ? LIMIT 1'
        );
        $stmt->execute([$collegeId, $code, $exceptRoomId]);
    } else {
        $stmt = db()->prepare(
            'SELECT id FROM rooms WHERE COALESCE(is_gened,0) = 0 AND college_id = ? AND room_code = ? LIMIT 1'
        );
        $stmt->execute([$collegeId, $code]);
    }
    return $stmt->fetchColumn() !== false;
}

/**
 * Whether room_code is already used for another Gen Ed room.
 */
function room_code_taken_for_gened(string $roomCode, ?int $exceptRoomId = null): bool
{
    $code = trim($roomCode);
    if ($code === '') {
        return false;
    }
}

```

```

if ($exceptRoomId !== null && $exceptRoomId > 0) {
    $stmt = db()->prepare(
        'SELECT id FROM rooms WHERE COALESCE(is_gened,0) = 1 AND room_code = ? AND id <> ? LIMIT 1'
    );
    $stmt->execute([$code, $exceptRoomId]);
} else {
    $stmt = db()->prepare(
        'SELECT id FROM rooms WHERE COALESCE(is_gened,0) = 1 AND room_code = ? LIMIT 1'
    );
    $stmt->execute([$code]);
}
return $stmt->fetchColumn() !== false;
}

/**
 * Short uppercase prefix for auto room codes from a college name (fallback C{id}).
 */
function room_auto_code_prefix_from_name(string $collegeName, int $fallbackId, int $maxLen = 6): string
{
    $s = strtoupper((string) preg_replace('/[^A-Za-z0-9]+/', '', $collegeName));
    if ($s === '') {
        $s = 'C' . max(1, $fallbackId);
    }
    if (strlen($s) > $maxLen) {
        $s = substr($s, 0, $maxLen);
    }
    return $s;
}

/**
 * Next auto room code for a college: PREFIX-001, PREFIX-002, ... (unique within that college).
 */
function next_auto_room_code_for_college(int $collegeId, string $collegeName): string
{
    if ($collegeId < 1) {
        throw new RuntimeException('Invalid college for room code.');
```



```

$st = db()->prepare(
    "SELECT room_code FROM rooms WHERE COALESCE(is_gened,0) = 1 AND room_code LIKE ?"
);
$st->execute([$prefix . '-%']);
$max = 0;
$pattern = '/^' . preg_quote($prefix, '/') . '-(\d+)$/';
foreach ($st->fetchAll(PDO::FETCH_COLUMN) as $code) {
    if (preg_match($pattern, (string) $code, $m)) {
        $max = max($max, (int) $m[1]);
    }
}
for ($n = $max + 1; $n < $max + 10000; $n++) {
    $suffix = str_pad((string) $n, 3, '0', STR_PAD_LEFT);
    $candidate = $prefix . '-' . $suffix;
    if (strlen($candidate) > 20) {
        $candidate = $prefix . '-' . $n;
        $candidate = substr($candidate, 0, 20);
    }
    if (!room_code_taken_for_gened($candidate)) {
        return $candidate;
    }
}
throw new RuntimeException('Could not allocate a unique GE room code. Use a custom code.');
```

```

}

function detect_cross_college_room_conflicts(
    int $roomId,
    int $collegeId,
    string $day,
    string $semester,
    string $schoolYear,
    int $st,
    int $en,
    ?int $excludeId = null
): array {
    if (room_status_allows_overlap($roomId)) {
        return [];
    }

    $sql = "SELECT s.id, s.start_time, s.end_time, c.college_name, COALESCE(f.full_name, '') AS faculty_name
    FROM schedules s
    LEFT JOIN colleges c ON c.id = s.college_id
    LEFT JOIN faculty f ON f.id = s.faculty_id
    WHERE s.room_id = ? AND s.college_id IS NOT NULL AND s.college_id <> ?
    AND s.semester = ? AND s.school_year = ?
    AND FIND_IN_SET(?, s.day_of_week) > 0";
    $params = [$roomId, $collegeId, $semester, $schoolYear, $day];
    if ($excludeId !== null) {
        $sql .= ' AND s.id != ?';
        $params[] = $excludeId;
    }
    $stmt = db()->prepare($sql);
    $stmt->execute($params);
    $rows = $stmt->fetchAll();

    $cross = [];
    foreach ($rows as $row) {
        $rs = time_to_minutes(substr((string) $row['start_time'], 0, 8));
        $re = time_to_minutes(substr((string) $row['end_time'], 0, 8));
        if (intervals_overlap($st, $en, $rs, $re)) {
            $collegeName = (string) ($row['college_name'] ?: 'Other college');
            $bookedBy = trim((string) ($row['faculty_name'] ?? ''));
            $byClause = $bookedBy !== '' ? " ({ $bookedBy })" : '';
            $cross[] = "Cross-college room conflict on { $day}: room is used by { $collegeName } { $byClause }.";
        }
    }
    return array_values(array_unique($cross));
}

/**
 * Check gap and consecutive-hour rules for faculty on a day when adding [st,en).
 * @param array<int,array{start_time:string,end_time:string}> $existing
 * @return string[]
 */
function check_faculty_gaps_and_consecutive(array $existing, int $st, int $en): array
{
    $errors = [];
    $intervals = [];
    foreach ($existing as $row) {
        $intervals[] = [

```

```

        time_to_minutes(substr($row['start_time'], 0, 8)),
        time_to_minutes(substr($row['end_time'], 0, 8)),
    ];
}
$intervals[] = [$st, $en];
usort($intervals, static fn ($x, $y) => $x[0] <=> $y[0]);

for ($i = 0; $i < count($intervals) - 1; $i++) {
    $end1 = $intervals[$i][1];
    $start2 = $intervals[$i + 1][0];
    $gap = $start2 - $end1;
    if ($gap < 0) {
        continue;
    }
    if (MIN_GAP_MINUTES > 0 && $gap > 0 && $gap < MIN_GAP_MINUTES) {
        $errors[] = 'There must be at least ' . MIN_GAP_MINUTES . ' minutes between classes on the same day.';
    }
}

$idx = 0;
while ($idx < count($intervals)) {
    $chainStart = $intervals[$idx][0];
    $chainEnd = $intervals[$idx][1];
    $chainCount = 1;
    $k = $idx;
    while ($k < count($intervals) - 1) {
        $g = $intervals[$k + 1][0] - $intervals[$k][1];
        if ($g === 0) {
            $chainEnd = $intervals[$k + 1][1];
            $chainCount++;
            $k++;
        } else {
            break;
        }
    }
    if ($chainCount > 1 && ($chainEnd - $chainStart > MAX_CONSECUTIVE_HOURS * 60)) {
        $breakPhrase = MIN_GAP_MINUTES > 0
            ? MIN_GAP_MINUTES . '-minute break'
            : 'break between class periods';
        $errors[] = 'Faculty cannot teach more than ' . MAX_CONSECUTIVE_HOURS . ' consecutive hours without a '
            . $breakPhrase . '.';
    }
    $idx = $k + 1;
}

return array_values(array_unique($errors));
}

/**
 * Main conflict checker.
 *
 * @param string[] $days
 * @return array<int,array{type:string,description:string,scope:string}>
 */
function checkConflicts(
    int $faculty_id,
    int $room_id,
    array $days,
    string $start_time,
    string $end_time,
    string $semester,
    string $school_year,
    ?int $exclude_id = null,
    ?int $college_id = null,
    bool $allow_long_block = false
): array {
    $conflicts = [];
    $st = time_to_minutes(substr($start_time, 0, 5) . ':00');
    $en = time_to_minutes(substr($end_time, 0, 5) . ':00');
    $ignoreRoomOverlap = room_status_allows_overlap($room_id);

    $facultyNameStmt = db()->prepare('SELECT full_name FROM faculty WHERE id = ? LIMIT 1');
    $facultyNameStmt->execute([$faculty_id]);
    $facultyName = trim((string) ($facultyNameStmt->fetchColumn() ?: ''));
    if ($facultyName === '') {
        $facultyName = 'Faculty';
    }

    foreach ($days as $day) {
        // Faculty overlap (always internal because faculty belongs to one college)

```

```

$frows = fetch_faculty_day_schedules($faculty_id, $day, $semester, $school_year, $exclude_id);
foreach ($frows as $row) {
    $rs = time_to_minutes(substr($row['start_time'], 0, 8));
    $re = time_to_minutes(substr($row['end_time'], 0, 8));
    if (intervals_overlap($st, $en, $rs, $re)) {
        $existingStart = substr((string) $row['start_time'], 0, 8);
        $existingEnd = substr((string) $row['end_time'], 0, 8);
        $conflicts[] = [
            'type' => 'faculty',
            'description' => "{$facultyName} already has a class on {$day} overlapping {$start_time}-{$end_time} (existing:
{$existingStart}-{$existingEnd}).",
            'scope' => 'internal',
        ];
    }
}

// Room overlap (same-college, term scoped)
if (!$ignoreRoomOverlap) {
    $rrows = fetch_room_day_schedules($room_id, $day, $semester, $school_year, $exclude_id);
    foreach ($rrows as $row) {
        $rs = time_to_minutes(substr($row['start_time'], 0, 8));
        $re = time_to_minutes(substr($row['end_time'], 0, 8));
        if (intervals_overlap($st, $en, $rs, $re)) {
            $bookedBy = trim((string) ($row['faculty_name'] ?? ''));
            $byClause = $bookedBy != '' ? " by {$bookedBy}" : '';
            $conflicts[] = [
                'type' => 'room',
                'description' => "Room is already booked on {$day} during this time{$byClause}.",
                'scope' => 'internal',
            ];
        }
    }
}

// Daily hour limit
$stmt = db()->prepare('SELECT max_hours_per_day FROM faculty WHERE id = ?');
$stmt->execute([$faculty_id]);
$maxH = (int) ($stmt->fetchColumn() ?: 8);
$maxMin = $maxH * 60;
$existingMin = 0;
foreach ($frows as $row) {
    $a = time_to_minutes(substr($row['start_time'], 0, 8));
    $b = time_to_minutes(substr($row['end_time'], 0, 8));
    $existingMin += ($b - $a);
}
$newMin = $en - $st;
/* Only count new slot once per day */
if ($existingMin + $newMin > $maxMin) {
    $conflicts[] = [
        'type' => 'time',
        'description' => "{$facultyName} would exceed {$maxH} hours on {$day}.",
        'scope' => 'internal',
    ];
}

// Gap / consecutive rules
$merged = fetch_faculty_day_schedules($faculty_id, $day, $semester, $school_year, $exclude_id);
$gapErrors = check_faculty_gaps_and_consecutive($merged, $st, $en);
foreach ($gapErrors as $msg) {
    if ($allow_long_block && str_contains($msg, 'consecutive hours')) {
        continue;
    }
    $conflicts[] = ['type' => 'time', 'description' => $msg . " ({$day})", 'scope' => 'internal'];
}

// Cross-college room overlap (request required for dean workflow)
if ($college_id !== null && !$ignoreRoomOverlap) {
    $cross = detect_cross_college_room_conflicts(
        $room_id,
        $college_id,
        $day,
        $semester,
        $school_year,
        $st,
        $en,
        $exclude_id
    );
    foreach ($cross as $msg) {
        $conflicts[] = ['type' => 'room', 'description' => $msg, 'scope' => 'cross_college'];
    }
}

```

```

    }

}

// Deduplicate by type+description+scope
$unique = [];
foreach ($conflicts as $c) {
    $k = $c['type'] . '|' . $c['scope'] . '|' . $c['description'];
    $unique[$k] = $c;
}
return array_values($unique);
}

/**
 * Log conflicts to conflict_logs (schedule_id nullable until saved).
 */
function log_conflicts(?int $scheduleId, array $conflicts, bool $resolved = false): void
{
    $stmt = db()->prepare(
        'INSERT INTO conflict_logs (schedule_id, conflict_type, conflict_description, resolved) VALUES (?, ?, ?, ?)'
    );
    foreach ($conflicts as $c) {
        $stmt->execute([$scheduleId, $c['type'], $c['description'], $resolved ? 1 : 0]);
    }
}

function create_conflict_request(array $payload): int
{
    $stmt = db()->prepare(
        'INSERT INTO conflict_requests
        (requested_by, college_id, faculty_id, course_id, room_id, schedule_type, day_of_week, start_time, end_time, semester, school_year,
        academic_year, reason)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)'
    );
    $stmt->execute([
        (int) $payload['requested_by'],
        (int) $payload['college_id'],
        (int) $payload['faculty_id'],
        (int) $payload['course_id'],
        (int) $payload['room_id'],
        (string) $payload['schedule_type'],
        (string) $payload['day_of_week'],
        (string) $payload['start_time'],
        (string) $payload['end_time'],
        (string) $payload['semester'],
        (string) $payload['school_year'],
        (string) ($payload['academic_year'] ?? ''),
        (string) $payload['reason'],
    ]);
    return (int) db()->lastInsertId();
}

/**
 * Find another faculty member who already holds this course load for the term.
 *
 * @param array{college_id?:int,program_name?:string,year_level?:string,section?:string}|null $geTarget
 * @return array{faculty_id:int,full_name:string,faculty_code:string}|null
 */
function find_course_load_assigned_faculty(
    int $courseId,
    string $semester,
    string $schoolYear,
    int $exceptFacultyId,
    ?array $geTarget = null
): ?array {
    if ($courseId < 1 || $semester === '' || $schoolYear === '') {
        return null;
    }

    $useGeTarget = $geTarget !== null
        && db_table_exists('ge_schedule_targets')
        && (int) ($geTarget['college_id'] ?? 0) > 0
        && trim((string) ($geTarget['program_name'] ?? '')) !== ''
        && trim((string) ($geTarget['year_level'] ?? '')) !== ''
        && trim((string) ($geTarget['section'] ?? '')) !== '';

    if ($useGeTarget) {
        $st = db()->prepare(
            'SELECT DISTINCT f.id AS faculty_id, f.full_name, f.faculty_id AS faculty_code
            FROM schedules s

```

```

        INNER JOIN faculty f ON f.id = s.faculty_id
        INNER JOIN ge_schedule_targets gst ON gst.schedule_id = s.id
        WHERE s.course_id = ?
            AND s.semester = ?
            AND s.school_year = ?
            AND s.faculty_id <> ?
            AND gst.college_id = ?
            AND gst.program_name = ?
            AND gst.year_level = ?
            AND gst.section = ?
        LIMIT 1'
    );
    $st->execute([
        $courseId,
        $semester,
        $schoolYear,
        $exceptFacultyId,
        (int) $geTarget['college_id'],
        trim((string) $geTarget['program_name']),
        trim((string) $geTarget['year_level']),
        trim((string) $geTarget['section']),
    ]);
} else {
    $st = db()->prepare(
        'SELECT DISTINCT f.id AS faculty_id, f.full_name, f.faculty_id AS faculty_code
        FROM schedules s
        INNER JOIN faculty f ON f.id = s.faculty_id
        WHERE s.course_id = ?
            AND s.semester = ?
            AND s.school_year = ?
            AND s.faculty_id <> ?
        LIMIT 1'
    );
    $st->execute([$courseId, $semester, $schoolYear, $exceptFacultyId]);
}

$row = $st->fetch(PDO::FETCH_ASSOC);
if (!$row) {
    return null;
}

return [
    'faculty_id' => (int) ($row['faculty_id'] ?? 0),
    'full_name' => trim((string) ($row['full_name'] ?? '')),
    'faculty_code' => trim((string) ($row['faculty_code'] ?? '')),
];
}

/**
 * @param array{faculty_id:int,full_name:string,faculty_code:string} $assignedFaculty
 * @param array{college_id?:int,program_name?:string,year_level?:string,section?:string}|null $geTarget
 */
function course_load_assignment_conflict_message(
    array $assignedFaculty,
    int $courseId = 0,
    ?array $geTarget = null
): string {
    $name = trim((string) ($assignedFaculty['full_name'] ?? ''));
    $code = trim((string) ($assignedFaculty['faculty_code'] ?? ''));
    $who = $name !== '' ? $name : 'another faculty member';
    if ($code !== '') {
        $who .= ' (' . $code . ')';
    }

    $sectionLabel = '';
    if ($geTarget !== null) {
        $program = trim((string) ($geTarget['program_name'] ?? ''));
        $yearLevel = trim((string) ($geTarget['year_level'] ?? ''));
        $section = trim((string) ($geTarget['section'] ?? ''));
        if ($program !== '' && $yearLevel !== '' && $section !== '') {
            $sectionLabel = $program . ' Y' . $yearLevel . '-' . $section;
        }
    }
} elseif ($courseId > 0 && db_column_exists('courses', 'section') && db_column_exists('courses', 'year_level')) {
    $st = db()->prepare('SELECT course_code, year_level, section FROM courses WHERE id=? LIMIT 1');
    $st->execute([$courseId]);
    $crow = $st->fetch(PDO::FETCH_ASSOC);
    if ($crow) {
        $courseCode = trim((string) ($crow['course_code'] ?? ''));
        $yearLevel = trim((string) ($crow['year_level'] ?? ''));
        $section = trim((string) ($crow['section'] ?? ''));
    }
}

```

```

        if ($yearLevel !== '' && $section !== '') {
            $sectionLabel = ($courseCode !== '' ? $courseCode . ', ' : '') . 'Y' . $yearLevel . '-' . $section;
        } elseif ($courseCode !== '') {
            $sectionLabel = $courseCode;
        }
    }
}

if ($sectionLabel !== '') {
    return 'This course load for ' . $sectionLabel . ' is already assigned to ' . $who . '.';
}

return 'This course load is already assigned to ' . $who . '.';
}

/** Full name of the administrator who signs as VPAA on teaching load memoranda. */
function vpaa_signatory_full_name(): string
{
    static $resolved = null;
    if ($resolved !== null) {
        return $resolved;
    }

    if (defined('VPAA_ADMIN_USER_ID') && (int) VPAA_ADMIN_USER_ID > 0) {
        $st = db()->prepare(
            'SELECT full_name FROM users WHERE id = ? AND is_active = 1 AND role IN (\admin\, \super_admin\)' LIMIT 1'
        );
        $st->execute([(int) VPAA_ADMIN_USER_ID]);
        $name = trim((string) ($st->fetchColumn() ?: ''));
        if ($name !== '') {
            $resolved = $name;
        }

        return $resolved;
    }

    if (db_column_exists('users', 'admin_log_title')) {
        $st = db()->query(
            'SELECT full_name, admin_log_title
            FROM users
            WHERE is_active = 1 AND role = \admin\' AND admin_log_title != \'\'
            ORDER BY id ASC'
        );
        foreach ($st->fetchAll(PDO::FETCH_ASSOC) ?: [] as $row) {
            $title = strtolower(trim((string) ($row['admin_log_title'] ?? '')));
            $isVpaa = ($title !== '' && str_contains($title, 'vice president') && str_contains($title, 'academic affairs'))
                || $title === 'vpaa'
                || str_contains($title, 'vp for academic affairs');
            if (!$isVpaa) {
                continue;
            }
            $name = trim((string) ($row['full_name'] ?? ''));
            if ($name !== '') {
                $resolved = $name;
            }

            return $resolved;
        }
    }

    $st = db()->query(
        'SELECT full_name FROM users WHERE is_active = 1 AND role = \admin\' ORDER BY id ASC LIMIT 1'
    );
    $resolved = trim((string) ($st->fetchColumn() ?: ''));

    return $resolved;
}

/**
 * Show schedule validation/conflict errors as a warning popup modal.
 */
@param list<string> $errors
*/
function render_schedule_errors_warning_popup(
    array $errors,
    string $title = 'Warning – Please fix the following',
    string $okLabel = 'OK'
): void {
    if ($errors === []) {
        return;
    }

```

```

}
$modalId = 'scheduleErrorsWarningModal';
?>
<div class="modal fade" id="<? = htmlspecialchars($modalId) ?>" tabindex="-1" aria-labelledby="<? = htmlspecialchars($modalId) ?>Label"
aria-hidden="true" data-bs-backdrop="static">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content border-warning">
      <div class="modal-header bg-warning-subtle">
        <h5 class="modal-title text-warning-emphasis" id="<? = htmlspecialchars($modalId) ?>Label">
          <i class="fa-solid fa-triangle-exclamation me-2"></i><? = htmlspecialchars($title) ?>
        </h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
      </div>
      <div class="modal-body">
        <ul class="mb-0">
          <?php foreach ($errors as $e): ?>
            <li><? = htmlspecialchars((string) $e) ?></li>
          <?php endforeach; ?>
        </ul>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-warning" data-bs-dismiss="modal"><? = htmlspecialchars($okLabel) ?></button>
      </div>
    </div>
  </div>
</div>
</div>
<script>
document.addEventListener('DOMContentLoaded', function () {
  var modalEl = document.getElementById(<? = json_encode($modalId) ?>);
  if (!modalEl) return;
  if (typeof bootstrap !== 'undefined' && bootstrap.Modal) {
    bootstrap.Modal.getOrCreateInstance(modalEl).show();
    return;
  }
  var items = [];
  modalEl.querySelectorAll('.modal-body li').forEach(function (li) {
    items.push(li.textContent.trim());
  });
  window.alert(<? = json_encode($title) ?> + ':\\n\\n- ' + items.join('\\n- '));
});
</script>
<?php
}

/**
 * Show a single notice as an Information popup modal.
 */
function render_information_popup(
  string $message,
  string $title = 'Information',
  string $okLabel = 'OK'
): void {
  $message = trim($message);
  if ($message === '') {
    return;
  }
  $modalId = 'appInformationModal';
  ?>
  <div class="modal fade no-print" id="<? = htmlspecialchars($modalId) ?>" tabindex="-1" aria-labelledby="<? = htmlspecialchars($modalId) ?
>Label" aria-hidden="true" data-bs-backdrop="static">
    <div class="modal-dialog modal-dialog-centered">
      <div class="modal-content border-info">
        <div class="modal-header bg-info-subtle">
          <h5 class="modal-title text-info-emphasis" id="<? = htmlspecialchars($modalId) ?>Label">
            <i class="fa-solid fa-circle-info me-2"></i><? = htmlspecialchars($title) ?>
          </h5>
          <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
        </div>
        <div class="modal-body">
          <? = htmlspecialchars($message) ?>
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-info" data-bs-dismiss="modal"><? = htmlspecialchars($okLabel) ?></button>
        </div>
      </div>
    </div>
  </div>
</div>
<script>
document.addEventListener('DOMContentLoaded', function () {
  var modalEl = document.getElementById(<? = json_encode($modalId) ?>);

```

```

        if (!modalEl) return;
        if (typeof bootstrap !== 'undefined' && bootstrap.Modal) {
            bootstrap.Modal.getOrCreateInstance(modalEl).show();
            return;
        }
        var body = modalEl.querySelector('.modal-body');
        window.alert(<?= json_encode($title) ?> + ':\n\n' + (body ? body.textContent.trim() : ''));
    });
</script>
<?php
}

```

FILE: includes/header.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/messaging_helpers.php';
require_once __DIR__ . '/student_tooltip.php';
require_once __DIR__ . '/profile_photo_helpers.php';

$pageTitle = $pageTitle ?? 'WPU SABLAE Portal';
$mainContainerClass = $mainContainerClass ?? 'container py-4 py-md-5 app-main';
$u = current_user();
$messagingNavUnread = ($u && messaging_table_exists()) ? messaging_unread_count((int) $u['id']) : 0;
$appSidebarShell = false;
$adminNavPage = basename($_SERVER['SCRIPT_NAME']) ?? '';
$bodyRoleClass = '';
if ($u) {
    $bodyRoleClass = preg_replace('/[^\a-z0-9_-]/', '', (string) ($u['role'] ?? ''));
}
$appCursorTooltips = $u && app_cursor_tooltips_active();
$styleCssVersion = @filemtime(__DIR__ . '/../assets/css/style.css') ?: 1;
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script>
        (function () {
            try {
                var k = 'app_theme', t = localStorage.getItem(k);
                if (t !== 'dark' && t !== 'light') {
                    t = localStorage.getItem('schedule_theme');
                    if (t === 'dark' || t === 'light') localStorage.setItem(k, t);
                }
                if (t !== 'dark' && t !== 'light') {
                    t = (window.matchMedia && matchMedia('(prefers-color-scheme: dark)').matches) ? 'dark' : 'light';
                }
                document.documentElement.setAttribute('data-bs-theme', t);
            } catch (e) {
                document.documentElement.setAttribute('data-bs-theme', 'light');
            }
        })();
    </script>
    <title><?= htmlspecialchars($pageTitle) ?></title>
    <?php require __DIR__ . '/favicon.php'; ?>
    <link href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css" rel="stylesheet">
    <link href="assets/css/style.css?v=<?= (int) $styleCssVersion ?>" rel="stylesheet">
    <link href="assets/css/print.css" rel="stylesheet" media="print">
</head>
<body class="app-shell"<?= $u ? ' app-shell-sidebar' . ($bodyRoleClass !== '' ? ' app-shell-sidebar--' . htmlspecialchars($bodyRoleClass, ENT_QUOTES, 'UTF-8') : '') : '' ?><?= !empty($bodyPageClass) ? ' ' . htmlspecialchars((string) $bodyPageClass, ENT_QUOTES, 'UTF-8') : '' ?>">
<a class="visually-hidden-focusable btn btn-primary position-fixed top-0 start-0 m-2 px-3 py-2 rounded-3 shadow z-3" href="#main-content" style="z-index: 1080;"<?= $appCursorTooltips ? app_tooltip_attr('Jumps past the sidebar and menus to the main page content. Use this for faster keyboard access to what you are reading.') : '' ?>>Skip to main content</a>
<?php if ($u): ?>
<?php
$appSidebarShell = true;
require_once __DIR__ . '/admin_nav.php';
$navRole = (string) ($u['role'] ?? '');
$topbarProfilePhotoUrl = profile_photo_url((int) ($u['id'] ?? 0));
$pcAssignedPrograms = [];
$pcActiveProgram = '';
if ($navRole === 'program_chair') {

```



```

$pcAssignedPrograms = is_array($_SESSION['assigned_programs']) ?? null)
    ? array_values(array_filter(array_map('strval', $_SESSION['assigned_programs'])))
    : program_chair_assigned_programs((int) ($u['id'] ?? 0));
$pcActiveProgram = trim((string) ($_SESSION['assigned_program'] ?? ''));
}
$pcProgramSwitcherRedirect = basename((string) ($_SERVER['SCRIPT_NAME'] ?? 'dashboard.php'));
$queryString = trim((string) ($_SERVER['QUERY_STRING'] ?? ''));
if ($queryString !== '') {
    $pcProgramSwitcherRedirect .= '?' . $queryString;
}
?>
<div class="admin-layout d-flex min-vh-100 w-100">
    <aside class="admin-sidebar d-none d-lg-flex flex-column flex-shrink-0 no-print" aria-label="Main navigation">
        <div class="admin-sidebar-brand">
            <a class="admin-sidebar-brand-link" href="dashboard.php">?<= $appCursorTooltips ? app_tooltip_attr('Returns to your dashboard
home. Use this when you want a quick way back from another page.'): '' ?>>
                <span class="admin-sidebar-brand-icon"></span>
                <span class="admin-sidebar-brand-text">WPU SABLAE Portal</span>
            </a>
            <div class="admin-sidebar-brand-meta"><?<= htmlspecialchars(app_sidebar_brand_meta($navRole)) ?></div>
        </div>
        <nav class="admin-sidebar-nav flex-grow-1" aria-label="Primary sections">
            <?php if ($navRole === 'admin'): ?>
                <?php render_admin_nav_sections($adminNavPage, $messagingNavUnread); ?>
            <?php elseif ($navRole === 'super_admin'): ?>
                <?php render_super_admin_nav_sections($adminNavPage, $messagingNavUnread); ?>
            <?php else: ?>
                <?php render_role_nav_sections($navRole, $adminNavPage, $messagingNavUnread); ?>
            <?php endif; ?>
        </nav>
        <div class="admin-sidebar-footer">
            <a class="admin-sidebar-foot-link" href="settings.php">?<= $appCursorTooltips ? app_tooltip_attr('Opens account settings where
you can change your password. Same as Settings in the top menu.'): '' ?>><i class="fa-solid fa-gear me-2" aria-hidden="true">
</i>Settings</a>
            <a class="admin-sidebar-foot-link admin-sidebar-logout" href="logout.php">?<= $appCursorTooltips ? app_tooltip_attr('Signs you
out of the system. Use when you are done or when someone else needs this device.'): '' ?>><i class="fa-solid fa-right-from-bracket me-2"
aria-hidden="true"></i>Logout</a>
        </div>
    </aside>
    <div class="admin-main-wrap flex-grow-1 d-flex flex-column min-vh-100 min-w-0">
        <header class="admin-topbar border-bottom bg-body shadow-sm sticky-top no-print">
            <div class="d-flex align-items-center gap-3 px-3 px-lg-4 py-2 py-lg-3">
                <label id="adminNavOpenBtn" for="adminNavToggle" class="btn btn-outline-secondary btn-sm d-lg-none rounded-3 d-inline-flex
align-items-center justify-content-center" role="button" tabindex="0" aria-controls="adminNavOffcanvas" aria-label="Open navigation menu">?<=
$appCursorTooltips ? app_tooltip_attr('Opens the navigation menu on phones and small screens. Use this when the left sidebar is hidden to
reach Dashboard, Messages, schedules, and other app sections.'): '' ?>>
                    <i class="fa-solid fa-bars"></i>
                </label>
                <div class="flex-grow-1 min-w-0">
                    <div class="admin-topbar-title text-truncate"><?<= htmlspecialchars($pageTitle) ?></div>
                    <div class="admin-topbar-subtitle small text-muted text-truncate d-none d-sm-block">Western Philippines University</div>
                </div>
                <?php if ($navRole === 'program_chair' && count($pcAssignedPrograms) > 1): ?>
                    <form method="post" class="d-none d-md-flex align-items-center gap-2 no-print" style="max-width: 18rem;">
                        <input type="hidden" name="action" value="switch_active_program">
                        <input type="hidden" name="redirect" value="<?<= htmlspecialchars($pcProgramSwitcherRedirect) ?>">
                        <label class="small text-muted text-nowrap mb-0" for="pcActiveProgramSelect">Active program</label>
                        <select class="form-select form-select-sm" id="pcActiveProgramSelect" name="active_program"
onchange="this.form.submit()">?<= $appCursorTooltips ? app_tooltip_attr('Switch which assigned program you are managing right now. Schedules,
faculty, and students follow this selection.'): '' ?>>
                            <?php foreach ($pcAssignedPrograms as $progName): ?>
                                <option value="<?<= htmlspecialchars($progName) ?>">?<= $pcActiveProgram === $progName ? 'selected' : '' ?>>
                                <?php htmlspecialchars($progName) ?></option>
                            <?php endforeach; ?>
                        </select>
                    </form>
                    <?php endif; ?>
                    <button type="button" class="btn app-theme-toggle align-items-center gap-1 d-inline-flex no-print" id="appThemeToggle" aria-
label="Switch color theme" title="Switch color theme" aria-pressed="false">?<= $appCursorTooltips ? app_tooltip_attr('Switches between dark
and light appearance for the whole app. Your choice is remembered on this device.'): '' ?>>
                        <i class="fa-solid fa-moon app-theme-icon-dark" aria-hidden="true"></i>
                        <i class="fa-solid fa-sun app-theme-icon-light d-none" aria-hidden="true"></i>
                        <span class="small" id="appThemeToggleLabel">Dark</span>
                    </button>
                    <div class="dropdown admin-topbar-user">
                        <button class="btn btn-outline-secondary rounded-pill d-flex align-items-center gap-2 py-1 px-2 px-sm-3" type="button"
id="adminUserMenuBtn" data-bs-toggle="dropdown" aria-expanded="false" aria-haspopup="true">?<= $appCursorTooltips ? app_tooltip_attr('Opens
your account menu for Settings and Logout. Use this to change your password or sign out when you are done.'): '' ?>>
                            <?php app_render_topbar_avatar($navRole, $topbarProfilePhotoUrl); ?>

```

```

        <span class="d-none d-md-inline text-start lh-sm">
            <span class="d-block small fw-semibold text-truncate" style="max-width: 10rem;"><?=
htmlspecialchars($u['full_name']) ?></span>
            <span class="d-block small text-muted text-uppercase" style="font-size: 0.65rem;"><?= htmlspecialchars($navRole)
?></span>
        </span>
    </button>
    <ul class="dropdown-menu dropdown-menu-end shadow border-0 rounded-3 py-2" aria-labelledby="adminUserMenuBtn"
style="min-width: 12rem;">
        <?php if ($navRole === 'program_chair' && count($pcAssignedPrograms) > 1): ?>
            <li class="px-3 py-1 d-md-none">
                <form method="post">
                    <input type="hidden" name="action" value="switch_active_program">
                    <input type="hidden" name="redirect" value="<?= htmlspecialchars($pcProgramSwitcherRedirect) ?>">
                    <label class="form-label small text-muted mb-1" for="pcActiveProgramSelectMobile">Active program</label>
                    <select class="form-select form-select-sm" id="pcActiveProgramSelectMobile" name="active_program"
onchange="this.form.submit()">
                        <?php foreach ($pcAssignedPrograms as $progName): ?>
                            <option value="<?= htmlspecialchars($progName) ?>" <?= $pcActiveProgram === $progName ?
'selected' : '' ?>><?= htmlspecialchars($progName) ?></option>
                        <?php endforeach; ?>
                    </select>
                </form>
            </li>
            <li class="d-md-none"><hr class="dropdown-divider my-1"></li>
            <?php endif; ?>
            <li><a class="dropdown-item rounded-2" href="settings.php"><?= $appCursorTooltips ? app_tooltip_attr('Opens account
settings where you can change your password. Use this to keep your login secure on shared or public devices.') : '' ?>><i class="fa-solid
fa-gear me-2 text-muted"></i>Settings</a></li>
            <li><hr class="dropdown-divider my-1"></li>
            <li><a class="dropdown-item rounded-2 text-danger" href="logout.php"><?= $appCursorTooltips ? app_tooltip_attr('Signs
you out of the scheduling system on this browser. Use this when you finish on a shared computer or switch accounts.') : '' ?>><i class="fa-
solid fa-right-from-bracket me-2"></i>Logout</a></li>
        </ul>
    </div>
</div>
</header>
<?php endif; ?>
<main id="main-content" class="<?= htmlspecialchars(trim($mainContainerClass . ($appSidebarShell ? ' flex-grow-1' : '')), ENT_QUOTES, 'UTF-
8') ?>" tabindex="-1">

```

```

-----
FILE: includes/login_slideshow_helpers.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/functions.php';

function login_slideshow_table_ready(): bool
{
    static $ready = null;
    if ($ready === null) {
        $ready = db_table_exists('login_slideshow_images');
    }
    return $ready;
}

function login_slideshow_dir(): string
{
    return BASE_PATH . DIRECTORY_SEPARATOR . 'uploads' . DIRECTORY_SEPARATOR . 'login_slideshow';
}

function login_slideshow_path(string $storedName): string
{
    return login_slideshow_dir() . DIRECTORY_SEPARATOR . basename($storedName);
}

/**
 * @return list<array{id:int, caption:string, sort_order:int, url:string}>
 */
function login_slideshow_active_images(): array
{
    if (!login_slideshow_table_ready()) {
        return [];
    }

    $rows = db()->query(
        'SELECT id, caption, sort_order, stored_name

```

```

        FROM login_slideshow_images
        WHERE is_active = 1
        ORDER BY sort_order ASC, id ASC'
    )->fetchAll(PDO::FETCH_ASSOC);

    $images = [];
    foreach ($rows as $row) {
        $stored = trim((string) ($row['stored_name'] ?? ''));
        if ($stored === '' || !is_file(login_slideshow_path($stored))) {
            continue;
        }
        $images[] = [
            'id' => (int) $row['id'],
            'caption' => trim((string) ($row['caption'] ?? '')),
            'sort_order' => (int) ($row['sort_order'] ?? 0),
            'url' => 'login_slideshow_image.php?id=' . (int) $row['id'],
        ];
    }

    return $images;
}

/**
 * @return list<array<string,mixed>>
 */
function login_slideshow_all_images(): array
{
    if (!login_slideshow_table_ready()) {
        return [];
    }

    return db()->query(
        'SELECT id, stored_name, caption, sort_order, is_active, created_at
        FROM login_slideshow_images
        ORDER BY sort_order ASC, id ASC'
    )->fetchAll(PDO::FETCH_ASSOC);
}

function login_slideshow_mime_for_stored(string $storedName): string
{
    return match (strtolower(pathinfo($storedName, PATHINFO_EXTENSION))) {
        'jpg', 'jpeg' => 'image/jpeg',
        'png' => 'image/png',
        'webp' => 'image/webp',
        default => 'application/octet-stream',
    };
}

function login_slideshow_delete_file(string $storedName): void
{
    if ($storedName === '') {
        return;
    }
    $path = login_slideshow_path($storedName);
    if (is_file($path)) {
        @unlink($path);
    }
}

/**
 * @param array<string,mixed> $file
 */
function login_slideshow_store_upload(array $file, string $caption, int $uploadedBy): int
{
    if (!login_slideshow_table_ready()) {
        throw new RuntimeException('Login slideshow is not installed. Run upgrade_roles.php once.');
```

```

    }

    $tmp = (string) ($file['tmp_name'] ?? '');
    if ($tmp === '' || !is_uploaded_file($tmp)) {
        throw new RuntimeException('Invalid upload.');
```

```

    }

    $imageInfo = @getimagesize($tmp);
    if ($imageInfo === false) {
        throw new RuntimeException('File must be a JPEG, PNG, or WebP image.');
```

```

    }

    $mime = (string) ($imageInfo['mime'] ?? '');
    $ext = match ($mime) {
        'image/jpeg' => 'jpg',
        'image/png' => 'png',
        'image/webp' => 'webp',
        default => '',
    };
    if ($ext === '') {
        throw new RuntimeException('Only JPEG, PNG, and WebP images are allowed.');
```

```

    }

    $dir = login_slideshow_dir();
    if (!is_dir($dir) && !mkdir($dir, 0775, true) && !is_dir($dir)) {
        throw new RuntimeException('Unable to create slideshow storage directory.');
```

```

    }

    $stored = 'slide_' . bin2hex(random_bytes(12)) . '.' . $ext;
    $dest = login_slideshow_path($stored);
    if (!move_uploaded_file($tmp, $dest)) {
        throw new RuntimeException('Failed to save image.');
```

```

    }

    $maxOrder = (int) db()->query('SELECT COALESCE(MAX(sort_order), 0) FROM login_slideshow_images')->fetchColumn();
    $st = db()->prepare(
        'INSERT INTO login_slideshow_images (stored_name, caption, sort_order, is_active, uploaded_by)
        VALUES (?, ?, ?, 1, ?)'
    );
    $st->execute([
        $stored,
        trim($caption),
        $maxOrder + 1,
        $uploadedBy > 0 ? $uploadedBy : null,
    ]);
```

```

    return (int) db()->lastInsertId();
}

/**
 * Store multiple uploaded slideshow images.
 *
 * @param array<string,mixed> $files $_FILES entry (supports single or multi file input)
 * @return array{uploaded:int, ids:list<int>, errors:list<string>}
 */
function login_slideshow_store_uploads(array $files, string $caption, int $uploadedBy): array
{
    $fileList = [];
    if (isset($files['name']) && is_array($files['name'])) {
        $count = count($files['name']);
        for ($i = 0; $i < $count; $i++) {
            $fileList[] = [
                'name' => $files['name'][$i] ?? '',
                'type' => $files['type'][$i] ?? '',
                'tmp_name' => $files['tmp_name'][$i] ?? '',
                'error' => $files['error'][$i] ?? UPLOAD_ERR_NO_FILE,
                'size' => $files['size'][$i] ?? 0,
            ];
        }
    }
    elseif (isset($files['name']) && ($files['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_NO_FILE) {
        $fileList[] = $files;
    }

    $fileList = array_values(array_filter($fileList, static function (array $file): bool {
        return (int) ($file['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_NO_FILE;
    }));

    if ($fileList === []) {
        throw new RuntimeException('Please choose at least one image to upload.');
```

```

    }
}
```

```

if (count($fileList) > 20) {
    throw new RuntimeException('You can upload up to 20 images at a time.');
```

```

}

$sids = [];
$errors = [];
foreach ($fileList as $i => $file) {
    $label = trim((string) ($file['name'] ?? ''));
    if ($label === '') {
        $label = 'Image ' . ($i + 1);
    }
    try {
        $sids[] = login_slideshow_store_upload($file, $caption, $uploadedBy);
    } catch (Throwable $e) {
        $errors[] = $label . ': ' . $e->getMessage();
    }
}

if ($sids === [] && $errors !== []) {
    throw new RuntimeException(implode(' ', $errors));
}

return [
    'uploaded' => count($sids),
    'ids' => $sids,
    'errors' => $errors,
];
}

function login_slideshow_delete(int $id): void
{
    if (!login_slideshow_table_ready()) {
        throw new RuntimeException('Login slideshow is not installed. Run upgrade_roles.php once.');
```

```

    }
    if ($id < 1) {
        throw new RuntimeException('Invalid image.');
```

```

    }

    $st = db()->prepare('SELECT stored_name FROM login_slideshow_images WHERE id = ? LIMIT 1');
    $st->execute([$id]);
    $stored = trim((string) ($st->fetchColumn() ?: ''));
    if ($stored === '') {
        throw new RuntimeException('Image not found.');
```

```

    }

    db()->prepare('DELETE FROM login_slideshow_images WHERE id = ?')->execute([$id]);
    login_slideshow_delete_file($stored);
}

function login_slideshow_set_active(int $id, bool $active): void
{
    if (!login_slideshow_table_ready()) {
        throw new RuntimeException('Login slideshow is not installed. Run upgrade_roles.php once.');
```

```

    }
    if ($id < 1) {
        throw new RuntimeException('Invalid image.');
```

```

    }

    db()->prepare('UPDATE login_slideshow_images SET is_active = ? WHERE id = ?')
        ->execute([$active ? 1 : 0, $id]);
}

function login_slideshow_update_caption(int $id, string $caption): void
{
    if (!login_slideshow_table_ready()) {
        throw new RuntimeException('Login slideshow is not installed. Run upgrade_roles.php once.');
```

```

    }
    if ($id < 1) {
        throw new RuntimeException('Invalid image.');
```

```

    }

    db()->prepare('UPDATE login_slideshow_images SET caption = ? WHERE id = ?')
        ->execute([trim($caption), $id]);
}

function login_slideshow_move(int $id, string $direction): void
{
    if (!login_slideshow_table_ready()) {
        throw new RuntimeException('Login slideshow is not installed. Run upgrade_roles.php once.');
```

```

    }
    if ($id < 1) {
        throw new RuntimeException('Invalid image.');
```

```
    }

```

```

    $rows = login_slideshow_all_images();
    $index = null;
    foreach ($rows as $i => $row) {
        if ((int) $row['id'] === $id) {
            $index = $i;
            break;
        }
    }
    if ($index === null) {
        throw new RuntimeException('Image not found.');
```

```
    }

```

```

    $swapIndex = $direction === 'up' ? $index - 1 : $index + 1;
    if ($swapIndex < 0 || $swapIndex >= count($rows)) {
        return;
    }

```

```

    $current = $rows[$index];
    $other = $rows[$swapIndex];
    $currentOrder = (int) $current['sort_order'];
    $otherOrder = (int) $other['sort_order'];

```

```

    $pdo = db();
    $pdo->prepare('UPDATE login_slideshow_images SET sort_order = ? WHERE id = ?')->execute([$otherOrder, (int) $current['id']]);
    $pdo->prepare('UPDATE login_slideshow_images SET sort_order = ? WHERE id = ?')->execute([$currentOrder, (int) $other['id']]);
}

```

```

function login_slideshow_image_row(int $id): ?array
{
    if (!login_slideshow_table_ready() || $id < 1) {
        return null;
    }

```

```

    $st = db()->prepare(
        'SELECT id, stored_name, caption, is_active
        FROM login_slideshow_images
        WHERE id = ? LIMIT 1'
    );
    $st->execute([$id]);
    $row = $st->fetch(PDO::FETCH_ASSOC);

```

```

    return $row ?: null;
}

```

```

-----
FILE: includes/mail_helpers.php
-----

```

```

<?php
declare(strict_types=1);

```

```

require_once __DIR__ . '/../config/config.php';

```

```

/**
 * Generate a random temporary password (alphanumeric, URL-safe subset).
 */

```

```

function generate_temp_password(int $length = 14): string
{
    $chars = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ23456789';
    $out = '';
    $max = strlen($chars) - 1;
    for ($i = 0; $i < $length; $i++) {
        $out .= $chars[random_int(0, $max)];
    }
    return $out;
}

```

```

/**
 * @return array{0:int,1:string} [code, full reply]
 */

```

```

function smtp_read_response($fp): array
{
    $reply = '';
    while ($line = @fgets($fp, 8192)) {
        $reply .= $line;
    }
}

```

```

        if (strlen($line) >= 4 && $line[3] === ' ') {
            break;
        }
    }
    $code = (int) substr($reply, 0, 3);

    return [$code, $reply];
}

/**
 * Resolve login URL for credential emails (config override, else APP_BASE_URL + /login.php).
 */
function mail_login_page_url(): string
{
    if (defined('MAIL_LOGIN_URL') && trim((string) MAIL_LOGIN_URL) !== '') {
        return rtrim((string) MAIL_LOGIN_URL, '/');
    }
    $base = defined('APP_BASE_URL') ? rtrim((string) APP_BASE_URL, '/') : '';

    return $base !== '' ? $base . '/login.php' : '/login.php';
}

/**
 * Absolute URL for the organization logo in HTML emails.
 */
function mail_logo_url(): string
{
    if (defined('MAIL_LOGO_URL') && trim((string) MAIL_LOGO_URL) !== '') {
        return rtrim((string) MAIL_LOGO_URL, '/');
    }
    $base = defined('APP_BASE_URL') ? rtrim((string) APP_BASE_URL, '/') : '';

    return $base !== '' ? $base . '/assets/logo.png' : '/assets/logo.png';
}

function mail_organization_name(): string
{
    if (defined('MAIL_ORGANIZATION_NAME') && trim((string) MAIL_ORGANIZATION_NAME) !== '') {
        return trim((string) MAIL_ORGANIZATION_NAME);
    }

    return defined('MAIL_FROM_NAME') ? (string) MAIL_FROM_NAME : 'WPU SABLAe Portal';
}

function mail_support_contact(): string
{
    if (defined('MAIL_SUPPORT_CONTACT') && trim((string) MAIL_SUPPORT_CONTACT) !== '') {
        return trim((string) MAIL_SUPPORT_CONTACT);
    }

    return 'Please contact your system administrator or IT help desk for assistance.';
}

function mail_credentials_subject(): string
{
    return 'Account Registration Successful - Your Login Credentials';
}

/**
 * SMTP dot-stuffing for DATA payload.
 */
function smtp_escape_data_body(string $body): string
{
    $normBody = str_replace(["\r\n", "\r"], "\n", $body);
    $lines = explode("\n", $normBody);
    $escaped = [];
    foreach ($lines as $line) {
        if (isset($line[0]) && $line[0] === '.') {
            $escaped[] = '.' . $line;
        } else {
            $escaped[] = $line;
        }
    }

    return implode("\r\n", $escaped);
}

/**
 * @return array{role_title: string, plain_intro: string, html_intro: string}
 */

```

```

function account_credentials_role_parts(string $role): array
{
    $role = strtolower(trim($role));
    $map = [
        'super_admin' => 'Super Administrator',
        'dean' => 'Dean',
        'program_chair' => 'Program Chair',
        'faculty' => 'Faculty',
        'gened' => 'General Education',
        'admin' => 'Administrator',
        'student' => 'Student',
        'student_registration' => 'Student',
    ];
    $roleTitle = $map[$role] ?? ucfirst(str_replace('_', ' ', $role));

    if ($role === 'student_registration') {
        $intro = 'Your registration has been approved and your account has been successfully created.';
    } else {
        $intro = 'Your account has been successfully created.';
    }

    return [
        'role_title' => $roleTitle,
        'plain_intro' => $intro,
        'html_intro' => $intro,
    ];
}

function mail_format_registration_datetime(?DateTimeInterface $registeredAt = null): string
{
    $dt = $registeredAt ?? new DateTimeImmutable('now');

    return $dt->format('F j, Y \a\t g:i A');
}

/**
 * Build plain-text + HTML bodies for new-account credential emails.
 *
 * @return array{0:string,1:string} [plain, html]
 */
function build_account_credentials_bodies(
    string $fullName,
    string $username,
    string $plainPassword,
    string $role,
    string $registeredEmail = '',
    ?DateTimeInterface $registeredAt = null
): array {
    $parts = account_credentials_role_parts($role);
    $loginUrl = mail_login_page_url();
    $logoUrl = mail_logo_url();
    $orgName = mail_organization_name();
    $support = mail_support_contact();
    $registeredOn = mail_format_registration_datetime($registeredAt);
    $roleKey = strtolower(trim($role));
    $isStudentCredentials = in_array($roleKey, ['student', 'student_registration'], true);
    $emailLine = $registeredEmail !== '' ? $registeredEmail : '-';

    $studentPlainExtra = $isStudentCredentials
        ? "\r\nAfter you sign in, open My Classes and enter each instructor's class join code.\r\n"
        : '';

    $plain =
        'Hello ' . $fullName . ",\r\n\r\n"
        . $parts['plain_intro'] . "\r\n\r\n"
        . "You can now log in using the following credentials:\r\n\r\n"
        . 'Username: ' . $username . "\r\n"
        . 'Temporary Password: ' . $plainPassword . "\r\n"
        . 'Registered email: ' . $emailLine . "\r\n"
        . 'Account type: ' . $parts['role_title'] . "\r\n"
        . 'Registered on: ' . $registeredOn . "\r\n\r\n"
        . 'Login Page: ' . $loginUrl . "\r\n\r\n"
        . "For security purposes, please change your password immediately after your first login.\r\n\r\n"
        . 'Technical support: ' . $support . $studentPlainExtra . "\r\n\r\n"
        . "Thank you,\r\nSystem Administrator\r\n" . $orgName . "\r\n\r\n"
        . "This is an automated email. Please do not reply.\r\n";

    $e = static function (string $s): string {
        return htmlspecialchars($s, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
    };
}

```



```

$studentHtmlExtra = $isStudentCredentials
? '<p style="margin:16px 0 0 0;color:#495057;font-size:14px;">After you sign in, open <strong>My Classes</strong> and enter each
instructor&rsquo;s class join code.</p>'
: '';

$html =
'<!DOCTYPE html><html><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,initial-scale=1">'
. '<title>' . $(mail_credentials_subject()) . '</title></head>'
. '<body style="margin:0;padding:0;background:#f4f6f9;font-family:Segoe UI,Roboto,Helvetica,Arial,sans-serif;font-size:15px;line-
height:1.6;color:#212529;">'
. '<table role="presentation" cellpadding="0" cellspacing="0" border="0" width="100%" style="background:#f4f6f9;padding:24px
12px;">'
. '<tr><td align="center">'
. '<table role="presentation" cellpadding="0" cellspacing="0" border="0" width="100%" style="max-
width:600px;background:#ffffff;border-radius:10px;box-shadow:0 2px 8px rgba(0,0,0,.08);overflow:hidden;">'
. '<tr><td style="padding:28px 28px 16px 28px;text-align:center;border-bottom:1px solid #e9ecef;">'
. ''
. '<div style="font-size:20px;font-weight:700;color:#0d6efd;">' . $(orgName) . '</div>'
. '</td></tr>'
. '<tr><td style="padding:24px 28px 8px 28px;">Hello <strong>' . $(fullName) . '</strong></td></tr>'
. '<tr><td style="padding:0 28px 20px 28px;color:#495057;">' . $(parts['html_intro']) . '</td></tr>'
. '<tr><td style="padding:0 28px 24px 28px;">'
. '<table role="presentation" cellpadding="0" cellspacing="0" border="0" width="100%" style="background:#f8f9fa;border:1px solid
#dee2e6;border-left:4px solid #0d6efd;border-radius:8px;">'
. '<tr><td style="padding:20px 22px;">'
. '<div style="font-size:13px;font-weight:700;color:#0d6efd;text-transform:uppercase;letter-spacing:.04em;margin-bottom:12px;">Your
login credentials</div>'
. '<table role="presentation" cellpadding="0" cellspacing="0" border="0" width="100%" style="font-size:14px;">'
. '<tr><td style="padding:6px 0;color:#6c757d;width:38%;">Username</td><td style="padding:6px 0;font-
family:Consolas,Monaco,monospace;font-weight:600;">' . $(username) . '</td></tr>'
. '<tr><td style="padding:6px 0;color:#6c757d;">Temporary Password</td><td style="padding:6px 0;font-
family:Consolas,Monaco,monospace;font-weight:600;">' . $(plainPassword) . '</td></tr>'
. '<tr><td style="padding:6px 0;color:#6c757d;">Registered email</td><td style="padding:6px 0;">' . $(emailLine) . '</td></tr>'
. '<tr><td style="padding:6px 0;color:#6c757d;">Account type</td><td style="padding:6px 0;">' . $(parts['role_title']) . '</td>
</tr>'
. '<tr><td style="padding:6px 0;color:#6c757d;">Registered on</td><td style="padding:6px 0;">' . $(registeredOn) . '</td></tr>'
. '</table></td></tr></table></td></tr>'
. '<tr><td style="padding:0 28px 24px 28px;" align="center">'
. '<table role="presentation" cellpadding="0" cellspacing="0" border="0"><tr><td style="border-radius:6px;background:#0d6efd;">'
. '<a href="' . $(loginUrl) . '" style="display:inline-block;padding:12px 28px;font-weight:600;color:#ffffff;text-
decoration:none;border-radius:6px;">Sign in</a>'
. '</td></tr></table>'
. '<p style="margin:12px 0 0 0;font-size:13px;color:#6c757d;word-break:break-all;">' . $(loginUrl) . '</p>'
. '</td></tr>'
. '<tr><td style="padding:0 28px 24px 28px;color:#495057;font-size:14px;">'
. '<strong>Important:</strong> For security purposes, please change your password immediately after your first login.'
. $studentHtmlExtra
. '<p style="margin:16px 0 0 0;"><strong>Technical support:</strong>' . $(support) . '</p>'
. '</td></tr>'
. '<tr><td style="padding:20px 28px;border-top:1px solid #e9ecef;color:#6c757d;font-size:13px;">'
. '<Thank you,<br><strong>System Administrator</strong><br>' . $(orgName)
. '<p style="margin:16px 0 0 0;font-size:12px;color:#adb5bd;">This is an automated email. Please do not reply.</p>'
. '</td></tr></table></td></tr></table></body></html>';

return [$plain, $html];
}

/**
 * Send email over SMTP (AUTH LOGIN + optional STARTTLS on port 587).
 * Pass $bodyHtml for multipart/alternative (plain + HTML).
 */
function smtp_send_mail(string $to, string $subject, string $body, ?string $bodyHtml = null): bool
{
    $host = defined('MAIL_SMTP_HOST') ? (string) MAIL_SMTP_HOST : '';
    $port = defined('MAIL_SMTP_PORT') ? (int) MAIL_SMTP_PORT : 587;
    $user = defined('MAIL_SMTP_USER') ? (string) MAIL_SMTP_USER : '';
    $pass = defined('MAIL_SMTP_PASS') ? (string) MAIL_SMTP_PASS : '';
    $from = defined('MAIL_FROM_ADDRESS') ? MAIL_FROM_ADDRESS : 'noreply@localhost';
    $fromName = defined('MAIL_FROM_NAME') ? MAIL_FROM_NAME : 'WPU SABLAe Portal';
    $useTls = defined('MAIL_SMTP_TLS') && MAIL_SMTP_TLS;

    if ($host === '') {
        return false;
    }

    $errno = 0;
    $errstr = '';
    $fp = @stream_socket_client(

```

```

        "tcp://{ $host }: { $port }",
        $errno,
        $errstr,
        25,
        STREAM_CLIENT_CONNECT
    );
    if (!$fp) {
        return false;
    }
    stream_set_timeout($fp, 20);

    $write = static function ($fp, string $cmd): void {
        fwrite($fp, $cmd . "\r\n");
    };

    [$code, $greet] = smtp_read_response($fp);
    if ($code !== 220) {
        fclose($fp);

        return false;
    }

    $ehloName = gethostname() ?: 'localhost';
    $write($fp, 'EHLO ' . $ehloName);
    [$code] = smtp_read_response($fp);
    if ($code !== 250) {
        fclose($fp);

        return false;
    }

    if ($useTls && $port === 587) {
        $write($fp, 'STARTTLS');
        [$code] = smtp_read_response($fp);
        if ($code !== 220) {
            fclose($fp);

            return false;
        }
        $cryptoOk = @stream_socket_enable_crypto($fp, true, STREAM_CRYPTO_METHOD_TLS_CLIENT);
        if (!$cryptoOk) {
            fclose($fp);

            return false;
        }
        $ehloName = gethostname() ?: 'localhost';
        $write($fp, 'EHLO ' . $ehloName);
        [$code] = smtp_read_response($fp);
        if ($code !== 250) {
            fclose($fp);

            return false;
        }
    }

    if ($user !== '' && $pass !== '') {
        $write($fp, 'AUTH LOGIN');
        [$code] = smtp_read_response($fp);
        if ($code !== 334) {
            fclose($fp);

            return false;
        }
        $write($fp, base64_encode($user));
        [$code] = smtp_read_response($fp);
        if ($code !== 334) {
            fclose($fp);

            return false;
        }
        $write($fp, base64_encode($pass));
        [$code] = smtp_read_response($fp);
        if ($code !== 235) {
            fclose($fp);

            return false;
        }
    }

    $write($fp, 'MAIL FROM:<' . $from . '>');

```

```

[code] = smtp_read_response($fp);
if ($code !== 250) {
    fclose($fp);

    return false;
}

fwrite($fp, 'RCPT TO:<' . $to . '>');
[code] = smtp_read_response($fp);
if ($code !== 250 && $code !== 251) {
    fclose($fp);

    return false;
}

fwrite($fp, 'DATA');
[code] = smtp_read_response($fp);
if ($code !== 354) {
    fclose($fp);

    return false;
}

$subjLine = preg_match('/[^\x20-\x7E]/', $subject)
    ? '?UTF-8?B?' . base64_encode($subject) . '?='
    : $subject;

$mimeHeaders = "MIME-Version: 1.0\r\n";
if ($bodyHtml !== null && $bodyHtml !== '') {
    $boundary = 'b_' . bin2hex(random_bytes(16));
    $mimeHeaders .= 'Content-Type: multipart/alternative; boundary="' . $boundary . "\"\r\n";
    $inner =
        '--' . $boundary . "\r\n"
        . "Content-Type: text/plain; charset=UTF-8\r\n"
        . "Content-Transfer-Encoding: 8bit\r\n\r\n"
        . $body . "\r\n\r\n"
        . '--' . $boundary . "\r\n"
        . "Content-Type: text/html; charset=UTF-8\r\n"
        . "Content-Transfer-Encoding: 8bit\r\n\r\n"
        . $bodyHtml . "\r\n\r\n"
        . '--' . $boundary . "--\r\n";
    $bodyOut = smtp_escape_data_body($inner);
} else {
    $mimeHeaders .= "Content-Type: text/plain; charset=UTF-8\r\n";
    $bodyOut = smtp_escape_data_body($body);
}

$payload =
    'From: ' . $fromName . ' <' . $from . ">\r\n"
    . 'To: ' . $to . "\r\n"
    . 'Subject: ' . $subjLine . "\r\n"
    . $mimeHeaders
    . 'Date: ' . date('r') . "\r\n"
    . "\r\n"
    . $bodyOut . "\r\n\r\n";

fwrite($fp, $payload);
[code] = smtp_read_response($fp);
if ($code !== 250) {
    fclose($fp);

    return false;
}

fwrite($fp, 'QUIT');
fclose($fp);

return true;
}

/**
 * Send email: authenticated SMTP if configured, else PHP mail() with optional Windows SMTP ini.
 * Optional $bodyHtml sends multipart/alternative (plain + HTML).
 */
function send_system_mail(string $to, string $subject, string $body, ?string $bodyHtml = null): bool
{
    $to = trim($to);
    if ($to === '' || !filter_var($to, FILTER_VALIDATE_EMAIL)) {
        return false;
    }

```

```

if (!defined('MAIL_ENABLED') || !MAIL_ENABLED) {
    return false;
}

$smtpHost = defined('MAIL_SMTP_HOST') ? trim((string) MAIL_SMTP_HOST) : '';
$smtpUser = defined('MAIL_SMTP_USER') ? trim((string) MAIL_SMTP_USER) : '';

if ($smtpHost !== '' && $smtpUser !== '') {
    return smtp_send_mail($to, $subject, $body, $bodyHtml);
}

$fromAddr = defined('MAIL_FROM_ADDRESS') ? MAIL_FROM_ADDRESS : 'noreply@localhost';
$fromName = defined('MAIL_FROM_NAME') ? MAIL_FROM_NAME : 'WPU SABLAE Portal';

if ($smtpHost !== '') {
    ini_set('SMTP', $smtpHost);
    ini_set('smtp_port', (string) (defined('MAIL_SMTP_PORT') ? (int) MAIL_SMTP_PORT : 25));
}
if (defined('MAIL_SENDMAIL_FROM') && MAIL_SENDMAIL_FROM !== '') {
    ini_set('sendmail_from', MAIL_SENDMAIL_FROM);
}

$subjectLine = $subject;
if (preg_match('/[^\x20-\x7E]/', $subject)) {
    $subjectLine = '=?UTF-8?B?' . base64_encode($subject) . '?=';
}

if ($bodyHtml !== null && $bodyHtml !== '') {
    $boundary = 'b_' . bin2hex(random_bytes(16));
    $headers = [
        'MIME-Version: 1.0',
        'From: ' . $fromName . ' <' . $fromAddr . '>',
        'Reply-To: ' . $fromAddr,
        'Content-Type: multipart/alternative; boundary="' . $boundary . '"',
    ];
    $message =
        '--' . $boundary . "\r\n"
        . "Content-Type: text/plain; charset=UTF-8\r\n"
        . "Content-Transfer-Encoding: 8bit\r\n\r\n"
        . $body . "\r\n\r\n"
        . '--' . $boundary . "\r\n"
        . "Content-Type: text/html; charset=UTF-8\r\n"
        . "Content-Transfer-Encoding: 8bit\r\n\r\n"
        . $bodyHtml . "\r\n\r\n"
        . '--' . $boundary . "--";

    return @mail($to, $subjectLine, $message, implode("\r\n", $headers));
}

$headers = [
    'MIME-Version: 1.0',
    'Content-type: text/plain; charset=UTF-8',
    'From: ' . $fromName . ' <' . $fromAddr . '>',
    'Reply-To: ' . $fromAddr,
];

return @mail($to, $subjectLine, $body, implode("\r\n", $headers));
}

/**
 * Email account credentials after a new account is registered (role-specific content, unified subject).
 *
 * @param string $role One of: super_admin, dean, program_chair, faculty, gened, admin, student, student_registration
 */
function send_account_credentials_mail(
    string $toEmail,
    string $fullName,
    string $username,
    string $plainPassword,
    string $role = 'dean',
    string $registeredEmail = ''
): bool {
    $registeredEmail = trim($registeredEmail) !== '' ? trim($registeredEmail) : trim($toEmail);
    [$plain, $html] = build_account_credentials_bodies(
        $fullName,
        $username,
        $plainPassword,
        $role,
        $registeredEmail
    );
}

```

```

    return send_system_mail($toEmail, mail_credentials_subject(), $plain, $html);
}

```

```

/**
 * @deprecated Use send_account_credentials_mail(..., 'dean') for explicit role.
 */
function send_dean_credentials_mail(string $toEmail, string $fullName, string $username, string $plainPassword): bool
{
    return send_account_credentials_mail($toEmail, $fullName, $username, $plainPassword, 'dean');
}

```

```

-----
FILE: includes/makeup_helpers.php
-----

```

```

<?php
declare(strict_types=1);

```

```

/**
 * Makeup-class helpers (shared). Requires db(), db_column_exists(), db_table_exists(),
 * parse_day_set(), days_to_set(), validate_schedule_rules(), checkConflicts().
 */

```

```

/**
 * Ensure makeup-class columns exist on schedules and schedule_change_requests.
 */

```

```

function ensure_makeup_schedule_support(): bool
{
    static $ready = null;
    if ($ready !== null) {
        return $ready;
    }
    $ready = false;
    if (!db_table_exists('schedules') || !db_table_exists('schedule_change_requests')) {
        return false;
    }
    try {
        $pdo = db();
        if (!db_column_exists('schedules', 'is_makeup')) {
            $pdo->exec('ALTER TABLE schedules ADD COLUMN is_makeup TINYINT(1) NOT NULL DEFAULT 0');
        }
        if (!db_column_exists('schedules', 'makeup_for_schedule_id')) {
            $pdo->exec('ALTER TABLE schedules ADD COLUMN makeup_for_schedule_id INT NULL');
        }
        if (!db_column_exists('schedule_change_requests', 'request_type')) {
            $pdo->exec(
                "ALTER TABLE schedule_change_requests
                ADD COLUMN request_type ENUM('change','makeup') NOT NULL DEFAULT 'change'"
            );
        }
        if (!db_column_exists('schedule_change_requests', 'proposed_day_of_week')) {
            $pdo->exec(
                "ALTER TABLE schedule_change_requests
                ADD COLUMN proposed_day_of_week SET('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday') NULL"
            );
        }
        if (!db_column_exists('schedule_change_requests', 'proposed_start_time')) {
            $pdo->exec('ALTER TABLE schedule_change_requests ADD COLUMN proposed_start_time TIME NULL');
        }
        if (!db_column_exists('schedule_change_requests', 'proposed_end_time')) {
            $pdo->exec('ALTER TABLE schedule_change_requests ADD COLUMN proposed_end_time TIME NULL');
        }
        if (!db_column_exists('schedule_change_requests', 'proposed_room_id')) {
            $pdo->exec('ALTER TABLE schedule_change_requests ADD COLUMN proposed_room_id INT NULL');
        }
        $ready = db_column_exists('schedules', 'is_makeup')
            && db_column_exists('schedule_change_requests', 'request_type');
    } catch (Throwable $e) {
        $ready = false;
    }
    return $ready;
}

```

```

/** Dean teaching-role requests are reviewed by institution admin, not the college dean. */
function schedule_change_needs_admin_review(?string $submitterRole = null): bool
{
    return ($submitterRole ?? (string) ($_SESSION['role'] ?? '')) === 'dean';
}

```

```

function schedule_change_reviewer_label(?string $submitterRole = null): string
{
    return schedule_change_needs_admin_review($submitterRole) ? 'Administration' : 'your dean';
}

/**
 * Faculty/room conflict messages for a proposed makeup slot.
 *
 * @param list<string> $days
 * @return list<string>
 */
function makeup_hard_conflict_messages(
    int $facultyId,
    int $roomId,
    array $days,
    string $startTime,
    string $endTime,
    string $semester,
    string $schoolYear,
    ?int $collegeId = null
): array {
    if ($facultyId < 1 || $roomId < 1 || $days === []) {
        return [];
    }
    $conflicts = checkConflicts(
        $facultyId,
        $roomId,
        $days,
        $startTime,
        $endTime,
        $semester,
        $schoolYear,
        null,
        $collegeId,
        true
    );
    $out = [];
    foreach ($conflicts as $c) {
        if (!is_array($c)) {
            continue;
        }
        $type = (string) ($c['type'] ?? '');
        if ($type !== 'faculty' && $type !== 'room') {
            continue;
        }
        $desc = trim((string) ($c['description'] ?? ''));
        if ($desc !== '') {
            $out[] = $desc;
        }
    }
    return array_values(array_unique($out));
}

/**
 * Create a temporary makeup schedule row from an approved makeup request.
 *
 * @param array<string,mixed> $request
 * @param array<string,mixed> $baseSchedule
 * @return array{ok:bool,schedule_id?:int,error?:string,conflicts?:list<array<string,mixed>>}
 */
function create_makeup_schedule_from_request(
    array $request,
    array $baseSchedule,
    int $createdByUserId,
    bool $forceDespiteConflicts = false
): array {
    if (!ensure_makeup_schedule_support()) {
        return ['ok' => false, 'error' => 'Makeup schedule support is not available. Run upgrade_roles.php.'];
    }

    $day = trim((string) ($request['proposed_day_of_week'] ?? ''));
    $startRaw = substr((string) ($request['proposed_start_time'] ?? ''), 0, 5);
    $endRaw = substr((string) ($request['proposed_end_time'] ?? ''), 0, 5);
    if ($startRaw === '' || $startRaw === '0') {
        $startRaw = substr((string) ($baseSchedule['start_time'] ?? ''), 0, 5);
    }
    if ($endRaw === '' || $endRaw === '0') {
        $endRaw = substr((string) ($baseSchedule['end_time'] ?? ''), 0, 5);
    }
    $startTime = $startRaw !== '' ? ($startRaw . ':00') : '';

```

```

$endTime = $endRaw != '' ? ($endRaw . ':00') : '';
$roomId = (int) ($request['proposed_room_id'] ?? 0);
if ($roomId < 1) {
    $roomId = (int) ($baseSchedule['room_id'] ?? 0);
}

$days = parse_day_set($day);
if ($days === [] && $day === '' && !empty($baseSchedule['day_of_week'])) {
    $days = parse_day_set((string) $baseSchedule['day_of_week']);
    if (count($days) > 1) {
        $days = [$days[0]];
    }
}
if ($days === [] || $roomId < 1 || $startTime === '' || $endTime === '') {
    return ['ok' => false, 'error' => 'Makeup day, time, and room are required.'];
}

$ruleCheck = validate_schedule_rules('Custom', $days, $startTime, $endTime);
if (empty($ruleCheck['ok'])) {
    $errs = $ruleCheck['errors'] ?? [];
    return ['ok' => false, 'error' => implode(' ', is_array($errs) ? $errs : [])];
}

$facultyId = (int) ($baseSchedule['faculty_id'] ?? 0);
$collegeId = isset($baseSchedule['college_id']) && $baseSchedule['college_id'] !== null
    ? (int) $baseSchedule['college_id']
    : null;
$semester = (string) ($baseSchedule['semester'] ?? '');
$schoolYear = (string) ($baseSchedule['school_year'] ?? '');
$conflicts = checkConflicts(
    $facultyId,
    $roomId,
    $days,
    $startTime,
    $endTime,
    $semester,
    $schoolYear,
    null,
    $collegeId,
    true
);
// Makeup slots are temporary – only hard faculty/room overlaps block by default.
$hardConflicts = [];
$descriptions = makeup_hard_conflict_messages(
    $facultyId,
    $roomId,
    $days,
    $startTime,
    $endTime,
    $semester,
    $schoolYear,
    $collegeId
);
foreach ($conflicts as $c) {
    $type = is_array($c) ? (string) ($c['type'] ?? '') : '';
    if ($type === 'faculty' || $type === 'room') {
        $hardConflicts[] = $c;
    }
}
if ($hardConflicts !== [] && !$forceDespiteConflicts) {
    return [
        'ok' => false,
        'error' => 'Conflicts prevent creating the makeup slot.'
        . ($descriptions !== [] ? ' ' . implode(' ', $descriptions) : '')
        . ' Use “Approve anyway” if you still want to create it.',
        'conflicts' => $hardConflicts,
        'conflict_messages' => $descriptions,
    ];
}

$daySet = days_to_set($days);
$cols = [
    'faculty_id', 'course_id', 'room_id', 'college_id', 'schedule_type', 'day_of_week',
    'start_time', 'end_time', 'semester', 'school_year', 'academic_year', 'created_by',
    'is_makeup', 'makeup_for_schedule_id',
];
$val = [
    $facultyId,
    (int) ($baseSchedule['course_id'] ?? 0),
    $roomId,

```

```

        $collegeId,
        'Custom',
        $daySet,
        $startTime,
        $endTime,
        $semester,
        $schoolYear,
        (string) ($baseSchedule['academic_year'] ?? ''),
        $createdByUserId,
        1,
        (int) ($baseSchedule['id'] ?? 0) ?: null,
    ];

    foreach (['program', 'year_level', 'section', 'session_kind'] as $optionalCol) {
        if (db_column_exists('schedules', $optionalCol) && array_key_exists($optionalCol, $baseSchedule)) {
            $cols[] = $optionalCol;
            $vals[] = $baseSchedule[$optionalCol];
        }
    }
    if (db_column_exists('schedules', 'online_class_url') && !empty($baseSchedule['online_class_url'])) {
        $cols[] = 'online_class_url';
        $vals[] = $baseSchedule['online_class_url'];
    }

    try {
        $placeholders = implode(',', array_fill(0, count($cols), '?'));
        $sql = 'INSERT INTO schedules (' . implode(' ', $cols) . ') VALUES (' . $placeholders . ')';
        db()->prepare($sql)->execute($vals);
        $newId = (int) db()->lastInsertId();
    } catch (Throwable $e) {
        return ['ok' => false, 'error' => 'Could not save makeup schedule: ' . $e->getMessage()];
    }

    if ($newId < 1) {
        return ['ok' => false, 'error' => 'Makeup schedule insert did not return an id.'];
    }

    return ['ok' => true, 'schedule_id' => $newId];
}

```

FILE: includes/messaging_helpers.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/db.php';

function messaging_has_assigned_program_column(): bool
{
    static $cache = null;
    if ($cache !== null) {
        return $cache;
    }
    $stmt = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.COLUMNS
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ? AND COLUMN_NAME = ?'
    );
    $stmt->execute([DB_NAME, 'users', 'assigned_program']);
    $cache = (int) $stmt->fetchColumn() > 0;
    return $cache;
}

function messaging_table_exists(): bool
{
    static $cache = null;
    if ($cache !== null) {
        return $cache;
    }
    $stmt = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.TABLES WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?'
    );
    $stmt->execute([DB_NAME, 'internal_messages']);
    $cache = (int) $stmt->fetchColumn() > 0;
    return $cache;
}

function messaging_thread_max_messages(): int
{

```



```

    if (defined('MESSAGING_THREAD_MAX_MESSAGES')) {
        return max(0, (int) MESSAGING_THREAD_MAX_MESSAGES);
    }
    return 10;
}

/** Drop oldest messages in a two-user thread when over the configured FIFO capacity. */
function messaging_enforce_thread_fifo(int $userA, int $userB): void
{
    if (!messaging_table_exists()) {
        return;
    }
    $max = messaging_thread_max_messages();
    if ($max < 1) {
        return;
    }

    $st = db()->prepare(
        'SELECT COUNT(*) FROM internal_messages
        WHERE (sender_user_id = ? AND recipient_user_id = ?)
        OR (sender_user_id = ? AND recipient_user_id = ?)'
    );
    $st->execute([$userA, $userB, $userB, $userA]);
    $count = (int) $st->fetchColumn();
    if ($count <= $max) {
        return;
    }

    $excess = $count - $max;
    $attachmentSelect = messaging_has_memo_columns() ? ', attachment_stored_name' : '';
    $st = db()->prepare(
        'SELECT id' . $attachmentSelect . ' FROM internal_messages
        WHERE (sender_user_id = ? AND recipient_user_id = ?)
        OR (sender_user_id = ? AND recipient_user_id = ?)
        ORDER BY created_at ASC, id ASC
        LIMIT ' . $excess
    );
    $st->execute([$userA, $userB, $userB, $userA]);
    foreach ($st->fetchAll(PDO::FETCH_ASSOC) ?: [] as $row) {
        $messageId = (int) ($row['id'] ?? 0);
        if ($messageId < 1) {
            continue;
        }
        $storedName = trim((string) ($row['attachment_stored_name'] ?? ''));
        db()->prepare('DELETE FROM internal_messages WHERE id = ?')->execute([$messageId]);
        if ($storedName !== '') {
            messaging_delete_attachment_if_unused($storedName);
        }
    }
}

function messaging_db_table_exists(string $tableName): bool
{
    $st = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.TABLES WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?'
    );
    $st->execute([DB_NAME, $tableName]);
    return (int) $st->fetchColumn() > 0;
}

function messaging_has_memo_columns(): bool
{
    static $cache = null;
    if ($cache !== null) {
        return $cache;
    }
    if (!messaging_table_exists()) {
        $cache = false;
        return false;
    }
    $stmt = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.COLUMNS
        WHERE TABLE_SCHEMA = ?
        AND TABLE_NAME = ?
        AND COLUMN_NAME IN (?, ?, ?, ?, ?)'
    );
    $stmt->execute([
        DB_NAME,
        'internal_messages',
        'subject',
    ]);
}

```

```

        'is_memo',
        'attachment_original_name',
        'attachment_stored_name',
        'attachment_mime',
    ]]);
    $cache = (int) $stmt->fetchColumn() === 5;
    return $cache;
}

function messaging_select_columns(): string
{
    if (messaging_has_memo_columns()) {
        return 'id, sender_user_id, recipient_user_id, subject, body, is_memo, attachment_original_name, attachment_stored_name, attachment_mime, created_at';
    }
    return 'id, sender_user_id, recipient_user_id, "" AS subject, body, 0 AS is_memo, "" AS attachment_original_name, "" AS attachment_stored_name, "" AS attachment_mime, created_at';
}

function messaging_user_department_by_user_id(int $userId): string
{
    $st = db()->prepare('SELECT department FROM faculty WHERE user_id = ? LIMIT 1');
    $st->execute([$userId]);
    return trim((string) $st->fetchColumn());
}

function messaging_faculty_id_by_user_id(int $userId): ?int
{
    $st = db()->prepare('SELECT id FROM faculty WHERE user_id = ? LIMIT 1');
    $st->execute([$userId]);
    $id = $st->fetchColumn();
    return $id !== false ? (int) $id : null;
}

function messaging_student_id_by_user_id(int $userId): ?int
{
    $st = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.TABLES
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?'
    );
    $st->execute([DB_NAME, 'classroom_students']);
    if ((int) $st->fetchColumn() < 1) {
        return null;
    }

    $st = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.COLUMNS
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ? AND COLUMN_NAME = ?'
    );
    $st->execute([DB_NAME, 'classroom_students', 'user_id']);
    if ((int) $st->fetchColumn() < 1) {
        return null;
    }

    $st = db()->prepare('SELECT id FROM classroom_students WHERE user_id = ? LIMIT 1');
    $st->execute([$userId]);
    $id = $st->fetchColumn();
    return $id !== false ? (int) $id : null;
}

function messaging_is_gened_faculty_user(int $userId): bool
{
    $st = db()->prepare('SELECT COALESCE(is_gened,0) FROM faculty WHERE user_id = ? LIMIT 1');
    $st->execute([$userId]);
    return (int) $st->fetchColumn() === 1;
}

function messaging_ge_dean_name_matches_hint(string $fullName, string $hint): bool
{
    $normalize = static function (string $value): string {
        $value = strtolower($value);
        return trim((string) preg_replace('/^[a-z0-9]+/', ' ', $value));
    };
    $name = $normalize($fullName);
    $hintNorm = $normalize($hint);
    if ($hintNorm === '') {
        return false;
    }
    foreach (array_filter(explode(' ', $hintNorm)) as $token) {
        if (!str_contains($name, $token)) {

```

```

        return false;
    }
}
return true;
}

/** Active dean user id designated for GE faculty messaging (Francis Alterado by default). */
function messaging_ge_dean_user_id(): ?int
{
    static $resolved = null;
    static $done = false;
    if ($done) {
        return $resolved;
    }
    $done = true;

    if (defined('GE_DEAN_USER_ID') && (int) GE_DEAN_USER_ID > 0) {
        $id = (int) GE_DEAN_USER_ID;
        $u = messaging_user_row($id);
        if ($u && $u['role'] === 'dean' && $u['is_active']) {
            $resolved = $id;
            return $resolved;
        }
    }

    $hint = defined('GE_DEAN_NAME_HINT') ? trim((string) GE_DEAN_NAME_HINT) : 'Francis Alterado';
    if ($hint === '') {
        return null;
    }

    $st = db()->query("SELECT id, full_name FROM users WHERE is_active = 1 AND role = 'dean'");
    foreach ($st->fetchAll(PDO::FETCH_ASSOC) ?: [] as $row) {
        if (messaging_ge_dean_name_matches_hint((string) ($row['full_name'] ?? ''), $hint)) {
            $resolved = (int) ($row['id'] ?? 0);
            return $resolved > 0 ? $resolved : null;
        }
    }

    return null;
}

function messaging_is_ge_dean_user(int $userId): bool
{
    $geDeanId = messaging_ge_dean_user_id();
    return $geDeanId !== null && $userId === $geDeanId;
}

function messaging_faculty_can_message_student(int $facultyUserId, int $studentUserId): bool
{
    if (!messaging_db_table_exists('online_classrooms') || !messaging_db_table_exists('classroom_enrollments')) {
        return false;
    }

    $facultyId = messaging_faculty_id_by_user_id($facultyUserId);
    $studentId = messaging_student_id_by_user_id($studentUserId);
    if ($facultyId === null || $studentId === null) {
        return false;
    }

    $st = db()->prepare(
        'SELECT COUNT(*)
        FROM classroom_enrollments ce
        INNER JOIN online_classrooms oc ON oc.id = ce.classroom_id
        WHERE oc.faculty_id = ? AND ce.student_id = ?'
    );
    $st->execute([$facultyId, $studentId]);
    return (int) $st->fetchColumn() > 0;
}

function messaging_student_can_message_faculty(int $studentUserId, int $facultyUserId): bool
{
    return messaging_faculty_can_message_student($facultyUserId, $studentUserId);
}

/** @return array{id:int,role:string,college_id:?int,is_active:int,assigned_program:string,full_name:string}|null */
function messaging_user_row(int $id): ?array
{
    $st = db()->prepare(
        'SELECT u.id, u.full_name, u.role, COALESCE(u.college_id, f.college_id) AS college_id, u.is_active, '
        . (messaging_has_assigned_program_column() ? 'assigned_program' : '"" AS assigned_program')
    );

```

```

        . ' FROM users u
        LEFT JOIN faculty f ON f.user_id = u.id
        WHERE u.id = ? LIMIT 1'
    );
    $st->execute([$id]);
    $r = $st->fetch(PDO::FETCH_ASSOC);
    return $r ? [
        'id' => (int) $r['id'],
        'full_name' => (string) $r['full_name'],
        'role' => (string) $r['role'],
        'college_id' => $r['college_id'] !== null ? (int) $r['college_id'] : null,
        'is_active' => (int) $r['is_active'],
        'assigned_program' => (string) ($r['assigned_program'] ?? ''),
    ] : null;
}

function messaging_can_open_thread(int $viewerId, int $otherId): bool
{
    if ($viewerId === $otherId) {
        return false;
    }
    if (messaging_can_send($viewerId, $otherId) || messaging_can_send($otherId, $viewerId)) {
        return true;
    }
    if (!messaging_table_exists()) {
        return false;
    }
    $st = db()->prepare(
        'SELECT COUNT(*) FROM internal_messages
        WHERE (sender_user_id = ? AND recipient_user_id = ?)
        OR (sender_user_id = ? AND recipient_user_id = ?)'
    );
    $st->execute([$viewerId, $otherId, $otherId, $viewerId]);
    return (int) $st->fetchColumn() > 0;
}

function messaging_can_send(int $senderId, int $recipientId): bool
{
    if ($senderId === $recipientId) {
        return false;
    }
    $s = messaging_user_row($senderId);
    $r = messaging_user_row($recipientId);
    if (!$s || !$r || !$s['is_active'] || !$r['is_active']) {
        return false;
    }
    $sr = $s['role'];
    $rr = $r['role'];
    $sc = $s['college_id'];
    $rc = $r['college_id'];
    $sp = trim((string) ($s['assigned_program'] ?? ''));
    $rp = trim((string) ($r['assigned_program'] ?? ''));

    if ($sr === 'admin') {
        if (in_array($rr, ['dean', 'program_chair', 'gened'], true)) {
            return true;
        }
        return $rr === 'faculty';
    }
    if ($sr === 'dean') {
        if ($rr === 'admin' || $rr === 'gened') {
            return true;
        }
        if ($rr === 'program_chair' && $sc !== null && $rc !== null && $sc === $rc) {
            return true;
        }
        if ($rr === 'faculty' && $sc !== null && $rc !== null && $sc === $rc) {
            if (messaging_is_gened_faculty_user($recipientId)) {
                return messaging_is_ge_dean_user($senderId);
            }
            return true;
        }
        return false;
    }
    if ($sr === 'program_chair') {
        if ($rr === 'admin') {
            return true;
        }
        if ($rr === 'dean' && $sc !== null && $rc !== null && $sc === $rc) {
            return true;
        }
    }
}

```

```

    }
    if ($rr === 'faculty' && $sc !== null && $rc !== null && $sc === $rc && $sp !== '') {
        return messaging_user_department_by_user_id($recipientId) === $sp;
    }
    return false;
}
if ($sr === 'faculty') {
    if ($rr === 'student') {
        return messaging_faculty_can_message_student($senderId, $recipientId);
    }
    if ($rr === 'dean') {
        if (messaging_is_gened_faculty_user($senderId)) {
            $geDeanId = messaging_ge_dean_user_id();
            return $geDeanId !== null && $recipientId === $geDeanId;
        }
        return $sc !== null && $rc !== null && $sc === $rc;
    }
    if ($rr === 'program_chair' && $sc !== null && $rc !== null && $sc === $rc && $rp !== '') {
        return messaging_user_department_by_user_id($senderId) === $rp;
    }
    if ($rr === 'gened') {
        return messaging_is_gened_faculty_user($senderId);
    }
    return false;
}
if ($sr === 'student') {
    if ($rr === 'faculty') {
        return messaging_student_can_message_faculty($senderId, $recipientId);
    }
    return false;
}
if ($sr === 'gened') {
    if (in_array($rr, ['dean', 'admin'], true)) {
        return true;
    }
    return $rr === 'faculty' && messaging_is_gened_faculty_user($recipientId);
}
return false;
}

/**
 * Users the current user may start (or continue) a conversation with.
 * @return list<array{id:int,full_name:string,role:string,college_id:?int}>
 */
function messaging_allowed_recipients(int $forUserId): array
{
    $me = messaging_user_row($forUserId);
    if (!$me || !$me['is_active']) {
        return [];
    }
    $role = $me['role'];
    $cid = $me['college_id'];

    if ($role === 'admin') {
        $st = db()->query(
            "SELECT id, full_name, role, college_id FROM users
            WHERE is_active = 1 AND role IN ('dean','program_chair','gened')
            ORDER BY role, full_name"
        );
        return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
    }
    if ($role === 'dean' && $cid !== null) {
        $includeGenedFaculty = messaging_is_ge_dean_user($forUserId);
        $st = db()->prepare(
            "SELECT u.id, u.full_name, u.role, u.college_id
            FROM users u
            LEFT JOIN faculty f ON f.user_id = u.id
            WHERE u.is_active = 1 AND (
                u.role = 'admin'
                OR u.role = 'gened'
                OR (u.role = 'program_chair' AND u.college_id = ?)
                OR (u.role = 'faculty' AND u.college_id = ? AND (? = 1 OR COALESCE(f.is_gened,0) = 0))
            )
            ORDER BY u.role, u.full_name"
        );
        $st->execute([$cid, $cid, $includeGenedFaculty ? 1 : 0]);
        return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
    }
    if ($role === 'dean' && $cid === null) {
        $st = db()->query(

```

```

        "SELECT id, full_name, role, college_id FROM users
        WHERE is_active = 1 AND role IN ('admin','gened')
        ORDER BY role, full_name"
    );
    return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
}
if ($role === 'program_chair' && $cid !== null) {
    $rows = [];
    $st = db()->prepare(
        "SELECT id, full_name, role, college_id FROM users
        WHERE is_active = 1 AND (
            role = 'admin'
            OR (role = 'dean' AND college_id = ?)
        )
        ORDER BY role, full_name"
    );
    $st->execute([$cid]);
    $rows = $st->fetchAll(PDO::FETCH_ASSOC) ?: [];

    $programs = function_exists('program_chair_assigned_programs')
        ? program_chair_assigned_programs((int) ($me['id'] ?? 0))
        : [];
    if ($programs === []) {
        $program = trim((string) ($me['assigned_program'] ?? ''));
        if ($program !== '') {
            $programs = [$program];
        }
    }
    if ($programs !== []) {
        $holds = implode(',', array_fill(0, count($programs), '?'));
        $st = db()->prepare(
            "SELECT u.id, u.full_name, u.role, u.college_id
            FROM users u
            INNER JOIN faculty f ON f.user_id = u.id
            WHERE u.is_active = 1 AND u.role = 'faculty' AND u.college_id = ? AND f.department IN ({$holds})
            ORDER BY u.full_name"
        );
        $st->execute(array_merge([$cid], $programs));
        $rows = array_merge($rows, $st->fetchAll(PDO::FETCH_ASSOC) ?: []);
    }
    return $rows;
}
if ($role === 'faculty' && $cid !== null) {
    $facultyId = messaging_faculty_id_by_user_id($forUserId);
    if (
        $facultyId !== null
        && messaging_db_table_exists('classroom_students')
        && messaging_db_table_exists('classroom_enrollments')
        && messaging_db_table_exists('online_classrooms')
    ) {
        $st = db()->prepare(
            "SELECT DISTINCT u.id, u.full_name, u.role, u.college_id
            FROM users u
            INNER JOIN classroom_students cs ON cs.user_id = u.id
            INNER JOIN classroom_enrollments ce ON ce.student_id = cs.id
            INNER JOIN online_classrooms oc ON oc.id = ce.classroom_id
            WHERE u.is_active = 1 AND u.role = 'student' AND oc.faculty_id = ?
            ORDER BY u.full_name"
        );
        $st->execute([$facultyId]);
        return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
    }

    return [];
}
if ($role === 'gened') {
    $st = db()->query(
        "SELECT id, full_name, role, college_id FROM users
        WHERE is_active = 1 AND role IN ('admin','dean')
        ORDER BY role, full_name"
    );
    $rows = $st->fetchAll(PDO::FETCH_ASSOC) ?: [];

    $st = db()->query(
        "SELECT u.id, u.full_name, u.role, u.college_id
        FROM users u
        INNER JOIN faculty f ON f.user_id = u.id
        WHERE u.is_active = 1 AND u.role = 'faculty' AND COALESCE(f.is_gened,0) = 1
        ORDER BY u.full_name"
    );
}

```

```

        return array_merge($rows, $st->fetchAll(PDO::FETCH_ASSOC) ?: []);
    }
    if (
        $role === 'student'
        && messaging_db_table_exists('classroom_students')
        && messaging_db_table_exists('classroom_enrollments')
        && messaging_db_table_exists('online_classrooms')
    ) {
        $studentId = messaging_student_id_by_user_id($forUserId);
        if ($studentId === null) {
            return [];
        }

        $st = db()->prepare(
            "SELECT DISTINCT u.id, u.full_name, u.role, u.college_id
            FROM users u
            INNER JOIN faculty f ON f.user_id = u.id
            INNER JOIN online_classrooms oc ON oc.faculty_id = f.id
            INNER JOIN classroom_enrollments ce ON ce.classroom_id = oc.id
            WHERE u.is_active = 1 AND u.role = 'faculty' AND ce.student_id = ?
            ORDER BY u.full_name"
        );
        $st->execute([$studentId]);
        return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
    }
    return [];
}

/**
 * Dean, program chair, and GEN ED contacts a faculty member may message in Inbox.
 *
 * @return list<array{id:int,full_name:string,role:string,college_id:int}>
 */
function messaging_faculty_inbox_recipients(int $forUserId): array
{
    $me = messaging_user_row($forUserId);
    if (!$me || $me['role'] !== 'faculty' || !$me['is_active']) {
        return [];
    }

    $cid = $me['college_id'];
    $department = messaging_user_department_by_user_id($forUserId);
    $isGenedFaculty = messaging_is_gened_faculty_user($forUserId);
    $rows = [];

    if ($isGenedFaculty) {
        $geDeanId = messaging_ge_dean_user_id();
        if ($geDeanId !== null) {
            $geDean = messaging_user_row($geDeanId);
            if ($geDean) {
                $rows[] = [
                    'id' => $geDean['id'],
                    'full_name' => $geDean['full_name'],
                    'role' => $geDean['role'],
                    'college_id' => $geDean['college_id'],
                ];
            }
        }
    }
    $st = db()->query(
        "SELECT id, full_name, role, college_id FROM users
        WHERE is_active = 1 AND role = 'gened'
        ORDER BY full_name"
    );
    $rows = array_merge($rows, $st->fetchAll(PDO::FETCH_ASSOC) ?: []);
} elseif ($cid !== null) {
    $st = db()->prepare(
        "SELECT id, full_name, role, college_id FROM users
        WHERE is_active = 1 AND role = 'dean' AND college_id = ?
        ORDER BY full_name"
    );
    $st->execute([$cid]);
    $rows = $st->fetchAll(PDO::FETCH_ASSOC) ?: [];

    if ($department !== '' && messaging_has_assigned_program_column()) {
        $st = db()->prepare(
            "SELECT id, full_name, role, college_id FROM users
            WHERE is_active = 1 AND role = 'program_chair' AND college_id = ? AND assigned_program = ?
            ORDER BY full_name"
        );
        $st->execute([$cid, $department]);
    }
}

```

```

$rows = array_merge($rows, $st->fetchAll(PDO::FETCH_ASSOC) ?: []);

if (function_exists('program_chair_programs_table_ready') && program_chair_programs_table_ready()) {
    $st = db()->prepare(
        "SELECT u.id, u.full_name, u.role, u.college_id
        FROM users u
        INNER JOIN program_chair_programs pcp ON pcp.user_id = u.id
        WHERE u.is_active = 1 AND u.role = 'program_chair' AND u.college_id = ? AND pcp.program_name = ?
        ORDER BY u.full_name"
    );
    $st->execute([$cid, $department]);
    $rows = array_merge($rows, $st->fetchAll(PDO::FETCH_ASSOC) ?: []);
}
}

$seen = [];
$out = [];
foreach ($rows as $row) {
    $id = (int) ($row['id'] ?? 0);
    if ($id < 1 || isset($seen[$id]) || !messaging_can_send($forUserId, $id)) {
        continue;
    }
    $seen[$id] = true;
    $out[] = [
        'id' => $id,
        'full_name' => (string) ($row['full_name'] ?? ''),
        'role' => (string) ($row['role'] ?? ''),
        'college_id' => isset($row['college_id']) && $row['college_id'] !== null ? (int) $row['college_id'] : null,
    ];
}

return $out;
}

/**
 * @return list<array{id:int,full_name:string,role:string,college_id:?int,department:string}>
 */
function messaging_memo_recipients(int $forUserId): array
{
    $me = messaging_user_row($forUserId);
    if (!$me || !$me['is_active']) {
        return [];
    }
    $role = $me['role'];
    $cid = $me['college_id'];

    if ($role === 'admin') {
        $st = db()->query(
            "SELECT u.id, u.full_name, u.role, u.college_id, COALESCE(f.department, '') AS department
            FROM users u
            INNER JOIN faculty f ON f.user_id = u.id
            WHERE u.is_active = 1 AND u.role = 'faculty'
            ORDER BY u.full_name"
        );
        return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
    }
    if ($role === 'dean' && $cid !== null) {
        $st = db()->prepare(
            "SELECT u.id, u.full_name, u.role, u.college_id, COALESCE(f.department, '') AS department
            FROM users u
            INNER JOIN faculty f ON f.user_id = u.id
            WHERE u.is_active = 1 AND u.role = 'faculty' AND u.college_id = ? AND COALESCE(f.is_gened, 0) = 0
            ORDER BY u.full_name"
        );
        $st->execute([$cid]);
        return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
    }
    if ($role === 'program_chair' && $cid !== null) {
        $programs = function_exists('program_chair_assigned_programs')
            ? program_chair_assigned_programs((int) ($me['id'] ?? 0))
            : [];
        if ($programs === []) {
            $program = trim((string) ($me['assigned_program'] ?? ''));
            if ($program !== '') {
                $programs = [$program];
            }
        }
        if ($programs === []) {
            return [];
        }
    }
}

```



```

    }
    $holds = implode(',', array_fill(0, count($programs), '?'));
    $st = db()->prepare(
        "SELECT u.id, u.full_name, u.role, u.college_id, COALESCE(f.department, '') AS department
        FROM users u
        INNER JOIN faculty f ON f.user_id = u.id
        WHERE u.is_active = 1 AND u.role = 'faculty' AND u.college_id = ? AND f.department IN ({$holds})
        ORDER BY u.full_name"
    );
    $st->execute(array_merge([$cid], $programs));
    return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
}
if ($role === 'gened') {
    $st = db()->query(
        "SELECT u.id, u.full_name, u.role, u.college_id, 'General Education' AS department
        FROM users u
        INNER JOIN faculty f ON f.user_id = u.id
        WHERE u.is_active = 1 AND u.role = 'faculty' AND COALESCE(f.is_gened, 0) = 1
        ORDER BY u.full_name"
    );
    return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
}
return [];
}

function messaging_unread_count(int $userId): int
{
    if (!messaging_table_exists()) {
        return 0;
    }
    $st = db()->prepare('SELECT COUNT(*) FROM internal_messages WHERE recipient_user_id = ? AND read_at IS NULL');
    $st->execute([$userId]);
    return (int) $st->fetchColumn();
}

/**
 * @return list<array{partner_id:int,full_name:string,role:string,last_at:string,unread:int,preview:string}>
 */
function messaging_conversation_list(int $userId): array
{
    $st = db()->prepare(
        'SELECT DISTINCT CASE WHEN sender_user_id = ? THEN recipient_user_id ELSE sender_user_id END AS partner_id
        FROM internal_messages
        WHERE sender_user_id = ? OR recipient_user_id = ?'
    );
    $st->execute([$userId, $userId, $userId]);
    $partnerIds = array_map('intval', $st->fetchAll(PDO::FETCH_COLUMN) ?: []);
    if ($partnerIds === []) {
        return [];
    }

    $list = [];
    foreach ($partnerIds as $pid) {
        $u = messaging_user_row($pid);
        if (!$u) {
            continue;
        }
        $st2 = db()->prepare(
            'SELECT ' . messaging_select_columns() . '
            FROM internal_messages
            WHERE (sender_user_id = ? AND recipient_user_id = ?) OR (sender_user_id = ? AND recipient_user_id = ?)
            ORDER BY created_at DESC LIMIT 1'
        );
        $st2->execute([$userId, $pid, $pid, $userId]);
        $last = $st2->fetch(PDO::FETCH_ASSOC);
        $st3 = db()->prepare(
            'SELECT COUNT(*) FROM internal_messages WHERE sender_user_id = ? AND recipient_user_id = ? AND read_at IS NULL'
        );
        $st3->execute([$pid, $userId]);
        $unread = (int) $st3->fetchColumn();
        if ($last) {
            $preview = (int) ($last['is_memo'] ?? 0) === 1
                ? ('MEMO: ' . trim((string) ($last['subject'] ?? '')))
                : (string) ($last['body'] ?? '');
        } else {
            $preview = '';
        }
        if (function_exists('mb_substr')) {
            $preview = mb_strlen($preview) > 80 ? mb_substr($preview, 0, 80) . '...' : $preview;
        } else {

```

```

        $preview = strlen($preview) > 80 ? substr($preview, 0, 80) . '...' : $preview;
    }
    $list[] = [
        'partner_id' => $pid,
        'full_name' => $u['full_name'],
        'role' => $u['role'],
        'last_at' => $last ? (string) $last['created_at'] : '',
        'unread' => $unread,
        'preview' => $preview,
    ];
}

usort($list, static function (array $a, array $b): int {
    return strcmp($b['last_at'], $a['last_at']);
});

return $list;
}

/**
 * @return
list<array{id:int,sender_user_id:int,recipient_user_id:int,subject:string,body:string,is_memo:int,attachment_original_name:string,attachment_stored_name:string,attachment_mime:string,created_at:string,mime:bool}>
 */
function messaging_thread(int $userId, int $otherId, ?int $limit = null): array
{
    $max = messaging_thread_max_messages();
    if ($limit === null) {
        $limit = $max > 0 ? $max : 500;
    }
    $st = db()->prepare(
        'SELECT ' . messaging_select_columns() . '
        FROM internal_messages
        WHERE (sender_user_id = ? AND recipient_user_id = ?) OR (sender_user_id = ? AND recipient_user_id = ?)
        ORDER BY created_at ASC
        LIMIT ' . max(1, min(500, $limit))
    );
    $st->execute([$userId, $otherId, $otherId, $userId]);
    $rows = $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
    $out = [];
    foreach ($rows as $r) {
        $sid = (int) $r['sender_user_id'];
        $out[] = [
            'id' => (int) $r['id'],
            'sender_user_id' => $sid,
            'recipient_user_id' => (int) $r['recipient_user_id'],
            'subject' => (string) ($r['subject'] ?? ''),
            'body' => (string) $r['body'],
            'is_memo' => (int) ($r['is_memo'] ?? 0),
            'attachment_original_name' => (string) ($r['attachment_original_name'] ?? ''),
            'attachment_stored_name' => (string) ($r['attachment_stored_name'] ?? ''),
            'attachment_mime' => (string) ($r['attachment_mime'] ?? ''),
            'created_at' => (string) $r['created_at'],
            'mime' => $sid === $userId,
        ];
    }
    return $out;
}

function messaging_mark_read(int $viewerId, int $otherId): void
{
    db()->prepare(
        'UPDATE internal_messages SET read_at = CURRENT_TIMESTAMP
        WHERE recipient_user_id = ? AND sender_user_id = ? AND read_at IS NULL'
    )->execute([$viewerId, $otherId]);
}

function messaging_attachment_dir(): string
{
    return BASE_PATH . DIRECTORY_SEPARATOR . 'uploads' . DIRECTORY_SEPARATOR . 'message_attachments';
}

function messaging_attachment_path(string $storedName): string
{
    return messaging_attachment_dir() . DIRECTORY_SEPARATOR . basename($storedName);
}

/**
 * @param array<string,mixed> $file
 * @return array{original_name:string,stored_name:string,mime:string}|null
 */

```

```

*/
function messaging_store_attachment(array $file): ?array
{
    if (($file['error'] ?? UPLOAD_ERR_NO_FILE) === UPLOAD_ERR_NO_FILE) {
        return null;
    }
    if (($file['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_OK) {
        throw new RuntimeException('Attachment upload failed.');
```

```

        db()->prepare(
            'INSERT INTO internal_messages (sender_user_id, recipient_user_id, subject, body, is_memo, attachment_original_name,
            attachment_stored_name, attachment_mime)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)'
        )->execute([
            $fromId,
            $toId,
            $subject,
            $body,
            $isMemo ? 1 : 0,
            (string) ($attachment['original_name'] ?? ''),
            (string) ($attachment['stored_name'] ?? ''),
            (string) ($attachment['mime'] ?? ''),
        ]);
        messaging_enforce_thread_fifo($fromId, $toId);
        return;
    }

    db()->prepare(
        'INSERT INTO internal_messages (sender_user_id, recipient_user_id, body) VALUES (?, ?, ?)'
    )->execute([$fromId, $toId, $body]);
    messaging_enforce_thread_fifo($fromId, $toId);
}

/**
 * @param int[] $recipientIds
 * @param array<string,mixed>|null $file
 */
function messaging_send_memo(int $fromId, array $recipientIds, string $subject, string $body, ?array $file): int
{
    $sender = messaging_user_row($fromId);
    if (!$sender || !in_array((string) $sender['role'], ['admin', 'dean', 'gened', 'program_chair'], true)) {
        throw new RuntimeException('You cannot send memos.');
```

```

    if ($storedName === '') {
        return;
    }

    $st = db()->prepare(
        'SELECT COUNT(*) FROM internal_messages WHERE attachment_stored_name = ?'
    );
    $st->execute([$storedName]);
    if ((int) $st->fetchColumn() > 0) {
        return;
    }

    $path = messaging_attachment_path($storedName);
    if (is_file($path)) {
        @unlink($path);
    }
}

function messaging_delete_message(int $messageId, int $userId): bool
{
    if (!messaging_table_exists()) {
        return false;
    }

    $st = db()->prepare(
        'SELECT id, attachment_stored_name
        FROM internal_messages
        WHERE id = ? AND sender_user_id = ?
        LIMIT 1'
    );
    $st->execute([$messageId, $userId]);
    $row = $st->fetch(PDO::FETCH_ASSOC);
    if (!$row) {
        return false;
    }

    $storedName = trim((string) ($row['attachment_stored_name'] ?? ''));
    db()->prepare(
        'DELETE FROM internal_messages WHERE id = ? AND sender_user_id = ?'
    )->execute([$messageId, $userId]);

    if ($storedName !== '') {
        messaging_delete_attachment_if_unused($storedName);
    }

    return true;
}

/** @return array<string,mixed>|null */
function messaging_message_for_user(int $messageId, int $userId): ?array
{
    if (!messaging_table_exists()) {
        return null;
    }
    $st = db()->prepare(
        'SELECT ' . messaging_select_columns() . '
        FROM internal_messages
        WHERE id = ? AND (sender_user_id = ? OR recipient_user_id = ?)
        LIMIT 1'
    );
    $st->execute([$messageId, $userId, $userId]);
    $row = $st->fetch(PDO::FETCH_ASSOC);
    return $row ?: null;
}

function messaging_role_badge_class(string $role): string
{
    return match ($role) {
        'admin' => 'danger',
        'dean' => 'primary',
        'program_chair' => 'warning',
        'faculty' => 'success',
        'student' => 'secondary',
        'gened' => 'info',
        default => 'secondary',
    };
}

```

FILE: includes/profile_photo_helpers.php

```
-----
<?php
declare(strict_types=1);

require_once __DIR__ . '/functions.php';

function profile_photo_column_ready(): bool
{
    static $ready = null;
    if ($ready === null) {
        $ready = db_column_exists('users', 'profile_photo');
    }
    return $ready;
}

function profile_photo_dir(): string
{
    return BASE_PATH . DIRECTORY_SEPARATOR . 'uploads' . DIRECTORY_SEPARATOR . 'profile_photos';
}

function profile_photo_path(string $storedName): string
{
    return profile_photo_dir() . DIRECTORY_SEPARATOR . basename($storedName);
}

function profile_photo_stored_name(int $userId): string
{
    if ($userId < 1 || !profile_photo_column_ready()) {
        return '';
    }

    static $cache = [];
    if (array_key_exists($userId, $cache)) {
        return $cache[$userId];
    }

    $st = db()->prepare('SELECT profile_photo FROM users WHERE id = ? AND is_active = 1 LIMIT 1');
    $st->execute([$userId]);
    $stored = trim((string) ($st->fetchColumn() ?: ''));
    $cache[$userId] = $stored;

    return $stored;
}

function profile_photo_url(int $userId): ?string
{
    $stored = profile_photo_stored_name($userId);
    if ($stored === '') {
        return null;
    }
    $path = profile_photo_path($stored);
    if (!is_file($path)) {
        return null;
    }

    return 'profile_photo.php?u=' . $userId;
}

/**
 * @param array<string,mixed> $file
 */
function profile_photo_store(int $userId, array $file): string
{
    if ($userId < 1) {
        throw new RuntimeException('Invalid user.');
```

```

    }
    if ($size > 2 * 1024 * 1024) {
        throw new RuntimeException('Photo is too large (max 2 MB).');
    }

    $tmp = (string) ($file['tmp_name'] ?? '');
    if ($tmp === '' || !is_uploaded_file($tmp)) {
        throw new RuntimeException('Invalid upload.');
```

```

    }

    $imageInfo = @getimagesize($tmp);
    if ($imageInfo === false) {
        throw new RuntimeException('File must be a JPEG, PNG, or WebP image.');
```

```

    }

    $mime = (string) ($imageInfo['mime'] ?? '');
    $ext = match ($mime) {
        'image/jpeg' => 'jpg',
        'image/png' => 'png',
        'image/webp' => 'webp',
        default => '',
    };
    if ($ext === '') {
        throw new RuntimeException('Only JPEG, PNG, and WebP images are allowed.');
```

```

    }

    $dir = profile_photo_dir();
    if (!is_dir($dir) && !mkdir($dir, 0775, true) && !is_dir($dir)) {
        throw new RuntimeException('Unable to create photo storage directory.');
```

```

    }

    profile_photo_delete_file(profile_photo_stored_name($userId));
```

```

    $stored = 'user_' . $userId . '_' . bin2hex(random_bytes(12)) . '.' . $ext;
    $dest = profile_photo_path($stored);
    if (!move_uploaded_file($tmp, $dest)) {
        throw new RuntimeException('Failed to save photo.');
```

```

    }

    db()->prepare('UPDATE users SET profile_photo = ? WHERE id = ?')->execute([$stored, $userId]);
```

```

    return $stored;
}
```

```
function profile_photo_delete_file(string $storedName): void
```

```

{
    if ($storedName === '') {
        return;
    }
    $path = profile_photo_path($storedName);
    if (is_file($path)) {
        @unlink($path);
    }
}
```

```
function profile_photo_remove(int $userId): void
```

```

{
    if ($userId < 1) {
        throw new RuntimeException('Invalid user.');
```

```

    }
    if (!profile_photo_column_ready()) {
        throw new RuntimeException('Profile photos are not installed. Run upgrade_roles.php once.');
```

```

    }

    profile_photo_delete_file(profile_photo_stored_name($userId));
    db()->prepare('UPDATE users SET profile_photo = NULL WHERE id = ?')->execute([$userId]);
}
```

```
function profile_photo_mime_for_stored(string $storedName): string
```

```

{
    return match (strtolower(pathinfo($storedName, PATHINFO_EXTENSION))) {
        'jpg', 'jpeg' => 'image/jpeg',
        'png' => 'image/png',
        'webp' => 'image/webp',
        default => 'application/octet-stream',
    };
}
```

FILE: includes/schedule_helpers.php

```
-----
<?php
declare(strict_types=1);

require_once __DIR__ . '/functions.php';

function schedule_program_scope_key(int $collegeId, string $programName): string
{
    return $collegeId . '|' . trim($programName);
}

function schedule_program_option_value(int $collegeId, string $programName, bool $useCollegeScope): string
{
    $programName = trim($programName);
    if ($useCollegeScope && $collegeId > 0) {
        return $collegeId . '::' . $programName;
    }

    return $programName;
}

/** @return array{college_id:int,program_name:string} */
function schedule_program_option_parse(string $value): array
{
    $value = trim($value);
    if ($value !== '' && preg_match('/^(\\d+):(\\.+)$/s', $value, $m)) {
        return ['college_id' => (int) $m[1], 'program_name' => trim($m[2])];
    }

    return ['college_id' => 0, 'program_name' => $value];
}

/**
 * @param array<string,mixed> $defaults
 * @param array<string,mixed> $options Pass `edit_single_row` => true when editing one schedule row (no lab split block).
 */
function render_schedule_form(array $defaults = [], array $options = []): void
{
    $editSingleRow = !empty($options['edit_single_row']);
    $unlockProgram = !empty($options['unlock_program']);
    $allCollegePrograms = !empty($options['all_college_programs']);
    $role = (string) ($_SESSION['role'] ?? '');
    $collegeId = isset($_SESSION['college_id']) ? (int) $_SESSION['college_id'] : null;
    $programScope = $role === 'program_chair' ? current_program_scope() : null;
    $effectiveProgramScope = $unlockProgram ? null : $programScope;
    $hasLabFlag = db_column_exists('courses', 'is_laboratory');
    $hasProgramsTable = db_table_exists('programs');
    $hasYearLevel = db_column_exists('courses', 'year_level');
    $hasSection = db_column_exists('courses', 'section');
    $geProgramChair = is_ge_program_scope($programScope);
    $showScopedRole = in_array($role, ['dean', 'program_chair'], true) && $collegeId;
    $showGetTargetScope = $geProgramChair && $hasProgramsTable;
    $showBlockScope = $showScopedRole && $hasYearLevel && $hasSection && !$geProgramChair;
    $showProgramFilter = $hasProgramsTable && $collegeId && ($role === 'dean' || ($unlockProgram && $role === 'program_chair'));
    $showProgramLocked = !$unlockProgram && $hasProgramsTable && $role === 'program_chair' && $collegeId && $programScope !== null;
    $programOptionsList = [];
    if ($showProgramFilter || $showProgramLocked) {
        if ($allCollegePrograms) {
            $st = db()->query(
                "SELECT p.college_id, p.program_name, c.college_code
                FROM programs p
                INNER JOIN colleges c ON c.id = p.college_id
                WHERE p.status='active'
                ORDER BY c.college_code, p.program_name"
            );
            foreach ($st->fetchAll(PDO::FETCH_ASSOC) as $row) {
                $name = (string) ($row['program_name'] ?? '');
                $progCollegeId = (int) ($row['college_id'] ?? 0);
                if ($name === '' || $progCollegeId < 1) {
                    continue;
                }
                $programOptionsList[] = [
                    'college_id' => $progCollegeId,
                    'name' => $name,
                    'value' => schedule_program_option_value($progCollegeId, $name, true),
                    'label' => (string) ($row['college_code'] ?? '') . ' - ' . $name,
                ];
            }
        } else {

```



```

    $st = db()->prepare("SELECT program_name FROM programs WHERE college_id=? AND status='active' ORDER BY program_name");
    $st->execute([$collegeId]);
    foreach ($st->fetchAll(PDO::FETCH_COLUMN) as $pn) {
        $name = (string) $pn;
        if ($name === '') {
            continue;
        }
        $programOptionsList[] = [
            'college_id' => (int) $collegeId,
            'name' => $name,
            'value' => schedule_program_option_value((int) $collegeId, $name, false),
            'label' => $name,
        ];
    }
}

$selectedProgram = (!$unlockProgram && $programScope !== null)
    ? schedule_program_option_value((int) $collegeId, (string) $programScope, $allCollegePrograms)
    : (string) ($defaults['program'] ?? '');
$yearLevelOptions = ['1', '2', '3', '4', '5'];
$sectionOptions = [];
$programYearLevelsByName = [];
$courseYearLevelsByProgramName = [];
$sectionsByScope = [];
$mergedDeanYearLevelsCollege = [];

if ($showBlockScope) {
    $hasProgYlTable = db_table_exists('programs_year_levels');
    if ($hasProgYlTable && $collegeId) {
        $programYearLevelsByName = dean_program_year_levels_map($collegeId);
        foreach ($programYearLevelsByName as $arr) {
            foreach ($arr as $yv) {
                $yv = trim((string) $yv);
                if ($yv !== '') {
                    $mergedDeanYearLevelsCollege[] = $yv;
                }
            }
        }
        $mergedDeanYearLevelsCollege = sort_schedule_year_levels($mergedDeanYearLevelsCollege);
    }

    $scopeCourseParams = [];
    $scopeCourseSqlExtra = '';
    if ($effectiveProgramScope !== null) {
        $scopeCourseSqlExtra = ' AND department=? ';
        $scopeCourseParams[] = $effectiveProgramScope;
    }
    if ($allCollegePrograms) {
        $scopeCourseSql = 'SELECT DISTINCT college_id, TRIM(COALESCE(department,\'\'\'\'')) AS dep, TRIM(year_level) AS yl, TRIM(section) AS
sec
        FROM courses
        WHERE college_id IS NOT NULL
        AND TRIM(COALESCE(year_level,\'\'\'\''))<>\'\'\'\'
        AND TRIM(COALESCE(department,\'\'\'\''))<>\'\'\'\'
        ' . $scopeCourseSqlExtra . '
        ORDER BY college_id, dep, yl, sec';
    } else {
        $scopeCourseParams[] = $collegeId;
        $scopeCourseSql = 'SELECT DISTINCT college_id, TRIM(COALESCE(department,\'\'\'\'')) AS dep, TRIM(year_level) AS yl, TRIM(section) AS
sec
        FROM courses
        WHERE college_id=?
        AND TRIM(COALESCE(year_level,\'\'\'\''))<>\'\'\'\'
        AND TRIM(COALESCE(department,\'\'\'\''))<>\'\'\'\'
        ' . $scopeCourseSqlExtra . '
        ORDER BY dep, yl, sec';
    }

    $st = db()->prepare($scopeCourseSql);
    $st->execute($scopeCourseParams);
    while ($crow = $st->fetch(PDO::FETCH_ASSOC)) {
        $cid = (int) ($crow['college_id'] ?? 0);
        $dp = trim((string) ($crow['dep'] ?? ''));
        $yl = trim((string) ($crow['yl'] ?? ''));
        $sec = trim((string) ($crow['sec'] ?? ''));
        if ($cid < 1 || $dp === '' || $yl === '') {
            continue;
        }
        $scopeKey = $allCollegePrograms ? schedule_program_scope_key($cid, $dp) : $dp;
        if (!isset($courseYearLevelsByProgramName[$scopeKey])) {

```

```

        $courseYearLevelsByProgramName[$scopeKey] = [];
    }
    if (!in_array($yl, $courseYearLevelsByProgramName[$scopeKey], true)) {
        $courseYearLevelsByProgramName[$scopeKey][] = $yl;
    }
    if ($sec !== '') {
        $sectionKey = $scopeKey . '|' . $yl;
        if (!isset($sectionsByScope[$sectionKey])) {
            $sectionsByScope[$sectionKey] = [];
        }
        if (!in_array($sec, $sectionsByScope[$sectionKey], true)) {
            $sectionsByScope[$sectionKey][] = $sec;
        }
    }
}
foreach ($courseYearLevelsByProgramName as $k => $_) {
    $courseYearLevelsByProgramName[$k] = sort_schedule_year_levels($courseYearLevelsByProgramName[$k]);
}

$courseYearLevelsCollegeWide = [];
foreach ($courseYearLevelsByProgramName as $arrs) {
    foreach ($arrs as $yv) {
        $courseYearLevelsCollegeWide[] = $yv;
    }
}
$courseYearLevelsCollegeWide = sort_schedule_year_levels($courseYearLevelsCollegeWide);

$selectedProgramParsed = schedule_program_option_parse($selectedProgram);
$progFocus = $selectedProgramParsed['program_name'];
$progFocusCollegeId = $selectedProgramParsed['college_id'];
$progFocusKey = ($allCollegePrograms && $progFocusCollegeId > 0 && $progFocus !== '')
    ? schedule_program_scope_key($progFocusCollegeId, $progFocus)
    : $progFocus;

$pick = [];
if ($progFocus !== '' && !$allCollegePrograms && isset($programYearLevelsByName[$progFocus]) && $programYearLevelsByName[$progFocus]
    !== []) {
    $pick = $programYearLevelsByName[$progFocus];
} elseif ($progFocusKey !== '' && isset($courseYearLevelsByProgramName[$progFocusKey]) &&
    $courseYearLevelsByProgramName[$progFocusKey] !== []) {
    $pick = $courseYearLevelsByProgramName[$progFocusKey];
} elseif ($progFocus === '' && $mergedDeanYearLevelsCollege !== []) {
    $pick = $mergedDeanYearLevelsCollege;
} elseif ($courseYearLevelsCollegeWide !== []) {
    $pick = $courseYearLevelsCollegeWide;
} else {
    $pick = $yearLevelOptions;
}

$yearLevelOptions = $pick;

if ($progFocusKey !== '' && $sectionsByScope !== []) {
    $mergedSections = [];
    foreach ($sectionsByScope as $sectionKey => $secs) {
        if (str_starts_with($sectionKey, $progFocusKey . '|')) {
            foreach ($secs as $sec) {
                if (!in_array($sec, $mergedSections, true)) {
                    $mergedSections[] = $sec;
                }
            }
        }
    }
    $sectionOptions = $mergedSections;
} else {
    $secSql = "SELECT DISTINCT TRIM(section) AS sec
    FROM courses
    WHERE TRIM(COALESCE(section, ''))<>''";
    $secParams = [];
    if ($allCollegePrograms) {
        $secSql .= ' AND college_id IS NOT NULL';
    } else {
        $secSql .= ' AND college_id=?';
        $secParams[] = $collegeId;
    }
    if ($effectiveProgramScope !== null) {
        $secSql .= ' AND department=?';
        $secParams[] = $effectiveProgramScope;
    }
    $secSql .= ' ORDER BY sec';
    $st = db()->prepare($secSql);

```

```

        $st->execute($secParams);
        $sectionOptions = array_map('strval', $st->fetchAll(PDO::FETCH_COLUMN));
    }
    if ($sectionOptions === []) {
        $sectionOptions = ['A', 'B', 'C', 'D', 'E'];
    }
}
$collegeCodesById = [];
if ($allCollegePrograms) {
    foreach (db()->query("SELECT id, college_code FROM colleges WHERE status='active'")->fetchAll(PDO::FETCH_ASSOC) as $colRow) {
        $collegeCodesById[(int) ($colRow['id'] ?? 0)] = (string) ($colRow['college_code'] ?? '');
    }
}
$colPick = $showBlockScope
    ? 'col-12 col-sm-6 col-lg-2'
    : (($showProgramFilter || $showProgramLocked) && $programOptionsList !== [] ? 'col-md-3' : 'col-md-4');
$days = schedule_days_list();
$selDays = isset($defaults['day_array']) ? $defaults['day_array'] : [];
$selLabDays = isset($defaults['lab_day_array']) ? $defaults['lab_day_array'] : [];
if (!$selDays && !empty($defaults['day_of_week'])) {
    $selDays = parse_day_set((string) $defaults['day_of_week']);
}
if (!$selLabDays && !empty($defaults['lab_day_of_week'])) {
    $selLabDays = parse_day_set((string) $defaults['lab_day_of_week']);
}
if ($showScopedRole) {
    $facultySql = "SELECT id, faculty_id, full_name FROM faculty WHERE status='active' AND college_id=?";
    $facultyParams = [$collegeId];
    if ($effectiveProgramScope !== null) {
        $facultySql .= " AND (department=? OR department='')";
        $facultyParams[] = $effectiveProgramScope;
    }
    $facultySql .= " ORDER BY full_name";
    $st = db()->prepare($facultySql);
    $st->execute($facultyParams);
    $faculty = $st->fetchAll();
    $blockCols = $showBlockScope ? ', year_level, section' : '';
    $courses = [];
    if ($geProgramChair && $collegeId && db_column_exists('courses', 'is_gened')) {
        foreach (ge_courses_offered_to_college($collegeId) as $geCourse) {
            $gid = (int) ($geCourse['id'] ?? 0);
            if ($gid < 1) {
                continue;
            }
            $courses[] = [
                'id' => $gid,
                'course_code' => (string) ($geCourse['course_code'] ?? ''),
                'course_name' => (string) ($geCourse['course_name'] ?? ''),
                'is_laboratory' => !empty($geCourse['is_laboratory']) ? 1 : 0,
                'department' => (string) ($geCourse['department'] ?? ge_program_chair_label()),
                'year_level' => (string) ($geCourse['year_level'] ?? ''),
                'section' => (string) ($geCourse['section'] ?? ''),
            ];
        }
        usort($courses, static fn ($a, $b) => strcmp((string) $a['course_code'], (string) $b['course_code']));
    } else {
        if ($allCollegePrograms) {
            $courseWhere = ' WHERE college_id IS NOT NULL';
            $courseParams = [];
        } else {
            $courseWhere = ' WHERE college_id=?';
            $courseParams = [$collegeId];
        }
        if ($effectiveProgramScope !== null) {
            $courseWhere .= ' AND department=?';
            $courseParams[] = $effectiveProgramScope;
        }
        $collegeCol = $allCollegePrograms ? ', college_id' : '';
        if ($hasLabFlag) {
            $st = db()->prepare("SELECT id, course_code, course_name, is_laboratory, department{$blockCols}{$collegeCol} FROM
courses{$courseWhere} ORDER BY course_code");
        } else {
            $st = db()->prepare("SELECT id, course_code, course_name, 0 AS is_laboratory, department{$blockCols}{$collegeCol} FROM
courses{$courseWhere} ORDER BY course_code");
        }
        $st->execute($courseParams);
        $courses = $st->fetchAll();
    }
}
$roomSql = "SELECT id, room_code, room_name FROM rooms WHERE status IN ('available','tba')";
$roomParams = [];

```

```

        if ($collegeId) {
            $roomSql .= " AND college_id=?";
            $roomParams[] = $collegeId;
        }
        $roomSql .= " ORDER BY room_code";
        $st = db()->prepare($roomSql);
        $st->execute($roomParams);
        $rooms = $st->fetchAll();
    } else {
        $faculty = db()->query("SELECT id, faculty_id, full_name FROM faculty WHERE status='active' ORDER BY full_name")->fetchAll();
        if ($hasLabFlag) {
            $courses = db()->query('SELECT id, course_code, course_name, is_laboratory, department FROM courses ORDER BY course_code')->fetchAll();
        } else {
            $courses = db()->query('SELECT id, course_code, course_name, 0 AS is_laboratory, department FROM courses ORDER BY course_code')->fetchAll();
        }
        $rooms = db()->query("SELECT id, room_code, room_name FROM rooms WHERE status IN ('available','tba') ORDER BY room_code")->fetchAll();
    }
    $semesters = ['1st Semester', '2nd Semester', 'Summer'];
    $types = ['MW', 'TTH', 'MWF', 'TTHS', 'Saturday', 'Sunday', 'MW_TTH', 'Custom'];
    $scheduleYlDyn = null;
    $geTargetProgramsByCollege = [];
    $geTargetSectionsByScope = [];
    $geTargetCollegeList = [];
    if ($showGeTargetScope) {
        $geTargetCollegeList = db()->query("SELECT id, college_code, college_name FROM colleges WHERE status='active' ORDER BY college_code")->fetchAll();
        $programRows = db()->query(
            "SELECT p.college_id, p.program_name, c.college_code
            FROM programs p
            INNER JOIN colleges c ON c.id = p.college_id
            WHERE p.status='active'
            ORDER BY c.college_code, p.program_name"
        )->fetchAll();
        $hasProgYlTable = db_table_exists('programs_year_levels');
        $programYearLevelsByCollege = [];
        if ($hasProgYlTable) {
            $ylSt = db()->query(
                'SELECT p.college_id AS cid, p.program_name AS pname, pyl.year_level AS yl
                FROM programs_year_levels pyl
                INNER JOIN programs p ON p.id = pyl.program_id'
            );
            while ($rw = $ylSt->fetch(PDO::FETCH_ASSOC)) {
                $cid = (int) ($rw['cid'] ?? 0);
                $pname = trim((string) ($rw['pname'] ?? ''));
                $yl = trim((string) ($rw['yl'] ?? ''));
                if ($cid < 1 || $pname === '' || $yl === '') {
                    continue;
                }
                if (!isset($programYearLevelsByCollege[$cid])) {
                    $programYearLevelsByCollege[$cid] = [];
                }
                if (!isset($programYearLevelsByCollege[$cid][$pname])) {
                    $programYearLevelsByCollege[$cid][$pname] = [];
                }
                if (!in_array($yl, $programYearLevelsByCollege[$cid][$pname], true)) {
                    $programYearLevelsByCollege[$cid][$pname][] = $yl;
                }
            }
        }
        foreach ($programRows as $pr) {
            $cid = (int) $pr['college_id'];
            if (!isset($geTargetProgramsByCollege[$cid])) {
                $geTargetProgramsByCollege[$cid] = [];
            }
            $nm = (string) $pr['program_name'];
            $deanYl = isset($programYearLevelsByCollege[$cid][$nm])
                ? sort_schedule_year_levels($programYearLevelsByCollege[$cid][$nm])
                : [];
            $geTargetProgramsByCollege[$cid][] = [
                'name' => $nm,
                'college_code' => (string) $pr['college_code'],
                'year_levels' => ($deanYl !== []) ? $deanYl : ['1', '2', '3', '4', '5'],
            ];
        }
    }
    if ($hasYearLevel && $hasSection) {
        $deanSectionRows = db()->query(
            "SELECT DISTINCT college_id, department, year_level, section

```

```

FROM courses
WHERE COALESCE(is_gened, 0) = 0
      AND college_id IS NOT NULL
      AND TRIM(COALESCE(department, '')) <> ''
      AND TRIM(COALESCE(section, '')) <> ''
      AND TRIM(COALESCE(year_level, '')) <> ''
ORDER BY college_id, department, year_level, section"
)->fetchAll();
foreach ($deanSectionRows as $sr) {
    $cid = (int) ($sr['college_id'] ?? 0);
    $program = (string) ($sr['department'] ?? '');
    $yl = (string) ($sr['year_level'] ?? '');
    $sec = (string) ($sr['section'] ?? '');
    if ($cid < 1 || $program === '' || $yl === '' || $sec === '') {
        continue;
    }
    if (!isset($geTargetSectionsByScope[$cid])) {
        $geTargetSectionsByScope[$cid] = [];
    }
    if (!isset($geTargetSectionsByScope[$cid][$program])) {
        $geTargetSectionsByScope[$cid][$program] = [];
    }
    if (!isset($geTargetSectionsByScope[$cid][$program][$yl])) {
        $geTargetSectionsByScope[$cid][$program][$yl] = [];
    }
    if (!in_array($sec, $geTargetSectionsByScope[$cid][$program][$yl], true)) {
        $geTargetSectionsByScope[$cid][$program][$yl][] = $sec;
    }
}
}
}
if ($showBlockScope) {
    $scheduleYlDyn = [
        'byProgramDean' => $programYearLevelsByName,
        'courseByProgram' => $courseYearLevelsByProgramName,
        'sectionsByScope' => $sectionsByScope,
        'deanCollegeUnion' => $mergedDeanYearLevelsCollege,
        'courseCollegeWide' => $courseYearLevelsCollegeWide ?? [],
        'numericFallback' => ['1', '2', '3', '4', '5'],
        'lockedProgram' => $effectiveProgramScope !== null ? trim((string) $effectiveProgramScope) : '',
        'useCollegeScope' => $allCollegePrograms,
    ];
}
?>
<div class="row g-3">
    <?php if ($showGeTargetScope): ?>
        <div class="col-12">
            <div class="alert alert-light border small mb-0 py-2">
                Schedule a <strong>General Education</strong> class for students in any college program. Teaching is assigned to faculty
in your college (<? = htmlspecialchars(college_name_by_id($collegeId)) ?>).
            </div>
        </div>
        <div class="col-md-4">
            <label class="form-label">Target college</label>
            <select name="target_college_id" id="ge_target_college_id" class="form-select" required>
                <option value="">— Select —</option>
                <?php foreach ($geTargetCollegeList as $c): ?>
                    <option value="<? = (int) $c['id'] ?>" <? = (int) ($defaults['target_college_id'] ?? 0) === (int) $c['id'] ?
'selected' : '' ?>>
                        <? = htmlspecialchars($c['college_code'] . ' - ' . $c['college_name']) ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </div>
        <div class="col-md-4">
            <label class="form-label">Target program</label>
            <select name="target_program" id="ge_target_program" class="form-select" required>
                <option value="">— Select program —</option>
            </select>
        </div>
        <div class="col-md-2">
            <label class="form-label">Target year level</label>
            <select name="target_year_level" id="ge_target_year_level" class="form-select" required>
                <option value="">— Select —</option>
            </select>
        </div>
        <div class="col-md-2">
            <label class="form-label">Target section</label>
            <?php if ($hasYearLevel && $hasSection): ?>
                <select name="target_section" id="ge_target_section" class="form-select" required>

```

```

        <option value="">— Select —</option>
    </select>
    <?php else: ??>
        <input type="text" name="target_section" class="form-control" maxlength="20" required value="<?=
htmlspecialchars((string) ($defaults['target_section'] ?? '')) ?>"
    <?php endif; ??>
</div>
<?php endif; ??>
<?php if (($showProgramFilter || $showProgramLocked) && $programOptionsList === []): ??>
    <div class="col-12">
        <div class="alert alert-light border small mb-0 py-2">No active programs yet. Add them under <strong>Programs</strong> to
filter courses by program on this form.</div>
    </div>
<?php endif; ??>
<?php if ($showBlockScope): ??>
    <div class="col-12">
        <div class="alert alert-light border small mb-0 py-2">Select <strong>year level</strong> and <strong>section</strong> to
filter courses (must match the course record). Year levels listed here match what the dean configures under Programs for each program.</div>
    </div>
<?php endif; ??>
<div class="<?= $colPick ?>">
    <label class="form-label">Faculty</label>
    <select name="faculty_id" class="form-select" required>
        <option value="">— Select —</option>
        <?php foreach ($faculty as $f): ??>
            <option value="<?= (int) $f['id'] ?>" <?= (int) ($defaults['faculty_id'] ?? 0) === (int) $f['id'] ? 'selected' : '' ?>>
                <?= htmlspecialchars($f['full_name'] . ' ( ' . $f['faculty_id'] . ')') ?>
            </option>
        <?php endforeach; ??>
    </select>
</div>
<?php if ($showProgramFilter && $programOptionsList !== []): ??>
<div class="<?= $colPick ?>">
    <label class="form-label">Program</label>
    <select name="program" id="schedule_program_filter" class="form-select">
        <option value="">All programs</option>
        <?php foreach ($programOptionsList as $po): ??>
            <option value="<?= htmlspecialchars($po['value']) ?>" <?= $selectedProgram === $po['value'] ? 'selected' : '' ?>><?=
htmlspecialchars($po['label']) ?></option>
        <?php endforeach; ??>
    </select>
    <div class="form-text"><?= $allCollegePrograms ? 'Filter courses by program (all colleges).' : 'Filter courses by program (same
list as in Courses).' ?></div>
</div>
<?php endif; ??>
<?php if ($showProgramLocked): ??>
<div class="<?= $colPick ?>">
    <label class="form-label">Program</label>
    <input type="text" class="form-control" value="<?= htmlspecialchars((string) $programScope) ?>" readonly>
    <?php if (!$geProgramChair): ??>
        <input type="hidden" name="program" value="<?= htmlspecialchars((string) $programScope) ?>"
    <?php endif; ??>
    <div class="form-text"><?= $geProgramChair ? 'GE scheduling uses the target program above.' : 'Locked to your assigned program.'
?></div>
</div>
<?php endif; ??>
<?php if ($showBlockScope): ??>
<div class="<?= $colPick ?>">
    <label class="form-label">Year level</label>
    <select name="year_level" id="schedule_year_level" class="form-select" required>
        <option value="">— Select —</option>
        <?php foreach ($yearLevelOptions as $yl): ??>
            <option value="<?= htmlspecialchars((string) $yl) ?>" <?= ((string) ($defaults['year_level'] ?? '')) === (string) $yl ?
'selected' : '' ?>><?= htmlspecialchars((string) $yl) ?></option>
        <?php endforeach; ??>
    </select>
</div>
<div class="<?= $colPick ?>">
    <label class="form-label">Section</label>
    <select name="section" id="schedule_section" class="form-select" required>
        <option value="">— Select —</option>
        <?php foreach ($sectionOptions as $sec): ??>
            <option value="<?= htmlspecialchars((string) $sec) ?>" <?= ((string) ($defaults['section'] ?? '')) === (string) $sec ?
'selected' : '' ?>><?= htmlspecialchars((string) $sec) ?></option>
        <?php endforeach; ??>
    </select>
</div>
<?php endif; ??>
<div class="<?= $colPick ?>">
    <label class="form-label">Course</label>

```

```

<select name="course_id" id="course_id" class="form-select" required>
  <option value="">- Select -</option>
  <?php foreach ($courses as $c): ?>
    <?php
      $dep = trim((string) ($c['department'] ?? ''));
      $cy = trim((string) ($c['year_level'] ?? ''));
      $cs = trim((string) ($c['section'] ?? ''));
      $courseCollegeId = (int) ($c['college_id'] ?? $collegeId ?? 0);
      $courseLabel = $c['course_code'] . ' - ' . $c['course_name'];
      if ($allCollegePrograms && $courseCollegeId > 0) {
        $courseCodePrefix = $collegeCodesById[$courseCollegeId] ?? '';
        if ($courseCodePrefix !== '') {
          $courseLabel = $courseCodePrefix . ' - ' . $courseLabel;
        }
      }
    <?php
      <option value="<?=(int) $c['id'] ?>" data-is-lab="<?= !empty($c['is_laboratory']) ? '1' : '0' ?>" data-department="<?=
htmlspecialchars($dep) ?>" data-college-id="<?= $courseCollegeId ?>"<?php if ($showBlockScope): ?> data-year-level="<?=
htmlspecialchars($cy) ?>" data-section="<?= htmlspecialchars($cs) ?>"<?php endif; ?> <?=(int) ($defaults['course_id'] ?? 0) === (int)
$c['id'] ? 'selected' : '' ?>>
        <?=(htmlspecialchars($courseLabel) ?>
      </option>
    <?php endforeach; ?>
  </select>
</div>
<div class="<?= $colPick ?>">
  <label class="form-label" id="scheduleMainRoomLabel">Room</label>
  <select name="room_id" class="form-select">
    <option value="">- Select -</option>
    <?php foreach ($rooms as $r): ?>
      <option value="<?= (int) $r['id'] ?>" <?=(int) ($defaults['room_id'] ?? 0) === (int) $r['id'] ? 'selected' : '' ?>>
        <?=(htmlspecialchars($r['room_code'] . ($r['room_name'] ? ' - ' . $r['room_name'] : '')) ?>
      </option>
    <?php endforeach; ?>
  </select>
</div>
<div class="col-md-4">
  <label class="form-label">Schedule type</label>
  <select name="schedule_type" class="form-select" id="schedule_type">
    <?php foreach ($types as $t): ?>
      <option value="<?= $t ?>" <?=( $defaults['schedule_type'] ?? 'Custom') === $t ? 'selected' : '' ?>><?=(
htmlspecialchars($t) ?></option>
    <?php endforeach; ?>
  </select>
</div>
<div class="col-12">
  <label class="form-label">Days</label>
  <div class="d-flex flex-wrap gap-3">
    <?php foreach ($days as $d): ?>
      <div class="form-check">
        <input class="form-check-input" type="checkbox" name="days[]" value="<?= htmlspecialchars($d) ?>"
          id="day_<?= htmlspecialchars($d) ?>"
          <?=( in_array($d, $selDays, true) ? 'checked' : '' ?>>
        <label class="form-check-label" for="day_<?= htmlspecialchars($d) ?>"><?=(htmlspecialchars($d) ?></label>
      </div>
    <?php endforeach; ?>
  </div>
</div>
<div class="col-md-3">
  <label class="form-label">Start time</label>
  <input type="time" name="start_time" class="form-control" required
    value="<?= htmlspecialchars(substr((string) ($defaults['start_time'] ?? '08:00'), 0, 5)) ?>">
</div>
<div class="col-md-3">
  <label class="form-label">End time</label>
  <input type="time" name="end_time" class="form-control" required
    value="<?= htmlspecialchars(substr((string) ($defaults['end_time'] ?? '09:00'), 0, 5)) ?>">
</div>
<div class="col-md-3">
  <label class="form-label">Semester</label>
  <select name="semester" class="form-select" required>
    <?php foreach ($semesters as $s): ?>
      <option value="<?= htmlspecialchars($s) ?>" <?=( $defaults['semester'] ?? '') === $s ? 'selected' : '' ?>><?=(
htmlspecialchars($s) ?></option>
    <?php endforeach; ?>
  </select>
</div>
<div class="col-md-3">
  <label class="form-label">School year</label>
  <input type="text" name="school_year" class="form-control" required placeholder="e.g. 2025-2026"

```

```

        value="<?= htmlspecialchars((string) ($defaults['school_year'] ?? '')) ?>"
    </div>
    <div class="col-md-6">
        <label class="form-label">Academic year</label>
        <input type="text" name="academic_year" class="form-control" placeholder="Optional label"
            value="<?= htmlspecialchars((string) ($defaults['academic_year'] ?? '')) ?>"
        </div>
    </div>
    <?php if (!$editSingleRow): ?>
    <div class="mt-4 p-3 border rounded bg-light" id="labScheduleSection" style="display:none;">
        <h6 class="mb-3">Laboratory Schedule (separate days/time)</h6>
        <div class="row g-3">
            <div class="col-md-4">
                <label class="form-label">Laboratory room</label>
                <select name="lab_room_id" class="form-select">
                    <option value="">— Select lab room —</option>
                    <?php foreach ($rooms as $r): ?>
                        <option value="<?= (int) $r['id'] ?>" <?= (int) ($defaults['lab_room_id'] ?? 0) === (int) $r['id'] ? 'selected' : ''
                            <?= htmlspecialchars($r['room_code'] . ($r['room_name'] ? ' - ' . $r['room_name'] : '')) ?>
                        </option>
                    </select>
                </div>
                <div class="col-12">
                    <label class="form-label">Laboratory days</label>
                    <div class="d-flex flex-wrap gap-3">
                        <?php foreach ($days as $d): ?>
                            <div class="form-check">
                                <input class="form-check-input" type="checkbox" name="lab_days[]" value="<?= htmlspecialchars($d) ?>"
                                    id="lab_day_<?= htmlspecialchars($d) ?>" <?= in_array($d, $sellLabDays, true) ? 'checked' : '' ?>>
                                <label class="form-check-label" for="lab_day_<?= htmlspecialchars($d) ?>"><?= htmlspecialchars($d) ?></label>
                            </div>
                        </div>
                    </div>
                </div>
                <div class="col-md-3">
                    <label class="form-label">Laboratory start time</label>
                    <input type="time" name="lab_start_time" class="form-control" value="<?= htmlspecialchars(substr((string)
($defaults['lab_start_time'] ?? '13:00'), 0, 5)) ?>"
                </div>
                <div class="col-md-3">
                    <label class="form-label">Laboratory end time</label>
                    <input type="time" name="lab_end_time" class="form-control" value="<?= htmlspecialchars(substr((string)
($defaults['lab_end_time'] ?? '16:00'), 0, 5)) ?>"
                </div>
            </div>
        </div>
        <?php endif; ?>
        <?php if (($role ?? '') === 'admin'): ?>
        <div class="mt-4 p-3 border rounded bg-light">
            <div class="form-check mb-2">
                <input class="form-check-input" type="checkbox" name="override_conflicts" value="1" id="override_conflicts">
                <label class="form-check-label" for="override_conflicts">Override conflicts (admin password required)</label>
            </div>
            <label class="form-label small text-muted">Admin password</label>
            <input type="password" name="admin_password" class="form-control form-control-sm" style="max-width:320px" autocomplete="new-
password"
                placeholder="Only if overriding">
            </div>
        </div>
        <?php endif; ?>
    <script>
    (function () {
        const courseSel = document.getElementById('course_id');
        if (!courseSel) return;
        const labSection = document.getElementById('labScheduleSection');
        const programSel = document.getElementById('schedule_program_filter');
        const yearSel = document.getElementById('schedule_year_level');
        const sectionSel = document.getElementById('schedule_section');
        const yIDyn = <?= ($scheduleYIDyn !== null ? json_encode($scheduleYIDyn, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES) :
'null') ?>;

        function syncLabSection() {
            if (!labSection) return;
            const opt = courseSel.options[courseSel.selectedIndex];
            const isLab = opt && opt.getAttribute('data-is-lab') === '1';
            labSection.style.display = isLab ? 'block' : 'none';
            const roomLabel = document.getElementById('scheduleMainRoomLabel');
            if (roomLabel) {
                roomLabel.textContent = isLab ? 'Lecture room' : 'Room';
            }
        }
    })();
    </script>

```



```

    }
}

function currentProgramSelection() {
    if (ylDyn && ylDyn.lockedProgram && String(ylDyn.lockedProgram).trim()) {
        return { collegeId: '', program: String(ylDyn.lockedProgram).trim() };
    }
    if (!programSel) {
        return { collegeId: '', program: '' };
    }
    const val = (programSel.value || '').trim();
    if (!val) {
        return { collegeId: '', program: '' };
    }
    if (ylDyn && ylDyn.useCollegeScope && val.includes(':')) {
        const splitAt = val.indexOf(':');
        return {
            collegeId: val.slice(0, splitAt),
            program: val.slice(splitAt + 2),
        };
    }
    return { collegeId: '', program: val };
}

function programScopeKey(collegeId, program) {
    if (!program) {
        return '';
    }
    if (ylDyn && ylDyn.useCollegeScope && collegeId) {
        return String(collegeId) + '|' + program;
    }
    return program;
}

function resolveLevelsList(collegeId, progRaw) {
    if (!ylDyn) {
        return null;
    }
    const scopeKey = programScopeKey(collegeId, progRaw);
    const deanMap = ylDyn.byProgramDean || {};
    const courseMap = ylDyn.courseByProgram || {};
    if (scopeKey && Array.isArray(courseMap[scopeKey]) && courseMap[scopeKey].length) {
        return courseMap[scopeKey];
    }
    if (progRaw && Array.isArray(deanMap[progRaw]) && deanMap[progRaw].length) {
        return deanMap[progRaw];
    }
    if (progRaw && Array.isArray(courseMap[progRaw]) && courseMap[progRaw].length) {
        return courseMap[progRaw];
    }
    if (!progRaw && Array.isArray(ylDyn.deanCollegeUnion) && ylDyn.deanCollegeUnion.length) {
        return ylDyn.deanCollegeUnion;
    }
    if (Array.isArray(ylDyn.courseCollegeWide) && ylDyn.courseCollegeWide.length) {
        return ylDyn.courseCollegeWide;
    }
    return ylDyn.numericFallback || [];
}

function refreshSectionOptions() {
    if (!sectionSel || !ylDyn || !ylDyn.sectionsByScope) {
        filterCourses();
        return;
    }
    const prev = String(sectionSel.value || '');
    const selection = currentProgramSelection();
    const year = yearSel ? (yearSel.value || '').trim() : '';
    const scopeKey = programScopeKey(selection.collegeId, selection.program);
    const sectionKey = scopeKey && year ? (scopeKey + '|' + year) : '';
    const list = sectionKey ? (ylDyn.sectionsByScope[sectionKey] || []) : [];
    sectionSel.innerHTML = '';
    const z = document.createElement('option');
    z.value = '';
    z.textContent = '\u2014 Select \u2014';
    sectionSel.appendChild(z);
    for (let i = 0; i < list.length; i++) {
        const sec = String(list[i]);
        const opt = document.createElement('option');
        opt.value = sec;
        opt.textContent = sec;
    }
}

```

```

        sectionSel.appendChild(opt);
    }
    if (prev && list.indexOf(prev) !== -1) {
        sectionSel.value = prev;
    } else {
        sectionSel.value = '';
    }
    filterCourses();
}

function refreshYearLevelOptions() {
    if (!yearSel || !ylDyn) {
        filterCourses();
        return;
    }
    const prev = String(yearSel.value || '');
    const selection = currentProgramSelection();
    const list = resolveLevelsList(selection.collegeId, selection.program) || [];
    yearSel.innerHTML = '';
    const z = document.createElement('option');
    z.value = '';
    z.textContent = '\u2014 Select \u2014';
    yearSel.appendChild(z);
    for (let i = 0; i < list.length; i++) {
        const lvl = String(list[i]);
        const opt = document.createElement('option');
        opt.value = lvl;
        opt.textContent = lvl;
        yearSel.appendChild(opt);
    }
    if (prev && list.indexOf(prev) !== -1) {
        yearSel.value = prev;
    } else {
        yearSel.value = '';
    }
    refreshSectionOptions();
}

function filterCourses() {
    const selection = currentProgramSelection();
    const program = selection.program;
    const programCollegeId = selection.collegeId;
    let programEffective = program;
    if (!programSel || program === '' && ylDyn && ylDyn.lockedProgram) {
        programEffective = String(ylDyn.lockedProgram).trim();
    }
    const year = yearSel ? (yearSel.value || '').trim() : '';
    const section = sectionSel ? (sectionSel.value || '').trim() : '';
    for (let i = 0; i < courseSel.options.length; i++) {
        const opt = courseSel.options[i];
        if (!opt.value) {
            opt.hidden = false;
            continue;
        }
        const dep = (opt.getAttribute('data-department') || '').trim();
        const yl = (opt.getAttribute('data-year-level') || '').trim();
        const sec = (opt.getAttribute('data-section') || '').trim();
        const courseCollegeId = (opt.getAttribute('data-college-id') || '').trim();
        let hide = false;
        if (programSel && program) {
            if (dep !== program) {
                hide = true;
            }
            if (!hide && programCollegeId && courseCollegeId && courseCollegeId !== programCollegeId) {
                hide = true;
            }
        }
        if (!programSel && ylDyn && ylDyn.lockedProgram && dep !== programEffective) {
            hide = true;
        }
        if (yearSel && year && yl !== year) {
            hide = true;
        }
        if (sectionSel && section && sec !== section) {
            hide = true;
        }
        opt.hidden = hide;
    }
    const sel = courseSel.options[courseSel.selectedIndex];
    if (sel && sel.hidden) {

```

```

        courseSel.value = '';
    }
    syncLabSection();
}
if (labSection) {
    courseSel.addEventListener('change', syncLabSection);
}
if (programSel) {
    programSel.addEventListener('change', function () {
        if (ylDyn) {
            refreshYearLevelOptions();
        } else {
            filterCourses();
        }
    });
}
if (yearSel) {
    yearSel.addEventListener('change', function () {
        if (ylDyn && ylDyn.sectionsByScope) {
            refreshSectionOptions();
        } else {
            filterCourses();
        }
    });
}
if (sectionSel) {
    sectionSel.addEventListener('change', filterCourses);
}
if (yearSel && ylDyn) {
    refreshYearLevelOptions();
} else {
    filterCourses();
}
})();
</script>
<?php if ($showGeTargetScope): ?>
<script>
((() => {
    const data = <?= json_encode($geTargetProgramsByCollege, JSON_UNESCAPED_SLASHES) ?>;
    const sectionData = <?= json_encode($geTargetSectionsByScope, JSON_UNESCAPED_SLASHES) ?>;
    const collegeSel = document.getElementById('ge_target_college_id');
    const programSel = document.getElementById('ge_target_program');
    const yearSel = document.getElementById('ge_target_year_level');
    const sectionSel = document.getElementById('ge_target_section');
    if (!collegeSel || !programSel || !yearSel) return;
    const selectedProgram = <?= json_encode((string) ($defaults['target_program'] ?? ''), JSON_UNESCAPED_SLASHES) ?>;
    const selectedSection = <?= json_encode((string) ($defaults['target_section'] ?? ''), JSON_UNESCAPED_SLASHES) ?>;
    const selectedYearInitial = <?= json_encode((string) ($defaults['target_year_level'] ?? ''), JSON_UNESCAPED_SLASHES) ?>;
    let yearInitialApplied = false;

    function yearLevelsForCurrentTarget() {
        const cid = collegeSel.value;
        const pname = programSel.value || '';
        const rows = data[cid] || [];
        for (let i = 0; i < rows.length; i++) {
            if (rows[i].name === pname) {
                return Array.isArray(rows[i].year_levels) && rows[i].year_levels.length ? rows[i].year_levels : ['1','2','3','4','5'];
            }
        }
        return ['1','2','3','4','5'];
    }

    function renderYearLevels() {
        const prev = yearSel.value || '';
        const list = yearLevelsForCurrentTarget();
        yearSel.innerHTML = '';
        const z = document.createElement('option');
        z.value = '';
        z.textContent = '\u2014 Select \u2014';
        yearSel.appendChild(z);
        for (let i = 0; i < list.length; i++) {
            const lvl = String(list[i]);
            const opt = document.createElement('option');
            opt.value = lvl;
            opt.textContent = lvl;
            yearSel.appendChild(opt);
        }
        let pick = '';
        if (!yearInitialApplied && selectedYearInitial && list.indexOf(selectedYearInitial) !== -1) {
            pick = selectedYearInitial;

```

```

        yearInitialApplied = true;
    } else if (prev && list.indexOf(prev) !== -1) {
        pick = prev;
    }
    yearSel.value = pick || '';
}

function renderPrograms() {
    const cid = collegeSel.value;
    const rows = data[cid] || [];
    programSel.innerHTML = '';
    const first = document.createElement('option');
    first.value = '';
    first.textContent = '- Select program -';
    programSel.appendChild(first);
    rows.forEach((r) => {
        const opt = document.createElement('option');
        opt.value = r.name;
        opt.textContent = r.name;
        if (selectedProgram && selectedProgram === r.name) {
            opt.selected = true;
        }
        programSel.appendChild(opt);
    });
}

function renderSections() {
    if (!sectionSel) return;
    const cid = collegeSel.value;
    const program = programSel.value;
    const yl = yearSel.value;
    const rows = (((sectionData[cid] || {})[program] || {})[yl] || []);
    sectionSel.innerHTML = '';
    const first = document.createElement('option');
    first.value = '';
    first.textContent = '- Select section -';
    sectionSel.appendChild(first);
    rows.forEach((s) => {
        const opt = document.createElement('option');
        opt.value = s;
        opt.textContent = s;
        if (selectedSection && selectedSection === s) {
            opt.selected = true;
        }
        sectionSel.appendChild(opt);
    });
}

collegeSel.addEventListener('change', () => {
    renderPrograms();
    renderYearLevels();
    renderSections();
});
programSel.addEventListener('change', () => {
    renderYearLevels();
    renderSections();
});
yearSel.addEventListener('change', renderSections);

renderPrograms();
renderYearLevels();
renderSections();
})();
</script>
<?php endif; ?>
<?php if (in_array($role, ['dean', 'program_chair'], true)): ?>
<script>
(() => {
    const facultySel = document.querySelector('select[name="faculty_id"]');
    const courseSel = document.getElementById('course_id');
    const semesterSel = document.querySelector('select[name="semester"]');
    const schoolYearInput = document.querySelector('input[name="school_year"]');
    if (!facultySel || !courseSel || !semesterSel || !schoolYearInput) {
        return;
    }

    const geCollegeSel = document.getElementById('ge_target_college_id');
    const geProgramSel = document.getElementById('ge_target_program');
    const geYearSel = document.getElementById('ge_target_year_level');
    const geSectionSel = document.getElementById('ge_target_section');

```

```

let lastAlertKey = '';

function currentLoadCheckKey() {
    const parts = [
        facultySel.value || '',
        courseSel.value || '',
        semesterSel.value || '',
        schoolYearInput.value || ''
    ];
    if (geCollegeSel) parts.push(geCollegeSel.value || '');
    if (geProgramSel) parts.push(geProgramSel.value || '');
    if (geYearSel) parts.push(geYearSel.value || '');
    if (geSectionSel) parts.push(geSectionSel.value || '');
    return parts.join('|');
}

function checkCourseLoadAssignment() {
    const courseId = parseInt(courseSel.value || '0', 10);
    const facultyId = parseInt(facultySel.value || '0', 10);
    const semester = (semesterSel.value || '').trim();
    const schoolYear = (schoolYearInput.value || '').trim();
    if (courseId < 1 || facultyId < 1 || semester === '' || schoolYear === '') {
        return;
    }

    const params = new URLSearchParams({
        course_id: String(courseId),
        faculty_id: String(facultyId),
        semester,
        school_year: schoolYear,
    });

    if (geCollegeSel && geCollegeSel.value) {
        params.set('target_college_id', geCollegeSel.value);
    }
    if (geProgramSel && geProgramSel.value) {
        params.set('target_program', geProgramSel.value);
    }
    if (geYearSel && geYearSel.value) {
        params.set('target_year_level', geYearSel.value);
    }
    if (geSectionSel && geSectionSel.value) {
        params.set('target_section', geSectionSel.value);
    }

    fetch('api/check_course_load_assignment.php?' + params.toString(), {
        headers: { 'Accept': 'application/json' },
        credentials: 'same-origin',
    })
        .then((res) => res.json())
        .then((data) => {
            if (!data || !data.assigned || !data.message) {
                lastAlertKey = '';
                return;
            }
            const key = currentLoadCheckKey() + '|' + (data.faculty_code || data.faculty_name || '');
            if (key === lastAlertKey) {
                return;
            }
            lastAlertKey = key;
            window.alert(data.message);
        })
        .catch(() => {});
}

const triggerFields = [facultySel, courseSel, semesterSel, schoolYearInput];
if (geCollegeSel) triggerFields.push(geCollegeSel);
if (geProgramSel) triggerFields.push(geProgramSel);
if (geYearSel) triggerFields.push(geYearSel);
if (geSectionSel) triggerFields.push(geSectionSel);

triggerFields.forEach((el) => {
    el.addEventListener('change', checkCourseLoadAssignment);
});
schoolYearInput.addEventListener('blur', checkCourseLoadAssignment);
})();
</script>
<?php endif; ?>
<?php
}

```

FILE: includes/student_materials_reviewer.php

```
<?php
declare(strict_types=1);

/**
 * Weekly course-materials reviewer: build context and generate practice questions.
 */

function student_materials_reviewer_plain_text(string $html): string
{
    $text = strip_tags($html);
    $text = html_entity_decode($text, ENT_QUOTES | ENT_HTML5, 'UTF-8');
    $text = preg_replace('/\s+/u', ' ', $text) ?? $text;

    return trim($text);
}

/**
 * @return array<string, mixed>|null
 */
function student_materials_reviewer_fetch_classroom(int $studentId, int $classroomId): ?array
{
    if ($studentId < 1 || $classroomId < 1 || !db_table_exists('classroom_enrollments')) {
        return null;
    }

    $st = db()->prepare(
        'SELECT oc.id, c.course_code, c.course_name, f.full_name AS faculty_name
        FROM classroom_enrollments ce
        INNER JOIN online_classrooms oc ON oc.id = ce.classroom_id
        INNER JOIN courses c ON c.id = oc.course_id
        INNER JOIN faculty f ON f.id = oc.faculty_id
        WHERE oc.id = ? AND ce.student_id = ?
        LIMIT 1'
    );
    $st->execute([$classroomId, $studentId]);
    $row = $st->fetch();

    return is_array($row) ? $row : null;
}

/**
 * @return list<array{label:string,count:int,items:array<int,array<string,mixed>>>>
 */
function student_materials_reviewer_week_groups(int $classroomId): array
{
    if ($classroomId < 1 || !db_table_exists('classroom_content')) {
        return [];
    }

    $st = db()->prepare(
        'SELECT *
        FROM classroom_content
        WHERE classroom_id = ? AND content_type <> "announcement"
        ORDER BY created_at DESC'
    );
    $st->execute([$classroomId]);
    $materials = $st->fetchAll();

    return classroom_content_group_by_week($materials);
}

/**
 * @return list<array<string, mixed>>
 */
function student_materials_reviewer_week_items(int $classroomId, string $weekLabel): array
{
    foreach (student_materials_reviewer_week_groups($classroomId) as $group) {
        if ((string) $group['label'] === $weekLabel) {
            return classroom_content_enrich_week_items($group['items']);
        }
    }

    return [];
}
```

```

/**
 * @param list<array<string, mixed>> $items
 */
function student_materials_reviewer_build_source_text(array $items, array $classroom, string $weekLabel): string
{
    $lines = [
        'Course: ' . trim((string) ($classroom['course_code'] ?? '')) . ' - ' . trim((string) ($classroom['course_name'] ?? '')),
        'Week: ' . $weekLabel,
        'Instructor: ' . trim((string) ($classroom['faculty_name'] ?? '')),
        '',
    ];

    foreach ($items as $idx => $item) {
        $n = (int) $idx + 1;
        $title = trim((string) ($item['title'] ?? 'Untitled'));
        $type = trim((string) ($item['content_type'] ?? 'material'));
        $body = student_materials_reviewer_plain_text((string) ($item['body'] ?? ''));
        if (mb_strlen($body, 'UTF-8') > 1200) {
            $body = mb_substr($body, 0, 1197, 'UTF-8') . '...';
        }

        $lines[] = "Material {$n}: {$title} ({$type})";
        if ($body !== '') {
            $lines[] = $body;
        }

        $attachments = is_array($item['attachments'] ?? null) ? $item['attachments'] : [];
        foreach ($attachments as $attachment) {
            if (!is_array($attachment)) {
                continue;
            }
            $attachName = trim((string) ($attachment['original_name'] ?? 'Attachment'));
            $lines[] = 'Attachment: ' . $attachName;
        }

        $attachmentText = trim((string) ($item['attachment_text'] ?? ''));
        if ($attachmentText !== '') {
            if (mb_strlen($attachmentText, 'UTF-8') > 2000) {
                $attachmentText = mb_substr($attachmentText, 0, 1997, 'UTF-8') . '...';
            }
            $lines[] = 'Extract from attachments: ' . $attachmentText;
        }

        $url = trim((string) ($item['resource_url'] ?? ''));
        if ($url !== '' && !classroom_content_is_attachment($url)) {
            $lines[] = 'Resource link: ' . $url;
        }
        $lines[] = '';
    }

    $text = trim(implode("\n", $lines));
    if (mb_strlen($text, 'UTF-8') > 8000) {
        $text = mb_substr($text, 0, 7997, 'UTF-8') . '...';
    }

    return $text;
}

function student_materials_reviewer_unlimited_count(int $count): bool
{
    return $count <= 0;
}

/**
 * @param list<string> $excludeNormalized
 */
function student_materials_reviewer_question_excluded(string $question, array $excludeNormalized): bool
{
    if ($excludeNormalized === []) {
        return false;
    }

    $norm = mb_strtolower(trim($question), 'UTF-8');

    return in_array($norm, $excludeNormalized, true);
}

/**
 * @param list<string> $exclude
 * @return list<string>

```

```

*/
function student_materials_reviewer_normalize_exclude_list(array $exclude): array
{
    $out = [];
    foreach ($exclude as $text) {
        $norm = mb_strtolower(trim((string) $text), 'UTF-8');
        if ($norm !== '') {
            $out[] = $norm;
        }
    }

    return array_values(array_unique($out));
}

/**
 * @param list<array{id:int,question:string,type:string,choices:list<string>,answer:string,source:string}> $questions
 * @return list<array{id:int,question:string,type:string,choices:list<string>,answer:string,source:string}>
 */
function student_materials_reviewer_renumber_questions(array $questions, int $startId = 1): array
{
    $out = [];
    $id = $startId;
    foreach ($questions as $row) {
        $row['id'] = $id++;
        $out[] = $row;
    }

    return $out;
}

/**
 * @param list<array<string, mixed>> $items
 * @param list<string> $excludeNormalized
 * @return list<array{id:int,question:string,type:string,choices:list<string>,answer:string,source:string}>
 */
function student_materials_reviewer_builtin_questions(
    string $weekLabel,
    array $items,
    int $count = 0,
    array $excludeNormalized = []
): array {
    $unlimited = student_materials_reviewer_unlimited_count($count);
    $questions = [];
    $id = 1;

    $add = static function (array $row) use (
        &$questions,
        &$id,
        $unlimited,
        $count,
        $excludeNormalized
    ): bool {
        if (student_materials_reviewer_question_excluded((string) $row['question'], $excludeNormalized)) {
            return false;
        }
        if (!$unlimited && count($questions) >= $count) {
            return false;
        }

        $row['id'] = $id++;
        $row['source'] = 'builtin';
        $questions[] = $row;

        return true;
    };

    if ($items === []) {
        return [
            'id' => 1,
            'question' => 'No materials were posted for ' . $weekLabel . ' yet. Check back after your instructor uploads topics for this
week.',
            'type' => 'short_answer',
            'choices' => [],
            'answer' => 'Review will be available once course materials are posted.',
            'source' => 'builtin',
        ];
    }

    foreach ($items as $item) {
        if (!$unlimited && count($questions) >= $count) {

```



```

        break;
    }

    $title = trim((string) ($item['title'] ?? 'this topic'));
    if ($title === '') {
        $title = 'this topic';
    }
    $type = trim((string) ($item['content_type'] ?? 'material'));
    $body = student_materials_reviewer_plain_text((string) ($item['body'] ?? ''));

    $add([
        'question' => 'For ' . $weekLabel . ', what are the main ideas you should remember from "' . $title . '"?',
        'type' => 'short_answer',
        'choices' => [],
        'answer' => $body !== ''
            ? 'Focus on the posted notes for "' . $title . '". Key excerpt: ' . mb_substr($body, 0, 280, 'UTF-8')
            : 'Review the posted "' . $title . '" material and list the concepts your instructor highlighted.',
    ]);

    $typeChoices = array_values(array_unique(array_filter([
        $type,
        'reading',
        'video',
        'link',
        'worksheet',
        'slides',
    ])));
    if (count($typeChoices) < 4) {
        $typeChoices = array_values(array_unique(array_merge($typeChoices, ['reading', 'video', 'link', 'worksheet', 'slides'])));
    }
    $typeChoices = array_slice($typeChoices, 0, 4);
    if (!in_array($type, $typeChoices, true)) {
        $typeChoices[0] = $type;
    }

    $add([
        'question' => 'What kind of resource did your instructor post for "' . $title . '" in ' . $weekLabel . '?',
        'type' => 'multiple_choice',
        'choices' => $typeChoices,
        'answer' => $type,
    ]);

    $add([
        'question' => 'Explain "' . $title . '" from ' . $weekLabel . ' in your own words, as if teaching a classmate.',
        'type' => 'short_answer',
        'choices' => [],
        'answer' => $body !== ''
            ? 'Your explanation should cover the core ideas from the posted material on "' . $title . '".'
            : 'Summarize the main lesson your instructor intended for "' . $title . '"',
    ]);

    $add([
        'question' => 'Why is "' . $title . '" important for the lessons in ' . $weekLabel . '?',
        'type' => 'short_answer',
        'choices' => [],
        'answer' => 'Connect this topic to the weekly learning goals and how it supports later lessons.',
    ]);

    if ($body !== '') {
        $add([
            'question' => 'From the notes on "' . $title . '", what is one detail you should not forget for the exam or recitation?',
            'type' => 'short_answer',
            'choices' => [],
            'answer' => 'Pick a specific fact or concept from the material: ' . mb_substr($body, 0, 220, 'UTF-8'),
        ]);
    }
}

$itemCount = count($items);
for ($i = 0; $i < $itemCount; ++$i) {
    if (!$unlimited && count($questions) >= $count) {
        break;
    }
    for ($j = $i + 1; $j < $itemCount; ++$j) {
        if (!$unlimited && count($questions) >= $count) {
            break 2;
        }
        $a = $items[$i];
        $b = $items[$j];
        $add([

```

```

        'question' => 'How do "' . trim((string) ($a['title'] ?? 'Topic A')) . '" and "' . trim((string) ($b['title'] ?? 'Topic B'))
        . '" connect within ' . $weekLabel . '?',
        'type' => 'short_answer',
        'choices' => [],
        'answer' => 'Explain how both topics support the learning goals for this week.',
    ]]);
}
}

$titles = array_values(array_filter(array_map(
    static fn (array $item): string => trim((string) ($item['title'] ?? '')),
    $items
)));
$titleList = $titles !== [] ? implode(' ', array_slice($titles, 0, 8)) : 'each posted material';

$weekQuestions = [
    [
        'question' => 'Summarize everything you studied in ' . $weekLabel . ' in five sentences or fewer.',
        'type' => 'short_answer',
        'choices' => [],
        'answer' => 'Your summary should mention each posted material and how it fits the week\'s lessons.',
    ],
    [
        'question' => 'What is the overall learning goal for ' . $weekLabel . ' in this subject?',
        'type' => 'short_answer',
        'choices' => [],
        'answer' => 'Combine the themes from: ' . $titleList . '.',
    ],
    [
        'question' => 'Which material from ' . $weekLabel . ' should you review first before the others, and why?',
        'type' => 'short_answer',
        'choices' => [],
        'answer' => 'Choose the foundational topic first, then build toward more advanced materials in the week.',
    ],
];

foreach ($weekQuestions as $row) {
    if (!$unlimited && count($questions) >= $count) {
        break;
    }
    $add($row);
}

if ($excludeNormalized !== []) {
    $extraTemplates = [
        static fn (string $title): array => [
            'question' => 'What exam-style question might your instructor ask about "' . $title . '" from ' . $weekLabel . '?',
            'type' => 'short_answer',
            'choices' => [],
            'answer' => 'Write a question that tests understanding of the core ideas in "' . $title . '", then answer it.',
        ],
        static fn (string $title): array => [
            'question' => 'List three important terms from "' . $title . '" and define each in one sentence.',
            'type' => 'short_answer',
            'choices' => [],
            'answer' => 'Use vocabulary and concepts directly from the posted material on "' . $title . '".',
        ],
        static fn (string $title): array => [
            'question' => 'How would you apply what you learned in "' . $title . '" to a real-world example?',
            'type' => 'short_answer',
            'choices' => [],
            'answer' => 'Give a concrete example that shows you understood the lesson in "' . $title . '".',
        ],
        static fn (string $title): array => [
            'question' => 'What is still unclear to you about "' . $title . '" , and what would you ask your instructor?',
            'type' => 'short_answer',
            'choices' => [],
            'answer' => 'Name a specific concept from "' . $title . '" you would clarify in class or office hours.',
        ],
    ],
];

foreach ($items as $item) {
    if (!$unlimited && count($questions) >= $count) {
        break;
    }
    $title = trim((string) ($item['title'] ?? 'this topic'));
    if ($title === '') {
        $title = 'this topic';
    }
    foreach ($extraTemplates as $template) {

```

```

        if (!$unlimited && count($questions) >= $count) {
            break 2;
        }
        $add($template($title));
    }
}

if (!$unlimited) {
    return array_slice($questions, 0, max(1, $count));
}

return $questions;
}

/**
 * @return list<array{id:int,question:string,type:string,choices:list<string>,answer:string,source:string}>
 */
function student_materials_reviewer_parse_ai_questions(string $raw, int $maxCount = 0, array $excludeNormalized = []): array
{
    if (preg_match('/\{[\s\S]*\}/', $raw, $m) === 1) {
        $raw = $m[0];
    }

    try {
        $decoded = json_decode($raw, true, 512, JSON_THROW_ON_ERROR);
    } catch (JsonException $e) {
        return [];
    }

    if (!is_array($decoded) || !isset($decoded['questions']) || !is_array($decoded['questions'])) {
        return [];
    }

    $unlimited = student_materials_reviewer_unlimited_count($maxCount);
    $out = [];
    foreach ($decoded['questions'] as $row) {
        if (!$unlimited && count($out) >= $maxCount) {
            break;
        }
        if (!is_array($row)) {
            continue;
        }
        $question = trim((string) ($row['question'] ?? ''));
        if ($question === '' || student_materials_reviewer_question_excluded($question, $excludeNormalized)) {
            continue;
        }
        $type = strtolower(trim((string) ($row['type'] ?? 'short_answer')));
        if ($type !== 'multiple_choice') {
            $type = 'short_answer';
        }
        $choices = [];
        if (isset($row['choices']) && is_array($row['choices'])) {
            foreach ($row['choices'] as $choice) {
                $c = trim((string) $choice);
                if ($c !== '') {
                    $choices[] = $c;
                }
            }
        }
        $answer = trim((string) ($row['answer'] ?? ''));
        if ($answer === '') {
            continue;
        }

        $out[] = [
            'id' => 0,
            'question' => $question,
            'type' => $type,
            'choices' => array_slice($choices, 0, 6),
            'answer' => $answer,
            'source' => 'ai',
        ];
        $excludeNormalized[] = mb_strtolower($question, 'UTF-8');
    }

    return $out;
}

/**

```

```

* @param list<string> $exclude
* @return list<array{id:int,question:string,type:string,choices:list<string>,answer:string,source:string}>
*/
function student_materials_reviewer_ai_questions_batch(
    string $sourceText,
    string $weekLabel,
    string $courseName,
    int $batchSize,
    array $exclude = []
): array {
    $excludeNormalized = student_materials_reviewer_normalize_exclude_list($exclude);
    $batchSize = max(1, $batchSize);

    $excludeBlock = '';
    if ($exclude != []) {
        $sample = array_slice($exclude, -40);
        $excludeBlock = "\nDo NOT repeat or closely rephrase these existing questions:\n- "
            . implode("\n- ", array_map(static fn (string $q): string => mb_substr($q, 0, 200, 'UTF-8'), $sample));
    }

    $system = <<<PROMPT
You create short practice review questions for university students based ONLY on the course materials provided.

Week: {$weekLabel}
Course: {$courseName}

Rules:
- Generate exactly {$batchSize} new questions grounded in the materials (no outside facts).
- Mix multiple_choice and short_answer types.
- For multiple_choice, provide exactly 4 choices and one correct answer string matching one choice.
- Keep questions clear, varied, and study-focused.
- Return ONLY valid JSON with this shape (no markdown fences):
{"questions":[{"question":"...", "type":"multiple_choice"|"short_answer", "choices":["A","B","C","D"], "answer":"..."}]}
PROMPT;

    $user = "Course materials for review:\n\n" . $sourceText . $excludeBlock;
    $raw = student_materials_reviewer_ai_call($system, $user, $batchSize > 12);
    if ($raw === null || $raw === '') {
        return [];
    }

    return student_materials_reviewer_parse_ai_questions($raw, $batchSize, $excludeNormalized);
}

/**
* @param list<string> $exclude
* @return list<array{id:int,question:string,type:string,choices:list<string>,answer:string,source:string}>|null
*/
function student_materials_reviewer_ai_questions(
    string $sourceText,
    string $weekLabel,
    string $courseName,
    int $count = 0,
    array $exclude = []
): ?array {
    if (!wellness_ai_is_enabled() || trim($sourceText) === '') {
        return null;
    }

    $unlimited = student_materials_reviewer_unlimited_count($count);
    $batchSize = $unlimited ? 15 : max(1, $count);
    $excludeTexts = $exclude;
    $all = [];

    $maxBatches = $unlimited ? 40 : 1;
    for ($batch = 0; $batch < $maxBatches; ++$batch) {
        $remaining = $unlimited ? $batchSize : max(0, $count - count($all));
        if (!$unlimited && $remaining < 1) {
            break;
        }

        $requestSize = $unlimited ? $batchSize : $remaining;
        $batchQuestions = student_materials_reviewer_ai_questions_batch(
            $sourceText,
            $weekLabel,
            $courseName,
            $requestSize,
            $excludeTexts
        );
        if ($batchQuestions === []) {

```

```

        break;
    }

    foreach ($batchQuestions as $row) {
        $all[] = $row;
        $excludeTexts[] = (string) $row['question'];
    }

    if (!$unlimited) {
        break;
    }
    if (count($batchQuestions) < $requestSize) {
        break;
    }
}

return $all !== [] ? student_materials_reviewer_renumber_questions($all) : null;
}

function student_materials_reviewer_ai_call(string $system, string $user, bool $largeBatch = false): ?string
{
    require_once dirname(__DIR__) . '/includes/wellness_ai.php';

    $messages = [
        ['role' => 'system', 'content' => $system],
        ['role' => 'user', 'content' => $user],
    ];

    $provider = wellness_ai_provider();
    if ($provider === 'auto') {
        if (wellness_ollama_reachable()) {
            $text = student_materials_reviewer_ai_ollama($messages, $largeBatch);
            if ($text !== null) {
                return $text;
            }
        }
        if (wellness_ai_has_cloud_key()) {
            return student_materials_reviewer_ai_openai_compatible($messages, $largeBatch);
        }

        return null;
    }

    if ($provider === 'ollama') {
        return student_materials_reviewer_ai_ollama($messages, $largeBatch);
    }

    if ($provider === 'gemini') {
        return student_materials_reviewer_ai_gemini($system, $user, $largeBatch);
    }

    return student_materials_reviewer_ai_openai_compatible($messages, $largeBatch);
}

/**
 * @param list<array{role:string,content:string}> $messages
 */
function student_materials_reviewer_ai_openai_compatible(array $messages, bool $largeBatch = false): ?string
{
    if (!wellness_ai_has_cloud_key()) {
        return null;
    }

    $baseUrl = rtrim((string) (defined('WELLNESS_AI_BASE_URL') ? WELLNESS_AI_BASE_URL : 'https://api.groq.com/openai/v1'), '/');
    $model = (string) (defined('WELLNESS_AI_MODEL') ? WELLNESS_AI_MODEL : 'llama-3.3-70b-versatile');
    $payload = [
        'model' => $model,
        'temperature' => 0.35,
        'max_tokens' => $largeBatch ? 2400 : 900,
        'messages' => $messages,
    ];

    return wellness_ai_http_json_post($baseUrl . '/chat/completions', [
        'Authorization: Bearer ' . trim((string) WELLNESS_AI_API_KEY),
    ], $payload, static function (array $data): ?string {
        return trim((string) ($data['choices'][0]['message']['content'] ?? ''));
    });
}

/**

```

```

* @param list<array{role:string,content:string}> $messages
*/
function student_materials_reviewer_ai_ollama(array $messages, bool $largeBatch = false): ?string
{
    $payload = [
        'model' => wellness_ollama_model(),
        'stream' => false,
        'options' => ['temperature' => 0.35, 'num_predict' => $largeBatch ? 2400 : 900],
        'messages' => $messages,
    ];

    return wellness_ai_http_json_post(wellness_ollama_url() . '/api/chat', [], $payload, static function (array $data): ?string {
        return trim((string) ($data['message']['content'] ?? ''));
    });
}

function student_materials_reviewer_ai_gemini(string $system, string $user, bool $largeBatch = false): ?string
{
    if (!wellness_ai_has_cloud_key()) {
        return null;
    }

    $model = (string) (defined('WELLNESS_AI_MODEL') ? WELLNESS_AI_MODEL : 'gemini-2.0-flash');
    $key = trim((string) WELLNESS_AI_API_KEY);
    $url = 'https://generativelanguage.googleapis.com/v1beta/models/'
        . rawurlencode($model) . ':generateContent?key=' . rawurlencode($key);

    $payload = [
        'systemInstruction' => ['parts' => [['text' => $system]]],
        'contents' => [['role' => 'user', 'parts' => [['text' => $user]]],
        'generationConfig' => ['temperature' => 0.35, 'maxOutputTokens' => $largeBatch ? 2400 : 900],
    ];

    return wellness_ai_http_json_post($url, [], $payload, static function (array $data): ?string {
        $candidates = $data['candidates'] ?? [];
        if (is_array($candidates) || $candidates === []) {
            return null;
        }
        $partsOut = $candidates[0]['content']['parts'] ?? [];
        $text = '';
        if (is_array($partsOut)) {
            foreach ($partsOut as $p) {
                $text .= (string) ($p['text'] ?? '');
            }
        }

        return trim($text);
    });
}

/**
 * @return array{
 *     ok: bool,
 *     week: string,
 *     classroom_id: int,
 *     course_code: string,
 *     course_name: string,
 *     material_count: int,
 *     source: string,
 *     ai_available: bool,
 *     questions: list<array{id:int,question:string,type:string,choices:list<string>,answer:string,source:string}>
 * }
 */
function student_materials_reviewer_generate(
    int $studentId,
    int $classroomId,
    string $weekLabel,
    int $count = 0,
    array $exclude = []
): array {
    $classroom = student_materials_reviewer_fetch_classroom($studentId, $classroomId);
    if ($classroom === null) {
        throw new RuntimeException('Classroom not found or you are not enrolled in it.');
```

```

$items = student_materials_reviewer_week_items($classroomId, $weekLabel);
$sourceText = student_materials_reviewer_build_source_text($items, $classroom, $weekLabel);
$courseName = trim((string) ($classroom['course_code'] ?? '')) . ' - ' . trim((string) ($classroom['course_name'] ?? ''));
$excludeNormalized = student_materials_reviewer_normalize_exclude_list($exclude);

$aiAvailable = wellness_ai_is_enabled();
$questions = null;
$source = 'builtin';
if ($aiAvailable && $items !== []) {
    $questions = student_materials_reviewer_ai_questions($sourceText, $weekLabel, $courseName, $count, $exclude);
    if ($questions !== null) {
        $source = 'ai';
    }
}

if ($questions === null) {
    $questions = student_materials_reviewer_builtin_questions($weekLabel, $items, $count, $excludeNormalized);
}

$questions = student_materials_reviewer_renumber_questions($questions);

return [
    'ok' => true,
    'week' => $weekLabel,
    'classroom_id' => $classroomId,
    'course_code' => (string) ($classroom['course_code'] ?? ''),
    'course_name' => (string) ($classroom['course_name'] ?? ''),
    'material_count' => count($items),
    'unlimited' => student_materials_reviewer_unlimited_count($count),
    'source' => $source,
    'ai_available' => $aiAvailable,
    'questions' => $questions,
];
}

```

FILE: includes/student_registration_helpers.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/functions.php';
require_once __DIR__ . '/mail_helpers.php';
require_once __DIR__ . '/account_registration_helpers.php';

function student_registration_table_ready(): bool
{
    return db_table_exists('student_registration_requests');
}

/**
 * @return array<int, array{id: int, college_code: string, college_name: string}>
 */
function active_colleges_for_registration(): array
{
    if (!db_table_exists('colleges')) {
        return [];
    }
    $st = db()->query(
        "SELECT id, college_code, college_name FROM colleges WHERE status = 'active' ORDER BY college_name"
    );
    return $st->fetchAll(PDO::FETCH_ASSOC) ?: [];
}

/**
 * @return string[]
 */
function active_programs_for_college(int $collegeId): array
{
    if ($collegeId < 1 || !db_table_exists('programs')) {
        return [];
    }
    $st = db()->prepare(
        "SELECT program_name FROM programs WHERE college_id = ? AND status = 'active' ORDER BY program_name"
    );
    $st->execute([$collegeId]);
    return $st->fetchAll(PDO::FETCH_COLUMN) ?: [];
}

```

```

/**
 * @return list<string>
 */
function active_year_levels_for_program(int $collegeId, string $programName): array
{
    $defaults = sort_schedule_year_levels(['1', '2', '3', '4', '5']);
    if ($collegeId < 1 || $programName === '' || !db_table_exists('programs')) {
        return $defaults;
    }
    $st = db()->prepare(
        "SELECT id FROM programs WHERE college_id = ? AND program_name = ? AND status = 'active' LIMIT 1"
    );
    $st->execute([$collegeId, $programName]);
    $programId = (int) ($st->fetchColumn() ?: 0);
    if ($programId < 1) {
        return $defaults;
    }
    $levels = program_defined_year_levels($programId);
    return $levels !== [] ? $levels : $defaults;
}

/**
 * @return array<string, list<string>>
 */
function active_year_levels_by_program_for_college(int $collegeId): array
{
    if ($collegeId < 1 || !db_table_exists('programs')) {
        return [];
    }
    if (db_table_exists('programs_year_levels')) {
        return dean_program_year_levels_map($collegeId);
    }
    $defaults = sort_schedule_year_levels(['1', '2', '3', '4', '5']);
    $out = [];
    foreach (active_programs_for_college($collegeId) as $programName) {
        $out[(string) $programName] = $defaults;
    }
    return $out;
}

function validate_student_registration_input(
    string $username,
    string $fullName,
    string $email,
    string $studentNumber,
    int $collegeId,
    string $programName,
    string $yearLevel = ''
): void {
    if ($username === '' || $fullName === '') {
        throw new RuntimeException('Username and full name are required.');
```



```

        if (!in_array($programName, active_programs_for_college($collegeId), true)) {
            throw new RuntimeException('Invalid program for the selected college.');
```

```
        }
    }
}
```

```
function username_taken_for_registration(string $username): bool
```

```
{
    $st = db()->prepare('SELECT COUNT(*) FROM users WHERE username = ?');
    $st->execute([$username]);
    if ((int) $st->fetchColumn() > 0) {
        return true;
    }
    if (!student_registration_table_ready()) {
        return false;
    }
    $pending = db()->prepare(
        "SELECT COUNT(*) FROM student_registration_requests WHERE username = ? AND status = 'pending'"
    );
    $pending->execute([$username]);
    return (int) $pending->fetchColumn() > 0;
}
```

```
function submit_student_registration(
```

```
    string $username,
    string $fullName,
    string $email,
    string $studentNumber,
    int $collegeId,
    string $programName,
    string $yearLevel
): void {
    if (!student_registration_table_ready()) {
        throw new RuntimeException('Student registration is not available yet. Ask your administrator to run upgrade_roles.php.');
```

```
    }
    if (!db_table_exists('classroom_students') || !db_column_exists('users', 'assigned_program')) {
        throw new RuntimeException('Student accounts are not fully configured. Run upgrade_roles.php first.');
```

```
    }

    $username = trim($username);
    $fullName = trim($fullName);
    $email = trim($email);
    $studentNumber = trim($studentNumber);
    $programName = trim($programName);
    $yearLevel = trim($yearLevel);

    validate_student_registration_input(
        $username,
        $fullName,
        $email,
        $studentNumber,
        $collegeId,
        $programName,
        $yearLevel
    );
}
```

```
    if (!db_column_exists('student_registration_requests', 'year_level')) {
        throw new RuntimeException('Student registration year level is not available yet. Ask your administrator to run
upgrade_roles.php.');
```

```
    }

    if (username_taken_for_registration($username)) {
        $s = suggest_available_usernames($username, 3);
        throw new RuntimeException(
            'Username is already taken or pending approval.'
            . ($s ? ' Try: ' . implode(' ', $s) : '')
        );
    }
}
```

```
$placeholderPassword = password_hash(bin2hex(random_bytes(16)), PASSWORD_DEFAULT);
```

```
db()->prepare(
    'INSERT INTO student_registration_requests
    (username, password, full_name, email, student_number, college_id, program_name, year_level, status)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)'
)->execute([
    $username,
    $placeholderPassword,
    $fullName,
    $email,
    $studentNumber,
```

```

        $collegeId,
        $programName,
        $yearLevel,
        'pending',
    ]]);
}

function registration_status_message_for_username(string $username): ?string
{
    if ($username === '' || !student_registration_table_ready()) {
        return null;
    }
    $st = db()->prepare(
        "SELECT status, rejection_reason FROM student_registration_requests
        WHERE username = ? ORDER BY id DESC LIMIT 1"
    );
    $st->execute([trim($username)]);
    $row = $st->fetch();
    if (!$row) {
        return null;
    }
    $status = (string) ($row['status'] ?? '');
    if ($status === 'pending') {
        return 'Your registration is pending approval by your Dean. You will receive an email with your sign-in credentials once approved.';
    }
    if ($status === 'rejected') {
        $reason = trim((string) ($row['rejection_reason'] ?? ''));
        return 'Your registration was not approved.'
            . ($reason !== '' ? ' Reason: ' . $reason : ' You may register again with corrected details.');
```

```

$yearLevel = trim($yearLevel);
if (db_column_exists('classroom_students', 'year_level')) {
    db()->prepare(
        'INSERT INTO classroom_students (user_id, student_number, full_name, email, year_level) VALUES (?, ?, ?, ?, ?)'
    )->execute([
        $uid,
        $studentNumber,
        $fullName,
        $hasUserEmail ? $email : '',
        $yearLevel,
    ]);
} else {
    db()->prepare(
        'INSERT INTO classroom_students (user_id, student_number, full_name, email) VALUES (?, ?, ?, ?)'
    )->execute([
        $uid,
        $studentNumber,
        $fullName,
        $hasUserEmail ? $email : '',
    ]);
}

db()->commit();
return $uid;
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    throw $e;
}
}

/**
 * Load a registration request scoped to a college, optionally to one program.
 * Pass null for $programScope to allow any program in the college (dean jurisdiction).
 *
 * @return array<string, mixed>
 */
function load_student_registration_request_for_reviewer(int $requestId, int $collegeId, ?string $programScope): array
{
    $st = db()->prepare('SELECT * FROM student_registration_requests WHERE id = ?');
    $st->execute([$requestId]);
    $req = $st->fetch(PDO::FETCH_ASSOC);
    if (!$req) {
        throw new RuntimeException('Registration request not found.');
```

```

$req = load_student_registration_request_for_reviewer($requestId, $collegeId, $programScope);
$status = (string) ($req['status'] ?? '');
$username = (string) $req['username'];
$email = trim((string) ($req['email'] ?? ''));
$fullName = (string) $req['full_name'];

if ($status === 'rejected') {
    throw new RuntimeException('This registration was already rejected.');
```

```

}

if ($status === 'approved') {
    if (student_user_exists($username)) {
        return [
            'email' => $email,
            'mail_sent' => false,
            'already_approved' => true,
        ];
    }
} elseif ($status !== 'pending') {
    throw new RuntimeException('Registration request is not pending approval.');
```

```

}

if (student_user_exists($username)) {
    throw new RuntimeException('A user with this username already exists. Reject this request or contact admin.');
```

```

}

$plainPassword = generate_temp_password();

$uid = create_student_account(
    $username,
    $plainPassword,
    $fullName,
    $email,
    (string) ($req['student_number'] ?? ''),
    (int) $req['college_id'],
    (string) $req['program_name'],
    false,
    (string) ($req['year_level'] ?? '')
);

if ($status === 'pending') {
    db()->prepare(
        "UPDATE student_registration_requests
        SET status = 'approved', reviewed_by_user_id = ?, reviewed_at = NOW()
        WHERE id = ?"
    )->execute([$reviewerUserId, $requestId]);
}

$mailSent = false;
if (db_column_exists('users', 'email') && $email !== '' && filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $notifyResult = notify_new_account_credentials(
        $uid,
        $email,
        $fullName,
        $username,
        $plainPassword,
        'student_registration'
    );
    $mailSent = $notifyResult === true;
} else {
    mark_new_account_requires_password_change($uid);
}

return [
    'email' => $email,
    'mail_sent' => $mailSent,
    'user_id' => $uid,
];
}

function reject_student_registration_request(
    int $requestId,
    int $reviewerUserId,
    int $collegeId,
    ?string $programScope,
    string $reason = ''
): void {
    if (!student_registration_table_ready()) {
        throw new RuntimeException('Registration requests are not available.');
```

```

    }

    $req = load_student_registration_request_for_reviewer($requestId, $collegeId, $programScope);
    if ((string) ($req['status'] ?? '') === 'rejected') {
        throw new RuntimeException('This registration was already rejected.');
```

```

    }
    if ((string) ($req['status'] ?? '') !== 'pending') {
        throw new RuntimeException('This registration is not pending approval.');
```

```

    }

    db()->prepare(
        "UPDATE student_registration_requests
        SET status = 'rejected', reviewed_by_user_id = ?, reviewed_at = NOW(), rejection_reason = ?
        WHERE id = ?"
    )->execute([$reviewerUserId, trim($reason), $requestId]);
}

/**
 * @return array<int, array<string, mixed>>
 */
function pending_registrations_for_college(int $collegeId, ?string $programScope = null): array
{
    if (!student_registration_table_ready()) {
        return [];
    }
    $sql = "SELECT r.*, c.college_name
    FROM student_registration_requests r
    INNER JOIN colleges c ON c.id = r.college_id
    WHERE r.status = 'pending' AND r.college_id = ?";
    $params = [$collegeId];
    if ($programScope !== null) {
        $sql .= ' AND r.program_name = ?';
        $params[] = $programScope;
    }
    $sql .= ' ORDER BY r.program_name ASC, r.created_at ASC';
    $st = db()->prepare($sql);
    $st->execute($params);
    return $st->fetchAll(PDO::FETCH_ASSOC) ? : [];
}

/**
 * @return array<int, array<string, mixed>>
 */
function pending_registrations_for_program(int $collegeId, string $programScope): array
{
    return pending_registrations_for_college($collegeId, $programScope);
}

function count_pending_registrations_for_college(int $collegeId, ?string $programScope = null): int
{
    if (!student_registration_table_ready()) {
        return 0;
    }
    $sql = "SELECT COUNT(*) FROM student_registration_requests
    WHERE status = 'pending' AND college_id = ?";
    $params = [$collegeId];
    if ($programScope !== null) {
        $sql .= ' AND program_name = ?';
        $params[] = $programScope;
    }
    $st = db()->prepare($sql);
    $st->execute($params);
    return (int) $st->fetchColumn();
}

function count_pending_registrations_for_program(int $collegeId, string $programScope): int
{
    return count_pending_registrations_for_college($collegeId, $programScope);
}

/**
 * Approve every pending registration in scope. Individual failures do not stop the batch.
 *
 * @return array{approved: int, already_approved: int, mail_sent: int, failed: int, errors: list<string>}
 */
function approve_all_pending_student_registrations(int $reviewerUserId, int $collegeId, ?string $programScope): array
{
    $result = [
        'approved' => 0,
        'already_approved' => 0,
```

```

        'mail_sent' => 0,
        'failed' => 0,
        'errors' => [],
    ];

    $pending = $programScope !== null
        ? pending_registrations_for_program($collegeId, $programScope)
        : pending_registrations_for_college($collegeId, null);

    foreach ($pending as $row) {
        $requestId = (int) ($row['id'] ?? 0);
        if ($requestId < 1) {
            continue;
        }

        try {
            $approval = approve_student_registration_request(
                $requestId,
                $reviewerUserId,
                $collegeId,
                $programScope
            );
            if (!empty($approval['already_approved'])) {
                $result['already_approved']++;
                continue;
            }
            $result['approved']++;
            if (!empty($approval['mail_sent'])) {
                $result['mail_sent']++;
            }
        } catch (Throwable $e) {
            $result['failed']++;
            $label = trim((string) ($row['full_name'] ?? ''));
            if ($label === '') {
                $label = (string) ($row['username'] ?? 'request #' . $requestId);
            }
            $result['errors'][] = $label . ': ' . $e->getMessage();
        }
    }

    return $result;
}

function bulk_student_registration_approval_flash_message(array $result): string
{
    $approved = (int) ($result['approved'] ?? 0);
    $alreadyApproved = (int) ($result['already_approved'] ?? 0);
    $mailSent = (int) ($result['mail_sent'] ?? 0);
    $failed = (int) ($result['failed'] ?? 0);
    $errors = $result['errors'] ?? [];

    if ($approved === 0 && $alreadyApproved === 0 && $failed === 0) {
        return 'No pending student registrations to approve.';
    }

    $parts = [];
    if ($approved > 0) {
        $parts[] = $approved === 1
            ? '1 registration approved'
            : $approved . ' registrations approved';
        if ($mailSent > 0) {
            $parts[] = $mailSent === 1
                ? 'credentials emailed to 1 student'
                : 'credentials emailed to ' . $mailSent . ' students';
        } elseif ($approved > 0) {
            $parts[] = 'share credentials manually where no email was on file';
        }
    }
    if ($alreadyApproved > 0) {
        $parts[] = $alreadyApproved === 1
            ? '1 was already approved'
            : $alreadyApproved . ' were already approved';
    }
    if ($failed > 0) {
        $parts[] = $failed === 1
            ? '1 could not be approved'
            : $failed . ' could not be approved';
    }

    $message = ucfirst(implode('; ', $parts)) . '.';

```

```

if ($failed > 0 && is_array($errors) && $errors !== []) {
    $shown = array_slice($errors, 0, 3);
    $message .= ' ' . implode(' ', $shown);
    if (count($errors) > 3) {
        $message .= ' (+ ' . (count($errors) - 3) . ' more)';
    }
}

return $message;
}

```

FILE: includes/student_tooltip.php

```

<?php
declare(strict_types=1);

/**
 * Cursor tooltips near the pointer (see assets/js/app_tooltips.js).
 * Active for admin, dean, gened, faculty, and student roles in the app shell.
 */
function app_cursor_tooltips_active(): bool
{
    if (session_status() !== PHP_SESSION_ACTIVE) {
        return false;
    }
    $r = (string) ($_SESSION['role'] ?? '');

    return in_array($r, ['super_admin', 'admin', 'dean', 'gened', 'faculty', 'student'], true);
}

function app_tooltip_attr(string $text): string
{
    if (!app_cursor_tooltips_active()) {
        return '';
    }

    return ' data-app-tooltip="' . htmlspecialchars($text, ENT_QUOTES, 'UTF-8') . '"';
}

/** @deprecated Use app_tooltip_attr */
function student_tooltip_attr(string $text): string
{
    return app_tooltip_attr($text);
}

```

FILE: includes/student_wellness_ui.php

```

<?php
declare(strict_types=1);

/**
 * Wellness companion chat UI (included by student_wellness.php).
 */
?>
<div class="student-wellness-root">
    <style>
        /* Theme-aware tokens (light defaults; dark overrides below) */
        .student-wellness-root {
            --wellness-accent: #0d9488;
            --wellness-accent-hover: #0f766e;
            --wellness-accent-soft: rgba(13, 148, 136, 0.18);
            --wellness-surface: var(--bs-body-bg, #ffffff);
            --wellness-surface-2: var(--bs-tertiary-bg, #f8fafc);
            --wellness-border: var(--bs-border-color, #e2e8f0);
            --wellness-text: var(--bs-body-color, #1e293b);
            --wellness-text-muted: var(--bs-secondary-color, #64748b);
            --wellness-header-bg: linear-gradient(135deg, #f0fdfa 0%, #ecfeff 50%, #f8fafc 100%);
            --wellness-alert-bg: #ffffeb;
            --wellness-alert-border: #fcd34d;
            --wellness-alert-text: #422006;
            --wellness-bubble-bg: var(--bs-body-bg, #ffffff);
            --wellness-bubble-user-bg: #eef2ff;
            --wellness-bubble-user-border: #c7d2fe;
            --wellness-bubble-user-text: var(--bs-body-color, #1e293b);
            --wellness-crisis-bg: #fef2f2;
            --wellness-crisis-border: #fecaca;
            --wellness-crisis-text: #7f1d1d;

```

```

--wellness-shadow: 0 8px 32px rgba(15, 23, 42, 0.06);
--wellness-bubble-shadow: 0 4px 14px rgba(15, 23, 42, 0.04);
--wellness-input-bg: var(--bs-body-bg, #ffffff);
--wellness-typing-dot: #94a3b8;
max-width: 820px;
margin: 0 auto;
color: var(--wellness-text);
}

html[data-bs-theme="dark"] .student-wellness-root {
  --wellness-accent: #2dd4bf;
  --wellness-accent-hover: #14b8a6;
  --wellness-accent-soft: rgba(45, 212, 191, 0.22);
  --wellness-surface: var(--app-surface, #151b24);
  --wellness-surface-2: var(--app-bg, #0f1419);
  --wellness-border: var(--app-border-strong, rgba(255, 255, 255, 0.12));
  --wellness-text: var(--app-text, #e2e8f0);
  --wellness-text-muted: var(--app-text-muted, #94a3b8);
  --wellness-header-bg: linear-gradient(135deg, #134e4a 0%, #1e293b 45%, #151b24 100%);
  --wellness-alert-bg: rgba(251, 191, 36, 0.12);
  --wellness-alert-border: rgba(251, 191, 36, 0.45);
  --wellness-alert-text: #fde68a;
  --wellness-bubble-bg: #1a2230;
  --wellness-bubble-user-bg: rgba(99, 102, 241, 0.22);
  --wellness-bubble-user-border: rgba(129, 140, 248, 0.45);
  --wellness-bubble-user-text: #e0e7ff;
  --wellness-crisis-bg: rgba(239, 68, 68, 0.14);
  --wellness-crisis-border: rgba(248, 113, 113, 0.5);
  --wellness-crisis-text: #fecaca;
  --wellness-shadow: var(--app-shadow-md, 0 8px 24px rgba(0, 0, 0, 0.35));
  --wellness-bubble-shadow: 0 4px 12px rgba(0, 0, 0, 0.25);
  --wellness-input-bg: #101722;
  --wellness-typing-dot: #94a3b8;
}

.student-wellness-card {
  border-radius: 16px;
  border: 1px solid var(--wellness-border);
  background: var(--wellness-surface);
  box-shadow: var(--wellness-shadow);
  overflow: hidden;
  display: flex;
  flex-direction: column;
  min-height: min(72vh, 680px);
}

.student-wellness-card-header {
  padding: 1rem 1.25rem;
  background: var(--wellness-header-bg);
  border-bottom: 1px solid var(--wellness-border);
  color: var(--wellness-text);
}

.student-wellness-card-header .text-muted {
  color: var(--wellness-text-muted) !important;
}

.student-wellness-alert {
  font-size: 0.76rem;
  line-height: 1.55;
  background: var(--wellness-alert-bg);
  border: 1px solid var(--wellness-alert-border);
  border-radius: 12px;
  padding: 0.7rem 0.9rem;
  margin-top: 0.75rem;
  color: var(--wellness-alert-text);
}

.student-wellness-thread {
  flex: 1;
  display: flex;
  flex-direction: column;
  gap: 12px;
  padding: 1rem 1.25rem;
  overflow-y: auto;
  min-height: 280px;
  max-height: min(48vh, 480px);
  background: var(--wellness-surface-2);
}

```



```

.student-wellness-bubble {
  max-width: min(92%, 560px);
  align-self: flex-start;
  border-radius: 14px;
  padding: 0.65rem 0.95rem;
  font-size: 0.875rem;
  line-height: 1.55;
  background: var(--wellness-bubble-bg);
  color: var(--wellness-text);
  border: 1px solid var(--wellness-border);
  box-shadow: var(--wellness-bubble-shadow);
}

.student-wellness-bubble strong {
  color: var(--wellness-text);
}

.student-wellness-bubble.user {
  align-self: flex-end;
  background: var(--wellness-bubble-user-bg);
  border-color: var(--wellness-bubble-user-border);
  color: var(--wellness-bubble-user-text);
}

.student-wellness-bubble.user strong {
  color: var(--wellness-bubble-user-text);
}

.student-wellness-bubble.crisis {
  align-self: stretch;
  background: var(--wellness-crisis-bg);
  border-color: var(--wellness-crisis-border);
  color: var(--wellness-crisis-text);
}

.student-wellness-bubble.crisis strong {
  color: var(--wellness-crisis-text);
}

.student-wellness-bubble-meta {
  font-size: 0.65rem;
  text-transform: uppercase;
  letter-spacing: 0.04em;
  color: var(--wellness-text-muted);
  margin-bottom: 0.25rem;
}

.student-wellness-compose {
  padding: 1rem 1.25rem 1.25rem;
  border-top: 1px solid var(--wellness-border);
  background: var(--wellness-surface);
}

.student-wellness-compose textarea {
  width: 100%;
  min-height: 88px;
  border-radius: 12px;
  border: 1px solid var(--wellness-border);
  background: var(--wellness-input-bg);
  color: var(--wellness-text);
  padding: 0.7rem 0.9rem;
  font-size: 0.9rem;
  resize: vertical;
}

.student-wellness-compose textarea::placeholder {
  color: var(--wellness-text-muted);
  opacity: 0.85;
}

.student-wellness-compose textarea:focus {
  border-color: var(--wellness-accent);
  outline: none;
  box-shadow: 0 0 3px var(--wellness-accent-soft);
}

.student-wellness-actions {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
}

```

```

    gap: 10px;
    margin-top: 0.6rem;
}

.student-wellness-send {
    border: none;
    border-radius: 999px;
    background: var(--wellness-accent);
    color: #0f172a;
    font-weight: 600;
    font-size: 0.875rem;
    padding: 0.5rem 1.4rem;
}

html[data-bs-theme="dark"] .student-wellness-send {
    color: #042f2e;
}

.student-wellness-send:hover:not(:disabled) {
    background: var(--wellness-accent-hover);
}

.student-wellness-send:disabled {
    opacity: 0.55;
}

.student-wellness-status {
    font-size: 0.75rem;
    color: var(--wellness-text-muted);
}

.student-wellness-status--warn {
    color: #b45309;
}

.student-wellness-status--ok {
    color: #15803d;
}

html[data-bs-theme="dark"] .student-wellness-status--warn {
    color: #fbbf24;
}

html[data-bs-theme="dark"] .student-wellness-status--ok {
    color: #4ade80;
}

.student-wellness-hint {
    font-size: 0.72rem;
    color: var(--wellness-text-muted);
    margin: 0.5rem 0 0;
}

.student-wellness-typing {
    display: none;
    align-self: flex-start;
    max-width: min(92%, 560px);
}

.student-wellness-typing.is-visible {
    display: block;
}

.student-wellness-typing .student-wellness-bubble {
    padding: 0.75rem 1rem;
    min-width: 4.5rem;
}

.student-wellness-typing-dots {
    display: inline-flex;
    align-items: center;
    gap: 5px;
    height: 1.25rem;
}

.student-wellness-typing-dots span {
    width: 7px;
    height: 7px;
    border-radius: 50%;
    background: var(--wellness-typing-dot);
}

```

```

    animation: wellness-typing-bounce 1.2s ease-in-out infinite;
  }

.student-wellness-typing-dots span:nth-child(2) {
  animation-delay: 0.15s;
}

.student-wellness-typing-dots span:nth-child(3) {
  animation-delay: 0.3s;
}

@keyframes wellness-typing-bounce {
  0%, 60%, 100% {
    transform: translateY(0);
    opacity: 0.45;
  }
  30% {
    transform: translateY(-5px);
    opacity: 1;
  }
}

/* Page title above card */
html[data-bs-theme="dark"] .student-wellness-page .text-success {
  color: var(--wellness-accent) !important;
}

@media (max-width: 575.98px) {
  .student-wellness-root {
    max-width: 100%;
  }

  .student-wellness-card {
    min-height: min(82vh, 640px);
    border-radius: 12px;
  }

  .student-wellness-card-header,
  .student-wellness-compose,
  .student-wellness-thread {
    padding-left: 0.875rem;
    padding-right: 0.875rem;
  }

  .student-wellness-thread {
    max-height: min(44vh, 360px);
    min-height: 220px;
  }

  .student-wellness-send {
    flex: 1 1 100%;
    text-align: center;
  }

  .student-wellness-bubble {
    max-width: 96%;
  }
}
</style>

<div class="student-wellness-card" aria-label="Wellness chat">
  <div class="student-wellness-card-header">
    <div class="d-flex align-items-center gap-2">
      <span class="rounded-circle d-inline-flex align-items-center justify-content-center"
        style="width:40px;height:40px;background:var(--wellness-accent-soft);color:var(--wellness-accent);">
        <i class="fa-solid fa-heart-pulse" aria-hidden="true"></i>
      </span>
      <div>
        <div class="fw-semibold">Wellness companion</div>
        <div class="text-muted small">Private support chat · English / Filipino</div>
      </div>
    </div>
    <div class="student-wellness-alert">
      <strong>Not therapy or emergency care.</strong> For crisis or thoughts of self-harm, use the hotlines in any crisis response—not
      this chat alone.
    </div>
  </div>

  <div class="student-wellness-thread" id="wellnessThread" aria-live="polite"></div>

```

```

<div class="student-wellness-compose">
  <label class="visually-hidden" for="wellnessMessageInput">Your message</label>
  <textarea id="wellnessMessageInput" placeholder="Hal. Nababalisa ako sa exam bukas...&#10;Eg. I can't sleep and feel alone on campus.">
</textarea>
  <div class="student-wellness-actions">
    <button type="button" class="student-wellness-send" id="wellnessSendBtn">Send</button>
    <span class="student-wellness-status" id="wellnessStatus"></span>
  </div>
  <p class="student-wellness-hint mb-0">Ctrl+Enter to send · Conversation is tied to your student login session.</p>
</div>
</div>
</div>

<script>
(function () {
  var thread = document.getElementById('wellnessThread');
  var ta = document.getElementById('wellnessMessageInput');
  var go = document.getElementById('wellnessSendBtn');
  var st = document.getElementById('wellnessStatus');
  if (!thread || !ta || !go) return;

  var apiUrl = String(new URL('api/student_wellness_chat.php', window.location.href)).split(/[?#]/)[0];
  var wellnessHistory = [];
  var typingBubble = null;

  function showTypingBubble() {
    if (!typingBubble) {
      typingBubble = document.createElement('div');
      typingBubble.className = 'student-wellness-typing';
      typingBubble.id = 'wellnessTypingBubble';
      typingBubble.setAttribute('aria-label', 'Wellness companion is typing');
      typingBubble.innerHTML =
        '<article class="student-wellness-bubble">' +
        '<div class="student-wellness-bubble-meta">Wellness companion</div>' +
        '<div class="student-wellness-typing-dots" aria-hidden="true">' +
        '<span></span><span></span><span></span>' +
        '</div>' +
        '</article>';
    }
    typingBubble.classList.add('is-visible');
    typingBubble.setAttribute('aria-hidden', 'false');
    thread.appendChild(typingBubble);
    thread.scrollTop = thread.scrollHeight;
  }

  function hideTypingBubble() {
    if (!typingBubble) return;
    typingBubble.classList.remove('is-visible');
    typingBubble.setAttribute('aria-hidden', 'true');
  }

  function scrollThreadToEnd() {
    thread.scrollTop = thread.scrollHeight;
  }

  function escapeHtml(s) {
    var d = document.createElement('div');
    d.textContent = s == null ? '' : String(s);
    return d.innerHTML;
  }

  function formatMd(s) {
    var r = escapeHtml(String(s).trim());
    r = r.replace(/\*\[s\S]+\?\]\*\//g, '<strong>$1</strong>');
    r = r.replace(/\n- /g, '<br />\u2022 ');
    r = r.replace(/\n(\d+)\. /g, '<br />$1. ');
    r = r.replace(/\n\n?/g, '<br />');
    return r;
  }

  function appendBubble(side, text, crisis) {
    hideTypingBubble();
    var bub = document.createElement('article');
    var meta = document.createElement('div');
    meta.className = 'student-wellness-bubble-meta';
    meta.textContent = side === 'user' ? 'You' : (crisis ? 'Crisis support – Philippines' : 'Wellness companion');
    var inner = document.createElement('div');
    inner.innerHTML = formatMd(text);
    bub.appendChild(meta);
    bub.appendChild(inner);
  }
}

```

```

bub.className = 'student-wellness-bubble' + (side === 'user' ? ' user' : '') + (crisis ? ' crisis' : '');
thread.appendChild(bub);
scrollThreadToEnd();
}

fetch(apiUrl, { credentials: 'same-origin', headers: { Accept: 'application/json' } })
.then(function (r) { return r.json(); })
.then(function (info) {
  if (!info || !st) return;
  if (info.setup_hint) {
    st.textContent = info.setup_hint;
    st.className = 'student-wellness-status student-wellness-status--warn';
  } else if (info.groq && info.groq.ready) {
    st.textContent = 'Groq AI ready';
    st.className = 'student-wellness-status student-wellness-status--ok';
  }
})
.catch(function () {});

appendBubble(
  'assistant',
  'Kumusta! Sabihin kung ano ang pinaka-mabigat ngayon-exam, tulog, kalungkutan, pamilya, o relasyon. Sasagutin ko sa Tagalog o English. (Suporta lang-hindi therapy.)',
  false
);

function setBusy(busy, msg) {
  go.disabled = busy;
  ta.readOnly = busy;
  if (busy) {
    showTypingBubble();
    if (st && msg != null) st.textContent = msg;
    else if (st) st.textContent = '';
  } else {
    hideTypingBubble();
    if (st && msg != null) {
      st.textContent = msg;
    } else if (st && !st.classList.contains('student-wellness-status--warn') && !st.classList.contains('student-wellness-status--ok')) {
      st.textContent = '';
      st.className = 'student-wellness-status';
    }
  }
}

function sendOnce() {
  var body = ta.value.trim();
  if (!body) {
    if (st) {
      st.textContent = 'Type a message first.';
      setTimeout(function () { if (st && st.textContent === 'Type a message first.') st.textContent = ''; }, 3000);
    }
    return;
  }
  ta.value = '';
  appendBubble('user', body, false);
  setBusy(true);

  fetch(apiUrl, {
    method: 'POST',
    credentials: 'same-origin',
    headers: { 'Content-Type': 'application/json', Accept: 'application/json' },
    body: JSON.stringify({ message: body, language_hint: 'auto', history: wellnessHistory }),
  })
  .then(function (r) {
    return r.json().then(function (j) { return { ok: r.ok, j: j, status: r.status }; });
  })
  .then(function (pack) {
    if (!pack.ok || !pack.j || pack.j.ok === false) {
      appendBubble(
        'assistant',
        pack.status === 401
          ? '**Please log in as a student** and open this page again.'
          : '**Could not get a reply.** ' + (pack.j && pack.j.error ? String(pack.j.error) : 'Try again.'),
        false
      );
      setBusy(false, '');
      return;
    }
    var reply = String(pack.j.reply || '');
    appendBubble('assistant', reply, Boolean(pack.j.crisis));
  });
}

```

```

    if (!pack.j.crisis) {
        wellnessHistory.push({ role: 'user', content: body });
        wellnessHistory.push({ role: 'assistant', content: reply });
        if (wellnessHistory.length > 16) wellnessHistory = wellnessHistory.slice(-16);
    }
    if (st) {
        var bits = [];
        if (pack.j.detected_language === 'tl') bits.push('Filipino');
        else if (pack.j.detected_language === 'en') bits.push('English');
        var src = pack.j.reply_source || 'smart';
        if (src === 'groq' || src === 'ai' || src === 'gemini' || src === 'ollama') bits.push('Groq AI');
        else bits.push('Smart');
        st.textContent = bits.join(' \u00b7 ');
        st.className = 'student-wellness-status';
    }
    setBusy(false, '');
})
.catch(function () {
    appendBubble('assistant', '**Network error.** Check your connection and try again.', false);
    setBusy(false, '');
});
});
}

go.addEventListener('click', sendOnce);
ta.addEventListener('keydown', function (e) {
    if ((e.ctrlKey || e.metaKey) && e.key === 'Enter') {
        e.preventDefault();
        sendOnce();
    }
});
ta.focus();
})();
</script>

```

FILE: includes/wellness_ai.php

```

<?php
declare(strict_types=1);

/**
 * Wellness AI: Groq, OpenAI, Gemini, or local Ollama (OpenAI-compatible).
 * Crisis checks stay in wellness_chatbot.php.
 */

function wellness_ai_provider(): string
{
    if (!defined('WELLNESS_AI_PROVIDER')) {
        return 'auto';
    }
    return strtolower(trim((string) WELLNESS_AI_PROVIDER));
}

function wellness_ai_has_cloud_key(): bool
{
    if (!defined('WELLNESS_AI_API_KEY')) {
        return false;
    }
    $key = trim((string) WELLNESS_AI_API_KEY);

    return $key !== '' && str_starts_with($key, 'gsk_');
}

function wellness_ai_is_enabled(): bool
{
    if (defined('WELLNESS_AI_ENABLED') && !WELLNESS_AI_ENABLED) {
        return false;
    }
    $p = wellness_ai_provider();
    if ($p === 'off' || $p === 'builtin') {
        return false;
    }
    if ($p === 'ollama') {
        return true;
    }
    if ($p === 'auto') {
        return wellness_ai_has_cloud_key() || wellness_ollama_reachable();
    }
}

```

```

    return wellness_ai_has_cloud_key();
}

function wellness_ollama_url(): string
{
    return rtrim((string) (defined('WELLNESS_OLLAMA_URL') ? WELLNESS_OLLAMA_URL : 'http://127.0.0.1:11434'), '/');
}

function wellness_ollama_model(): string
{
    return (string) (defined('WELLNESS_OLLAMA_MODEL') ? WELLNESS_OLLAMA_MODEL : 'llama3.2');
}

function wellness_ollama_reachable(): bool
{
    static $cache = null;
    if ($cache !== null) {
        return $cache;
    }

    $ch = curl_init(wellness_ollama_url() . '/api/tags');
    if ($ch === false) {
        $cache = false;
        return false;
    }
    curl_setopt_array($ch, [
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_TIMEOUT => 2,
        CURLOPT_CONNECTTIMEOUT => 1,
    ]);
    $raw = curl_exec($ch);
    $code = (int) curl_getinfo($ch, CURLINFO_HTTP_CODE);
    curl_close($ch);

    $cache = is_string($raw) && $code >= 200 && $code < 300;
    return $cache;
}

/**
 * @param list<array{role: string, content: string}> $history
 * @param list<string> $scenarios
 */
function wellness_ai_system_prompt(string $lang, array $scenarios, string $userMessage): string
{
    $focus = $scenarios !== [] ? implode(', ', array_slice($scenarios, 0, 3)) : 'general support';
    $langRule = $lang === 'tl'
        ? 'Sumagot sa natural na Filipino o Taglish–tulad ng kausap na ate/kuya sa campus, hindi textbook.'
        : 'Reply in warm, natural English–like a supportive peer mentor, not a textbook.';

    return <<<PROMPT
You are "Wellness Companion" for university students in the Philippines (WPU-style campus).

{$langRule}

The student's latest message (respond to THIS specifically):
---
{$userMessage}
---

Detected themes: {$focus}.

Your job:
1. First sentence: reflect what they feel or face, in fresh words (never copy their sentence in quotes).
2. Then answer their question directly with 2-4 short, practical steps they can do today or tomorrow.
3. End with one gentle line: you are not a therapist; suggest campus guidance if this lasts weeks or feels overwhelming.

Hard rules:
- Max ~150 words. Short paragraphs or bullets.
- Never diagnose (no "you have depression/anxiety disorder").
- Never prescribe medicine.
- No headers like "Direct answer:" or "Sagot:".
- If they mix English and Tagalog, match their style.
- Be kind, specific, and human–not generic wellness pamphlet text.
PROMPT;
}

/**
 * @param list<array{role: string, content: string}> $history
 * @return list<array{role: string, content: string}>
 */

```

```

function wellness_ai_sanitize_history(array $history): array
{
    $out = [];
    foreach ($history as $turn) {
        if (!is_array($turn)) {
            continue;
        }
        $role = (string) ($turn['role'] ?? '');
        $content = trim((string) ($turn['content'] ?? ''));
        if ($content === '' || mb_strlen($content, 'UTF-8') > 2000) {
            continue;
        }
        if ($role !== 'user' && $role !== 'assistant') {
            continue;
        }
        $out[] = ['role' => $role, 'content' => $content];
    }

    return array_slice($out, -8);
}

/**
 * @param list<array{role: string, content: string}> $history
 * @param list<string> $scenarios
 */
function wellness_ai_chat(string $userMessage, string $lang, array $scenarios, array $history = []): ?string
{
    if (!wellness_ai_is_enabled()) {
        return null;
    }

    $provider = wellness_ai_provider();
    if ($provider === 'auto') {
        if (wellness_ollama_reachable()) {
            $text = wellness_ai_call_ollama($userMessage, $lang, $scenarios, $history);
            if ($text !== null) {
                return $text;
            }
        }
        if (wellness_ai_has_cloud_key()) {
            return wellness_ai_call_openai_compatible($userMessage, $lang, $scenarios, $history);
        }
        return null;
    }

    if ($provider === 'ollama') {
        return wellness_ai_call_ollama($userMessage, $lang, $scenarios, $history);
    }

    if ($provider === 'gemini') {
        return wellness_ai_call_gemini($userMessage, $lang, $scenarios, $history);
    }

    if ($provider === 'groq' || $provider === 'openai') {
        return wellness_ai_call_openai_compatible($userMessage, $lang, $scenarios, $history);
    }

    return wellness_ai_call_openai_compatible($userMessage, $lang, $scenarios, $history);
}

/** Quick connectivity check for admin GET / status (does not log the key). */
function wellness_groq_status(): array
{
    $ready = wellness_ai_provider() === 'groq' && wellness_ai_has_cloud_key() && wellness_ai_is_enabled();

    return [
        'provider' => wellness_ai_provider(),
        'configured' => wellness_ai_has_cloud_key(),
        'ready' => $ready,
        'model' => defined('WELLNESS_AI_MODEL') ? (string) WELLNESS_AI_MODEL : '',
        'message' => $ready
            ? 'Groq is configured. Wellness chat will use cloud AI.'
            : 'Add your Groq API key (gsk_...) in config/config.php or set GROQ_API_KEY environment variable.',
    ];
}

/**
 * @param list<array{role: string, content: string}> $history
 * @param list<string> $scenarios
 */

```



```

*/
function wellness_ai_build_messages(string $userMessage, string $lang, array $scenarios, array $history): array
{
    $messages = [
        ['role' => 'system', 'content' => wellness_ai_system_prompt($lang, $scenarios, $userMessage)],
    ];
    foreach (wellness_ai_sanitize_history($history) as $turn) {
        $messages[] = $turn;
    }
    $messages[] = ['role' => 'user', 'content' => $userMessage];

    return $messages;
}

/**
 * @param list<array{role: string, content: string}> $history
 * @param list<string> $scenarios
 */
function wellness_ai_call_openai_compatible(string $userMessage, string $lang, array $scenarios, array $history): ?string
{
    if (!wellness_ai_has_cloud_key()) {
        return null;
    }

    $baseUrl = rtrim((string) (defined('WELLNESS_AI_BASE_URL') ? WELLNESS_AI_BASE_URL : 'https://api.groq.com/openai/v1'), '/');
    $model = (string) (defined('WELLNESS_AI_MODEL') ? WELLNESS_AI_MODEL : 'llama-3.3-70b-versatile');

    $payload = [
        'model' => $model,
        'temperature' => 0.72,
        'max_tokens' => 520,
        'messages' => wellness_ai_build_messages($userMessage, $lang, $scenarios, $history),
    ];

    return wellness_ai_http_json_post($baseUrl . '/chat/completions', [
        'Authorization: Bearer ' . trim((string) WELLNESS_AI_API_KEY),
    ], $payload, static function (array $data): ?string {
        return trim((string) ($data['choices'][0]['message']['content'] ?? ''));
    });
}

/**
 * @param list<array{role: string, content: string}> $history
 * @param list<string> $scenarios
 */
function wellness_ai_call_ollama(string $userMessage, string $lang, array $scenarios, array $history): ?string
{
    $payload = [
        'model' => wellness_ollama_model(),
        'stream' => false,
        'options' => ['temperature' => 0.72, 'num_predict' => 520],
        'messages' => wellness_ai_build_messages($userMessage, $lang, $scenarios, $history),
    ];

    return wellness_ai_http_json_post(wellness_ollama_url() . '/api/chat', [], $payload, static function (array $data): ?string {
        return trim((string) ($data['message']['content'] ?? ''));
    });
}

/**
 * @param list<array{role: string, content: string}> $history
 * @param list<string> $scenarios
 */
function wellness_ai_call_gemini(string $userMessage, string $lang, array $scenarios, array $history): ?string
{
    if (!wellness_ai_has_cloud_key()) {
        return null;
    }

    $model = (string) (defined('WELLNESS_AI_MODEL') ? WELLNESS_AI_MODEL : 'gemini-2.0-flash');
    $key = trim((string) WELLNESS_AI_API_KEY);
    $url = 'https://generativelanguage.googleapis.com/v1beta/models/'
        . rawurlencode($model) . ':generateContent?key=' . rawurlencode($key);

    $parts = [];
    foreach (wellness_ai_sanitize_history($history) as $turn) {
        $role = $turn['role'] === 'assistant' ? 'model' : 'user';
        $parts[] = ['role' => $role, 'parts' => [['text' => $turn['content']]]];
    }
    $parts[] = ['role' => 'user', 'parts' => [['text' => $userMessage]]];
}

```

```

$payload = [
    'systemInstruction' => ['parts' => [['text' => wellness_ai_system_prompt($lang, $scenarios, $userMessage)]]],
    'contents' => $parts,
    'generationConfig' => ['temperature' => 0.72, 'maxOutputTokens' => 520],
];

return wellness_ai_http_json_post($url, [], $payload, static function (array $data): ?string {
    $candidates = $data['candidates'] ?? [];
    if (!is_array($candidates) || $candidates === []) {
        return null;
    }
    $partsOut = $candidates[0]['content']['parts'] ?? [];
    $text = '';
    if (is_array($partsOut)) {
        foreach ($partsOut as $p) {
            $text .= (string) ($p['text'] ?? '');
        }
    }

    return trim($text);
});
}

/**
 * @param array<string, mixed> $payload
 */
function wellness_ai_http_json_post(string $url, array $extraHeaders, array $payload, callable $extract): ?string
{
    try {
        $json = json_encode($payload, JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR);
    } catch (JsonException $e) {
        return null;
    }

    $ch = curl_init($url);
    if ($ch === false) {
        return null;
    }

    $headers = array_merge(['Content-Type: application/json'], $extraHeaders);
    curl_setopt_array($ch, [
        CURLOPT_POST => true,
        CURLOPT_HTTPHEADER => $headers,
        CURLOPT_POSTFIELDS => $json,
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_TIMEOUT => 45,
        CURLOPT_CONNECTTIMEOUT => 8,
    ]);

    $raw = curl_exec($ch);
    $httpCode = (int) curl_getinfo($ch, CURLINFO_HTTP_CODE);
    curl_close($ch);

    if (!is_string($raw) || $raw === '' || $httpCode < 200 || $httpCode >= 300) {
        return null;
    }

    try {
        $data = json_decode($raw, true, 512, JSON_THROW_ON_ERROR);
    } catch (JsonException $e) {
        return null;
    }

    if (!is_array($data)) {
        return null;
    }

    $text = $extract($data);
    if ($text === null || $text === '') {
        return null;
    }

    if (mb_strlen($text, 'UTF-8') > 2400) {
        $text = mb_substr($text, 0, 2397, 'UTF-8') . '...';
    }

    return $text;
}

```

```
-----  
FILE: includes/wellness_chatbot.php  
-----
```

```
<?php  
declare(strict_types=1);  
  
/**  
 * Wellness chat: question-aware replies, crisis escalation (PH), EN/TL.  
 */  
  
function wellness_normalize_for_match(string $text): string  
{  
    $t = mb_strtolower(trim($text), 'UTF-8');  
    return preg_replace('/\s+/u', ' ', $t) ?: '';  
}  
  
/** @return 'en'|'tl' */  
function wellness_resolve_language(?string $clientHint, string $messageNorm): string  
{  
    $hint = $clientHint !== null ? mb_strtolower(trim($clientHint), 'UTF-8') : '';  
    if (in_array($hint, ['tl', 'fil', 'tagalog'], true)) {  
        return 'tl';  
    }  
    if (in_array($hint, ['en', 'english'], true)) {  
        return 'en';  
    }  
  
    $tlPatterns = [  
        'ako ', ' ako', 'hindi ako', 'hindi ko', 'saan ', ' bakit ', 'kasi ', 'dahil ',  
        'nababalisa', 'nababahalala', 'nalulungkot', 'pakiramdam', 'pagod', 'wala akong',  
        'gusto kong', 'naman ', 'po ', 'opo', 'ayoko ', 'tulong', 'nakakatakot',  
        'nakaka-stress', 'sama ng loob', 'kabado', 'kinakabahan', 'mag-isa', 'nag-iisa',  
        'paano ', 'ano ang', 'bakit ',  
    ];  
    $tlLex = ['ako', 'hindi', 'ba', 'ang', 'ng', 'kung', 'pero', 'ito', 'yung', 'naman', 'lang'];  
  
    $score = 0;  
    foreach ($tlPatterns as $p) {  
        if (mb_strpos($messageNorm, $p, 0, 'UTF-8') !== false) {  
            $score += 3;  
        }  
    }  
    foreach (preg_split('/[^\p{L}\p{N}]+/u', $messageNorm, -1, PREG_SPLIT_NO_EMPTY) ?: [] as $w) {  
        if (in_array($w, $tlLex, true)) {  
            $score += 1;  
        }  
    }  
  
    return $score >= 4 ? 'tl' : 'en';  
}  
  
function wellness_is_crisis_message(string $messageNorm): bool  
{  
    if ($messageNorm === '') {  
        return false;  
    }  
  
    $needles = [  
        'suicidal', 'suicide', 'kill myself', 'end my life', 'end it all',  
        'want to die', 'dont want to live', 'don\'t want to live',  
        'better off dead', 'self harm', 'selfharm', 'self-harm', 'hurt myself',  
        'cant go on', 'cannot go on', 'cant take it anymore', 'plan to kill',  
        'magpakamatay', 'magsuicide', 'patay na lang', 'mamatay na lang',  
        'gusto ko nang mamatay', 'gusto kong mamatay', 'wakasan ang buhay',  
        'tapusin ang buhay', 'ayoko na mabuhay', 'ayoko nang mabuhay',  
        'wala akong dahilan para mabuhay',  
    ];  
  
    foreach ($needles as $n) {  
        if (mb_strpos($messageNorm, $n, 0, 'UTF-8') !== false) {  
            return true;  
        }  
    }  
  
    return false;  
}  
  
function wellness_message_is_question(string $norm): bool  
{
```

```

if (str_contains($norm, '?')) {
    return true;
}
$starters = [
    'how ', 'what ', 'why ', 'when ', 'where ', 'can i ', 'should i ', 'is it ',
    'paano ', 'bakit ', 'ano ', 'saan ', 'kailan ', 'puwede ba ', 'pwede ba ',
    'may paraan ba', 'normal ba ',
];
foreach ($starters as $s) {
    if (str_starts_with($norm, $s)) {
        return true;
    }
}

return false;
}

/**
 * Score scenario keys from message content (higher = stronger match).
 *
 * @return array<string, int>
 */
function wellness_score_scenarios(string $norm): array
{
    $rules = [
        'exam_school' => [
            'exam', 'quiz', 'midterm', 'finals', 'thesis', 'assignment', 'grades', 'gpa',
            'class', 'prof', 'school', 'aral', 'pagsusulit', 'quiz', 'projekto', 'report',
            'deadline', 'pasahan',
        ],
        'sleep' => [
            'cant sleep', 'can\'t sleep', 'cannot sleep', 'insomnia', 'sleepless', 'no sleep',
            'hard to sleep', 'trouble sleeping', 'hindi makatulog', 'di makatulog', 'puyat',
            'gising', '3am', '4am', 'tulog', 'sleep ', 'sleeping',
        ],
        'loneliness' => [
            'lonely', 'loneliness', 'alone', 'no friends', 'walang kaibigan', 'mag-isa',
            'nag-iisa', 'walang kasama', 'isolated', 'nobody cares', 'wala akong kaibigan',
        ],
        'family' => [
            'parents', 'parent', 'mom', 'dad', 'family', 'magulang', 'ina', 'ama', 'nanay',
            'tatay', 'pamilya', 'pressure sa bahay', 'hindi ako maintindihan',
        ],
        'relationship' => [
            'breakup', 'broke up', 'ex ', 'boyfriend', 'girlfriend', 'partner', 'relationship',
            'iniwan ako', 'hiwalay', 'kasintahan', 'jowa', 'heartbreak', 'broken heart',
        ],
        'anxiety' => [
            'anxiety', 'anxious', 'panic', 'panicking', 'worried', 'worry', 'nervous',
            'nababahala', 'kabado', 'kinakabahan', 'overthink', 'takot na takot',
        ],
        'low_mood' => [
            'sad', 'depressed', 'depression', 'hopeless', 'empty', 'down', 'miserable',
            'lungkot', 'malungkot', 'nalulungkot', 'hindi masaya', 'walang gana', 'bigat',
        ],
        'stress_burnout' => [
            'stress', 'stressed', 'burnout', 'burned out', 'overwhelm', 'overloaded',
            'pagod na pagod', 'sobrang pagod', 'nakakastress', 'pressure', 'overwhelmed',
        ],
        'help_seeking' => [
            'counselor', 'therapist', 'psychologist', 'guidance', 'professional help',
            'konsultasyon', 'magpa-check', 'saan ako lalapit', 'saan puwedeng lumapit',
            'need help', 'help me', 'tulong po', 'tulong sa',
        ],
        'coping_how' => [
            'how do i cope', 'how can i cope', 'how to cope', 'what should i do',
            'paano ko haharapin', 'paano ko ito haharapin', 'ano ang gagawin ko',
            'tips', 'advice', 'suggestions',
        ],
        'why_feel' => [
            'why do i feel', 'why am i', 'bakit ako', 'bakit ganito', 'bakit parang',
            'normal ba na', 'is it normal',
        ],
        'social_fear' => [
            'social anxiety', 'presentation', 'public speaking', 'recit', 'recitation',
            'nahihiya', 'nahihiya ako', 'takot magsalita', 'group work',
        ],
    ];

    $scores = [];

```

```

foreach ($rules as $key => $patterns) {
    $s = 0;
    foreach ($patterns as $p) {
        if (mb_strpos($norm, $p, 0, 'UTF-8') !== false) {
            $s += mb_strlen($p, 'UTF-8') >= 8 ? 3 : 2;
        }
    }
    if ($s > 0) {
        $scores[$key] = $s;
    }
}

if (wellness_message_is_question($norm)) {
    if (preg_match('/\b(how|paano)\b/u', $norm)) {
        $scores['coping_how'] = ($scores['coping_how'] ?? 0) + 2;
    }
    $domainScore = 0;
    foreach ($scores as $k => $v) {
        if (!in_array($k, ['coping_how', 'why_feel', 'help_seeking'], true)) {
            $domainScore = max($domainScore, $v);
        }
    }
    if (preg_match('/\b(why|bakit)\b/u', $norm) && $domainScore < 3) {
        $scores['why_feel'] = ($scores['why_feel'] ?? 0) + 2;
    }
}

arsort($scores);

return $scores;
}

/**
 * @return list<string> topic tags for API metadata
 */
function wellness_topics_from_scenarios(array $scores): array
{
    $map = [
        'exam_school' => 'stress',
        'sleep' => 'stress',
        'loneliness' => 'loneliness',
        'family' => 'stress',
        'relationship' => 'low_mood',
        'anxiety' => 'anxiety',
        'low_mood' => 'low_mood',
        'stress_burnout' => 'stress',
        'help_seeking' => 'support',
        'coping_how' => 'support',
        'why_feel' => 'support',
        'social_fear' => 'anxiety',
    ];

    $topics = [];
    foreach (array_keys($scores) as $sc) {
        if (isset($map[$sc])) {
            $topics[] = $map[$sc];
        }
    }

    return array_values(array_unique($topics));
}

/**
 * @return array{intent: string, scenarios: list<string>}
 */
function wellness_classify(string $norm): array
{
    $scores = wellness_score_scenarios($norm);
    $scenarios = array_keys($scores);
    $primary = $scenarios[0] ?? 'general_support';
    $intent = $primary;
    if (wellness_message_is_question($norm)) {
        if (($scores['coping_how'] ?? 0) > 0 || ($scores['help_seeking'] ?? 0) > 0) {
            $intent = 'question_coping';
        } elseif (($scores['why_feel'] ?? 0) > 0) {
            $intent = 'question_explainer';
        } else {
            $intent = 'question_' . $primary;
        }
    }
}

```

```

    return ['intent' => $intent, 'scenarios' => $scenarios != [] ? $scenarios : ['general_support']];
}

function wellness_snippet_safe(string $message, int $max = 120): string
{
    $s = trim(preg_replace('/\s+/u', ' ', $message) ?? '');
    $s = preg_replace('/["\x00-\x1F]/u', ' ', $s) ?? '';
    if (mb_strlen($s, 'UTF-8') > $max) {
        return mb_substr($s, 0, $max - 1, 'UTF-8') . '...';
    }

    return $s;
}

/** @return array{how: bool, why: bool} */
function wellness_question_flavor(string $norm): array
{
    return [
        'how' => (bool) preg_match('/\b(how|paano|ano ang gagawin|what should i)\b/u', $norm),
        'why' => (bool) preg_match('/\b(why|bakit|normal ba)\b/u', $norm),
    ];
}

/**
 * Natural, conversational reply – no quote-back, no "Direct answer" headers.
 *
 * @param list<string> $scenarios
 * @param array<string, int> $scores
 */
function wellness_compose_natural_reply(string $primary, array $scenarios, array $scores, string $lang, string $norm): string
{
    $flavor = wellness_question_flavor($norm);
    $how = $flavor['how'];
    $why = $flavor['why'];

    // If generic "how to cope" but a domain scenario is stronger, answer the domain
    if ($primary === 'coping_how' && isset($scenarios[1])) {
        $second = $scenarios[1];
        if (($scores[$second] ?? 0) >= ($scores['coping_how'] ?? 0)) {
            $primary = $second;
        }
    }

    $templates = wellness_natural_templates($lang, $how, $why);
    $body = $templates[$primary] ?? $templates['general_support'];

    return trim($body);
}

/**
 * @return array<string, string>
 */
function wellness_natural_templates(string $lang, bool $how, bool $why): array
{
    if ($lang === 'tl') {
        return [
            'exam_school' => $how || $why
                ? "Nakakapagod talaga kapag exam ang nakabantay–parang lahat ng enerhiya mo napupunta sa takot na mabagsak, hindi lang sa pag-aaral.\n\n"
                . "Subukan ito ngayong gabi at bukas:\n"
                . "- **Isang bagay lang muna:** 25 minutong review, 5 minutong pahinga–huwag buong syllabus sa isang upuan.\n"
                . "- **Bago pumasok sa exam room:** box breathing (hinga 4, hold 4, labas 4, hold 4) ng isang minuto.\n"
                . "- **Sa loob ng exam:** \"good enough\" answers muna–may partial credit pa rin.\n\n"
                . "Kung halos araw-araw ganito at hindi ka na makatulog o kumain nang maayos ng **dalawang linggo**, mas okay ang campus guidance–hindi ako makakapag-diagnose, pero sila makakapagbigay ng plano para sa iyo."
                : "Mabigat ang pressure ng school, lalo na kapag sunod-sunod ang requirements–hindi ibig sabihin hindi ka kayang mag-aral, overload ka lang.\n\n"
                . "Ngayong linggo: piliin **isang subject** na uunahin, mag-message sa isang kaklase o TA kung saan ka stuck, at protektahan ang tulog bago ang huling gabing cram. Kung gusto mo ng mas specific na tips, sabihin kung exam ba, project, o grades ang pinaka-nakakatakot.",
            'sleep' => $why
                ? "Madalas hindi lang \"tamad mag-sleep\"–naka-on pa ang utak mo sa mga dapat gawin bukas, o may stress na hindi pa nalalabasan.\n\n"
                . "Pwede ring kulang ng routine, sobrang screen bago matulog, o caffeine late in the day. Subukan 3–5 gabi: **parehong oras ng gising**, isulat sa papel ang 3 priority bukas, at iwas screen 45–60 min bago matulog.\n\n"
                . "Kung tuloy-tuloy na ito at hirap ka nang function sa araw, nurse o guidance sa campus–hindi disorder agad, pero kailangan ng tao na makakasama mag-ayos."
                : "Pagod ka na pero gising pa ang isip–nakakaiyak at nakakapagod iyon.\n\n"

```

```

        . "1. **Gising** sa parehong oras araw-araw (kahit kulang tulog).\n"
        . "2. **Screen off** 45-60 min bago matulog; ilipat worries sa papel.\n"
        . "3. **Walang mabigat na caffeine** pag hapon na.\n\n"
        . "Kung 2 linggo nang ganito at sabog ang araw mo, guidance counselor-mas malaki ang tulong kaysa mag-isang laban.",

'loneliness' => "Masakit ang pakiramdam na wala kang kasama sa campus-maraming estudyante ang nag-iisip na sila lang ang ganito,
pero hindi totoo.\n\n"
        . "Hindi kailangan maging popular. Subukan:\n"
        . "- **Isang mensahe** sa kaklase tungkol sa assignment (mababang pressure).\n"
        . "- **Isang org o group** na pupuntahan mo ng isang beses lang muna.\n"
        . "- **Isang tao** na i-text mo linggo-linggo, kahit short lang.\n\n"
        . "Kung gusto mo, sabihin kung mas hirap ka sa bagong environment, breakup, o hindi ka iniimbiba-iba ang angkop na
hakbang.",

'family' => "Mabigat kapag ang pamilya ang source ng pressure-minsan mahal ka nila pero hindi nila alam kung paano suportahan
ang load mo ngayon.\n\n"
        . "Pwede mong sabihin nang kalmado: *\"Naririnig ko kayo; kailangan ko mag-focus sa [subject/week] muna.\"* Huminto kung
mainit ang usapan; bumalik kapag kalmado na.\n\n"
        . "Kung **hindi ligtas** ang bahay o may pang-aabuso, kailangan ng tao/propesyunal agad-iba iyon sa normal na stress. Campus
guidance makakatulong mag-plan.",

'relationship' => "Valid ang sakit sa relasyon o breakup-hindi \"OA\" ang nararamdaman mo; parang mini-grief iyan.\n\n"
        . "Ngayon: huwag pilitin ang \"okay na ako\" agad; kumain at matulog kahit basic lang; bawasan ang stalking sa social kung
masakit. May isang taong mapagkakatiwalaan na pwede mong kausapin.\n\n"
        . "Kung ilang araw ka nang hindi kumakain, natutulog, o hindi makapunta sa klase, guidance counselor-relationship pain
pwedeng magpatagal ng low mood.",

'anxiety' => $how
        ? "Kapag sobrang kaba, ang katawan mo ay naghahanda para sa panganib-kahit exam o recit lang ang nangyayari.\n\n"
        . "***Ngayon mismo:**\n"
        . "- 5-4-3-2-1 grounding (5 nakikita, 4 nahahawakan...)\n"
        . "- Box breathing: 4 in, 4 hold, 4 out, 4 hold\n"
        . "- I-schedule ang worry ng 15 min **mamaya**, hindi buong araw\n\n"
        . "Kung halos araw-araw panic at hirap huminga, sabihin sa counselor-may skills silang ituturo, hindi ako magbibigay ng
diagnosis."
        : "Nakakapagod ang laging naka-alert-hindi ka mahina, overloaded lang ang nervous system mo.\n\n"
        . "Subukan grounding o box breathing bago klase; bawasan caffeine; isang maliliit na task lang sa isang oras. Kung
tumatagal ng linggo at sabog ang routine mo, campus guidance.",

'low_mood' => $why
        ? "Maraming dahilan kung bakit mabigat ang araw-kulang tulog, stress sa school, kalungkutan, away, o sunod-sunod na
disappointments. **Hindi** ibig sabihin may \"sakit\" ka agad; ibig sabihin overloaded ka.\n\n"
        . "Ngayon: **isang maliit na gawain** (ligo, lakad, isang pagkain). Iwas malaking desisyon habang pinakamababa ang
mood.\n\n"
        . "Kung 2 linggo+ na halos araw-araw ganito, propesyunal-common ang paghingi ng tulong at may mga option sa campus."
        : "Malungkot at walang gana-tunay iyan, at hindi ka nag-iisa dito.\n\n"
        . "Ngayong araw: isang maliit na win lang (pagkain, 10-min walk, isang message sa taong safe). Bukas: ulitin. Kung hindi
gumagalaw ang mood ng ilang linggo, guidance counselor-hindi ako magdi-diagnose, pero sila makakasama.",

'stress_burnout' => "Pagod na pagod ka-burnout ang tawag ng marami, pero ibig sabihin lang nun sobra kang nagbigay nang kulang
ang pahinga.\n\n"
        . "Ngayong linggo:\n"
        . "- **Alisin o ipagpaliban** ang isang commitment na hindi critical.\n"
        . "- **Protektahan** ang isang oras ng rest tulad ng klase.\n"
        . "- **Huwag** magdagdag ng bagong responsibility hanggang may konting gana ka na.\n\n"
        . "Recovery ay productive, hindi tamad.",

'help_seeking' => "Magandang hakbang na naghahanap ka kung saan lalapit.\n\n"
        . "***Sa campus:** guidance / counseling office o student affairs-sila ang unang linya para sa talk therapy referrals at
student support.\n\n"
        . "***Sa Pilipinas (urgent emotional crisis):** NCMH **1553**, o mobile lines sa crisis response ng app na ito.\n\n"
        . "Ako ay chat companion lang-hindi ako makakapag-intervene sa emergency.",

'coping_how' => "Narito ang simpleng plan na gumagana sa maraming estudyante:\n\n"
        . "1. **Isulat** ang problema sa isang pangungusap.\n"
        . "2. **Gawin** ang isang aksyon na kaya sa 30 minuto.\n"
        . "3. **Sabihin** sa isang tao (kahit chat lang).\n"
        . "4. **Gabi:** ano ang 5% na nakatulong? Ulitin bukas.\n\n"
        . "Sabihin kung school, tulog, tao, o mood ang pinaka-mabigat para mas iangkop ko ang susunod na sagot.",

'why_feel' => "Valid ang tanong na *bakit* ganito-hindi ka nag-iisa sa pagtataka.\n\n"
        . "Karaniwang contributors: **stress**, **kulang tulog**, **exam pressure**, **kalungkutan**, o **away** sa
pamilya/relasyon. Hindi ibig sabihin may disorder ka-pattern lang ng buhay mo ngayon.\n\n"
        . "Kung matindi, araw-araw, at **2 linggo+**, mas tama ang counselor para mag-explore ng dahilan kaysa umasa sa chatbot.",

'social_fear' => "Takot sa recit o group work-super common, lalo na kung feeling mo huhusgahan ka.\n\n"
        . "- Maghanda ng **isang pangungusap** na sasabihin mo; basahin sa bahay nang malakas isang beses.\n"
        . "- Umupo malapit sa friendly na kaklase.\n"
        . "- Gantimpalaan ang sarili pagkatapos ng klase, kahit maliit.\n\n"
        . "Kung racing heart at feeling mo mamatay ka sa panic, counselor-may exposure techniques sila.",

```

```

'general_support' => "Salamat sa pagbabahagi-naririnig kita.\n\n"
. "Para mas makatulong ako, sabihin kung ano ang pinaka-mabigat ngayon: **exam/school**, **tulog**, **kalungkutan**,
**relasyon/pamilya**, o **kabog/panic**. Isang paragraph lang, sasagutin ko nang specific-hindi therapy, pero may praktikal na hakbang.",
];
}

return [
'exam_school' => $how || $why
? "Exam stress can make your brain treat a test like real danger—that is why your heart races even when you \"know\" the
material.\n\n"
. "Try this tonight and tomorrow:\n"
. "- **One block only:** 25 minutes review, 5 minutes off—not the whole course tonight.\n"
. "- **Before you walk in:** one minute box breathing (in 4, hold 4, out 4, hold 4).\n"
. "- **During the exam:** aim for *good enough* answers first—partial credit still counts.\n\n"
. "If this is daily for **two+ weeks** and you cannot sleep or eat normally, your campus guidance office can help with a
real plan—I cannot diagnose you, but they can support you."
: "School pressure stacking up is exhausting—it usually means overload, not that you are incapable.\n\n"
. "This week: pick **one subject** to prioritize, message one classmate or TA where you are stuck, and protect sleep before
a cram night. Tell me if exams, projects, or grades are the main fear and I can narrow tips further.",

'sleep' => $why
? "When you are tired but wired, it is often a stressed nervous system—not \"bad sleep habits\" alone.\n\n"
. "Common contributors: unfinished tasks looping in your head, late screens, caffeine, or anxiety. For 3–5 nights try:
**same wake time**, **paper list** of tomorrow's top 3, **no screens** 45–60 minutes before bed.\n\n"
. "If this runs for weeks and days feel broken, see campus nurse or guidance—not a label from me, just human support."
: "Being exhausted but unable to switch off is brutal.\n\n"
. "1. **Wake** at the same time daily (even after short sleep).\n"
. "2. **Off screens** 45–60 min before bed; write worries on paper.\n"
. "3. **No heavy caffeine** in the afternoon.\n\n"
. "If 2+ weeks of this wreck your days, a counselor can help more than struggling alone.",

'loneliness' => "Feeling alone on campus hurts—and many students hide the same feeling.\n\n"
. "You do not need a big friend group. Try:\n"
. "- **One message** to a classmate about coursework (low pressure).\n"
. "- **One club or meeting** you attend just once.\n"
. "- **One person** you check in with weekly, even briefly.\n\n"
. "If you tell me whether it is new environment, breakup, or never being invited, I can suggest a tighter next step.",

'family' => "Family pressure often mixes love with fear of letting people down—that is heavy to carry while studying.\n\n"
. "You can say calmly: *\"I hear you; I need to focus on [X] this week.\"* Pause heated talks; return when cooler.\n\n"
. "If home is **unsafe** or abusive, you need a person/professional—not tips from a chatbot. Campus guidance can help you plan
next steps.",

'relationship' => "Relationship pain and breakups are real grief—not overreacting.\n\n"
. "For now: do not force *\"I'm fine\"*; eat and sleep at a basic level; reduce painful social media triggers. Talk to one trusted
person if you can.\n\n"
. "If you cannot eat, sleep, or attend class for many days, see a counselor—heartbreak can deepen low mood.",

'anxiety' => $how
? "When anxiety spikes, your body thinks there is danger—even if it is only a quiz or recit.\n\n"
. "**Right now:**\n"
. "- 5-4-3-2-1 grounding\n"
. "- Box breathing: 4 in, 4 hold, 4 out, 4 hold\n"
. "- Schedule a 15-minute **worry time later**, not all day\n\n"
. "If panic is frequent and hard to breathe through, tell a counselor—they teach skills; I do not diagnose."
: "Living on high alert is tiring—you are not weak, you are overloaded.\n\n"
. "Try grounding or box breathing before class, cut afternoon caffeine, and do one small task per hour. If this runs for
weeks, campus guidance helps.",

'low_mood' => $why
? "Low mood often comes from stacked stress—sleep debt, exams, loneliness, conflict—not necessarily a disorder.\n\n"
. "Today: **one small action** (shower, short walk, one meal). Avoid big decisions at the bottom.\n\n"
. "If most days feel like this for **2+ weeks**, a professional check-in is wise and common on campus."
: "Heavy, sad days are real—and you are not alone in this.\n\n"
. "Today: one small win (food, 10-minute walk, one safe message). Repeat tomorrow. If nothing shifts for weeks, guidance
counselor—I will not diagnose, but they can walk with you.",

'stress_burnout' => "You sound burned out—that usually means too much output, too little recovery, not laziness.\n\n"
. "This week:\n"
. "- **Drop or delay** one non-essential commitment.\n"
. "- **Protect** one rest hour like a class.\n"
. "- **Pause** new responsibilities until you have a little energy back.\n\n"
. "Rest is part of performance, not the opposite.",

'help_seeking' => "Reaching out is a strong move.\n\n"
. "**On campus:** start with guidance/counseling or student affairs—they know local referrals and student support.\n\n"
. "**Philippines (urgent emotional crisis):** NCMH **1553** and numbers in this app's crisis response.\n\n"
. "I am a chat companion only—I cannot handle emergencies.",

```



```

'coping_how' => "Here is a simple plan many students use:\n\n"
    . "1. Write the problem in one sentence.\n"
    . "2. Do one action you can finish in 30 minutes.\n"
    . "3. Tell one person (even a short chat).\n"
    . "4. Tonight what helped 5%? Repeat tomorrow.\n\n"
    . "Share whether school, sleep, people, or mood is hardest and I will tailor the next reply.",

'why_feel' => "Asking why you feel this way is completely fair.\n\n"
    . "Common contributors: stress, poor sleep, exam pressure, loneliness, or conflict at home or in a
relationship. That does not automatically mean a disorder—it can be your situation right now.\n\n"
    . "If it is intense, daily, and lasts 2+ weeks, a counselor can explore causes properly—better than relying on a chatbot.",

'social_fear' => "Fear of recitation or being judged in class is very common.\n\n"
    . "- Prepare one sentence you will say; say it aloud once at home.\n"
    . "- Sit near someone friendly.\n"
    . "- Reward yourself after class, even small.\n\n"
    . "If panic feels physical and scary (racing heart, faint), mention it to a counselor.",

'general_support' => "Thank you for opening up—I am listening.\n\n"
    . "To answer you better: what hurts most right now—school/exams, sleep, loneliness, family/relationship, or
anxiety/panic? A few sentences is enough and I will respond with specific steps (support only, not therapy).",
];

function wellness_ph_crises_resources_public(): array
{
    return [
        'cards' => [
            [
                'type' => 'ncmh',
                'title' => 'National Center for Mental Health (NCMH) Crisis Hotline',
                'lines' => [
                    'Toll-free (landline nationwide): 1553',
                    'Globe/TM mobile: 0917-899-8727 (USAP)',
                    'Mobile / Smart: 0966-351-4518, 0919-057-1553',
                ],
            ],
            [
                'type' => 'hopeline_touch',
                'title' => 'HOPELINE (In Touch Crisis Line)',
                'lines' => [
                    '0917-558-HOPE (4673)',
                    '2919 (Globe/TM toll-free shortcut, when available)',
                ],
            ],
            [
                'type' => 'emergency',
                'title' => 'Immediate danger – Philippines',
                'lines' => [
                    'If harm is imminent, call emergency services (911) or go to your nearest ER/hospital.',
                ],
            ],
        ],
    ];
}

/** @param 'en'|'tl' $lang */
function wellness_disclaimer_banner(string $lang): string
{
    if ($lang === 'tl') {
        return 'Paalala: Hindi ako therapist, hindi nagdi-diagnose, at hindi pamalit sa emergency. '
            . 'Sa krisis, gamitin ang hotline sa Pilipinas (makikita sa crisis response).';
    }

    return 'Reminder: I am not a therapist, I do not diagnose, and I am not for emergencies. '
        . 'In crisis, use Philippines hotlines (included in crisis responses).';
}

/** @param 'en'|'tl' $lang */
function wellness_footer_disclaimer(string $lang): string
{
    return $lang === 'tl'
        ? 'Ito ay pang-edukasyon at suporta lamang—hindi medikal na payo.'
        : 'Educational support only—not medical advice.';
}

/** @param 'en'|'tl' $lang */
function wellness_crisis_response(string $lang): array
{
    $res = wellness_ph_crises_resources_public();
}

```

```

$linesBullet = '';
foreach ($res['cards'] as $card) {
    $linesBullet .= "### {$card['title']}\n";
    foreach ($card['lines'] as $ln) {
        $linesBullet .= '- ' . $ln . "\n";
    }
    $linesBullet .= "\n";
}

if ($lang === 'tl') {
    $reply = "**Salamat at sinabi mo ito.** Kung pinag-iisipan mong saktan ang sarili o tapusin ang buhay, kailangan mo ng **totoong tao** ngayon.\n\n"
        . "- **Agad na panganib:** tumawag sa **911** o ER/hospital.\n"
        . "- **Pilipinas – crisis lines:**\n\n"
        . $linesBullet
        . 'Huwag mag-isang maghintay kung hindi ligtas–manatili sa linya o kasama ng taong pinagkakatiwalaan mo.';
} else {
    $reply = "**Thank you for telling someone—even here.** If you might harm yourself or end your life, please reach a **real person** now.\n\n"
        . "- **Immediate danger:** call **911** or go to the nearest ER.\n"
        . "- **Philippines crisis lines:**\n\n"
        . $linesBullet
        . 'Do not wait alone if you feel unsafe—stay on the line or with someone you trust.';
}

return ['reply' => trim($reply), 'resources' => $res];
}

/**
 * @return array{
 *     reply: string,
 *     reply_body: string,
 *     crisis: bool,
 *     topics: list<string>,
 *     intent: string,
 *     scenarios: list<string>,
 *     is_question: bool,
 *     disclaimer_banner: string,
 *     footer_disclaimer: string,
 *     language: string,
 *     reply_source: string,
 *     resources?: array
 * }
 */
/**
 * @param list<array{role: string, content: string}> $history
 */
function wellness_chat_orchestrate(string $message, ?string $langHint, array $history = []): array
{
    $norm = wellness_normalize_for_match($message);
    $language = wellness_resolve_language($langHint, $norm);
    /** @var 'en'|'tl' $language */

    $banner = wellness_disclaimer_banner($language);
    $footer = wellness_footer_disclaimer($language);
    $isQuestion = wellness_message_is_question($norm);

    if ($norm === '') {
        $body = $language === 'tl'
            ? 'Magtanong o magkwentong konti-hal. *Paano ko haharapin ang exam anxiety?* o *Sobrang pagod na ako sa school.* Sasagutin ko ayon sa tanong mo (Tagalog/English).'
            : 'Ask or share a little—e.g. *How do I handle exam anxiety?* or *I am burned out from school.* I will answer based on your question (English or Tagalog).';

        return [
            'reply' => $body,
            'reply_body' => $body,
            'crisis' => false,
            'topics' => [],
            'intent' => 'prompt',
            'scenarios' => [],
            'is_question' => false,
            'disclaimer_banner' => $banner,
            'footer_disclaimer' => $footer,
            'language' => $language,
            'reply_source' => 'builtin',
        ];
    }

    if (wellness_is_crisis_message($norm)) {

```

```

$cr = wellness_crisis_response($language);

return [
    'reply' => $cr['reply'],
    'reply_body' => $cr['reply'],
    'crisis' => true,
    'topics' => [],
    'intent' => 'crisis',
    'scenarios' => ['crisis'],
    'is_question' => $isQuestion,
    'disclaimer_banner' => $banner,
    'footer_disclaimer' => $footer,
    'language' => $language,
    'resources' => $cr['resources'],
    'reply_source' => 'crisis',
];
}

$class = wellness_classify($norm);
$scores = wellness_score_scenarios($norm);
$primary = $class['scenarios'][0] ?? 'general_support';
$topics = wellness_topics_from_scenarios($scores);
$messageTrim = trim($message);

$replySource = 'smart';
$replyBody = null;

require_once dirname(__DIR__) . '/includes/wellness_ai.php';
if (wellness_ai_is_enabled()) {
    $aiReply = wellness_ai_chat($messageTrim, $language, $class['scenarios'], $history);
    if ($aiReply !== null && $aiReply !== '') {
        $replyBody = $aiReply;
        $provider = wellness_ai_provider();
        if ($provider === 'groq') {
            $replySource = 'groq';
        } elseif ($provider === 'ollama' || ($provider === 'auto' && wellness_ollama_reachable() && !wellness_ai_has_cloud_key())) {
            $replySource = 'ollama';
        } elseif ($provider === 'gemini') {
            $replySource = 'gemini';
        } else {
            $replySource = 'ai';
        }
    }
}

if ($replyBody === null) {
    require_once dirname(__DIR__) . '/includes/wellness_engine.php';
    $replyBody = wellness_engine_reply($messageTrim, $language, $primary, $class['scenarios'], $norm);
    $replySource = 'smart';
}

return [
    'reply' => $replyBody,
    'reply_body' => $replyBody,
    'crisis' => false,
    'topics' => $topics,
    'intent' => $class['intent'],
    'scenarios' => $class['scenarios'],
    'is_question' => $isQuestion,
    'disclaimer_banner' => $banner,
    'footer_disclaimer' => $footer,
    'language' => $language,
    'reply_source' => $replySource,
];
}

```

FILE: includes/wellness_engine.php

```

<?php
declare(strict_types=1);

/**
 * Dynamic wellness replies without external AI – reads the student's message and answers directly.
 */

/**
 * @return array{
 *     emotions: list<string>,

```

```

*   topics: list<string>,
*   time_hint: ?string,
*   question: ?string,
*   intensity: string
* }
*/
function wellness_parse_student_message(string $message, string $norm): array
{
    $emotions = [];
    $emotionMap = [
        'anxious' => ['anxious', 'anxiety', 'nababalisa', 'kabado', 'kinakabahan', 'panic', 'takot', 'nerbiyos'],
        'sad' => ['sad', 'depressed', 'malungkot', 'lungkot', 'iyak', 'umiiyak', 'empty', 'walang gana'],
        'tired' => ['tired', 'exhausted', 'pagod', 'burnout', 'drained', 'walang lakas'],
        'lonely' => ['lonely', 'alone', 'mag-isa', 'nag-iisa', 'walang kaibigan', 'isolated'],
        'angry' => ['angry', 'galit', 'frustrated', 'inis', 'naiirita'],
        'overwhelmed' => ['overwhelm', 'sobra', 'daming', 'lahat', 'hindi ko na kaya', 'cant handle'],
    ];
    foreach ($emotionMap as $key => $words) {
        foreach ($words as $w) {
            if (mb_strpos($norm, $w, 0, 'UTF-8') !== false) {
                $emotions[] = $key;
                break;
            }
        }
    }

    $topics = [];
    $topicMap = [
        'exam' => ['exam', 'quiz', 'test', 'midterm', 'final', 'pagsusulit'],
        'school' => ['school', 'class', 'prof', 'assignment', 'project', 'grades', 'aral', 'pasahan'],
        'sleep' => ['sleep', 'tulog', 'insomnia', 'makakatulog', 'puyat', 'gising'],
        'family' => ['family', 'parents', 'mom', 'dad', 'nanay', 'tatay', 'magulang'],
        'relationship' => ['breakup', 'boyfriend', 'girlfriend', 'jowa', 'ex ', 'relasyon'],
        'friends' => ['friend', 'kaibigan', 'classmate'],
    ];
    foreach ($topicMap as $key => $words) {
        foreach ($words as $w) {
            if (mb_strpos($norm, $w, 0, 'UTF-8') !== false) {
                $topics[] = $key;
                break;
            }
        }
    }

    $timeHint = null;
    foreach (['bukas', 'tomorrow', 'tonight', 'ngayong gabi', 'later today', 'this week', 'ngayon'] as $t) {
        if (mb_strpos($norm, $t, 0, 'UTF-8') !== false) {
            $timeHint = $t;
            break;
        }
    }

    $question = null;
    if (preg_match('/^\s*(how|paano|what|ano|why|bakit|can i|pwede|should i)/u', $norm)) {
        $question = preg_match('/\b(why|bakit)\b/u', $norm) ? 'why' : (preg_match('/\b(how|paano)\b/u', $norm) ? 'how' : 'what');
    } elseif (str_contains($norm, '?')) {
        $question = preg_match('/\b(why|bakit)\b/u', $norm) ? 'why' : (preg_match('/\b(how|paano)\b/u', $norm) ? 'how' : 'general');
    }

    $intensity = 'moderate';
    foreach (['sobra', 'very', 'really', 'so ', 'labis', 'hindi ko na', 'cant', "can't"] as $i) {
        if (mb_strpos($norm, $i, 0, 'UTF-8') !== false) {
            $intensity = 'high';
            break;
        }
    }

    return [
        'emotions' => array_values(array_unique($emotions)),
        'topics' => array_values(array_unique($topics)),
        'time_hint' => $timeHint,
        'question' => $question,
        'intensity' => $intensity,
    ];
}

/** @param array{emotions: list<string>, topics: list<string>, time_hint: ?string, question: ?string, intensity: string} $ctx */
function wellness_empathy_opening(array $ctx, string $lang, string $primary): string
{
    $e = $ctx['emotions'][0] ?? null;

```

```

    $t = $ctx['topics'][0] ?? null;
    $time = $ctx['time_hint'];

    if ($lang === 'tl') {
        if ($t === 'exam' && $time !== null && (str_contains((string) $time, 'bukas') || str_contains((string) $time, 'tomorrow'))) {
            return 'Naiintindihan ko-bukas ang exam at mabigat na ang pakiramdam mo ngayon. Normal ang kaba, pero pwedeng gawing mas kaya ang gabi at umaga mo.';
        }
        if ($t === 'sleep') {
            return 'Nakakapagod kapag gusto mong magpahinga pero hindi sumusuko ang utak mo-valid ang pagod mo.';
        }
        if ($t === 'friends' || $e === 'lonely') {
            return 'Masakit ang pakiramdam na wala kang kasama o hindi ka makaconnect-maraming estudyante ang dumadaan dito sa campus.';
        }
        if ($e === 'anxious') {
            return 'Kapag sobrang kaba, parang laging may mangyayaring masama-hindi ka mahina, overloaded lang ang katawan mo.';
        }
        if ($e === 'sad') {
            return 'Malungkot at mabigat ang araw mo-naririnig kita at hindi mo kailangang magdala nito nang mag-isa.';
        }
        if ($e === 'tired') {
            return 'Pagod na pagod ka-sign ito na kailangan mo ng pahinga, hindi ng mas maraming pressure sa sarili.';
        }

        return 'Salamat sa pagbabahagi-naririnig kita at valid ang nararamdaman mo.';
    }

    if ($t === 'exam' && $time !== null && (str_contains((string) $time, 'tomorrow') || str_contains((string) $time, 'bukas'))) {
        return 'I hear you-tomorrow\'s exam is weighing on you tonight. That kind of dread is common, and you can still make the next hours more manageable.';
    }
    if ($t === 'sleep') {
        return 'Wanting rest while your mind stays awake is exhausting-and it makes sense you feel worn down.';
    }
    if ($t === 'friends' || $e === 'lonely') {
        return 'Feeling disconnected or without people who get you really hurts-many students feel this on campus, even when it looks like everyone else is fine.';
    }
    if ($e === 'anxious') {
        return 'When anxiety spikes, it can feel like something bad is about to happen-even if you logically know you\'re "fine." That\'s your nervous system on high alert, not weakness.';
    }
    if ($e === 'sad') {
        return 'Heavy, sad days are real-and you don\'t have to pretend they\'re not.';
    }
    if ($e === 'tired') {
        return 'You sound deeply tired-that usually means you need recovery, not more self-criticism.';
    }

    return 'Thank you for trusting me with this-I\'m listening, and what you feel matters.';
}

/**
 * @param array{emotions: list<string>, topics: list<string>, time_hint: ?string, question: ?string, intensity: string} $ctx
 */
function wellness_core_steps(array $ctx, string $lang, string $primary): string
{
    $q = $ctx['question'];
    $topics = $ctx['topics'];
    $top = $topics[0] ?? $primary;

    if ($lang === 'tl') {
        if ($top === 'exam' || $primary === 'exam_school') {
            if ($q === 'how') {
                return "***Ngayong gabi:**\n"
                    . "1. Piliin **isang topic** lang na ire-review (hindi buong course).\n"
                    . "2. **25 min** study → **5 min** walk/tubig.\n"
                    . "3. Isulat sa papel: *\"Bukas, first thing I do is...\"* (isang konkretong hakbang).\n\n"
                    . "***Bago exam:** 1 minutong box breathing; dalhin tubig; target **good enough**, hindi perpekto.\n\n"
                    . "Kung halos araw-araw ganito at hindi ka makatulong ng 2+ linggo, campus guidance makakatulong.";
            }

            return "***Maaaring tumulong:**\n"
                . "• Hatiin ang aral sa maliliit na bloke-huwag buong syllabus sa isang upuan.\n"
                . "• Magtanong sa kaklase/TA kung saan stuck ka.\n"
                . "• Protektahan ang tulog bago huling gabing cram.";
        }

        if ($top === 'sleep' || $primary === 'sleep') {
            return "***Subukan ngayon:**\n"
                . "1. Isulat sa papel ang **3 priority bukas** – para hindi umiikot sa utak sa kama.\n"

```

```

        . "2. **Screen off** 45-60 min bago matulog.\n"
        . "3. **Parehong oras ng gising** kahit kulang tulog (3-5 araw).\n\n"
        . "Kung 2+ linggo nang sabog ang araw mo, nurse o guidance-hindi ito diagnosis, pero kailangan ng tao.";
    }
    if (($ctx['emotions'][0] ?? '') === 'lonely' || $top === 'friends' || $primary === 'loneliness') {
        return ***Maliit na hakbang ngayong linggo:**\n
        . "1. **Isang mensahe** sa kaklase tungkol sa subject (mababang pressure).\n"
        . "2. **Isang org/meeting** - dumalo lang ng isang beses.\n"
        . "3. **Isang tao** na i-check-in mo kahit once a week.\n\n"
        . "Hindi kailangan maging close sa lahat-isa lang na consistent ay malaki na.";
    }
    if ($primary === 'anxiety' || ($ctx['emotions'][0] ?? '') === 'anxious') {
        return ***Para sa kaba ngayon:**\n
        . "1. **Box breathing** - 4 in, 4 hold, 4 out, 4 hold (3 ulit).\n"
        . "2. **Grounding** - 5 bagay na nakikita, 4 nahahawakan.\n"
        . "3. **Worry window** - 15 min mamaya ang pag-iisip, hindi ngayon.\n\n"
        . "Bawasan caffeine bago klase. Kung halos araw-araw panic, guidance counselor.";
    }
    if ($primary === 'family') {
        return ***Sa pressure ng pamilya:**\n
        . "1. Sabihin ang limit mo nang kalmado: *\Naririnig kita; kailangan ko mag-focus sa [X] ngayong linggo.*\n"
        . "2. Huminto kung mainit ang usapan; bumalik kapag kalmado.\n"
        . "3. Guidance kung paulit-ulit at hindi ka na makapag-aral.\n\n"
        . "Kung **hindi ligtas** ang bahay, kailangan ng tao/propesyunal-hindi chatbot.";
    }
    if ($primary === 'relationship') {
        return ***Sa sakit ng relasyon/breakup:**\n
        . "1. Payagan ang sarili na magluksa-walang deadline sa *\move on.*\n"
        . "2. Bawasan ang social media kung trigger.\n"
        . "3. Isang taong mapagkakatiwalaan ang kausapin.\n\n"
        . "Kung hindi kumakain/matulog ng maraming araw, guidance counselor.";
    }
    if ($primary === 'stress_burnout' || ($ctx['emotions'][0] ?? '') === 'tired') {
        return ***Sa burnout/pagod:**\n
        . "1. **Alisin o ipagpaliban** ang isang hindi critical na commitment ngayong linggo.\n"
        . "2. **Isang oras** ng pahinga-tulad ng importance ng klase.\n"
        . "3. **Huwag** magdagdag ng bagong task hanggang may konting gana.\n\n"
        . "Ang pahinga ay parte ng productivity, hindi katamaran.";
    }
    if ($q === 'why') {
        return "Madalas ang bigat ng mood dahil sa **stack**: kulang tulog, exam pressure, kalungkutan, o away-**hindi** agad ibig sabihin may *\sakit\ ka.\n\n"
        . "Kung **araw-araw** at **2+ linggo** na, mas tama ang counselor para mag-explore-ako pang-edukasyon lang.";
    }
    }

    return ***Ngayon:**\n
    . "1. Pangalanan ang pinaka-mabigat (aral / tulog / tao / mood).\n"
    . "2. Gawin **isang** aksyon na kaya sa 30 min.\n"
    . "3. Sabihin sa **isang tao** kung safe.\n\n"
    . "Sabihin kung ano ang pinaka-mabigat para mas i-target ang susunod na sagot.";
}

// English
if ($top === 'exam' || $primary === 'exam_school') {
    if ($q === 'how') {
        return ***Tonight:**\n
        . "1. Pick **one topic** to review-not the whole course.\n"
        . "2. **25 min** study → **5 min** break (water/walk).\n"
        . "3. On paper, write: *\Tomorrow, the first thing I do is...*\ (one concrete step).\n\n"
        . ***Before the exam:** 1 minute box breathing; bring water; aim for **good enough**, not perfect.\n\n"
        . "If this is daily for **2+ weeks** and sleep is wrecked, campus guidance can help with a real plan.";
    }

    return ***What often helps:**\n
    . "• Break study into small blocks-avoid cramming the entire syllabus at once.\n"
    . "• Ask a classmate or TA where you're stuck.\n"
    . "• Protect sleep the night before-recall beats last-minute panic.";
}
if ($top === 'sleep' || $primary === 'sleep') {
    return ***Try tonight:**\n
    . "1. Write tomorrow's **top 3 tasks** on paper so your brain can stop looping.\n"
    . "2. **No screens** 45-60 minutes before bed.\n"
    . "3. Same **wake time** daily for 3-5 days, even after short sleep.\n\n"
    . "If this lasts **2+ weeks** and days feel broken, see campus nurse or guidance-not a diagnosis from me, just human support.";
}
if (($ctx['emotions'][0] ?? '') === 'lonely' || $top === 'friends' || $primary === 'loneliness') {
    return ***Small steps this week:**\n
    . "1. **One message** to a classmate about coursework (low pressure).\n"
    . "2. **One club or meeting**-show up once.\n"
    . "3. **One person** to check in with weekly.\n\n"

```

```

        . "You don't need a big friend group—one steady connection can shift how alone you feel.";
    }
    if ($primary === 'anxiety' || ($ctx['emotions'][0] ?? '') === 'anxious') {
        return "**For anxiety right now:**\n"
            . "1. **Box breathing** – in 4, hold 4, out 4, hold 4 (repeat 3 times).\n"
            . "2. **Grounding** – name 5 things you see, 4 you can touch.\n"
            . "3. **Worry window** – schedule 15 minutes later for rumination, not all day.\n\n"
            . "Cut afternoon caffeine before class. If panic is frequent, see guidance counselor.";
    }
    if ($primary === 'family') {
        return "**With family pressure:**\n"
            . "1. State a calm limit: *\"I hear you; I need to focus on [X] this week.\"*\n"
            . "2. Pause heated talks; return when cooler.\n"
            . "3. Use guidance if it keeps blocking school.\n\n"
            . "If home is **unsafe**, you need a person/professional—not chat tips.";
    }
    if ($primary === 'relationship') {
        return "**With relationship pain or breakup:**\n"
            . "1. Let yourself grieve—no forced timeline to \"be over it.\"\n"
            . "2. Reduce social media if it's a trigger.\n"
            . "3. Talk to one trusted person.\n\n"
            . "If you can't eat, sleep, or attend class for many days, see a counselor.";
    }
    if ($primary === 'stress_burnout' || ($ctx['emotions'][0] ?? '') === 'tired') {
        return "**For burnout and exhaustion:**\n"
            . "1. **Drop or delay** one non-critical commitment this week.\n"
            . "2. **Protect one hour** of rest like a class.\n"
            . "3. **Don't** add new tasks until you regain a little energy.\n\n"
            . "Rest is part of doing well, not the opposite.";
    }
    if ($q === 'why') {
        return "Low mood or worry often comes from a **stack** of stress, poor sleep, exams, loneliness, or conflict—that doesn't automatically mean a disorder.\n\n"
            . "If it's **most days for 2+ weeks**, a counselor can explore causes properly; I'm only here for general support.";
    }

    return "**Right now:**\n"
        . "1. Name what's heaviest (school / sleep / people / mood).\n"
        . "2. Do **one** action you can finish in 30 minutes.\n"
        . "3. Tell **one safe person** if you can.\n\n"
        . "Share which area hurts most if you want a more targeted reply next.";
}

/** @param array{emotions: list<string>, topics: list<string>, time_hint: ?string, question: ?string, intensity: string} $ctx */
function wellness_soft_close(array $ctx, string $lang): string
{
    if ($lang === 'tl') {
        return 'Hindi ako therapist o emergency line—kung lumala o tumagal ng linggo, campus guidance ang tamang susunod na hakbang.';
    }

    return 'I\'m not a therapist or emergency line—if this stays intense for weeks, your campus guidance office is the right next step.';
}

/**
 * Best-effort local reply tailored to the student's words.
 */
function wellness_engine_reply(string $message, string $lang, string $primary, array $scenarios, string $norm): string
{
    $ctx = wellness_parse_student_message($message, $norm);

    if ($primary === 'coping_how' && isset($scenarios[1])) {
        $primary = $scenarios[1];
    }

    $parts = [
        wellness_empathy_opening($ctx, $lang, $primary),
        wellness_core_steps($ctx, $lang, $primary),
        wellness_soft_close($ctx, $lang),
    ];

    return trim(implode("\n\n", $parts));
}

```

FILE: index.php

<?php

```

declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';

if (!empty($_SESSION['user_id'])) {
    header('Location: dashboard.php');
} else {
    header('Location: login.php');
}
exit;

-----
FILE: install.php
-----
<?php
/**
 * One-time database installer. Open in browser after configuring config/config.php
 */
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';

$message = '';
$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST' || isset($_GET['run'])) {
    try {
        $pdo = new PDO(
            sprintf('mysql:host=%s;port=%d;charset=%s', DB_HOST, DB_PORT, DB_CHARSET),
            DB_USER,
            DB_PASS,
            [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
        );
        $pdo->exec('CREATE DATABASE IF NOT EXISTS `'. str_replace('`', '``', DB_NAME) . '` CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci');

        $pdo = new PDO(
            sprintf('mysql:host=%s;port=%d;dbname=%s;charset=%s', DB_HOST, DB_PORT, DB_NAME, DB_CHARSET),
            DB_USER,
            DB_PASS,
            [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
        );

        $sql = file_get_contents(__DIR__ . '/install/schema.sql');
        if ($sql === false) {
            throw new RuntimeException('Could not read install/schema.sql');
        }
        $sql = preg_replace('/--.*$/m', '', $sql);
        $parts = array_filter(array_map('trim', explode(';', $sql)));
        foreach ($parts as $stmt) {
            if ($stmt === '') {
                continue;
            }
            $pdo->exec($stmt);
        }

        $check = (int) $pdo->query("SELECT COUNT(*) FROM users WHERE role = 'admin'")->fetchColumn();
        if ($check === 0) {
            $hash = password_hash(DEFAULT_ADMIN_PASSWORD, PASSWORD_DEFAULT);
            $st = $pdo->prepare(
                'INSERT INTO users (username, password, full_name, role) VALUES (?, ?, ?, ?)'
            );
            $st->execute([DEFAULT_ADMIN_USERNAME, $hash, DEFAULT_ADMIN_FULL_NAME, 'admin']);
        }
        $checkGened = (int) $pdo->query("SELECT COUNT(*) FROM users WHERE role = 'gened'")->fetchColumn();
        if ($checkGened === 0) {
            $hash = password_hash(DEFAULT_GENED_PASSWORD, PASSWORD_DEFAULT);
            $st = $pdo->prepare(
                'INSERT INTO users (username, password, full_name, role) VALUES (?, ?, ?, ?)'
            );
            $st->execute([DEFAULT_GENED_USERNAME, $hash, DEFAULT_GENED_FULL_NAME, 'gened']);
        }

        $message = 'Database installed successfully. Default login: ' . htmlspecialchars(DEFAULT_ADMIN_USERNAME)
            . ' / ' . htmlspecialchars(DEFAULT_ADMIN_PASSWORD)
            . ' and ' . htmlspecialchars(DEFAULT_GENED_USERNAME) . ' / ' . htmlspecialchars(DEFAULT_GENED_PASSWORD)
            . ' - change passwords after first login.';
    } catch (Throwable $e) {
        $error = $e->getMessage();
    }
}

```



```

    }
} else {
    $message = 'Click the button below to create tables and the default admin user.';
}

?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Install – WPU SABLAE Portal</title>
    <?php require __DIR__ . '/includes/favicon.php'; ?>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
<div class="container py-5" style="max-width:640px">
    <h1 class="h3 mb-4">WPU SABLAE Portal – Install</h1>
    <?php if ($message && !$error): ?>
        <div class="alert alert-success"><?=$message ?></div>
        <a href="login.php" class="btn btn-primary">Go to login</a>
    <?php elseif ($error): ?>
        <div class="alert alert-danger"><?= htmlspecialchars($error) ?></div>
        <a href="install.php" class="btn btn-secondary">Retry</a>
    <?php else: ?>
        <p><?= htmlspecialchars($message) ?></p>
        <form method="post">
            <button type="submit" class="btn btn-primary">Run installation</button>
        </form>
    <?php endif; ?>
</div>
</body>
</html>

```

```

-----
FILE: install/schema.sql
-----

```

```

-- WPU SABLAE Portal – run via install.php or mysql client
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

```

```

CREATE TABLE IF NOT EXISTS colleges (
    id INT AUTO_INCREMENT PRIMARY KEY,
    college_code VARCHAR(20) NOT NULL UNIQUE,
    college_name VARCHAR(120) NOT NULL,
    status ENUM('active','inactive') NOT NULL DEFAULT 'active',
    dean_user_id INT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(190) NOT NULL DEFAULT '',
    role ENUM('super_admin','admin','dean','program_chain','faculty','gened','student') NOT NULL DEFAULT 'faculty',
    assigned_program VARCHAR(120) NOT NULL DEFAULT '',
    admin_log_title VARCHAR(120) NOT NULL DEFAULT '',
    college_id INT NULL,
    is_active TINYINT(1) NOT NULL DEFAULT 1,
    last_login_at DATETIME NULL,
    last_seen_at DATETIME NULL,
    last_logout_at DATETIME NULL,
    profile_photo VARCHAR(80) NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_sched_users_college_id FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

CREATE TABLE IF NOT EXISTS programs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    college_id INT NOT NULL,
    program_name VARCHAR(120) NOT NULL,
    status ENUM('active','inactive') NOT NULL DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_program_college_name (college_id, program_name),
    CONSTRAINT fk_program_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-- Programs a Program Chair account may manage (users.assigned_program = active/primary).
CREATE TABLE IF NOT EXISTS program_chair_programs (
  user_id INT NOT NULL,
  program_name VARCHAR(120) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id, program_name),
  CONSTRAINT fk_pcp_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Year levels offered per dean program (drives schedule / GE target year dropdowns).
CREATE TABLE IF NOT EXISTS programs_year_levels (
  program_id INT NOT NULL,
  year_level VARCHAR(20) NOT NULL,
  PRIMARY KEY (program_id, year_level),
  CONSTRAINT fk_prog_y1_program FOREIGN KEY (program_id) REFERENCES programs(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS college_programs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  college_id INT NOT NULL,
  program_code VARCHAR(30) NOT NULL,
  program_name VARCHAR(120) NOT NULL DEFAULT '',
  status ENUM('active','inactive') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_college_program_code (college_id, program_code),
  CONSTRAINT fk_cp_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS program_year_levels (
  id INT AUTO_INCREMENT PRIMARY KEY,
  program_id INT NOT NULL,
  year_level VARCHAR(20) NOT NULL,
  status ENUM('active','inactive') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_program_year (program_id, year_level),
  CONSTRAINT fk_py1_program FOREIGN KEY (program_id) REFERENCES college_programs(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS program_sections (
  id INT AUTO_INCREMENT PRIMARY KEY,
  year_level_id INT NOT NULL,
  section_name VARCHAR(20) NOT NULL,
  status ENUM('active','inactive') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_year_section (year_level_id, section_name),
  CONSTRAINT fk_ps_year FOREIGN KEY (year_level_id) REFERENCES program_year_levels(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS faculty (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NULL UNIQUE,
  faculty_id VARCHAR(20) NOT NULL UNIQUE,
  full_name VARCHAR(100) NOT NULL,
  department VARCHAR(100) NOT NULL DEFAULT '',
  email VARCHAR(100) NOT NULL DEFAULT '',
  max_hours_per_day INT NOT NULL DEFAULT 8,
  employment_status ENUM('Permanent','Contract of Service','Temporary') NOT NULL DEFAULT 'Permanent',
  is_gened TINYINT(1) NOT NULL DEFAULT 0,
  college_id INT NULL,
  status ENUM('active','inactive') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_faculty_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
  CONSTRAINT fk_faculty_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS courses (
  id INT AUTO_INCREMENT PRIMARY KEY,
  course_code VARCHAR(20) NOT NULL,
  course_name VARCHAR(100) NOT NULL,
  units DECIMAL(3,1) NOT NULL DEFAULT 3.0,
  lecture_units DECIMAL(3,1) NOT NULL DEFAULT 3.0,
  laboratory_units DECIMAL(3,1) NOT NULL DEFAULT 0.0,
  is_laboratory TINYINT(1) NOT NULL DEFAULT 0,
  is_gened TINYINT(1) NOT NULL DEFAULT 0,
  year_level VARCHAR(20) NOT NULL DEFAULT '',
  section VARCHAR(20) NOT NULL DEFAULT '',
  department VARCHAR(100) NOT NULL DEFAULT '',
  college_id INT NULL,
  classroom_code VARCHAR(16) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

```

KEY idx_course_code (course_code),
UNIQUE KEY uq_courses_classroom_code (classroom_code),
CONSTRAINT fk_course_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS rooms (
  id INT AUTO_INCREMENT PRIMARY KEY,
  room_code VARCHAR(20) NOT NULL,
  room_name VARCHAR(50) NOT NULL DEFAULT '',
  capacity INT NOT NULL DEFAULT 0,
  type ENUM('lecture','laboratory','conference','tba') NOT NULL DEFAULT 'lecture',
  status ENUM('available','tba','maintenance') NOT NULL DEFAULT 'available',
  is_gened TINYINT(1) NOT NULL DEFAULT 0,
  college_id INT NULL,
  room_code_scope VARCHAR(64) GENERATED ALWAYS AS (
    IF(COALESCE(is_gened,0) = 1,
      CONCAT('G|', room_code),
      CONCAT('C|', IFNULL(college_id, 0), '|', room_code))
  ) VIRTUAL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_rooms_scope (room_code_scope),
  KEY idx_room_code (room_code),
  CONSTRAINT fk_room_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS schedules (
  id INT AUTO_INCREMENT PRIMARY KEY,
  faculty_id INT NOT NULL,
  course_id INT NOT NULL,
  room_id INT NOT NULL,
  college_id INT NULL,
  schedule_type ENUM('MW','TTH','MWF','TTHS','Saturday','Sunday','MW_TTH','Custom') NOT NULL DEFAULT 'Custom',
  day_of_week SET('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday') NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  semester VARCHAR(20) NOT NULL,
  school_year VARCHAR(20) NOT NULL,
  academic_year VARCHAR(20) NOT NULL DEFAULT '',
  program VARCHAR(100) NOT NULL DEFAULT '',
  year_level VARCHAR(20) NOT NULL DEFAULT '',
  section VARCHAR(20) NOT NULL DEFAULT '',
  online_class_url VARCHAR(512) NULL DEFAULT NULL,
  online_live_at DATETIME NULL DEFAULT NULL,
  is_makeup TINYINT(1) NOT NULL DEFAULT 0,
  makeup_for_schedule_id INT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  created_by INT NOT NULL,
  CONSTRAINT fk_sched_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE,
  CONSTRAINT fk_sched_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE,
  CONSTRAINT fk_sched_room FOREIGN KEY (room_id) REFERENCES rooms(id) ON DELETE CASCADE,
  CONSTRAINT fk_sched_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE SET NULL,
  CONSTRAINT fk_sched_user FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS conflict_logs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  schedule_id INT NULL,
  conflict_type ENUM('faculty','room','time') NOT NULL,
  conflict_description TEXT NOT NULL,
  resolved TINYINT(1) NOT NULL DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_conf_sched FOREIGN KEY (schedule_id) REFERENCES schedules(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS conflict_requests (
  id INT AUTO_INCREMENT PRIMARY KEY,
  requested_by INT NOT NULL,
  college_id INT NOT NULL,
  faculty_id INT NOT NULL,
  course_id INT NOT NULL,
  room_id INT NOT NULL,
  schedule_type VARCHAR(20) NOT NULL,
  day_of_week VARCHAR(120) NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  semester VARCHAR(20) NOT NULL,
  school_year VARCHAR(20) NOT NULL,
  academic_year VARCHAR(20) NOT NULL DEFAULT '',
  program VARCHAR(100) NOT NULL DEFAULT '',
  year_level VARCHAR(20) NOT NULL DEFAULT '',

```

```

section VARCHAR(20) NOT NULL DEFAULT '',
reason TEXT NOT NULL,
status ENUM('pending','approved','rejected') NOT NULL DEFAULT 'pending',
admin_remarks TEXT NULL,
reviewed_by INT NULL,
reviewed_at DATETIME NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_cr_user FOREIGN KEY (requested_by) REFERENCES users(id) ON DELETE CASCADE,
CONSTRAINT fk_cr_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE,
CONSTRAINT fk_cr_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE,
CONSTRAINT fk_cr_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE,
CONSTRAINT fk_cr_room FOREIGN KEY (room_id) REFERENCES rooms(id) ON DELETE CASCADE,
CONSTRAINT fk_cr_reviewer FOREIGN KEY (reviewed_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

CREATE TABLE IF NOT EXISTS dean_activity_logs (
id INT AUTO_INCREMENT PRIMARY KEY,
dean_user_id INT NOT NULL,
college_id INT NOT NULL,
action_type VARCHAR(60) NOT NULL,
action_details TEXT NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_dal_user FOREIGN KEY (dean_user_id) REFERENCES users(id) ON DELETE CASCADE,
CONSTRAINT fk_dal_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

CREATE TABLE IF NOT EXISTS admin_activity_logs (
id INT AUTO_INCREMENT PRIMARY KEY,
admin_user_id INT NOT NULL,
admin_username VARCHAR(50) NOT NULL,
actor_full_name VARCHAR(100) NOT NULL DEFAULT '',
actor_log_title VARCHAR(120) NOT NULL DEFAULT '',
user_role VARCHAR(32) NOT NULL DEFAULT 'admin',
action_type VARCHAR(20) NOT NULL,
target_module VARCHAR(120) NOT NULL,
target_reference VARCHAR(255) NOT NULL DEFAULT '',
details_json LONGTEXT NULL,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
KEY idx_admin_activity_created (created_at),
KEY idx_admin_activity_user_created (admin_user_id, created_at),
KEY idx_admin_activity_role_created (user_role, created_at),
CONSTRAINT fk_aal_admin_user FOREIGN KEY (admin_user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

CREATE TABLE IF NOT EXISTS schedule_change_requests (
id INT AUTO_INCREMENT PRIMARY KEY,
faculty_user_id INT NOT NULL,
schedule_id INT NOT NULL,
request_type ENUM('change','makeup') NOT NULL DEFAULT 'change',
message TEXT NOT NULL,
proposed_day_of_week SET('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday') NULL,
proposed_start_time TIME NULL,
proposed_end_time TIME NULL,
proposed_room_id INT NULL,
status ENUM('pending','approved','rejected') NOT NULL DEFAULT 'pending',
dean_remarks TEXT NULL,
reviewed_by INT NULL,
reviewed_at DATETIME NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_scr_user FOREIGN KEY (faculty_user_id) REFERENCES users(id) ON DELETE CASCADE,
CONSTRAINT fk_scr_sched FOREIGN KEY (schedule_id) REFERENCES schedules(id) ON DELETE CASCADE,
CONSTRAINT fk_scr_reviewer FOREIGN KEY (reviewed_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

CREATE TABLE IF NOT EXISTS faculty_specializations (
id INT AUTO_INCREMENT PRIMARY KEY,
faculty_id INT NOT NULL,
course_id INT NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
UNIQUE KEY uq_faculty_course (faculty_id, course_id),
CONSTRAINT fk_fs_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE,
CONSTRAINT fk_fs_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

CREATE TABLE IF NOT EXISTS faculty_course_colors (
id INT AUTO_INCREMENT PRIMARY KEY,
faculty_id INT NOT NULL,
course_id INT NOT NULL,
color_index TINYINT UNSIGNED NOT NULL DEFAULT 0,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

```

```

    UNIQUE KEY uq_fcc_faculty_course (faculty_id, course_id),
    CONSTRAINT fk_fcc_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE,
    CONSTRAINT fk_fcc_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS ge_course_colleges (
    id INT AUTO_INCREMENT PRIMARY KEY,
    course_id INT NOT NULL,
    college_id INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_ge_course_college (course_id, college_id),
    CONSTRAINT fk_gecc_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE,
    CONSTRAINT fk_gecc_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS ge_schedule_targets (
    id INT AUTO_INCREMENT PRIMARY KEY,
    schedule_id INT NOT NULL UNIQUE,
    college_id INT NOT NULL,
    program_name VARCHAR(120) NOT NULL,
    year_level VARCHAR(20) NOT NULL DEFAULT '',
    section VARCHAR(20) NOT NULL DEFAULT '',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    KEY idx_ge_target_scope (college_id, program_name, year_level, section),
    CONSTRAINT fk_gst_schedule FOREIGN KEY (schedule_id) REFERENCES schedules(id) ON DELETE CASCADE,
    CONSTRAINT fk_gst_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS internal_messages (
    id INT AUTO_INCREMENT PRIMARY KEY,
    sender_user_id INT NOT NULL,
    recipient_user_id INT NOT NULL,
    subject VARCHAR(255) NOT NULL DEFAULT '',
    body TEXT NOT NULL,
    is_memo TINYINT(1) NOT NULL DEFAULT 0,
    attachment_original_name VARCHAR(255) NOT NULL DEFAULT '',
    attachment_stored_name VARCHAR(255) NOT NULL DEFAULT '',
    attachment_mime VARCHAR(120) NOT NULL DEFAULT '',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    read_at TIMESTAMP NULL DEFAULT NULL,
    KEY idx_im_recipient_unread (recipient_user_id, read_at),
    KEY idx_im_pair_time (sender_user_id, recipient_user_id, created_at),
    CONSTRAINT fk_im_sender FOREIGN KEY (sender_user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT fk_im_recipient FOREIGN KEY (recipient_user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS online_classrooms (
    id INT AUTO_INCREMENT PRIMARY KEY,
    schedule_id INT NOT NULL UNIQUE,
    faculty_id INT NOT NULL,
    course_id INT NOT NULL,
    title VARCHAR(150) NOT NULL,
    description TEXT NULL,
    meet_link VARCHAR(512) NOT NULL DEFAULT '',
    status ENUM('active','archived') NOT NULL DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    written_work_percentage DECIMAL(5,2) NOT NULL DEFAULT 50.00,
    performance_task_percentage DECIMAL(5,2) NOT NULL DEFAULT 50.00,
    join_code VARCHAR(16) NULL,
    syllabus_stored_name VARCHAR(255) NULL,
    syllabus_original_name VARCHAR(255) NULL,
    syllabus_mime VARCHAR(120) NULL,
    banner_stored_name VARCHAR(255) NULL,
    banner_original_name VARCHAR(255) NULL,
    banner_mime VARCHAR(120) NULL,
    UNIQUE KEY uq_oc_join_code (join_code),
    CONSTRAINT fk_oc_schedule FOREIGN KEY (schedule_id) REFERENCES schedules(id) ON DELETE CASCADE,
    CONSTRAINT fk_oc_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE,
    CONSTRAINT fk_oc_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS classroom_students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NULL UNIQUE,
    student_number VARCHAR(30) NOT NULL DEFAULT '',
    full_name VARCHAR(120) NOT NULL,
    email VARCHAR(190) NOT NULL DEFAULT '',
    year_level VARCHAR(20) NOT NULL DEFAULT '',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

```
CONSTRAINT fk_cls_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE IF NOT EXISTS student_registration_requests (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  password VARCHAR(255) NOT NULL,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(190) NOT NULL DEFAULT '',
  student_number VARCHAR(30) NOT NULL DEFAULT '',
  college_id INT NOT NULL,
  program_name VARCHAR(120) NOT NULL,
  year_level VARCHAR(20) NOT NULL DEFAULT '',
  status ENUM('pending','approved','rejected') NOT NULL DEFAULT 'pending',
  reviewed_by_user_id INT NULL,
  reviewed_at DATETIME NULL,
  rejection_reason VARCHAR(500) NOT NULL DEFAULT '',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_srr_pending_scope (status, college_id, program_name),
  INDEX idx_srr_username (username),
  CONSTRAINT fk_srr_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE,
  CONSTRAINT fk_srr_reviewer FOREIGN KEY (reviewed_by_user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE IF NOT EXISTS classroom_enrollments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  classroom_id INT NOT NULL,
  student_id INT NOT NULL,
  enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_classroom_student (classroom_id, student_id),
  CONSTRAINT fk_ce_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
  CONSTRAINT fk_ce_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE IF NOT EXISTS classroom_content (
  id INT AUTO_INCREMENT PRIMARY KEY,
  classroom_id INT NOT NULL,
  faculty_id INT NOT NULL,
  content_type ENUM('material','link','announcement') NOT NULL DEFAULT 'material',
  title VARCHAR(150) NOT NULL,
  body TEXT NULL,
  weeks VARCHAR(100) NOT NULL DEFAULT '',
  days_per_topic VARCHAR(100) NOT NULL DEFAULT '',
  resource_url VARCHAR(512) NOT NULL DEFAULT '',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_cc_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
  CONSTRAINT fk_cc_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE IF NOT EXISTS classroom_content_attachments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  content_id INT NOT NULL,
  original_name VARCHAR(255) NOT NULL DEFAULT '',
  stored_name VARCHAR(255) NOT NULL DEFAULT '',
  mime VARCHAR(120) NOT NULL DEFAULT '',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  KEY idx_cca_content (content_id),
  CONSTRAINT fk_cca_content FOREIGN KEY (content_id) REFERENCES classroom_content(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE IF NOT EXISTS classroom_messages (
  id INT AUTO_INCREMENT PRIMARY KEY,
  classroom_id INT NOT NULL,
  sender_user_id INT NOT NULL,
  body TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  KEY idx_cm_classroom_time (classroom_id, created_at),
  CONSTRAINT fk_cm_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
  CONSTRAINT fk_cm_sender FOREIGN KEY (sender_user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE IF NOT EXISTS classroom_assessments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  classroom_id INT NOT NULL,
  faculty_id INT NOT NULL,
  assessment_type ENUM('written_work','performance_task','assignment','quiz') NOT NULL DEFAULT 'written_work',
  title VARCHAR(150) NOT NULL,
  description TEXT NULL,
  total_points DECIMAL(8,2) NOT NULL DEFAULT 100.00,
  due_at DATETIME NULL,
```

```

time_limit_minutes INT UNSIGNED NULL DEFAULT NULL,
credited_week VARCHAR(100) NOT NULL DEFAULT '',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_ca_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
CONSTRAINT fk_ca_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS classroom_scores (
  id INT AUTO_INCREMENT PRIMARY KEY,
  assessment_id INT NOT NULL,
  student_id INT NOT NULL,
  score DECIMAL(8,2) NULL,
  feedback TEXT NULL,
  graded_at DATETIME NULL,
  UNIQUE KEY uq_assessment_student (assessment_id, student_id),
  CONSTRAINT fk_cs_assessment FOREIGN KEY (assessment_id) REFERENCES classroom_assessments(id) ON DELETE CASCADE,
  CONSTRAINT fk_cs_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS classroom_submissions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  assessment_id INT NOT NULL,
  student_id INT NOT NULL,
  answer_text LONGTEXT NULL,
  status ENUM('submitted','resubmitted') NOT NULL DEFAULT 'submitted',
  submitted_at DATETIME NOT NULL,
  updated_at DATETIME NOT NULL,
  UNIQUE KEY uq_submission_assessment_student (assessment_id, student_id),
  CONSTRAINT fk_csub_assessment FOREIGN KEY (assessment_id) REFERENCES classroom_assessments(id) ON DELETE CASCADE,
  CONSTRAINT fk_csub_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS classroom_attendance_sessions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  classroom_id INT NOT NULL,
  faculty_id INT NOT NULL,
  attendance_date DATE NOT NULL,
  session_start_at DATETIME NOT NULL,
  session_end_at DATETIME NOT NULL,
  source ENUM('auto_login_online') NOT NULL DEFAULT 'auto_login_online',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uq_classroom_attendance_day (classroom_id, attendance_date),
  CONSTRAINT fk_cas_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
  CONSTRAINT fk_cas_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS classroom_attendance_records (
  id INT AUTO_INCREMENT PRIMARY KEY,
  session_id INT NOT NULL,
  student_id INT NOT NULL,
  status ENUM('present','absent') NOT NULL DEFAULT 'absent',
  source ENUM('auto','manual') NOT NULL DEFAULT 'auto',
  evidence_login_at DATETIME NULL,
  evidence_seen_at DATETIME NULL,
  evidence_logout_at DATETIME NULL,
  checked_at DATETIME NOT NULL,
  notes VARCHAR(255) NOT NULL DEFAULT '',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uq_attendance_session_student (session_id, student_id),
  KEY idx_attendance_status (session_id, status),
  CONSTRAINT fk_car_session FOREIGN KEY (session_id) REFERENCES classroom_attendance_sessions(id) ON DELETE CASCADE,
  CONSTRAINT fk_car_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS login_slideshow_images (
  id INT AUTO_INCREMENT PRIMARY KEY,
  stored_name VARCHAR(120) NOT NULL,
  caption VARCHAR(255) NOT NULL DEFAULT '',
  sort_order INT NOT NULL DEFAULT 0,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  uploaded_by INT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  KEY idx_login_slide_order (sort_order, id),
  CONSTRAINT fk_login_slide_uploader FOREIGN KEY (uploaded_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;

```

FILE: login.php

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/student_registration_helpers.php';
require_once __DIR__ . '/includes/login_slideshow_helpers.php';

if (!empty($_SESSION['user_id'])) {
    header('Location: dashboard.php');
    exit;
}

$loginRoles = [
    'student' => [
        'label' => 'Student',
        'icon' => 'fa-user-graduate',
        'roles' => ['student'],
        'subtitle' => 'Sign in to join classes, view schedules, and use EduTools.',
    ],
    'admin' => [
        'label' => 'Admin',
        'icon' => 'fa-user-shield',
        'roles' => ['admin'],
        'subtitle' => 'Day-to-day scheduling, colleges, deans, and institution-wide settings.',
    ],
    'super_admin' => [
        'label' => 'Super Admin',
        'short' => 'Super',
        'icon' => 'fa-user-secret',
        'roles' => ['super_admin'],
        'subtitle' => 'Provision administrator accounts, faculty inventory, and institution-wide reports.',
    ],
    'program_chair' => [
        'label' => 'Program Chair',
        'icon' => 'fa-user-tie',
        'roles' => ['program_chair'],
        'subtitle' => 'Manage faculty, students, courses, and program schedules.',
    ],
    'dean' => [
        'label' => 'Dean',
        'icon' => 'fa-building-columns',
        'roles' => ['dean'],
        'subtitle' => 'College oversight, program chairs, faculty, and scheduling.',
    ],
    'faculty' => [
        'label' => 'Faculty',
        'icon' => 'fa-chalkboard-user',
        'roles' => ['faculty'],
        'subtitle' => 'Classrooms, teaching load, schedules, and academic tools.',
    ],
    'gened' => [
        'label' => 'General Education',
        'short' => 'Gen Ed',
        'icon' => 'fa-book-open',
        'roles' => ['gened'],
        'subtitle' => 'Institution-wide GE courses, faculty, rooms, and scheduling.',
        'accent' => '#059669',
        'accent_bg' => '#ecfdf5',
        'accent_border' => '#6ee7b7',
    ],
];

$selectedRole = (string) ($_GET['role'] ?? $_POST['login_role'] ?? 'student');
if (!isset($loginRoles[$selectedRole])) {
    $selectedRole = 'student';
}

$viewMode = (string) ($_GET['mode'] ?? $_POST['form_mode'] ?? 'login');
if ($viewMode !== 'register') {
    $viewMode = 'login';
}
if ($viewMode === 'register' && $selectedRole !== 'student') {
    $viewMode = 'login';
}
```



```

$error = '';
$success = '';
$usernameValue = '';
$regFullName = '';
$regEmail = '';
$regStudentNumber = '';
$regCollegeId = 0;
$regProgramName = '';
$regYearLevel = '';
$hasAssignedProgram = false;
$hasAdminLogTitle = false;
$registrationReady = student_registration_table_ready();
$colleges = $registrationReady ? active_colleges_for_registration() : [];
$programsForCollege = $regCollegeId > 0 ? active_programs_for_college($regCollegeId) : [];

try {
    $colStmt = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.COLUMNS
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ? AND COLUMN_NAME = ?'
    );
    $colStmt->execute([DB_NAME, 'users', 'assigned_program']);
    $hasAssignedProgram = (int) $colStmt->fetchColumn() > 0;
    $colStmt->execute([DB_NAME, 'users', 'admin_log_title']);
    $hasAdminLogTitle = (int) $colStmt->fetchColumn() > 0;
} catch (Throwable $e) {
    $hasAssignedProgram = false;
    $hasAdminLogTitle = false;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $formMode = (string) ($_POST['form_mode'] ?? 'login');
    $postedRole = (string) ($_POST['login_role'] ?? $selectedRole);
    if (!isset($loginRoles[$postedRole])) {
        $postedRole = 'student';
    }
    $selectedRole = $postedRole;
    $allowedRoles = $loginRoles[$selectedRole]['roles'];

    if ($formMode === 'register') {
        $viewMode = 'register';
        $usernameValue = trim((string) ($_POST['username'] ?? ''));
        $regFullName = trim((string) ($_POST['full_name'] ?? ''));
        $regEmail = trim((string) ($_POST['email'] ?? ''));
        $regStudentNumber = trim((string) ($_POST['student_number'] ?? ''));
        $regCollegeId = (int) ($_POST['college_id'] ?? 0);
        $regProgramName = trim((string) ($_POST['program_name'] ?? ''));
        $regYearLevel = trim((string) ($_POST['year_level'] ?? ''));
        try {
            submit_student_registration(
                $usernameValue,
                $regFullName,
                $regEmail,
                $regStudentNumber,
                $regCollegeId,
                $regProgramName,
                $regYearLevel
            );
            $success = 'Registration submitted successfully. Your Dean will review your request. Once approved, you will receive an email
with your username and temporary password.';
            $viewMode = 'login';
            $usernameValue = '';
            $regFullName = '';
            $regEmail = '';
            $regStudentNumber = '';
            $regCollegeId = 0;
            $regProgramName = '';
            $regYearLevel = '';
        } catch (Throwable $e) {
            $error = $e->getMessage();
            $programsForCollege = $regCollegeId > 0 ? active_programs_for_college($regCollegeId) : [];
        }
    } else {
        $viewMode = 'login';
        $username = trim((string) ($_POST['username'] ?? ''));
        $usernameValue = $username;
        $password = (string) ($_POST['password'] ?? '');
        if ($username === '' || $password === '') {
            $error = 'Please enter username and password.';
        } else {
            try {

```

```

$pendingMsg = registration_status_message_for_username($username);
if ($pendingMsg !== null) {
    $error = $pendingMsg;
} else {
    $userSql = 'SELECT u.id, u.username, u.password, u.full_name, u.role, '
        . ($hasAssignedProgram ? 'u.assigned_program' : '"" AS assigned_program') . ', '
        . ($hasAdminLogTitle ? 'u.admin_log_title' : '"" AS admin_log_title') . ', '
        . u.college_id, u.is_active, f.id AS faculty_id
    FROM users u
    LEFT JOIN faculty f ON f.user_id = u.id
    WHERE u.username = ? LIMIT 1';
    $user = false;
    foreach (login_username_lookup_variants($username) as $usernameTry) {
        $stmt = db()->prepare($userSql);
        $stmt->execute([$usernameTry]);
        $found = $stmt->fetch();
        if ($found) {
            $user = $found;
            break;
        }
    }
    if ($user && (int) $user['is_active'] === 1 && password_verify($password, $user['password'])) {
        $userRole = (string) $user['role'];
        if (!in_array($userRole, $allowedRoles, true)) {
            if ($userRole === 'super_admin' && $postedRole === 'admin') {
                $error = 'This is a Super Administrator account. Select the Super Admin tab and sign in again.';
            } elseif ($userRole === 'admin' && $postedRole === 'super_admin') {
                $error = 'This is an Administrator (scheduling) account. Select the Admin tab and sign in again.';
            } else {
                $error = 'This account is not a ' . strtolower($loginRoles[$selectedRole]['label'])
                    . ' account. Please select the correct role tab and try again.';
            }
        }
    } else {
        $resolvedFacultyId = $user['faculty_id'] !== null ? (int) $user['faculty_id'] : null;
        $resolvedStudentId = null;
        if (in_array($userRole, ['faculty', 'program_chair', 'dean', 'gened'], true) && $resolvedFacultyId === null) {
            $resolvedFacultyId = resolve_faculty_id_for_user((int) $user['id']);
        }
        if ($userRole === 'student') {
            $resolvedStudentId = resolve_student_id_for_user((int) $user['id']);
        }
        $_SESSION['user_id'] = (int) $user['id'];
        $_SESSION['username'] = $user['username'];
        $_SESSION['full_name'] = $user['full_name'];
        $_SESSION['role'] = $user['role'];
        $_SESSION['assigned_program'] = (string) ($user['assigned_program'] ?? '');
        $_SESSION['assigned_programs'] = [];
        $_SESSION['admin_log_title'] = trim((string) ($user['admin_log_title'] ?? ''));
        $_SESSION['college_id'] = $user['college_id'] !== null ? (int) $user['college_id'] : null;
        $_SESSION['faculty_id'] = $resolvedFacultyId;
        $_SESSION['student_id'] = $resolvedStudentId;
        if ($userRole === 'program_chair') {
            ensure_program_chair_programs_table();
            program_chair_sync_session_programs((int) $user['id']);
        }
        try {
            if (db_column_exists('users', 'last_logout_at')) {
                db()->prepare('UPDATE users SET last_login_at = NOW(), last_seen_at = NOW(), last_logout_at = NULL WHERE
id = ?')
                    ->execute([(int) $user['id']]);
            } else {
                db()->prepare('UPDATE users SET last_login_at = NOW(), last_seen_at = NOW() WHERE id = ?')
                    ->execute([(int) $user['id']]);
            }
        } catch (Throwable $e) {
            // Ignore if legacy schema does not yet have activity columns.
        }
        require_once __DIR__ . '/includes/admin_activity_log.php';
        $loginMeta = [
            'role' => $userRole,
            'ip' => (string) ($_SERVER['REMOTE_ADDR'] ?? ''),
        ];
        if ($hasAdminLogTitle) {
            $loginMeta['log_title'] = trim((string) ($user['admin_log_title'] ?? ''));
        }
        log_user_activity(
            'login',
            'Authentication',
            'Signed in',
            null,

```

```

        $loginMeta
    );
    $mustChangePassword = user_must_change_password((int) $user['id']);
    $_SESSION['must_change_password'] = $mustChangePassword;
    if ($mustChangePassword) {
        header('Location: settings.php?force_password=1');
        exit;
    }
    header('Location: dashboard.php');
    exit;
}
} else {
    $error = $user && (int) $user['is_active'] != 1
        ? 'Your account is disabled. Please contact your dean/admin.'
        : 'Invalid username or password.';
}
}
} catch (Throwable $e) {
    $prev = $e->getPrevious();
    $error = $e instanceof RuntimeException && $prev instanceof PDOException
        ? $e->getMessage()
        : 'Database error: ' . $e->getMessage();
}
}
}

if ($regCollegeId > 0) {
    $programsForCollege = active_programs_for_college($regCollegeId);
}

$roleSubtitle = $loginRoles[$selectedRole]['subtitle'];
$loginSlideshowImages = login_slideshow_active_images();

$sealDataUri = '';
$sealCandidates = [
    __DIR__ . '/assets/images/wpu-seal.png',
    'C:\Users\wpu-0\cursor\projects\c-xampp\htdocs\CLASS\assets\c__Users_wpu-
0_AppData_Roaming_Cursor_User_workspaceStorage_f94ec644ac39e85d4f1f3f887fe3104f_images_A_LOGO-ba32c997-80f6-4298-97e0-4ca3b192b5d0.png',
];
foreach ($sealCandidates as $sealPath) {
    if (is_file($sealPath)) {
        $sealRaw = @file_get_contents($sealPath);
        if ($sealRaw !== false) {
            $sealDataUri = 'data:image/png;base64,' . base64_encode($sealRaw);
            break;
        }
    }
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=yes">
    <title>WPU SABLA ePortal | Smart Academic Hub</title>
    <?php require __DIR__ . '/includes/favicon.php'; ?>
    <link href="https://fonts.googleapis.com/css2?
family=Inter:opsz,wght@14..32,300;14..32,400;14..32,500;14..32,600;14..32,700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css">
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        :root {
            --portal-accent: #4f46e5;
            --portal-accent-soft: #eef2ff;
            --portal-accent-border: #c7d2fe;
        }
        body {
            font-family: 'Inter', system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
            background: linear-gradient(145deg, #eef2ff 0%, #e0e7ff 45%, #ecfdf5 100%);
            min-height: 100vh;
            display: flex;
            align-items: center;
            justify-content: center;
            padding: 1.5rem;
            position: relative;
        }
        body::before {
            content: "";
            position: fixed;

```

```

inset: 0;
background-image:
  radial-gradient(circle at 12% 22%, rgba(79, 70, 229, 0.06) 0%, transparent 42%),
  radial-gradient(circle at 88% 78%, rgba(5, 150, 105, 0.05) 0%, transparent 38%),
  radial-gradient(circle at 10% 30%, rgba(79, 70, 229, 0.03) 2%, transparent 2.5%);
background-size: auto, auto, 48px 48px;
pointer-events: none;
z-index: 0;
}
.portal-container {
  max-width: 1320px;
  width: 100%;
  background: #fff;
  border-radius: 2rem;
  box-shadow: 0 25px 50px -16px rgba(15, 23, 42, 0.28), 0 8px 20px rgba(15, 23, 42, 0.06);
  overflow: hidden;
  display: flex;
  flex-wrap: wrap;
  z-index: 2;
  border: 1px solid rgba(255, 255, 255, 0.7);
}
.hero-panel {
  flex: 1.15;
  background: linear-gradient(155deg, #1e293b 0%, #0f172a 55%, #134e4a 100%);
  padding: 2rem 2rem;
  color: #fff;
  display: flex;
  flex-direction: column;
  position: relative;
  overflow: hidden;
}
.hero-panel::before {
  content: "";
  position: absolute;
  inset: 0;
  background: radial-gradient(ellipse 80% 60% at 100% 100%, rgba(52, 211, 153, 0.12), transparent 55%);
  pointer-events: none;
}
.hero-panel::after {
  content: "◆";
  font-size: 280px;
  opacity: 0.05;
  position: absolute;
  bottom: -50px;
  right: -40px;
  pointer-events: none;
}
}
.logo-area { margin-bottom: 1.25rem; display: flex; align-items: center; gap: 0.75rem; position: relative; z-index: 1; }
.logo-seal {
  width: 58px; height: 58px; border-radius: 50%; object-fit: contain;
  background: #fff; padding: 5px; flex-shrink: 0;
  box-shadow: 0 10px 28px rgba(0, 0, 0, 0.22);
  border: 2px solid rgba(255, 255, 255, 0.28);
}
}
.logo-text h1 { font-size: 1.35rem; font-weight: 700; letter-spacing: -0.35px; line-height: 1.2; }
.logo-text span { font-size: 0.68rem; opacity: 0.78; display: block; margin-top: 0.15rem; letter-spacing: 0.02em; }
.hero-quote { margin-top: auto; margin-bottom: 1rem; position: relative; z-index: 1; }
.hero-quote h2 {
  font-size: 1.35rem; font-weight: 600; line-height: 1.3; margin-bottom: 0.5rem;
  background: linear-gradient(120deg, #fff 0%, #c7d2fe 55%, #6ee7b7 100%);
  background-clip: text; -webkit-background-clip: text; color: transparent;
}
}
.hero-quote p { font-size: 0.8rem; color: #cbd5e1; max-width: 90%; line-height: 1.45; }
.feature-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 0.45rem;
  margin-top: 1rem;
  position: relative;
  z-index: 1;
}
}
.feature-item {
  display: flex; align-items: center; gap: 0.45rem; font-size: 0.72rem;
  background: rgba(255, 255, 255, 0.06); padding: 0.45rem 0.7rem; border-radius: 0.75rem;
  border: 1px solid rgba(255, 255, 255, 0.08);
  backdrop-filter: blur(4px);
}
}
.feature-item i { color: #a5b4fc; width: 1rem; text-align: center; flex-shrink: 0; }
.feature-item.feature-gened i { color: #6ee7b7; }
.hero-slideshow {

```

```

    flex: 1 1 auto;
    margin: 0.5rem 0 1rem;
    position: relative;
    z-index: 1;
    width: 100%;
    min-height: clamp(260px, 42vh, 420px);
    height: clamp(260px, 42vh, 420px);
    border-radius: 1.1rem;
    overflow: hidden;
    border: 1px solid rgba(255, 255, 255, 0.14);
    background: rgba(15, 23, 42, 0.45);
    box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.06);
}
.hero-slide {
    position: absolute;
    inset: 0;
    opacity: 0;
    transition: opacity 1.1s ease-in-out;
    pointer-events: none;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 0.15rem;
}
.hero-slide.active {
    opacity: 1;
    pointer-events: auto;
}
.hero-slide img {
    max-width: 100%;
    max-height: 100%;
    width: auto;
    height: auto;
    object-fit: contain;
    object-position: center;
    display: block;
    border-radius: 0.65rem;
}
.hero-slide-caption {
    position: absolute;
    left: 0;
    right: 0;
    bottom: 0;
    padding: 0.55rem 0.75rem;
    font-size: 0.72rem;
    line-height: 1.4;
    color: #f8fafc;
    background: linear-gradient(180deg, transparent, rgba(15, 23, 42, 0.82));
}
.hero-slideshow-dots {
    position: absolute;
    top: 0.65rem;
    right: 0.75rem;
    display: flex;
    gap: 0.35rem;
    z-index: 2;
}
.hero-slideshow-dot {
    width: 7px;
    height: 7px;
    border-radius: 50%;
    border: none;
    padding: 0;
    background: rgba(255, 255, 255, 0.35);
    cursor: pointer;
    transition: background 0.2s, transform 0.2s;
}
.hero-slideshow-dot.active {
    background: #fff;
    transform: scale(1.15);
}
.form-panel {
    flex: 1;
    padding: 2.75rem 2.25rem;
    display: flex;
    flex-direction: column;
    justify-content: center;
    background: linear-gradient(180deg, #ffffff 0%, #f8fafc 100%);
}
.welcome-header { margin-bottom: 1.6rem; }

```

```

.welcome-header h3 { font-size: 1.95rem; font-weight: 700; color: #0f172a; letter-spacing: -0.02em; }
.welcome-header p { color: #475569; font-size: 0.9rem; margin-top: 0.55rem; line-height: 1.55; max-width: 36rem; }
.role-section-label {
  font-size: 0.72rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  color: #94a3b8;
  margin-bottom: 0.65rem;
}
.role-tabs {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 0.55rem;
  margin-bottom: 1.5rem;
}
.role-btn {
  border: 1.5px solid #e2e8f0;
  padding: 0.7rem 0.45rem;
  border-radius: 0.95rem;
  font-weight: 600;
  font-size: 0.72rem;
  line-height: 1.25;
  cursor: pointer;
  color: #475569;
  font-family: 'Inter', sans-serif;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 0.35rem;
  background: #f8fafc;
  transition: border-color 0.2s, background 0.2s, color 0.2s, box-shadow 0.2s, transform 0.15s;
  min-height: 4.1rem;
}
.role-btn i { font-size: 1.05rem; color: #64748b; transition: color 0.2s; }
.role-btn .role-short { display: none; }
.role-btn.active {
  background: var(--portal-accent-soft);
  color: var(--portal-accent);
  border-color: var(--portal-accent-border);
  box-shadow: 0 4px 14px rgba(79, 70, 229, 0.12);
  transform: translateY(-1px);
}
.role-btn.active i { color: var(--portal-accent); }
.role-btn:not(.active):hover {
  background: #f1f5f9;
  border-color: #cbd5e1;
}
.auth-switch {
  display: flex; gap: 0.5rem; background: #f1f5f9; padding: 0.4rem;
  border-radius: 60px; margin-bottom: 1.5rem;
}
.auth-switch.hidden { display: none; }
.auth-switch-btn {
  flex: 1; border: none; background: transparent; padding: 0.6rem 1rem;
  border-radius: 40px; font-weight: 600; font-size: 0.85rem; color: #334155;
  cursor: pointer; font-family: 'Inter', sans-serif;
}
.auth-switch-btn.active {
  background: #fff; color: #4f46e5;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.05); border: 1px solid #e2e8f0;
}
.auth-switch-btn.disabled { opacity: 0.5; cursor: not-allowed; }
.alert-banner {
  padding: 0.85rem 1rem; border-radius: 1rem; font-size: 0.88rem;
  margin-bottom: 1.25rem; line-height: 1.45;
}
.alert-banner.success { background: #ecfdf5; color: #065f46; border: 1px solid #a7f3d0; }
.alert-banner.error { background: #fef2f2; color: #991b1b; border: 1px solid #fecaca; }
.input-group { margin-bottom: 1.25rem; }
.input-label {
  display: flex; align-items: center; gap: 8px; font-weight: 500;
  font-size: 0.85rem; margin-bottom: 0.5rem; color: #1e293b;
}
.input-label i { color: #64748b; width: 18px; }
.input-field {
  width: 100%; padding: 0.9rem 1rem; border: 1.5px solid #e2e8f0;
  border-radius: 1rem; font-size: 0.95rem; font-family: 'Inter', sans-serif;
  background: #fefefe; transition: all 0.2s;
}

```

```

}
.input-field:focus {
  outline: none; border-color: #818cf8;
  box-shadow: 0 0 3px rgba(79, 70, 229, 0.1);
}
.password-wrapper { position: relative; width: 100%; }
.password-wrapper .input-field { padding-right: 2.75rem; }
.pwd-toggle {
  position: absolute; right: 14px; top: 50%; transform: translateY(-50%);
  cursor: pointer; color: #94a3b8;
}
.form-row { display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; }
.login-panel-form.hide, .register-panel { display: none; }
.register-panel.show { display: block; }
.login-btn {
  width: 100%; background: var(--portal-accent); border: none; padding: 0.95rem;
  border-radius: 1rem; font-weight: 700; font-size: 1rem; color: #fff;
  font-family: 'Inter', sans-serif; display: flex; align-items: center;
  justify-content: center; gap: 0.5rem; cursor: pointer; margin-bottom: 1.25rem;
  box-shadow: 0 6px 16px rgba(79, 70, 229, 0.22); transition: all 0.25s;
}
.login-btn:hover:not(:disabled) { background: #4338ca; transform: translateY(-1px); box-shadow: 0 8px 20px rgba(79, 70, 229, 0.28); }

.login-btn:disabled { opacity: 0.55; cursor: not-allowed; }
.register-note { text-align: center; color: #64748b; font-size: 0.85rem; line-height: 1.45; }
.portal-footer {
  margin-top: 1.5rem;
  padding-top: 1rem;
  border-top: 1px solid #e2e8f0;
  font-size: 0.75rem;
  color: #94a3b8;
  text-align: center;
}

@media (max-width: 880px) {
  .portal-container { flex-direction: column; border-radius: 1.5rem; }
  .hero-panel, .form-panel { padding: 2rem 1.5rem; }
  .hero-quote h2 { font-size: 1.2rem; }
  .hero-slideshow { min-height: clamp(200px, 32vh, 300px); height: clamp(200px, 32vh, 300px); }
  .hero-quote p { max-width: 100%; }
  .form-row { grid-template-columns: 1fr; }
  .feature-grid { grid-template-columns: 1fr 1fr; }
}

@media (max-width: 550px) {
  .role-tabs { grid-template-columns: repeat(2, minmax(0, 1fr)); }
  .role-btn.role-full { display: none; }
  .role-btn.role-short { display: inline; }
  .role-btn { min-height: 3.6rem; font-size: 0.68rem; }
  .welcome-header h3 { font-size: 1.55rem; }
  .feature-grid { grid-template-columns: 1fr; }
}
</style>
</head>
<body>

<div class="portal-container">
  <div class="hero-panel">
    <div class="logo-area">
      <?php if ($sealDataUri !== ''): ?>
        
      <?php else: ?>
        
      <?php endif; ?>
      <div class="logo-text">
        <h1>WPU SABLA ePortal</h1>
        <span>scheduling &amp; LMS · unified workspace</span>
      </div>
    </div>
    <?php if ($loginSlideshowImages !== []): ?>
      <div class="hero-slideshow" id="heroSlideshow" aria-live="polite" aria-label="Campus highlights">
        <?php foreach ($loginSlideshowImages as $i => $slide): ?>
          <div class="hero-slide<?= $i === 0 ? ' active' : '' ?>" data-slide="<?= $i ?>">
            " loading="<?= $i === 0 ? 'eager' : 'lazy' ?>" decoding="async">
            <?php if ($slide['caption'] !== ''): ?>
              <div class="hero-slide-caption"><?= htmlspecialchars($slide['caption']) ?></div>
            <?php endif; ?>
          </div>
        <?php endforeach; ?>
        <?php if (count($loginSlideshowImages) > 1): ?>
          <div class="hero-slideshow-dots" role="tablist" aria-label="Slideshow navigation">

```

```

        <?php foreach ($loginSlideshowImages as $i => $slide): ?>
            <button type="button" class="hero-slideshow-dot<?=$i === 0 ? ' active' : '' ?>" data-slide="<?=$i ?>" aria-
label="Show slide <?=$i + 1 ?>"<?=$i === 0 ? ' aria-selected="true" : '' ?>></button>
        <?php endforeach; ?>
    </div>
<?php endif; ?>
</div>
<?php endif; ?>
<div class="hero-quote">
    <h2>Academic scheduling, classrooms, and General Education – unified in one hub.</h2>
    <p>Seamless access for students, faculty, deans, program chairs, and the GE coordinator – all in one intelligent workspace.</p>
</div>
<div class="feature-grid">
    <div class="feature-item"><i class="fas fa-calendar-alt"></i> <span>Smart scheduling</span></div>
    <div class="feature-item"><i class="fas fa-chalkboard"></i> <span>EduTools integrated</span></div>
    <div class="feature-item feature-gened"><i class="fas fa-book-open"></i> <span>General Education</span></div>
    <div class="feature-item"><i class="fas fa-shield-alt"></i> <span>Secure workspace</span></div>
</div>
</div>
<div class="form-panel">
    <div class="welcome-header">
        <h3 id="authHeading"><?=$viewMode === 'register' ? 'Create student account' : 'Welcome back' ?></h3>
        <p id="authSubtitle"><?=$htmlspecialchars($viewMode === 'register' ? 'Register under your college and program. Your Dean must
approve before you can sign in. You will receive an email with your temporary password once approved.' : $roleSubtitle) ?></p>
    </div>

    <div class="role-section-label">Sign in as</div>
    <div class="role-tabs" id="roleTabs" role="tablist" aria-label="Sign in as">
        <?php foreach ($loginRoles as $roleKey => $roleMeta): ?>
            <button type="button"
                class="role-btn<?=$selectedRole === $roleKey ? ' active' : '' ?>"
                data-role="<?=$htmlspecialchars($roleKey) ?>"
                aria-pressed="<?=$selectedRole === $roleKey ? 'true' : 'false' ?>"
                <i class="fa-solid <?=$htmlspecialchars($roleMeta['icon']) ?>"></i>
                <span class="role-full"><?=$htmlspecialchars($roleMeta['label']) ?></span>
            <?php if (!empty($roleMeta['short'])): ?>
                <span class="role-short"><?=$htmlspecialchars($roleMeta['short']) ?></span>
            <?php else: ?>
                <span class="role-short"><?=$htmlspecialchars($roleMeta['label']) ?></span>
            <?php endif; ?>
        </button>
    <?php endforeach; ?>
</div>

    <div class="auth-switch<?=$selectedRole !== 'student' ? ' hidden' : '' ?>" id="studentAuthSwitch">
        <button type="button" class="auth-switch-btn<?=$viewMode === 'login' ? ' active' : '' ?>" data-mode="login">Sign in</button>
        <button type="button" class="auth-switch-btn<?=$viewMode === 'register' ? ' active' : '' ?>" data-mode="register"
            <?=$!$registrationReady ? ' disabled title="Run upgrade_roles.php first" : '' ?>>Register</button>
    </div>

    <?php if ($success): ?>
        <div class="alert-banner success"><?=$htmlspecialchars($success) ?></div>
    <?php endif; ?>
    <?php if ($error): ?>
        <div class="alert-banner error"><?=$htmlspecialchars($error) ?></div>
    <?php endif; ?>

    <form method="post" autocomplete="on" class="login-panel-form<?=$viewMode === 'register' ? ' hide' : '' ?>" id="loginForm">
        <input type="hidden" name="form_mode" value="login">
        <input type="hidden" name="login_role" id="loginRoleField" value="<?=$htmlspecialchars($selectedRole) ?>">
        <div class="input-group">
            <div class="input-label"><i class="fas fa-user-circle"></i> <span>Username</span></div>
            <input type="text" name="username" id="username" class="input-field" value="<?=$htmlspecialchars($usernameValue) ?>"
placeholder="Enter your username" required autofocus>
        </div>
        <div class="input-group">
            <div class="input-label"><i class="fas fa-lock"></i> <span>Password</span></div>
            <input type="password" name="password" id="password" class="input-field" placeholder="••••••••" autocomplete="current-
password" required>
        </div>
        <button type="submit" class="login-btn">
            <i class="fas fa-arrow-right-to-bracket"></i> Sign in
        </button>
    </form>

    <form method="post" autocomplete="on" class="register-panel<?=$viewMode === 'register' ? ' show' : '' ?>" id="registerForm">
        <input type="hidden" name="form_mode" value="register">
        <input type="hidden" name="login_role" value="student">
        <div class="input-group">

```



```

        <div class="input-label"><i class="fas fa-id-card"></i> <span>Full name</span></div>
        <input type="text" name="full_name" id="reg_full_name" class="input-field" value="<?> htmlspecialchars($regFullName) ?>"
required>
    </div>
    <div class="input-group">
        <div class="input-label"><i class="fas fa-hashtag"></i> <span>Student number</span></div>
        <input type="text" name="student_number" id="reg_student_number" class="input-field" value="<?>
htmlspecialchars($regStudentNumber) ?>" required>
    </div>
    <div class="form-row">
        <div class="input-group">
            <div class="input-label"><i class="fas fa-building"></i> <span>College</span></div>
            <select name="college_id" id="reg_college_id" class="input-field" required>
                <option value="">Select college</option>
                <?php foreach ($colleges as $college): ?>
                    <option value="<?> (int) $college['id'] ?>"<?> $regCollegeId === (int) $college['id'] ? ' selected' : '' ?>>
                        <?> htmlspecialchars((string) $college['college_name']) ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </div>
        <div class="input-group">
            <div class="input-label"><i class="fas fa-graduation-cap"></i> <span>Program</span></div>
            <select name="program_name" id="reg_program_name" class="input-field" required>
                <option value="">Select program</option>
                <?php foreach ($programsForCollege as $program): ?>
                    <option value="<?> htmlspecialchars((string) $program) ?>"<?> $regProgramName === (string) $program ? '
selected' : '' ?>>
                        <?> htmlspecialchars((string) $program) ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </div>
    </div>
    <div class="input-group">
        <div class="input-label"><i class="fas fa-layer-group"></i> <span>Year level</span></div>
        <select name="year_level" id="reg_year_level" class="input-field" required>
            <option value="">Select year level</option>
            <?php
                $yearLevelsForForm = $regCollegeId > 0 && $regProgramName !== ''
                ? active_year_levels_for_program($regCollegeId, $regProgramName)
                : [];
            foreach ($yearLevelsForForm as $yl):
                <option value="<?> htmlspecialchars((string) $yl) ?>"<?> $regYearLevel === (string) $yl ? ' selected' : '' ?>>
                    Year <?> htmlspecialchars((string) $yl) ?>
                </option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="input-group">
        <div class="input-label"><i class="fas fa-user"></i> <span>Username</span></div>
        <input type="text" name="username" id="reg_username" class="input-field" value="<?> htmlspecialchars($usernameValue) ?>"
required pattern="[A-Za-z0-9._-]{3,50}">
    </div>
    <?php if (db_column_exists('users', 'email')): ?>
    <div class="input-group">
        <div class="input-label"><i class="fas fa-envelope"></i> <span>Email</span></div>
        <input type="email" name="email" id="reg_email" class="input-field" value="<?> htmlspecialchars($regEmail) ?>" required>
    </div>
    <?php endif; ?>
    <button type="submit" class="login-btn"<?> !$registrationReady ? ' disabled' : '' ?>>
        <i class="fas fa-user-plus"></i> Submit registration
    </button>
    <p class="register-note">Your Dean will review your registration before you can sign in.</p>
</form>

    <p class="portal-footer">Western Philippines University · WPU SABLA ePortal</p>
</div>
</div>

<script>
(function () {
    var roleSubtitles = <?> json_encode(array_map(static fn ($r) => $r['subtitle'], $loginRoles), JSON_HEX_TAG | JSON_HEX_APOS |
JSON_HEX_AMP | JSON_UNESCAPED_UNICODE) ?>;
    var roleAccents = <?> json_encode(array_map(static function ($r) {
        return [
            'accent' => $r['accent'] ?? '#4f46e5',
            'soft' => $r['accent_bg'] ?? '#eef2ff',
            'border' => $r['accent_border'] ?? '#c7d2fe',

```

```

    ];
    }, $loginRoles), JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_AMP) ?>;
    var selectedRole = <?=json_encode($selectedRole, JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_AMP) ?>;
    var viewMode = <?=json_encode($viewMode, JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_AMP) ?>;

    var roleTabs = document.getElementById('roleTabs');
    var loginRoleField = document.getElementById('loginRoleField');
    var authHeading = document.getElementById('authHeading');
    var authSubtitle = document.getElementById('authSubtitle');
    var studentAuthSwitch = document.getElementById('studentAuthSwitch');
    var loginForm = document.getElementById('loginForm');
    var registerForm = document.getElementById('registerForm');
    var collegeSelect = document.getElementById('reg_college_id');
    var programSelect = document.getElementById('reg_program_name');
    var yearLevelSelect = document.getElementById('reg_year_level');
    var yearLevelsByProgram = {};
    var selectedYearInitial = <?=json_encode($regYearLevel, JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_AMP) ?>;

    function setViewMode(mode) {
        viewMode = mode;
        var isRegister = mode === 'register';
        loginForm.classList.toggle('hide', isRegister);
        registerForm.classList.toggle('show', isRegister);
        authHeading.textContent = isRegister ? 'Create student account' : 'Welcome back';
        authSubtitle.textContent = isRegister
            ? 'Register under your college and program. Your Dean must approve before you can sign in. You will receive an email with your
temporary password once approved.'
            : (roleSubtitles[selectedRole] || '');
        document.querySelectorAll('.auth-switch-btn').forEach(function (btn) {
            btn.classList.toggle('active', btn.getAttribute('data-mode') === mode);
        });
    }

    function applyRoleAccent(role) {
        var accent = roleAccents[role] || roleAccents.student || { accent: '#4f46e5', soft: '#eef2ff', border: '#c7d2fe' };
        document.documentElement.style.setProperty('--portal-accent', accent.accent);
        document.documentElement.style.setProperty('--portal-accent-soft', accent.soft);
        document.documentElement.style.setProperty('--portal-accent-border', accent.border);
    }

    function setRole(role) {
        selectedRole = role;
        if (loginRoleField) loginRoleField.value = role;
        document.querySelectorAll('.role-btn').forEach(function (tab) {
            var active = tab.getAttribute('data-role') === role;
            tab.classList.toggle('active', active);
            tab.setAttribute('aria-pressed', active ? 'true' : 'false');
        });
        applyRoleAccent(role);
        if (studentAuthSwitch) {
            studentAuthSwitch.classList.toggle('hidden', role !== 'student');
        }
        if (role !== 'student' && viewMode === 'register') {
            setViewMode('login');
        } else if (viewMode === 'login') {
            authSubtitle.textContent = roleSubtitles[role] || '';
        }
    }

    if (roleTabs) {
        roleTabs.addEventListener('click', function (e) {
            var tab = e.target.closest('.role-btn');
            if (!tab) return;
            setRole(tab.getAttribute('data-role'));
        });
    }

    if (studentAuthSwitch) {
        studentAuthSwitch.addEventListener('click', function (e) {
            var btn = e.target.closest('.auth-switch-btn');
            if (!btn || btn.disabled) return;
            setViewMode(btn.getAttribute('data-mode'));
        });
    }

    function renderYearLevels(selectedProgram, selectedYear) {
        if (!yearLevelSelect) return;
        var list = ['1', '2', '3', '4', '5'];
        if (selectedProgram && yearLevelsByProgram[selectedProgram] && yearLevelsByProgram[selectedProgram].length) {
            list = yearLevelsByProgram[selectedProgram];
        }
    }

```

```

    }
    var html = '<option value="">Select year level</option>';
    list.forEach(function (yl) {
        var selected = selectedYear === yl ? ' selected' : '';
        html += '<option value="' + String(yl).replace(/"/g, '&quot;') + '">' + selected + '>Year ' + yl + '</option>';
    });
    yearLevelSelect.innerHTML = html;
}

function loadPrograms(collegeId, selectedProgram, selectedYear) {
    if (!programSelect || !collegeId) {
        if (programSelect) programSelect.innerHTML = '<option value="">Select program</option>';
        yearLevelsByProgram = {};
        renderYearLevels('', '');
        return;
    }
    programSelect.innerHTML = '<option value="">Loading...</option>';
    if (yearLevelSelect) yearLevelSelect.innerHTML = '<option value="">Loading...</option>';
    fetch('api/programs_by_college.php?college_id=' + encodeURIComponent(collegeId))
        .then(function (res) { return res.json(); })
        .then(function (data) {
            yearLevelsByProgram = data.year_levels_by_program || {};
            var html = '<option value="">Select program</option>';
            (data.programs || []).forEach(function (name) {
                var selected = selectedProgram === name ? ' selected' : '';
                html += '<option value="' + name.replace(/"/g, '&quot;') + '">' + selected + '>' + name + '</option>';
            });
            programSelect.innerHTML = html;
            renderYearLevels(selectedProgram, selectedYear);
        })
        .catch(function () {
            programSelect.innerHTML = '<option value="">Select program</option>';
            yearLevelsByProgram = {};
            renderYearLevels('', '');
        });
}

if (collegeSelect) {
    collegeSelect.addEventListener('change', function () {
        loadPrograms(collegeSelect.value, '', '');
    });
}

if (programSelect) {
    programSelect.addEventListener('change', function () {
        renderYearLevels(programSelect.value, '');
    });
}

if (collegeSelect && collegeSelect.value) {
    loadPrograms(collegeSelect.value, programSelect ? programSelect.value : '', selectedYearInitial);
}

var passwordInput = document.getElementById('password');
if (passwordInput) {
    var wrapperDiv = document.createElement('div');
    wrapperDiv.className = 'password-wrapper';
    passwordInput.parentNode.insertBefore(wrapperDiv, passwordInput);
    wrapperDiv.appendChild(passwordInput);
    var toggleEye = document.createElement('i');
    toggleEye.className = 'far fa-eye-slash pwd-toggle';
    toggleEye.setAttribute('role', 'button');
    toggleEye.setAttribute('tabindex', '0');
    toggleEye.setAttribute('aria-label', 'Toggle password visibility');
    wrapperDiv.appendChild(toggleEye);
    var pwdVisible = false;
    toggleEye.addEventListener('click', function () {
        pwdVisible = !pwdVisible;
        passwordInput.type = pwdVisible ? 'text' : 'password';
        toggleEye.className = pwdVisible ? 'far fa-eye pwd-toggle' : 'far fa-eye-slash pwd-toggle';
    });
}

setRole(selectedRole);
setViewMode(viewMode);

var slideshow = document.getElementById('heroSlideshow');
if (slideshow) {
    var slides = slideshow.querySelectorAll('.hero-slide');
    var dots = slideshow.querySelectorAll('.hero-slideshow-dot');

```

```

var current = 0;
var timer = null;
var intervalMs = 5000;

function showSlide(index) {
    if (!slides.length) return;
    current = (index + slides.length) % slides.length;
    slides.forEach(function (slide, i) {
        slide.classList.toggle('active', i === current);
    });
    dots.forEach(function (dot, i) {
        dot.classList.toggle('active', i === current);
        dot.setAttribute('aria-selected', i === current ? 'true' : 'false');
    });
}

function nextSlide() {
    showSlide(current + 1);
}

function resetTimer() {
    if (timer) clearInterval(timer);
    if (slides.length > 1) {
        timer = setInterval(nextSlide, intervalMs);
    }
}

dots.forEach(function (dot) {
    dot.addEventListener('click', function () {
        var idx = parseInt(dot.getAttribute('data-slide'), 10);
        if (!isNaN(idx)) {
            showSlide(idx);
            resetTimer();
        }
    });
});

resetTimer();
}
})();
</script>
</body>
</html>

```

FILE: login1.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';

if (!empty($_SESSION['user_id'])) {
    header('Location: dashboard.php');
    exit;
}

$error = '';
$usernameValue = '';
$hasAssignedProgram = false;
try {
    $colStmt = db()->prepare(
        'SELECT COUNT(*) FROM information_schema.COLUMNS
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ? AND COLUMN_NAME = ?'
    );
    $colStmt->execute([DB_NAME, 'users', 'assigned_program']);
    $hasAssignedProgram = (int) $colStmt->fetchColumn() > 0;
} catch (Throwable $e) {
    $hasAssignedProgram = false;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = trim((string) ($_POST['username'] ?? ''));
    $usernameValue = $username;
    $password = (string) ($_POST['password'] ?? '');
    if ($username === '' || $password === '') {
        $error = 'Please enter username and password.';
    } else {
        try {

```

```

$stmt = db()->prepare(
    'SELECT u.id, u.username, u.password, u.full_name, u.role, '
    . ($hasAssignedProgram ? 'u.assigned_program' : '' AS assigned_program') . ', '
    . u.college_id, u.is_active, f.id AS faculty_id
    FROM users u
    LEFT JOIN faculty f ON f.user_id = u.id
    WHERE u.username = ? LIMIT 1'
);
$stmt->execute([$username]);
$user = $stmt->fetch();
if ($user && (int) $user['is_active'] === 1 && password_verify($password, $user['password'])) {
    $resolvedFacultyId = $user['faculty_id'] !== null ? (int) $user['faculty_id'] : null;
    $resolvedStudentId = null;
    if ((string) $user['role'] === 'faculty' && $resolvedFacultyId === null) {
        $resolvedFacultyId = resolve_faculty_id_for_user((int) $user['id']);
    }
    if ((string) $user['role'] === 'student') {
        $resolvedStudentId = resolve_student_id_for_user((int) $user['id']);
    }
    $_SESSION['user_id'] = (int) $user['id'];
    $_SESSION['username'] = $user['username'];
    $_SESSION['full_name'] = $user['full_name'];
    $_SESSION['role'] = $user['role'];
    $_SESSION['assigned_program'] = (string) ($user['assigned_program'] ?? '');
    $_SESSION['college_id'] = $user['college_id'] !== null ? (int) $user['college_id'] : null;
    $_SESSION['faculty_id'] = $resolvedFacultyId;
    $_SESSION['student_id'] = $resolvedStudentId;
    try {
        if (db_column_exists('users', 'last_logout_at')) {
            db()->prepare('UPDATE users SET last_login_at = NOW(), last_seen_at = NOW(), last_logout_at = NULL WHERE id = ?')
                ->execute([(int) $user['id']]);
        } else {
            db()->prepare('UPDATE users SET last_login_at = NOW(), last_seen_at = NOW() WHERE id = ?')
                ->execute([(int) $user['id']]);
        }
    } catch (Throwable $e) {
        // Ignore if legacy schema does not yet have activity columns.
    }
    header('Location: dashboard.php');
    exit;
}
$error = $user && (int) $user['is_active'] !== 1
    ? 'Your account is disabled. Please contact your dean/admin.'
    : 'Invalid username or password.';
} catch (Throwable $e) {
    $prev = $e->getPrevious();
    $error = $e instanceof RuntimeException && $prev instanceof PDOException
        ? $e->getMessage()
        : 'Database error: ' . $e->getMessage();
}
}
}

$sealDataUri = '';
$sealCandidates = [
    __DIR__ . '/assets/images/wpu-seal.png',
    'C:\Users\wpu-0\cursor\projects\c-xampp\htdocs\CLASS\assets\c_Users_wpu-0_AppData_Roaming_Cursor_User_workspaceStorage_f94ec644ac39e85d4f1f3f887fe3104f_images_A_LOGO-ba32c997-80f6-4298-97e0-4ca3b192b5d0.png',
];
foreach ($sealCandidates as $sealPath) {
    if (is_file($sealPath)) {
        $sealRaw = @file_get_contents($sealPath);
        if ($sealRaw !== false) {
            $sealDataUri = 'data:image/png;base64,' . base64_encode($sealRaw);
            break;
        }
    }
}
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login - WPU SABLAe Portal</title>
    <?php require __DIR__ . '/includes/favicon.php'; ?>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css" rel="stylesheet">
    <style>
        :root {

```

```

--hero-dark: #082032;
--hero-green: #1f6f43;
--hero-light: #f4f8f6;
--hero-card: rgba(255, 255, 255, 0.92);
--hero-shadow: 0 24px 60px rgba(8, 32, 50, 0.22);
}

body.login-page {
  min-height: 100vh;
  margin: 0;
  background:
    radial-gradient(circle at top left, rgba(255, 255, 255, 0.22), transparent 28%),
    radial-gradient(circle at bottom right, rgba(56, 161, 105, 0.24), transparent 25%),
    linear-gradient(135deg, #061826 0%, var(--hero-dark) 42%, #114b2b 100%);
}

.login-shell {
  min-height: 100vh;
  display: flex;
  align-items: center;
  padding: 32px 0;
}

.login-panel {
  overflow: hidden;
  border: 0;
  border-radius: 28px;
  background: var(--hero-card);
  box-shadow: var(--hero-shadow);
  backdrop-filter: blur(10px);
}

.login-brand {
  height: 100%;
  color: #fff;
  background:
    linear-gradient(180deg, rgba(255, 255, 255, 0.08), rgba(255, 255, 255, 0)),
    linear-gradient(145deg, #0a2235 0%, #12394d 45%, #1f6f43 100%);
  padding: 48px 38px;
  position: relative;
}

.login-brand::after {
  content: "";
  position: absolute;
  inset: auto -70px -70px auto;
  width: 180px;
  height: 180px;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.08);
  filter: blur(2px);
}

.seal-wrap {
  width: clamp(168px, 38vw, 220px);
  height: clamp(168px, 38vw, 220px);
  flex-shrink: 0;
  border-radius: 50%;
  margin-left: auto;
  margin-right: auto;
  margin-bottom: 28px;
  display: flex;
  align-items: center;
  justify-content: center;
  background: rgba(255, 255, 255, 0.14);
  border: 1px solid rgba(255, 255, 255, 0.28);
  box-shadow: 0 16px 35px rgba(0, 0, 0, 0.18);
}

.login-brand .seal-wrap {
  width: min(82%, 320px);
  height: min(82%, 320px);
  max-width: 320px;
  max-height: 320px;
  margin-bottom: 32px;
}

.seal-wrap img {
  width: 90%;
  height: 90%;
}

```

```

    max-width: 100%;
    max-height: 100%;
    object-fit: contain;
    border-radius: 50%;
    background: #fff;
    padding: 8px;
}

.seal-fallback {
    font-size: 3rem;
    color: #fff;
}

.brand-chip {
    display: inline-flex;
    align-items: center;
    gap: 8px;
    padding: 8px 14px;
    border-radius: 999px;
    background: rgba(255, 255, 255, 0.12);
    border: 1px solid rgba(255, 255, 255, 0.18);
    font-size: 0.85rem;
    letter-spacing: 0.04em;
    text-transform: uppercase;
}

.brand-title {
    font-size: clamp(1.8rem, 4vw, 2.5rem);
    font-weight: 700;
    line-height: 1.15;
    margin: 16px 0 12px;
}

.brand-text {
    max-width: 440px;
    color: rgba(255, 255, 255, 0.78);
    font-size: 1rem;
    margin-bottom: 0;
}

.login-form-side {
    padding: 42px 38px;
    background: linear-gradient(180deg, rgba(255,255,255,0.96), rgba(246,249,247,0.98));
}

.login-kicker {
    color: var(--hero-green);
    font-weight: 700;
    letter-spacing: 0.08em;
    text-transform: uppercase;
    font-size: 0.78rem;
    margin-bottom: 8px;
}

.login-heading {
    font-size: 1.95rem;
    font-weight: 700;
    color: #163043;
    margin-bottom: 8px;
}

.login-subtitle {
    color: #6c7a86;
    margin-bottom: 28px;
}

.input-group-text {
    border-radius: 16px 0 0 16px;
    border-right: 0;
    background: #f3f6f5;
    color: #5a6874;
}

.form-control {
    min-height: 52px;
    border-radius: 0 16px 16px 0;
    border-left: 0;
    box-shadow: none;
    background: #fbfcfc;
}

```

```

.form-control:focus {
  border-color: #9bcb00;
  box-shadow: 0 0 0 0.2rem rgba(31, 111, 67, 0.12);
}

.field-stack .form-control,
.field-stack .input-group-text {
  border-color: #d8e4dd;
}

.btn-login {
  min-height: 54px;
  border: 0;
  border-radius: 16px;
  background: linear-gradient(135deg, #1f6f43 0%, #2f8e57 100%);
  box-shadow: 0 14px 28px rgba(31, 111, 67, 0.22);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.btn-login:hover {
  transform: translateY(-1px);
  box-shadow: 0 18px 30px rgba(31, 111, 67, 0.26);
}

.login-note {
  color: #6c7a86;
}

.login-note a {
  color: var(--hero-green);
  font-weight: 600;
  text-decoration: none;
}

.login-note a:hover {
  text-decoration: underline;
}

@media (max-width: 991.98px) {
  .login-brand,
  .login-form-side {
    padding: 32px 26px;
  }

  .login-form-side .seal-wrap {
    width: clamp(152px, 44vw, 220px);
    height: clamp(152px, 44vw, 220px);
  }
}
</style>
</head>
<body class="login-page">
<div class="container login-shell">
  <div class="login-panel w-100">
    <div class="row g-0 align-items-stretch">
      <div class="col-lg-6 d-none d-lg-block">
        <div class="login-brand d-flex flex-column justify-content-center">
          <div class="seal-wrap">
            <?php if ($sealDataUri !== ''): ?>
              
            <?php else: ?>
              <span class="seal-fallback"><i class="fa-solid fa-building-columns"></i></span>
            <?php endif; ?>
          </div>
          <div class="brand-chip">
            <i class="fa-solid fa-shield-halved"></i>
            SAFE WPU Scheduling and LMS Portal
          </div>
          <h1 class="brand-title">WPU SABLAE Portal</h1>
          <p class="brand-text">A smoother way to manage academic schedules, faculty coordination, and college operations in one
secure workspace.</p>
        </div>
      </div>
      <div class="col-lg-6">
        <div class="login-form-side h-100 d-flex flex-column justify-content-center">
          <div class="d-lg-none text-center mb-4">
            <div class="seal-wrap mx-auto">
              <?php if ($sealDataUri !== ''): ?>
                
              <?php else: ?>
                <span class="seal-fallback"><i class="fa-solid fa-building-columns"></i></span>
              <?php endif; ?>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</body>

```



```

        <?php else: ?>
            <span class="seal-fallback text-dark"><i class="fa-solid fa-building-columns"></i></span>
        <?php endif; ?>
    </div>
</div>
<div class="login-kicker">Welcome back</div>
<h2 class="login-heading">Sign in to continue</h2>
<p class="login-subtitle">Access faculty schedules, reports, announcements, and academic management tools.</p>

<?php if ($error): ?>
    <div class="alert alert-danger rounded-4"><?= htmlspecialchars($error) ?></div>
<?php endif; ?>

<form method="post" autocomplete="on">
    <div class="mb-3 field-stack">
        <label class="form-label fw-semibold" for="username">Username</label>
        <div class="input-group">
            <span class="input-group-text"><i class="fa-solid fa-user"></i></span>
            <input type="text" name="username" id="username" class="form-control" value="<?=
htmlspecialchars($usernameValue) ?>" required autofocus>
        </div>
    </div>
    <div class="mb-4 field-stack">
        <label class="form-label fw-semibold" for="password">Password</label>
        <div class="input-group">
            <span class="input-group-text"><i class="fa-solid fa-lock"></i></span>
            <input type="password" name="password" id="password" class="form-control" required>
        </div>
    </div>
    <button type="submit" class="btn btn-success btn-login w-100">
        <i class="fa-solid fa-right-to-bracket me-2"></i>Login
    </button>
</form>

<p class="login-note text-center small mt-4 mb-0">First time setup? Run <a href="install.php">install.php</a> to create
the database.</p>
    </div>
</div>
</div>
</div>
</div>
</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

FILE: login_slideshow_image.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/login_slideshow_helpers.php';

$id = isset($_GET['id']) ? (int) $_GET['id'] : 0;
if ($id < 1) {
    http_response_code(404);
    echo 'Image not found.';
    exit;
}

$row = login_slideshow_image_row($id);
if ($row === null) {
    http_response_code(404);
    echo 'Image not found.';
    exit;
}

$isActive = (int) ($row['is_active'] ?? 0) === 1;
$isAdminPreview = !empty($_SESSION['user_id']) && in_array((string) ($_SESSION['role'] ?? ''), ['admin', 'super_admin'], true);
if (!$isActive && !$isAdminPreview) {
    http_response_code(404);
    echo 'Image not found.';
    exit;
}

$stored = trim((string) ($row['stored_name'] ?? ''));
$path = login_slideshow_path($stored);
if ($stored === '' || !is_file($path)) {

```

```

    http_response_code(404);
    echo 'Image file is missing.';
    exit;
}

$mime = login_slideshow_mime_for_stored($stored);
if ($mime === 'application/octet-stream') {
    http_response_code(404);
    echo 'Image not found.';
    exit;
}

header('Content-Type: ' . $mime);
header('Content-Length: ' . (string) filesize($path));
header('Cache-Control: public, max-age=3600');
header('X-Content-Type-Options: nosniff');
readfile($path);
exit;

-----
FILE: logout.php
-----
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';

$userId = isset($_SESSION['user_id']) ? (int) $_SESSION['user_id'] : 0;
$logoutUsername = (string) ($_SESSION['username'] ?? '');
$logoutRole = (string) ($_SESSION['role'] ?? '');
if ($userId > 0) {
    try {
        $nowTs = date('Y-m-d H:i:s');
        if (db_column_exists('users', 'last_logout_at')) {
            db()->prepare('UPDATE users SET last_logout_at = ?, last_seen_at = ? WHERE id = ?')
                ->execute([$nowTs, $nowTs, $userId]);
        } elseif (db_column_exists('users', 'last_seen_at')) {
            db()->prepare('UPDATE users SET last_seen_at = ? WHERE id = ?')->execute([$nowTs, $userId]);
        }

        // If attendance has already been generated for today's classes, capture logout time on those records.
        if (
            db_table_exists('classroom_attendance_sessions')
            && db_table_exists('classroom_attendance_records')
            && db_table_exists('classroom_students')
            && db_column_exists('classroom_attendance_records', 'evidence_logout_at')
            && db_column_exists('classroom_students', 'user_id')
        ) {
            db()->prepare(
                'UPDATE classroom_attendance_records ar
                INNER JOIN classroom_attendance_sessions s ON s.id = ar.session_id
                INNER JOIN classroom_students cs ON cs.id = ar.student_id
                SET ar.evidence_logout_at = ?, ar.checked_at = ?, ar.updated_at = CURRENT_TIMESTAMP,
                    ar.notes = CONCAT(TRIM(ar.notes), CASE WHEN TRIM(ar.notes) = "" THEN "" ELSE " " END, "Logout captured at sign-out.")
                WHERE cs.user_id = ?
                    AND s.attendance_date = ?
                    AND ar.evidence_logout_at IS NULL'
            )->execute([$nowTs, $nowTs, $userId, date('Y-m-d')]);
        }
    } catch (Throwable $e) {
        // Best effort logout timestamp update.
    }
    require_once __DIR__ . '/includes/admin_activity_log.php';
    log_user_activity(
        'logout',
        'Authentication',
        'Signed out',
        null,
        null,
        $userId,
        $logoutUsername,
        $logoutRole
    );
}

$_SESSION = [];
if (ini_get('session.use_cookies')) {
    $p = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000, $p['path'], $p['domain'], $p['secure'], $p['httponly']);
}

```

```

}
session_destroy();
header('Location: login.php');
exit;

```

FILE: messages.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/messaging_helpers.php';
require_once __DIR__ . '/includes/classroom_discussion_helpers.php';

require_role(['admin', 'dean', 'program_chair', 'gened', 'faculty', 'student']);

$usesClassroomDiscussions = in_array((string) ($_SESSION['role'] ?? ''), ['faculty', 'student'], true);
/** Faculty also receive dean/admin memos in internal_messages; badge counts those but classroom UI hid them. */
$showFacultyInbox = is_faculty() && messaging_table_exists();

if (!$usesClassroomDiscussions && !messaging_table_exists()) {
    http_response_code(503);
    echo '<!DOCTYPE html><html><head><meta charset="utf-8"><title>Messages</title></head><body class="p-4">';
    echo '<p>Messaging is not installed. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once.</p>';
    echo '</body></html>';
    exit;
}

$userId = (int) $_SESSION['user_id'];
$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    if ($usesClassroomDiscussions && $action === 'post_classroom_message') {
        $classroomId = (int) ($_POST['classroom_id'] ?? 0);
        $body = (string) ($_POST['body'] ?? '');
        try {
            classroom_discussion_post($classroomId, $userId, $body);
            $_SESSION['flash'] = 'Message sent to the class conversation.';
            header('Location: messages.php?classroom=' . $classroomId);
            exit;
        } catch (Throwable $e) {
            $_SESSION['flash'] = 'Error: ' . $e->getMessage();
            header('Location: messages.php' . ($classroomId > 0 ? '?classroom=' . $classroomId : ''));
            exit;
        }
    }
    if ($action === 'send' && (!$usesClassroomDiscussions || $showFacultyInbox)) {
        $toId = (int) ($_POST['recipient_id'] ?? 0);
        $body = (string) ($_POST['body'] ?? '');
        try {
            messaging_send($userId, $toId, $body);
            $_SESSION['flash'] = 'Message sent.';
            header('Location: messages.php?with=' . $toId);
            exit;
        } catch (Throwable $e) {
            $_SESSION['flash'] = 'Error: ' . $e->getMessage();
            header('Location: messages.php' . ($toId > 0 ? '?with=' . $toId : ''));
            exit;
        }
    }
    if (!$usesClassroomDiscussions && $action === 'delete_message') {
        $messageId = (int) ($_POST['message_id'] ?? 0);
        $returnWith = (int) ($_POST['with'] ?? 0);
        try {
            if (!messaging_delete_message($messageId, $userId)) {
                throw new RuntimeException('Message not found or you cannot delete it.');
            }
            $_SESSION['flash'] = 'Message deleted.';
        } catch (Throwable $e) {
            $_SESSION['flash'] = 'Error: ' . $e->getMessage();
        }
        header('Location: messages.php' . ($returnWith > 0 ? '?with=' . $returnWith : ''));
        exit;
    }
    if (!$usesClassroomDiscussions && $action === 'send_memo') {
        $recipientIds = $_POST['recipient_ids'] ?? [];
    }
}

```

```

$subject = (string) ($_POST['subject'] ?? '');
$body = (string) ($_POST['body'] ?? '');
$attachment = isset($_FILES['attachment']) && is_array($_FILES['attachment']) ? $_FILES['attachment'] : null;
try {
    $sentCount = messaging_send_memo($userId, is_array($recipientIds) ? $recipientIds : [], $subject, $body, $attachment);
    $_SESSION['flash'] = $sentCount === 1 ? 'Memo sent to 1 faculty member.' : 'Memo sent to ' . $sentCount . ' faculty members.';
    $firstRecipient = is_array($recipientIds) && $recipientIds !== [] ? (int) $recipientIds[0] : 0;
    header('Location: messages.php' . ($firstRecipient > 0 ? '?with=' . $firstRecipient : ''));
    exit;
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
    header('Location: messages.php');
    exit;
}
}

$allowInternalThreads = !$usesClassroomDiscussions || $showFacultyInbox;
$withId = $allowInternalThreads ? (int) ($_GET['with'] ?? 0) : 0;
$threadUser = $allowInternalThreads && $withId > 0 ? messaging_user_row($withId) : null;
$canOpen = $allowInternalThreads && $withId > 0 && $threadUser && messaging_can_open_thread($userId, $withId);
if ($canOpen) {
    messaging_mark_read($userId, $withId);
}

$classroomId = $usesClassroomDiscussions ? (int) ($_GET['classroom'] ?? 0) : 0;
$classroomThreads = $usesClassroomDiscussions ? classroom_discussion_threads_for_user($userId) : [];
$activeClassroom = $usesClassroomDiscussions && $classroomId > 0 ? classroom_discussion_thread_row($classroomId, $userId) : null;
$classroomMessages = $activeClassroom ? classroom_discussion_messages($classroomId, $userId) : [];
$hasClassroomDiscussionTable = classroom_discussions_table_exists();

$conversations = $allowInternalThreads ? messaging_conversation_list($userId) : [];
$inboxRecipients = $showFacultyInbox ? messaging_faculty_inbox_recipients($userId) : [];
$recipients = !$usesClassroomDiscussions ? messaging_allowed_recipients($userId) : [];
$memoRecipients = !$usesClassroomDiscussions ? messaging_memo_recipients($userId) : [];
$thread = $canOpen ? messaging_thread($userId, $withId) : [];
$showInternalThread = $canOpen && $threadUser;
$canSendTo = $withId > 0 && $threadUser && ($showInternalThread || !$usesClassroomDiscussions)
    && messaging_can_send($userId, $withId);
$canSendMemo = !$usesClassroomDiscussions && in_array((string) ($_SESSION['role'] ?? ''), ['admin', 'dean', 'gened', 'program_chair'], true);
$memoReady = !$usesClassroomDiscussions && messaging_has_memo_columns();

$pageTitle = 'Messages';
require_once __DIR__ . '/includes/header.php';

$invalidWith = $withId > 0 && (!$threadUser || !$canOpen);
$invalidClassroom = $usesClassroomDiscussions && $classroomId > 0 && !$activeClassroom;
$studentMessagesThreadOpen = is_student()
    && $usesClassroomDiscussions
    && $activeClassroom !== null
    && !$invalidClassroom;
$messagesLayoutClass = 'row g-3 student-messages-layout'
    . ($studentMessagesThreadOpen ? ' student-messages-thread-open' : '');
?>
<h1 class="h3 mb-3"><i class="fa-solid fa-comments me-2 text-primary"></i>Messages</h1>
<p class="text-muted small mb-3">
    <?php if (is_admin()): ?>
        You can message <strong>deans</strong>, <strong>program chairs</strong>, and <strong>GEN ED</strong> coordinators. You can also send
        memo announcements with attachments to <strong>faculty</strong>.
    <?php elseif (is_dean()): ?>
        Message <strong>faculty</strong> in your college, <strong>GEN ED</strong>, and <strong>admin</strong> (faculty can reply). Memo
        announcements can be sent to your college faculty.
    <?php elseif (is_program_chair()): ?>
        Message your college <strong>dean</strong>, <strong>admin</strong>, and <strong>faculty</strong> in your assigned program (faculty
        can reply). Memo announcements can be sent to your program faculty.
    <?php elseif (is_faculty()): ?>
        Use <strong>Inbox</strong> to message your dean, program chair, or <strong>GEN ED</strong> coordinator. <strong>Subjects:</strong>
        class conversations with students.
    <?php elseif (is_gened()): ?>
        Message <strong>deans</strong>, <strong>admin</strong>, and <strong>GE faculty</strong> (faculty can reply). Memo announcements can
        be sent to GE faculty.
    <?php elseif (is_student()): ?>
        Choose one of your enrolled <strong>subjects</strong> to reply in the shared class conversation.
    <?php endif; ?>
</p>
<?php if (messaging_thread_max_messages() > 0): ?>
    <p class="text-muted small mb-3"><i class="fa-solid fa-layer-group me-1"></i>Each conversation keeps the latest <strong><?= (int)
        messaging_thread_max_messages() ?></strong> messages; older ones are removed automatically (FIFO).</p>
<?php endif; ?>

```

```

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
<?php if ($invalidWith): ?>
    <div class="alert alert-warning">You cannot open that conversation.</div>
<?php endif; ?>
<?php if ($invalidClassroom): ?>
    <div class="alert alert-warning">You cannot open that subject conversation.</div>
<?php endif; ?>
<?php if ($usesClassroomDiscussions && !$hasClassroomDiscussionTable): ?>
    <div class="alert alert-warning py-2">Classroom discussion is not installed yet. Run <code>upgrade_roles.php</code> once.</div>
<?php endif; ?>
<?php if ($canSendMemo && !$memoReady): ?>
    <div class="alert alert-warning py-2">Memo attachments are not installed yet. Run <code>upgrade_roles.php</code> once to enable memo
fields.</div>
<?php endif; ?>

<div class="<?php htmlspecialchars($messagesLayoutClass, ENT_QUOTES, 'UTF-8') ?>">
    <div class="col-lg-4 student-messages-sidebar">
        <div class="card shadow-sm h-100">
            <div class="card-header bg-white d-flex justify-content-between align-items-center flex-wrap gap-2">
                <strong><?php $usesClassroomDiscussions ? 'Subjects' : 'Conversations' ?></strong>
            </div>
            <div class="list-group list-group-flush" style="max-height: 70vh; overflow-y: auto;">
                <?php if ($showFacultyInbox): ?>
                    <div class="list-group-item list-group-item-secondary py-1 small fw-semibold text-uppercase">Inbox</div>
                    <?php if (!$conversations): ?>
                        <div class="list-group-item text-muted small">No messages from administration yet.</div>
                    <?php endif; ?>
                    <?php foreach ($conversations as $c): ?>
                        <a class="list-group-item list-group-item-action d-flex justify-content-between align-items-start <?php $withId ===
(int) $c['partner_id'] ? 'active' : '' ?>"
                            href="messages.php?with=<?php (int) $c['partner_id'] ?>"<?php app_tooltip_attr('Opens your inbox thread with this
dean, program chair, or sender. Unread items clear after you open it.') ?>>
                            <div class="me-2">
                                <div class="fw-semibold"><?php htmlspecialchars($c['full_name']) ?></div>
                                <div class="small opacity-75 text-truncate" style="max-width: 220px;"><?php htmlspecialchars($c['preview']) !==
'' ? $c['preview'] : 'Open conversation' ?></div>
                            </div>
                            <?php if ($c['unread'] > 0): ?>
                                <span class="badge bg-danger rounded-pill"><?php (int) $c['unread'] ?></span>
                            <?php endif; ?>
                        </a>
                    <?php endforeach; ?>
                    <div class="list-group-item list-group-item-secondary py-1 small fw-semibold text-uppercase">Subjects</div>
                <?php endif; ?>
                <?php if ($usesClassroomDiscussions): ?>
                    <?php if (!$classroomThreads): ?>
                        <div class="list-group-item text-muted small">No subject conversations available yet.</div>
                    <?php endif; ?>
                    <?php foreach ($classroomThreads as $classThread): ?>
                        <a class="list-group-item list-group-item-action <?php $classroomId === (int) $classThread['classroom_id'] ? 'active'
: '' ?>"
                            href="messages.php?classroom=<?php (int) $classThread['classroom_id'] ?>"<?php app_tooltip_attr(is_student() ? 'Opens
the shared class conversation for this subject. Use this to read and send messages with your instructor and classmates.' : 'Opens the shared
class conversation for this subject. Use this to coordinate with enrolled students and track the thread.') ?>>
                            <div class="fw-semibold">
                                <?php htmlspecialchars((string) $classThread['course_code']) ?>
                                <?php if (trim((string) $classThread['title']) !== ''): ?>
                                    <span class="small fw-normal">· <?php htmlspecialchars((string) $classThread['title']) ?></span>
                                <?php endif; ?>
                            </div>
                            <div class="small opacity-75"><?php htmlspecialchars((string) $classThread['course_name']) ?></div>
                            <div class="small opacity-75 text-truncate" style="max-width: 240px;">
                                <?php htmlspecialchars($classThread['preview']) !== '' ? $classThread['preview'] : 'Open this subject
conversation' ?>
                            </div>
                        </a>
                    <?php endforeach; ?>
                <?php else: ?>
                    <?php if (!$conversations): ?>
                        <div class="list-group-item text-muted small">No messages yet. Start below.</div>
                    <?php endif; ?>
                    <?php foreach ($conversations as $c): ?>
                        <a class="list-group-item list-group-item-action d-flex justify-content-between align-items-start <?php $withId ===
(int) $c['partner_id'] ? 'active' : '' ?>"
                            href="messages.php?with=<?php (int) $c['partner_id'] ?>"<?php app_tooltip_attr(is_student() ? 'Opens your private
message thread with this partner. Use this to read replies and continue the conversation.' : 'Opens your direct message thread with this
colleague. Use this to read replies and continue the conversation.') ?>>
                            <div class="me-2">
                                <div class="fw-semibold"><?php htmlspecialchars($c['full_name']) ?></div>
                                <div class="small opacity-75 text-truncate" style="max-width: 220px;"><?php htmlspecialchars($c['preview']) ?>

```

```

</div>
        </div>
        <?php if ($c['unread'] > 0): ?>
            <span class="badge bg-danger rounded-pill"><?= (int) $c['unread'] ?></span>
        <?php endif; ?>
    </a>
    <?php endforeach; ?>
<?php endif; ?>
</div>
<div class="card-body border-top bg-body-tertiary">
    <?php if ($showFacultyInbox): ?>
        <label class="form-label small text-muted mb-1"><?= messaging_is_gened_faculty_user($userId) ? 'Message Dean Francis
Alterado or GEN ED' : 'Message dean, program chair, or GEN ED' ?></label>
        <form method="get" class="d-flex gap-2 flex-wrap mb-3">
            <select name="with" class="form-select form-select-sm" onchange="if(this.value) window.location='messages.php?
with='+this.value">
                <option value="">-- Select recipient --</option>
                <?php foreach ($inboxRecipients as $r): ?>
                    <option value="<?= (int) $r['id'] ?>" <?= $withId === (int) $r['id'] ? 'selected' : '' ?>>
                        <?= htmlspecialchars($r['full_name']) ?> (<?= htmlspecialchars(strtoupper((string) $r['role'])) ?>)
                    </option>
                <?php endforeach; ?>
            </select>
        </form>
        <?php if (!$inboxRecipients): ?>
            <p class="small text-muted mb-0">No dean, program chair, or GEN ED contact is available for your account.</p>
        <?php endif; ?>
    <?php endif; ?>
    <?php if ($usesClassroomDiscussions): ?>
        <label class="form-label small text-muted mb-1">Open subject conversation</label>
        <form method="get" class="d-flex gap-2 flex-wrap">
            <select name="classroom" class="form-select form-select-sm" onchange="if(this.value) window.location='messages.php?
classroom='+this.value">
                <option value="">-- Select subject --</option>
                <?php foreach ($classroomThreads as $classThread): ?>
                    <option value="<?= (int) $classThread['classroom_id'] ?>" <?= $classroomId === (int)
$classThread['classroom_id'] ? 'selected' : '' ?>>
                        <?= htmlspecialchars((string) $classThread['course_code']) ?> - <?= htmlspecialchars((string)
$classThread['course_name']) ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </form>
        <?php if (!$classroomThreads): ?>
            <p class="small text-muted mb-0 mt-2">
                <?= is_faculty() ? 'No subject conversations are available for the classes you handle.' : 'No subject
conversations are available for your enrolled classes.' ?>
            </p>
        <?php endif; ?>
    <?php else: ?>
        <label class="form-label small text-muted mb-1">New message to</label>
        <form method="get" class="d-flex gap-2 flex-wrap">
            <select name="with" class="form-select form-select-sm" onchange="if(this.value) window.location='messages.php?
with='+this.value">
                <option value="">-- Select recipient --</option>
                <?php foreach ($recipients as $r): ?>
                    <option value="<?= (int) $r['id'] ?>" <?= $withId === (int) $r['id'] ? 'selected' : '' ?>>
                        <?= htmlspecialchars($r['full_name']) ?> (<?= htmlspecialchars(strtoupper((string) $r['role'])) ?>)
                    </option>
                <?php endforeach; ?>
            </select>
        </form>
        <?php if (!$recipients): ?>
            <p class="small text-muted mb-0 mt-2">
                <?= is_student() ? 'No faculty recipients are available for your enrolled subjects.' : 'No recipients available
(check college assignment, classroom enrollment, or active users).' ?>
            </p>
        <?php endif; ?>
    <?php endif; ?>
</div>
</div>
<div class="col-lg-8 student-messages-panel">
    <div class="card shadow-sm">
        <div class="card-header bg-white d-flex justify-content-between align-items-center flex-wrap gap-2">
            <?php if ($studentMessagesThreadOpen): ?>
                <a href="messages.php" class="btn btn-outline-secondary btn-sm student-messages-back"><?= app_tooltip_attr('Returns to
your subject list on phones and small screens.') ?><i class="fa-solid fa-arrow-left"></i> Subjects</a>
            <?php endif; ?>
            <?php if ($showInternalThread): ?>

```

```

        <div>
            <strong><? = htmlspecialchars($threadUser['full_name']) ?></strong>
            <span class="badge bg-<? = htmlspecialchars(messaging_role_badge_class($threadUser['role'])) ?> ms-1"><? =
htmlspecialchars(strtoupper($threadUser['role'])) ?></span>
        </div>
        <?php elseif ($usesClassroomDiscussions && $activeClassroom): ?>
            <div>
                <strong><? = htmlspecialchars($activeClassroom['title']) ?></strong>
                <span class="badge bg-primary ms-1"><? = htmlspecialchars($activeClassroom['course_code']) ?></span>
                <div class="small text-muted mt-1"><? = htmlspecialchars($activeClassroom['course_name']) ?></div>
            </div>
            <?php elseif ($canOpen && $threadUser): ?>
                <div>
                    <strong><? = htmlspecialchars($threadUser['full_name']) ?></strong>
                    <span class="badge bg-<? = htmlspecialchars(messaging_role_badge_class($threadUser['role'])) ?> ms-1"><? =
htmlspecialchars(strtoupper($threadUser['role'])) ?></span>
                </div>
                <?php else: ?>
                    <span class="text-muted"><? = $showFacultyInbox ? 'Select an inbox message or subject' : ($usesClassroomDiscussions ?
'Select a subject conversation' : 'Select a conversation or recipient') ?></span>
                <?php endif; ?>
            </div>
            <?php if ($showInternalThread && !$canSendTo && $thread): ?>
                <div class="alert alert-secondary border-0 rounded-0 mb-0 small py-2">You can read this thread but cannot send new messages
in this direction.</div>
            <?php endif; ?>
            <div class="card-body bg-light" style="min-height: 320px; max-height: 55vh; overflow-y: auto;" id="msgThread">
                <?php if ($showInternalThread): ?>
                    <?php foreach ($thread as $m): ?>
                        <div class="d-flex mb-3 <? = $m['mine'] ? 'justify-content-end' : 'justify-content-start' ?>">
                            <div class="rounded-3 px-3 py-2 shadow-sm <? = $m['mine'] ? 'bg-primary text-white' : 'bg-white border' ?>"
style="max-width: 85%;">
                                <div class="small <? = $m['mine'] ? 'text-white-50' : 'text-muted' ?> mb-1">
                                    <? = $m['mine'] ? 'You' : htmlspecialchars($threadUser['full_name']) ?>
                                    · <? = htmlspecialchars(substr($m['created_at'], 0, 16)) ?>
                                </div>
                                <?php if ((int) $m['is_memo'] === 1): ?>
                                    <div class="mb-2">
                                        <span class="badge bg-warning text-dark me-1">MEMO</span>
                                        <?php if ($m['subject'] !== ''): ?>
                                            <strong><? = htmlspecialchars($m['subject']) ?></strong>
                                        <?php endif; ?>
                                    </div>
                                <?php endif; ?>
                                <?php if (trim((string) $m['body']) !== ''): ?>
                                    <div class="message-body"><? = nl2br(htmlspecialchars($m['body'])) ?></div>
                                <?php endif; ?>
                                <?php if ($m['attachment_original_name'] !== ''): ?>
                                    <div class="mt-2">
                                        <a class="btn btn-sm <? = $m['mine'] ? 'btn-light' : 'btn-outline-secondary' ?>"
href="message_attachment.php?id=<? = (int) $m['id'] ?>"><? = app_tooltip_attr('Downloads or opens the file attached to this message.') ?>>
                                        <i class="fa-solid fa-paperclip me-1"></i><? = htmlspecialchars($m['attachment_original_name']) ?>
                                    </a>
                                </div>
                                <?php endif; ?>
                            </div>
                        </div>
                    </div>
                <?php endforeach; ?>
                <?php if ($thread === []): ?>
                    <p class="text-muted text-center py-5 mb-0">No messages in this conversation yet.</p>
                <?php endif; ?>
            <?php elseif ($usesClassroomDiscussions && $activeClassroom): ?>
                <?php if ($classroomMessages === []): ?>
                    <p class="text-muted text-center py-5 mb-0">No messages yet in this subject conversation.</p>
                <?php else: ?>
                    <?php foreach ($classroomMessages as $message): ?>
                        <div class="d-flex mb-3 <? = $message['mine'] ? 'justify-content-end' : 'justify-content-start' ?>">
                            <div class="rounded-3 px-3 py-2 shadow-sm <? = $message['mine'] ? 'bg-primary text-white' : 'bg-white border'
?>" style="max-width: 85%;">
                                <div class="small <? = $message['mine'] ? 'text-white-50' : 'text-muted' ?> mb-1">
                                    <? = $message['mine'] ? 'You' : htmlspecialchars((string) $message['full_name']) ?>
                                    <span class="badge bg-<? = htmlspecialchars(classroom_discussion_role_badge_class((string)
$message['role'])) ?> ms-1"><? = htmlspecialchars(strtoupper((string) $message['role'])) ?></span>
                                    · <? = htmlspecialchars(substr((string) $message['created_at'], 0, 16)) ?>
                                </div>
                                <div class="message-body"><? = nl2br(htmlspecialchars((string) $message['body'])) ?></div>
                            </div>
                        </div>
                    <?php endforeach; ?>
                </div>
            <?php endforeach; ?>

```

```

        <?php endif; ?>
    <?php elseif ($canOpen && $threadUser): ?>
        <?php foreach ($thread as $m): ?>
            <div class="d-flex mb-3 <?= $m['mine'] ? 'justify-content-end' : 'justify-content-start' ?>">
                <div class="rounded-3 px-3 py-2 shadow-sm <?= $m['mine'] ? 'bg-primary text-white' : 'bg-white border' ?>"
                    style="max-width: 85%;">
                    <div class="small <?= $m['mine'] ? 'text-white-50' : 'text-muted' ?> mb-1">
                        <?= $m['mine'] ? 'You' : htmlspecialchars($threadUser['full_name']) ?>
                        . <?= htmlspecialchars(substr($m['created_at'], 0, 16)) ?>
                    </div>
                    <?php if ((int) $m['is_memo'] === 1): ?>
                        <div class="mb-2">
                            <span class="badge bg-warning text-dark me-1">MEMO</span>
                            <?php if ($m['subject'] !== ''): ?>
                                <strong><?= htmlspecialchars($m['subject']) ?></strong>
                            <?php endif; ?>
                        </div>
                    <?php endif; ?>
                    <?php if (trim((string) $m['body']) !== ''): ?>
                        <div class="message-body"><?= nl2br(htmlspecialchars($m['body'])) ?></div>
                    <?php endif; ?>
                    <?php if ($m['attachment_original_name'] !== ''): ?>
                        <div class="mt-2">
                            <a class="btn btn-sm <?= $m['mine'] ? 'btn-light' : 'btn-outline-secondary' ?>"
                                href="message_attachment.php?id=<?= (int) $m['id'] ?>"<?= app_tooltip_attr('Downloads or opens the file attached to this message. Use this
                                to view documents shared in this conversation.') ?>>
                                <i class="fa-solid fa-paperclip me-1"></i><?= htmlspecialchars($m['attachment_original_name']) ?
                            </a>
                        </div>
                    <?php endif; ?>
                    <?php if ($m['mine']): ?>
                        <form method="post" class="mt-2 text-end" onsubmit="return confirm('Delete this message?');">
                            <input type="hidden" name="action" value="delete_message">
                            <input type="hidden" name="message_id" value="<?= (int) $m['id'] ?>">
                            <input type="hidden" name="with" value="<?= $withId ?>">
                            <button type="submit" class="btn btn-sm btn-outline-light"<?= app_tooltip_attr('Removes this message
                                from the thread after you confirm. Use this only for messages you sent by mistake.') ?>>
                                <i class="fa-solid fa-trash me-1"></i>Delete
                            </button>
                        </form>
                    <?php endif; ?>
                </div>
            <?php endforeach; ?>
        <?php else: ?>
            <p class="text-muted text-center py-5 mb-0"><?= $showFacultyInbox ? 'Choose an inbox sender or a subject from the left.'
: ($usesClassroomDiscussions ? 'Choose a subject from the left to open the shared conversation.' : 'Choose someone from the left to read or
send messages.') ?></p>
        <?php endif; ?>
    </div>
    <?php if ($showInternalThread && $canSendTo): ?>
        <div class="card-footer bg-white">
            <form method="post" class="d-flex flex-column gap-2">
                <input type="hidden" name="action" value="send">
                <input type="hidden" name="recipient_id" value="<?= $withId ?>">
                <label class="visually-hidden" for="msgBodyInbox">Reply</label>
                <textarea name="body" id="msgBodyInbox" class="form-control" rows="3" placeholder="Type your reply to <?=
htmlspecialchars($threadUser['full_name']) ?>..." required maxlength="8000"></textarea>
                <div class="text-end">
                    <button type="submit" class="btn btn-primary"<?= app_tooltip_attr('Sends your message in this inbox thread. Your
dean, program chair, or GEN ED coordinator will see it under Messages.') ?>><i class="fa-solid fa-paper-plane me-1"></i>Send</button>
                </div>
            </form>
        </div>
    <?php elseif ($usesClassroomDiscussions && $activeClassroom && $hasClassroomDiscussionTable): ?>
        <div class="card-footer bg-white">
            <form method="post" class="d-flex flex-column gap-2">
                <input type="hidden" name="action" value="post_classroom_message">
                <input type="hidden" name="classroom_id" value="<?= $classroomId ?>">
                <label class="visually-hidden" for="msgBody">Message</label>
                <textarea name="body" id="msgBody" class="form-control" rows="3" placeholder="Type your message to this subject
conversation..." required maxlength="8000"></textarea>
                <div class="text-end">
                    <button type="submit" class="btn btn-primary"<?= app_tooltip_attr(is_student() ? 'Posts your message to this
subject's shared conversation. Use this to ask questions or reply after choosing the class on the left.' : 'Posts your message to this
subject's shared conversation. Use this to reply to students or share updates for the selected class.') ?>><i class="fa-solid fa-paper-plane
me-1"></i>Send</button>
                </div>
            </form>
        </div>
    </div>

```



```

        </div>
        <?php elseif ($canOpen && $threadUser && $canSendTo): ?>
            <div class="card-footer bg-white">
                <form method="post" class="d-flex flex-column gap-2">
                    <input type="hidden" name="action" value="send">
                    <input type="hidden" name="recipient_id" value="<?= $withId ?>">
                    <label class="visually-hidden" for="msgBody">Message</label>
                    <textarea name="body" id="msgBody" class="form-control" rows="3" placeholder="Type your message..." required
maxlength="8000"></textarea>
                    <div class="text-end">
                        <button type="submit" class="btn btn-primary"<?= app_tooltip_attr(is_student() ? 'Sends your private message in
this conversation. Use this after you select someone on the left and type your message.' : 'Sends your message in this thread. Use this
after you select a recipient on the left and type your note.') ?><i class="fa-solid fa-paper-plane me-1"></i>Send</button>
                    </div>
                </form>
            </div>
        <?php endif; ?>
    </div>

    <?php if ($canSendMemo): ?>
        <div class="card shadow-sm mt-3">
            <div class="card-header bg-white">
                <strong>Memo / Announcement</strong>
            </div>
            <div class="card-body">
                <?php if (!$memoReady): ?>
                    <p class="text-muted mb-0">Run <code>upgrade_roles.php</code> to enable memo subjects and document attachments.</p>
                <?php elseif (!$memoRecipients): ?>
                    <p class="text-muted mb-0">No faculty recipients are available for your account scope.</p>
                <?php else: ?>
                    <form method="post" enctype="multipart/form-data" class="row g-3">
                        <input type="hidden" name="action" value="send_memo">
                        <div class="col-12">
                            <label class="form-label">Faculty recipients</label>
                            <select name="recipient_ids[]" class="form-select" size="8" multiple required>
                                <?php foreach ($memoRecipients as $r): ?>
                                    <option value="<?= (int) $r['id'] ?>">
                                        <?= htmlspecialchars($r['full_name']) ?><?= $r['department'] !== '' ? ' - ' : '' .
htmlspecialchars($r['department']) : '' ?>
                                    </option>
                                <?php endforeach; ?>
                            </select>
                            <div class="form-text">Hold Ctrl or Cmd to select multiple faculty members.</div>
                        </div>
                        <div class="col-12">
                            <label class="form-label" for="memoSubject">Memo subject</label>
                            <input type="text" name="subject" id="memoSubject" class="form-control" maxlength="255" required>
                        </div>
                        <div class="col-12">
                            <label class="form-label" for="memoBody">Memo details</label>
                            <textarea name="body" id="memoBody" class="form-control" rows="4" maxlength="8000" placeholder="Write the
announcement details here..."></textarea>
                        </div>
                        <div class="col-md-7">
                            <label class="form-label" for="memoAttachment">Attachment</label>
                            <input type="file" name="attachment" id="memoAttachment" class="form-control"
accept=".pdf,.doc,.docx,.xls,.xlsx,.ppt,.pptx,.txt,.csv,.jpg,.jpeg,.png">
                            <div class="form-text">Allowed: PDF, Office documents, text, CSV, JPG, PNG. Max 10 MB.</div>
                        </div>
                        <div class="col-md-5 d-flex align-items-end justify-content-md-end">
                            <button type="submit" class="btn btn-warning"<?= app_tooltip_attr('Sends a memo announcement to the selected
faculty inboxes. Use this for official notices with optional attachments.') ?>>
                                <i class="fa-solid fa-bullhorn me-1"></i>Send memo
                            </button>
                        </div>
                    </form>
                <?php endif; ?>
            </div>
        </div>
    <?php endif; ?>
</div>

<?php if ($showInternalThread || ($usesClassroomDiscussions && $activeClassroom) || ($canOpen && $threadUser)): ?>
    <script>
        document.getElementById('msgThread')?.scrollTo(0, document.getElementById('msgThread').scrollHeight);
    </script>
<?php endif; ?>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: message_attachment.php

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/messaging_helpers.php';

require_login();

$messageId = isset($_GET['id']) ? (int) $_GET['id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
$message = $messageId > 0 ? messaging_message_for_user($messageId, $userId) : null;

if (!$message || trim((string) ($message['attachment_stored_name'] ?? '')) === '') {
    http_response_code(404);
    echo 'Attachment not found.';
    exit;
}

$path = messaging_attachment_path((string) $message['attachment_stored_name']);
if (!is_file($path)) {
    http_response_code(404);
    echo 'Attachment file is missing.';
    exit;
}

$downloadName = basename((string) ($message['attachment_original_name'] ?? 'attachment'));
$mime = trim((string) ($message['attachment_mime'] ?? 'application/octet-stream'));
if ($mime === '') {
    $mime = 'application/octet-stream';
}

header('Content-Type: ' . $mime);
header('Content-Length: ' . (string) filesize($path));
header('Content-Disposition: attachment; filename="' . addslashes($downloadName, '"\'') . '"');
header('X-Content-Type-Options: nosniff');
readfile($path);
exit;
```

FILE: profile_photo.php

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/profile_photo_helpers.php';

require_login();

$userId = isset($_GET['u']) ? (int) $_GET['u'] : (int) ($_SESSION['user_id'] ?? 0);
if ($userId < 1) {
    http_response_code(404);
    echo 'Photo not found.';
    exit;
}

$stored = profile_photo_stored_name($userId);
if ($stored === '') {
    http_response_code(404);
    echo 'Photo not found.';
    exit;
}

$path = profile_photo_path($stored);
if (!is_file($path)) {
    http_response_code(404);
    echo 'Photo file is missing.';
    exit;
}

$mime = profile_photo_mime_for_stored($stored);
if ($mime === 'application/octet-stream') {
    http_response_code(404);
    echo 'Photo not found.';
    exit;
}
```

```

}

header('Content-Type: ' . $mime);
header('Content-Length: ' . (string) filesize($path));
header('Cache-Control: private, max-age=3600');
header('X-Content-Type-Options: nosniff');
readfile($path);
exit;

-----
FILE: program_chairs.php
-----

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/mail_helpers.php';
require_once __DIR__ . '/includes/account_registration_helpers.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['dean']);
$collegeId = dean_college_id_or_fail();
$collegeName = college_name_by_id($collegeId);
$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

$hasUserEmail = db_column_exists('users', 'email');
$hasAssignedProgram = db_column_exists('users', 'assigned_program');
$hasProgramsTable = db_table_exists('programs');
$hasChairProgramsTable = ensure_program_chair_programs_table();

$programOptions = [];
if ($hasProgramsTable) {
    $st = db()->prepare("SELECT program_name FROM programs WHERE college_id=? AND status='active' ORDER BY program_name");
    $st->execute([$collegeId]);
    $programOptions = array_map('strval', $st->fetchAll(PDO::FETCH_COLUMN) ?: []);
}

/**
 * @param mixed $raw
 * @return list<string>
 */
$normalizeAssignedPrograms = static function ($raw) use ($programOptions): array {
    if (!is_array($raw)) {
        $raw = $raw !== null && $raw !== '' ? [$raw] : [];
    }
    $out = [];
    foreach ($raw as $name) {
        $name = trim((string) $name);
        if ($name !== '' && in_array($name, $programOptions, true) && !in_array($name, $out, true)) {
            $out[] = $name;
        }
    }
    return $out;
};

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if (!$hasAssignedProgram) {
            throw new RuntimeException('Run upgrade_roles.php first to enable Program Chair accounts.');
```

```

if ($username === '' || $fullName === '' || $assignedPrograms === []) {
    throw new RuntimeException('Username, full name, and at least one program are required.');
```

```

}
if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    throw new RuntimeException('Invalid email address.');
```

```

}

$primaryProgram = $assignedPrograms[0];
$cred = prepare_new_account_password($password, $hasValidEmail);
$password = $cred['plain'];
$plainForMail = $hasValidEmail ? $cred['plain'] : '';
```

```

$exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username=?');
$exists->execute([$username]);
if ((int) $exists->fetchColumn() > 0) {
    $s = suggest_available_usernames($username, 3);
    throw new RuntimeException('Username already exists.' . ($s ? ' Try: ' . implode(', ', $s) : ''));
}
```

```

$sql = $hasUserEmail
    ? 'INSERT INTO users (username,password,full_name,email,role,assigned_program,college_id,is_active) VALUES
(?,?,?,?,?,?,?,?)'
    : 'INSERT INTO users (username,password,full_name,role,assigned_program,college_id,is_active) VALUES (?,?,?,?,?,?,?)';
$params = $hasUserEmail
    ? [$username, $cred['hash'], $fullName, $email, 'program_chair', $primaryProgram, $collegeId, !empty($_POST['is_active']) ?
1 : 0]
```

```

    : [$username, $cred['hash'], $fullName, 'program_chair', $primaryProgram, $collegeId, !empty($_POST['is_active']) ? 1 : 0];
db()->prepare($sql)->execute($params);
$newPcId = (int) db()->lastInsertId();
program_chair_set_assigned_programs($newPcId, $assignedPrograms, $primaryProgram);
$stmtPc = db()->prepare(
    'SELECT id, username, full_name, email, role, assigned_program, college_id, is_active FROM users WHERE id = ? LIMIT 1'
);
$stmtPc->execute([$newPcId]);
$afterPc = $stmtPc->fetch(PDO::FETCH_ASSOC);
if ($afterPc) {
    $afterPc['password'] = '[set at creation]';
    $afterPc['assigned_programs'] = implode(', ', $assignedPrograms);
}
log_user_activity('add', 'Program chairs', 'Program chair user #' . $newPcId, null, $afterPc ? (array) $afterPc : null);
log_dean_activity('program_chair_create', 'Created program chair for ' . implode(', ', $assignedPrograms));
```

```

$mailOk = $hasValidEmail
    ? notify_new_account_credentials($newPcId, $email, $fullName, $username, $plainForMail, 'program_chair')
    : null;
if (!$hasValidEmail) {
    mark_new_account_requires_password_change($newPcId);
}
```

```

$_SESSION['flash'] = registration_email_flash_message($mailOk, $email, 'Program Chair account');
} elseif ($action === 'edit' && isset($_POST['id'])) {
    $id = (int) $_POST['id'];
    $username = trim((string) ($_POST['username'] ?? ''));
    $fullName = trim((string) ($_POST['full_name'] ?? ''));
    $assignedPrograms = $normalizeAssignedPrograms($_POST['assigned_programs'] ?? []);
    $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
    $resetPassword = trim((string) ($_POST['reset_password'] ?? ''));
    $generateAndEmail = !empty($_POST['generate_temp_password_email']);
```

```

if ($username === '' || $fullName === '' || $assignedPrograms === []) {
    throw new RuntimeException('Username, full name, and at least one program are required.');
```

```

}
if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    throw new RuntimeException('Invalid email address.');
```

```

}

$stmt = db()->prepare(
    'SELECT id, username, full_name, email, role, assigned_program, college_id, is_active FROM users WHERE id=? AND
role="program_chair" AND college_id=?'
);
$stmt->execute([$id, $collegeId]);
$beforePc = $stmt->fetch(PDO::FETCH_ASSOC);
$row = $beforePc ? ['username' => $beforePc['username'], 'full_name' => $beforePc['full_name']] : false;
if (!$row) {
    throw new RuntimeException('Program Chair account not found.');
```

```

}

if (strcasecmp($username, (string) $beforePc['username']) !== 0) {
    $exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username=? AND id<>?');
    $exists->execute([$username, $id]);
    if ((int) $exists->fetchColumn() > 0) {
```

```

        $s = suggest_available_usernames($username, 3);
        throw new RuntimeException('Username already exists.' . ($s ? ' Try: ' . implode(' ', $s) : ''));
    }
}

$previousPrograms = program_chair_assigned_programs($id);
$previousPrimary = trim((string) ($beforePc['assigned_program'] ?? ''));
$primaryProgram = in_array($previousPrimary, $assignedPrograms, true)
    ? $previousPrimary
    : $assignedPrograms[0];

$sql = $hasUserEmail
    ? 'UPDATE users SET username=?, full_name=?, email=?, assigned_program=?, is_active=? WHERE id=? AND role="program_chair"
AND college_id=?'
    : 'UPDATE users SET username=?, full_name=?, assigned_program=?, is_active=? WHERE id=? AND role="program_chair" AND
college_id=?';
$params = $hasUserEmail
    ? [$username, $fullName, $email, $primaryProgram, !empty($_POST['is_active']) ? 1 : 0, $id, $collegeId]
    : [$username, $fullName, $primaryProgram, !empty($_POST['is_active']) ? 1 : 0, $id, $collegeId];
db()->prepare($sql)->execute($params);
program_chair_set_assigned_programs($id, $assignedPrograms, $primaryProgram);

$plainForMail = '';
if ($generateAndEmail) {
    if (!$hasUserEmail || $email === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        throw new RuntimeException('A valid email is required to generate and send a temporary password.');
```

```

        $beforeDelPc['assigned_programs'] = implode(' ', program_chair_assigned_programs($id));
    }
    $st = db()->prepare('DELETE FROM users WHERE id=? AND role="program_chair" AND college_id=?');
    $st->execute([$id, $collegeId]);
    if ($st->rowCount() < 1) {
        throw new RuntimeException('Program Chair account not found.');
```

```

    }
    log_user_activity('delete', 'Program chairs', 'Program chair user #' . $id, $beforeDelPc ? (array) $beforeDelPc : null, null);
    log_dean_activity('program_chair_delete', 'Deleted program chair #' . $id);
    $_SESSION['flash'] = 'Program Chair account deleted.';
}
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: program_chairs.php');
exit;
}
}

$editRow = null;
$editPrograms = [];
if (isset($_GET['edit'])) {
    $st = db()->prepare('SELECT * FROM users WHERE id=? AND role="program_chair" AND college_id=?');
    $st->execute([(int) $_GET['edit'], $collegeId]);
    $editRow = $st->fetch() ?: null;
    if ($editRow) {
        $editPrograms = program_chair_assigned_programs((int) $editRow['id']);
        if ($editPrograms === []) {
            $legacy = trim((string) ($editRow['assigned_program'] ?? ''));
            if ($legacy !== '') {
                $editPrograms = [$legacy];
            }
        }
    }
}

$chairs = [];
if ($hasAssignedProgram) {
    $st = db()->prepare(
        'SELECT id, username, full_name, email, assigned_program, is_active
        FROM users
        WHERE role="program_chair" AND college_id=?
        ORDER BY full_name, assigned_program'
    );
    $st->execute([$collegeId]);
    $chairs = $st->fetchAll();
    foreach ($chairs as $chairRow) {
        $programs = program_chair_assigned_programs((int) $chairRow['id']);
        if ($programs === []) {
            $legacy = trim((string) ($chairRow['assigned_program'] ?? ''));
            $programs = $legacy !== '' ? [$legacy] : [];
        }
        $chairRow['assigned_programs_list'] = $programs;
        $chairRow['assigned_programs_label'] = $programs !== [] ? implode(' ', $programs) : '-';
    }
    unset($chairRow);
}

$currentUser = current_user() ?? [];
$deanName = trim((string) ($currentUser['full_name'] ?? 'College Administrator'));
$deanInitials = '';
foreach (preg_split('/\s+/', $deanName) ? : [] as $part) {
    $token = trim((string) $part, " \t\n\r\0\x0B");
    if ($token !== '') {
        $deanInitials .= strtoupper(substr($token, 0, 1));
    }
}
$deanInitials = $deanInitials !== '' ? substr($deanInitials, 0, 2) : 'PC';
$totalProgramChairs = count($chairs);
$activeProgramChairs = 0;
foreach ($chairs as $chairRow) {
    if ((int) ($chairRow['is_active'] ?? 0) === 1) {
        $activeProgramChairs++;
    }
}
$programCount = count($programOptions);

$pageTitle = 'Program Chairs';
require_once __DIR__ . '/includes/header.php';
?>
<style>
```

```

.pc-dashboard-container { max-width: 1440px; margin: 0 auto; }
.pc-university-header { display: flex; justify-content: space-between; align-items: flex-end; flex-wrap: wrap; margin-bottom: 28px; border-bottom: 2px solid rgba(30,64,95,.2); padding-bottom: 16px; gap: 12px; }
.pc-title-section h1 { font-size: 1.9rem; font-weight: 700; background: linear-gradient(135deg, #1e405f, #2a6f8f); -webkit-background-clip: text; background-clip: text; color: transparent; letter-spacing: -.3px; }
.pc-title-section p { font-size: .9rem; color: #4a627a; margin-top: 6px; font-weight: 500; }
.pc-dean-card { background: #fff; padding: 10px 24px; border-radius: 60px; box-shadow: 0 4px 10px rgba(0,0,0,.02), 0 1px 2px rgba(0,0,0,.05); display: flex; align-items: center; gap: 16px; border: 1px solid #e2edf2; }
.pc-dean-avatar { background: #1e5a6f; width: 44px; height: 44px; border-radius: 50%; display: flex; align-items: center; justify-content: center; color: #fff; font-weight: 600; font-size: 1.2rem; }
.pc-dean-info h4 { font-weight: 600; font-size: .9rem; color: #2c4c6e; margin: 0; }
.pc-dean-info p { font-size: .75rem; color: #4c6a82; font-weight: 500; margin: 0; }
.pc-stats-row { display: flex; gap: 20px; margin-bottom: 32px; flex-wrap: wrap; }
.pc-stat-card { background: #fff; border-radius: 24px; padding: 14px 28px; box-shadow: 0 2px 6px rgba(0,0,0,.04); display: flex; align-items: center; gap: 14px; border: 1px solid #eef3f8; flex: 1 1 auto; min-width: 200px; }
.pc-stat-icon { background: #eaf4fc; width: 48px; height: 48px; border-radius: 32px; display: flex; align-items: center; justify-content: center; color: #1f6e8c; font-size: 1.4rem; }
.pc-stat-info h3 { font-size: 1.6rem; font-weight: 700; color: #1e405f; margin: 0; }
.pc-stat-info span { font-size: .75rem; color: #5e7f9a; font-weight: 500; }
.pc-layout { display: flex; flex-direction: column; gap: 32px; }
.pc-card { background: #fff; border-radius: 28px; box-shadow: 0 12px 28px rgba(0,0,0,.05), 0 0 1px rgba(0,0,0,.02); overflow: hidden; }
.pc-card-header { padding: 20px 28px 12px; border-bottom: 1px solid #edf2f7; display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 10px; }
.pc-card-header h2 { font-size: 1.35rem; font-weight: 600; color: #1f3b4c; display: flex; align-items: center; gap: 10px; margin: 0; }
.pc-badge-manage { background: #eef2ff; padding: 6px 14px; border-radius: 30px; font-size: .7rem; font-weight: 600; color: #2c5f8a; }
.pc-form-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 20px 28px; padding: 28px; }
.pc-input-group { display: flex; flex-direction: column; gap: 8px; }
.pc-input-group label { font-size: .75rem; font-weight: 600; text-transform: uppercase; letter-spacing: .5px; color: #4a6b85; }
.pc-input-group input, .pc-input-group select { padding: 12px 14px; border-radius: 16px; border: 1.5px solid #e2edf2; background: #fefefe; font-size: .9rem; transition: all .2s; }
.pc-input-group input:focus, .pc-input-group select:focus { outline: none; border-color: #2c8cac; box-shadow: 0 0 3px rgba(44,140,172,.2); }

.pc-program-checks { display: flex; flex-direction: column; gap: 8px; max-height: 220px; overflow: auto; padding: 12px 14px; border-radius: 16px; border: 1.5px solid #e2edf2; background: #fefefe; }
.pc-program-checks label { text-transform: none; letter-spacing: 0; font-weight: 500; font-size: .85rem; color: #2a445b; display: flex; align-items: center; gap: 8px; margin: 0; }
.pc-program-checks input[type="checkbox"] { width: 16px; height: 16px; accent-color: #1f6e8c; flex-shrink: 0; }
.pc-program-hint { font-size: .75rem; color: #6b89a0; font-weight: 500; text-transform: none; letter-spacing: 0; }
.pc-checkbox-group { display: flex; align-items: center; gap: 8px; color: #2a445b; margin-top: 8px; font-size: .85rem; }
.pc-checkbox-group input[type="checkbox"] { width: 16px; height: 16px; accent-color: #1f6e8c; }
.pc-save-btn { background: #1f6e8c; border: none; padding: 12px 24px; border-radius: 40px; font-weight: 600; color: #fff; font-size: .9rem; display: inline-flex; align-items: center; gap: 12px; cursor: pointer; transition: .2s; }
.pc-save-btn:hover { background: #0e556f; transform: translateY(-1px); box-shadow: 0 6px 12px rgba(0,0,0,.05); }
.pc-cancel-link { border-radius: 40px; padding: 10px 20px; }
.pc-table-wrapper { overflow-x: auto; padding: 0 0 8px; }
.pc-table { width: 100%; border-collapse: collapse; font-size: .85rem; }
.pc-table th { text-align: left; padding: 18px 16px; background: #f9fbfd; font-weight: 600; color: #2c4c6e; border-bottom: 1px solid #e6edf2; }
.pc-table td { padding: 14px 16px; border-bottom: 1px solid #eef3f8; vertical-align: middle; color: #2a445b; }
.pc-table tr:hover td { background: #fafcff; }
.pc-program-pills { display: flex; flex-wrap: wrap; gap: 6px; }
.pc-program-pill { display: inline-block; padding: 4px 10px; border-radius: 50px; font-size: .7rem; font-weight: 600; background: #eaf4fc; color: #1f6e8c; }
.pc-status-badge { display: inline-block; padding: 4px 12px; border-radius: 50px; font-size: .7rem; font-weight: 600; text-transform: capitalize; }
.pc-status-active { background: #e0f2e9; color: #1f7840; }
.pc-status-inactive { background: #ffe6e5; color: #b33; }
.pc-action-buttons { display: flex; gap: 8px; align-items: center; }
.pc-icon-btn { border: none; border-radius: 30px; background: transparent; color: #2c7da0; padding: 6px 9px; }
.pc-icon-btn.delete { color: #bc6c6c; }
.pc-icon-btn:hover { background: #eef2ff; color: #0f5c7c; }
.pc-icon-btn.delete:hover { background: #fff0f0; color: #b13e3e; }
.pc-footer-note { margin-top: 32px; text-align: center; font-size: .7rem; color: #6b89a0; border-top: 1px solid #e0eaf0; padding-top: 24px; }
html[data-bs-theme="dark"] .pc-title-section h1 { background: linear-gradient(135deg, #c5e6ff, #79c5e8); -webkit-background-clip: text; background-clip: text; color: transparent; }
html[data-bs-theme="dark"] .pc-title-section p { color: #9db6c8; }
html[data-bs-theme="dark"] .pc-dean-card,
html[data-bs-theme="dark"] .pc-stat-card,
html[data-bs-theme="dark"] .pc-card { background: #16202a; border-color: #2a3947; box-shadow: 0 10px 24px rgba(0,0,0,.35); }
html[data-bs-theme="dark"] .pc-dean-info h4,
html[data-bs-theme="dark"] .pc-card-header h2,
html[data-bs-theme="dark"] .pc-table th { color: #d3e7f6; }
html[data-bs-theme="dark"] .pc-dean-info p,
html[data-bs-theme="dark"] .pc-stat-info span,
html[data-bs-theme="dark"] .pc-input-group label,
html[data-bs-theme="dark"] .pc-footer-note { color: #9ab0c2; }
html[data-bs-theme="dark"] .pc-stat-icon { background: #203346; color: #87c8ea; }
html[data-bs-theme="dark"] .pc-stat-info h3 { color: #e3f3ff; }
html[data-bs-theme="dark"] .pc-card-header { border-bottom-color: #2b3a49; }
html[data-bs-theme="dark"] .pc-input-group input,
html[data-bs-theme="dark"] .pc-input-group select,

```

```

html[data-bs-theme="dark"] .pc-program-checks { background:#0f1821; border-color:#2e4152; color:#deebf5; }
html[data-bs-theme="dark"] .pc-program-checks label { color:#c5d9e7; }
html[data-bs-theme="dark"] .pc-program-hint { color:#9ab0c2; }
html[data-bs-theme="dark"] .pc-input-group input:focus,
html[data-bs-theme="dark"] .pc-input-group select:focus { border-color:#4ea4c6; box-shadow:0 0 3px rgba(78,164,198,.28); }
html[data-bs-theme="dark"] .pc-checkbox-group { color:#c5d9e7; }
html[data-bs-theme="dark"] .pc-badge-manage { background:#223243; color:#a9d2ea; }
html[data-bs-theme="dark"] .pc-table th { background:#1b2834; border-bottom-color:#30414f; }
html[data-bs-theme="dark"] .pc-table td { color:#c8dced; border-bottom-color:#2b3a47; }
html[data-bs-theme="dark"] .pc-table tr:hover td { background:#1a2530; }
html[data-bs-theme="dark"] .pc-program-pill { background:#203346; color:#87c8ea; }
html[data-bs-theme="dark"] .pc-status-active { background:#1d3a2b; color:#8fddb1; }
html[data-bs-theme="dark"] .pc-status-inactive { background:#45262a; color:#f4aab1; }
html[data-bs-theme="dark"] .pc-icon-btn { color:#90c7e5; }
html[data-bs-theme="dark"] .pc-icon-btn:hover { background:#233344; color:#c4e6f9; }
html[data-bs-theme="dark"] .pc-icon-btn.delete { color:#e09a9a; }
html[data-bs-theme="dark"] .pc-icon-btn.delete:hover { background:#3f262b; color:#ffc4c4; }
html[data-bs-theme="dark"] .pc-footer-note { border-top-color:#2a3a49; }
@media (max-width: 700px) {
    .pc-card-header { flex-direction:column; align-items:flex-start; }
    .pc-stat-card { padding:10px 20px; }
}
</style>

<div class="pc-dashboard-container">
    <div class="pc-university-header">
        <div class="pc-title-section">
            <h1><i class="fa-solid fa-user-tie" style="color:#2c7da0; margin-right:8px;"></i>Program Chairs</h1>
            <p><?= htmlspecialchars($collegeName) ?> · Western Philippines University</p>
        </div>
        <div class="pc-dean-card">
            <div class="pc-dean-avatar"><?= htmlspecialchars($deanInitials) ?></div>
            <div class="pc-dean-info">
                <h4><?= htmlspecialchars(strtoupper($deanName)) ?></h4>
                <p>Dean, <?= htmlspecialchars($collegeName) ?> <i class="fa-solid fa-circle-check" style="font-size:.7rem; color:#1f8a4c;">
            </i></p>
        </div>
    </div>
    <div>

        <?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

        <?php if (!$hasAssignedProgram || !$hasProgramsTable): ?>
            <div class="alert alert-warning">Program Chair accounts require the upgraded schema. Run <a
href="upgrade_roles.php">upgrade_roles.php</a> first.</div>
        <?php else: ?>
            <div class="pc-stats-row">
                <div class="pc-stat-card">
                    <div class="pc-stat-icon"><i class="fa-solid fa-user-tie"></i></div>
                    <div class="pc-stat-info"><h3><?= $totalProgramChairs ?></h3><span>Total Program Chairs</span></div>
                </div>
                <div class="pc-stat-card">
                    <div class="pc-stat-icon"><i class="fa-solid fa-user-check"></i></div>
                    <div class="pc-stat-info"><h3><?= $activeProgramChairs ?></h3><span>Active Chairs</span></div>
                </div>
                <div class="pc-stat-card">
                    <div class="pc-stat-icon"><i class="fa-solid fa-layer-group"></i></div>
                    <div class="pc-stat-info"><h3><?= $programCount ?></h3><span>Active Programs</span></div>
                </div>
            </div>
        </div>

        <div class="pc-layout">
            <div class="pc-card">
                <div class="pc-card-header">
                    <h2><i class="fa-solid fa-user-plus"></i> <?= $editRow ? 'Edit Program Chair' : 'Add Program Chair' ?></h2>
                    <span class="pc-badge-manage"><i class="fa-solid fa-pen me-1"></i>Manage chair records</span>
                </div>
                <form method="post" class="pc-form-grid">
                    <input type="hidden" name="action" value="<?= $editRow ? 'edit' : 'add' ?>">
                    <?php if ($editRow): ?><input type="hidden" name="id" value="<?= (int) $editRow['id'] ?>"><?php endif; ?>

                    <div class="pc-input-group"><label>Username</label><input name="username" maxlength="50" required value="<?=
htmlspecialchars((string) ($editRow['username'] ?? '')) ?>"></div>
                    <div class="pc-input-group"><label>Full Name</label><input name="full_name" required value="<?= htmlspecialchars((string)
($editRow['full_name'] ?? '')) ?>"></div>
                    <div class="pc-input-group" style="grid-column:1 / -1;">
                        <label>Assigned Programs</label>
                        <span class="pc-program-hint">Select one or more programs this chair will handle.</span>
                        <div class="pc-program-checks">
                            <?php if ($programOptions === []): ?>

```



```

        <span class="text-muted">No active programs yet. Add programs first.</span>
    <?php else: ??
        <?php foreach ($programOptions as $programName): ??
            <label>
                <input type="checkbox" name="assigned_programs[]" value="<?= htmlspecialchars($programName) ??"
                    <?= in_array($programName, $editPrograms, true) ? 'checked' : '' ??">
                <span><?= htmlspecialchars($programName) ?"></span>
            </label>
        <?php endforeach; ??
    <?php endif; ??
</div>
</div>
<?php if ($hasUserEmail): ??
    <div class="pc-input-group"><label>Email</label><input type="email" name="email" value="<?= htmlspecialchars((string)
($editRow['email'] ?? '') ?)"></div>
<?php endif; ??

<?php if (!$editRow): ??
    <div class="pc-input-group">
        <label>Password</label>
        <input name="password" id="pcPassword" autocomplete="new-password" placeholder="Required if not emailing">
        <?php if ($hasUserEmail): ??
            <label class="pc-checkbox-group">
                <input type="checkbox" name="send_credentials_email" id="pc_send_credentials_email" value="1" checked>
                <span>Generate temporary password and email it</span>
            </label>
        <?php endif; ??
    </div>
<?php else: ??
    <div class="pc-input-group">
        <label>Reset Password (optional)</label>
        <input name="reset_password" autocomplete="new-password">
        <?php if ($hasUserEmail): ??
            <label class="pc-checkbox-group">
                <input type="checkbox" name="email_reset_password" id="pc_email_reset_password" value="1">
                <span>Email this new password</span>
            </label>
            <label class="pc-checkbox-group">
                <input type="checkbox" name="generate_temp_password_email" id="pc_generate_temp_password_email" value="1">
                <span>Generate temporary password and email it</span>
            </label>
        <?php endif; ??
    </div>
<?php endif; ??

<div class="pc-input-group">
    <label>Status</label>
    <label class="pc-checkbox-group">
        <input type="checkbox" name="is_active" value="1" <?= !isset($editRow['is_active']) || (int) ($editRow['is_active']
?? 1) === 1 ? 'checked' : '' ??">
        <span>Active account</span>
    </label>
</div>

<div style="display:flex; justify-content:flex-end; gap:10px; grid-column:1 / -1;">
    <?php if ($editRow): ??<a class="btn btn-outline-secondary pc-cancel-link" href="program_chairs.php">Cancel</a><?php
endif; ??
    <button class="pc-save-btn" type="submit"><i class="fa-solid <?= $editRow ? 'fa-pen' : 'fa-save' ?"></i> <?= $editRow ?
'Update Program Chair' : 'Save Program Chair' ?"></button>
</div>
</form>
<?php if (!$editRow && $hasUserEmail): ??
<script>
    (function () {
        const cb = document.getElementById('pc_send_credentials_email');
        const pw = document.getElementById('pcPassword');
        if (!cb || !pw) return;
        function sync() {
            pw.required = !cb.checked;
            pw.disabled = cb.checked;
            if (cb.checked) pw.value = '';
        }
        cb.addEventListener('change', sync);
        sync();
    })();
</script>
<?php endif; ??
</div>

<div class="pc-card">

```

```

<div class="pc-card-header">
    <h2><i class="fa-solid fa-list-ul"></i> Program chair roster</h2>
    <span class="pc-badge-manage"><i class="fa-solid fa-users me-1"></i>Account directory</span>
</div>
<div class="pc-table-wrapper">
    <table class="pc-table">
        <thead>
            <tr>
                <th>Username</th>
                <th>Name</th>
                <th>Programs</th>
                <?php if ($hasUserEmail): ?><th>Email</th><?php endif; ?>
                <th>Status</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            <?php foreach ($chairs as $chair): ?>
                <tr>
                    <td><? = htmlspecialchars((string) $chair['username']) ?></td>
                    <td><? = htmlspecialchars((string) $chair['full_name']) ?></td>
                    <td>
                        <div class="pc-program-pills">
                            <?php foreach (($chair['assigned_programs_list'] ?? []) as $progName): ?>
                                <span class="pc-program-pill"><? = htmlspecialchars((string) $progName) ?></span>
                            <?php endforeach; ?>
                            <?php if (($chair['assigned_programs_list'] ?? []) === []): ?>
                                <span class="text-muted"></span>
                            <?php endif; ?>
                        </div>
                    </td>
                    <?php if ($hasUserEmail): ?><td><? = htmlspecialchars((string) ($chair['email'] ?? '')) ?></td><?php endif; ?>
                    <td>
                        <span class="pc-status-badge <? = (int) $chair['is_active'] === 1 ? 'pc-status-active' : 'pc-status-inactive'
?>">
                            <? = (int) $chair['is_active'] === 1 ? 'Active' : 'Disabled' ?>
                        </span>
                    </td>
                    <td class="pc-action-buttons">
                        <a href="program_chairs.php?edit=<? = (int) $chair['id'] ?>" class="pc-icon-btn" title="Edit Program Chair">
<i class="fa-solid fa-pen"></i></a>
                        <form method="post" class="d-inline" onsubmit="return confirm('Delete this Program Chair account?');">
                            <input type="hidden" name="action" value="delete">
                            <input type="hidden" name="id" value="<? = (int) $chair['id'] ?>">
                            <button type="submit" class="pc-icon-btn delete" title="Delete Program Chair"><i class="fa-solid fa-
trash-can"></i></button>
                        </form>
                    </td>
                </tr>
            <?php endforeach; ?>
            <?php if ($chairs === []): ?>
                <tr><td colspan="<? = $hasUserEmail ? '6' : '5' ?>" class="text-center text-muted py-4">No Program Chair accounts
yet.</td></tr>
            <?php endif; ?>
        </tbody>
    </table>
</div>
</div>
</div>

<div class="pc-footer-note">
    <i class="fa-solid fa-database"></i> Program chair directory · Real-time management | Western Philippines University
</div>
<?php endif; ?>
</div>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: program_chair_students.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/mail_helpers.php';
require_once __DIR__ . '/includes/account_registration_helpers.php';

```

```

require_role(['program_chair']);
$collegeId = dean_or_program_chair_college_id_or_fail();
$programScope = program_scope_or_fail();
$collegeName = college_name_by_id($collegeId);

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

$hasUserEmail = db_column_exists('users', 'email');
$hasAssignedProgram = db_column_exists('users', 'assigned_program');

if (!$hasAssignedProgram) {
    exit('Run upgrade_roles.php first to enable student accounts for your program.');
```

```

}
if (!db_table_exists('classroom_students')) {
    exit('Classroom student profiles are not installed. Run upgrade_roles.php once.');
```

```

}

$hasClassroomEnrollments = db_table_exists('classroom_enrollments') && db_table_exists('online_classrooms');

if (isset($_GET['download_students_csv_template'])) {
    header('Content-Type: text/csv; charset=utf-8');
    header('Content-Disposition: attachment; filename="students_import_template.csv"');
    $out = fopen('php://output', 'w');
    if ($hasUserEmail) {
        fputcsv($out, ['username', 'password', 'full_name', 'student_number', 'email']);
        fputcsv($out, ['jdoe', 'ChangeMe123', 'Jane Doe', '2024-0001', 'jane@example.com']);
    } else {
        fputcsv($out, ['username', 'password', 'full_name', 'student_number']);
        fputcsv($out, ['jdoe', 'ChangeMe123', 'Jane Doe', '2024-0001']);
    }
    fclose($out);
    exit;
}

/**
 * @param array<int, string> $headerCells
 * @return array<string, int> canonical key => column index
 */
function program_chair_students_csv_column_map(array $headerCells, bool $includeEmail): array
{
    $aliases = [
        'username' => ['username', 'user_name', 'login'],
        'password' => ['password', 'pass', 'passwd'],
        'full_name' => ['full_name', 'fullname', 'name', 'full name'],
        'student_number' => ['student_number', 'studentnumber', 'student_id', 'student_no', 'student_#', 'id_number'],
        'email' => ['email', 'e_mail'],
    ];
    $norm = [];
    foreach ($headerCells as $i => $cell) {
        $h = strtolower(trim((string) $cell));
        if ($i === 0 && strpos($h, "\xEF\xBB\xBF", 3) === 0) {
            $h = substr($h, 3);
        }
        $h = str_replace([' ', '-', '_'], '_', $h);
        $norm[$i] = $h;
    }
    $map = [];
    foreach ($aliases as $key => $names) {
        if ($key === 'email' && !$includeEmail) {
            continue;
        }
        foreach ($norm as $idx => $h) {
            if (in_array($h, $names, true)) {
                $map[$key] = $idx;
                break;
            }
        }
    }
    return $map;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if ($action === 'add') {
            $username = trim((string) ($_POST['username'] ?? ''));
            $password = (string) ($_POST['password'] ?? '');
            $fullName = trim((string) ($_POST['full_name'] ?? ''));
            $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';

```

```

$studentNumber = trim((string) ($_POST['student_number'] ?? ''));
$hasValidEmail = $hasUserEmail && $email !== '' && filter_var($email, FILTER_VALIDATE_EMAIL);
$plainForMail = '';

if ($username === '' || $fullName === '') {
    throw new RuntimeException('Username and full name are required.');
```

```

}
if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    throw new RuntimeException('Invalid email address.');
```

```

}

$cred = prepare_new_account_password($password, $hasValidEmail);
$password = $cred['plain'];
$plainForMail = $hasValidEmail ? $cred['plain'] : '';

$exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username=?');
$exists->execute([$username]);
if ((int) $exists->fetchColumn() > 0) {
    $s = suggest_available_usernames($username, 3);
    throw new RuntimeException('Username already exists.' . ($s ? ' Try: ' . implode(', ', $s) : ''));
}

db()->beginTransaction();
if ($hasUserEmail) {
    db()->prepare(
        'INSERT INTO users (username, password, full_name, email, role, assigned_program, college_id, is_active)
        VALUES (?, ?, ?, ?, ?, ?, ?, 1)'
    )->execute([
        $username,
        $cred['hash'],
        $fullName,
        $email,
        'student',
        $programScope,
        $collegeId,
    ]);
} else {
    db()->prepare(
        'INSERT INTO users (username, password, full_name, role, assigned_program, college_id, is_active)
        VALUES (?, ?, ?, ?, ?, ?, 1)'
    )->execute([
        $username,
        $cred['hash'],
        $fullName,
        'student',
        $programScope,
        $collegeId,
    ]);
}
$uid = (int) db()->lastInsertId();

db()->prepare(
    'INSERT INTO classroom_students (user_id, student_number, full_name, email) VALUES (?, ?, ?, ?)'
)->execute([
    $uid,
    $studentNumber,
    $fullName,
    $hasUserEmail ? $email : '',
]);

db()->commit();
log_dean_activity('student_create', 'Created student login for program ' . $programScope . ': ' . $username);

$mailOk = $hasValidEmail
    ? notify_new_account_credentials($uid, $email, $fullName, $username, $plainForMail, 'student')
    : null;
if (!$hasValidEmail) {
    mark_new_account_requires_password_change($uid);
}
$_SESSION['flash'] = registration_email_flash_message($mailOk, $email, 'Student account');
} elseif ($action === 'edit' && isset($_POST['id'])) {
    $userId = (int) $_POST['id'];
    $fullName = trim((string) ($_POST['full_name'] ?? ''));
    $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
    $studentNumber = trim((string) ($_POST['student_number'] ?? ''));
    $resetPassword = trim((string) ($_POST['reset_password'] ?? ''));

    if ($fullName === '') {
        throw new RuntimeException('Full name is required.');
```

```

    }
}

```

```

if ($hasUserEmail && $email != '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    throw new RuntimeException('Invalid email address.');
```

```

}
```

```

$st = db()->prepare(
    'SELECT u.id, u.username FROM users u
    WHERE u.id = ? AND u.role = ? AND u.college_id = ? AND u.assigned_program = ?'
);
```

```

$st->execute([$userId, 'student', $collegeId, $programScope]);
```

```

$studentRow = $st->fetch();
```

```

if (!$studentRow) {
    throw new RuntimeException('Student not found in your program.');
```

```

}
```

```

$username = (string) $studentRow['username'];
```

```

if ($hasUserEmail) {
    db()->prepare('UPDATE users SET full_name = ?, email = ?, is_active = ? WHERE id = ? AND role = ? AND college_id = ? AND
assigned_program = ?')
```

```

    ->execute([
        $fullName,
        $email,
        !empty($_POST['is_active']) ? 1 : 0,
        $userId,
        'student',
        $collegeId,
        $programScope,
    ]);
```

```

} else {
    db()->prepare('UPDATE users SET full_name = ?, is_active = ? WHERE id = ? AND role = ? AND college_id = ? AND
assigned_program = ?')
```

```

    ->execute([
        $fullName,
        !empty($_POST['is_active']) ? 1 : 0,
        $userId,
        'student',
        $collegeId,
        $programScope,
    ]);
```

```

}
```

```

db()->prepare(
    'UPDATE classroom_students SET full_name = ?, email = ?, student_number = ? WHERE user_id = ?'
)->execute([
```

```

    $fullName,
    $hasUserEmail ? $email : '',
    $studentNumber,
    $userId,
]);
```

```

$generateAndEmail = $hasUserEmail && !empty($_POST['generate_temp_password_email']);
$plainForMail = '';
```

```

if ($generateAndEmail) {
    if ($email === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        throw new RuntimeException('A valid email is required to generate and send a temporary password.');
```

```

    }
    $plainForMail = generate_temp_password();
    db()->prepare('UPDATE users SET password = ? WHERE id = ?')->execute([
        password_hash($plainForMail, PASSWORD_DEFAULT),
        $userId,
    ]);
```

```

} elseif ($resetPassword !== '') {
    if (strlen($resetPassword) < 8) {
        throw new RuntimeException('New password must be at least 8 characters.');
```

```

    }
    db()->prepare('UPDATE users SET password = ? WHERE id = ?')->execute([
        password_hash($resetPassword, PASSWORD_DEFAULT),
        $userId,
    ]);
```

```

    if (!empty($_POST['email_reset_password']) && $hasUserEmail && $email != '' && filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $plainForMail = $resetPassword;
    }
}
```

```

log_dean_activity('student_update', 'Updated student user #' . $userId);
```

```

if ($plainForMail !== '') {
    $mailOk = send_account_credentials_mail($email, $fullName, $username, $plainForMail, 'student', $email);
    $_SESSION['flash'] = credentials_email_flash_message($mailOk, $email, 'Student updated');
```

```

} else {
    $_SESSION['flash'] = 'Student updated.';
}
```

```

    }
} elseif ($action === 'delete' && isset($_POST['id'])) {
    $userId = (int) $_POST['id'];
    $st = db()->prepare(
        'SELECT u.id FROM users u
        WHERE u.id = ? AND u.role = ? AND u.college_id = ? AND u.assigned_program = ?'
    );
    $st->execute([$userId, 'student', $collegeId, $programScope]);
    if (!$st->fetch()) {
        throw new RuntimeException('Student not found in your program.');
```

```

        continue;
    }
    if ($password === '' || strlen($password) < 8) {
        $failures[] = 'Row ' . $rowNum . ': password must be at least 8 characters.';
        continue;
    }
    if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $failures[] = 'Row ' . $rowNum . ': invalid email.';
        continue;
    }
    $lower = strtolower($username);
    if (isset($seenLowerUsernames[$lower])) {
        $failures[] = 'Row ' . $rowNum . ': duplicate username in this file (' . $username . ').';
        continue;
    }
    $seenLowerUsernames[$lower] = true;

    $exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username=?');
    $exists->execute([$username]);
    if ((int) $exists->fetchColumn() > 0) {
        $failures[] = 'Row ' . $rowNum . ': username already exists (' . $username . ').';
        continue;
    }

    try {
        db()->beginTransaction();
        if ($hasUserEmail) {
            db()->prepare(
                'INSERT INTO users (username, password, full_name, email, role, assigned_program, college_id, is_active)
                VALUES (?, ?, ?, ?, ?, ?, ?, 1)'
            )->execute([
                $username,
                password_hash($password, PASSWORD_DEFAULT),
                $fullName,
                $email,
                'student',
                $programScope,
                $collegeId,
            ]);
        } else {
            db()->prepare(
                'INSERT INTO users (username, password, full_name, role, assigned_program, college_id, is_active)
                VALUES (?, ?, ?, ?, ?, ?, 1)'
            )->execute([
                $username,
                password_hash($password, PASSWORD_DEFAULT),
                $fullName,
                'student',
                $programScope,
                $collegeId,
            ]);
        }
        $uid = (int) db()->lastInsertId();
        db()->prepare(
            'INSERT INTO classroom_students (user_id, student_number, full_name, email) VALUES (?, ?, ?, ?)'
        )->execute([
            $uid,
            $studentNumber,
            $fullName,
            $hasUserEmail ? $email : '',
        ]);
        db()->commit();
        $created++;

        $hasRowEmail = $hasUserEmail && $email !== '' && filter_var($email, FILTER_VALIDATE_EMAIL);
        if ($hasRowEmail) {
            notify_new_account_credentials($uid, $email, $fullName, $username, $password, 'student');
        } else {
            mark_new_account_requires_password_change($uid);
        }
    } catch (Throwable $rowEx) {
        if (db()->inTransaction()) {
            db()->rollBack();
        }
        $failures[] = 'Row ' . $rowNum . ': ' . $rowEx->getMessage();
    }
}
fclose($fh);
$parts = ['Batch import finished: ' . $created . ' student account(s) created.'];
if ($failures !== []) {

```

```

        $show = array_slice($failures, 0, 15);
        $parts[] = count($failures) . ' row(s) skipped: ' . implode(' ', $show);
        if (count($failures) > 15) {
            $parts[] = '(and ' . (count($failures) - 15) . ' more)';
        }
    }
    log_dean_activity('student_batch_import', 'Batch CSV import for program ' . $programScope . ': ' . $created . ' created');
    $_SESSION['flash'] = implode(' ', $parts);
}
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: program_chair_students.php');
exit;
}

$editRow = null;
if (isset($_GET['edit'])) {
    $editId = (int) $_GET['edit'];
    $st = db()->prepare(
        'SELECT u.id, u.username, u.full_name, u.email, u.is_active, cs.student_number
        FROM users u
        INNER JOIN classroom_students cs ON cs.user_id = u.id
        WHERE u.id = ? AND u.role = ? AND u.college_id = ? AND u.assigned_program = ?'
    );
    $st->execute([$editId, 'student', $collegeId, $programScope]);
    $editRow = $st->fetch() ?: null;
}

$enrolledFacultySelect = $hasClassroomEnrollments
    ? ', (SELECT GROUP_CONCAT(DISTINCT CONCAT(f.full_name, \' \', f.faculty_id) SEPARATOR \' \')
        FROM classroom_enrollments ce
        INNER JOIN online_classrooms oc ON oc.id = ce.classroom_id
        INNER JOIN faculty f ON f.id = oc.faculty_id
        WHERE ce.student_id = cs.id) AS enrolled_faculty_text'
    : '';
$st = db()->prepare(
    'SELECT u.id, u.username, u.full_name, u.email, u.is_active, cs.student_number' . $enrolledFacultySelect . '
    FROM users u
    INNER JOIN classroom_students cs ON cs.user_id = u.id
    WHERE u.role = ? AND u.college_id = ? AND u.assigned_program = ?
    ORDER BY u.full_name ASC'
);
$st->execute(['student', $collegeId, $programScope]);
$list = $st->fetchAll();

$pcsStudentsPayload = [];
foreach ($list as $r) {
    $pcsStudentsPayload[] = [
        'id' => (int) $r['id'],
        'full_name' => (string) $r['full_name'],
        'username' => (string) $r['username'],
        'student_number' => (string) ($r['student_number'] ?? ''),
        'email' => $hasUserEmail ? (string) ($r['email'] ?? '') : '',
        'is_active' => (int) ($r['is_active'] ?? 1),
        'enrolled_faculty_text' => $hasClassroomEnrollments ? (string) ($r['enrolled_faculty_text'] ?? '') : '',
    ];
}

$pageTitle = 'Student management';
$mainContainerClass = 'container-fluid py-4 py-md-5 app-main';
require_once __DIR__ . '/includes/header.php';
?>
<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<link href="https://fonts.googleapis.com/css2?family=Inter:opsz,wght@14..32,300;14..32,400;14..32,500;14..32,600;14..32,700&display=swap"
rel="stylesheet">
<style>
.pcs-app {
    --pcs-bg: #f1f5f9;
    --pcs-text: #0f172a;
    --pcs-muted: #475569;
    --pcs-border: #e2e8f0;
    max-width: 1400px;
    margin: 0 auto;
    font-family: 'Inter', system-ui, sans-serif;
    color: var(--pcs-text);

```



```

    line-height: 1.5;
}
.pcs-app * { box-sizing: border-box; }
.pcs-top-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    margin-bottom: 28px;
    gap: 16px;
}
.pcs-logo-area h1 {
    font-size: 1.8rem;
    font-weight: 700;
    letter-spacing: -0.3px;
    background: linear-gradient(135deg, #1e293b, #2d3a4e);
    -webkit-background-clip: text;
    background-clip: text;
    color: transparent;
    margin: 0;
}
.pcs-logo-area p {
    font-size: 0.85rem;
    color: var(--pcs-muted);
    margin: 4px 0 0;
}
.pcs-college-badge {
    background: #fff;
    padding: 8px 18px;
    border-radius: 40px;
    box-shadow: 0 1px 3px rgba(0,0,0,0.05);
    font-size: 0.9rem;
    font-weight: 500;
    border: 1px solid var(--pcs-border);
}
.pcs-college-badge i { margin-right: 6px; color: #3b82f6; }
.pcs-nav-tabs {
    display: flex;
    gap: 8px;
    border-bottom: 2px solid var(--pcs-border);
    margin-bottom: 28px;
    flex-wrap: wrap;
}
.pcs-nav-tab {
    padding: 10px 20px;
    font-weight: 600;
    font-size: 0.95rem;
    border-radius: 30px 30px 0 0;
    color: #64748b;
    text-decoration: none;
    transition: color 0.2s, background 0.2s;
}
.pcs-nav-tab:hover { color: #1e40af; background: #ffffff80; }
.pcs-nav-tab.pcs-active {
    color: #1e40af;
    border-bottom: 3px solid #1e40af;
    background: #ffffff80;
    cursor: default;
    pointer-events: none;
}
.pcs-nav-tab i { margin-right: 8px; }
.pcs-card {
    background: #fff;
    border-radius: 24px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.03), 0 1px 2px rgba(0,0,0,0.05);
    margin-bottom: 32px;
    border: 1px solid #eef2f6;
    overflow: hidden;
}
.pcs-card-header {
    padding: 20px 28px;
    border-bottom: 1px solid #edf2f7;
}
.pcs-card-header h2 {
    font-size: 1.4rem;
    font-weight: 600;
    margin: 0;
    display: flex;
    align-items: center;
    gap: 10px;
}

```

```

}
.pcs-card-header h2 i { color: #3b82f6; font-size: 1.3rem; }
.pcs-card-header .pcs-lead {
  font-size: 0.8rem;
  color: var(--pcs-muted);
  margin: 6px 0 0;
}
.pcs-card-body { padding: 24px 28px; }
.pcs-form-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 20px 28px;
}
.pcs-field { display: flex; flex-direction: column; gap: 6px; }
.pcs-field.pcs-full { grid-column: span 2; }
.pcs-field label {
  font-size: 0.85rem;
  font-weight: 600;
  color: #334155;
  letter-spacing: 0.3px;
}
.pcs-field label .pcs-opt {
  font-weight: normal;
  color: #94a3b8;
  font-size: 0.75rem;
  margin-left: 6px;
}
.pcs-field input, .pcs-field select {
  padding: 12px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 14px;
  font-family: 'Inter', system-ui, sans-serif;
  font-size: 0.9rem;
  transition: border-color 0.2s, box-shadow 0.2s;
  background: #fff;
  width: 100%;
}
.pcs-field input:focus, .pcs-field select:focus {
  outline: none;
  border-color: #3b82f6;
  box-shadow: 0 0 3px rgba(59,130,246,0.2);
}
.pcs-password-hint { font-size: 0.7rem; color: #5b6e8c; margin-top: 4px; }
.pcs-btn-group { display: flex; gap: 12px; align-items: center; margin-top: 12px; flex-wrap: wrap; }
.pcs-btn-primary {
  background: #1e40af;
  border: none;
  padding: 10px 20px;
  border-radius: 40px;
  font-weight: 600;
  color: #fff;
  font-size: 0.85rem;
  cursor: pointer;
  transition: 0.2s;
  display: inline-flex;
  align-items: center;
  gap: 8px;
  text-decoration: none;
}
.pcs-btn-primary:hover { background: #1e3a8a; color: #fff; transform: translateY(-1px); box-shadow: 0 6px 12px -8px #1e3a8a50; }
.pcs-btn-secondary {
  background: #f1f5f9;
  border: 1px solid #cbd5e1;
  padding: 9px 18px;
  border-radius: 40px;
  font-weight: 500;
  color: #1f2937;
  cursor: pointer;
  transition: 0.2s;
  display: inline-flex;
  align-items: center;
  gap: 6px;
  text-decoration: none;
}
.pcs-btn-secondary:hover { background: #e6edf4; border-color: #94a3b8; color: #1f2937; }
.pcs-btn-outline {
  background: transparent;
  border: 1px solid #cbd5e1;
  padding: 9px 18px;
  border-radius: 40px;
}

```

```

    font-weight: 500;
    cursor: pointer;
    transition: 0.2s;
    color: #1f2937;
    display: inline-flex;
    align-items: center;
    gap: 6px;
    text-decoration: none;
}
.pcs-btn-outline i { margin-right: 0; }
.pcs-btn-outline:hover { background: #f8fafc; color: #1f2937; }
.pcs-batch-zone { background: #fafcff; border-radius: 20px; padding: 0; }
.pcs-batch-inner { background: #f9fbfe; border-radius: 18px; padding: 12px 16px; }
.pcs-batch-inner > p { font-size: 0.85rem; margin-bottom: 12px; }
.pcs-file-row { display: flex; gap: 16px; flex-wrap: wrap; align-items: center; margin: 0; }
.pcs-fake-file {
    background: #f1f5f9;
    padding: 10px 16px;
    border-radius: 40px;
    font-size: 0.85rem;
    color: #334155;
    border: 1px dashed #94a3b8;
    flex: 1;
    min-width: 140px;
}
.pcs-table-wrap { overflow-x: auto; border-radius: 20px; }
.pcs-table { width: 100%; border-collapse: collapse; font-size: 0.85rem; }
.pcs-table th {
    text-align: left;
    padding: 16px 12px;
    background: #f8fafc;
    font-weight: 600;
    color: #1e293b;
    border-bottom: 1px solid var(--pcs-border);
}
.pcs-table td {
    padding: 14px 12px;
    border-bottom: 1px solid #ecf3fa;
    vertical-align: middle;
}
.pcs-status {
    display: inline-flex;
    align-items: center;
    gap: 6px;
    padding: 4px 10px;
    border-radius: 30px;
    font-size: 0.75rem;
    font-weight: 600;
}
.pcs-status-active { background: #e6f7ec; color: #15803d; }
.pcs-status-off { background: #f1f5f9; color: #64748b; }
.pcs-actions { display: flex; gap: 12px; }
.pcs-actions a, .pcs-actions button {
    background: none;
    border: none;
    cursor: pointer;
    font-size: 1.1rem;
    padding: 4px;
    border-radius: 30px;
    width: 28px;
    height: 28px;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    color: #5b6e8c;
    text-decoration: none;
    transition: 0.1s;
}
.pcs-actions a:hover, .pcs-actions button:hover { background: #f1f5f9; transform: scale(1.05); }
.pcs-edit:hover { color: #2563eb; }
.pcs-del:hover { color: #dc2626; }
.pcs-controls {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    flex-wrap: wrap;
    gap: 12px;
}
.pcs-search {

```

```

        display: flex;
        align-items: center;
        background: #fff;
        border: 1px solid var(--pcs-border);
        border-radius: 40px;
        padding: 6px 16px;
        gap: 8px;
    }
    .pcs-search i { color: #94a3b8; }
    .pcs-search input {
        border: none;
        padding: 8px 0;
        outline: none;
        background: transparent;
        width: min(220px, 50vw);
        font-family: inherit;
        font-size: 0.9rem;
    }
    }
    .pcs-total-pill {
        font-size: 0.75rem;
        background: #eef2ff;
        padding: 6px 12px;
        border-radius: 40px;
    }
    }
    .pcs-pagination { display: flex; gap: 8px; align-items: center; margin-top: 20px; justify-content: flex-end; flex-wrap: wrap; }
    .pcs-page-btn {
        background: #fff;
        border: 1px solid var(--pcs-border);
        padding: 6px 12px;
        border-radius: 30px;
        font-size: 0.8rem;
        cursor: pointer;
        transition: 0.1s;
    }
    }
    .pcs-page-btn.pcs-page-on { background: #1e40af; color: #fff; border-color: #1e40af; }
    .pcs-modal {
        display: none;
        position: fixed;
        inset: 0;
        background: rgba(0,0,0,0.5);
        align-items: center;
        justify-content: center;
        z-index: 1000;
    }
    }
    .pcs-modal.pcs-open { display: flex; }
    .pcs-modal-card {
        background: #fff;
        max-width: 400px;
        width: 90%;
        border-radius: 32px;
        padding: 24px;
        text-align: center;
    }
    }
    .pcs-modal-btns { display: flex; gap: 12px; justify-content: center; margin-top: 24px; flex-wrap: wrap; }
    .pcs-modal-btns .pcs-btn-primary.pcs-danger { background: #b91c1c; }
    .pcs-modal-btns .pcs-btn-primary.pcs-danger:hover { background: #991b1b; }
    .pcs-toast {
        position: fixed;
        bottom: 24px;
        right: 24px;
        color: #fff;
        padding: 12px 20px;
        border-radius: 40px;
        font-size: 0.85rem;
        z-index: 1100;
        box-shadow: 0 8px 20px rgba(0,0,0,0.1);
        max-width: min(420px, calc(100vw - 48px));
    }
    }
    .pcs-toast.pcs-toast-err { background: #b91c1c; }
    .pcs-toast:not(.pcs-toast-err) { background: #1f2937; }
    .pcs-empty { text-align: center; color: #64748b; padding: 28px 16px; font-size: 0.9rem; }
    @media (max-width: 760px) {
        .pcs-form-grid { grid-template-columns: 1fr; }
        .pcs-field.pcs-full { grid-column: span 1; }
        .pcs-card-body { padding: 20px; }
    }
    }
</style>

<div class="pcs-app no-print">
    <div class="pcs-top-header">

```

```

<div class="pcs-logo-area">
    <h1><i class="fa-solid fa-chalkboard-user" aria-hidden="true"></i> WPU SABLAe Portal</h1>
    <p>Course enrollment & student management</p>
</div>
<div class="pcs-college-badge">
    <i class="fa-solid fa-building-columns" aria-hidden="true"></i>
    <?= htmlspecialchars($collegeName) ?> &nbsp;&nbsp;&nbsp;
    <strong><?= htmlspecialchars($programScope) ?></strong>
</div>
</div>

<nav class="pcs-nav-tabs" aria-label="Program chair sections">
    <span class="pcs-nav-tab pcs-active"><i class="fa-solid fa-users" aria-hidden="true"></i> Students</span>
    <a class="pcs-nav-tab" href="schedule.php"><i class="fa-solid fa-calendar-alt" aria-hidden="true"></i> Schedules</a>
    <a class="pcs-nav-tab" href="courses.php"><i class="fa-solid fa-book" aria-hidden="true"></i> Courses</a>
</nav>

<div class="pcs-card">
    <div class="pcs-card-header">
        <h2><i class="fa-solid fa-user-plus" aria-hidden="true"></i> <?= $editRow ? 'Edit student' : 'Add student account' ?></h2>
        <?php if (!$editRow): ?>
            <p class="pcs-lead">After signing in, students use &quot;My Classes&quot; and enter each instructor's class join code.</p>
        <?php endif; ?>
    </div>
    <div class="pcs-card-body">
        <form method="post" id="pcs-add-form" autocomplete="off">
            <input type="hidden" name="action" value="<?= $editRow ? 'edit' : 'add' ?>">
            <?php if ($editRow): ?><input type="hidden" name="id" value="(int) $editRow['id'] ?>"><?php endif; ?>
            <div class="pcs-form-grid">
                <div class="pcs-field">
                    <label for="pcs-username">Username <span class="pcs-opt">*</span></label>
                    <input id="pcs-username" type="text" name="username" maxlength="50" <?= $editRow ? 'readonly' : 'required' ?>
                    value="<?= htmlspecialchars((string) ($editRow['username'] ?? '')) ?>" autocomplete="username">
                </div>
                <div class="pcs-field">
                    <label for="pcs-password"><?= $editRow ? 'New password' : 'Password' ?> <span class="pcs-opt"><?= $editRow ?
'optional' : '*' ?></span></label>
                    <input id="pcs-password" type="password" name="<?= $editRow ? 'reset_password' : 'password' ?>" <?= $editRow ? '' :
'required minlength="8"' ?> autocomplete="new-password" minlength="<?= $editRow ? '0' : '8' ?>">
                    <?php if (!$editRow): ?><div class="pcs-password-hint">At least 8 characters.</div><?php endif; ?>
                    <?php if ($editRow && $hasUserEmail): ?>
                        <div class="pcs-field pcs-full" style="margin-top:10px;">
                            <label class="pcs-field" style="flex-direction:row;align-items:center;gap:8px;font-weight:500;">
                                <input type="checkbox" name="email_reset_password" id="pcs-email-reset-pw" value="1" class="form-check-
input" style="width:auto;">
                                <span>Email this new password</span>
                            </label>
                            <label class="pcs-field" style="flex-direction:row;align-items:center;gap:8px;font-weight:500;margin-
top:6px;">
                                <input type="checkbox" name="generate_temp_password_email" id="pcs-gen-email-pw" value="1" class="form-
check-input" style="width:auto;">
                                <span>Generate temporary password and email it</span>
                            </label>
                        </div>
                    <?php endif; ?>
                </div>
                <div class="pcs-field">
                    <label for="pcs-fullname">Full name <span class="pcs-opt">*</span></label>
                    <input id="pcs-fullname" type="text" name="full_name" required maxlength="100" value="<?= htmlspecialchars((string)
($editRow['full_name'] ?? '')) ?>" autocomplete="name">
                </div>
                <div class="pcs-field">
                    <label for="pcs-stunum">Student number <span class="pcs-opt">(optional)</span></label>
                    <input id="pcs-stunum" type="text" name="student_number" maxlength="30" value="<?= htmlspecialchars((string)
($editRow['student_number'] ?? '')) ?>">
                </div>
                <?php if ($hasUserEmail): ?>
                    <div class="pcs-field pcs-full">
                        <label for="pcs-email">Email <span class="pcs-opt">(optional)</span></label>
                        <input id="pcs-email" type="email" name="email" maxlength="190" value="<?= htmlspecialchars((string)
($editRow['email'] ?? '')) ?>" autocomplete="email">
                        <?php if (!$editRow): ?>
                            <div class="pcs-field" style="margin-top:10px;">
                                <label class="pcs-field" style="flex-direction:row;align-items:center;gap:8px;font-weight:500;">
                                    <input type="checkbox" name="send_credentials_email" id="pcs_send_credentials_email" value="1"
class="form-check-input" style="width:auto;" checked="">
                                    <span>Generate temporary password and email it</span>
                                </label>
                            </div>
                        <?php endif; ?>
                    </div>
                </div>
            </div>
        </form>
    </div>
</div>

```

```

        </div>
        <?php endif; ?>
        <?php if ($editRow): ?>
            <div class="pcs-field">
                <label for="pcs-status">Account status</label>
                <select id="pcs-status" name="is_active">
                    <option value="1" <?= (int) ($editRow['is_active']) ?? 1) === 1 ? 'selected' : '' ?>>Active</option>
                    <option value="0" <?= (int) ($editRow['is_active']) ?? 1) === 0 ? 'selected' : '' ?>>Disabled</option>
                </select>
            </div>
            <?php endif; ?>
        </div>
        <div class="pcs-btn-group">
            <button type="submit" class="pcs-btn-primary"><i class="fa-solid fa-floppy-disk" aria-hidden="true"></i> Save</button>
            <?php if ($editRow): ?><a href="program_chair_students.php" class="pcs-btn-secondary">Cancel</a><?php endif; ?>
        </div>
    </form>
</div>
</div>

<?php if (!$editRow && $hasUserEmail): ?>
<script>
(function () {
    var cb = document.getElementById('pcs_send_credentials_email');
    var pw = document.getElementById('pcs-password');
    var em = document.getElementById('pcs-email');
    if (!cb || !pw) return;
    function sync() {
        var on = cb.checked;
        pw.required = !on;
        pw.disabled = on;
        if (on) pw.value = '';
        if (em) em.required = on;
    }
    cb.addEventListener('change', sync);
    sync();
})();
</script>
<?php endif; ?>

<?php if ($editRow && $hasUserEmail): ?>
<script>
(function () {
    var gen = document.getElementById('pcs-gen-email-pw');
    var pw = document.getElementById('pcs-password');
    if (!gen || !pw) return;
    function sync() {
        if (gen.checked) {
            pw.value = '';
            pw.disabled = true;
        } else {
            pw.disabled = false;
        }
    }
    gen.addEventListener('change', sync);
    sync();
})();
</script>
<?php endif; ?>

<?php if (!$editRow): ?>
<div class="pcs-card">
    <div class="pcs-card-header">
        <h2><i class="fa-solid fa-upload" aria-hidden="true"></i> Batch upload (CSV)</h2>
    </div>
    <div class="pcs-card-body pcs-batch-zone">
        <div class="pcs-batch-inner">
            <p><i class="fa-solid fa-circle-info" aria-hidden="true"></i> Upload a spreadsheet saved as CSV. First row must match:
                <strong>username, password, full_name</strong><?= $hasUserEmail ? ', <strong>email</strong> (optional)' : '' ?>,
                <strong>student_number</strong> (optional). Up to 1000 rows, max 2&nbsp;MB.</p>
            <div class="pcs-file-row">
                <a href="program_chair_students.php?download_students_csv_template=1" class="pcs-btn-outline"><i class="fa-solid fa-download" aria-hidden="true"></i> Download CSV template</a>
                <form method="post" enctype="multipart/form-data" id="pcs-batch-form" class="pcs-file-row" style="flex:1;min-width:200px;">
                    <input type="hidden" name="action" value="batch_upload">
                    <label for="pcs-csv" class="pcs-btn-secondary" style="cursor:pointer;margin:0;"><i class="fa-solid fa-folder-open" aria-hidden="true"></i> Choose file</label>
                    <input id="pcs-csv" type="file" name="batch_csv" accept=".csv,text/csv" required class="visually-hidden">
                    <span id="pcs-file-label" class="pcs-fake-file">No file chosen</span>
                </form>
            </div>
        </div>
    </div>
</div>
</div>

```

```

        <button type="submit" class="pcs-btn-primary"><i class="fa-solid fa-users" aria-hidden="true"></i> Import
students</button>
    </form>
</div>
</div>
</div>
</div>
<?php endif; ?>

<div class="pcs-card" style="margin-bottom:0;">
    <div class="pcs-card-header">
        <h2><i class="fa-solid fa-list-ul" aria-hidden="true"></i> Enrolled students</h2>
    </div>
    <div class="pcs-card-body">
        <div class="pcs-controls">
            <div class="pcs-search">
                <i class="fa-solid fa-magnifying-glass" aria-hidden="true"></i>
                <label class="visually-hidden" for="pcs-search-input">Search students</label>
                <input type="search" id="pcs-search-input" placeholder="Search name, username, email, instructor..." autocomplete="off">
            </div>
            <div>
                <span class="pcs-total-pill">Total: <span id="pcs-total">0</span> students</span>
            </div>
        </div>
        <div class="pcs-table-wrap">
            <table class="pcs-table" id="pcs-student-table">
                <thead>
                    <tr>
                        <th scope="col">Name</th>
                        <th scope="col">Username</th>
                        <th scope="col">Student #</th>
                        <?php if ($hasUserEmail): ?><th scope="col">Email</th><?php endif; ?>
                        <th scope="col">Status</th>
                        <th scope="col">Actions</th>
                    </tr>
                </thead>
                <tbody id="pcs-table-body"></tbody>
            </table>
        </div>
        <div class="pcs-pagination" id="pcs-pagination" aria-label="Pagination"></div>
    </div>
</div>
</div>

<form id="pcs-delete-form" method="post" class="visually-hidden" aria-hidden="true">
    <input type="hidden" name="action" value="delete">
    <input type="hidden" name="id" id="pcs-delete-id" value="">
</form>

<div id="pcs-delete-modal" class="pcs-modal no-print" role="dialog" aria-modal="true" aria-labelledby="pcs-del-title" aria-hidden="true">
    <div class="pcs-modal-card">
        <i class="fa-solid fa-triangle-exclamation" style="font-size:2rem;color:#e11d48;margin-bottom:10px;" aria-hidden="true"></i>
        <h3 id="pcs-del-title" style="margin:0 0 8px;font-size:1.1rem;">Confirm delete</h3>
        <p id="pcs-delete-text" style="margin:0;color:#475569;font-size:0.9rem;">Remove this student account and their class enrollments?
    </p>
        <div class="pcs-modal-btns">
            <button type="button" class="pcs-btn-secondary" id="pcs-del-cancel">Cancel</button>
            <button type="button" class="pcs-btn-primary pcs-danger" id="pcs-del-confirm">Delete</button>
        </div>
    </div>
</div>
</div>

<script>
(function() {
    var students = <?= json_encode($pcsStudentsPayload, JSON_UNESCAPED_UNICODE | JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_AMP) ?>;
    var hasEmail = <?= $hasUserEmail ? 'true' : 'false' ?>;
    var rowsPerPage = 5;
    var currentPage = 1;
    var searchQuery = '';
    var pendingDeleteId = null;

    function escapeHtml(s) {
        if (!s) return '';
        var d = document.createElement('div');
        d.textContent = s;
        return d.innerHTML;
    }

    function filteredList() {
        var q = searchQuery;

```

```

return students.filter(function(s) {
    if (!q) return true;
    var hay = (s.full_name + ' ' + s.username + ' ' + (s.email || '') + ' ' + (s.student_number || '') + ' ' +
(s.enrolled_faculty_text || '')).toLowerCase();
    return hay.indexOf(q) !== -1;
});
}

function renderTable() {
    var filtered = filteredList();
    var total = filtered.length;
    var totalEl = document.getElementById('pcs-total');
    if (totalEl) totalEl.textContent = String(total);

    var totalPages = Math.max(1, Math.ceil(total / rowsPerPage));
    if (currentPage > totalPages) currentPage = totalPages;
    var start = (currentPage - 1) * rowsPerPage;
    var pageRows = filtered.slice(start, start + rowsPerPage);

    var tbody = document.getElementById('pcs-table-body');
    tbody.innerHTML = '';

    if (pageRows.length === 0) {
        var tr = document.createElement('tr');
        var td = document.createElement('td');
        td.colSpan = hasEmail ? 6 : 5;
        td.className = 'pcs-empty';
        td.textContent = total === 0 && !searchQuery
            ? 'No student accounts yet. Add a student above or import a CSV.'
            : 'No students match your search.';
        tr.appendChild(td);
        tbody.appendChild(tr);
    } else {
        pageRows.forEach(function(s) {
            var tr = document.createElement('tr');
            var active = s.is_active === 1;
            var statusHtml = active
                ? '<span class="pcs-status pcs-status-active"><i class="fa-solid fa-circle" style="font-size:0.5rem;" aria-
hidden="true"></i> Active</span>'
                : '<span class="pcs-status pcs-status-off"><i class="fa-solid fa-circle" style="font-size:0.5rem;" aria-hidden="true">
</i> Disabled</span>';
            var emailCell = hasEmail ? '<td>' + (s.email ? escapeHtml(s.email) : '-') + '</td>' : '';
            tr.innerHTML =
                '<td>' + escapeHtml(s.full_name) + '</td>' +
                '<td>' + escapeHtml(s.username) + '</td>' +
                '<td>' + (s.student_number ? escapeHtml(s.student_number) : '-') + '</td>' +
                emailCell +
                '<td>' + statusHtml + '</td>' +
                '<td class="pcs-actions">' +
                '<a class="pcs-edit" href="program_chair_students.php?edit=' + s.id + '" title="Edit"><i class="fa-solid fa-pen" aria-
hidden="true"></i><span class="visually-hidden">Edit</span></a>' +
                '<button type="button" class="pcs-del" data-id="' + s.id + '" title="Delete"><i class="fa-solid fa-trash-can" aria-
hidden="true"></i><span class="visually-hidden">Delete</span></button>' +
                '</td>';
            tbody.appendChild(tr);
        });
    }

    tbody.querySelectorAll('.pcs-del').forEach(function(btn) {
        btn.addEventListener('click', function() {
            var id = parseInt(btn.getAttribute('data-id'), 10);
            pendingDeleteId = id;
            var st = students.find(function(x) { return x.id === id; });
            var msg = document.getElementById('pcs-delete-text');
            if (msg) msg.textContent = st
                ? 'Delete "' + st.full_name + '"? This removes the account and class enrollments.'
                : 'Remove this student account and their class enrollments?';
            var m = document.getElementById('pcs-delete-modal');
            if (m) { m.classList.add('pcs-open'); m.setAttribute('aria-hidden', 'false'); }
        });
    });

    var pag = document.getElementById('pcs-pagination');
    pag.innerHTML = '';
    for (var i = 1; i <= totalPages; i++) {
        var b = document.createElement('button');
        b.type = 'button';
        b.className = 'pcs-page-btn' + (i === currentPage ? ' pcs-page-on' : '');
        b.textContent = String(i);
        if (i === currentPage) b.setAttribute('aria-current', 'page');
    }

```



```

        else b.removeAttribute('aria-current');
        (function(pi) {
            b.addEventListener('click', function() { currentPage = pi; renderTable(); });
        })(i);
        pag.appendChild(b);
    }
}

var searchInput = document.getElementById('pcs-search-input');
if (searchInput) {
    searchInput.addEventListener('input', function() {
        searchQuery = (searchInput.value || '').toLowerCase().trim();
        currentPage = 1;
        renderTable();
    });
}

document.getElementById('pcs-del-cancel').addEventListener('click', function() {
    pendingDeleteId = null;
    var m = document.getElementById('pcs-delete-modal');
    m.classList.remove('pcs-open');
    m.setAttribute('aria-hidden', 'true');
});
document.getElementById('pcs-del-confirm').addEventListener('click', function() {
    if (pendingDeleteId == null) return;
    document.getElementById('pcs-delete-id').value = String(pendingDeleteId);
    document.getElementById('pcs-delete-form').submit();
});
document.getElementById('pcs-delete-modal').addEventListener('click', function(e) {
    if (e.target === this) {
        pendingDeleteId = null;
        this.classList.remove('pcs-open');
        this.setAttribute('aria-hidden', 'true');
    }
});

var csvInput = document.getElementById('pcs-csv');
var fileLabel = document.getElementById('pcs-file-label');
if (csvInput && fileLabel) {
    csvInput.addEventListener('change', function() {
        var f = csvInput.files && csvInput.files[0];
        fileLabel.textContent = f ? f.name : 'No file chosen';
    });
}

function pcsToast(msg, isError) {
    var old = document.querySelector('.pcs-toast');
    if (old) old.remove();
    var t = document.createElement('div');
    t.className = 'pcs-toast' + (isError ? ' pcs-toast-err' : '');
    t.setAttribute('role', 'status');
    t.innerHTML = '<i class="fa-solid ' + (isError ? 'fa-circle-exclamation' : 'fa-circle-check') + '" aria-hidden="true"></i> ' +
escapeHtml(msg);
    document.body.appendChild(t);
    setTimeout(function() { t.remove(); }, 3200);
}

renderTable();
})();
</script>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: program_chair_student_registrations.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/student_registration_helpers.php';

require_role(['program_chair']);
$collegeId = dean_or_program_chair_college_id_or_fail();
$programScope = program_scope_or_fail();
$collegeName = college_name_by_id($collegeId);

if (!student_registration_table_ready()) {

```

```

        exit('Run upgrade_roles.php first to enable student registration approvals.');
```

```

    }

```

```

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

```

```

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    $requestId = (int) ($_POST['request_id'] ?? 0);
    $redirectUrl = 'program_chair_student_registrations.php';
    try {
        if ($action === 'approve_all') {
            $bulk = approve_all_pending_student_registrations(
                (int) $_SESSION['user_id'],
                $collegeId,
                $programScope
            );
            if (($bulk['approved'] ?? 0) > 0) {
                log_dean_activity(
                    'student_registration_approve_all',
                    'Bulk approved ' . (int) $bulk['approved'] . ' student registration(s) for ' . $programScope
                );
            }
            $_SESSION['flash'] = bulk_student_registration_approval_flash_message($bulk);
        } elseif ($action === 'approve' && $requestId > 0) {
            $approval = approve_student_registration_request(
                $requestId,
                (int) $_SESSION['user_id'],
                $collegeId,
                $programScope
            );
            if (!empty($approval['already_approved'])) {
                $_SESSION['flash'] = 'This registration was already approved. The student can sign in with their existing account.';
            } else {
                log_dean_activity(
                    'student_registration_approve',
                    'Approved student registration for ' . $programScope . ' (request #' . $requestId . ')'
                );
                $studentEmail = trim((string) ($approval['email'] ?? ''));
                if ($studentEmail !== '' && filter_var($studentEmail, FILTER_VALIDATE_EMAIL)) {
                    $_SESSION['flash'] = !empty($approval['mail_sent'])
                        ? 'Registration approved. Login credentials sent to ' . $studentEmail . '.'
                        : 'The account has been created successfully, but the email notification could not be sent.';
                } else {
                    $_SESSION['flash'] = 'Registration approved. No email on file – share credentials manually with the student.';
                }
            }
        } elseif ($action === 'reject' && $requestId > 0) {
            $reason = trim((string) ($_POST['rejection_reason'] ?? ''));
            reject_student_registration_request(
                $requestId,
                (int) $_SESSION['user_id'],
                $collegeId,
                $programScope,
                $reason
            );
            log_dean_activity(
                'student_registration_reject',
                'Rejected student registration for ' . $programScope . ' (request #' . $requestId . ')'
            );
            $_SESSION['flash'] = 'Registration request rejected.';
        } else {
            throw new RuntimeException('Invalid action.');
```

```

        }
    } catch (Throwable $e) {
        $_SESSION['flash'] = 'Error: ' . $e->getMessage();
    }
    header('Location: ' . $redirectUrl);
    exit;
}

```

```

$pending = pending_registrations_for_program($collegeId, $programScope);

```

```

$pageTitle = 'Student registration approvals';
$mainContainerClass = 'container-fluid py-4 py-md-5 app-main';
require_once __DIR__ . '/includes/header.php';
?>

```

```

<div class="container" style="max-width: 1100px;">
    <div class="d-flex flex-wrap justify-content-between align-items-center gap-3 mb-4">
        <div>

```

```

        <h1 class="h3 mb-1">Student registration approvals</h1>
        <p class="text-muted mb-0">
            <?= htmlspecialchars($collegeName) ?> - <?= htmlspecialchars($programScope) ?>
        </p>
    </div>
    <?php if ($pending !== []): ?>
        <form method="post" class="d-inline">
            <input type="hidden" name="action" value="approve_all">
            <button type="submit" class="btn btn-success"
                onclick="return confirm('Approve all <?= (int) count($pending) ?> pending registration(s)? Each student will receive
an email with a temporary password when an email address is on file.');">
                <i class="fa-solid fa-check-double me-1"></i> Approve all (<?= (int) count($pending) ?>)
            </button>
        </form>
    <?php endif; ?>
</div>

<?php if ($flash !== ''): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if ($pending === []): ?>
    <div class="card border-0 shadow-sm rounded-4">
        <div class="card-body text-center py-5 text-muted">
            <i class="fa-solid fa-inbox fa-2x mb-3 d-block opacity-50"></i>
            No pending student registrations for <?= htmlspecialchars($programScope) ?>.
        </div>
    </div>
<?php else: ?>
    <div class="card border-0 shadow-sm rounded-4">
        <div class="table-responsive">
            <table class="table table-hover align-middle mb-0">
                <thead class="table-light">
                    <tr>
                        <th>Student</th>
                        <th>Username</th>
                        <th>Student #</th>
                        <th>Year</th>
                        <th>Email</th>
                        <th>Submitted</th>
                        <th class="text-end">Actions</th>
                    </tr>
                </thead>
                <tbody>
                    <?php foreach ($pending as $row): ?>
                        <tr>
                            <td class="fw-semibold"><?= htmlspecialchars((string) $row['full_name']) ?></td>
                            <td><code><?= htmlspecialchars((string) $row['username']) ?></code></td>
                            <td><?= htmlspecialchars((string) $row['student_number']) ?></td>
                            <td><?= htmlspecialchars((string) ($row['year_level'] ?? '')) ?>: '-'</td>
                            <td><?= htmlspecialchars((string) ($row['email'] ?? '')) ?>: '-'</td>
                            <td class="text-muted small"><?= htmlspecialchars((string) ($row['created_at'] ?? '')) ?></td>
                            <td class="text-end text-nowrap">
                                <form method="post" class="d-inline">
                                    <input type="hidden" name="action" value="approve">
                                    <input type="hidden" name="request_id" value="<?= (int) $row['id'] ?>">
                                    <button type="submit" class="btn btn-success btn-sm"
                                        onclick="return confirm('Approve this student registration? They will receive an email with
a temporary password to sign in.');">
                                        <i class="fa-solid fa-check me-1"></i> Approve
                                    </button>
                                </form>
                                <button type="button" class="btn btn-outline-danger btn-sm ms-1"
                                    data-bs-toggle="modal" data-bs-target="#rejectModal<?= (int) $row['id'] ?>">
                                    <i class="fa-solid fa-xmark me-1"></i> Reject
                                </button>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                </tbody>
            </table>
        </div>
    </div>
    <?php foreach ($pending as $row): ?>
        <div class="modal fade" id="rejectModal<?= (int) $row['id'] ?>" tabindex="-1" aria-hidden="true">
            <div class="modal-dialog">
                <div class="modal-content rounded-4">
                    <form method="post">
                        <input type="hidden" name="action" value="reject">
                        <input type="hidden" name="request_id" value="<?= (int) $row['id'] ?>">
                        <div class="modal-header">
                            <h5 class="modal-title">Reject registration</h5>

```

```

        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
    </div>
    <div class="modal-body">
        <p>Reject registration for <strong><?= htmlspecialchars((string) $row['full_name']) ?></strong></p>
        <label class="form-label" for="reason<?= (int) $row['id'] ?>">Reason (optional)</label>
        <textarea class="form-control" id="reason<?= (int) $row['id'] ?>" name="rejection_reason" rows="3"
            placeholder="e.g. Wrong program or invalid student number"></textarea>
    </div>
    <div class="modal-footer">
        <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Cancel</button>
        <button type="submit" class="btn btn-danger">Reject</button>
    </div>
</form>
</div>
</div>
</div>
<?php endforeach; ?>
<?php endif; ?>
</div>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: reports.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['admin', 'dean', 'program_chair', 'gened', 'faculty']);
$role = (string) ($_SESSION['role'] ?? '');
$collegeId = current_college_id();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
if (is_dean()) {
    $collegeId = dean_college_id_or_fail();
} elseif (is_program_chair()) {
    $collegeId = dean_or_program_chair_college_id_or_fail();
}
$facultySelfId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;

$sem = trim((string) ($_GET['semester'] ?? ''));
$sy = trim((string) ($_GET['school_year'] ?? ''));
$facultySearch = trim((string) ($_GET['faculty_search'] ?? ''));
$adminCollegeId = (int) ($_GET['college_id'] ?? 0);

$reportsListParams = array_filter([
    'semester' => $sem !== '' ? $sem : null,
    'school_year' => $sy !== '' ? $sy : null,
    'faculty_search' => ($facultySearch !== '' && $role !== 'faculty') ? $facultySearch : null,
    'college_id' => ($role === 'admin' && $adminCollegeId > 0) ? (string) $adminCollegeId : null,
], static fn ($v) => $v !== null && $v !== '');
$reportsListUrl = 'reports.php' . ($reportsListParams !== [] ? '?' . http_build_query($reportsListParams) : '');

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['delete_ids'])) {
    $ids = array_values(array_unique(array_filter(
        array_map('intval', preg_split('/\s*,\s*/', (string) $_POST['delete_ids']))
    )));
    $deletedCount = 0;
    $lastMessage = '';

    foreach ($ids as $sid) {
        if ($sid < 1) {
            continue;
        }
        if (is_admin()) {
            $snap = db()->prepare(
                'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year,
                    s.day_of_week, s.start_time, s.end_time, s.schedule_type,
                    f.full_name AS faculty_name, c.course_code, c.course_name, r.room_code,
                    COALESCE(col.college_code, "") AS college_code
                FROM schedules s
                INNER JOIN faculty f ON f.id = s.faculty_id
                INNER JOIN courses c ON c.id = s.course_id
                INNER JOIN rooms r ON r.id = s.room_id
            ');

```

```

        LEFT JOIN colleges col ON col.id = s.college_id
        WHERE s.id = ? LIMIT 1'
    );
    $snap->execute([$sid]);
    $schedBefore = $snap->fetch(PDO::FETCH_ASSOC);
    $stmt = db()->prepare('DELETE FROM schedules WHERE id=?');
    $stmt->execute([$sid]);
    if ($schedBefore) {
        log_admin_activity(
            'delete',
            'Schedules',
            'Schedule #' . $sid,
            (array) $schedBefore,
            null
        );
        $deletedCount++;
    }
    $lastMessage = 'Course schedule deleted by admin emergency override.';
} elseif ((is_dean() || is_program_chair()) && $collegeId) {
    $check = db()->prepare(
        'SELECT s.id, COALESCE(u.role, "") AS creator_role,
            c.department AS course_dept, f.department AS faculty_dept
        FROM schedules s
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN faculty f ON f.id = s.faculty_id
        LEFT JOIN users u ON u.id = s.created_by
        WHERE s.id=? AND s.college_id=? LIMIT 1'
    );
    $check->execute([$sid, $collegeId]);
    $row = $check->fetch();
    if (!$row) {
        $lastMessage = 'Schedule not found.';
    } elseif ($programScope !== null
        && trim((string) ($row['course_dept'] ?? '')) !== $programScope
        && trim((string) ($row['faculty_dept'] ?? '')) !== $programScope) {
        $lastMessage = 'This schedule is outside your assigned program.';
    } elseif ((string) $row['creator_role'] === 'gened') {
        $lastMessage = 'This GE-created schedule is read-only for Deans.';
    } else {
        $snapPc = db()->prepare(
            'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year,
                s.day_of_week, s.start_time, s.end_time, s.schedule_type,
                f.full_name AS faculty_name, c.course_code, c.course_name, r.room_code
            FROM schedules s
            INNER JOIN faculty f ON f.id = s.faculty_id
            INNER JOIN courses c ON c.id = s.course_id
            INNER JOIN rooms r ON r.id = s.room_id
            WHERE s.id = ? AND s.college_id = ? LIMIT 1'
        );
        $snapPc->execute([$sid, $collegeId]);
        $schedBeforePc = $snapPc->fetch(PDO::FETCH_ASSOC);
        $stmt = db()->prepare('DELETE FROM schedules WHERE id=? AND college_id=?');
        $stmt->execute([$sid, $collegeId]);
        if ($schedBeforePc) {
            log_user_activity('delete', 'Schedules', 'Schedule #' . $sid, (array) $schedBeforePc, null);
            $deletedCount++;
        }
        log_dean_activity('schedule_delete', 'Deleted schedule #' . $sid);
        $lastMessage = 'Course schedule deleted.';
    }
} elseif (is_gened()) {
    $snapGe = db()->prepare(
        'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year,
            s.day_of_week, s.start_time, s.end_time, s.schedule_type,
            f.full_name AS faculty_name, c.course_code, c.course_name, r.room_code
        FROM schedules s
        INNER JOIN users u ON u.id = s.created_by
        INNER JOIN faculty f ON f.id = s.faculty_id
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN rooms r ON r.id = s.room_id
        WHERE s.id = ? AND u.role = "gened" LIMIT 1'
    );
    $snapGe->execute([$sid]);
    $schedBeforeGe = $snapGe->fetch(PDO::FETCH_ASSOC);
    $stmt = db()->prepare(
        'DELETE s FROM schedules s
        INNER JOIN users u ON u.id = s.created_by
        WHERE s.id=? AND u.role="gened"'
    );
    $stmt->execute([$sid]);
}

```

```

        if ($stmt->rowCount() > 0 && $schedBeforeGe) {
            log_user_activity('delete', 'Schedules', 'Schedule #' . $sid, (array) $schedBeforeGe, null);
            $deletedCount++;
            $lastMessage = 'GE course schedule deleted.';
        } else {
            $lastMessage = 'Only GE-created schedules can be deleted by GEN ED account.';
        }
    }
}

if ($deletedCount > 0) {
    $_SESSION['flash'] = $deletedCount === 1
        ? ($lastMessage !== '' ? $lastMessage : 'Schedule deleted.')
        : 'Course schedule deleted (' . $deletedCount . ' rows).';
} elseif ($lastMessage !== '') {
    $_SESSION['flash'] = $lastMessage;
} else {
    $_SESSION['flash'] = 'No schedule rows were deleted.';
}
header('Location: ' . $reportsListUrl);
exit;
}

$canDeleteSchedules = is_admin() || is_dean() || is_program_chair() || is_gened();

$collegeFilterOptions = [];
if ($role === 'admin') {
    $collegeFilterOptions = db()->query(
        'SELECT id, college_code, college_name FROM colleges ORDER BY college_code'
    )->fetchAll();
}

$hasLectureUnits = db_column_exists('courses', 'lecture_units');
$hasLaboratoryUnits = db_column_exists('courses', 'laboratory_units');
$hasIsGenedCourse = db_column_exists('courses', 'is_gened');
$hasGeTargetsTable = db_table_exists('ge_schedule_targets');
$targetJoin = $hasGeTargetsTable ? ' LEFT JOIN ge_schedule_targets gst ON gst.schedule_id = s.id ' : '';

$courseUnitsSelect = ', c.units AS course_units_total';
if ($hasLectureUnits) {
    $courseUnitsSelect .= ', c.lecture_units';
}
if ($hasLaboratoryUnits) {
    $courseUnitsSelect .= ', c.laboratory_units';
}
$hasCourseLabFlag = db_column_exists('courses', 'is_laboratory');
$courseLabSelect = $hasCourseLabFlag ? ', c.is_laboratory' : '';
$hasSessionKind = db_column_exists('schedules', 'session_kind');
$sessionKindSelect = $hasSessionKind ? ', s.session_kind' : '';

$sql = "SELECT s.*, f.full_name AS faculty_name, f.department AS fac_dept, c.course_code, c.course_name, c.department AS course_dept,
r.room_code, r.type AS room_type,
    COALESCE(u.role, '') AS creator_role
    {$courseUnitsSelect}
    {$courseLabSelect}
    {$sessionKindSelect}
FROM schedules s
INNER JOIN faculty f ON f.id = s.faculty_id
INNER JOIN courses c ON c.id = s.course_id
INNER JOIN rooms r ON r.id = s.room_id
LEFT JOIN users u ON u.id = s.created_by
    {$targetJoin}
WHERE 1=1";

$params = [];
if ($role === 'dean' && $collegeId) {
    $hasIsGenedCourseCol = db_column_exists('courses', 'is_gened');
    $sql .= dean_schedule_scope_sql($collegeId, $hasIsGenedCourseCol, $params, $hasGeTargetsTable);
} elseif ($programScope !== null && $collegeId) {
    $sql .= program_chair_schedule_scope_sql($collegeId, $programScope, $hasGeTargetsTable, $params);
} elseif ($role === 'faculty' && $facultySelfId > 0) {
    $sql .= ' AND s.faculty_id = ?';
    $params[] = $facultySelfId;
} elseif ($role === 'admin') {
    if ($adminCollegeId > 0) {
        $sql .= ' AND s.college_id = ?';
        $params[] = $adminCollegeId;
    }
}
if ($role === 'gened') {
    if ($hasIsGenedCourse) {

```

```

        $sql .= ' AND COALESCE(c.is_gened, 0) = 1';
    } else {
        $sql .= ' AND c.department = ?';
        $params[] = 'General Education';
    }
}
if (is_program_chair()) {
    if ($hasIsGenedCourse) {
        $sql .= ' AND COALESCE(c.is_gened, 0) = 0';
    } else {
        $sql .= ' AND COALESCE(u.role, "") != "gened"';
    }
}
if ($facultySearch != '' && $role != 'faculty') {
    $sql .= ' AND (f.full_name LIKE ? OR f.faculty_id LIKE ?)';
    $params[] = '%' . $facultySearch . '%';
    $params[] = '%' . $facultySearch . '%';
}
if ($sem != '') {
    $sql .= ' AND s.semester = ?';
    $params[] = $sem;
}
if ($sy != '') {
    $sql .= ' AND s.school_year = ?';
    $params[] = $sy;
}
$sql .= ' ORDER BY f.full_name, s.start_time';

$stmt = db()->prepare($sql);
$stmt->execute($params);
$rows = $stmt->fetchAll();

$sems = db()->query('SELECT DISTINCT semester FROM schedules ORDER BY semester')->fetchAll(PDO::FETCH_COLUMN);
$years = db()->query('SELECT DISTINCT school_year FROM schedules ORDER BY school_year DESC')->fetchAll(PDO::FETCH_COLUMN);

$byFaculty = [];
foreach ($rows as $r) {
    $k = $r['faculty_name'];
    if (!isset($byFaculty[$k])) {
        $byFaculty[$k] = [];
    }
    $byFaculty[$k][] = $r;
}

$formatTime12h = static function (?string $time): string {
    $raw = substr((string) $time, 0, 5);
    $dt = DateTime::createFromFormat('H:i', $raw);
    return $dt ? $dt->format('g:i A') : $raw;
};

$sessionHoursBetween = static function (?string $start, ?string $end): float {
    $rawS = substr((string) $start, 0, 8);
    $rawE = substr((string) $end, 0, 8);
    $ds = DateTime::createFromFormat('H:i:s', $rawS) ?: DateTime::createFromFormat('H:i', substr((string) $start, 0, 5));
    $de = DateTime::createFromFormat('H:i:s', $rawE) ?: DateTime::createFromFormat('H:i', substr((string) $end, 0, 5));
    if (!$ds || !$de) {
        return 0.0;
    }
    $secs = $de->getTimestamp() - $ds->getTimestamp();
    if ($secs <= 0) {
        return 0.0;
    }
    return round($secs / 3600, 2);
};

$courseTotalUnits = static function (array $row) use ($hasLectureUnits, $hasLaboratoryUnits): float {
    if ($hasLectureUnits && $hasLaboratoryUnits) {
        return (float) ($row['lecture_units'] ?? 0) + (float) ($row['laboratory_units'] ?? 0);
    }
    return (float) ($row['course_units_total'] ?? 0);
};

/** One row per scheduled offering (course may have multiple time slots). */
$scheduleOfferingKey = static function (array $row): string {
    return (int) ($row['faculty_id'] ?? 0) . '|'
        . (int) ($row['course_id'] ?? 0) . '|'
        . (string) ($row['semester'] ?? '') . '|'
        . (string) ($row['school_year'] ?? '') . '|'
        . (int) ($row['college_id'] ?? 0);
};

```

```

$mergeOfferingRows = static function (array $group) use (
    $courseTotalUnits,
    $sessionHoursBetween,
    $formatTime12h
): array {
    $r0 = $group[0];
    $lecHours = 0.0;
    $labHours = 0.0;
    $dayParts = [];
    $timeParts = [];
    $roomParts = [];
    $roomSeen = [];
    foreach ($group as $r) {
        $sessionH = $sessionHoursBetween((string) ($r['start_time'] ?? ''), (string) ($r['end_time'] ?? ''));
        $dayCount = count(parse_day_set((string) ($r['day_of_week'] ?? '')));
        if ($dayCount < 1) {
            $dayCount = 1;
        }
        $hours = $sessionH * $dayCount;
        if (schedule_session_is_laboratory(
            (string) ($r['start_time'] ?? ''),
            (string) ($r['end_time'] ?? ''),
            (string) ($r['room_type'] ?? ''),
            isset($r['session_kind']) ? (string) $r['session_kind'] : null,
            array_key_exists('is_laboratory', $r) ? ((int) ($r['is_laboratory'] ?? 0) === 1) : null
        )) {
            $labHours += $hours;
        } else {
            $lecHours += $hours;
        }
        $dayParts[] = format_day_set_abbrev((string) ($r['day_of_week'] ?? ''));
        $timeParts[] = $formatTime12h((string) ($r['start_time'] ?? ''))
            . ' - '
            . $formatTime12h((string) ($r['end_time'] ?? ''));
        $roomCode = trim((string) ($r['room_code'] ?? ''));
        if ($roomCode !== '' && !isset($roomSeen[$roomCode])) {
            $roomSeen[$roomCode] = true;
            $roomParts[] = $roomCode;
        }
    }

    return [
        'course_code' => (string) ($r0['course_code'] ?? ''),
        'course_name' => (string) ($r0['course_name'] ?? ''),
        'units' => $courseTotalUnits($r0),
        'lec_hours' => $lecHours,
        'lab_hours' => $labHours,
        'total_hours' => $lecHours + $labHours,
        'day_display' => implode(' / ', array_values(array_unique($dayParts))),
        'time_display' => implode('; ', array_values(array_unique($timeParts))),
        'room_display' => $roomParts !== [] ? implode('; ', $roomParts) : '-',
    ];
};

$pageTitle = 'Schedule reports';
require_once __DIR__ . '/includes/header.php';
?>
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-4">
    <h1 class="h3 mb-0"><i class="fa-solid fa-file-lines me-2 text-primary"></i>Schedule reports</h1>
    <button type="button" class="btn btn-outline-secondary no-print" onclick="window.print()"><? app_tooltip_attr('Prints the report using
your browser. Use this for a clean paper or PDF copy.') ?><i class="fa-solid fa-print me-1"></i>Print</button>
</div>
<?php if ($programScope !== null): ?>
    <p class="text-muted">
        Program scope: <strong><? htmlspecialchars($programScope) ?></strong>.
        <?php if (is_program_chair()): ?>
            Includes courses in this program and loads for your program's faculty (including subjects the Dean assigned in other
programs). GE schedules are excluded.
        <?php endif; ?>
    </p>
<?php endif; ?>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<form class="row g-2 mb-4 no-print align-items-end" method="get">
    <div class="col-md-3">
        <label class="form-label small mb-0">Semester</label>
        <select name="semester" class="form-select form-select-sm">
            <option value="">All semesters</option>

```



```

        <?php foreach ($sems as $s): ?>
            <option value="<?> htmlspecialchars((string) $s) ?>" <?> $sem === (string) $s ? 'selected' : '' ?>><?>
htmlspecialchars((string) $s) ?></option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-3">
    <label class="form-label small mb-0">School year</label>
    <select name="school_year" class="form-select form-select-sm">
        <option value="">All years</option>
        <?php foreach ($years as $y): ?>
            <option value="<?> htmlspecialchars((string) $y) ?>" <?> $sy === (string) $y ? 'selected' : '' ?>><?>
htmlspecialchars((string) $y) ?></option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-md-3">
    <button type="submit" class="btn btn-outline-primary btn-sm"<?> app_tooltip_attr('Reloads the report with the selected semester,
year, and filters. Use this after changing criteria.') ?>>Generate</button>
    <a href="reports.php" class="btn btn-outline-secondary btn-sm"<?> app_tooltip_attr('Clears filters and shows the default report view
again.') ?>>Clear</a>
</div>
<?php if ($role !== 'faculty'): ?>
    <div class="col-md-3">
        <label class="form-label small mb-0">Faculty search</label>
        <input type="text" name="faculty_search" class="form-control form-control-sm" placeholder="Name or faculty ID" value="<?>
htmlspecialchars($facultySearch) ?>">
    </div>
<?php endif; ?>
<?php if ($role === 'admin'): ?>
    <div class="col-md-3">
        <label class="form-label small mb-0">College</label>
        <select name="college_id" class="form-select form-select-sm">
            <option value="0">All colleges</option>
            <?php foreach ($collegeFilterOptions as $c): ?>
                <option value="<?> (int) $c['id'] ?>" <?> $adminCollegeId === (int) $c['id'] ? 'selected' : '' ?>>
                    <?> htmlspecialchars($c['college_code'] . ' - ' . $c['college_name']) ?>
                </option>
            <?php endforeach; ?>
        </select>
    </div>
<?php endif; ?>
</form>

<?php if (($facultySearch !== '' && $role !== 'faculty') || ($role === 'admin' && $adminCollegeId > 0)): ?>
    <div class="small text-muted mb-3">
        Active filters:
        <?php if ($facultySearch !== '' && $role !== 'faculty'): ?><span class="badge text-bg-light border me-1">Faculty: <?>
htmlspecialchars($facultySearch) ?></span><?php endif; ?>
        <?php if ($role === 'admin' && $adminCollegeId > 0):
            $selectedCollege = '';
            foreach ($collegeFilterOptions as $c) {
                if ((int) $c['id'] === $adminCollegeId) {
                    $selectedCollege = (string) ($c['college_code'] . ' - ' . $c['college_name']);
                    break;
                }
            }
        ?>
        <span class="badge text-bg-light border">College: <?> htmlspecialchars($selectedCollege) ?></span>
    <?php endif; ?>
</div>
<?php endif; ?>

<?php foreach ($byFaculty as $fname => $items): ?>
    <div class="card shadow-sm mb-4">
        <div class="card-header bg-primary text-white">
            <strong><?> htmlspecialchars($fname) ?></strong>
            <span class="small opacity-75 ms-2"><?> htmlspecialchars($items[0]['fac_dept'] ?? '') ?></span>
        </div>
        <div class="card-body p-0">
            <div class="table-responsive">
                <table class="table table-sm mb-0">
                    <thead class="table-light">
                        <tr>
                            <th>Course</th>
                            <th class="text-end">Units</th>
                            <th class="text-end">LEC hrs</th>
                            <th class="text-end">LAB hrs</th>
                            <th class="text-end">Total hrs</th>
                            <th>Days</th>
                        </tr>
                    </thead>
                </table>
            </div>
        </div>
    </div>
</foreach>

```

```

        <th>Time</th>
        <th>Room</th>
        <?php if ($canDeleteSchedules): ?>
            <th class="no-print text-end">Delete</th>
        <?php endif; ?>
    </tr>
</thead>
<tbody>
    <?php
        $facultyTotalUnits = 0.0;
        $facultyLecHours = 0.0;
        $facultyLabHours = 0.0;
        $offeringGroups = [];
        foreach ($items as $r) {
            $offeringGroups[$scheduleOfferingKey($r)][] = $r;
        }
        ?>
        <?php foreach ($offeringGroups as $group): ?>
            <?php
                $merged = $mergeOfferingRows($group);
                $facultyTotalUnits += $merged['units'];
                $facultyLecHours += $merged['lec_hours'];
                $facultyLabHours += $merged['lab_hours'];
            ?>
            <tr>
                <td><? = htmlspecialchars($merged['course_code']) ?> - <? = htmlspecialchars($merged['course_name']) ?></td>
                <td class="text-end"><? = htmlspecialchars(number_format($merged['units'], 1)) ?></td>
                <td class="text-end"><? = htmlspecialchars(number_format($merged['lec_hours'], 1)) ?></td>
                <td class="text-end"><? = htmlspecialchars(number_format($merged['lab_hours'], 1)) ?></td>
                <td class="text-end"><? = htmlspecialchars(number_format($merged['total_hours'], 1)) ?></td>
                <td><? = htmlspecialchars($merged['day_display']) ?></td>
                <td><? = htmlspecialchars($merged['time_display']) ?></td>
                <td><? = htmlspecialchars($merged['room_display']) ?></td>
                <?php if ($canDeleteSchedules): ?>
                    <?php
                        $deletableIds = [];
                        foreach ($group as $r) {
                            $isGeCreated = ((string) ($r['creator_role'] ?? '')) === 'gened';
                            $courseInProgram = $programScope === null
                                || trim((string) ($r['course_dept'] ?? '')) === $programScope;
                            $canDeleteRow = is_admin()
                                || (is_dean() && !$isGeCreated)
                                || (is_program_chair() && !$isGeCreated && $courseInProgram)
                                || (is_gened() && $isGeCreated);
                            if ($canDeleteRow) {
                                $deletableIds[] = (int) $r['id'];
                            }
                        }
                    ?>
                    <td class="no-print text-end text-nowrap">
                        <?php if ($deletableIds !== []): ?>
                            <form method="post" action="<? = htmlspecialchars($reportsListUrl) ?>" class="d-inline"
                                onsubmit="return confirm('Delete all schedule rows for this course (lecture and laboratory)?');">
                                <input type="hidden" name="delete_ids" value="<? = htmlspecialchars(implode(', ',
                                    $deletableIds)) ?>">
                                <button type="submit" class="btn btn-sm btn-outline-danger"><? = app_tooltip_attr('Deletes all
                                    schedule rows for this course offering (lecture and laboratory) after confirmation.') ?><i class="fa-solid fa-trash"></i></button>
                            </form>
                        <?php endif; ?>
                    </td>
                <?php endif; ?>
            </tr>
        <?php endforeach; ?>
    <tr class="table-light fw-semibold">
        <td class="text-end">TOTAL</td>
        <td class="text-end"><? = htmlspecialchars(number_format($facultyTotalUnits, 1)) ?></td>
        <td class="text-end"><? = htmlspecialchars(number_format($facultyLecHours, 1)) ?></td>
        <td class="text-end"><? = htmlspecialchars(number_format($facultyLabHours, 1)) ?></td>
        <td class="text-end"><? = htmlspecialchars(number_format($facultyLecHours + $facultyLabHours, 1)) ?></td>
        <td colspan="<? = $canDeleteSchedules ? 4 : 3 ?>"></td>
    </tr>
</tbody>
</table>
</div>
</div>
</div>
<?php endforeach; ?>

<?php if ($byFaculty === []): ?>
    <p class="text-muted">No data for the selected filters.</p>

```

```
<?php endif; ?>
```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>
```

```
-----  
FILE: rooms.php  
-----
```

```
<?php  
declare(strict_types=1);  
  
require_once __DIR__ . '/includes/auth.php';  
require_once __DIR__ . '/includes/functions.php';  
require_once __DIR__ . '/includes/admin_activity_log.php';  
  
require_role(['dean']);  
$collegeId = dean_college_id_or_fail();  
$collegeName = college_name_by_id($collegeId);  
  
$flash = $_SESSION['flash'] ?? '';  
unset($_SESSION['flash']);  
$roomTypes = ['lecture', 'laboratory', 'conference', 'tba'];  
$roomStatuses = ['available', 'tba', 'maintenance'];  
  
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    $action = $_POST['action'] ?? '';  
    try {  
        $type = strtolower(trim((string) ($_POST['type'] ?? 'lecture')));  
        $status = strtolower(trim((string) ($_POST['status'] ?? 'available')));  
        if (in_array($type, $roomTypes, true)) {  
            $type = 'lecture';  
        }  
        if (!in_array($status, $roomStatuses, true)) {  
            $status = 'available';  
        }  
        if ($action === 'add') {  
            $useCustomCode = isset($_POST['custom_room_code']) && (string) $_POST['custom_room_code'] === '1';  
            if ($useCustomCode) {  
                $roomCode = trim((string) ($_POST['room_code'] ?? ''));  
                if ($roomCode === '') {  
                    throw new RuntimeException('Enter a room code, or leave Custom code unchecked for auto-generated code.');                }  
            } else {  
                $roomCode = next_auto_room_code_for_college($collegeId, $collegeName);  
            }  
            if (room_code_taken_for_college($roomCode, $collegeId)) {  
                throw new RuntimeException(  
                    'Room code "' . $roomCode . '" is already used in your college. Pick a different code.'  
                );  
            }  
            $stmt = db()->prepare(  
                'INSERT INTO rooms (room_code, room_name, capacity, type, status, college_id) VALUES (?, ?, ?, ?, ?, ?)'  
            );  
            $stmt->execute([  
                $roomCode,  
                trim((string) $_POST['room_name']),  
                (int) $_POST['capacity'],  
                $type,  
                $status,  
                $collegeId,  
            ]);  
            $newRid = (int) db()->lastInsertId();  
            $stR = db()->prepare('SELECT * FROM rooms WHERE id = ? LIMIT 1');  
            $stR->execute([$newRid]);  
            $afterR = $stR->fetch(PDO::FETCH_ASSOC);  
            log_user_activity('add', 'Rooms', 'Room #' . $newRid, null, $afterR ? (array) $afterR : null);  
            log_dean_activity('room_create', 'Created room ' . $roomCode);  
            $_SESSION['flash'] = 'Room added.';  
        } elseif ($action === 'edit' && isset($_POST['id'])) {  
            $roomId = (int) $_POST['id'];  
            $stRb = db()->prepare('SELECT * FROM rooms WHERE id=? AND college_id=? LIMIT 1');  
            $stRb->execute([$roomId, $collegeId]);  
            $beforeR = $stRb->fetch(PDO::FETCH_ASSOC);  
            $roomCode = trim((string) $_POST['room_code']);  
            if (room_code_taken_for_college($roomCode, $collegeId, $roomId)) {  
                throw new RuntimeException(  
                    'Room code "' . $roomCode . '" is already used in your college. Pick a different code.'  
                );  
            }  
            $stmt = db()->prepare(  

```

```

        'UPDATE rooms SET room_code=?, room_name=?, capacity=?, type=?, status=? WHERE id=? AND college_id=?'
    );
    $stmt->execute([
        $roomCode,
        trim((string) $_POST['room_name']),
        (int) $_POST['capacity'],
        $type,
        $status,
        $roomId,
        $collegeId,
    ]);
    $stRa = db()->prepare('SELECT * FROM rooms WHERE id = ? LIMIT 1');
    $stRa->execute([$roomId]);
    $afterR = $stRa->fetch(PDO::FETCH_ASSOC);
    log_user_activity('edit', 'Rooms', 'Room #' . $roomId, $beforeR ? (array) $beforeR : null, $afterR ? (array) $afterR : null);
    log_dean_activity('room_update', 'Updated room ID #' . $roomId);
    $_SESSION['flash'] = 'Room updated.';
} elseif ($action === 'delete' && isset($_POST['id'])) {
    $ridDel = (int) $_POST['id'];
    $stRd = db()->prepare('SELECT * FROM rooms WHERE id=? AND college_id=? LIMIT 1');
    $stRd->execute([$ridDel, $collegeId]);
    $beforeDelR = $stRd->fetch(PDO::FETCH_ASSOC);
    $stmt = db()->prepare('DELETE FROM rooms WHERE id=? AND college_id=?');
    $stmt->execute([$ridDel, $collegeId]);
    log_user_activity('delete', 'Rooms', 'Room #' . $ridDel, $beforeDelR ? (array) $beforeDelR : null, null);
    log_dean_activity('room_delete', 'Deleted room ID #' . $ridDel);
    $_SESSION['flash'] = 'Room removed.';
}
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: rooms.php');
exit;
}

$editRow = null;
if (isset($_GET['edit'])) {
    $st = db()->prepare('SELECT * FROM rooms WHERE id=? AND college_id=?');
    $st->execute([(int) $_GET['edit'], $collegeId]);
    $editRow = $st->fetch() ?: null;
}

$st = db()->prepare('SELECT * FROM rooms WHERE college_id=? ORDER BY room_code ASC');
$st->execute([$collegeId]);
$list = $st->fetchAll();

$nextAutoRoomCode = null;
if (!$editRow) {
    try {
        $nextAutoRoomCode = next_auto_room_code_for_college($collegeId, $collegeName);
    } catch (Throwable) {
        $nextAutoRoomCode = null;
    }
}

$pageTitle = 'Rooms';
require_once __DIR__ . '/includes/header.php';
?>
<h1 class="h3 mb-4"><i class="fa-solid fa-door-open me-2 text-primary"></i>Rooms</h1>
<p class="text-muted">Managing: <strong><? = htmlspecialchars($collegeName) ?></strong></p>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<div class="card shadow-sm mb-4">
    <div class="card-header bg-white">
        <strong><? = $editRow ? 'Edit room' : 'Add room' ?></strong>
    </div>
    <div class="card-body">
        <form method="post" class="row g-3">
            <input type="hidden" name="action" value="<? = $editRow ? 'edit' : 'add' ?>">
            <?php if ($editRow): ?>
                <input type="hidden" name="id" value="<? = (int) $editRow['id'] ?>">
            <?php endif; ?>
            <div class="col-md-3">
                <label class="form-label">Code</label>
                <?php if (!$editRow): ?>
                    <div class="form-check mb-2">
                        <input class="form-check-input" type="checkbox" name="custom_room_code" value="1" id="customRoomCode">
                        <label class="form-check-label small" for="customRoomCode">Custom code</label>
                    </div>
                </div>
            </div>
        </form>
    </div>
</div>

```

```

        <input type="text" name="room_code" id="roomCodeInput" class="form-control" maxlength="20"
            value="" placeholder="Auto if unchecked" autocomplete="off" disabled>
        <?php if ($nextAutoRoomCode != null): ?>
            <div class="form-text small" id="autoCodeHint">Next auto code: <strong><? = htmlspecialchars($nextAutoRoomCode) ?>
</strong></div>
        <?php endif; ?>
    <?php else: ?>
        <input type="text" name="room_code" class="form-control" required maxlength="20"
            value="<? = htmlspecialchars($editRow['room_code']) ?? '' ?>">
        <?php endif; ?>
    </div>
    <div class="col-md-3">
        <label class="form-label">Name</label>
        <input type="text" name="room_name" class="form-control" maxlength="50"
            value="<? = htmlspecialchars($editRow['room_name']) ?? '' ?>">
    </div>
    <div class="col-md-2">
        <label class="form-label">Capacity</label>
        <input type="number" name="capacity" class="form-control" min="0" max="500"
            value="<? = (int) ($editRow['capacity']) ?? 0 ?>">
    </div>
    <div class="col-md-2">
        <label class="form-label">Type</label>
        <select name="type" class="form-select">
            <?php foreach ($roomTypes as $t): ?>
                <?php $label = $t === 'tba' ? 'TBA (To be arranged)' : ucfirst($t); ?>
                <option value="<? = $t ?>" <? = ($editRow['type']) ?? 'lecture' ?> === $t ? 'selected' : '' ?><? = $label ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="col-md-2">
        <label class="form-label">Status</label>
        <select name="status" class="form-select">
            <option value="available" <? = ($editRow['status']) ?? '' ?> === 'available' ? 'selected' : '' ?>>Available</option>
            <option value="tba" <? = ($editRow['status']) ?? '' ?> === 'tba' ? 'selected' : '' ?>>TBA (To be arranged)</option>
            <option value="maintenance" <? = ($editRow['status']) ?? '' ?> === 'maintenance' ? 'selected' : '' ?>>Maintenance</option>
        </select>
    </div>
    <div class="col-12">
        <button type="submit" class="btn btn-primary"><? = app_tooltip_attr($editRow ? 'Saves changes to this room. Use this after
editing code, capacity, type, or status.' : 'Adds the room to your college inventory. Use this so scheduling and auto-assign can pick it.')
?><i class="fa-solid fa-floppy-disk me-1"></i>Save</button>
        <?php if ($editRow): ?>
            <a href="rooms.php" class="btn btn-outline-secondary"><? = app_tooltip_attr('Closes edit mode without saving further
changes.') ?>>Cancel</a>
        <?php endif; ?>
    </div>
</form>
</div>
</div>

<div class="card shadow-sm">
    <div class="card-body p-0">
        <div class="table-responsive">
            <table class="table table-striped table-hover mb-0">
                <thead class="table-light">
                    <tr>
                        <th>Code</th>
                        <th>Name</th>
                        <th>Capacity</th>
                        <th>Type</th>
                        <th>Status</th>
                        <th class="no-print"></th>
                    </tr>
                </thead>
                <tbody>
                    <?php foreach ($list as $r): ?>
                        <tr>
                            <td><? = htmlspecialchars($r['room_code']) ?></td>
                            <td><? = htmlspecialchars($r['room_name']) ?></td>
                            <td><? = (int) $r['capacity'] ?></td>
                            <td><? = htmlspecialchars($r['type']) ?></td>
                            <td>
                                <?php $badge = $r['status'] === 'available' ? 'success' : ($r['status'] === 'tba' ? 'secondary' :
'warning'); ?>
                                <span class="badge bg-<? = $badge ?>"><? = htmlspecialchars($r['status']) ?></span>
                            </td>
                            <td class="no-print text-nowrap">
                                <a href="rooms.php?edit=<? = (int) $r['id'] ?>" class="btn btn-sm btn-outline-primary"><? =
app_tooltip_attr('Edits this room's code, capacity, or availability. Use this when the physical space changes.') ?>><i class="fa-solid fa-

```

```

pen"></i></a>
                <form method="post" class="d-inline" onsubmit="return confirm('Delete this room?');">
                    <input type="hidden" name="action" value="delete">
                    <input type="hidden" name="id" value="<?= (int) $r['id'] ?>">
                    <button type="submit" class="btn btn-sm btn-outline-danger"<?= app_tooltip_attr('Deletes this room after
confirmation. Use only if no schedules should reference it.') ?><i class="fa-solid fa-trash"></i></button>
                </form>
            </td>
        </tr>
    <?php endforeach; ?>
</tbody>
</table>
</div>
</div>
</div>

<?php if (!$editRow): ?>
<script>
(function () {
    var cb = document.getElementById('customRoomCode');
    var inp = document.getElementById('roomCodeInput');
    if (!cb || !inp) return;
    cb.addEventListener('change', function () {
        inp.disabled = !cb.checked;
        inp.required = cb.checked;
        if (!cb.checked) inp.value = '';
    });
})();
</script>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

-----
FILE: schedule.php
-----

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['admin', 'dean', 'program_chair', 'gened', 'faculty']);
$role = (string) ($_SESSION['role'] ?? '');
$collegeId = current_college_id();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
$facultySelfId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$hasGeTargetsTable = db_table_exists('ge_schedule_targets');
$hasCourseLabFlag = db_column_exists('courses', 'is_laboratory');
$hasCourseYearLevel = db_column_exists('courses', 'year_level');
$hasCourseSection = db_column_exists('courses', 'section');
$hasLectureUnits = db_column_exists('courses', 'lecture_units');
$hasLaboratoryUnits = db_column_exists('courses', 'laboratory_units');

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

/**
 * Delete one schedule row when the current user is allowed. Returns true when a row was removed.
 */
$scheduleDeleteOne = static function (int $sid) use ($collegeId, $programScope): bool {
    if (is_admin()) {
        $snap = db()->prepare(
            'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year,
            s.day_of_week, s.start_time, s.end_time, s.schedule_type,
            f.full_name AS faculty_name, c.course_code, c.course_name, r.room_code,
            COALESCE(col.college_code, "") AS college_code
            FROM schedules s
            INNER JOIN faculty f ON f.id = s.faculty_id
            INNER JOIN courses c ON c.id = s.course_id
            INNER JOIN rooms r ON r.id = s.room_id
            LEFT JOIN colleges col ON col.id = s.college_id
            WHERE s.id = ? LIMIT 1'
        );
    }
    $snap->execute([$sid]);
    $schedBefore = $snap->fetch(PDO::FETCH_ASSOC);
    $stmt = db()->prepare('DELETE FROM schedules WHERE id=?');
    $stmt->execute([$sid]);

```

```

        if ($schedBefore) {
            log_admin_activity(
                'delete',
                'Schedules',
                'Schedule #' . $sid,
                (array) $schedBefore,
                null
            );
        }

        return $stmt->rowCount() > 0;
    }
}

if ((is_dean() || is_program_chair()) && $collegeId) {
    $check = db()->prepare(
        'SELECT s.id, COALESCE(u.role, "") AS creator_role,
            c.department AS course_dept, f.department AS faculty_dept
        FROM schedules s
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN faculty f ON f.id = s.faculty_id
        LEFT JOIN users u ON u.id = s.created_by
        WHERE s.id=? AND s.college_id=? LIMIT 1'
    );
    $check->execute([$sid, $collegeId]);
    $row = $check->fetch();
    if (!$row) {
        return false;
    }
    // Match list visibility: program chairs may act when faculty OR course is in their program.
    if ($programScope !== null) {
        $courseDept = trim((string) ($row['course_dept'] ?? ''));
        $facultyDept = trim((string) ($row['faculty_dept'] ?? ''));
        if ($courseDept !== $programScope && $facultyDept !== $programScope) {
            return false;
        }
    }
    if ((string) $row['creator_role'] === 'gened') {
        return false;
    }
    $snapPc = db()->prepare(
        'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year,
            s.day_of_week, s.start_time, s.end_time, s.schedule_type,
            f.full_name AS faculty_name, c.course_code, c.course_name, r.room_code
        FROM schedules s
        INNER JOIN faculty f ON f.id = s.faculty_id
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN rooms r ON r.id = s.room_id
        WHERE s.id = ? AND s.college_id = ? LIMIT 1'
    );
    $snapPc->execute([$sid, $collegeId]);
    $schedBeforePc = $snapPc->fetch(PDO::FETCH_ASSOC);
    $stmt = db()->prepare('DELETE FROM schedules WHERE id=? AND college_id=?');
    $stmt->execute([$sid, $collegeId]);
    if ($stmt->rowCount() > 0 && $schedBeforePc) {
        log_user_activity('delete', 'Schedules', 'Schedule #' . $sid, (array) $schedBeforePc, null);
        log_dean_activity('schedule_delete', 'Deleted schedule #' . $sid);
    }

    return $stmt->rowCount() > 0;
}

if (is_gened()) {
    $snapGe = db()->prepare(
        'SELECT s.id, s.faculty_id, s.course_id, s.room_id, s.college_id, s.semester, s.school_year,
            s.day_of_week, s.start_time, s.end_time, s.schedule_type,
            f.full_name AS faculty_name, c.course_code, c.course_name, r.room_code
        FROM schedules s
        INNER JOIN users u ON u.id = s.created_by
        INNER JOIN faculty f ON f.id = s.faculty_id
        INNER JOIN courses c ON c.id = s.course_id
        INNER JOIN rooms r ON r.id = s.room_id
        WHERE s.id = ? AND u.role = "gened" LIMIT 1'
    );
    $snapGe->execute([$sid]);
    $schedBeforeGe = $snapGe->fetch(PDO::FETCH_ASSOC);
    $stmt = db()->prepare(
        'DELETE s FROM schedules s
        INNER JOIN users u ON u.id = s.created_by
        WHERE s.id=? AND u.role="gened"'
    );
    $stmt->execute([$sid]);
    if ($stmt->rowCount() > 0 && $schedBeforeGe) {

```

```

        log_user_activity('delete', 'Schedules', 'Schedule #' . $sid, (array) $schedBeforeGe, null);
    }

    return $stmt->rowCount() > 0;
}

return false;
};

$scheduleListRedirect = static function (array $returnParams = []): string {
    $query = [];
    foreach (['dept', 'semester', 'school_year', 'faculty_search', 'course'] as $key) {
        $value = trim((string) ($returnParams[$key] ?? ''));
        if ($value !== '') {
            $query[$key] = $value;
        }
    }

    return 'schedule.php' . ($query !== [] ? '?' . http_build_query($query) : '');
};

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $returnParams = [
        'dept' => (string) ($_POST['return_dept'] ?? ''),
        'semester' => (string) ($_POST['return_semester'] ?? ''),
        'school_year' => (string) ($_POST['return_school_year'] ?? ''),
        'faculty_search' => (string) ($_POST['return_faculty_search'] ?? ''),
        'course' => (string) ($_POST['return_course'] ?? ''),
    ];

    if (isset($_POST['delete_id'])) {
        $sid = (int) $_POST['delete_id'];
        if ($scheduleDeleteOne($sid)) {
            if (is_admin()) {
                $_SESSION['flash'] = 'Schedule deleted by admin emergency override.';
            } elseif (is_gened()) {
                $_SESSION['flash'] = 'GE schedule deleted.';
            } else {
                $_SESSION['flash'] = 'Schedule deleted.';
            }
        } else {
            if ((is_dean() || is_program_chair()) && $collegeId) {
                $check = db()->prepare(
                    'SELECT s.id, COALESCE(u.role, "") AS creator_role,
                     c.department AS course_dept, f.department AS faculty_dept
                    FROM schedules s
                    INNER JOIN courses c ON c.id = s.course_id
                    INNER JOIN faculty f ON f.id = s.faculty_id
                    LEFT JOIN users u ON u.id = s.created_by
                    WHERE s.id=? AND s.college_id=? LIMIT 1'
                );
                $check->execute([$sid, $collegeId]);
                $row = $check->fetch();
                if (!$row) {
                    $_SESSION['flash'] = 'Schedule not found.';
                } elseif ($programScope !== null) {
                    $courseDept = trim((string) ($row['course_dept'] ?? ''));
                    $facultyDept = trim((string) ($row['faculty_dept'] ?? ''));
                    if ($courseDept !== $programScope && $facultyDept !== $programScope) {
                        $_SESSION['flash'] = 'This schedule is outside your assigned program.';
                    } elseif ((string) $row['creator_role'] === 'gened') {
                        $_SESSION['flash'] = 'This GE-created schedule is read-only for Deans.';
                    } else {
                        $_SESSION['flash'] = 'Schedule could not be deleted.';
                    }
                } elseif ((string) $row['creator_role'] === 'gened') {
                    $_SESSION['flash'] = 'This GE-created schedule is read-only for Deans.';
                } else {
                    $_SESSION['flash'] = 'Schedule could not be deleted.';
                }
            } elseif (is_gened()) {
                $_SESSION['flash'] = 'Only GE-created schedules can be deleted by GEN ED account.';
            } else {
                $_SESSION['flash'] = 'Schedule could not be deleted.';
            }
        }
    }
    header('Location: ' . $scheduleListRedirect($returnParams));
    exit;
}

```



```

if (isset($_POST['delete_bulk'])) {
    $ids = isset($_POST['ids']) && is_array($_POST['ids'])
        ? array_values(array_unique(array_filter(array_map('intval', $_POST['ids']), static fn (int $id): bool => $id > 0)))
        : [];
    if ($ids === []) {
        $_SESSION['flash'] = 'Select at least one schedule to delete.';
    } else {
        $deleted = 0;
        foreach ($ids as $sid) {
            if ($scheduleDeleteOne($sid)) {
                $deleted++;
            }
        }
        if ($deleted === 0) {
            $_SESSION['flash'] = 'No selected schedules could be deleted.';
        } elseif ($deleted === 1) {
            $_SESSION['flash'] = '1 schedule deleted.';
        } else {
            $_SESSION['flash'] = $deleted . ' schedules deleted.';
        }
    }
    header('Location: ' . $scheduleListRedirect($returnParams));
    exit;
}

$dept = trim((string) ($_GET['dept'] ?? ''));
$sem = trim((string) ($_GET['semester'] ?? ''));
$sy = trim((string) ($_GET['school_year'] ?? ''));
$facultySearch = trim((string) ($_GET['faculty_search'] ?? ''));
$courseSearch = trim((string) ($_GET['course'] ?? ''));

$targetSelect = $hasGeTargetsTable
    ? ", gst.program_name AS target_program, gst.year_level AS target_year_level, gst.section AS target_section"
    : '';
$targetJoin = $hasGeTargetsTable ? ' LEFT JOIN ge_schedule_targets gst ON gst.schedule_id = s.id ' : '';
$courseLabSelect = $hasCourseLabFlag ? ', c.is_laboratory' : '';
$courseBlockSelect = '';
if ($hasCourseYearLevel || $hasCourseSection) {
    $courseBlockSelect = ', c.department AS course_block_department';
    if ($hasCourseYearLevel) {
        $courseBlockSelect .= ', c.year_level AS course_block_year_level';
    }
    if ($hasCourseSection) {
        $courseBlockSelect .= ', c.section AS course_block_section';
    }
}

$courseUnitsSelect = ', c.units AS course_units_total';
if ($hasLectureUnits) {
    $courseUnitsSelect .= ', c.lecture_units';
}
if ($hasLaboratoryUnits) {
    $courseUnitsSelect .= ', c.laboratory_units';
}
$hasSessionKind = db_column_exists('schedules', 'session_kind');
$sessionKindSelect = $hasSessionKind ? ', s.session_kind' : '';
$hasMakeupCol = db_column_exists('schedules', 'is_makeup');
$makeupSelect = $hasMakeupCol ? ', s.is_makeup' : '';

$sql = "SELECT s.id, s.faculty_id, s.course_id, s.schedule_type, s.day_of_week, s.start_time, s.end_time, s.semester, s.school_year,
    s.college_id AS sched_college_id, c.college_id AS course_college_id,
    f.full_name AS faculty_name, f.department AS fac_dept, c.course_code, c.course_name, r.room_code, r.type AS room_type,
    col.college_name, course_col.college_name AS course_college_name, COALESCE(uc.role, '') AS creator_role
    {$courseUnitsSelect}
    {$courseLabSelect}
    {$courseBlockSelect}
    {$sessionKindSelect}
    {$makeupSelect}
    {$targetSelect}
    FROM schedules s
    INNER JOIN faculty f ON f.id = s.faculty_id
    INNER JOIN courses c ON c.id = s.course_id
    INNER JOIN rooms r ON r.id = s.room_id
    LEFT JOIN users uc ON uc.id = s.created_by
    LEFT JOIN colleges col ON col.id = s.college_id
    LEFT JOIN colleges course_col ON course_col.id = c.college_id
    {$targetJoin}
    WHERE 1=1";

$params = [];

```

```

if (is_dean() && $collegeId) {
    $hasIsGenedCourseCol = db_column_exists('courses', 'is_gened');
    $sql .= dean_schedule_scope_sql($collegeId, $hasIsGenedCourseCol, $params, $hasGeTargetsTable);
} elseif ($programScope !== null && $collegeId) {
    $sql .= program_chair_schedule_scope_sql($collegeId, $programScope, $hasGeTargetsTable, $params);
} elseif (is_faculty() && $facultySelfId > 0) {
    $sql .= ' AND s.faculty_id = ?';
    $params[] = $facultySelfId;
}
if ($dept !== '') {
    if ($hasGeTargetsTable) {
        $sql .= ' AND (f.department = ? OR c.department = ? OR gst.program_name = ?)';
        $params[] = $dept;
        $params[] = $dept;
        $params[] = $dept;
    } else {
        $sql .= ' AND (f.department = ? OR c.department = ?)';
        $params[] = $dept;
        $params[] = $dept;
    }
}
if ($sem !== '') {
    $sql .= ' AND s.semester = ?';
    $params[] = $sem;
}
if ($sy !== '') {
    $sql .= ' AND s.school_year = ?';
    $params[] = $sy;
}
if (is_program_chair() && $facultySearch !== '') {
    $sql .= ' AND (f.full_name LIKE ? OR f.faculty_id LIKE ?)';
    $params[] = '%' . $facultySearch . '%';
    $params[] = '%' . $facultySearch . '%';
}
if ($courseSearch !== '') {
    $sql .= ' AND (c.course_code LIKE ? OR c.course_name LIKE ?)';
    $courseLike = '%' . $courseSearch . '%';
    $params[] = $courseLike;
    $params[] = $courseLike;
}
$sql .= ' ORDER BY s.school_year DESC, s.semester, f.full_name, s.start_time';

$stmt = db()->prepare($sql);
$stmt->execute($params);
$rows = $stmt->fetchAll();

/** Group lecture + lab lines for the same course offering into one list row (laboratory courses only). */
$scheduleListGroup = [];
foreach ($rows as $r) {
    $k = (int) $r['faculty_id'] . '|' . (int) $r['course_id'] . '|' . (string) $r['semester'] . '|' . (string) $r['school_year'];
    $scheduleListGroup[$k] = $r;
}
foreach ($scheduleListGroup as &$g) {
    usort($g, static function (array $a, array $b): int {
        $ta = strtolower((string) ($a['room_type'] ?? ''));
        $tb = strtolower((string) ($b['room_type'] ?? ''));
        if ($ta === 'laboratory' && $tb !== 'laboratory') {
            return 1;
        }
        if ($tb === 'laboratory' && $ta !== 'laboratory') {
            return -1;
        }
        return strcmp((string) $a['start_time'], (string) $b['start_time']);
    });
}
unset($g);

$scheduleListDisplay = [];
foreach ($scheduleListGroup as $group) {
    $isLabCourse = $hasCourseLabFlag && isset($group[0]['is_laboratory']) && (int) $group[0]['is_laboratory'] === 1;
    if (count($group) > 1 && $isLabCourse) {
        $scheduleListDisplay[] = ['merged' => true, 'lines' => $group];
    } else {
        foreach ($group as $r) {
            $scheduleListDisplay[] = ['merged' => false, 'lines' => [$r]];
        }
    }
}

$canDeleteSchedule = static function (array $r): bool {

```

```

$isGeCreated = ((string) ($r['creator_role'] ?? '') === 'gened');
if (is_admin()) {
    return true;
}
if ((is_dean() || is_program_chair()) && !$isGeCreated) {
    return true;
}

return is_gened() && $isGeCreated;
};

$scheduleShowBulkDelete = is_admin() || is_dean() || is_program_chair() || is_gened();
$scheduleHasDeletableRows = false;
if ($scheduleShowBulkDelete) {
    foreach ($rows as $r) {
        if ($scanDeleteSchedule($r)) {
            $scheduleHasDeletableRows = true;
            break;
        }
    }
}

$scheduleReturnFilterFields = [
    'return_dept' => $dept,
    'return_semester' => $sem,
    'return_school_year' => $sy,
    'return_faculty_search' => $facultySearch,
    'return_course' => $courseSearch,
];

$formatTime12h = static function (?string $time): string {
    $raw = substr((string) $time, 0, 5);
    $dt = DateTime::createFromFormat('H:i', $raw);
    return $dt ? $dt->format('g:i A') : $raw;
};

/** Laboratory vs lecture badge: prefer session_kind; pure lecture courses stay Lecture even at 3h. */
$segmentIsLaboratory = static function (array $r) use ($hasCourseLabFlag): bool {
    $courseIsLab = null;
    if ($hasCourseLabFlag && array_key_exists('is_laboratory', $r)) {
        $courseIsLab = (int) ($r['is_laboratory'] ?? 0) === 1;
    }
    return schedule_session_is_laboratory(
        (string) ($r['start_time'] ?? ''),
        (string) ($r['end_time'] ?? ''),
        (string) ($r['room_type'] ?? ''),
        isset($r['session_kind']) ? (string) $r['session_kind'] : null,
        $courseIsLab
    );
};

/** Block label from course program/year/section (Dean module) when row is not a GE target. */
$courseBlockLabelFromRow = static function (array $r) use ($hasCourseYearLevel, $hasCourseSection): ?string {
    if (!$hasCourseYearLevel && !$hasCourseSection) {
        return null;
    }
    $d = trim((string) ($r['course_block_department'] ?? ''));
    if ($d === '') {
        return null;
    }
    $y1 = $hasCourseYearLevel ? trim((string) ($r['course_block_year_level'] ?? '')) : '';
    $sec = $hasCourseSection ? trim((string) ($r['course_block_section'] ?? '')) : '';
    if ($y1 === '' && $sec === '') {
        return null;
    }
    if ($y1 !== '' && $sec !== '') {
        return $d . ' Y' . $y1 . '-' . $sec;
    }
    if ($sec !== '') {
        return $d . ' . Sec ' . $sec;
    }
    return $d . ' . Y' . $y1;
};

if ((is_dean() || is_program_chair()) && $collegeId) {
    $d = db()->prepare('SELECT DISTINCT department FROM faculty WHERE college_id=? AND department != "" ORDER BY department');
    $d->execute([$collegeId]);
    $depts = $d->fetchAll(PDO::FETCH_COLUMN);
} else {

```

```

    $depts = db()->query('SELECT DISTINCT department FROM faculty WHERE department != "" ORDER BY department')->fetchAll(PDO::FETCH_COLUMN);
}
if ($programScope !== null) {
    $depts = [$programScope];
    if ($dept === '') {
        $dept = $programScope;
    }
}
$sems = db()->query('SELECT DISTINCT semester FROM schedules ORDER BY semester')->fetchAll(PDO::FETCH_COLUMN);
$years = db()->query('SELECT DISTINCT school_year FROM schedules ORDER BY school_year DESC')->fetchAll(PDO::FETCH_COLUMN);
$scheduleFilterDeptSemSyCol = is_program_chair() ? 'col-md-2' : 'col-md-3';

$pageTitle = 'Schedules';
require_once __DIR__ . '/includes/header.php';
?>
<style>
    .schedule-list-page {
        --schedule-bg-body: #f1f5f9;
        --schedule-card-bg: #ffffff;
        --schedule-table-header-bg: #f8fafc;
        --schedule-border-color: #e2e8f0;
        --schedule-text-primary: #0f172a;
        --schedule-text-secondary: #334155;
        --schedule-hover-bg: #f1f5f9;
        --schedule-shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.03), 0 1px 2px rgba(0, 0, 0, 0.05);
        background: var(--schedule-bg-body);
        color: var(--schedule-text-primary);
        border-radius: 20px;
        padding: 16px;
    }

    html[data-bs-theme="dark"] .schedule-list-page {
        --schedule-bg-body: #0a0f1c;
        --schedule-card-bg: #0f172a;
        --schedule-table-header-bg: #1e293b;
        --schedule-border-color: #1e2a3e;
        --schedule-text-primary: #e2e8f0;
        --schedule-text-secondary: #94a3b8;
        --schedule-hover-bg: #1e293b;
        --schedule-shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.45);
    }

    .schedule-container {
        max-width: 1400px;
        margin: 0 auto;
    }

    .schedule-header {
        display: flex;
        justify-content: space-between;
        align-items: baseline;
        flex-wrap: wrap;
        margin-bottom: 1.25rem;
        gap: 12px;
    }

    .schedule-title {
        font-size: 1.1rem;
        font-weight: 600;
        letter-spacing: -0.2px;
        color: var(--schedule-text-primary);
        background: var(--schedule-card-bg);
        display: inline-block;
        padding: 0.3rem 0.8rem;
        border-radius: 40px;
        border: 1px solid var(--schedule-border-color);
    }

    .schedule-list-page .schedule-actions .btn {
        border-radius: 999px;
        font-size: 0.75rem;
        padding: 0.35rem 0.75rem;
    }

    .schedule-list-page .schedule-filters {
        background: var(--schedule-card-bg);
        border: 1px solid var(--schedule-border-color);
        border-radius: 20px;
        box-shadow: var(--schedule-shadow-sm);
        padding: 14px;
    }

```

```

}

.schedule-list-page .schedule-filters .form-label {
  font-size: 0.72rem;
  color: var(--schedule-text-secondary);
  text-transform: uppercase;
  letter-spacing: 0.2px;
}

.schedule-list-page .form-select-sm,
.schedule-list-page .btn-sm {
  font-size: 0.75rem;
}

.schedule-list-wrapper {
  overflow-x: auto;
  border-radius: 20px;
  background: var(--schedule-card-bg);
  border: 1px solid var(--schedule-border-color);
  box-shadow: var(--schedule-shadow-sm);
}

.schedule-table {
  width: 100%;
  font-size: 0.75rem;
  font-weight: 450;
  border-collapse: collapse;
  background: var(--schedule-card-bg);
  color: var(--schedule-text-primary);
  min-width: 980px;
  margin-bottom: 0;
}

.schedule-table thead th {
  text-align: left;
  padding: 12px 8px;
  background-color: var(--schedule-table-header-bg);
  font-weight: 600;
  font-size: 0.7rem;
  text-transform: uppercase;
  letter-spacing: 0.3px;
  color: var(--schedule-text-secondary);
  border-bottom: 1px solid var(--schedule-border-color);
}

.schedule-table td {
  padding: 10px 8px;
  border-bottom: 1px solid var(--schedule-border-color);
  vertical-align: middle;
  font-size: 0.73rem;
  color: var(--schedule-text-primary);
}

.schedule-table tr:last-child td {
  border-bottom: none;
}

.schedule-table tbody tr:hover td {
  background-color: var(--schedule-hover-bg);
  transition: 0.1s;
}

.schedule-table th:first-child, .schedule-table td:first-child {
  padding-left: 14px;
}

.schedule-table .schedule-col-select {
  width: 36px;
  text-align: center;
  vertical-align: middle;
}

.schedule-bulk-bar {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: flex-end;
  gap: 8px;
  margin-bottom: 10px;
}

```

```

.schedule-table th:last-child, .schedule-table td:last-child {
  padding-right: 14px;
}

.schedule-table .badge {
  font-size: 0.65rem;
  font-weight: 600;
}

.schedule-table .text-muted,
.schedule-list-page .text-muted {
  color: var(--schedule-text-secondary) !important;
}

.schedule-info-note {
  margin-top: 16px;
  text-align: center;
  font-size: 0.65rem;
  color: var(--schedule-text-secondary);
  border-top: 1px solid var(--schedule-border-color);
  padding-top: 14px;
  letter-spacing: 0.2px;
}

.schedule-info-note span {
  background: var(--schedule-card-bg);
  padding: 2px 8px;
  border-radius: 30px;
}

@media (max-width: 780px) {
  .schedule-list-page {
    padding: 12px;
  }

  .schedule-table thead {
    display: none;
  }

  .schedule-table,
  .schedule-table tbody,
  .schedule-table tr,
  .schedule-table td {
    display: block;
    width: 100%;
  }

  .schedule-table tr {
    background: var(--schedule-card-bg);
    margin-bottom: 16px;
    border: 1px solid var(--schedule-border-color);
    border-radius: 20px;
    padding: 12px 14px;
    box-shadow: var(--schedule-shadow-sm);
  }

  .schedule-table td {
    display: flex;
    justify-content: space-between;
    align-items: baseline;
    gap: 12px;
    padding: 8px 0;
    border-bottom: 1px dashed var(--schedule-border-color);
    font-size: 0.7rem;
    line-height: 1.35;
    word-break: break-word;
  }

  .schedule-table td:last-child {
    border-bottom: none;
  }

  .schedule-table td::before {
    content: attr(data-label);
    font-weight: 600;
    width: 40%;
    min-width: 95px;
    color: var(--schedule-text-secondary);
    font-size: 0.66rem;
  }
}

```

```

        letter-spacing: 0.2px;
        text-transform: uppercase;
        flex-shrink: 0;
    }

    .schedule-table td > * {
        width: 60%;
        text-align: right;
    }

    .schedule-table td[data-label="ACTIONS"] > * {
        width: auto;
        margin-left: auto;
    }
}
</style>
<div class="schedule-list-page">
<div class="schedule-container">
<div class="schedule-header">
    <h1 class="schedule-title"><i class="fa-solid fa-list me-2 text-primary"></i>Schedule list</h1>
    <?php if (is_dean() || is_program_chair() || is_gened()): ?>
        <div class="d-flex gap-2 schedule-actions">
            <?php if (is_dean() || is_program_chair()): ?>
                <a href="add_schedule.php" class="btn btn-primary no-print">? app_tooltip_attr('Creates a new schedule row for a course
section. Use this when manually placing a class in time and room.') ?><i class="fa-solid fa-plus me-1"></i>Add schedule</a>
                <?php if (is_dean()): ?>
                    <a href="auto_schedule.php" class="btn btn-outline-primary no-print">? app_tooltip_attr('Runs automated placement for your
college. Use this to generate or refresh draft schedules.') ?><i class="fa-solid fa-wand-magic-sparkles me-1"></i>Auto schedule</a>
                <?php endif; ?>
            <?php else: ?>
                <a href="gened_schedule.php" class="btn btn-primary no-print">? app_tooltip_attr('Adds a General Education schedule entry.
Use this for GE-specific time and room placement.') ?><i class="fa-solid fa-plus me-1"></i>Add GE schedule</a>
            <?php endif; ?>
        </div>
    <?php endif; ?>
</div>
<?php if ($programScope !== null): ?>
    <p class="text-muted">Program scope: <strong><? htmlspecialchars($programScope) ?></strong></p>
<?php endif; ?>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<form class="row g-2 mb-4 no-print align-items-end schedule-filters" method="get">
    <div class="<? htmlspecialchars($scheduleFilterDeptSemSyCol) ?>">
        <label class="form-label small mb-0">Program</label>
        <select name="dept" class="form-select form-select-sm">
            <option value="">All</option>
            <?php foreach ($depts as $d): ?>
                <option value="<? htmlspecialchars((string) $d) ?>" <? $dept === (string) $d ? 'selected' : '' ?><?
htmlspecialchars((string) $d) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="<? htmlspecialchars($scheduleFilterDeptSemSyCol) ?>">
        <label class="form-label small mb-0">Semester</label>
        <select name="semester" class="form-select form-select-sm">
            <option value="">All</option>
            <?php foreach ($sems as $s): ?>
                <option value="<? htmlspecialchars((string) $s) ?>" <? $sem === (string) $s ? 'selected' : '' ?><?
htmlspecialchars((string) $s) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="<? htmlspecialchars($scheduleFilterDeptSemSyCol) ?>">
        <label class="form-label small mb-0">School year</label>
        <select name="school_year" class="form-select form-select-sm">
            <option value="">All</option>
            <?php foreach ($years as $y): ?>
                <option value="<? htmlspecialchars((string) $y) ?>" <? $sy === (string) $y ? 'selected' : '' ?><?
htmlspecialchars((string) $y) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="col-12 col-md-6 col-xl-4">
        <label class="form-label small mb-0">Course</label>
        <input type="search" name="course" class="form-control form-control-sm" value="<? htmlspecialchars($courseSearch) ?>"
placeholder="Code or title..." maxlength="160" autocomplete="off">? app_tooltip_attr('Narrows the list to schedules whose course code or
course title contains this text (partial match).') ?>
    </div>
    <?php if (is_program_chair()): ?>

```



```

        <?php endif; ?>
    </td>
<?php endif; ?>
<td data-label="TIME / DAYS">
    <?php foreach ($lines as $idx => $r): ?>
        <?php
            $isLabTime = $segmentIsLaboratory($r);
            $segLabel = $isLabTime ? 'Laboratory' : 'Lecture';
            $segBadgeClass = $isLabTime ? 'bg-warning text-dark' : 'bg-info text-dark';
            ?>
            <div class="<?=$idx > 0 ? 'mt-2 pt-2 border-top border-light' : '' ?>">
                <span class="badge bg-secondary"><?=$htmlspecialchars($r['schedule_type']) ?></span>
                <span class="badge <?=$segBadgeClass ?>"><?=$htmlspecialchars($segLabel) ?></span><br>
                <small><?=$htmlspecialchars(str_replace(',', ' ', (string) $r['day_of_week'])) ?></small><br>
                <?=$htmlspecialchars($formatTime12h((string) $r['start_time'])) ?> - <?=$
htmlspecialchars($formatTime12h((string) $r['end_time'])) ?>
            </div>
        <?php endforeach; ?>
    </td>
<td data-label="COURSE"><strong><?=$htmlspecialchars($r0['course_code']) ?></strong><?php if
(!empty($r0['is_makeup'])): ?> <span class="badge bg-warning text-dark">Makeup</span><?php endif; ?><br><span class="small text-muted"><?=$
htmlspecialchars($r0['course_name']) ?></span></td>
<td data-label="FACULTY"><?=$htmlspecialchars($r0['faculty_name']) ?></td>
<td data-label="ROOM">
    <?php foreach ($lines as $idx => $r): ?>
        <?php $roomIsLab = $segmentIsLaboratory($r); ?>
        <div class="<?=$idx > 0 ? 'mt-1' : '' ?>">
            <?=$htmlspecialchars($r['room_code']) ?>
            <?php if ($roomIsLab): ?>
                <span class="badge bg-warning text-dark ms-1">Laboratory</span>
            <?php endif; ?>
        </div>
    <?php endforeach; ?>
</td>
<td data-label="COLLEGE">
    <?php
        $schedCollegeName = trim((string) ($r0['college_name'] ?? ''));
        $courseCollegeName = trim((string) ($r0['course_college_name'] ?? ''));
        $schedCollegeId = (int) ($r0['sched_college_id'] ?? 0);
        $courseCollegeId = (int) ($r0['course_college_id'] ?? 0);
        $isCrossCollegeCourse = $courseCollegeId > 0 && $schedCollegeId > 0 && $courseCollegeId !== $schedCollegeId;
        ?>
        <?=$htmlspecialchars($schedCollegeName) ?>
        <?php if ($isCrossCollegeCourse && $courseCollegeName !== ''): ?>
            <br><span class="badge bg-secondary mt-1">Course · <?=$htmlspecialchars($courseCollegeName) ?></span>
        <?php endif; ?>
    </td>
<?php if ($hasGetTargetsTable): ?>
    <td data-label="TARGET BLOCK">
        <?php
            $shownTarget = false;
            foreach ($lines as $r) {
                if (empty($r['target_program'])) {
                    $shownTarget = true;
                    ?>
                    <span class="small d-block"><?=$htmlspecialchars((string) $r['target_program']) ?> Y<?=$
htmlspecialchars((string) ($r['target_year_level'] ?? '')) ?>-<?=$htmlspecialchars((string) ($r['target_section'] ?? '')) ?></span>
                    <?php
                        break;
                    }
                }
            }
            if (!$shownTarget) {
                $blockLbl = $courseBlockLabelFromRow($r0);
                if ($blockLbl !== null) {
                    echo '<span class="small d-block">' . htmlspecialchars($blockLbl) . '</span>';
                } else {
                    echo '<span class="text-muted small">-</span>';
                }
            }
        <?php
            ?>
        </td>
    <?php endif; ?>
<td data-label="SEMESTER"><?=$htmlspecialchars($r0['semester']) ?></td>
<td data-label="SCHOOL YEAR"><?=$htmlspecialchars($r0['school_year']) ?></td>
<td data-label="ACTIONS" class="no-print text-nowrap">
    <?php if ($deletableIds !== []): ?>
        <form method="post" class="d-inline" onsubmit="return confirm('Delete this schedule<?=$count($deletableIds)
> 1 ? ' (all linked lecture/lab rows)' : '' ?> This cannot be undone.');">
            <input type="hidden" name="delete_bulk" value="1">
            <?php foreach ($deletableIds as $delId): ?>

```

```

        <input type="hidden" name="ids[]" value="<?= $delId ?>">
        <?php endforeach; ?>
        <?php foreach ($scheduleReturnFilterFields as $fieldName => $fieldValue): ?>
            <input type="hidden" name="<?= htmlspecialchars($fieldName) ?>" value="<?= htmlspecialchars((string)
$fieldValue) ?>">
            <?php endforeach; ?>
            <button type="submit" class="btn btn-sm btn-outline-danger" title="Delete"<?=
app_tooltip_attr(count($deletableIds) > 1 ? 'Deletes all linked lecture and laboratory rows for this course offering after confirmation.' :
'Deletes this schedule row after confirmation. Use this when a section is cancelled or duplicated by mistake.') ?><i class="fa-solid fa-
trash"></i></button>
        </form>
        <?php endif; ?>
    </td>
</tr>
<?php endforeach; ?>
</tbody>
</table>
</div>
<div class="schedule-info-note">
    <span>Compact table • mobile cards • dark mode ready</span>
</div>
</div>
</div>

<?php if ($scheduleShowBulkDelete && $scheduleHasDeletableRows): ?>
<script>
(function () {
    var selectAll = document.getElementById('schedule-select-all');
    var deleteBtn = document.getElementById('schedule-bulk-delete-btn');
    if (!selectAll || !deleteBtn) {
        return;
    }

    var rowChecks = function () {
        return Array.prototype.slice.call(document.querySelectorAll('.schedule-row-select'));
    };

    var groupChecks = function () {
        return Array.prototype.slice.call(document.querySelectorAll('.schedule-group-select'));
    };

    var setGroupIdsChecked = function (groupEl, checked) {
        var ids = (groupEl.getAttribute('data-ids') || '').split(',').filter(Boolean);
        rowChecks().forEach(function (el) {
            if (ids.indexOf(String(el.value)) !== -1) {
                el.checked = checked;
            }
        });
        groupEl.checked = checked;
        groupEl.indeterminate = false;
    };

    var syncGroupFromRows = function (groupEl) {
        var ids = (groupEl.getAttribute('data-ids') || '').split(',').filter(Boolean);
        var matched = rowChecks().filter(function (el) {
            return ids.indexOf(String(el.value)) !== -1;
        });
        var selected = matched.filter(function (el) { return el.checked; });
        groupEl.checked = matched.length > 0 && selected.length === matched.length;
        groupEl.indeterminate = selected.length > 0 && selected.length < matched.length;
    };

    var syncBulkDelete = function () {
        var checks = rowChecks();
        var selected = checks.filter(function (el) { return el.checked; });
        deleteBtn.disabled = selected.length === 0;
        selectAll.indeterminate = selected.length > 0 && selected.length < checks.length;
        selectAll.checked = checks.length > 0 && selected.length === checks.length;
        groupChecks().forEach(syncGroupFromRows);
    };

    selectAll.addEventListener('change', function () {
        var checked = selectAll.checked;
        rowChecks().forEach(function (el) {
            el.checked = checked;
        });
        groupChecks().forEach(function (el) {
            el.checked = checked;
            el.indeterminate = false;
        });
    });
});

```

```

        syncBulkDelete();
    });

    groupChecks().forEach(function (el) {
        el.addEventListener('change', function () {
            setGroupIdsChecked(el, el.checked);
            syncBulkDelete();
        });
    });
});

syncBulkDelete();
})();
</script>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: settings.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/admin_activity_log.php';
require_once __DIR__ . '/includes/profile_photo_helpers.php';

require_role(['super_admin', 'admin', 'dean', 'program_chair', 'gened', 'faculty', 'student']);

if (is_admin() || is_super_admin()) {
    admin_activity_log_cleanup();
}

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);
$forcePasswordChange = !empty($_SESSION['must_change_password']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $userId = (int) ($_SESSION['user_id'] ?? 0);
    $settingsAction = (string) ($_POST['settings_action'] ?? 'password');

    try {
        if ($userId < 1) {
            throw new RuntimeException('Invalid user session. Please log in again.');
```

```

if ($settingsAction === 'change_username') {
    if (is_admin() || is_super_admin()) {
        throw new RuntimeException('Administrators cannot change their username here.');
```

```

    }
    if ($forcePasswordChange) {
        throw new RuntimeException('Please change your temporary password before updating your username.');
```

```

    }

    $newUsername = trim((string) ($_POST['new_username'] ?? ''));
    $confirmPassword = (string) ($_POST['confirm_password'] ?? '');

```

```

    if ($newUsername === '' || $confirmPassword === '') {
        throw new RuntimeException('New username and current password are required.');
```

```

    }
    if (!preg_match('/^[a-zA-Z0-9._-]{3,50}$/ ', $newUsername)) {
        throw new RuntimeException('Username must be 3-50 characters (letters, numbers, . _ -).');
```

```

    }

    $st = db()->prepare('SELECT username, password FROM users WHERE id = ? AND is_active = 1 LIMIT 1');
    $st->execute([$userId]);
    $row = $st->fetch(PDO::FETCH_ASSOC);
    if (!$row) {
        throw new RuntimeException('Invalid user session. Please log in again.');
```

```

    }

    $currentUsername = (string) ($row['username'] ?? '');
    $hash = (string) ($row['password'] ?? '');
    if ($hash === '' || !password_verify($confirmPassword, $hash)) {
        throw new RuntimeException('Current password is incorrect.');
```

```

    }
    if (strcasecmp($newUsername, $currentUsername) === 0) {
        throw new RuntimeException('New username must be different from your current username.');
```

```

    }

    $exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username = ? AND id <> ?');
    $exists->execute([$newUsername, $userId]);
    if ((int) $exists->fetchColumn() > 0) {
        $suggestions = suggest_available_usernames($newUsername, 3);
        $msg = 'Username already exists.';
        if ($suggestions) {
            $msg .= ' Try: ' . implode(', ', $suggestions);
        }
        throw new RuntimeException($msg);
    }

```

```

    db()->prepare('UPDATE users SET username = ? WHERE id = ?')
    ->execute([$newUsername, $userId]);

    $_SESSION['username'] = $newUsername;

    log_user_activity(
        'edit',
        'Account',
        'Username changed (own account)',
        null,
        ['username' => ['before' => $currentUsername, 'after' => $newUsername]]
    );
    $_SESSION['flash'] = 'Username updated successfully.';
    header('Location: settings.php');
    exit;
}

```

```

}

$currentPassword = (string) ($_POST['current_password'] ?? '');
$newPassword = (string) ($_POST['new_password'] ?? '');
$confirmPassword = (string) ($_POST['confirm_password'] ?? '');

if ($currentPassword === '' || $newPassword === '' || $confirmPassword === '') {
    throw new RuntimeException('All password fields are required.');
```

```

}
if ($newPassword !== $confirmPassword) {
    throw new RuntimeException('New password and confirmation do not match.');
```

```

}
if (strlen($newPassword) < 8) {
    throw new RuntimeException('New password must be at least 8 characters.');
```

```

}
if ($newPassword === $currentPassword) {
    throw new RuntimeException('New password must be different from current password.');
```

```

    $st = db()->prepare('SELECT password FROM users WHERE id = ? AND is_active = 1 LIMIT 1');
    $st->execute([$userId]);
    $hash = (string) ($st->fetchColumn() ?? '');
    if ($hash === '' || !password_verify($currentPassword, $hash)) {
        throw new RuntimeException('Current password is incorrect.');
```

```

    }

    db()->prepare('UPDATE users SET password = ? WHERE id = ?')
        ->execute([password_hash($newPassword, PASSWORD_DEFAULT), $userId]);
```

```

    clear_must_change_password($userId);
    unset($_SESSION['must_change_password']);
```

```

    log_user_activity(
        'edit',
        'Account',
        'Password changed (own account)',
        null,
        ['password' => '[changed]']
    );
    $_SESSION['flash'] = 'Password updated successfully.';
    header('Location: ' . ($forcePasswordChange ? 'dashboard.php' : 'settings.php'));
    exit;
```

```

} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
    header('Location: settings.php');
    exit;
}
}
```

```

$activityLogOrder = (string) ($_GET['log_sort'] ?? 'newest');
if ($activityLogOrder !== 'oldest') {
    $activityLogOrder = 'newest';
}
$activityLogTableReady = false;
$activityLogs = [];
if (is_admin() || is_super_admin()) {
    $activityLogTableReady = admin_activity_log_table_ready();
    $activityLogs = $activityLogTableReady ? admin_activity_log_list_sorted($activityLogOrder) : [];
}
}
```

```

$userId = (int) ($_SESSION['user_id'] ?? 0);
$canChangeUsername = !is_admin() && !is_super_admin();
$currentUsername = (string) ($_SESSION['username'] ?? '');
$profilePhotoReady = profile_photo_column_ready();
$profilePhotoUrl = $profilePhotoReady ? profile_photo_url($userId) : null;
```

```

$pageTitle = 'Settings';
require_once __DIR__ . '/includes/header.php';
?>
```

```

<h1 class="h3 mb-4"><i class="fa-solid fa-gear me-2 app-role-icon"></i>Account Settings</h1>
<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
<?php if ($forcePasswordChange): ?>
<div class="alert alert-warning shadow-sm" style="max-width: 640px;">
    <strong>Password change required.</strong> You signed in with a temporary password. Choose a new password below before using the rest of
    the system.
</div>
<?php endif; ?>
```

```

<?php if (!$forcePasswordChange): ?>
<?php if ($canChangeUsername): ?>
<div class="card shadow-sm mb-4" style="max-width: 640px;">
    <div class="card-header bg-white"><strong>Change Username</strong></div>
    <div class="card-body">
        <form method="post" class="row g-3">
            <input type="hidden" name="settings_action" value="change_username">
            <div class="col-12">
                <label class="form-label">Current username</label>
                <input type="text" class="form-control" value="<?= htmlspecialchars($currentUsername) ?>" disabled autocomplete="username">
            </div>
            <div class="col-12">
                <label class="form-label" for="new_username">New username</label>
                <input type="text" name="new_username" id="new_username" class="form-control" required minlength="3" maxlength="50"
                pattern="[A-Za-z0-9._-]{3,50}" autocomplete="username">
                <div class="form-text">3-50 characters: letters, numbers, period, underscore, or hyphen.</div>
            </div>
            <div class="col-12">
                <label class="form-label" for="username_confirm_password">Current password</label>
                <input type="password" name="confirm_password" id="username_confirm_password" class="form-control" required
                autocomplete="current-password">
            </div>
        </form>
    </div>
</div>
```

```

        </div>
        <div class="col-12">
            <button type="submit" class="btn btn-primary">= app_tooltip_attr('Updates your login username after verifying your current
password.') ?&gt;&gt;Update Username&lt;/button&gt;
        &lt;/div&gt;
    &lt;/form&gt;
&lt;/div&gt;
&lt;/div&gt;
&lt;?php endif; ?&gt;

&lt;div class="card shadow-sm mb-4" style="max-width: 640px;"&gt;
    &lt;div class="card-header bg-white"&gt;&lt;strong&gt;Profile Photo&lt;/strong&gt;&lt;/div&gt;
    &lt;div class="card-body"&gt;
        &lt;?php if (!$profilePhotoReady): ?&gt;
            &lt;div class="alert alert-warning mb-0 py-2 small"&gt;
                Profile photos are not installed yet. Run &lt;a href="upgrade_roles.php" class="alert-link"&gt;upgrade_roles.php&lt;/a&gt; once, then
reload this page.
            &lt;/div&gt;
            &lt;?php else: ?&gt;
                &lt;div class="d-flex flex-wrap align-items-center gap-3 mb-3"&gt;
                    &lt;?php if ($profilePhotoUrl): ?&gt;
                        &lt;img src="&lt;?= htmlspecialchars($profilePhotoUrl, ENT_QUOTES, 'UTF-8') ?&gt;" alt="" class="rounded-circle border settings-
profile-preview" width="80" height="80"&gt;
                    &lt;?php else: ?&gt;
                        &lt;span class="admin-topbar-avatar rounded-circle d-inline-flex align-items-center justify-content-center settings-
profile-preview" style="width: 5rem; height: 5rem; font-size: 1.5rem;"&gt;
                            &lt;?php
                                $previewRole = (string) ($_SESSION['role'] ?? '');
                                $previewIcon = app_sidebar_topbar_icon_class($previewRole);
                                ?&gt;
                                &lt;i class="fa-solid &lt;?= htmlspecialchars($previewIcon, ENT_QUOTES, 'UTF-8') ?&gt; admin-topbar-avatar-icon" aria-
hidden="true"&gt;&lt;/i&gt;
                            &lt;/span&gt;
                        &lt;?php endif; ?&gt;
                    &lt;div class="small text-muted"&gt;
                        This photo appears in the top bar instead of the role icon. JPEG, PNG, or WebP, max 2&amp;nbsp;MB.
                    &lt;/div&gt;
                &lt;/div&gt;
                &lt;form method="post" enctype="multipart/form-data" class="row g-3 mb-2"&gt;
                    &lt;input type="hidden" name="settings_action" value="upload_profile_photo"&gt;
                    &lt;div class="col-12"&gt;
                        &lt;label class="form-label" for="profile_photo"&gt;Choose photo&lt;/label&gt;
                        &lt;input type="file" name="profile_photo" id="profile_photo" class="form-control" accept="image/jpeg,image/png,image/webp"
required&gt;
                    &lt;/div&gt;
                    &lt;div class="col-12"&gt;
                        &lt;button type="submit" class="btn btn-primary"&gt;<?= app_tooltip_attr('Uploads your profile photo and shows it in the top
bar menu.') ?&gt;&gt;Upload Photo&lt;/button&gt;
                    &lt;/div&gt;
                &lt;/form&gt;
                &lt;?php if ($profilePhotoUrl): ?&gt;
                    &lt;form method="post" class="mt-2" onsubmit="return confirm('Remove your profile photo and use the default icon again?');"&gt;
                        &lt;input type="hidden" name="settings_action" value="remove_profile_photo"&gt;
                        &lt;button type="submit" class="btn btn-outline-danger btn-sm"&gt;<?= app_tooltip_attr('Deletes your profile photo and restores
the role icon in the top bar.') ?&gt;&gt;Remove Photo&lt;/button&gt;
                    &lt;/form&gt;
                &lt;?php endif; ?&gt;
            &lt;?php endif; ?&gt;
        &lt;/div&gt;
    &lt;/div&gt;
&lt;?php endif; ?&gt;

&lt;div class="card shadow-sm" style="max-width: 640px;"&gt;
    &lt;div class="card-header bg-white"&gt;&lt;strong&gt;Change Password&lt;/strong&gt;&lt;/div&gt;
    &lt;div class="card-body"&gt;
        &lt;form method="post" class="row g-3"&gt;
            &lt;div class="col-12"&gt;
                &lt;label class="form-label"&gt;Current password&lt;/label&gt;
                &lt;input type="password" name="current_password" class="form-control" required autocomplete="current-password"&gt;
            &lt;/div&gt;
            &lt;div class="col-md-6"&gt;
                &lt;label class="form-label"&gt;New password&lt;/label&gt;
                &lt;input type="password" name="new_password" class="form-control" required minlength="8" autocomplete="new-password"&gt;
            &lt;/div&gt;
            &lt;div class="col-md-6"&gt;
                &lt;label class="form-label"&gt;Confirm new password&lt;/label&gt;
                &lt;input type="password" name="confirm_password" class="form-control" required minlength="8" autocomplete="new-password"&gt;
            &lt;/div&gt;
            &lt;div class="col-12"&gt;
                &lt;button type="submit" class="btn btn-primary"&gt;<?= app_tooltip_attr('Saves your new password after verifying your current
</pre

```

```

password. Use this to strengthen security or meet institutional rules.') ?>>Update Password</button>
    </div>
</form>
</div>
</div>

<?php if (is_admin() || is_super_admin()): ?>
<div class="card shadow-sm mt-4">
    <div class="card-header bg-white d-flex flex-wrap justify-content-between align-items-center gap-2">
        <strong><i class="fa-solid fa-clipboard-list me-2 text-primary" aria-hidden="true"></i>Log Activity Monitoring</strong>
        <?php if ($activityLogTableReady): ?>
            <form method="get" class="d-flex align-items-center gap-2">
                <label class="small text-muted mb-0" for="log_sort">Sort</label>
                <select name="log_sort" id="log_sort" class="form-select form-select-sm" style="width: auto;" onchange="this.form.submit()">
                    <option value="newest" <?=$activityLogOrder === 'newest' ? 'selected' : '' ?>>Newest first</option>
                    <option value="oldest" <?=$activityLogOrder === 'oldest' ? 'selected' : '' ?>>Oldest first</option>
                </select>
                <noscript><button type="submit" class="btn btn-sm btn-outline-secondary">Apply</button></noscript>
            </form>
        <?php endif; ?>
    </div>
    <div class="card-body p-0">
        <p class="text-muted small px-3 pt-3 mb-2">Read-only audit trail for <strong>all signed-in roles</strong> (admin, dean, program chair, GEN ED, faculty, student, etc.). Data changes and sign-in / sign-out are recorded where instrumented. Entries older than <?=(int) ADMIN_ACTIVITY_LOG_RETENTION_DAYS ?> days are removed automatically.</p>
        <?php if (!$activityLogTableReady): ?>
            <div class="alert alert-warning mx-3 mb-3 py-2 small" role="alert">
                The activity log table is not installed yet. Run <a href="upgrade_roles.php" class="alert-link">upgrade_roles.php</a> once (or re-run install) so admin actions can be recorded and shown here.
            </div>
        <?php elseif (!$activityLogs): ?>
            <div class="px-3 pb-3 text-muted small">No activity has been logged yet. Entries appear when users perform recorded actions (for example saving courses, rooms, schedules, or signing in).</div>
        <?php else: ?>
            <div class="table-responsive">
                <table class="table table-sm table-striped mb-0 align-middle">
                    <thead class="table-light">
                        <tr>
                            <th scope="col">When</th>
                            <th scope="col">Account</th>
                            <th scope="col">Action</th>
                            <th scope="col">Module</th>
                            <th scope="col">Target</th>
                            <th scope="col">Details</th>
                        </tr>
                    </thead>
                    <tbody>
                        <?php foreach ($activityLogs as $log): ?>
                            <?php
                                $rawDetails = (string) ($log['details_json'] ?? '');
                                [$dBefore, $dAfter, $detailsInvalid] = admin_activity_log_decode_details_for_display($rawDetails);
                                $showBefore = admin_activity_log_detail_block_nonempty($dBefore);
                                $showAfter = admin_activity_log_detail_block_nonempty($dAfter);
                                $hasStructured = $showBefore || $showAfter;
                                $summaryLabel = $showBefore && $showAfter
                                    ? 'Before / after'
                                    : ($showBefore ? 'Before' : ($showAfter ? 'After' : ''));
                                $explainLines = admin_activity_log_explain_human_lines($log, $dBefore, $dAfter);
                            <?>
                            <tr>
                                <td class="text-nowrap small"><?=$htmlspecialchars((string) ($log['created_at'] ?? '')) ?></td>
                                <td class="small">
                                    <?php
                                        $logActorName = trim((string) ($log['actor_full_name'] ?? ''));
                                        $logActorTitle = trim((string) ($log['actor_log_title'] ?? ''));
                                    <?>
                                    <?php if ($logActorName !== ''): ?>
                                        <div class="fw-semibold"><?=$htmlspecialchars($logActorName) ?></div>
                                    <?php endif; ?>
                                </td>
                                <td>
                                    <?=$htmlspecialchars((string) ($log['admin_username'] ?? '')) ?>
                                    <span class="text-muted">(<?=(int) ($log['admin_user_id'] ?? 0) ?>)</span>
                                </td>
                                <?php if ($logActorTitle !== ''): ?>
                                    <div class="text-muted"><?=$htmlspecialchars($logActorTitle) ?></div>
                                <?php endif; ?>
                                <span class="badge bg-light text-dark border mt-1"><?=$htmlspecialchars((string) ($log['user_role'] ?? ''))
                            </span>
                        </td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>
    <td>

```

```

        <?php
        $at = (string) ($log['action_type'] ?? '');
        $badgeClass = match ($at) {
            'login' => 'bg-success',
            'logout' => 'bg-info text-dark',
            'delete' => 'bg-danger',
            'add' => 'bg-primary',
            default => 'bg-secondary',
        };
        <?>
        <span class="badge <?= $badgeClass ?>"><?= htmlspecialchars($at) ?></span>
    </td>
    <td class="small"><?= htmlspecialchars((string) ($log['target_module'] ?? '')) ?></td>
    <td class="small"><?= htmlspecialchars((string) ($log['target_reference'] ?? '')) ?></td>
    <td class="small align-top" style="min-width: 260px; max-width: 520px;">
        <div class="activity-log-explain border-start border-primary border-2 ps-2 mb-2">
            <?php foreach ($explainLines as $i => $sentence): ?>
                <?php if ($i === 0): ?>
                    <p class="small fw-semibold mb-1"><?= htmlspecialchars($sentence) ?></p>
                <?php else: ?>
                    <div class="small text-body-secondary mb-1"><?= htmlspecialchars($sentence) ?></div>
                <?php endif; ?>
            <?php endforeach; ?>
        </div>
        <?php if ($hasStructured || ($detailsInvalid && $rawDetails !== '')): ?>
            <details class="activity-log-details mt-1">
                <summary class="d-flex flex-wrap align-items-center gap-1 text-muted user-select-none small"
                    style="cursor: pointer;">
                    <span>Technical payload (JSON)</span>
                    <?php if ($summaryLabel !== ''): ?>
                        <span class="text-body-secondary"></span>
                        <span><?= htmlspecialchars($summaryLabel) ?></span>
                    <?php endif; ?>
                    <?php if ($showBefore): ?><span class="badge rounded-pill bg-secondary-subtle text-secondary-emphasis border">Before</span><?php endif; ?>
                    <?php if ($showAfter): ?><span class="badge rounded-pill bg-primary-subtle text-primary-emphasis border">After</span><?php endif; ?>
                    <?php if ($detailsInvalid): ?><span class="badge rounded-pill bg-warning-subtle text-warning-emphasis border">Raw</span><?php endif; ?>
                </summary>
                <div class="border rounded mt-2 mb-0 bg-body-secondary overflow-hidden" style="max-height: 240px;
                    overflow-y: auto;">
                    <?php if ($showBefore): ?>
                        <div class="border-bottom border-secondary-subtle p-2">
                            <div class="fw-semibold text-uppercase small text-muted mb-1">Before</div>
                            <pre class="small font-monospace mb-0 text-wrap" style="white-space: pre-wrap;"><?=
                                htmlspecialchars(admin_activity_log_format_json_pretty($dBefore)) ?></pre>
                        </div>
                    <?php endif; ?>
                    <?php if ($showAfter): ?>
                        <div class="p-2">
                            <div class="fw-semibold text-uppercase small text-muted mb-1">After</div>
                            <pre class="small font-monospace mb-0 text-wrap" style="white-space: pre-wrap;"><?=
                                htmlspecialchars(admin_activity_log_format_json_pretty($dAfter)) ?></pre>
                        </div>
                    <?php endif; ?>
                    <?php if ($detailsInvalid && $rawDetails !== ''): ?>
                        <div class="p-2 <?= ($showBefore || $showAfter) ? 'border-top border-secondary-subtle' : ''
                            ?>">
                            <div class="fw-semibold text-uppercase small text-muted mb-1">Raw JSON</div>
                            <pre class="small font-monospace mb-0 text-wrap" style="white-space: pre-wrap;"><?=
                                htmlspecialchars($rawDetails) ?></pre>
                        </div>
                    <?php endif; ?>
                </div>
            </details>
            <?php endif; ?>
        </td>
    </tr>
    <?php endforeach; ?>
</tbody>
</table>
</div>
<?php endif; ?>
</div>
<?php endif; ?>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: student_calculator.php

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['student']);

$pageTitle = 'Scientific Calculator';
$bodyPageClass = 'page-student-calculator';
$mainContainerClass = 'container-fluid px-3 px-lg-4 py-3 py-md-4 flex-grow-1 d-flex flex-column min-w-0 app-main';
require_once __DIR__ . '/includes/header.php';
?>
<div class="mb-3">
    <h1 class="h4 mb-1"><i class="fa-solid fa-calculator me-2 text-primary"></i>Scientific Calculator</h1>
    <p class="text-muted small mb-0">Full-featured scientific calculator with trigonometry, logarithms, powers, and more.</p>
</div>

<div class="sci-calc-wrapper d-flex justify-content-center">
<div class="sci-calc" id="sciCalc">
    <div class="sci-calc-display">
        <div class="sci-calc-expression" id="calcExpression">&nbsp;</div>
        <div class="sci-calc-result" id="calcResult">0</div>
    </div>
    <div class="sci-calc-mode-bar">
        <button type="button" class="sci-calc-mode-btn active" data-angle="deg">DEG</button>
        <button type="button" class="sci-calc-mode-btn" data-angle="rad">RAD</button>
        <span class="sci-calc-memory-indicator" id="calcMemInd"></span>
    </div>
    <div class="sci-calc-buttons">
        <!-- Row 1: Scientific functions -->
        <button type="button" class="sci-btn func" data-action="sin">sin</button>
        <button type="button" class="sci-btn func" data-action="cos">cos</button>
        <button type="button" class="sci-btn func" data-action="tan">tan</button>
        <button type="button" class="sci-btn func" data-action="log">log</button>
        <button type="button" class="sci-btn func" data-action="ln">ln</button>

        <!-- Row 2: More functions -->
        <button type="button" class="sci-btn func" data-action="sqrt">&#8730;</button>
        <button type="button" class="sci-btn func" data-action="cbrt">&#8731;</button>
        <button type="button" class="sci-btn func" data-action="pow">x<sup>y</sup></button>
        <button type="button" class="sci-btn func" data-action="square">x<sup>2</sup></button>
        <button type="button" class="sci-btn func" data-action="inv">1/x</button>

        <!-- Row 3: Constants and parens -->
        <button type="button" class="sci-btn func" data-action="pi">&pi;</button>
        <button type="button" class="sci-btn func" data-action="e">e</button>
        <button type="button" class="sci-btn func" data-action="fact">n!</button>
        <button type="button" class="sci-btn paren" data-action="(">(</button>
        <button type="button" class="sci-btn paren" data-action=")">)</button>

        <!-- Row 4: 7 8 9 DEL AC -->
        <button type="button" class="sci-btn num" data-action="7">7</button>
        <button type="button" class="sci-btn num" data-action="8">8</button>
        <button type="button" class="sci-btn num" data-action="9">9</button>
        <button type="button" class="sci-btn del" data-action="del">DEL</button>
        <button type="button" class="sci-btn clear" data-action="ac">AC</button>

        <!-- Row 5: 4 5 6 * / -->
        <button type="button" class="sci-btn num" data-action="4">4</button>
        <button type="button" class="sci-btn num" data-action="5">5</button>
        <button type="button" class="sci-btn num" data-action="6">6</button>
        <button type="button" class="sci-btn op" data-action="*">&times;</button>
        <button type="button" class="sci-btn op" data-action="/">&divide;</button>

        <!-- Row 6: 1 2 3 + - -->
        <button type="button" class="sci-btn num" data-action="1">1</button>
        <button type="button" class="sci-btn num" data-action="2">2</button>
        <button type="button" class="sci-btn num" data-action="3">3</button>
        <button type="button" class="sci-btn op" data-action="+">+</button>
        <button type="button" class="sci-btn op" data-action="-">&minus;</button>

        <!-- Row 7: 0 . +/- % -->
        <button type="button" class="sci-btn num" data-action="0">0</button>
        <button type="button" class="sci-btn num" data-action=".">.</button>
        <button type="button" class="sci-btn func" data-action="negate">+&minus;</button>
```

```

        <button type="button" class="sci-btn func" data-action="percent">%</button>
        <button type="button" class="sci-btn equals" data-action="=">=</button>
    </div>
    <div class="sci-calc-history-toggle">
        <button type="button" class="btn btn-sm btn-outline-secondary rounded-pill" id="calcHistToggle">
            <i class="fa-solid fa-clock-rotate-left me-1"></i>History
        </button>
    </div>
    <div class="sci-calc-history" id="calcHistory" style="display:none;">
        <div class="sci-calc-history-list" id="calcHistoryList"></div>
        <button type="button" class="btn btn-sm btn-outline-danger rounded-pill mt-2" id="calcHistClear">Clear history</button>
    </div>
</div>
</div>

<style>
.sci-calc-wrapper {
    padding-bottom: 2rem;
}
.sci-calc {
    width: 100%;
    max-width: 420px;
    background: #1e293b;
    border-radius: 20px;
    padding: 20px;
    box-shadow: 0 20px 60px rgba(0,0,0,0.3), 0 0 0 1px rgba(255,255,255,0.05);
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', system-ui, sans-serif;
}
.sci-calc-display {
    background: #0f172a;
    border-radius: 14px;
    padding: 16px 20px;
    margin-bottom: 14px;
    min-height: 90px;
    display: flex;
    flex-direction: column;
    justify-content: flex-end;
    align-items: flex-end;
    overflow: hidden;
    border: 1px solid rgba(255,255,255,0.06);
}
.sci-calc-expression {
    font-size: 0.85rem;
    color: #94a3b8;
    word-break: break-all;
    text-align: right;
    width: 100%;
    min-height: 1.2em;
    line-height: 1.3;
}
.sci-calc-result {
    font-size: 2.2rem;
    font-weight: 700;
    color: #f1f5f9;
    word-break: break-all;
    text-align: right;
    width: 100%;
    line-height: 1.2;
    margin-top: 4px;
}
.sci-calc-mode-bar {
    display: flex;
    align-items: center;
    gap: 6px;
    margin-bottom: 12px;
    padding: 0 2px;
}
.sci-calc-mode-btn {
    background: #334155;
    border: none;
    color: #94a3b8;
    font-size: 0.7rem;
    font-weight: 700;
    padding: 4px 10px;
    border-radius: 6px;
    cursor: pointer;
    letter-spacing: 0.05em;
    transition: all 0.15s;
}
.sci-calc-mode-btn.active {

```

```

        background: #3b82f6;
        color: #fff;
    }
    .sci-calc-memory-indicator {
        margin-left: auto;
        font-size: 0.7rem;
        color: #fbbf24;
        font-weight: 600;
    }
    .sci-calc-buttons {
        display: grid;
        grid-template-columns: repeat(5, 1fr);
        gap: 8px;
    }
    .sci-btn {
        border: none;
        border-radius: 12px;
        padding: 14px 4px;
        font-size: 0.92rem;
        font-weight: 600;
        cursor: pointer;
        transition: all 0.12s ease;
        display: flex;
        align-items: center;
        justify-content: center;
        user-select: none;
        -webkit-tap-highlight-color: transparent;
        line-height: 1;
    }
    .sci-btn.num {
        background: #334155;
        color: #f1f5f9;
    }
    .sci-btn.num:hover { background: #475569; }
    .sci-btn.num:active { background: #1e293b; transform: scale(0.95); }

    .sci-btn.op {
        background: #1d4ed8;
        color: #fff;
    }
    .sci-btn.op:hover { background: #2563eb; }
    .sci-btn.op:active { background: #1e40af; transform: scale(0.95); }

    .sci-btn.func {
        background: #1e3a5f;
        color: #93c5fd;
    }
    .sci-btn.func:hover { background: #1e4a7a; }
    .sci-btn.func:active { background: #172e4a; transform: scale(0.95); }

    .sci-btn.paren {
        background: #1e3a5f;
        color: #93c5fd;
    }
    .sci-btn.paren:hover { background: #1e4a7a; }

    .sci-btn.del {
        background: #7c2d12;
        color: #fecaca;
    }
    .sci-btn.del:hover { background: #9a3412; }

    .sci-btn.clear {
        background: #991b1b;
        color: #fff;
    }
    .sci-btn.clear:hover { background: #b91c1c; }

    .sci-btn.equals {
        background: #16a34a;
        color: #fff;
        font-size: 1.2rem;
    }
    .sci-btn.equals:hover { background: #15803d; }
    .sci-btn.equals:active { background: #166534; transform: scale(0.95); }

    .sci-calc-history-toggle {
        text-align: center;
        margin-top: 14px;
    }

```

```

.sci-calc-history-toggle .btn {
  color: #94a3b8;
  border-color: #475569;
  font-size: 0.78rem;
}
.sci-calc-history-toggle .btn:hover {
  color: #f1f5f9;
  border-color: #64748b;
  background: #334155;
}
.sci-calc-history {
  margin-top: 12px;
  background: #0f172a;
  border-radius: 12px;
  padding: 12px 14px;
  max-height: 200px;
  overflow-y: auto;
  border: 1px solid rgba(255,255,255,0.06);
}
.sci-calc-history-list {
  font-size: 0.8rem;
  color: #94a3b8;
}
.sci-calc-history-list .hist-item {
  padding: 5px 0;
  border-bottom: 1px solid #1e293b;
  cursor: pointer;
  transition: color 0.15s;
}
.sci-calc-history-list .hist-item:hover { color: #e2e8f0; }
.sci-calc-history-list .hist-item:last-child { border-bottom: none; }
.sci-calc-history-list .hist-expr { color: #64748b; }
.sci-calc-history-list .hist-ans { color: #93c5fd; font-weight: 600; margin-left: 6px; }

```

```

@media (max-width: 480px) {
  .sci-calc { padding: 14px; border-radius: 16px; }
  .sci-calc-buttons { gap: 5px; }
  .sci-btn { padding: 12px 2px; font-size: 0.82rem; border-radius: 10px; }
  .sci-calc-result { font-size: 1.8rem; }
}

```

</style>

```

<script>
(function() {
  var angleMode = 'deg';
  var expression = '';
  var resultEl = document.getElementById('calcResult');
  var exprEl = document.getElementById('calcExpression');
  var histList = document.getElementById('calcHistoryList');
  var histPanel = document.getElementById('calcHistory');
  var histToggle = document.getElementById('calcHistToggle');
  var histClear = document.getElementById('calcHistClear');
  var history = JSON.parse(localStorage.getItem('sci_calc_history') || '[]');
  var lastAnswer = null;

  function toRad(deg) { return deg * Math.PI / 180; }
  function toDeg(rad) { return rad * 180 / Math.PI; }

  function factorial(n) {
    if (n < 0) return NaN;
    if (n === 0 || n === 1) return 1;
    if (n > 170) return Infinity;
    var r = 1;
    for (var i = 2; i <= n; i++) r *= i;
    return r;
  }

  function formatNum(n) {
    if (typeof n !== 'number' || isNaN(n)) return 'Error';
    if (!isFinite(n)) return n > 0 ? 'Infinity' : '-Infinity';
    var s = n.toPrecision(12);
    return parseFloat(s).toString();
  }

  function evalExpression(expr) {
    expr = expr.replace(/π/g, '(' + Math.PI + ')');
    expr = expr.replace(/e(?:![xp])/g, '(' + Math.E + ')');

    expr = expr.replace(/(\d+(?:\.\d+)?)!/g, function(_, num) {
      return 'factorial(' + num + ')';
    });
  }

```

```

});

expr = expr.replace(/sin\(((\[^\)]+)\))/g, function(_, inner) {
    if (angleMode === 'deg') return 'Math.sin(toRad(' + inner + '))';
    return 'Math.sin(' + inner + ')';
});
expr = expr.replace(/cos\(((\[^\)]+)\))/g, function(_, inner) {
    if (angleMode === 'deg') return 'Math.cos(toRad(' + inner + '))';
    return 'Math.cos(' + inner + ')';
});
expr = expr.replace(/tan\(((\[^\)]+)\))/g, function(_, inner) {
    if (angleMode === 'deg') return 'Math.tan(toRad(' + inner + '))';
    return 'Math.tan(' + inner + ')';
});
expr = expr.replace(/log\(((\[^\)]+)\))/g, 'Math.log10($1)');
expr = expr.replace(/ln\(((\[^\)]+)\))/g, 'Math.log($1)');
expr = expr.replace(/sqrt\(((\[^\)]+)\))/g, 'Math.sqrt($1)');
expr = expr.replace(/cbrt\(((\[^\)]+)\))/g, 'Math.cbrt($1)');

var fn = new Function('factorial', 'toRad', 'toDeg', 'return (' + expr + ');');
return fn(factorial, toRad, toDeg);
}

function updateDisplay() {
    exprEl.textContent = expression || '\u00a0';
    if (lastAnswer !== null) {
        resultEl.textContent = formatNum(lastAnswer);
    }
}

function addHistory(expr, ans) {
    history.unshift({ expr: expr, ans: formatNum(ans) });
    if (history.length > 50) history.pop();
    localStorage.setItem('sci_calc_history', JSON.stringify(history));
    renderHistory();
}

function renderHistory() {
    if (!histList) return;
    histList.innerHTML = '';
    history.forEach(function(item) {
        var div = document.createElement('div');
        div.className = 'hist-item';
        div.innerHTML = '<span class="hist-expr">' + escapeHtml(item.expr) + '</span><span class="hist-ans">' + escapeHtml(item.ans) +
        '</span>';
        div.addEventListener('click', function() {
            expression = item.ans;
            lastAnswer = parseFloat(item.ans);
            updateDisplay();
        });
        histList.appendChild(div);
    });
}

function escapeHtml(s) {
    var d = document.createElement('div');
    d.textContent = s;
    return d.innerHTML;
}

function handleAction(action) {
    switch (action) {
        case 'ac':
            expression = '';
            lastAnswer = null;
            resultEl.textContent = '0';
            exprEl.textContent = '\u00a0';
            return;
        case 'del':
            expression = expression.slice(0, -1);
            updateDisplay();
            return;
        case '=':
            try {
                var result = evalExpression(expression);
                if (typeof result === 'number') {
                    addHistory(expression, result);
                    lastAnswer = result;
                    exprEl.textContent = expression;
                    resultEl.textContent = formatNum(result);
                }
            }
    }
}

```

```

        expression = formatNum(result);
    }
} catch (e) {
    resultEl.textContent = 'Error';
    lastAnswer = null;
}
return;
case 'sin':
case 'cos':
case 'tan':
case 'log':
case 'ln':
case 'sqrt':
case 'cbrt':
    expression += action + '(';
    break;
case 'pow':
    expression += '**';
    break;
case 'square':
    expression += '**2';
    break;
case 'inv':
    expression = '1/(' + expression + ')';
    break;
case 'pi':
    expression += 'π';
    break;
case 'e':
    expression += 'e';
    break;
case 'fact':
    expression += '!';
    break;
case 'negate':
    if (expression && expression.charAt(0) === '-') {
        expression = expression.substring(1);
    } else if (expression) {
        expression = '-' + expression;
    }
    break;
case 'percent':
    try {
        var pv = evalExpression(expression);
        if (typeof pv === 'number') {
            expression = formatNum(pv / 100);
            lastAnswer = pv / 100;
        }
    } catch (e) { /* ignore */ }
    break;
default:
    expression += action;
    break;
}
updateDisplay();
}

document.querySelectorAll('#sciCalc .sci-btn').forEach(function(btn) {
    btn.addEventListener('click', function() {
        var action = btn.getAttribute('data-action');
        if (action) handleAction(action);
    });
});

document.querySelectorAll('#sciCalc .sci-calc-mode-btn').forEach(function(btn) {
    btn.addEventListener('click', function() {
        angleMode = btn.getAttribute('data-angle') || 'deg';
        document.querySelectorAll('#sciCalc .sci-calc-mode-btn').forEach(function(b) {
            b.classList.toggle('active', b === btn);
        });
    });
});

if (histToggle) {
    histToggle.addEventListener('click', function() {
        var showing = histPanel.style.display !== 'none';
        histPanel.style.display = showing ? 'none' : 'block';
    });
}

```

```

    if (histClear) {
        histClear.addEventListener('click', function() {
            history = [];
            localStorage.setItem('sci_calc_history', JSON.stringify(history));
            renderHistory();
        });
    }

    document.addEventListener('keydown', function(e) {
        if (e.target.tagName === 'INPUT' || e.target.tagName === 'TEXTAREA') return;
        var key = e.key;
        if ('0123456789.+*/()'.indexOf(key) !== -1) {
            e.preventDefault();
            handleAction(key);
        } else if (key === 'Enter' || key === '=') {
            e.preventDefault();
            handleAction('=');
        } else if (key === 'Backspace') {
            e.preventDefault();
            handleAction('del');
        } else if (key === 'Escape') {
            e.preventDefault();
            handleAction('ac');
        } else if (key === '^') {
            e.preventDefault();
            handleAction('pow');
        }
    });

    renderHistory();
})();
</script>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: student_classroom.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/classroom_discussion_helpers.php';
require_once __DIR__ . '/includes/assessment_retake_helpers.php';

require_role(['student']);

assessment_retake_ensure_table();

$studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
$hasLiveAt = db_column_exists('schedules', 'online_live_at');
$hasContentAttachments = db_table_exists('classroom_content_attachments');
$hasClassroomDiscussions = classroom_discussions_table_exists();
$hasContentWeeks = db_column_exists('classroom_content', 'weeks');
$hasContentDaysPerTopic = db_column_exists('classroom_content', 'days_per_topic');
$hasContentTopicSchedule = $hasContentWeeks && $hasContentDaysPerTopic;
$hasSyllabusCols = db_column_exists('online_classrooms', 'syllabus_stored_name');
if (db_table_exists('classroom_assessments') && !db_column_exists('classroom_assessments', 'time_limit_minutes')) {
    try {
        db()->exec('ALTER TABLE classroom_assessments ADD COLUMN time_limit_minutes INT UNSIGNED NULL DEFAULT NULL');
    } catch (Throwable $e) {
        // Column may already exist under a concurrent request.
    }
}
if (db_table_exists('classroom_submissions') && !db_column_exists('classroom_submissions', 'integrity_locked')) {
    try {
        db()->exec('ALTER TABLE classroom_submissions ADD COLUMN integrity_locked TINYINT(1) NOT NULL DEFAULT 0');
    } catch (Throwable $e) {
        // Column may already exist under a concurrent request.
    }
}
$hasCreditedWeek = db_column_exists('classroom_assessments', 'credited_week');
$hasTimeLimit = db_column_exists('classroom_assessments', 'time_limit_minutes');
$hasIntegrityLocked = db_column_exists('classroom_submissions', 'integrity_locked');
$hasAttendanceSessionsTable = db_table_exists('classroom_attendance_sessions');
$hasAttendanceRecordsTable = db_table_exists('classroom_attendance_records');
$hasAttendanceTables = $hasAttendanceSessionsTable && $hasAttendanceRecordsTable;
$hasAttendanceLogoutColumn = $hasAttendanceTables && db_column_exists('classroom_attendance_records', 'evidence_logout_at');

```

```

$formatDateTime12h = static function (?string $dateTime): string {
    $raw = trim((string) $dateTime);
    if ($raw === '') {
        return '-';
    }
    $ts = strtotime($raw);
    if ($ts === false) {
        return $raw;
    }
    return date('M j, Y g:i A', $ts);
};

$classIsLive = static function (?string $liveAt): bool {
    if ($liveAt === null || $liveAt === '') {
        return false;
    }

    $liveTs = strtotime($liveAt);
    if ($liveTs === false) {
        return false;
    }

    return (time() - $liveTs) <= 2 * 3600;
};

if ($studentId < 1) {
    $studentId = resolve_student_id_for_user($userId) ?? 0;
    $_SESSION['student_id'] = $studentId > 0 ? $studentId : null;
}
if ($studentId < 1) {
    exit('Student profile not linked to this account. Ask your instructor to create or link your student profile.');
```



```

if ($_SERVER['REQUEST_METHOD'] === 'POST' && $missingTables === [] && $classroom) {
    $action = (string) ($_POST['action'] ?? '');

    try {
        if ($action === 'submit_assessment') {
            $assessmentId = (int) ($_POST['assessment_id'] ?? 0);
            $integrityViolation = (string) ($_POST['integrity_violation'] ?? '') === '1';
            if ($assessmentId < 1) {
                throw new RuntimeException('Invalid assessment.');
```

```

db()->beginTransaction();
if ($hasIntegrityLocked) {
    db()->prepare(
        'INSERT INTO classroom_submissions (assessment_id, student_id, answer_text, status, auto_total_score,
requires_manual_grade, integrity_locked, submitted_at, updated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, NOW(), NOW())
        ON DUPLICATE KEY UPDATE answer_text = VALUES(answer_text), status = VALUES(status),
        auto_total_score = VALUES(auto_total_score), requires_manual_grade = VALUES(requires_manual_grade),
        integrity_locked = GREATEST(integrity_locked, VALUES(integrity_locked)), updated_at = NOW()'
    )->execute([
        $assessmentId,
        $studentId,
        json_encode($answerSummary, JSON_UNESCAPED_UNICODE),
        $submissionStatus,
        $autoTotal,
        $requiresManual ? 1 : 0,
        $lockFlag,
    ]);
} else {
    db()->prepare(
        'INSERT INTO classroom_submissions (assessment_id, student_id, answer_text, status, auto_total_score,
requires_manual_grade, submitted_at, updated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, NOW(), NOW())
        ON DUPLICATE KEY UPDATE answer_text = VALUES(answer_text), status = VALUES(status),
        auto_total_score = VALUES(auto_total_score), requires_manual_grade = VALUES(requires_manual_grade), updated_at =
NOW()'
    )->execute([
        $assessmentId,
        $studentId,
        json_encode($answerSummary, JSON_UNESCAPED_UNICODE),
        $submissionStatus,
        $autoTotal,
        $requiresManual ? 1 : 0,
    ]);
}
$subId = (int) db()->lastInsertId();
if ($subId < 1) {
    $subSt = db()->prepare('SELECT id FROM classroom_submissions WHERE assessment_id = ? AND student_id = ? LIMIT 1');
    $subSt->execute([$assessmentId, $studentId]);
    $subId = (int) ($subSt->fetchColumn() ?: 0);
}
if ($subId < 1) {
    throw new RuntimeException('Failed to save submission.');
```

```

        $studentId,
        $autoTotal,
        $scoreFeedback,
    ]);
    db()->commit();

    if ($integrityViolation) {
        $_SESSION['flash'] = 'Assessment ended and locked: you left or minimized the assessment window, or switched to another
tab/application. Your answers were submitted and Update answer is now disabled.';
    } else {
        $_SESSION['flash'] = 'Assessment submitted successfully. Update answer is now disabled.';
    }
} elseif ($action === 'request_retake') {
    $assessmentId = (int) ($_POST['assessment_id'] ?? 0);
    $reason = trim((string) ($_POST['reason'] ?? ''));
    if ($assessmentId < 1) {
        throw new RuntimeException('Invalid assessment.');
```

```

    }

    $st = db()->prepare(
        'SELECT id FROM classroom_assessments WHERE id = ? AND classroom_id = ? LIMIT 1'
    );
    $st->execute([$assessmentId, $classroomId]);
    if (!$st->fetchColumn()) {
        throw new RuntimeException('Assessment not found.');
```

```

    }
    create_assessment_retake_request($classroomId, $assessmentId, $studentId, $reason);
    $_SESSION['flash'] = 'Your request for another attempt has been sent to your instructor for approval.';
} elseif ($action === 'attendance_login' || $action === 'attendance_logout') {
    if (!$hasAttendanceTables) {
        throw new RuntimeException('Attendance module needs a database update. Run upgrade_roles.php once.');
```

```

    }

    $today = date('Y-m-d');
    $sessionStart = $today . ' ' . substr((string) ($classroom['start_time'] ?? '00:00:00'), 0, 8);
    $sessionEnd = $today . ' ' . substr((string) ($classroom['end_time'] ?? '23:59:59'), 0, 8);
    $now = date('Y-m-d H:i:s');
```

```

    db()->beginTransaction();
    try {
        db()->prepare(
            'INSERT INTO classroom_attendance_sessions (classroom_id, faculty_id, attendance_date, session_start_at, session_end_at,
source)
            VALUES (?, ?, ?, ?, ?, ?)
            ON DUPLICATE KEY UPDATE
                session_start_at = VALUES(session_start_at),
                session_end_at = VALUES(session_end_at),
                source = VALUES(source),
                updated_at = CURRENT_TIMESTAMP'
        )->execute([
            $classroomId,
            (int) $classroom['faculty_id'],
            $today,
            $sessionStart,
            $sessionEnd,
            'auto_login_online',
        ]);
```

```

        $sidStmt = db()->prepare(
            'SELECT id FROM classroom_attendance_sessions WHERE classroom_id = ? AND attendance_date = ? LIMIT 1'
        );
        $sidStmt->execute([$classroomId, $today]);
        $sessionId = (int) ($sidStmt->fetchColumn() ?? 0);
        if ($sessionId < 1) {
            throw new RuntimeException('Unable to initialize attendance session.');
```

```

        }

        if ($action === 'attendance_login') {
            $loginWindow = classroom_attendance_login_allowed($classroom);
            if (!$loginWindow['allowed']) {
                throw new RuntimeException(
                    $loginWindow['reason'] !== ''
                    ? $loginWindow['reason']
                    : 'You can only log in during scheduled class time.'
                );
            }
        }
    }

    if ($hasAttendanceLogoutColumn) {
        db()->prepare(
```

notes)

```
'INSERT INTO classroom_attendance_records
(session_id, student_id, status, source, evidence_login_at, evidence_seen_at, evidence_logout_at, checked_at,
VALUES (?, ?, ?, ?, ?, ?, ?, ?)
ON DUPLICATE KEY UPDATE
    status = VALUES(status),
    source = VALUES(source),
    evidence_login_at = VALUES(evidence_login_at),
    evidence_seen_at = VALUES(evidence_seen_at),
    checked_at = VALUES(checked_at),
    notes = VALUES(notes),
    updated_at = CURRENT_TIMESTAMP'
)->execute([
    $sessionId,
    $studentId,
    'present',
    'manual',
    $now,
    $now,
    null,
    $now,
    'Student tapped class login.',
]);
} else {
    db()->prepare(
        'INSERT INTO classroom_attendance_records
        (session_id, student_id, status, source, evidence_login_at, evidence_seen_at, checked_at, notes)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        ON DUPLICATE KEY UPDATE
            status = VALUES(status),
            source = VALUES(source),
            evidence_login_at = VALUES(evidence_login_at),
            evidence_seen_at = VALUES(evidence_seen_at),
            checked_at = VALUES(checked_at),
            notes = VALUES(notes),
            updated_at = CURRENT_TIMESTAMP'
    )->execute([
        $sessionId,
        $studentId,
        'present',
        'manual',
        $now,
        $now,
        $now,
        $now,
        'Student tapped class login.',
    ]);
}
$_SESSION['flash'] = 'Attendance login recorded for this class.';
} else {
    if ($hasAttendanceLogoutColumn) {
        db()->prepare(
            'INSERT INTO classroom_attendance_records
            (session_id, student_id, status, source, evidence_login_at, evidence_seen_at, evidence_logout_at, checked_at,
VALUES (?, ?, ?, ?, ?, ?, ?, ?)
ON DUPLICATE KEY UPDATE
    source = "manual",
    evidence_logout_at = VALUES(evidence_logout_at),
    checked_at = VALUES(checked_at),
    notes = VALUES(notes),
    updated_at = CURRENT_TIMESTAMP'
        )->execute([
            $sessionId,
            $studentId,
            'present',
            'manual',
            null,
            null,
            $now,
            $now,
            'Student tapped class logout.',
        ]);
    } else {
        db()->prepare(
            'INSERT INTO classroom_attendance_records
            (session_id, student_id, status, source, evidence_login_at, evidence_seen_at, checked_at, notes)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            ON DUPLICATE KEY UPDATE
                source = "manual",
                checked_at = VALUES(checked_at),

```

notes)

```

                notes = VALUES(notes),
                updated_at = CURRENT_TIMESTAMP'
            )->execute([
                $sessionId,
                $studentId,
                'present',
                'manual',
                null,
                null,
                $now,
                'Student tapped class logout.',
            ]);
        }
        $_SESSION['flash'] = 'Attendance logout recorded for this class.';
    }

    db()->commit();
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    throw $e;
}
} elseif ($action === 'post_discussion_message') {
    classroom_discussion_post($classroomId, $userId, (string) ($_POST['message_body'] ?? ''));
    $_SESSION['flash'] = 'Class discussion message posted.';
} elseif ($action === 'upload_banner') {
    if (!classroom_banner_column_ready()) {
        throw new RuntimeException('Header backgrounds need a database update. Run upgrade_roles.php once.');
```

```

    }
    if (!isset($_FILES['banner']) || !is_array($_FILES['banner'])) {
        throw new RuntimeException('Please choose a background image to upload.');
```

```

    }
    $f = $_FILES['banner'];
    classroom_banner_store_for_classroom($classroomId, [
        'name' => (string) ($f['name'] ?? ''),
        'type' => (string) ($f['type'] ?? ''),
        'tmp_name' => (string) ($f['tmp_name'] ?? ''),
        'error' => (int) ($f['error'] ?? UPLOAD_ERR_NO_FILE),
        'size' => (int) ($f['size'] ?? 0),
    ]);
    $_SESSION['flash'] = 'Classroom header background updated.';
} elseif ($action === 'delete_banner') {
    if (!classroom_banner_column_ready()) {
        throw new RuntimeException('Header backgrounds are not installed.');
```

```

    }
    classroom_banner_remove_for_classroom($classroomId);
    $_SESSION['flash'] = 'Classroom header background removed.';
}
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}

header('Location: student_classroom.php?id=' . $classroomId);
exit;
}

$announcements = [];
$materials = [];
$contentAttachmentMap = [];
$discussionMessages = [];
$assessments = [];
$materialWeeks = [];
$attendanceTodayLoginAt = null;
$attendanceTodayLogoutAt = null;
$assessmentQuestionMap = [];
$submissionQuestionMap = [];
$retakeRequestMap = [];

if ($missingTables === []) {
    $st = db()->prepare(
        'SELECT *
        FROM classroom_content
        WHERE classroom_id = ? AND content_type = "announcement"
        ORDER BY created_at DESC'
    );
    $st->execute([$classroomId]);
}

```

```

$announcements = $st->fetchAll();

$st = db()->prepare(
    'SELECT *
    FROM classroom_content
    WHERE classroom_id = ? AND content_type <> "announcement"
    ORDER BY created_at DESC'
);
$st->execute([$classroomId]);
$materials = $st->fetchAll();
$materialWeeks = classroom_content_group_by_week($materials);
if ($hasContentAttachments) {
    $contentAttachmentMap = classroom_content_attachment_map(array_merge(
        array_column($announcements, 'id'),
        array_column($materials, 'id')
    ));
}
if ($hasClassroomDiscussions) {
    $discussionMessages = classroom_discussion_messages($classroomId, $userId);
}

$assessmentSelect = 'SELECT ca.*, sc.score, sc.feedback, sc.graded_at, sub.answer_text, sub.status AS submission_status,
sub.submitted_at';
if ($hasIntegrityLocked) {
    $assessmentSelect .= ', sub.integrity_locked';
}
$st = db()->prepare(
    $assessmentSelect . '
    FROM classroom_assessments ca
    LEFT JOIN classroom_scores sc ON sc.assessment_id = ca.id AND sc.student_id = ?
    LEFT JOIN classroom_submissions sub ON sub.assessment_id = ca.id AND sub.student_id = ?
    WHERE ca.classroom_id = ?
    ORDER BY ca.created_at DESC'
);
$st->execute([$studentId, $studentId, $classroomId]);
$assessments = $st->fetchAll();

if ($assessments !== []) {
    $assessmentIds = array_map(static fn (array $r): int => (int) $r['id'], $assessments);
    $placeholders = implode(',', array_fill(0, count($assessmentIds), '?'));
    $qst = db()->prepare(
        "SELECT * FROM classroom_assessment_questions
        WHERE assessment_id IN ($placeholders)
        ORDER BY assessment_id ASC, position ASC, id ASC"
    );
    $qst->execute($assessmentIds);
    foreach ($qst->fetchAll() as $q) {
        $aid = (int) ($q['assessment_id'] ?? 0);
        if ($aid > 0) {
            $assessmentQuestionMap[$aid][] = $q;
        }
    }

    $sst = db()->prepare(
        "SELECT sub.assessment_id, qa.question_id, qa.answer_text, qa.answer_steps, qa.is_correct, qa.auto_score
        FROM classroom_submissions sub
        INNER JOIN classroom_submission_question_answers qa ON qa.submission_id = sub.id
        WHERE sub.student_id = ? AND sub.assessment_id IN ($placeholders)"
    );
    $sst->execute(array_merge([$studentId], $assessmentIds));
    foreach ($sst->fetchAll() as $row) {
        $submissionQuestionMap[(int) $row['assessment_id']][(int) $row['question_id']] = $row;
    }

    if (assessment_retake_table_ready()) {
        $retakeRequestMap = assessment_retake_requests_for_student(
            $studentId,
            array_map(static fn (array $r): int => (int) $r['id'], $assessments)
        );
    }
}

$assessmentsByWeek = [];
if ($hasCreditedWeek) {
    foreach ($assessments as $a) {
        $wk = trim((string) ($a['credited_week'] ?? ''));
        if ($wk !== '') {
            $assessmentsByWeek[$wk][] = $a;
        }
    }
}

```

```

}

if ($hasAttendanceTables) {
    $attendanceTodayDate = date('Y-m-d');
    if ($hasAttendanceLogoutColumn) {
        $st = db()->prepare(
            'SELECT ar.evidence_login_at, ar.evidence_logout_at
            FROM classroom_attendance_sessions s
            LEFT JOIN classroom_attendance_records ar ON ar.session_id = s.id AND ar.student_id = ?
            WHERE s.classroom_id = ? AND s.attendance_date = ?
            LIMIT 1'
        );
        $st->execute([$studentId, $classroomId, $attendanceTodayDate]);
        $attendanceRow = $st->fetch() ?: null;
        if ($attendanceRow) {
            $attendanceTodayLoginAt = $attendanceRow['evidence_login_at'] ?? null;
            $attendanceTodayLogoutAt = $attendanceRow['evidence_logout_at'] ?? null;
        }
    } else {
        $st = db()->prepare(
            'SELECT ar.evidence_login_at
            FROM classroom_attendance_sessions s
            LEFT JOIN classroom_attendance_records ar ON ar.session_id = s.id AND ar.student_id = ?
            WHERE s.classroom_id = ? AND s.attendance_date = ?
            LIMIT 1'
        );
        $st->execute([$studentId, $classroomId, $attendanceTodayDate]);
        $attendanceRow = $st->fetch() ?: null;
        if ($attendanceRow) {
            $attendanceTodayLoginAt = $attendanceRow['evidence_login_at'] ?? null;
        }
    }
}

}

$pageTitle = 'Classroom';
require_once __DIR__ . '/includes/header.php';
?>
<?php
$classroomLiveAt = $hasLiveAt && $classroom ? (string) ($classroom['online_live_at'] ?? '') : '';
$classroomIsLive = $hasLiveAt && $classroomIsLive($classroomLiveAt);
$studentSyllabusReady = $classroom && $hasSyllabusCols && trim((string) ($classroom['syllabus_stored_name'] ?? '')) !== '';
$classroomBannerUrl = $classroom ? classroom_banner_url_for($classroom) : null;
$hasBannerCols = classroom_banner_column_ready();
$attendanceHasLoginToday = trim((string) ($attendanceTodayLoginAt ?? '')) !== '';
$attendanceHasLogoutToday = $hasAttendanceLogoutColumn && trim((string) ($attendanceTodayLogoutAt ?? '')) !== '';
$attendanceLoginWindow = $classroom ? classroom_attendance_login_allowed($classroom) : [
    'allowed' => false,
    'reason' => '',
    'is_scheduled_day' => false,
    'is_within_window' => false,
    'session_start' => '',
    'session_end' => '',
];
$attendanceCanLogin = (!$attendanceHasLoginToday || $attendanceHasLogoutToday) && $attendanceLoginWindow['allowed'];
$attendanceLoginTooltip = $attendanceLoginWindow['allowed']
    ? "Records that you joined today's class session for attendance. Use this when class starts or when your instructor expects a login
    check."
    : ($attendanceLoginWindow['reason'] !== ''
        ? $attendanceLoginWindow['reason']
        : 'Class login is only available during your scheduled class time.');
```

```

$attendanceCanLogout = $hasAttendanceLogoutColumn
    ? ($attendanceHasLoginToday && !$attendanceHasLogoutToday)
    : true;
$attendanceStatusClass = 'text-secondary';
$attendanceStatusLabel = 'Not logged in yet';
if ($attendanceHasLogoutToday) {
    $attendanceStatusClass = 'text-danger';
    $attendanceStatusLabel = 'Logged out';
} elseif ($attendanceHasLoginToday) {
    $attendanceStatusClass = 'text-success';
    $attendanceStatusLabel = $hasAttendanceLogoutColumn ? 'In class' : 'Login recorded';
}
?>
<div class="classroom-banner mb-3">
    <?php if ($classroomBannerUrl ? ' has-photo' : '' ?>
        <?php if ($classroomBannerUrl ? ' style="--classroom-banner-image:
        url(\'' . htmlspecialchars($classroomBannerUrl, ENT_QUOTES) . '\')' : '' ?>
        <div class="classroom-banner__overlay">
            <h1 class="classroom-banner__title">
                <?php if ($classroomIsLive): ?>
                    <?php if ($classroomIsLive): ?>

```

```

        <span class="badge bg-danger live-pulse-badg align-middle ms-2">LIVE</span>
    <?php endif; ?>
</h1>
<?php if ($classroom): ?>
    <div class="classroom-banner__meta">
        <? = htmlspecialchars((string) $classroom['course_code']) ?> - <? = htmlspecialchars((string) $classroom['course_name']) ?>
        <span class="classroom-banner__sep">&middot;</span>
        Instructor: <? = htmlspecialchars((string) $classroom['faculty_name']) ?>
    </div>
    <?php if ($classroomIsLive): ?>
        <div class="classroom-banner__live-note">
            <i class="fa-solid fa-circle-play me-1"></i>Your instructor is live now.
        </div>
    <?php endif; ?>
<?php endif; ?>
</div>
<?php if ($classroom && $hasBannerCols): ?>
    <div class="classroom-banner__actions">
        <form method="post" enctype="multipart/form-data" class="classroom-banner__upload-form" id="studentBannerUploadForm">
            <input type="hidden" name="action" value="upload_banner">
            <input type="hidden" name="classroom_id" value="<? = (int) $classroomId ?>">
            <label class="btn btn-light btn-sm classroom-banner__upload-btn" for="studentBannerFile"><? = student_tooltip_attr('Choose a
JPEG, PNG, or WebP image to use as this class header background.') ?>>
                <i class="fa-solid fa-image me-1"></i><? = $classroomBannerUrl ? 'Change background' : 'Upload background' ?>
            </label>
            <input id="studentBannerFile" type="file" name="banner" class="visually-hidden"
accept="image/jpeg,image/png,image/webp,.jpg,.jpeg,.png,.webp" required>
        </form>
        <?php if ($classroomBannerUrl): ?>
            <form method="post" class="d-inline" onsubmit="return confirm('Remove the header background image?');">
                <input type="hidden" name="action" value="delete_banner">
                <input type="hidden" name="classroom_id" value="<? = (int) $classroomId ?>">
                <button type="submit" class="btn btn-outline-light btn-sm"><? = student_tooltip_attr('Removes the custom background and
restores the default header style.') ?>><i class="fa-solid fa-trash me-1"></i>Remove</button>
            </form>
        <?php endif; ?>
    </div>
<?php elseif ($classroom && !$hasBannerCols): ?>
    <div class="classroom-banner__actions">
        <span class="badge text-bg-warning">Run upgrade_roles.php to enable background upload</span>
    </div>
<?php endif; ?>
</div>

<div class="d-flex flex-wrap justify-content-between align-items-start gap-2 mb-4 student-page-header student-page-header--below-banner">
    <div class="min-w-0 d-none d-md-block">
        <div class="text-muted small">Class workspace</div>
    </div>
    <div class="d-flex flex-wrap align-items-start align-items-md-center gap-2 student-page-header__actions">
        <?php if ($studentSyllabusReady): ?>
            <a href="<? = htmlspecialchars(classroom_syllabus_href((int) $classroomId)) ?>" target="_blank" rel="noopener noreferrer"
class="btn btn-primary btn-sm px-3"><? = student_tooltip_attr('Opens the official course syllabus in a new browser tab so you can read it
alongside the class page.') ?>><i class="fa-solid fa-file-contract me-1"></i>Syllabus</a>
        <?php elseif ($hasSyllabusCols && $classroom): ?>
            <span class="badge text-bg-secondary align-self-center">Syllabus pending</span>
        <?php endif; ?>
        <button type="button" class="btn btn-outline-primary btn-sm px-3" data-offline-save="<? = (int) $classroomId ?>" data-offline-
merge="1"><? = student_tooltip_attr('Saves this classroom faculty announcements and materials on your device so you can read them without
internet.') ?>>
            <i class="fa-solid fa-cloud-arrow-down me-1"></i>Save for offline
        </button>
        <a href="student_offline.php" class="btn btn-outline-secondary btn-sm px-3"><? = student_tooltip_attr('Opens your saved offline copy
of faculty posts. Works even without internet after you save once.') ?>>
            <i class="fa-solid fa-book-open me-1"></i>Read offline
        </a>
        <a href="student_classrooms.php" class="btn btn-outline-secondary btn-sm px-3"><? = student_tooltip_attr('Returns to the list of all
your enrolled classes. Use this when you are done with this class or want to switch subjects.') ?>>Back to My Classes</a>
    <?php if ($hasAttendanceTables): ?>
        <div class="border rounded-3 px-2 py-2 bg-light-subtle d-flex flex-column align-items-start gap-1 student-attendance-panel">
            <div class="small fw-semibold <? = htmlspecialchars($attendanceStatusClass) ?>">
                <i class="fa-solid fa-user-check me-1"></i><? = htmlspecialchars($attendanceStatusLabel) ?>
            </div>
            <div class="d-flex flex-wrap gap-2">
                <form method="post" class="d-inline">
                    <input type="hidden" name="action" value="attendance_login">
                    <input type="hidden" name="classroom_id" value="<? = (int) $classroomId ?>">
                    <button type="submit" class="btn btn-primary btn-sm" <? = $attendanceCanLogin ? '' : 'disabled' ?> <? =
student_tooltip_attr($attendanceLoginTooltip) ?>><i class="fa-solid fa-right-to-bracket me-1"></i>Class Login</button>
                </form>
                <form method="post" class="d-inline">

```



```

        <input type="hidden" name="action" value="attendance_logout">
        <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
        <button type="submit" class="btn btn-outline-danger btn-sm" <?= $attendanceCanLogout ? '' : 'disabled' ?> <?=
student_tooltip_attr('Records that you left the class session. Use this when your instructor tracks end-of-class attendance or when you must
leave early.') ?><i class="fa-solid fa-right-from-bracket me-1"></i>Class Logout</button>
    </form>
</div>
<?php if (!$attendanceLoginWindow['allowed'] && $attendanceLoginWindow['reason'] != ''): ?>
    <div class="small text-muted"><?= htmlspecialchars($attendanceLoginWindow['reason']) ?></div>
<?php endif; ?>
<div class="small text-muted">
    Last class login: <?= htmlspecialchars($formatDateTime12h($attendanceTodayLoginAt != null ? (string)
$attendanceTodayLoginAt : null)) ?>
    <?php if ($hasAttendanceLogoutColumn): ?>
        <br>Last class logout: <?= htmlspecialchars($formatDateTime12h($attendanceTodayLogoutAt != null ? (string)
$attendanceTodayLogoutAt : null)) ?>
    <?php endif; ?>
</div>
</div>
<?php endif; ?>
<?php if ($classroom && trim((string) $classroom['meet_link']) != ''): ?>
    <?php if ($classroomIsLive): ?>
        <a href="<?= htmlspecialchars((string) $classroom['meet_link']) ?>" target="_blank" rel="noopener noreferrer" class="btn
btn-success btn-sm px-3"><?= student_tooltip_attr('Your instructor is live. Opens the video meeting for this class in a new tab.') ?><i
class="fa-solid fa-video me-1"></i>Join Meet</a>
    <?php else: ?>
        <button type="button" class="btn btn-outline-secondary btn-sm px-3" disabled<?= student_tooltip_attr('Join Meet is available
only when your instructor goes live for this class.') ?><i class="fa-solid fa-video me-1"></i>Join Meet</button>
    <?php endif; ?>
<?php endif; ?>
</div>
</div>
<p class="small text-muted mb-3" data-offline-status="auto"></p>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if ($missingTables != []): ?>
    <div class="alert alert-warning">
        Student classroom features are not installed yet. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this page.
    </div>
<?php else: ?>
    <div class="row g-4">
        <div class="col-lg-4 order-lg-1 order-2">
            <div class="card shadow-sm mb-4">
                <div class="card-header bg-white"><strong>Announcements</strong></div>
                <div class="card-body">
                    <?php if ($announcements == []): ?>
                        <p class="text-muted mb-0">No announcements yet.</p>
                    <?php else: ?>
                        <?php foreach ($announcements as $item): ?>
                            <div class="border-bottom pb-3 mb-3">
                                <div class="fw-semibold"><?= htmlspecialchars((string) $item['title']) ?></div>
                                <?php if (trim((string) ($item['body'] ?? '')) != ''): ?>
                                    <div class="small mt-1 classroom-content-body"><?= classroom_content_render_body((string) $item['body'])
?></div>
                                <?php endif; ?>
                                <?php if ($hasContentTopicSchedule && (trim((string) ($item['weeks'] ?? '')) != '' || trim((string)
($item['days_per_topic'] ?? '')) != '')): ?>
                                    <div class="small text-muted mt-1">
                                        <?php if (trim((string) ($item['weeks'] ?? '')) != ''): ?>
                                            <span class="me-2"><i class="fa-regular fa-calendar me-1"></i>Weeks: <?=
htmlspecialchars((string) $item['weeks']) ?></span>
                                        <?php endif; ?>
                                        <?php if (trim((string) ($item['days_per_topic'] ?? '')) != ''): ?>
                                            <span><i class="fa-regular fa-clock me-1"></i>Days/topic: <?= htmlspecialchars((string)
$item['days_per_topic']) ?></span>
                                        <?php endif; ?>
                                    </div>
                                <?php endif; ?>
                            </div>
                        <?php if (trim((string) ($item['resource_url'] ?? '')) != ''): ?>
                            <?php $resourceUrl = trim((string) $item['resource_url']); ?>
                            <?php if (classroom_content_is_attachment($resourceUrl)): ?>
                                <div class="small mt-2">
                                    <a href="<?= htmlspecialchars(classroom_content_resource_href((int) $item['id'], $resourceUrl))
?>"><?= student_tooltip_attr('Downloads or opens a file your instructor attached to this announcement. Use this to read handouts or slides
they shared.') ?>>
                                        <i class="fa-solid fa-paperclip me-1"></i><?=
htmlspecialchars(classroom_content_attachment_name($resourceUrl)) ?>
                                    </a>
                                </div>
                            </div>

```

```

                <?php else: ?>
                    <div class="small mt-2"><a href="<?=$htmlspecialchars(classroom_content_resource_href((int)
$item['id'], $resourceUrl)) ?>" target="_blank" rel="noopener noreferrer"><?=$student_tooltip_attr('Opens the linked resource in a new
browser tab. Use this for external sites or documents your instructor linked here.') ?>>Open resource</a></div>
                <?php endif; ?>
            <?php endif; ?>
            <?php foreach ($contentAttachmentMap[(int) $item['id']] ?? [] as $attachment): ?>
                <div class="small mt-2">
                    <a href="<?=$htmlspecialchars(classroom_content_attachment_href((int) $attachment['id'])) ?>"><?=$
student_tooltip_attr('Downloads or opens an extra file attached to this post. Use this when your instructor added more than one document.')
?>>
                        <i class="fa-solid fa-paperclip me-1"></i><?=$
htmlspecialchars(classroom_content_attachment_download_name((string) $attachment['original_name'], (string) $attachment['stored_name'])) ?>
                    </a>
                </div>
            <?php endforeach; ?>
            <div class="small text-muted mt-1"><?=$htmlspecialchars((string) $item['created_at']) ?></div>
        </div>
    <?php endforeach; ?>
<?php endif; ?>
</div>
</div>

<div class="card shadow-sm">
    <div class="card-header bg-white"><strong>Course Materials</strong></div>
    <div class="card-body">
        <?php if ($materials === []): ?>
            <p class="text-muted mb-0">No course materials yet.</p>
        <?php else: ?>
            <p class="small text-muted">Open a week to view only that week's faculty topics, materials, links, and attachments.

</p>
            <div class="list-group list-group-flush">
                <?php foreach ($materialWeeks as $group): ?>
                    <?php $weekAssessments = $assessmentsByWeek[(string) $group['label']] ?? []; ?>
                    <a class="list-group-item list-group-item-action d-flex justify-content-between align-items-center px-0"
href="student_classroom_week.php?id=<?=(int) $classroomId ?>&week=<?=$rawurlencode((string) $group['label']) ?>"><?=$
student_tooltip_attr('Opens materials and topics for this week only. Use this to focus on what your instructor posted for a specific week.')
?>>
                        <span>
                            <i class="fa-regular fa-calendar me-2 text-primary"></i>
                            <strong><?=$htmlspecialchars((string) $group['label']) ?></strong>
                        </span>
                        <span class="d-flex gap-1">
                            <span class="badge text-bg-primary rounded-pill"><?=(int) $group['count'] ?></span>
                            <?php if ($weekAssessments !== []): ?>
                                <span class="badge text-bg-warning rounded-pill"><?=$count($weekAssessments) ?> <?=$
count($weekAssessments) === 1 ? 'assessment' : 'assessments' ?></span>
                            <?php endif; ?>
                        </span>
                    </a>
                <?php endforeach; ?>
            </div>
        <?php endif; ?>
    </div>
</div>
</div>

<div class="col-lg-8 order-lg-2 order-1 student-classroom-main">
    <div class="card shadow-sm mb-4">
        <div class="card-header bg-white"><strong>Class Discussion</strong></div>
        <div class="card-body">
            <?php if (!$hasClassroomDiscussions): ?>
                <div class="alert alert-warning mb-0">Run <a href="upgrade_roles.php">upgrade_roles.php</a> once to enable classroom
discussion.</div>
            <?php else: ?>
                <div class="border rounded p-3 bg-light mb-3" style="max-height: 320px; overflow-y: auto;"
id="studentClassDiscussionThread">
                    <?php if ($discussionMessages === []): ?>
                        <p class="text-muted mb-0">No discussion messages yet. Start the conversation for this class.</p>
                    <?php else: ?>
                        <?php foreach ($discussionMessages as $message): ?>
                            <div class="d-flex mb-3 <?=$message['mine'] ? 'justify-content-end' : 'justify-content-start' ?>">
                                <div class="rounded-3 px-3 py-2 shadow-sm <?=$message['mine'] ? 'bg-primary text-white' : 'bg-white
border' ?>" style="max-width: 85%;">
                                    <div class="small <?=$message['mine'] ? 'text-white-50' : 'text-muted' ?> mb-1">
                                        <?=$message['mine'] ? 'You' : htmlspecialchars($message['full_name']) ?>
                                        <span class="badge bg-<?=$htmlspecialchars(classroom_discussion_role_badge_class((string)
$message['role'])) ?> ms-1"><?=$htmlspecialchars(strtoupper((string) $message['role'])) ?></span>
                                        A. <?=$htmlspecialchars(substr((string) $message['created_at'], 0, 16)) ?>
                                    </div>
                                </div>
                            </?php endforeach; ?>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

                <div><? n12br(htmlspecialchars((string) $message['body'])) ?></div>
            </div>
        </div>
        <?php endforeach; ?>
    <?php endif; ?>
</div>
<form method="post">
    <input type="hidden" name="action" value="post_discussion_message">
    <input type="hidden" name="classroom_id" value="<? (int) $classroomId ?>">
    <label class="visually-hidden" for="studentDiscussionBody">Discussion message</label>
    <textarea name="message_body" id="studentDiscussionBody" class="form-control mb-2" rows="3" maxlength="8000"
placeholder="Send a message to your class discussion..." required></textarea>
    <div class="text-end">
        <button type="submit" class="btn btn-primary"<? student_tooltip_attr('Sends your text to the class
discussion thread. Use this to ask questions or respond to classmates and your instructor.') ?><i class="fa-solid fa-paper-plane me-1">
</i>Post message</button>
    </div>
</form>
<?php endif; ?>
</div>
</div>

<div class="card shadow-sm">
    <div class="card-header bg-white"><strong>Assessments</strong></div>
    <div class="card-body">
        <?php if ($assessments === []): ?>
            <p class="text-muted mb-0">No assessments yet.</p>
        <?php endif; ?>

        <?php foreach ($assessments as $assessment): ?>
            <?php
                $timeLimitMins = $hasTimeLimit ? (int) ($assessment['time_limit_minutes'] ?? 0) : 0;
            >
            <div class="d-flex flex-wrap justify-content-between align-items-center border rounded p-3 mb-2" id="assessment-<?
(int) $assessment['id'] ?>">
                <div class="min-w-0 me-3">
                    <div>
                        <strong><? htmlspecialchars((string) $assessment['title']) ?></strong>
                        <span class="badge <? classroom_assessment_type_badge_class((string) ($assessment['assessment_type'] ??
'essay')) ?> ms-1">
                            <? htmlspecialchars(classroom_assessment_type_label((string) ($assessment['assessment_type'] ??
'essay'))) ?>
                        </span>
                        <?php if ($timeLimitMins > 0): ?>
                            <span class="badge bg-dark ms-1"><i class="fa-regular fa-clock me-1"></i><? $timeLimitMins >= 60 ?
((int) floor($timeLimitMins / 60) . 'h ' . ($timeLimitMins % 60) . 'm') : ($timeLimitMins . ' min') ?></span>
                        <?php endif; ?>
                    </div>
                    <div class="small text-muted">
                        <? number_format((float) $assessment['total_points'], 2) ?> points
                        <?php if (!empty($assessment['due_at'])): ?>
                            | Due: <? htmlspecialchars((string) $assessment['due_at']) ?>
                        <?php endif; ?>
                    </div>
                </div>
                <div class="d-flex align-items-center gap-2">
                    <?php
                        $listIntegrityLocked = ($hasIntegrityLocked && (int) ($assessment['integrity_locked'] ?? 0) === 1)
                        || strpos((string) ($assessment['feedback'] ?? ''), 'left or minimized the assessment window') !==
false;

                        $listRetakeReq = $retakeRequestMap[(string) (int) $assessment['id']] ?? null;
                        $listRetakeStatus = is_array($listRetakeReq) ? (string) ($listRetakeReq['status'] ?? '') : '';
                    >
                    <?php if ($listIntegrityLocked): ?>
                        <span class="badge bg-danger">Locked</span>
                        <?php if ($listRetakeStatus === 'pending'): ?>
                            <span class="badge bg-warning text-dark">Retake pending</span>
                        <?php elseif ($listRetakeStatus === 'rejected'): ?>
                            <span class="badge bg-secondary">Retake denied</span>
                        <?php endif; ?>
                    <?php elseif ($listRetakeStatus === 'approved' && empty($assessment['submitted_at'])): ?>
                        <span class="badge bg-info text-dark">Retake approved</span>
                    <?php elseif ($assessment['score'] !== null): ?>
                        <span class="badge bg-success"><? number_format((float) $assessment['score'], 2) ?> / <?
number_format((float) $assessment['total_points'], 2) ?></span>
                    <?php elseif (!empty($assessment['submitted_at'])): ?>
                        <span class="badge bg-info text-dark">Submitted</span>
                    <?php endif; ?>
                    <button type="button" class="btn btn-primary btn-sm" data-bs-toggle="modal" data-bs-
target="#assessmentModal<? (int) $assessment['id'] ?>"<? student_tooltip_attr('Opens this assessment in a focused popup window where you

```

```

can read the questions, write your answers, and submit.') ?>>
        <i class="fa-solid fa-up-right-from-square me-1"></i>Open
    </button>
</div>
</div>
</div>
<?php endforeach; ?>
</div>
</div>
</div>
</div>

<?php foreach ($assessments as $assessment): ?>
<?php
$aid = (int) $assessment['id'];
$questions = $assessmentQuestionMap[$aid] ?? [];
$timeLimitMins = $hasTimeLimit ? (int) ($assessment['time_limit_minutes'] ?? 0) : 0;
$alreadySubmitted = !empty($assessment['submitted_at']);
$integrityLocked = ($hasIntegrityLocked && (int) ($assessment['integrity_locked'] ?? 0) === 1)
    || strpos((string) ($assessment['feedback'] ?? ''), 'left or minimized the assessment window') !== false;
$retakeReq = $retakeRequestMap[(string) $aid] ?? null;
$retakeStatus = is_array($retakeReq) ? (string) ($retakeReq['status'] ?? '') : '';
$retakeRemarks = is_array($retakeReq) ? trim((string) ($retakeReq['faculty_remarks'] ?? '')) : '';
$clearTimer = $retakeStatus === 'approved' && !$alreadySubmitted;
$timedAttempt = $timeLimitMins > 0 && !$alreadySubmitted && !$integrityLocked && $questions !== [];
$timerLabel = '';
if ($timeLimitMins > 0) {
    $th = intval($timeLimitMins, 60);
    $tm = $timeLimitMins % 60;
    if ($th > 0 && $tm > 0) {
        $timerLabel = $th . ' hr ' . $tm . ' min';
    } elseif ($th > 0) {
        $timerLabel = $th . ' hr';
    } else {
        $timerLabel = $tm . ' min';
    }
}
?>
<div class="modal fade assessment-take-modal" id="assessmentModal<?= $aid ?>" tabindex="-1" aria-labelledby="assessmentModalLabel<?=
$aid ?>" aria-hidden="true"
    data-bs-backdrop="static"
    data-bs-keyboard="false"
    data-assessment-id="<?= $aid ?>"
    data-time-limit-minutes="<?= $timeLimitMins ?>"
    data-student-id="<?= (int) $studentId ?>"
    data-already-submitted="<?= $alreadySubmitted ? '1' : '0' ?>"
    data-integrity-locked="<?= $integrityLocked ? '1' : '0' ?>"
    data-clear-timer="<?= $clearTimer ? '1' : '0' ?>"
    data-timed-attempt="<?= $timedAttempt ? '1' : '0' ?>"
    <div class="modal-dialog modal-xl modal-fullscreen-sm-down modal-dialog-scrollable">
        <div class="modal-content">
            <div class="modal-header flex-wrap gap-2">
                <h5 class="modal-title me-auto" id="assessmentModalLabel<?= $aid ?>">
                    <i class="fa-solid fa-file-pen me-2 text-primary"></i><?= htmlspecialchars((string) $assessment['title']) ?>
                    <span class="badge <?= classroom_assessment_type_badge_class((string) ($assessment['assessment_type'] ?? 'essay')) ?>
> ms-2 align-middle">
                        <?= htmlspecialchars(classroom_assessment_type_label((string) ($assessment['assessment_type'] ?? 'essay'))) ?>
                    </span>
                </h5>
                <?php if ($timeLimitMins > 0): ?>
                    <div class="assessment-digital-timer d-none" data-timer-display role="timer" aria-live="polite" aria-atomic="true">
                        <span class="assessment-digital-timer_label">Time left</span>
                        <span class="assessment-digital-timer_digits" data-timer-digits>00:00:00</span>
                    </div>
                <?php endif; ?>
                <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
            </div>
            <div class="modal-body">
                <div class="mb-3">
                    <div class="text-muted">
                        <?= number_format((float) $assessment['total_points'], 2) ?> points
                        <?php if (!empty($assessment['due_at'])): ?>
                            | Due: <?= htmlspecialchars((string) $assessment['due_at']) ?>
                        <?php endif; ?>
                        <?php if ($timeLimitMins > 0): ?>
                            | <i class="fa-regular fa-clock"></i> Time limit: <?= htmlspecialchars($timerLabel) ?>
                        <?php endif; ?>
                    </div>
                    <?php if (trim((string) ($assessment['description'] ?? '')) !== ''): ?>
                        <div class="mt-1 classroom-assessment-desc"><?= classroom_content_render_body((string)
$assessment['description']) ?></div>

```

```

        <?php endif; ?>
    </div>

    <?php if ($assessment['score'] != null): ?>
        <div class="alert alert-success small">
            <i class="fa-solid fa-check-circle me-1"></i><strong>Score:</strong>
            <?= number_format((float) $assessment['score'], 2) ?> / <?= number_format((float) $assessment['total_points'],
2) ?>
            <?php if (trim((string) ($assessment['feedback'] ?? '')) != ''): ?>
                <br>Feedback: <?= htmlspecialchars((string) $assessment['feedback']) ?>
            <?php endif; ?>
        </div>
    <?php endif; ?>

    <?php if (!empty($assessment['submitted_at'])): ?>
        <div class="alert alert-light border small">
            Submitted: <?= htmlspecialchars((string) $assessment['submitted_at']) ?>
            <?php if (!empty($assessment['submission_status'])): ?>
                | Status: <?= htmlspecialchars((string) $assessment['submission_status']) ?>
            <?php endif; ?>
        </div>
    <?php endif; ?>

    <?php if ($integrityLocked): ?>
        <div class="alert alert-danger small">
            <i class="fa-solid fa-lock me-1"></i>
            This assessment is locked. You left or minimized the window (or switched tabs), so <strong>Update
answer</strong> is disabled.
        </div>
        <?php if ($retakeStatus === 'pending'): ?>
            <div class="alert alert-warning small">
                <i class="fa-solid fa-hourglass-half me-1"></i>
                Your request for another attempt is <strong>pending</strong> faculty approval.
            </div>
        <?php elseif ($retakeStatus === 'rejected'): ?>
            <div class="alert alert-secondary small">
                <i class="fa-solid fa-circle-xmark me-1"></i>
                Your retake request was <strong>denied</strong>.
                <?php if ($retakeRemarks != ''): ?>
                    <br>Instructor note: <?= htmlspecialchars($retakeRemarks) ?>
                <?php endif; ?>
            </div>
            <div class="border rounded p-3 mb-3 bg-light">
                <h6 class="mb-2"><i class="fa-solid fa-rotate-left me-1"></i>Request another attempt</h6>
                <p class="small text-muted mb-2">You may submit a new request with additional explanation.</p>
                <form method="post">
                    <input type="hidden" name="action" value="request_retake">
                    <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
                    <input type="hidden" name="assessment_id" value="<?= $aid ?>">
                    <label class="form-label small" for="retakeReason<?= $aid ?>">Your explanation</label>
                    <textarea class="form-control form-control-sm mb-2" name="reason" id="retakeReason<?= $aid ?>" rows="3"
maxlength="2000" required placeholder="Briefly explain what happened and why you are requesting another chance..."></textarea>
                    <div class="text-end">
                        <button type="submit" class="btn btn-outline-primary btn-sm">
                            <i class="fa-solid fa-paper-plane me-1"></i>Send new request
                        </button>
                    </div>
                </form>
            </div>
        <?php else: ?>
            <div class="border rounded p-3 mb-3 bg-light">
                <h6 class="mb-2"><i class="fa-solid fa-rotate-left me-1"></i>Request another attempt</h6>
                <p class="small text-muted mb-2">Explain why you believe you should be allowed to retake this assessment.
Your instructor will review your request.</p>
                <form method="post">
                    <input type="hidden" name="action" value="request_retake">
                    <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
                    <input type="hidden" name="assessment_id" value="<?= $aid ?>">
                    <label class="form-label small" for="retakeReason<?= $aid ?>">Your explanation</label>
                    <textarea class="form-control form-control-sm mb-2" name="reason" id="retakeReason<?= $aid ?>" rows="3"
maxlength="2000" required placeholder="Briefly explain what happened and why you are requesting another chance..."></textarea>
                    <div class="text-end">
                        <button type="submit" class="btn btn-outline-primary btn-sm">
                            <i class="fa-solid fa-paper-plane me-1"></i>Send request to instructor
                        </button>
                    </div>
                </form>
            </div>
        <?php endif; ?>
    <?php elseif ($retakeStatus === 'approved' && !$alreadySubmitted): ?>

```

```

        <div class="alert alert-success small">
            <i class="fa-solid fa-check-circle me-1"></i>
            Your instructor approved another attempt. You may take this assessment again.
        </div>
    <?php elseif ($alreadySubmitted): ?>
        <div class="alert alert-secondary small">
            <i class="fa-solid fa-lock me-1"></i>
            This assessment is already submitted. <strong>Update answer</strong> is disabled.
        </div>
    <?php endif; ?>

    <?php if ($timedAttempt): ?>
        <div class="assessment-timer-gate text-center py-4" data-timer-gate>
            <div class="assessment-digital-timer assessment-digital-timer--hero mb-3" aria-hidden="true">
                <span class="assessment-digital-timer__label">Time limit</span>
                <span class="assessment-digital-timer__digits"><?= sprintf('%02d:%02d:00', intval($timeLimitMins, 60),
$timeLimitMins % 60) ?></span>
            </div>
            <p class="mb-3 text-muted">Once you start, the countdown begins. Your answers will be submitted automatically when
time runs out.</p>
            <p class="mb-3 small text-danger"><i class="fa-solid fa-triangle-exclamation me-1"></i>Leaving, minimizing, closing,
or switching to another tab/window will end this assessment immediately and disable further updates.</p>
            <button type="button" class="btn btn-primary btn-lg px-4" data-timer-start>
                <i class="fa-solid fa-play me-2"></i>Start assessment
            </button>
        </div>
    <?php endif; ?>

    <div class="assessment-timer-body" data-timer-body<?= $timedAttempt ? ' hidden' : '' ?>>
    <?php if (!$timedAttempt && !$integrityLocked && !$alreadySubmitted): ?>
        <div class="alert alert-warning small py-2">
            <i class="fa-solid fa-triangle-exclamation me-1"></i>
            Stay on this assessment window. Leaving, minimizing, closing, or switching tabs/apps will end your assessment,
submit your answers, and disable updates.
        </div>
    <?php endif; ?>
    <?php
    $answersReadOnly = $integrityLocked || $alreadySubmitted;
    $disabledAttr = $answersReadOnly ? ' disabled' : '';
    ?>
    <form method="post" data-assessment-form<?= $answersReadOnly ? ' onsubmit="return false;" : '' ?>>
        <input type="hidden" name="action" value="submit_assessment">
        <input type="hidden" name="classroom_id" value="<?= (int) $classroomId ?>">
        <input type="hidden" name="assessment_id" value="<?= $aid ?>">
        <input type="hidden" name="integrity_violation" value="0" data-integrity-field>
        <?php if ($questions === []): ?>
            <div class="alert alert-warning small mb-2">This assessment has no questions yet.</div>
        <?php endif; ?>
        <?php foreach ($questions as $idx => $q): ?>
            <?php
            $qid = (int) ($q['id'] ?? 0);
            $qType = classroom_question_normalize_type((string) ($q['question_type'] ?? 'essay'));
            $savedAnswer = (string) (($submissionQuestionMap[$aid][$qid]['answer_text'] ?? ''));
            $savedSteps = (string) (($submissionQuestionMap[$aid][$qid]['answer_steps'] ?? ''));
            $options = json_decode((string) ($q['options_json'] ?? '[]'), true);
            if (!is_array($options)) {
                $options = [];
            }
            ?>
            <div class="mb-3 border rounded p-3 bg-light-subtle">
                <div class="fw-semibold mb-1">Q<?= (int) ($idx + 1) ?>. <?= htmlspecialchars((string) $q['question_text']) ?
></div>
                <div class="small text-muted mb-2"><?= htmlspecialchars(classroom_question_type_label($qType)) ?> &bull; <?=
number_format((float) ($q['points'] ?? 0), 2) ?> pts</div>
                <?php if ($qType === 'multiple_choice'): ?>
                    <?php foreach ($options as $oIdx => $opt): ?>
                        <?php $val = (string) chr(65 + (int) $oIdx); ?>
                        <div class="form-check">
                            <input class="form-check-input" type="radio" name="answers[<?= $qid ?>]" id="qm<?= $aid ?>_<?=
$qid ?>_<?= $val ?>" value="<?= htmlspecialchars($val) ?>" <?= $savedAnswer === $val ? 'checked' : '' ?><?= $answersReadOnly ? '' : '
required' ?><?= $disabledAttr ?>>
                            <label class="form-check-label" for="qm<?= $aid ?>_<?= $qid ?>_<?= $val ?>"><?=
htmlspecialchars($val . ' . ' . (string) $opt) ?></label>
                        </div>
                    <?php endforeach; ?>
                <?php elseif ($qType === 'true_false'): ?>
                    <?php foreach (['true' => 'True', 'false' => 'False'] as $tfVal => $tfLabel): ?>
                        <div class="form-check">
                            <input class="form-check-input" type="radio" name="answers[<?= $qid ?>]" id="qm<?= $aid ?>_<?=
$qqid ?>_<?= $tfVal ?>" value="<?= $tfVal ?>" <?= strtolower($savedAnswer) === $tfVal ? 'checked' : '' ?><?= $answersReadOnly ? '' : '

```

```

required' ?><?= $disabledAttr ?>>
                                <label class="form-check-label" for="qm<?= $aid ?>_<?= $qid ?>_<?= $tfVal ?>"><?= $tfLabel ?>
</label>
                                </div>
                                <?php endforeach; ?>
                                <?php else: ?>
                                    <textarea class="form-control mb-2" name="answers[<?= $qid ?>]" rows="<?= $qType === 'essay' ? 5 : 3 ?>"
<?= (int) ($q['char_limit'] ?? 0) > 0 ? 'maxlength="' . (int) $q['char_limit'] . '" : '' ?><?= $answersReadOnly ? '' : ' required' ?><?=
$disabledAttr ?>><?= htmlspecialchars($savedAnswer) ?></textarea>
                                    <?php if ((int) ($q['word_limit'] ?? 0) > 0): ?>
                                        <div class="small text-muted">Word limit: <?= (int) $q['word_limit'] ?></div>
                                    <?php endif; ?>
                                <?php endif; ?>
                                <?php if ((int) ($q['allow_steps'] ?? 0) === 1): ?>
                                    <label class="form-label small mt-2">Optional step-by-step work</label>
                                    <textarea class="form-control" name="answer_steps[<?= $qid ?>]" rows="3" placeholder="Show your steps
for teacher review"><?= $disabledAttr ?>><?= htmlspecialchars($savedSteps) ?></textarea>
                                <?php endif; ?>
                                </div>
                                <?php endforeach; ?>
                                <?php if ($questions !== [] && !$answersReadOnly): ?>
                                    <div class="text-end">
                                        <button type="submit" class="btn btn-primary"><?= student_tooltip_attr('Sends your written answer to your
instructor for grading. Use this when you are ready to submit.') ?>>
                                            <i class="fa-solid fa-paper-plane me-1"></i>Submit answer
                                        </button>
                                    </div>
                                <?php elseif ($answersReadOnly): ?>
                                    <div class="text-end">
                                        <button type="button" class="btn btn-secondary" disabled aria-disabled="true">
                                            <i class="fa-solid fa-lock me-1"></i>Update answer disabled
                                        </button>
                                    </div>
                                <?php endif; ?>
                            </form>
                        </div>
                    </div>
                </div>
            </div>
        <?php endforeach; ?>
    <?php endif; ?>

    <?php if ($hasClassroomDiscussions): ?>
    <script>
document.getElementById('studentClassDiscussionThread')?.scrollTo(0, document.getElementById('studentClassDiscussionThread').scrollHeight);
    </script>
    <?php endif; ?>

    <script>
document.getElementById('studentBannerFile')?.addEventListener('change', function () {
    if (this.files && this.files.length > 0) {
        document.getElementById('studentBannerUploadForm')?.submit();
    }
});
    </script>

    <style>
.assessment-digital-timer {
    display: inline-flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    background: linear-gradient(160deg, #0b1220 0%, #111827 100%);
    color: #22d3ee;
    border-radius: 0.65rem;
    padding: 0.35rem 0.85rem;
    min-width: 7.5rem;
    box-shadow: inset 0 0 0 1px rgba(34, 211, 238, 0.28), 0 6px 16px rgba(15, 23, 42, 0.18);
}
.assessment-digital-timer__label {
    font-size: 0.65rem;
    letter-spacing: 0.12em;
    text-transform: uppercase;
    color: #94a3b8;
    line-height: 1.2;
}
.assessment-digital-timer__digits {
    font-family: "Consolas", "Courier New", monospace;
    font-size: 1.45rem;

```

```

    font-weight: 700;
    letter-spacing: 0.08em;
    line-height: 1.2;
}
.assessment-digital-timer.is-warning .assessment-digital-timer__digits {
    color: #fbbf24;
}
.assessment-digital-timer.is-danger .assessment-digital-timer__digits {
    color: #f87171;
    animation: assessment-timer-pulse 1s ease-in-out infinite;
}
.assessment-digital-timer--hero {
    margin: 0 auto;
    padding: 0.75rem 1.25rem;
    min-width: 10rem;
}
.assessment-digital-timer--hero .assessment-digital-timer__digits {
    font-size: 2rem;
}
@keyframes assessment-timer-pulse {
    0%, 100% { opacity: 1; }
    50% { opacity: 0.55; }
}
</style>

```

```

<div class="modal fade" id="assessmentIntegrityWarningModal" tabindex="-1" aria-labelledby="assessmentIntegrityWarningLabel" aria-
hidden="true" data-bs-backdrop="static" data-bs-keyboard="false">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content border-warning">
      <div class="modal-header bg-warning-subtle">
        <h5 class="modal-title" id="assessmentIntegrityWarningLabel">
          <i class="fa-solid fa-triangle-exclamation me-2 text-warning"></i>Assessment ended
        </h5>
      </div>
      <div class="modal-body">
        Your assessment has ended because you left or minimized the assessment window, closed it, or switched to another tab or
application.
        <div class="mt-2 small text-muted">Your answers are being submitted now.</div>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-warning" data-integrity-ack>OK</button>
      </div>
    </div>
  </div>
</div>
<script>
document.querySelectorAll('[data-wordpad]').forEach((shell) => {
  const fieldName = shell.getAttribute('data-wordpad-name') || 'body';
  const textarea = shell.querySelector('textarea[name="' + fieldName + '"]');
  const editor = shell.querySelector('.wordpad-editor');
  const toolbar = shell.querySelector('.wordpad-toolbar');
  const blockSelect = shell.querySelector('[data-wordpad-block]');
  const form = shell.closest('form');

  if (!textarea || !editor || !toolbar || !form) {
    return;
  }

  toolbar.classList.remove('d-none');
  editor.classList.remove('d-none');
  editor.innerHTML = textarea.value.trim() !== '' ? textarea.value : '';
  textarea.classList.add('d-none');

  const syncEditor = () => {
    const html = editor.innerHTML
      .replace(/<div><br></div>/gi, '')
      .replace(/&nbsp;/gi, ' ')
      .trim();
    textarea.value = html;
  };

  const runCommand = (command, value = null) => {
    editor.focus();
    document.execCommand('styleWithCSS', false, false);
    document.execCommand(command, false, value);
    syncEditor();
  };

  toolbar.addEventListener('click', (event) => {
    const button = event.target.closest('button[data-command]');

```



```

        if (!button) {
            return;
        }

        event.preventDefault();
        const command = button.getAttribute('data-command') || '';

        if (command === 'createLink') {
            const url = window.prompt('Enter link URL', 'https://');
            if (url) {
                runCommand(command, url);
            }
            return;
        }

        runCommand(command);
    });

    blockSelect?.addEventListener('change', (event) => {
        const value = event.target.value || '<p>';
        runCommand('formatBlock', value);
    });

    editor.addEventListener('input', syncEditor);
    editor.addEventListener('blur', syncEditor);
    form.addEventListener('submit', syncEditor);
});
</script>

```

```

<script src="assets/js/student_offline.js" defer></script>
<?php
if ($assessmentAlreadySubmittedWarning !== '') {
    render_schedule_errors_warning_popup(
        [$assessmentAlreadySubmittedWarning],
        'Warning - Assessment already submitted',
        'OK'
    );
}
$footerScripts = ['assets/js/assessment_integrity.js?v=20260721c'];
require_once __DIR__ . '/includes/footer.php';
?>

```

FILE: student_classrooms.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['student']);

$studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
$hasLiveAt = db_column_exists('schedules', 'online_live_at');
$classIsLive = static function (?string $liveAt): bool {
    if ($liveAt === null || $liveAt === '') {
        return false;
    }

    $liveTs = strtotime($liveAt);
    if ($liveTs === false) {
        return false;
    }

    return (time() - $liveTs) <= 2 * 3600;
};
$formatTime12h = static function (?string $time): string {
    $raw = substr((string) $time, 0, 5);
    $dt = DateTime::createFromFormat('H:i', $raw);
    return $dt ? $dt->format('g:i A') : $raw;
};
$isLabSession = static function (?string $start, ?string $end, ?string $roomType = null, ?string $sessionKind = null, ?bool $courseIsLab = null): bool {
    return schedule_session_is_laboratory($start, $end, $roomType, $sessionKind, $courseIsLab);
};

if ($studentId < 1) {

```

```

        $studentId = resolve_student_id_for_user($userId) ?? 0;
        $_SESSION['student_id'] = $studentId > 0 ? $studentId : null;
    }
    if ($studentId < 1) {
        exit('Student profile not linked to this account. Ask your instructor to create or link your student profile.');
```

```

    }

    $requiredTables = [
        'online_classrooms',
        'classroom_students',
        'classroom_enrollments',
        'classroom_content',
        'classroom_assessments',
        'classroom_submissions',
        'classroom_scores',
    ];
    $missingTables = array_values(array_filter(
        $requiredTables,
        static fn (string $table): bool => !db_table_exists($table)
    ));
    $hasJoinCode = db_table_exists('online_classrooms') && db_column_exists('online_classrooms', 'join_code');
```

```

    if ($_SERVER['REQUEST_METHOD'] === 'POST' && $missingTables === [] && $hasJoinCode) {
        $action = (string) ($_POST['action'] ?? '');
        if ($action === 'join_with_code') {
            try {
                $code = classroom_normalize_join_code((string) ($_POST['join_code'] ?? ''));
                if (strlen($code) < 4) {
                    throw new RuntimeException('Enter the class code your instructor shared.');
```

```

                }
                $st = db()->prepare(
                    'SELECT oc.id, oc.status, s.college_id AS schedule_college_id
                     FROM online_classrooms oc
                     INNER JOIN schedules s ON s.id = oc.schedule_id
                     WHERE oc.join_code = ? LIMIT 1'
                );
                $st->execute([$code]);
                $row = $st->fetch();
                if (!$row) {
                    throw new RuntimeException('No class matches that code. Check with your instructor and try again.');
```

```

                }
                if ((string) $row['status'] !== 'active') {
                    throw new RuntimeException('That class is not accepting new enrollments.');
```

```

                }
                $schedCollege = $row['schedule_college_id'] !== null ? (int) $row['schedule_college_id'] : null;
                $userCollege = current_college_id();
                if ($schedCollege !== null && ($userCollege === null || $userCollege !== $schedCollege)) {
                    throw new RuntimeException('That class is for a different college.');
```

```

                }
                $joinClassroomId = (int) $row['id'];
                $chk = db()->prepare('SELECT COUNT(*) FROM classroom_enrollments WHERE classroom_id = ? AND student_id = ?');
                $chk->execute([$joinClassroomId, $studentId]);
                if ((int) $chk->fetchColumn() > 0) {
                    throw new RuntimeException('You are already enrolled in that class.');
```

```

                }
                db()->prepare('INSERT INTO classroom_enrollments (classroom_id, student_id) VALUES (?,?)')
                    ->execute([$joinClassroomId, $studentId]);
                $_SESSION['flash'] = 'You joined the class successfully.';
            } catch (Throwable $e) {
                $_SESSION['flash'] = 'Error: ' . $e->getMessage();
            }
            header('Location: student_classrooms.php');
            exit;
        }
    }
}
```

```

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);
```

```

$classes = [];
$announcements = [];
```

```

$hasCourseLabFlag = db_column_exists('courses', 'is_laboratory');
$hasSessionKind = db_column_exists('schedules', 'session_kind');
$labKindSelect = ($hasCourseLabFlag ? ', c.is_laboratory' : '')
    . ($hasSessionKind ? ', s.session_kind' : '');
```

```

if ($missingTables === []) {
    $st = db()->prepare(
        'SELECT oc.id, oc.title, oc.description, oc.meet_link, oc.status,
```

```

        c.course_code, c.course_name, f.full_name AS faculty_name,
        s.semester, s.school_year, s.day_of_week, s.start_time, s.end_time, s.online_live_at'
    . $labKindSelect . ',
    (SELECT COUNT(*) FROM classroom_content cc WHERE cc.classroom_id = oc.id AND cc.content_type = "announcement") AS
announcement_count,
    (SELECT COUNT(*) FROM classroom_assessments ca WHERE ca.classroom_id = oc.id) AS assessment_count,
    (SELECT COUNT(*)
        FROM classroom_assessments ca
        LEFT JOIN classroom_submissions sub
            ON sub.assessment_id = ca.id AND sub.student_id = ?
        WHERE ca.classroom_id = oc.id AND sub.id IS NULL) AS pending_count
    FROM classroom_enrollments ce
    INNER JOIN online_classrooms oc ON oc.id = ce.classroom_id
    INNER JOIN schedules s ON s.id = oc.schedule_id
    INNER JOIN courses c ON c.id = oc.course_id
    INNER JOIN faculty f ON f.id = oc.faculty_id
    WHERE ce.student_id = ?
    ORDER BY s.school_year DESC, s.semester, c.course_code'
);
$st->execute([$studentId, $studentId]);
$classess = $st->fetchAll();

$st = db()->prepare(
'SELECT oc.id AS classroom_id, oc.title AS classroom_title, cc.title, cc.body, cc.created_at
FROM classroom_content cc
INNER JOIN online_classrooms oc ON oc.id = cc.classroom_id
INNER JOIN classroom_enrollments ce ON ce.classroom_id = oc.id
WHERE ce.student_id = ? AND cc.content_type = "announcement"
ORDER BY cc.created_at DESC
LIMIT 10'
);
$st->execute([$studentId]);
$announcements = $st->fetchAll();
}

$pageTitle = 'My Classes';
require_once __DIR__ . '/includes/header.php';
?>
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-4 student-page-header">
    <h1 class="h3 mb-0"><i class="fa-solid fa-user-graduate me-2 text-primary"></i>My Classes</h1>
    <div class="d-flex flex-wrap gap-2 align-items-center">
        <button type="button" class="btn btn-outline-primary btn-sm" data-offline-save=""><?=> student_tooltip_attr('Downloads all faculty
announcements and materials from your enrolled classes so you can read them without internet.') ?>>
            <i class="fa-solid fa-cloud-arrow-down me-1"></i>Save all for offline
        </button>
        <a href="student_offline.php" class="btn btn-outline-secondary btn-sm"><?=> student_tooltip_attr('Opens your saved offline copy of
faculty posts. Works even without internet after you save once.') ?>>
            <i class="fa-solid fa-book-open me-1"></i>Read offline
        </a>
    </div>
</div>
</div>
<p class="small mb-3" data-offline-status="auto"></p>

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if ($missingTables !== []): ?>
    <div class="alert alert-warning">
        Student classroom features are not installed yet. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this page.
    </div>
<?php else: ?>
    <?php if ($hasJoinCode): ?>
        <div class="card shadow-sm mb-4">
            <div class="card-header bg-white"><strong>Join a class with a code</strong></div>
            <div class="card-body">
                <p class="small text-muted mb-3">Your program chair or instructor gives you a code. Enter it here to add the class to your
list.</p>
                <form method="post" class="row g-2 align-items-end student-join-form">
                    <input type="hidden" name="action" value="join_with_code">
                    <div class="col-12 col-md-6 col-lg-4">
                        <label class="form-label" for="join-code-input">Class code</label>
                        <input id="join-code-input" type="text" name="join_code" class="form-control font-monospace text-uppercase"
maxlength="16" placeholder="e.g. AB12CD34" autocomplete="off" required>
                    </div>
                    <div class="col-12 col-sm-auto">
                        <button type="submit" class="btn btn-primary"><?=> student_tooltip_attr('Enrolls you in the class that matches the
code you entered. Use this after your instructor or program chair shares a join code with you.') ?>>Join class</button>
                    </div>
                </form>
            </div>
        </div>
    </div>
</div>

```

```

<?php endif; ?>
<div class="row g-4">
    <div class="col-lg-8">
        <div class="row g-3">
            <?php if ($classes === []): ?>
                <div class="col-12">
                    <div class="card shadow-sm">
                        <div class="card-body text-muted">You are not enrolled in any online classroom yet.</div>
                    </div>
                </div>
            <?php endif; ?>
            <?php foreach ($classes as $class): ?>
                <div class="col-12 col-md-6">
                    <?php
                    $liveAt = $hasLiveAt ? (string) ($class['online_live_at'] ?? '') : '';
                    $isLive = $hasLiveAt && $classIsLive($liveAt);
                    ?>
                    <div class="card shadow-sm h-100">
                        <div class="card-body">
                            <div class="d-flex justify-content-between align-items-start gap-2">
                                <div>
                                    <h2 class="h5 mb-1"><? = htmlspecialchars((string) $class['title']) ?></h2>
                                    <div class="text-muted small"><? = htmlspecialchars((string) $class['course_code']) ?> - <? =
htmlspecialchars((string) $class['course_name']) ?></div>
                                </div>
                                <div class="d-flex flex-wrap justify-content-end gap-1">
                                    <?php
                                    $classIsLab = array_key_exists('is_laboratory', $class)
                                        ? ((int) ($class['is_laboratory'] ?? 0) === 1)
                                        : null;
                                    $classKind = isset($class['session_kind']) ? (string) $class['session_kind'] : null;
                                    if ($isLabSession(
                                        (string) ($class['start_time'] ?? ''),
                                        (string) ($class['end_time'] ?? ''),
                                        null,
                                        $classKind,
                                        $classIsLab
                                    )):
                                    ?>
                                        <span class="badge bg-warning text-dark"><i class="fa-solid fa-flask me-1"></i>Laboratory</span>
                                    <?php else: ?>
                                        <span class="badge bg-info text-dark"><i class="fa-solid fa-chalkboard me-1"></i>Lecture</span>
                                    <?php endif; ?>
                                    <?php if ($isLive): ?>
                                        <span class="badge bg-danger live-pulse-badge">LIVE</span>
                                    <?php endif; ?>
                                    <span class="badge <? = (string) $class['status'] === 'archived' ? 'bg-secondary' : 'bg-success' ?>>">
<? = htmlspecialchars((string) $class['status']) ?></span>
                                </div>
                            </div>
                            <p class="small mt-3 mb-2">Instructor: <? = htmlspecialchars((string) $class['faculty_name']) ?></p>
                            <p class="small text-muted mb-3"><? = htmlspecialchars((string) $class['semester']) ?> / <? =
htmlspecialchars((string) $class['school_year']) ?></p>
                            <p class="small text-muted mb-3">
                                <i class="fa-solid fa-calendar-days me-1"></i><? = htmlspecialchars(str_replace(',', ' ', (string)
($class['day_of_week'] ?? ''))) ?>
                                <br>
                                <i class="fa-solid fa-clock me-1"></i><? = htmlspecialchars($formatTime12h((string) ($class['start_time']
?? ''))) ?> - <? = htmlspecialchars($formatTime12h((string) ($class['end_time'] ?? ''))) ?>
                            </p>
                            <?php if ($isLive): ?>
                                <div class="small text-danger fw-semibold mb-3">
                                    <i class="fa-solid fa-circle-play me-1"></i>Your instructor is live now.
                                </div>
                            <?php endif; ?>
                            <div class="small mb-3 student-class-card-stats">
                                <span class="me-2">Announcements: <strong><? = (int) $class['announcement_count'] ?></strong></span>
                                <span class="me-2">Assessments: <strong><? = (int) $class['assessment_count'] ?></strong></span>
                                <span>Pending: <strong><? = (int) $class['pending_count'] ?></strong></span>
                            </div>
                            <div class="d-flex gap-2 flex-wrap">
                                <a href="student_classroom.php?id=<? = (int) $class['id'] ?>" class="btn btn-primary btn-sm"><? =
student_tooltip_attr('Opens this class workspace with announcements, materials, discussion, and assessments. Use this as your main entry for
daily class work.') ?>>Open class</a>
                                <button type="button" class="btn btn-outline-primary btn-sm" data-offline-save="<? = (int) $class['id'] ?
>" data-offline-merge="1"><? = student_tooltip_attr('Saves this classroom faculty announcements and materials on your device for offline
reading.') ?>>
                                    <i class="fa-solid fa-download me-1"></i>Save offline
                                </button>
                                <?php if (trim((string) $class['meet_link']) != ''): ?>

```

```

                <?php if ($isLive): ?>
                    <a href="<? = htmlspecialchars((string) $class['meet_link']) ?>" target="_blank" rel="noopener
norereferrer" class="btn btn-success btn-sm">? = student_tooltip_attr('Your instructor is live. Opens the video meeting for this class in a new
tab.') ?>><i class="fa-solid fa-video me-1"></i>Join Meet</a>
                <?php else: ?>
                    <button type="button" class="btn btn-outline-secondary btn-sm" disabled? =
student_tooltip_attr('Join Meet is available only when your instructor goes live for this class.') ?>><i class="fa-solid fa-video me-1">
</i>Join Meet</button>
                <?php endif; ?>
            <?php endif; ?>
        </div>
    </div>
</div>
<?php endforeach; ?>
</div>
</div>
<div class="col-lg-4">
    <div class="card shadow-sm">
        <div class="card-header bg-white"><strong>Latest Announcements</strong></div>
        <div class="card-body">
            <?php if ($announcements === []): ?>
                <p class="text-muted mb-0">No announcements yet.</p>
            <?php else: ?>
                <?php foreach ($announcements as $item): ?>
                    <div class="border-bottom pb-3 mb-3">
                        <div class="small text-muted mb-1"><? = htmlspecialchars((string) $item['classroom_title']) ?></div>
                        <div class="fw-semibold"><? = htmlspecialchars((string) $item['title']) ?></div>
                        <?php if (trim((string) ($item['body'] ?? '')) !== ''): ?>
                            <div class="small mt-1"><? = nl2br(htmlspecialchars((string) $item['body'])) ?></div>
                        <?php endif; ?>
                        <div class="small text-muted mt-1"><? = htmlspecialchars((string) $item['created_at']) ?></div>
                    </div>
                <?php endforeach; ?>
            <?php endif; ?>
        </div>
    </div>
</div>
</div>
<?php endif; ?>

<script src="assets/js/student_offline.js" defer></script>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: student_classroom_week.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['student']);

$studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
$hasLiveAt = db_column_exists('schedules', 'online_live_at');
$hasContentAttachments = db_table_exists('classroom_content_attachments');
$hasContentWeeks = db_column_exists('classroom_content', 'weeks');
$hasContentDaysPerTopic = db_column_exists('classroom_content', 'days_per_topic');
$hasContentTopicSchedule = $hasContentWeeks && $hasContentDaysPerTopic;
$hasSyllabusCols = db_column_exists('online_classrooms', 'syllabus_stored_name');
$hasCreditedWeek = db_column_exists('classroom_assessments', 'credited_week');
$hasAssessmentTables = db_table_exists('classroom_assessments') && db_table_exists('classroom_scores') &&
db_table_exists('classroom_submissions');
$classIsLive = static function (?string $liveAt): bool {
    if ($liveAt === null || $liveAt === '') {
        return false;
    }

    $liveTs = strtotime($liveAt);
    if ($liveTs === false) {
        return false;
    }

    return (time() - $liveTs) <= 2 * 3600;
};

```

```

if ($studentId < 1) {
    $studentId = resolve_student_id_for_user($userId) ?? 0;
    $_SESSION['student_id'] = $studentId > 0 ? $studentId : null;
}
if ($studentId < 1) {
    exit('Student profile not linked to this account. Ask your instructor to create or link your student profile.');
```

```

$className = (int) ($_GET['id'] ?? 0);
$requestedWeek = trim((string) ($_GET['week'] ?? ''));
$requiredTables = [
    'online_classrooms',
    'classroom_students',
    'classroom_enrollments',
    'classroom_content',
];
$missingTables = array_values(array_filter(
    $requiredTables,
    static fn (string $table): bool => !db_table_exists($table)
));
```

```

$className = null;
$weekItems = [];
$contentAttachmentMap = [];
$weekAssessments = [];
```

```

if ($className > 0 && $missingTables === []) {
    $st = db()->prepare(
        'SELECT oc.*, c.course_code, c.course_name, f.full_name AS faculty_name, s.semester, s.school_year, s.online_live_at
        FROM classroom_enrollments ce
        INNER JOIN online_classrooms oc ON oc.id = ce.classroom_id
        INNER JOIN courses c ON c.id = oc.course_id
        INNER JOIN faculty f ON f.id = oc.faculty_id
        INNER JOIN schedules s ON s.id = oc.schedule_id
        WHERE oc.id = ? AND ce.student_id = ?
        LIMIT 1'
    );
    $st->execute([$className, $studentId]);
    $className = $st->fetch() ?: null;
}
```

```

if ($missingTables === [] && !$className) {
    http_response_code(404);
    exit('Classroom not found or you are not enrolled in it.');
```

```

if ($missingTables === []) {
    $st = db()->prepare(
        'SELECT *
        FROM classroom_content
        WHERE classroom_id = ? AND content_type <> "announcement"
        ORDER BY created_at DESC'
    );
    $st->execute([$className]);
    $materials = $st->fetchAll();
    $materialWeeks = classroom_content_group_by_week($materials);
    $weekLookup = [];
    foreach ($materialWeeks as $group) {
        $weekLookup[(string) $group['label']] = $group;
    }
```

```

    if ($requestedWeek === '') {
        $requestedWeek = $materialWeeks !== [] ? (string) $materialWeeks[0]['label'] : '';
    }
```

```

    if ($requestedWeek !== '' && isset($weekLookup[$requestedWeek])) {
        $weekItems = $weekLookup[$requestedWeek]['items'];
    } elseif ($materials !== []) {
        http_response_code(404);
        exit('Week not found for this classroom.');
```

```

    if ($hasContentAttachments) {
        $contentAttachmentMap = classroom_content_attachment_map(array_column($weekItems, 'id'));
    }
```

```

    if ($hasCreditedWeek && $hasAssessmentTables && $requestedWeek !== '') {
        $st = db()->prepare(
            'SELECT ca.*, sc.score, sc.feedback, sc.graded_at, sub.status AS submission_status, sub.submitted_at
```

```

        FROM classroom_assessments ca
        LEFT JOIN classroom_scores sc ON sc.assessment_id = ca.id AND sc.student_id = ?
        LEFT JOIN classroom_submissions sub ON sub.assessment_id = ca.id AND sub.student_id = ?
        WHERE ca.classroom_id = ? AND ca.credited_week = ?
        ORDER BY ca.created_at DESC'
    );
    $st->execute([$studentId, $studentId, $classroomId, $requestedWeek]);
    $weekAssessments = $st->fetchAll();
}
}

$pageTitle = $requestedWeek !== '' ? 'Materials - ' . $requestedWeek : 'Materials by Week';
require_once __DIR__ . '/includes/header.php';
?>
<?php
$classroomLiveAt = $hasLiveAt && $classroom ? (string) ($classroom['online_live_at'] ?? '') : '';
$classroomIsLive = $hasLiveAt && $classIsLive($classroomLiveAt);
$studentWeekSyllabusReady = $classroom && $hasSyllabusCols && trim((string) ($classroom['syllabus_stored_name'] ?? '')) !== '';
?>
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-4 student-page-header">
    <div class="min-w-0">
        <h1 class="h3 mb-1">
            <i class="fa-solid fa-calendar-week me-2 text-primary"></i><? = htmlspecialchars($requestedWeek !== '' ? $requestedWeek : 'Course
Materials') ?>
            <?php if ($classroomIsLive): ?>
                <span class="badge bg-danger live-pulse-badg align-middle ms-2">LIVE</span>
            <?php endif; ?>
        </h1>
        <?php if ($classroom): ?>
            <div class="text-muted student-class-meta">
                <? = htmlspecialchars((string) $classroom['course_code']) ?> - <? = htmlspecialchars((string) $classroom['course_name']) ?>
                | Instructor: <? = htmlspecialchars((string) $classroom['faculty_name']) ?>
            </div>
        <?php endif; ?>
    </div>
    <div class="d-flex flex-wrap gap-2 student-page-header__actions">
        <?php if ($studentWeekSyllabusReady): ?>
            <a href="<? = htmlspecialchars(classroom_syllabus_href((int) $classroomId)) ?>" target="_blank" rel="noopener noreferrer"
class="btn btn-primary btn-sm"><? = student_tooltip_attr('Opens the course syllabus in a new tab.') ?><i class="fa-solid fa-file-contract me-
1"></i>Syllabus</a>
        <?php endif; ?>
            <a href="student_materials_reviewer.php?classroom_id=<? = (int) $classroomId ?>&week=<? = rawurlencode($requestedWeek) ?>" class="btn
btn-outline-primary btn-sm"><? = student_tooltip_attr('Generate practice review questions from this week\'s posted materials.') ?><i
class="fa-solid fa-clipboard-question me-1"></i>Review this week</a>
            <a href="student_classroom.php?id=<? = (int) $classroomId ?>" class="btn btn-outline-secondary btn-sm"><? =
student_tooltip_attr('Returns to the full classroom page with announcements, discussion, and assessments. Use this when you are done
browsing this week\'s materials.') ?>>Back to Classroom</a>
            <a href="student_classrooms.php" class="btn btn-outline-primary btn-sm"><? = student_tooltip_attr('Opens the list of all your enrolled
classes. Use this to switch subjects or join another Meet.') ?>>My Classes</a>
        </div>
    </div>

<?php if ($missingTables !== []): ?>
    <div class="alert alert-warning">
        Student classroom features are not installed yet. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this page.
    </div>
<?php else: ?>
    <div class="card shadow-sm">
        <div class="card-header bg-white d-flex justify-content-between align-items-center flex-wrap gap-2">
            <strong>Week Materials and Topics</strong>
            <?php if ($weekItems !== []): ?>
                <span class="badge text-bg-primary"><? = count($weekItems) ?> item<? = count($weekItems) === 1 ? '' : 's' ?></span>
            <?php endif; ?>
        </div>
        <div class="card-body">
            <?php if ($weekItems === []): ?>
                <p class="text-muted mb-0">No course materials were posted for this week yet.</p>
            <?php else: ?>
                <?php foreach ($weekItems as $item): ?>
                    <div class="border rounded p-3 mb-3">
                        <div class="d-flex justify-content-between align-items-start gap-3 flex-wrap">
                            <div>
                                <div class="fw-semibold"><? = htmlspecialchars((string) $item['title']) ?></div>
                                <div class="small text-muted mb-1"><? = htmlspecialchars((string) $item['content_type']) ?></div>
                            </div>
                            <div class="small text-muted"><? = htmlspecialchars((string) $item['created_at']) ?></div>
                        </div>

                        <?php if (trim((string) ($item['body'] ?? '')) !== ''): ?>
                            <div class="small mt-2 classroom-content-body"><? = classroom_content_render_body((string) $item['body']) ?>

```

```

</div>

<?php endif; ?>

<?php if ($hasContentTopicSchedule && (trim((string) ($item['weeks'] ?? '')) != '' || trim((string)
($item['days_per_topic'] ?? '')) != '')): ?>
    <div class="small text-muted mt-2">
        <?php if (trim((string) ($item['weeks'] ?? '')) != ''): ?>
            <span class="me-2"><i class="fa-regular fa-calendar me-1"></i>Weeks: <?= htmlspecialchars((string)
$item['weeks']) ?></span>
            <?php endif; ?>
            <?php if (trim((string) ($item['days_per_topic'] ?? '')) != ''): ?>
                <span><i class="fa-regular fa-clock me-1"></i>Days/topic: <?= htmlspecialchars((string)
$item['days_per_topic']) ?></span>
            <?php endif; ?>
        </div>
    <?php endif; ?>

    <?php if (trim((string) ($item['resource_url'] ?? '')) != ''): ?>
        <?php $resourceUrl = trim((string) $item['resource_url']); ?>
        <?php if (classroom_content_is_attachment($resourceUrl)): ?>
            <?php
            $legacyName = classroom_content_attachment_name($resourceUrl);
            $legacyHref = classroom_content_resource_href((int) $item['id'], $resourceUrl);
            $legacyIsImage = classroom_content_is_image_filename($legacyName);
            ?>
            <?php if ($legacyIsImage): ?>
                <div class="mt-2 classroom-content-attachment-image">
                    <a href="<?= htmlspecialchars($legacyHref) ?>" target="_blank" rel="noopener noreferrer"<?=
student_tooltip_attr('Opens the image your instructor attached for this week's topic.') ?>>
                        " loading="lazy" decoding="async">
                    </a>
                    <div class="small mt-1">
                        <a href="<?= htmlspecialchars($legacyHref) ?>"<?= student_tooltip_attr('Downloads or opens a
file linked to this week's topic.') ?>>
                            <i class="fa-solid fa-paperclip me-1"></i><?= htmlspecialchars($legacyName) ?>
                        </a>
                    </div>
                </div>
            <?php else: ?>
                <div class="small mt-2">
                    <a href="<?= htmlspecialchars($legacyHref) ?>"<?= student_tooltip_attr('Downloads or opens a file
linked to this week's topic. Use this for readings, slides, or worksheets your instructor posted.') ?>>
                        <i class="fa-solid fa-paperclip me-1"></i><?= htmlspecialchars($legacyName) ?>
                    </a>
                </div>
            <?php endif; ?>
        <?php else: ?>
            <div class="small mt-2">
                <a href="<?= htmlspecialchars(classroom_content_resource_href((int) $item['id'], $resourceUrl)) ?>"
target="_blank" rel="noopener noreferrer"<?= student_tooltip_attr('Opens the linked resource in a new tab. Use this for external sites or
documents your instructor linked for this week.') ?>>Open resource</a>
            </div>
        <?php endif; ?>
    <?php endif; ?>

    <?php foreach ($contentAttachmentMap[(int) $item['id']] ?? [] as $attachment): ?>
        <?php
        $attachName = classroom_content_attachment_download_name((string) $attachment['original_name'], (string)
$attachment['stored_name']);
        $attachHref = classroom_content_attachment_href((int) $attachment['id']);
        $attachInlineHref = classroom_content_attachment_href((int) $attachment['id'], true);
        ?>
        <?php if (classroom_content_attachment_is_image($attachment)): ?>
            <div class="mt-2 classroom-content-attachment-image">
                <a href="<?= htmlspecialchars($attachHref) ?>" target="_blank" rel="noopener noreferrer"<?=
student_tooltip_attr('Opens the image your instructor attached for this week's topic.') ?>>
                    "
loading="lazy" decoding="async">
                </a>
                <div class="small mt-1">
                    <a href="<?= htmlspecialchars($attachHref) ?>"<?= student_tooltip_attr('Downloads or opens an extra
attachment for this week's item.') ?>>
                        <i class="fa-solid fa-paperclip me-1"></i><?= htmlspecialchars($attachName) ?>
                    </a>
                </div>
            </div>
        <?php else: ?>
            <div class="small mt-2">
                <a href="<?= htmlspecialchars($attachHref) ?>"<?= student_tooltip_attr('Downloads or opens an extra

```



```

attachment for this week's item. Use this when there are multiple files for the same topic.') ?>>
        <i class="fa-solid fa-paperclip me-1"></i><?>= htmlspecialchars($attachName) ?>
    </a>
</div>
<?php endif; ?>
<?php endforeach; ?>
</div>
<?php endforeach; ?>
<?php endif; ?>
</div>
</div>

<?php if ($weekAssessments != []): ?>
    <div class="card shadow-sm mt-4">
        <div class="card-header bg-white d-flex justify-content-between align-items-center flex-wrap gap-2">
            <strong><i class="fa-solid fa-clipboard-check me-1 text-warning"></i>Assessments for <?>= htmlspecialchars($requestedWeek) ?>
        </strong>
        <span class="badge text-bg-warning"><?>= count($weekAssessments) ?> <?>= count($weekAssessments) == 1 ? 'assessment' :
'assessments' ?></span>
        </div>
        <div class="card-body">
            <?php foreach ($weekAssessments as $wa): ?>
                <a href="student_classroom.php?id=<?>= (int) $classroomId ?>#assessment-<?>= (int) $wa['id'] ?>" class="border rounded p-3
mb-3 d-block text-decoration-none text-body hover-shadow-sm">
                    <div class="d-flex justify-content-between align-items-start gap-3 flex-wrap">
                        <div>
                            <div class="fw-semibold">
                                <i class="fa-solid fa-file-pen me-1 text-warning"></i><?>= htmlspecialchars((string) $wa['title']) ?>
                            </div>
                            <span class="badge <?>= classroom_assessment_type_badge_class((string) ($wa['assessment_type'] ?? 'essay')) ?>
> mt-1">
                                <?>= htmlspecialchars(classroom_assessment_type_label((string) ($wa['assessment_type'] ?? 'essay'))) ?>
                            </span>
                            <div class="small text-muted mt-1">
                                <?>= number_format((float) $wa['total_points'], 2) ?> points
                                <?php if (!empty($wa['due_at'])): ?>
                                    | Due: <?>= htmlspecialchars((string) $wa['due_at']) ?>
                                <?php endif; ?>
                            </div>
                        </div>
                        <div class="text-end">
                            <?php if ($wa['score'] != null): ?>
                                <span class="badge bg-success">Graded</span>
                                <div class="small mt-1"><?>= number_format((float) $wa['score'], 2) ?> / <?>= number_format((float)
$wa['total_points'], 2) ?></div>
                            <?php elseif (!empty($wa['submitted_at'])): ?>
                                <span class="badge bg-info">Submitted</span>
                            <?php else: ?>
                                <span class="badge bg-secondary">Not submitted</span>
                            <?php endif; ?>
                        </div>
                    </div>
                    <?php if (trim((string) ($wa['description'] ?? '')) != ''): ?>
                        <div class="small text-muted mt-2"><?>= mb_strimwidth(strip_tags((string) $wa['description']), 0, 150, '...') ?>
                    </div>
                <?php endif; ?>
            </a>
        <?php endforeach; ?>
    </div>
</div>
<?php endif; ?>
</div>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: student_edutools.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['student']);

$userId = (int) ($_SESSION['user_id'] ?? 0);
$storageNs = 'u' . ($userId > 0 ? (string) $userId : '0');

```

```

$eduToolsCatalog = [
    ['id' => 'notebook', 'name' => 'Notebook', 'icon' => '📓', 'category' => 'notebooks'],
    ['id' => 'notebooklm', 'name' => 'NotebookLM', 'icon' => '🗨️', 'category' => 'notebooks'],
    ['id' => 'chatgpt', 'name' => 'ChatGPT', 'icon' => '🤖', 'category' => 'ai'],
    ['id' => 'perplexity', 'name' => 'Perplexity', 'icon' => '🔍', 'category' => 'ai'],
    ['id' => 'wolfram', 'name' => 'Wolfram', 'icon' => '📊', 'category' => 'reference'],
    ['id' => 'khan', 'name' => 'Khan Academy', 'icon' => '🎓', 'category' => 'reference'],
    ['id' => 'notion', 'name' => 'Notion', 'icon' => '📌', 'category' => 'reference'],
];

$pageTitle = 'Learning tools (EduTools)';
$bodyPageClass = 'page-student-edutools';
$mainContainerClass = 'container-fluid px-3 px-lg-4 py-3 py-md-4 flex-grow-1 d-flex flex-column min-w-0 app-main student-edutools-page';
require_once __DIR__ . '/includes/header.php';
?>
<div class="mb-3">
    <h1 class="h4 mb-1"><i class="fa-solid fa-wand-magic-sparkles me-2 text-primary"></i>EduTools</h1>
    <p class="text-muted small mb-0 d-none d-lg-block">AI helpers, references, and a personal notebook for your studies. Use the sidebar to
    pick a tool; <kbd class="small">Ctrl</kbd>+<kbd class="small">E</kbd> toggles the tool menu.</p>
    <p class="text-muted small mb-0 d-lg-none">Swipe the tool chips at the top, then use the notebook or launch button below.</p>
</div>

<div class="student-edutools-root flex-grow-1 d-flex flex-column min-h-0" data-storage-ns="<?> htmlspecialchars($storageNs, ENT_QUOTES,
'UTF-8') ?>">
    <style>
        .student-edutools-root * {
            box-sizing: border-box;
        }

        .student-edutools-root .app-container {
            display: flex;
            width: 100%;
            position: relative;
            min-height: min(720px, calc(100vh - 14rem));
            border-radius: 12px;
            overflow: hidden;
            border: 1px solid #e2e8f0;
            background: #ffffff;
        }

        .student-edutools-root .tools-panel {
            width: 280px;
            max-width: 92vw;
            background: rgba(255, 255, 255, 0.96);
            backdrop-filter: blur(2px);
            border-right: 1px solid #e2e8f0;
            display: flex;
            flex-direction: column;
            transition: transform 0.25s ease, width 0.2s;
            z-index: 30;
            box-shadow: 2px 0 12px rgba(0, 0, 0, 0.03);
            flex-shrink: 0;
            height: auto;
            align-self: stretch;
            min-height: min(680px, calc(100vh - 15rem));
            overflow-y: auto;
        }

        .student-edutools-root .tools-panel.collapsed {
            transform: translateX(-100%);
            position: absolute;
            height: 100%;
            min-height: 100%;
        }

        .student-edutools-root .main-area {
            flex: 1;
            display: flex;
            flex-direction: column;
            overflow: hidden;
            background: #ffffff;
            transition: all 0.2s;
            min-width: 0;
            min-height: min(680px, calc(100vh - 15rem));
        }

        .student-edutools-root .top-bar {
            background: white;
            padding: 12px 16px;
            border-bottom: 1px solid #e9eef3;

```

```

display: flex;
align-items: center;
gap: 12px;
flex-wrap: wrap;
box-shadow: 0 1px 2px rgba(0,0,0,0.02);
}

.student-edutools-root .menu-toggle-btn {
background: #f8fafc;
border: 1px solid #cbd5e1;
border-radius: 40px;
padding: 8px 14px;
font-size: 0.9rem;
font-weight: 500;
cursor: pointer;
display: flex;
align-items: center;
gap: 8px;
color: #1e293b;
transition: all 0.2s;
}

.student-edutools-root .menu-toggle-btn:hover {
background: #eef2ff;
border-color: #94a3b8;
}

.student-edutools-root .active-badge {
background: #eef2ff;
padding: 6px 14px;
border-radius: 40px;
font-size: 0.85rem;
color: #1e40af;
font-weight: 500;
display: flex;
align-items: center;
gap: 8px;
}

.student-edutools-root .active-badge span:first-child {
font-weight: 400;
color: #475569;
}

.student-edutools-root .key-hint {
background: #e2e8f0;
padding: 2px 8px;
border-radius: 20px;
font-family: ui-monospace, monospace;
font-size: 0.75rem;
color: #0f172a;
}

.student-edutools-root .tool-viewport {
flex: 1;
position: relative;
background: #fefefe;
overflow: auto;
min-height: 0;
}

.student-edutools-root #eduDynamicContent {
min-height: 100%;
}

.student-edutools-root .mobile-tools-strip {
display: none;
gap: 8px;
padding: 8px 12px;
overflow-x: auto;
overflow-y: hidden;
border-bottom: 1px solid #e9eef3;
background: #f8fafc;
-webkit-overflow-scrolling: touch;
scrollbar-width: thin;
flex-shrink: 0;
}

.student-edutools-root .mobile-tool-chip {
flex: 0 0 auto;

```

```

display: inline-flex;
align-items: center;
gap: 6px;
padding: 7px 12px;
border-radius: 999px;
border: 1px solid #cbd5e1;
background: #ffffff;
color: #1e293b;
font-size: 0.78rem;
font-weight: 600;
white-space: nowrap;
cursor: pointer;
transition: background 0.15s ease, border-color 0.15s ease, color 0.15s ease;
}

.student-edutools-root .mobile-tool-chip:hover {
  background: #f1f5f9;
}

.student-edutools-root .mobile-tool-chip.active {
  background: #eef2ff;
  border-color: #93c5fd;
  color: #1e40af;
}

.student-edutools-root iframe {
  width: 100%;
  height: 100%;
  min-height: 420px;
  border: none;
  display: block;
}

.student-edutools-root .notebook-container {
  padding: 20px 22px;
  height: 100%;
  overflow-y: auto;
  background: #fefcf5;
}

.student-edutools-root .notebook-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 12px;
  margin-bottom: 20px;
  border-bottom: 2px solid #e2e8f0;
  padding-bottom: 12px;
}

.student-edutools-root .notebook-header h2 {
  font-size: 1.35rem;
  font-weight: 600;
  background: linear-gradient(135deg, #0f172a, #334155);
  background-clip: text;
  -webkit-background-clip: text;
  color: transparent;
  margin: 0;
}

.student-edutools-root .note-actions {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
}

.student-edutools-root .note-btn {
  background: white;
  border: 1px solid #cbd5e1;
  padding: 6px 12px;
  border-radius: 30px;
  font-weight: 500;
  font-size: 0.8rem;
  cursor: pointer;
  transition: 0.2s;
}

.student-edutools-root .note-btn.primary {
  background: #0f172a;

```

```

    color: white;
    border: none;
}

.student-edutools-root .note-btn.primary:hover {
    background: #1e293b;
}

.student-edutools-root .note-btn:hover {
    background: #f1f5f9;
}

.student-edutools-root textarea.notepad-textarea {
    width: 100%;
    min-height: 42vh;
    padding: 16px;
    font-family: ui-monospace, 'Courier New', monospace;
    font-size: 0.95rem;
    line-height: 1.5;
    border: 1px solid #e2e8f0;
    border-radius: 16px;
    background: #ffffff;
    resize: vertical;
    box-shadow: inset 0 1px 3px rgba(0,0,0,0.03);
}

.student-edutools-root .status-msg {
    font-size: 0.75rem;
    margin-top: 10px;
    color: #2d6a4f;
}

.student-edutools-root .menu-header {
    padding: 18px 16px 6px 16px;
    font-weight: 600;
    font-size: 0.7rem;
    letter-spacing: 0.06em;
    color: #5b6e8c;
    text-transform: uppercase;
}

.student-edutools-root .tool-list {
    list-style: none;
    margin: 6px 10px 16px 10px;
    padding: 0;
}

.student-edutools-root .tool-item {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 10px 14px;
    margin: 4px 0;
    border-radius: 14px;
    cursor: pointer;
    transition: all 0.15s;
    font-weight: 500;
    font-size: 0.9rem;
    color: #1e293b;
}

.student-edutools-root .tool-item:hover {
    background: #f1f5f9;
}

.student-edutools-root .tool-item.active {
    background: #eef2ff;
    color: #1e40af;
    border-left: 3px solid #3b82f6;
}

.student-edutools-root .tool-icon {
    font-size: 1.25rem;
    line-height: 1;
}

.student-edutools-root .tool-name {
    flex: 1;
}

```

```

.student-edutools-root .badge-external {
  font-size: 0.65rem;
  background: #e2e8f0;
  padding: 2px 8px;
  border-radius: 30px;
  color: #334155;
}

.student-edutools-root .edutools-hr {
  margin: 8px 14px;
  border: 0;
  border-top: 1px solid #edf2f7;
}

.student-edutools-root .shortcut-toast {
  position: absolute;
  bottom: 12px;
  right: 12px;
  background: #0f172ad9;
  backdrop-filter: blur(8px);
  color: white;
  padding: 6px 12px;
  border-radius: 40px;
  font-size: 0.68rem;
  font-family: ui-monospace, monospace;
  pointer-events: none;
  z-index: 25;
  max-width: 90%;
}

.student-edutools-root .badge-google {
  background: #e8eef9;
  color: #1a73e8;
}

.student-edutools-root .tools-panel-backdrop {
  display: none;
  position: absolute;
  inset: 0;
  margin: 0;
  padding: 0;
  border: 0;
  background: rgba(15, 23, 42, 0.45);
  z-index: 35;
  cursor: pointer;
  -webkit-tap-highlight-color: transparent;
}

.student-edutools-root .tools-panel-backdrop.is-visible {
  display: block;
}

@media (max-width: 991.98px) {
  .student-edutools-root .app-container {
    position: relative;
    flex: 1 1 auto;
    flex-direction: column;
    min-height: min(560px, calc(100dvh - 11rem));
    overflow: visible;
  }

  .student-edutools-root .tools-panel,
  .student-edutools-root .tools-panel-backdrop,
  .student-edutools-root .menu-toggle-btn,
  .student-edutools-root .key-hint {
    display: none !important;
  }

  .student-edutools-root .mobile-tools-strip {
    display: flex !important;
    order: 1;
    width: 100%;
  }

  .student-edutools-root .main-area {
    order: 2;
    width: 100%;
    flex: 1 1 auto;
    max-width: 100%;
    min-height: min(520px, calc(100dvh - 12rem));
  }
}

```

```

        overflow: visible;
    }

    .student-edutools-root .top-bar {
        padding: 8px 12px;
    }

    .student-edutools-root .active-badge {
        flex: 1 1 auto;
        min-width: 0;
        font-size: 0.78rem;
    }

    .student-edutools-root .tool-viewport {
        flex: 1 1 auto;
        min-height: min(460px, calc(100dvh - 14rem));
        overflow: auto;
        -webkit-overflow-scrolling: touch;
    }

    .student-edutools-root #eduDynamicContent {
        min-height: min(460px, calc(100dvh - 14rem));
    }

    .student-edutools-root textarea.notepad-textarea {
        min-height: min(360px, calc(100dvh - 17rem));
    }

    .student-edutools-root .notebook-container {
        padding: 14px;
    }

    .student-edutools-root .shortcut-toast {
        display: none;
    }

    .student-edutools-root iframe {
        min-height: 360px;
    }
}

@media (max-width: 480px) {
    .student-edutools-root .note-actions .note-btn {
        flex: 1 1 calc(50% - 0.25rem);
        text-align: center;
    }
}

</style>

<div class="app-container">
    <div class="mobile-tools-strip" id="eduMobileToolsStrip" aria-label="Quick tool picker">
        <?php foreach ($eduToolsCatalog as $tool): ?>
            <button type="button" class="mobile-tool-chip" <?=$tool['id'] == 'notebook' ? ' active' : '' ?>" data-tool-id="<?=
htmlspecialchars($tool['id'], ENT_QUOTES, 'UTF-8') ?>">
                <span aria-hidden="true"><?=$tool['icon'] ?></span>
                <span><?= htmlspecialchars($tool['name'], ENT_QUOTES, 'UTF-8') ?></span>
            </button>
        <?php endforeach; ?>
    </div>
    <button type="button" class="tools-panel-backdrop" id="eduToolsBackdrop" aria-label="Close tool menu" tabindex="-1" aria-hidden="true">
</button>
    <aside class="tools-panel" id="eduToolsPanel" aria-label="Learning tools">
        <div style="padding: 16px 16px 0 16px;">
            <h2 style="font-size: 1.35rem; font-weight: 700; margin: 0; background: linear-gradient(145deg, #0f2b3d, #1e4a76); background-clip:
text; -webkit-background-clip: text; color: transparent;">EduTools</h2>
            <p style="font-size: 0.75rem; color: #4b5563; margin-top: 6px;">Smart learning panel & education sources</p>
        </div>
        <div class="menu-header">AI & learning tools</div>
        <ul class="tool-list" id="eduToolMenuList"></ul>
        <div class="menu-header" style="margin-top: 6px;">Notebooks</div>
        <ul class="tool-list" id="eduNotebookSectionList"></ul>
        <hr class="edutools-hr" />
        <div style="padding: 10px 16px 16px 16px; font-size: 0.68rem; color: #6c757d;">
            <kbd>Ctrl</kbd> + <kbd>E</kbd> toggles this tool menu
        </div>
    </aside>

    <div class="main-area">
        <div class="top-bar">

```

[illegible]


```

}

function syncToolsBackdrop() {
  if (!toolsBackdrop) return;
  var open = isMobileToolsLayout() && !toolsPanel.classList.contains('collapsed');
  toolsBackdrop.classList.toggle('is-visible', open);
  toolsBackdrop.setAttribute('aria-hidden', open ? 'false' : 'true');
  toolsBackdrop.tabIndex = open ? 0 : -1;
}

function setPanelCollapsed(collapsed, persist) {
  toolsPanel.classList.toggle('collapsed', collapsed);
  if (persist !== false) {
    localStorage.setItem(PANEL_COLLAPSED_KEY, collapsed ? 'true' : 'false');
  }
  syncToolsBackdrop();
}

function renderMenu() {
  toolMenuList.innerHTML = '';
  notebookSectionList.innerHTML = '';
  if (mobileToolsStrip) {
    mobileToolsStrip.innerHTML = '';
  }
  const aiTools = toolsConfig.filter(function (t) { return t.category === 'ai'; });
  const refTools = toolsConfig.filter(function (t) { return t.category === 'reference'; });
  const notebookTools = toolsConfig.filter(function (t) { return t.category === 'notebooks'; });
  aiTools.forEach(function (tool) { toolMenuList.appendChild(createMenuItem(tool)); });
  refTools.forEach(function (tool) { toolMenuList.appendChild(createMenuItem(tool)); });
  notebookTools.forEach(function (tool) { notebookSectionList.appendChild(createMenuItem(tool)); });
  toolsConfig.forEach(function (tool) {
    if (!mobileToolsStrip) return;
    var existing = mobileToolsStrip.querySelector('[data-tool-id="' + tool.id + '"]');
    if (!existing) {
      mobileToolsStrip.appendChild(createMobileChip(tool));
    }
  });
}

function createMobileChip(tool) {
  var btn = document.createElement('button');
  btn.type = 'button';
  btn.className = 'mobile-tool-chip' + (currentToolId === tool.id ? ' active' : '');
  btn.setAttribute('data-tool-id', tool.id);
  btn.innerHTML = '<span aria-hidden="true">' + tool.icon + '</span><span>' + escapeHtml(tool.name) + '</span>';
  btn.addEventListener('click', function () {
    activateTool(tool.id);
    if (isMobileToolsLayout()) {
      setPanelCollapsed(true);
    }
  });
  return btn;
}

function createMenuItem(tool) {
  const li = document.createElement('li');
  li.className = 'tool-item' + (currentToolId === tool.id ? ' active' : '');
  li.setAttribute('data-tool-id', tool.id);
  const isNotebookLM = tool.id === 'notebooklm';
  li.innerHTML =
    '<span class="tool-icon">' + tool.icon + '</span>' +
    '<span class="tool-name">' + escapeHtml(tool.name) + '</span>' +
    '<span class="badge-external" + (isNotebookLM ? ' badge-google' : '') + ">' + (tool.openNewTab ? '\u2197\uFE0F' : '\uD83D\uDCD3') +
    '</span>';
  li.addEventListener('click', function (e) {
    e.stopPropagation();
    activateTool(tool.id);
    if (isMobileToolsLayout()) {
      setPanelCollapsed(true);
    }
  });
  return li;
}

function escapeHtml(s) {
  const d = document.createElement('div');
  d.textContent = s;
  return d.innerHTML;
}

```

```

function renderNotebook() {
  const saved = localStorage.getItem(NOTEBOOK_STORAGE_KEY) || '# Welcome to your Smart Notebook\n\nTake notes, draft ideas, and save study resources.\n\n## Tips\n- Notes auto-save while you type\n- Export as .md or .txt\n- Open NotebookLM from the menu for AI on your sources';
  notebookContent = saved;
  toolViewport.innerHTML =
    '<div class="notebook-container">' +
    '<div class="notebook-header">' +
    '<h2>Personal learning notebook</h2>' +
    '<div class="note-actions">' +
    '<button type="button" id="eduExportMdBtn" class="note-btn">Export .md</button>' +
    '<button type="button" id="eduExportTxtBtn" class="note-btn">Export .txt</button>' +
    '<button type="button" id="eduClearNotebookBtn" class="note-btn">Clear</button>' +
    '</div>' +
    '</div>' +
    '<textarea id="eduNotebookTextarea" class="notepad-textarea" placeholder="Write your notes here..."></textarea>' +
    '<div class="status-msg" id="eduNoteStatus">Auto-save enabled</div>' +
    '</div>';

  const textarea = document.getElementById('eduNotebookTextarea');
  if (textarea) {
    textarea.value = notebookContent;
    notebookTextarea = textarea;
    var saveTimeout;
    textarea.addEventListener('input', function () {
      if (saveTimeout) clearTimeout(saveTimeout);
      saveTimeout = setTimeout(function () {
        var newContent = textarea.value;
        localStorage.setItem(NOTEBOOK_STORAGE_KEY, newContent);
        notebookContent = newContent;
        var statusDiv = document.getElementById('eduNoteStatus');
        if (statusDiv) {
          statusDiv.textContent = 'Saved at ' + new Date().toLocaleTimeString();
          setTimeout(function () {
            if (statusDiv) statusDiv.textContent = 'Auto-save active';
          }, 1500);
        }
      }, 500);
    });
  }
  var exportMd = document.getElementById('eduExportMdBtn');
  var exportTxt = document.getElementById('eduExportTxtBtn');
  var clearBtn = document.getElementById('eduClearNotebookBtn');
  if (exportMd) exportMd.addEventListener('click', function () { exportNotebook('md'); });
  if (exportTxt) exportTxt.addEventListener('click', function () { exportNotebook('txt'); });
  if (clearBtn) {
    clearBtn.addEventListener('click', function () {
      if (confirm('Clear all notebook content? This cannot be undone.')) {
        if (notebookTextarea) notebookTextarea.value = '';
        localStorage.setItem(NOTEBOOK_STORAGE_KEY, '');
        notebookContent = '';
        var statusDiv = document.getElementById('eduNoteStatus');
        if (statusDiv) statusDiv.textContent = 'Notebook cleared.';
      }
    });
  }
}

function exportNotebook(format) {
  var content = '';
  if (notebookTextarea) {
    content = notebookTextarea.value;
  } else {
    content = localStorage.getItem(NOTEBOOK_STORAGE_KEY) || '';
  }
  var blob = new Blob([content], { type: format === 'md' ? 'text/markdown' : 'text/plain' });
  var link = document.createElement('a');
  var filename = 'my_learning_notes.' + (format === 'md' ? 'md' : 'txt');
  link.href = URL.createObjectURL(blob);
  link.download = filename;
  link.click();
  URL.revokeObjectURL(link.href);
  var statusDiv = document.getElementById('eduNoteStatus');
  if (statusDiv) statusDiv.textContent = 'Exported as ' + filename;
}

function renderExternalHelper(tool) {
  var isNotebookLM = tool.id === 'notebooklm';
  var gradientBg = isNotebookLM ? 'linear-gradient(135deg, #f5f0ff 0%, #e8eefa 100%)' : '#fafcff';
  var accentColor = isNotebookLM ? '#8b5cf6' : '#0f172a';

```

```

var specialNote = isNotebookLM
? "NotebookLM is Google's AI notebook. Upload sources (PDFs, docs, web) and ask questions, get summaries, and connect ideas."
: '';
toolViewport.innerHTML =
  '<div style="display:flex;align-items:center;justify-content:center;min-height:100%;background:' + gradientBg + ';padding:1.5rem;">' +
  '<div style="max-width:520px;text-align:center;background:white;border-radius:1.5rem;padding:1.75rem;box-shadow:0 12px 30px' +
  'rgba(0,0,0,0.08);border:1px solid #eef2ff;">' +
  '<div style="font-size:3rem;line-height:1;">' + tool.icon + '</div>' +
  '<h2 style="margin:0.75rem 0 0.35rem;font-size:1.25rem;color:' + accentColor + ';">' + escapeHtml(tool.name) + '</h2>' +
  '<p style="color:#4b5563;margin-bottom:0.75rem;font-size:0.95rem;">' + escapeHtml(tool.description || 'Learning assistant') +
  '</p>' +
  (specialNote ? '<p style="background:#f3e8ff;padding:8px 12px;border-radius:16px;font-size:0.8rem;color:#6b21a5;margin:12px 0;">' +
  + escapeHtml(specialNote) + '</p>' : '') +
  '<button type="button" id="eduOpenExternalBtn" class="note-btn primary" style="padding:10px 28px;border-radius:60px;font-size:1rem;margin-top:8px;">Launch ' + escapeHtml(tool.name) + ' \u2197</button>' +
  '<p style="margin-top:16px;font-size:0.7rem;color:#6c757d;">Opens in a new tab. ' + (isNotebookLM ? 'Sign in with Google to use'
  NotebookLM.' : 'Best experienced on the official site.') + '</p>' +
  '</div>' +
  '</div>';
var openBtn = document.getElementById('eduOpenExternalBtn');
if (openBtn && tool.url) {
  openBtn.addEventListener('click', function () {
    window.open(tool.url, '_blank', 'noopener,noreferrer');
  });
}

function activateTool(toolId) {
  var tool = toolsConfig.find(function (t) { return t.id === toolId; });
  if (!tool) return;
  currentToolId = toolId;
  activeToolNameSpan.textContent = tool.name;
  document.querySelectorAll('.student-edutools-root .tool-item').forEach(function (item) {
    if (item.getAttribute('data-tool-id') === toolId) {
      item.classList.add('active');
    } else {
      item.classList.remove('active');
    }
  });
  document.querySelectorAll('.student-edutools-root .mobile-tool-chip').forEach(function (chip) {
    if (chip.getAttribute('data-tool-id') === toolId) {
      chip.classList.add('active');
    } else {
      chip.classList.remove('active');
    }
  });
  if (tool.type === 'notebook') {
    renderNotebook();
    setTimeout(function () {
      notebookTextarea = document.getElementById('eduNotebookTextarea');
      if (notebookTextarea && notebookContent) notebookTextarea.value = notebookContent;
    }, 10);
  } else if (tool.type === 'external') {
    renderExternalHelper(tool);
  } else if (tool.type === 'iframe' && tool.url) {
    toolViewport.innerHTML = '<iframe src="' + escapeHtml(tool.url) + '" title="' + escapeHtml(tool.name) + '" sandbox="allow-same-origin' +
    allow-scripts allow-popups allow-forms"></iframe>';
    localStorage.setItem(LAST_TOOL_KEY, toolId);
  }
}

function togglePanel() {
  setPanelCollapsed(!toolsPanel.classList.contains('collapsed'));
}

function restoreState() {
  var lastTool = localStorage.getItem(LAST_TOOL_KEY);
  var storedCollapsed = localStorage.getItem(PANEL_COLLAPSED_KEY);
  var panelCollapsed = storedCollapsed === 'true';
  if (isMobileToolsLayout()) {
    panelCollapsed = true;
  } else if (storedCollapsed === null) {
    panelCollapsed = false;
  }
  setPanelCollapsed(panelCollapsed, false);
  if (lastTool && toolsConfig.some(function (t) { return t.id === lastTool; })) {
    activateTool(lastTool);
  } else {
    activateTool('notebook');
  }
}

```

```

        toolsPanel.classList.add('tools-panel-ready');
        syncToolsBackdrop();
    }

    function handleKeydown(e) {
        if ((e.ctrlKey || e.metaKey) && (e.key === 'e' || e.key === 'E')) {
            e.preventDefault();
            togglePanel();
        }
    }

    window.addEventListener('beforeunload', function () {
        if (currentToolId === 'notebook' && notebookTextarea) {
            localStorage.setItem(NOTEBOOK_STORAGE_KEY, notebookTextarea.value);
        }
    });

    renderMenu();
    restoreState();
    document.querySelectorAll('.student-edutools-root .mobile-tool-chip[data-tool-id]').forEach(function (chip) {
        if (chip.getAttribute('data-edu-bound') === '1') return;
        chip.setAttribute('data-edu-bound', '1');
        chip.addEventListener('click', function () {
            var toolId = chip.getAttribute('data-tool-id');
            if (toolId) {
                activateTool(toolId);
                if (isMobileToolsLayout()) {
                    setPanelCollapsed(true);
                }
            }
        });
    });
    if (toggleBtn) {
        toggleBtn.addEventListener('click', togglePanel);
    }
    if (toolsBackdrop) {
        toolsBackdrop.addEventListener('click', function () {
            if (!toolsPanel.classList.contains('collapsed')) {
                setPanelCollapsed(true);
            }
        });
    }
    if (typeof mobileToolsQuery.addEventListener === 'function') {
        mobileToolsQuery.addEventListener('change', function () {
            if (isMobileToolsLayout()) {
                setPanelCollapsed(true);
            } else {
                syncToolsBackdrop();
            }
        });
    }
    } else if (typeof mobileToolsQuery.addListener === 'function') {
        mobileToolsQuery.addListener(function () {
            if (isMobileToolsLayout()) {
                setPanelCollapsed(true);
            } else {
                syncToolsBackdrop();
            }
        });
    }
    window.addEventListener('keydown', handleKeydown);

    setInterval(function () {
        if (currentToolId === 'notebook' && notebookTextarea && document.getElementById('eduNotebookTextarea')) {
            var fresh = notebookTextarea.value;
            localStorage.setItem(NOTEBOOK_STORAGE_KEY, fresh);
            notebookContent = fresh;
        }
    }, 2000);
})();
</script>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: student_materials_reviewer.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';

```

```

require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/student_materials_reviewer.php';
require_once __DIR__ . '/includes/wellness_ai.php';

require_role(['student']);

$studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);

if ($studentId < 1) {
    $studentId = resolve_student_id_for_user($userId) ?? 0;
    $_SESSION['student_id'] = $studentId > 0 ? $studentId : null;
}
if ($studentId < 1) {
    exit('Student profile not linked to this account. Ask your instructor to create or link your student profile.');
```

```

$pageTitle = 'Weekly materials reviewer';
require_once __DIR__ . '/includes/header.php';
?>
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-4 student-page-header">
    <div class="min-w-0">
        <h1 class="h3 mb-1">
            <i class="fa-solid fa-clipboard-question me-2 text-primary"></i>Weekly materials reviewer
        </h1>
        <p class="text-muted small mb-0">
            Generate practice questions from your instructor's posted course materials, one week at a time.
            <?php if ($aiAvailable): ?>
                AI-enhanced questions are enabled when materials are available.
            <?php else: ?>
                Questions are generated from your weekly materials (enable Wellness AI in config for richer AI questions).
            <?php endif; ?>
        </p>
    </div>
    <div class="d-flex flex-wrap gap-2 student-page-header__actions">
        <a href="student_classrooms.php" class="btn btn-outline-primary btn-sm"<? = student_tooltip_attr('Opens all your enrolled classes.')
?>><i class="fa-solid fa-user-graduate me-1"></i>My Classes</a>
    </div>
</div>

<?php if ($missingTables !== []): ?>
    <div class="alert alert-warning">
        Classroom features are not installed yet. Run <a href="upgrade_roles.php">upgrade_roles.php</a> once, then reload this page.
    </div>
<?php elseif ($classes === []): ?>
    <div class="card shadow-sm">
        <div class="card-body">
            <p class="text-muted mb-3">You are not enrolled in any class yet. Join a class first, then return here to review materials by
week.</p>
            <a href="student_classrooms.php" class="btn btn-primary btn-sm">Go to My Classes</a>
        </div>
    </div>
<?php else: ?>
    <div class="row g-4 student-materials-reviewer-layout">
        <div class="col-lg-4 order-lg-1 order-1">
            <div class="card shadow-sm h-100">
                <div class="card-header bg-white"><strong>Select class and week</strong></div>
                <div class="card-body">
                    <form method="get" action="student_materials_reviewer.php" class="mb-3">
                        <label class="form-label small text-muted" for="classroom_id">Class</label>
                        <select class="form-select form-select-sm mb-3" id="classroom_id" name="classroom_id" onchange="this.form.submit()">
                            <?php foreach ($classes as $row): ?>
                                <option value="<? = (int) $row['id'] ?>"<? = (int) $row['id'] === $selectedClassroomId ? ' selected' : '' ?>>
                                    <? = htmlspecialchars((string) $row['course_code']) ?> - <? = htmlspecialchars((string)
$row['course_name']) ?>
                                </option>
                            <?php endforeach; ?>
                        </select>

                        <?php if ($weekGroups !== []): ?>
                            <label class="form-label small text-muted" for="week">Week</label>
                            <select class="form-select form-select-sm" id="week" name="week" onchange="this.form.submit()">
                                <?php foreach ($weekGroups as $group): ?>
                                    <?php $label = (string) $group['label']; ?>
                                    <option value="<? = htmlspecialchars($label, ENT_QUOTES, 'UTF-8') ?>"<? = $label === $selectedWeek ? '
selected' : '' ?>>
                                        <? = htmlspecialchars($label) ?> (<? = (int) $group['count'] ?> item<? = (int) $group['count'] === 1 ?
'' : 's' ?>)
                                    </option>
                                <?php endforeach; ?>
                            </select>
                        <?php else: ?>
                            <p class="small text-muted mb-0">No weekly materials posted for this class yet.</p>
                        <?php endif; ?>
                    </form>

                    <?php if ($selectedClassroomId): ?>
                        <div class="small text-muted border-top pt-3">
                            <div><strong>Instructor:</strong> <? = htmlspecialchars((string) $selectedClassroomId['faculty_name']) ?></div>
                            <?php if ($selectedWeek !== ''): ?>
                                <div class="mt-1"><strong>Selected:</strong> <? = htmlspecialchars($selectedWeek) ?></div>
                            <?php endif; ?>
                        </div>
                    <?php endif; ?>

                    <?php if ($selectedClassroomId > 0 && $selectedWeek !== ''): ?>

```

```

        <div class="mt-3 d-grid gap-2">
            <a href="student_classroom_week.php?id=<?=(int) $selectedClassroomId ?>&week=<?=(int) $selectedWeek ?>"
            >" class="btn btn-outline-secondary btn-sm"<?=(int) $selectedWeek ?>" class="fa-regular fa-folder-open me-1"></i>View week materials
            </a>
        </div>
    </div>
    <?php endif; ?>
</div>
</div>
</div>

<div class="col-lg-8 order-lg-2 order-2">
    <div class="card shadow-sm" id="reviewer-panel"
        data-classroom-id="<?=(int) $selectedClassroomId ?>"
        data-week="<?=(int) $selectedWeek ?>"
        data-api="<?=(int) $selectedWeek ?>"
    <div class="card-header bg-white d-flex flex-wrap justify-content-between align-items-center gap-2">
        <strong>Practice questions</strong>
        <?php if ($selectedWeek != ''): ?>
            <span class="badge text-bg-light border"><?=(int) $selectedWeek ?></span>
        <?php endif; ?>
    </div>
    <div class="card-body">
        <?php if ($weekGroups === []): ?>
            <p class="text-muted mb-0">Your instructor has not posted course materials for this class yet. Check back after
materials are uploaded.</p>
        <?php else: ?>
            <p class="small text-muted">Generate as many review questions as needed from materials posted for the selected week.
Click again to add more.</p>
            <div class="d-flex flex-wrap gap-2 mb-3 student-reviewer-actions">
                <button type="button" class="btn btn-primary btn-sm" id="reviewer-generate-btn"<?=(int) $selectedWeek ?>"
                >Generate questions
            </button>
            <button type="button" class="btn btn-outline-primary btn-sm d-none" id="reviewer-more-btn"<?=(int) $selectedWeek ?>"
            >Generate more
            </button>
            </div>
            <div id="reviewer-status" class="small text-muted mb-2 d-none"></div>
            <div id="reviewer-questions" class="reviewer-questions-list"></div>
        <?php endif; ?>
    </div>
</div>
</div>
</div>

<script>
(function () {
    const panel = document.getElementById('reviewer-panel');
    const btn = document.getElementById('reviewer-generate-btn');
    const moreBtn = document.getElementById('reviewer-more-btn');
    const statusEl = document.getElementById('reviewer-status');
    const listEl = document.getElementById('reviewer-questions');
    if (!panel || !btn || !listEl) {
        return;
    }

    let existingQuestions = [];
    let metaEl = null;

    function setStatus(text, isError) {
        if (!statusEl) {
            return;
        }
        statusEl.textContent = text;
        statusEl.classList.toggle('text-danger', !!isError);
        statusEl.classList.toggle('text-muted', !isError);
        statusEl.classList.remove('d-none');
    }

    function escapeHtml(value) {
        return String(value)
            .replace(/&/g, '&amp;')
            .replace(/</g, '&lt;')
            .replace(/>/g, '&gt;')
            .replace(/"/g, '&quot;');
    }

    function buildQuestionCard(q, displayIndex) {

```

```

const card = document.createElement('div');
card.className = 'reviewer-question-card';

let inner = '<div class="fw-semibold mb-2">Q' + displayIndex + '. ' + escapeHtml(q.question || '') + '</div>';
if (q.type === 'multiple_choice' && Array.isArray(q.choices) && q.choices.length) {
  inner += '<div class="small mb-2">';
  q.choices.forEach(function (choice) {
    inner += '<label class="reviewer-choice"><input type="radio" disabled name="q' + q.id + '" class="me-2">'
      + escapeHtml(choice) + '</label>';
  });
  inner += '</div>';
}
inner += '<button type="button" class="btn btn-outline-secondary btn-sm reviewer-reveal-btn">Show answer</button>';
inner += '<div class="reviewer-answer small"><strong>Answer:</strong> ' + escapeHtml(q.answer || '') + '</div>';
card.innerHTML = inner;

const revealBtn = card.querySelector('.reviewer-reveal-btn');
if (revealBtn) {
  revealBtn.addEventListener('click', function () {
    card.classList.toggle('is-revealed');
    revealBtn.textContent = card.classList.contains('is-revealed') ? 'Hide answer' : 'Show answer';
  });
}

return card;
}

function updateMeta(totalCount, source, materialCount, appendNote) {
  if (!metaEl) {
    metaEl = document.createElement('p');
    metaEl.className = 'small text-muted reviewer-meta';
    listEl.prepend(metaEl);
  }
  let text = totalCount + ' question' + (totalCount === 1 ? '' : 's') + ' shown';
  if (source === 'ai') {
    text += ' (AI-assisted)';
  }
  if (materialCount !== null) {
    text += ' • ' + materialCount + ' material(s) in this week';
  }
  if (appendNote) {
    text += ' • added ' + appendNote + ' more';
  }
  metaEl.textContent = text;
}

function renderAllQuestions() {
  const cards = listEl.querySelectorAll('.reviewer-question-card');
  cards.forEach(function (card) {
    card.remove();
  });

  existingQuestions.forEach(function (q, idx) {
    listEl.appendChild(buildQuestionCard(q, idx + 1));
  });

  if (existingQuestions.length === 0) {
    if (metaEl) {
      metaEl.remove();
      metaEl = null;
    }
    return;
  }

  updateMeta(existingQuestions.length, existingQuestions[0].source || 'builtin', null, '');
}

async function requestQuestions(append) {
  const classroomId = parseInt(panel.dataset.classroomId || '0', 10);
  const week = panel.dataset.week || '';
  const api = panel.dataset.api || '';
  if (!classroomId || !week || !api) {
    setStatus('Select a class and week first.', true);
    return;
  }

  const activeBtn = append ? moreBtn : btn;
  if (activeBtn) {
    activeBtn.disabled = true;
  }
}

```



```

    if (!append) {
      btn.disabled = true;
    }
    setStatus(append ? 'Generating more questions...' : 'Generating questions...', false);

    if (!append) {
      existingQuestions = [];
      listEl.innerHTML = '';
      metaEl = null;
    }

    try {
      const exclude = append
        ? existingQuestions.map(function (q) { return q.question || ''; })
        : [];
      const res = await fetch(api, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        credentials: 'same-origin',
        body: JSON.stringify({
          classroom_id: classroomId,
          week: week,
          count: 0,
          exclude: exclude,
        })
      });
      const data = await res.json();
      if (!res.ok || !data.ok) {
        throw new Error(data.error || 'Could not generate questions.');
```

```

        requestQuestions(true);
    });
}
})();
</script>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: student_offline.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['student']);

$studentId = isset($_SESSION['student_id']) ? (int) $_SESSION['student_id'] : 0;
$userId = (int) ($_SESSION['user_id'] ?? 0);
if ($studentId < 1) {
    $studentId = resolve_student_id_for_user($userId) ?? 0;
    $_SESSION['student_id'] = $studentId > 0 ? $studentId : null;
}

$pageTitle = 'Offline reading';
require_once __DIR__ . '/includes/header.php';
?>
<div class="d-flex flex-wrap justify-content-between align-items-start gap-2 mb-4 student-page-header">
    <div class="min-w-0">
        <h1 class="h3 mb-1"><i class="fa-solid fa-cloud-arrow-down me-2 text-primary"></i>Offline reading</h1>
        <p class="text-muted small mb-0">Click a course box to see saved items, then choose one to review offline.</p>
    </div>
    <div class="d-flex flex-wrap gap-2 student-page-header__actions">
        <button type="button" class="btn btn-primary btn-sm px-3" data-offline-save=""><?=> student_tooltip_attr('Downloads all faculty posts
from your enrolled classes and stores them on this device for offline reading.') ?>>
            <i class="fa-solid fa-download me-1"></i>Save all classes
        </button>
        <a href="student_classrooms.php" class="btn btn-outline-secondary btn-sm px-3"><?=> student_tooltip_attr('Returns to your enrolled
class list.') ?>>My classes</a>
    </div>
</div>

<p class="small mb-3" data-offline-status="auto">Checking saved offline copy...</p>

<div data-offline-reader></div>

<script src="assets/js/student_offline.js?v=2" defer></script>
<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

FILE: student_wellness.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['student']);

$pageTitle = 'Wellness companion';
$mainContainerClass = 'container-fluid px-3 px-lg-4 py-3 py-md-4 flex-grow-1 student-wellness-page';
require_once __DIR__ . '/includes/header.php';
?>
<div class="mb-3 student-page-header">
    <h1 class="h4 mb-1">
        <i class="fa-solid fa-heart-pulse me-2 text-success"></i>Wellness companion
    </h1>
    <p class="text-muted small mb-0">
        Empathic support for stress, anxiety, loneliness, and low mood—English or Filipino.
        Not a substitute for therapy or emergency services.
    </p>
</div>

<?php require_once __DIR__ . '/includes/student_wellness_ui.php'; ?>

```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>
```

```
-----  
FILE: super_admin_accounts.php  
-----
```

```
<?php  
declare(strict_types=1);  
  
require_once __DIR__ . '/includes/auth.php';  
require_once __DIR__ . '/includes/functions.php';  
require_once __DIR__ . '/includes/admin_activity_log.php';  
require_once __DIR__ . '/includes/account_registration_helpers.php';  
require_once __DIR__ . '/includes/mail_helpers.php';  
require_once __DIR__ . '/includes/student_registration_helpers.php';  
  
require_role(['super_admin']);  
  
$flash = $_SESSION['flash'] ?? '';  
unset($_SESSION['flash']);  
  
$currentUserId = (int) ($_SESSION['user_id'] ?? 0);  
$hasUserEmail = db_column_exists('users', 'email');  
$hasMustChange = db_column_exists('users', 'must_change_password');  
$hasLastLogin = db_column_exists('users', 'last_login_at');  
$hasCollegeId = db_column_exists('users', 'college_id');  
$hasAssignedProgram = db_column_exists('users', 'assigned_program');  
$hasAdminLogTitle = db_column_exists('users', 'admin_log_title');  
$hasProgramsTable = db_table_exists('programs');  
$hasCollegesTable = db_table_exists('colleges');  
$hasFacultyTable = db_table_exists('faculty');  
$hasEmploymentStatus = $hasFacultyTable && db_column_exists('faculty', 'employment_status');  
$hasChairProgramsTable = ensure_program_chair_programs_table();  
  
/** @var array<string, string> */  
$roleLabels = [  
    'super_admin' => 'Super Administrator',  
    'admin' => 'Administrator',  
    'dean' => 'Dean',  
    'program_chair' => 'Program Chair',  
    'faculty' => 'Faculty',  
    'gened' => 'General Education',  
    'student' => 'Student',  
];  
  
/** Roles Super Admin may create from this page. */  
$creatableRoles = array_keys($roleLabels);  
  
function super_admin_active_count(): int  
{  
    return (int) db()->query("SELECT COUNT(*) FROM users WHERE role = 'super_admin' AND is_active = 1")->fetchColumn();  
}  
  
function sa_role_label(string $role, array $roleLabels): string  
{  
    return $roleLabels[$role] ?? ucfirst(str_replace('_', ' ', $role));  
}  
  
/**  
 * Delete one user account and related role rows. Returns true when deleted.  
 */  
* @param array<string, string> $roleLabels  
* @throws RuntimeException  
*/  
function sa_delete_account(int $id, int $currentUserId, array $roleLabels): bool  
{  
    if ($id < 1) {  
        throw new RuntimeException('Invalid account.');    }  
    if ($id === $currentUserId) {  
        throw new RuntimeException('You cannot delete your own account.');    }  
  
    $selCols = 'id, username, full_name, role, is_active';  
    if (db_column_exists('users', 'email')) {  
        $selCols .= ', email';  
    }  
    if (db_column_exists('users', 'college_id')) {  
        $selCols .= ', college_id';  
    }  
}
```

```

    }
    if (db_column_exists('users', 'assigned_program')) {
        $selCols .= ', assigned_program';
    }
    $st = db()->prepare("SELECT {$selCols} FROM users WHERE id = ? LIMIT 1");
    $st->execute([$id]);
    $before = $st->fetch(PDO::FETCH_ASSOC);
    if (!$before) {
        return false;
    }

    $role = (string) ($before['role'] ?? '');
    if ($role === 'super_admin' && (int) ($before['is_active'] ?? 0) === 1 && super_admin_active_count() <= 1) {
        throw new RuntimeException('At least one active Super Administrator account is required.');
```

```

    }

    db()->beginTransaction();
    try {
        if ($role === 'dean' && db_table_exists('colleges') && db_column_exists('colleges', 'dean_user_id')) {
            db()->prepare('UPDATE colleges SET dean_user_id = NULL WHERE dean_user_id = ?')->execute([$id]);
        }
        if ($role === 'program_chair' && function_exists('program_chair_programs_table_ready') && program_chair_programs_table_ready()) {
            db()->prepare('DELETE FROM program_chair_programs WHERE user_id = ?')->execute([$id]);
        }
        if ($role === 'faculty' && db_table_exists('faculty')) {
            db()->prepare('DELETE FROM faculty WHERE user_id = ?')->execute([$id]);
        }
        if ($role === 'student' && db_table_exists('classroom_students')) {
            db()->prepare('DELETE FROM classroom_students WHERE user_id = ?')->execute([$id]);
        }

        $del = db()->prepare('DELETE FROM users WHERE id = ?');
        $del->execute([$id]);
        if ($del->rowCount() < 1) {
            throw new RuntimeException('Account could not be deleted.');
```

```

        }
        db()->commit();
    } catch (Throwable $e) {
        if (db()->inTransaction()) {
            db()->rollBack();
        }
        throw $e;
    }
}

$roleLabel = sa_role_label($role, $roleLabels);
log_admin_activity(
    'delete',
    'Account monitor',
    $roleLabel . ' user #' . $id . ' (' . (string) $before['username'] . ')',
    (array) $before,
    null
);
);

return true;
}

```

```

$tab = (string) ($_GET['tab'] ?? 'monitor');
if (!in_array($tab, ['monitor', 'create', 'super_admins'], true)) {
    $tab = 'monitor';
}
$selectRole = trim((string) ($_GET['role'] ?? ''));
if ($selectRole !== '' && !isset($roleLabels[$selectRole])) {
    $selectRole = '';
}

```

```

$editId = isset($_GET['edit']) ? (int) $_GET['edit'] : 0;
$editRow = null;
if ($editId > 0) {
    $tab = 'super_admins';
    $selCols = 'id, username, full_name, is_active';
    if ($hasUserEmail) {
        $selCols .= ', email';
    }
    $st = db()->prepare("SELECT {$selCols} FROM users WHERE id = ? AND role = 'super_admin' LIMIT 1");
    $st->execute([$editId]);
    $editRow = $st->fetch(PDO::FETCH_ASSOC) ?: null;
    if (!$editRow) {
        $editId = 0;
    }
}
}

```

```

$resetId = isset($_GET['reset']) ? (int) $_GET['reset'] : 0;
$resetRow = null;
if ($resetId > 0) {
    $tab = 'monitor';
    $selCols = 'id, username, full_name, role, is_active';
    if ($hasUserEmail) {
        $selCols .= ', email';
    }
    $st = db()->prepare("SELECT {$selCols} FROM users WHERE id = ? LIMIT 1");
    $st->execute([$resetId]);
    $resetRow = $st->fetch(PDO::FETCH_ASSOC) ?: null;
    if (!$resetRow) {
        $resetId = 0;
    }
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if ($action === 'add') {
            $role = trim((string) ($_POST['role'] ?? ''));
            $username = trim((string) ($_POST['username'] ?? ''));
            $fullName = trim((string) ($_POST['full_name'] ?? ''));
            $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
            $hasValidEmail = $hasUserEmail && $email !== '' && filter_var($email, FILTER_VALIDATE_EMAIL);
            $password = (string) ($_POST['password'] ?? '');
            $isActive = !empty($_POST['is_active']) ? 1 : 0;
            $collegeIdRaw = trim((string) ($_POST['college_id'] ?? ''));
            $collegeId = $collegeIdRaw !== '' ? (int) $collegeIdRaw : 0;
            $assignedProgram = trim((string) ($_POST['assigned_program'] ?? ''));
            $adminLogTitle = $hasAdminLogTitle ? trim((string) ($_POST['admin_log_title'] ?? '')) : '';
            $facultyCode = trim((string) ($_POST['faculty_id'] ?? ''));
            $studentNumber = trim((string) ($_POST['student_number'] ?? ''));
            $yearLevel = trim((string) ($_POST['year_level'] ?? ''));

            if (!in_array($role, $creatableRoles, true)) {
                throw new RuntimeException('Invalid role.');
```

```

if ($role === 'faculty') {
    if (!$hasFacultyTable) {
        throw new RuntimeException('Faculty table is missing. Run upgrade_roles.php first.');
```

```

    }
    if ($facultyCode === '') {
        throw new RuntimeException('Faculty ID is required.');
```

```

    }
    $chkFac = db()->prepare('SELECT COUNT(*) FROM faculty WHERE faculty_id = ?');
    $chkFac->execute([$facultyCode]);
    if ((int) $chkFac->fetchColumn() > 0) {
        throw new RuntimeException('Faculty ID already exists.');
```

```

    }
}
```

```

if ($role === 'student' && $studentNumber === '') {
    throw new RuntimeException('Student number is required.');
```

```

}
if ($role === 'student' && !db_table_exists('classroom_students')) {
    throw new RuntimeException('Student accounts require classroom_students. Run upgrade_roles.php first.');
```

```

}
if ($role === 'program_chair' && !$hasChairProgramsTable) {
    throw new RuntimeException('Could not prepare program_chair_programs. Run upgrade_roles.php.');
```

```

}

$cred = prepare_new_account_password($password, $hasValidEmail);
$plainForMail = $hasValidEmail ? $cred['plain'] : '';
```

```

$exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username = ?');
$exists->execute([$username]);
if ((int) $exists->fetchColumn() > 0) {
    $s = suggest_available_usernames($username, 3);
    $msg = 'Username already exists.';
    if ($s) {
        $msg .= ' Try: ' . implode(', ', $s);
    }
    throw new RuntimeException($msg);
}
```

```

$newId = 0;
```

```

if ($role === 'student') {
    $newId = create_student_account(
        $username,
        $cred['hash'],
        $fullName,
        $email,
        $studentNumber,
        $collegeId,
        $assignedProgram,
        true,
        $yearLevel
    );
    if ($isActive === 0) {
        db()->prepare('UPDATE users SET is_active = 0 WHERE id = ?')->execute([$newId]);
    }
}
```

```

} else {
    $fields = ['username', 'password', 'full_name'];
    $params = [$username, $cred['hash'], $fullName];
    if ($hasUserEmail) {
        $fields[] = 'email';
        $params[] = $email;
    }
    if ($role === 'admin' && $hasAdminLogTitle) {
        $fields[] = 'admin_log_title';
        $params[] = $adminLogTitle;
    }
    $fields[] = 'role';
    $params[] = $role;
    if ($needsCollege || ($role === 'gened' && $collegeId > 0 && $hasCollegeId)) {
        $fields[] = 'college_id';
        $params[] = $collegeId > 0 ? $collegeId : null;
    }
    if (in_array($role, ['program_chair', 'faculty'], true) && $hasAssignedProgram) {
        $fields[] = 'assigned_program';
        $params[] = $assignedProgram;
    }
    $fields[] = 'is_active';
    $params[] = $isActive;
}
```

```

    $ph = implode(' ', array_fill(0, count($fields), '?'));
    $sql = 'INSERT INTO users (' . implode(' ', $fields) . ') VALUES (' . $ph . ')';
    db()->prepare($sql)->execute($params);
    $newId = (int) db()->lastInsertId();

    if ($role === 'dean' && $collegeId > 0) {
        db()->prepare('UPDATE colleges SET dean_user_id = ? WHERE id = ?')->execute([$newId, $collegeId]);
    }
    if ($role === 'program_chair') {
        program_chair_set_assigned_programs($newId, [$assignedProgram], $assignedProgram);
    }
    if ($role === 'faculty') {
        $facStatus = $isActive === 1 ? 'active' : 'inactive';
        if ($hasEmploymentStatus) {
            db()->prepare(
                'INSERT INTO faculty (user_id, faculty_id, full_name, department, email, max_hours_per_day, college_id, status,
employment_status, is_generated)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)'
            )->execute([
                $newId,
                $facultyCode,
                $fullName,
                $assignedProgram,
                $email,
                8,
                $collegeId,
                $facStatus,
                'Permanent',
                is_generated
            ]);
        } else {
            db()->prepare(
                'INSERT INTO faculty (user_id, faculty_id, full_name, department, email, max_hours_per_day, college_id, status,
is_generated)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)'
            )->execute([
                $newId,
                $facultyCode,
                $fullName,
                $assignedProgram,
                $email,
                8,
                $collegeId,
                $facStatus,
                is_generated
            ]);
        }
    }
}

    $selAfter = 'id, username, full_name, role, is_active';
    if ($hasUserEmail) {
        $selAfter .= ', email';
    }
    if ($hasCollegeId) {
        $selAfter .= ', college_id';
    }
    if ($hasAssignedProgram) {
        $selAfter .= ', assigned_program';
    }
    $stAfter = db()->prepare("SELECT {$selAfter} FROM users WHERE id = ? LIMIT 1");
    $stAfter->execute([$newId]);
    $afterRow = $stAfter->fetch(PDO::FETCH_ASSOC) ?: [];
    if ($afterRow !== []) {
        $afterRow['password'] = '[set at creation]';
    }
    $roleLabel = sa_role_label($role, $roleLabels);
    log_admin_activity(
        'add',
        'Account creation',
        $roleLabel . ' user #' . $newId,
        null,
        $afterRow !== [] ? $afterRow : null
    );

    $mailOk = $hasValidEmail
        ? notify_new_account_credentials($newId, $email, $fullName, $username, $plainForMail, $role)
        : null;
    if (!$hasValidEmail) {
        mark_new_account_requires_password_change($newId);
    }
    $_SESSION['flash'] = registration_email_flash_message($mailOk, $email, $roleLabel . ' account');

```

```

$redirectTab = $role === 'super_admin' ? 'super_admins' : 'create';
header('Location: super_admin_accounts.php?tab=' . $redirectTab . ($role !== 'super_admin' ? '&role=' . rawurlencode($role) :
''));
exit;
}

if ($action === 'edit') {
    $id = (int) ($_POST['id'] ?? 0);
    $fullName = trim((string) ($_POST['full_name'] ?? ''));
    $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
    $resetPassword = (string) ($_POST['reset_password'] ?? '');
    $isActive = !empty($_POST['is_active']) ? 1 : 0;

    if ($id < 1) {
        throw new RuntimeException('Invalid account.');
```



```

        if ($hasUserEmail) {
            $selAfter .= ', email';
        }
        $stAfter = db()->prepare("SELECT {$selAfter} FROM users WHERE id = ? LIMIT 1");
        $stAfter->execute([$id]);
        $afterRow = $stAfter->fetch(PDO::FETCH_ASSOC) ?: [];
        if ($afterRow !== []) {
            $afterRow['password'] = $resetPassword !== '' ? '[changed]' : '[unchanged]';
        }
        log_admin_activity(
            'edit',
            'Super Administrator accounts',
            'Super Admin user #' . $id,
            $before ? (array) $before : null,
            $afterRow !== [] ? $afterRow : null
        );

        $_SESSION['flash'] = 'Super Administrator account updated.';
        header('Location: super_admin_accounts.php?tab=super_admins');
        exit;
    }

    if ($action === 'reset_password') {
        $id = (int) ($_POST['id'] ?? 0);
        $manualPassword = trim((string) ($_POST['new_password'] ?? ''));
        $confirmPassword = trim((string) ($_POST['confirm_password'] ?? ''));
        $generateAndEmail = !empty($_POST['generate_temp_password_email']);
        $emailReset = !empty($_POST['email_reset_password']);
        $forceChange = !empty($_POST['force_password_change']);

        if ($id < 1) {
            throw new RuntimeException('Invalid account.');
```

```

    } elseif ($forceChange || $generateAndEmail || $plainForMail !== '') {
        mark_new_account_requires_password_change($id);
    } else {
        clear_must_change_password($id);
    }

    $afterRow = $before;
    $afterRow['password'] = '[changed]';
    log_admin_activity(
        'edit',
        'Account monitor',
        'Password reset for user #' . $id . ' (' . (string) $before['username'] . ')',
        ['id' => $id, 'username' => $before['username'], 'role' => $before['role'], 'password' => '[unchanged]'],
        $afterRow
    );

    $roleKey = (string) ($before['role'] ?? 'admin');
    $displayName = (string) ($before['full_name'] ?? '');
    $uname = (string) ($before['username'] ?? '');

    if ($plainForMail !== '') {
        $mailOk = send_account_credentials_mail($email, $displayName, $uname, $plainForMail, $roleKey, $email);
        $_SESSION['flash'] = credentials_email_flash_message($mailOk, $email, 'Password updated for ' . $uname);
    } else {
        $_SESSION['flash'] = 'Password updated for ' . $uname . '.';
    }

    $q = http_build_query(array_filter([
        'tab' => 'monitor',
        'q' => trim((string) ($_POST['return_q'] ?? '')),
        'role' => trim((string) ($_POST['return_role'] ?? '')),
        'status' => trim((string) ($_POST['return_status'] ?? '')),
        'page' => (int) ($_POST['return_page'] ?? 1) > 1 ? (int) ($_POST['return_page'] ?? 1) : null,
    ], static fn ($v) => $v !== null && $v !== '');
    header('Location: super_admin_accounts.php' . ($q !== '' ? '?' . $q : ''));
    exit;
}

if ($action === 'delete') {
    $id = (int) ($_POST['id'] ?? 0);
    if (!sa_delete_account($id, $currentUserId, $roleLabels)) {
        throw new RuntimeException('Account not found.');
```

```

        $flash = $deleted === 1 ? 'Account deleted.' : $deleted . ' accounts deleted.';
        if ($skipped !== []) {
            $flash .= ' Some were skipped: ' . $skipped[0];
        }
        $_SESSION['flash'] = $flash;
        $q = http_build_query(array_filter([
            'tab' => 'monitor',
            'q' => trim((string) ($_POST['return_q'] ?? '')),
            'role' => trim((string) ($_POST['return_role'] ?? '')),
            'status' => trim((string) ($_POST['return_status'] ?? '')),
            'page' => (int) ($_POST['return_page'] ?? 1) > 1 ? (int) ($_POST['return_page'] ?? 1) : null,
        ], static fn ($v) => $v !== null && $v !== '');
        header('Location: super_admin_accounts.php' . ($q !== '' ? '?' . $q : ''));
        exit;
    }

    throw new RuntimeException('Unknown action.');
} catch (Throwable $e) {
    if (db()->inTransaction()) {
        db()->rollBack();
    }
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
    if ($action === 'reset_password') {
        $rid = (int) ($_POST['id'] ?? 0);
        header('Location: super_admin_accounts.php?tab=monitor' . ($rid > 0 ? '&reset=' . $rid : ''));
    } elseif ($action === 'edit') {
        header('Location: super_admin_accounts.php?tab=super_admins' . ($editId > 0 ? '&edit=' . $editId : ''));
    } elseif ($action === 'add') {
        $failRole = trim((string) ($_POST['role'] ?? ''));
        $q = 'tab=create';
        if ($failRole !== '' && isset($roleLabels[$failRole])) {
            $q .= '&role=' . rawurlencode($failRole);
        }
        header('Location: super_admin_accounts.php?' . $q);
    } elseif (in_array($action, ['delete', 'delete_bulk'], true)) {
        $q = http_build_query(array_filter([
            'tab' => 'monitor',
            'q' => trim((string) ($_POST['return_q'] ?? '')),
            'role' => trim((string) ($_POST['return_role'] ?? '')),
            'status' => trim((string) ($_POST['return_status'] ?? '')),
            'page' => (int) ($_POST['return_page'] ?? 1) > 1 ? (int) ($_POST['return_page'] ?? 1) : null,
        ], static fn ($v) => $v !== null && $v !== ''));
        header('Location: super_admin_accounts.php' . ($q !== '' ? '?' . $q : ''));
    } else {
        header('Location: super_admin_accounts.php');
    }
    exit;
}
}

// --- Monitor filters ---
$filterQ = trim((string) ($_GET['q'] ?? ''));
$filterRole = trim((string) ($_GET['role'] ?? ''));
$filterStatus = trim((string) ($_GET['status'] ?? ''));
$page = max(1, (int) ($_GET['page'] ?? 1));
$perPage = 40;

$where = ['1=1'];
$params = [];
if ($filterQ !== '') {
    $where[] = '(u.username LIKE ? OR u.full_name LIKE ?' . ($hasUserEmail ? ' OR u.email LIKE ?' : '') . ')';
    $like = '%' . $filterQ . '%';
    $params[] = $like;
    $params[] = $like;
    if ($hasUserEmail) {
        $params[] = $like;
    }
}
if ($filterRole !== '' && isset($roleLabels[$filterRole])) {
    $where[] = 'u.role = ?';
    $params[] = $filterRole;
}
if ($filterStatus === 'active') {
    $where[] = 'u.is_active = 1';
} elseif ($filterStatus === 'disabled') {
    $where[] = 'u.is_active = 0';
}

$whereSql = implode(' AND ', $where);

```

```

$countSt = db()->prepare("SELECT COUNT(*) FROM users u WHERE {$whereSql}");
$countSt->execute($params);
$totalAccounts = (int) $countSt->fetchColumn();
$totalPages = max(1, (int) ceil($totalAccounts / $perPage));
if ($page > $totalPages) {
    $page = $totalPages;
}
$offset = ($page - 1) * $perPage;

$listCols = 'u.id, u.username, u.full_name, u.role, u.is_active';
if ($hasUserEmail) {
    $listCols .= ', u.email';
}
if ($hasMustChange) {
    $listCols .= ', u.must_change_password';
}
if ($hasLastLogin) {
    $listCols .= ', u.last_login_at';
}
if ($hasCollegeId) {
    $listCols .= ', c.college_name';
}

$joinCollege = ($hasCollegeId && db_table_exists('colleges'))
    ? 'LEFT JOIN colleges c ON c.id = u.college_id'
    : '';
if ($joinCollege === '' && $hasCollegeId) {
    // college table missing - drop college_name from select
    $listCols = str_replace(', c.college_name', '', $listCols);
}

$listSql = "SELECT {$listCols} FROM users u {$joinCollege} WHERE {$whereSql} ORDER BY u.role ASC, u.username ASC LIMIT {$perPage} OFFSET
{$offset}";
$listSt = db()->prepare($listSql);
$listSt->execute($params);
$accounts = $listSt->fetchAll(PDO::FETCH_ASSOC) ?: [];

$roleCounts = db()->query(
    "SELECT role, COUNT(*) AS cnt FROM users GROUP BY role ORDER BY role"
)->fetchAll(PDO::FETCH_ASSOC) ?: [];

$listColsSa = 'id, username, full_name, is_active';
if ($hasUserEmail) {
    $listColsSa .= ', email';
}
$superAdmins = db()->query("SELECT {$listColsSa} FROM users WHERE role = 'super_admin' ORDER BY username")->fetchAll(PDO::FETCH_ASSOC) ?:
[];

$colleges = [];
if ($hasCollegesTable) {
    $colleges = db()->query('SELECT id, college_code, college_name FROM colleges ORDER BY college_code')->fetchAll(PDO::FETCH_ASSOC) ?: [];
}

/** @var array<int, list<string>> */
$programsByCollege = [];
if ($hasProgramsTable) {
    $progRows = db()->query(
        "SELECT college_id, program_name FROM programs WHERE status = 'active' ORDER BY program_name"
    )->fetchAll(PDO::FETCH_ASSOC) ?: [];
    foreach ($progRows as $pr) {
        $cid = (int) ($pr['college_id'] ?? 0);
        $pname = trim((string) ($pr['program_name'] ?? ''));
        if ($cid > 0 && $pname !== '') {
            $programsByCollege[$cid][] = $pname;
        }
    }
}

$monitorQueryBase = array_filter([
    'tab' => 'monitor',
    'q' => $filterQ !== '' ? $filterQ : null,
    'role' => $filterRole !== '' ? $filterRole : null,
    'status' => $filterStatus !== '' ? $filterStatus : null,
], static fn ($v) => $v !== null);

$pageTitle = 'Account monitor';
require_once __DIR__ . '/includes/header.php';
?>
<h1 class="h3 mb-2"><i class="fa-solid fa-users-gear me-2 text-primary"></i>Account monitor</h1>
<p class="text-muted mb-3">Review every login, create accounts for any role, and manage Super Administrator logins.</p>

```

```

<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<ul class="nav nav-tabs mb-4">
    <li class="nav-item">
        <a class="nav-link<?= $tab === 'monitor' ? ' active' : '' ?>" href="super_admin_accounts.php?tab=monitor">
            <i class="fa-solid fa-desktop me-1"></i>All accounts
        </a>
    </li>
    <li class="nav-item">
        <a class="nav-link<?= $tab === 'create' ? ' active' : '' ?>" href="super_admin_accounts.php?tab=create">
            <i class="fa-solid fa-user-plus me-1"></i>Create account
        </a>
    </li>
    <li class="nav-item">
        <a class="nav-link<?= $tab === 'super_admins' ? ' active' : '' ?>" href="super_admin_accounts.php?tab=super_admins">
            <i class="fa-solid fa-user-secret me-1"></i>Super Administrators
        </a>
    </li>
</ul>

<?php if ($tab === 'monitor'): ?>
    <div class="row g-3 mb-3">
        <?php foreach ($roleCounts as $rc): ?>
            <?php
                $r = (string) ($rc['role'] ?? '');
                $cnt = (int) ($rc['cnt'] ?? 0);
                $activeFilter = $filterRole === $r;
                $href = 'super_admin_accounts.php?' . http_build_query(array_filter([
                    'tab' => 'monitor',
                    'role' => $activeFilter ? null : $r,
                    'q' => $filterQ !== '' ? $filterQ : null,
                    'status' => $filterStatus !== '' ? $filterStatus : null,
                ], static fn ($v) => $v !== null));
            ?>
            <div class="col-6 col-md-4 col-xl-auto">
                <a href="<?= htmlspecialchars($href) ?>" class="text-decoration-none">
                    <div class="card shadow-sm h-100<?= $activeFilter ? ' border-primary' : '' ?>">
                        <div class="card-body py-2 px-3">
                            <div class="small text-muted"><?= htmlspecialchars(sa_role_label($r, $roleLabels)) ?></div>
                            <div class="fs-5 fw-semibold"><?= number_format($cnt) ?></div>
                        </div>
                    </div>
                </a>
            </div>
        <?php endforeach; ?>
    </div>

    <div class="card shadow-sm mb-3">
        <div class="card-body">
            <form method="get" class="row g-2 align-items-end">
                <input type="hidden" name="tab" value="monitor">
                <div class="col-md-4">
                    <label class="form-label">Search</label>
                    <input type="search" name="q" class="form-control" value="<?= htmlspecialchars($filterQ) ?>" placeholder="Username,
name<?= $hasUserEmail ? ', or email' : '' ?>" autocomplete="off">
                </div>
                <div class="col-md-3">
                    <label class="form-label">Role</label>
                    <select name="role" class="form-select">
                        <option value="">All roles</option>
                        <?php foreach ($roleLabels as $rk => $rl): ?>
                            <option value="<?= htmlspecialchars($rk) ?>"<?= $filterRole === $rk ? ' selected' : '' ?><?=
htmlspecialchars($rl) ?></option>
                        <?php endforeach; ?>
                    </select>
                </div>
                <div class="col-md-2">
                    <label class="form-label">Status</label>
                    <select name="status" class="form-select">
                        <option value="">All</option>
                        <option value="active"<?= $filterStatus === 'active' ? ' selected' : '' ?>>Active</option>
                        <option value="disabled"<?= $filterStatus === 'disabled' ? ' selected' : '' ?>>Disabled</option>
                    </select>
                </div>
                <div class="col-md-3 d-flex flex-wrap gap-2">
                    <button type="submit" class="btn btn-primary">Filter</button>
                    <a class="btn btn-outline-secondary" href="super_admin_accounts.php?tab=monitor">Clear</a>
                </div>
            </form>
        </div>
    </div>

```

```

</div>

<?php if ($resetRow): ?>
    <div class="card shadow-sm border-warning mb-4">
        <div class="card-header bg-white">
            <strong><i class="fa-solid fa-key me-1 text-warning"></i>Reset / change password</strong>
            - <? = htmlspecialchars((string) $resetRow['username']) ?>
            <span class="badge bg-secondary ms-1"><? = htmlspecialchars(sa_role_label((string) $resetRow['role'], $roleLabels)) ?></span>
        </div>
        <div class="card-body">
            <p class="text-muted small mb-3">
                <? = htmlspecialchars((string) $resetRow['full_name']) ?>
                <?php if ($hasUserEmail && trim((string) ($resetRow['email'] ?? '')) !== ''): ?>
                    . <? = htmlspecialchars((string) $resetRow['email']) ?>
                <?php endif; ?>
            </p>
            <form method="post" class="row g-3">
                <input type="hidden" name="action" value="reset_password">
                <input type="hidden" name="id" value="<? = (int) $resetRow['id'] ?>">
                <input type="hidden" name="return_q" value="<? = htmlspecialchars($filterQ) ?>">
                <input type="hidden" name="return_role" value="<? = htmlspecialchars($filterRole) ?>">
                <input type="hidden" name="return_status" value="<? = htmlspecialchars($filterStatus) ?>">
                <input type="hidden" name="return_page" value="<? = (int) $page ?>">

                <div class="col-md-4">
                    <label class="form-label">New password</label>
                    <input type="password" name="new_password" class="form-control" minlength="8" autocomplete="new-password"
id="saNewPassword">

                    <div class="form-text">At least 8 characters (unless generating a temporary password below).</div>
                </div>
                <div class="col-md-4">
                    <label class="form-label">Confirm password</label>
                    <input type="password" name="confirm_password" class="form-control" minlength="8" autocomplete="new-password"
id="saConfirmPassword">
                </div>
                <div class="col-md-4">
                    <label class="form-label d-block">&nbsp;</label>
                    <div class="form-check mb-2">
                        <input class="form-check-input" type="checkbox" name="force_password_change" value="1" id="saForceChange"
checked>

                        <label class="form-check-label" for="saForceChange">Require password change on next login</label>
                    </div>
                    <?php if ($hasUserEmail): ?>
                        <div class="form-check mb-2">
                            <input class="form-check-input" type="checkbox" name="email_reset_password" value="1" id="saEmailReset">
                            <label class="form-check-label" for="saEmailReset">Email this new password</label>
                        </div>
                        <div class="form-check">
                            <input class="form-check-input" type="checkbox" name="generate_temp_password_email" value="1"
id="saGenerateTemp">

                            <label class="form-check-label" for="saGenerateTemp">Generate temporary password and email it</label>
                        </div>
                    <?php endif; ?>
                </div>
            </form>
            <div class="col-12 d-flex flex-wrap gap-2">
                <button type="submit" class="btn btn-warning">
                    <i class="fa-solid fa-key me-1"></i>Update password
                </button>
                <a class="btn btn-outline-secondary" href="super_admin_accounts.php?<? =
htmlspecialchars(http_build_query($monitorQueryBase)) ?>">Cancel</a>
            </div>
        </div>
    </div>
</div>
<script>
    (function () {
        var gen = document.getElementById('saGenerateTemp');
        var pw = document.getElementById('saNewPassword');
        var cf = document.getElementById('saConfirmPassword');
        if (!gen || !pw || !cf) return;
        function sync() {
            var on = gen.checked;
            pw.required = !on;
            cf.required = !on;
            pw.disabled = on;
            cf.disabled = on;
            if (on) { pw.value = ''; cf.value = ''; }
        }
        gen.addEventListener('change', sync);
        sync();
    })();

```

```

    });
</script>
<?php endif; ?>

<div class="card shadow-sm">
    <div class="card-header bg-white d-flex flex-wrap justify-content-between align-items-center gap-2">
        <strong>All accounts</strong>
        <div class="d-flex flex-wrap align-items-center gap-2">
            <?php if ($accounts): ?>
                <form id="sa-bulk-delete-form" method="post" class="d-inline" onsubmit="return confirm('Delete the selected accounts? This cannot be undone.');">
                    <input type="hidden" name="action" value="delete_bulk">
                    <input type="hidden" name="return_q" value="<? = htmlspecialchars($filterQ) ?>">
                    <input type="hidden" name="return_role" value="<? = htmlspecialchars($filterRole) ?>">
                    <input type="hidden" name="return_status" value="<? = htmlspecialchars($filterStatus) ?>">
                    <input type="hidden" name="return_page" value="<? = (int) $page ?>">
                    <button type="submit" id="sa-bulk-delete-btn" class="btn btn-sm btn-outline-danger" disabled>
                        <i class="fa-solid fa-trash me-1"></i>Delete selected
                    </button>
                </form>
            <?php endif; ?>
            <span class="text-muted small"><? = number_format($totalAccounts) ?> account<? = $totalAccounts === 1 ? '' : 's' ?></span>
        </div>
    </div>
    <div class="card-body p-0">
        <?php if (!$accounts): ?>
            <p class="text-muted p-3 mb-0">No accounts match the current filters.</p>
        <?php else: ?>
            <div class="table-responsive">
                <table class="table table-sm table-striped mb-0 align-middle">
                    <thead class="table-light">
                        <tr>
                            <th scope="col" style="width:2.25rem">
                                <input type="checkbox" class="form-check-input" id="sa-select-all" aria-label="Select all deletable
                                accounts on this page">
                            </th>
                            <th>Username</th>
                            <th>Full name</th>
                            <th>Role</th>
                            <?php if ($hasUserEmail): ?><th>Email</th><?php endif; ?>
                            <?php if ($hasCollegeId && $joinCollege !== ''): ?><th>College</th><?php endif; ?>
                            <th>Status</th>
                            <?php if ($hasMustChange): ?><th>Password</th><?php endif; ?>
                            <?php if ($hasLastLogin): ?><th>Last login</th><?php endif; ?>
                            <th></th>
                        </tr>
                    </thead>
                    <tbody>
                        <?php foreach ($accounts as $acc): ?>
                            <?php
                                $accId = (int) $acc['id'];
                                $isSelf = $accId === $currentUserId;
                                $resetHref = 'super_admin_accounts.php?' . http_build_query(array_merge($monitorQueryBase, [
                                    'reset' => $accId,
                                    'page' => $page > 1 ? $page : null,
                                ]));
                            <?>
                            <tr<? = $resetId === $accId ? ' class="table-warning"' : '' ?>>
                                <td>
                                    <?php if (!$isSelf): ?>
                                        <input type="checkbox" class="form-check-input sa-row-select" name="ids[]" form="sa-bulk-delete-
                                        form" value="<? = $accId ?>" aria-label="Select <? = htmlspecialchars((string) $acc['username']) ?>">
                                    <?php endif; ?>
                                </td>
                                <td>
                                    <? = htmlspecialchars((string) $acc['username']) ?>
                                    <?php if ($isSelf): ?>
                                        <span class="badge bg-primary-subtle text-primary-emphasis border ms-1">You</span>
                                    <?php endif; ?>
                                </td>
                                <td><? = htmlspecialchars((string) $acc['full_name']) ?></td>
                                <td><span class="badge text-bg-light border"><? = htmlspecialchars(sa_role_label((string) $acc['role'],
                                $roleLabels)) ?></span></td>
                                <?php if ($hasUserEmail): ?>
                                    <td class="small"><? = htmlspecialchars((string) ($acc['email'] ?? '')) ?></td>
                                <?php endif; ?>
                                <?php if ($hasCollegeId && $joinCollege !== ''): ?>
                                    <td class="small"><? = htmlspecialchars((string) ($acc['college_name'] ?? '-')) ?></td>
                                <?php endif; ?>
                                <td>

```

```

                <?php if ((int) $acc['is_active'] === 1): ?>
                    <span class="badge bg-success">Active</span>
                <?php else: ?>
                    <span class="badge bg-secondary">Disabled</span>
                <?php endif; ?>
            </td>
            <?php if ($hasMustChange): ?>
                <td>
                    <?php if ((int) ($acc['must_change_password'] ?? 0) === 1): ?>
                        <span class="badge bg-warning text-dark">Must change</span>
                    <?php else: ?>
                        <span class="text-muted small">OK</span>
                    <?php endif; ?>
                </td>
            <?php endif; ?>
            <?php if ($hasLastLogin): ?>
                <td class="small text-muted">
                    <?php
                        $l1 = (string) ($acc['last_login_at'] ?? '');
                        echo $l1 !== '' ? htmlspecialchars(date('M j, Y g:i A', strtotime($l1))) : '-';
                    <?php
                </td>
            <?php endif; ?>
            <td class="text-end text-nowrap">
                <a class="btn btn-sm btn-outline-warning" href="<?== htmlspecialchars($resetHref) ?&gt;"&gt;
                    &lt;i class="fa-solid fa-key me-1"&gt;&lt;/i&gt;Password
                &lt;/a&gt;
                &lt;?php if (!$isSelf): ?&gt;
                    &lt;form method="post" class="d-inline" onsubmit="return confirm('Delete this account? This cannot be
undone.');"&gt;
                        &lt;input type="hidden" name="action" value="delete"&gt;
                        &lt;input type="hidden" name="id" value="&lt;?=<?= $accId ?&gt;"&gt;
                        &lt;input type="hidden" name="return_q" value="&lt;?=<?= htmlspecialchars($filterQ) ?&gt;"&gt;
                        &lt;input type="hidden" name="return_role" value="&lt;?=<?= htmlspecialchars($filterRole) ?&gt;"&gt;
                        &lt;input type="hidden" name="return_status" value="&lt;?=<?= htmlspecialchars($filterStatus) ?&gt;"&gt;
                        &lt;input type="hidden" name="return_page" value="&lt;?=<?= (int) $page ?&gt;"&gt;
                        &lt;button type="submit" class="btn btn-sm btn-outline-danger" title="Delete"&gt;
                            &lt;i class="fa-solid fa-trash"&gt;&lt;/i&gt;
                        &lt;/button&gt;
                    &lt;/form&gt;
                &lt;?php endif; ?&gt;
            &lt;/td&gt;
        &lt;/tr&gt;
    &lt;?php endforeach; ?&gt;
&lt;/tbody&gt;
&lt;/table&gt;
&lt;/div&gt;
&lt;?php endif; ?&gt;
&lt;/div&gt;
&lt;?php if ($totalPages &gt; 1): ?&gt;
    &lt;div class="card-footer bg-white d-flex flex-wrap justify-content-between align-items-center gap-2"&gt;
        &lt;span class="text-muted small"&gt;Page &lt;?=<?= (int) $page ?&gt; of &lt;?=<?= (int) $totalPages ?&gt;&lt;/span&gt;
        &lt;div class="btn-group"&gt;
            &lt;?php if ($page &gt; 1): ?&gt;
                &lt;a class="btn btn-sm btn-outline-secondary" href="super_admin_accounts.php?&lt;?=<?=
htmlspecialchars(http_build_query(array_merge($monitorQueryBase, ['page' =&gt; $page - 1]))) ?&gt;"&gt;Previous&lt;/a&gt;
            &lt;?php endif; ?&gt;
            &lt;?php if ($page &lt; $totalPages): ?&gt;
                &lt;a class="btn btn-sm btn-outline-secondary" href="super_admin_accounts.php?&lt;?=<?=
htmlspecialchars(http_build_query(array_merge($monitorQueryBase, ['page' =&gt; $page + 1]))) ?&gt;"&gt;Next&lt;/a&gt;
            &lt;?php endif; ?&gt;
        &lt;/div&gt;
    &lt;/div&gt;
&lt;/div&gt;
&lt;?php endif; ?&gt;
&lt;/div&gt;
&lt;?php if ($accounts): ?&gt;
    &lt;script&gt;
        (function () {
            var selectAll = document.getElementById('sa-select-all');
            var deleteBtn = document.getElementById('sa-bulk-delete-btn');
            if (!selectAll || !deleteBtn) return;

            var rowChecks = function () {
                return Array.prototype.slice.call(document.querySelectorAll('.sa-row-select'));
            };

            var syncBulkDelete = function () {
                var checks = rowChecks();
                var selected = checks.filter(function (el) { return el.checked; });
                deleteBtn.disabled = selected.length === 0;
            };
        })();
    &lt;/script&gt;
</pre

```



```

        selectAll.indeterminate = selected.length > 0 && selected.length < checks.length;
        selectAll.checked = checks.length > 0 && selected.length === checks.length;
    });

    selectAll.addEventListener('change', function () {
        var checked = selectAll.checked;
        rowChecks().forEach(function (el) { el.checked = checked; });
        syncBulkDelete();
    });

    rowChecks().forEach(function (el) {
        el.addEventListener('change', syncBulkDelete);
    });

    syncBulkDelete();
    })());
</script>
<?php endif; ?>

<?php elseif ($tab === 'create'): ?>
    <?php
    $createRole = $preselectRole !== '' ? $preselectRole : 'admin';
    ?>
    <p class="text-muted">Create a login for any role. College and program fields appear when the selected role needs them.</p>

    <div class="row g-4">
        <div class="col-lg-7 col-xl-6">
            <div class="card shadow-sm">
                <div class="card-header bg-white"><strong>New account</strong></div>
                <div class="card-body">
                    <form method="post" class="row g-3" id="saCreateAccountForm">
                        <input type="hidden" name="action" value="add">
                        <div class="col-12">
                            <label class="form-label">Role</label>
                            <select name="role" id="saCreateRole" class="form-select" required>
                                <?php foreach ($roleLabels as $rk => $rl): ?>
                                    <option value="<?=" htmlspecialchars($rk) ?">"<?=" $createRole === $rk ? ' selected' : '' ?">"<?="
htmlspecialchars($rl) ?"></option>
                                <?php endforeach; ?>
                            </select>
                        </div>
                        <div class="col-md-6">
                            <label class="form-label">Username</label>
                            <input type="text" name="username" class="form-control" required maxlength="50" autocomplete="username">
                        </div>
                        <div class="col-md-6">
                            <label class="form-label">Full name</label>
                            <input type="text" name="full_name" class="form-control" required maxlength="100" autocomplete="name">
                        </div>
                        <?php if ($hasUserEmail): ?>
                            <div class="col-12">
                                <label class="form-label">Email <span class="text-muted">(optional)</span></label>
                                <input type="email" name="email" class="form-control" maxlength="190" autocomplete="email">
                                <div class="form-text">If provided, a temporary password can be emailed automatically when the password
field is left blank.</div>
                            </div>
                        <?php endif; ?>
                        <div class="col-12">
                            <label class="form-label">Password</label>
                            <input type="password" name="password" class="form-control" minlength="8" autocomplete="new-password"
id="saCreatePassword">
                            <div class="form-text">Required when no email is provided (min. 8 characters).</div>
                        </div>

                        <div class="col-12 sa-role-field" data-roles="admin"<?=" $hasAdminLogTitle ? '' : ' hidden' ?">>
                            <label class="form-label">Activity log title <span class="text-muted">(optional)</span></label>
                            <input type="text" name="admin_log_title" class="form-control" maxlength="120" placeholder="e.g. Scheduling
Office">
                        </div>

                        <div class="col-12 sa-role-field" data-roles="dean program_chair faculty student">
                            <label class="form-label">College</label>
                            <select name="college_id" id="saCreateCollege" class="form-select">
                                <option value="">Select college...</option>
                                <?php foreach ($colleges as $c): ?>
                                    <option value="<?=" (int) $c['id'] ?">">
                                        <?=" htmlspecialchars((string) $c['college_code'] . ' - ' . (string) $c['college_name']) ?">
                                    </option>
                                <?php endforeach; ?>
                            </select>

```

```

        <?php if ($colleges === []): ?>
            <div class="form-text text-warning">No colleges found. Add colleges first.</div>
        <?php endif; ?>
    </div>

    <div class="col-12 sa-role-field" data-roles="program_chair faculty student">
        <label class="form-label">Program</label>
        <select name="assigned_program" id="saCreateProgram" class="form-select">
            <option value="">Select college first...</option>
        </select>
    </div>

    <div class="col-md-6 sa-role-field" data-roles="faculty">
        <label class="form-label">Faculty ID</label>
        <input type="text" name="faculty_id" class="form-control" maxlength="50" placeholder="Employee / faculty code">
    </div>

    <div class="col-md-6 sa-role-field" data-roles="student">
        <label class="form-label">Student number</label>
        <input type="text" name="student_number" class="form-control" maxlength="50">
    </div>

    <div class="col-md-6 sa-role-field" data-roles="student">
        <label class="form-label">Year level <span class="text-muted">(optional)</span></label>
        <input type="text" name="year_level" class="form-control" maxlength="40" placeholder="e.g. 1st Year">
    </div>

    <div class="col-12">
        <div class="form-check">
            <input class="form-check-input" type="checkbox" name="is_active" value="1" id="saCreateActive" checked>
            <label class="form-check-label" for="saCreateActive">Account active</label>
        </div>
    </div>
    <div class="col-12">
        <button type="submit" class="btn btn-primary">
            <i class="fa-solid fa-user-plus me-1"></i>Create account
        </button>
    </div>
</form>
</div>
</div>
</div>
<div class="col-lg-5">
    <div class="card shadow-sm border-0 bg-light">
        <div class="card-body">
            <h2 class="h6">Role notes</h2>
            <ul class="small text-muted mb-0 ps-3">
                <li><strong>Super Administrator</strong> – institution-wide oversight</li>
                <li><strong>Administrator</strong> – day-to-day scheduling admin</li>
                <li><strong>Dean</strong> – requires a college; linked as that college's dean</li>
                <li><strong>Program Chair</strong> – college + program</li>
                <li><strong>Faculty</strong> – college, program, and Faculty ID (creates faculty profile)</li>
                <li><strong>General Education</strong> – institution-wide GE coordinator</li>
                <li><strong>Student</strong> – college, program, and student number</li>
            </ul>
            <p class="small text-muted mt-3 mb-0">
                Prefer dedicated pages for bulk work:
                <a href="super_admin_admins.php">Administrators</a>
                • college deans / chairs are usually managed by Admin or Dean roles.
            </p>
        </div>
    </div>
</div>
</div>
</div>
<script>
    (function () {
        var programsByCollege = <?= json_encode($programsByCollege, JSON_UNESCAPED_UNICODE) ?>;
        var roleEl = document.getElementById('saCreateRole');
        var collegeEl = document.getElementById('saCreateCollege');
        var programEl = document.getElementById('saCreateProgram');
        if (!roleEl) return;

        function syncRoleFields() {
            var role = roleEl.value;
            document.querySelectorAll('.sa-role-field').forEach(function (el) {
                var roles = (el.getAttribute('data-roles') || '').split(/\s+/);
                var show = roles.indexOf(role) !== -1;
                el.classList.toggle('d-none', !show);
                el.querySelectorAll('input, select').forEach(function (inp) {
                    if (inp.name === 'admin_log_title') return;
                    if (show && (inp.name === 'college_id' || inp.name === 'assigned_program' || inp.name === 'faculty_id' || inp.name

```

```

=== 'student_number')) {
    inp.required = true;
} else if (!show) {
    inp.required = false;
}
});
});
}

function fillPrograms() {
    if (!programEl || !collegeEl) return;
    var cid = collegeEl.value;
    var list = programsByCollege[cid] || programsByCollege[String(cid)] || [];
    var prev = programEl.value;
    programEl.innerHTML = '';
    var opt0 = document.createElement('option');
    opt0.value = '';
    opt0.textContent = list.length ? 'Select program...' : (cid ? 'No active programs for this college' : 'Select college
first...');
    programEl.appendChild(opt0);
    list.forEach(function (name) {
        var opt = document.createElement('option');
        opt.value = name;
        opt.textContent = name;
        if (name === prev) opt.selected = true;
        programEl.appendChild(opt);
    });
}

roleEl.addEventListener('change', syncRoleFields);
if (collegeEl) collegeEl.addEventListener('change', fillPrograms);
syncRoleFields();
fillPrograms();
})();
</script>

<?php else: ?>
<p class="text-muted">Update existing <strong>super_admin</strong> logins. To create a new Super Administrator (or any other role), use
the <a href="super_admin_accounts.php?tab=create&role=super_admin">Create account</a> tab.</p>

<div class="row g-4">
    <div class="col-lg-5">
        <div class="card shadow-sm">
            <div class="card-header bg-white"><strong><?= $editRow ? 'Edit Super Administrator' : 'Super Administrator' ?></strong>
</div>
            <div class="card-body">
                <?php if ($editRow): ?>
                    <form method="post" class="row g-3">
                        <input type="hidden" name="action" value="edit">
                        <input type="hidden" name="id" value="<?= (int) $editRow['id'] ?>">
                        <div class="col-12">
                            <label class="form-label">Username</label>
                            <input type="text" class="form-control" value="<?= htmlspecialchars((string) $editRow['username']) ?>"
disabled autocomplete="username">
                            <div class="form-text">Username cannot be changed here.</div>
                        </div>
                        <div class="col-12">
                            <label class="form-label">Full name</label>
                            <input type="text" name="full_name" class="form-control" required value="<?= htmlspecialchars((string)
$editRow['full_name']) ?>" autocomplete="name">
                        </div>
                        <?php if ($hasUserEmail): ?>
                            <div class="col-12">
                                <label class="form-label">Email</label>
                                <input type="email" name="email" class="form-control" value="<?= htmlspecialchars((string)
($editRow['email'] ?? '')) ?>" autocomplete="email">
                            </div>
                        <?php endif; ?>
                        <div class="col-12">
                            <label class="form-label">New password</label>
                            <input type="password" name="reset_password" class="form-control" minlength="8" autocomplete="new-password">
                            <div class="form-text">Leave blank to keep the current password.</div>
                        </div>
                        <div class="col-12">
                            <div class="form-check">
                                <input class="form-check-input" type="checkbox" name="is_active" value="1" id="is_active_edit" <?= (int)
$editRow['is_active'] === 1 ? 'checked' : '' ?><?= super_admin_active_count() <= 1 && (int) $editRow['is_active'] === 1 ? ' disabled' : '' ?
>>
                                <label class="form-check-label" for="is_active_edit">Account active</label>
                                <?php if (super_admin_active_count() <= 1 && (int) $editRow['is_active'] === 1): ?>

```

```

        <input type="hidden" name="is_active" value="1">
        <div class="form-text">The last active Super Administrator cannot be disabled.</div>
    <?php endif; ?>
</div>
</div>
<div class="col-12 d-flex flex-wrap gap-2">
    <button type="submit" class="btn btn-primary">Save changes</button>
    <a class="btn btn-outline-secondary" href="super_admin_accounts.php?tab=super_admins">Cancel</a>
</div>
</form>
<?php else: ?>
    <p class="text-muted mb-3">Select <strong>Edit</strong> on an account in the list, or create a new Super
Administrator from the Create account tab.</p>
    <a class="btn btn-primary" href="super_admin_accounts.php?tab=create&role=super_admin">
        <i class="fa-solid fa-user-plus me-1"></i>Create Super Administrator
    </a>
    <?php endif; ?>
</div>
</div>
<div class="col-lg-7">
    <div class="card shadow-sm">
        <div class="card-header bg-white"><strong>Existing Super Administrators</strong></div>
        <div class="card-body p-0">
            <?php if (!$superAdmins): ?>
                <p class="text-muted p-3 mb-0">No Super Administrator accounts found. Run <a
href="upgrade_roles.php">upgrade_roles.php</a> or <a href="super_admin_accounts.php?tab=create&role=super_admin">create one</a>.</p>
            <?php else: ?>
                <div class="table-responsive">
                    <table class="table table-sm table-striped mb-0 align-middle">
                        <thead class="table-light">
                            <tr>
                                <th>Username</th>
                                <th>Full name</th>
                                <th>Status</th>
                                <th></th>
                            </tr>
                        </thead>
                        <tbody>
                            <?php foreach ($superAdmins as $sa): ?>
                                <tr>
                                    <td>
                                        <?= htmlspecialchars((string) $sa['username']) ?>
                                        <?php if ((int) $sa['id'] === $currentUserId): ?>
                                            <span class="badge bg-primary-subtle text-primary-emphasis border ms-1">You</span>
                                        <?php endif; ?>
                                    </td>
                                    <td><?= htmlspecialchars((string) $sa['full_name']) ?></td>
                                    <td>
                                        <?php if ((int) $sa['is_active'] === 1): ?>
                                            <span class="badge bg-success">Active</span>
                                        <?php else: ?>
                                            <span class="badge bg-secondary">Disabled</span>
                                        <?php endif; ?>
                                    </td>
                                    <td class="text-end">
                                        <a class="btn btn-sm btn-outline-primary" href="super_admin_accounts.php?tab=super_admins&edit=
                                        <?= (int) $sa['id'] ?>">Edit</a>
                                        <a class="btn btn-sm btn-outline-warning" href="super_admin_accounts.php?tab=monitor&reset=<?=
                                        (int) $sa['id'] ?>">Password</a>
                                    </td>
                                </tr>
                            <?php endforeach; ?>
                        </tbody>
                    </table>
                </div>
            <?php endif; ?>
        </div>
    </div>
</div>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php';

```

```

-----
FILE: super_admin_admins.php
-----

```

```

<?php

```

```

declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';
require_once __DIR__ . '/includes/account_registration_helpers.php';

require_role(['super_admin']);

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

$hasUserEmail = db_column_exists('users', 'email');
$hasAdminLogTitle = db_column_exists('users', 'admin_log_title');

$editId = isset($_GET['edit']) ? (int) $_GET['edit'] : 0;
$editRow = null;
if ($editId > 0) {
    $selCols = 'id, username, full_name, is_active';
    if ($hasUserEmail) {
        $selCols .= ', email';
    }
    if ($hasAdminLogTitle) {
        $selCols .= ', admin_log_title';
    }
    $st = db()->prepare("SELECT {$selCols} FROM users WHERE id = ? AND role = 'admin' LIMIT 1");
    $st->execute([$editId]);
    $editRow = $st->fetch(PDO::FETCH_ASSOC) ?: null;
    if (!$editRow) {
        $editId = 0;
    }
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if ($action === 'add') {
            $username = trim((string) ($_POST['username'] ?? ''));
            $fullName = trim((string) ($_POST['full_name'] ?? ''));
            $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
            $adminLogTitle = $hasAdminLogTitle ? trim((string) ($_POST['admin_log_title'] ?? '')) : '';
            $password = (string) ($_POST['password'] ?? '');
            $hasValidEmail = $hasUserEmail && $email !== '' && filter_var($email, FILTER_VALIDATE_EMAIL);
            $isActive = !empty($_POST['is_active']) ? 1 : 0;

            if ($username === '' || $fullName === '') {
                throw new RuntimeException('Username and full name are required.');
            }
            if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
                throw new RuntimeException('Invalid email address.');
            }

            $cred = prepare_new_account_password($password, $hasValidEmail);
            $plainForMail = $hasValidEmail ? $cred['plain'] : '';

            $exists = db()->prepare('SELECT COUNT(*) FROM users WHERE username = ?');
            $exists->execute([$username]);
            if ((int) $exists->fetchColumn() > 0) {
                $s = suggest_available_usernames($username, 3);
                $msg = 'Username already exists.';
                if ($s) {
                    $msg .= ' Try: ' . implode(', ', $s);
                }
                throw new RuntimeException($msg);
            }

            $fields = ['username', 'password', 'full_name'];
            $params = [
                $username,
                $cred['hash'],
                $fullName,
            ];
            if ($hasUserEmail) {
                $fields[] = 'email';
                $params[] = $email;
            }
            if ($hasAdminLogTitle) {
                $fields[] = 'admin_log_title';
                $params[] = $adminLogTitle;
            }
        }
    }
}

```

```

$fields[] = 'role';
$params[] = 'admin';
$fields[] = 'is_active';
$params[] = $isActive;

$ph = implode(' ', array_fill(0, count($fields), '?'));
$sql = 'INSERT INTO users (' . implode(' ', $fields) . ') VALUES (' . $ph . ')';
db()->prepare($sql)->execute($params);
$newId = (int) db()->lastInsertId();

$selAfter = 'id, username, full_name, role, is_active';
if ($hasUserEmail) {
    $selAfter .= ', email';
}
if ($hasAdminLogTitle) {
    $selAfter .= ', admin_log_title';
}
$stmtAfter = db()->prepare("SELECT {$selAfter} FROM users WHERE id = ? LIMIT 1");
$stmtAfter->execute([$newId]);
$afterRow = $stmtAfter->fetch(PDO::FETCH_ASSOC) ?: [];
if ($afterRow !== []) {
    $afterRow['password'] = '[set at creation]';
}
log_admin_activity('add', 'Administrator accounts', 'Admin user #' . $newId, null, $afterRow !== [] ? $afterRow : null);

$mailOk = $hasValidEmail
    ? notify_new_account_credentials($newId, $email, $fullName, $username, $plainForMail, 'admin')
    : null;
if (!$hasValidEmail) {
    mark_new_account_requires_password_change($newId);
}
$_SESSION['flash'] = registration_email_flash_message($mailOk, $email, 'Administrator account');
header('Location: super_admin_admins.php');
exit;
}

```

```

if ($action === 'edit') {
    $id = (int) ($_POST['id'] ?? 0);
    $fullName = trim((string) ($_POST['full_name'] ?? ''));
    $email = $hasUserEmail ? trim((string) ($_POST['email'] ?? '')) : '';
    $adminLogTitle = $hasAdminLogTitle ? trim((string) ($_POST['admin_log_title'] ?? '')) : '';
    $resetPassword = (string) ($_POST['reset_password'] ?? '');
    $isActive = !empty($_POST['is_active']) ? 1 : 0;

    if ($id < 1) {
        throw new RuntimeException('Invalid account.');
```

```

    }
    if ($fullName === '') {
        throw new RuntimeException('Full name is required.');
```

```

    }
    if ($hasUserEmail && $email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        throw new RuntimeException('Invalid email address.');
```

```

    }
    if ($resetPassword !== '' && strlen($resetPassword) < 8) {
        throw new RuntimeException('New password must be at least 8 characters.');
```

```

    }

    $selCols = 'id, username, full_name, is_active';
    if ($hasUserEmail) {
        $selCols .= ', email';
    }
    if ($hasAdminLogTitle) {
        $selCols .= ', admin_log_title';
    }
    $st = db()->prepare("SELECT {$selCols} FROM users WHERE id = ? AND role = 'admin' LIMIT 1");
    $st->execute([$id]);
    $before = $st->fetch(PDO::FETCH_ASSOC);
    if (!$before) {
        throw new RuntimeException('Administrator not found.');
```

```

    }

    if ($resetPassword !== '') {
        if ($hasUserEmail && $hasAdminLogTitle) {
            db()->prepare(
                'UPDATE users SET full_name = ?, email = ?, admin_log_title = ?, password = ?, is_active = ? WHERE id = ? AND role =
                '
            )->execute([
                $fullName,
                $email,
                $adminLogTitle,

```

```

        password_hash($resetPassword, PASSWORD_DEFAULT),
        $isActive,
        $id,
        'admin',
    ]);
} elseif ($hasUserEmail) {
    db()->prepare(
        'UPDATE users SET full_name = ?, email = ?, password = ?, is_active = ? WHERE id = ? AND role = ?'
    )->execute([
        $fullName,
        $email,
        password_hash($resetPassword, PASSWORD_DEFAULT),
        $isActive,
        $id,
        'admin',
    ]);
} elseif ($hasAdminLogTitle) {
    db()->prepare(
        'UPDATE users SET full_name = ?, admin_log_title = ?, password = ?, is_active = ? WHERE id = ? AND role = ?'
    )->execute([
        $fullName,
        $adminLogTitle,
        password_hash($resetPassword, PASSWORD_DEFAULT),
        $isActive,
        $id,
        'admin',
    ]);
} else {
    db()->prepare(
        'UPDATE users SET full_name = ?, password = ?, is_active = ? WHERE id = ? AND role = ?'
    )->execute([
        $fullName,
        password_hash($resetPassword, PASSWORD_DEFAULT),
        $isActive,
        $id,
        'admin',
    ]);
}
} else {
    if ($hasUserEmail && $hasAdminLogTitle) {
        db()->prepare(
            'UPDATE users SET full_name = ?, email = ?, admin_log_title = ?, is_active = ? WHERE id = ? AND role = ?'
        )->execute([$fullName, $email, $adminLogTitle, $isActive, $id, 'admin']);
    } elseif ($hasUserEmail) {
        db()->prepare(
            'UPDATE users SET full_name = ?, email = ?, is_active = ? WHERE id = ? AND role = ?'
        )->execute([$fullName, $email, $isActive, $id, 'admin']);
    } elseif ($hasAdminLogTitle) {
        db()->prepare(
            'UPDATE users SET full_name = ?, admin_log_title = ?, is_active = ? WHERE id = ? AND role = ?'
        )->execute([$fullName, $adminLogTitle, $isActive, $id, 'admin']);
    } else {
        db()->prepare(
            'UPDATE users SET full_name = ?, is_active = ? WHERE id = ? AND role = ?'
        )->execute([$fullName, $isActive, $id, 'admin']);
    }
}

$selAfter = 'id, username, full_name, role, is_active';
if ($hasUserEmail) {
    $selAfter .= ', email';
}
if ($hasAdminLogTitle) {
    $selAfter .= ', admin_log_title';
}
}
$stmtAfter = db()->prepare("SELECT {$selAfter} FROM users WHERE id = ? LIMIT 1");
$stmtAfter->execute([$id]);
$afterRow = $stmtAfter->fetch(PDO::FETCH_ASSOC) ?: [];
if ($afterRow !== []) {
    $afterRow['password'] = $resetPassword !== '' ? '[changed]' : '[unchanged]';
}
log_admin_activity(
    'edit',
    'Administrator accounts',
    'Admin user #' . $id,
    $before ? (array) $before : null,
    $afterRow !== [] ? $afterRow : null
);

$_SESSION['flash'] = 'Administrator account updated.';

```

```

        header('Location: super_admin_admins.php');
        exit;
    }

    throw new RuntimeException('Unknown action.');
```

```

} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
    header('Location: super_admin_admins.php' . ($editId > 0 ? '?edit=' . $editId : ''));
    exit;
}
}

$listCols = 'id, username, full_name, is_active';
if ($hasUserEmail) {
    $listCols .= ', email';
}
if ($hasAdminLogTitle) {
    $listCols .= ', admin_log_title';
}
$admins = db()->query("SELECT {$listCols} FROM users WHERE role = 'admin' ORDER BY username")->fetchAll(PDO::FETCH_ASSOC) ?: [];

$pageTitle = 'Administrator accounts';
require_once __DIR__ . '/includes/header.php';
?>
<h1 class="h3 mb-4"><i class="fa-solid fa-users-gear me-2 text-primary"></i>Administrator accounts</h1>
<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<p class="text-muted">Create or update <strong>admin</strong> logins for day-to-day scheduling. Use <strong>Log title / office</strong> so
activity in Settings shows which administrator performed each action (shown with full name and username).</p>

<div class="row g-4">
    <div class="col-lg-5">
        <div class="card shadow-sm">
            <div class="card-header bg-white"><strong><?=$editRow ? 'Edit administrator' : 'Add administrator' ?></strong></div>
            <div class="card-body">
                <?php if ($editRow): ?>
                    <form method="post" class="row g-3">
                        <input type="hidden" name="action" value="edit">
                        <input type="hidden" name="id" value="<?=(int) $editRow['id'] ?>">
                        <div class="col-12">
                            <label class="form-label">Username</label>
                            <input type="text" class="form-control" value="<?= htmlspecialchars((string) $editRow['username']) ?>" disabled
autocomplete="username">
                            <div class="form-text">Username cannot be changed here.</div>
                        </div>
                        <div class="col-12">
                            <label class="form-label">Full name</label>
                            <input type="text" name="full_name" class="form-control" required value="<?= htmlspecialchars((string)
$editRow['full_name']) ?>" autocomplete="name">
                        </div>
                        <?php if ($hasUserEmail): ?>
                            <div class="col-12">
                                <label class="form-label">Email</label>
                                <input type="email" name="email" class="form-control" value="<?= htmlspecialchars((string)
($editRow['email'] ?? '')) ?>" autocomplete="email">
                            </div>
                        <?php endif; ?>
                        <?php if ($hasAdminLogTitle): ?>
                            <div class="col-12">
                                <label class="form-label">Log title / office</label>
                                <input type="text" name="admin_log_title" class="form-control" maxlength="120" value="<?=
htmlspecialchars((string) ($editRow['admin_log_title'] ?? '')) ?>" placeholder="e.g. Registrar, ICT – Scheduling">
                                <div class="form-text">Shown in Settings → activity log next to this person's name.</div>
                            </div>
                        <?php endif; ?>
                        <div class="col-12">
                            <label class="form-label">New password</label>
                            <input type="password" name="reset_password" class="form-control" minlength="8" autocomplete="new-password">
                            <div class="form-text">Leave blank to keep the current password.</div>
                        </div>
                        <div class="col-12">
                            <div class="form-check">
                                <input class="form-check-input" type="checkbox" name="is_active" value="1" id="is_active_edit" <?=(int)
$editRow['is_active'] === 1 ? 'checked' : '' ?>>
                                <label class="form-check-label" for="is_active_edit">Account active</label>
                            </div>
                        </div>
                        <div class="col-12 d-flex flex-wrap gap-2">
                            <button type="submit" class="btn btn-primary">Save changes</button>
                            <a class="btn btn-outline-secondary" href="super_admin_admins.php">Cancel</a>
                        </div>
                    </form>
                <?php endif; ?>
            </div>
        </div>
    </div>

```



```

        </div>
    </form>
    <?php else: ?>
        <form method="post" class="row g-3">
            <input type="hidden" name="action" value="add">
            <div class="col-12">
                <label class="form-label">Username</label>
                <input type="text" name="username" class="form-control" required maxlength="50" autocomplete="username">
            </div>
            <div class="col-12">
                <label class="form-label">Full name</label>
                <input type="text" name="full_name" class="form-control" required maxlength="100" autocomplete="name">
            </div>
            <?php if ($hasUserEmail): ?>
                <div class="col-12">
                    <label class="form-label">Email <span class="text-muted">(optional)</span></label>
                    <input type="email" name="email" class="form-control" maxlength="190" autocomplete="email">
                </div>
            <?php endif; ?>
            <?php if ($hasAdminLogTitle): ?>
                <div class="col-12">
                    <label class="form-label">Log title / office <span class="text-muted">(optional)</span></label>
                    <input type="text" name="admin_log_title" class="form-control" maxlength="120" placeholder="e.g. Registrar,
ICT – Scheduling">
                    <div class="form-text">Helps identify this admin in the activity log.</div>
                </div>
            <?php endif; ?>
            <div class="col-12">
                <label class="form-label">Password</label>
                <input type="password" name="password" class="form-control" required minlength="8" autocomplete="new-password">
            </div>
            <div class="col-12">
                <div class="form-check">
                    <input class="form-check-input" type="checkbox" name="is_active" value="1" id="is_active_add" checked>
                    <label class="form-check-label" for="is_active_add">Account active</label>
                </div>
            </div>
            <div class="col-12">
                <button type="submit" class="btn btn-primary">Create administrator</button>
            </div>
        </form>
    <?php endif; ?>
</div>
</div>
</div>
<div class="col-lg-7">
    <div class="card shadow-sm">
        <div class="card-header bg-white"><strong>Existing administrators</strong></div>
        <div class="card-body p-0">
            <?php if (!$admins): ?>
                <p class="text-muted p-3 mb-0">No administrator accounts yet. Create one using the form.</p>
            <?php else: ?>
                <div class="table-responsive">
                    <table class="table table-sm table-striped mb-0 align-middle">
                        <thead class="table-light">
                            <tr>
                                <th>Username</th>
                                <th>Full name</th>
                                <?php if ($hasAdminLogTitle): ?><th>Log title</th><?php endif; ?>
                                <th>Status</th>
                                <th></th>
                            </tr>
                        </thead>
                        <tbody>
                            <?php foreach ($admins as $a): ?>
                                <tr>
                                    <td><?= htmlspecialchars((string) $a['username']) ?></td>
                                    <td><?= htmlspecialchars((string) $a['full_name']) ?></td>
                                    <?php if ($hasAdminLogTitle): ?>
                                        <td class="small text-muted"><?= htmlspecialchars(trim((string) ($a['admin_log_title'] ?? ''))) ?>
                                        <?php endif; ?>
                                    <td>
                                        <?php if ((int) $a['is_active'] === 1): ?>
                                            <span class="badge bg-success">Active</span>
                                        <?php else: ?>
                                            <span class="badge bg-secondary">Disabled</span>
                                        <?php endif; ?>
                                    </td>
                                </tr>
                            </tbody>
                        </table>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

        <a class="btn btn-sm btn-outline-primary" href="super_admin_admins.php?edit=<?= (int) $a['id'] ?
>">Edit</a>
    </td>
</tr>
<?php endforeach; ?>
</tbody>
</table>
</div>
<?php endif; ?>
</div>
</div>
</div>
</div>

```

```

<?php require_once __DIR__ . '/includes/footer.php';

```

```

-----
FILE: super_admin_faculty_inventory.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['super_admin']);

$collegeId = (int) ($_GET['college_id'] ?? 0);
$scope = trim((string) ($_GET['scope'] ?? 'all'));
$search = trim((string) ($_GET['q'] ?? ''));
$allowedScopes = ['all', 'ge', 'non_ge'];
if (!in_array($scope, $allowedScopes, true)) {
    $scope = 'all';
}

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

if ($_SERVER['REQUEST_METHOD'] === 'POST' && (string) ($_POST['action'] ?? '') === 'delete' && isset($_POST['id'])) {
    $facultyRowId = (int) $_POST['id'];
    try {
        db()->beginTransaction();
        $stFaculty = db()->prepare('SELECT id, user_id FROM faculty WHERE id = ? LIMIT 1');
        $stFaculty->execute([$facultyRowId]);
        $facultyRow = $stFaculty->fetch(PDO::FETCH_ASSOC) ?: null;
        if (!$facultyRow) {
            throw new RuntimeException('Faculty record not found.');
```

```

}

$hasEmploymentStatusColumn = db_column_exists('faculty', 'employment_status');
$employmentSelect = $hasEmploymentStatusColumn
    ? 'f.employment_status'
    : "'Permanent' AS employment_status";

$sql = "SELECT f.id, f.user_id, f.faculty_id, f.full_name, f.department, f.email, f.max_hours_per_day, f.status,
             COALESCE(f.is_gened,0) AS is_gened, f.college_id, {$employmentSelect},
             u.username, u.is_active,
             c.college_code, c.college_name
        FROM faculty f
        LEFT JOIN users u ON u.id = f.user_id
        LEFT JOIN colleges c ON c.id = f.college_id
        WHERE 1=1";

$params = [];
if ($collegeId > 0) {
    $sql .= ' AND f.college_id = ?';
    $params[] = $collegeId;
}
if ($scope === 'ge') {
    $sql .= ' AND COALESCE(f.is_gened,0) = 1';
} elseif ($scope === 'non_ge') {
    $sql .= ' AND COALESCE(f.is_gened,0) = 0';
}
if ($search !== '') {
    $sql .= ' AND (f.full_name LIKE ? OR f.faculty_id LIKE ? OR f.department LIKE ? OR u.username LIKE ?';
    if ($hasEmploymentStatusColumn) {
        $sql .= ' OR f.employment_status LIKE ?';
    }
    $sql .= ')';
    $needle = '%' . $search . '%';
    $params[] = $needle;
    $params[] = $needle;
    $params[] = $needle;
    $params[] = $needle;
    if ($hasEmploymentStatusColumn) {
        $params[] = $needle;
    }
}
$sql .= ' ORDER BY c.college_code ASC, f.full_name ASC';
$stmt = db()->prepare($sql);
$stmt->execute($params);
$list = $stmt->fetchAll(PDO::FETCH_ASSOC) ?: [];

$collegeOptions = db()->query(
    'SELECT id, college_code, college_name FROM colleges ORDER BY college_code, college_name'
)->fetchAll(PDO::FETCH_ASSOC) ?: [];

$export = trim((string) ($_GET['export'] ?? ''));
if ($export === 'excel') {
    $filename = 'faculty_inventory_' . date('Ymd_His') . '.csv';
    header('Content-Type: text/csv; charset=UTF-8');
    header('Content-Disposition: attachment; filename="' . $filename . '"');
    header('Pragma: no-cache');
    header('Expires: 0');

    $out = fopen('php://output', 'wb');
    if ($out === false) {
        http_response_code(500);
        exit('Unable to generate export file.');
```

```

$row = [
    (string) ($r['faculty_id'] ?? ''),
    (string) ($r['full_name'] ?? ''),
    $collegeLabel,
    (string) ($r['department'] ?? ''),
    ((int) ($r['is_gened'] ?? 0) === 1) ? 'Yes' : 'No',
];
if ($hasEmploymentStatusColumn) {
    $row[] = (string) ($r['employment_status'] ?? 'Permanent');
}
$row[] = (string) ($r['email'] ?? '');
$row[] = (string) ($r['status'] ?? '');
fputcsv($out, $row);
}
fclose($out);
exit;
}

```

```

$pageTitle = 'Faculty inventory';
$mainContainerClass = 'container py-4 py-md-5 app-main';
require_once __DIR__ . '/includes/header.php';
?>

```

```

<style>
:root {
    --fi-surface: #ffffff;
    --fi-surface-soft: #fafcff;
    --fi-border: #e9edf2;
    --fi-border-soft: #f0f4f9;
    --fi-text: #1e293b;
    --fi-text-muted: #475569;
    --fi-chip-bg: #eef2ff;
    --fi-chip-text: #1f543b;
    --fi-chip-strong: #0f3b2a;
    --fi-college-bg: #eef2ff;
    --fi-college-text: #1e3a5f;
}
[data-bs-theme="dark"] {
    --fi-surface: #111827;
    --fi-surface-soft: #1f2937;
    --fi-border: #334155;
    --fi-border-soft: #293548;
    --fi-text: #e5e7eb;
    --fi-text-muted: #cbd5e1;
    --fi-chip-bg: #1f2937;
    --fi-chip-text: #86efac;
    --fi-chip-strong: #bbf7d0;
    --fi-college-bg: #1e293b;
    --fi-college-text: #b9dbfe;
}
@media (prefers-color-scheme: dark) {
    :root:not([data-bs-theme="light"]) {
        --fi-surface: #111827;
        --fi-surface-soft: #1f2937;
        --fi-border: #334155;
        --fi-border-soft: #293548;
        --fi-text: #e5e7eb;
        --fi-text-muted: #cbd5e1;
        --fi-chip-bg: #1f2937;
        --fi-chip-text: #86efac;
        --fi-chip-strong: #bbf7d0;
        --fi-college-bg: #1e293b;
        --fi-college-text: #b9dbfe;
    }
}
.faculty-dashboard {
    max-width: 1440px;
    margin: 0 auto;
    color: var(--fi-text);
}
.faculty-header {
    display: flex;
    justify-content: space-between;
    align-items: flex-end;
    flex-wrap: wrap;
    gap: 1rem;
    margin-bottom: 1.75rem;
}
.faculty-title h1 {
    margin: 0;
}

```

```

    font-size: 1.8rem;
    font-weight: 700;
    letter-spacing: -0.3px;
    background: linear-gradient(135deg, #1e3c2c, #2b5e3b);
    -webkit-background-clip: text;
    background-clip: text;
    color: transparent;
}
.faculty-title p {
    margin: 0.4rem 0 0;
    font-size: 0.9rem;
    color: var(--fi-text-muted);
    display: flex;
    align-items: center;
    gap: 8px;
}
}
.admin-chip {
    background: var(--fi-surface);
    border: 1px solid var(--fi-border);
    border-radius: 40px;
    padding: 0.4rem 1rem;
    font-size: 0.8rem;
    font-weight: 500;
    color: var(--fi-text);
    display: inline-flex;
    align-items: center;
    gap: 8px;
    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
}
}
.filter-card {
    background: var(--fi-surface);
    border: 1px solid var(--fi-border);
    border-radius: 24px;
    padding: 1.25rem 1.5rem;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.03), 0 1px 2px rgba(0, 0, 0, 0.05);
    margin-bottom: 1.75rem;
}
}
.filter-label {
    display: block;
    margin-bottom: 0.4rem;
    font-size: 0.75rem;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    color: var(--fi-text-muted);
}
}
.filter-card .form-select,
.filter-card .form-control {
    border-radius: 16px;
    border: 1px solid var(--fi-border);
    padding: 0.7rem 0.9rem;
    background: var(--fi-surface);
    color: var(--fi-text);
}
}
.filter-card .form-select:focus,
.filter-card .form-control:focus {
    border-color: #2c7a4b;
    box-shadow: 0 0 3px rgba(44, 122, 75, 0.2);
}
}
.search-wrapper {
    position: relative;
}
}
.search-wrapper .search-icon {
    position: absolute;
    left: 14px;
    top: 50%;
    transform: translateY(-50%);
    color: var(--fi-text-muted);
    font-size: 0.9rem;
}
}
.search-wrapper input {
    padding-left: 2.3rem;
}
}
.btn-apply {
    background: #1f543b;
    border: none;
    border-radius: 40px;
    padding: 0.7rem 1.4rem;
    color: #fff;
    font-weight: 600;
}

```

```

    font-size: 0.85rem;
  }
  .btn-apply:hover {
    background: #13472e;
    color: #fff;
  }
  .stats-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: 0.75rem;
    margin-bottom: 1rem;
  }
  .result-count {
    background: var(--fi-chip-bg);
    color: var(--fi-chip-text);
    border-radius: 40px;
    padding: 0.3rem 1rem;
    font-size: 0.8rem;
    font-weight: 500;
  }
  .result-count span {
    font-weight: 800;
    color: var(--fi-chip-strong);
  }
  .table-shell {
    background: var(--fi-surface);
    border: 1px solid var(--fi-border);
    border-radius: 24px;
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.02);
    overflow: hidden;
  }
  .table-shell .table {
    margin-bottom: 0;
    min-width: 980px;
  }
  .table-shell thead th {
    background: var(--fi-surface-soft);
    border-bottom: 1px solid var(--fi-border);
    color: var(--fi-text-muted);
    font-size: 0.8rem;
    letter-spacing: 0.3px;
    font-weight: 600;
    white-space: nowrap;
    padding: 1rem;
  }
  .table-shell tbody td {
    border-color: var(--fi-border-soft);
    color: var(--fi-text);
    padding: 1rem;
  }
  .table-shell tbody tr:hover td {
    background: color-mix(in srgb, var(--fi-surface-soft) 75%, #1f543b 25%);
  }
  .college-badge {
    background: var(--fi-college-bg);
    color: var(--fi-college-text);
    border-radius: 20px;
    padding: 0.2rem 0.7rem;
    display: inline-block;
    font-size: 0.75rem;
    font-weight: 500;
  }
  .ge-badge {
    border-radius: 30px;
    padding: 0.2rem 0.6rem;
    font-size: 0.7rem;
    font-weight: 600;
    display: inline-block;
  }
  .ge-badge.yes {
    background: #dcfce7;
    color: #15803d;
  }
  .ge-badge.no {
    background: #fee2e2;
    color: #b91c1c;
  }
  .employment-tag {

```

```

        background: var(--fi-surface-soft);
        border-radius: 30px;
        padding: 0.2rem 0.7rem;
        font-size: 0.75rem;
        font-weight: 500;
        color: var(--fi-text-muted);
        display: inline-block;
    }
    .status-badge {
        display: inline-flex;
        align-items: center;
        gap: 6px;
        background: #e6f7ec;
        color: #166534;
        border-radius: 40px;
        padding: 0.25rem 0.7rem;
        font-weight: 500;
        font-size: 0.75rem;
    }
    [data-bs-theme="dark"] .text-muted,
    [data-bs-theme="dark"] .small {
        color: var(--fi-text-muted) !important;
    }
    [data-bs-theme="dark"] .btn-outline-success {
        border-color: #34d399;
        color: #86efac;
    }
    [data-bs-theme="dark"] .btn-outline-success:hover {
        background: #14532d;
        border-color: #22c55e;
        color: #dcfce7;
    }
    @media (max-width: 767.98px) {
        .faculty-title h1 {
            font-size: 1.5rem;
        }
        .btn-apply {
            width: 100%;
        }
    }
}
</style>

<div class="faculty-dashboard">
    <div class="faculty-header">
        <div class="faculty-title">
            <h1><i class="fa-solid fa-chalkboard-user me-2" style="color:#2d6a4f;"></i>Faculty inventory</h1>
            <p>Western Philippines University - Super admin view <i class="fa-solid fa-university"></i></p>
        </div>
        <div class="admin-chip">
            <i class="fa-solid fa-crown"></i>
            <span>Administrator - Full access</span>
            <i class="fa-solid fa-chevron-down" style="font-size:0.7rem;"></i>
        </div>
    </div>
    <?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

    <div class="filter-card">
        <div>
            <form method="get" class="row g-3 align-items-end">
                <div class="col-md-4">
                    <label class="filter-label"><i class="fa-solid fa-building me-1"></i>College</label>
                    <select name="college_id" class="form-select">
                        <option value="0">All colleges</option>
                        <?php foreach ($collegeOptions as $c): ?>
                            <option value="<?= (int) $c['id'] ?"><?= $collegeId == (int) $c['id'] ? ' selected' : '' ?>>
                                <?= htmlspecialchars(trim((string) ($c['college_code'] ?? '')) . ' - ' . trim((string) ($c['college_name'] ??
''))) ?>
                            </option>
                        <?php endforeach; ?>
                    </select>
                </div>
                <div class="col-md-3">
                    <label class="filter-label"><i class="fa-solid fa-globe me-1"></i>Scope</label>
                    <select name="scope" class="form-select">
                        <option value="all"<?= $scope == 'all' ? ' selected' : '' ?>>All faculty</option>
                        <option value="ge"<?= $scope == 'ge' ? ' selected' : '' ?>>GE only</option>
                        <option value="non_ge"<?= $scope == 'non_ge' ? ' selected' : '' ?>>Non-GE only</option>
                    </select>
                </div>
            </div>
            <div class="col-md-3">

```

```

        <label class="filter-label"><i class="fa-solid fa-magnifying-glass me-1"></i>Search</label>
        <div class="search-wrapper">
            <i class="fa-solid fa-magnifying-glass search-icon"></i>
            <input type="text" name="q" class="form-control" value="<?= htmlspecialchars($search) ?>" placeholder="Name, ID,
program, username">
        </div>
    </div>
    <div class="col-md-2 d-grid">
        <button type="submit" class="btn btn-apply"><i class="fa-solid fa-sliders me-1"></i>Apply filters</button>
    </div>
    <div class="col-md-2 d-grid">
        <a
            class="btn btn-outline-success"
            href="super_admin_faculty_inventory.php?<?= htmlspecialchars(http_build_query([
                'college_id' => $collegeId,
                'scope' => $scope,
                'q' => $search,
                'export' => 'excel',
            ])) ?>"
        >
            <i class="fa-solid fa-file-excel me-1"></i>Download Excel
        </a>
    </div>
</form>
</div>
</div>

<div class="stats-row">
    <div class="result-count"><i class="fa-solid fa-users me-1"></i><span><?= count($list) ?></span> faculty records</div>
    <div class="text-muted small"><i class="fa-solid fa-database me-1"></i>Last sync: today</div>
</div>

<div class="table-shell">
    <div class="table-responsive">
        <table class="table align-middle">
            <thead>
                <tr>
                    <th>Faculty ID</th>
                    <th>Name</th>
                    <th>College</th>
                    <th>Program</th>
                    <th>GE</th>
                    <?php if ($hasEmploymentStatusColumn): ?><th>Employment</th><?php endif; ?>
                    <th>Email</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($list as $r): ?>
                    <?php
                        $collegeCode = trim((string) ($r['college_code'] ?? ''));
                        $collegeName = trim((string) ($r['college_name'] ?? ''));
                        $collegeLabel = $collegeCode != '' ? $collegeCode : 'N/A';
                        if ($collegeName != '') {
                            $collegeLabel .= ' - ' . $collegeName;
                        }
                        $isGened = (int) ($r['is_gened'] ?? 0) === 1;
                        $statusRaw = trim((string) ($r['status'] ?? ''));
                        $statusLabel = $statusRaw != '' ? ucfirst(strtolower($statusRaw)) : 'Active';
                        $statusIconClass = strtolower($statusLabel) === 'active'
                            ? 'fa-solid fa-circle-check text-success'
                            : 'fa-solid fa-circle-exclamation';
                    <?php
                <tr>
                    <td><strong><?= htmlspecialchars((string) ($r['faculty_id'] ?? '')) ?></strong></td>
                    <td><?= htmlspecialchars((string) ($r['full_name'] ?? '')) ?></td>
                    <td><span class="college-badge"><?= htmlspecialchars($collegeLabel) ?></span></td>
                    <td><?= htmlspecialchars((string) ($r['department'] ?? '')) ?></td>
                    <td>
                        <span class="ge-badge <?= $isGened ? 'yes' : 'no' ?>">
                            <?= $isGened ? 'Yes' : 'No' ?>
                        </span>
                    </td>
                    <?php if ($hasEmploymentStatusColumn): ?>
                        <td><span class="employment-tag"><?= htmlspecialchars((string) ($r['employment_status'] ?? 'Permanent')) ?></span>
                    </td>
                    <?php endif; ?>
                <td>
                    <?php $email = trim((string) ($r['email'] ?? '')); ?>

```



```

        <?php if ($email != ''): ?>
            <a href="mailto:<?= htmlspecialchars($email) ?>" style="color:#2c7a4b; text-decoration:none;">
                <?= htmlspecialchars($email) ?>
            </a>
        <?php endif; ?>
    </td>
    <td>
        <span class="status-badge"><i class="<?= $statusIconClass ?>"></i><?= htmlspecialchars($statusLabel) ?></span>
    </td>
    <td class="text-nowrap">
        <form method="post" class="d-inline" onsubmit="return confirm('Delete this faculty record?');">
            <input type="hidden" name="action" value="delete">
            <input type="hidden" name="id" value="<?= (int) ($r['id']) ?? 0 ?>">
            <button type="submit" class="btn btn-sm btn-outline-danger" title="Delete">
                <i class="fa-solid fa-trash"></i>
            </button>
        </form>
    </td>
</tr>
<?php endforeach; ?>
<?php if ($list === []): ?>
    <tr>
        <td colspan="<?= $hasEmploymentStatusColumn ? '9' : '8' ?>" class="text-center text-muted py-5">
            <i class="fa-solid fa-user-slash me-1"></i>No faculty records found for the selected filters.
        </td>
    </tr>
<?php endif; ?>
</tbody>
</table>
</div>
</div>

```

```

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: super_admin_programs.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/admin_activity_log.php';

require_role(['super_admin']);

$hasProgramsTable = db_table_exists('programs');
$hasProgYlTable = db_table_exists('programs_year_levels');
$hasCollegesTable = db_table_exists('colleges');

$filterCollegeId = isset($_GET['college_id']) ? (int) $_GET['college_id'] : 0;
$statusFilter = trim((string) ($_GET['status'] ?? 'all'));
if (!in_array($statusFilter, ['all', 'active', 'inactive'], true)) {
    $statusFilter = 'all';
}

$flash = $_SESSION['flash'] ?? '';
unset($_SESSION['flash']);

$redirectSelf = static function () use ($filterCollegeId, $statusFilter): never {
    $query = [];
    if ($filterCollegeId > 0) {
        $query['college_id'] = (string) $filterCollegeId;
    }
    if ($statusFilter !== 'all') {
        $query['status'] = $statusFilter;
    }
    $url = 'super_admin_programs.php';
    if ($query !== []) {
        $url .= '?' . http_build_query($query);
    }
    header('Location: ' . $url);
    exit;
};

$colleges = [];
if ($hasCollegesTable) {
    $colleges = db()->query(

```

```

        "SELECT id, college_code, college_name, status FROM colleges ORDER BY college_code"
    )->fetchAll(PDO::FETCH_ASSOC) ?: [];
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $action = (string) ($_POST['action'] ?? '');
    try {
        if (!$hasProgramsTable) {
            throw new RuntimeException('Programs module is unavailable until you run upgrade_roles.php.');
        }
        if (!$hasCollegesTable) {
            throw new RuntimeException('Colleges table is missing.');
```

```

        $st->execute([$collegeId, $programName, $status, $id]);
        if ($hasProgYlTable) {
            program_year_levels_replace($id, parse_dean_program_year_levels_post($_POST));
        }
        log_admin_activity(
            'edit',
            'Program offers',
            'Program #' . $id,
            $before,
            [
                'id' => $id,
                'college_id' => $collegeId,
                'program_name' => $programName,
                'status' => $status,
            ]
        );
        $_SESSION['flash'] = 'Program offer updated.';
    } elseif ($action === 'delete' && isset($_POST['id'])) {
        $id = (int) $_POST['id'];
        if ($id < 1) {
            throw new RuntimeException('Invalid program.');
```

```

$ids = array_map(static fn ($r): int => (int) ($r['id'] ?? 0), $list);
$ids = array_values(array_filter(array_unique($ids), static fn (int $i): bool => $i > 0));
if ($ids !== []) {
    $holds = implode(',', array_fill(0, count($ids), ''));
    $q = db()->prepare("SELECT program_id, year_level FROM programs_year_levels WHERE program_id IN ({$holds}) ORDER BY year_level");
    $q->execute($ids);
    while ($lr = $q->fetch(PDO::FETCH_ASSOC)) {
        $pid = (int) ($lr['program_id'] ?? 0);
        $yl = trim((string) ($lr['year_level'] ?? ''));
        if ($pid < 1 || $yl === '') {
            continue;
        }
        if (!isset($levelsByProgId[$pid])) {
            $levelsByProgId[$pid] = [];
        }
        $levelsByProgId[$pid][] = $yl;
    }
}

$prefillCanon = [];
$prefillExtraCsv = '';
if ($hasProgYlTable && $editRow) {
    $defined = program_defined_year_levels((int) $editRow['id']);
    $canonLabels = ['1', '2', '3', '4', '5'];
    foreach ($defined as $d) {
        if (in_array($d, $canonLabels, true)) {
            $prefillCanon[] = $d;
        }
    }
    $extra = [];
    foreach ($defined as $d) {
        if (!in_array($d, $canonLabels, true)) {
            $extra[] = $d;
        }
    }
    $prefillExtraCsv = $extra !== [] ? implode(', ', $extra) : '';
}

$listQuery = [];
if ($filterCollegeId > 0) {
    $listQuery['college_id'] = (string) $filterCollegeId;
}
if ($statusFilter !== 'all') {
    $listQuery['status'] = $statusFilter;
}
$listBaseUrl = 'super_admin_programs.php' . ($listQuery !== [] ? '?' . http_build_query($listQuery) : '');
$editBaseUrl = 'super_admin_programs.php';
$editQuery = $listQuery;

$pageTitle = 'Program offers';
require_once __DIR__ . '/includes/header.php';

$canonicalYl = ['1', '2', '3', '4', '5'];
$editCollegeId = (int) ($editRow['college_id'] ?? 0);
?>
<h1 class="h3 mb-2"><i class="fa-solid fa-graduation-cap me-2 text-primary"></i>Program offers</h1>
<p class="text-muted mb-4">University-wide catalog of programs offered by each college. Super Admin can add, edit, or delete any offer.</p>
<?php if ($flash): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>
<?php if (!$hasProgramsTable || !$hasCollegesTable): ?>
    <div class="alert alert-warning">Programs module is unavailable. Please run <a href="upgrade_roles.php">upgrade_roles.php</a>.</div>
<?php else: ?>
<?php if (!$hasProgYlTable): ?>
    <div class="alert alert-warning">Run <a href="upgrade_roles.php">upgrade_roles.php</a> to enable program year levels.</div>
<?php endif; ?>

<form method="get" class="card shadow-sm mb-4">
    <div class="card-body row g-3 align-items-end">
        <div class="col-md-5">
            <label class="form-label">Filter by college</label>
            <select name="college_id" class="form-select">
                <option value="0">All colleges</option>
                <?php foreach ($colleges as $c): ?>
                    <option value="<?=(int) $c['id'] ?>" <?=$filterCollegeId === (int) $c['id'] ? 'selected' : '' ?>>
                        <?=$htmlspecialchars((string) $c['college_code']) ?> - <?=$htmlspecialchars((string) $c['college_name']) ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </div>
        <div class="col-md-3">

```

```

        <label class="form-label">Status</label>
        <select name="status" class="form-select">
            <option value="all" <?= $statusFilter === 'all' ? 'selected' : '' ?>>All</option>
            <option value="active" <?= $statusFilter === 'active' ? 'selected' : '' ?>>Active</option>
            <option value="inactive" <?= $statusFilter === 'inactive' ? 'selected' : '' ?>>Inactive</option>
        </select>
    </div>
    <div class="col-md-4">
        <button type="submit" class="btn btn-outline-primary">Apply filters</button>
        <?php if ($filterCollegeId > 0 || $statusFilter !== 'all'): ?>
            <a href="super_admin_programs.php" class="btn btn-outline-secondary">Clear</a>
        <?php endif; ?>
    </div>
</div>
</form>

<div class="card shadow-sm mb-4"><div class="card-body">
    <h2 class="h5 mb-3"><?= $editRow ? 'Edit program offer' : 'Add program offer' ?></h2>
    <form method="post" class="row g-3">
        <input type="hidden" name="action" value="<?= $editRow ? 'edit' : 'add' ?>">
        <?php if ($editRow): ?><input type="hidden" name="id" value="(int) $editRow['id'] ?>"><?php endif; ?>
        <div class="col-md-5">
            <label class="form-label">College</label>
            <select name="college_id" class="form-select" required>
                <option value="">Select college...</option>
                <?php foreach ($colleges as $c): ?>
                    <option value="(int) $c['id'] ?>" <?= $editCollegeId === (int) $c['id'] ? 'selected' : '' ?>
                        <?= htmlspecialchars((string) $c['college_code']) ?> - <?= htmlspecialchars((string) $c['college_name']) ?>
                        <?= ((string) ($c['status'] ?? '')) === 'inactive' ? ' (inactive)' : '' ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </div>
        <div class="col-md-5">
            <label class="form-label">Program name</label>
            <input name="program_name" class="form-control" required maxlength="120" placeholder="e.g. BS Social Work" value="<?=
htmlspecialchars((string) ($editRow['program_name'] ?? '')) ?>">
        </div>
        <div class="col-md-2">
            <label class="form-label">Status</label>
            <select name="status" class="form-select">
                <option value="active" <?= (($editRow['status'] ?? 'active') === 'active') ? 'selected' : '' ?>>Active</option>
                <option value="inactive" <?= (($editRow['status'] ?? 'active') === 'inactive') ? 'selected' : '' ?>>Inactive</option>
            </select>
        </div>
        <?php if ($hasProgYlTable): ?>
        <div class="col-12">
            <label class="form-label">Year levels offered</label>
            <div class="d-flex flex-wrap gap-3 mb-2">
                <?php foreach ($canonicalYl as $yl): ?>
                    <div class="form-check">
                        <input class="form-check-input" type="checkbox" name="program_year_levels[]" value="<?= htmlspecialchars($yl) ?>"
                            id="sa_yl_<?= htmlspecialchars($yl) ?>" <?= in_array($yl, $prefillCanon, true) ? 'checked' : '' ?>>
                        <label class="form-check-label" for="sa_yl_<?= htmlspecialchars($yl) ?>">Year <?= htmlspecialchars($yl) ?></label>
                    </div>
                <?php endforeach; ?>
            </div>
            <label class="form-label small text-muted">Additional levels (comma-separated)</label>
            <input type="text" name="program_year_level_extra" class="form-control" maxlength="200" placeholder="e.g. 6, grad1"
                value="<?= htmlspecialchars($prefillExtraCsv) ?>">
            <div class="form-text">These levels appear in scheduling forms for the college and when GEN ED coordinators target the program.
        </div>
        </div>
        <?php endif; ?>
        <div class="col-12">
            <button type="submit" class="btn btn-primary"<?= app_tooltip_attr($editRow ? 'Saves changes to this university program offer.' :
'Adds a new program offer under the selected college.') ?>>Save</button>
            <?php if ($editRow): ?>
                <a class="btn btn-outline-secondary" href="<?= htmlspecialchars($listBaseUrl) ?>"<?= app_tooltip_attr('Closes the editor
without saving.') ?>>Cancel</a>
            <?php endif; ?>
        </div>
    </form>
</div></div>

<div class="card shadow-sm"><div class="table-responsive">
    <table class="table mb-0 align-middle">
        <thead class="table-light">
            <tr>
                <th>College</th>

```

```

        <th>Program</th>
        <?php if ($hasProgYlTable): ?><th>Year levels</th><?php endif; ?>
        <th>Status</th>
        <th></th>
    </tr>
</thead>
<tbody>
<?php foreach ($list as $r): ?>
    <?php
    $editQuery['edit'] = (string) (int) $r['id'];
    $editHref = $editBaseUrl . '?' . http_build_query($editQuery);
    ?>
    <tr>
        <td>
            <div class="fw-semibold"><?= htmlspecialchars((string) ($r['college_code'] ?? '-')) ?></div>
            <div class="small text-muted"><?= htmlspecialchars((string) ($r['college_name'] ?? '')) ?></div>
        </td>
        <td><?= htmlspecialchars((string) $r['program_name']) ?></td>
        <?php if ($hasProgYlTable): ?>
        <td>
            <?php
            $lid = (int) $r['id'];
            $yrs = isset($levelsByProgId[$lid]) ? sort_schedule_year_levels($levelsByProgId[$lid]) : [];
            ?>
            <?php if ($yrs === []): ?>
                <span class="text-muted small">— Not configured</span>
            <?php else: ?>
                <?php foreach ($yrs as $y): ?>
                    <span class="badge bg-primary-subtle text-primary-emphasis me-1 mb-1"><?= htmlspecialchars((string) $y) ?>
                </span>
                <?php endforeach; ?>
            <?php endif; ?>
        </td>
        <?php endif; ?>
    </tr>
    <?php endif; ?>
    <td>
        <span class="badge <?= ($r['status'] === 'active') ? 'bg-success-subtle text-success-emphasis' : 'bg-secondary-subtle text-secondary-emphasis' ?>">
            <?= htmlspecialchars((string) $r['status']) ?>
        </span>
    </td>
    <td class="text-nowrap">
        <a class="btn btn-sm btn-outline-primary" href="<?= htmlspecialchars($editHref) ?>"><?= app_tooltip_attr('Edits this program offer (college, name, status, or year levels).') ?>>Edit</a>
        <form method="post" class="d-inline" onsubmit="return confirm('Delete this program offer from the university catalog?');">
            <input type="hidden" name="action" value="delete">
            <input type="hidden" name="id" value="<?= (int) $r['id'] ?>">
            <button type="submit" class="btn btn-sm btn-outline-danger"><?= app_tooltip_attr('Removes this program offer after confirmation.') ?>>Delete</button>
        </form>
    </td>
    </tr>
</tbody>
<?php endforeach; ?>
<?php if (!$list): ?>
    <tr>
        <td colspan="<?= $hasProgYlTable ? '5' : '4' ?>" class="text-center text-muted py-4">
            No program offers found<?= ($filterCollegeId > 0 || $statusFilter !== 'all') ? ' for the current filters' : '' ?>.
        </td>
    </tr>
</tbody>
<?php endif; ?>
</table>
</div></div>
<?php endif; ?>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

```

```

-----
FILE: super_admin_teaching_load_report.php
-----

```

```

<?php
declare(strict_types=1);

/**
 * Super Admin: weekly teaching contact hours by program (faculty.department),
 * flagged under load (<18 hrs/week) or overload (>27). Lecture hours use catalog
 * lecture units (x1); laboratory hours use the scheduled lab block when the course
 * has lab units; otherwise session length x meeting days.
 */

```

```

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['super_admin']);
$hasEmploymentStatusColumn = db_column_exists('faculty', 'employment_status');
$hasLectureUnits = db_column_exists('courses', 'lecture_units');
$hasLaboratoryUnits = db_column_exists('courses', 'laboratory_units');

$adminCollegeId = (int) ($_GET['college_id'] ?? 0);
$sem = trim((string) ($_GET['semester'] ?? ''));
$sy = trim((string) ($_GET['school_year'] ?? ''));
$programSearch = trim((string) ($_GET['program_search'] ?? ''));
$employmentStatusFilter = trim((string) ($_GET['employment_status'] ?? ''));
$allowedEmploymentStatuses = ['Permanent', 'Contract of Service', 'Temporary'];
if (!in_array($employmentStatusFilter, $allowedEmploymentStatuses, true)) {
    $employmentStatusFilter = '';
}

$latestTerm = db()->query(
    'SELECT semester, school_year FROM schedules ORDER BY school_year DESC, id DESC LIMIT 1'
)->fetch(PDO::FETCH_ASSOC) ?: null;
if ($sem === '' && $latestTerm) {
    $sem = (string) ($latestTerm['semester'] ?? '');
}
if ($sy === '' && $latestTerm) {
    $sy = (string) ($latestTerm['school_year'] ?? '');
}

$courseUnitsSelect = ', c.units AS course_units_total';
if ($hasLectureUnits) {
    $courseUnitsSelect .= ', c.lecture_units';
}
if ($hasLaboratoryUnits) {
    $courseUnitsSelect .= ', c.laboratory_units';
}
$hasCourseLabFlag = db_column_exists('courses', 'is_laboratory');
$courseLabSelect = $hasCourseLabFlag ? ', c.is_laboratory' : '';
$hasSessionKind = db_column_exists('schedules', 'session_kind');
$sessionKindSelect = $hasSessionKind ? ', s.session_kind' : '';

$sql = "SELECT s.id, s.faculty_id, s.course_id, s.college_id AS sched_college_id,
    s.semester, s.school_year,
    s.start_time, s.end_time, s.day_of_week,
    f.full_name AS faculty_name, f.department AS faculty_department,
    c.course_code, r.type AS room_type{$courseUnitsSelect}{$courseLabSelect}{$sessionKindSelect}
    FROM schedules s
    INNER JOIN faculty f ON f.id = s.faculty_id
    INNER JOIN courses c ON c.id = s.course_id
    LEFT JOIN rooms r ON r.id = s.room_id
    WHERE 1=1";

$params = [];
if ($adminCollegeId > 0) {
    $sql .= ' AND s.college_id = ?';
    $params[] = $adminCollegeId;
}
if ($sem !== '') {
    $sql .= ' AND s.semester = ?';
    $params[] = $sem;
}
if ($sy !== '') {
    $sql .= ' AND s.school_year = ?';
    $params[] = $sy;
}
$sql .= ' ORDER BY s.school_year DESC, s.semester, f.full_name, s.start_time';

$stmt = db()->prepare($sql);
$stmt->execute($params);
$schedRows = $stmt->fetchAll(PDO::FETCH_ASSOC) ?: [];

/** Normalize MySQL TIME / DATETIME-ish values to HH:MM:SS for minute math (host TZ-safe). */
$normalizeScheduleTime = static function (string $value): string {
    $value = trim($value);
    if ($value === '') {
        return '';
    }
    // "YYYY-mm-dd HH:MM:SS" or "HH:MM:SS[fraction]" / "HH:MM"
    if (preg_match('/(\d{1,2}):(\d{2})(?:.(\d{2}))?/', $value, $m)) {
        return sprintf('%02d:%02d:%02d', (int) $m[1], (int) $m[2], (int) ($m[3] ?? 0));
    }
    return $value;
};

```

```

};

/** Contact hours between start/end – uses minutes, not DateTime (avoids server TZ / format quirks). */
$sessionHoursBetween = static function (string $start, string $end) use ($normalizeScheduleTime): float {
    $rawS = $normalizeScheduleTime($start);
    $rawE = $normalizeScheduleTime($end);
    if ($rawS === '' || $rawE === '') {
        return 0.0;
    }
    $mins = time_to_minutes($rawE) - time_to_minutes($rawS);
    if ($mins <= 0) {
        return 0.0;
    }
    return round($mins / 60, 2);
};

$weeklyContactFromGroup = static function (array $group) use (
    $sessionHoursBetween,
    $normalizeScheduleTime,
    $hasLectureUnits,
    $hasLaboratoryUnits
): array {
    $r0 = $group[0];
    $lecU = (float) ($r0['lecture_units'] ?? 0);
    $labU = (float) ($r0['laboratory_units'] ?? 0);
    $noLabComponent = $hasLaboratoryUnits && $labU <= 0;

    $lec = 0.0;
    $lab = 0.0;
    foreach ($group as $r) {
        $start = $normalizeScheduleTime((string) ($r['start_time'] ?? ''));
        $end = $normalizeScheduleTime((string) ($r['end_time'] ?? ''));
        $h = $sessionHoursBetween($start, $end);
        $dayCount = count(parse_day_set((string) ($r['day_of_week'] ?? '')));
        if ($dayCount < 1) {
            $dayCount = 1;
        }
        $weeklyH = $h * $dayCount;
        $isLabSlot = !$noLabComponent && schedule_session_is_laboratory(
            $start,
            $end,
            (string) ($r['room_type'] ?? ''),
            isset($r['session_kind']) ? (string) $r['session_kind'] : null,
            array_key_exists('is_laboratory', $r) ? ((int) ($r['is_laboratory'] ?? 0) === 1) : null
        );
        if ($isLabSlot) {
            $lab += $weeklyH;
        } else {
            $lec += $weeklyH;
        }
    }

    // Match faculty_teaching_load: catalog lecture units x1; lab from timetable when course has lab units.
    if ($hasLectureUnits && $hasLaboratoryUnits && ($lecU > 0 || $labU > 0)) {
        $lec = $lecU;
        $lab = $labU > 0 ? $lab : 0.0;
    }

    return [
        'lec' => round($lec, 2),
        'lab' => round($lab, 2),
        'total' => round($lec + $lab, 2),
    ];
};

$courseOfferingUnits = static function (array $r) use ($hasLectureUnits, $hasLaboratoryUnits): float {
    $catalog = (float) ($r['course_units_total'] ?? 0);
    if ($hasLectureUnits && $hasLaboratoryUnits) {
        $sum = (float) ($r['lecture_units'] ?? 0) + (float) ($r['laboratory_units'] ?? 0);
        // Server DBs may have lec/lab columns present but still empty after transfer.
        return $sum > 0 ? $sum : $catalog;
    }

    return $catalog;
};

$courseOfferingLecUnits = static function (array $r) use ($hasLectureUnits, $hasLaboratoryUnits, $courseOfferingUnits): float {
    if ($hasLectureUnits && $hasLaboratoryUnits) {
        $lec = (float) ($r['lecture_units'] ?? 0);
        $lab = (float) ($r['laboratory_units'] ?? 0);
    }

```



```

        if ($lec > 0 || $lab > 0) {
            return $lec;
        }
    }

    return $courseOfferingUnits($r);
};

$courseOfferingLabUnits = static function (array $r) use ($hasLectureUnits, $hasLaboratoryUnits): float {
    if ($hasLectureUnits && $hasLaboratoryUnits) {
        return (float) ($r['laboratory_units'] ?? 0);
    }

    return 0.0;
};

$formatUnitCell = static function (float $units): string {
    return $units > 0 ? number_format($units, 2) : '-';
};

/** Permanent/Temporary load equivalent: lecture contact hours x2, laboratory x1.5. */
$weightedTeachingHours = static function (float $lecHours, float $labHours): float {
    return round(($lecHours * 2.0) + ($labHours * 1.5), 2);
};

$scheduleListGroup = [];
foreach ($schedRows as $r) {
    $cid = (int) ($r['sched_college_id'] ?? 0);
    $k = (int) $r['faculty_id'] . '|' . (int) $r['course_id'] . '|' . (string) $r['semester'] . '|' . (string) $r['school_year'] . '|';
    $cid;
    $scheduleListGroup[$k] = $r;
}
foreach ($scheduleListGroup as &$g) {
    usort(
        $g,
        static function (array $a, array $b) use ($normalizeScheduleTime): int {
            $aLab = schedule_session_is_laboratory(
                $normalizeScheduleTime((string) ($a['start_time'] ?? '')),
                $normalizeScheduleTime((string) ($a['end_time'] ?? '')),
                (string) ($a['room_type'] ?? ''),
                isset($a['session_kind']) ? (string) $a['session_kind'] : null,
                array_key_exists('is_laboratory', $a) ? ((int) ($a['is_laboratory'] ?? 0) === 1) : null
            );
            $bLab = schedule_session_is_laboratory(
                $normalizeScheduleTime((string) ($b['start_time'] ?? '')),
                $normalizeScheduleTime((string) ($b['end_time'] ?? '')),
                (string) ($b['room_type'] ?? ''),
                isset($b['session_kind']) ? (string) $b['session_kind'] : null,
                array_key_exists('is_laboratory', $b) ? ((int) ($b['is_laboratory'] ?? 0) === 1) : null
            );
            if ($aLab !== $bLab) {
                return $aLab ? 1 : -1;
            }
            return strcmp((string) ($a['id'] ?? ''), (string) ($b['id'] ?? ''));
        }
    );
}
unset($g);

$hoursByFaculty = [];
$lecHoursByFaculty = [];
$labHoursByFaculty = [];
$unitsByFaculty = [];
$lecUnitsByFaculty = [];
$labUnitsByFaculty = [];
foreach ($scheduleListGroup as $group) {
    $r0 = $group[0];
    $fid = (int) $r0['faculty_id'];
    $contact = $weeklyContactFromGroup($group);
    $units = $courseOfferingUnits($r0);
    $lecUnits = $courseOfferingLecUnits($r0);
    $labUnits = $courseOfferingLabUnits($r0);
    if (!isset($hoursByFaculty[$fid])) {
        $hoursByFaculty[$fid] = 0.0;
        $lecHoursByFaculty[$fid] = 0.0;
        $labHoursByFaculty[$fid] = 0.0;
        $unitsByFaculty[$fid] = 0.0;
        $lecUnitsByFaculty[$fid] = 0.0;
        $labUnitsByFaculty[$fid] = 0.0;
    }
}

```

```

$hoursByFaculty[$fid] = round($hoursByFaculty[$fid] + $contact['total'], 2);
$lecHoursByFaculty[$fid] = round($lecHoursByFaculty[$fid] + $contact['lec'], 2);
$labHoursByFaculty[$fid] = round($labHoursByFaculty[$fid] + $contact['lab'], 2);
$unitsByFaculty[$fid] = round($unitsByFaculty[$fid] + $units, 2);
$lecUnitsByFaculty[$fid] = round($lecUnitsByFaculty[$fid] + $lecUnits, 2);
$labUnitsByFaculty[$fid] = round($labUnitsByFaculty[$fid] + $labUnits, 2);
}

$underMax = 18.0;
$overMin = 27.0;

$facultyEmploymentSelect = $hasEmploymentStatusColumn
    ? 'f.employment_status'
    : "'Permanent' AS employment_status";
$facultySql = "SELECT f.id, f.full_name, f.department, {$facultyEmploymentSelect}, f.college_id,
    c.college_code, c.college_name
    FROM faculty f
    LEFT JOIN colleges c ON c.id = f.college_id
    WHERE f.status = 'active'";
$facultyParams = [];
if ($adminCollegeId > 0) {
    $facultySql .= ' AND f.college_id = ?';
    $facultyParams[] = $adminCollegeId;
}
if ($programSearch !== '') {
    $facultySql .= ' AND f.department = ?';
    $facultyParams[] = $programSearch;
}
if ($employmentStatusFilter !== '' && $hasEmploymentStatusColumn) {
    $facultySql .= ' AND f.employment_status = ?';
    $facultyParams[] = $employmentStatusFilter;
}
$facultySql .= ' ORDER BY f.department, f.full_name';
$facStmt = db()->prepare($facultySql);
$facStmt->execute($facultyParams);
$allFaculty = $facStmt->fetchAll(PDO::FETCH_ASSOC) ?: [];

$byProgram = [];
foreach ($allFaculty as $fr) {
    $prog = trim((string) ($fr['department'] ?? ''));
    if ($prog === '') {
        $prog = '- (no program / department)';
    }
    $fid = (int) $fr['id'];
    $hrs = $hoursByFaculty[$fid] ?? 0.0;
    $lecHrs = $lecHoursByFaculty[$fid] ?? 0.0;
    $labHrs = $labHoursByFaculty[$fid] ?? 0.0;
    $units = $unitsByFaculty[$fid] ?? 0.0;
    $lecUnits = $lecUnitsByFaculty[$fid] ?? 0.0;
    $labUnits = $labUnitsByFaculty[$fid] ?? 0.0;
    if ($hrs < $underMax) {
        $cat = 'under';
    } elseif ($hrs > $overMin) {
        $cat = 'over';
    } else {
        $cat = 'normal';
    }
    if (!isset($byProgram[$prog])) {
        $byProgram[$prog] = ['under' => [], 'normal' => [], 'over' => []];
    }
    $cc = trim((string) ($fr['college_code'] ?? ''));
    $cn = trim((string) ($fr['college_name'] ?? ''));
    $collegeLabel = ($cc === '' && $cn === '') ? '-' : ($cc !== '' ? $cc . ' - ' . $cn : $cn);
    $byProgram[$prog][$cat][] = [
        'name' => (string) $fr['full_name'],
        'employment_status' => trim((string) ($fr['employment_status'] ?? '')) !== '' ? (string) $fr['employment_status'] : 'Permanent',
        'hours' => $hrs,
        'lec_hours' => $lecHrs,
        'lab_hours' => $labHrs,
        'units' => $units,
        'lec_units' => $lecUnits,
        'lab_units' => $labUnits,
        'college' => $collegeLabel,
    ];
}
uksort($byProgram, 'strcasecmp');

$collegeOpts = db()->query(
    'SELECT id, college_code, college_name FROM colleges ORDER BY college_code'
)->fetchAll(PDO::FETCH_ASSOC) ?: [];

```

```

$programOptsSql = 'SELECT DISTINCT department FROM faculty WHERE department <> ""';
$programOptsParams = [];
if ($adminCollegeId > 0) {
    $programOptsSql .= ' AND college_id = ?';
    $programOptsParams[] = $adminCollegeId;
}
$programOptsSql .= ' ORDER BY department';
$programOptsStmt = db()->prepare($programOptsSql);
$programOptsStmt->execute($programOptsParams);
$programOpts = $programOptsStmt->fetchAll(PDO::FETCH_COLUMN) ?: [];

$sems = db()->query('SELECT DISTINCT semester FROM schedules ORDER BY semester')->fetchAll(PDO::FETCH_COLUMN) ?: [];
$years = db()->query('SELECT DISTINCT school_year FROM schedules ORDER BY school_year DESC')->fetchAll(PDO::FETCH_COLUMN) ?: [];

$pageTitle = 'Teaching load report (all programs)';
$mainContainerClass = 'container py-4 py-md-5 app-main';
require_once __DIR__ . '/includes/header.php';
?>
<style>
:root {
    --bg: #f0f4f8;
    --text: #1e2a3e;
    --muted: #5b6e8c;
    --primary: #1f5e6b;
    --primary-dark: #12464f;
    --surface: #ffffff;
    --line: #e2e8f0;
    --surface-soft: #f9fbfd;
    --table-text: #1f2f3e;
    --title-strong: #1f4e5c;
    --name-strong: #13505b;
}

body {
    background: var(--bg);
    color: var(--text);
}

.report-dashboard {
    max-width: 1440px;
    margin: 0 auto;
}

.report-header {
    display: flex;
    justify-content: space-between;
    align-items: flex-end;
    flex-wrap: wrap;
    gap: 0.75rem;
    margin-bottom: 1.5rem;
    border-bottom: 2px solid var(--line);
    padding-bottom: 1rem;
}

.report-title {
    font-size: 1.9rem;
    font-weight: 700;
    letter-spacing: -0.3px;
    margin: 0;
    background: linear-gradient(135deg, #1e3c3f 0%, #2b5e6b 100%);
    background-clip: text;
    -webkit-background-clip: text;
    color: transparent;
}

.report-subtitle {
    margin: 0.35rem 0 0 0;
    color: #4a627a;
    font-weight: 500;
    font-size: 0.9rem;
}

.export-badge {
    background: var(--surface);
    padding: 0.5rem 1rem;
    border-radius: 999px;
    font-size: 0.8rem;
    font-weight: 600;
    color: #2c5f6e;
    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
}

```

```

border: 1px solid #cbd5e1;
display: inline-flex;
align-items: center;
gap: 0.45rem;
}

.filter-card {
background: var(--surface);
border-radius: 24px;
box-shadow: 0 8px 20px rgba(0, 0, 0, 0.03), 0 2px 6px rgba(0, 0, 0, 0.05);
padding: 1.35rem 1.6rem;
margin-bottom: 1.8rem;
border: 1px solid #e9edf2;
}

.filter-label {
display: block;
font-size: 0.75rem;
font-weight: 700;
text-transform: uppercase;
letter-spacing: 0.45px;
color: var(--muted);
margin-bottom: 0.45rem;
}

.filter-control {
border-radius: 16px;
border: 1px solid #cfdedf;
font-size: 0.9rem;
font-weight: 500;
color: #1f2f40;
min-height: 42px;
}

.filter-control:focus {
border-color: #2c8f9c;
box-shadow: 0 0 3px rgba(44, 143, 156, 0.18);
}

.btn-pill {
border-radius: 999px;
padding: 0.6rem 1.2rem;
font-size: 0.85rem;
font-weight: 600;
}

.program-card {
background: var(--surface);
border-radius: 22px;
border: 1px solid #eef2f8;
box-shadow: 0 4px 14px rgba(0, 0, 0, 0.02);
padding: 1rem;
margin-bottom: 1.25rem;
}

.program-head {
display: flex;
justify-content: space-between;
align-items: center;
flex-wrap: wrap;
gap: 0.8rem;
margin-bottom: 0.9rem;
}

.program-name {
margin: 0;
font-size: 1.3rem;
color: var(--title-strong);
font-weight: 700;
}

.stats-row {
display: flex;
gap: 0.7rem;
flex-wrap: wrap;
}

.stat-chip {
background: var(--surface-soft);
border: 1px solid var(--line);

```

```

border-radius: 999px;
padding: 0.45rem 0.85rem;
display: inline-flex;
align-items: center;
gap: 0.5rem;
font-size: 0.8rem;
}

.stat-chip .value {
  font-weight: 700;
  color: #1f3b45;
}

.table-shell {
  border: 1px solid #e6edf4;
  border-radius: 16px;
  overflow: auto;
}

.report-table {
  width: 100%;
  min-width: 980px;
  border-collapse: collapse;
  font-size: 0.86rem;
}

.report-table th {
  text-align: left;
  padding: 0.9rem 0.85rem;
  background: var(--surface-soft);
  color: #2c4b5c;
  border-bottom: 1px solid #e4eaf1;
  font-weight: 700;
  font-size: 0.79rem;
  text-transform: uppercase;
  letter-spacing: 0.3px;
}

.report-table td {
  padding: 0.85rem;
  border-bottom: 1px solid #edf2f7;
  color: var(--table-text);
  font-weight: 500;
  vertical-align: middle;
}

.report-table tbody tr:last-child td {
  border-bottom: none;
}

.report-table tbody tr:hover td {
  background: #fafcff;
}

.faculty-name {
  font-weight: 700;
  color: var(--name-strong);
}

.status-pill {
  display: inline-flex;
  padding: 0.25rem 0.6rem;
  border-radius: 999px;
  font-size: 0.72rem;
  font-weight: 700;
  border: 1px solid transparent;
}

.status-under {
  background: #fff6e5;
  color: #9a6200;
  border-color: #f6d7a6;
}

.status-normal {
  background: #e8f7ee;
  color: #0d6a39;
  border-color: #bde7cd;
}

```

```

.status-over {
  background: #feecec;
  color: #a11a1a;
  border-color: #f5c3c3;
}

.empty-state {
  text-align: center;
  padding: 2rem 1rem;
  color: #8ea0b3;
  font-style: italic;
}

.footer-note {
  margin-top: 1rem;
  font-size: 0.74rem;
  color: #7c8ea0;
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 0.8rem;
  flex-wrap: wrap;
}

[data-bs-theme="dark"] {
  --bg: #0f1722;
  --text: #d9e4f0;
  --muted: #98adc4;
  --surface: #111c28;
  --surface-soft: #162435;
  --line: #2a3a4d;
  --table-text: #d3e0ec;
  --title-strong: #cfe8ff;
  --name-strong: #a7deef;
}

[data-bs-theme="dark"] .report-subtitle {
  color: #9db4cc;
}

[data-bs-theme="dark"] .export-badge {
  background: #152334;
  border-color: #2a4159;
  color: #b9d7ec;
  box-shadow: none;
}

[data-bs-theme="dark"] .filter-card,
[data-bs-theme="dark"] .program-card {
  box-shadow: none;
  border-color: #2a3a4d;
}

[data-bs-theme="dark"] .filter-control {
  background: #0f1a27;
  border-color: #2e435b;
  color: #d7e6f5;
}

[data-bs-theme="dark"] .filter-control:focus {
  border-color: #4d7fa3;
  box-shadow: 0 0 3px rgba(93, 146, 186, 0.25);
}

[data-bs-theme="dark"] .report-table th {
  color: #bcd1e4;
  border-bottom-color: #334b61;
}

[data-bs-theme="dark"] .report-table td {
  border-bottom-color: #24384c;
}

[data-bs-theme="dark"] .report-table tbody tr:hover td {
  background: #18283a;
}

[data-bs-theme="dark"] .table-shell {
  border-color: #2a3d53;
}

```

```

[data-bs-theme="dark"] .footer-note {
  color: #8ea4ba;
}

@media print {
  @page { size: A4 portrait; margin: 8mm; }

  .no-print { display: none !important; }
  .footer-note { display: none !important; }
  .app-main { padding-top: 0 !important; padding-bottom: 0 !important; }
  .stats-row { display: none !important; }
  .export-badge { display: none !important; }
  .report-subtitle { margin-top: 0.15rem; font-size: 8pt; }
  .report-header { margin-bottom: 0.4rem; padding-bottom: 0.35rem; }
  .program-head { margin-bottom: 0.35rem; }
  .program-name { font-size: 1rem; }
  .program-card { break-inside: auto; page-break-inside: auto; box-shadow: none; border-color: #aab2bd; margin-bottom: 0.3rem;
padding: 0.4rem; }
  body { font-size: 9pt; background: #fff; }
  .report-title {
    color: #000 !important;
    background: none !important;
    background-clip: initial !important;
    -webkit-background-clip: initial !important;
    font-size: 14pt;
    letter-spacing: 0;
  }
  .report-title i,
  .program-name i {
    display: none !important;
  }
  .table-shell { border-radius: 10px; }
  .report-table { font-size: 7.8pt; }
  .report-table th, .report-table td { padding: 0.2rem 0.24rem; line-height: 1.1; }
  .report-table th { color: #111; background: #fff; border-bottom: 1px solid #333; }
  .report-table td { color: #111; }
  .status-pill { padding: 0.08rem 0.25rem; font-size: 0.58rem; border-color: #666 !important; background: #fff !important; color: #111
!important; }
  .faculty-name i { display: none; }
  .empty-state { padding: 0.8rem 0.5rem; font-style: normal; }

  /* Remove any QR-related widgets/images from print to prevent extra pages */
  [id*="qr" i],
  [class*="qr" i],
  [data-qr],
  .qrcode, .qr-code, .qr-wrapper, .qr-section, .qr-container,
  img[src*="qr" i], img[alt*="qr" i], svg[id*="qr" i], canvas[id*="qr" i],
  iframe[src*="qr" i], object[data*="qr" i], embed[src*="qr" i],
  .print-hide-qr {
    display: none !important;
    visibility: hidden !important;
  }

  .print-single-program .program-card { display: none !important; }
  .print-single-program .program-card.selected-for-print { display: block !important; }
}

@media (max-width: 760px) {
  .filter-card {
    padding: 1rem;
  }
}
</style>
<div class="report-dashboard">
  <div class="report-header">
    <div>
      <h1 class="report-title"><i class="fa-solid fa-chalkboard-user me-2" style="color:#2f7c8a;"></i>Teaching load report</h1>
      <p class="report-subtitle">Western Philippines University · official academic load summary</p>
    </div>
    <button type="button" class="export-badge no-print" onclick="downloadExcelReport()">
      <i class="fa-solid fa-file-excel"></i>Download Excel
    </button>
  </div>

  <form method="get" class="filter-card no-print">
    <div class="row g-3 align-items-end">
      <div class="col-lg-2 col-md-4">
        <label class="filter-label"><i class="fa-solid fa-building-columns me-1"></i>College</label>
        <select name="college_id" class="form-select filter-control">

```

```

        <option value="0">All colleges</option>
        <?php foreach ($collegeOpts as $co): ?>
            <option value="<?= (int) $co['id'] ?>"<?= $adminCollegeId === (int) $co['id'] ? ' selected' : '' ?>>
                <?= htmlspecialchars((string) $co['college_code']) ?> - <?= htmlspecialchars((string) $co['college_name']) ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-lg-2 col-md-4">
    <label class="filter-label"><i class="fa-solid fa-calendar-days me-1"></i>Semester</label>
    <select name="semester" class="form-select filter-control">
        <option value="">(auto from data)</option>
        <?php foreach ($sems as $s): ?>
            <option value="<?= htmlspecialchars((string) $s) ?>"<?= $sem === (string) $s ? ' selected' : '' ?>><?=
htmlspecialchars((string) $s) ?></option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-lg-2 col-md-4">
    <label class="filter-label"><i class="fa-solid fa-graduation-cap me-1"></i>School year</label>
    <select name="school_year" class="form-select filter-control">
        <option value="">(auto from data)</option>
        <?php foreach ($years as $y): ?>
            <option value="<?= htmlspecialchars((string) $y) ?>"<?= $sy === (string) $y ? ' selected' : '' ?>><?=
htmlspecialchars((string) $y) ?></option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-lg-3 col-md-6">
    <label class="filter-label"><i class="fa-solid fa-laptop-code me-1"></i>Program</label>
    <select name="program_search" class="form-select filter-control">
        <option value="">All programs</option>
        <?php foreach ($programOpts as $program): ?>
            <option value="<?= htmlspecialchars((string) $program) ?>"<?= $programSearch === (string) $program ? ' selected' :
'' ?>>
                <?= htmlspecialchars((string) $program) ?>
            </option>
        <?php endforeach; ?>
    </select>
</div>
<div class="col-lg-3 col-md-6">
    <label class="filter-label"><i class="fa-solid fa-user-check me-1"></i>Employment status</label>
    <select name="employment_status" class="form-select filter-control"<?= $hasEmploymentStatusColumn ? '' : ' disabled' ?>>
        <option value="">All statuses</option>
        <?php foreach ($allowedEmploymentStatuses as $status): ?>
            <option value="<?= htmlspecialchars($status) ?>"<?= $employmentStatusFilter === $status ? ' selected' : '' ?>>
                <?= htmlspecialchars($status) ?>
            </option>
        <?php endforeach; ?>
    </select>
    <?php if (!$hasEmploymentStatusColumn): ?>
        <div class="form-text small text-warning">Run <a href="upgrade_roles.php">upgrade_roles.php</a> to enable this filter.
    </div>
    <?php endif; ?>
</div>
</div>
<div class="col-12 d-flex gap-2">
    <button type="submit" class="btn btn-primary btn-pill"><i class="fa-solid fa-filter me-1"></i>Apply filters</button>
    <a href="super_admin_teaching_load_report.php" class="btn btn-outline-secondary btn-pill"><i class="fa-solid fa-rotate-right
me-1"></i>Reset</a>
</div>
</div>
</form>

<?php if ($sem === '' && $sy === '' && $schedRows === []): ?>
    <div class="alert alert-warning">No schedules were found. Add schedules first or select a term.</div>
<?php else: ?>
    <?php if ($byProgram === []): ?>
        <div class="alert alert-info">
            No matching programs found<?= $programSearch !== '' ? ' for "' . htmlspecialchars($programSearch) . '"' : '' ?>.
        </div>
    <?php endif; ?>

    <?php foreach ($byProgram as $programName => $buckets): ?>
        <?php
            $allRows = array_merge($buckets['under'], $buckets['normal'], $buckets['over']);
            usort($allRows, static fn(array $a, array $b): int => strcasecmp((string) $a['name'], (string) $b['name']));
            $totalFaculty = count($allRows);
            $totalUnits = round(array_reduce($allRows, static fn(float $carry, array $row): float => $carry + (float) ($row['units']) ??
0), 0.0), 2);
            $totalLecUnits = round(array_reduce($allRows, static fn(float $carry, array $row): float => $carry + (float)

```



```

($row['lec_units'] ?? 0), 0.0), 2);
    $totalLabUnits = round(array_reduce($allRows, static fn(float $carry, array $row): float => $carry + (float)
($row['lab_units'] ?? 0), 0.0), 2);
    $totalHours = round(array_reduce($allRows, static fn(float $carry, array $row): float => $carry + (float) ($row['hours'] ??
0), 0.0), 2);
    ?>
    <section class="program-card">= $programSearch != '' && $programSearch == $programName ? ' selected-for-print' : '' ?> data-
program-name="<?>= htmlspecialchars($programName, ENT_QUOTES) ?>" aria-labelledby="prog-<?>= htmlspecialchars(md5($programName)) ?>">
    <div class="program-head">
        <h2 class="program-name" id="prog-<?>= htmlspecialchars(md5($programName)) ?>">
            <i class="fa-solid fa-graduation-cap me-2"></i><?>= htmlspecialchars($programName) ?>
        </h2>
    <div class="stats-row">
        <span class="stat-chip"><i class="fa-solid fa-users"></i> Faculty <span class="value"><?>= $totalFaculty ?></span>
    </span>
        <span class="stat-chip"><i class="fa-solid fa-book-open"></i> Lecture <span class="value"><?>=
$formatUnitCell($totalLecUnits) ?></span></span>
        <span class="stat-chip"><i class="fa-solid fa-flask"></i> Laboratory <span class="value"><?>=
$formatUnitCell($totalLabUnits) ?></span></span>
        <span class="stat-chip"><i class="fa-solid fa-book"></i> Units <span class="value"><?>= number_format($totalUnits, 2)
?></span></span>
        <span class="stat-chip"><i class="fa-solid fa-clock"></i> Hours/week <span class="value"><?>=
number_format($totalHours, 2) ?></span></span>
    </div>
</div>

<div class="table-shell">
    <table class="report-table">
        <thead>
            <tr>
                <th>Faculty</th>
                <th>Employment Status</th>
                <th>College</th>
                <th>Load Status</th>
                <th class="text-end">Lecture</th>
                <th class="text-end">Laboratory</th>
                <th class="text-end">Total Units</th>
                <th class="text-end">Lec h/wk</th>
                <th class="text-end">Lab h/wk</th>
                <th class="text-end">Hours/week</th>
                <th class="text-end">Total load hours</th>
            </tr>
        </thead>
        <tbody>
            <?php if ($allRows == []): ?>
                <tr><td colspan="11" class="empty-state">No faculty records found for this program.</td></tr>
            <?php else: ?>
                <?php foreach ($allRows as $row): ?>
                    <?php
                        $hours = (float) ($row['hours'] ?? 0);
                        $lecHours = (float) ($row['lec_hours'] ?? 0);
                        $labHours = (float) ($row['lab_hours'] ?? 0);
                        $lecUnits = (float) ($row['lec_units'] ?? 0);
                        $labUnits = (float) ($row['lab_units'] ?? 0);
                        $statusClass = $hours < $underMax ? 'status-under' : ($hours > $overMin ? 'status-over' : 'status-
normal');
                        $statusLabel = $hours < $underMax ? 'Under load' : ($hours > $overMin ? 'Over load' : 'Normal
load');
                        $employmentStatus = trim((string) ($row['employment_status'] ?? ''));
                        $employmentKey = strtolower($employmentStatus);
                        $weightedHours = in_array($employmentKey, ['permanent', 'temporary'], true)
                            ? number_format($weightedTeachingHours($lecHours, $labHours), 2)
                            : '-';
                    <?>
                    <tr>
                        <td class="faculty-name"><i class="fa-solid fa-user-graduate me-1" style="color:#568c9b;"></i><?>=
htmlspecialchars((string) $row['name']) ?></td>
                        <td><?>= htmlspecialchars((string) $row['employment_status']) ?></td>
                        <td><?>= htmlspecialchars((string) $row['college']) ?></td>
                        <td><span class="status-pill <?>= $statusClass ?>"><?>= $statusLabel ?></span></td>
                        <td class="text-end font-monospace"><?>= $formatUnitCell($lecUnits) ?></td>
                        <td class="text-end font-monospace"><?>= $formatUnitCell($labUnits) ?></td>
                        <td class="text-end font-monospace"><strong><?>= number_format((float) ($row['units'] ?? 0), 2) ?>
                    </strong></td>
                        <td class="text-end font-monospace"><?>= $lecHours > 0 ? number_format($lecHours, 2) : '-' ?></td>
                        <td class="text-end font-monospace"><?>= $labHours > 0 ? number_format($labHours, 2) : '-' ?></td>
                        <td class="text-end font-monospace"><?>= number_format($hours, 2) ?></td>
                        <td class="text-end font-monospace"><?>= $weightedHours ?></td>
                    </tr>
                <?php endforeach; ?>
            </tbody>
        </table>
    </div>

```

```

                <?php endif; ?>
            </tbody>
        </table>
    </div>
</section>
<?php endforeach; ?>
<?php endif; ?>

<div class="footer-note">
    <span><i class="fa-regular fa-note-sticky"></i> Load summary based on official teaching assignments. Total load hours: lecture x2 +
laboratory x1.5 (Permanent/Temporary only).</span>
    <span><i class="fa-solid fa-download"></i> Western Philippines University · Teaching Load System</span>
</div>
</div>

<script>
function downloadExcelReport() {
    const cards = Array.from(document.querySelectorAll('.program-card'));
    if (cards.length === 0) {
        window.alert('No report data available to export.');
```

return;

```
    }

    const parts = [];
    parts.push('<html xmlns:o="urn:schemas-microsoft-com:office:office" xmlns:x="urn:schemas-microsoft-com:office:excel"
xmlns="http://www.w3.org/TR/REC-html40">');
    parts.push('<head><meta charset="UTF-8"><style>');
    parts.push('table{border-collapse:collapse;width:100%;font-family:Arial,sans-serif;font-size:12px;});
    parts.push('th,td{border:1px solid #333;padding:6px;text-align:left;});
    parts.push('th{background:#f0f0f0;font-weight:700;});
    parts.push('h2{margin:14px 0 6px 0;font-family:Arial,sans-serif;font-size:14px;});
    parts.push('p{margin:0 0 6px 0;font-family:Arial,sans-serif;font-size:11px;color:#333;});
    parts.push('</style></head><body>');
    parts.push('<h2>Western Philippines University - Teaching Load Report</h2>');
    parts.push('<p>Generated: ' + new Date().toLocaleString() + '</p>');

    cards.forEach((card) => {
        const titleNode = card.querySelector('.program-name');
        const tableNode = card.querySelector('.report-table');
        if (!tableNode) {
            return;
        }
        const programTitle = titleNode ? titleNode.textContent.trim() : 'Program';
        parts.push('<h2>' + escapeHtml(programTitle) + '</h2>');
        parts.push(tableNode.outerHTML);
    });

    parts.push('</body></html>');
    const blob = new Blob([parts.join('')], { type: 'application/vnd.ms-excel;charset=utf-8;' });
    const fileName = 'teaching_load_report_' + new Date().toISOString().slice(0, 10) + '.xls';
    const link = document.createElement('a');
    link.href = URL.createObjectURL(blob);
    link.download = fileName;
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
    URL.revokeObjectURL(link.href);
}

function escapeHtml(value) {
    return String(value)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&#039;');
}
</script>

<?php require_once __DIR__ . '/includes/footer.php'; ?>

-----
FILE: sw-faculty-offline.js
-----
/* Faculty offline shell – cache queue page + assets for offline upload drafting. */
var CACHE_NAME = 'class-faculty-offline-v1';
var PRECACHE = [
    './faculty_offline.php',
    './assets/js/faculty_offline.js',

```

```

        './assets/css/style.css',
        './assets/wpu-logo.png',
        'https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css',
        'https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js',
        'https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css',
    ];

    self.addEventListener('install', function (event) {
        event.waitUntil(
            caches
                .open(CACHE_NAME)
                .then(function (cache) {
                    return Promise.all(
                        PRECACHE.map(function (url) {
                            return cache.add(url).catch(function () {
                                return null;
                            });
                        })
                    );
                })
                .then(function () {
                    return self.skipWaiting();
                })
        );
    });

    self.addEventListener('activate', function (event) {
        event.waitUntil(
            caches
                .keys()
                .then(function (keys) {
                    return Promise.all(
                        keys.map(function (key) {
                            if (key !== CACHE_NAME) {
                                return caches.delete(key);
                            }
                            return null;
                        })
                    );
                })
                .then(function () {
                    return self.clients.claim();
                })
        );
    });

    self.addEventListener('sync', function (event) {
        if (event.tag === 'faculty-offline-upload-sync') {
            event.waitUntil(
                self.clients.matchAll({ type: 'window', includeUncontrolled: true }).then(function (clients) {
                    clients.forEach(function (client) {
                        client.postMessage({ type: 'faculty-offline-flush' });
                    });
                })
            );
        }
    });

    self.addEventListener('fetch', function (event) {
        var req = event.request;
        if (req.method !== 'GET') {
            return;
        }

        var url = new URL(req.url);
        var path = url.pathname || '';
        var isOfflinePage = /\ faculty_offline\.php$/i.test(path);
        var isOfflineAsset =
            /\ assets\/js\/faculty_offline\.js$/i.test(path) ||
            /\ assets\/css\/style\.css$/i.test(path) ||
            /\ assets\/wpu-logo\.png$/i.test(path) ||
            /bootstrap@5\.3\.3/i.test(url.href) ||
            /font-awesome\/6\.5\.1/i.test(url.href);

        if (!isOfflinePage && !isOfflineAsset) {
            return;
        }

        event.respondWith(
            fetch(req)
        )
    });

```

```

.then(function (res) {
  if (res && res.ok) {
    var copy = res.clone();
    caches.open(CACHE_NAME).then(function (cache) {
      cache.put(req, copy);
    });
  }
  return res;
})
.catch(function () {
  return caches.match(req).then(function (cached) {
    if (cached) {
      return cached;
    }
    if (isOfflinePage) {
      return caches.match('./faculty_offline.php');
    }
    return Response.error();
  });
});
});
});

```

FILE: sw-student-offline.js

```

/* Student offline shell – cache reader page + assets for offline faculty-item reading. */
var CACHE_NAME = 'class-student-offline-v2';
var PRECACHE = [
  './student_offline.php',
  './assets/js/student_offline.js',
  './assets/css/style.css',
  './assets/wpu-logo.png',
  'https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css',
  'https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js',
  'https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css',
];

```

```

self.addEventListener('install', function (event) {
  event.waitUntil(
    caches
      .open(CACHE_NAME)
      .then(function (cache) {
        return Promise.all(
          PRECACHE.map(function (url) {
            return cache.add(url).catch(function () {
              return null;
            });
          })
        );
      })
      .then(function () {
        return self.skipWaiting();
      })
  );
});

```

```

self.addEventListener('activate', function (event) {
  event.waitUntil(
    caches
      .keys()
      .then(function (keys) {
        return Promise.all(
          keys.map(function (key) {
            if (key !== CACHE_NAME) {
              return caches.delete(key);
            }
            return null;
          })
        );
      })
      .then(function () {
        return self.clients.claim();
      })
  );
});

```

```

self.addEventListener('fetch', function (event) {
  var req = event.request;

```

```

if (req.method !== 'GET') {
    return;
}

var url = new URL(req.url);
var path = url.pathname || '';
var isOfflinePage = /\student_offline\.php$/i.test(path);
var isOfflineAsset =
    /\assets\/js\/student_offline\.js$/i.test(path) ||
    /\assets\/css\/style\.css$/i.test(path) ||
    /\assets\/wpu-logo\.png$/i.test(path) ||
    /bootstrap@5\.3\.3\/i.test(url.href) ||
    /font-awesome\/6\.5\.1\/i.test(url.href);

if (!isOfflinePage && !isOfflineAsset) {
    return;
}

event.respondWith(
    fetch(req)
        .then(function (res) {
            if (res && res.ok) {
                var copy = res.clone();
                caches.open(CACHE_NAME).then(function (cache) {
                    cache.put(req, copy);
                });
            }
            return res;
        })
        .catch(function () {
            return caches.match(req).then(function (cached) {
                if (cached) {
                    return cached;
                }
                if (isOfflinePage) {
                    return caches.match('./student_offline.php');
                }
                return Response.error();
            });
        })
);
});

```

```

-----
FILE: tools/seed_jomari_thursday.php
-----

```

```

<?php
declare(strict_types=1);

/**
 * Adds Thursday sample schedules for Jomari (faculty_id 10).
 * Run: c:\xampp\php\php.exe tools/seed_jomari_thursday.php
 */
require_once dirname(__DIR__) . '/includes/db.php';
require_once dirname(__DIR__) . '/includes/functions.php';

$pdo = db();
$facultyId = 10;
$collegeId = 2;
$createdBy = 2;
$semester = '1st Semester';
$schoolYear = '2026-2027';
$academicYear = '2026-2027';
$hasSessionKind = db_column_exists('schedules', 'session_kind');
$hasJoinCode = db_table_exists('online_classrooms') && db_column_exists('online_classrooms', 'join_code');

/** @var list<array{course_id:int,room_id:int,schedule_type:string,days:list<string>,start:string,end:string,meet_link:string}> */
$samples = [
    [
        'course_id' => 54, // CC104 - already assigned Sunday; separate Thursday section
        'room_id' => 11,
        'schedule_type' => 'Custom',
        'days' => ['Thursday'],
        'start' => '13:00:00',
        'end' => '15:00:00',
        'meet_link' => 'https://meet.google.com/jomari-thu-cc104',
    ],
];

```

```

$existsStmt = $pdo->prepare(
    'SELECT id FROM schedules
    WHERE faculty_id = ? AND course_id = ? AND day_of_week = ? AND start_time = ? AND end_time = ?
    AND semester = ? AND school_year = ?
    LIMIT 1'
);

$insertSchedule = $pdo->prepare(
    'INSERT INTO schedules (faculty_id, course_id, room_id, college_id, schedule_type, day_of_week, start_time, end_time, semester,
    school_year, academic_year, created_by'
    . ($hasSessionKind ? ', session_kind' : '')
    . ', online_class_url) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?'
    . ($hasSessionKind ? ', ?' : '')
    . ', ?)'
);

foreach ($samples as $row) {
    $daySet = days_to_set($row['days']);
    $existsStmt->execute([$facultyId, $row['course_id'], $daySet, $row['start'], $row['end'], $semester, $schoolYear]);
    if ($existsStmt->fetchColumn()) {
        echo "Already exists: {$daySet} {$row['start']}-{$row['end']}\n";
        continue;
    }

    $params = [
        $facultyId, $row['course_id'], $row['room_id'], $collegeId,
        $row['schedule_type'], $daySet, $row['start'], $row['end'],
        $semester, $schoolYear, $academicYear, $createdBy,
    ];
    if ($hasSessionKind) {
        $params[] = 'lecture';
    }
    $params[] = $row['meet_link'];

    $insertSchedule->execute($params);
    $scheduleId = (int) $pdo->lastInsertId();

    $course = $pdo->prepare('SELECT course_code, course_name FROM courses WHERE id = ?');
    $course->execute([$row['course_id']]);
    $c = $course->fetch(PDO::FETCH_ASSOC) ?: ['course_code' => '?', 'course_name' => '?'];
    $title = trim((string) $c['course_code']) . ' - ' . trim((string) $c['course_name']);

    if (db_table_exists('online_classrooms')) {
        $meetLink = $row['meet_link'];
        if ($hasJoinCode) {
            $joinCode = classroom_alloc_unique_join_code();
            $pdo->prepare(
                'INSERT INTO online_classrooms (schedule_id, faculty_id, course_id, title, description, meet_link, join_code)
                VALUES (?, ?, ?, ?, ?, ?, ?)'
            )->execute([$scheduleId, $facultyId, $row['course_id'], $title, null, $meetLink, $joinCode]);
        } else {
            $pdo->prepare(
                'INSERT INTO online_classrooms (schedule_id, faculty_id, course_id, title, description, meet_link)
                VALUES (?, ?, ?, ?, ?, ?)'
            )->execute([$scheduleId, $facultyId, $row['course_id'], $title, null, $meetLink]);
        }
    }

    echo "Inserted #{$scheduleId}: {$title} - {$daySet} {$row['start']}-{$row['end']}\n";
}

echo "\nLog in as Jomari → http://localhost/CLASS/faculty_schedule.php\n";

-----
FILE: tools/seed_thursday_schedule.php
-----
<?php
declare(strict_types=1);

/**
 * Inserts sample Thursday schedules for CCAS faculty (college_id 2).
 * Run: c:\xampp\php\php.exe tools/seed_thursday_schedule.php
 */
require_once dirname(__DIR__) . '/includes/db.php';
require_once dirname(__DIR__) . '/includes/functions.php';

$pdo = db();
$hasSessionKind = db_column_exists('schedules', 'session_kind');
$hasJoinCode = db_table_exists('online_classrooms') && db_column_exists('online_classrooms', 'join_code');

```

```

$semester = '1st Semester';
$schoolYear = '2026-2027';
$academicYear = '2026-2027';
$createdBy = 2; // dean user
$collegeId = 2;
$roomA = 10; // COLLEG-001
$roomB = 11; // COLLEG-002

/** @var
list<array{faculty_id:int,course_id:int,room_id:int,schedule_type:string,days:list<string>,start:string,end:string,meet_link?:string}> */
$samples = [
    ['faculty_id' => 8, 'course_id' => 53, 'room_id' => $roomA, 'schedule_type' => 'TTH', 'days' => ['Tuesday', 'Thursday'], 'start' =>
'08:00:00', 'end' => '09:30:00'],
    ['faculty_id' => 9, 'course_id' => 54, 'room_id' => $roomB, 'schedule_type' => 'TTH', 'days' => ['Tuesday', 'Thursday'], 'start' =>
'08:00:00', 'end' => '09:30:00'],
    ['faculty_id' => 10, 'course_id' => 55, 'room_id' => $roomA, 'schedule_type' => 'TTH', 'days' => ['Tuesday', 'Thursday'], 'start' =>
'10:30:00', 'end' => '12:00:00'],
    ['faculty_id' => 11, 'course_id' => 56, 'room_id' => $roomB, 'schedule_type' => 'TTH', 'days' => ['Tuesday', 'Thursday'], 'start' =>
'10:30:00', 'end' => '12:00:00'],
    ['faculty_id' => 12, 'course_id' => 57, 'room_id' => $roomA, 'schedule_type' => 'Custom', 'days' => ['Thursday'], 'start' => '13:00:00',
'end' => '14:30:00'],
    ['faculty_id' => 13, 'course_id' => 58, 'room_id' => $roomB, 'schedule_type' => 'Custom', 'days' => ['Thursday'], 'start' => '13:00:00',
'end' => '14:30:00'],
    // Active-now window on Thursdays for go-live / attendance testing (12:00-14:00 PHT)
    ['faculty_id' => 8, 'course_id' => 63, 'room_id' => $roomA, 'schedule_type' => 'Custom', 'days' => ['Thursday'], 'start' => '12:00:00',
'end' => '14:00:00', 'meet_link' => 'https://meet.google.com/sample-thu-elijah'],
    ['faculty_id' => 7, 'course_id' => 52, 'room_id' => $roomB, 'schedule_type' => 'Custom', 'days' => ['Thursday'], 'start' => '15:00:00',
'end' => '16:30:00'],
];

$insertSchedule = $pdo->prepare(
    'INSERT INTO schedules (faculty_id, course_id, room_id, college_id, schedule_type, day_of_week, start_time, end_time, semester,
school_year, academic_year, created_by'
    . ($hasSessionKind ? ', session_kind' : '')
    . ', online_class_url) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?'
    . ($hasSessionKind ? ', ?' : '')
    . ', ?)');

);

$existsStmt = $pdo->prepare(
    'SELECT id FROM schedules
    WHERE faculty_id = ? AND course_id = ? AND day_of_week = ? AND start_time = ? AND end_time = ?
    AND semester = ? AND school_year = ?
    LIMIT 1'
);

);

$inserted = 0;
$skipped = 0;

foreach ($samples as $row) {
    $daySet = days_to_set($row['days']);
    $existsStmt->execute([
        $row['faculty_id'],
        $row['course_id'],
        $daySet,
        $row['start'],
        $row['end'],
        $semester,
        $schoolYear,
    ]);
    if ($existsStmt->fetchColumn()) {
        $skipped++;
        echo "Skip (exists): faculty {$row['faculty_id']} course {$row['course_id']} {$daySet} {$row['start']}-{$row['end']}\n";
        continue;
    }

    $params = [
        $row['faculty_id'],
        $row['course_id'],
        $row['room_id'],
        $collegeId,
        $row['schedule_type'],
        $daySet,
        $row['start'],
        $row['end'],
        $semester,
        $schoolYear,
        $academicYear,
        $createdBy,
    ];

```

```

];
if ($hasSessionKind) {
    $params[] = 'lecture';
}
$params[] = $row['meet_link'] ?? null;

$insertSchedule->execute($params);
$scheduleId = (int) $pdo->lastInsertId();
$inserted++;

$course = $pdo->prepare('SELECT course_code, course_name FROM courses WHERE id = ?');
$course->execute([$row['course_id']]);
$courseRow = $course->fetch(PDO::FETCH_ASSOC) ?: ['course_code' => '?', 'course_name' => '?'];
$title = trim((string) $courseRow['course_code']) . ' - ' . trim((string) $courseRow['course_name']);

if (db_table_exists('online_classrooms')) {
    $chk = $pdo->prepare('SELECT COUNT(*) FROM online_classrooms WHERE schedule_id = ?');
    $chk->execute([$scheduleId]);
    if ((int) $chk->fetchColumn() === 0) {
        $meetLink = trim((string) ($row['meet_link'] ?? ''));
        if ($hasJoinCode) {
            $joinCode = classroom_alloc_unique_join_code();
            $pdo->prepare(
                'INSERT INTO online_classrooms (schedule_id, faculty_id, course_id, title, description, meet_link, join_code)
                VALUES (?, ?, ?, ?, ?, ?, ?)'
            )->execute([$scheduleId, $row['faculty_id'], $row['course_id'], $title, null, $meetLink, $joinCode]);
        } else {
            $pdo->prepare(
                'INSERT INTO online_classrooms (schedule_id, faculty_id, course_id, title, description, meet_link)
                VALUES (?, ?, ?, ?, ?, ?)'
            )->execute([$scheduleId, $row['faculty_id'], $row['course_id'], $title, null, $meetLink]);
        }
    }
}

echo "Inserted schedule #{$scheduleId}: faculty {$row['faculty_id']} {$title} - {$daySet} {$row['start']}-{$row['end']}\n";
}

```

```

echo "\nDone. Inserted {$inserted}, skipped {$skipped}.\n";
echo "Open http://localhost/CLASS/faculty_schedule.php (log in as a CCAS faculty user, e.g. Elijah).\n";

```

FILE: upgrade_roles.php

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/config/config.php';

function exec_safe(PDO $pdo, string $sql): void
{
    try {
        $pdo->exec($sql);
    } catch (Throwable $e) {
        // Ignore idempotent alter errors for already-existing columns/indexes.
        $msg = strtolower($e->getMessage());
        if (
            str_contains($msg, 'duplicate column')
            // Index already exists (do not match 1062 "Duplicate entry ... for key ...").
            || str_contains($msg, 'duplicate key name')
            // InnoDB: duplicate FK constraint identifier (errno 121) or duplicate FK definition.
            || str_contains($msg, 'errno: 121')
            || str_contains($msg, 'duplicate key on write or update')
            || str_contains($msg, 'duplicate foreign key')
            || str_contains($msg, 'already exists')
            || str_contains($msg, 'check that column/key exists')
            || str_contains($msg, 'check that it exists')
            || str_contains($msg, "can't drop index")
            || str_contains($msg, 'cannot drop')
            || str_contains($msg, 'unknown column')
            || str_contains($msg, "can't drop field")
        ) {
            return;
        }
        throw $e;
    }
}

$ok = '';

```



```

$err = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    try {
        $pdo = new PDO(
            sprintf('mysql:host=%s;port=%d;dbname=%s;charset=%s', DB_HOST, DB_PORT, DB_NAME, DB_CHARSET),
            DB_USER,
            DB_PASS,
            [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
        );

        $schema = file_get_contents(__DIR__ . '/install/schema.sql');
        if ($schema === false) {
            throw new RuntimeException('Unable to read install/schema.sql');
        }
        $schema = preg_replace('/--.*$/m', '', $schema);
        foreach (array_filter(array_map('trim', explode(';', (string) $schema))) as $stmt) {
            if ($stmt !== '') {
                exec_safe($pdo, $stmt);
            }
        }

        exec_safe(
            $pdo,
            "ALTER TABLE users MODIFY role ENUM('super_admin','admin','dean','program_chain','faculty','gened','student') NOT NULL DEFAULT
'faculty'");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN admin_log_title VARCHAR(120) NOT NULL DEFAULT ''");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN email VARCHAR(190) NOT NULL DEFAULT ''");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN assigned_program VARCHAR(120) NOT NULL DEFAULT ''");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN college_id INT NULL");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN is_active TINYINT(1) NOT NULL DEFAULT 1");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN last_login_at DATETIME NULL");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN last_seen_at DATETIME NULL");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN last_logout_at DATETIME NULL");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN profile_photo VARCHAR(80) NULL");
        exec_safe($pdo, "ALTER TABLE users ADD COLUMN must_change_password TINYINT(1) NOT NULL DEFAULT 0");
        // Legacy installs used fk_users_college; that identifier can collide globally (errno 121) on some servers.
        exec_safe($pdo, "ALTER TABLE users DROP FOREIGN KEY fk_users_college");
        exec_safe($pdo, "ALTER TABLE users ADD CONSTRAINT fk_sched_users_college_id FOREIGN KEY (college_id) REFERENCES colleges(id) ON
DELETE SET NULL");

        exec_safe($pdo, "ALTER TABLE faculty ADD COLUMN user_id INT NULL UNIQUE");
        exec_safe($pdo, "ALTER TABLE faculty ADD COLUMN college_id INT NULL");
        exec_safe($pdo, "ALTER TABLE faculty ADD COLUMN is_gened TINYINT(1) NOT NULL DEFAULT 0");
        exec_safe($pdo, "ALTER TABLE faculty ADD COLUMN employment_status ENUM('Permanent','Contract of Service','Temporary') NOT NULL
DEFAULT 'Permanent'");
        exec_safe($pdo, "ALTER TABLE faculty MODIFY COLUMN employment_status ENUM('Permanent','Contract of Service','Temporary') NOT NULL
DEFAULT 'Permanent'");
        exec_safe($pdo, "ALTER TABLE faculty ADD CONSTRAINT fk_faculty_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL");
        exec_safe($pdo, "ALTER TABLE faculty ADD CONSTRAINT fk_faculty_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE
SET NULL");

        exec_safe($pdo, "ALTER TABLE courses ADD COLUMN college_id INT NULL");
        exec_safe($pdo, "ALTER TABLE courses ADD COLUMN is_laboratory TINYINT(1) NOT NULL DEFAULT 0");
        exec_safe($pdo, "ALTER TABLE courses ADD COLUMN is_gened TINYINT(1) NOT NULL DEFAULT 0");
        exec_safe($pdo, "ALTER TABLE courses ADD COLUMN year_level VARCHAR(20) NOT NULL DEFAULT ''");
        exec_safe($pdo, "ALTER TABLE courses ADD COLUMN section VARCHAR(20) NOT NULL DEFAULT ''");
        exec_safe($pdo, "ALTER TABLE courses ADD COLUMN lecture_units DECIMAL(3,1) NOT NULL DEFAULT 3.0");
        exec_safe($pdo, "ALTER TABLE courses ADD COLUMN laboratory_units DECIMAL(3,1) NOT NULL DEFAULT 0.0");
        // Old schema used global UNIQUE(course_code); remove to allow per-scope codes.
        exec_safe($pdo, "ALTER TABLE courses DROP INDEX course_code");
        exec_safe($pdo, "ALTER TABLE courses DROP INDEX uq_course_code");
        exec_safe($pdo, "ALTER TABLE courses DROP INDEX uq_courses_course_code");
        $idxStmt = $pdo->prepare(
            "SELECT DISTINCT INDEX_NAME
            FROM information_schema.STATISTICS
            WHERE TABLE_SCHEMA = ?
              AND TABLE_NAME = 'courses'
              AND COLUMN_NAME = 'course_code'
              AND NON_UNIQUE = 0
              AND INDEX_NAME <> 'PRIMARY'"
        );
        $idxStmt->execute([DB_NAME]);
        $uniqueIndexes = $idxStmt->fetchAll(PDO::FETCH_COLUMN) ?: [];
        foreach ($uniqueIndexes as $idxName) {
            $safeIdx = str_replace('_', '', (string) $idxName);
            exec_safe($pdo, "ALTER TABLE courses DROP INDEX `{$safeIdx}`");
        }
        exec_safe($pdo, "ALTER TABLE courses ADD INDEX idx_course_code (course_code)");
        exec_safe($pdo, "UPDATE courses SET lecture_units = units WHERE lecture_units IS NULL OR lecture_units = 0");
    } catch (Exception $e) {
        $err = $e->getMessage();
    }
}

```

```

exec_safe($pdo, "UPDATE courses SET laboratory_units = units WHERE is_laboratory = 1 AND (laboratory_units IS NULL OR
laboratory_units = 0)");
exec_safe($pdo, "ALTER TABLE courses ADD CONSTRAINT fk_course_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE SET
NULL");
exec_safe($pdo, 'ALTER TABLE courses ADD COLUMN classroom_code VARCHAR(16) NULL');
exec_safe($pdo, 'ALTER TABLE courses ADD UNIQUE KEY uq_courses_classroom_code (classroom_code)');

exec_safe($pdo, "ALTER TABLE rooms ADD COLUMN college_id INT NULL");
exec_safe($pdo, "ALTER TABLE rooms ADD COLUMN is_genet TINYINT(1) NOT NULL DEFAULT 0");
exec_safe($pdo, "ALTER TABLE rooms MODIFY type ENUM('lecture','laboratory','conference','tba') NOT NULL DEFAULT 'lecture'");
exec_safe($pdo, "ALTER TABLE rooms MODIFY status ENUM('available','tba','maintenance') NOT NULL DEFAULT 'available'");
exec_safe($pdo, "ALTER TABLE rooms ADD CONSTRAINT fk_room_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE SET
NULL");

// Old schema used global UNIQUE(room_code); use scoped uniqueness (per college / Gen Ed).
exec_safe($pdo, 'ALTER TABLE rooms DROP INDEX room_code');
exec_safe($pdo, 'ALTER TABLE rooms DROP INDEX uq_rooms_scope');
$roomIdxStmt = $pdo->prepare(
    "SELECT DISTINCT INDEX_NAME
    FROM information_schema.STATISTICS
    WHERE TABLE_SCHEMA = ?
    AND TABLE_NAME = 'rooms'
    AND COLUMN_NAME = 'room_code'
    AND NON_UNIQUE = 0
    AND INDEX_NAME <> 'PRIMARY'"
);
$roomIdxStmt->execute([DB_NAME]);
$roomUniqueIndexes = $roomIdxStmt->fetchAll(PDO::FETCH_COLUMN) ?: [];
foreach ($roomUniqueIndexes as $roomIdxName) {
    $safeRoomIdx = str_replace('`', '``', (string) $roomIdxName);
    exec_safe($pdo, "ALTER TABLE rooms DROP INDEX ``{$safeRoomIdx}`");
}
exec_safe($pdo, 'ALTER TABLE rooms ADD INDEX idx_room_code (room_code)');
exec_safe(
    $pdo,
    "ALTER TABLE rooms ADD COLUMN room_code_scope VARCHAR(64) GENERATED ALWAYS AS (
        IF(COALESCE(is_genet,0) = 1,
        CONCAT('G|', room_code),
        CONCAT('C|', IFNULL(college_id, 0), '|', room_code))
    ) STORED"
);
exec_safe($pdo, 'ALTER TABLE rooms ADD UNIQUE KEY uq_rooms_scope (room_code_scope)');

exec_safe($pdo, "ALTER TABLE schedules ADD COLUMN college_id INT NULL");
exec_safe($pdo, "ALTER TABLE schedules ADD COLUMN online_class_url VARCHAR(512) NULL DEFAULT NULL");
exec_safe($pdo, "ALTER TABLE schedules ADD COLUMN online_live_at DATETIME NULL DEFAULT NULL");
exec_safe($pdo, "ALTER TABLE schedules ADD CONSTRAINT fk_sched_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE
SET NULL");
exec_safe($pdo, "ALTER TABLE schedules ADD COLUMN program VARCHAR(100) NOT NULL DEFAULT ''");
exec_safe($pdo, "ALTER TABLE schedules ADD COLUMN year_level VARCHAR(20) NOT NULL DEFAULT ''");
exec_safe($pdo, "ALTER TABLE schedules ADD COLUMN section VARCHAR(20) NOT NULL DEFAULT ''");
exec_safe($pdo, "ALTER TABLE schedules ADD COLUMN session_kind ENUM('lecture','laboratory') NULL DEFAULT NULL");
exec_safe($pdo, "ALTER TABLE schedules ADD COLUMN is_makeup TINYINT(1) NOT NULL DEFAULT 0");
exec_safe($pdo, 'ALTER TABLE schedules ADD COLUMN makeup_for_schedule_id INT NULL');
exec_safe(
    $pdo,
    "ALTER TABLE schedule_change_requests
    ADD COLUMN request_type ENUM('change','makeup') NOT NULL DEFAULT 'change'"
);
exec_safe(
    $pdo,
    "ALTER TABLE schedule_change_requests
    ADD COLUMN proposed_day_of_week SET('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday') NULL"
);
exec_safe($pdo, 'ALTER TABLE schedule_change_requests ADD COLUMN proposed_start_time TIME NULL');
exec_safe($pdo, 'ALTER TABLE schedule_change_requests ADD COLUMN proposed_end_time TIME NULL');
exec_safe($pdo, 'ALTER TABLE schedule_change_requests ADD COLUMN proposed_room_id INT NULL');
exec_safe(
    $pdo,
    "UPDATE schedules s
    INNER JOIN courses c ON c.id = s.course_id
    SET s.session_kind = CASE
        WHEN TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time) BETWEEN 180 AND 240 THEN 'laboratory'
        ELSE 'lecture'
    END
    WHERE c.is_laboratory = 1 AND s.session_kind IS NULL"
);
// Pure lecture courses: never leave session_kind null (avoids 3h blocks showing as Laboratory).
exec_safe(
    $pdo,

```

```

"UPDATE schedules s
INNER JOIN courses c ON c.id = s.course_id
SET s.session_kind = 'lecture'
WHERE COALESCE(c.is_laboratory, 0) = 0 AND s.session_kind IS NULL"
);

exec_safe($pdo, "ALTER TABLE conflict_requests ADD COLUMN program VARCHAR(100) NOT NULL DEFAULT ''");
exec_safe($pdo, "ALTER TABLE conflict_requests ADD COLUMN year_level VARCHAR(20) NOT NULL DEFAULT ''");
exec_safe($pdo, "ALTER TABLE conflict_requests ADD COLUMN section VARCHAR(20) NOT NULL DEFAULT ''");

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS programs (
        id INT AUTO_INCREMENT PRIMARY KEY,
        college_id INT NOT NULL,
        program_name VARCHAR(120) NOT NULL,
        status ENUM('active','inactive') NOT NULL DEFAULT 'active',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        UNIQUE KEY uq_program_college_name (college_id, program_name),
        CONSTRAINT fk_program_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS program_chair_programs (
        user_id INT NOT NULL,
        program_name VARCHAR(120) NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (user_id, program_name),
        CONSTRAINT fk_pcp_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

try {
    $pdo->exec(
        "INSERT IGNORE INTO program_chair_programs (user_id, program_name)
        SELECT id, assigned_program
        FROM users
        WHERE role = 'program_chair'
        AND TRIM(assigned_program) <> ''"
    );
} catch (Throwable $e) {
    // Ignore when column/table not ready yet on very old installs.
}

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS programs_year_levels (
        program_id INT NOT NULL,
        year_level VARCHAR(20) NOT NULL,
        PRIMARY KEY (program_id, year_level),
        CONSTRAINT fk_prog_y1_program FOREIGN KEY (program_id) REFERENCES programs(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS college_programs (
        id INT AUTO_INCREMENT PRIMARY KEY,
        college_id INT NOT NULL,
        program_code VARCHAR(30) NOT NULL,
        program_name VARCHAR(120) NOT NULL DEFAULT '',
        status ENUM('active','inactive') NOT NULL DEFAULT 'active',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        UNIQUE KEY uq_college_program_code (college_id, program_code),
        CONSTRAINT fk_cp_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS program_year_levels (
        id INT AUTO_INCREMENT PRIMARY KEY,
        program_id INT NOT NULL,
        year_level VARCHAR(20) NOT NULL,
        status ENUM('active','inactive') NOT NULL DEFAULT 'active',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        UNIQUE KEY uq_program_year (program_id, year_level),
        CONSTRAINT fk_pyl_program FOREIGN KEY (program_id) REFERENCES college_programs(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

```

```

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS program_sections (
        id INT AUTO_INCREMENT PRIMARY KEY,
        year_level_id INT NOT NULL,
        section_name VARCHAR(20) NOT NULL,
        status ENUM('active','inactive') NOT NULL DEFAULT 'active',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        UNIQUE KEY uq_year_section (year_level_id, section_name),
        CONSTRAINT fk_ps_year FOREIGN KEY (year_level_id) REFERENCES program_year_levels(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS faculty_specializations (
        id INT AUTO_INCREMENT PRIMARY KEY,
        faculty_id INT NOT NULL,
        course_id INT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        UNIQUE KEY uq_faculty_course (faculty_id, course_id),
        CONSTRAINT fk_fs_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE,
        CONSTRAINT fk_fs_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS faculty_course_colors (
        id INT AUTO_INCREMENT PRIMARY KEY,
        faculty_id INT NOT NULL,
        course_id INT NOT NULL,
        color_index TINYINT UNSIGNED NOT NULL DEFAULT 0,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        UNIQUE KEY uq_fcc_faculty_course (faculty_id, course_id),
        CONSTRAINT fk_fcc_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE,
        CONSTRAINT fk_fcc_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS ge_course_colleges (
        id INT AUTO_INCREMENT PRIMARY KEY,
        course_id INT NOT NULL,
        college_id INT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        UNIQUE KEY uq_ge_course_college (course_id, college_id),
        CONSTRAINT fk_gecc_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE,
        CONSTRAINT fk_gecc_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS ge_schedule_targets (
        id INT AUTO_INCREMENT PRIMARY KEY,
        schedule_id INT NOT NULL UNIQUE,
        college_id INT NOT NULL,
        program_name VARCHAR(120) NOT NULL,
        year_level VARCHAR(20) NOT NULL DEFAULT '',
        section VARCHAR(20) NOT NULL DEFAULT '',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        KEY idx_ge_target_scope (college_id, program_name, year_level, section),
        CONSTRAINT fk_gst_schedule FOREIGN KEY (schedule_id) REFERENCES schedules(id) ON DELETE CASCADE,
        CONSTRAINT fk_gst_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS internal_messages (
        id INT AUTO_INCREMENT PRIMARY KEY,
        sender_user_id INT NOT NULL,
        recipient_user_id INT NOT NULL,
        subject VARCHAR(255) NOT NULL DEFAULT '',
        body TEXT NOT NULL,
        is_memo TINYINT(1) NOT NULL DEFAULT 0,
        attachment_original_name VARCHAR(255) NOT NULL DEFAULT '',
        attachment_stored_name VARCHAR(255) NOT NULL DEFAULT '',
        attachment_mime VARCHAR(120) NOT NULL DEFAULT '',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        read_at TIMESTAMP NULL DEFAULT NULL,
        KEY idx_im_recipient_unread (recipient_user_id, read_at),

```

```

        KEY idx_im_pair_time (sender_user_id, recipient_user_id, created_at),
        CONSTRAINT fk_im_sender FOREIGN KEY (sender_user_id) REFERENCES users(id) ON DELETE CASCADE,
        CONSTRAINT fk_im_recipient FOREIGN KEY (recipient_user_id) REFERENCES users(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe($pdo, "ALTER TABLE internal_messages ADD COLUMN subject VARCHAR(255) NOT NULL DEFAULT '');"
exec_safe($pdo, "ALTER TABLE internal_messages ADD COLUMN is_memo TINYINT(1) NOT NULL DEFAULT 0");
exec_safe($pdo, "ALTER TABLE internal_messages ADD COLUMN attachment_original_name VARCHAR(255) NOT NULL DEFAULT '');"
exec_safe($pdo, "ALTER TABLE internal_messages ADD COLUMN attachment_stored_name VARCHAR(255) NOT NULL DEFAULT '');"
exec_safe($pdo, "ALTER TABLE internal_messages ADD COLUMN attachment_mime VARCHAR(120) NOT NULL DEFAULT '');"

exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS online_classrooms (
        id INT AUTO_INCREMENT PRIMARY KEY,
        schedule_id INT NOT NULL UNIQUE,
        faculty_id INT NOT NULL,
        course_id INT NOT NULL,
        title VARCHAR(150) NOT NULL,
        description TEXT NULL,
        meet_link VARCHAR(512) NOT NULL DEFAULT '',
        join_code VARCHAR(16) NULL,
        status ENUM('active','archived') NOT NULL DEFAULT 'active',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        UNIQUE KEY uq_oc_join_code (join_code),
        CONSTRAINT fk_oc_schedule FOREIGN KEY (schedule_id) REFERENCES schedules(id) ON DELETE CASCADE,
        CONSTRAINT fk_oc_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE,
        CONSTRAINT fk_oc_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_students (
        id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT NOT NULL UNIQUE,
        student_number VARCHAR(30) NOT NULL DEFAULT '',
        full_name VARCHAR(120) NOT NULL,
        email VARCHAR(190) NOT NULL DEFAULT '',
        year_level VARCHAR(20) NOT NULL DEFAULT '',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        CONSTRAINT fk_cls_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe($pdo, "ALTER TABLE classroom_students ADD COLUMN user_id INT NOT NULL UNIQUE");
exec_safe($pdo, "ALTER TABLE classroom_students ADD COLUMN year_level VARCHAR(20) NOT NULL DEFAULT '');"
exec_safe($pdo, "ALTER TABLE classroom_students ADD CONSTRAINT fk_cls_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET
NULL");
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS student_registration_requests (
        id INT AUTO_INCREMENT PRIMARY KEY,
        username VARCHAR(50) NOT NULL,
        password VARCHAR(255) NOT NULL,
        full_name VARCHAR(120) NOT NULL,
        email VARCHAR(190) NOT NULL DEFAULT '',
        student_number VARCHAR(30) NOT NULL DEFAULT '',
        college_id INT NOT NULL,
        program_name VARCHAR(120) NOT NULL,
        year_level VARCHAR(20) NOT NULL DEFAULT '',
        status ENUM('pending','approved','rejected') NOT NULL DEFAULT 'pending',
        reviewed_by_user_id INT NULL,
        reviewed_at DATETIME NULL,
        rejection_reason VARCHAR(500) NOT NULL DEFAULT '',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_srr_pending_scope (status, college_id, program_name),
        INDEX idx_srr_username (username),
        CONSTRAINT fk_srr_college FOREIGN KEY (college_id) REFERENCES colleges(id) ON DELETE CASCADE,
        CONSTRAINT fk_srr_reviewer FOREIGN KEY (reviewed_by_user_id) REFERENCES users(id) ON DELETE SET NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe($pdo, "ALTER TABLE student_registration_requests ADD COLUMN year_level VARCHAR(20) NOT NULL DEFAULT '');"
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_enrollments (
        id INT AUTO_INCREMENT PRIMARY KEY,
        classroom_id INT NOT NULL,
        student_id INT NOT NULL,
        enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        UNIQUE KEY uq_classroom_student (classroom_id, student_id),

```

```

        CONSTRAINT fk_ce_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
        CONSTRAINT fk_ce_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_content (
        id INT AUTO_INCREMENT PRIMARY KEY,
        classroom_id INT NOT NULL,
        faculty_id INT NOT NULL,
        content_type ENUM('material','link','announcement') NOT NULL DEFAULT 'material',
        title VARCHAR(150) NOT NULL,
        body TEXT NULL,
        weeks VARCHAR(100) NOT NULL DEFAULT '',
        days_per_topic VARCHAR(100) NOT NULL DEFAULT '',
        resource_url VARCHAR(512) NOT NULL DEFAULT '',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        CONSTRAINT fk_cc_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
        CONSTRAINT fk_cc_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe($pdo, "ALTER TABLE classroom_content ADD COLUMN weeks VARCHAR(100) NOT NULL DEFAULT ''");
exec_safe($pdo, "ALTER TABLE classroom_content ADD COLUMN days_per_topic VARCHAR(100) NOT NULL DEFAULT ''");
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_messages (
        id INT AUTO_INCREMENT PRIMARY KEY,
        classroom_id INT NOT NULL,
        sender_user_id INT NOT NULL,
        body TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        KEY idx_cm_classroom_time (classroom_id, created_at),
        CONSTRAINT fk_cm_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
        CONSTRAINT fk_cm_sender FOREIGN KEY (sender_user_id) REFERENCES users(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_content_attachments (
        id INT AUTO_INCREMENT PRIMARY KEY,
        content_id INT NOT NULL,
        original_name VARCHAR(255) NOT NULL DEFAULT '',
        stored_name VARCHAR(255) NOT NULL DEFAULT '',
        mime VARCHAR(120) NOT NULL DEFAULT '',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        KEY idx_cca_content (content_id),
        CONSTRAINT fk_cca_content FOREIGN KEY (content_id) REFERENCES classroom_content(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_assessments (
        id INT AUTO_INCREMENT PRIMARY KEY,
        classroom_id INT NOT NULL,
        faculty_id INT NOT NULL,
        assessment_type ENUM('multiple_choice','true_false','essay','problem_solving') NOT NULL DEFAULT 'essay',
        title VARCHAR(150) NOT NULL,
        description TEXT NULL,
        total_points DECIMAL(8,2) NOT NULL DEFAULT 100.00,
        due_at DATETIME NULL,
        time_limit_minutes INT UNSIGNED NULL DEFAULT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        CONSTRAINT fk_ca_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
        CONSTRAINT fk_ca_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "UPDATE classroom_assessments
    SET assessment_type = CASE assessment_type
        WHEN 'written_work' THEN 'essay'
        WHEN 'assignment' THEN 'essay'
        WHEN 'performance_task' THEN 'problem_solving'
        WHEN 'quiz' THEN 'multiple_choice'
        ELSE assessment_type
    END"
);
exec_safe($pdo, "ALTER TABLE classroom_assessments MODIFY COLUMN assessment_type
ENUM('multiple_choice','true_false','essay','problem_solving') NOT NULL DEFAULT 'essay'");
exec_safe($pdo, "ALTER TABLE classroom_assessments ADD COLUMN credited_week VARCHAR(100) NOT NULL DEFAULT ''");

```

```

exec_safe($pdo, "ALTER TABLE classroom_assessments ADD COLUMN time_limit_minutes INT UNSIGNED NULL DEFAULT NULL");
exec_safe($pdo, "ALTER TABLE online_classrooms ADD COLUMN written_work_percentage DECIMAL(5,2) NOT NULL DEFAULT 50.00");
exec_safe($pdo, "ALTER TABLE online_classrooms ADD COLUMN performance_task_percentage DECIMAL(5,2) NOT NULL DEFAULT 50.00");
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_scores (
        id INT AUTO_INCREMENT PRIMARY KEY,
        assessment_id INT NOT NULL,
        student_id INT NOT NULL,
        score DECIMAL(8,2) NULL,
        feedback TEXT NULL,
        graded_at DATETIME NULL,
        UNIQUE KEY uq_assessment_student (assessment_id, student_id),
        CONSTRAINT fk_cs_assessment FOREIGN KEY (assessment_id) REFERENCES classroom_assessments(id) ON DELETE CASCADE,
        CONSTRAINT fk_cs_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_submissions (
        id INT AUTO_INCREMENT PRIMARY KEY,
        assessment_id INT NOT NULL,
        student_id INT NOT NULL,
        answer_text LONGTEXT NULL,
        status ENUM('submitted','resubmitted') NOT NULL DEFAULT 'submitted',
        auto_total_score DECIMAL(8,2) NULL,
        requires_manual_grade TINYINT(1) NOT NULL DEFAULT 1,
        submitted_at DATETIME NOT NULL,
        updated_at DATETIME NOT NULL,
        UNIQUE KEY uq_submission_assessment_student (assessment_id, student_id),
        CONSTRAINT fk_csub_assessment FOREIGN KEY (assessment_id) REFERENCES classroom_assessments(id) ON DELETE CASCADE,
        CONSTRAINT fk_csub_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe($pdo, "ALTER TABLE classroom_submissions ADD COLUMN auto_total_score DECIMAL(8,2) NULL");
exec_safe($pdo, "ALTER TABLE classroom_submissions ADD COLUMN requires_manual_grade TINYINT(1) NOT NULL DEFAULT 1");
exec_safe($pdo, "ALTER TABLE classroom_submissions ADD COLUMN integrity_locked TINYINT(1) NOT NULL DEFAULT 0");
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS assessment_retake_requests (
        id INT AUTO_INCREMENT PRIMARY KEY,
        classroom_id INT NOT NULL,
        assessment_id INT NOT NULL,
        student_id INT NOT NULL,
        submission_id INT NULL,
        reason TEXT NOT NULL,
        status ENUM('pending','approved','rejected') NOT NULL DEFAULT 'pending',
        reviewed_by_user_id INT NULL,
        reviewed_at DATETIME NULL,
        faculty_remarks VARCHAR(500) NOT NULL DEFAULT '',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_arr_pending_classroom (status, classroom_id),
        INDEX idx_arr_student_assessment (student_id, assessment_id),
        CONSTRAINT fk_arr_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
        CONSTRAINT fk_arr_assessment FOREIGN KEY (assessment_id) REFERENCES classroom_assessments(id) ON DELETE CASCADE,
        CONSTRAINT fk_arr_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_assessment_questions (
        id INT AUTO_INCREMENT PRIMARY KEY,
        assessment_id INT NOT NULL,
        question_type ENUM('multiple_choice','true_false','essay','problem_solving') NOT NULL DEFAULT 'essay',
        question_text LONGTEXT NOT NULL,
        options_json LONGTEXT NULL,
        answer_key VARCHAR(255) NULL,
        points DECIMAL(8,2) NOT NULL DEFAULT 1.00,
        position INT NOT NULL DEFAULT 1,
        word_limit INT NULL,
        char_limit INT NULL,
        allow_steps TINYINT(1) NOT NULL DEFAULT 0,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        KEY idx_caq_assessment_pos (assessment_id, position),
        CONSTRAINT fk_caq_assessment FOREIGN KEY (assessment_id) REFERENCES classroom_assessments(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_submission_question_answers (

```

```

        id INT AUTO_INCREMENT PRIMARY KEY,
        submission_id INT NOT NULL,
        question_id INT NOT NULL,
        answer_text LONGTEXT NULL,
        answer_steps LONGTEXT NULL,
        is_correct TINYINT(1) NULL,
        auto_score DECIMAL(8,2) NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        UNIQUE KEY uq_csqa_submission_question (submission_id, question_id),
        KEY idx_csqa_question (question_id),
        CONSTRAINT fk_csqa_submission FOREIGN KEY (submission_id) REFERENCES classroom_submissions(id) ON DELETE CASCADE,
        CONSTRAINT fk_csqa_question FOREIGN KEY (question_id) REFERENCES classroom_assessment_questions(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_attendance_sessions (
        id INT AUTO_INCREMENT PRIMARY KEY,
        classroom_id INT NOT NULL,
        faculty_id INT NOT NULL,
        attendance_date DATE NOT NULL,
        session_start_at DATETIME NOT NULL,
        session_end_at DATETIME NOT NULL,
        source ENUM('auto_login_online') NOT NULL DEFAULT 'auto_login_online',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        UNIQUE KEY uq_classroom_attendance_day (classroom_id, attendance_date),
        CONSTRAINT fk_cas_classroom FOREIGN KEY (classroom_id) REFERENCES online_classrooms(id) ON DELETE CASCADE,
        CONSTRAINT fk_cas_faculty FOREIGN KEY (faculty_id) REFERENCES faculty(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe(
    $pdo,
    "CREATE TABLE IF NOT EXISTS classroom_attendance_records (
        id INT AUTO_INCREMENT PRIMARY KEY,
        session_id INT NOT NULL,
        student_id INT NOT NULL,
        status ENUM('present','absent') NOT NULL DEFAULT 'absent',
        source ENUM('auto','manual') NOT NULL DEFAULT 'auto',
        evidence_login_at DATETIME NULL,
        evidence_seen_at DATETIME NULL,
        evidence_logout_at DATETIME NULL,
        checked_at DATETIME NOT NULL,
        notes VARCHAR(255) NOT NULL DEFAULT '',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        UNIQUE KEY uq_attendance_session_student (session_id, student_id),
        KEY idx_attendance_status (session_id, status),
        CONSTRAINT fk_car_session FOREIGN KEY (session_id) REFERENCES classroom_attendance_sessions(id) ON DELETE CASCADE,
        CONSTRAINT fk_car_student FOREIGN KEY (student_id) REFERENCES classroom_students(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
);
exec_safe($pdo, "ALTER TABLE classroom_attendance_records ADD COLUMN evidence_logout_at DATETIME NULL");

exec_safe($pdo, 'ALTER TABLE online_classrooms ADD COLUMN join_code VARCHAR(16) NULL');
exec_safe($pdo, 'ALTER TABLE online_classrooms ADD UNIQUE KEY uq_oc_join_code (join_code)');
exec_safe($pdo, 'ALTER TABLE online_classrooms ADD COLUMN syllabus_stored_name VARCHAR(255) NULL');
exec_safe($pdo, 'ALTER TABLE online_classrooms ADD COLUMN syllabus_original_name VARCHAR(255) NULL');
exec_safe($pdo, 'ALTER TABLE online_classrooms ADD COLUMN syllabus_mime VARCHAR(120) NULL');
exec_safe($pdo, 'ALTER TABLE online_classrooms ADD COLUMN banner_stored_name VARCHAR(255) NULL');
exec_safe($pdo, 'ALTER TABLE online_classrooms ADD COLUMN banner_original_name VARCHAR(255) NULL');
exec_safe($pdo, 'ALTER TABLE online_classrooms ADD COLUMN banner_mime VARCHAR(120) NULL');

$superAdminId = (int) $pdo->query("SELECT id FROM users WHERE role='super_admin' ORDER BY id LIMIT 1")->fetchColumn();
if ($superAdminId < 1) {
    $st = $pdo->prepare('INSERT INTO users (username,password,full_name,role,is_active) VALUES (?,?,,?,1)');
    $st->execute([
        DEFAULT_SUPER_ADMIN_USERNAME,
        password_hash(DEFAULT_SUPER_ADMIN_PASSWORD, PASSWORD_DEFAULT),
        DEFAULT_SUPER_ADMIN_FULL_NAME,
        'super_admin',
    ]);
}

exec_safe($pdo, "ALTER TABLE admin_activity_logs ADD COLUMN actor_full_name VARCHAR(100) NOT NULL DEFAULT ''");
exec_safe($pdo, "ALTER TABLE admin_activity_logs ADD COLUMN actor_log_title VARCHAR(120) NOT NULL DEFAULT ''");

$adminId = (int) $pdo->query("SELECT id FROM users WHERE role='admin' ORDER BY id LIMIT 1")->fetchColumn();
if ($adminId < 1) {

```



```

        $st = $pdo->prepare('INSERT INTO users (username,password,full_name,role,is_active) VALUES (?,?,,?,1)');
        $st->execute([DEFAULT_ADMIN_USERNAME, password_hash(DEFAULT_ADMIN_PASSWORD, PASSWORD_DEFAULT), DEFAULT_ADMIN_FULL_NAME,
'admin']));
    }

    $genedId = (int) $pdo->query("SELECT id FROM users WHERE role='gened' ORDER BY id LIMIT 1")->fetchColumn();
    if ($genedId < 1) {
        $st = $pdo->prepare('INSERT INTO users (username,password,full_name,role,is_active) VALUES (?,?,,?,1)');
        $st->execute([DEFAULT_GENED_USERNAME, password_hash(DEFAULT_GENED_PASSWORD, PASSWORD_DEFAULT), DEFAULT_GENED_FULL_NAME,
'gened']));
    }

    $joinCodeCol = $pdo->prepare(
        'SELECT COUNT(*) FROM information_schema.COLUMNS
        WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ? AND COLUMN_NAME = ?'
    );
    $joinCodeCol->execute([DB_NAME, 'online_classrooms', 'join_code']);
    if ((int) $joinCodeCol->fetchColumn() > 0) {
        $idsStmt = $pdo->query("SELECT id FROM online_classrooms WHERE join_code IS NULL OR join_code = '');
        $missingCodes = $idsStmt->fetchAll(PDO::FETCH_COLUMN) : [];
        $chars = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
        foreach ($missingCodes as $oid) {
            $oid = (int) $oid;
            for ($t = 0; $t < 50; $t++) {
                $code = '';
                for ($i = 0; $i < 8; $i++) {
                    $code .= $chars[random_int(0, strlen($chars) - 1)];
                }
                $c = $pdo->prepare('SELECT COUNT(*) FROM online_classrooms WHERE join_code = ?');
                $c->execute([$code]);
                if ((int) $c->fetchColumn() === 0) {
                    $u = $pdo->prepare('UPDATE online_classrooms SET join_code = ? WHERE id = ?');
                    $u->execute([$code, $oid]);
                    break;
                }
            }
        }
    }

    exec_safe(
        $pdo,
        "CREATE TABLE IF NOT EXISTS login_slideshow_images (
            id INT AUTO_INCREMENT PRIMARY KEY,
            stored_name VARCHAR(120) NOT NULL,
            caption VARCHAR(255) NOT NULL DEFAULT '',
            sort_order INT NOT NULL DEFAULT 0,
            is_active TINYINT(1) NOT NULL DEFAULT 1,
            uploaded_by INT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            KEY idx_login_slide_order (sort_order, id),
            CONSTRAINT fk_login_slide_uploader FOREIGN KEY (uploaded_by) REFERENCES users(id) ON DELETE SET NULL
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci"
    );

    // Remove campuses feature: drop FKs/columns/table and restore pre-campus room_code_scope (idempotent).
    exec_safe($pdo, 'ALTER TABLE schedules DROP FOREIGN KEY fk_sched_campus');
    exec_safe($pdo, 'ALTER TABLE schedules DROP COLUMN campus_id');
    exec_safe($pdo, 'ALTER TABLE rooms DROP INDEX uq_rooms_scope');
    exec_safe($pdo, 'ALTER TABLE rooms DROP COLUMN room_code_scope');
    exec_safe($pdo, 'ALTER TABLE rooms DROP FOREIGN KEY fk_room_campus');
    exec_safe($pdo, 'ALTER TABLE rooms DROP INDEX idx_rooms_campus');
    exec_safe($pdo, 'ALTER TABLE rooms DROP COLUMN campus_id');
    exec_safe(
        $pdo,
        "ALTER TABLE rooms ADD COLUMN room_code_scope VARCHAR(64) GENERATED ALWAYS AS (
            IF(COALESCE(is_gened,0) = 1,
            CONCAT('G|', room_code),
            CONCAT('C|', IFNULL(college_id, 0), '|', room_code))
        ) STORED"
    );
    exec_safe($pdo, 'ALTER TABLE rooms ADD UNIQUE KEY uq_rooms_scope (room_code_scope)');
    exec_safe($pdo, 'DROP TABLE IF EXISTS campuses');

    $ok = 'Role upgrade complete. You can now use admin/dean/faculty/gened workflows.';
} catch (Throwable $e) {
    $err = $e->getMessage();
}
}
?>
<!doctype html>

```

```

<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Upgrade Roles</title>
  <?php require __DIR__ . '/includes/favicon.php'; ?>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
  <link href="https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.5.1/css/all.min.css" rel="stylesheet">
</head>
<body class="bg-light">
<div class="container py-5" style="max-width: 700px;">
  <h1 class="h4 mb-3">Database Role Upgrade</h1>
  <p class="text-muted">Runs idempotent schema updates for admin/dean/faculty role support.</p>
  <?php if ($ok): ?>
    <div class="alert alert-success"><?php echo htmlspecialchars($ok) ?></div>
    <div class="card border-primary shadow-sm mb-4">
      <div class="card-header bg-primary text-white">
        <strong><i class="fa-solid fa-user-secret me-2" aria-hidden="true"></i>Super Admin – first-time sign-in</strong>
      </div>
      <div class="card-body">
        <p class="mb-2">Super Admin uses the <strong>same login page</strong> (<code>login.php</code>). There is no separate URL.
      </p>
        <p class="mb-2"><strong>Default username:</strong> <code><?php echo htmlspecialchars(DEFAULT_SUPER_ADMIN_USERNAME) ?></code></p>
        <p class="mb-2"><strong>Default password:</strong> see <code>config/config.php</code> →
        <code>DEFAULT_SUPER_ADMIN_PASSWORD</code>. Change it in the file and under <strong>Settings</strong> after you sign in.</p>
        <p class="mb-0 small text-muted">After signing in: sidebar → <strong>Administrator accounts</strong> → create or edit
        <code>admin</code> users for day-to-day scheduling.</p>
      </div>
    </div>
    <a class="btn btn-primary btn-lg" href="login.php">Go to login</a>
  <?php else: ?>
    <?php if ($err): ?>
      <div class="alert alert-danger"><?php echo htmlspecialchars($err) ?></div>
    <?php endif; ?>
    <form method="post">
      <button class="btn btn-primary" type="submit">Run upgrade</button>
    </form>
  <?php endif; ?>
</div>
</body>
</html>

```

```

-----
FILE: view_schedule.php
-----

```

```

<?php
declare(strict_types=1);

require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

require_role(['super_admin', 'admin', 'dean', 'program_chair', 'gened', 'faculty']);
$role = (string) ($_SESSION['role'] ?? '');
$collegeId = current_college_id();
$programScope = is_program_chair() ? program_scope_or_fail() : null;
$facultySelfId = isset($_SESSION['faculty_id']) ? (int) $_SESSION['faculty_id'] : 0;
$facultyCollegeId = ($role === 'faculty' && $facultySelfId > 0) ? faculty_college_id($facultySelfId) : null;

$hasCourseColors = ensure_faculty_course_colors_table();

if (
  $_SERVER['REQUEST_METHOD'] === 'POST'
  && $role === 'faculty'
  && $facultySelfId > 0
  && (string) ($_POST['action'] ?? '') === 'save_course_color'
) {
  $redirectQs = [];
  foreach (['dept', 'semester', 'school_year'] as $qk) {
    $qv = trim((string) ($_POST[$qk] ?? $_GET[$qk] ?? ''));
    if ($qv !== '') {
      $redirectQs[$qk] = $qv;
    }
  }
  $redirectUrl = 'view_schedule.php' . ($redirectQs !== [] ? ('?' . http_build_query($redirectQs)) : '');
  try {
    save_faculty_course_color(
      $facultySelfId,
      (int) ($_POST['course_id'] ?? 0),
      (int) ($_POST['color_index'] ?? 0),

```

```

        $facultyCollegeId
    );
    $_SESSION['flash'] = 'Course color saved. Matching blocks on your weekly view use this color.';
} catch (Throwable $e) {
    $_SESSION['flash'] = 'Error: ' . $e->getMessage();
}
header('Location: ' . $redirectUrl);
exit;
}

$flash = '';
if (!empty($_SESSION['flash'])) {
    $flash = (string) $_SESSION['flash'];
    unset($_SESSION['flash']);
}

$dept = trim((string) ($_GET['dept'] ?? ''));
$sem = trim((string) ($_GET['semester'] ?? ''));
$sy = trim((string) ($_GET['school_year'] ?? ''));
$facultyFilter = (int) ($_GET['faculty_id'] ?? 0);
$roomFilter = (int) ($_GET['room_id'] ?? 0);
$collegeFilter = (int) ($_GET['college_id'] ?? 0);

$hasOnlineUrlCol = db_column_exists('schedules', 'online_class_url');
$hasLiveAtCol = db_column_exists('schedules', 'online_live_at');
$hasGeTargetsTable = db_table_exists('ge_schedule_targets');
$hasCourseIsGenedCol = db_column_exists('courses', 'is_gened');
$targetJoin = $hasGeTargetsTable ? ' LEFT JOIN ge_schedule_targets gst ON gst.schedule_id = s.id' : '';
$hasOcTable = db_table_exists('online_classrooms');
$hasSyllabusOnOc = $hasOcTable && db_column_exists('online_classrooms', 'syllabus_stored_name');
$ocJoin = $hasOcTable ? ' LEFT JOIN online_classrooms oc ON oc.schedule_id = s.id' : '';
$ocSelect = '';
if ($hasOcTable) {
    $ocSelect = ', oc.id AS oc_classroom_id';
    if ($hasSyllabusOnOc) {
        $ocSelect .= ', oc.syllabus_stored_name AS oc_syllabus_stored';
    }
}
$courseIsGenedSelect = $hasCourseIsGenedCol ? ', c.is_gened AS course_is_gened' : '';
$geTargetSelect = $hasGeTargetsTable ? ', gst.program_name AS ge_target_program' : '';

$hostCollegeJoin = ' LEFT JOIN colleges sched_col ON sched_col.id = s.college_id';
$hostCollegeSelect = ', sched_col.college_code AS host_college_code';

$sql = "SELECT DISTINCT s.*, f.full_name AS faculty_name, c.course_code, c.course_name, c.department AS
course_department{$courseIsGenedSelect}{$geTargetSelect}, r.room_code, r.room_name{$hostCollegeSelect}{$ocSelect}
FROM schedules s
INNER JOIN faculty f ON f.id = s.faculty_id
INNER JOIN courses c ON c.id = s.course_id
INNER JOIN rooms r ON r.id = s.room_id
{$hostCollegeJoin}
{$targetJoin}
{$ocJoin}
WHERE 1=1";
$params = [];
if ($role === 'faculty' && $facultySelfId > 0) {
    $sql .= ' AND s.faculty_id = ?';
    $params[] = $facultySelfId;
    if ($facultyCollegeId !== null) {
        $sql .= ' AND s.college_id = ?';
        $params[] = $facultyCollegeId;
    }
} elseif ($programScope !== null && $collegeId) {
    $sql .= program_chair_schedule_scope_sql($collegeId, $programScope, $hasGeTargetsTable, $params);
} elseif (is_dean() && $collegeId) {
    $sql .= dean_schedule_scope_sql($collegeId, $hasCourseIsGenedCol, $params, $hasGeTargetsTable);
}
if ($collegeFilter > 0 && !is_dean() && $role !== 'faculty') {
    $sql .= ' AND s.college_id = ?';
    $params[] = $collegeFilter;
}
if ($dept !== '') {
    if ($hasGeTargetsTable) {
        $sql .= ' AND (f.department = ? OR c.department = ? OR gst.program_name = ?)';
        $params[] = $dept;
        $params[] = $dept;
        $params[] = $dept;
    } else {
        $sql .= ' AND (f.department = ? OR c.department = ?)';
        $params[] = $dept;
    }
}

```

```

        $params[] = $dept;
    }
}
if ($sem != '') {
    $sql .= ' AND s.semester = ?';
    $params[] = $sem;
}
if ($sy != '') {
    $sql .= ' AND s.school_year = ?';
    $params[] = $sy;
}
if ($facultyFilter > 0) {
    $sql .= ' AND s.faculty_id = ?';
    $params[] = $facultyFilter;
}
if ($roomFilter > 0) {
    $sql .= ' AND s.room_id = ?';
    $params[] = $roomFilter;
}
$sql .= ' ORDER BY s.start_time, f.full_name';

$stmt = db()->prepare($sql);
$stmt->execute($params);
$schedules = $stmt->fetchAll();

/** @var array<int, array<int, int>> $facultyColorMaps faculty_id => (course_id => color_index) */
$facultyColorMaps = [];
if ($hasCourseColors && $schedules) {
    $facultyIdsForColors = [];
    foreach ($schedules as $sRow) {
        $fid = (int) ($sRow['faculty_id'] ?? 0);
        if ($fid > 0) {
            $facultyIdsForColors[$fid] = true;
        }
    }
    $ids = array_keys($facultyIdsForColors);
    if ($ids !== []) {
        $placeholders = implode(',', array_fill(0, count($ids), '?'));
        $stColors = db()->prepare(
            "SELECT faculty_id, course_id, color_index
            FROM faculty_course_colors
            WHERE faculty_id IN ({$placeholders})"
        );
        $stColors->execute($ids);
        foreach ($stColors->fetchAll(PDO::FETCH_ASSOC) as $colorRow) {
            $fid = (int) $colorRow['faculty_id'];
            $cid = (int) $colorRow['course_id'];
            $facultyColorMaps[$fid][$cid] = max(0, min(5, (int) $colorRow['color_index']));
        }
    }
}

$facultyCourseLegend = [];
if ($role === 'faculty' && $facultySelfId > 0) {
    foreach ($schedules as $sRow) {
        $cid = (int) ($sRow['course_id'] ?? 0);
        if ($cid < 1 || isset($facultyCourseLegend[$cid])) {
            continue;
        }
        $facultyCourseLegend[$cid] = [
            'course_code' => (string) ($sRow['course_code'] ?? ''),
            'course_name' => (string) ($sRow['course_name'] ?? ''),
            'color_index' => (int) ($facultyColorMaps[$facultySelfId][$cid] ?? ($cid % 6)),
        ];
    }
    uasort($facultyCourseLegend, static function (array $a, array $b): int {
        return strcmp($a['course_code'], $b['course_code']);
    });
}
$colorPalette = schedule_color_palette();

$formatTime12h = static function (?string $time): string {
    $raw = substr((string) $time, 0, 5);
    $dt = DateTime::createFromFormat('H:i', $raw);
    return $dt ? $dt->format('g:i A') : $raw;
};

$byDay = [];
foreach (schedule_days_list() as $d) {
    $byDay[$d] = [];
}

```

```

}
foreach ($schedules as $s) {
    foreach (parse_day_set((string) $s['day_of_week']) as $d) {
        if (isset($byDay[$d])) {
            $byDay[$d][] = $s;
        }
    }
}
foreach ($byDay as $d => $list) {
    usort($byDay[$d], static function ($a, $b) {
        return strcmp((string) $a['start_time'], (string) $b['start_time']);
    });
}

/** Hourly grid bounds so vacant slots are visible against the time axis. */
$gridStartMin = defined('TIME_MIN') ? time_to_minutes((string) TIME_MIN) : 7 * 60;
$gridEndMin = defined('TIME_MAX') ? time_to_minutes((string) TIME_MAX) : 21 * 60;
foreach ($schedules as $s) {
    $st = time_to_minutes(substr((string) $s['start_time'], 0, 8));
    $en = time_to_minutes(substr((string) $s['end_time'], 0, 8));
    if ($st < $gridStartMin) {
        $gridStartMin = $st;
    }
    if ($en > $gridEndMin) {
        $gridEndMin = $en;
    }
}
$gridStartHour = (int) floor($gridStartMin / 60);
$gridEndHour = (int) max($gridStartHour + 1, (int) ceil($gridEndMin / 60));
$timeSlots = [];
for ($h = $gridStartHour; $h < $gridEndHour; $h++) {
    $timeSlots[] = $h;
}

$scheduleOverlapsHour = static function (array $scheduleRow, int $hour): bool {
    $slotStart = $hour * 60;
    $slotEnd = ($hour + 1) * 60;
    $st = time_to_minutes(substr((string) $scheduleRow['start_time'], 0, 8));
    $en = time_to_minutes(substr((string) $scheduleRow['end_time'], 0, 8));
    return $st < $slotEnd && $en > $slotStart;
};

$scheduleStartsInHour = static function (array $scheduleRow, int $hour): bool {
    $st = time_to_minutes(substr((string) $scheduleRow['start_time'], 0, 8));
    return $st >= ($hour * 60) && $st < (($hour + 1) * 60);
};

/**
 * Merge a straight multi-hour class into one tall cell (rowspan) when the next
 * hour(s) are only a continuation of the same block (no other class begins there).
 * @var array<string, array<int, array{startHour:int, span:int}>> $rowspanPlans
 * @var array<string, array<int, int>> $coveredByRowspan day => hour => startHour that owns the rowspan
 */
$rowspanPlans = [];
$coveredByRowspan = [];
foreach (schedule_days_list() as $dayName) {
    $dayList = $byDay[$dayName] ?? [];
    foreach ($dayList as $schedRow) {
        $sid = (int) ($schedRow['id'] ?? 0);
        if ($sid < 1) {
            continue;
        }
        $overlapHours = [];
        foreach ($timeSlots as $hour) {
            if ($scheduleOverlapsHour($schedRow, $hour)) {
                $overlapHours[] = $hour;
            }
        }
        if (count($overlapHours) < 2) {
            continue;
        }
        $startHour = $overlapHours[0];
        $span = 1;
        for ($i = 1, $n = count($overlapHours); $i < $n; $i++) {
            if ($overlapHours[$i] !== $overlapHours[$i - 1] + 1) {
                break;
            }
        }
        $nextHour = $overlapHours[$i];
        $hourClear = true;
        foreach ($dayList as $other) {

```

```

        if (!$scheduleOverlapsHour($other, $nextHour)) {
            continue;
        }
        // Another class appears in this next hour but not at the start → stop merging here.
        if (!$scheduleOverlapsHour($other, $startHour)) {
            $hourClear = false;
            break;
        }
    }
    if (!$hourClear) {
        break;
    }
    $span++;
}
if ($span < 2) {
    continue;
}
// Only block if a different start-hour already claimed a continuation slot.
$blocked = false;
for ($h = $startHour + 1; $h < $startHour + $span; $h++) {
    if (isset($coveredByRowspan[$dayName][$h]) && $coveredByRowspan[$dayName][$h] != $startHour) {
        $blocked = true;
        break;
    }
}
if ($blocked) {
    continue;
}
$rowspanPlans[$dayName][$sid] = ['startHour' => $startHour, 'span' => $span];
for ($h = $startHour + 1; $h < $startHour + $span; $h++) {
    $coveredByRowspan[$dayName][$h] = $startHour;
}
}
}

$depts = ($role === 'dean' || $role === 'program_chair') && $collegeId
? (function () use ($collegeId) {
    $st = db()->prepare('SELECT DISTINCT department FROM faculty WHERE department != "" AND college_id=? ORDER BY department');
    $st->execute([$collegeId]);
    return $st->fetchAll(PDO::FETCH_COLUMN);
})()
: db()->query('SELECT DISTINCT department FROM faculty WHERE department != "" ORDER BY department')->fetchAll(PDO::FETCH_COLUMN);
$depts = $programScope !== null ? [$programScope] : $depts;
$sems = db()->query('SELECT DISTINCT semester FROM schedules ORDER BY semester')->fetchAll(PDO::FETCH_COLUMN);
$years = db()->query('SELECT DISTINCT school_year FROM schedules ORDER BY school_year DESC')->fetchAll(PDO::FETCH_COLUMN);
if (($role === 'dean' || $role === 'program_chair') && $collegeId) {
    $sql = "SELECT id, full_name FROM faculty WHERE status='active' AND college_id=?";
    $params = [$collegeId];
    if ($programScope !== null) {
        $sql .= " AND department=?";
        $params[] = $programScope;
    }
    $sql .= " ORDER BY full_name";
    $st = db()->prepare($sql);
    $st->execute($params);
    $facultyList = $st->fetchAll();
    $st = db()->prepare("SELECT id, room_code FROM rooms WHERE status IN ('available','tba') AND college_id=? ORDER BY room_code");
    $st->execute([$collegeId]);
    $roomList = $st->fetchAll();
} elseif ($role === 'faculty') {
    $facultyList = [];
    $roomList = [];
} else {
    $facultyList = db()->query("SELECT id, full_name FROM faculty WHERE status='active' ORDER BY full_name")->fetchAll();
    $roomList = db()->query("SELECT id, room_code FROM rooms WHERE status IN ('available','tba') ORDER BY room_code")->fetchAll();
}
$collegeList = db()->query("SELECT id, college_code, college_name FROM colleges WHERE status='active' ORDER BY college_code")->fetchAll();

$pageTitle = 'Weekly schedule view';
require_once __DIR__ . '/includes/header.php';
?>
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-3">
    <h1 class="h3 mb-0"><i class="fa-solid fa-calendar-week me-2 text-primary"></i>Weekly schedule</h1>
    <button type="button" class="btn btn-outline-secondary no-print" onclick="window.print()"><? app_tooltip_attr('Opens the print dialog
for this weekly view. Use this for a paper copy or PDF without sidebar clutter.') ?>><i class="fa-solid fa-print me-1"></i>Print</button>
</div>

<?php if ($flash !== ''): ?><?php render_information_popup((string) $flash); ?><?php endif; ?>

<?php if ($role === 'faculty' && $hasCourseColors && $facultyCourseLegend !== []): ?>

```

```

<?php
$firstCourseId = (int) array_key_first($facultyCourseLegend);
$firstCourseColor = (int) ($facultyCourseLegend[$firstCourseId]['color_index'] ?? 0);
?>
<div class="card border-0 shadow-sm mb-3 no-print course-color-panel">
  <div class="card-body py-3">
    <div class="mb-2">
      <h2 class="h6 mb-1"><i class="fa-solid fa-palette me-1 text-primary"></i>Course colors</h2>
      <p class="small text-muted mb-0">Choose a course and a color, then apply it to the weekly grid.</p>
    </div>
    <form method="post" class="row g-2 align-items-end course-color-form" id="course-color-form">
      <input type="hidden" name="action" value="save_course_color">
      <?php if ($dept !== ''): ?><input type="hidden" name="dept" value="<?= htmlspecialchars($dept) ?>"><?php endif; ?>
      <?php if ($sem !== ''): ?><input type="hidden" name="semester" value="<?= htmlspecialchars($sem) ?>"><?php endif; ?>
      <?php if ($sy !== ''): ?><input type="hidden" name="school_year" value="<?= htmlspecialchars($sy) ?>"><?php endif; ?>
      <div class="col-12 col-sm-5 col-md-4">
        <label class="form-label small mb-0" for="course-color-course">Course</label>
        <select id="course-color-course" name="course_id" class="form-select form-select-sm" required>
          <?php foreach ($facultyCourseLegend as $courseId => $courseInfo): ?>
            <option
              value="<?= (int) $courseId ?>"
              data-color-index="<?= (int) $courseInfo['color_index'] ?>"
              <?= (int) $courseId === $firstCourseId ? 'selected' : '' ?>
            >
              <?= htmlspecialchars($courseInfo['course_code']) ?><?= $courseInfo['course_name'] !== '' ? ' - ' : '' ?>
              <?= htmlspecialchars($courseInfo['course_name']) : '' ?>
            </option>
          <?php endforeach; ?>
        </select>
      </div>
      <div class="col-8 col-sm-4 col-md-3">
        <label class="form-label small mb-0" for="course-color-index">Color</label>
        <div class="d-flex align-items-center gap-2">
          <span
            id="course-color-preview"
            class="course-color-preview"
            style="--swatch-bg: <?= htmlspecialchars($colorPalette[$firstCourseColor]['bg']) ?> '#e2e8f0' ?>; --swatch-border: <?= htmlspecialchars($colorPalette[$firstCourseColor]['border']) ?> '#94a3b8' ?>;"
            aria-hidden="true"
          ></span>
          <select id="course-color-index" name="color_index" class="form-select form-select-sm" required>
            <?php foreach ($colorPalette as $swatch): ?>
              <option
                value="<?= (int) $swatch['index'] ?>"
                data-bg="<?= htmlspecialchars($swatch['bg']) ?>"
                data-border="<?= htmlspecialchars($swatch['border']) ?>"
                <?= (int) $swatch['index'] === $firstCourseColor ? 'selected' : '' ?>
              >
                <?= htmlspecialchars($swatch['label']) ?>
              </option>
            <?php endforeach; ?>
          </select>
        </div>
      </div>
      <div class="col-4 col-sm-3 col-md-2">
        <button type="submit" class="btn btn-sm btn-primary w-100"><?= app_tooltip_attr('Saves the selected color for the chosen course on your weekly schedule.') ?>>Apply</button>
      </div>
    </form>
    <?php if (count($facultyCourseLegend) > 0): ?>
      <div class="course-color-legend mt-3 pt-2 border-top">
        <div class="small text-muted mb-2">Current colors</div>
        <div class="d-flex flex-wrap gap-2">
          <?php foreach ($facultyCourseLegend as $courseId => $courseInfo): ?>
            <?php
              $ci = (int) $courseInfo['color_index'];
              $sw = $colorPalette[$ci] ?? $colorPalette[0];
            >
            <span class="course-color-legend-item">
              <span
                class="course-color-preview course-color-preview--sm"
                style="--swatch-bg: <?= htmlspecialchars($sw['bg']) ?> '#e2e8f0' ?>; --swatch-border: <?= htmlspecialchars($sw['border']) ?> '#94a3b8' ?>;"
                aria-hidden="true"
              ></span>
              <span class="small"><?= htmlspecialchars($courseInfo['course_code']) ?></span>
            </span>
          <?php endforeach; ?>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

        <?php endif; ?>
    </div>
</div>
<script>
(function () {
    var courseSelect = document.getElementById('course-color-course');
    var colorSelect = document.getElementById('course-color-index');
    var preview = document.getElementById('course-color-preview');
    if (!courseSelect || !colorSelect || !preview) return;

    function syncPreview() {
        var opt = colorSelect.options[colorSelect.selectedIndex];
        if (!opt) return;
        preview.style.setProperty('--swatch-bg', opt.getAttribute('data-bg') || '#e2e8f0');
        preview.style.setProperty('--swatch-border', opt.getAttribute('data-border') || '#94a3b8');
    }

    courseSelect.addEventListener('change', function () {
        var opt = courseSelect.options[courseSelect.selectedIndex];
        if (!opt) return;
        var idx = opt.getAttribute('data-color-index');
        if (idx !== null && idx !== '') {
            colorSelect.value = String(idx);
        }
        syncPreview();
    });
    colorSelect.addEventListener('change', syncPreview);
})();
</script>
<?php elseif ($role === 'faculty' && !$hasCourseColors): ?>
    <div class="alert alert-warning no-print">Course color coding needs a database update. Ask your administrator to run <a
href="upgrade_roles.php">upgrade_roles.php</a> once.</div>
<?php endif; ?>

<form class="row g-2 mb-4 no-print align-items-end" method="get">
    <div class="col-6 col-md-2">
        <label class="form-label small mb-0">Program</label>
        <select name="dept" class="form-select form-select-sm">
            <option value="">All</option>
            <?php foreach ($depts as $d): ?>
                <option value="<?= htmlspecialchars((string) $d) ?>" <?= $dept === (string) $d ? 'selected' : '' ?>><?=
htmlspecialchars((string) $d) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="col-6 col-md-2">
        <label class="form-label small mb-0">Semester</label>
        <select name="semester" class="form-select form-select-sm">
            <option value="">All</option>
            <?php foreach ($sems as $s): ?>
                <option value="<?= htmlspecialchars((string) $s) ?>" <?= $sem === (string) $s ? 'selected' : '' ?>><?=
htmlspecialchars((string) $s) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="col-6 col-md-2">
        <label class="form-label small mb-0">School year</label>
        <select name="school_year" class="form-select form-select-sm">
            <option value="">All</option>
            <?php foreach ($years as $y): ?>
                <option value="<?= htmlspecialchars((string) $y) ?>" <?= $sy === (string) $y ? 'selected' : '' ?>><?=
htmlspecialchars((string) $y) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <?php if ($role !== 'faculty' && $programScope === null && ($role === 'admin' || $role === 'gened')): ?>
        <div class="col-6 col-md-2">
            <label class="form-label small mb-0">College</label>
            <select name="college_id" class="form-select form-select-sm">
                <option value="0">All colleges</option>
                <?php foreach ($collegeList as $c): ?>
                    <option value="<?= (int) $c['id'] ?>" <?= $collegeFilter === (int) $c['id'] ? 'selected' : '' ?>>
                        <?= htmlspecialchars($c['college_code']) ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </div>
    <?php endif; ?>
    <?php if ($role !== 'faculty'): ?>
        <div class="col-6 col-md-3">

```



```

        <label class="form-label small mb-0">Faculty</label>
        <select name="faculty_id" class="form-select form-select-sm">
            <option value="0">All faculty</option>
            <?php foreach ($facultyList as $f): ?>
                <option value="<?=(int) $f['id'] ?>" <?=$facultyFilter === (int) $f['id'] ? 'selected' : '' ?>><?=$
htmlspecialchars($f['full_name']) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <div class="col-6 col-md-3">
        <label class="form-label small mb-0">Room</label>
        <select name="room_id" class="form-select form-select-sm">
            <option value="0">All rooms</option>
            <?php foreach ($roomList as $r): ?>
                <option value="<?=(int) $r['id'] ?>" <?=$roomFilter === (int) $r['id'] ? 'selected' : '' ?>><?=$
htmlspecialchars($r['room_code']) ?></option>
            <?php endforeach; ?>
        </select>
    </div>
    <?php endif; ?>
    <div class="col-12">
        <button type="submit" class="btn btn-outline-primary btn-sm"<?=$app_tooltip_attr('Reloads the grid using program, term, faculty, and
room filters. Use this after narrowing what you need to review.') ?>>Apply filters</button>
        <a href="view_schedule.php" class="btn btn-outline-secondary btn-sm"<?=$app_tooltip_attr('Clears filters and shows the default
weekly view. Use this when you want to start selection over.') ?>>Reset</a>
    </div>
</form>

<p class="text-muted small mb-2 no-print">Times on the left mark each hour. Empty cells are vacant. A straight class (for example 8:00 AM-
10:00 AM) shows as one tall block across those hours when the next slot is only a continuation.</p>
<div class="table-responsive">
    <table class="table table-bordered bg-body schedule-weekly schedule-weekly--timed">
        <thead class="table-primary">
            <tr>
                <th class="text-center schedule-time-col" scope="col">Time</th>
                <?php foreach (schedule_days_list() as $day): ?>
                    <th class="text-center" style="min-width:140px"><?=$htmlspecialchars($day) ?></th>
                <?php endforeach; ?>
            </tr>
        </thead>
        <tbody>
            <?php foreach ($timeSlots as $hour): ?>
                <?php
                $slotLabel = $formatTime12h(sprintf('%02d:00', $hour));
                $slotEndLabel = $formatTime12h(sprintf('%02d:00', $hour + 1));
                ?>
                <tr>
                    <th class="schedule-time-col text-end align-top" scope="row">
                        <div class="schedule-time-label fw-semibold"><?=$htmlspecialchars($slotLabel) ?></div>
                        <div class="schedule-time-end small text-muted"><?=$htmlspecialchars($slotEndLabel) ?></div>
                    </th>
                    <?php foreach (schedule_days_list() as $day): ?>
                        <?php
                        // Cell already occupied by a multi-hour rowspan from an earlier hour.
                        if (isset($coveredByRowspan[$day][$hour])) {
                            continue;
                        }
                        $slotClasses = [];
                        foreach ($byDay[$day] as $s) {
                            if ($scheduleOverlapsHour($s, $hour)) {
                                $slotClasses[] = $s;
                            }
                        }
                        $isVacant = $slotClasses === [];
                        $cellRowspan = 1;
                        foreach ($slotClasses as $s) {
                            $sid = (int) ($s['id'] ?? 0);
                            if ($sid > 0 && isset($rowspanPlans[$day][$sid]) && $rowspanPlans[$day][$sid]['startHour'] === $hour) {
                                $cellRowspan = max($cellRowspan, (int) $rowspanPlans[$day][$sid]['span']);
                            }
                        }
                        ?>
                        <td class="align-top schedule-cell p-2"<?=$isVacant ? ' schedule-cell--vacant' : '' ?><?=$cellRowspan > 1 ? '
schedule-cell--spanned' : '' ?><?=$cellRowspan > 1 ? ' rowspan="' . $cellRowspan . '"' : '' ?>>
                            <?php if ($isVacant): ?>
                                <span class="schedule-vacant-label">Vacant</span>
                            <?php else: ?>
                                <?php foreach ($slotClasses as $s): ?>
                                    <?php
                                    $startsHere = $scheduleStartsInHour($s, $hour);

```

```

        $sid = (int) ($s['id'] ?? 0);
        $blockSpan = ($sid > 0 && isset($rowspanPlans[$day][$sid]) && $rowspanPlans[$day][$sid]['startHour'] ==
$hour)
            ? (int) $rowspanPlans[$day][$sid]['span']
            : 1;
        // Skip "(continues)" only inside the merged rowspan range.
        if (!$startsHere && $sid > 0 && isset($rowspanPlans[$day][$sid])) {
            $plan = $rowspanPlans[$day][$sid];
            if ($hour >= $plan['startHour'] && $hour < $plan['startHour'] + $plan['span']) {
                continue;
            }
        }
        $blockFacultyId = (int) ($s['faculty_id'] ?? 0);
        $blockCourseId = (int) ($s['course_id'] ?? 0);
        $c = schedule_block_color_class(
            $blockCourseId,
            $facultyColorMaps[$blockFacultyId] ?? null
        );
        $isGeCourse = $hasCourseIsGenedCol && (int) ($s['course_is_gened'] ?? 0) == 1;
        $hostCollegeCode = trim((string) ($s['host_college_code'] ?? ''));
        $showHostCollege = is_dean() && $collegeId && $isGeCourse && $hostCollegeCode !== ''
            && (int) ($s['college_id'] ?? 0) !== $collegeId;
    }>
    <?php
        $roomLabel = trim((string) ($s['room_code'] ?? ''));
        $roomName = trim((string) ($s['room_name'] ?? ''));
        if ($roomLabel !== '' && $roomName !== '' && strcasecmp($roomLabel, $roomName) !== 0) {
            $roomLabel .= ' - ' . $roomName;
        } elseif ($roomLabel === '' && $roomName !== '') {
            $roomLabel = $roomName;
        }
    }>
    <?php
        $liveDisplayMode = weekly_schedule_online_live_mode($role, $s, $collegeFilter, $dept);
        $onlineUrl = $hasOnlineUrlCol ? trim((string) ($s['online_class_url'] ?? '')) : '';
        $liveAtStr = $hasLiveAtCol ? ($s['online_live_at'] ?? null) : null;
        $liveAtStr = $liveAtStr !== null && $liveAtStr !== '' ? (string) $liveAtStr : null;
        $isLive = $hasLiveAtCol && schedule_display_is_faculty_live($s);
        $showLiveIndicator = $isLive && $liveDisplayMode !== 'hidden';
    }>
    <?php if (!$startsHere): ?>
        <div class="schedule-block schedule-block--continued <?= $c ?>">
            <div class="small fw-semibold">
                <?= htmlspecialchars($s['course_code']) ?> <span class="text-muted fw-normal">(continues)
</span>

                <?php if ($showLiveIndicator): ?>
                    <span class="badge bg-danger live-pulse-badge rounded-pill ms-1" style="font-
size:0.65rem"><i class="fa-solid fa-circle me-1" style="font-size:0.5rem;vertical-align:middle;"></i>LIVE</span>
                <?php endif; ?>
            </div>
            <div class="small text-muted"><?= htmlspecialchars($formatTime12h((string) $s['start_time'])) ?>
- <?= htmlspecialchars($formatTime12h((string) $s['end_time'])) ?></div>
            <?php if ($roomLabel !== ''): ?>
                <div class="small"><i class="fa-solid fa-door-open me-1"></i><?=
htmlspecialchars($roomLabel) ?></div>

                <?php endif; ?>
            </div>
        <?php else: ?>
            <div class="schedule-block <?= $c ?><?= $blockSpan > 1 ? ' schedule-block--spanned' : '' ?>">
$blockSpan > 1 ? ' style="--block-span:' . $blockSpan . '"" : '' ?>
            <div class="fw-semibold"><?= htmlspecialchars($formatTime12h((string) $s['start_time'])) ?> - <?
= htmlspecialchars($formatTime12h((string) $s['end_time'])) ?></div>
            <div><?= htmlspecialchars($s['course_code']) ?><?php if ($isGeCourse): ?> <span class="badge bg-
info text-dark" style="font-size:0.65rem">GE</span><?php endif; ?><?php if (!empty($s['is_makeup'])): ?> <span class="badge bg-warning text-
dark" style="font-size:0.65rem">Makeup</span><?php endif; ?></div>
            <?php if ($showHostCollege): ?>
                <div class="small text-muted"><i class="fa-solid fa-building-columns me-1"></i><?=
htmlspecialchars($hostCollegeCode) ?></div>

            <?php endif; ?>
            <div class="small text-muted"><?= htmlspecialchars($s['faculty_name']) ?></div>
            <?php if ($roomLabel !== ''): ?>
                <div class="small"><i class="fa-solid fa-door-open me-1"></i><?=
htmlspecialchars($roomLabel) ?></div>

            <?php endif; ?>
            <?php if ($hasOnlineUrlCol && $onlineUrl !== '' && $liveDisplayMode !== 'hidden'): ?>
                <div class="mt-1 pt-1 border-top border-secondary-subtle small">
                    <?php if ($liveDisplayMode === 'unauthorized' && $hasLiveAtCol && $isLive): ?>
                        <span class="badge bg-danger live-pulse-badge rounded-pill me-1"><i class="fa-solid
fa-circle me-1" style="font-size:0.5rem;vertical-align:middle;"></i>LIVE</span>
                    <?php

```

```

        $unauthTip = $role === 'dean'
            ? 'Live GE classes can only be joined by the Dean of the College of Arts and
Sciences. You can still monitor schedules and conflicts in this view.'
            : 'General education classes cannot be joined from a program chair account. Use
the dean or GEN ED weekly view for oversight.';
    }>
    <span class="badge bg-warning text-dark"> app_tooltip_attr($unauthTip) ?

>>Unauthorized</span>

    <?php elseif ($liveDisplayMode === 'normal' && $hasLiveAtCol && $isLive): ?>
        <span class="badge bg-danger live-pulse-badge rounded-pill me-1"><i class="fa-solid
fa-circle me-1" style="font-size:0.5rem;vertical-align:middle;"></i>LIVE</span>
        <a class="btn btn-sm btn-success text-white py-0 px-2" href="<? =
htmlspecialchars($onlineUrl) ?>" target="_blank" rel="noopener noreferrer"> app_tooltip_attr('Opens the live online class while the
instructor is broadcasting. Use this during scheduled class time.') ?>>Join online</a>
    <?php elseif ($liveDisplayMode === 'normal' && $hasLiveAtCol): ?>
        <span class="badge bg-secondary me-1">Not live</span>
        <span class="text-muted">Faculty has not gone live</span>
    <?php elseif ($liveDisplayMode === 'normal'): ?>
        <a class="btn btn-sm btn-outline-primary py-0 px-2" href="<? =
htmlspecialchars($onlineUrl) ?>" target="_blank" rel="noopener noreferrer"> app_tooltip_attr('Opens the faculty's online meeting link in a
new tab. Use this to join the virtual room for this block.') ?>><i class="fa-solid fa-video me-1"></i>Online class</a>
    <?php endif; ?>
</div>
<?php endif; ?>
<?php
$ocPortalId = isset($s['oc_classroom_id']) ? (int) $s['oc_classroom_id'] : 0;
$ocHasSyllabus = $hasSyllabusOnOc && $ocPortalId > 0 && trim((string) ($s['oc_syllabus_stored']
?? '')) !== '';

$hidePcGeButtons = $role === 'program_chair' && $isGeCourse;
$hideGeNonGeButtons = $role === 'gened' && !$isGeCourse;
$hideButtons = $hidePcGeButtons || $hideGeNonGeButtons;
$syllabusRoles = ['admin', 'dean', 'program_chair', 'gened'];
$monitorRoles = ['admin', 'dean', 'program_chair', 'gened'];
$monitorHref = 'classroom_materials_monitor.php?id=' . $ocPortalId;
if ($role === 'gened') {
    $monitorHref .= '&monitor_college=' . (string) $collegeFilter;
    $monitorHref .= '&monitor_program=' . rawurlencode($dept);
}
?>
<?php if ($ocHasSyllabus && in_array($role, $syllabusRoles, true) && !$hideButtons): ?>
    <div class="mt-1">
        <a class="btn btn-sm btn-outline-secondary py-0 px-2" href="<? =
htmlspecialchars(classroom_syllabus_href($ocPortalId)) ?>" onclick="var w=400,h=300,l=(screen.width-w)/2,t=(screen.height-
h)/2;window.open(this.href,'syllabusWin','width='+w+',height='+h+',left='+l+',top='+t+',scrollbars=yes,resizable=yes');return false;">
app_tooltip_attr('Opens the faculty-uploaded syllabus for this section in a small window (oversight).') ?>><i class="fa-solid fa-file-
contract me-1"></i>Syllabus</a>

    </div>
    <?php endif; ?>
    <?php if ($ocPortalId > 0 && in_array($role, $monitorRoles, true) && !$hideButtons): ?>
        <div class="mt-1">
            <a class="btn btn-sm btn-outline-dark py-0 px-2" href="<? =
htmlspecialchars($monitorHref) ?>" onclick="var w=400,h=300,l=(screen.width-w)/2,t=(screen.height-
h)/2;window.open(this.href,'monitorWin','width='+w+',height='+h+',left='+l+',top='+t+',scrollbars=yes,resizable=yes');return false;">
app_tooltip_attr('Opens read-only monitoring of posted materials and week topics for this classroom.') ?>><i class="fa-solid fa-list-check
me-1"></i>Monitor</a>

        </div>
    <?php endif; ?>
    </div>
    <?php endif; ?>
    <?php endforeach; ?>
    </tr>
    <?php endforeach; ?>
    </tbody>
    </table>
</div>

<script>
(function () {
    if (!document.querySelector('.live-pulse-badge')) {
        return;
    }
    window.setInterval(function () {
        window.location.reload();
    }, 45000);
})();
</script>

```

```
<?php require_once __DIR__ . '/includes/footer.php'; ?>
```